

BEKO Pager Firmware

Wygenerowano za pomocą Doxygen 1.15.0

1 BEKO Pager Firmware	1
1.1 Wprowadzenie	1
1.2 Punkty startowe	1
1.3 Strony opisowe	2
1.4 Generowanie dokumentacji	2
1.5 Generowanie dokumentacji	2
1.5.1 Wymagania	2
1.5.2 Komenda	2
1.5.3 Zakres wejścia	2
1.5.4 Polityka plików wynikowych	3
1.6 Architektura firmware	3
1.6.1 Warstwy	3
1.6.2 Zadania RTOS	3
1.6.3 Wspólna magistrala I2C	3
1.6.4 Runtime configuration	4
1.7 Mapa modułów	4
1.7.1 Core application	4
1.7.2 Radio	4
1.7.3 Protocol and crypto	4
1.7.4 Security and storage	4
1.7.5 User interface	4
1.7.6 Sensors	5
1.8 Bezpieczeństwo	5
1.8.1 LAVIET_FRAME_V1	5
1.8.2 Kryptografia	5
1.8.3 TPM	5
1.8.4 Storage	5
2 BEKO Pager Network (pager-rtos)	7
2.1 1. Opis systemu	7
2.2 2. Adresacja	7
2.3 3. Role użytkowników	7
2.3.1 Administrator	8
2.3.2 Operator	8
2.3.3 Użytkownik	8
2.3.4 Serwisant	8
2.4 4. Ochrona ustawień lokalnych	8
2.5 5. Ramka LAVIET_FRAME_V1	8
2.5.1 Struktura ramki	8
2.5.2 Rozmiar ramki	9
2.6 6. Znaczenie pól <code>ver_type</code> i <code>flags</code>	9
2.6.1 <code>ver_type</code>	9

2.6.2 flags	9
2.7 7. Zabezpieczenia	10
2.7.1 HMAC-SHA256	10
2.7.2 Szyfrowanie wiadomości	10
2.8 8. TPM i klucze	10
2.9 9. Anti-replay i synchronizacja licznika	11
2.10 10. Parowanie i rotacja kluczy	11
2.10.1 Parowanie	11
2.10.2 Rotacja kluczy	12
2.11 11. Logi UART i kontrola startu	12
2.12 12. Główny graf systemu	12
2.13 13. Główne przepływy komunikacji	13
2.13.1 Zwykła wiadomość z ACK	13
2.13.2 Parowanie	13
2.13.3 Synchronizacja licznika	13
2.13.4 Rotacja kluczy	13
2.14 14. Dokumentacja Doxygen	13
3 BEKO Security Architecture	15
3.1 1. Skrót	15
3.2 2. Cele bezpieczeństwa	16
3.3 3. Model atakującego	16
3.4 4. Granice bezpieczeństwa	17
3.5 5. Komponenty implementacji	17
3.6 6. TPM jako root-of-trust	18
3.6.1 6.1. Dlaczego TPM jest bezpieczniejszy niż EEPROM	18
3.7 7. TPM init	18
3.7.1 7.1. Verbose logi TPM	19
3.8 8. Root seed	19
3.8.1 8.1. Dlaczego seed nie jest w firmware	19
3.8.2 8.2. Lifecycle root seeda	19
3.8.3 8.3. Physical Presence	20
3.8.4 8.4. Ograniczenie: seed ma 8 bajtów	20
3.9 9. Entropia i generowanie seeda	21
3.10 10. Hierarchia kluczy	21
3.11 11. Klucze ramek radiowych	21
3.11.1 11.1. Pair link key	22
3.11.2 11.2. Shared mode	22
3.12 12. Ramka LAVIET_FRAME_V1	23
3.13 13. Walidacja ramek	24
3.14 14. HMAC-SHA256	24
3.14.1 14.1. Kolejność RX	25

3.14.2 14.2. Stałoczasowe porównanie MAC	25
3.15 15. AES-CTR	25
3.15.1 15.1. Counter block AES-CTR	25
3.15.2 15.2. AES-CTR nie jest autentykacją	26
3.16 16. Hardware crypto STM32	26
3.16.1 16.1. Dlaczego self-test jest ważny	27
3.16.2 16.2. Pakowanie big-endian do AES	27
3.17 17. Software fallback	27
3.18 18. EEPROM secret storage	28
3.18.1 18.1. Szyfrowanie secret slots	28
3.19 19. Trusted devices	29
3.20 20. Pairing	29
3.21 21. ACK	30
3.22 22. Counter i anti-replay	30
3.23 23. COUNTER_SYNC	31
3.24 24. Broadcast	32
3.25 25. Lokalny PIN i role	32
3.26 26. Logi UART	33
3.27 27. Dlaczego ta architektura jest bezpieczna	33
3.27.1 27.1. Atak: podsłuch radiowy	34
3.27.2 27.2. Atak: zmiana payloadu	34
3.27.3 27.3. Atak: zmiana adresu docelowego	34
3.27.4 27.4. Atak: fałszywy ACK	34
3.27.5 27.5. Atak: odczyt EEPROM	35
3.27.6 27.6. Atak: zdalna rotacja root seeda	35
3.27.7 27.7. Atak: błąd hardware AES/HASH	35
3.27.8 27.8. Atak: przypadkowe wejście w menu admin	35
3.28 28. Ograniczenia obecnego wdrożenia	36
3.29 29. Zalecane utwardzenia	36
3.30 30. Przykłady	37
3.30.1 30.1. Przykład: budowa zaszyfrowanej ramki DATA	37
3.30.2 30.2. Przykład: odbiór zaszyfrowanej ramki	37
3.30.3 30.3. Przykład: odrzucenie zmienionej ramki	37
3.30.4 30.4. Przykład: rotacja root seeda	37
3.30.5 30.5. Przykład: hardware crypto self-test	37
3.30.6 30.6. Przykład: EEPROM secret slot	38
3.31 31. Checklist dla testów	38
3.32 32. Najkrótsza odpowiedź "dlaczego to jest bezpieczne"	38
4 STRIDE / CIA Analysis	39
4.1 STRIDE	39
4.2 CIA	41

4.3 Najważniejsze priorytety	41
5 Threat Model	43
5.1 Assets	43
5.2 Attacker goals	43
5.3 Current mitigations	43
5.4 Remaining gaps	43
6 Security	45
6.1 Pytania	45
6.2 Obszary	45
6.3 Security TODO	45
6.3.1 DoS / Replay	46
7 Struktura katalogów	47
7.1 Katalogi	47
8 Indeks struktur danych	51
8.1 Struktury danych	51
9 Indeks plików	53
9.1 Lista plików	53
10 Dokumentacja katalogów	55
10.1 Dokumentacja katalogu Core/App	55
10.1.1 Opis szczegółowy	56
10.1.2 Core/App - FreeRTOS Application Layer	56
10.1.3 Startup	56
10.1.4 Radio i UI	57
10.1.5 Build note	57
10.2 Dokumentacja katalogu Core/App/i2c_mem_store_lib	57
10.2.1 Opis szczegółowy	57
10.2.2 i2c_mem_store_lib	57
10.2.3 Założenia sprzętowe	57
10.2.4 Układ danych w pamięci	57
10.2.5 Bezpieczeństwo	58
10.3 Dokumentacja katalogu Core/App/led_array_lib	58
10.3.1 Opis szczegółowy	58
10.3.2 led_array_lib	58
10.3.3 Files	58
10.3.4 Design	59
10.3.5 API Overview	59
10.3.6 LED Masks	59
10.3.7 Usage (polling mode)	59

10.3.8 Usage (ISR mode)	59
10.3.9 Notes	60
10.4 Dokumentacja katalogu Core/App/radio_lib	60
10.4.1 Opis szczegółowy	60
10.4.2 radio_lib	60
10.4.3 Cele projektu	60
10.4.4 Struktura katalogu	60
10.4.5 Wybór aktywnej modulacji	61
10.4.6 Model obsługi przerwań (IRQ/EXTI)	61
10.4.7 Integracja w aplikacji	61
10.4.8 Status backendów	61
10.4.9 Dokumentacja Doxygen	62
10.5 Dokumentacja katalogu Core/App/security	62
10.5.1 Opis szczegółowy	62
10.5.2 Security	62
10.5.3 Aktualny model	62
10.5.4 DoS / Flood Defense	63
10.5.4.1 1. Rate limiting per source	63
10.5.4.2 2. Global TX budget	63
10.5.4.3 3. Separate limits for control traffic	63
10.5.4.4 4. Replay-aware abuse scoring	63
10.5.4.5 5. Strict frame policy	63
10.5.4.6 6. Bounded parsing cost	63
10.5.4.7 7. Queue and UI protection	63
10.5.4.8 8. Channel occupancy protection	64
10.5.5 Najbardziej opłacalna kolejność wdrożenia	64
10.6 Dokumentacja katalogu Core/App/st33ktpm2x_lib	64
10.6.1 Opis szczegółowy	64
10.6.2 st33ktpm2x_lib	64
10.6.3 Założenia	64
10.6.4 Co zawiera	64
10.6.5 Użycie w BEKO	65
11 Dokumentacja struktur danych	67
11.1 Dokumentacja struktury bmp280_api_data_t	67
11.1.1 Opis szczegółowy	67
11.1.2 Dokumentacja pól	67
11.1.2.1 pressure_hpa	67
11.1.2.2 pressure_pa	67
11.1.2.3 temperature_c	67
11.1.2.4 timestamp_ms	68
11.1.2.5 valid	68

11.2 Dokumentacja struktury BMP280_HandleTypedef	68
11.2.1 Opis szczegółowy	69
11.2.2 Dokumentacja pól	69
11.2.2.1 addr	69
11.2.2.2 dig_H1	69
11.2.2.3 dig_H2	69
11.2.2.4 dig_H3	69
11.2.2.5 dig_H4	69
11.2.2.6 dig_H5	69
11.2.2.7 dig_H6	70
11.2.2.8 dig_P1	70
11.2.2.9 dig_P2	70
11.2.2.10 dig_P3	70
11.2.2.11 dig_P4	70
11.2.2.12 dig_P5	70
11.2.2.13 dig_P6	70
11.2.2.14 dig_P7	70
11.2.2.15 dig_P8	70
11.2.2.16 dig_P9	70
11.2.2.17 dig_T1	71
11.2.2.18 dig_T2	71
11.2.2.19 dig_T3	71
11.2.2.20 i2c	71
11.2.2.21 id	71
11.2.2.22 params	71
11.3 Dokumentacja struktury bmp280_params_t	71
11.3.1 Opis szczegółowy	72
11.3.2 Dokumentacja pól	72
11.3.2.1 filter	72
11.3.2.2 mode	72
11.3.2.3 oversampling_humidity	72
11.3.2.4 oversampling_pressure	72
11.3.2.5 oversampling_temperature	72
11.3.2.6 standby	72
11.4 Dokumentacja struktury button_hw_t	72
11.4.1 Opis szczegółowy	73
11.4.2 Dokumentacja pól	73
11.4.2.1 active_low	73
11.4.2.2 pin	73
11.4.2.3 port	73
11.5 Dokumentacja struktury button_state_t	73
11.5.1 Opis szczegółowy	73

11.5.2 Dokumentacja pól	73
11.5.2.1 long_reported	73
11.5.2.2 pressed_since_ms	73
11.5.2.3 raw_pressed	73
11.5.2.4 stable_count	73
11.5.2.5 stable_pressed	74
11.6 Dokumentacja struktury i2c_mem_store_cfg_t	74
11.6.1 Opis szczegółowy	74
11.6.2 Dokumentacja pól	74
11.6.2.1 crypto_key	74
11.6.2.2 hi2c	74
11.6.2.3 i2c_addr_7bit	75
11.6.2.4 mem_addr_size	75
11.6.2.5 page_bytes	75
11.6.2.6 secret_area_bytes	75
11.6.2.7 total_bytes	75
11.6.2.8 write_cycle_ms	75
11.7 Dokumentacja struktury i2c_mem_store_log_record_t	75
11.7.1 Opis szczegółowy	76
11.7.2 Dokumentacja pól	76
11.7.2.1 length	76
11.7.2.2 payload	76
11.7.2.3 rssi_dbm	76
11.7.2.4 sequence	76
11.7.2.5 timestamp_ms	76
11.8 Dokumentacja struktury i2c_mem_store_meta_wire_t	77
11.8.1 Opis szczegółowy	77
11.8.2 Dokumentacja pól	77
11.8.2.1 crc16	77
11.8.2.2 magic	77
11.8.2.3 next_sequence	77
11.8.2.4 reserved	77
11.8.2.5 reserved2	77
11.8.2.6 secret_counter	77
11.8.2.7 valid_count	77
11.8.2.8 version	77
11.8.2.9 write_slot	78
11.9 Dokumentacja struktury i2c_mem_store_t	78
11.9.1 Opis szczegółowy	78
11.9.2 Dokumentacja pól	78
11.9.2.1 cfg	78
11.9.2.2 initialized	78

11.9.2.3 next_sequence	79
11.9.2.4 secret_counter	79
11.9.2.5 secret_slot_count	79
11.9.2.6 slot_count	79
11.9.2.7 valid_count	79
11.9.2.8 write_slot	79
11.10 Dokumentacja struktury i2c_mem_store_trusted_device_t	79
11.10.1 Opis szczegółowy	80
11.10.2 Dokumentacja pól	80
11.10.2.1 code	80
11.10.2.2 code_len	80
11.10.2.3 id	80
11.10.2.4 id_len	80
11.11 Dokumentacja struktury laviet_frame_t	80
11.11.1 Opis szczegółowy	81
11.11.2 Dokumentacja pól	81
11.11.2.1 counter	81
11.11.2.2 dst_id	81
11.11.2.3 flags	81
11.11.2.4 mac_tag	81
11.11.2.5 msg_id	81
11.11.2.6 payload	82
11.11.2.7 payload_len	82
11.11.2.8 src_id	82
11.11.2.9 ver_type	82
11.12 Dokumentacja struktury laviet_sha256_ctx_t	82
11.12.1 Opis szczegółowy	82
11.12.2 Dokumentacja pól	82
11.12.2.1 bitlen	82
11.12.2.2 data	82
11.12.2.3 datalen	83
11.12.2.4 state	83
11.13 Dokumentacja struktury lcd_main_msg_t	83
11.13.1 Opis szczegółowy	83
11.13.2 Dokumentacja pól	83
11.13.2.1 line_index	83
11.13.2.2 mode	83
11.13.2.3 text0	83
11.13.2.4 text1	83
11.13.2.5 text2	83
11.13.2.6 text3	83
11.13.2.7 type	84

11.14 Dokumentacja struktury <code>lcd_state_t</code>	84
11.14.1 Opis szczegółowy	84
11.14.2 Dokumentacja pól	84
11.14.2.1 <code>addr_7bit</code>	84
11.14.2.2 <code>backlight</code>	84
11.14.2.3 <code>initialized</code>	84
11.15 Dokumentacja struktury <code>led_array_ctx_t</code>	84
11.15.1 Opis szczegółowy	85
11.15.2 Dokumentacja pól	85
11.15.2.1 <code>active_brightness</code>	85
11.15.2.2 <code>duration_ms</code>	85
11.15.2.3 <code>effect</code>	85
11.15.2.4 <code>elapsed_ms</code>	85
11.15.2.5 <code>initialized</code>	85
11.15.2.6 <code>last_tick_ms</code>	85
11.15.2.7 <code>manual_brightness</code>	85
11.15.2.8 <code>mask</code>	86
11.15.2.9 <code>max_pct</code>	86
11.15.2.10 <code>min_pct</code>	86
11.15.2.11 <code>period_ms</code>	86
11.15.2.12 <code>phase_accum_ms</code>	86
11.15.2.13 <code>phase_base</code>	86
11.15.2.14 <code>step_ms</code>	86
11.15.2.15 <code>tick_period_ms</code>	86
11.15.2.16 <code>timer_running</code>	86
11.16 Dokumentacja struktury <code>menu_item_t</code>	86
11.16.1 Opis szczegółowy	86
11.16.2 Dokumentacja pól	87
11.16.2.1 <code>action</code>	87
11.16.2.2 <code>child_page</code>	87
11.16.2.3 <code>label</code>	87
11.17 Dokumentacja struktury <code>menu_notification_t</code>	87
11.17.1 Opis szczegółowy	87
11.17.2 Dokumentacja pól	87
11.17.2.1 <code>device_code</code>	87
11.17.2.2 <code>reply_device_code</code>	87
11.17.2.3 <code>rsi_dbm</code>	88
11.17.2.4 <code>text</code>	88
11.17.2.5 <code>type</code>	88
11.18 Dokumentacja struktury <code>menu_page_t</code>	88
11.18.1 Opis szczegółowy	88
11.18.2 Dokumentacja pól	88

11.18.2.1 item_count	88
11.18.2.2 items	88
11.18.2.3 parent	88
11.18.2.4 title	88
11.19 Dokumentacja struktury menu_radio_action_binding_t	89
11.19.1 Opis szczegółowy	89
11.19.2 Dokumentacja pól	89
11.19.2.1 action	89
11.19.2.2 err_text	89
11.19.2.3 modulation	89
11.19.2.4 ok_text	89
11.19.2.5 option	89
11.19.2.6 persist_lora_preset	89
11.19.2.7 set_modulation	89
11.19.2.8 set_option	89
11.19.2.9 value	90
11.20 Dokumentacja struktury menu_state_t	90
11.20.1 Opis szczegółowy	90
11.20.2 Dokumentacja pól	90
11.20.2.1 admin_unlock_until_ms	90
11.20.2.2 current_page	90
11.20.2.3 last_input_ms	90
11.20.2.4 led_mode	91
11.20.2.5 modal	91
11.20.2.6 operator_unlock_until_ms	91
11.20.2.7 pairing_network_mode	91
11.20.2.8 pending_action	91
11.20.2.9 pending_auth_level	91
11.20.2.10 pending_auth_page	91
11.20.2.11 pending_target_node_id	91
11.20.2.12 pin_digits	91
11.20.2.13 pin_index	91
11.20.2.14 popup_enabled	91
11.20.2.15 quick_reply_deadline_ms	91
11.20.2.16 quick_reply_last_seconds	92
11.20.2.17 quick_reply_src_id	92
11.20.2.18 quick_reply_text	92
11.20.2.19 selected_device_slot	92
11.20.2.20 selected_idx	92
11.20.2.21 send_target_action	92
11.20.2.22 transient_parent_page	92
11.20.2.23 tx_ack_waiting	92

11.20.2.24 tx_sent_text	92
11.21 Dokumentacja struktury radio_context_t	92
11.21.1 Opis szczegółowy	93
11.21.2 Dokumentacja pól	93
11.21.2.1 bus	93
11.21.2.2 callback	93
11.21.2.3 callback_ctx	93
11.21.2.4 dio_pending_mask	93
11.21.2.5 event_flags	93
11.21.2.6 hw	93
11.21.2.7 initialized	93
11.21.2.8 last_packet	93
11.21.2.9 lora	93
11.21.2.10 rx_single_timeout_symbols	94
11.21.2.11 state	94
11.21.2.12 tx_resume_state	94
11.22 Dokumentacja struktury radio_fsk_cfg_t	94
11.22.1 Opis szczegółowy	94
11.22.2 Dokumentacja pól	95
11.22.2.1 address_filter	95
11.22.2.2 bitrate_bps	95
11.22.2.3 crc_type	95
11.22.2.4 data_whitening	95
11.22.2.5 filter	95
11.22.2.6 frequency_hz	95
11.22.2.7 preamble_len	95
11.22.2.8 rx_bandwidth	95
11.22.2.9 shaping	96
11.22.2.10 sync_word	96
11.22.2.11 sync_word_len	96
11.22.2.12 tx_power_dbm	96
11.23 Dokumentacja struktury radio_fsk_context_t	96
11.23.1 Opis szczegółowy	96
11.23.2 Dokumentacja pól	96
11.23.2.1 bus	96
11.23.2.2 callback	97
11.23.2.3 callback_ctx	97
11.23.2.4 cfg	97
11.23.2.5 dio_pending_mask	97
11.23.2.6 event_flags	97
11.23.2.7 hw	97
11.23.2.8 initialized	97

11.23.2.9 last_packet	97
11.23.2.10 rx_single_deadline_ms	97
11.23.2.11 state	97
11.23.2.12 tx_deadline_ms	97
11.23.2.13 tx_resume_state	97
11.24 Dokumentacja struktury radio_gpio_t	98
11.24.1 Opis szczegółowy	98
11.24.2 Dokumentacja pól	98
11.24.2.1 pin	98
11.24.2.2 port	98
11.25 Dokumentacja struktury radio_hw_cfg_t	98
11.25.1 Opis szczegółowy	98
11.25.2 Dokumentacja pól	99
11.25.2.1 dio	99
11.25.2.2 hspi	99
11.25.2.3 nss	99
11.25.2.4 reset	99
11.25.2.5 spi_timeout_ms	99
11.26 Dokumentacja struktury radio_lora_cfg_t	99
11.26.1 Opis szczegółowy	100
11.26.2 Dokumentacja pól	100
11.26.2.1 bandwidth	100
11.26.2.2 coding_rate	100
11.26.2.3 crc_on	100
11.26.2.4 frequency_hz	100
11.26.2.5 implicit_header	100
11.26.2.6 invert_iq	101
11.26.2.7 payload_len	101
11.26.2.8 preamble_len	101
11.26.2.9 spreading_factor	101
11.26.2.10 sync_word	101
11.26.2.11 tx_power_dbm	101
11.27 Dokumentacja struktury radio_main_cmd_sync_t	101
11.27.1 Opis szczegółowy	101
11.27.2 Dokumentacja pól	102
11.27.2.1 done	102
11.27.2.2 result	102
11.27.2.3 runtime_cfg	102
11.28 Dokumentacja struktury radio_main_cmd_t	102
11.28.1 Opis szczegółowy	103
11.28.2 Dokumentacja pól	103
11.28.2.1 data	103

11.28.2.2 dst_id	103
11.28.2.3 enabled	103
11.28.2.4 group_id	103
11.28.2.5 id	103
11.28.2.6 len	103
11.28.2.7 msg_id	103
11.28.2.8 network_mode	103
11.28.2.9 option	103
11.28.2.10 sync	103
11.28.2.11 text	104
11.28.2.12 timeout_ms	104
11.28.2.13 value [1/2]	104
11.28.2.14 value [2/2]	104
11.29 Dokumentacja struktury radio_main_ctx_t	104
11.29.1 Opis szczegółowy	105
11.29.2 Dokumentacja pól	105
11.29.2.1 active	105
11.29.2.2 auto_ping_enabled	106
11.29.2.3 auto_ping_mode	106
11.29.2.4 auto_ping_period_ms	106
11.29.2.5 backend_cfg	106
11.29.2.6 backend_modulation_id	106
11.29.2.7 broadcast_group_aes_key	106
11.29.2.8 broadcast_group_epoch	106
11.29.2.9 broadcast_group_hmac_key	106
11.29.2.10 broadcast_group_key_valid	106
11.29.2.11 broadcast_group_pending_epoch	106
11.29.2.12 broadcast_group_pending_key	106
11.29.2.13 broadcast_group_pending_mask	106
11.29.2.14 broadcast_group_pending_valid	107
11.29.2.15 coding_enabled	107
11.29.2.16 counter	107
11.29.2.17 crypto_ready	107
11.29.2.18 deadline_ms	107
11.29.2.19 expires_ms	107
11.29.2.20 fh_enabled	107
11.29.2.21 fh_period_ms	107
11.29.2.22 fsk_cfg	107
11.29.2.23 gateway_key_mode	107
11.29.2.24 gateway_key_mode_known	107
11.29.2.25 gateway_pair_code	107
11.29.2.26 gateway_pair_code_len	108

11.29.2.27 gateway_pair_code_valid	108
11.29.2.28 gateway_rx_counter	108
11.29.2.29 gateway_tx_counter	108
11.29.2.30 hop_idx	108
11.29.2.31 hw	108
11.29.2.32 initialized	108
11.29.2.33 last_error_text	108
11.29.2.34 last_hop_ms	108
11.29.2.35 last_ping_ms	108
11.29.2.36 lora_cfg	108
11.29.2.37 lora_preset	108
11.29.2.38 modulation_id	109
11.29.2.39 msg_id	109
11.29.2.40 next_msg_id	109
11.29.2.41 node_id	109
11.29.2.42 ook_cfg	109
11.29.2.43 pairing_active	109
11.29.2.44 pairing_network_mode	109
11.29.2.45 pairing_outgoing_code	109
11.29.2.46 pairing_outgoing_code_len	109
11.29.2.47 pairing_outgoing_network	109
11.29.2.48 pairing_outgoing_pending	109
11.29.2.49 pairing_pending	109
11.29.2.50 pairing_pending_code	110
11.29.2.51 pairing_pending_code_len	110
11.29.2.52 pairing_pending_network	110
11.29.2.53 pairing_pending_node	110
11.29.2.54 pairing_until_ms	110
11.29.2.55 peer_id	110
11.29.2.56 raw	110
11.29.2.57 raw_len	110
11.29.2.58 retries_done	110
11.29.2.59 timed_out	110
11.29.2.60 tx_deadline_ms	110
11.29.2.61 tx_in_progress	110
11.29.2.62 tx_silent	111
11.29.2.63 valid	111
11.29.2.64 wait_ms	111
11.30 Dokumentacja struktury radio_main_fsk_cfg_t	111
11.30.1 Opis szczegółowy	111
11.30.2 Dokumentacja pól	112
11.30.2.1 address_filter	112

11.30.2.2	bitrate_bps	112
11.30.2.3	crc_type	112
11.30.2.4	data_whitening	112
11.30.2.5	filter	112
11.30.2.6	frequency_hz	112
11.30.2.7	preamble_len	112
11.30.2.8	rx_bandwidth	112
11.30.2.9	shaping	113
11.30.2.10	sync_word	113
11.30.2.11	sync_word_len	113
11.30.2.12	tx_power_dbm	113
11.31	Dokumentacja struktury radio_main_ook_cfg_t	113
11.31.1	Opis szczegółowy	114
11.31.2	Dokumentacja pól	114
11.31.2.1	bitrate_bps	114
11.31.2.2	frequency_hz	114
11.31.2.3	preamble_len	114
11.31.2.4	rx_bandwidth	114
11.31.2.5	sync_word	114
11.31.2.6	sync_word_len	114
11.31.2.7	threshold	114
11.31.2.8	threshold_value	114
11.31.2.9	tx_power_dbm	115
11.32	Dokumentacja struktury radio_main_runtime_cfg_t	115
11.32.1	Opis szczegółowy	115
11.32.2	Dokumentacja pól	115
11.32.2.1	active_modulation	115
11.32.2.2	auto_ping_enabled	116
11.32.2.3	auto_ping_mode	116
11.32.2.4	auto_ping_period_ms	116
11.32.2.5	coding_enabled	116
11.32.2.6	fh_enabled	116
11.32.2.7	fh_period_ms	116
11.32.2.8	fsk	116
11.32.2.9	lora	116
11.32.2.10	ook	116
11.33	Dokumentacja struktury radio_ook_cfg_t	117
11.33.1	Opis szczegółowy	117
11.33.2	Dokumentacja pól	117
11.33.2.1	bitrate_bps	117
11.33.2.2	frequency_hz	117
11.33.2.3	preamble_len	117

11.33.2.4 rx_bandwidth	118
11.33.2.5 sync_word	118
11.33.2.6 sync_word_len	118
11.33.2.7 threshold	118
11.33.2.8 threshold_value	118
11.33.2.9 tx_power_dbm	118
11.34 Dokumentacja struktury radio_ook_context_t	118
11.34.1 Opis szczegółowy	119
11.34.2 Dokumentacja pól	119
11.34.2.1 bus	119
11.34.2.2 callback	119
11.34.2.3 callback_ctx	119
11.34.2.4 cfg	119
11.34.2.5 dio_pending_mask	119
11.34.2.6 event_flags	119
11.34.2.7 hw	119
11.34.2.8 initialized	119
11.34.2.9 last_packet	119
11.34.2.10 rx_single_deadline_ms	119
11.34.2.11 state	120
11.34.2.12 tx_deadline_ms	120
11.34.2.13 tx_resume_state	120
11.35 Dokumentacja struktury radio_packet_t	120
11.35.1 Opis szczegółowy	120
11.35.2 Dokumentacja pól	120
11.35.2.1 data	120
11.35.2.2 length	121
11.35.2.3 rssi_dbm	121
11.35.2.4 snr_db	121
11.35.2.5 timestamp_ms	121
11.35.2.6 valid	121
11.36 Dokumentacja struktury security_cmd_sync_t	121
11.36.1 Opis szczegółowy	121
11.36.2 Dokumentacja pól	121
11.36.2.1 done	121
11.36.2.2 result	122
11.36.2.3 runtime_out	122
11.36.2.4 trusted_out	122
11.37 Dokumentacja struktury security_cmd_t	122
11.37.1 Opis szczegółowy	123
11.37.2 Dokumentacja pól	123
11.37.2.1 cfg	123

11.37.2.2 code	123
11.37.2.3 code_len	123
11.37.2.4 enabled	123
11.37.2.5 gateway_slot	123
11.37.2.6 id	123
11.37.2.7 index	123
11.37.2.8 len	123
11.37.2.9 mode	123
11.37.2.10 node_id	123
11.37.2.11 payload	123
11.37.2.12 preset_id	124
11.37.2.13 rssi_dbm	124
11.37.2.14 sync	124
11.37.2.15 value	124
11.38 Dokumentacja struktury security_key_wire_t	124
11.38.1 Opis szczegółowy	124
11.38.2 Dokumentacja pól	124
11.38.2.1 crc	124
11.38.2.2 magic	124
11.38.2.3 seed	124
11.39 Dokumentacja struktury security_radio_store_wire_t	125
11.39.1 Opis szczegółowy	125
11.39.2 Dokumentacja pól	125
11.39.2.1 magic	125
11.39.2.2 packed	125
11.39.2.3 version	125
11.40 Dokumentacja struktury security_runtime_cfg_t	125
11.40.1 Opis szczegółowy	126
11.40.2 Dokumentacja pól	126
11.40.2.1 active_modulation	126
11.40.2.2 auto_ping_enabled	126
11.40.2.3 auto_ping_mode	126
11.40.2.4 auto_ping_period_ms	126
11.40.2.5 coding_enabled	126
11.40.2.6 fh_enabled	126
11.40.2.7 fh_period_ms	126
11.40.2.8 fsk	127
11.40.2.9 lora	127
11.40.2.10 lora_preset	127
11.40.2.11 notify_mode	127
11.40.2.12 ook	127
11.40.2.13 radio_profiles_persisted	127

11.41 Dokumentacja struktury <code>security_settings_wire_t</code>	127
11.41.1 Opis szczegółowy	127
11.41.2 Dokumentacja pól	128
11.41.2.1 <code>crc</code>	128
11.41.2.2 <code>flags</code>	128
11.41.2.3 <code>lora_preset</code>	128
11.41.2.4 <code>magic</code>	128
11.41.2.5 <code>notify_mode</code>	128
11.41.2.6 <code>reserved0</code>	128
11.41.2.7 <code>reserved1</code>	128
11.41.2.8 <code>version</code>	128
11.42 Dokumentacja struktury <code>security_store_wire_t</code>	128
11.42.1 Opis szczegółowy	128
11.42.2 Dokumentacja pól	129
11.42.2.1 <code>flags</code>	129
11.42.2.2 <code>lora_preset</code>	129
11.42.2.3 <code>magic</code>	129
11.42.2.4 <code>modulation_fh</code>	129
11.42.2.5 <code>notify_mode</code>	129
11.42.2.6 <code>seed</code>	129
11.42.2.7 <code>version</code>	129
11.43 Dokumentacja struktury <code>security_trusted_entry_t</code>	129
11.43.1 Opis szczegółowy	129
11.43.2 Dokumentacja pól	129
11.43.2.1 <code>code</code>	129
11.43.2.2 <code>code_len</code>	130
11.43.2.3 <code>in_use</code>	130
11.43.2.4 <code>is_master</code>	130
11.43.2.5 <code>node_id</code>	130
11.44 Dokumentacja struktury <code>st33ktpm2x_cfg_t</code>	130
11.44.1 Opis szczegółowy	131
11.44.2 Dokumentacja pól	131
11.44.2.1 <code>burst_timeout_ms</code>	131
11.44.2.2 <code>davint_active_state</code>	131
11.44.2.3 <code>davint_pin</code>	131
11.44.2.4 <code>davint_port</code>	131
11.44.2.5 <code>hi2c</code>	131
11.44.2.6 <code>i2c_addr_7bit</code>	131
11.44.2.7 <code>io_timeout_ms</code>	132
11.44.2.8 <code>locality_timeout_ms</code>	132
11.44.2.9 <code>pp_active_state</code>	132
11.44.2.10 <code>pp_pin</code>	132

11.44.2.11 pp_port	132
11.44.2.12 reset_pin	132
11.44.2.13 reset_port	132
11.44.2.14 reset_pulse_ms	132
11.44.2.15 reset_recovery_ms	132
11.45 Dokumentacja struktury st33ktpm2x_t	133
11.45.1 Opis szczegółowy	133
11.45.2 Dokumentacja pól	133
11.45.2.1 cfg	133
11.45.2.2 initialized	133
11.45.2.3 locality0_acquired	133
11.46 Dokumentacja struktury sx1276_bus_t	134
11.46.1 Opis szczegółowy	134
11.46.2 Dokumentacja pól	134
11.46.2.1 hspi	134
11.46.2.2 nss_pin	134
11.46.2.3 nss_port	134
11.46.2.4 spi_timeout_ms	134
11.47 Dokumentacja struktury trusted_info_t	134
11.47.1 Opis szczegółowy	135
11.47.2 Dokumentacja pól	135
11.47.2.1 code	135
11.47.2.2 code_len	135
11.47.2.3 in_use	135
11.47.2.4 is_master	135
11.47.2.5 node_id	135
11.47.2.6 slot	135
12 Dokumentacja plików	137
12.1 app.c	137
12.2 Dokumentacja pliku Core/App/app.h	141
12.2.1 Opis szczegółowy	142
12.2.2 Dokumentacja funkcji	142
12.2.2.1 app_freertos_init()	142
12.2.2.2 app_i2c_lock()	142
12.2.2.3 app_i2c_master_receive()	142
12.2.2.4 app_i2c_master_transmit()	143
12.2.2.5 app_i2c_mem_read()	143
12.2.2.6 app_i2c_mem_write()	144
12.2.2.7 app_i2c_unlock()	144
12.2.2.8 app_init()	145
12.2.2.9 app_main()	145

12.3 app.h	145
12.4 bmp280.c	146
12.5 Dokumentacja pliku Core/App/bmp280_lib/bmp280.h	149
12.5.1 Opis szczegółowy	150
12.5.2 Dokumentacja definicji	150
12.5.2.1 BME280_CHIP_ID	150
12.5.2.2 BMP280_CHIP_ID	151
12.5.2.3 BMP280_I2C_ADDRESS_0	151
12.5.2.4 BMP280_I2C_ADDRESS_1	151
12.5.3 Dokumentacja typów wyliczanych	151
12.5.3.1 BMP280_Filter	151
12.5.3.2 BMP280_Mode	151
12.5.3.3 BMP280_Oversampling	151
12.5.3.4 BMP280_StandbyTime	152
12.5.4 Dokumentacja funkcji	152
12.5.4.1 bmp280_force_measurement()	152
12.5.4.2 bmp280_init()	152
12.5.4.3 bmp280_init_default_params()	152
12.5.4.4 bmp280_is_measuring()	153
12.5.4.5 bmp280_read_fixed()	153
12.5.4.6 bmp280_read_float()	154
12.6 bmp280.h	154
12.7 bmp280_api.c	155
12.8 Dokumentacja pliku Core/App/bmp280_lib/bmp280_api.h	159
12.8.1 Opis szczegółowy	159
12.8.2 Dokumentacja funkcji	159
12.8.2.1 bmp280_api_get_last_data()	159
12.8.2.2 bmp280_api_init()	160
12.8.2.3 bmp280_api_measure_all()	160
12.8.2.4 bmp280_api_measure_pressure()	160
12.8.2.5 bmp280_api_measure_temperature()	161
12.8.2.6 bmp280_api_sleep()	161
12.9 bmp280_api.h	161
12.10 bmp280_main.c	162
12.11 Dokumentacja pliku Core/App/bmp280_main.h	164
12.11.1 Opis szczegółowy	164
12.11.2 Dokumentacja funkcji	164
12.11.2.1 bmp280_main_create_task()	164
12.11.2.2 bmp280_main_get_last()	164
12.12 bmp280_main.h	164
12.13 button_main.c	165
12.14 Dokumentacja pliku Core/App/button_main.h	167

12.14.1 Opis szczegółowy	168
12.14.2 Dokumentacja typów wyliczanych	168
12.14.2.1 button_event_t	168
12.14.3 Dokumentacja funkcji	168
12.14.3.1 button_main_create_task()	168
12.14.3.2 button_main_get_event()	168
12.15 button_main.h	169
12.16 Dokumentacja pliku Core/App/i2c_mem_store_lib/i2c_mem_store.c	169
12.16.1 Opis szczegółowy	171
12.16.2 Dokumentacja definicji	171
12.16.2.1 I2C_MEM_STORE_LOG_CRC_OFFSET	171
12.16.2.2 I2C_MEM_STORE_LOG_HEADER_SIZE	171
12.16.2.3 I2C_MEM_STORE_LOG_MAGIC	171
12.16.2.4 I2C_MEM_STORE_LOG_START_ADDR	171
12.16.2.5 I2C_MEM_STORE_META_MAGIC	172
12.16.2.6 I2C_MEM_STORE_META_MIRROR_ADDR	172
12.16.2.7 I2C_MEM_STORE_META_PRIMARY_ADDR	172
12.16.2.8 I2C_MEM_STORE_META_VERSION	172
12.16.2.9 I2C_MEM_STORE_META_WIRE_SIZE	172
12.16.2.10 I2C_MEM_STORE_SECRET_CRC_OFFSET	172
12.16.2.11 I2C_MEM_STORE_SECRET_HEADER_SIZE	172
12.16.2.12 I2C_MEM_STORE_SECRET_MAGIC	172
12.16.3 Dokumentacja funkcji	172
12.16.3.1 i2c_mem_store_append_message()	172
12.16.3.2 i2c_mem_store_crc16_ccitt()	173
12.16.3.3 i2c_mem_store_default_cfg()	173
12.16.3.4 i2c_mem_store_default_cfg_m24c01r()	173
12.16.3.5 i2c_mem_store_format()	174
12.16.3.6 i2c_mem_store_get_log_count()	174
12.16.3.7 i2c_mem_store_hal_to_status()	174
12.16.3.8 i2c_mem_store_init()	174
12.16.3.9 i2c_mem_store_layout_valid()	175
12.16.3.10 i2c_mem_store_le16_read()	175
12.16.3.11 i2c_mem_store_le16_write()	175
12.16.3.12 i2c_mem_store_le32_read()	175
12.16.3.13 i2c_mem_store_le32_write()	175
12.16.3.14 i2c_mem_store_log_base_addr()	176
12.16.3.15 i2c_mem_store_log_slot_addr()	176
12.16.3.16 i2c_mem_store_meta_decode()	176
12.16.3.17 i2c_mem_store_meta_encode()	176
12.16.3.18 i2c_mem_store_meta_load()	176
12.16.3.19 i2c_mem_store_meta_save()	176

12.16.3.20	i2c_mem_store_min_u16()	177
12.16.3.21	i2c_mem_store_read_message()	177
12.16.3.22	i2c_mem_store_read_raw()	177
12.16.3.23	i2c_mem_store_secret_base_addr()	177
12.16.3.24	i2c_mem_store_secret_erase()	178
12.16.3.25	i2c_mem_store_secret_read()	178
12.16.3.26	i2c_mem_store_secret_slot_addr()	179
12.16.3.27	i2c_mem_store_secret_write()	179
12.16.3.28	i2c_mem_store_trusted_device_read()	179
12.16.3.29	i2c_mem_store_trusted_device_write()	180
12.16.3.30	i2c_mem_store_write_raw()	180
12.16.3.31	i2c_mem_store_xtea_ctr_crypt()	180
12.16.3.32	i2c_mem_store_xtea_encrypt_block()	181
12.17	i2c_mem_store.c	181
12.18	Dokumentacja pliku Core/App/i2c_mem_store_lib/i2c_mem_store.h	190
12.18.1	Opis szczegółowy	192
12.18.2	Dokumentacja definicji	192
12.18.2.1	I2C_MEM_STORE_I2C_ADDR_DEFAULT	192
12.18.2.2	I2C_MEM_STORE_LOG_PAYLOAD_MAX	192
12.18.2.3	I2C_MEM_STORE_LOG_RECORD_SIZE	192
12.18.2.4	I2C_MEM_STORE_PAGE_BYTES_DEFAULT	192
12.18.2.5	I2C_MEM_STORE_SECRET_AREA_BYTES_DEFAULT	192
12.18.2.6	I2C_MEM_STORE_SECRET_PAYLOAD_MAX	192
12.18.2.7	I2C_MEM_STORE_SECRET_SLOT_SIZE	192
12.18.2.8	I2C_MEM_STORE_TOTAL_BYTES_DEFAULT	192
12.18.2.9	I2C_MEM_STORE_TRUSTED_CODE_MAX	192
12.18.2.10	I2C_MEM_STORE_TRUSTED_ID_MAX	193
12.18.2.11	I2C_MEM_STORE_WRITE_CYCLE_MS_DEFAULT	193
12.18.3	Dokumentacja typów wyliczanych	193
12.18.3.1	i2c_mem_store_status_t	193
12.18.4	Dokumentacja funkcji	193
12.18.4.1	i2c_mem_store_append_message()	193
12.18.4.2	i2c_mem_store_default_cfg()	194
12.18.4.3	i2c_mem_store_default_cfg_m24c01r()	194
12.18.4.4	i2c_mem_store_format()	194
12.18.4.5	i2c_mem_store_get_log_count()	195
12.18.4.6	i2c_mem_store_init()	195
12.18.4.7	i2c_mem_store_read_message()	195
12.18.4.8	i2c_mem_store_secret_erase()	196
12.18.4.9	i2c_mem_store_secret_read()	196
12.18.4.10	i2c_mem_store_secret_write()	197
12.18.4.11	i2c_mem_store_trusted_device_read()	197

12.18.4.12 i2c_mem_store_trusted_device_write()	198
12.19 i2c_mem_store.h	198
12.20 Dokumentacja pliku Core/App/laviet_crypto.c	200
12.20.1 Opis szczegółowy	202
12.20.2 Dokumentacja definicji	202
12.20.2.1 LAVIET_CRYPTO_TIMEOUT_MS	202
12.20.2.2 LAVIET_TEST_FIXED_FRAME_MAC	202
12.20.2.3 LAVIET_TEST_FIXED_FRAME_MAC_BYTE	202
12.20.3 Dokumentacja funkcji	202
12.20.3.1 laviet_aes_add_round_key()	202
12.20.3.2 laviet_aes_ctr_crypt()	202
12.20.3.3 laviet_aes_ctr_crypt_hw_locked()	203
12.20.3.4 laviet_aes_ctr_crypt_sw()	203
12.20.3.5 laviet_aes_encrypt_block()	203
12.20.3.6 laviet_aes_key_expansion()	203
12.20.3.7 laviet_aes_mix_columns()	203
12.20.3.8 laviet_aes_shift_rows()	204
12.20.3.9 laviet_aes_sub_bytes()	204
12.20.3.10 laviet_aes_xtime()	204
12.20.3.11 laviet_crypto_init()	204
12.20.3.12 laviet_crypto_lock()	204
12.20.3.13 laviet_crypto_log_mismatch()	204
12.20.3.14 laviet_crypto_self_test_locked()	205
12.20.3.15 laviet_crypto_unlock()	205
12.20.3.16 laviet_frame_hmac_sha256()	205
12.20.3.17 laviet_hal_status_text()	205
12.20.3.18 laviet_hmac_sha256()	205
12.20.3.19 laviet_hmac_sha256_hw_locked()	206
12.20.3.20 laviet_hmac_sha256_sw()	206
12.20.3.21 laviet_mac_equal()	206
12.20.3.22 laviet_make_counter_block()	207
12.20.3.23 laviet_rotr32()	207
12.20.3.24 laviet_secure_zero()	207
12.20.3.25 laviet_sha256_final()	207
12.20.3.26 laviet_sha256_init()	207
12.20.3.27 laviet_sha256_transform()	208
12.20.3.28 laviet_sha256_update()	208
12.20.4 Dokumentacja zmiennych	208
12.20.4.1 s_aes_rcon	208
12.20.4.2 s_aes_sbox	208
12.20.4.3 s_crypto_hw_ready	208
12.20.4.4 s_crypto_mutex	208

12.20.4.5 s_sha256_k	208
12.21 laviet_crypto.c	209
12.22 Dokumentacja pliku Core/App/laviet_crypto.h	222
12.22.1 Opis szczegółowy	223
12.22.2 Dokumentacja definicji	223
12.22.2.1 LAVIET_AES_KEY_LEN	223
12.22.2.2 LAVIET_HMAC_KEY_LEN	223
12.22.2.3 LAVIET_SHA256_LEN	223
12.22.3 Dokumentacja funkcji	224
12.22.3.1 laviet_aes_ctr_crypt()	224
12.22.3.2 laviet_crypto_init()	224
12.22.3.3 laviet_frame_hmac_sha256()	224
12.22.3.4 laviet_hmac_sha256()	225
12.22.3.5 laviet_mac_equal()	225
12.22.3.6 laviet_secure_zero()	226
12.23 laviet_crypto.h	226
12.24 Dokumentacja pliku Core/App/laviet_frame.c	226
12.24.1 Opis szczegółowy	227
12.24.2 Dokumentacja funkcji	227
12.24.2.1 laviet_be16_read()	227
12.24.2.2 laviet_be16_write()	228
12.24.2.3 laviet_be32_read()	228
12.24.2.4 laviet_be32_write()	228
12.24.2.5 laviet_frame_build_mac_input()	228
12.24.2.6 laviet_frame_decode()	228
12.24.2.7 laviet_frame_encode()	229
12.24.2.8 laviet_frame_is_broadcast()	229
12.24.2.9 laviet_frame_read_ack_payload()	230
12.24.2.10 laviet_frame_type()	230
12.24.2.11 laviet_frame_type_known()	230
12.24.2.12 laviet_frame_validate_plain()	231
12.24.2.13 laviet_frame_ver_type()	231
12.24.2.14 laviet_frame_version_from_ver_type()	231
12.24.2.15 laviet_frame_write_ack_payload()	231
12.24.2.16 laviet_local_node_id()	232
12.25 laviet_frame.c	232
12.26 Dokumentacja pliku Core/App/laviet_frame.h	237
12.26.1 Opis szczegółowy	239
12.26.2 Dokumentacja definicji	239
12.26.2.1 LAVIET_ACK_PAYLOAD_LEN	239
12.26.2.2 LAVIET_BROADCAST_ID	239
12.26.2.3 LAVIET_COUNTER_SYNC_PAYLOAD_LEN	239

12.26.2.4 LAVIET_ERROR_PAYLOAD_LEN	240
12.26.2.5 LAVIET_FLAG_ACK_REQUIRED	240
12.26.2.6 LAVIET_FLAG_BROADCAST	240
12.26.2.7 LAVIET_FLAG_CONFIG_ACCESS	240
12.26.2.8 LAVIET_FLAG_COUNTER_OVERRIDE	240
12.26.2.9 LAVIET_FLAG_ENCRYPTED	240
12.26.2.10 LAVIET_FLAG_IS_ACK	240
12.26.2.11 LAVIET_FLAG_KEY_UPDATE	240
12.26.2.12 LAVIET_FLAG_PAIRING	240
12.26.2.13 LAVIET_FRAME_HEADER_LEN	241
12.26.2.14 LAVIET_FRAME_MAX_LEN	241
12.26.2.15 LAVIET_FRAME_MIN_LEN	241
12.26.2.16 LAVIET_FRAME_VERSION	241
12.26.2.17 LAVIET_GATEWAY_ID	241
12.26.2.18 LAVIET_MAC_TAG_LEN	241
12.26.2.19 LAVIET_MAX_PAYLOAD	241
12.26.2.20 LAVIET_PAIR_PAYLOAD_LEN	241
12.26.3 Dokumentacja typów wyliczanych	242
12.26.3.1 laviet_frame_status_t	242
12.26.3.2 laviet_frame_type_t	242
12.26.4 Dokumentacja funkcji	242
12.26.4.1 laviet_frame_build_mac_input()	242
12.26.4.2 laviet_frame_decode()	243
12.26.4.3 laviet_frame_encode()	243
12.26.4.4 laviet_frame_is_broadcast()	244
12.26.4.5 laviet_frame_read_ack_payload()	244
12.26.4.6 laviet_frame_type()	245
12.26.4.7 laviet_frame_validate_plain()	245
12.26.4.8 laviet_frame_ver_type()	245
12.26.4.9 laviet_frame_write_ack_payload()	246
12.26.4.10 laviet_local_node_id()	246
12.27 laviet_frame.h	246
12.28 lcd.c	248
12.29 Dokumentacja pliku Core/App/lcd_library/lcd.h	255
12.29.1 Opis szczegółowy	255
12.29.2 Dokumentacja funkcji	255
12.29.2.1 lcd_animation_hello_beko()	255
12.29.2.2 lcd_backlight()	255
12.29.2.3 lcd_clear()	256
12.29.2.4 lcd_demo()	256
12.29.2.5 lcd_init()	256
12.29.2.6 lcd_set_cursor()	256

12.29.2.7	lcd_write_line()	256
12.29.2.8	lcd_write_string()	257
12.30	lcd.h	257
12.31	lcd_main.c	257
12.32	Dokumentacja pliku Core/App/lcd_main.h	265
12.32.1	Opis szczegółowy	266
12.32.2	Dokumentacja typów wyliczanych	266
12.32.2.1	lcd_main_mode_t	266
12.32.3	Dokumentacja funkcji	266
12.32.3.1	lcd_main_create_task()	266
12.32.3.2	lcd_main_monitor_scroll_down()	266
12.32.3.3	lcd_main_monitor_scroll_up()	266
12.32.3.4	lcd_main_push_message()	267
12.32.3.5	lcd_main_push_message_from()	267
12.32.3.6	lcd_main_set_line()	267
12.32.3.7	lcd_main_set_lines()	268
12.32.3.8	lcd_main_set_mode()	268
12.32.3.9	lcd_main_show_boot_hello()	268
12.32.3.10	lcd_main_show_menu()	268
12.32.3.11	lcd_main_show_popup()	269
12.33	lcd_main.h	269
12.34	Dokumentacja pliku Core/App/led_array_lib/led_array_lib.c	270
12.34.1	Opis szczegółowy	272
12.34.2	Dokumentacja definicji	272
12.34.2.1	LED_ARRAY_COUNT	272
12.34.2.2	LED_ARRAY_PWM_FREQ_HZ	272
12.34.2.3	LED_ARRAY_PWM_STEPS	272
12.34.2.4	LED_ARRAY_RAINBOW_PHASE_INC	272
12.34.2.5	LED_ARRAY_VALID_MASK	272
12.34.3	Dokumentacja funkcji	272
12.34.3.1	led_array_apply_outputs()	272
12.34.3.2	led_array_clamp_pct()	272
12.34.3.3	led_array_deinit()	272
12.34.3.4	led_array_effect_breath_update()	273
12.34.3.5	led_array_effect_fade_update()	273
12.34.3.6	led_array_effect_none()	273
12.34.3.7	led_array_effect_rainbow_update()	273
12.34.3.8	led_array_fire_fade_out()	273
12.34.3.9	led_array_get_effect()	273
12.34.3.10	led_array_get_tim_clock()	274
12.34.3.11	led_array_hw_init()	274
12.34.3.12	led_array_init()	274

12.34.3.13	<code>led_array_is_initialized()</code>	274
12.34.3.14	<code>led_array_mask_valid()</code>	274
12.34.3.15	<code>led_array_now_ms()</code>	274
12.34.3.16	<code>led_array_off()</code>	275
12.34.3.17	<code>led_array_on()</code>	275
12.34.3.18	<code>led_array_process()</code>	275
12.34.3.19	<code>led_array_set_brightness()</code>	275
12.34.3.20	<code>led_array_set_hw_duty()</code>	276
12.34.3.21	<code>led_array_start_breath()</code>	276
12.34.3.22	<code>led_array_start_rainbow()</code>	276
12.34.3.23	<code>led_array_stop_effect()</code>	277
12.34.3.24	<code>led_array_tick_core()</code>	277
12.34.3.25	<code>led_array_timer_base_init()</code>	277
12.34.3.26	<code>led_array_timer_deinit()</code>	277
12.34.3.27	<code>led_array_timer_init()</code>	277
12.34.3.28	<code>led_array_timer_pwm_ch_init()</code>	278
12.34.3.29	<code>led_array_timer_tick()</code>	278
12.34.3.30	<code>led_array_tri8()</code>	278
12.34.3.31	<code>led_array_write_pin()</code>	278
12.34.4	Dokumentacja zmiennych	278
12.34.4.1	<code>s_led</code>	278
12.35	<code>led_array_lib.c</code>	278
12.36	Dokumentacja pliku <code>Core/App/led_array_lib/led_array_lib.h</code>	287
12.36.1	Opis szczegółowy	288
12.36.2	Dokumentacja typów wyliczanych	288
12.36.2.1	<code>led_array_effect_t</code>	288
12.36.2.2	<code>led_array_led_mask_t</code>	288
12.36.2.3	<code>led_array_status_t</code>	288
12.36.3	Dokumentacja funkcji	288
12.36.3.1	<code>led_array_deinit()</code>	288
12.36.3.2	<code>led_array_fire_fade_out()</code>	289
12.36.3.3	<code>led_array_get_effect()</code>	289
12.36.3.4	<code>led_array_init()</code>	289
12.36.3.5	<code>led_array_is_initialized()</code>	289
12.36.3.6	<code>led_array_off()</code>	290
12.36.3.7	<code>led_array_on()</code>	290
12.36.3.8	<code>led_array_process()</code>	290
12.36.3.9	<code>led_array_set_brightness()</code>	290
12.36.3.10	<code>led_array_start_breath()</code>	291
12.36.3.11	<code>led_array_start_rainbow()</code>	291
12.36.3.12	<code>led_array_stop_effect()</code>	291
12.36.3.13	<code>led_array_timer_deinit()</code>	292

12.36.3.14 led_array_timer_init()	292
12.36.3.15 led_array_timer_tick()	292
12.37 led_array_lib.h	292
12.38 Dokumentacja pliku Core/App/led_array_lib/led_array_lib_config.h	293
12.38.1 Opis szczegółowy	294
12.38.2 Dokumentacja definicji	294
12.38.2.1 LED_ARRAY_LIB_DEFAULT_TICK_MS	294
12.38.2.2 LED_ARRAY_LIB_OWNS_HAL_SYSTICK_CALLBACK	294
12.39 led_array_lib_config.h	294
12.40 led_array_main.c	294
12.41 Dokumentacja pliku Core/App/led_array_main.h	295
12.41.1 Opis szczegółowy	295
12.41.2 Dokumentacja funkcji	295
12.41.2.1 led_array_main_create_task()	295
12.42 led_array_main.h	295
12.43 menu_main.c	296
12.44 Dokumentacja pliku Core/App/menu_main.h	349
12.44.1 Opis szczegółowy	349
12.44.2 Dokumentacja typów wyliczanych	349
12.44.2.1 menu_notification_type_t	349
12.44.3 Dokumentacja funkcji	350
12.44.3.1 menu_main_create_task()	350
12.44.3.2 menu_main_post_notification()	350
12.45 menu_main.h	350
12.46 Dokumentacja pliku Core/App/pager_config.h	351
12.46.1 Opis szczegółowy	351
12.46.2 Dokumentacja definicji	351
12.46.2.1 PAGER_CONFIG_BMP280_MOUNTED	351
12.46.2.2 PAGER_CONFIG_UART_SENSITIVE_LOGS	351
12.46.2.3 SERVICE_UART	351
12.47 pager_config.h	351
12.48 Dokumentacja pliku Core/App/radio_lib/common/sx1276/radio_sx1276.c	352
12.48.1 Opis szczegółowy	353
12.48.2 Dokumentacja funkcji	353
12.48.2.1 sx1276_clear_irq_flags()	353
12.48.2.2 sx1276_deselect()	353
12.48.2.3 sx1276_get_irq_flags()	353
12.48.2.4 sx1276_get_packet_rssi_raw()	353
12.48.2.5 sx1276_get_packet_snr_raw()	354
12.48.2.6 sx1276_get_rx_current_addr()	354
12.48.2.7 sx1276_get_rx_nb_bytes()	354
12.48.2.8 sx1276_get_version()	354

12.48.2.9 <code>sx1276_map_dio()</code>	354
12.48.2.10 <code>sx1276_read_burst()</code>	354
12.48.2.11 <code>sx1276_read_fifo()</code>	355
12.48.2.12 <code>sx1276_read_reg()</code>	355
12.48.2.13 <code>sx1276_select()</code>	356
12.48.2.14 <code>sx1276_set_fifo_addr_ptr()</code>	356
12.48.2.15 <code>sx1276_set_fifo_base_addrs()</code>	356
12.48.2.16 <code>sx1276_set_frequency()</code>	356
12.48.2.17 <code>sx1276_set_invert_iq()</code>	356
12.48.2.18 <code>sx1276_set_lora_modem()</code>	356
12.48.2.19 <code>sx1276_set_op_mode()</code>	357
12.48.2.20 <code>sx1276_set_pa_output_power()</code>	357
12.48.2.21 <code>sx1276_set_payload_length()</code>	357
12.48.2.22 <code>sx1276_set_preamble()</code>	357
12.48.2.23 <code>sx1276_set_symbol_timeout()</code>	357
12.48.2.24 <code>sx1276_set_sync_word()</code>	358
12.48.2.25 <code>sx1276_write_burst()</code>	358
12.48.2.26 <code>sx1276_write_fifo()</code>	358
12.48.2.27 <code>sx1276_write_reg()</code>	358
12.49 <code>radio_sx1276.c</code>	359
12.50 Dokumentacja pliku <code>Core/App/radio_lib/common/sx1276/radio_sx1276.h</code>	363
12.50.1 Opis szczegółowy	364
12.50.2 Dokumentacja funkcji	365
12.50.2.1 <code>sx1276_clear_irq_flags()</code>	365
12.50.2.2 <code>sx1276_get_irq_flags()</code>	365
12.50.2.3 <code>sx1276_get_packet_rssi_raw()</code>	365
12.50.2.4 <code>sx1276_get_packet_snr_raw()</code>	365
12.50.2.5 <code>sx1276_get_rx_current_addr()</code>	365
12.50.2.6 <code>sx1276_get_rx_nb_bytes()</code>	365
12.50.2.7 <code>sx1276_get_version()</code>	366
12.50.2.8 <code>sx1276_map_dio()</code>	366
12.50.2.9 <code>sx1276_read_burst()</code>	366
12.50.2.10 <code>sx1276_read_fifo()</code>	366
12.50.2.11 <code>sx1276_read_reg()</code>	367
12.50.2.12 <code>sx1276_set_fifo_addr_ptr()</code>	367
12.50.2.13 <code>sx1276_set_fifo_base_addrs()</code>	367
12.50.2.14 <code>sx1276_set_frequency()</code>	367
12.50.2.15 <code>sx1276_set_invert_iq()</code>	368
12.50.2.16 <code>sx1276_set_lora_modem()</code>	368
12.50.2.17 <code>sx1276_set_op_mode()</code>	368
12.50.2.18 <code>sx1276_set_pa_output_power()</code>	368
12.50.2.19 <code>sx1276_set_payload_length()</code>	368

12.50.2.20 sx1276_set_preamble()	369
12.50.2.21 sx1276_set_symbol_timeout()	369
12.50.2.22 sx1276_set_sync_word()	369
12.50.2.23 sx1276_write_burst()	369
12.50.2.24 sx1276_write_fifo()	369
12.50.2.25 sx1276_write_reg()	370
12.51 radio_sx1276.h	370
12.52 Dokumentacja pliku Core/App/radio_lib/common/sx1276/radio_sx1276_regs.h	371
12.52.1 Opis szczegółowy	374
12.52.2 Dokumentacja definicji	374
12.52.2.1 SX1276_DIO0_MAP_CAD_DONE	374
12.52.2.2 SX1276_DIO0_MAP_FSK_PACKET_SENT	374
12.52.2.3 SX1276_DIO0_MAP_FSK_PAYLOAD_READY	374
12.52.2.4 SX1276_DIO0_MAP_RX_DONE	374
12.52.2.5 SX1276_DIO0_MAP_TX_DONE	374
12.52.2.6 SX1276_DIO1_MAP_CAD_DETECTED	374
12.52.2.7 SX1276_DIO1_MAP_FHSS_CHANGE	374
12.52.2.8 SX1276_DIO1_MAP_RX_TIMEOUT	374
12.52.2.9 SX1276_FIFO_THRESH_TX_START_NOT_EMPTY	374
12.52.2.10 SX1276_IRQ1_MODE_READY	375
12.52.2.11 SX1276_IRQ1_PLL_LOCK	375
12.52.2.12 SX1276_IRQ1_PREAMBLE_DETECT	375
12.52.2.13 SX1276_IRQ1_RSSI	375
12.52.2.14 SX1276_IRQ1_RX_READY	375
12.52.2.15 SX1276_IRQ1_SYNC_ADDRESS_MATCH	375
12.52.2.16 SX1276_IRQ1_TIMEOUT	375
12.52.2.17 SX1276_IRQ1_TX_READY	375
12.52.2.18 SX1276_IRQ2_CRC_OK	375
12.52.2.19 SX1276_IRQ2_FIFO_FULL	375
12.52.2.20 SX1276_IRQ2_FIFO_LEVEL	375
12.52.2.21 SX1276_IRQ2_FIFO_NOT_EMPTY	375
12.52.2.22 SX1276_IRQ2_FIFO_OVERRUN	376
12.52.2.23 SX1276_IRQ2_LOW_BAT	376
12.52.2.24 SX1276_IRQ2_PACKET_SENT	376
12.52.2.25 SX1276_IRQ2_PAYLOAD_READY	376
12.52.2.26 SX1276_IRQ_CAD_DETECTED	376
12.52.2.27 SX1276_IRQ_CAD_DONE	376
12.52.2.28 SX1276_IRQ_FHSS_CHANGE_CHANNEL	376
12.52.2.29 SX1276_IRQ_PAYLOAD_CRC_ERROR	376
12.52.2.30 SX1276_IRQ_RX_DONE	376
12.52.2.31 SX1276_IRQ_RX_TIMEOUT	376
12.52.2.32 SX1276_IRQ_TX_DONE	376

12.52.2.33 SX1276_IRQ_VALID_HEADER	376
12.52.2.34 SX1276_MODE_CAD	377
12.52.2.35 SX1276_MODE_FSRX	377
12.52.2.36 SX1276_MODE_FSTX	377
12.52.2.37 SX1276_MODE_RXCONTINUOUS	377
12.52.2.38 SX1276_MODE_RXSINGLE	377
12.52.2.39 SX1276_MODE_SLEEP	377
12.52.2.40 SX1276_MODE_STDBY	377
12.52.2.41 SX1276_MODE_TX	377
12.52.2.42 SX1276_OPMODE_ACCESS_SHARED_REG	377
12.52.2.43 SX1276_OPMODE_LONG_RANGE_MODE	377
12.52.2.44 SX1276_OPMODE_LOW_FREQ_MODE_ON	377
12.52.2.45 SX1276_OPMODE_MODE_MASK	377
12.52.2.46 SX1276_OPMODE_MODULATION_FSK	378
12.52.2.47 SX1276_OPMODE_MODULATION_MASK	378
12.52.2.48 SX1276_OPMODE_MODULATION_OOK	378
12.52.2.49 SX1276_OPMODE_SHAPING_FSK_BT03	378
12.52.2.50 SX1276_OPMODE_SHAPING_FSK_BT05	378
12.52.2.51 SX1276_OPMODE_SHAPING_FSK_BT10	378
12.52.2.52 SX1276_OPMODE_SHAPING_MASK	378
12.52.2.53 SX1276_OPMODE_SHAPING_NONE	378
12.52.2.54 SX1276_OPMODE_SHAPING_OOK_FILTER	378
12.52.2.55 SX1276_PA_DAC_DISABLE_17DBM	378
12.52.2.56 SX1276_PA_DAC_ENABLE_20DBM	378
12.52.2.57 SX1276_PACKET_ADDR_FILTER_NODE	378
12.52.2.58 SX1276_PACKET_ADDR_FILTER_NODE_BC	379
12.52.2.59 SX1276_PACKET_ADDR_FILTER_OFF	379
12.52.2.60 SX1276_PACKET_CRC_OFF	379
12.52.2.61 SX1276_PACKET_CRC_ON	379
12.52.2.62 SX1276_PACKET_CRC_WHITENING_CCITT	379
12.52.2.63 SX1276_PACKET_CRC_WHITENING_IBM	379
12.52.2.64 SX1276_PACKET_DATA_MODE_CONTINUOUS	379
12.52.2.65 SX1276_PACKET_DATA_MODE_PACKET	379
12.52.2.66 SX1276_PACKET_DC_FREE_NONE	379
12.52.2.67 SX1276_PACKET_DC_FREE_WHITENING	379
12.52.2.68 SX1276_PACKET_FORMAT_FIXED	379
12.52.2.69 SX1276_PACKET_FORMAT_VARIABLE	379
12.52.2.70 SX1276_REG_AFC_BW	380
12.52.2.71 SX1276_REG_BITRATE_LSB	380
12.52.2.72 SX1276_REG_BITRATE_MSB	380
12.52.2.73 SX1276_REG_BROADCAST_ADDRESS	380
12.52.2.74 SX1276_REG_DETECTION_OPTIMIZE	380

12.52.2.75 SX1276_REG_DETECTION_THRESHOLD	380
12.52.2.76 SX1276_REG_DIO_MAPPING_1	380
12.52.2.77 SX1276_REG_DIO_MAPPING_2	380
12.52.2.78 SX1276_REG_FDEV_LSB	380
12.52.2.79 SX1276_REG_FDEV_MSB	380
12.52.2.80 SX1276_REG_FIFO	380
12.52.2.81 SX1276_REG_FIFO_ADDR_PTR	380
12.52.2.82 SX1276_REG_FIFO_RX_BASE_ADDR	381
12.52.2.83 SX1276_REG_FIFO_RX_CURRENT_ADDR	381
12.52.2.84 SX1276_REG_FIFO_THRESH	381
12.52.2.85 SX1276_REG_FIFO_TX_BASE_ADDR	381
12.52.2.86 SX1276_REG_FRF_LSB	381
12.52.2.87 SX1276_REG_FRF_MID	381
12.52.2.88 SX1276_REG_FRF_MSB	381
12.52.2.89 SX1276_REG_INVERT_IQ	381
12.52.2.90 SX1276_REG_INVERT_IQ2	381
12.52.2.91 SX1276_REG_IRQ_FLAGS	381
12.52.2.92 SX1276_REG_IRQ_FLAGS_1	381
12.52.2.93 SX1276_REG_IRQ_FLAGS_2	381
12.52.2.94 SX1276_REG_IRQ_FLAGS_MASK	382
12.52.2.95 SX1276_REG_LNA	382
12.52.2.96 SX1276_REG_MODEM_CONFIG_1	382
12.52.2.97 SX1276_REG_MODEM_CONFIG_2	382
12.52.2.98 SX1276_REG_MODEM_CONFIG_3	382
12.52.2.99 SX1276_REG_NODE_ADDRESS	382
12.52.2.100 SX1276_REG_OOK_AVG	382
12.52.2.101 SX1276_REG_OOK_FIX	382
12.52.2.102 SX1276_REG_OOK_PEAK	382
12.52.2.103 SX1276_REG_OP_MODE	382
12.52.2.104 SX1276_REG_PA_CONFIG	382
12.52.2.105 SX1276_REG_PA_DAC	382
12.52.2.106 SX1276_REG_PACKET_CONFIG_1	383
12.52.2.107 SX1276_REG_PACKET_CONFIG_2	383
12.52.2.108 SX1276_REG_PAYLOAD_LENGTH	383
12.52.2.109 SX1276_REG_PAYLOAD_LENGTH_FSK	383
12.52.2.110 SX1276_REG_PKT_RSSI_VALUE	383
12.52.2.111 SX1276_REG_PKT_SNR_VALUE	383
12.52.2.112 SX1276_REG_PREAMBLE_DETECT	383
12.52.2.113 SX1276_REG_PREAMBLE_LSB	383
12.52.2.114 SX1276_REG_PREAMBLE_LSB_FSK	383
12.52.2.115 SX1276_REG_PREAMBLE_MSB	383
12.52.2.116 SX1276_REG_PREAMBLE_MSB_FSK	383

12.52.2.117 SX1276_REG_RSSI_CONFIG	383
12.52.2.118 SX1276_REG_RSSI_VALUE_FSK	384
12.52.2.119 SX1276_REG_RX_BW	384
12.52.2.120 SX1276_REG_RX_CONFIG	384
12.52.2.121 SX1276_REG_RX_NB_BYTES	384
12.52.2.122 SX1276_REG_SYMB_TIMEOUT_LSB	384
12.52.2.123 SX1276_REG_SYNC_CONFIG	384
12.52.2.124 SX1276_REG_SYNC_VALUE_1	384
12.52.2.125 SX1276_REG_SYNC_WORD	384
12.52.2.126 SX1276_REG_VERSION	384
12.52.2.127 SX1276_RXCONFIG_AFC_AUTO_ON	384
12.52.2.128 SX1276_RXCONFIG_AGC_AUTO_ON	384
12.52.2.129 SX1276_RXCONFIG_RESTART_RX_NO_PLL_LOCK	384
12.52.2.130 SX1276_RXCONFIG_TRIGGER_RSSI	385
12.52.2.131 SX1276_SPI_READ_MASK	385
12.52.2.132 SX1276_SPI_WRITE_MASK	385
12.52.2.133 SX1276_SYNC_AUTO_RESTART_NO_PLL	385
12.52.2.134 SX1276_SYNC_AUTO_RESTART_OFF	385
12.52.2.135 SX1276_SYNC_AUTO_RESTART_PLL	385
12.52.2.136 SX1276_SYNC_FIFOFILL_AUTO	385
12.52.2.137 SX1276_SYNC_FIFOFILL_MANUAL	385
12.52.2.138 SX1276_SYNC_OFF	385
12.52.2.139 SX1276_SYNC_ON	385
12.52.2.140 SX1276_SYNC_PREAMBLE_POLARITY_55	385
12.52.2.141 SX1276_SYNC_PREAMBLE_POLARITY_AA	385
12.52.2.142 SX1276_VERSION_ID	386
12.53 radio_sx1276_regs.h	386
12.54 Dokumentacja pliku Core/App/radio_lib/modulations/fsk/radio_fsk.c	388
12.54.1 Opis szczegółowy	390
12.54.2 Dokumentacja definicji	390
12.54.2.1 RADIO_FSK_BROADCAST_ADDRESS	390
12.54.2.2 RADIO_FSK_FIFO_MAX_BYTES	390
12.54.2.3 RADIO_FSK_FIFO_THRESH_LEVEL	390
12.54.2.4 RADIO_FSK_NODE_ADDRESS_DEFAULT	390
12.54.2.5 RADIO_FSK_RESET_HIGH_DELAY_MS	390
12.54.2.6 RADIO_FSK_RESET_LOW_DELAY_MS	390
12.54.2.7 RADIO_FSK_TX_GUARD_MS	390
12.54.3 Dokumentacja funkcji	391
12.54.3.1 radio_apply_fsk_config()	391
12.54.3.2 radio_cfg_valid()	391
12.54.3.3 radio_fsk_default_hw_cfg()	391
12.54.3.4 radio_fsk_default_lora_cfg()	391

12.54.3.5 radio_fsk_deinit()	391
12.54.3.6 radio_fsk_get_last_packet()	391
12.54.3.7 radio_fsk_get_state()	392
12.54.3.8 radio_fsk_init()	392
12.54.3.9 radio_fsk_on_exti()	392
12.54.3.10 radio_fsk_process()	392
12.54.3.11 radio_fsk_raw_read_burst()	392
12.54.3.12 radio_fsk_raw_read_reg()	392
12.54.3.13 radio_fsk_raw_write_burst()	393
12.54.3.14 radio_fsk_raw_write_reg()	393
12.54.3.15 radio_fsk_send_async()	393
12.54.3.16 radio_fsk_set_runtime_cfg()	393
12.54.3.17 radio_fsk_sleep()	393
12.54.3.18 radio_fsk_standby()	394
12.54.3.19 radio_fsk_start_rx_continuous()	394
12.54.3.20 radio_fsk_start_rx_single()	394
12.54.3.21 radio_fsk_take_events()	394
12.54.3.22 radio_get_frequency_deviation()	394
12.54.3.23 radio_get_packet_config_1()	394
12.54.3.24 radio_get_rx_bw_reg_value()	395
12.54.3.25 radio_get_shaping_bits()	395
12.54.3.26 radio_handle_exti_pin()	395
12.54.3.27 radio_hw_reset()	395
12.54.3.28 radio_irq_restore()	395
12.54.3.29 radio_irq_save()	395
12.54.3.30 radio_read_rx_packet()	395
12.54.3.31 radio_resume_after_tx()	396
12.54.3.32 radio_rx_single_timeout_ms()	396
12.54.3.33 radio_set_event_flags()	396
12.54.3.34 radio_set_op_mode_ready()	396
12.54.3.35 radio_set_state()	396
12.54.3.36 radio_tx_timeout_ms()	396
12.54.3.37 radio_write_sync_word()	396
12.54.4 Dokumentacja zmiennych	397
12.54.4.1 s_radio	397
12.54.4.2 s_runtime_cfg	397
12.54.4.3 s_runtime_cfg_ready	397
12.55 radio_fsk.c	397
12.56 Dokumentacja pliku Core/App/radio_lib/modulations/fsk/radio_fsk.h	407
12.56.1 Opis szczegółowy	408
12.56.2 Dokumentacja funkcji	408
12.56.2.1 radio_fsk_default_hw_cfg()	408

12.56.2.2 radio_fsk_default_lora_cfg()	408
12.56.2.3 radio_fsk_deinit()	409
12.56.2.4 radio_fsk_get_last_packet()	409
12.56.2.5 radio_fsk_get_state()	409
12.56.2.6 radio_fsk_init()	409
12.56.2.7 radio_fsk_on_exti()	409
12.56.2.8 radio_fsk_process()	409
12.56.2.9 radio_fsk_raw_read_burst()	410
12.56.2.10 radio_fsk_raw_read_reg()	410
12.56.2.11 radio_fsk_raw_write_burst()	410
12.56.2.12 radio_fsk_raw_write_reg()	410
12.56.2.13 radio_fsk_send_async()	410
12.56.2.14 radio_fsk_set_runtime_cfg()	410
12.56.2.15 radio_fsk_sleep()	411
12.56.2.16 radio_fsk_standby()	411
12.56.2.17 radio_fsk_start_rx_continuous()	411
12.56.2.18 radio_fsk_start_rx_single()	411
12.56.2.19 radio_fsk_take_events()	411
12.57 radio_fsk.h	411
12.58 Dokumentacja pliku Core/App/radio_lib/modulations/lora/radio_lora.c	412
12.58.1 Opis szczegółowy	414
12.58.2 Dokumentacja definicji	414
12.58.2.1 RADIO_DEFAULT_RX_TIMEOUT_SYMBOLS	414
12.58.2.2 RADIO_RESET_HIGH_DELAY_MS	414
12.58.2.3 RADIO_RESET_LOW_DELAY_MS	414
12.58.2.4 RADIO_SX1276_FIFO_RX_BASE_ADDR	414
12.58.2.5 RADIO_SX1276_FIFO_TX_BASE_ADDR	414
12.58.3 Dokumentacja funkcji	414
12.58.3.1 radio_apply_lora_config()	414
12.58.3.2 radio_cfg_valid()	415
12.58.3.3 radio_config_dio_for_rx()	415
12.58.3.4 radio_config_dio_for_tx()	415
12.58.3.5 radio_handle_exti_pin()	415
12.58.3.6 radio_hw_reset()	415
12.58.3.7 radio_irq_restore()	415
12.58.3.8 radio_irq_save()	416
12.58.3.9 radio_lora_default_hw_cfg()	416
12.58.3.10 radio_lora_default_lora_cfg()	416
12.58.3.11 radio_lora_deinit()	416
12.58.3.12 radio_lora_get_last_packet()	417
12.58.3.13 radio_lora_get_state()	417
12.58.3.14 radio_lora_init()	417

12.58.3.15 radio_lora_on_exti()	418
12.58.3.16 radio_lora_process()	418
12.58.3.17 radio_lora_raw_read_burst()	418
12.58.3.18 radio_lora_raw_read_reg()	418
12.58.3.19 radio_lora_raw_write_burst()	419
12.58.3.20 radio_lora_raw_write_reg()	419
12.58.3.21 radio_lora_send_async()	420
12.58.3.22 radio_lora_sleep()	420
12.58.3.23 radio_lora_standby()	420
12.58.3.24 radio_lora_start_rx_continuous()	420
12.58.3.25 radio_lora_start_rx_single()	421
12.58.3.26 radio_lora_take_events()	421
12.58.3.27 radio_low_data_rate_optimize_required()	421
12.58.3.28 radio_read_irq_flags_retry()	421
12.58.3.29 radio_read_rx_packet()	422
12.58.3.30 radio_resume_after_tx()	422
12.58.3.31 radio_set_event_flags()	422
12.58.3.32 radio_set_state()	422
12.58.3.33 radio_wait_for_tx_enter_or_done()	422
12.58.4 Dokumentacja zmiennych	422
12.58.4.1 s_radio	422
12.59 radio_lora.c	422
12.60 Dokumentacja pliku Core/App/radio_lib/modulations/lora/radio_lora.h	432
12.60.1 Opis szczegółowy	432
12.60.2 Dokumentacja funkcji	433
12.60.2.1 radio_lora_default_hw_cfg()	433
12.60.2.2 radio_lora_default_lora_cfg()	433
12.60.2.3 radio_lora_deinit()	433
12.60.2.4 radio_lora_get_last_packet()	433
12.60.2.5 radio_lora_get_state()	434
12.60.2.6 radio_lora_init()	434
12.60.2.7 radio_lora_on_exti()	434
12.60.2.8 radio_lora_process()	435
12.60.2.9 radio_lora_raw_read_burst()	435
12.60.2.10 radio_lora_raw_read_reg()	435
12.60.2.11 radio_lora_raw_write_burst()	436
12.60.2.12 radio_lora_raw_write_reg()	436
12.60.2.13 radio_lora_send_async()	436
12.60.2.14 radio_lora_sleep()	437
12.60.2.15 radio_lora_standby()	437
12.60.2.16 radio_lora_start_rx_continuous()	437
12.60.2.17 radio_lora_start_rx_single()	438

12.60.2.18 radio_lora_take_events()	438
12.61 radio_lora.h	438
12.62 Dokumentacja pliku Core/App/radio_lib/modulations/ook/radio_ook.c	439
12.62.1 Opis szczegółowy	441
12.62.2 Dokumentacja definicji	441
12.62.2.1 RADIO_OOK_FIFO_MAX_BYTES	441
12.62.2.2 RADIO_OOK_FIFO_THRESH_LEVEL	441
12.62.2.3 RADIO_OOK_RESET_HIGH_DELAY_MS	441
12.62.2.4 RADIO_OOK_RESET_LOW_DELAY_MS	441
12.62.2.5 RADIO_OOK_TX_GUARD_MS	441
12.62.3 Dokumentacja funkcji	441
12.62.3.1 radio_apply_ook_config()	441
12.62.3.2 radio_apply_threshold()	442
12.62.3.3 radio_cfg_valid()	442
12.62.3.4 radio_get_rx_bw_reg_value()	442
12.62.3.5 radio_handle_exti_pin()	442
12.62.3.6 radio_hw_reset()	442
12.62.3.7 radio_irq_restore()	442
12.62.3.8 radio_irq_save()	442
12.62.3.9 radio_ook_default_hw_cfg()	443
12.62.3.10 radio_ook_default_lora_cfg()	443
12.62.3.11 radio_ook_deinit()	443
12.62.3.12 radio_ook_get_last_packet()	443
12.62.3.13 radio_ook_get_state()	443
12.62.3.14 radio_ook_init()	443
12.62.3.15 radio_ook_on_exti()	444
12.62.3.16 radio_ook_process()	444
12.62.3.17 radio_ook_raw_read_burst()	444
12.62.3.18 radio_ook_raw_read_reg()	444
12.62.3.19 radio_ook_raw_write_burst()	444
12.62.3.20 radio_ook_raw_write_reg()	444
12.62.3.21 radio_ook_send_async()	445
12.62.3.22 radio_ook_set_runtime_cfg()	445
12.62.3.23 radio_ook_sleep()	445
12.62.3.24 radio_ook_standby()	445
12.62.3.25 radio_ook_start_rx_continuous()	445
12.62.3.26 radio_ook_start_rx_single()	445
12.62.3.27 radio_ook_take_events()	446
12.62.3.28 radio_read_rx_packet()	446
12.62.3.29 radio_resume_after_tx()	446
12.62.3.30 radio_rx_single_timeout_ms()	446
12.62.3.31 radio_set_event_flags()	446

12.62.3.32 radio_set_op_mode_ready()	446
12.62.3.33 radio_set_state()	446
12.62.3.34 radio_tx_timeout_ms()	447
12.62.3.35 radio_write_sync_word()	447
12.62.4 Dokumentacja zmiennych	447
12.62.4.1 s_radio	447
12.62.4.2 s_runtime_cfg	447
12.62.4.3 s_runtime_cfg_ready	447
12.63 radio_ook.c	447
12.64 Dokumentacja pliku Core/App/radio_lib/modulations/ook/radio_ook.h	457
12.64.1 Opis szczegółowy	457
12.64.2 Dokumentacja funkcji	458
12.64.2.1 radio_ook_default_hw_cfg()	458
12.64.2.2 radio_ook_default_lora_cfg()	458
12.64.2.3 radio_ook_deinit()	458
12.64.2.4 radio_ook_get_last_packet()	458
12.64.2.5 radio_ook_get_state()	458
12.64.2.6 radio_ook_init()	458
12.64.2.7 radio_ook_on_exti()	459
12.64.2.8 radio_ook_process()	459
12.64.2.9 radio_ook_raw_read_burst()	459
12.64.2.10 radio_ook_raw_read_reg()	459
12.64.2.11 radio_ook_raw_write_burst()	459
12.64.2.12 radio_ook_raw_write_reg()	459
12.64.2.13 radio_ook_send_async()	460
12.64.2.14 radio_ook_set_runtime_cfg()	460
12.64.2.15 radio_ook_sleep()	460
12.64.2.16 radio_ook_standby()	460
12.64.2.17 radio_ook_start_rx_continuous()	460
12.64.2.18 radio_ook_start_rx_single()	460
12.64.2.19 radio_ook_take_events()	461
12.65 radio_ook.h	461
12.66 Dokumentacja pliku Core/App/radio_lib/radio_lib.c	461
12.66.1 Opis szczegółowy	463
12.66.2 Dokumentacja funkcji	463
12.66.2.1 HAL_GPIO_EXTI_Callback()	463
12.66.2.2 radio_default_fsk_cfg()	463
12.66.2.3 radio_default_hw_cfg()	463
12.66.2.4 radio_default_lora_cfg()	463
12.66.2.5 radio_default_ook_cfg()	464
12.66.2.6 radio_deinit()	464
12.66.2.7 radio_get_backend()	464

12.66.2.8 radio_get_last_packet()	464
12.66.2.9 radio_get_state()	465
12.66.2.10 radio_init()	465
12.66.2.11 radio_normalize_backend()	465
12.66.2.12 radio_on_exti()	466
12.66.2.13 radio_process()	466
12.66.2.14 radio_raw_read_burst()	466
12.66.2.15 radio_raw_read_reg()	466
12.66.2.16 radio_raw_write_burst()	467
12.66.2.17 radio_raw_write_reg()	467
12.66.2.18 radio_select_backend()	468
12.66.2.19 radio_send_async()	468
12.66.2.20 radio_set_fsk_cfg()	468
12.66.2.21 radio_set_ook_cfg()	468
12.66.2.22 radio_sleep()	469
12.66.2.23 radio_standby()	469
12.66.2.24 radio_start_rx_continuous()	469
12.66.2.25 radio_start_rx_single()	469
12.66.2.26 radio_take_events()	470
12.66.3 Dokumentacja zmiennych	470
12.66.3.1 s_active_backend	470
12.67 radio_lib.c	470
12.68 Dokumentacja pliku Core/App/radio_lib/radio_lib.h	474
12.68.1 Opis szczegółowy	477
12.68.2 Dokumentacja definicji typów	477
12.68.2.1 radio_event_cb_t	477
12.68.3 Dokumentacja typów wyliczanych	477
12.68.3.1 radio_address_filter_t	477
12.68.3.2 radio_event_mask_t	477
12.68.3.3 radio_fsk_filter_t	478
12.68.3.4 radio_fsk_shaping_t	478
12.68.3.5 radio_lora_bw_t	478
12.68.3.6 radio_ook_threshold_t	478
12.68.3.7 radio_packet_crc_t	478
12.68.3.8 radio_state_t	478
12.68.3.9 radio_status_t	479
12.68.4 Dokumentacja funkcji	479
12.68.4.1 radio_default_fsk_cfg()	479
12.68.4.2 radio_default_hw_cfg()	479
12.68.4.3 radio_default_lora_cfg()	480
12.68.4.4 radio_default_ook_cfg()	480
12.68.4.5 radio_deinit()	480

12.68.4.6 radio_get_backend()	480
12.68.4.7 radio_get_last_packet()	480
12.68.4.8 radio_get_state()	481
12.68.4.9 radio_init()	481
12.68.4.10 radio_on_exti()	481
12.68.4.11 radio_process()	482
12.68.4.12 radio_raw_read_burst()	482
12.68.4.13 radio_raw_read_reg()	482
12.68.4.14 radio_raw_write_burst()	483
12.68.4.15 radio_raw_write_reg()	483
12.68.4.16 radio_select_backend()	483
12.68.4.17 radio_send_async()	484
12.68.4.18 radio_set_fsk_cfg()	484
12.68.4.19 radio_set_ook_cfg()	484
12.68.4.20 radio_sleep()	485
12.68.4.21 radio_standby()	485
12.68.4.22 radio_start_rx_continuous()	485
12.68.4.23 radio_start_rx_single()	485
12.68.4.24 radio_take_events()	486
12.69 radio_lib.h	486
12.70 Dokumentacja pliku Core/App/radio_lib/radio_lib_config.h	489
12.70.1 Opis szczegółowy	489
12.70.2 Dokumentacja definicji	489
12.70.2.1 RADIO_LIB_ACTIVE_MODULATION	489
12.70.2.2 RADIO_LIB_DEFAULT_FREQ_HZ	490
12.70.2.3 RADIO_LIB_DEFAULT_SYNC_WORD	490
12.70.2.4 RADIO_LIB_ENABLE_DIO2_EXTI	490
12.70.2.5 RADIO_LIB_ENABLE_DIO3_EXTI	490
12.70.2.6 RADIO_LIB_ENABLE_DIO4_EXTI	490
12.70.2.7 RADIO_LIB_ENABLE_DIO5_EXTI	490
12.70.2.8 RADIO_LIB_MAX_PAYLOAD	490
12.70.2.9 RADIO_LIB_MODULATION_FSK	490
12.70.2.10 RADIO_LIB_MODULATION_LORA	490
12.70.2.11 RADIO_LIB_MODULATION_OOK	491
12.70.2.12 RADIO_LIB_OWNS_HAL_EXTI_CALLBACK	491
12.70.2.13 RADIO_LIB_SPI_TIMEOUT_MS	491
12.71 radio_lib_config.h	491
12.72 Dokumentacja pliku Core/App/radio_lib/test/radio_test.c	492
12.72.1 Opis szczegółowy	493
12.72.2 Dokumentacja funkcji	493
12.72.2.1 radio_test_demo_init()	493
12.72.2.2 radio_test_demo_process()	493

12.72.2.3 radio_test_dump_basic()	493
12.72.2.4 radio_test_dump_tx_debug()	494
12.72.2.5 radio_test_event_cb()	494
12.72.2.6 radio_test_irq_restore()	494
12.72.2.7 radio_test_irq_save()	494
12.72.2.8 radio_test_now_ms()	494
12.72.2.9 radio_test_print_packet()	494
12.72.2.10 radio_test_print_packet_text()	495
12.72.2.11 radio_test_probe()	495
12.72.2.12 radio_test_publish_packet_to_lcd()	495
12.72.2.13 radio_test_send_ping()	495
12.72.2.14 radio_test_store_packet_in_i2c_memory()	496
12.72.3 Dokumentacja zmiennych	496
12.72.3.1 s_demo_initialized	496
12.72.3.2 s_msg_store	496
12.72.3.3 s_msg_store_ready	496
12.72.3.4 s_radio_cb_events	496
12.72.3.5 s_radio_last_ping_ms	496
12.72.3.6 s_radio_last_tx_start_ms	496
12.73 radio_test.c	496
12.74 Dokumentacja pliku Core/App/radio_lib/test/radio_test.h	501
12.74.1 Opis szczegółowy	501
12.74.2 Dokumentacja funkcji	501
12.74.2.1 radio_test_demo_init()	501
12.74.2.2 radio_test_demo_process()	502
12.74.2.3 radio_test_dump_basic()	502
12.74.2.4 radio_test_probe()	502
12.74.2.5 radio_test_send_ping()	502
12.75 radio_test.h	503
12.76 radio_main.c	503
12.77 Dokumentacja pliku Core/App/radio_main.h	560
12.77.1 Opis szczegółowy	562
12.77.2 Dokumentacja typów wyliczanych	562
12.77.2.1 radio_main_address_filter_t	562
12.77.2.2 radio_main_auto_ping_mode_t	563
12.77.2.3 radio_main_crc_type_t	563
12.77.2.4 radio_main_filter_t	563
12.77.2.5 radio_main_fsk_shaping_t	563
12.77.2.6 radio_main_header_mode_t	564
12.77.2.7 radio_main_modulation_t	564
12.77.2.8 radio_main_ook_threshold_t	564
12.77.2.9 radio_main_option_t	564

12.77.3 Dokumentacja funkcji	565
12.77.3.1 radio_main_cmd_pairing_accept()	565
12.77.3.2 radio_main_cmd_reset_module()	566
12.77.3.3 radio_main_cmd_send_network_pair_req()	566
12.77.3.4 radio_main_cmd_send_pair_error()	566
12.77.3.5 radio_main_cmd_send_pair_req()	566
12.77.3.6 radio_main_cmd_send_raw()	567
12.77.3.7 radio_main_cmd_send_template()	567
12.77.3.8 radio_main_cmd_send_user_text()	567
12.77.3.9 radio_main_cmd_set_auto_ping()	568
12.77.3.10 radio_main_cmd_set_auto_ping_mode()	568
12.77.3.11 radio_main_cmd_set_auto_ping_period()	568
12.77.3.12 radio_main_cmd_set_coding()	569
12.77.3.13 radio_main_cmd_set_fh()	569
12.77.3.14 radio_main_cmd_set_fh_period()	569
12.77.3.15 radio_main_cmd_set_lora_preset()	569
12.77.3.16 radio_main_cmd_set_modulation()	570
12.77.3.17 radio_main_cmd_set_modulation_bw()	570
12.77.3.18 radio_main_cmd_set_modulation_freq()	570
12.77.3.19 radio_main_cmd_set_option()	571
12.77.3.20 radio_main_cmd_start_network_pairing()	571
12.77.3.21 radio_main_cmd_start_pairing()	571
12.77.3.22 radio_main_create_task()	571
12.77.3.23 radio_main_get_auto_ping_period_ms()	572
12.77.3.24 radio_main_get_last_error_text()	572
12.77.3.25 radio_main_get_node_id()	572
12.77.3.26 radio_main_get_runtime_cfg()	572
12.77.3.27 radio_main_load_default_fsk_profile()	573
12.77.3.28 radio_main_load_default_lora_preset()	573
12.77.3.29 radio_main_load_default_ook_profile()	573
12.78 radio_main.h	574
12.79 security_main.c	576
12.80 Dokumentacja pliku Core/App/security_main.h	613
12.80.1 Opis szczegółowy	614
12.80.2 Dokumentacja typów wyliczanych	614
12.80.2.1 security_frame_key_mode_t	614
12.80.2.2 security_notify_mode_t	615
12.80.3 Dokumentacja funkcji	615
12.80.3.1 security_main_cmd_add_device()	615
12.80.3.2 security_main_cmd_add_gateway()	615
12.80.3.3 security_main_cmd_delete_device()	616
12.80.3.4 security_main_cmd_get_device()	616

12.80.3.5 security_main_cmd_get_runtime_cfg()	616
12.80.3.6 security_main_cmd_rotate_key()	617
12.80.3.7 security_main_cmd_set_auto_ping()	617
12.80.3.8 security_main_cmd_set_auto_ping_mode()	617
12.80.3.9 security_main_cmd_set_auto_ping_period()	617
12.80.3.10 security_main_cmd_set_coding()	618
12.80.3.11 security_main_cmd_set_fh()	618
12.80.3.12 security_main_cmd_set_fh_period()	618
12.80.3.13 security_main_cmd_set_lora_preset()	619
12.80.3.14 security_main_cmd_set_notify_mode()	619
12.80.3.15 security_main_cmd_set_radio_runtime_cfg()	619
12.80.3.16 security_main_commit_gateway_counter()	619
12.80.3.17 security_main_create_task()	620
12.80.3.18 security_main_get_frame_keys()	620
12.80.3.19 security_main_get_frame_keys_for_code()	620
12.80.3.20 security_main_get_frame_keys_mode()	621
12.80.3.21 security_main_get_gateway_counter()	621
12.80.3.22 security_main_get_network_key()	622
12.80.3.23 security_main_get_peer_link_key()	622
12.80.3.24 security_main_get_tpm_ready()	622
12.80.3.25 security_main_log_message()	623
12.81 security_main.h	623
12.82 service.c	625
12.83 Dokumentacja pliku Core/App/service.h	637
12.83.1 Opis szczegółowy	638
12.83.2 Dokumentacja funkcji	638
12.83.2.1 service_uart_create_task()	638
12.84 service.h	638
12.85 Dokumentacja pliku Core/App/st33ktpm2x_lib/st33ktpm2x.c	638
12.85.1 Opis szczegółowy	641
12.85.2 Dokumentacja definicji	641
12.85.2.1 ST33_ACCESS_ACTIVE_LOCALITY	641
12.85.2.2 ST33_ACCESS_READ_ZERO	641
12.85.2.3 ST33_ACCESS_REQUEST_USE	642
12.85.2.4 ST33_ACCESS_VALID	642
12.85.2.5 ST33_I2C_MAX_RETRIES	642
12.85.2.6 ST33_REG_ACCESS	642
12.85.2.7 ST33_REG_DATA_FIFO	642
12.85.2.8 ST33_REG_DID_VID	642
12.85.2.9 ST33_REG_INTERFACE_CAPABILITY	642
12.85.2.10 ST33_REG_LOC_SEL	642
12.85.2.11 ST33_REG_RID	642

12.85.2.12 ST33_REG_STS	642
12.85.2.13 ST33_STS_COMMAND_READY	642
12.85.2.14 ST33_STS_DATA_AVAIL	642
12.85.2.15 ST33_STS_EXPECT	643
12.85.2.16 ST33_STS_READ_ZERO	643
12.85.2.17 ST33_STS_TPM_GO	643
12.85.2.18 ST33_STS_VALID	643
12.85.2.19 ST33_TPM_HEADER_SIZE	643
12.85.2.20 ST33_TPM_NV_TRANSFER_MAX_BYTES	643
12.85.2.21 ST33_TPM_PW_AUTH_SIZE	643
12.85.2.22 ST33_TPM_RANDOM_MAX_BYTES	643
12.85.2.23 ST33_TPM_RH_OWNER	643
12.85.2.24 ST33_TPM_RH_PLATFORM	643
12.85.2.25 ST33_TPM_RS_PW	643
12.85.2.26 ST33_TRACE	644
12.85.3 Dokumentacja funkcji	644
12.85.3.1 st33_be16_read()	644
12.85.3.2 st33_be16_write()	644
12.85.3.3 st33_be32_read()	644
12.85.3.4 st33_be32_write()	644
12.85.3.5 st33_fifo_read()	644
12.85.3.6 st33_fifo_write()	645
12.85.3.7 st33_i2c_read()	645
12.85.3.8 st33_i2c_write()	645
12.85.3.9 st33_identity_is_valid()	645
12.85.3.10 st33_le32_read()	645
12.85.3.11 st33_min_u16()	645
12.85.3.12 st33_read_access()	646
12.85.3.13 st33_read_response_stream()	646
12.85.3.14 st33_read_sts()	646
12.85.3.15 st33_status_text()	646
12.85.3.16 st33_tpm2_nv_write_auth()	646
12.85.3.17 st33_trace_fail()	647
12.85.3.18 st33_trace_tis()	647
12.85.3.19 st33_wait_access_bits()	647
12.85.3.20 st33_wait_burst()	647
12.85.3.21 st33_wait_sts_bits()	647
12.85.3.22 st33_write_access()	648
12.85.3.23 st33_write_empty_password_auth()	648
12.85.3.24 st33_write_loc_sel()	648
12.85.3.25 st33_write_sts()	648
12.85.3.26 st33tpm2x_davint_is_asserted()	648

12.85.3.27 st33ktpm2x_default_cfg()	649
12.85.3.28 st33ktpm2x_deinit()	649
12.85.3.29 st33ktpm2x_hard_reset()	649
12.85.3.30 st33ktpm2x_init()	650
12.85.3.31 st33ktpm2x_pp_is_pressed()	650
12.85.3.32 st33ktpm2x_pp_wait_pressed()	650
12.85.3.33 st33ktpm2x_read_identity()	651
12.85.3.34 st33ktpm2x_release_locality0()	651
12.85.3.35 st33ktpm2x_request_locality0()	652
12.85.3.36 st33ktpm2x_tpm2_flush_context()	652
12.85.3.37 st33ktpm2x_tpm2_get_capability()	652
12.85.3.38 st33ktpm2x_tpm2_get_random()	653
12.85.3.39 st33ktpm2x_tpm2_nv_define()	653
12.85.3.40 st33ktpm2x_tpm2_nv_read()	654
12.85.3.41 st33ktpm2x_tpm2_nv_write()	655
12.85.3.42 st33ktpm2x_tpm2_nv_write_platform_pp()	655
12.85.3.43 st33ktpm2x_tpm2_pcr_read_sha256()	656
12.85.3.44 st33ktpm2x_tpm2_policy_command_code()	656
12.85.3.45 st33ktpm2x_tpm2_policy_or()	656
12.85.3.46 st33ktpm2x_tpm2_policy_pcr()	657
12.85.3.47 st33ktpm2x_tpm2_policy_physical_presence()	657
12.85.3.48 st33ktpm2x_tpm2_self_test()	658
12.85.3.49 st33ktpm2x_tpm2_start_auth_session()	658
12.85.3.50 st33ktpm2x_tpm2_startup()	659
12.85.3.51 st33ktpm2x_transceive()	659
12.85.4 Dokumentacja zmiennych	660
12.85.4.1 s_st33_i2c_tx	660
12.86 st33ktpm2x.c	660
12.87 Dokumentacja pliku Core/App/st33ktpm2x_lib/st33ktpm2x.h	677
12.87.1 Opis szczegółowy	680
12.87.2 Dokumentacja definicji	680
12.87.2.1 ST33KTPM2X_CMD_MAX_BYTES	680
12.87.2.2 ST33KTPM2X_I2C_ADDR_DEFAULT	680
12.87.2.3 ST33KTPM2X_RSP_MAX_BYTES	680
12.87.2.4 ST33KTPM2X_TPM2_ALG_SHA256	680
12.87.2.5 ST33KTPM2X_TPM2_CAP_ALGS	680
12.87.2.6 ST33KTPM2X_TPM2_CAP_COMMANDS	680
12.87.2.7 ST33KTPM2X_TPM2_CAP_HANDLES	680
12.87.2.8 ST33KTPM2X_TPM2_CAP_PCERS	681
12.87.2.9 ST33KTPM2X_TPM2_CAP_TPM_PROPERTIES	681
12.87.2.10 ST33KTPM2X_TPM2_CC_GET_CAPABILITY	681
12.87.2.11 ST33KTPM2X_TPM2_CC_GET_RANDOM	681

12.87.2.12 ST33KTPM2X_TPM2_CC_NV_DEFINE_SPACE	681
12.87.2.13 ST33KTPM2X_TPM2_CC_NV_READ	681
12.87.2.14 ST33KTPM2X_TPM2_CC_NV_WRITE	681
12.87.2.15 ST33KTPM2X_TPM2_CC_PCR_READ	681
12.87.2.16 ST33KTPM2X_TPM2_CC_SELF_TEST	681
12.87.2.17 ST33KTPM2X_TPM2_CC_STARTUP	681
12.87.2.18 ST33KTPM2X_TPM2_RC_SUCCESS	681
12.87.2.19 ST33KTPM2X_TPM2_ST_NO_SESSIONS	681
12.87.2.20 ST33KTPM2X_TPM2_ST_SESSIONS	682
12.87.2.21 ST33KTPM2X_TPM2_SU_CLEAR	682
12.87.2.22 ST33KTPM2X_TPM2_SU_STATE	682
12.87.3 Dokumentacja typów wyliczanych	682
12.87.3.1 st33ktpm2x_status_t	682
12.87.4 Dokumentacja funkcji	682
12.87.4.1 st33ktpm2x_davint_is_asserted()	682
12.87.4.2 st33ktpm2x_default_cfg()	683
12.87.4.3 st33ktpm2x_deinit()	683
12.87.4.4 st33ktpm2x_hard_reset()	683
12.87.4.5 st33ktpm2x_init()	684
12.87.4.6 st33ktpm2x_pp_is_pressed()	684
12.87.4.7 st33ktpm2x_pp_wait_pressed()	684
12.87.4.8 st33ktpm2x_read_identity()	685
12.87.4.9 st33ktpm2x_release_locality0()	685
12.87.4.10 st33ktpm2x_request_locality0()	686
12.87.4.11 st33ktpm2x_tpm2_flush_context()	686
12.87.4.12 st33ktpm2x_tpm2_get_capability()	686
12.87.4.13 st33ktpm2x_tpm2_get_random()	687
12.87.4.14 st33ktpm2x_tpm2_nv_define()	687
12.87.4.15 st33ktpm2x_tpm2_nv_read()	688
12.87.4.16 st33ktpm2x_tpm2_nv_write()	689
12.87.4.17 st33ktpm2x_tpm2_nv_write_platform_pp()	689
12.87.4.18 st33ktpm2x_tpm2_pcr_read_sha256()	690
12.87.4.19 st33ktpm2x_tpm2_policy_command_code()	690
12.87.4.20 st33ktpm2x_tpm2_policy_or()	690
12.87.4.21 st33ktpm2x_tpm2_policy_pcr()	691
12.87.4.22 st33ktpm2x_tpm2_policy_physical_presence()	691
12.87.4.23 st33ktpm2x_tpm2_self_test()	692
12.87.4.24 st33ktpm2x_tpm2_start_auth_session()	692
12.87.4.25 st33ktpm2x_tpm2_startup()	693
12.87.4.26 st33ktpm2x_transceive()	693
12.88 st33ktpm2x.h	694
12.89 tof_main.c	696

12.90 Dokumentacja pliku Core/App/tof_main.h	697
12.90.1 Opis szczegółowy	697
12.90.2 Dokumentacja funkcji	698
12.90.2.1 tof_main_create_task()	698
12.90.2.2 tof_main_get_last_distance()	698
12.91 tof_main.h	698
12.92 vl53l3cx_lib.c	698
12.93 Dokumentacja pliku Core/App/vl53l3cx_lib/vl53l3cx_lib.h	701
12.93.1 Opis szczegółowy	701
12.93.2 Dokumentacja funkcji	702
12.93.2.1 tof_get_distance()	702
12.93.2.2 tof_init()	702
12.93.2.3 VL53L3CX_TestOnce()	702
12.94 vl53l3cx_lib.h	702
Skorowidz	703

Rozdział 1

BEKO Pager Firmware

1.1 Wprowadzenie

BEKO Pager Firmware to aplikacja STM32 dla bezprzewodowego noda systemu pagerowego. Kod aplikacyjny znajduje się w `Core/App` i obejmuje:

- warstwie radiowa SX1276/RFM95 z backendami LoRa, FSK i OOK,
- protokół ramki `LAVIET_FRAME_V1`,
- kryptografii AES-CTR oraz HMAC-SHA256,
- zadanie security z TPM ST33KTPM2X i magazynem I2C EEPROM/FRAM,
- lokalny interfejs LCD, menu, przyciski i LED,
- sensory BMP280 oraz VL53L3CX,
- opcjonalna konsola serwisowa UART.

1.2 Punkty startowe

Najważniejsze pliki publicznego API:

- `Core/App/app.h` - inicjalizacja aplikacji i wspólne wrappery I2C,
- `Core/App/radio_main.h` - zadanie radiowe i komendy runtime,
- `Core/App/security_main.h` - zadanie security, klucze i trusted storage,
- `Core/App/laviet_frame.h` - parser i serializer ramki LAVIET,
- `Core/App/laviet_crypto.h` - HMAC-SHA256, AES-CTR i porównanie MAC,
- `Core/App/menu_main.h` oraz `Core/App/lcd_main.h` - lokalny UI.

1.3 Strony opisowe

- [Generowanie dokumentacji](#)
- [Architektura firmware](#)
- [Mapa modułow](#)
- [Bezpieczeństwo](#)

1.4 Generowanie dokumentacji

Z katalogu głównego projektu:

```
doxygen Doxyfile
```

Plik startowy HTML powstaje w:

```
docs/doxygen/output/html/index.html
```

1.5 Generowanie dokumentacji

1.5.1 Wymagania

Do wygenerowania dokumentacji HTML potrzebny jest Doxygen. Do pliku PDF potrzebny jest też LaTeX z `pdflatex` i `makeindex`.

Na Ubuntu zestaw narzędzi można zainstalować komendą:

```
sudo apt-get install doxygen graphviz texlive-latex-base texlive-latex-extra texlive-fonts-recommended texlive-lang-polish
```

1.5.2 Komenda

Uruchom generator HTML i LaTeX z katalogu głównego repozytorium:

```
doxygen Doxyfile
```

Wynik HTML:

```
docs/doxygen/output/html/index.html
```

PDF powstaje z wygenerowanych plików LaTeX:

```
make -C docs/doxygen/output/latex
```

Wynik PDF:

```
docs/doxygen/output/latex/refman.pdf
```

1.5.3 Zakres wejścia

Konfiguracja dokumentuje:

- `README.md`,
- `Core/App`,
- `docs/doxygen`.

Katalogi `Drivers`, `Middlewares`, `Debug`, `.git` i `.metadata` są wykluczone. Dzięki temu dokumentacja skupia się na kodzie aplikacji BEKO, a nie na wygenerowanym lub zewnętrznym kodzie HAL/BSP/FreeRTOS.

1.5.4 Polityka plików wynikowych

Wygenerowany HTML jest ignorowany przez Git przez wpis:

```
docs/doxygen/output/
```

W repo zostają tylko źródła dokumentacji i konfiguracja.

1.6 Architektura firmware

1.6.1 Warstwy

Firmware jest podzielony na kilka warstw aplikacyjnych:

- warstwa startowa i wspólne zasoby: `app.c`, `app.h`,
- warstwa radiowa aplikacji: `radio_main.c`, `radio_main.h`,
- biblioteka radiowa i backendy modulacji: `radio_lib/`,
- ramka i kryptografia LAVIET: `laviet_frame.*`, `laviet_crypto.*`,
- security i storage: `security_main.*`, `i2c_mem_store_lib/`, `st33ktpm2x_lib/`,
- lokalny UI: `menu_main.*`, `lcd_main.*`, `button_main.*`, `lcd_library/`,
- sensory i sygnalizacja: `bmp280_*`, `tof_*`, `led_array_*`,
- konsola serwisowa: `service.*`.

1.6.2 Zadania RTOS

Główne moduły aplikacji działają jako zadania CMSIS-RTOS2/FreeRTOS:

- `radio_main_create_task()` tworzy zadanie radia i kolejkę komend,
- `security_main_create_task()` tworzy zadanie security i kolejkę komend,
- `menu_main_create_task()` obsługuje ekran menu oraz notyfikacje,
- `lcd_main_create_task()` zarządza fizycznym odświeżaniem LCD,
- `button_main_create_task()` wykonuje polling/debounce przycisków,
- `service_uart_create_task()` tworzy opcjonalną konsolę UART,
- `bmp280_main_create_task()`, `tof_main_create_task()` i `led_array_main_create_task()` uruchamiają zadania peryferyjne.

1.6.3 Wspólna magistrala I2C

Dostęp do I2C jest serializowany przez helpery z `app.h`. Moduły korzystają z `app_i2c_master_transmit()`, `app_i2c_master_receive()`, `app_i2c_mem_write()` i `app_i2c_mem_read()`, żeby uniknąć kolizji na wspólnej magistrali oraz mieć jedno miejsce odzyskiwania po błędach magistrali.

1.6.4 Runtime configuration

[radio_main_runtime_cfg_t](#) opisuje pełny snapshot profili radiowych i flag runtime. [security_runtime_cfg_t](#) rozszerza go o ustawienia security i dane utrwalane w magazynie.

1.7 Mapa modułów

1.7.1 Core application

- [app.h](#) - inicjalizacja aplikacji, start RTOS i wrappery I2C.
- [pager_config.h](#) - przełączniki build/board.
- [service.h](#) - opcjonalna konsola UART.

1.7.2 Radio

- [radio_main.h](#) - wysokopoziomowe komendy: wysyłanie, pairing, hopping, auto-ping i rekonfiguracja profili.
- [radio_lib/radio_lib.h](#) - wspólna fasada na backendy modulacji.
- [radio_lib/modulations/loro/radio_lora.h](#) - backend LoRa.
- [radio_lib/modulations/fsk/radio_fsk.h](#) - backend FSK/GFSK/MSK/GMSK.
- [radio_lib/modulations/ook/radio_ook.h](#) - backend OOK.
- [radio_lib/common/sx1276/radio_sx1276.h](#) - niskopoziomowy dostęp SPI do rejestrów SX1276.

1.7.3 Protocol and crypto

- [laviet_frame.h](#) - struktura, walidacja, kodowanie i dekodowanie ramki.
- [laviet_crypto.h](#) - HMAC-SHA256, AES-CTR, self-test i software fallback.

1.7.4 Security and storage

- [security_main.h](#) - trusted devices, klucze, country, TPM bootstrap.
- [st33ktpm2x_lib/st33ktpm2x.h](#) - sterownik TPM2/TIS/FIFO przez I2C.
- [i2c_mem_store_lib/i2c_mem_store.h](#) - EEPROM/FRAM log i encrypted secret slots.

1.7.5 User interface

- [menu_main.h](#) - stan menu i notyfikacje z innych subsystemów.
- [lcd_main.h](#) - kolejka renderowania ekranów.
- [lcd_library/lcd.h](#) - driver LCD HD44780 po ekspanderze I2C.
- [button_main.h](#) - polling i debounce przycisków.
- [led_array_lib/led_array_lib.h](#) - efekty i PWM LED.

1.7.6 Sensors

- [bmp280_lib/bmp280.h](#) - niskopoziomowy driver BMP280/BME280.
- [bmp280_lib/bmp280_api.h](#) - aplikacyjny wrapper single-shot.
- [vl53l3cx_lib/vl53l3cx_lib.h](#) - aplikacyjny wrapper VL53L3CX.

1.8 Bezpieczeństwo

1.8.1 LAVIET_FRAME_V1

Ramka LAVIET zawiera:

- wersje i typ,
- flagi,
- identyfikatory źródła i celu,
- `msg_id`,
- licznik anti-replay,
- payload do 16 bajtów,
- 32-bajtowy `mac_tag`.

Szczegóły API są w [laviet_frame.h](#).

1.8.2 Kryptografia

`laviet_crypto` udostępnia:

- HMAC-SHA256 dla integralności i autentyczności ramek,
- AES-CTR dla szyfrowania payloadu bez paddingu,
- stały czas porównania MAC przez `laviet_mac_equal()`,
- bezpieczne czyszczenie buforów przez `laviet_secure_zero()`.

Moduł próbuje użyć sprzętowych bloków HASH/CRYPT, ale zachowuje software fallback, gdy sprzętowy self-test nie przejdzie.

1.8.3 TPM

`st33ktpm2x` implementuje transport ST33KTPM2X przez I2C/TIS/FIFO i podstawowe komendy TPM2. `security_main` używa TPM jako punktu zaufania dla root seed, materiału kluczowego i operacji wymagających Physical Presence.

1.8.4 Storage

`i2c_mem_store` dzieli zewnętrzną pamięć I2C na:

- ring log odebranych wiadomości,
- szyfrowane sloty sekretów/trusted devices.

Sekretny payloady są chronione przez XTEA-CTR i CRC w warstwie storage.

Rozdział 2

BEKO Pager Network (pager-rtos)

2.1 1. Opis systemu

BEKO to bezprzewodowy system pagerowy pracujący w topologii **gwiazdy**.
W systemie występuje:

- jeden węzeł centralny: **Raspberry Pi Gateway**,
- adresy node'ów STM32 w zakresie 0x0002 . . 0xFFFFE,
- komunikacja radiowa przez obecny backend **SX1276/RFM95**.

Firmware noda przechowuje obecnie do **15 zaufanych urządzeń** w lokalnych slotach trusted storage. Nie jest to limit adresacji radiowej.

Gateway wysyła wiadomości do wybranego node'a albo do całej sieci.

Każda poprawnie odebrana wiadomość z gatewaya musi zostać potwierdzona przez node ramką ACK.

2.2 2. Adresacja

Adresacja w ramce LAVIET_FRAME_V1 jest 16-bitowa:

- 0x0000 – wartość nieważna,
- 0x0001 – Raspberry Pi Gateway,
- 0x0002 . . 0xFFFFE – adresy node'ów,
- 0xFFFF – broadcast.

W systemie może działać:

- **1 gateway**,
 - wiele urządzeń końcowych w zakresie adresów 0x0002 . . 0xFFFFE,
 - aktualnie do **15 sparowanych wpisów trusted** na jednym nodzie.
-

2.3 3. Role użytkowników

W systemie przewidziane są cztery role:

2.3.1 Administrator

Ma pełny dostęp do systemu.

Może zmieniać konfigurację gatewaya i node'ów, zarządzać parowaniem, licznikami bezpieczeństwa, kluczami i ustawieniami krytycznymi.

Dostęp wymaga podania odpowiedniego PIN-u administratora.

2.3.2 Operator

Ma ograniczony dostęp administracyjny.

Może wykonywać działania, które nie naruszają integralności całego systemu, np. wysyłać wiadomości, zmieniać wybrane ustawienia użytkowe i obsługiwać system operacyjnie.

2.3.3 Użytkownik

Ma dostęp tylko do podstawowych funkcji node'a.

Może odczytać wiadomość, wysłać odpowiedź przyciskami i korzystać z urządzenia w normalnym trybie pracy.

2.3.4 Serwisant

Ma fizyczny dostęp do urządzenia.

Może podłączyć się do node'a lub gatewaya w celu diagnostyki, odczytu logów, testów i czynności serwisowych.

Dostęp serwisowy powinien być kontrolowany i logowany.

2.4 4. Ochrona ustawień lokalnych

Każdy node posiada lokalny tryb konfiguracji.

Dostęp do ustawień wymaga podania **4-cyfrowego PIN-u**.

PIN:

- chroni lokalną konfigurację,
- może zostać zmieniony,
- nie może być przechowywany w postaci jawnej.

Powinien być przechowywany w postaci zabezpieczonej, np. jako wartość chroniona z użyciem HMAC lub materiału powiązanego z TPM.

PIN służy do kontroli dostępu, a nie jako główny sekret kryptograficzny systemu.

2.5 5. Ramka LAVIET_FRAME_V1

System wykorzystuje zwartą ramkę aplikacyjną LAVIET_FRAME_V1.

2.5.1 Struktura ramki

- `ver_type` – 1 B
- `flags` – 1 B
- `src_id` – 2 B
- `dst_id` – 2 B
- `msg_id` – 2 B
- `counter` – 4 B
- `payload_len` – 1 B
- `payload` – 0–16 B
- `mac_tag` – 32 B

2.5.2 Rozmiar ramki

Część stała zajmuje:

- $1 + 1 + 2 + 2 + 2 + 4 + 1 + 32 = 45$ B

Całkowity rozmiar ramki:

- minimum: 45 B
- maksimum: 61 B

Ramka mieści się w limicie 64 B i zostawia 3 B rezerwy.

2.6 6. Znaczenie pól `ver_type` i `flags`

2.6.1 `ver_type`

Pole 1-bajtowe:

- starsze 4 bity – wersja protokołu,
- młodsze 4 bity – typ wiadomości.

Przykładowe typy:

- 0x1 – DATA
- 0x2 – ACK
- 0x3 – RESP
- 0x4 – PAIR_REQ
- 0x5 – PAIR_RESP
- 0x6 – CFG
- 0x7 – COUNTER_SYNC
- 0x8 – KEY_ROTATE
- 0x9 – ERROR

2.6.2 `flags`

Pole 1-bajtowe:

- bit 0 – ENCRYPTED
 - bit 1 – ACK_REQUIRED
 - bit 2 – IS_ACK
 - bit 3 – PAIRING
 - bit 4 – CONFIG_ACCESS
 - bit 5 – BROADCAST
 - bit 6 – COUNTER_OVERRIDE
 - bit 7 – KEY_UPDATE
-

2.7 7. Zabezpieczenia

Szczegółowy opis mechanizmów kryptografii, TPM, EEPROM, pairingu, gatewaya i znanych ograniczeń jest w `gateway_docs/SECURITY_CRYPTO_OVERVIEW.md`.

System chroni:

- poufność wiadomości,
- integralność ramek,
- autentyczność nadawcy,
- licznik anti-replay,
- konfigurację urządzenia,
- klucze kryptograficzne.

2.7.1 HMAC-SHA256

Każda ramka zawiera `mac_tag` o długości **32 B**.

Do uwierzytelniania i kontroli integralności używany jest **HMAC-SHA256**.

HMAC liczony jest po:

`ver_type || flags || src_id || dst_id || msg_id || counter || payload_len || payload`

Firmware wykonuje self-test backendu **CRYPT/HASH** STM32U545. Jeżeli test przejdzie, używany jest backend sprzętowy; jeżeli nie, `laviet_crypto` przechodzi na programowy fallback, zachowując ten sam kontrakt `LAVIET_FRAME_V1`.

2.7.2 Szyfrowanie wiadomości

Payload wiadomości szyfrowany jest w trybie:

- **AES-CTR**

Zalety:

- brak paddingu,
- ciphertext ma taką samą długość jak plaintext,
- dobrze pasuje do krótkiego payloadu 0–16 B.

Jeżeli flaga `ENCRYPTED` jest ustawiona, odbiornik musi:

1. zweryfikować HMAC,
2. sprawdzić licznik `counter`,
3. dopiero wtedy odszyfrować payload.

2.8 8. TPM i klucze

TPM jest głównym punktem zaufania w systemie.

Wszystkie operacje bezpieczeństwa, które mogą być wykonane w TPM i które są wspierane w przyjętej architekturze, powinny być wykonywane właśnie tam.

TPM jest używany do:

- przechowywania sekretu głównego urządzenia,
- ochrony kluczy,
- generowania danych losowych,
- ochrony sekretów EEPROM,

- wsparcia lokalnej rotacji root seeda.

Aktualny lifecycle sekretu:

1. Przy starcie firmware inicjalizuje TPM przed storage.
2. Root seed jest odczytywany z TPM NV.
3. Główny, PP-protected index to 0x01C10101.
4. Starszy index 0x01C10100 jest odczytywany tylko jako ścieżka migracyjna.
5. Jeśli seed musi zostać utworzony albo nadpisany, zapis do TPM NV wymaga aktywnego pinu **TPM PP**.
6. Pin PP jest podłączony do samego TPM, pin 7, aktywny stanem VDD; nie jest odczytywany jako GPIO STM32.
7. Klucz szyfrowania sekretów EEPROM jest wyprowadzany z TPM-backed root seeda.

Index 0x01C10101 jest definiowany z PPWRITE, OWNERREAD i NO_DA. W praktyce rotacja klucza z menu *Security -> Keys* wymaga trzymania przycisku TPM PP; bez tego UI pokaże Hold TPM PP button.

Klucze nie mogą być przechowywane w firmware w postaci jawnej.

Są wyprowadzane lub ładowane bezpiecznie przy starcie, a następnie używane tylko tymczasowo.

2.9 9. Anti-replay i synchronizacja licznika

Każda ramka zawiera pole `counter` o długości 4 B.

Licznik:

- rośnie monotonicznie,
- służy do ochrony przed replay,
- jest sprawdzany po stronie odbiorcy.

Gateway może wysłać specjalną wiadomość `COUNTER_SYNC`, aby nadpisać licznik node'a.

Do tego celu używana jest flaga `COUNTER_OVERRIDE`.

Taka operacja:

- jest dostępna tylko dla gatewaya,
 - musi być zabezpieczona HMAC,
 - powinna być logowana.
-

2.10 10. Parowanie i rotacja kluczy

2.10.1 Parowanie

Parowanie odbywa się wyłącznie z gatewayem. Próg RSSI -20 dBm jest traktowany jako ostrzeżenie diagnostyczne do walidacji praktycznej, a nie jako twarda blokada w pierwszym wdrożeniu.

Przebieg:

1. Gateway wysyła `PAIR_REQ`.
 2. Node przechodzi w tryb parowania.
 3. Użytkownik lokalnie potwierdza operację.
 4. TPM wspiera ustanowienie materiału kryptograficznego.
 5. Zapisywana jest relacja zaufania.
 6. Wyprowadzane są klucze robocze.
-

2.10.2 Rotacja kluczy

System powinien wspierać rotację kluczy z użyciem **Diffie–Hellman**. Rotacja może być uruchamiana:

- okresowo,
- po określonej liczbie wiadomości,
- na żądanie administratora,
- po incydencie bezpieczeństwa.

Lokalna rotacja root seeda na nodzie jest już zabezpieczona TPM PP. Radiowy protokół KEY_ROTATE i docelowa wymiana DH/ECDH nadal wymagają dokończenia jako osobny przepływ.

2.11 11. Logi UART i kontrola startu

Podczas uruchamiania urządzenia przez UART powinny być wypisywane informacje o inicjalizacji:

- MCU,
- TPM,
- SX1276/RFM95,
- wyświetlacza,
- przycisków,
- LED,
- EEPROM,
- materiału kluczowego,
- konfiguracji bezpieczeństwa,
- zgodności firmware.

Dodatkowo czasy inicjalizacji ważnych modułów powinny być mierzone.

Jeżeli któryś element uruchamia się poza oczekiwanym zakresem czasowym, system powinien zgłosić anomalię na UART.

2.12 12. Główny graf systemu

```
graph TD
    A["Administrator"] --> WEB["Panel / interfejs systemu"]
    O["Operator"] --> WEB
    U["Użytkownik"] --> NODE["Node STM32"]
    A --> NODE
    S["Serwisant"] --> DEV["Dostęp fizyczny / UART / serwis"]

    WEB --> GW["Raspberry Pi Gateway"]

    GW --> N1["Node STM32 #1"]
    GW --> N2["Node STM32 #X"]
    GW --> N3["Node STM32 #15 trusted"]

    N1 --> UI1["Wyświetlacz"]
    N2 --> UI2["Wyświetlacz"]
    N3 --> UI3["Wyświetlacz"]

    DEV --> GW
    DEV --> N2
```

2.13 13. Główne przepływy komunikacji

2.13.1 Zwykła wiadomość z ACK

```
sequenceDiagram
    participant O as Operator / Administrator
    participant GW as Raspberry Pi Gateway
    participant N as Node STM32
    participant U as Użytkownik

    O->>GW: Wysłanie wiadomości
    GW->>GW: Budowa ramki
    GW->>GW: AES-CTR payload
    GW->>GW: HMAC-SHA256
    GW->>N: DATA
    N->>N: Weryfikacja HMAC i counter
    N->>N: Odszyfrowanie payloadu
    N->>U: Wyświetlenie wiadomości
    N->>GW: ACK
    GW->>O: Wynik operacji
```

2.13.2 Parowanie

```
sequenceDiagram
    participant A as Administrator
    participant GW as Raspberry Pi Gateway
    participant N as Node STM32
    participant TPM as TPM
    participant U as Użytkownik

    A->>GW: Uruchomienie parowania
    GW->>N: PAIR_REQ
    N->>U: Prośba o potwierdzenie
    U->>N: Potwierdzenie lokalne
    GW->>TPM: Operacje bezpieczeństwa
    N->>TPM: Operacje bezpieczeństwa
    GW->>N: Wymiana danych parowania
    N->>GW: PAIR_RESP
    GW->>A: Wynik operacji
```

2.13.3 Synchronizacja licznika

```
sequenceDiagram
    participant A as Administrator
    participant GW as Raspberry Pi Gateway
    participant N as Node STM32

    A->>GW: Żądanie synchronizacji licznika
    GW->>N: COUNTER_SYNC
    N->>N: Weryfikacja HMAC
    N->>N: Aktualizacja counter
    N->>GW: ACK
    GW->>A: Wynik operacji
```

2.13.4 Rotacja kluczy

```
sequenceDiagram
    participant A as Administrator
    participant GW as Raspberry Pi Gateway
    participant N as Node STM32
    participant TPM as TPM

    A->>GW: Uruchomienie rotacji kluczy
    GW->>N: KEY_ROTATE
    N->>TPM: Wymagana aktywna linia PP dla zapisu root seeda
    GW->>TPM: Operacje DH / materiał kluczowy
    N->>TPM: Operacje DH / materiał kluczowy
    GW->>N: Wymiana danych
    N->>GW: Odpowiedź
    N->>GW: ACK
    GW->>A: Wynik operacji
```

2.14 14. Dokumentacja Doxygen

Projekt zawiera konfigurację Doxygen w pliku Doxyfile.
Generowanie dokumentacji:

doxygen Doxyfile

Wynik HTML:

docs/doxygen/output/html/index.html

Wynik PDF:

make -C docs/doxygen/output/latex

docs/doxygen/output/latex/refman.pdf

Zakres dokumentacji obejmuje README.md, Core/App oraz strony opisowe w docs/doxygen. Katalogi wygenerowane i zewnętrzne (Debug, Drivers, Middlewares) są celowo wykluczone.

Rozdział 3

BEKO Security Architecture

Ten dokument opisuje, jak firmware BEKO chroni komunikację radiową, sekrety lokalne, TPM, AES/HMAC, EEPROM, liczniki anti-replay i dostęp lokalny przez menu.

Dokument jest pisany z perspektywy aktualnego kodu firmware STM32 w katalogu `Core/App`. Jeżeli backend gatewaya albo frontend RPi zmieni protokół, ten plik należy zaktualizować razem z kodem.

3.1 1. Skróty

Najważniejsze decyzje bezpieczeństwa:

- TPM jest lokalnym root-of-trust dla noda.
- Root seed urządzenia jest przechowywany w TPM NV index `0x01C10101`.
- Zapis root seeda do głównego indexu wymaga TPM Physical Presence (PPWRITE).
- TPM PP jest podłączony bezpośrednio do TPM, pin 7, aktywny stanem VDD.
- TPM komunikuje się z MCU przez I2C3, PC0=SCL, PC1=SDA.
- `TPM_RESET#` jest na PB0, aktywny niskim stanem.
- `TPM_DAVINT#` / `PIRQ` jest na PH0, aktywny niskim stanem.
- Sekrety w EEPROM są szyfrowane kluczem wyprowadzonym z root seeda TPM.
- Payload radiowy jest szyfrowany AES-CTR.
- Integralność i autentyczność ramki zapewnia HMAC-SHA256.
- HMAC obejmuje nagłówek i payload, czyli także `src_id`, `dst_id`, `msg_id` i `counter`.
- Odbiornik weryfikuje HMAC przed odszyfrowaniem payloadu.
- Porównanie MAC jest stałoczasowe względem długości tagu.
- Backend crypto używa sprzętowego AES/HASH STM32, jeżeli self-test przejdzie.
- Jeżeli hardware crypto nie przejdzie self-testu, firmware używa software fallback.
- Klucze i bufor tymczasowe są zerowane po użyciu przez `laviet_secure_zero()`.
- Dostęp do menu chronią role lokalne i PIN operator/admin.
- Logi TPM TIS można włączyć przez `TPM_INIT_LOG`; domyślnie verbose TPM init jest schowany.
- Logowanie wrażliwych payloadów na UART jest kontrolowane przez `PAGER_CONFIG_UART_SENSITIVE_LOGS`.

W uproszczeniu:

```
TPM NV root seed
|
+-- HMAC(seed, "SEC:NET:ROOT") -> lokalny network key
|
+-- HMAC(seed, "SEC:EEPROM:KEY") -> klucz szyfrowania sekretów EEPROM

Pair code + local_id + peer_id
|
+-- peer link base key
|
+-- HMAC(base, "LV1K" + domain + 0x01) -> AES-CTR enc key
|
+-- HMAC(base, "LV1K" + domain + 0x02) -> HMAC-SHA256 key
```

3.2 2. Cele bezpieczeństwa

System chroni cztery główne obszary:

1. Poufność wiadomości radiowych.
2. Integralność i autentyczność ramek.
3. Trwałe sekrety noda: root seed, trusted devices, pair code, country.
4. Lokalny dostęp operatora/administratora.

System nie zakłada, że radio jest zaufane. Kanał radiowy jest traktowany jako publiczny:

- każdy może podsłuchać,
- każdy może wysłać bajty,
- każdy może powtórzyć starą transmisję,
- każdy może próbować floodować,
- pakiety mogą zginąć albo przyjść po czasie.

Dlatego bezpieczeństwo nie opiera się na LoRa/FSK/OOK jako takich. Radio jest tylko transportem. Ochrona jest w warstwie aplikacyjnej LAVIET_FRAME_V1.

3.3 3. Model atakującego

Zakładany atakujący może:

- słuchać kanału radiowego,
- wysłać własne pakiety SX1276/RFM95,
- powtarzać stare ramki,
- próbować zgadnąć `src_id`,
- próbować podszyć się pod gateway albo node,
- odczytać zewnętrzny EEPROM po fizycznym dostępie do płytki,
- obserwować UART, jeżeli ma dostęp serwisowy,
- zresetować zasilanie w niekorzystnym momencie.

Zakładany atakujący nie powinien móc:

- odczytać sekretów z TPM bez obejścia zabezpieczeń TPM,
- zapisać nowego root seeda bez fizycznej obecności TPM PP,

- wygenerować poprawnego HMAC bez właściwego klucza,
- zmodyfikować zaszyfrowanej wiadomości tak, żeby przeszła HMAC,
- udawać sparowanego peer-a po zakończonym pairingu bez pair code.

Ważne: jeżeli atakujący ma pełny dostęp debug/SWD do działającego MCU i może czytać RAM, może próbować odczytywać klucze w czasie użycia. Ten projekt ogranicza czas życia sekretów w RAM, ale nie zastępuje pełnej ochrony debug portu, secure boot ani TrustZone policy.

3.4 4. Granice bezpieczeństwa

System ma kilka granic zaufania:

Granica	Co jest zaufane	Co jest niezaufane
MCU firmware	kod firmware, jeśli flash/debug są kontrolowane	radio RX, payloady, zewnętrzny świat
TPM	root seed i operacje NV/PP	I2C jako transport, dopóki komenda nie przejdzie TPM
EEPROM	nośnik danych	zawartość EEPROM bez weryfikacji CRC i deszyfrowania
Radio	nic samo z siebie	każda ramka przed HMAC
Menu lokalne	świadoma interakcja użytkownika	przypadkowe naciśnięcia i nieuprawniony operator

Najważniejsza zasada: żadna ramka radiowa nie powinna dostać się do logiki aplikacji jako zaufana, dopóki:

1. parser nie potwierdzi formatu,
2. HMAC nie pasuje,
3. polityka adresacji i typu nie jest poprawna,
4. counter/replay policy nie jest spełniona,
5. payload, jeżeli zaszyfrowany, nie zostanie odszyfrowany poprawnym kluczem.

3.5 5. Komponenty implementacji

Główne pliki:

Plik	Rola
Core/App/security_main.c	TPM bootstrap, root seed, storage key, trusted devices, runtime security config
Core/App/st33ktpm2x_lib/st33ktpm2x.c	driver ST33KTPM2X/TIS over I2C, komendy TPM2
Core/App/laviet_crypto.c	AES-CTR, HMAC-SHA256, hardware backend, fall-back software, self-test
Core/App/laviet_frame.c	parser, serializer, walidacja LAVIET_FRAME_V1, input do HMAC
Core/App/radio_main.c	TX/RX ramek, ACK, pairing, counter, szyfrowanie i MAC
Core/App/i2c_mem_store_lib/i2c_mem_store.c	EEPROM storage, CRC, secret slots, XTEA-CTR dla małych rekordów

Plik	Rola
<code>Core/App/menu_main.c</code>	role lokalne, PIN operator/admin, dostęp do menu Security
<code>Core/App/pager_config.h</code>	flagi produkcyjne/logowania

3.6 6. TPM jako root-of-trust

TPM jest używany po to, żeby główny sekret urządzenia nie był stałą w firmware ani zwykłym bajtem w EEPROM. Obecny TPM:

- model/rodzina: ST33KTPM2X,
- adres I2C 7-bit: 0x2E,
- magistrala: I2C3,
- linie: PC0=SCL, PC1=SDA,
- reset: TPM_RESET# na PB0, aktywny GND,
- interrupt/data available: TPM_DAVINT# na PH0, aktywny GND,
- physical presence: TPM pin 7, aktywny VDD, podłączony do TPM, nie do GPIO MCU.

3.6.1 6.1. Dlaczego TPM jest bezpieczniejszy niż EEPROM

EEPROM jest tylko pamięcią. Jeśli ktoś fizycznie odczyta EEPROM, dostaje zapisane bajty. TPM jest układem bezpieczeństwa:

- ma własne komendy i polityki autoryzacji,
- może wymagać platform auth albo physical presence,
- nie udostępnia NV write bez spełnienia atrybutów indexu,
- wykonuje TPM2_GetRandom,
- separuje trwały sekret od zwykłej pamięci aplikacji,
- pozwala przechowywać root seed poza firmware.

W tym projekcie TPM nie jest używany jako “magiczne szyfrowanie wszystkiego”. TPM jest korzeniem hierarchii kluczy. To oznacza, że z niego pochodzi materiał, który potem zasila klucze robocze.

3.7 7. TPM init

Podczas startu `security_main` wykonuje:

1. Konfigurację drivera TPM przez `st33ktpm2x_default_cfg(&cfg, &hi2c3)`.
2. Hard reset TPM przez `TPM_RESET#`.
3. Opcjonalny odczyt `DAVINT#`.
4. Skan/probe I2C3, jeśli `TPM_INIT_LOG` jest aktywny.
5. Odczyt identyfikacji TPM (`DID_VID`, `RID`).
6. `TPM2_Startup(ST_CLEAR)`.
7. `TPM2_SelfTest(full_test=false)`.
8. Ustawienie `s_tpm_ready=true`, jeśli transport i podstawowe komendy działają.

Przykładowe logi poprawnego startu:

```
SEC: TPM ready DIDVID=0x0003104A RID=0x01 self_test_tpm_rc=0x0000090A
SEC: TPM NV read idx=0x01C10101 rc=0(OK) tpm_rc=0x00000000 len=8 nonzero=1
SEC: root seed loaded from TPM
SEC: MEM secret key derived from TPM root seed
```

`self_test_tpm_rc=0x0000090A` oznacza odpowiedź TPM na self-test. Firmware traktuje transport TPM jako gotowy, jeżeli komendy krytyczne, np. NV read, działają poprawnie. To dlatego root seed może zostać odczytany mimo diagnostycznego kodu z self-testu.

3.7.1 7.1. Verbose logi TPM

Pełne logi TIS/FIFO są ukryte za `TPM_INIT_LOG`.

Domyślnie nie są wypisywane:

```
TPM TIS transceive start ...
TPM TIS request locality ...
TPM TIS data_avail ...
SEC: TPM reg8 DIDVID ...
```

Dlaczego to jest dobre:

- normalny UART nie jest zalewany danymi,
- mniej ryzyka przypadkowego wypisania informacji serwisowej,
- debug magistrali można włączyć tylko świadomie podczas diagnostyki.

3.8 8. Root seed

Root seed jest głównym trwałym sekretem noda.

Parametry:

Parametr	Wartość
Rozmiar	8 B (SECURITY_KEY_SEED_BYTES)
Główny TPM NV index	0x01C10101
Legacy TPM NV index	0x01C10100
Atrybuty głównego indexu	PPWRITE, OWNERREAD, NO_DA
Zapis	wymaga TPM Physical Presence
Odczyt	owner read

3.8.1 8.1. Dlaczego seed nie jest w firmware

Gdyby klucz był wpisany jako stała w kodzie:

- każdy egzemplarz miałby ten sam sekret,
- wyciek firmware ujawniłby sekret,
- nie byłoby sensownej rotacji,
- nie dałoby się związać sekretu z konkretną płytką.

W obecnym modelu firmware zna tylko algorytm:

```
root seed -> HMAC labels -> klucze robocze
```

Sam root seed jest ładowany z TPM podczas startu.

3.8.2 8.2. Lifecycle root seeda

Przy starcie firmware wykonuje:

1. Próbuje odczytać 0x01C10101.

2. Jeśli główny index nie ma danych, próbuje legacy index 0x01C10100.
3. Jeśli legacy seed istnieje, firmware próbuje migrować go do PP-protected index.
4. Jeśli nie ma żadnego seeda, firmware generuje nowy seed.
5. Jeśli TPM jest gotowy, nowy seed jest zapisywany do TPM NV.
6. Zapis do 0x01C10101 wymaga aktywnego TPM PP.
7. Seed jest używany do wyprowadzenia kluczy.
8. Bufory tymczasowe z seedem są zerowane.

Przykład:

```
SEC: TPM NV read idx=0x01C10101 rc=0 (OK) tpm_rc=0x00000000 len=8 nonzero=1
SEC: root seed loaded from TPM
SEC: MEM secret key derived from TPM root seed
```

Przykład, gdy zapis wymaga fizycznej obecności:

```
SEC: TPM NV write PP idx=0x01C10101 rc=7 (ETPM_RC) tpm_rc=...
SEC: root seed write requires TPM PP button
```

3.8.3 8.3. Physical Presence

Physical Presence (PP) jest ważne, bo chroni operacje, które zmieniają root seeda.

W obecnym hardware:

- przycisk PP jest podłączony do TPM pin 7,
- aktywny poziom to VDD,
- STM32 nie odczytuje PP jako GPIO,
- TPM sam egzekwuje warunek dla operacji PPWRITE.

To oznacza:

- firmware nie może "udawać" fizycznej obecności przez samo ustawienie GPIO,
- zdalny atak radiowy nie powinien móc nadpisać root seeda,
- operator musi fizycznie nacisnąć/aktywować PP podczas rotacji wymagającej zapisu.

3.8.4 8.4. Ograniczenie: seed ma 8 bajtów

Aktualny root seed ma 8 B, czyli 64 bity.

To jest istotne ograniczenie. Architektura hierarchii kluczy jest poprawna, ale dla długoterminowego bezpieczeństwa produkcyjnego lepiej zwiększyć seed do 16 B albo 32 B.

Dlaczego mimo tego obecny model jest lepszy niż stały klucz:

- seed nie jest stałą firmware,
- seed może być inny per urządzenie,
- seed jest w TPM,
- zapis wymaga PP,
- klucze robocze są separowane labelami HMAC.

Co poprawić docelowo:

```
SECURITY_KEY_SEED_BYTES: 8 -> 16 albo 32
TPM NV data_size:      8 -> 16 albo 32
test migracji EEPROM i TPM NV
```

3.9 9. Entropia i generowanie seeda

Firmware używa dwóch źródeł entropii:

1. Preferowane: TPM2_GetRandom.
2. Fallback: STM32 HAL RNG (HAL_RNG_GenerateRandomNumber).

Przepływ:

```
security_get_entropy_bytes()
    if TPM ready:
        TPM2_GetRandom(len)
        if OK: return
        else: log, disable TPM ready for entropy path

    HAL_RNG_GenerateRandomNumber()
```

Dlaczego to jest bezpieczne:

- TPM RNG jest niezależny od MCU,
- HAL RNG daje fallback, gdy TPM random nie jest dostępny,
- seed nie jest deterministyczny,
- brak RNG powoduje błąd init/rotacji, a nie użycie stałego seeda.

3.10 10. Hierarchia kluczy

Firmware nie używa jednego klucza do wszystkiego.

Z root seeda powstają oddzielne domeny:

Domena	Funkcja	Label
Network/root key	lokalny klucz sieciowy noda	SEC:NET:ROOT
EEPROM secret key	szyfrowanie secret slots EEPROM	SEC:EEPROM:KEY

Implementacyjnie:

```
network_key = HMAC-SHA256(seed, "SEC:NET:ROOT") [0..15]
store_key   = HMAC-SHA256(seed, "SEC:EEPROM:KEY") [0..15]
```

Dlaczego labelowanie jest bezpieczne:

- ten sam root seed nie jest używany bezpośrednio jako klucz AES,
- różne zastosowania dostają różne klucze,
- wyciek jednego klucza roboczego nie musi ujawniać drugiego,
- label utrudnia przypadkowe reuse materiału między protokołami.

Przykład:

```
seed = 8 losowych bajtów z TPM

HMAC(seed, "SEC:NET:ROOT")
-> digest 32 B
-> pierwsze 16 B jako network_key

HMAC(seed, "SEC:EEPROM:KEY")
-> digest 32 B
-> pierwsze 16 B jako EEPROM secret key
```

3.11 11. Klucze ramek radiowych

Ramki radiowe nie używają bezpośrednio root seeda TPM. Używają kluczy ramek wyprowadzanych z trybu relacji peer.

Są dwa główne tryby:

Tryb	Kiedy	Materiał bazowy
<code>SECURITY_FRAME_KEY_MODE_PAIR</code>	zwykła komunikacja sparowany peer <-> node	pair code / trusted entry
<code>SECURITY_FRAME_KEY_MODE_SHARED</code>	pairing, error, broadcast/control fall-back	stały shared frame root key

3.11.1 11.1. Pair link key

Dla sparowanego urządzenia firmware pobiera zapisany `pair code` z trusted storage i wyprowadza z niego klucz relacji:

```
lo = min(local_node_id, peer_node_id)
hi = max(local_node_id, peer_node_id)
base_key = HMAC-SHA256(pair_code, "SEC:PAIR:V1" || lo || hi)[0..15]
```

Relacja jest zależna od:

- lokalnego `node_id`,
- `peer_id`,
- `pair code`,
- trybu derivation.

Następnie powstają dwa oddzielne klucze:

```
info = "LV1K" || domain_id || 0x00 || purpose
```

```
purpose=0x01 -> AES-CTR enc_key 16 B
purpose=0x02 -> HMAC-SHA256 hmac_key 32 B
domain_id:
```

- dla broadcastu: `0xFFFF`,
- dla unicastu: mniejszy z `local_id` i `peer_id`.

Dlaczego to jest bezpieczne:

- AES i HMAC nie używają tego samego klucza,
- klucze zależą od pary urządzeń,
- ten sam `pair code` nie daje identycznych kluczy dla każdej relacji, jeśli identyfikatory są inne,
- HMAC jako KDF daje separację domen.

3.11.2 11.2. Shared mode

`SECURITY_FRAME_KEY_MODE_SHARED` używa stałego root key:

```
"LAVIET_SHARED_V1"
```

Ten tryb jest używany dla ramek, które muszą istnieć przed pełnym zaufaniem albo poza relacją `pair`:

- pairing,
- część ramek control/error,
- broadcast.

To nie daje tej samej siły co `pair-specific key`. Shared mode należy traktować jako tryb bootstrap/compatibility, a nie docelową ochronę poufnych danych użytkownika.

Bezpieczna zasada:

- zwykle wiadomości i odpowiedzi powinny używać `pair-derived keys`,
- broadcast powinien nie przenosić sekretów,
- pairing powinien mieć lokalne potwierdzenie użytkownika,
- docelowo pairing powinien dostać challenge-response lub TPM policy.

3.12 12. Ramka LAVIET_FRAME_V1

Struktura:

```

ver_type    1 B
flags       1 B
src_id      2 B
dst_id      2 B
msg_id      2 B
counter     4 B
payload_len 1 B
payload     0..16 B
mac_tag     32 B

```

Rozmiar:

```

min = 45 B
max = 61 B

```

Adresacja:

Adres	Znaczenie
0x0000	nieważny
0x0001	gateway
0x0002..0xFFFE	node
0xFFFF	broadcast

Typy:

Typ	Znaczenie
DATA	zwykła wiadomość
ACK	potwierdzenie dostarczenia
RESP	odpowiedź
PAIR_REQ	żądanie parowania
PAIR_RESP	odpowiedź parowania
CFG	konfiguracja
COUNTER_SYNC	synchronizacja countera
KEY_ROTATE	rotacja klucza
ERROR	błąd/protokół

Flagi:

Flaga	Znaczenie
ENCRYPTED	payload jest szyfrowany AES-CTR
ACK_REQUIRED	odbiorca ma odesłać ACK
IS_ACK	ramka jest ACK
PAIRING	ramka należy do pairingu
CONFIG_ACCESS	ramka konfiguracyjna
BROADCAST	dst to broadcast
COUNTER_OVERRIDE	specjalna zmiana countera
KEY_UPDATE	rotacja klucza

3.13 13. Walidacja ramek

`laviet_frame_validate_plain()` odrzuca:

- złą wersję protokołu,
- nieznany typ,
- `src_id=0`,
- `src_id=0xFFFF`,
- `dst_id=0`,
- `msg_id=0`,
- payload dłuższy niż 16 B,
- broadcast bez flagi `BROADCAST`,
- broadcast z `ACK_REQUIRED`,
- niepoprawne kombinacje flag dla typu.

Przykład:

```
DATA + IS_ACK -> błąd flag
ACK + payload_len != 6 -> błąd payload
COUNTER_SYNC bez COUNTER_OVERRIDE -> błąd flag
BROADCAST + ACK_REQUIRED -> błąd flag
```

Dlaczego to jest bezpieczne:

- tanie błędy są odrzucane przed kosztowną logiką,
- niepoprawne typy nie trafiają do handlerów,
- broadcast nie może wymusić ACK,
- `src_id` i `dst_id` mają ścisłą semantykę,
- payload ma twardy limit 16 B.

3.14 14. HMAC-SHA256

Każda ramka ma `mac_tag` 32 B.

MAC liczony jest po:

```
ver_type ||
flags ||
src_id ||
dst_id ||
msg_id ||
counter ||
payload_len ||
payload
```

`mac_tag` nie jest częścią inputu do HMAC.

Dlaczego to jest bezpieczne:

- zmiana payloadu zmienia HMAC,
- zmiana adresu nadawcy zmienia HMAC,
- zmiana adresu odbiorcy zmienia HMAC,
- zmiana `msg_id` zmienia HMAC,
- zmiana `counter` zmienia HMAC,
- zmiana flag, np. dopisanie `ENCRYPTED`, zmienia HMAC.

Atakujący może wysłać losową ramkę, ale bez `hmac_key` nie powinien być w stanie policzyć poprawnego `mac_tag`.

3.14.1 14.1. Kolejność RX

W RX kolejność jest krytyczna:

decode bytes -> validate plain -> derive keys -> verify HMAC -> decrypt -> handle

Payload jest odszyfrowywany dopiero po HMAC.

Dlaczego:

- AES-CTR jest strumieniowy i sam nie daje integralności,
- odszyfrowywanie losowych danych nie powinno sterować logiką,
- HMAC chroni ciphertext i nagłówki.

3.14.2 14.2. Stałoczasowe porównanie MAC

`laviet_mac_equal()` porównuje wszystkie 32 bajty i akumuluje różnice:

```
diff |= a[i] ^ b[i];
```

Nie kończy po pierwszej różnicy.

Dlaczego:

- ogranicza leak timingowy mówiący, ile bajtów MAC było poprawnych,
- jest prostą i właściwą praktyką przy tagach MAC.

3.15 15. AES-CTR

Payload jest szyfrowany AES-CTR, gdy flaga `ENCRYPTED` jest ustawiona.

Właściwości AES-CTR:

- nie wymaga paddingu,
- działa dla payloadów 1..16 B,
- ten sam algorytm szyfruje i deszyfruje,
- ciphertext ma tę samą długość co plaintext,
- wymaga unikalnego counter block dla danego klucza.

3.15.1 15.1. Counter block AES-CTR

Counter block ma 16 B:

```
0..2  "LV1"
3      0x00
4..5  src_id
6..7  dst_id
8..9  msg_id
10..13 frame.counter
14..15 block_index
```

Dla obecnego payloadu max 16 B `block_index` zwykle wynosi 0.

Przykład:

```
src_id  = 0x1234
dst_id  = 0x0001
msg_id  = 0x4567
counter = 0x89ABCDEF
block   = 0
```

counter block:

```
4C 56 31 00 12 34 00 01 45 67 89 AB CD EF 00 00
```

Dlaczego to jest bezpieczne:

- nonce/counter zależy od nadawcy, odbiorcy, `msg_id` i countera,
- powtórzenie tego samego counter block przy tym samym kluczu jest utrudnione przez monotoniczny counter i `msg_id`,
- HMAC obejmuje te same pola, więc atakujący nie może zmienić nonce bez uszkodzenia MAC.

3.15.2 15.2. AES-CTR nie jest autentykacją

AES-CTR sam nie chroni przed modyfikacją bitów.

Przykład problemu bez HMAC:

```
plaintext = "OK"
ciphertext = AES_CTR(plaintext)
atakujący zmienia bit ciphertextu
odbiorca dostaje zmieniony plaintext
```

W BEKO ten atak nie powinien przejść, bo HMAC jest liczony po zaszyfrowanym payloadzie i nagłówku. Zmieniony ciphertext da inny HMAC.

Dlatego bezpieczeństwo wiadomości to:

```
Poufność:      AES-CTR
Integralność:   HMAC-SHA256
Autentyczność: HMAC-SHA256 z kluczem relacji
Anti-replay:   counter policy
```

3.16 16. Hardware crypto STM32

`laviet_crypto_init()` sprawdza, czy:

- `hcrip.Instance == AES`,
- AES HAL jest READY,
- HASH HAL jest READY.

Potem wykonuje self-test:

1. HMAC software.
2. HMAC hardware.
3. Porównanie HMAC.
4. AES-CTR software dla 0 B.
5. AES-CTR hardware dla 0 B.
6. AES-CTR software dla 1 B.
7. AES-CTR hardware dla 1 B.
8. Porównanie 1 B.
9. AES-CTR software dla 16 B.
10. AES-CTR hardware dla 16 B.
11. Porównanie 16 B.
12. Roundtrip encrypt/decrypt.

Poprawny log:

```
CRYPTO: AES pre_self_test inst=0x420c0000 state=1(READY) err=0x00000000 ...
CRYPTO: HASH pre_self_test state=1(READY) err=0x00000000 ...
CRYPTO: hardware backend ready
```

Jeżeli test nie przejdzie:

```
CRYPTO: self-test mismatch stage=aes_1B_compare index=0 expected=0xAE actual=0x41 len=1
CRYPTO: hardware self-test failed, using software fallback
```

Firmware wtedy nadal działa, ale używa software fallback.

3.16.1 16.1. Dlaczego self-test jest ważny

AES/HASH HAL może być `READY`, ale wynik może być zły, np. przez:

- błędny `DataType`,
- złą endianowość `key/IV/input`,
- złą długość jednostki danych,
- błąd konfiguracji `.ioc`.

Self-test porównuje hardware z implementacją software i dopiero wtedy włącza `s_crypto_hw_ready`.

3.16.2 16.2. Pakowanie big-endian do AES

STM32 AES przyjmuje `key/IV/data` jako `uint32_t *`.

Firmware jawnie pakuje bajty protokołu do słów big-endian:

```
bytes[0..3] -> word = b0<<24 | b1<<16 | b2<<8 | b3
```

Po AES wynik jest rozpakowywany z powrotem do bajtów.

Dlaczego:

- ramka LAVIET jest big-endian,
- AES test vector software działa na kolejności bajtowej protokołu,
- rzutowanie `uint8_t[16]` na `uint32_t *` dawało odwrócenie bajtów na little-endian MCU,
- jawne pakowanie usuwa niejednoznaczność.

3.17 17. Software fallback

Software fallback istnieje jako zabezpieczenie dostępności.

Jeżeli hardware AES/HASH:

- nie jest gotowy,
- nie przejdzie self-testu,
- zwróci błąd HAL,
- nie da się zablokować mutexa,

to firmware używa implementacji software.

Dlaczego to jest bezpieczne:

- system nie przechodzi na "brak crypto",
- nadal liczy HMAC,
- nadal szyfruje AES-CTR,
- hardware path jest tylko akceleratorem, nie jedynym źródłem poprawności.

Ograniczenie:

- software AES/HMAC działa w RAM MCU i może być wolniejszy,
- przy pełnym modelu produkcyjnym trzeba też chronić debug/SWD i pamięć.

3.18 18. EEPROM secret storage

Zewnętrzny EEPROM jest traktowany jako niezaufany nośnik.

Przechowywane tam dane:

- metadata store,
- trusted devices,
- pair code,
- gateway counters,
- opcjonalne logi,
- runtime config.

Na M24C01-R pamięć ma tylko 128 B, więc obecnie:

```
secret_area_bytes = 72 B
secret_slot_size  = 24 B
secret_slots      = 3
log_slots         = 0
```

Przy tym układzie slot 2 jest zarezerwowany na gateway counters, więc trwale trusted devices mają pojemność:

```
trusted_slot 0 = gateway
trusted_slot 1 = dodatkowy node
```

Firmware nie powinien akceptować trusted device jako zapisanego, jeżeli wpis nie został rzeczywiście utwalony. Zapis trusted wykonuje write -> read-back -> compare; dopiero po poprawnej weryfikacji wpis trafia do RAM jako zaufany.

Log:

```
SEC: MEM store ready log_slots=0 secret_slots=3
SEC: MEM log disabled, storage reserved for trusted devices
```

3.18.1 18.1. Szyfrowanie secret slots

i2c_mem_store używa XTEA-CTR dla secret slots.

Klucz:

```
store_key = HMAC(root_seed, "SEC:EEPROM:KEY") [0..15]
```

Slot zawiera:

```
magic      2 B
length     1 B
slot_id    1 B
nonce      4 B
ciphertext 14 B
crc16      2 B
```

Dlaczego to jest bezpieczniejsze niż plaintext EEPROM:

- odczyt EEPROM nie ujawnia pair code bez store key,
- store key nie jest w EEPROM,
- store key pochodzi z TPM root seed,
- każdy zapis dostaje nonce z secret_counter,
- CRC wykrywa przypadkowe uszkodzenia i proste manipulacje formatu.

Ważne ograniczenie:

- CRC nie jest MAC.
- Integralność kryptograficzna EEPROM secret slots nie jest tak mocna jak HMAC.
- Docelowo warto dodać HMAC/AEAD dla rekordów EEPROM albo rozszerzyć format.

3.19 19. Trusted devices

Trusted entry zawiera:

```
node_id
code_len
pair_code
is_master
in_use
```

Slot 0 jest traktowany jako gateway/master slot.

Po sparowaniu:

1. pair code jest zapisany w trusted storage,
2. trusted storage trafia do encrypted secret slot EEPROM,
3. późniejsze ramki unicast używają pair-derived keys,
4. RX od nieznanego źródła jest odrzucany dla DATA i RESP.

Wyjątek: gateway (`src_id=0x0001`) może wysłać DATA/RESP jako broadcast (`dst_id=0xFFFF`) po poprawnym HMAC w domenie broadcast/shared. Taka ramka nie wymaga wpisu w trusted slotach, ale nie dostaje ACK i nie powinna przenosić sekretów. Unicast DATA/RESP nadal wymaga sparowanego źródła.

Przykład RX spoza sieci:

```
RADIO RX outside-network drop src=0x1234 dst=0x77CD type=DATA
```

Dlaczego to jest bezpieczne:

- samo zgadnięcie `src_id` nie wystarczy,
- trzeba mieć pair code, żeby wyprowadzić poprawny HMAC key,
- nawet po HMAC dodatkowo sprawdzana jest lista trusted dla zwykłych wiadomości.

3.20 20. Pairing

Pairing ma dwa warianty:

- zwykły pairing node-device,
- network pairing z gatewayem.

Ogólny przepływ:

1. Lokalnie uruchamiasz tryb parowania.
2. Node otwiera okno czasowe.
3. Druga strona wysyła PAIR_REQ albo PAIR_RESP.
4. Firmware parsuje pair payload.
5. Użytkownik lokalnie akceptuje.
6. Pair code trafia do trusted storage.
7. Kolejne ramki używają pair-derived keys.

Dlaczego pairing nie jest tylko radiowy:

- wymaga aktywnego lokalnego okna,
- wymaga lokalnej akceptacji,
- dla gatewaya slot może być zablokowany po zapisaniu,
- RX spoza okna parowania jest ignorowany.

Ograniczenie:

- shared pairing key jest statyczny,
- pair code ma ograniczoną długość,
- docelowo pairing powinien dostać challenge-response i/lub TPM policy session.

3.21 21. ACK

ACK jest osobną ramką `LAVIET_TYPE_ACK`.

Payload ACK:

```
acked_msg_id 2 B
acked_counter 4 B
```

Nadawca akceptuje ACK tylko, jeśli pasują:

- `src_id`,
- `msg_id`,
- `counter`.

Tu `msg_id` i `counter` oznaczają pola z payloadu ACK, czyli `acked_msg_id` i `acked_counter`. Zewnętrzne pole `counter` samej ramki ACK jest osobnym licznikiem TX nadawcy ACK i nie musi być równe `acked_counter`.

Przykład:

```
RADIO ACK wait dst=0x0001 msg=0x0042 counter=123 timeout=6000 ms
RADIO RX OK type=ACK src=0x0001 dst=0x77CD msg=0x2F5E counter=456 flags=0x04 len=6
RADIO RX ACK src=0x0001 ack_msg=0x0042 ack_counter=123
RADIO ACK delivered src=0x0001 msg=0x0042 counter=123 retries=0
```

Dlaczego to jest bezpieczne:

- przypadkowy ACK nie zamyka pending TX,
- opóźniony ACK jest rozpoznawany jako late/duplicate,
- ACK nie jest generowany dla broadcastu,
- ACK nie jest szyfrowany, ale jest MACowany jak każda ramka.

3.22 22. Counter i anti-replay

Ramka ma 32-bitowy `counter`.

Firmware przechowuje:

```
s_gateway_rx_counter
s_gateway_tx_counter
```

Gateway/backend powinien przechowywać analogiczne, rozdzielone wartości:

```
gateway_tx_counter      ostatni counter użyty przez gateway w ramce do noda
node_rx_counter[node_id] ostatni counter zaakceptowany od danego noda
```

`gateway_tx_counter` musi być trwały i monotoniczny. Backend nie powinien zerować go przy restarcie procesu. Dla każdej ramki `gateway -> node`:

```
counter = gateway_tx_counter + 1
persist gateway_tx_counter = counter
zbuduj nonce/AES/HMAC z tym counterem
wyślij ramkę
```

Dla ACK wysyłanego przez gateway zewnętrzny `frame.counter` też pochodzi z `gateway_tx_counter`, natomiast payload ACK musi zawierać `msg_id` i `counter` oryginalnej ramki noda:

```
ack_payload = acked_msg_id || acked_counter
```

Przy odbiorze ACK od noda backend musi dopasowywać potwierdzenie po payloadzie ACK (`acked_msg_id`, `acked_counter`), a nie po zewnętrznym `frame.counter` ramki ACK.

W EEPROM slot counter ma:

```
magic 0xC7
version 1
rx 4 B
tx 4 B
```

Dlaczego counter jest potrzebny:

- AES-CTR wymaga unikalnego counter block,
- HMAC chroni counter przed modyfikacją,
- odbiornik może wykryć stare ramki,
- ACK może być dopasowany do konkretnej transmisji.

Przykład replay:

```
last gateway counter = 100
przychodzi ramka DATA z counter = 95
odbiornik wykrywa replay/stary counter
```

W obecnym kodzie istnieje diagnostyczna ścieżka RADIO RX REPLAY bypass, co jest zaznaczone w `todo.md` jako zadanie do przywrócenia finalnej polityki anti-replay. Docelowo ramki z `counter <= last_` counter powinny być odrzucane po poprawnym HMAC, przed wykonaniem akcji aplikacyjnej.

Bezpieczna docelowa polityka:

```
if src == gateway and type != COUNTER_SYNC:
    if counter <= stored_rx_counter:
        drop as replay
    else:
        process
        persist new counter
```

3.23 23. COUNTER_SYNC

COUNTER_SYNC jest specjalną ramką gateway -> node.

Wymagania:

- `src_id == 0x0001`,
- typ COUNTER_SYNC,
- flaga COUNTER_OVERRIDE,
- payload 4 B,
- poprawny HMAC,
- payload zaszyfrowany.

Użycie:

```
payload = new_counter_be32
```

Jeżeli backend utracił trwały `gateway_tx_counter` albo node zapamiętał wyższy licznik niż backend, nie trzeba restartować firmware noda. Backend powinien wysłać kontrolną ramkę COUNTER_SYNC:

```
ver_type = 0x17                                # v1 + COUNTER_SYNC
flags    = 0x41                                # ENCRYPTED | COUNTER_OVERRIDE
flags    = 0x43                                # ENCRYPTED | ACK_REQUIRED | COUNTER_OVERRIDE, gdy backend chce ACK
src_id   = 0x0001
dst_id   = node_id
payload  = AES_CTR(new_counter_be32)
mac      = HMAC(header || encrypted_payload)
```

Po wysłaniu COUNTER_SYNC backend ustawia swój `gateway_tx_counter` na co najmniej `new_counter`, a następna zwykła ramka gateway -> node musi mieć `counter > new_counter`. `new_counter` nie powinien być mniejszy od wartości, którą node mógł już zapamiętać; w praktyce wybiera się aktualny trwały licznik backendu albo większą wartość z zapasem.

Dlaczego to jest bezpieczne:

- tylko gateway może wysłać poprawnie uwierzytelnioną synchronizację,
- counter sync nie jest zwykłą wiadomością,
- flaga COUNTER_OVERRIDE odróżnia operację od zwykłego DATA,
- HMAC chroni nową wartość countera.

Ryzyko:

- jeżeli gateway zostanie przejęty, może zsynchronizować counter.
- to jest akceptowany model: gateway jest centralnym zaufanym węzłem topologii gwiazdy.

3.24 24. Broadcast

Broadcast ma adres 0xFFFF.

Zasady:

- broadcast nie wymaga ACK,
- broadcast nie może mieć ACK_REQUIRED,
- node nie wysyła zwykłych wiadomości na broadcast,
- broadcast może być używany przez gateway,
- gateway broadcast DATA/RESP jest akceptowany po poprawnym HMAC shared/broadcast nawet przed trusted-list,
- broadcast nie powinien przenosić sekretów.

Gateway z branchu gateway dla coded=true broadcast używa broadcast_group_key, nie samego LAVIET_SHARED_V1. Node musi najpierw odebrać unicastowe KEY_ROTATE od gatewaya:

```
B7 01 epoch_be32 fragment_index fragment_count fragment[8]
B7 01 epoch_be32 fragment_index fragment_count fragment[8]
B7 02 epoch_be32 00 00 00 00 00 00 00 00 00 00 00 00
```

Po aktywacji node wyprowadza:

```
broadcast_base_key = HMAC_SHA256(
    key = broadcast_group_key[16],
    data = "LAVIET:BCAST:GROUP:V1" || epoch_be32 || 0xFFFF
)[0..15]
```

```
broadcast_aes_key = HMAC_SHA256(broadcast_base_key, "LV1K" || 0xFFFF || 00 || 01)[0..15]
broadcast_hmac_key = HMAC_SHA256(broadcast_base_key, "LV1K" || 0xFFFF || 00 || 02)
```

Dla zaszyfowanego broadcastu RX próbuje najpierw aktywny group key, a potem fallback SHARED. Jeśli KEY↔_ROTATE nie dotarł albo nie został aktywowany, flags=0x21 (ENCRYPTED | BROADCAST) zakończy się HMAC drop.

Dlaczego broadcast jest słabszy:

- wiele urządzeń odbiera tę samą ramkę,
- nie ma per-node ACK,
- w praktyce używa shared/broadcast key domain,
- nie daje takiego samego potwierdzenia dostarczenia jak unicast.

Bezpieczne użycie:

```
broadcast: krótkie komunikaty, wezwania, prośby o odpowiedź
unicast: dane wrażliwe, konfiguracja, ACK, odpowiedzi
```

3.25 25. Lokalny PIN i role

Menu ma role:

- operator,
- admin.

Domyślne PIN-y w kodzie:

```
operator: 0000
admin:    9999
```

PIN:

- ma 4 cyfry,
- może być włączony/wyłączony,
- może zostać zmieniony w Security -> PIN settings,

- unlock trwa 60000 ms,
- admin unlock odblokowuje też poziom operatora.

Dlaczego PIN jest użyteczny:

- ogranicza przypadkowe albo nieuprawnione wejście w menu,
- oddziela normalny tryb użytkownika od konfiguracji,
- wymaga świadomej lokalnej interakcji.

Ograniczenie:

- PIN w obecnym kodzie jest runtime/static, nie jest jeszcze trwale zapisany jako HMAC/TPM-protected secret.
- PIN nie jest głównym sekretem kryptograficznym.
- PIN nie zastępuje TPM PP.

Docelowo PIN powinien być zapisany jako:

```
pin_hash = HMAC(TPM-derived local auth key, salt || pin)
```

3.26 26. Logi UART

Logi są potrzebne do uruchamiania embedded, ale mogą ujawniać za dużo.

Bezpieczne zasady:

- nie logować root seeda,
- nie logować pełnych kluczy,
- nie logować pair code w trybie produkcyjnym,
- debug HMAC/payload trzymać za flagą produkcyjną,
- TPM verbose logi tylko z TPM_INIT_LOG.

Aktualne mechanizmy:

- TPM_INIT_LOG ukrywa szczegółowe logi TPM TIS,
- [PAGER_CONFIG_UART_SENSITIVE_LOGS](#) kontroluje wrażliwe logi UART,
- startup pokazuje statusy, ale nie powinien drukować sekretów.

Przykład dobrego startupu:

```
APP: FreeRTOS bootstrap
CRYPTO: hardware backend ready
SEC: TPM ready DIDVID=0x0003104A RID=0x01 self_test_tpm_rc=0x0000090A
SEC: TPM NV read idx=0x01C10101 rc=0(OK) tpm_rc=0x00000000 len=8 nonzero=1
SEC: root seed loaded from TPM
SEC: MEM secret key derived from TPM root seed
SEC: task ready
MENU: task ready
RADIO: init OK laviet_node=0x77CD
```

3.27 27. Dlaczego ta architektura jest bezpieczna

Bezpieczeństwo wynika z warstw, a nie z jednego mechanizmu.

3.27.1 27.1. Atak: podsłuch radiowy

Atakujący słyszy ciphertext.

Ochrona:

- payload jest szyfrowany AES-CTR,
- klucz AES jest pair-derived,
- counter block zawiera pola ramki,
- HMAC chroni nagłówek i ciphertext.

Efekt:

- podsłuch nie daje plaintextu,
- modyfikacja ciphertextu psuje HMAC.

3.27.2 27.2. Atak: zmiana payloadu

Atakujący zmienia bajt payloadu.

Ochrona:

- HMAC obejmuje payload,
- porównanie MAC wykrywa zmianę.

Efekt:

```
RADIO RX HMAC drop src=...
```

3.27.3 27.3. Atak: zmiana adresu docelowego

Atakujący próbuje zmienić `dst_id`.

Ochrona:

- `dst_id` jest w HMAC input,
- zmiana `dst_id` psuje HMAC,
- parser sprawdza poprawność adresacji.

Efekt:

- ramka zostaje odrzucona.

3.27.4 27.4. Atak: fałszywy ACK

Atakujący wysyła ACK.

Ochrona:

- ACK też jest ramką z HMAC,
- ACK payload zawiera `acked_msg_id` i `acked_counter`,
- tracker wymaga zgodności `src_id`, `msg_id`, `counter`.

Efekt:

- losowy ACK nie zamyka transmisji.

3.27.5 27.5. Atak: odczyt EEPROM

Atakujący odczytuje zewnętrzny EEPROM.

Ochrona:

- secret slots są szyfrowane,
- klucz EEPROM pochodzi z TPM root seed,
- root seed nie jest w EEPROM,
- rekordy mają CRC.

Efekt:

- attacker widzi ciphertext i metadata, nie pair code wprost.

3.27.6 27.6. Atak: zdalna rotacja root seeda

Atakujący próbuje zdalnie wymusić zmianę root seeda.

Ochrona:

- zapis do TPM NV index 0x01C10101 wymaga PPWRITE,
- PP jest egzekwowane przez TPM,
- PP jest fizyczne, pin TPM 7, aktywny VDD,
- STM32 nie może tego zastąpić zwykłą zmienną.

Efekt:

- bez fizycznej obecności zapis seeda powinien się nie udać.

3.27.7 27.7. Atak: błąd hardware AES/HASH

Problem:

- HAL może być READY, ale wynik AES może być błędny przez endianness.

Ochrona:

- self-test porównuje hardware i software,
- niezgodność wyłącza hardware backend,
- fallback nadal wykonuje crypto.

Efekt:

- system nie używa cichego, błędnego AES.

3.27.8 27.8. Atak: przypadkowe wejście w menu admin

Ochrona:

- strony menu mają wymagany poziom auth,
- PIN operator/admin,
- unlock wygasa po czasie,
- użytkownik bez PIN może czytać/odpowiadać, ale nie konfigurować.

Efekt:

- konfiguracja nie jest dostępna z normalnego ekranu pracy.

3.28 28. Ograniczenia obecnego wdrożenia

To są punkty, których nie wolno ukrywać:

1. Root seed ma obecnie 8 B. Dla produkcji lepiej zwiększyć do 16/32 B.
2. Shared mode używa stałego firmware key i jest słabszy niż pair-specific mode.
3. Secret slots EEPROM używają CRC, nie kryptograficznego MAC.
4. Finalna polityka replay-drop jest oznaczona w TODO jako praca do domknięcia.
5. PIN nie jest jeszcze trwale zapisany jako TPM/HMAC-protected secret.
6. Brakuje pełnego secure boot / debug lock / TrustZone policy.
7. Pairing powinien docelowo dostać challenge-response albo TPM policy session.
8. KEY_ROTATE jako pełny radiowy protokół rotacji nie jest jeszcze domknięty.
9. Brakuje per-source rate limiting i abuse scoring dla flood/replay.
10. Broadcast ma niższy poziom pewności niż unicast.

Te ograniczenia nie unieważniają architektury. One pokazują, gdzie są granice obecnego poziomu bezpieczeństwa.

3.29 29. Zalecane utwardzenia

Najważniejsze następne kroki:

1. Zwiększyć SECURITY_KEY_SEED_BYTES do 16 albo 32.
2. Dodać HMAC/AEAD dla EEPROM secret slots.
3. Przywrócić finalny drop dla replay ramek gatewaya.
4. Dodać test replay attack.
5. Dodać test odtwarzania sekretów po reboocie.
6. Dodać test migracji 0x01C10100 -> 0x01C10101.
7. Przenieść PIN do trwałego TPM-derived hash.
8. Dodać rate limiting per `src_id`.
9. Dodać abuse counters: bad HMAC, replay, malformed frame.
10. Domknąć KEY_ROTATE jako protokół radiowy.
11. Dodać policy/session TPM dla pairingu lub operacji admin.
12. Wyłączyć/maskować wrażliwe UART logi w buildzie produkcyjnym.
13. Zablokować debug/SWD w urządzeniach produkcyjnych.

3.30 30. Przykłady

3.30.1 30.1. Przykład: budowa zaszyfrowanej ramki DATA

Założenia:

```
src_id = 0x77CD
dst_id = 0x0001
msg_id = 0x0042
counter = 0x00000010
payload = "STS:OK"
```

Kroki:

```
1. frame.payload = "STS:OK"
2. frame.flags = ACK_REQUIRED | ENCRYPTED
3. enc_key, hmac_key = derive_pair_keys(0x77CD, 0x0001, pair_code)
4. AES-CTR encrypt payload
5. HMAC(header || encrypted_payload)
6. encode frame
7. send raw bytes
8. wait for ACK(msg_id=0x0042, counter=0x00000010)
```

3.30.2 30.2. Przykład: odbiór zaszyfrowanej ramki

```
1. radio RX bytes
2. laviet_frame_decode()
3. derive expected HMAC key
4. calculate HMAC over header || payload
5. constant-time compare with mac_tag
6. if HMAC OK: AES-CTR decrypt payload
7. if dst_id == local or broadcast: handle
8. if ACK_REQUIRED and unicast: send ACK
9. persist latest counter
```

3.30.3 30.3. Przykład: odrzucenie zmienionej ramki

Atakujący zmienia dst_id:

```
original dst_id = 0x77CD
tampered dst_id = 0x1234
```

HMAC input zmienił się, ale mac_tag został stary.
Odbiornik liczy expected MAC i dostaje inną wartość.

Wynik:

```
RADIO RX HMAC drop src=...
```

3.30.4 30.4. Przykład: rotacja root seeda

```
1. Admin wchodzi do Security -> Keys.
2. Firmware prosi o aktywne TPM PP.
3. Użytkownik trzyma przycisk PP.
4. Firmware generuje seed z TPM RNG albo HAL RNG.
5. Firmware zapisuje seed do NV index 0x01C10101.
6. TPM egzekwuje PPWRITE.
7. Firmware wyprowadza network_key i EEPROM key.
8. EEPROM secret store dostaje nowy klucz.
```

Bez PP:

```
SEC: root seed write requires TPM PP button
```

3.30.5 30.5. Przykład: hardware crypto self-test

Poprawny:

```
CRYPTO: AES pre_self_test inst=0x420c0000 state=1(READY) err=0x00000000
CRYPTO: HASH pre_self_test state=1(READY) err=0x00000000
CRYPTO: hardware backend ready
```

Błędna endianowość AES:

```
CRYPTO: self-test mismatch stage=aes_1B_compare index=0 expected=0xAE actual=0x41 len=1
CRYPTO: hardware self-test failed, using software fallback
```

Po poprawce pakowania big-endian:

```
CRYPTO: hardware backend ready
```

3.30.6 30.6. Przykład: EEPROM secret slot

Plain trusted payload:

```
id_len    = 4
code_len  = 8
id        = node_id as 4 B
code      = pair code 8 B
```

Po zapisie:

```
magic      = C0 DE
len        = 14
slot       = 0
nonce      = monotonic secret_counter
ciphertext = XTEA-CTR(plain, store_key, nonce)
crc16      = CRC rekordu
```

Atakujący z EEPROM widzi ciphertext, nie pair code.

3.31 31. Checklist dla testów

Po zmianie security trzeba wykonać:

- boot z TPM podłączonym,
- boot bez aktywnego TPM_INIT_LOG,
- boot z TPM_INIT_LOG, jeśli diagnozujemy I2C,
- potwierdzenie CRYPTO: hardware backend ready,
- odczyt root seed z 0x01C10101,
- reboot i ponowny odczyt tego samego stanu trusted/counter,
- re-pair gateway,
- DATA -> ACK gateway -> node,
- RESP -> ACK node -> gateway,
- broadcast RX bez ACK,
- próba ramki spoza trusted network,
- próba starego countera,
- próba złego HMAC,
- rotacja root seeda z PP,
- rotacja root seeda bez PP, oczekiwany błąd,
- test zaniku zasilania podczas zapisu EEPROM.

3.32 32. Najkrótsza odpowiedź “dlaczego to jest bezpieczne”

Bo sekret główny nie jest w firmware ani w EEPROM, tylko w TPM; zapis sekretu wymaga fizycznej obecności; klucze robocze są wyprowadzane z separacją domen; każda ramka jest uwierzytelniana HMAC po nagłówku i payloadzie; payload jest szyfrowany AES-CTR z nonce/counter zależnym od ramki; błędny hardware crypto jest wykrywany self-testem; EEPROM trzyma sekrety zaszyfrowane kluczem wyprowadzonym z TPM; a lokalne operacje konfiguracyjne są chronione PIN-em i menu role.

Nie jest to jednak koniec prac. Największe produkcyjne utwardzenia to: seed 16/32 B, replay-drop bez bypassu diagnostycznego, MAC/AEAD dla EEPROM secret slots, TPM policy dla pairingu/admin i trwały TPM-derived PIN hash.

Rozdział 4

STRIDE / CIA Analysis

Ten dokument porządkuje źródła ataku na system BEKO według dwóch perspektyw:

- STRIDE: klasy ataków na tożsamość, integralność, dostępność i kontrolę systemu,
- CIA: wpływ na poufność, integralność i dostępność.

Tabela ma trzy cele:

- pokazać, jakie klasy ataków są już częściowo adresowane,
- zaznaczyć, czego jeszcze brakuje w obecnym kodzie,
- dać listę praktycznych kierunków wdrożenia.

4.1 STRIDE

Typ ataku	Wdrożone rozwiązanie	Możliwy sposób rozwiązania / dalsze utwardzenie
S Spoofing: podszycie się pod inny węzeł przez fałszywy <code>src_id</code>	HMAC-SHA256 dla <code>LAVIET_↔FRAME_V1</code> , lokalny <code>node_id</code> , pairing/trust lista, root seed w TPM NV	Dodać policy dla operacji administracyjnych i pełny flow admin/service
S Spoofing: podszycie się pod zaufane urządzenie podczas pairingu	<code>PAIR_REQ</code> / <code>PAIR_RESP</code> , zapis trusted devices, lokalna zgoda użytkownika	Uwierzytelnić pairing challenge-response, generować pair code z TPM RNG, dodać policy TPM dla zgody fizycznej
T Tampering: modyfikacja payloadu w locie	HMAC-SHA256 dla każdej ramki i AES-CTR dla payloadu szyfrowanego	Podmienić portable crypto backend na sprzętowy HAL CRYPT/HASH/↔RNG
T Tampering: zmiana pól adresacji (<code>dst_id</code> , <code>msg_id</code> , <code>counter</code>)	Pola nagłówka są w HMAC, a parser waliduje typy/flagi/długości	Dodać pełną trwałość counterów i diagnostykę przyczyn odrzucenia
T Tampering: manipulacja stanem EEPROM / listą trusted	Częściowe formaty z CRC i wersjonowaniem danych, klucz EEPROM wyprowadzany z TPM-backed root seeda	Dodać integralność rekordów security, licznik wersji i odporność na rollback
R Repudiation: brak możliwości wykazania, kto wysłał wiadomość	<code>src_id</code> , <code>msg_id</code> , log RX do store, logi terminala	Dodać trwalszy audit log security, powód odrzucenia, liczniki replay/bad-auth, znaczniki czasu lub monotoniczne liczniki

Typ ataku	Wdrożone rozwiązanie	Możliwy sposób rozwiązania / dalsze utwardzenie
R Repudiation: brak dowodu akceptacji pairingu lub rotacji klucza	Logi runtime i menu popup	Zapisywać zdarzenia security w trwałym logu z typem operacji, <code>src_id</code> , wynikiem i przyczyną
I Information Disclosure: podsłuch payloadu OPERATOR	<code>coding</code> dla secure frames	Domyślnie wymusić secure mode dla ruchu operatora, rotować klucz sieciowy, użyć mocniejszej prymitywy niż obecny software cipher
I Information Disclosure: wyciek klucza z EEPROM/firmware	Root seed jest trzymany w TPM NV <code>0x01C10101</code> , zapis wymaga TPM PP, EEPROM key jest wyrowadzany z root seeda	Ograniczyć obecność klucza w RAM, wyczyścić bufor po użyciu, rozważyć sealing/policy sessions
I Information Disclosure: podgląd pair code lub danych trusted	Trusted store i menu urządzenia	Ukryć lub skrócić ekspozycję pair code na UI, nie logować pełnych sekretów na UART, ograniczyć odczyt danych trusted do trybu serwisowego
D Denial of Service: replay powodujący lokalne akcje	Drop ramek gateway->node ze starym <code>counter</code>	Dodać abuse score per source, cooldown, agregację alertów security i liczniki replay drop
D Denial of Service: flood DATA i broadcastów	Broadcast tylko od gatewaya, brak ACK dla broadcastu, duty-cycle dla auto ping	Per-source rate limiting, globalny budżet TX i limity dla ruchu control
D Denial of Service: flood ruchem control / pairing	Pairing jest rozdzielony typami ramek	Dodać osobny rate limiting dla <code>PAIR_*</code> , krótkie okno serwisowe, lokalną zgodę TPM/PP dla akceptacji
D Denial of Service: zapchanie UI i kolejek komunikatami security	Częściowa ochrona kolejek UI/↔ LCD, ograniczenie popupów w menu	Osobna kolejka security, agregowanie zdarzeń flood, nie generować popupu dla każdego błędu/replay
D Denial of Service: zawieszanie backendu radiowego przez nietypowe TX/FSK	Watchdog TX, recovery ścieżki radiowej, limity payload dla FSK/OOK	Dodać licznik recoveries, auto-disable wadliwego trybu po serii błędów, twarde ograniczenia konfiguracji backendu
E Elevation of Privilege: zdalne wykonanie operacji administracyjnej	Część operacji wymaga wejścia do odpowiedniego menu/flow; lokalna rotacja root seeda wymaga TPM PP	Rozdzielić role: operator/admin/service, chronić więcej operacji security przez TPM policy/local presence i osobne komendy admin
E Elevation of Privilege: przejęcie pairingu bez fizycznej obecności	Pairing nadal wymaga lokalnego flow UI, ale TPM PP chroni obecnie tylko zapis root seeda	Wdrożyć TPM Policy↔ <code>PhysicalPresence</code> , policy session i ograniczenie pairingu do lokalnie autoryzowanego okna
E Elevation of Privilege: użycie starych ramek sterujących do wymuszenia stanu	Replay protection na <code>counter</code>	Dodać pełną trwałość counterów oraz oddzielne polityki dla <code>CFG</code> , <code>COUNTER_SYNC</code> i <code>KEY_ROTATE</code>

4.2 CIA

Obszar CIA	Źródło ataku	Wdrożone rozwiązanie	Możliwy sposób rozwiązania / dalsze utwardzenie
C Confidentiality	Podśluch radiowy OPERATOR	coding dla secure ramek	Wymusić secure-by-default, mocniejsza kryptografia, regularna rotacja klucza sieciowego
C Confidentiality	Odczyt sekretów z pamięci trwalej	Root seed w TPM NV, EEPROM key z TPM-backed root seeda, migracja legacy EEPROM key	Sealed storage z TPM, policy sessions, ograniczenie debug/logów sekretów
C Confidentiality	Wyciek przez UART/logi	Terminal pokazuje szczegóły TX/RX	Dodać tryb produkcyjny bez pełnych dumpów, maskować wrażliwe dane i pair code
I Integrity	Modyfikacja ramek w locie	HMAC-SHA256 dla LAVIET_FRAME_V1	Sprzętowy HASH i dokładniejsza telemetryka bad-auth
I Integrity	Replay starych ramek	Monotoniczny counter gateway-node	Trwały zapis odporny na roll-back i telemetryka security
I Integrity	Fałszywe trusted device / nieautoryzowane parowanie	Pairing flow i trusted list	Pairing challenge-response, TPM RNG, lokalna zgoda fizyczna przez TPM policy
I Integrity	Fałszywa konfiguracja security po restarcie	Store versioning i częściowe CRC	Ochrona integralności rekordów security, monotoniczny counter, binding do TPM
A Availability	Flood radiowy	Brak mesh relay, replay drop, broadcast bez ACK	Token bucket per source, globalny budżet TX, adaptive backoff
A Availability	Zużycie duty-cycle / baterii	Auto ping respektuje duty-cycle	Budżet TX dla control traffic i automatyczne obniżanie priorytetu flood sources
A Availability	Zawieszenie toru radiowego lub backendu	Recovery watchdog i reset ścieżki radiowej	Eskalacja recovery, licznik awarii per modulacja, bezpieczny fallback do LoRa preset
A Availability	Zablokowanie UI/obsługi operatora przez spam zdarzeń	Częściowe utwardzenie UI i kolejek	Osobna ścieżka alertów security, status zbiorczy zamiast popup spam

4.3 Najważniejsze priorytety

1. Przenieść crypto backend na sprzętowy HAL CRYPT/HASH/RNG.
2. Dodać per-source rate limiting i globalny budżet TX.
3. Rozszerzyć ochronę TPM PP/policy z root seeda na pairing i operacje administracyjne.
4. Dalej ograniczać ekspozycję sekretów w RAM i logach UART.
5. Dodać trwałą telemetrykę security: `replay_drop`, `bad_auth`, `duplicate_drop`, `recovery_count`.

Rozdział 5

Threat Model

5.1 Assets

- Klucz sieciowy i klucze peer.
- Integralność ramek `LAVIET_FRAME_V1`.
- Dostępność kanału radiowego i budżetu duty-cycle.
- Lista urządzeń zaufanych.
- EEPROM z konfiguracją security.

5.2 Attacker goals

- Replay starych ramek, aby wywołać ponowną lokalną akcję lub fałszywy `ACK`.
- Flood broadcastami, żeby zużyć duty-cycle i baterię.
- Spam pairing/control traffic, żeby blokować UI i task radiowy.
- Wstrzykiwanie śmieciowych ramek o poprawnej strukturze, ale bez sensu logicznego.

5.3 Current mitigations

- HMAC-SHA256 dla każdej ramki `LAVIET_FRAME_V1`.
- AES-CTR dla payloadu z flagą `ENCRYPTED`.
- Monotoniczny `counter gateway-node`.
- Brak relay mesh/TTL w bieżącej topologii gwiazdy.
- Broadcast tylko od gatewaya i bez `ACK_REQUIRED`.

5.4 Remaining gaps

- Brak per-source token bucket.
- Brak pełnego trwałego modelu counterów dla wielu relacji.
- Brak reputacji źródeł i czasowego banowania.
- Brak agregacji alertów security przy floodzie.
- Brak rozdziału priorytetów dla ruchu lokalnego i administracyjnego.

Rozdział 6

Security

6.1 Pytania

- Jakie są wymagania dla odporności na replay?
- Czy każda ramka `OPERATOR` ma wymagać auth i szyfrowania?
- Jak wygląda odzyskiwanie zaufania po rotacji klucza?
- Czy tryb serwisowy ma mieć osobne uprawnienia i osobny klucz?

6.2 Obszary

- Radio link LoRa/FSK/OOK
- Pairing nowych urządzeń
- Magazyn EEPROM
- Integracja z TPM i TPM PP

6.3 Security TODO

- Spisać aktorów i poziomy zaufania.
- Zmapować dane wrażliwe i ich cykl życia.
- Zdefiniować minimalny zestaw testów bezpieczeństwa.
- Opisać dokładny threat model dla komunikacji lokalnej i radiowej.
- Rozdzielić warstwę policy/security od aktualnej logiki runtime w `security_main`.
- Dokończyć testy migracji EEPROM po przejściu na klucz wyprowadzany z TPM.
- Zdefiniować wersjonowanie protokołu secure frame.
- Rozszerzyć dokumentację pairing i trust removal.
- Dodać testy regresyjne dla auth tag, coding i migracji EEPROM.
- Przetestować restart z root seedem w TPM NV `0x01C10101`.
- Przetestować ścieżkę migracji z legacy TPM NV `0x01C10100`.
- Rozszerzyć TPM PP/policy z lokalnej rotacji root seeda na pairing i operacje admin.

6.3.1 DoS / Replay

Co można dodać

- Dodać per-source rate limiting dla OPERATOR.
- Dodać osobny rate limiting dla ramek control/pairing.
- Dodać globalny budżet TX niezależny od local TX.
- Dodać licznik nadużyć (`replay`, `bad auth`, `unknown src`) i czasowy cooldown.
- Agregować alerty security zamiast popupu dla każdego zdarzenia flood.
- Rozważyć telemetrykę security: liczba replay drop, duplicate drop, throttle hit.

Rozdział 7

Struktura katalogów

7.1 Katalogi

App	55
bmp280_lib	??
bmp280.c	
bmp280.h	149
bmp280_api.c	
bmp280_api.h	159
i2c_mem_store_lib	57
i2c_mem_store.c	169
i2c_mem_store.h	190
lcd_library	??
lcd.c	
lcd.h	255
led_array_lib	58
led_array_lib.c	270
led_array_lib.h	287
led_array_lib_config.h	293
radio_lib	60
common	??
sx1276	??
radio_sx1276.c	352
radio_sx1276.h	363
radio_sx1276_regs.h	371
modulations	??
fsk	??
radio_fsk.c	388
radio_fsk.h	407
lora	??
radio_lora.c	412
radio_lora.h	432
ook	??
radio_ook.c	439
radio_ook.h	457
test	??
radio_test.c	492
radio_test.h	501
radio_lib.c	461
radio_lib.h	474
radio_lib_config.h	489
security	62
st33ktpm2x_lib	64

st33ktpm2x.c	638
st33ktpm2x.h	677
vl53l3cx_lib	??
vl53l3cx_lib.c	
vl53l3cx_lib.h	701
app.c	
app.h	141
bmp280_main.c	
bmp280_main.h	164
button_main.c	
button_main.h	167
laviet_crypto.c	200
laviet_crypto.h	222
laviet_frame.c	226
laviet_frame.h	237
lcd_main.c	
lcd_main.h	265
led_array_main.c	
led_array_main.h	295
menu_main.c	
menu_main.h	349
pager_config.h	351
radio_main.c	
radio_main.h	560
security_main.c	
security_main.h	613
service.c	
service.h	637
tof_main.c	
tof_main.h	697
i2c_mem_store_lib	57
i2c_mem_store.c	169
i2c_mem_store.h	190
led_array_lib	58
led_array_lib.c	270
led_array_lib.h	287
led_array_lib_config.h	293
radio_lib	60
common	??
sx1276	??
radio_sx1276.c	352
radio_sx1276.h	363
radio_sx1276_regs.h	371
modulations	??
fsk	??
radio_fsk.c	388
radio_fsk.h	407
lora	??
radio_lora.c	412
radio_lora.h	432
ook	??
radio_ook.c	439
radio_ook.h	457
test	??
radio_test.c	492
radio_test.h	501

radio_lib.c	461
radio_lib.h	474
radio_lib_config.h	489
security	62
st33ktpm2x_lib	64
st33ktpm2x.c	638
st33ktpm2x.h	677

Rozdział 8

Indeks struktur danych

8.1 Struktury danych

Tutaj znajdują się struktury danych wraz z ich krótkimi opisami:

bmp280_api_data_t	67
Cached measurement container	
BMP280_HandleTypeDef	68
Driver handle with calibration constants and active configuration	
bmp280_params_t	71
Configuration parameters for BMP280 module	
button_hw_t	72
button_state_t	73
i2c_mem_store_cfg_t	74
Runtime configuration for external I2C memory	
i2c_mem_store_log_record_t	75
One logged message record returned to caller	
i2c_mem_store_meta_wire_t	77
i2c_mem_store_t	78
Driver context	
i2c_mem_store_trusted_device_t	79
Trusted-device object stored in encrypted partition	
laviet_frame_t	80
Parsed LAVIET frame representation	
laviet_sha256_ctx_t	82
lcd_main_msg_t	83
lcd_state_t	84
led_array_ctx_t	84
menu_item_t	86
menu_notification_t	87
Message passed from application subsystems to the menu task	
menu_page_t	88
menu_radio_action_binding_t	89
menu_state_t	90
radio_context_t	92
radio_fsk_cfg_t	94
Konfiguracja backendu FSK/GFSK/MSK/GMSK	
radio_fsk_context_t	96
radio_gpio_t	98
Opis pojedynczego pinu GPIO	
radio_hw_cfg_t	98
Konfiguracja sprzętowa modułu radiowego	
radio_lora_cfg_t	99
Konfiguracja profilu LoRa	
radio_main_cmd_sync_t	101

radio_main_cmd_t	102
radio_main_ctx_t	104
radio_main_fsk_cfg_t	
Profil rodziny FSK/GFSK/MSK/GMSK trzymany przez warstwę aplikacji	111
radio_main_ook_cfg_t	
Profil rodziny OOK przechowywany przez warstwę aplikacji	113
radio_main_runtime_cfg_t	
Snapshot pełnej konfiguracji radiowej widzianej przez aplikację	115
radio_ook_cfg_t	
Konfiguracja backendu OOK	117
radio_ook_context_t	118
radio_packet_t	
Bufor ostatnio odebranej ramki	120
security_cmd_sync_t	121
security_cmd_t	122
security_key_wire_t	124
security_radio_store_wire_t	125
security_runtime_cfg_t	
Persistent and runtime security settings snapshot	125
security_settings_wire_t	127
security_store_wire_t	128
security_trusted_entry_t	129
st33ktpm2x_cfg_t	
Runtime configuration for ST33KTPM2X instance	130
st33ktpm2x_t	
Driver context for one TPM device	133
sx1276_bus_t	
Kontekst magistrali SPI dla układu SX1276	134
trusted_info_t	
Public descriptor of one trusted pairing entry	134

Rozdział 9

Indeks plików

9.1 Lista plików

Tutaj znajduje się lista wszystkich udokumentowanych plików wraz z ich krótkimi opisami:

Core/App/app.c	137
Core/App/app.h	
Main application entry points and shared I2C bus helpers	141
Core/App/bmp280_main.c	162
Core/App/bmp280_main.h	
RTOS task facade for periodic BMP280 measurements	164
Core/App/button_main.c	165
Core/App/button_main.h	
Polled button task (25 ms) with event queue	167
Core/App/laviet_crypto.c	
Portable AES-CTR and HMAC-SHA256 backend for LAVIET_FRAME_V1	200
Core/App/laviet_crypto.h	
Crypto helpers for LAVIET_FRAME_V1	222
Core/App/laviet_frame.c	
LAVIET_FRAME_V1 parser, serializer and cheap validation	226
Core/App/laviet_frame.h	
LAVIET_FRAME_V1 wire protocol helpers	237
Core/App/lcd_main.c	257
Core/App/lcd_main.h	
RTOS-facing LCD rendering task API	265
Core/App/led_array_main.c	294
Core/App/led_array_main.h	
RTOS task facade for LED array status effects	295
Core/App/menu_main.c	296
Core/App/menu_main.h	
LCD menu controller task	349
Core/App/pager_config.h	
Application-level build and board configuration switches	351
Core/App/radio_main.c	503
Core/App/radio_main.h	
Application radio task, runtime profile and command API	560
Core/App/security_main.c	576
Core/App/security_main.h	
Security domain task and command API	613
Core/App/service.c	625
Core/App/service.h	
Optional UART service console task entry point	637
Core/App/tof_main.c	696
Core/App/tof_main.h	
RTOS task facade for the VL53L3CX time-of-flight sensor	697

Core/App/bmp280_lib/bmp280.c	146
Core/App/bmp280_lib/bmp280.h	
Low-level BMP280/BME280 driver API	149
Core/App/bmp280_lib/bmp280_api.c	155
Core/App/bmp280_lib/bmp280_api.h	
Single-instance BMP280 measurement wrapper used by the application	159
Core/App/i2c_mem_store_lib/i2c_mem_store.c	
Implementation of external I2C memory storage helper	169
Core/App/i2c_mem_store_lib/i2c_mem_store.h	
I2C NVM storage library for received messages and encrypted service codes	190
Core/App/lcd_library/lcd.c	248
Core/App/lcd_library/lcd.h	
HD44780-compatible 20x4 LCD driver over I2C expander	255
Core/App/led_array_lib/led_array_lib.c	
LED array driver implementation	270
Core/App/led_array_lib/led_array_lib.h	
LED array driver for 5 LEDs (PC7, PA8, PB10, PB4, PB5)	287
Core/App/led_array_lib/led_array_lib_config.h	
Compile-time configuration for led_array_lib	293
Core/App/radio_lib/radio_lib.c	
Fasada biblioteki radia delegująca wywołania do backendu wybranego w runtime	461
Core/App/radio_lib/radio_lib.h	
Główne API biblioteki radiowej	474
Core/App/radio_lib/radio_lib_config.h	
Konfiguracja kompilacyjna biblioteki radio_lib	489
Core/App/radio_lib/common/sx1276/radio_sx1276.c	
Implementacja niskopoziomowego dostępu do rejestrów SX1276	352
Core/App/radio_lib/common/sx1276/radio_sx1276.h	
Niskopoziomowe API dostępu do rejestrów SX1276	363
Core/App/radio_lib/common/sx1276/radio_sx1276_regs.h	
Mapa rejestrów SX1276 i definicje bitów używane w bibliotece	371
Core/App/radio_lib/modulations/fsk/radio_fsk.c	
Backend modulacji FSK/GFSK/MSK/GMSK dla SX1276	388
Core/App/radio_lib/modulations/fsk/radio_fsk.h	
Interfejs backendu modulacji FSK dla SX1276	407
Core/App/radio_lib/modulations/lora/radio_lora.c	
Implementacja backendu LoRa (SX1276/RFM95)	412
Core/App/radio_lib/modulations/lora/radio_lora.h	
Backend modulacji LoRa dla SX1276/RFM95	432
Core/App/radio_lib/modulations/ook/radio_ook.c	
Backend modulacji OOK dla SX1276	439
Core/App/radio_lib/modulations/ook/radio_ook.h	
Interfejs backendu modulacji OOK dla SX1276	457
Core/App/radio_lib/test/radio_test.c	
Implementacja narzędzi testowych i diagnostycznych radio_lib	492
Core/App/radio_lib/test/radio_test.h	
Narzędzia testowe i diagnostyczne dla radio_lib	501
Core/App/st33ktpm2x_lib/st33ktpm2x.c	
ST33KTPM2X driver implementation (TIS over I2C)	638
Core/App/st33ktpm2x_lib/st33ktpm2x.h	
High-level and transport-level driver for ST33KTPM2X over I2C (TIS/FIFO)	677
Core/App/vl53l3cx_lib/vl53l3cx_lib.c	698
Core/App/vl53l3cx_lib/vl53l3cx_lib.h	
Application wrapper for the VL53L3CX time-of-flight sensor	701

Rozdział 10

Dokumentacja katalogów

10.1 Dokumentacja katalogu Core/App

Katalogi

- katalog [bmp280_lib](#)
- katalog [i2c_mem_store_lib](#)
- katalog [lcd_library](#)
- katalog [led_array_lib](#)
- katalog [radio_lib](#)
- katalog [security](#)
- katalog [st33ktpm2x_lib](#)
- katalog [vl53l3cx_lib](#)

Pliki

- plik **app.c**
- plik [app.h](#)
Main application entry points and shared I2C bus helpers.
- plik **bmp280_main.c**
- plik [bmp280_main.h](#)
RTOS task facade for periodic BMP280 measurements.
- plik **button_main.c**
- plik [button_main.h](#)
Polled button task (25 ms) with event queue.
- plik [laviet_crypto.c](#)
Portable AES-CTR and HMAC-SHA256 backend for LAVIET_FRAME_V1.
- plik [laviet_crypto.h](#)
Crypto helpers for LAVIET_FRAME_V1.
- plik [laviet_frame.c](#)
LAVIET_FRAME_V1 parser, serializer and cheap validation.
- plik [laviet_frame.h](#)
LAVIET_FRAME_V1 wire protocol helpers.
- plik **lcd_main.c**
- plik [lcd_main.h](#)
RTOS-facing LCD rendering task API.
- plik **led_array_main.c**
- plik [led_array_main.h](#)
RTOS task facade for LED array status effects.
- plik **menu_main.c**
- plik [menu_main.h](#)

- *LCD menu controller task.*
- plik [pager_config.h](#)
 - *Application-level build and board configuration switches.*
- plik **radio_main.c**
- plik [radio_main.h](#)
 - *Application radio task, runtime profile and command API.*
- plik **security_main.c**
- plik [security_main.h](#)
 - *Security domain task and command API.*
- plik **service.c**
- plik [service.h](#)
 - *Optional UART service console task entry point.*
- plik **tof_main.c**
- plik [tof_main.h](#)
 - *RTOS task facade for the VL53L3CX time-of-flight sensor.*

10.1.1 Opis szczegółowy

10.1.2 Core/App - FreeRTOS Application Layer

Aktualna architektura jest oparta o zadania RTOS i Active Object (kolejki komend/zdarzeń):

- `button_main` - polling 3 przycisków co 25 ms (`DIG_B1/B2/B3`), debounce, short/long press.
- `menu_main` - kontroler UI na LCD 20x4 (pager/menu/popup), auto-exit 60 s.
- `lcd_main` - dedykowany task LCD z trybami MONITOR/MENU/POPUP, scroll wiadomości RX.
- `radio_main` - task radiowy z protokołem LAVIET_FRAME_V1, ACK, HMAC/AES-CTR, counter anti-replay i presetami LoRa.
- `security_main` - task domeny security: runtime config, rotacja klucza, TPM bootstrap, log RX do EEPROM.
- `bmp280_main`, `tof_main`, `led_array_main` - zadania sprzętowe jak wcześniej.

[app.c](#) pozostaje orchestratorem: inicjuje współdzielony lock I2C i uruchamia taski.

10.1.3 Startup

1. `main.c` inicjalizuje HAL/perferia.
2. `app_init()`.
3. `osKernelInitialize()`.
4. `MX_FREERTOS_Init()` -> `app_freertos_init()` tworzy taski.
5. `osKernelStart()`.

LCD pokazuje animację HELLO BEKO, następnie przechodzi do monitora RX.

10.1.4 Radio i UI

- Ramki systemowe: LAVIET_FRAME_V1, 45-61 B, payload 0-16 B, mac_tag 32 B.
- node_id jest 16-bitowe i wyliczane z UID MCU z pominięciem wartości zarezerwowanych.
- Topologia jest gwiazdą: node nie wykonuje mesh relay ani TTL forwarding.
- Każda poprawna unicastowa ramka gateway->node dostaje ACK z msg_id i counter.
- Każda wiadomość RX:
 - print na UART,
 - jedna linia LCD (RSSI:payload, 20 znaków),
 - log do EEPROM (przez security_main).

10.1.5 Build note

Jeśli IDE nie podchwyci nowych plików automatycznie, zrób:

1. Refresh projektu
2. Clean
3. Rebuild

10.2 Dokumentacja katalogu Core/App/i2c_mem_store_lib

Pliki

- plik [i2c_mem_store.c](#)
Implementation of external I2C memory storage helper.
- plik [i2c_mem_store.h](#)
I2C NVM storage library for received messages and encrypted service codes.

10.2.1 Opis szczegółowy

10.2.2 i2c_mem_store_lib

Biblioteka pamięci zewnętrznej na I2C1 do:

- trwałego logowania odebranych wiadomości,
- przechowywania zaszyfrowanych kodów/rekordów dla bezpieczeństwa.

10.2.3 Założenia sprzętowe

- pamięć na I2C1 (hi2c1),
- typ: EEPROM/FRAM z adresowaniem 16-bit (I2C_MEMADD_SIZE_16BIT domyślnie),
- domyślny adres I2C: 0x50 (7-bit).

10.2.4 Układ danych w pamięci

- 0x0000 metadata (primary + mirror),
- od 0x0100 ring log wiadomości (sloty 128 B),
- końcowa część pamięci: partycja secret (domyślnie 4096 B, sloty 96 B).

10.2.5 Bezpieczeństwo

W partycji `secret` dane są szyfrowane software'owo (XTEA-CTR, klucz 128-bit z konfiguracji).
Dla produkcji:

- nie trzymaj klucza na stałe w firmware,
- klucz ładuj/derywuj z TPM (np. NV + policy, albo sealed material),
- rotuj klucze i wersjonuj format rekordu.

10.3 Dokumentacja katalogu Core/App/led_array_lib

Pliki

- plik [led_array_lib.c](#)
LED array driver implementation.
- plik [led_array_lib.h](#)
LED array driver for 5 LEDs (PC7, PA8, PB10, PB4, PB5).
- plik [led_array_lib_config.h](#)
Compile-time configuration for led_array_lib.

10.3.1 Opis szczegółowy

10.3.2 led_array_lib

Driver for 5 LEDs connected to:

1. PC7
2. PA8
3. PB10
4. PB4
5. PB5

The library provides:

- init/deinit,
- ON/OFF control,
- brightness 0 . . 100%,
- one-shot fade-out ("fire and dim"),
- breath effect,
- circulating rainbow-like wave across all LEDs.

10.3.3 Files

- [led_array_lib.h](#) - public API.
- [led_array_lib.c](#) - implementation.
- [led_array_lib_config.h](#) - compile-time options.

10.3.4 Design

- Uses hardware PWM with high carrier frequency (~20 kHz).
- Timers used:
 - TIM8_CH2 -> PC7
 - TIM1_CH1 -> PA8
 - TIM2_CH3 -> PB10
 - TIM3_CH1 -> PB4
 - TIM3_CH2 -> PB5
- Effect timing (breath/rainbow/fade progression) is still updated by library tick.

10.3.5 API Overview

- `led_array_init()`, `led_array_deinit()`
- `led_array_timer_init(tick_ms)`, `led_array_timer_deinit()`
- `led_array_timer_tick()` (for ISR scheduling)
- `led_array_process()` (polling scheduling via HAL_GetTick)
- `led_array_on(mask)`, `led_array_off(mask)`
- `led_array_set_brightness(mask, pct)`
- `led_array_fire_fade_out(mask, fade_ms)`
- `led_array_start_breath(mask, period_ms, min_pct, max_pct)`
- `led_array_start_rainbow(step_ms, min_pct, max_pct)`
- `led_array_stop_effect()`

10.3.6 LED Masks

- `LED_ARRAY_LED_1` ... `LED_ARRAY_LED_5`
- `LED_ARRAY_LED_ALL`

Masks can be OR-ed, for example:

`LED_ARRAY_LED_1 | LED_ARRAY_LED_3`

10.3.7 Usage (polling mode)

```
#include "led_array_lib/led_array_lib.h"

void app_init(void)
{
    led_array_init();
    led_array_timer_init(1); /* 1 ms tick */
    led_array_start_rainbow(20, 5, 100);
}

void app_main(void)
{
    led_array_process();
}
```

10.3.8 Usage (ISR mode)

Call `led_array_timer_tick()` from SysTick or a hardware timer ISR.

If you want the library to own HAL_SYSTICK_Callback, set:

`LED_ARRAY_LIB_OWNS_HAL_SYSTICK_CALLBACK = 1`
in `led_array_lib_config.h` (or compiler defines).

10.3.9 Notes

- For smooth dimming, use `tick_ms = 1`.
- Effects are global (one active effect at a time).
- Manual brightness APIs (`on/off/set_brightness`) stop active effect.

10.4 Dokumentacja katalogu Core/App/radio_lib

Katalogi

- katalog [test](#)

Pliki

- plik [radio_lib.c](#)
Fasada biblioteki radia delegująca wywołania do backendu wybranego w runtime.
- plik [radio_lib.h](#)
Główne API biblioteki radiowej.
- plik [radio_lib_config.h](#)
Konfiguracja kompilacyjna biblioteki `radio_lib`.

10.4.1 Opis szczegółowy

10.4.2 radio_lib

Modułowa biblioteka radiowa dla układów SX1276/RFM95 (np. RFM95W-862S2) dla STM32. Warstwa aplikacyjna korzysta z jednego API ([radio_lib.h](#)), a backend może być wybierany zarówno kompilacyjnie, jak i przełączany w runtime.

10.4.3 Cele projektu

- Stabilne API do inicjalizacji, TX/RX i obsługi zdarzeń IRQ.
- Rozdzielenie warstwy sprzętowej (rejstry SX1276) od logiki modulacji.
- Możliwość rozbudowy o kolejne backendy bez zmian w aplikacji.
- Utrzymanie zgodności z kodem generowanym przez CubeMX.

10.4.4 Struktura katalogu

Ścieżka	Rola
radio_lib.h	Publiczne API biblioteki (statusy, stany, konfiguracje, funkcje).
radio_lib.c	Fasada/dispatcher delegująca wywołania do aktywnego backendu modulacji.
radio_lib_config.h	Konfiguracja kompilacyjna: wybór modulacji, timeouty, limity, EXTI.
common/sx1276/radio_sx1276_regs.h	Definicje rejestrów SX1276 i masek bitowych.
common/sx1276/radio_sx1276.h	Publiczny interfejs niskopoziomowy (SPI/rejstry/FIFO/↔ IRQ).
common/sx1276/radio_sx1276.c	Implementacja dostępu do rejestrów SX1276.
modulations/loro/radio_lora.h	Kontrakt backendu LoRa.

Ścieżka	Rola
<code>modulations/lora/radio_lora.c</code>	Implementacja LoRa: init, TX/RX, zdarzenia, DIO/IRQ, recovery.
<code>modulations/fsk/radio_fsk.h</code>	Kontrakt backendu FSK.
<code>modulations/fsk/radio_fsk.c</code>	Implementacja backendu FSK/GFSK/MSK/GMSK w packet-mode SX1276.
<code>modulations/ook/radio_ook.h</code>	Kontrakt backendu OOK.
<code>modulations/ook/radio_ook.c</code>	Implementacja backendu OOK w packet-mode SX1276.
<code>test/radio_test.h</code>	Publiczne narzędzia testowe i diagnostyczne.
<code>test/radio_test.c</code>	Scenariusz demo: probe, dump rejestrów, okresowy PING, logowanie zdarzeń.

10.4.5 Wybór aktywnej modulacji

W pliku `radio_lib_config.h` ustaw makro:

- `RADIO_LIB_ACTIVE_MODULATION = RADIO_LIB_MODULATION_LORA` (domyślnie),
- `RADIO_LIB_ACTIVE_MODULATION = RADIO_LIB_MODULATION_FSK`,
- `RADIO_LIB_ACTIVE_MODULATION = RADIO_LIB_MODULATION_OOK`.

Aplikacja zawsze używa `radio_lib.h`; backend można też przełączyć w runtime przez `radio_select_backend(...)` i przekazać profil przez `radio_set_fsk_cfg(...)` lub `radio_set_ook_cfg(...)`.

10.4.6 Model obsługi przerwań (IRQ/EXTI)

- EXTI pozostaje po stronie kodu CubeMX (`HAL_GPIO_EXTI_IRQHandler(...)`).
- `HAL_GPIO_EXTI_Callback` może być implementowany przez bibliotekę (`RADIO_LIB_OWNS_HAL_EXTI_CALLBACK = 1`) albo przez aplikację (`= 0`) z przekazaniem pinu do `radio_on_exti(...)`.
- ISR ustawia tylko lekkie flagi; operacje SPI i logika pakietów są wykonywane w `radio_process()`.

10.4.7 Integracja w aplikacji

Minimalny przepływ:

1. `radio_default_hw_cfg(...)` i `radio_default_lora_cfg(...)`.
2. `radio_init(...)`.
3. `radio_start_rx_continuous()` lub `radio_start_rx_single(...)`.
4. W pętli głównej: `radio_process()` oraz `radio_take_events()`.

Do szybkiego uruchomienia można użyć `test/radio_test.c`:

- `radio_test_demo_init(&hspi1)`,
- `radio_test_demo_process()` cyklicznie w pętli głównej.

W tym projekcie warstwa aplikacyjna używa obecnie `radio_main` (task RTOS), a `radio_test.c` jest traktowany jako moduł diagnostyczny.

10.4.8 Status backendów

- LoRa: implementacja produkcyjna (SX1276/RFM95).
- FSK: działający backend SX1276 z TX/RX, FIFO, IRQ i packet engine.
- OOK: działający backend SX1276 z TX/RX, FIFO, IRQ i packet engine.

10.4.9 Dokumentacja Doxygen

Pliki `.h` zawierają pełną dokumentację publicznego API w stylu Doxygen. Pliki `.c` zawierają dokumentację modułów oraz sekcji implementacyjnych.

Przykładowe znaczniki użyte w projekcie:

- `@file`, `@brief`,
- `@param`, `@return`,
- `@name i @{ ... @}` dla grupowania API.

10.5 Dokumentacja katalogu Core/App/security

10.5.1 Opis szczegółowy

10.5.2 Security

Ten katalog zbiera założenia i zadania dla bezpieczeństwa komunikacji BEKO.

Dokumenty pomocnicze:

- `SECURITY_ARCHITECTURE.md` - pełny opis architektury bezpieczeństwa: TPM, AES/HMAC, root seed, EEPROM, ramki, ACK, anti-replay, PIN i przykłady,
- `THREAT_MODEL.md` - skrócony model zagrożeń dla bieżącej architektury,
- `STRIDE_CIA_ANALYSIS.md` - tabela klas ataków, obecnych zabezpieczeń i dalszych mitigacji.
- `../../../../gateway_docs/SECURITY_CRYPTO_OVERVIEW.md` - przekrojowy opis kryptografii i ochrony całego systemu, w tym gatewaya.

10.5.3 Aktualny model

- `LAVIET_FRAME_V1` używa `src_id`, `dst_id`, `msg_id`, `counter`, `payload_len` i `mac_tag`.
- Ramki są uwierzytelniane HMAC-SHA256, a payload może być szyfrowany AES-CTR.
- Relay mesh/TTL został usunięty z bieżącej architektury; node działa w topologii gwiazdy z gatewayem.
- Ochrona przed replay w pierwszym wdrożeniu używa monotonicznego `counter` dla relacji gateway-node i zapisuje go w małym slotcie `secret` NVM.
- TPM PP w obecnym hardware jest podłączony do samego modułu TPM, nie do GPIO MCU. Pin TPM PP to pin 7, aktywny stanem VDD.
- TPM jest podłączony przez I2C3 (`PC0 = SCL`, `PC1 = SDA`); `reset TPM_RESET#` jest na `PB0`, a `TPM_DAVINT# / PIRQ` na `PH0` jako `EXTI0`, aktywny niskim stanem.
- Firmware używa TPM jako źródła trwałego root seeda:
 - główny NV index: `0x01C10101`,
 - legacy read-only/migration index: `0x01C10100`,
 - atrybuty głównego indexu: `PPWRITE`, `OWNERREAD`, `NO_DA`.
- Zapis lub rotacja root seeda wymaga aktywnego TPM PP. STM32 nie odczytuje tego pinu jako GPIO; wymuszenie odbywa się przez TPM authorization (`TPM_RH_PLATFORM` dla `NV_Write`).
- Klucz szyfrowania sekretów EEPROM jest wyprowadzany z TPM-backed root seeda. Przy pierwszym starcie po zmianie firmware próbuje przepisać stare wpisy EEPROM z dawnego stałego klucza na klucz wyprowadzony z TPM.
- PP nie zabezpiecza jeszcze całego pairingu radiowego. Jeśli pairing albo inne operacje administracyjne mają wymagać fizycznej obecności, trzeba dodać osobne TPM policy/session albo równoważny flow oparty o TPM.

10.5.4 DoS / Flood Defense

Poniżej są mechanizmy, które można wdrażać warstwowo. Nie trzeba robić wszystkiego naraz.

10.5.4.1 1. Rate limiting per source

- Trzymać licznik ramek na `src_id` w krótkim oknie czasu, np. $N / 10$ s.
- Po przekroczeniu limitu przestać obsługiwać ruch z tego źródła.
- Dla bardzo agresywnych źródeł wejść w czasowe `cooldown`.

10.5.4.2 2. Global TX budget

- Osobno limitować ruch użytkownika i ruch administracyjny.
- Węzeł nie powinien poświęcić całego duty-cycle na flood control traffic.
- Praktycznie: token bucket dla TX, np. osobny budżet na minutę.

10.5.4.3 3. Separate limits for control traffic

- `PAIR_REQ`, `PAIR_RESP`, `ERROR`, `CFG`, `COUNTER_SYNC`, `KEY_ROTATE` i `ACK` powinny mieć niższe limity niż zwykle `DATA`.
- Pairing powinien działać tylko w krótkim oknie serwisowym.
- Powtarzane `PAIR_REQ` od jednego źródła powinny być szybko wyciszane.

10.5.4.4 4. Replay-aware abuse scoring

- Każde wykrycie replay zwiększa licznik nadużyć dla `src_id`.
- Po kilku replayach z rzędu można:
 - zablokować obsługę źródła,
 - obniżyć priorytet,
 - logować zdarzenie security.

10.5.4.5 5. Strict frame policy

- Przyjmować tylko ramki, które pasują do topologii gateway-node.
- Nie generować `ACK` dla broadcastu ani dla błędnego HMAC/replay.
- Walidować typ, flagi, `payload_len`, `src_id` i `dst_id` przed kryptografią.

10.5.4.6 6. Bounded parsing cost

- Odrzucać ramki z nieprawidłowym `src_id`, `dst_id`, `msg_id`, `counter` lub długością przed cięższą logiką.
- Nie robić drogich operacji kryptograficznych dla ruchu, który już wygląda na śmieciowy.
- Przy floodzie najtańsze filtry powinny działać jako pierwsze.

10.5.4.7 7. Queue and UI protection

- Oddzielić logi security od ścieżki krytycznej UI.
- Nie generować popupu dla każdego zdarzenia flood/replay.
- Zamiast tego agregować liczniki i wyświetlać skrócony status.

10.5.4.8 8. Channel occupancy protection

- Auto-ping i control traffic powinny respektować duty-cycle i własny budżet TX.
- W stanie przeciążenia najpierw ograniczać ruch administracyjny/testowy, dopiero potem ruch użytkownika.
- Dobrze działa też adaptacyjne wydłużanie odstępów TX po wykryciu floodu.

10.5.5 Najbardziej opłacalna kolejność wdrożenia

1. Per-source rate limiting.
2. Global TX budget.
3. Abuse scoring dla replay/flood.
4. Ograniczenia pairing/control traffic.
5. Agregowane logowanie security zamiast popup spam.

10.6 Dokumentacja katalogu Core/App/st33ktpm2x_lib

Pliki

- plik [st33ktpm2x.c](#)
ST33KTPM2X driver implementation (TIS over I2C).
- plik [st33ktpm2x.h](#)
High-level and transport-level driver for ST33KTPM2X over I2C (TIS/FIFO).

10.6.1 Opis szczegółowy

10.6.2 st33ktpm2x_lib

Biblioteka dla ST33KTPM2X (TPM 2.0) po I2C (TIS/FIFO) pod STM32 HAL.

10.6.3 Założenia

- Interfejs TPM: I2C3 (`hi2c3`, `PC0 = SCL`, `PC1 = SDA`).
- Pin reset TPM: `PB0` (`TPM_RESET__Pin/TPM_RESET__GPIO_Port` z `main.h`), aktywny niskim stanem.
- Pin `TPM_DAVINT# / PIRQ: PH0` (`TPM_DAVINT__Pin/TPM_DAVINT__GPIO_Port`), wejście `EXTIO`, aktywne niskim stanem.
- Jeśli PP jest wyprowadzony do STM32 GPIO, może być konfigurowany w [st33ktpm2x_cfg_t](#).
- W aktualnym hardware BEKO PP jest podłączony do samego TPM: pin 7, aktywny stanem `VDD`. Tego pinu nie czyta STM32 jako zwykłego przycisku; jest używany przez TPM przy komendach wymagających fizycznej obecności.

10.6.4 Co zawiera

- transport TPM TIS over I2C:
 - lokalność `locality 0`,
 - FIFO TX/RX,
 - parsowanie nagłówka TPM2 i `TPM_RC`.
- funkcje bazowe TPM2:
 - `TPM2_Startup`,

- TPM2_SelfTest,
 - TPM2_GetRandom,
 - TPM2_GetCapability,
 - TPM2_PCR_Read (SHA-256, pojedynczy PCR).
- funkcje NV:
 - TPM2_NV_DefineSpace,
 - TPM2_NV_Read,
 - TPM2_NV_Write z owner authorization,
 - TPM2_NV_Write z platform authorization dla PP-protected NV.
- wsparcie dla PP jako GPIO, gdy hardware tak je podłączy:
 - odczyt stanu przycisku,
 - czekanie na naciśnięcie.

10.6.5 Użycie w BEKO

security_main używa biblioteki TPM do:

- inicjalizacji TPM przed EEPROM,
- pobierania losowości z TPM2_GetRandom,
- przechowywania root seeda w NV indexie 0x01C10101,
- migracyjnego odczytu starego NV indexu 0x01C10100,
- wymuszenia aktywnego PP przy zapisie root seeda przez NV_Write z TPM_RH_PLATFORM.

Root seed nie powinien być już zapisywany jawnie do EEPROM, gdy TPM jest gotowy. EEPROM przechowuje wpisy trusted/country zaszyfrowane kluczem wyprowadzonym z TPM-backed root seeda.

Rozdział 11

Dokumentacja struktur danych

11.1 Dokumentacja struktury `bmp280_api_data_t`

Cached measurement container.

```
#include <Core/App/bmp280_lib/bmp280_api.h>
```

Pola danych

- float `temperature_c`
Temperature in degrees Celsius.
- float `pressure_pa`
Pressure in Pascals.
- float `pressure_hpa`
Pressure in hPa/mbar.
- uint32_t `timestamp_ms`
HAL_GetTick() timestamp of the last successful measurement.
- bool `valid`
Set after the first successful measurement.

11.1.1 Opis szczegółowy

Cached measurement container.

Definicja w linii 20 pliku `bmp280_api.h`.

11.1.2 Dokumentacja pól

11.1.2.1 `pressure_hpa`

```
float bmp280_api_data_t::pressure_hpa
```

Pressure in hPa/mbar.

Definicja w linii 24 pliku `bmp280_api.h`.

11.1.2.2 `pressure_pa`

```
float bmp280_api_data_t::pressure_pa
```

Pressure in Pascals.

Definicja w linii 23 pliku `bmp280_api.h`.

11.1.2.3 `temperature_c`

```
float bmp280_api_data_t::temperature_c
```

Temperature in degrees Celsius.

Definicja w linii 22 pliku `bmp280_api.h`.

11.1.2.4 timestamp_ms

`uint32_t bmp280_api_data_t::timestamp_ms`

`HAL_GetTick()` timestamp of the last successful measurement.

Definicja w linii 25 pliku [bmp280_api.h](#).

11.1.2.5 valid

`bool bmp280_api_data_t::valid`

Set after the first successful measurement.

Definicja w linii 26 pliku [bmp280_api.h](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/bmp280_lib/bmp280_api.h](#)

11.2 Dokumentacja struktury BMP280_HandleTypedef

Driver handle with calibration constants and active configuration.

```
#include <Core/App/bmp280_lib/bmp280.h>
```

Pola danych

- `uint16_t dig_T1`
Temperature calibration coefficient T1.
- `int16_t dig_T2`
Temperature calibration coefficient T2.
- `int16_t dig_T3`
Temperature calibration coefficient T3.
- `uint16_t dig_P1`
Pressure calibration coefficient P1.
- `int16_t dig_P2`
Pressure calibration coefficient P2.
- `int16_t dig_P3`
Pressure calibration coefficient P3.
- `int16_t dig_P4`
Pressure calibration coefficient P4.
- `int16_t dig_P5`
Pressure calibration coefficient P5.
- `int16_t dig_P6`
Pressure calibration coefficient P6.
- `int16_t dig_P7`
Pressure calibration coefficient P7.
- `int16_t dig_P8`
Pressure calibration coefficient P8.
- `int16_t dig_P9`
Pressure calibration coefficient P9.
- `uint8_t dig_H1`
Humidity calibration coefficient H1.
- `int16_t dig_H2`
Humidity calibration coefficient H2.
- `uint8_t dig_H3`
Humidity calibration coefficient H3.
- `int16_t dig_H4`
Humidity calibration coefficient H4.

- `int16_t dig_H5`
Humidity calibration coefficient H5.
- `int8_t dig_H6`
Humidity calibration coefficient H6.
- `uint16_t addr`
7-bit I2C device address.
- `I2C_HandleTypeDef * i2c`
I2C peripheral handle.
- `bmp280_params_t params`
Active sensor configuration.
- `uint8_t id`
Chip ID read from the sensor.

11.2.1 Opis szczegółowy

Driver handle with calibration constants and active configuration.
Definicja w linii 97 pliku [bmp280.h](#).

11.2.2 Dokumentacja pól

11.2.2.1 addr

```
uint16_t BMP280_HandleTypeDef::addr
```

7-bit I2C device address.
Definicja w linii 119 pliku [bmp280.h](#).
Odwołania w [bmp280_init\(\)](#).

11.2.2.2 dig_H1

```
uint8_t BMP280_HandleTypeDef::dig_H1
```

Humidity calibration coefficient H1.
Definicja w linii 112 pliku [bmp280.h](#).

11.2.2.3 dig_H2

```
int16_t BMP280_HandleTypeDef::dig_H2
```

Humidity calibration coefficient H2.
Definicja w linii 113 pliku [bmp280.h](#).

11.2.2.4 dig_H3

```
uint8_t BMP280_HandleTypeDef::dig_H3
```

Humidity calibration coefficient H3.
Definicja w linii 114 pliku [bmp280.h](#).

11.2.2.5 dig_H4

```
int16_t BMP280_HandleTypeDef::dig_H4
```

Humidity calibration coefficient H4.
Definicja w linii 115 pliku [bmp280.h](#).

11.2.2.6 dig_H5

```
int16_t BMP280_HandleTypeDef::dig_H5
```

Humidity calibration coefficient H5.
Definicja w linii 116 pliku [bmp280.h](#).

11.2.2.7 dig_H6

```
int8_t BMP280_HandleTypedef::dig_H6
```

Humidity calibration coefficient H6.

Definicja w linii 117 pliku [bmp280.h](#).

11.2.2.8 dig_P1

```
uint16_t BMP280_HandleTypedef::dig_P1
```

Pressure calibration coefficient P1.

Definicja w linii 101 pliku [bmp280.h](#).

11.2.2.9 dig_P2

```
int16_t BMP280_HandleTypedef::dig_P2
```

Pressure calibration coefficient P2.

Definicja w linii 102 pliku [bmp280.h](#).

11.2.2.10 dig_P3

```
int16_t BMP280_HandleTypedef::dig_P3
```

Pressure calibration coefficient P3.

Definicja w linii 103 pliku [bmp280.h](#).

11.2.2.11 dig_P4

```
int16_t BMP280_HandleTypedef::dig_P4
```

Pressure calibration coefficient P4.

Definicja w linii 104 pliku [bmp280.h](#).

11.2.2.12 dig_P5

```
int16_t BMP280_HandleTypedef::dig_P5
```

Pressure calibration coefficient P5.

Definicja w linii 105 pliku [bmp280.h](#).

11.2.2.13 dig_P6

```
int16_t BMP280_HandleTypedef::dig_P6
```

Pressure calibration coefficient P6.

Definicja w linii 106 pliku [bmp280.h](#).

11.2.2.14 dig_P7

```
int16_t BMP280_HandleTypedef::dig_P7
```

Pressure calibration coefficient P7.

Definicja w linii 107 pliku [bmp280.h](#).

11.2.2.15 dig_P8

```
int16_t BMP280_HandleTypedef::dig_P8
```

Pressure calibration coefficient P8.

Definicja w linii 108 pliku [bmp280.h](#).

11.2.2.16 dig_P9

```
int16_t BMP280_HandleTypedef::dig_P9
```

Pressure calibration coefficient P9.

Definicja w linii 109 pliku [bmp280.h](#).

11.2.2.17 dig_T1

uint16_t BMP280_HandleTypeDef::dig_T1

Temperature calibration coefficient T1.

Definicja w linii 98 pliku [bmp280.h](#).

11.2.2.18 dig_T2

int16_t BMP280_HandleTypeDef::dig_T2

Temperature calibration coefficient T2.

Definicja w linii 99 pliku [bmp280.h](#).

11.2.2.19 dig_T3

int16_t BMP280_HandleTypeDef::dig_T3

Temperature calibration coefficient T3.

Definicja w linii 100 pliku [bmp280.h](#).

11.2.2.20 i2c

I2C_HandleTypeDef* BMP280_HandleTypeDef::i2c

I2C peripheral handle.

Definicja w linii 121 pliku [bmp280.h](#).

11.2.2.21 id

uint8_t BMP280_HandleTypeDef::id

Chip ID read from the sensor.

Definicja w linii 125 pliku [bmp280.h](#).

Odwwołania w [bmp280_init\(\)](#) i [bmp280_read_fixed\(\)](#).

11.2.2.22 params

[bmp280_params_t](#) BMP280_HandleTypeDef::params

Active sensor configuration.

Definicja w linii 123 pliku [bmp280.h](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/bmp280_lib/bmp280.h](#)

11.3 Dokumentacja struktury bmp280_params_t

Configuration parameters for BMP280 module.

```
#include <Core/App/bmp280_lib/bmp280.h>
```

Pola danych

- [BMP280_Mode mode](#)
Sensor operating mode.
- [BMP280_Filter filter](#)
IIR filter setting.
- [BMP280_Oversampling oversampling_pressure](#)
Pressure oversampling setting.
- [BMP280_Oversampling oversampling_temperature](#)
Temperature oversampling setting.
- [BMP280_Oversampling oversampling_humidity](#)
Humidity oversampling setting for BME280.
- [BMP280_StandbyTime standby](#)
Standby period in normal mode.

11.3.1 Opis szczegółowy

Configuration parameters for BMP280 module.

Use function `bmp280_init_default_params` to use default configuration.

Definicja w linii 84 pliku [bmp280.h](#).

11.3.2 Dokumentacja pól

11.3.2.1 filter

`BMP280_Filter` `bmp280_params_t::filter`

IIR filter setting.

Definicja w linii 86 pliku [bmp280.h](#).

Odwołania w [bmp280_api_init\(\)](#), [bmp280_init\(\)](#) i [bmp280_init_default_params\(\)](#).

11.3.2.2 mode

`BMP280_Mode` `bmp280_params_t::mode`

Sensor operating mode.

Definicja w linii 85 pliku [bmp280.h](#).

Odwołania w [bmp280_api_init\(\)](#), [bmp280_init\(\)](#) i [bmp280_init_default_params\(\)](#).

11.3.2.3 oversampling_humidity

`BMP280_Oversampling` `bmp280_params_t::oversampling_humidity`

Humidity oversampling setting for BME280.

Definicja w linii 89 pliku [bmp280.h](#).

Odwołania w [bmp280_init\(\)](#) i [bmp280_init_default_params\(\)](#).

11.3.2.4 oversampling_pressure

`BMP280_Oversampling` `bmp280_params_t::oversampling_pressure`

Pressure oversampling setting.

Definicja w linii 87 pliku [bmp280.h](#).

Odwołania w [bmp280_api_init\(\)](#), [bmp280_init\(\)](#) i [bmp280_init_default_params\(\)](#).

11.3.2.5 oversampling_temperature

`BMP280_Oversampling` `bmp280_params_t::oversampling_temperature`

Temperature oversampling setting.

Definicja w linii 88 pliku [bmp280.h](#).

Odwołania w [bmp280_api_init\(\)](#), [bmp280_init\(\)](#) i [bmp280_init_default_params\(\)](#).

11.3.2.6 standby

`BMP280_StandbyTime` `bmp280_params_t::standby`

Standby period in normal mode.

Definicja w linii 90 pliku [bmp280.h](#).

Odwołania w [bmp280_api_init\(\)](#), [bmp280_init\(\)](#) i [bmp280_init_default_params\(\)](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/bmp280_lib/bmp280.h](#)

11.4 Dokumentacja struktury `button_hw_t`

Pola danych

- `GPIO_TypeDef *` [port](#)
- `uint16_t` [pin](#)
- `bool` [active_low](#)

11.4.1 Opis szczegółowy

Definicja w linii 26 pliku `button_main.c`.

11.4.2 Dokumentacja pól

11.4.2.1 `active_low`

```
bool button_hw_t::active_low
```

Definicja w linii 30 pliku `button_main.c`.

11.4.2.2 `pin`

```
uint16_t button_hw_t::pin
```

Definicja w linii 29 pliku `button_main.c`.

11.4.2.3 `port`

```
GPIO_TypeDef* button_hw_t::port
```

Definicja w linii 28 pliku `button_main.c`.

Dokumentacja dla tej struktury została wygenerowana z pliku:

- `Core/App/button_main.c`

11.5 Dokumentacja struktury `button_state_t`

Pola danych

- bool `raw_pressed`
- bool `stable_pressed`
- uint8_t `stable_count`
- uint32_t `pressed_since_ms`
- bool `long_reported`

11.5.1 Opis szczegółowy

Definicja w linii 33 pliku `button_main.c`.

11.5.2 Dokumentacja pól

11.5.2.1 `long_reported`

```
bool button_state_t::long_reported
```

Definicja w linii 39 pliku `button_main.c`.

11.5.2.2 `pressed_since_ms`

```
uint32_t button_state_t::pressed_since_ms
```

Definicja w linii 38 pliku `button_main.c`.

11.5.2.3 `raw_pressed`

```
bool button_state_t::raw_pressed
```

Definicja w linii 35 pliku `button_main.c`.

11.5.2.4 `stable_count`

```
uint8_t button_state_t::stable_count
```

Definicja w linii 37 pliku `button_main.c`.

11.5.2.5 stable_pressed

`bool button_state_t::stable_pressed`

Definicja w linii 36 pliku `button_main.c`.

Dokumentacja dla tej struktury została wygenerowana z pliku:

- `Core/App/button_main.c`

11.6 Dokumentacja struktury `i2c_mem_store_cfg_t`

Runtime configuration for external I2C memory.

`#include <Core/App/i2c_mem_store_lib/i2c_mem_store.h>`

Pola danych

- `I2C_HandleTypeDef * hi2c`
I2C handle (use &hi2c1).
- `uint16_t i2c_addr_7bit`
7-bit memory address (default 0x50).
- `uint16_t mem_addr_size`
I2C memory address size (I2C_MEMADD_SIZE_8BIT/16BIT).
- `uint32_t total_bytes`
Total memory size in bytes.
- `uint16_t page_bytes`
Write page size in bytes (EEPROM page).
- `uint16_t write_cycle_ms`
EEPROM write cycle delay in ms.
- `uint32_t secret_area_bytes`
Size of encrypted partition at end of memory.
- `uint8_t crypto_key [16]`
128-bit XTEA key.

11.6.1 Opis szczegółowy

Runtime configuration for external I2C memory.

Definicja w linii 85 pliku `i2c_mem_store.h`.

11.6.2 Dokumentacja pól

11.6.2.1 crypto_key

`uint8_t i2c_mem_store_cfg_t::crypto_key[16]`

128-bit XTEA key.

Definicja w linii 94 pliku `i2c_mem_store.h`.

Odwolania w `i2c_mem_store_default_cfg()`, `i2c_mem_store_secret_read()` i `i2c_mem_store_secret_write()`.

11.6.2.2 hi2c

`I2C_HandleTypeDef* i2c_mem_store_cfg_t::hi2c`

I2C handle (use &hi2c1).

Definicja w linii 87 pliku `i2c_mem_store.h`.

Odwolania w `i2c_mem_store_default_cfg()`, `i2c_mem_store_layout_valid()`, `i2c_mem_store_read_raw()` i `i2c_mem_store_write_raw()`.

11.6.2.3 i2c_addr_7bit

uint16_t i2c_mem_store_cfg_t::i2c_addr_7bit

7-bit memory address (default 0x50).

Definicja w linii 88 pliku [i2c_mem_store.h](#).

Odwołania w [i2c_mem_store_default_cfg\(\)](#), [i2c_mem_store_default_cfg_m24c01r\(\)](#), [i2c_mem_store_read_raw\(\)](#) i [i2c_mem_store_write_raw\(\)](#).

11.6.2.4 mem_addr_size

uint16_t i2c_mem_store_cfg_t::mem_addr_size

I2C memory address size (I2C_MEMADD_SIZE_8BIT/16BIT).

Definicja w linii 89 pliku [i2c_mem_store.h](#).

Odwołania w [i2c_mem_store_default_cfg\(\)](#), [i2c_mem_store_default_cfg_m24c01r\(\)](#), [i2c_mem_store_read_raw\(\)](#) i [i2c_mem_store_write_raw\(\)](#).

11.6.2.5 page_bytes

uint16_t i2c_mem_store_cfg_t::page_bytes

Write page size in bytes (EEPROM page).

Definicja w linii 91 pliku [i2c_mem_store.h](#).

Odwołania w [i2c_mem_store_default_cfg\(\)](#), [i2c_mem_store_default_cfg_m24c01r\(\)](#), [i2c_mem_store_layout_valid\(\)](#) i [i2c_mem_store_write_raw\(\)](#).

11.6.2.6 secret_area_bytes

uint32_t i2c_mem_store_cfg_t::secret_area_bytes

Size of encrypted partition at end of memory.

Definicja w linii 93 pliku [i2c_mem_store.h](#).

Odwołania w [i2c_mem_store_default_cfg\(\)](#), [i2c_mem_store_default_cfg_m24c01r\(\)](#), [i2c_mem_store_layout_valid\(\)](#) i [i2c_mem_store_secret_base_addr\(\)](#).

11.6.2.7 total_bytes

uint32_t i2c_mem_store_cfg_t::total_bytes

Total memory size in bytes.

Definicja w linii 90 pliku [i2c_mem_store.h](#).

Odwołania w [i2c_mem_store_default_cfg\(\)](#), [i2c_mem_store_default_cfg_m24c01r\(\)](#), [i2c_mem_store_layout_valid\(\)](#) i [i2c_mem_store_secret_base_addr\(\)](#).

11.6.2.8 write_cycle_ms

uint16_t i2c_mem_store_cfg_t::write_cycle_ms

EEPROM write cycle delay in ms.

Definicja w linii 92 pliku [i2c_mem_store.h](#).

Odwołania w [i2c_mem_store_default_cfg\(\)](#), [i2c_mem_store_default_cfg_m24c01r\(\)](#) i [i2c_mem_store_write_raw\(\)](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/i2c_mem_store_lib/i2c_mem_store.h](#)

11.7 Dokumentacja struktury i2c_mem_store_log_record_t

One logged message record returned to caller.

```
#include <Core/App/i2c_mem_store_lib/i2c_mem_store.h>
```

Pola danych

- uint32_t [sequence](#)
Monotonic record sequence.

- `uint32_t timestamp_ms`
Timestamp from HAL_GetTick().
- `int16_t rssi_dbm`
RSSI attached to message.
- `uint8_t length`
Message payload length.
- `uint8_t payload [I2C_MEM_STORE_LOG_PAYLOAD_MAX]`
Message payload bytes.

11.7.1 Opis szczegółowy

One logged message record returned to caller.

Definicja w linii 62 pliku [i2c_mem_store.h](#).

11.7.2 Dokumentacja pól

11.7.2.1 length

```
uint8_t i2c_mem_store_log_record_t::length
```

Message payload length.

Definicja w linii 67 pliku [i2c_mem_store.h](#).

Odwwołania w [i2c_mem_store_read_message\(\)](#).

11.7.2.2 payload

```
uint8_t i2c_mem_store_log_record_t::payload[I2C_MEM_STORE_LOG_PAYLOAD_MAX]
```

Message payload bytes.

Definicja w linii 68 pliku [i2c_mem_store.h](#).

Odwwołania w [i2c_mem_store_read_message\(\)](#).

11.7.2.3 rssi_dbm

```
int16_t i2c_mem_store_log_record_t::rssi_dbm
```

RSSI attached to message.

Definicja w linii 66 pliku [i2c_mem_store.h](#).

Odwwołania w [i2c_mem_store_read_message\(\)](#).

11.7.2.4 sequence

```
uint32_t i2c_mem_store_log_record_t::sequence
```

Monotonic record sequence.

Definicja w linii 64 pliku [i2c_mem_store.h](#).

Odwwołania w [i2c_mem_store_read_message\(\)](#).

11.7.2.5 timestamp_ms

```
uint32_t i2c_mem_store_log_record_t::timestamp_ms
```

Timestamp from HAL_GetTick().

Definicja w linii 65 pliku [i2c_mem_store.h](#).

Odwwołania w [i2c_mem_store_read_message\(\)](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- `Core/App/i2c_mem_store_lib/i2c_mem_store.h`

11.8 Dokumentacja struktury i2c_mem_store_meta_wire_t

Pola danych

- uint32_t [magic](#)
- uint16_t [version](#)
- uint16_t [write_slot](#)
- uint16_t [valid_count](#)
- uint16_t [reserved](#)
- uint32_t [next_sequence](#)
- uint32_t [secret_counter](#)
- uint16_t [crc16](#)
- uint16_t [reserved2](#)

11.8.1 Opis szczegółowy

Definicja w linii 30 pliku [i2c_mem_store.c](#).

11.8.2 Dokumentacja pól

11.8.2.1 crc16

```
uint16_t i2c_mem_store_meta_wire_t::crc16
```

Definicja w linii 39 pliku [i2c_mem_store.c](#).

11.8.2.2 magic

```
uint32_t i2c_mem_store_meta_wire_t::magic
```

Definicja w linii 32 pliku [i2c_mem_store.c](#).

11.8.2.3 next_sequence

```
uint32_t i2c_mem_store_meta_wire_t::next_sequence
```

Definicja w linii 37 pliku [i2c_mem_store.c](#).

11.8.2.4 reserved

```
uint16_t i2c_mem_store_meta_wire_t::reserved
```

Definicja w linii 36 pliku [i2c_mem_store.c](#).

11.8.2.5 reserved2

```
uint16_t i2c_mem_store_meta_wire_t::reserved2
```

Definicja w linii 40 pliku [i2c_mem_store.c](#).

11.8.2.6 secret_counter

```
uint32_t i2c_mem_store_meta_wire_t::secret_counter
```

Definicja w linii 38 pliku [i2c_mem_store.c](#).

11.8.2.7 valid_count

```
uint16_t i2c_mem_store_meta_wire_t::valid_count
```

Definicja w linii 35 pliku [i2c_mem_store.c](#).

11.8.2.8 version

```
uint16_t i2c_mem_store_meta_wire_t::version
```

Definicja w linii 33 pliku [i2c_mem_store.c](#).

11.8.2.9 write_slot

`uint16_t i2c_mem_store_meta_wire_t::write_slot`

Definicja w linii 34 pliku [i2c_mem_store.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/i2c_mem_store_lib/i2c_mem_store.c](#)

11.9 Dokumentacja struktury i2c_mem_store_t

Driver context.

```
#include <Core/App/i2c_mem_store_lib/i2c_mem_store.h>
```

Pola danych

- [i2c_mem_store_cfg_t](#) `cfg`
Active configuration.
- `uint16_t` [slot_count](#)
Message log slot count.
- `uint16_t` [secret_slot_count](#)
Secret partition slot count.
- `uint16_t` [write_slot](#)
Next slot index to write in log partition.
- `uint16_t` [valid_count](#)
Number of valid log entries.
- `uint32_t` [next_sequence](#)
Next sequence number.
- `uint32_t` [secret_counter](#)
Nonce/counter source for secret encryption.
- `bool` [initialized](#)
Context initialization flag.

11.9.1 Opis szczegółowy

Driver context.

Definicja w linii 100 pliku [i2c_mem_store.h](#).

11.9.2 Dokumentacja pól

11.9.2.1 cfg

`i2c_mem_store_cfg_t i2c_mem_store_t::cfg`

Active configuration.

Definicja w linii 102 pliku [i2c_mem_store.h](#).

Odwołania w [i2c_mem_store_init\(\)](#), [i2c_mem_store_read_raw\(\)](#), [i2c_mem_store_secret_base_addr\(\)](#), [i2c_mem_store_secret_read\(\)](#), [i2c_mem_store_secret_write\(\)](#) i [i2c_mem_store_write_raw\(\)](#).

11.9.2.2 initialized

`bool i2c_mem_store_t::initialized`

Context initialization flag.

Definicja w linii 109 pliku [i2c_mem_store.h](#).

Odwołania w [i2c_mem_store_append_message\(\)](#), [i2c_mem_store_format\(\)](#), [i2c_mem_store_get_log_count\(\)](#), [i2c_mem_store_init\(\)](#), [i2c_mem_store_read_message\(\)](#), [i2c_mem_store_secret_erase\(\)](#), [i2c_mem_store_secret_read\(\)](#), [i2c_mem_store_secret_write\(\)](#), [i2c_mem_store_trusted_device_read\(\)](#) i [i2c_mem_store_trusted_device_write\(\)](#).

11.9.2.3 `next_sequence`

`uint32_t i2c_mem_store_t::next_sequence`

Next sequence number.

Definicja w linii 107 pliku `i2c_mem_store.h`.

Odwołania w `i2c_mem_store_append_message()`, `i2c_mem_store_format()`, `i2c_mem_store_meta_decode()` i `i2c_mem_store_meta_encode()`.

11.9.2.4 `secret_counter`

`uint32_t i2c_mem_store_t::secret_counter`

Nonce/counter source for secret encryption.

Definicja w linii 108 pliku `i2c_mem_store.h`.

Odwołania w `i2c_mem_store_format()`, `i2c_mem_store_meta_decode()`, `i2c_mem_store_meta_encode()` i `i2c_mem_store_secret_write()`.

11.9.2.5 `secret_slot_count`

`uint16_t i2c_mem_store_t::secret_slot_count`

Secret partition slot count.

Definicja w linii 104 pliku `i2c_mem_store.h`.

Odwołania w `i2c_mem_store_init()`, `i2c_mem_store_secret_erase()`, `i2c_mem_store_secret_read()` i `i2c_mem_store_secret_write()`.

11.9.2.6 `slot_count`

`uint16_t i2c_mem_store_t::slot_count`

Message log slot count.

Definicja w linii 103 pliku `i2c_mem_store.h`.

Odwołania w `i2c_mem_store_append_message()`, `i2c_mem_store_init()`, `i2c_mem_store_meta_decode()` i `i2c_mem_store_read_message()`.

11.9.2.7 `valid_count`

`uint16_t i2c_mem_store_t::valid_count`

Number of valid log entries.

Definicja w linii 106 pliku `i2c_mem_store.h`.

Odwołania w `i2c_mem_store_append_message()`, `i2c_mem_store_format()`, `i2c_mem_store_get_log_count()`, `i2c_mem_store_meta_decode()`, `i2c_mem_store_meta_encode()` i `i2c_mem_store_read_message()`.

11.9.2.8 `write_slot`

`uint16_t i2c_mem_store_t::write_slot`

Next slot index to write in log partition.

Definicja w linii 105 pliku `i2c_mem_store.h`.

Odwołania w `i2c_mem_store_append_message()`, `i2c_mem_store_format()`, `i2c_mem_store_meta_decode()`, `i2c_mem_store_meta_encode()` i `i2c_mem_store_read_message()`.

Dokumentacja dla tej struktury została wygenerowana z pliku:

- `Core/App/i2c_mem_store_lib/i2c_mem_store.h`

11.10 Dokumentacja struktury `i2c_mem_store_trusted_device_t`

Trusted-device object stored in encrypted partition.

```
#include <Core/App/i2c_mem_store_lib/i2c_mem_store.h>
```

Pola danych

- `uint8_t id_len`

Device identifier length.

- `uint8_t code_len`
Device/service code length.
- `uint8_t id` [I2C_MEM_STORE_TRUSTED_ID_MAX]
Device identifier bytes.
- `uint8_t code` [I2C_MEM_STORE_TRUSTED_CODE_MAX]
Encrypted content payload before encryption.

11.10.1 Opis szczegółowy

Trusted-device object stored in encrypted partition.
Definicja w linii 74 pliku [i2c_mem_store.h](#).

11.10.2 Dokumentacja pól

11.10.2.1 code

`uint8_t i2c_mem_store_trusted_device_t::code` [I2C_MEM_STORE_TRUSTED_CODE_MAX]
Encrypted content payload before encryption.
Definicja w linii 79 pliku [i2c_mem_store.h](#).
Odwołania w [i2c_mem_store_trusted_device_read\(\)](#) i [i2c_mem_store_trusted_device_write\(\)](#).

11.10.2.2 code_len

`uint8_t i2c_mem_store_trusted_device_t::code_len`
Device/service code length.
Definicja w linii 77 pliku [i2c_mem_store.h](#).
Odwołania w [i2c_mem_store_trusted_device_read\(\)](#) i [i2c_mem_store_trusted_device_write\(\)](#).

11.10.2.3 id

`uint8_t i2c_mem_store_trusted_device_t::id` [I2C_MEM_STORE_TRUSTED_ID_MAX]
Device identifier bytes.
Definicja w linii 78 pliku [i2c_mem_store.h](#).
Odwołania w [i2c_mem_store_trusted_device_read\(\)](#) i [i2c_mem_store_trusted_device_write\(\)](#).

11.10.2.4 id_len

`uint8_t i2c_mem_store_trusted_device_t::id_len`
Device identifier length.
Definicja w linii 76 pliku [i2c_mem_store.h](#).
Odwołania w [i2c_mem_store_trusted_device_read\(\)](#) i [i2c_mem_store_trusted_device_write\(\)](#).
Dokumentacja dla tej struktury została wygenerowana z pliku:

- `Core/App/i2c_mem_store_lib/i2c_mem_store.h`

11.11 Dokumentacja struktury `laviет_frame_t`

Parsed LAVIET frame representation.
`#include <Core/App/laviет_frame.h>`

Pola danych

- `uint8_t ver_type`
Version/type byte: high nibble version, low nibble type.
- `uint8_t flags`
Bitmask of LAVIET_FLAG_ values.*
- `uint16_t src_id`

- Source node identifier.*
 - `uint16_t dst_id`
- Destination node identifier.*
 - `uint16_t msg_id`
- Message identifier used by ACK tracking.*
 - `uint32_t counter`
- Monotonic frame counter.*
 - `uint8_t payload_len`
- Number of valid bytes in `payload`.*
 - `uint8_t payload` [`LAVIET_MAX_PAYLOAD`]
- Frame payload buffer.*
 - `uint8_t mac_tag` [`LAVIET_MAC_TAG_LEN`]
- Authentication tag stored on the wire.*

11.11.1 Opis szczegółowy

Parsed LAVIET frame representation.
Definicja w linii 95 pliku `laviet_frame.h`.

11.11.2 Dokumentacja pól

11.11.2.1 counter

```
uint32_t laviet_frame_t::counter
```

Monotonic frame counter.

Definicja w linii 102 pliku `laviet_frame.h`.

Odwołania w `laviet_frame_build_mac_input()`, `laviet_frame_decode()`, `laviet_frame_encode()` i `laviet_make_counter_block()`.

11.11.2.2 dst_id

```
uint16_t laviet_frame_t::dst_id
```

Destination node identifier.

Definicja w linii 100 pliku `laviet_frame.h`.

Odwołania w `laviet_frame_build_mac_input()`, `laviet_frame_decode()`, `laviet_frame_encode()`, `laviet_frame_is_broadcast()`, `laviet_frame_validate_plain()` i `laviet_make_counter_block()`.

11.11.2.3 flags

```
uint8_t laviet_frame_t::flags
```

Bitmask of `LAVIET_FLAG_*` values.

Definicja w linii 98 pliku `laviet_frame.h`.

Odwołania w `laviet_frame_build_mac_input()`, `laviet_frame_decode()`, `laviet_frame_encode()` i `laviet_frame_validate_plain()`.

11.11.2.4 mac_tag

```
uint8_t laviet_frame_t::mac_tag[LAVIET_MAC_TAG_LEN]
```

Authentication tag stored on the wire.

Definicja w linii 105 pliku `laviet_frame.h`.

Odwołania w `laviet_frame_decode()` i `laviet_frame_encode()`.

11.11.2.5 msg_id

```
uint16_t laviet_frame_t::msg_id
```

Message identifier used by ACK tracking.

Definicja w linii 101 pliku `laviet_frame.h`.

Odwołania w `laviet_frame_build_mac_input()`, `laviet_frame_decode()`, `laviet_frame_encode()`, `laviet_frame_validate_plain()` i `laviet_make_counter_block()`.

11.11.2.6 payload

```
uint8_t laviet_frame_t::payload[LAVIET_MAX_PAYLOAD]
```

Frame payload buffer.

Definicja w linii 104 pliku [laviet_frame.h](#).

Odwołania w [laviet_frame_build_mac_input\(\)](#), [laviet_frame_decode\(\)](#) i [laviet_frame_encode\(\)](#).

11.11.2.7 payload_len

```
uint8_t laviet_frame_t::payload_len
```

Number of valid bytes in [payload](#).

Definicja w linii 103 pliku [laviet_frame.h](#).

Odwołania w [laviet_frame_build_mac_input\(\)](#), [laviet_frame_decode\(\)](#), [laviet_frame_encode\(\)](#) i [laviet_frame_validate_plain\(\)](#).

11.11.2.8 src_id

```
uint16_t laviet_frame_t::src_id
```

Source node identifier.

Definicja w linii 99 pliku [laviet_frame.h](#).

Odwołania w [laviet_frame_build_mac_input\(\)](#), [laviet_frame_decode\(\)](#), [laviet_frame_encode\(\)](#), [laviet_frame_validate_plain\(\)](#) i [laviet_make_counter_block\(\)](#).

11.11.2.9 ver_type

```
uint8_t laviet_frame_t::ver_type
```

Version/type byte: high nibble version, low nibble type.

Definicja w linii 97 pliku [laviet_frame.h](#).

Odwołania w [laviet_frame_build_mac_input\(\)](#), [laviet_frame_decode\(\)](#), [laviet_frame_encode\(\)](#), [laviet_frame_type\(\)](#) i [laviet_frame_validate_plain\(\)](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/[laviet_frame.h](#)

11.12 Dokumentacja struktury laviet_sha256_ctx_t

Pola danych

- `uint8_t data` [64]
- `uint32_t datalen`
- `uint64_t bitlen`
- `uint32_t state` [8]

11.12.1 Opis szczegółowy

Definicja w linii 46 pliku [laviet_crypto.c](#).

11.12.2 Dokumentacja pól

11.12.2.1 bitlen

```
uint64_t laviet_sha256_ctx_t::bitlen
```

Definicja w linii 50 pliku [laviet_crypto.c](#).

11.12.2.2 data

```
uint8_t laviet_sha256_ctx_t::data[64]
```

Definicja w linii 48 pliku [laviet_crypto.c](#).

11.12.2.3 datalen

```
uint32_t laviet_sha256_ctx_t::datalen
```

Definicja w linii 49 pliku [laviet_crypto.c](#).

11.12.2.4 state

```
uint32_t laviet_sha256_ctx_t::state[8]
```

Definicja w linii 51 pliku [laviet_crypto.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/laviet_crypto.c](#)

11.13 Dokumentacja struktury `lcd_main_msg_t`

Pola danych

- `lcd_main_msg_type_t` [type](#)
- `uint8_t` [line_index](#)
- `lcd_main_mode_t` [mode](#)
- `char` [text0](#) [LCD_MAIN_COLS+1U]
- `char` [text1](#) [LCD_MAIN_COLS+1U]
- `char` [text2](#) [LCD_MAIN_COLS+1U]
- `char` [text3](#) [LCD_MAIN_COLS+1U]

11.13.1 Opis szczegółowy

Definicja w linii 34 pliku [lcd_main.c](#).

11.13.2 Dokumentacja pól

11.13.2.1 line_index

```
uint8_t lcd_main_msg_t::line_index
```

Definicja w linii 37 pliku [lcd_main.c](#).

11.13.2.2 mode

```
lcd_main_mode_t lcd_main_msg_t::mode
```

Definicja w linii 38 pliku [lcd_main.c](#).

11.13.2.3 text0

```
char lcd_main_msg_t::text0[LCD_MAIN_COLS+1U]
```

Definicja w linii 39 pliku [lcd_main.c](#).

11.13.2.4 text1

```
char lcd_main_msg_t::text1[LCD_MAIN_COLS+1U]
```

Definicja w linii 40 pliku [lcd_main.c](#).

11.13.2.5 text2

```
char lcd_main_msg_t::text2[LCD_MAIN_COLS+1U]
```

Definicja w linii 41 pliku [lcd_main.c](#).

11.13.2.6 text3

```
char lcd_main_msg_t::text3[LCD_MAIN_COLS+1U]
```

Definicja w linii 42 pliku [lcd_main.c](#).

11.13.2.7 type

`lcd_main_msg_type_t lcd_main_msg_t::type`

Definicja w linii 36 pliku [lcd_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/lcd_main.c](#)

11.14 Dokumentacja struktury `lcd_state_t`

Pola danych

- `bool` [initialized](#)
- `uint8_t` [backlight](#)
- `uint8_t` [addr_7bit](#)

11.14.1 Opis szczegółowy

Definicja w linii 32 pliku [lcd.c](#).

11.14.2 Dokumentacja pól

11.14.2.1 `addr_7bit`

`uint8_t lcd_state_t::addr_7bit`

Definicja w linii 36 pliku [lcd.c](#).

11.14.2.2 `backlight`

`uint8_t lcd_state_t::backlight`

Definicja w linii 35 pliku [lcd.c](#).

11.14.2.3 `initialized`

`bool lcd_state_t::initialized`

Definicja w linii 34 pliku [lcd.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/lcd_library/lcd.c](#)

11.15 Dokumentacja struktury `led_array_ctx_t`

Pola danych

- `bool` [initialized](#)
- `bool` [timer_running](#)
- `uint32_t` [tick_period_ms](#)
- `uint32_t` [last_tick_ms](#)
- `led_array_effect_t` [effect](#)
- `uint8_t` [manual_brightness](#) [LED_ARRAY_COUNT]
- `uint8_t` [active_brightness](#) [LED_ARRAY_COUNT]
- `struct` {
 - `uint8_t` [mask](#)
 - `uint16_t` [duration_ms](#)
 - `uint32_t` [elapsed_ms](#)
- `fade`

- struct {
 uint8_t mask
 uint16_t period_ms
 uint8_t min_pct
 uint8_t max_pct
 uint32_t elapsed_ms
} **breath**
- struct {
 uint16_t step_ms
 uint8_t min_pct
 uint8_t max_pct
 uint16_t phase_accum_ms
 uint8_t phase_base
} **rainbow**

11.15.1 Opis szczegółowy

Definicja w linii 18 pliku `led_array_lib.c`.

11.15.2 Dokumentacja pól

11.15.2.1 active_brightness

```
uint8_t led_array_ctx_t::active_brightness[LED_ARRAY_COUNT]
```

Definicja w linii 26 pliku `led_array_lib.c`.

11.15.2.2 duration_ms

```
uint16_t led_array_ctx_t::duration_ms
```

Definicja w linii 30 pliku `led_array_lib.c`.

11.15.2.3 effect

```
led_array_effect_t led_array_ctx_t::effect
```

Definicja w linii 24 pliku `led_array_lib.c`.

11.15.2.4 elapsed_ms

```
uint32_t led_array_ctx_t::elapsed_ms
```

Definicja w linii 31 pliku `led_array_lib.c`.

11.15.2.5 initialized

```
bool led_array_ctx_t::initialized
```

Definicja w linii 20 pliku `led_array_lib.c`.

11.15.2.6 last_tick_ms

```
uint32_t led_array_ctx_t::last_tick_ms
```

Definicja w linii 23 pliku `led_array_lib.c`.

11.15.2.7 manual_brightness

```
uint8_t led_array_ctx_t::manual_brightness[LED_ARRAY_COUNT]
```

Definicja w linii 25 pliku `led_array_lib.c`.

11.15.2.8 mask

```
uint8_t led_array_ctx_t::mask
```

Definicja w linii 29 pliku [led_array_lib.c](#).

11.15.2.9 max_pct

```
uint8_t led_array_ctx_t::max_pct
```

Definicja w linii 38 pliku [led_array_lib.c](#).

11.15.2.10 min_pct

```
uint8_t led_array_ctx_t::min_pct
```

Definicja w linii 37 pliku [led_array_lib.c](#).

11.15.2.11 period_ms

```
uint16_t led_array_ctx_t::period_ms
```

Definicja w linii 36 pliku [led_array_lib.c](#).

11.15.2.12 phase_accum_ms

```
uint16_t led_array_ctx_t::phase_accum_ms
```

Definicja w linii 46 pliku [led_array_lib.c](#).

11.15.2.13 phase_base

```
uint8_t led_array_ctx_t::phase_base
```

Definicja w linii 47 pliku [led_array_lib.c](#).

11.15.2.14 step_ms

```
uint16_t led_array_ctx_t::step_ms
```

Definicja w linii 43 pliku [led_array_lib.c](#).

11.15.2.15 tick_period_ms

```
uint32_t led_array_ctx_t::tick_period_ms
```

Definicja w linii 22 pliku [led_array_lib.c](#).

11.15.2.16 timer_running

```
bool led_array_ctx_t::timer_running
```

Definicja w linii 21 pliku [led_array_lib.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/led_array_lib/[led_array_lib.c](#)

11.16 Dokumentacja struktury menu_item_t

Pola danych

- const char * [label](#)
- menu_page_id_t [child_page](#)
- menu_action_t [action](#)

11.16.1 Opis szczegółowy

Definicja w linii 337 pliku [menu_main.c](#).

11.16.2 Dokumentacja pól

11.16.2.1 action

menu_action_t menu_item_t::action

Definicja w linii 341 pliku [menu_main.c](#).

11.16.2.2 child_page

menu_page_id_t menu_item_t::child_page

Definicja w linii 340 pliku [menu_main.c](#).

11.16.2.3 label

const char* menu_item_t::label

Definicja w linii 339 pliku [menu_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/menu_main.c

11.17 Dokumentacja struktury menu_notification_t

Message passed from application subsystems to the menu task.

```
#include <Core/App/menu_main.h>
```

Pola danych

- [menu_notification_type_t](#) type
Notification category.
- int16_t [rssi_dbm](#)
RSSI for RX notifications.
- uint32_t [device_code](#)
Source or related device identifier.
- uint32_t [reply_device_code](#)
Reply destination suggested by notification.
- char [text](#) [21]
LCD-width text payload, null-terminated when possible.

11.17.1 Opis szczegółowy

Message passed from application subsystems to the menu task.

Definicja w linii 28 pliku [menu_main.h](#).

11.17.2 Dokumentacja pól

11.17.2.1 device_code

uint32_t menu_notification_t::device_code

Source or related device identifier.

Definicja w linii 32 pliku [menu_main.h](#).

11.17.2.2 reply_device_code

uint32_t menu_notification_t::reply_device_code

Reply destination suggested by notification.

Definicja w linii 33 pliku [menu_main.h](#).

11.17.2.3 rssi_dbm

```
int16_t menu_notification_t::rssi_dbm
```

RSSI for RX notifications.

Definicja w linii 31 pliku [menu_main.h](#).

11.17.2.4 text

```
char menu_notification_t::text[21]
```

LCD-width text payload, null-terminated when possible.

Definicja w linii 34 pliku [menu_main.h](#).

11.17.2.5 type

```
menu_notification_type_t menu_notification_t::type
```

Notification category.

Definicja w linii 30 pliku [menu_main.h](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/menu_main.h](#)

11.18 Dokumentacja struktury menu_page_t

Pola danych

- const char * [title](#)
- menu_page_id_t [parent](#)
- const [menu_item_t](#) * [items](#)
- uint8_t [item_count](#)

11.18.1 Opis szczegółowy

Definicja w linii 344 pliku [menu_main.c](#).

11.18.2 Dokumentacja pól

11.18.2.1 item_count

```
uint8_t menu_page_t::item_count
```

Definicja w linii 349 pliku [menu_main.c](#).

11.18.2.2 items

```
const menu_item_t* menu_page_t::items
```

Definicja w linii 348 pliku [menu_main.c](#).

11.18.2.3 parent

```
menu_page_id_t menu_page_t::parent
```

Definicja w linii 347 pliku [menu_main.c](#).

11.18.2.4 title

```
const char* menu_page_t::title
```

Definicja w linii 346 pliku [menu_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/menu_main.c](#)

11.19 Dokumentacja struktury menu_radio_action_binding_t

Pola danych

- menu_action_t [action](#)
- bool [set_modulation](#)
- radio_main_modulation_t [modulation](#)
- bool [set_option](#)
- radio_main_option_t [option](#)
- uint32_t [value](#)
- bool [persist_lora_preset](#)
- const char * [ok_text](#)
- const char * [err_text](#)

11.19.1 Opis szczegółowy

Definicja w linii 398 pliku [menu_main.c](#).

11.19.2 Dokumentacja pól

11.19.2.1 action

```
menu_action_t menu_radio_action_binding_t::action
```

Definicja w linii 400 pliku [menu_main.c](#).

11.19.2.2 err_text

```
const char* menu_radio_action_binding_t::err_text
```

Definicja w linii 408 pliku [menu_main.c](#).

11.19.2.3 modulation

```
radio_main_modulation_t menu_radio_action_binding_t::modulation
```

Definicja w linii 402 pliku [menu_main.c](#).

11.19.2.4 ok_text

```
const char* menu_radio_action_binding_t::ok_text
```

Definicja w linii 407 pliku [menu_main.c](#).

11.19.2.5 option

```
radio_main_option_t menu_radio_action_binding_t::option
```

Definicja w linii 404 pliku [menu_main.c](#).

11.19.2.6 persist_lora_preset

```
bool menu_radio_action_binding_t::persist_lora_preset
```

Definicja w linii 406 pliku [menu_main.c](#).

11.19.2.7 set_modulation

```
bool menu_radio_action_binding_t::set_modulation
```

Definicja w linii 401 pliku [menu_main.c](#).

11.19.2.8 set_option

```
bool menu_radio_action_binding_t::set_option
```

Definicja w linii 403 pliku [menu_main.c](#).

11.19.2.9 value

`uint32_t menu_radio_action_binding_t::value`

Definicja w linii 405 pliku `menu_main.c`.

Dokumentacja dla tej struktury została wygenerowana z pliku:

- `Core/App/menu_main.c`

11.20 Dokumentacja struktury `menu_state_t`

Pola danych

- `menu_page_id_t` `current_page`
- `uint8_t` `selected_idx`
- `uint8_t` `led_mode`
- `uint8_t` `selected_device_slot`
- `uint32_t` `pending_target_node_id`
- `menu_action_t` `send_target_action`
- `menu_page_id_t` `transient_parent_page`
- `bool` `pairing_network_mode`
- `uint32_t` `quick_reply_src_id`
- `uint32_t` `quick_reply_deadline_ms`
- `uint32_t` `quick_reply_last_seconds`
- `char` `quick_reply_text` [MENU_LINE_BUF_SIZE]
- `bool` `tx_ack_waiting`
- `char` `tx_sent_text` [MENU_LINE_BUF_SIZE]
- `menu_page_id_t` `pending_auth_page`
- `menu_auth_level_t` `pending_auth_level`
- `uint8_t` `pin_digits` [MENU_SETTINGS_PIN_LEN]
- `uint8_t` `pin_index`
- `uint32_t` `operator_unlock_until_ms`
- `uint32_t` `admin_unlock_until_ms`
- `bool` `popup_enabled`
- `uint32_t` `last_input_ms`
- `menu_modal_t` `modal`
- `menu_action_t` `pending_action`

11.20.1 Opis szczegółowy

Definicja w linii 370 pliku `menu_main.c`.

11.20.2 Dokumentacja pól

11.20.2.1 admin_unlock_until_ms

`uint32_t menu_state_t::admin_unlock_until_ms`

Definicja w linii 391 pliku `menu_main.c`.

11.20.2.2 current_page

`menu_page_id_t menu_state_t::current_page`

Definicja w linii 372 pliku `menu_main.c`.

11.20.2.3 last_input_ms

`uint32_t menu_state_t::last_input_ms`

Definicja w linii 393 pliku `menu_main.c`.

11.20.2.4 led_mode

uint8_t menu_state_t::led_mode

Definicja w linii 374 pliku [menu_main.c](#).

11.20.2.5 modal

menu_modal_t menu_state_t::modal

Definicja w linii 394 pliku [menu_main.c](#).

11.20.2.6 operator_unlock_until_ms

uint32_t menu_state_t::operator_unlock_until_ms

Definicja w linii 390 pliku [menu_main.c](#).

11.20.2.7 pairing_network_mode

bool menu_state_t::pairing_network_mode

Definicja w linii 379 pliku [menu_main.c](#).

11.20.2.8 pending_action

menu_action_t menu_state_t::pending_action

Definicja w linii 395 pliku [menu_main.c](#).

11.20.2.9 pending_auth_level

menu_auth_level_t menu_state_t::pending_auth_level

Definicja w linii 387 pliku [menu_main.c](#).

11.20.2.10 pending_auth_page

menu_page_id_t menu_state_t::pending_auth_page

Definicja w linii 386 pliku [menu_main.c](#).

11.20.2.11 pending_target_node_id

uint32_t menu_state_t::pending_target_node_id

Definicja w linii 376 pliku [menu_main.c](#).

11.20.2.12 pin_digits

uint8_t menu_state_t::pin_digits[MENU_SETTINGS_PIN_LEN]

Definicja w linii 388 pliku [menu_main.c](#).

11.20.2.13 pin_index

uint8_t menu_state_t::pin_index

Definicja w linii 389 pliku [menu_main.c](#).

11.20.2.14 popup_enabled

bool menu_state_t::popup_enabled

Definicja w linii 392 pliku [menu_main.c](#).

11.20.2.15 quick_reply_deadline_ms

uint32_t menu_state_t::quick_reply_deadline_ms

Definicja w linii 381 pliku [menu_main.c](#).

11.20.2.16 quick_reply_last_seconds

`uint32_t menu_state_t::quick_reply_last_seconds`

Definicja w linii 382 pliku [menu_main.c](#).

11.20.2.17 quick_reply_src_id

`uint32_t menu_state_t::quick_reply_src_id`

Definicja w linii 380 pliku [menu_main.c](#).

11.20.2.18 quick_reply_text

`char menu_state_t::quick_reply_text[MENU_LINE_BUF_SIZE]`

Definicja w linii 383 pliku [menu_main.c](#).

11.20.2.19 selected_device_slot

`uint8_t menu_state_t::selected_device_slot`

Definicja w linii 375 pliku [menu_main.c](#).

11.20.2.20 selected_idx

`uint8_t menu_state_t::selected_idx`

Definicja w linii 373 pliku [menu_main.c](#).

11.20.2.21 send_target_action

`menu_action_t menu_state_t::send_target_action`

Definicja w linii 377 pliku [menu_main.c](#).

11.20.2.22 transient_parent_page

`menu_page_id_t menu_state_t::transient_parent_page`

Definicja w linii 378 pliku [menu_main.c](#).

11.20.2.23 tx_ack_waiting

`bool menu_state_t::tx_ack_waiting`

Definicja w linii 384 pliku [menu_main.c](#).

11.20.2.24 tx_sent_text

`char menu_state_t::tx_sent_text[MENU_LINE_BUF_SIZE]`

Definicja w linii 385 pliku [menu_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/menu_main.c](#)

11.21 Dokumentacja struktury `radio_context_t`

Pola danych

- `bool` [initialized](#)
- `sx1276_bus_t` [bus](#)
- `radio_hw_cfg_t` [hw](#)
- `radio_lora_cfg_t` [lora](#)
- `radio_event_cb_t` [callback](#)
- `void *` [callback_ctx](#)
- `volatile uint8_t` [dio_pending_mask](#)

- `volatile uint32_t event_flags`
- `radio_state_t state`
- `radio_state_t tx_resume_state`
- `uint16_t rx_single_timeout_symbols`
- `radio_packet_t last_packet`

11.21.1 Opis szczegółowy

Definicja w linii 21 pliku `radio_lora.c`.

11.21.2 Dokumentacja pól

11.21.2.1 bus

```
sx1276_bus_t radio_context_t::bus
```

Definicja w linii 24 pliku `radio_lora.c`.

11.21.2.2 callback

```
radio_event_cb_t radio_context_t::callback
```

Definicja w linii 27 pliku `radio_lora.c`.

11.21.2.3 callback_ctx

```
void* radio_context_t::callback_ctx
```

Definicja w linii 28 pliku `radio_lora.c`.

11.21.2.4 dio_pending_mask

```
volatile uint8_t radio_context_t::dio_pending_mask
```

Definicja w linii 29 pliku `radio_lora.c`.

11.21.2.5 event_flags

```
volatile uint32_t radio_context_t::event_flags
```

Definicja w linii 30 pliku `radio_lora.c`.

11.21.2.6 hw

```
radio_hw_cfg_t radio_context_t::hw
```

Definicja w linii 25 pliku `radio_lora.c`.

11.21.2.7 initialized

```
bool radio_context_t::initialized
```

Definicja w linii 23 pliku `radio_lora.c`.

11.21.2.8 last_packet

```
radio_packet_t radio_context_t::last_packet
```

Definicja w linii 34 pliku `radio_lora.c`.

11.21.2.9 lora

```
radio_lora_cfg_t radio_context_t::lora
```

Definicja w linii 26 pliku `radio_lora.c`.

11.21.2.10 rx_single_timeout_symbols

`uint16_t radio_context_t::rx_single_timeout_symbols`

Definicja w linii 33 pliku [radio_lora.c](#).

11.21.2.11 state

`radio_state_t radio_context_t::state`

Definicja w linii 31 pliku [radio_lora.c](#).

11.21.2.12 tx_resume_state

`radio_state_t radio_context_t::tx_resume_state`

Definicja w linii 32 pliku [radio_lora.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/radio_lib/modulations/lora/[radio_lora.c](#)

11.22 Dokumentacja struktury radio_fsk_cfg_t

Konfiguracja backendu FSK/GFSK/MSK/GMSK.

```
#include <Core/App/radio_lib/radio_lib.h>
```

Pola danych

- `uint32_t frequency_hz`
Częstotliwość RF [Hz].
- `uint32_t bitrate_bps`
Bitrate [bps].
- `radio_lora_bw_t rx_bandwidth`
Szerokość pasma RX zakodowana tym samym enumem co w LoRa.
- `radio_fsk_shaping_t shaping`
Odmiana/kanal shapingu modulacji.
- `radio_fsk_filter_t filter`
Typ filtra BT dla wariantów gaussowskich.
- `int8_t tx_power_dbm`
Moc TX [dBm].
- `uint16_t preamble_len`
Długość preambuły [B].
- `uint8_t sync_word_len`
Długość słowa synchronizacji [B].
- `uint32_t sync_word`
Wzorzec sync word, używane najmłodsze `sync_word_len` bajtów.
- `radio_address_filter_t address_filter`
Filtrowanie adresowe packet engine.
- `radio_packet_crc_t crc_type`
Typ CRC packet engine.
- `bool data_whitening`
Włączenie whitening-u danych.

11.22.1 Opis szczegółowy

Konfiguracja backendu FSK/GFSK/MSK/GMSK.

Definicja w linii 177 pliku [radio_lib.h](#).

11.22.2 Dokumentacja pól

11.22.2.1 address_filter

`radio_address_filter_t` `radio_fsk_cfg_t::address_filter`

Filtrowanie adresowe packet engine.

Definicja w linii 188 pliku `radio_lib.h`.

Odwołania w `radio_cfg_valid()`, `radio_default_fsk_cfg()` i `radio_get_packet_config_1()`.

11.22.2.2 bitrate_bps

`uint32_t` `radio_fsk_cfg_t::bitrate_bps`

Bitrate [bps].

Definicja w linii 180 pliku `radio_lib.h`.

Odwołania w `radio_apply_fsk_config()`, `radio_cfg_valid()`, `radio_default_fsk_cfg()` i `radio_get_frequency_deviation()`.

11.22.2.3 crc_type

`radio_packet_crc_t` `radio_fsk_cfg_t::crc_type`

Typ CRC packet engine.

Definicja w linii 189 pliku `radio_lib.h`.

Odwołania w `radio_cfg_valid()`, `radio_default_fsk_cfg()` i `radio_get_packet_config_1()`.

11.22.2.4 data_whitening

`bool` `radio_fsk_cfg_t::data_whitening`

Włączenie whitening-u danych.

Definicja w linii 190 pliku `radio_lib.h`.

Odwołania w `radio_default_fsk_cfg()` i `radio_get_packet_config_1()`.

11.22.2.5 filter

`radio_fsk_filter_t` `radio_fsk_cfg_t::filter`

Typ filtra BT dla wariantów gaussowskich.

Definicja w linii 183 pliku `radio_lib.h`.

Odwołania w `radio_cfg_valid()`, `radio_default_fsk_cfg()` i `radio_get_shaping_bits()`.

11.22.2.6 frequency_hz

`uint32_t` `radio_fsk_cfg_t::frequency_hz`

Częstotliwość RF [Hz].

Definicja w linii 179 pliku `radio_lib.h`.

Odwołania w `radio_apply_fsk_config()`, `radio_cfg_valid()` i `radio_default_fsk_cfg()`.

11.22.2.7 preamble_len

`uint16_t` `radio_fsk_cfg_t::preamble_len`

Długość preambuły [B].

Definicja w linii 185 pliku `radio_lib.h`.

Odwołania w `radio_apply_fsk_config()`, `radio_cfg_valid()` i `radio_default_fsk_cfg()`.

11.22.2.8 rx_bandwidth

`radio_lora_bw_t` `radio_fsk_cfg_t::rx_bandwidth`

Szerokość pasma RX zakodowana tym samym enumem co w LoRa.

Definicja w linii 181 pliku `radio_lib.h`.

Odwołania w `radio_apply_fsk_config()`, `radio_cfg_valid()` i `radio_default_fsk_cfg()`.

11.22.2.9 shaping

`radio_fsk_shaping_t` `radio_fsk_cfg_t::shaping`

Odmiana/kanal shapingu modulacji.

Definicja w linii 182 pliku [radio_lib.h](#).

Odwołania w [radio_cfg_valid\(\)](#), [radio_default_fsk_cfg\(\)](#), [radio_get_frequency_deviation\(\)](#) i [radio_get_shaping_bits\(\)](#).

11.22.2.10 sync_word

`uint32_t` `radio_fsk_cfg_t::sync_word`

Wzorzec sync word, używane najmłodsze `sync_word_len` bajtów.

Definicja w linii 187 pliku [radio_lib.h](#).

Odwołania w [radio_default_fsk_cfg\(\)](#) i [radio_write_sync_word\(\)](#).

11.22.2.11 sync_word_len

`uint8_t` `radio_fsk_cfg_t::sync_word_len`

Długość słowa synchronizacji [B].

Definicja w linii 186 pliku [radio_lib.h](#).

Odwołania w [radio_cfg_valid\(\)](#), [radio_default_fsk_cfg\(\)](#) i [radio_write_sync_word\(\)](#).

11.22.2.12 tx_power_dbm

`int8_t` `radio_fsk_cfg_t::tx_power_dbm`

Moc TX [dBm].

Definicja w linii 184 pliku [radio_lib.h](#).

Odwołania w [radio_apply_fsk_config\(\)](#) i [radio_default_fsk_cfg\(\)](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/radio_lib/radio_lib.h

11.23 Dokumentacja struktury `radio_fsk_context_t`

Pola danych

- bool `initialized`
- `sx1276_bus_t` `bus`
- `radio_hw_cfg_t` `hw`
- `radio_fsk_cfg_t` `cfg`
- `radio_event_cb_t` `callback`
- void * `callback_ctx`
- volatile `uint8_t` `dio_pending_mask`
- volatile `uint32_t` `event_flags`
- `radio_state_t` `state`
- `radio_state_t` `tx_resume_state`
- `uint32_t` `rx_single_deadline_ms`
- `uint32_t` `tx_deadline_ms`
- `radio_packet_t` `last_packet`

11.23.1 Opis szczegółowy

Definicja w linii 24 pliku [radio_fsk.c](#).

11.23.2 Dokumentacja pól

11.23.2.1 bus

`sx1276_bus_t` `radio_fsk_context_t::bus`

Definicja w linii 27 pliku [radio_fsk.c](#).

11.23.2.2 `callback`

`radio_event_cb_t` `radio_fsk_context_t::callback`
Definicja w linii 30 pliku [radio_fsk.c](#).

11.23.2.3 `callback_ctx`

`void*` `radio_fsk_context_t::callback_ctx`
Definicja w linii 31 pliku [radio_fsk.c](#).

11.23.2.4 `cfg`

`radio_fsk_cfg_t` `radio_fsk_context_t::cfg`
Definicja w linii 29 pliku [radio_fsk.c](#).

11.23.2.5 `dio_pending_mask`

`volatile uint8_t` `radio_fsk_context_t::dio_pending_mask`
Definicja w linii 32 pliku [radio_fsk.c](#).

11.23.2.6 `event_flags`

`volatile uint32_t` `radio_fsk_context_t::event_flags`
Definicja w linii 33 pliku [radio_fsk.c](#).

11.23.2.7 `hw`

`radio_hw_cfg_t` `radio_fsk_context_t::hw`
Definicja w linii 28 pliku [radio_fsk.c](#).

11.23.2.8 `initialized`

`bool` `radio_fsk_context_t::initialized`
Definicja w linii 26 pliku [radio_fsk.c](#).

11.23.2.9 `last_packet`

`radio_packet_t` `radio_fsk_context_t::last_packet`
Definicja w linii 38 pliku [radio_fsk.c](#).

11.23.2.10 `rx_single_deadline_ms`

`uint32_t` `radio_fsk_context_t::rx_single_deadline_ms`
Definicja w linii 36 pliku [radio_fsk.c](#).

11.23.2.11 `state`

`radio_state_t` `radio_fsk_context_t::state`
Definicja w linii 34 pliku [radio_fsk.c](#).

11.23.2.12 `tx_deadline_ms`

`uint32_t` `radio_fsk_context_t::tx_deadline_ms`
Definicja w linii 37 pliku [radio_fsk.c](#).

11.23.2.13 `tx_resume_state`

`radio_state_t` `radio_fsk_context_t::tx_resume_state`
Definicja w linii 35 pliku [radio_fsk.c](#).
Dokumentacja dla tej struktury została wygenerowana z pliku:

- `Core/App/radio_lib/modulations/fsk/radio_fsk.c`

11.24 Dokumentacja struktury `radio_gpio_t`

Opis pojedynczego pinu GPIO.

```
#include <Core/App/radio_lib/radio_lib.h>
```

Pola danych

- `GPIO_TypeDef * port`
Port GPIO.
- `uint16_t pin`
Maska pinu GPIO.

11.24.1 Opis szczegółowy

Opis pojedynczego pinu GPIO.

Definicja w linii 85 pliku `radio_lib.h`.

11.24.2 Dokumentacja pól

11.24.2.1 `pin`

```
uint16_t radio_gpio_t::pin
```

Maska pinu GPIO.

Definicja w linii 88 pliku `radio_lib.h`.

Odwwołania w `radio_fsk_init()`, `radio_lora_default_hw_cfg()`, `radio_lora_init()` i `radio_ook_init()`.

11.24.2.2 `port`

```
GPIO_TypeDef* radio_gpio_t::port
```

Port GPIO.

Definicja w linii 87 pliku `radio_lib.h`.

Odwwołania w `radio_fsk_init()`, `radio_lora_default_hw_cfg()`, `radio_lora_init()` i `radio_ook_init()`.

Dokumentacja dla tej struktury została wygenerowana z pliku:

- `Core/App/radio_lib/radio_lib.h`

11.25 Dokumentacja struktury `radio_hw_cfg_t`

Konfiguracja sprzętowa modułu radiowego.

```
#include <Core/App/radio_lib/radio_lib.h>
```

Pola danych

- `SPI_HandleTypeDef * hspi`
Uchwyt SPI używany przez transceiver.
- `radio_gpio_t nss`
Pin NSS/CS.
- `radio_gpio_t reset`
Pin RESET transceivera.
- `radio_gpio_t dio [6]`
Piny DIO0..DIO5 (IRQ/status).
- `uint32_t spi_timeout_ms`
Timeout blokujących transferów SPI.

11.25.1 Opis szczegółowy

Konfiguracja sprzętowa modułu radiowego.

Definicja w linii 94 pliku `radio_lib.h`.

11.25.2 Dokumentacja pól

11.25.2.1 dio

`radio_gpio_t` `radio_hw_cfg_t::dio[6]`

Piny DIO0..DIO5 (IRQ/status).

Definicja w linii 99 pliku `radio_lib.h`.

Odwołania w `radio_lora_default_hw_cfg()`.

11.25.2.2 hspi

`SPI_HandleTypeDef*` `radio_hw_cfg_t::hspi`

Uchwyt SPI używany przez transceiver.

Definicja w linii 96 pliku `radio_lib.h`.

Odwołania w `radio_fsk_init()`, `radio_lora_default_hw_cfg()`, `radio_lora_init()` i `radio_ook_init()`.

11.25.2.3 nss

`radio_gpio_t` `radio_hw_cfg_t::nss`

Pin NSS/CS.

Definicja w linii 97 pliku `radio_lib.h`.

Odwołania w `radio_fsk_init()`, `radio_lora_default_hw_cfg()`, `radio_lora_init()` i `radio_ook_init()`.

11.25.2.4 reset

`radio_gpio_t` `radio_hw_cfg_t::reset`

Pin RESET transceivera.

Definicja w linii 98 pliku `radio_lib.h`.

Odwołania w `radio_fsk_init()`, `radio_lora_default_hw_cfg()`, `radio_lora_init()` i `radio_ook_init()`.

11.25.2.5 spi_timeout_ms

`uint32_t` `radio_hw_cfg_t::spi_timeout_ms`

Timeout blokujących transferów SPI.

Definicja w linii 100 pliku `radio_lib.h`.

Odwołania w `radio_fsk_init()`, `radio_lora_default_hw_cfg()`, `radio_lora_init()` i `radio_ook_init()`.

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/radio_lib/`radio_lib.h`

11.26 Dokumentacja struktury radio_lora_cfg_t

Konfiguracja profilu LoRa.

```
#include <Core/App/radio_lib/radio_lib.h>
```

Pola danych

- `uint32_t` `frequency_hz`
Częstotliwość RF [Hz].
- `radio_lora_bw_t` `bandwidth`
Szerokość pasma.
- `uint8_t` `spreading_factor`
SF6..SF12.
- `uint8_t` `coding_rate`
Mianownik CR: 4/x, x=5..8.
- `uint16_t` `preamble_len`
Długość preambuły w symbolach.
- `uint8_t` `sync_word`

- `bool crc_on`
Włączenie CRC payloadu.
- `bool invert_iq`
Włączenie inwersji IQ.
- `int8_t tx_power_dbm`
Moc TX [dBm].
- `bool implicit_header`
Tryb nagłówka niejawnego.
- `uint8_t payload_len`
Stała długość payloadu (implicit header).

11.26.1 Opis szczegółowy

Konfiguracja profilu LoRa.

Definicja w linii 106 pliku [radio_lib.h](#).

11.26.2 Dokumentacja pól

11.26.2.1 bandwidth

```
radio_lora_bw_t radio_lora_cfg_t::bandwidth
```

Szerokość pasma.

Definicja w linii 109 pliku [radio_lib.h](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_cfg_valid\(\)](#), [radio_lora_default_lora_cfg\(\)](#), [radio_low_data_rate_optimize_required\(\)](#) i [radio_main_load_default_lora_preset\(\)](#).

11.26.2.2 coding_rate

```
uint8_t radio_lora_cfg_t::coding_rate
```

Mianownik CR: 4/x, x=5..8.

Definicja w linii 111 pliku [radio_lib.h](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_cfg_valid\(\)](#), [radio_lora_default_lora_cfg\(\)](#) i [radio_main_load_default_lora_preset\(\)](#).

11.26.2.3 crc_on

```
bool radio_lora_cfg_t::crc_on
```

Włączenie CRC payloadu.

Definicja w linii 114 pliku [radio_lib.h](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_lora_default_lora_cfg\(\)](#) i [radio_main_load_default_lora_preset\(\)](#).

11.26.2.4 frequency_hz

```
uint32_t radio_lora_cfg_t::frequency_hz
```

Częstotliwość RF [Hz].

Definicja w linii 108 pliku [radio_lib.h](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_cfg_valid\(\)](#), [radio_lora_default_lora_cfg\(\)](#) i [radio_main_load_default_lora_preset\(\)](#).

11.26.2.5 implicit_header

```
bool radio_lora_cfg_t::implicit_header
```

Tryb nagłówka niejawnego.

Definicja w linii 117 pliku [radio_lib.h](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_cfg_valid\(\)](#), [radio_lora_default_lora_cfg\(\)](#) i [radio_main_load_default_lora_preset\(\)](#).

11.26.2.6 invert_iq

`bool radio_lora_cfg_t::invert_iq`

Włączenie inwersji IQ.

Definicja w linii 115 pliku [radio_lib.h](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_lora_default_lora_cfg\(\)](#) i [radio_main_load_default_lora_preset\(\)](#).

11.26.2.7 payload_len

`uint8_t radio_lora_cfg_t::payload_len`

Stała długość payloadu (implicit header).

Definicja w linii 118 pliku [radio_lib.h](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_cfg_valid\(\)](#), [radio_lora_default_lora_cfg\(\)](#) i [radio_main_load_default_lora_preset\(\)](#).

11.26.2.8 preamble_len

`uint16_t radio_lora_cfg_t::preamble_len`

Długość preambuły w symbolach.

Definicja w linii 112 pliku [radio_lib.h](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_cfg_valid\(\)](#), [radio_lora_default_lora_cfg\(\)](#) i [radio_main_load_default_lora_preset\(\)](#).

11.26.2.9 spreading_factor

`uint8_t radio_lora_cfg_t::spreading_factor`

SF6..SF12.

Definicja w linii 110 pliku [radio_lib.h](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_cfg_valid\(\)](#), [radio_lora_default_lora_cfg\(\)](#), [radio_low_data_rate_optimize_required\(\)](#) i [radio_main_load_default_lora_preset\(\)](#).

11.26.2.10 sync_word

`uint8_t radio_lora_cfg_t::sync_word`

Sync word LoRa.

Definicja w linii 113 pliku [radio_lib.h](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_lora_default_lora_cfg\(\)](#) i [radio_main_load_default_lora_preset\(\)](#).

11.26.2.11 tx_power_dbm

`int8_t radio_lora_cfg_t::tx_power_dbm`

Moc TX [dBm].

Definicja w linii 116 pliku [radio_lib.h](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_lora_default_lora_cfg\(\)](#) i [radio_main_load_default_lora_preset\(\)](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/radio_lib/radio_lib.h](#)

11.27 Dokumentacja struktury radio_main_cmd_sync_t

Pola danych

- volatile bool [done](#)
- bool [result](#)
- [radio_main_runtime_cfg_t](#) [runtime_cfg](#)

11.27.1 Opis szczegółowy

Definicja w linii 87 pliku [radio_main.c](#).

11.27.2 Dokumentacja pól

11.27.2.1 done

`volatile bool radio_main_cmd_sync_t::done`

Definicja w linii 89 pliku [radio_main.c](#).

11.27.2.2 result

`bool radio_main_cmd_sync_t::result`

Definicja w linii 90 pliku [radio_main.c](#).

11.27.2.3 runtime_cfg

`radio_main_runtime_cfg_t radio_main_cmd_sync_t::runtime_cfg`

Definicja w linii 91 pliku [radio_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/radio_main.c

11.28 Dokumentacja struktury radio_main_cmd_t

Pola danych

- `radio_main_cmd_id_t id`
- `radio_main_cmd_sync_t * sync`
- union {
 - struct {
 - `uint8_t group_id`
 - `uint8_t msg_id`
 - `uint32_t dst_id`
 - send_template**
 - struct {
 - `uint32_t dst_id`
 - `uint8_t len`
 - `char text [21]`
 - send_text**
 - struct {
 - `const uint8_t * data`
 - `uint8_t len`
 - send_raw**
 - struct {
 - `uint8_t value`
 - set_u8**
 - struct {
 - `uint32_t value`
 - set_u32**
 - struct {
 - `radio_main_option_t option`
 - `uint32_t value`
 - set_option**
 - struct {
 - `bool enabled`
 - set_bool**
 - struct {
 - `uint32_t timeout_ms`
 - `bool network_mode`
 - pairing**
 - struct {


```
    uint32_t dst_id
} to_node
} u
```

11.28.1 Opis szczegółowy

Definicja w linii 94 pliku `radio_main.c`.

11.28.2 Dokumentacja pól

11.28.2.1 data

```
const uint8_t* radio_main_cmd_t::data
```

Definicja w linii 114 pliku `radio_main.c`.

11.28.2.2 dst_id

```
uint32_t radio_main_cmd_t::dst_id
```

Definicja w linii 104 pliku `radio_main.c`.

11.28.2.3 enabled

```
bool radio_main_cmd_t::enabled
```

Definicja w linii 132 pliku `radio_main.c`.

11.28.2.4 group_id

```
uint8_t radio_main_cmd_t::group_id
```

Definicja w linii 102 pliku `radio_main.c`.

11.28.2.5 id

```
radio_main_cmd_id_t radio_main_cmd_t::id
```

Definicja w linii 96 pliku `radio_main.c`.

11.28.2.6 len

```
uint8_t radio_main_cmd_t::len
```

Definicja w linii 109 pliku `radio_main.c`.

11.28.2.7 msg_id

```
uint8_t radio_main_cmd_t::msg_id
```

Definicja w linii 103 pliku `radio_main.c`.

11.28.2.8 network_mode

```
bool radio_main_cmd_t::network_mode
```

Definicja w linii 137 pliku `radio_main.c`.

11.28.2.9 option

```
radio_main_option_t radio_main_cmd_t::option
```

Definicja w linii 127 pliku `radio_main.c`.

11.28.2.10 sync

```
radio_main_cmd_sync_t* radio_main_cmd_t::sync
```

Definicja w linii 97 pliku `radio_main.c`.

11.28.2.11 text

```
char radio_main_cmd_t::text[21]
```

Definicja w linii 110 pliku [radio_main.c](#).

11.28.2.12 timeout_ms

```
uint32_t radio_main_cmd_t::timeout_ms
```

Definicja w linii 136 pliku [radio_main.c](#).

11.28.2.13 value [1/2]

```
uint32_t radio_main_cmd_t::value
```

Definicja w linii 123 pliku [radio_main.c](#).

11.28.2.14 value [2/2]

```
uint8_t radio_main_cmd_t::value
```

Definicja w linii 119 pliku [radio_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/radio_main.c

11.29 Dokumentacja struktury radio_main_ctx_t

Pola danych

- [radio_hw_cfg_t](#) hw
- [radio_lora_cfg_t](#) backend_cfg
- [radio_lora_cfg_t](#) lora_cfg
- [radio_main_modulation_t](#) modulation_id
- [radio_main_modulation_t](#) backend_modulation_id
- bool initialized
- bool fh_enabled
- uint32_t fh_period_ms
- bool coding_enabled
- bool auto_ping_enabled
- uint32_t auto_ping_period_ms
- [radio_main_auto_ping_mode_t](#) auto_ping_mode
- bool crypto_ready
- uint8_t lora_preset
- [radio_main_fsk_cfg_t](#) fsk_cfg
- [radio_main_ook_cfg_t](#) ook_cfg
- uint8_t hop_idx
- uint32_t last_hop_ms
- uint32_t last_ping_ms
- uint16_t node_id
- uint16_t next_msg_id
- uint32_t gateway_rx_counter
- uint32_t gateway_tx_counter
- bool gateway_key_mode_known
- [security_frame_key_mode_t](#) gateway_key_mode
- bool gateway_pair_code_valid
- uint8_t gateway_pair_code_len
- uint8_t gateway_pair_code [8]
- bool broadcast_group_key_valid
- uint32_t broadcast_group_epoch

- uint8_t broadcast_group_aes_key [16]
- uint8_t broadcast_group_hmac_key [32]
- bool broadcast_group_pending_valid
- uint32_t broadcast_group_pending_epoch
- uint8_t broadcast_group_pending_mask
- uint8_t broadcast_group_pending_key [RADIO_BCAST_GROUP_KEY_LEN]
- bool pairing_active
- bool pairing_network_mode
- uint32_t pairing_until_ms
- bool pairing_pending
- uint32_t pairing_pending_node
- uint8_t pairing_pending_code [8]
- uint8_t pairing_pending_code_len
- bool pairing_pending_network
- bool pairing_outgoing_pending
- uint8_t pairing_outgoing_code [8]
- uint8_t pairing_outgoing_code_len
- bool pairing_outgoing_network
- bool tx_in_progress
- bool tx_silent
- uint32_t tx_deadline_ms
- char last_error_text [21]
- struct {
 - bool active
 - uint16_t peer_id
 - uint16_t msg_id
 - uint32_t counter
 - uint8_t raw_len
 - uint8_t raw [LAVIET_FRAME_MAX_LEN]
 - uint8_t retries_done
 - uint32_t deadline_ms
 - uint32_t wait_ms
 } ack_pending
- struct {
 - bool valid
 - bool timed_out
 - uint16_t peer_id
 - uint16_t msg_id
 - uint32_t counter
 - uint32_t expires_ms
 } ack_recent

11.29.1 Opis szczegółowy

Definicja w linii 146 pliku radio_main.c.

11.29.2 Dokumentacja pól

11.29.2.1 active

```
bool radio_main_ctx_t::active
```

Definicja w linii 202 pliku radio_main.c.

11.29.2.2 auto_ping_enabled

`bool radio_main_ctx_t::auto_ping_enabled`

Definicja w linii 157 pliku [radio_main.c](#).

11.29.2.3 auto_ping_mode

`radio_main_auto_ping_mode_t radio_main_ctx_t::auto_ping_mode`

Definicja w linii 159 pliku [radio_main.c](#).

11.29.2.4 auto_ping_period_ms

`uint32_t radio_main_ctx_t::auto_ping_period_ms`

Definicja w linii 158 pliku [radio_main.c](#).

11.29.2.5 backend_cfg

`radio_lora_cfg_t radio_main_ctx_t::backend_cfg`

Definicja w linii 149 pliku [radio_main.c](#).

11.29.2.6 backend_modulation_id

`radio_main_modulation_t radio_main_ctx_t::backend_modulation_id`

Definicja w linii 152 pliku [radio_main.c](#).

11.29.2.7 broadcast_group_aes_key

`uint8_t radio_main_ctx_t::broadcast_group_aes_key[16]`

Definicja w linii 178 pliku [radio_main.c](#).

11.29.2.8 broadcast_group_epoch

`uint32_t radio_main_ctx_t::broadcast_group_epoch`

Definicja w linii 177 pliku [radio_main.c](#).

11.29.2.9 broadcast_group_hmac_key

`uint8_t radio_main_ctx_t::broadcast_group_hmac_key[32]`

Definicja w linii 179 pliku [radio_main.c](#).

11.29.2.10 broadcast_group_key_valid

`bool radio_main_ctx_t::broadcast_group_key_valid`

Definicja w linii 176 pliku [radio_main.c](#).

11.29.2.11 broadcast_group_pending_epoch

`uint32_t radio_main_ctx_t::broadcast_group_pending_epoch`

Definicja w linii 181 pliku [radio_main.c](#).

11.29.2.12 broadcast_group_pending_key

`uint8_t radio_main_ctx_t::broadcast_group_pending_key[RADIO_BCAST_GROUP_KEY_LEN]`

Definicja w linii 183 pliku [radio_main.c](#).

11.29.2.13 broadcast_group_pending_mask

`uint8_t radio_main_ctx_t::broadcast_group_pending_mask`

Definicja w linii 182 pliku [radio_main.c](#).

11.29.2.14 broadcast_group_pending_valid

`bool radio_main_ctx_t::broadcast_group_pending_valid`
Definicja w linii 180 pliku [radio_main.c](#).

11.29.2.15 coding_enabled

`bool radio_main_ctx_t::coding_enabled`
Definicja w linii 156 pliku [radio_main.c](#).

11.29.2.16 counter

`uint32_t radio_main_ctx_t::counter`
Definicja w linii 205 pliku [radio_main.c](#).

11.29.2.17 crypto_ready

`bool radio_main_ctx_t::crypto_ready`
Definicja w linii 160 pliku [radio_main.c](#).

11.29.2.18 deadline_ms

`uint32_t radio_main_ctx_t::deadline_ms`
Definicja w linii 209 pliku [radio_main.c](#).

11.29.2.19 expires_ms

`uint32_t radio_main_ctx_t::expires_ms`
Definicja w linii 219 pliku [radio_main.c](#).

11.29.2.20 fh_enabled

`bool radio_main_ctx_t::fh_enabled`
Definicja w linii 154 pliku [radio_main.c](#).

11.29.2.21 fh_period_ms

`uint32_t radio_main_ctx_t::fh_period_ms`
Definicja w linii 155 pliku [radio_main.c](#).

11.29.2.22 fsk_cfg

`radio_main_fsk_cfg_t radio_main_ctx_t::fsk_cfg`
Definicja w linii 162 pliku [radio_main.c](#).

11.29.2.23 gateway_key_mode

`security_frame_key_mode_t radio_main_ctx_t::gateway_key_mode`
Definicja w linii 172 pliku [radio_main.c](#).

11.29.2.24 gateway_key_mode_known

`bool radio_main_ctx_t::gateway_key_mode_known`
Definicja w linii 171 pliku [radio_main.c](#).

11.29.2.25 gateway_pair_code

`uint8_t radio_main_ctx_t::gateway_pair_code[8]`
Definicja w linii 175 pliku [radio_main.c](#).

11.29.2.26 gateway_pair_code_len

uint8_t radio_main_ctx_t::gateway_pair_code_len
Definicja w linii 174 pliku [radio_main.c](#).

11.29.2.27 gateway_pair_code_valid

bool radio_main_ctx_t::gateway_pair_code_valid
Definicja w linii 173 pliku [radio_main.c](#).

11.29.2.28 gateway_rx_counter

uint32_t radio_main_ctx_t::gateway_rx_counter
Definicja w linii 169 pliku [radio_main.c](#).

11.29.2.29 gateway_tx_counter

uint32_t radio_main_ctx_t::gateway_tx_counter
Definicja w linii 170 pliku [radio_main.c](#).

11.29.2.30 hop_idx

uint8_t radio_main_ctx_t::hop_idx
Definicja w linii 164 pliku [radio_main.c](#).

11.29.2.31 hw

[radio_hw_cfg_t](#) radio_main_ctx_t::hw
Definicja w linii 148 pliku [radio_main.c](#).

11.29.2.32 initialized

bool radio_main_ctx_t::initialized
Definicja w linii 153 pliku [radio_main.c](#).

11.29.2.33 last_error_text

char radio_main_ctx_t::last_error_text[21]
Definicja w linii 199 pliku [radio_main.c](#).

11.29.2.34 last_hop_ms

uint32_t radio_main_ctx_t::last_hop_ms
Definicja w linii 165 pliku [radio_main.c](#).

11.29.2.35 last_ping_ms

uint32_t radio_main_ctx_t::last_ping_ms
Definicja w linii 166 pliku [radio_main.c](#).

11.29.2.36 lora_cfg

[radio_lora_cfg_t](#) radio_main_ctx_t::lora_cfg
Definicja w linii 150 pliku [radio_main.c](#).

11.29.2.37 lora_preset

uint8_t radio_main_ctx_t::lora_preset
Definicja w linii 161 pliku [radio_main.c](#).

11.29.2.38 modulation_id

`radio_main_modulation_t radio_main_ctx_t::modulation_id`
Definicja w linii 151 pliku [radio_main.c](#).

11.29.2.39 msg_id

`uint16_t radio_main_ctx_t::msg_id`
Definicja w linii 204 pliku [radio_main.c](#).

11.29.2.40 next_msg_id

`uint16_t radio_main_ctx_t::next_msg_id`
Definicja w linii 168 pliku [radio_main.c](#).

11.29.2.41 node_id

`uint16_t radio_main_ctx_t::node_id`
Definicja w linii 167 pliku [radio_main.c](#).

11.29.2.42 ook_cfg

`radio_main_ook_cfg_t radio_main_ctx_t::ook_cfg`
Definicja w linii 163 pliku [radio_main.c](#).

11.29.2.43 pairing_active

`bool radio_main_ctx_t::pairing_active`
Definicja w linii 184 pliku [radio_main.c](#).

11.29.2.44 pairing_network_mode

`bool radio_main_ctx_t::pairing_network_mode`
Definicja w linii 185 pliku [radio_main.c](#).

11.29.2.45 pairing_outgoing_code

`uint8_t radio_main_ctx_t::pairing_outgoing_code[8]`
Definicja w linii 193 pliku [radio_main.c](#).

11.29.2.46 pairing_outgoing_code_len

`uint8_t radio_main_ctx_t::pairing_outgoing_code_len`
Definicja w linii 194 pliku [radio_main.c](#).

11.29.2.47 pairing_outgoing_network

`bool radio_main_ctx_t::pairing_outgoing_network`
Definicja w linii 195 pliku [radio_main.c](#).

11.29.2.48 pairing_outgoing_pending

`bool radio_main_ctx_t::pairing_outgoing_pending`
Definicja w linii 192 pliku [radio_main.c](#).

11.29.2.49 pairing_pending

`bool radio_main_ctx_t::pairing_pending`
Definicja w linii 187 pliku [radio_main.c](#).

11.29.2.50 pairing_pending_code

```
uint8_t radio_main_ctx_t::pairing_pending_code[8]
```

Definicja w linii 189 pliku [radio_main.c](#).

11.29.2.51 pairing_pending_code_len

```
uint8_t radio_main_ctx_t::pairing_pending_code_len
```

Definicja w linii 190 pliku [radio_main.c](#).

11.29.2.52 pairing_pending_network

```
bool radio_main_ctx_t::pairing_pending_network
```

Definicja w linii 191 pliku [radio_main.c](#).

11.29.2.53 pairing_pending_node

```
uint32_t radio_main_ctx_t::pairing_pending_node
```

Definicja w linii 188 pliku [radio_main.c](#).

11.29.2.54 pairing_until_ms

```
uint32_t radio_main_ctx_t::pairing_until_ms
```

Definicja w linii 186 pliku [radio_main.c](#).

11.29.2.55 peer_id

```
uint16_t radio_main_ctx_t::peer_id
```

Definicja w linii 203 pliku [radio_main.c](#).

11.29.2.56 raw

```
uint8_t radio_main_ctx_t::raw[LAVIET_FRAME_MAX_LEN]
```

Definicja w linii 207 pliku [radio_main.c](#).

11.29.2.57 raw_len

```
uint8_t radio_main_ctx_t::raw_len
```

Definicja w linii 206 pliku [radio_main.c](#).

11.29.2.58 retries_done

```
uint8_t radio_main_ctx_t::retries_done
```

Definicja w linii 208 pliku [radio_main.c](#).

11.29.2.59 timed_out

```
bool radio_main_ctx_t::timed_out
```

Definicja w linii 215 pliku [radio_main.c](#).

11.29.2.60 tx_deadline_ms

```
uint32_t radio_main_ctx_t::tx_deadline_ms
```

Definicja w linii 198 pliku [radio_main.c](#).

11.29.2.61 tx_in_progress

```
bool radio_main_ctx_t::tx_in_progress
```

Definicja w linii 196 pliku [radio_main.c](#).

11.29.2.62 tx_silent

bool radio_main_ctx_t::tx_silent

Definicja w linii 197 pliku [radio_main.c](#).

11.29.2.63 valid

bool radio_main_ctx_t::valid

Definicja w linii 214 pliku [radio_main.c](#).

11.29.2.64 wait_ms

uint32_t radio_main_ctx_t::wait_ms

Definicja w linii 210 pliku [radio_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/radio_main.c

11.30 Dokumentacja struktury radio_main_fsk_cfg_t

Profil rodziny FSK/GFSK/MSK/GMSK trzymany przez warstwę aplikacji.

```
#include <Core/App/radio_main.h>
```

Pola danych

- [radio_main_fsk_shaping_t](#) [shaping](#)
Wybrana odmiana modulacji FSK.
- uint32_t [frequency_hz](#)
Częstotliwość nośna [Hz].
- uint32_t [bitrate_bps](#)
Bitrate [bps].
- [radio_lora_bw_t](#) [rx_bandwidth](#)
Umowne kodowanie pasma RX na potrzeby UI/profilu.
- [radio_main_filter_t](#) [filter](#)
Filtr i BT dla gaussowskiego shapingu.
- int8_t [tx_power_dbm](#)
Moc TX [dBm].
- uint16_t [preamble_len](#)
Długość preambuły [bajty/symbole].
- uint8_t [sync_word_len](#)
Długość słowa synchronizacji [B].
- uint64_t [sync_word](#)
Wzorzec sync word, używane najmłodsze [sync_word_len](#) bajtów.
- [radio_main_address_filter_t](#) [address_filter](#)
Tryb adresowania pakietów.
- [radio_main_crc_type_t](#) [crc_type](#)
Typ CRC pakietowego.
- bool [data_whitening](#)
Włączenie whitening-u danych.

11.30.1 Opis szczegółowy

Profil rodziny FSK/GFSK/MSK/GMSK trzymany przez warstwę aplikacji.

Definicja w linii 152 pliku [radio_main.h](#).

11.30.2 Dokumentacja pól

11.30.2.1 address_filter

`radio_main_address_filter_t` `radio_main_fsk_cfg_t::address_filter`

Tryb adresowania pakietów.

Definicja w linii 163 pliku `radio_main.h`.

Odwołania w `radio_main_load_default_fsk_profile()`.

11.30.2.2 bitrate_bps

`uint32_t` `radio_main_fsk_cfg_t::bitrate_bps`

Bitrate [bps].

Definicja w linii 156 pliku `radio_main.h`.

Odwołania w `radio_main_load_default_fsk_profile()`.

11.30.2.3 crc_type

`radio_main_crc_type_t` `radio_main_fsk_cfg_t::crc_type`

Typ CRC pakietowego.

Definicja w linii 164 pliku `radio_main.h`.

Odwołania w `radio_main_load_default_fsk_profile()`.

11.30.2.4 data_whitening

`bool` `radio_main_fsk_cfg_t::data_whitening`

Włączenie whitening-u danych.

Definicja w linii 165 pliku `radio_main.h`.

Odwołania w `radio_main_load_default_fsk_profile()`.

11.30.2.5 filter

`radio_main_filter_t` `radio_main_fsk_cfg_t::filter`

Filtr i BT dla gaussowskiego shapingu.

Definicja w linii 158 pliku `radio_main.h`.

Odwołania w `radio_main_load_default_fsk_profile()`.

11.30.2.6 frequency_hz

`uint32_t` `radio_main_fsk_cfg_t::frequency_hz`

Częstotliwość nośna [Hz].

Definicja w linii 155 pliku `radio_main.h`.

Odwołania w `radio_main_load_default_fsk_profile()`.

11.30.2.7 preamble_len

`uint16_t` `radio_main_fsk_cfg_t::preamble_len`

Długość preambuły [bajty/symbole].

Definicja w linii 160 pliku `radio_main.h`.

Odwołania w `radio_main_load_default_fsk_profile()`.

11.30.2.8 rx_bandwidth

`radio_lora_bw_t` `radio_main_fsk_cfg_t::rx_bandwidth`

Umowne kodowanie pasma RX na potrzeby UI/profilu.

Definicja w linii 157 pliku `radio_main.h`.

Odwołania w `radio_main_load_default_fsk_profile()`.

11.30.2.9 shaping

`radio_main_fsk_shaping_t` `radio_main_fsk_cfg_t::shaping`

Wybrana odmiana modulacji FSK.

Definicja w linii 154 pliku `radio_main.h`.

Odwołania w `radio_main_load_default_fsk_profile()`.

11.30.2.10 sync_word

`uint64_t` `radio_main_fsk_cfg_t::sync_word`

Wzorzec sync word, używane najmłodsze `sync_word_len` bajtów.

Definicja w linii 162 pliku `radio_main.h`.

Odwołania w `radio_main_load_default_fsk_profile()`.

11.30.2.11 sync_word_len

`uint8_t` `radio_main_fsk_cfg_t::sync_word_len`

Długość słowa synchronizacji [B].

Definicja w linii 161 pliku `radio_main.h`.

Odwołania w `radio_main_load_default_fsk_profile()`.

11.30.2.12 tx_power_dbm

`int8_t` `radio_main_fsk_cfg_t::tx_power_dbm`

Moc TX [dBm].

Definicja w linii 159 pliku `radio_main.h`.

Odwołania w `radio_main_load_default_fsk_profile()`.

Dokumentacja dla tej struktury została wygenerowana z pliku:

- `Core/App/radio_main.h`

11.31 Dokumentacja struktury radio_main_ook_cfg_t

Profil rodziny OOK przechowywany przez warstwę aplikacji.

```
#include <Core/App/radio_main.h>
```

Pola danych

- `uint32_t` `frequency_hz`
Częstotliwość nośna [Hz].
- `uint32_t` `bitrate_bps`
Bitrate [bps].
- `radio_lora_bw_t` `rx_bandwidth`
Umowne kodowanie szerokości pasma RX.
- `int8_t` `tx_power_dbm`
Moc TX [dBm].
- `uint16_t` `preamble_len`
Długość preambuły [B].
- `uint8_t` `sync_word_len`
Długość słowa synchronizacji [B].
- `uint32_t` `sync_word`
Wzorzec sync word.
- `radio_main_ook_threshold_t` `threshold`
Tryb progu detekcji OOK.
- `uint8_t` `threshold_value`
Surowa wartość progu dla trybu wybranego powyżej.

11.31.1 Opis szczegółowy

Profil rodziny OOK przechowywany przez warstwę aplikacji.
Definicja w linii 171 pliku [radio_main.h](#).

11.31.2 Dokumentacja pól

11.31.2.1 bitrate_bps

```
uint32_t radio_main_ook_cfg_t::bitrate_bps
```

Bitrate [bps].
Definicja w linii 174 pliku [radio_main.h](#).
Odwołania w [radio_main_load_default_ook_profile\(\)](#).

11.31.2.2 frequency_hz

```
uint32_t radio_main_ook_cfg_t::frequency_hz
```

Częstotliwość nośna [Hz].
Definicja w linii 173 pliku [radio_main.h](#).
Odwołania w [radio_main_load_default_ook_profile\(\)](#).

11.31.2.3 preamble_len

```
uint16_t radio_main_ook_cfg_t::preamble_len
```

Długość preambuły [B].
Definicja w linii 177 pliku [radio_main.h](#).
Odwołania w [radio_main_load_default_ook_profile\(\)](#).

11.31.2.4 rx_bandwidth

```
radio_lora_bw_t radio_main_ook_cfg_t::rx_bandwidth
```

Umowne kodowanie szerokości pasma RX.
Definicja w linii 175 pliku [radio_main.h](#).
Odwołania w [radio_main_load_default_ook_profile\(\)](#).

11.31.2.5 sync_word

```
uint32_t radio_main_ook_cfg_t::sync_word
```

Wzorzec sync word.
Definicja w linii 179 pliku [radio_main.h](#).
Odwołania w [radio_main_load_default_ook_profile\(\)](#).

11.31.2.6 sync_word_len

```
uint8_t radio_main_ook_cfg_t::sync_word_len
```

Długość słowa synchronizacji [B].
Definicja w linii 178 pliku [radio_main.h](#).
Odwołania w [radio_main_load_default_ook_profile\(\)](#).

11.31.2.7 threshold

```
radio_main_ook_threshold_t radio_main_ook_cfg_t::threshold
```

Tryb progu detekcji OOK.
Definicja w linii 180 pliku [radio_main.h](#).
Odwołania w [radio_main_load_default_ook_profile\(\)](#).

11.31.2.8 threshold_value

```
uint8_t radio_main_ook_cfg_t::threshold_value
```

Surowa wartość progu dla trybu wybranego powyżej.

Definicja w linii 181 pliku [radio_main.h](#).

Odwołania w [radio_main_load_default_ook_profile\(\)](#).

11.31.2.9 tx_power_dbm

`int8_t radio_main_ook_cfg_t::tx_power_dbm`

Moc TX [dBm].

Definicja w linii 176 pliku [radio_main.h](#).

Odwołania w [radio_main_load_default_ook_profile\(\)](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/radio_main.h](#)

11.32 Dokumentacja struktury radio_main_runtime_cfg_t

Snapshot pełnej konfiguracji radiowej widzianej przez aplikację.

```
#include <Core/App/radio_main.h>
```

Pola danych

- [radio_main_modulation_t active_modulation](#)
Aktualnie wybrana rodzina modulacji.
- [radio_lora_cfg_t lora](#)
Profil LoRa.
- [radio_main_fsk_cfg_t fsk](#)
Profil FSK/GFSK/MSK/GMSK.
- [radio_main_ook_cfg_t ook](#)
Profil OOK.
- `bool fh_enabled`
Flaga frequency hopping.
- `uint32_t fh_period_ms`
Okres przeskoku kanału [ms] dla FH w LoRa.
- `bool coding_enabled`
Flaga szyfrowania ruchu OPERATOR.
- `bool auto_ping_enabled`
Flaga automatycznego PING.
- `uint32_t auto_ping_period_ms`
Wybrany okres automatycznego PING [ms].
- [radio_main_auto_ping_mode_t auto_ping_mode](#)
Sposób kodowania automatycznego PING.

11.32.1 Opis szczegółowy

Snapshot pełnej konfiguracji radiowej widzianej przez aplikację.

Definicja w linii 187 pliku [radio_main.h](#).

11.32.2 Dokumentacja pól

11.32.2.1 active_modulation

`radio_main_modulation_t radio_main_runtime_cfg_t::active_modulation`

Aktualnie wybrana rodzina modulacji.

Definicja w linii 189 pliku [radio_main.h](#).

11.32.2.2 auto_ping_enabled

`bool radio_main_runtime_cfg_t::auto_ping_enabled`

Flaga automatycznego PING.

Definicja w linii 196 pliku [radio_main.h](#).

11.32.2.3 auto_ping_mode

`radio_main_auto_ping_mode_t radio_main_runtime_cfg_t::auto_ping_mode`

Sposób kodowania automatycznego PING.

Definicja w linii 198 pliku [radio_main.h](#).

11.32.2.4 auto_ping_period_ms

`uint32_t radio_main_runtime_cfg_t::auto_ping_period_ms`

Wybrany okres automatycznego PING [ms].

Definicja w linii 197 pliku [radio_main.h](#).

11.32.2.5 coding_enabled

`bool radio_main_runtime_cfg_t::coding_enabled`

Flaga szyfrowania ruchu OPERATOR.

Definicja w linii 195 pliku [radio_main.h](#).

11.32.2.6 fh_enabled

`bool radio_main_runtime_cfg_t::fh_enabled`

Flaga frequency hopping.

Definicja w linii 193 pliku [radio_main.h](#).

11.32.2.7 fh_period_ms

`uint32_t radio_main_runtime_cfg_t::fh_period_ms`

Okres przeskoku kanału [ms] dla FH w LoRa.

Definicja w linii 194 pliku [radio_main.h](#).

11.32.2.8 fsk

`radio_main_fsk_cfg_t radio_main_runtime_cfg_t::fsk`

Profil FSK/GFSK/MSK/GMSK.

Definicja w linii 191 pliku [radio_main.h](#).

11.32.2.9 lora

`radio_lora_cfg_t radio_main_runtime_cfg_t::lora`

Profil LoRa.

Definicja w linii 190 pliku [radio_main.h](#).

11.32.2.10 ook

`radio_main_ook_cfg_t radio_main_runtime_cfg_t::ook`

Profil OOK.

Definicja w linii 192 pliku [radio_main.h](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/radio_main.h](#)

11.33 Dokumentacja struktury radio_ook_cfg_t

Konfiguracja backendu OOK.

```
#include <Core/App/radio_lib/radio_lib.h>
```

Pola danych

- uint32_t [frequency_hz](#)
Częstotliwość nośna [Hz].
- uint32_t [bitrate_bps](#)
Bitrate [bps].
- [radio_lora_bw_t rx_bandwidth](#)
Szerokość pasma RX.
- int8_t [tx_power_dbm](#)
Moc TX [dBm].
- uint16_t [preamble_len](#)
Długość preambuły [B].
- uint8_t [sync_word_len](#)
Długość słowa synchronizacji [B].
- uint32_t [sync_word](#)
Wzorec sync word, używane najmłodsze [sync_word_len](#) bajtów.
- [radio_ook_threshold_t threshold](#)
Tryb progu OOK.
- uint8_t [threshold_value](#)
Surowa wartość progu zależna od trybu.

11.33.1 Opis szczegółowy

Konfiguracja backendu OOK.

Definicja w linii 196 pliku [radio_lib.h](#).

11.33.2 Dokumentacja pól

11.33.2.1 bitrate_bps

```
uint32_t radio_ook_cfg_t::bitrate_bps
```

Bitrate [bps].

Definicja w linii 199 pliku [radio_lib.h](#).

Odwołania w [radio_apply_ook_config\(\)](#), [radio_cfg_valid\(\)](#) i [radio_default_ook_cfg\(\)](#).

11.33.2.2 frequency_hz

```
uint32_t radio_ook_cfg_t::frequency_hz
```

Częstotliwość nośna [Hz].

Definicja w linii 198 pliku [radio_lib.h](#).

Odwołania w [radio_apply_ook_config\(\)](#), [radio_cfg_valid\(\)](#) i [radio_default_ook_cfg\(\)](#).

11.33.2.3 preamble_len

```
uint16_t radio_ook_cfg_t::preamble_len
```

Długość preambuły [B].

Definicja w linii 202 pliku [radio_lib.h](#).

Odwołania w [radio_apply_ook_config\(\)](#), [radio_cfg_valid\(\)](#) i [radio_default_ook_cfg\(\)](#).

11.33.2.4 rx_bandwidth

`radio_lora_bw_t` `radio_ook_cfg_t::rx_bandwidth`

Szerokość pasma RX.

Definicja w linii 200 pliku `radio_lib.h`.

Odwołania w `radio_apply_ook_config()`, `radio_cfg_valid()` i `radio_default_ook_cfg()`.

11.33.2.5 sync_word

`uint32_t` `radio_ook_cfg_t::sync_word`

Wzorzec sync word, używane najmłodsze `sync_word_len` bajtów.

Definicja w linii 204 pliku `radio_lib.h`.

Odwołania w `radio_default_ook_cfg()` i `radio_write_sync_word()`.

11.33.2.6 sync_word_len

`uint8_t` `radio_ook_cfg_t::sync_word_len`

Długość słowa synchronizacji [B].

Definicja w linii 203 pliku `radio_lib.h`.

Odwołania w `radio_cfg_valid()`, `radio_default_ook_cfg()` i `radio_write_sync_word()`.

11.33.2.7 threshold

`radio_ook_threshold_t` `radio_ook_cfg_t::threshold`

Tryb progu OOK.

Definicja w linii 205 pliku `radio_lib.h`.

Odwołania w `radio_apply_threshold()`, `radio_cfg_valid()` i `radio_default_ook_cfg()`.

11.33.2.8 threshold_value

`uint8_t` `radio_ook_cfg_t::threshold_value`

Surowa wartość progu zależna od trybu.

Definicja w linii 206 pliku `radio_lib.h`.

Odwołania w `radio_apply_threshold()` i `radio_default_ook_cfg()`.

11.33.2.9 tx_power_dbm

`int8_t` `radio_ook_cfg_t::tx_power_dbm`

Moc TX [dBm].

Definicja w linii 201 pliku `radio_lib.h`.

Odwołania w `radio_apply_ook_config()` i `radio_default_ook_cfg()`.

Dokumentacja dla tej struktury została wygenerowana z pliku:

- `Core/App/radio_lib/radio_lib.h`

11.34 Dokumentacja struktury `radio_ook_context_t`

Pola danych

- `bool` `initialized`
- `sx1276_bus_t` `bus`
- `radio_hw_cfg_t` `hw`
- `radio_ook_cfg_t` `cfg`
- `radio_event_cb_t` `callback`
- `void *` `callback_ctx`
- `volatile uint8_t` `dio_pending_mask`
- `volatile uint32_t` `event_flags`
- `radio_state_t` `state`
- `radio_state_t` `tx_resume_state`

- `uint32_t rx_single_deadline_ms`
- `uint32_t tx_deadline_ms`
- `radio_packet_t last_packet`

11.34.1 Opis szczegółowy

Definicja w linii 21 pliku `radio_ook.c`.

11.34.2 Dokumentacja pól

11.34.2.1 bus

`sx1276_bus_t radio_ook_context_t::bus`

Definicja w linii 24 pliku `radio_ook.c`.

11.34.2.2 callback

`radio_event_cb_t radio_ook_context_t::callback`

Definicja w linii 27 pliku `radio_ook.c`.

11.34.2.3 callback_ctx

`void* radio_ook_context_t::callback_ctx`

Definicja w linii 28 pliku `radio_ook.c`.

11.34.2.4 cfg

`radio_ook_cfg_t radio_ook_context_t::cfg`

Definicja w linii 26 pliku `radio_ook.c`.

11.34.2.5 dio_pending_mask

`volatile uint8_t radio_ook_context_t::dio_pending_mask`

Definicja w linii 29 pliku `radio_ook.c`.

11.34.2.6 event_flags

`volatile uint32_t radio_ook_context_t::event_flags`

Definicja w linii 30 pliku `radio_ook.c`.

11.34.2.7 hw

`radio_hw_cfg_t radio_ook_context_t::hw`

Definicja w linii 25 pliku `radio_ook.c`.

11.34.2.8 initialized

`bool radio_ook_context_t::initialized`

Definicja w linii 23 pliku `radio_ook.c`.

11.34.2.9 last_packet

`radio_packet_t radio_ook_context_t::last_packet`

Definicja w linii 35 pliku `radio_ook.c`.

11.34.2.10 rx_single_deadline_ms

`uint32_t radio_ook_context_t::rx_single_deadline_ms`

Definicja w linii 33 pliku `radio_ook.c`.

11.34.2.11 state

`radio_state_t` `radio_ook_context_t::state`

Definicja w linii 31 pliku `radio_ook.c`.

11.34.2.12 tx_deadline_ms

`uint32_t` `radio_ook_context_t::tx_deadline_ms`

Definicja w linii 34 pliku `radio_ook.c`.

11.34.2.13 tx_resume_state

`radio_state_t` `radio_ook_context_t::tx_resume_state`

Definicja w linii 32 pliku `radio_ook.c`.

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/radio_lib/modulations/ook/`radio_ook.c`

11.35 Dokumentacja struktury `radio_packet_t`

Bufor ostatnio odebranej ramki.

```
#include <Core/App/radio_lib/radio_lib.h>
```

Pola danych

- `uint8_t` `data` [`RADIO_LIB_MAX_PAYLOAD`]
Payload ramki.
- `uint8_t` `length`
Długość payloadu.
- `int16_t` `rsi_dbm`
RSSI pakietu [dBm].
- `int8_t` `snr_db`
SNR pakietu [dB].
- `uint32_t` `timestamp_ms`
*Znacznik czasu (*HAL_GetTick*).*
- `bool` `valid`
Flaga poprawności danych bufora.

11.35.1 Opis szczegółowy

Bufor ostatnio odebranej ramki.

Definicja w linii 212 pliku `radio_lib.h`.

11.35.2 Dokumentacja pól

11.35.2.1 data

`uint8_t` `radio_packet_t::data` [`RADIO_LIB_MAX_PAYLOAD`]

Payload ramki.

Definicja w linii 214 pliku `radio_lib.h`.

Odwwołania w `radio_test_print_packet()`, `radio_test_print_packet_text()`, `radio_test_publish_packet_to_lcd()` i `radio_test_store_packet_in_i2c_memory()`.

11.35.2.2 length

`uint8_t radio_packet_t::length`

Długość payloadu.

Definicja w linii 215 pliku [radio_lib.h](#).

Odwolania w [radio_test_print_packet\(\)](#), [radio_test_print_packet_text\(\)](#), [radio_test_publish_packet_to_lcd\(\)](#) i [radio_test_store_packet_in_i2c_memory\(\)](#).

11.35.2.3 rssi_dbm

`int16_t radio_packet_t::rssi_dbm`

RSSI pakietu [dBm].

Definicja w linii 216 pliku [radio_lib.h](#).

Odwolania w [radio_test_print_packet\(\)](#), [radio_test_publish_packet_to_lcd\(\)](#) i [radio_test_store_packet_in_i2c_memory\(\)](#).

11.35.2.4 snr_db

`int8_t radio_packet_t::snr_db`

SNR pakietu [dB].

Definicja w linii 217 pliku [radio_lib.h](#).

Odwolania w [radio_test_print_packet\(\)](#).

11.35.2.5 timestamp_ms

`uint32_t radio_packet_t::timestamp_ms`

Znacznik czasu (`HAL_GetTick`).

Definicja w linii 218 pliku [radio_lib.h](#).

11.35.2.6 valid

`bool radio_packet_t::valid`

Flaga poprawności danych bufora.

Definicja w linii 219 pliku [radio_lib.h](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- `Core/App/radio_lib/radio_lib.h`

11.36 Dokumentacja struktury `security_cmd_sync_t`

Pola danych

- volatile bool [done](#)
- bool [result](#)
- [trusted_info_t](#) [trusted_out](#)
- [security_runtime_cfg_t](#) [runtime_out](#)

11.36.1 Opis szczegółowy

Definicja w linii 92 pliku [security_main.c](#).

11.36.2 Dokumentacja pól

11.36.2.1 done

`volatile bool security_cmd_sync_t::done`

Definicja w linii 94 pliku [security_main.c](#).

11.36.2.2 result

`bool security_cmd_sync_t::result`

Definicja w linii 95 pliku [security_main.c](#).

11.36.2.3 runtime_out

`security_runtime_cfg_t security_cmd_sync_t::runtime_out`

Definicja w linii 97 pliku [security_main.c](#).

11.36.2.4 trusted_out

`trusted_info_t security_cmd_sync_t::trusted_out`

Definicja w linii 96 pliku [security_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/security_main.c

11.37 Dokumentacja struktury security_cmd_t

Pola danych

- `security_cmd_id_t id`
- `security_cmd_sync_t * sync`
- union {
 - struct {
 - `uint32_t node_id`
 - `uint8_t code_len`
 - `uint8_t code [SECURITY_CODE_MAX]`
 - `bool gateway_slot`
 - add_device**
 - struct {
 - `uint32_t node_id`
 - del_device**
 - struct {
 - `uint8_t index`
 - get_device**
 - struct {
 - `bool enabled`
 - set_bool**
 - struct {
 - `uint32_t value`
 - set_u32**
 - struct {
 - `security_notify_mode_t mode`
 - set_notify**
 - struct {
 - `uint8_t preset_id`
 - set_preset**
 - struct {
 - `radio_main_runtime_cfg_t cfg`
 - set_radio**
 - struct {
 - `int16_t rssi_dbm`
 - `uint8_t len`
 - `uint8_t payload [I2C_MEM_STORE_LOG_PAYLOAD_MAX]`
 - log_msg**
- u**

11.37.1 Opis szczegółowy

Definicja w linii 100 pliku [security_main.c](#).

11.37.2 Dokumentacja pól

11.37.2.1 cfg

```
radio_main_runtime_cfg_t security_cmd_t::cfg
```

Definicja w linii 139 pliku [security_main.c](#).

11.37.2.2 code

```
uint8_t security_cmd_t::code[SECURITY_CODE_MAX]
```

Definicja w linii 110 pliku [security_main.c](#).

11.37.2.3 code_len

```
uint8_t security_cmd_t::code_len
```

Definicja w linii 109 pliku [security_main.c](#).

11.37.2.4 enabled

```
bool security_cmd_t::enabled
```

Definicja w linii 123 pliku [security_main.c](#).

11.37.2.5 gateway_slot

```
bool security_cmd_t::gateway_slot
```

Definicja w linii 111 pliku [security_main.c](#).

11.37.2.6 id

```
security_cmd_id_t security_cmd_t::id
```

Definicja w linii 102 pliku [security_main.c](#).

11.37.2.7 index

```
uint8_t security_cmd_t::index
```

Definicja w linii 119 pliku [security_main.c](#).

11.37.2.8 len

```
uint8_t security_cmd_t::len
```

Definicja w linii 144 pliku [security_main.c](#).

11.37.2.9 mode

```
security_notify_mode_t security_cmd_t::mode
```

Definicja w linii 131 pliku [security_main.c](#).

11.37.2.10 node_id

```
uint32_t security_cmd_t::node_id
```

Definicja w linii 108 pliku [security_main.c](#).

11.37.2.11 payload

```
uint8_t security_cmd_t::payload[I2C_MEM_STORE_LOG_PAYLOAD_MAX]
```

Definicja w linii 145 pliku [security_main.c](#).

11.37.2.12 preset_id

```
uint8_t security_cmd_t::preset_id
```

Definicja w linii 135 pliku [security_main.c](#).

11.37.2.13 rssi_dbm

```
int16_t security_cmd_t::rssi_dbm
```

Definicja w linii 143 pliku [security_main.c](#).

11.37.2.14 sync

```
security_cmd_sync_t* security_cmd_t::sync
```

Definicja w linii 103 pliku [security_main.c](#).

11.37.2.15 value

```
uint32_t security_cmd_t::value
```

Definicja w linii 127 pliku [security_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/security_main.c

11.38 Dokumentacja struktury security_key_wire_t

Pola danych

- uint16_t [magic](#)
- uint8_t [seed](#) [SECURITY_KEY_SEED_BYTES]
- uint16_t [crc](#)

11.38.1 Opis szczegółowy

Definicja w linii 189 pliku [security_main.c](#).

11.38.2 Dokumentacja pól

11.38.2.1 crc

```
uint16_t security_key_wire_t::crc
```

Definicja w linii 193 pliku [security_main.c](#).

11.38.2.2 magic

```
uint16_t security_key_wire_t::magic
```

Definicja w linii 191 pliku [security_main.c](#).

11.38.2.3 seed

```
uint8_t security_key_wire_t::seed[SECURITY_KEY_SEED_BYTES]
```

Definicja w linii 192 pliku [security_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/security_main.c

11.39 Dokumentacja struktury security_radio_store_wire_t

Pola danych

- uint8_t [magic](#)
- uint8_t [version](#)
- uint8_t [packed](#) [11]

11.39.1 Opis szczegółowy

Definicja w linii 170 pliku [security_main.c](#).

11.39.2 Dokumentacja pól

11.39.2.1 magic

```
uint8_t security_radio_store_wire_t::magic
```

Definicja w linii 172 pliku [security_main.c](#).

11.39.2.2 packed

```
uint8_t security_radio_store_wire_t::packed[11]
```

Definicja w linii 174 pliku [security_main.c](#).

11.39.2.3 version

```
uint8_t security_radio_store_wire_t::version
```

Definicja w linii 173 pliku [security_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/security_main.c

11.40 Dokumentacja struktury security_runtime_cfg_t

Persistent and runtime security settings snapshot.

```
#include <Core/App/security_main.h>
```

Pola danych

- bool [coding_enabled](#)
Protected/coded traffic setting.
- bool [fh_enabled](#)
Frequency hopping setting.
- uint32_t [fh_period_ms](#)
Frequency hopping period in milliseconds.
- bool [auto_ping_enabled](#)
Automatic ping setting.
- uint32_t [auto_ping_period_ms](#)
Automatic ping period in milliseconds.
- [radio_main_auto_ping_mode_t](#) [auto_ping_mode](#)
Automatic ping payload mode.
- [security_notify_mode_t](#) [notify_mode](#)
Security notification display mode.
- uint8_t [lora_preset](#)
Selected LoRa preset.
- [radio_main_modulation_t](#) [active_modulation](#)

- Active modulation family.*
 - [radio_lora_cfg_t](#) lora
 - Persisted LoRa profile.*
 - [radio_main_fsk_cfg_t](#) fsk
 - Persisted FSK profile.*
 - [radio_main_ook_cfg_t](#) ook
 - Persisted OOK profile.*
 - bool [radio_profiles_persisted](#)
- true when radio profiles were loaded from store.*

11.40.1 Opis szczegółowy

Persistent and runtime security settings snapshot.

Definicja w linii 48 pliku [security_main.h](#).

11.40.2 Dokumentacja pól

11.40.2.1 active_modulation

[radio_main_modulation_t](#) security_runtime_cfg_t::active_modulation

Active modulation family.

Definicja w linii 58 pliku [security_main.h](#).

11.40.2.2 auto_ping_enabled

bool security_runtime_cfg_t::auto_ping_enabled

Automatic ping setting.

Definicja w linii 53 pliku [security_main.h](#).

11.40.2.3 auto_ping_mode

[radio_main_auto_ping_mode_t](#) security_runtime_cfg_t::auto_ping_mode

Automatic ping payload mode.

Definicja w linii 55 pliku [security_main.h](#).

11.40.2.4 auto_ping_period_ms

uint32_t security_runtime_cfg_t::auto_ping_period_ms

Automatic ping period in milliseconds.

Definicja w linii 54 pliku [security_main.h](#).

11.40.2.5 coding_enabled

bool security_runtime_cfg_t::coding_enabled

Protected/coded traffic setting.

Definicja w linii 50 pliku [security_main.h](#).

11.40.2.6 fh_enabled

bool security_runtime_cfg_t::fh_enabled

Frequency hopping setting.

Definicja w linii 51 pliku [security_main.h](#).

11.40.2.7 fh_period_ms

uint32_t security_runtime_cfg_t::fh_period_ms

Frequency hopping period in milliseconds.

Definicja w linii 52 pliku [security_main.h](#).

11.40.2.8 fsk

`radio_main_fsk_cfg_t` `security_runtime_cfg_t::fsk`

Persisted FSK profile.

Definicja w linii 60 pliku [security_main.h](#).

11.40.2.9 lora

`radio_lora_cfg_t` `security_runtime_cfg_t::lora`

Persisted LoRa profile.

Definicja w linii 59 pliku [security_main.h](#).

11.40.2.10 lora_preset

`uint8_t` `security_runtime_cfg_t::lora_preset`

Selected LoRa preset.

Definicja w linii 57 pliku [security_main.h](#).

11.40.2.11 notify_mode

`security_notify_mode_t` `security_runtime_cfg_t::notify_mode`

Security notification display mode.

Definicja w linii 56 pliku [security_main.h](#).

11.40.2.12 ook

`radio_main_ook_cfg_t` `security_runtime_cfg_t::ook`

Persisted OOK profile.

Definicja w linii 61 pliku [security_main.h](#).

11.40.2.13 radio_profiles_persisted

`bool` `security_runtime_cfg_t::radio_profiles_persisted`

`true` when radio profiles were loaded from store.

Definicja w linii 62 pliku [security_main.h](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/[security_main.h](#)

11.41 Dokumentacja struktury security_settings_wire_t

Pola danych

- `uint16_t` [magic](#)
- `uint8_t` [version](#)
- `uint8_t` [flags](#)
- `uint8_t` [notify_mode](#)
- `uint8_t` [lora_preset](#)
- `uint8_t` [reserved0](#)
- `uint8_t` [reserved1](#)
- `uint16_t` [crc](#)

11.41.1 Opis szczegółowy

Definicja w linii 177 pliku [security_main.c](#).

11.41.2 Dokumentacja pól

11.41.2.1 crc

```
uint16_t security_settings_wire_t::crc
```

Definicja w linii 186 pliku [security_main.c](#).

11.41.2.2 flags

```
uint8_t security_settings_wire_t::flags
```

Definicja w linii 181 pliku [security_main.c](#).

11.41.2.3 lora_preset

```
uint8_t security_settings_wire_t::lora_preset
```

Definicja w linii 183 pliku [security_main.c](#).

11.41.2.4 magic

```
uint16_t security_settings_wire_t::magic
```

Definicja w linii 179 pliku [security_main.c](#).

11.41.2.5 notify_mode

```
uint8_t security_settings_wire_t::notify_mode
```

Definicja w linii 182 pliku [security_main.c](#).

11.41.2.6 reserved0

```
uint8_t security_settings_wire_t::reserved0
```

Definicja w linii 184 pliku [security_main.c](#).

11.41.2.7 reserved1

```
uint8_t security_settings_wire_t::reserved1
```

Definicja w linii 185 pliku [security_main.c](#).

11.41.2.8 version

```
uint8_t security_settings_wire_t::version
```

Definicja w linii 180 pliku [security_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/security_main.c

11.42 Dokumentacja struktury security_store_wire_t

Pola danych

- uint8_t [magic](#)
- uint8_t [version](#)
- uint8_t [flags](#)
- uint8_t [notify_mode](#)
- uint8_t [lora_preset](#)
- uint8_t [modulation_fh](#)
- uint8_t [seed](#) [SECURITY_KEY_SEED_BYTES]

11.42.1 Opis szczegółowy

Definicja w linii 159 pliku [security_main.c](#).

11.42.2 Dokumentacja pól

11.42.2.1 flags

```
uint8_t security_store_wire_t::flags
```

Definicja w linii 163 pliku [security_main.c](#).

11.42.2.2 lora_preset

```
uint8_t security_store_wire_t::lora_preset
```

Definicja w linii 165 pliku [security_main.c](#).

11.42.2.3 magic

```
uint8_t security_store_wire_t::magic
```

Definicja w linii 161 pliku [security_main.c](#).

11.42.2.4 modulation_fh

```
uint8_t security_store_wire_t::modulation_fh
```

Definicja w linii 166 pliku [security_main.c](#).

11.42.2.5 notify_mode

```
uint8_t security_store_wire_t::notify_mode
```

Definicja w linii 164 pliku [security_main.c](#).

11.42.2.6 seed

```
uint8_t security_store_wire_t::seed[SECURITY_KEY_SEED_BYTES]
```

Definicja w linii 167 pliku [security_main.c](#).

11.42.2.7 version

```
uint8_t security_store_wire_t::version
```

Definicja w linii 162 pliku [security_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/security_main.c

11.43 Dokumentacja struktury `security_trusted_entry_t`

Pola danych

- bool [in_use](#)
- bool [is_master](#)
- uint32_t [node_id](#)
- uint8_t [code_len](#)
- uint8_t [code](#) [SECURITY_CODE_MAX]

11.43.1 Opis szczegółowy

Definicja w linii 150 pliku [security_main.c](#).

11.43.2 Dokumentacja pól

11.43.2.1 code

```
uint8_t security_trusted_entry_t::code[SECURITY_CODE_MAX]
```

Definicja w linii 156 pliku [security_main.c](#).

11.43.2.2 code_len

uint8_t security_trusted_entry_t::code_len

Definicja w linii 155 pliku [security_main.c](#).

11.43.2.3 in_use

bool security_trusted_entry_t::in_use

Definicja w linii 152 pliku [security_main.c](#).

11.43.2.4 is_master

bool security_trusted_entry_t::is_master

Definicja w linii 153 pliku [security_main.c](#).

11.43.2.5 node_id

uint32_t security_trusted_entry_t::node_id

Definicja w linii 154 pliku [security_main.c](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/security_main.c

11.44 Dokumentacja struktury st33ktpm2x_cfg_t

Runtime configuration for ST33KTPM2X instance.

```
#include <Core/App/st33ktpm2x_lib/st33ktpm2x.h>
```

Pola danych

- I2C_HandleTypeDef * [hi2c](#)
I2C handle (use I2C3 in this project).
- uint16_t [i2c_addr_7bit](#)
7-bit I2C address of TPM.
- GPIO_TypeDef * [reset_port](#)
Reset GPIO port (PB0 in your setup).
- uint16_t [reset_pin](#)
Reset GPIO pin.
- uint32_t [reset_pulse_ms](#)
Reset low pulse width.
- uint32_t [reset_recovery_ms](#)
Delay after reset release.
- GPIO_TypeDef * [davint_port](#)
Optional TPM_DAVINT# / PIRQ port.
- uint16_t [davint_pin](#)
Optional TPM_DAVINT# / PIRQ pin.
- GPIO_PinState [davint_active_state](#)
Logical active level for DAVINT# (normally low).
- GPIO_TypeDef * [pp_port](#)
Optional Physical Presence button port.
- uint16_t [pp_pin](#)
Optional Physical Presence button pin.
- GPIO_PinState [pp_active_state](#)
Logical active level for PP button.
- uint32_t [io_timeout_ms](#)

I2C transfer timeout.

- `uint32_t locality_timeout_ms`

Timeout for locality acquisition.

- `uint32_t burst_timeout_ms`

Timeout for burst/data availability.

11.44.1 Opis szczegółowy

Runtime configuration for ST33KTPM2X instance.

Definicja w linii 83 pliku `st33ktpm2x.h`.

11.44.2 Dokumentacja pól

11.44.2.1 `burst_timeout_ms`

`uint32_t st33ktpm2x_cfg_t::burst_timeout_ms`

Timeout for burst/data availability.

Definicja w linii 103 pliku `st33ktpm2x.h`.

Odwołania w `st33_wait_burst()`, `st33ktpm2x_default_cfg()` i `st33ktpm2x_transceive()`.

11.44.2.2 `davint_active_state`

`GPIO_PinState st33ktpm2x_cfg_t::davint_active_state`

Logical active level for DAVINT# (normally low).

Definicja w linii 95 pliku `st33ktpm2x.h`.

Odwołania w `st33ktpm2x_davint_is_asserted()` i `st33ktpm2x_default_cfg()`.

11.44.2.3 `davint_pin`

`uint16_t st33ktpm2x_cfg_t::davint_pin`

Optional TPM_DAVINT# / PIRQ pin.

Definicja w linii 94 pliku `st33ktpm2x.h`.

Odwołania w `st33ktpm2x_davint_is_asserted()` i `st33ktpm2x_default_cfg()`.

11.44.2.4 `davint_port`

`GPIO_TypeDef* st33ktpm2x_cfg_t::davint_port`

Optional TPM_DAVINT# / PIRQ port.

Definicja w linii 93 pliku `st33ktpm2x.h`.

Odwołania w `st33ktpm2x_davint_is_asserted()` i `st33ktpm2x_default_cfg()`.

11.44.2.5 `hi2c`

`I2C_HandleTypeDef* st33ktpm2x_cfg_t::hi2c`

I2C handle (use I2C3 in this project).

Definicja w linii 85 pliku `st33ktpm2x.h`.

Odwołania w `st33_i2c_read()`, `st33_i2c_write()`, `st33_trace_tis()`, `st33ktpm2x_default_cfg()` i `st33ktpm2x_init()`.

11.44.2.6 `i2c_addr_7bit`

`uint16_t st33ktpm2x_cfg_t::i2c_addr_7bit`

7-bit I2C address of TPM.

Definicja w linii 86 pliku `st33ktpm2x.h`.

Odwołania w `st33_i2c_read()`, `st33_i2c_write()` i `st33ktpm2x_default_cfg()`.

11.44.2.7 io_timeout_ms

uint32_t st33ktpm2x_cfg_t::io_timeout_ms

I2C transfer timeout.

Definicja w linii 101 pliku [st33ktpm2x.h](#).

Odwolania w [st33_i2c_read\(\)](#), [st33_i2c_write\(\)](#) i [st33ktpm2x_default_cfg\(\)](#).

11.44.2.8 locality_timeout_ms

uint32_t st33ktpm2x_cfg_t::locality_timeout_ms

Timeout for locality acquisition.

Definicja w linii 102 pliku [st33ktpm2x.h](#).

Odwolania w [st33ktpm2x_default_cfg\(\)](#) i [st33ktpm2x_request_locality0\(\)](#).

11.44.2.9 pp_active_state

GPIO_PinState st33ktpm2x_cfg_t::pp_active_state

Logical active level for PP button.

Definicja w linii 99 pliku [st33ktpm2x.h](#).

Odwolania w [st33ktpm2x_default_cfg\(\)](#) i [st33ktpm2x_pp_is_pressed\(\)](#).

11.44.2.10 pp_pin

uint16_t st33ktpm2x_cfg_t::pp_pin

Optional Physical Presence button pin.

Definicja w linii 98 pliku [st33ktpm2x.h](#).

Odwolania w [st33ktpm2x_default_cfg\(\)](#) i [st33ktpm2x_pp_is_pressed\(\)](#).

11.44.2.11 pp_port

GPIO_TypeDef* st33ktpm2x_cfg_t::pp_port

Optional Physical Presence button port.

Definicja w linii 97 pliku [st33ktpm2x.h](#).

Odwolania w [st33ktpm2x_default_cfg\(\)](#) i [st33ktpm2x_pp_is_pressed\(\)](#).

11.44.2.12 reset_pin

uint16_t st33ktpm2x_cfg_t::reset_pin

Reset GPIO pin.

Definicja w linii 89 pliku [st33ktpm2x.h](#).

Odwolania w [st33ktpm2x_default_cfg\(\)](#), [st33ktpm2x_hard_reset\(\)](#) i [st33ktpm2x_init\(\)](#).

11.44.2.13 reset_port

GPIO_TypeDef* st33ktpm2x_cfg_t::reset_port

Reset GPIO port (PB0 in your setup).

Definicja w linii 88 pliku [st33ktpm2x.h](#).

Odwolania w [st33ktpm2x_default_cfg\(\)](#), [st33ktpm2x_hard_reset\(\)](#) i [st33ktpm2x_init\(\)](#).

11.44.2.14 reset_pulse_ms

uint32_t st33ktpm2x_cfg_t::reset_pulse_ms

Reset low pulse width.

Definicja w linii 90 pliku [st33ktpm2x.h](#).

Odwolania w [st33ktpm2x_default_cfg\(\)](#) i [st33ktpm2x_hard_reset\(\)](#).

11.44.2.15 reset_recovery_ms

uint32_t st33ktpm2x_cfg_t::reset_recovery_ms

Delay after reset release.

Definicja w linii 91 pliku [st33ktpm2x.h](#).

Odwołania w [st33ktpm2x_default_cfg\(\)](#) i [st33ktpm2x_hard_reset\(\)](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/st33ktpm2x_lib/st33ktpm2x.h](#)

11.45 Dokumentacja struktury st33ktpm2x_t

Driver context for one TPM device.

```
#include <Core/App/st33ktpm2x_lib/st33ktpm2x.h>
```

Pola danych

- [st33ktpm2x_cfg_t](#) *cfg*
Active runtime configuration.
- bool [initialized](#)
Driver init state.
- bool [locality0_acquired](#)
Locality 0 acquisition state.

11.45.1 Opis szczegółowy

Driver context for one TPM device.

Definicja w linii 109 pliku [st33ktpm2x.h](#).

11.45.2 Dokumentacja pól

11.45.2.1 cfg

```
st33ktpm2x_cfg_t st33ktpm2x_t::cfg
```

Active runtime configuration.

Definicja w linii 111 pliku [st33ktpm2x.h](#).

Odwołania w [st33_i2c_read\(\)](#), [st33_i2c_write\(\)](#), [st33_trace_tis\(\)](#), [st33_wait_burst\(\)](#), [st33ktpm2x_davint_is_asserted\(\)](#), [st33ktpm2x_hard_reset\(\)](#), [st33ktpm2x_init\(\)](#), [st33ktpm2x_pp_is_pressed\(\)](#), [st33ktpm2x_request_locality0\(\)](#) i [st33ktpm2x_transceive\(\)](#).

11.45.2.2 initialized

```
bool st33ktpm2x_t::initialized
```

Driver init state.

Definicja w linii 112 pliku [st33ktpm2x.h](#).

Odwołania w [st33_tpm2_nv_write_auth\(\)](#), [st33ktpm2x_davint_is_asserted\(\)](#), [st33ktpm2x_deinit\(\)](#), [st33ktpm2x_hard_reset\(\)](#), [st33ktpm2x_init\(\)](#), [st33ktpm2x_pp_is_pressed\(\)](#), [st33ktpm2x_pp_wait_pressed\(\)](#), [st33ktpm2x_read_identity\(\)](#), [st33ktpm2x_release_locality0\(\)](#), [st33ktpm2x_request_locality0\(\)](#), [st33ktpm2x_tpm2_nv_define\(\)](#), [st33ktpm2x_tpm2_nv_read\(\)](#) i [st33ktpm2x_transceive\(\)](#).

11.45.2.3 locality0_acquired

```
bool st33ktpm2x_t::locality0_acquired
```

Locality 0 acquisition state.

Definicja w linii 113 pliku [st33ktpm2x.h](#).

Odwołania w [st33_trace_tis\(\)](#), [st33ktpm2x_hard_reset\(\)](#), [st33ktpm2x_release_locality0\(\)](#) i [st33ktpm2x_request_locality0\(\)](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- [Core/App/st33ktpm2x_lib/st33ktpm2x.h](#)

11.46 Dokumentacja struktury `sx1276_bus_t`

Kontekst magistrali SPI dla układu SX1276.

```
#include <Core/App/radio_lib/common/sx1276/radio_sx1276.h>
```

Pola danych

- `SPI_HandleTypeDef * hspi`
Uchwyt SPI.
- `GPIO_TypeDef * nss_port`
Port pinu NSS/CS.
- `uint16_t nss_pin`
Pin NSS/CS.
- `uint32_t spi_timeout_ms`
Timeout transferu SPI.

11.46.1 Opis szczegółowy

Kontekst magistrali SPI dla układu SX1276.

Definicja w linii 17 pliku [radio_sx1276.h](#).

11.46.2 Dokumentacja pól

11.46.2.1 `hspi`

```
SPI_HandleTypeDef* sx1276_bus_t::hspi
```

Uchwyt SPI.

Definicja w linii 19 pliku [radio_sx1276.h](#).

Odwolania w [sx1276_read_burst\(\)](#), [sx1276_read_reg\(\)](#), [sx1276_select\(\)](#), [sx1276_write_burst\(\)](#) i [sx1276_write_reg\(\)](#).

11.46.2.2 `nss_pin`

```
uint16_t sx1276_bus_t::nss_pin
```

Pin NSS/CS.

Definicja w linii 21 pliku [radio_sx1276.h](#).

Odwolania w [sx1276_deselect\(\)](#) i [sx1276_select\(\)](#).

11.46.2.3 `nss_port`

```
GPIO_TypeDef* sx1276_bus_t::nss_port
```

Port pinu NSS/CS.

Definicja w linii 20 pliku [radio_sx1276.h](#).

Odwolania w [sx1276_deselect\(\)](#) i [sx1276_select\(\)](#).

11.46.2.4 `spi_timeout_ms`

```
uint32_t sx1276_bus_t::spi_timeout_ms
```

Timeout transferu SPI.

Definicja w linii 22 pliku [radio_sx1276.h](#).

Odwolania w [sx1276_read_burst\(\)](#), [sx1276_read_reg\(\)](#), [sx1276_write_burst\(\)](#) i [sx1276_write_reg\(\)](#).

Dokumentacja dla tej struktury została wygenerowana z pliku:

- `Core/App/radio_lib/common/sx1276/radio_sx1276.h`

11.47 Dokumentacja struktury `trusted_info_t`

Public descriptor of one trusted pairing entry.

```
#include <Core/App/security_main.h>
```


Pola danych

- bool `in_use`
Slot contains a valid trusted entry.
- bool `is_master`
Entry represents the gateway/master slot.
- uint8_t `slot`
Logical trusted-entry slot index.
- uint32_t `node_id`
Trusted node identifier.
- uint8_t `code_len`
Number of valid bytes in `code`.
- uint8_t `code`[8]
Pairing/service code bytes.

11.47.1 Opis szczegółowy

Public descriptor of one trusted pairing entry.
Definicja w linii 17 pliku [security_main.h](#).

11.47.2 Dokumentacja pól

11.47.2.1 code

```
uint8_t trusted_info_t::code[8]
```

Pairing/service code bytes.
Definicja w linii 24 pliku [security_main.h](#).

11.47.2.2 code_len

```
uint8_t trusted_info_t::code_len
```

Number of valid bytes in `code`.
Definicja w linii 23 pliku [security_main.h](#).

11.47.2.3 in_use

```
bool trusted_info_t::in_use
```

Slot contains a valid trusted entry.
Definicja w linii 19 pliku [security_main.h](#).

11.47.2.4 is_master

```
bool trusted_info_t::is_master
```

Entry represents the gateway/master slot.
Definicja w linii 20 pliku [security_main.h](#).

11.47.2.5 node_id

```
uint32_t trusted_info_t::node_id
```

Trusted node identifier.
Definicja w linii 22 pliku [security_main.h](#).

11.47.2.6 slot

```
uint8_t trusted_info_t::slot
```

Logical trusted-entry slot index.
Definicja w linii 21 pliku [security_main.h](#).
Dokumentacja dla tej struktury została wygenerowana z pliku:

- Core/App/[security_main.h](#)

Rozdział 12

Dokumentacja plików

12.1 app.c

```
00001 /*
00002  * app.c
00003  *
00004  * FreeRTOS-style application orchestrator (App-level "app_freertos").
00005  */
00006
00007 #include "app.h"
00008
00009 #include "bmp280_main.h"
00010 #include "button_main.h"
00011 #include "lcd_main.h"
00012 #include "led_array_main.h"
00013 #include "menu_main.h"
00014 #include "radio_main.h"
00015 #include "security_main.h"
00016 #include "service.h"
00017 #include "tof_main.h"
00018
00019 #include "app_delay.h"
00020 #include "cmsis_os2.h"
00021 #include "FreeRTOS.h"
00022 #include "semphr.h"
00023
00024 #include <stdio.h>
00025
00026 #define APP_I2C_DEFAULT_TIMEOUT_MS 100U
00027 #define APP_I2C_RETRY_COUNT 3U
00028 #define APP_I2C_RETRY_DELAY_MS 1U
00029
00030 static osMutexId_t s_app_i2c_mutex = NULL;
00031 static StaticSemaphore_t s_app_i2c_mutex_cb;
00032
00033 static const osMutexAttr_t s_app_i2c_mutex_attr =
00034 {
00035     .name = "app_i2c_mutex",
00036     .attr_bits = osMutexRecursive | osMutexPrioInherit,
00037     .cb_mem = &s_app_i2c_mutex_cb,
00038     .cb_size = sizeof(s_app_i2c_mutex_cb)
00039 };
00040
00041 static bool app_i2c_ensure_mutex(void);
00042 static uint32_t app_i2c_normalize_timeout(uint32_t timeout_ms);
00043 static bool app_i2c_should_recover(I2C_HandleTypeDef *hi2c, HAL_StatusTypeDef status);
00044 static bool app_i2c_recover_locked(I2C_HandleTypeDef *hi2c);
00045
00046 void app_init(void)
00047 {
00048     printf("APP: FreeRTOS bootstrap\r\n");
00049 }
00050
00051 void app_main(void)
00052 {
00053     /* Fallback path; should not execute in normal RTOS flow. */
00054     app_delay_ms(100U);
00055 }
00056
00057 void app_freertos_init(void)
00058 {
00059     (void)app_i2c_ensure_mutex();
00060
00061     lcd_main_create_task();
```

```

00062     security_main_create_task();
00063     button_main_create_task();
00064     menu_main_create_task();
00065     bmp280_main_create_task();
00066     tof_main_create_task();
00067     radio_main_create_task();
00068     service_uart_create_task();
00069     led_array_main_create_task();
00070 }
00071
00072 bool app_i2c_lock(uint32_t timeout_ms)
00073 {
00074     uint32_t timeout;
00075     osKernelState_t kernel_state;
00076
00077     if (!app_i2c_ensure_mutex())
00078     {
00079         kernel_state = osKernelGetState();
00080         return (kernel_state == osKernelInactive);
00081     }
00082
00083     timeout = (timeout_ms == 0U) ? osWaitForever : timeout_ms;
00084     return (osMutexAcquire(s_app_i2c_mutex, timeout) == osOK);
00085 }
00086
00087 void app_i2c_unlock(void)
00088 {
00089     if (s_app_i2c_mutex != NULL)
00090     {
00091         (void)osMutexRelease(s_app_i2c_mutex);
00092     }
00093 }
00094
00095 HAL_StatusTypeDef app_i2c_master_transmit(I2C_HandleTypeDef *hi2c,
00096                                           uint16_t device_address,
00097                                           const uint8_t *data,
00098                                           uint16_t size,
00099                                           uint32_t timeout_ms)
00100 {
00101     HAL_StatusTypeDef status;
00102     uint32_t attempt;
00103     uint8_t *tx_data;
00104
00105     if ((hi2c == NULL) || ((data == NULL) && (size > 0U)))
00106     {
00107         return HAL_ERROR;
00108     }
00109     if (!app_i2c_lock(0U))
00110     {
00111         return HAL_TIMEOUT;
00112     }
00113
00114     tx_data = (uint8_t *)data;
00115     status = HAL_ERROR;
00116
00117     for (attempt = 0U; attempt < APP_I2C_RETRY_COUNT; attempt++)
00118     {
00119         status = HAL_I2C_Master_Transmit(hi2c,
00120                                           device_address,
00121                                           tx_data,
00122                                           size,
00123                                           app_i2c_normalize_timeout(timeout_ms));
00124
00125         if (status == HAL_OK)
00126         {
00127             break;
00128         }
00129
00130         if (app_i2c_should_recover(hi2c, status))
00131         {
00132             (void)app_i2c_recover_locked(hi2c);
00133         }
00134
00135         if ((attempt + 1U) < APP_I2C_RETRY_COUNT)
00136         {
00137             app_delay_ms(APP_I2C_RETRY_DELAY_MS);
00138         }
00139     }
00140
00141     app_i2c_unlock();
00142     return status;
00143 }
00144
00145 HAL_StatusTypeDef app_i2c_master_receive(I2C_HandleTypeDef *hi2c,
00146                                           uint16_t device_address,
00147                                           uint8_t *data,
00148                                           uint16_t size,
00149                                           uint32_t timeout_ms)

```

```

00149 {
00150     HAL_StatusTypeDef status;
00151     uint32_t attempt;
00152
00153     if ((hi2c == NULL) || ((data == NULL) && (size > 0U)))
00154     {
00155         return HAL_ERROR;
00156     }
00157     if (!app_i2c_lock(0U))
00158     {
00159         return HAL_TIMEOUT;
00160     }
00161
00162     status = HAL_ERROR;
00163
00164     for (attempt = 0U; attempt < APP_I2C_RETRY_COUNT; attempt++)
00165     {
00166         status = HAL_I2C_Master_Receive(hi2c,
00167                                         device_address,
00168                                         data,
00169                                         size,
00170                                         app_i2c_normalize_timeout(timeout_ms));
00171
00172         if (status == HAL_OK)
00173         {
00174             break;
00175         }
00176
00177         if (app_i2c_should_recover(hi2c, status))
00178         {
00179             (void)app_i2c_recover_locked(hi2c);
00180         }
00181
00182         if ((attempt + 1U) < APP_I2C_RETRY_COUNT)
00183         {
00184             app_delay_ms(APP_I2C_RETRY_DELAY_MS);
00185         }
00186     }
00187     app_i2c_unlock();
00188     return status;
00189 }
00190
00191 HAL_StatusTypeDef app_i2c_mem_write(I2C_HandleTypeDef *hi2c,
00192                                     uint16_t device_address,
00193                                     uint16_t mem_address,
00194                                     uint16_t mem_address_size,
00195                                     const uint8_t *data,
00196                                     uint16_t size,
00197                                     uint32_t timeout_ms)
00198 {
00199     HAL_StatusTypeDef status;
00200     uint32_t attempt;
00201     uint8_t *tx_data;
00202
00203     if ((hi2c == NULL) || ((data == NULL) && (size > 0U)))
00204     {
00205         return HAL_ERROR;
00206     }
00207     if (!app_i2c_lock(0U))
00208     {
00209         return HAL_TIMEOUT;
00210     }
00211
00212     tx_data = (uint8_t *)data;
00213     status = HAL_ERROR;
00214
00215     for (attempt = 0U; attempt < APP_I2C_RETRY_COUNT; attempt++)
00216     {
00217         status = HAL_I2C_Mem_Write(hi2c,
00218                                    device_address,
00219                                    mem_address,
00220                                    mem_address_size,
00221                                    tx_data,
00222                                    size,
00223                                    app_i2c_normalize_timeout(timeout_ms));
00224
00225         if (status == HAL_OK)
00226         {
00227             break;
00228         }
00229
00230         if (app_i2c_should_recover(hi2c, status))
00231         {
00232             (void)app_i2c_recover_locked(hi2c);
00233         }
00234
00235         if ((attempt + 1U) < APP_I2C_RETRY_COUNT)
00236         {

```

```

00236         app_delay_ms(APP_I2C_RETRY_DELAY_MS);
00237     }
00238 }
00239
00240 app_i2c_unlock();
00241 return status;
00242 }
00243
00244 HAL_StatusTypeDef app_i2c_mem_read(I2C_HandleTypeDef *hi2c,
00245                                     uint16_t device_address,
00246                                     uint16_t mem_address,
00247                                     uint16_t mem_address_size,
00248                                     uint8_t *data,
00249                                     uint16_t size,
00250                                     uint32_t timeout_ms)
00251 {
00252     HAL_StatusTypeDef status;
00253     uint32_t attempt;
00254
00255     if ((hi2c == NULL) || ((data == NULL) && (size > 0U)))
00256     {
00257         return HAL_ERROR;
00258     }
00259     if (!app_i2c_lock(0U))
00260     {
00261         return HAL_TIMEOUT;
00262     }
00263
00264     status = HAL_ERROR;
00265
00266     for (attempt = 0U; attempt < APP_I2C_RETRY_COUNT; attempt++)
00267     {
00268         status = HAL_I2C_Mem_Read(hi2c,
00269                                     device_address,
00270                                     mem_address,
00271                                     mem_address_size,
00272                                     data,
00273                                     size,
00274                                     app_i2c_normalize_timeout(timeout_ms));
00275
00276         if (status == HAL_OK)
00277         {
00278             break;
00279         }
00280
00281         if (app_i2c_should_recover(hi2c, status))
00282         {
00283             (void)app_i2c_recover_locked(hi2c);
00284         }
00285
00286         if ((attempt + 1U) < APP_I2C_RETRY_COUNT)
00287         {
00288             app_delay_ms(APP_I2C_RETRY_DELAY_MS);
00289         }
00290     }
00291
00292     app_i2c_unlock();
00293     return status;
00294 }
00295
00296 static bool app_i2c_ensure_mutex(void)
00297 {
00298     osKernelState_t kernel_state;
00299
00300     if (s_app_i2c_mutex != NULL)
00301     {
00302         return true;
00303     }
00304
00305     kernel_state = osKernelGetState();
00306     if ((kernel_state == osKernelReady) || (kernel_state == osKernelRunning))
00307     {
00308         s_app_i2c_mutex = osMutexNew(&s_app_i2c_mutex_attr);
00309     }
00310
00311     return (s_app_i2c_mutex != NULL);
00312 }
00313
00315 static uint32_t app_i2c_normalize_timeout(uint32_t timeout_ms)
00316 {
00317     if (timeout_ms == 0U)
00318     {
00319         return APP_I2C_DEFAULT_TIMEOUT_MS;
00320     }
00321
00322     return timeout_ms;
00323 }
00324

```

```

00326 static bool app_i2c_should_recover(I2C_HandleTypeDef *hi2c, HAL_StatusTypeDef status)
00327 {
00328     uint32_t error_flags;
00329
00330     if ((hi2c == NULL) || (status == HAL_OK))
00331     {
00332         return false;
00333     }
00334
00335     if ((status == HAL_BUSY) || (status == HAL_TIMEOUT))
00336     {
00337         return true;
00338     }
00339
00340     error_flags = HAL_I2C_GetError(hi2c);
00341
00342     return ((error_flags & (HAL_I2C_ERROR_BERR |
00343                          HAL_I2C_ERROR_ARLO |
00344                          HAL_I2C_ERROR_OVR |
00345                          HAL_I2C_ERROR_DMA |
00346                          HAL_I2C_ERROR_TIMEOUT |
00347                          HAL_I2C_ERROR_SIZE |
00348                          HAL_I2C_ERROR_DMA_PARAM)) != 0U);
00349 }
00350
00352 static bool app_i2c_recover_locked(I2C_HandleTypeDef *hi2c)
00353 {
00354     if (hi2c == NULL)
00355     {
00356         return false;
00357     }
00358
00359     (void)HAL_I2C_DeInit(hi2c);
00360
00361     if (HAL_I2C_Init(hi2c) != HAL_OK)
00362     {
00363         return false;
00364     }
00365     if (HAL_I2CEx_ConfigAnalogFilter(hi2c, I2C_ANALOGFILTER_ENABLE) != HAL_OK)
00366     {
00367         return false;
00368     }
00369     if (HAL_I2CEx_ConfigDigitalFilter(hi2c, 0U) != HAL_OK)
00370     {
00371         return false;
00372     }
00373
00374     return true;
00375 }

```

12.2 Dokumentacja pliku Core/App/app.h

Main application entry points and shared I2C bus helpers.

```

#include <stdbool.h>
#include <stdint.h>
#include "stm32u5xx_hal.h"

```

Funkcje

- void **app_init** (void)
Initialize application-level peripherals and modules.
- void **app_main** (void)
Run the bare-metal application loop before the RTOS scheduler starts.
- void **app_freertos_init** (void)
Create application RTOS tasks and synchronization primitives.
- bool **app_i2c_lock** (uint32_t timeout_ms)
Lock the shared application I2C bus.
- void **app_i2c_unlock** (void)
Release the shared application I2C bus.
- HAL_StatusTypeDef **app_i2c_master_transmit** (I2C_HandleTypeDef *hi2c, uint16_t device_address, const uint8_t *data, uint16_t size, uint32_t timeout_ms)

Thread-safe wrapper for HAL_I2C_Master_Transmit.

- HAL_StatusTypeDef [app_i2c_master_receive](#) (I2C_HandleTypeDef *hi2c, uint16_t device_address, uint8_t *data, uint16_t size, uint32_t timeout_ms)

Thread-safe wrapper for HAL_I2C_Master_Receive.

- HAL_StatusTypeDef [app_i2c_mem_write](#) (I2C_HandleTypeDef *hi2c, uint16_t device_address, uint16_t mem_address, uint16_t mem_address_size, const uint8_t *data, uint16_t size, uint32_t timeout_ms)

Thread-safe wrapper for HAL_I2C_Mem_Write.

- HAL_StatusTypeDef [app_i2c_mem_read](#) (I2C_HandleTypeDef *hi2c, uint16_t device_address, uint16_t mem_address, uint16_t mem_address_size, uint8_t *data, uint16_t size, uint32_t timeout_ms)

Thread-safe wrapper for HAL_I2C_Mem_Read.

12.2.1 Opis szczegółowy

Main application entry points and shared I2C bus helpers.

Definicja w pliku [app.h](#).

12.2.2 Dokumentacja funkcji

12.2.2.1 app_freertos_init()

```
void app_freertos_init (
    void )
```

Create application RTOS tasks and synchronization primitives.

Definicja w linii 57 pliku [app.c](#).

Odwołuje się do [bmp280_main_create_task\(\)](#), [button_main_create_task\(\)](#), [lcd_main_create_task\(\)](#), [led_array_main_create_task\(\)](#), [menu_main_create_task\(\)](#), [radio_main_create_task\(\)](#), [security_main_create_task\(\)](#), [service_uart_create_task\(\)](#) i [tof_main_create_task\(\)](#).

12.2.2.2 app_i2c_lock()

```
bool app_i2c_lock (
    uint32_t timeout_ms)
```

Lock the shared application I2C bus.

Parametry

<i>timeout_ms</i>	Timeout passed to the RTOS mutex wait.
-------------------	--

Zwraca

`true` when the bus lock was acquired.

Definicja w linii 72 pliku [app.c](#).

12.2.2.3 app_i2c_master_receive()

```
HAL_StatusTypeDef app_i2c_master_receive (
    I2C_HandleTypeDef * hi2c,
    uint16_t device_address,
    uint8_t * data,
    uint16_t size,
    uint32_t timeout_ms)
```

Thread-safe wrapper for HAL_I2C_Master_Receive.

Parametry

<i>hi2c</i>	I2C peripheral handle.
<i>device_address</i>	8-bit HAL device address.
<i>data</i>	Receive buffer.
<i>size</i>	Number of bytes to receive.
<i>timeout_ms</i>	Transfer timeout in milliseconds.

Zwraca

HAL transfer status.

Definicja w linii 144 pliku [app.c](#).

Odwołuje się do [app_i2c_unlock\(\)](#).

12.2.2.4 app_i2c_master_transmit()

```
HAL_StatusTypeDef app_i2c_master_transmit (
    I2C_HandleTypeDef * hi2c,
    uint16_t device_address,
    const uint8_t * data,
    uint16_t size,
    uint32_t timeout_ms)
```

Thread-safe wrapper for HAL_I2C_Master_Transmit.

Parametry

<i>hi2c</i>	I2C peripheral handle.
<i>device_address</i>	8-bit HAL device address.
<i>data</i>	Data buffer to transmit.
<i>size</i>	Number of bytes to transmit.
<i>timeout_ms</i>	Transfer timeout in milliseconds.

Zwraca

HAL transfer status.

Definicja w linii 95 pliku [app.c](#).

Odwołuje się do [app_i2c_unlock\(\)](#).

12.2.2.5 app_i2c_mem_read()

```
HAL_StatusTypeDef app_i2c_mem_read (
    I2C_HandleTypeDef * hi2c,
    uint16_t device_address,
    uint16_t mem_address,
    uint16_t mem_address_size,
    uint8_t * data,
    uint16_t size,
    uint32_t timeout_ms)
```

Thread-safe wrapper for HAL_I2C_Mem_Read.

Parametry

<i>hi2c</i>	I2C peripheral handle.
-------------	------------------------

<i>device_address</i>	8-bit HAL device address.
<i>mem_address</i>	Memory/register address.
<i>mem_address_size</i>	HAL memory address size selector.
<i>data</i>	Receive buffer.
<i>size</i>	Number of bytes to read.
<i>timeout_ms</i>	Transfer timeout in milliseconds.

Zwraca

HAL transfer status.

Definicja w linii 244 pliku [app.c](#).

Odwołuje się do [app_i2c_unlock\(\)](#).

12.2.2.6 app_i2c_mem_write()

```
HAL_StatusTypeDef app_i2c_mem_write (
    I2C_HandleTypeDef * hi2c,
    uint16_t device_address,
    uint16_t mem_address,
    uint16_t mem_address_size,
    const uint8_t * data,
    uint16_t size,
    uint32_t timeout_ms)
```

Thread-safe wrapper for HAL_I2C_Mem_Write.

Parametry

<i>hi2c</i>	I2C peripheral handle.
<i>device_address</i>	8-bit HAL device address.
<i>mem_address</i>	Memory/register address.
<i>mem_address_size</i>	HAL memory address size selector.
<i>data</i>	Data buffer to write.
<i>size</i>	Number of bytes to write.
<i>timeout_ms</i>	Transfer timeout in milliseconds.

Zwraca

HAL transfer status.

Definicja w linii 191 pliku [app.c](#).

Odwołuje się do [app_i2c_unlock\(\)](#).

12.2.2.7 app_i2c_unlock()

```
void app_i2c_unlock (
    void )
```

Release the shared application I2C bus.

Definicja w linii 87 pliku [app.c](#).

Odwołania w [app_i2c_master_receive\(\)](#), [app_i2c_master_transmit\(\)](#), [app_i2c_mem_read\(\)](#), [app_i2c_mem_write\(\)](#), [lcd_backlight\(\)](#), [lcd_clear\(\)](#), [lcd_set_cursor\(\)](#), [lcd_write_line\(\)](#) i [lcd_write_string\(\)](#).

12.2.2.8 app_init()

```
void app_init (
    void )
```

Initialize application-level peripherals and modules.

Definicja w linii 46 pliku [app.c](#).

12.2.2.9 app_main()

```
void app_main (
    void )
```

Run the bare-metal application loop before the RTOS scheduler starts.

Definicja w linii 51 pliku [app.c](#).

12.3 app.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_APP_H_
00007 #define APP_APP_H_
00008
00009 #include <stdbool.h>
00010 #include <stdint.h>
00011 #include "stm32u5xx_hal.h"
00012
00016 void
00017 app_init( void );
00018
00022 void
00023 app_main( void );
00024
00028 void
00029 app_freertos_init( void );
00030
00036 bool
00037 app_i2c_lock( uint32_t timeout_ms );
00038
00042 void
00043 app_i2c_unlock( void );
00044
00054 HAL_StatusTypeDef
00055 app_i2c_master_transmit( I2C_HandleTypeDef *hi2c,
00056                          uint16_t device_address,
00057                          const uint8_t *data,
00058                          uint16_t size,
00059                          uint32_t timeout_ms );
00060
00070 HAL_StatusTypeDef
00071 app_i2c_master_receive( I2C_HandleTypeDef *hi2c,
00072                         uint16_t device_address,
00073                         uint8_t *data,
00074                         uint16_t size,
00075                         uint32_t timeout_ms );
00076
00088 HAL_StatusTypeDef
00089 app_i2c_mem_write( I2C_HandleTypeDef *hi2c,
00090                   uint16_t device_address,
00091                   uint16_t mem_address,
00092                   uint16_t mem_address_size,
00093                   const uint8_t *data,
00094                   uint16_t size,
00095                   uint32_t timeout_ms );
00096
00108 HAL_StatusTypeDef
00109 app_i2c_mem_read( I2C_HandleTypeDef *hi2c,
00110                  uint16_t device_address,
00111                  uint16_t mem_address,
00112                  uint16_t mem_address_size,
00113                  uint8_t *data,
00114                  uint16_t size,
00115                  uint32_t timeout_ms );
00116
00117 #endif /* APP_APP_H_ */
```

12.4 bmp280.c

```

00001
00002
00029 #include "bmp280.h"
00030 #include "../app.h"
00031
00035 #define BMP280_REG_TEMP_XLSB 0xFC /* bits: 7-4 */
00036 #define BMP280_REG_TEMP_LSB 0xFB
00037 #define BMP280_REG_TEMP_MSB 0xFA
00038 #define BMP280_REG_TEMP (BMP280_REG_TEMP_MSB)
00039 #define BMP280_REG_PRESS_XLSB 0xF9 /* bits: 7-4 */
00040 #define BMP280_REG_PRESS_LSB 0xF8
00041 #define BMP280_REG_PRESS_MSB 0xF7
00042 #define BMP280_REG_PRESSURE (BMP280_REG_PRESS_MSB)
00043 #define BMP280_REG_CONFIG 0xF5 /* bits: 7-5 t_sb; 4-2 filter; 0 spi3w_en */
00044 #define BMP280_REG_CTRL 0xF4 /* bits: 7-5 osrs_t; 4-2 osrs_p; 1-0 mode */
00045 #define BMP280_REG_STATUS 0xF3 /* bits: 3 measuring; 0 im_update */
00046 #define BMP280_REG_CTRL_HUM 0xF2 /* bits: 2-0 osrs_h; */
00047 #define BMP280_REG_RESET 0xE0
00048 #define BMP280_REG_ID 0xD0
00049 #define BMP280_REG_CALIB 0x88
00050 #define BMP280_REG_HUM_CALIB 0x88
00051
00052 #define BMP280_RESET_VALUE 0xB6
00053
00054
00055
00056 void bmp280_init_default_params(bmp280_params_t *params) {
00057     params->mode = BMP280_MODE_NORMAL;
00058     params->filter = BMP280_FILTER_OFF;
00059     params->oversampling_pressure = BMP280_STANDARD;
00060     params->oversampling_temperature = BMP280_STANDARD;
00061     params->oversampling_humidity = BMP280_STANDARD;
00062     params->standby = BMP280_STANDBY_250;
00063 }
00064
00066 static bool read_register16(BMP280_HandleTypedef *dev, uint8_t addr, uint16_t *value) {
00067     uint16_t tx_buff;
00068     uint8_t rx_buff[2];
00069     tx_buff = (dev->addr << 1);
00070
00071     if (app_i2c_mem_read(dev->i2c, tx_buff, addr, I2C_MEMADD_SIZE_8BIT, rx_buff, 2, 5000)
00072         == HAL_OK) {
00073         *value = (uint16_t) ((rx_buff[1] << 8) | rx_buff[0]);
00074         return true;
00075     } else
00076         return false;
00077
00078 }
00079
00081 static inline int read_data(BMP280_HandleTypedef *dev, uint8_t addr, uint8_t *value,
00082     uint8_t len) {
00083     uint16_t tx_buff;
00084     tx_buff = (dev->addr << 1);
00085     if (app_i2c_mem_read(dev->i2c, tx_buff, addr, I2C_MEMADD_SIZE_8BIT, value, len, 5000) == HAL_OK)
00086         return 0;
00087     else
00088         return 1;
00089
00090 }
00091
00093 static bool read_calibration_data(BMP280_HandleTypedef *dev) {
00094
00095     if (read_register16(dev, 0x88, &dev->dig_T1)
00096         && read_register16(dev, 0x8a, (uint16_t *) &dev->dig_T2)
00097         && read_register16(dev, 0x8c, (uint16_t *) &dev->dig_T3)
00098         && read_register16(dev, 0x8e, &dev->dig_P1)
00099         && read_register16(dev, 0x90, (uint16_t *) &dev->dig_P2)
00100         && read_register16(dev, 0x92, (uint16_t *) &dev->dig_P3)
00101         && read_register16(dev, 0x94, (uint16_t *) &dev->dig_P4)
00102         && read_register16(dev, 0x96, (uint16_t *) &dev->dig_P5)
00103         && read_register16(dev, 0x98, (uint16_t *) &dev->dig_P6)
00104         && read_register16(dev, 0x9a, (uint16_t *) &dev->dig_P7)
00105         && read_register16(dev, 0x9c, (uint16_t *) &dev->dig_P8)
00106         && read_register16(dev, 0x9e,
00107             (uint16_t *) &dev->dig_P9)) {
00108
00109         return true;
00110     }
00111
00112     return false;
00113 }
00114
00116 static bool read_hum_calibration_data(BMP280_HandleTypedef *dev) {
00117     uint16_t h4, h5;

```

```

00118
00119     if (!read_data(dev, 0xa1, &dev->dig_H1, 1)
00120         && read_register16(dev, 0xe1, (uint16_t *) &dev->dig_H2)
00121         && !read_data(dev, 0xe3, &dev->dig_H3, 1)
00122         && read_register16(dev, 0xe4, &h4)
00123         && read_register16(dev, 0xe5, &h5)
00124         && !read_data(dev, 0xe7, (uint8_t *) &dev->dig_H6, 1)) {
00125     dev->dig_H4 = (h4 & 0x00ff) << 4 | (h4 & 0x0f00) >> 8;
00126     dev->dig_H5 = h5 >> 4;
00127
00128     return true;
00129 }
00130
00131 return false;
00132 }
00133
00134 static int write_register8(BMP280_HandleTypedef *dev, uint8_t addr, uint8_t value) {
00135     uint16_t tx_buff;
00136
00137     tx_buff = (dev->addr << 1);
00138
00139     if (app_i2c_mem_write(dev->i2c, tx_buff, addr, I2C_MEMADD_SIZE_8BIT, &value, 1, 10000) == HAL_OK)
00140         return false;
00141     else
00142         return true;
00143 }
00144
00145 bool bmp280_init(BMP280_HandleTypedef *dev, bmp280_params_t *params) {
00146     if (dev->addr != BMP280_I2C_ADDRESS_0
00147         && dev->addr != BMP280_I2C_ADDRESS_1) {
00148         return false;
00149     }
00150
00151     if (read_data(dev, BMP280_REG_ID, &dev->id, 1)) {
00152         return false;
00153     }
00154
00155     if (dev->id != BMP280_CHIP_ID && dev->id != BME280_CHIP_ID) {
00156         return false;
00157     }
00158
00159     // Soft reset.
00160     if (write_register8(dev, BMP280_REG_RESET, BMP280_RESET_VALUE)) {
00161         return false;
00162     }
00163
00164     // Wait until finished copying over the NVP data.
00165     while (1) {
00166         uint8_t status;
00167         if (!read_data(dev, BMP280_REG_STATUS, &status, 1)
00168             && (status & 1) == 0)
00169             break;
00170     }
00171
00172     if (!read_calibration_data(dev)) {
00173         return false;
00174     }
00175
00176     if (dev->id == BME280_CHIP_ID && !read_hum_calibration_data(dev)) {
00177         return false;
00178     }
00179
00180     uint8_t config = (params->standby << 5) | (params->filter << 2);
00181     if (write_register8(dev, BMP280_REG_CONFIG, config)) {
00182         return false;
00183     }
00184
00185     if (params->mode == BMP280_MODE_FORCED) {
00186         params->mode = BMP280_MODE_SLEEP; // initial mode for forced is sleep
00187     }
00188
00189     uint8_t ctrl = (params->oversampling_temperature << 5)
00190         | (params->oversampling_pressure << 2) | (params->mode);
00191
00192     if (dev->id == BME280_CHIP_ID) {
00193         // Write ctrl hum reg first, only active after write to BMP280_REG_CTRL.
00194         uint8_t ctrl_hum = params->oversampling_humidity;
00195         if (write_register8(dev, BMP280_REG_CTRL_HUM, ctrl_hum)) {
00196             return false;
00197         }
00198     }
00199
00200     if (write_register8(dev, BMP280_REG_CTRL, ctrl)) {
00201         return false;
00202     }
00203
00204     return true;
00205 }

```

```

00206     }
00207
00208     return true;
00209 }
00210
00211 bool bmp280_force_measurement(BMP280_HandleTypeDef *dev) {
00212     uint8_t ctrl;
00213     if (read_data(dev, BMP280_REG_CTRL, &ctrl, 1))
00214         return false;
00215     ctrl &= ~0b11; // clear two lower bits
00216     ctrl |= BMP280_MODE_FORCED;
00217     if (write_register8(dev, BMP280_REG_CTRL, ctrl)) {
00218         return false;
00219     }
00220     return true;
00221 }
00222
00223 bool bmp280_is_measuring(BMP280_HandleTypeDef *dev) {
00224     uint8_t status;
00225     if (read_data(dev, BMP280_REG_STATUS, &status, 1))
00226         return false;
00227     if (status & (1 << 3)) {
00228         return true;
00229     }
00230     return false;
00231 }
00232
00233 static inline int32_t compensate_temperature(BMP280_HandleTypeDef *dev, int32_t adc_temp,
00234     int32_t *fine_temp) {
00235     int32_t var1, var2;
00236
00237     var1 = (((adc_temp >> 3) - ((int32_t) dev->dig_T1 << 1)))
00238         * (int32_t) dev->dig_T2 >> 11;
00239     var2 = (((((adc_temp >> 4) - (int32_t) dev->dig_T1)
00240         * ((adc_temp >> 4) - (int32_t) dev->dig_T1)) >> 12)
00241         * (int32_t) dev->dig_T3 >> 14;
00242
00243     *fine_temp = var1 + var2;
00244     return (*fine_temp * 5 + 128) >> 8;
00245 }
00246
00247 static inline uint32_t compensate_pressure(BMP280_HandleTypeDef *dev, int32_t adc_press,
00248     int32_t fine_temp) {
00249     int64_t var1, var2, p;
00250
00251     var1 = (int64_t) fine_temp - 128000;
00252     var2 = var1 * var1 * (int64_t) dev->dig_P6;
00253     var2 = var2 + ((var1 * (int64_t) dev->dig_P5 << 17);
00254     var2 = var2 + (((int64_t) dev->dig_P4 << 35);
00255     var1 = ((var1 * var1 * (int64_t) dev->dig_P3 >> 8)
00256         + ((var1 * (int64_t) dev->dig_P2 << 12);
00257     var1 = (((int64_t) 1 << 47) + var1) * ((int64_t) dev->dig_P1 >> 33;
00258
00259     if (var1 == 0) {
00260         return 0; // avoid exception caused by division by zero
00261     }
00262
00263     p = 1048576 - adc_press;
00264     p = (((p << 31) - var2) * 3125) / var1;
00265     var1 = ((int64_t) dev->dig_P9 * (p >> 13) * (p >> 13)) >> 25;
00266     var2 = ((int64_t) dev->dig_P8 * p) >> 19;
00267
00268     p = ((p + var1 + var2) >> 8) + ((int64_t) dev->dig_P7 << 4);
00269     return p;
00270 }
00271
00272 static inline uint32_t compensate_humidity(BMP280_HandleTypeDef *dev, int32_t adc_hum,
00273     int32_t v_x1_u32r;
00274
00275     v_x1_u32r = fine_temp - (int32_t) 76800;
00276     v_x1_u32r = (((adc_hum << 14) - ((int32_t) dev->dig_H4 << 20)
00277         - ((int32_t) dev->dig_H5 * v_x1_u32r)) + (int32_t) 16384) >> 15;
00278     * ((((((v_x1_u32r * (int32_t) dev->dig_H6 >> 10)
00279         * (((v_x1_u32r * (int32_t) dev->dig_H3 >> 11)
00280         + (int32_t) 32768)) >> 10) + (int32_t) 2097152)
00281         * (int32_t) dev->dig_H2 + 8192) >> 14);
00282     v_x1_u32r = v_x1_u32r
00283         - (((((v_x1_u32r >> 15) * (v_x1_u32r >> 15)) >> 7)
00284         * (int32_t) dev->dig_H1) >> 4);
00285     v_x1_u32r = v_x1_u32r < 0 ? 0 : v_x1_u32r;
00286     v_x1_u32r = v_x1_u32r > 419430400 ? 419430400 : v_x1_u32r;
00287     return v_x1_u32r >> 12;
00288 }
00289
00290 bool bmp280_read_fixed(BMP280_HandleTypeDef *dev, int32_t *temperature, uint32_t *pressure,
00291     uint32_t *humidity) {

```

```

00308     int32_t adc_pressure;
00309     int32_t adc_temp;
00310     uint8_t data[8];
00311
00312     // Only the BME280 supports reading the humidity.
00313     if (dev->id != BME280_CHIP_ID) {
00314         if (humidity)
00315             *humidity = 0;
00316         humidity = NULL;
00317     }
00318
00319     // Need to read in one sequence to ensure they match.
00320     size_t size = humidity ? 8 : 6;
00321     if (read_data(dev, 0xf7, data, size)) {
00322         return false;
00323     }
00324
00325     adc_pressure = data[0] << 12 | data[1] << 4 | data[2] >> 4;
00326     adc_temp = data[3] << 12 | data[4] << 4 | data[5] >> 4;
00327
00328     int32_t fine_temp;
00329     *temperature = compensate_temperature(dev, adc_temp, &fine_temp);
00330     *pressure = compensate_pressure(dev, adc_pressure, fine_temp);
00331
00332     if (humidity) {
00333         int32_t adc_humidity = data[6] << 8 | data[7];
00334         *humidity = compensate_humidity(dev, adc_humidity, fine_temp);
00335     }
00336
00337     return true;
00338 }
00339
00340 bool bmp280_read_float(BMP280_HandleTypedef *dev, float *temperature, float *pressure,
00341     float *humidity) {
00342     int32_t fixed_temperature;
00343     uint32_t fixed_pressure;
00344     uint32_t fixed_humidity;
00345     if (bmp280_read_fixed(dev, &fixed_temperature, &fixed_pressure,
00346         humidity ? &fixed_humidity : NULL)) {
00347         *temperature = (float) fixed_temperature / 100;
00348         *pressure = (float) fixed_pressure / 256;
00349         if (humidity)
00350             *humidity = (float) fixed_humidity / 1024;
00351         return true;
00352     }
00353
00354     return false;
00355 }

```

12.5 Dokumentacja pliku Core/App/bmp280_lib/bmp280.h

Low-level BMP280/BME280 driver API.

```

#include "stm32u5xx_hal.h"
#include <stdint.h>
#include <stdbool.h>

```

Struktury danych

- struct [bmp280_params_t](#)
Configuration parameters for BMP280 module.
- struct [BMP280_HandleTypedef](#)
Driver handle with calibration constants and active configuration.

Definicje

- #define [BMP280_I2C_ADDRESS_0](#) 0x76
Ciastkolog.pl (<https://github.com/ciastkolog>).
- #define [BMP280_I2C_ADDRESS_1](#) 0x77
I2C address when SDO pin is high.
- #define [BMP280_CHIP_ID](#) 0x58
Expected BMP280 chip identifier.

- `#define BME280_CHIP_ID 0x60`
Expected BME280 chip identifier.

Wyliczenia

- enum `BMP280_Mode` { `BMP280_MODE_SLEEP` = 0 , `BMP280_MODE_FORCED` = 1 , `BMP280_MODE_↵`
`_NORMAL` = 3 }
Mode of BMP280 module operation.
- enum `BMP280_Filter` {
`BMP280_FILTER_OFF` = 0 , `BMP280_FILTER_2` = 1 , `BMP280_FILTER_4` = 2 , `BMP280_FILTER_8` = 3 ,
`BMP280_FILTER_16` = 4 }
- enum `BMP280_Oversampling` {
`BMP280_SKIPPED` = 0 , `BMP280_ULTRA_LOW_POWER` = 1 , `BMP280_LOW_POWER` = 2 , `BMP280_↵`
`STANDARD` = 3 ,
`BMP280_HIGH_RES` = 4 , `BMP280_ULTRA_HIGH_RES` = 5 }
Pressure oversampling settings.
- enum `BMP280_StandbyTime` {
`BMP280_STANDBY_05` = 0 , `BMP280_STANDBY_62` = 1 , `BMP280_STANDBY_125` = 2 , `BMP280_↵`
`STANDBY_250` = 3 ,
`BMP280_STANDBY_500` = 4 , `BMP280_STANDBY_1000` = 5 , `BMP280_STANDBY_2000` = 6 , `BMP280_↵`
`_STANDBY_4000` = 7 }
Stand by time between measurements in normal mode.

Funkcje

- void `bmp280_init_default_params` (`bmp280_params_t` *params)
Initialize default BMP280 parameters.
- bool `bmp280_init` (`BMP280_HandleTypeDef` *dev, `bmp280_params_t` *params)
Initialize BMP280/BME280 module.
- bool `bmp280_force_measurement` (`BMP280_HandleTypeDef` *dev)
Start one measurement in forced mode.
- bool `bmp280_is_measuring` (`BMP280_HandleTypeDef` *dev)
Check whether BMP280 is busy measuring temperature or pressure.
- bool `bmp280_read_fixed` (`BMP280_HandleTypeDef` *dev, `int32_t` *temperature, `uint32_t` *pressure, `uint32_↵`
`_t` *humidity)
Read compensated fixed-point temperature, pressure and optional humidity.
- bool `bmp280_read_float` (`BMP280_HandleTypeDef` *dev, `float` *temperature, `float` *pressure, `float` *humidity)
Read compensated floating-point temperature, pressure and optional humidity.

12.5.1 Opis szczegółowy

Low-level BMP280/BME280 driver API.
Definicja w pliku [bmp280.h](#).

12.5.2 Dokumentacja definicji

12.5.2.1 BME280_CHIP_ID

`#define BME280_CHIP_ID 0x60`
Expected BME280 chip identifier.
Definicja w linii 33 pliku [bmp280.h](#).
Odwołania w [bmp280_init\(\)](#) i [bmp280_read_fixed\(\)](#).

12.5.2.2 BMP280_CHIP_ID

```
#define BMP280_CHIP_ID 0x58
```

Expected BMP280 chip identifier.

Definicja w linii 31 pliku [bmp280.h](#).

Odwołania w [bmp280_init\(\)](#).

12.5.2.3 BMP280_I2C_ADDRESS_0

```
#define BMP280_I2C_ADDRESS_0 0x76
```

Ciastkolog.pl (<https://github.com/ciastkolog>).

The MIT License (MIT) Copyright (c) 2016 sheinz (<https://github.com/sheinz>) BMP280 or BME280 address is 0x77 if SDO pin is high, and is 0x76 if SDO pin is low.

I2C address when SDO pin is low.

Definicja w linii 26 pliku [bmp280.h](#).

Odwołania w [bmp280_api_init\(\)](#) i [bmp280_init\(\)](#).

12.5.2.4 BMP280_I2C_ADDRESS_1

```
#define BMP280_I2C_ADDRESS_1 0x77
```

I2C address when SDO pin is high.

Definicja w linii 28 pliku [bmp280.h](#).

Odwołania w [bmp280_api_init\(\)](#) i [bmp280_init\(\)](#).

12.5.3 Dokumentacja typów wyliczanych

12.5.3.1 BMP280_Filter

```
enum BMP280_Filter
```

Wartości wyliczeń

BMP280_FILTER_OFF	IIR filter disabled.
BMP280_FILTER_2	IIR filter coefficient 2.
BMP280_FILTER_4	IIR filter coefficient 4.
BMP280_FILTER_8	IIR filter coefficient 8.
BMP280_FILTER_16	IIR filter coefficient 16.

Definicja w linii 46 pliku [bmp280.h](#).

12.5.3.2 BMP280_Mode

```
enum BMP280_Mode
```

Mode of BMP280 module operation.

Forced - Measurement is initiated by user. Normal - Continues measurement.

Definicja w linii 40 pliku [bmp280.h](#).

12.5.3.3 BMP280_Oversampling

```
enum BMP280_Oversampling
```

Pressure oversampling settings.

Definicja w linii 57 pliku [bmp280.h](#).

12.5.3.4 BMP280_StandbyTime

enum [BMP280_StandbyTime](#)

Stand by time between measurements in normal mode.

Definicja w linii 69 pliku [bmp280.h](#).

12.5.4 Dokumentacja funkcji

12.5.4.1 bmp280_force_measurement()

```
bool bmp280_force_measurement (
    BMP280_HandleTypeDef * dev)
```

Start one measurement in forced mode.

The module remains in forced mode after this call. Do not call this method in normal mode.

Parametry

<i>dev</i>	Driver handle.
------------	----------------

Zwraca

`true` when the measurement trigger was written.

Definicja w linii 211 pliku [bmp280.c](#).

12.5.4.2 bmp280_init()

```
bool bmp280_init (
    BMP280_HandleTypeDef * dev,
    bmp280_params_t * params)
```

Initialize BMP280/BME280 module.

Probes for the device, soft resets the device, reads the calibration constants, and configures the device using the supplied parameters.

The I2C address is assumed to have been initialized in the dev, and may be either `BMP280_I2C_ADDRESS_0` or `BMP280_I2C_ADDRESS_1`. If the I2C address is unknown then try initializing each in turn.

This may be called again to soft reset the device and initialize it again.

Parametry

<i>dev</i>	Driver handle with I2C handle and address already set.
<i>params</i>	Sensor configuration parameters.

Zwraca

`true` on successful initialization.

Definicja w linii 146 pliku [bmp280.c](#).

Odwołuje się do `BMP280_HandleTypeDef::addr`, `BME280_CHIP_ID`, `BMP280_CHIP_ID`, `BMP280_I2C_ADDRESS_0`, `BMP280_I2C_ADDRESS_1`, `bmp280_params_t::filter`, `BMP280_HandleTypeDef::id`, `bmp280_params_t::mode`, `bmp280_params_t::oversampling_humidity`, `bmp280_params_t::oversampling_pressure`, `bmp280_params_t::oversampling_temperature` i `bmp280_params_t::standby`.

12.5.4.3 bmp280_init_default_params()

```
void bmp280_init_default_params (
    bmp280_params_t * params)
```

Initialize default BMP280 parameters.

Default configuration:

- mode: NORMAL
- filter: OFF
- oversampling: x4
- standby time: 250ms

Parametry

<i>params</i>	[out] Parameter structure to initialize.
---------------	--

Definicja w linii 56 pliku [bmp280.c](#).

Odwołuje się do [BMP280_FILTER_OFF](#), [bmp280_params_t::filter](#), [bmp280_params_t::mode](#), [bmp280_params_t::oversampling_humidity](#), [bmp280_params_t::oversampling_pressure](#), [bmp280_params_t::oversampling_temperature](#) i [bmp280_params_t::standby](#).

12.5.4.4 bmp280_is_measuring()

```
bool bmp280_is_measuring (
    BMP280_HandleTypeDef * dev)
```

Check whether BMP280 is busy measuring temperature or pressure.

Parametry

<i>dev</i>	Driver handle.
------------	----------------

Zwraca

`true` when the sensor is currently measuring.

Definicja w linii 223 pliku [bmp280.c](#).

12.5.4.5 bmp280_read_fixed()

```
bool bmp280_read_fixed (
    BMP280_HandleTypeDef * dev,
    int32_t * temperature,
    uint32_t * pressure,
    uint32_t * humidity)
```

Read compensated fixed-point temperature, pressure and optional humidity.

Temperature in degrees Celsius times 100.

Pressure in Pascals in fixed point 24 bit integer 8 bit fraction format.

Humidity is optional and only read for the BME280, in percent relative humidity as a fixed point 22 bit integer and 10 bit fraction format.

Parametry

<i>dev</i>	Driver handle.
<i>temperature</i>	[out] Temperature in centi-degrees Celsius.
<i>pressure</i>	[out] Pressure in Pa using 24.8 fixed-point format.
<i>humidity</i>	[out] Optional humidity in BME280 fixed-point format.

Zwraca

`true` when data was read and compensated.

Definicja w linii 306 pliku [bmp280.c](#).

Odwołuje się do [BME280_CHIP_ID](#) i [BMP280_HandleTypeDef::id](#).

12.5.4.6 bmp280_read_float()

```
bool bmp280_read_float (
    BMP280_HandleTypeDef * dev,
    float * temperature,
    float * pressure,
    float * humidity)
```

Read compensated floating-point temperature, pressure and optional humidity.

Temperature in degrees Celsius. Pressure in Pascals. Humidity is optional and only read for the BME280, in percent relative humidity.

Parametry

<i>dev</i>	Driver handle.
<i>temperature</i>	[out] Temperature in degrees Celsius.
<i>pressure</i>	[out] Pressure in Pascals.
<i>humidity</i>	[out] Optional relative humidity in percent.

Zwraca

`true` when data was read and compensated.

Definicja w linii 340 pliku [bmp280.c](#).

12.6 bmp280.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00013 #ifndef __BMP280_H__
00014 #define __BMP280_H__
00015
00016 #include "stm32u5xx_hal.h"
00017 #include <stdint.h>
00018 #include <stdbool.h>
00019
00024
00026 #define BMP280_I2C_ADDRESS_0  0x76
00028 #define BMP280_I2C_ADDRESS_1  0x77
00029
00031 #define BMP280_CHIP_ID  0x58
00033 #define BME280_CHIP_ID  0x60
00034
00040 typedef enum {
00041     BMP280_MODE_SLEEP = 0,
00042     BMP280_MODE_FORCED = 1,
00043     BMP280_MODE_NORMAL = 3
00044 } BMP280_Mode;
00045
00046 typedef enum {
00047     BMP280_FILTER_OFF = 0,
00048     BMP280_FILTER_2 = 1,
00049     BMP280_FILTER_4 = 2,
00050     BMP280_FILTER_8 = 3,
00051     BMP280_FILTER_16 = 4
00052 } BMP280_Filter;
00053
00057 typedef enum {
00058     BMP280_SKIPPED = 0,          /* no measurement */
00059     BMP280_ULTRA_LOW_POWER = 1, /* oversampling x1 */
00060     BMP280_LOW_POWER = 2,       /* oversampling x2 */
00061     BMP280_STANDARD = 3,        /* oversampling x4 */
00062     BMP280_HIGH_RES = 4,        /* oversampling x8 */
00063     BMP280_ULTRA_HIGH_RES = 5   /* oversampling x16 */
00064 } BMP280_Oversampling;
00065
00069 typedef enum {
00070     BMP280_STANDBY_05 = 0,      /* stand by time 0.5ms */
00071     BMP280_STANDBY_62 = 1,      /* stand by time 62.5ms */
00072     BMP280_STANDBY_125 = 2,     /* stand by time 125ms */
00073     BMP280_STANDBY_250 = 3,     /* stand by time 250ms */
00074 }
```

```

00074     BMP280_STANDBY_500 = 4,      /* stand by time 500ms */
00075     BMP280_STANDBY_1000 = 5,     /* stand by time 1s */
00076     BMP280_STANDBY_2000 = 6,     /* stand by time 2s BMP280, 10ms BME280 */
00077     BMP280_STANDBY_4000 = 7,     /* stand by time 4s BMP280, 20ms BME280 */
00078 } BMP280_StandbyTime;
00079
00084 typedef struct {
00085     BMP280_Mode mode;
00086     BMP280_Filter filter;
00087     BMP280_Oversampling oversampling_pressure;
00088     BMP280_Oversampling oversampling_temperature;
00089     BMP280_Oversampling oversampling_humidity;
00090     BMP280_StandbyTime standby;
00091 } bmp280_params_t;
00092
00093
00097 typedef struct {
00098     uint16_t dig_T1;
00099     int16_t dig_T2;
00100     int16_t dig_T3;
00101     uint16_t dig_P1;
00102     int16_t dig_P2;
00103     int16_t dig_P3;
00104     int16_t dig_P4;
00105     int16_t dig_P5;
00106     int16_t dig_P6;
00107     int16_t dig_P7;
00108     int16_t dig_P8;
00109     int16_t dig_P9;
00110
00111     /* Humidity compensation for BME280 */
00112     uint8_t dig_H1;
00113     int16_t dig_H2;
00114     uint8_t dig_H3;
00115     int16_t dig_H4;
00116     int16_t dig_H5;
00117     int8_t dig_H6;
00118
00119     uint16_t addr;
00120
00121     I2C_HandleTypeDef* i2c;
00122
00123     bmp280_params_t params;
00124
00125     uint8_t id;
00126
00127 } BMP280_HandleTypeDef;
00128
00140 void bmp280_init_default_params(bmp280_params_t *params);
00141
00159 bool bmp280_init(BMP280_HandleTypeDef *dev, bmp280_params_t *params);
00160
00170 bool bmp280_force_measurement(BMP280_HandleTypeDef *dev);
00171
00177 bool bmp280_is_measuring(BMP280_HandleTypeDef *dev);
00178
00195 bool bmp280_read_fixed(BMP280_HandleTypeDef *dev, int32_t *temperature,
00196                        uint32_t *pressure, uint32_t *humidity);
00197
00212 bool bmp280_read_float(BMP280_HandleTypeDef *dev, float *temperature,
00213                        float *pressure, float *humidity);
00214
00215
00216 #endif // __BMP280_H__

```

12.7 bmp280_api.c

```

00001 #include "bmp280_api.h"
00002
00003 #include "../app.h"
00004
00005 #include <string.h>
00006
00007 #define BMP280_API_REG_CTRL          0xF4U
00008 #define BMP280_API_REG_STATUS        0xF3U
00009 #define BMP280_API_STATUS_MEASURING_MASK (1U << 3)
00010 #define BMP280_API_I2C_TIMEOUT_MS    100U
00011 #define BMP280_API_DEFAULT_MEAS_TIMEOUT_MS 100U
00012
00013 /*
00014  * Single-instance wrapper built on top of the user's existing bmp280.c/.h driver.
00015  * This keeps the public API small and uses one public data struct only.
00016  */
00017 static BMP280_HandleTypeDef s_bmp280_dev;

```

```
00018 static bmp280_api_data_t s_last_data;
00019 static bool s_initialized = false;
00020
00022 static bool bmp280_api_write_reg(uint8_t reg, uint8_t value)
00023 {
00024     uint16_t device_addr;
00025
00026     if (!s_initialized || (s_bmp280_dev.i2c == NULL))
00027     {
00028         return false;
00029     }
00030
00031     device_addr = (uint16_t)(s_bmp280_dev.addr << 1);
00032
00033     return (app_i2c_mem_write(s_bmp280_dev.i2c,
00034                             device_addr,
00035                             reg,
00036                             I2C_MEMADD_SIZE_8BIT,
00037                             &value,
00038                             1,
00039                             BMP280_API_I2C_TIMEOUT_MS) == HAL_OK);
00040 }
00041
00043 static bool bmp280_api_read_reg(uint8_t reg, uint8_t *value)
00044 {
00045     uint16_t device_addr;
00046
00047     if (!s_initialized || (s_bmp280_dev.i2c == NULL) || (value == NULL))
00048     {
00049         return false;
00050     }
00051
00052     device_addr = (uint16_t)(s_bmp280_dev.addr << 1);
00053
00054     return (app_i2c_mem_read(s_bmp280_dev.i2c,
00055                             device_addr,
00056                             reg,
00057                             I2C_MEMADD_SIZE_8BIT,
00058                             value,
00059                             1,
00060                             BMP280_API_I2C_TIMEOUT_MS) == HAL_OK);
00061 }
00062
00064 static bool bmp280_api_wait_for_measurement_done(uint32_t timeout_ms)
00065 {
00066     uint32_t start_tick;
00067     uint8_t status;
00068
00069     if (!s_initialized)
00070     {
00071         return false;
00072     }
00073
00074     if (timeout_ms == 0U)
00075     {
00076         timeout_ms = BMP280_API_DEFAULT_MEAS_TIMEOUT_MS;
00077     }
00078
00079     start_tick = HAL_GetTick();
00080
00081     while (1)
00082     {
00083         if (!bmp280_api_read_reg(BMP280_API_REG_STATUS, &status))
00084         {
00085             return false;
00086         }
00087
00088         if ((status & BMP280_API_STATUS_MEASURING_MASK) == 0U)
00089         {
00090             return true;
00091         }
00092
00093         if ((HAL_GetTick() - start_tick) >= timeout_ms)
00094         {
00095             return false;
00096         }
00097     }
00098 }
00099
00101 static bool bmp280_api_measure_internal(float *temperature_c, float *pressure_pa, uint32_t timeout_ms)
00102 {
00103     bool success = false;
00104     float local_temperature = 0.0f;
00105     float local_pressure = 0.0f;
00106
00107     if (!s_initialized)
00108     {
```

```

00109         return false;
00110     }
00111
00112     if (!bmp280_force_measurement(&s_bmp280_dev))
00113     {
00114         (void)bmp280_api_sleep();
00115         return false;
00116     }
00117
00118     if (!bmp280_api_wait_for_measurement_done(timeout_ms))
00119     {
00120         (void)bmp280_api_sleep();
00121         return false;
00122     }
00123
00124     if (!bmp280_read_float(&s_bmp280_dev, &local_temperature, &local_pressure, NULL))
00125     {
00126         (void)bmp280_api_sleep();
00127         return false;
00128     }
00129
00130     s_last_data.temperature_c = local_temperature;
00131     s_last_data.pressure_pa = local_pressure;
00132     s_last_data.pressure_hpa = local_pressure / 100.0f;
00133     s_last_data.timestamp_ms = HAL_GetTick();
00134     s_last_data.valid = true;
00135
00136     if (temperature_c != NULL)
00137     {
00138         *temperature_c = local_temperature;
00139     }
00140
00141     if (pressure_pa != NULL)
00142     {
00143         *pressure_pa = local_pressure;
00144     }
00145
00146     success = true;
00147
00148     (void)bmp280_api_sleep();
00149     return success;
00150 }
00151
00152 bool bmp280_api_init(I2C_HandleTypeDef *hi2c, uint8_t i2c_address)
00153 {
00154     bmp280_params_t params;
00155
00156     if ((hi2c == NULL) ||
00157         ((i2c_address != BMP280_I2C_ADDRESS_0) && (i2c_address != BMP280_I2C_ADDRESS_1)))
00158     {
00159         return false;
00160     }
00161
00162     s_initialized = false;
00163     memset(&s_bmp280_dev, 0, sizeof(s_bmp280_dev));
00164     memset(&s_last_data, 0, sizeof(s_last_data));
00165
00166     s_bmp280_dev.i2c = hi2c;
00167     s_bmp280_dev.addr = i2c_address;
00168
00169     bmp280_init_default_params(&params);
00170
00171     /*
00172      * Ultra-precision single-shot profile:
00173      * - forced mode for explicit one-shot measurements
00174      * - x1 pressure oversampling
00175      * - x1 temperature oversampling
00176      * - min IIR filter
00177      * - shortest standby (not relevant in forced mode, but kept deterministic)
00178      */
00179     params.mode = BMP280_MODE_FORCED;
00180     params.filter = BMP280_FILTER_OFF;
00181     params.oversampling_pressure = BMP280_ULTRA_LOW_POWER;
00182     params.oversampling_temperature = BMP280_ULTRA_LOW_POWER;
00183     params.standby = BMP280_STANDBY_05;
00184
00185     if (!bmp280_init(&s_bmp280_dev, &params))
00186     {
00187         memset(&s_bmp280_dev, 0, sizeof(s_bmp280_dev));
00188         memset(&s_last_data, 0, sizeof(s_last_data));
00189         s_initialized = false;
00190         return false;
00191     }
00192
00193     /* bmp280_init() changes FORCED to SLEEP internally. Cache the final configuration. */
00194     s_bmp280_dev.params = params;
00195     s_initialized = true;

```

```
00196
00197     bmp280_api_data_t dummy;
00198     bmp280_api_measure_all(&dummy, 50);
00199
00200     /* Explicitly request lowest-power state before returning. */
00201     (void)bmp280_api_sleep();
00202     return true;
00203 }
00204
00205 bool bmp280_api_measure_temperature(float *temperature_c, uint32_t timeout_ms)
00206 {
00207     bool result;
00208
00209     if (temperature_c == NULL)
00210     {
00211         (void)bmp280_api_sleep();
00212         return false;
00213     }
00214
00215     result = bmp280_api_measure_internal(temperature_c, NULL, timeout_ms);
00216     (void)bmp280_api_sleep();
00217     return result;
00218 }
00219
00220 bool bmp280_api_measure_pressure(float *pressure_pa, uint32_t timeout_ms)
00221 {
00222     bool result;
00223
00224     if (pressure_pa == NULL)
00225     {
00226         (void)bmp280_api_sleep();
00227         return false;
00228     }
00229
00230     result = bmp280_api_measure_internal(NULL, pressure_pa, timeout_ms);
00231     (void)bmp280_api_sleep();
00232     return result;
00233 }
00234
00235 bool bmp280_api_measure_all(bmp280_api_data_t *data, uint32_t timeout_ms)
00236 {
00237     bool result;
00238
00239     if (data == NULL)
00240     {
00241         (void)bmp280_api_sleep();
00242         return false;
00243     }
00244
00245     result = bmp280_api_measure_internal(NULL, NULL, timeout_ms);
00246
00247     if (result)
00248     {
00249         *data = s_last_data;
00250     }
00251
00252     (void)bmp280_api_sleep();
00253     return result;
00254 }
00255
00256 bool bmp280_api_sleep(void)
00257 {
00258     uint8_t ctrl;
00259
00260     if (!s_initialized)
00261     {
00262         return false;
00263     }
00264
00265     ctrl = (uint8_t)((uint8_t)s_bmp280_dev.params.oversampling_temperature < 5) |
00266           ((uint8_t)s_bmp280_dev.params.oversampling_pressure < 2) |
00267           (uint8_t)BMP280_MODE_SLEEP);
00268
00269     return bmp280_api_write_reg(BMP280_API_REG_CTRL, ctrl);
00270 }
00271
00272 bool bmp280_api_get_last_data(bmp280_api_data_t *data)
00273 {
00274     bool result = false;
00275
00276     if ((data != NULL) && s_last_data.valid)
00277     {
00278         *data = s_last_data;
00279         result = true;
00280     }
00281
00282     (void)bmp280_api_sleep();
```



```
00283     return result;
00284 }
```

12.8 Dokumentacja pliku Core/App/bmp280_lib/bmp280_api.h

Single-instance BMP280 measurement wrapper used by the application.

```
#include "bmp280.h"
#include <stdbool.h>
#include <stdint.h>
```

Struktury danych

- struct [bmp280_api_data_t](#)
Cached measurement container.

Funkcje

- bool [bmp280_api_init](#) (I2C_HandleTypeDef *hi2c, uint8_t i2c_address)
Initialize BMP280 for single-shot measurements.
- bool [bmp280_api_measure_temperature](#) (float *temperature_c, uint32_t timeout_ms)
Measure temperature using forced mode.
- bool [bmp280_api_measure_pressure](#) (float *pressure_pa, uint32_t timeout_ms)
Measure pressure using forced mode.
- bool [bmp280_api_measure_all](#) (bmp280_api_data_t *data, uint32_t timeout_ms)
Measure temperature and pressure using forced mode.
- bool [bmp280_api_sleep](#) (void)
Force the sensor into sleep mode.
- bool [bmp280_api_get_last_data](#) (bmp280_api_data_t *data)
Read back the last cached measurement data.

12.8.1 Opis szczegółowy

Single-instance BMP280 measurement wrapper used by the application.
Definicja w pliku [bmp280_api.h](#).

12.8.2 Dokumentacja funkcji

12.8.2.1 bmp280_api_get_last_data()

```
bool bmp280_api_get_last_data (
    bmp280_api_data_t * data)
```

Read back the last cached measurement data.

Parametry

<i>data</i>	[out] Cached measurement container.
-------------	-------------------------------------

Zwraca

`true` when valid cached data was copied.

Definicja w linii 272 pliku [bmp280_api.c](#).
Odwołuje się do [bmp280_api_sleep\(\)](#).

12.8.2.2 bmp280_api_init()

```
bool bmp280_api_init (
    I2C_HandleTypeDef * hi2c,
    uint8_t i2c_address)
```

Initialize BMP280 for single-shot measurements.
The device is left in sleep mode after initialization.

Parametry

<i>hi2c</i>	I2C peripheral handle.
<i>i2c_address</i>	BMP280_I2C_ADDRESS_0 (0x76) or BMP280_I2C_ADDRESS_1 (0x77).

Zwraca

`true` when the sensor was initialized.

Definicja w linii 152 pliku [bmp280_api.c](#).

Odwołuje się do [bmp280_api_sleep\(\)](#), [BMP280_FILTER_OFF](#), [BMP280_I2C_ADDRESS_0](#), [BMP280_I2C_ADDRESS_1](#), [bmp280_params_t::filter](#), [bmp280_params_t::mode](#), [bmp280_params_t::oversampling_pressure](#), [bmp280_params_t::oversampling_t](#) i [bmp280_params_t::standby](#).

12.8.2.3 bmp280_api_measure_all()

```
bool bmp280_api_measure_all (
    bmp280_api_data_t * data,
    uint32_t timeout_ms)
```

Measure temperature and pressure using forced mode.

Parametry

<i>data</i>	[out] Measurement container.
<i>timeout_ms</i>	Measurement timeout checked with <code>HAL_GetTick()</code> .

Zwraca

`true` when a fresh measurement was read.

Definicja w linii 235 pliku [bmp280_api.c](#).

Odwołuje się do [bmp280_api_sleep\(\)](#).

12.8.2.4 bmp280_api_measure_pressure()

```
bool bmp280_api_measure_pressure (
    float * pressure_pa,
    uint32_t timeout_ms)
```

Measure pressure using forced mode.

Parametry

<i>pressure_pa</i>	[out] Pressure in Pascals.
<i>timeout_ms</i>	Measurement timeout checked with <code>HAL_GetTick()</code> .

Zwraca

`true` when a fresh measurement was read.

Definicja w linii 220 pliku [bmp280_api.c](#).

Odwołuje się do [bmp280_api_sleep\(\)](#).

12.8.2.5 bmp280_api_measure_temperature()

```
bool bmp280_api_measure_temperature (
    float * temperature_c,
    uint32_t timeout_ms)
```

Measure temperature using forced mode.

Parametry

<i>temperature_c</i>	[out] Temperature in degrees Celsius.
<i>timeout_ms</i>	Measurement timeout checked with <code>HAL_GetTick()</code> .

Zwraca

`true` when a fresh measurement was read.

Definicja w linii 205 pliku [bmp280_api.c](#).

Odwołuje się do [bmp280_api_sleep\(\)](#).

12.8.2.6 bmp280_api_sleep()

```
bool bmp280_api_sleep (
    void )
```

Force the sensor into sleep mode.

Zwraca

`true` when the sleep command was written.

Definicja w linii 256 pliku [bmp280_api.c](#).

Odwołania w [bmp280_api_get_last_data\(\)](#), [bmp280_api_init\(\)](#), [bmp280_api_measure_all\(\)](#), [bmp280_api_measure_pressure\(\)](#) i [bmp280_api_measure_temperature\(\)](#).

12.9 bmp280_api.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef BMP280_API_H
00007 #define BMP280_API_H
00008
00009 #include "bmp280.h"
00010 #include <stdbool.h>
00011 #include <stdint.h>
00012
00013 #ifdef __cplusplus
00014 extern "C" {
00015 #endif
00016
00020 typedef struct
00021 {
00022     float temperature_c;
00023     float pressure_pa;
00024     float pressure_hpa;
00025     uint32_t timestamp_ms;
00026     bool valid;
00027 } bmp280_api_data_t;
00028
```

```

00038 bool bmp280_api_init(I2C_HandleTypeDef *hi2c, uint8_t i2c_address);
00039
00046 bool bmp280_api_measure_temperature(float *temperature_c, uint32_t timeout_ms);
00047
00054 bool bmp280_api_measure_pressure(float *pressure_pa, uint32_t timeout_ms);
00055
00062 bool bmp280_api_measure_all(bmp280_api_data_t *data, uint32_t timeout_ms);
00063
00068 bool bmp280_api_sleep(void);
00069
00075 bool bmp280_api_get_last_data(bmp280_api_data_t *data);
00076
00077 #ifdef __cplusplus
00078 }
00079 #endif
00080
00081 #endif /* BMP280_API_H */

```

12.10 bmp280_main.c

```

00001 #include "bmp280_main.h"
00002
00003 #include "pager_config.h"
00004
00005 #include <stdio.h>
00006
00007 #if PAGER_CONFIG_BMP280_MOUNTED
00008
00009 #include "app.h"
00010 #include "cmsis_os2.h"
00011 #include "FreeRTOS.h"
00012 #include "semphr.h"
00013 #include "task.h"
00014 #include "main.h"
00015
00016 #include <string.h>
00017
00018 extern I2C_HandleTypeDef hi2c1;
00019
00020 #define BMP280_TASK_STACK_SIZE 3072U
00021 #define BMP280_TASK_STACK_WORDS (BMP280_TASK_STACK_SIZE / sizeof(StackType_t))
00022
00023 static osThreadId_t s_bmp280_task = NULL;
00024 static osMutexId_t s_bmp280_data_mutex = NULL;
00025 static bmp280_api_data_t s_bmp280_last;
00026 static bool s_bmp280_has_data = false;
00027 static StaticTask_t s_bmp280_task_cb;
00028 static StackType_t s_bmp280_task_stack[BMP280_TASK_STACK_WORDS];
00029 static StaticSemaphore_t s_bmp280_data_mutex_cb;
00030
00031 static void bmp280_main_task_fn(void *argument);
00032
00033 static const osMutexAttr_t s_bmp280_data_mutex_attr =
00034 {
00035     .name = "bmp280_data_mutex",
00036     .cb_mem = &s_bmp280_data_mutex_cb,
00037     .cb_size = sizeof(s_bmp280_data_mutex_cb)
00038 };
00039
00040 static const osThreadAttr_t s_bmp280_task_attr =
00041 {
00042     .name = "bmp280_task",
00043     .priority = (osPriority_t)osPriorityBelowNormal,
00044     .stack_mem = s_bmp280_task_stack,
00045     .stack_size = sizeof(s_bmp280_task_stack),
00046     .cb_mem = &s_bmp280_task_cb,
00047     .cb_size = sizeof(s_bmp280_task_cb)
00048 };
00049
00050 #else
00051
00052 static bool s_bmp280_not_mounted_logged = false;
00053
00054 #endif
00055
00056 void bmp280_main_create_task(void)
00057 {
00058     #if PAGER_CONFIG_BMP280_MOUNTED
00059         if (s_bmp280_data_mutex == NULL)
00060         {
00061             s_bmp280_data_mutex = osMutexNew(&s_bmp280_data_mutex_attr);
00062         }
00063
00064         if (s_bmp280_task == NULL)

```

```

00065     {
00066         s_bmp280_task = osThreadNew(bmp280_main_task_fn, NULL, &s_bmp280_task_attr);
00067     }
00068 #else
00069     if (!s_bmp280_not_mounted_logged)
00070     {
00071         printf("BMP280: not mounted, task disabled\r\n");
00072         s_bmp280_not_mounted_logged = true;
00073     }
00074 #endif
00075 }
00076
00077 bool bmp280_main_get_last(bmp280_api_data_t *out_data)
00078 {
00079     #if PAGER_CONFIG_BMP280_MOUNTED
00080         bool has_data;
00081
00082         if ((out_data == NULL) || (s_bmp280_data_mutex == NULL))
00083         {
00084             return false;
00085         }
00086
00087         if (osMutexAcquire(s_bmp280_data_mutex, 50U) != osOK)
00088         {
00089             return false;
00090         }
00091
00092         has_data = s_bmp280_has_data;
00093         if (has_data)
00094         {
00095             *out_data = s_bmp280_last;
00096         }
00097
00098         (void)osMutexRelease(s_bmp280_data_mutex);
00099         return has_data;
00100     #else
00101         (void)out_data;
00102         return false;
00103     #endif
00104 }
00105
00106 #if PAGER_CONFIG_BMP280_MOUNTED
00107 static void bmp280_main_task_fn(void *argument)
00108 {
00109     bmp280_api_data_t sample;
00110     bool sensor_ready;
00111
00112     (void)argument;
00113
00114     if (app_i2c_lock(0U))
00115     {
00116         sensor_ready = bmp280_api_init(&hi2c1, BMP280_I2C_ADDRESS_1);
00117         app_i2c_unlock();
00118     }
00119     else
00120     {
00121         sensor_ready = false;
00122     }
00123
00124     if (!sensor_ready)
00125     {
00126         printf("BMP280 task: init failed\r\n");
00127     }
00128     else
00129     {
00130         printf("BMP280 task: init OK\r\n");
00131     }
00132
00133     memset(&sample, 0, sizeof(sample));
00134
00135     for (;;)
00136     {
00137         if (sensor_ready && app_i2c_lock(0U))
00138         {
00139             if (bmp280_api_measure_all(&sample, 80U))
00140             {
00141                 if (osMutexAcquire(s_bmp280_data_mutex, 50U) == osOK)
00142                 {
00143                     s_bmp280_last = sample;
00144                     s_bmp280_has_data = true;
00145                     (void)osMutexRelease(s_bmp280_data_mutex);
00146                 }
00147             }
00148             app_i2c_unlock();
00149         }
00150     }
00151
00152     osDelay(1000U);

```

```
00153     }
00154 }
00155 #endif
```

12.11 Dokumentacja pliku Core/App/bmp280_main.h

RTOS task facade for periodic BMP280 measurements.

```
#include "bmp280_lib/bmp280_api.h"
#include <stdbool.h>
```

Funkcje

- void [bmp280_main_create_task](#) (void)
Create the BMP280 sampling task when the sensor is enabled for the board.
- bool [bmp280_main_get_last](#) (bmp280_api_data_t *out_data)
Read the last cached BMP280 measurement from the task layer.

12.11.1 Opis szczegółowy

RTOS task facade for periodic BMP280 measurements.

Definicja w pliku [bmp280_main.h](#).

12.11.2 Dokumentacja funkcji

12.11.2.1 bmp280_main_create_task()

```
void bmp280_main_create_task (
    void )
```

Create the BMP280 sampling task when the sensor is enabled for the board.

Definicja w linii 56 pliku [bmp280_main.c](#).

Odwołania w [app_freertos_init\(\)](#).

12.11.2.2 bmp280_main_get_last()

```
bool bmp280_main_get_last (
    bmp280_api_data_t * out_data)
```

Read the last cached BMP280 measurement from the task layer.

Parametry

<i>out_data</i>	[out] Destination for the cached measurement.
-----------------	---

Zwraca

`true` when valid cached data was copied.

Definicja w linii 77 pliku [bmp280_main.c](#).

12.12 bmp280_main.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_BMP280_MAIN_H_
00007 #define APP_BMP280_MAIN_H_
00008
00009 #include "bmp280_lib/bmp280_api.h"
00010
```

```

00011 #include <stdbool.h>
00012
00016 void bmp280_main_create_task(void);
00017
00023 bool bmp280_main_get_last(bmp280_api_data_t *out_data);
00024
00025 #endif /* APP_BMP280_MAIN_H_ */

```

12.13 button_main.c

```

00001 #include "button_main.h"
00002
00003 #include "cmsis_os2.h"
00004 #include "FreeRTOS.h"
00005 #include "main.h"
00006 #include "task.h"
00007
00008 #include <stdio.h>
00009 #include <string.h>
00010
00011 #define BUTTON_TASK_STACK_SIZE          2048U
00012 #define BUTTON_TASK_STACK_WORDS        (BUTTON_TASK_STACK_SIZE / sizeof(StackType_t))
00013 #define BUTTON_POLL_MS                 25U
00014 #define BUTTON_DEBOUNCE_SAMPLES        2U
00015 #define BUTTON_OK_LONG_PRESS_MS        800U
00016 #define BUTTON_EVENT_QUEUE_DEPTH       16U
00017
00018 typedef enum
00019 {
00020     BUTTON_ID_UP = 0,
00021     BUTTON_ID_DOWN,
00022     BUTTON_ID_OK,
00023     BUTTON_ID_COUNT
00024 } button_id_t;
00025
00026 typedef struct
00027 {
00028     GPIO_TypeDef *port;
00029     uint16_t pin;
00030     bool active_low;
00031 } button_hw_t;
00032
00033 typedef struct
00034 {
00035     bool raw_pressed;
00036     bool stable_pressed;
00037     uint8_t stable_count;
00038     uint32_t pressed_since_ms;
00039     bool long_reported;
00040 } button_state_t;
00041
00042 static osThreadId_t s_button_task = NULL;
00043 static osMessageQueueId_t s_button_evt_queue = NULL;
00044 static StaticTask_t s_button_task_cb;
00045 static StackType_t s_button_task_stack[BUTTON_TASK_STACK_WORDS];
00046
00047 static const button_hw_t s_button_hw[BUTTON_ID_COUNT] =
00048 {
00049     [BUTTON_ID_UP] = { DIG_B1_GPIO_Port, DIG_B1_Pin, true },
00050     [BUTTON_ID_DOWN] = { DIG_B2_GPIO_Port, DIG_B2_Pin, true },
00051     [BUTTON_ID_OK] = { DIG_B3_GPIO_Port, DIG_B3_Pin, true }
00052 };
00053
00054 static void button_main_task_fn(void *argument);
00055 static bool button_hw_is_pressed(button_id_t id);
00056 static void button_try_emit(button_event_t evt);
00057
00058 static const osThreadAttr_t s_button_task_attr =
00059 {
00060     .name = "button_task",
00061     .priority = (osPriority_t)osPriorityBelowNormal,
00062     .stack_mem = s_button_task_stack,
00063     .stack_size = sizeof(s_button_task_stack),
00064     .cb_mem = &s_button_task_cb,
00065     .cb_size = sizeof(s_button_task_cb)
00066 };
00067
00068 void button_main_create_task(void)
00069 {
00070     if (s_button_evt_queue == NULL)
00071     {
00072         s_button_evt_queue = osMessageQueueNew(BUTTON_EVENT_QUEUE_DEPTH, sizeof(button_event_t),
00073         NULL);
00074     }
00075     if (s_button_evt_queue == NULL)

```

```

00074     {
00075         printf("BUTTON: queue create failed\r\n");
00076         return;
00077     }
00078 }
00079
00080 if (s_button_task == NULL)
00081 {
00082     s_button_task = osThreadNew(button_main_task_fn, NULL, &s_button_task_attr);
00083     if (s_button_task == NULL)
00084     {
00085         printf("BUTTON: task create failed\r\n");
00086     }
00087 }
00088 }
00089
00090 bool button_main_get_event(button_event_t *evt, uint32_t timeout_ms)
00091 {
00092     if ((evt == NULL) || (s_button_evt_queue == NULL))
00093     {
00094         return false;
00095     }
00096
00097     return (osMessageQueueGet(s_button_evt_queue, evt, NULL, timeout_ms) == osOK);
00098 }
00099
00100 static void button_main_task_fn(void *argument)
00101 {
00102     button_state_t st[BUTTON_ID_COUNT];
00103     uint32_t i;
00104
00105     (void)argument;
00106     memset(st, 0, sizeof(st));
00107
00108     for (i = 0U; i < BUTTON_ID_COUNT; i++)
00109     {
00110         st[i].raw_pressed = button_hw_is_pressed((button_id_t)i);
00111         st[i].stable_pressed = st[i].raw_pressed;
00112         st[i].stable_count = BUTTON_DEBOUNCE_SAMPLES;
00113     }
00114
00115     printf("BUTTON: task ready b1=%u b2=%u b3=%u\r\n",
00116           (unsigned int)HAL_GPIO_ReadPin(DIG_B1_GPIO_Port, DIG_B1_Pin),
00117           (unsigned int)HAL_GPIO_ReadPin(DIG_B2_GPIO_Port, DIG_B2_Pin),
00118           (unsigned int)HAL_GPIO_ReadPin(DIG_B3_GPIO_Port, DIG_B3_Pin));
00119
00120     for (;;)
00121     {
00122         uint32_t now_ms = HAL_GetTick();
00123
00124         for (i = 0U; i < BUTTON_ID_COUNT; i++)
00125         {
00126             bool current_pressed = button_hw_is_pressed((button_id_t)i);
00127             button_state_t *s = &st[i];
00128
00129             if (current_pressed == s->raw_pressed)
00130             {
00131                 if (s->stable_count < BUTTON_DEBOUNCE_SAMPLES)
00132                 {
00133                     s->stable_count++;
00134                 }
00135             }
00136             else
00137             {
00138                 s->raw_pressed = current_pressed;
00139                 s->stable_count = 0U;
00140             }
00141
00142             if ((s->stable_count >= BUTTON_DEBOUNCE_SAMPLES) && (s->stable_pressed != s->raw_pressed))
00143             {
00144                 s->stable_pressed = s->raw_pressed;
00145
00146                 if (s->stable_pressed)
00147                 {
00148                     s->pressed_since_ms = now_ms;
00149                     s->long_reported = false;
00150                 }
00151                 else
00152                 {
00153                     uint32_t press_time = now_ms - s->pressed_since_ms;
00154
00155                     if ((i == BUTTON_ID_OK) && s->long_reported)
00156                     {
00157                         /* Long event was already emitted while holding. */
00158                     }
00159                     else
00160                     {
00161

```



```

00162         switch ((button_id_t)i)
00163         {
00164             case BUTTON_ID_UP:
00165                 button_try_emit(BUTTON_EVENT_UP_SHORT);
00166                 break;
00167             case BUTTON_ID_DOWN:
00168                 button_try_emit(BUTTON_EVENT_DOWN_SHORT);
00169                 break;
00170             case BUTTON_ID_OK:
00171                 if (press_time >= BUTTON_OK_LONG_PRESS_MS)
00172                 {
00173                     button_try_emit(BUTTON_EVENT_OK_LONG);
00174                 }
00175                 else
00176                 {
00177                     button_try_emit(BUTTON_EVENT_OK_SHORT);
00178                 }
00179                 break;
00180             default:
00181                 break;
00182         }
00183     }
00184 }
00185 }
00186 else if ((i == BUTTON_ID_OK) && s->stable_pressed && (!s->long_reported))
00187 {
00188     if ((now_ms - s->pressed_since_ms) >= BUTTON_OK_LONG_PRESS_MS)
00189     {
00190         s->long_reported = true;
00191         button_try_emit(BUTTON_EVENT_OK_LONG);
00192     }
00193 }
00194 }
00195 }
00196 osDelay(BUTTON_POLL_MS);
00197 }
00198 }
00199
00201 static bool button_hw_is_pressed(button_id_t id)
00202 {
00203     GPIO_PinState raw;
00204     bool pressed;
00205
00206     if (id >= BUTTON_ID_COUNT)
00207     {
00208         return false;
00209     }
00210
00211     raw = HAL_GPIO_ReadPin(s_button_hw[id].port, s_button_hw[id].pin);
00212     pressed = (raw == GPIO_PIN_SET);
00213     if (s_button_hw[id].active_low)
00214     {
00215         pressed = !pressed;
00216     }
00217
00218     return pressed;
00219 }
00220
00222 static void button_try_emit(button_event_t evt)
00223 {
00224     if (s_button_evt_queue == NULL)
00225     {
00226         return;
00227     }
00228
00229     if (osMessageQueuePut(s_button_evt_queue, &evt, 0U, 0U) != osOK)
00230     {
00231         button_event_t dropped;
00232         if (osMessageQueueGet(s_button_evt_queue, &dropped, NULL, 0U) == osOK)
00233         {
00234             (void)osMessageQueuePut(s_button_evt_queue, &evt, 0U, 0U);
00235         }
00236     }
00237 }

```

12.14 Dokumentacja pliku Core/App/button_main.h

Polled button task (25 ms) with event queue.

```

#include <stdbool.h>
#include <stdint.h>

```

Wyliczenia

- enum `button_event_t` {
`BUTTON_EVENT_NONE` = 0 , `BUTTON_EVENT_UP_SHORT` , `BUTTON_EVENT_DOWN_SHORT` ,
`BUTTON_EVENT_OK_SHORT` ,
`BUTTON_EVENT_OK_LONG` }

Logical button events generated by the button task.

Funkcje

- void `button_main_create_task` (void)
Create the button polling task and event queue.
- bool `button_main_get_event` (`button_event_t` *evt, uint32_t timeout_ms)
Read one debounced button event from the queue.

12.14.1 Opis szczegółowy

Polled button task (25 ms) with event queue.

Definicja w pliku `button_main.h`.

12.14.2 Dokumentacja typów wyliczanych

12.14.2.1 `button_event_t`

enum `button_event_t`

Logical button events generated by the button task.

Wartości wyliczeń

<code>BUTTON_EVENT_NONE</code>	No button event.
<code>BUTTON_EVENT_UP_SHORT</code>	Short press of the UP button.
<code>BUTTON_EVENT_DOWN_SHORT</code>	Short press of the DOWN button.
<code>BUTTON_EVENT_OK_SHORT</code>	Short press of the OK button.
<code>BUTTON_EVENT_OK_LONG</code>	Long press of the OK button.

Definicja w linii 15 pliku `button_main.h`.

12.14.3 Dokumentacja funkcji

12.14.3.1 `button_main_create_task()`

```
void button_main_create_task (
    void )
```

Create the button polling task and event queue.

Definicja w linii 68 pliku `button_main.c`.

Odwołania w `app_freertos_init()`.

12.14.3.2 `button_main_get_event()`

```
bool button_main_get_event (
    button_event_t * evt,
    uint32_t timeout_ms)
```

Read one debounced button event from the queue.

Parametry

<i>evt</i>	[out] Destination for the event.
<i>timeout_ms</i>	RTOS queue wait timeout in milliseconds.

Zwraca

true when an event was received.

Definicja w linii 90 pliku [button_main.c](#).

12.15 button_main.h

[Idź do dokumentacji tego pliku.](#)

```

00001
00005
00006 #ifndef APP_BUTTON_MAIN_H_
00007 #define APP_BUTTON_MAIN_H_
00008
00009 #include <stdbool.h>
00010 #include <stdint.h>
00011
00015 typedef enum
00016 {
00017     BUTTON_EVENT_NONE = 0,
00018     BUTTON_EVENT_UP_SHORT,
00019     BUTTON_EVENT_DOWN_SHORT,
00020     BUTTON_EVENT_OK_SHORT,
00021     BUTTON_EVENT_OK_LONG
00022 } button_event_t;
00023
00027 void button_main_create_task(void);
00028
00035 bool button_main_get_event(button_event_t *evt, uint32_t timeout_ms);
00036
00037 #endif /* APP_BUTTON_MAIN_H_ */

```

12.16 Dokumentacja pliku Core/App/i2c_mem_store_lib/i2c_mem_store.c

Implementation of external I2C memory storage helper.

```

#include "i2c_mem_store.h"
#include "../app.h"
#include "app_delay.h"
#include <string.h>

```

Struktury danych

- struct [i2c_mem_store_meta_wire_t](#)

Definicje

- #define [I2C_MEM_STORE_META_PRIMARY_ADDR](#) 0x0000UL
- #define [I2C_MEM_STORE_META_MIRROR_ADDR](#) 0x0018UL
- #define [I2C_MEM_STORE_LOG_START_ADDR](#) 0x0030UL
- #define [I2C_MEM_STORE_META_MAGIC](#) 0x4D533331UL
- #define [I2C_MEM_STORE_META_VERSION](#) 0x0001U
- #define [I2C_MEM_STORE_LOG_MAGIC](#) 0xA55AU
- #define [I2C_MEM_STORE_SECRET_MAGIC](#) 0xC0DEU
- #define [I2C_MEM_STORE_LOG_HEADER_SIZE](#) 16U
- #define [I2C_MEM_STORE_LOG_CRC_OFFSET](#) (I2C_MEM_STORE_LOG_RECORD_SIZE - 2U)
- #define [I2C_MEM_STORE_SECRET_HEADER_SIZE](#) 8U
- #define [I2C_MEM_STORE_SECRET_CRC_OFFSET](#) (I2C_MEM_STORE_SECRET_SLOT_SIZE - 2U)
- #define [I2C_MEM_STORE_META_WIRE_SIZE](#) 24U

Funkcje

- static uint16_t [i2c_mem_store_min_u16](#) (uint16_t a, uint16_t b)
Internal helper: [i2c_mem_store_min_u16](#).
- static [i2c_mem_store_status_t](#) [i2c_mem_store_hal_to_status](#) (HAL_StatusTypeDef st)
Internal helper: [i2c_mem_store_hal_to_status](#).
- static uint16_t [i2c_mem_store_crc16_ccitt](#) (const uint8_t *data, uint16_t len)
Calculate CRC-16/CCITT used to protect metadata and stored records.
- static void [i2c_mem_store_le16_write](#) (uint8_t *dst, uint16_t value)
Internal helper: [i2c_mem_store_le16_write](#).
- static void [i2c_mem_store_le32_write](#) (uint8_t *dst, uint32_t value)
Internal helper: [i2c_mem_store_le32_write](#).
- static uint16_t [i2c_mem_store_le16_read](#) (const uint8_t *src)
Internal helper: [i2c_mem_store_le16_read](#).
- static uint32_t [i2c_mem_store_le32_read](#) (const uint8_t *src)
Internal helper: [i2c_mem_store_le32_read](#).
- static void [i2c_mem_store_xtea_encrypt_block](#) (uint32_t v[2], const uint32_t key[4])
Encrypt one 64-bit block with XTEA for the software CTR keystream.
- static void [i2c_mem_store_xtea_ctr_crypt](#) (uint8_t *data, uint16_t len, const uint8_t key_bytes[16], uint32_t nonce)
Apply XTEA-CTR in place to encrypt or decrypt secret-slot payload bytes.
- static uint32_t [i2c_mem_store_log_base_addr](#) (const [i2c_mem_store_t](#) *ctx)
Internal helper: [i2c_mem_store_log_base_addr](#).
- static uint32_t [i2c_mem_store_secret_base_addr](#) (const [i2c_mem_store_t](#) *ctx)
Internal helper: [i2c_mem_store_secret_base_addr](#).
- static uint32_t [i2c_mem_store_log_slot_addr](#) (const [i2c_mem_store_t](#) *ctx, uint16_t slot)
Internal helper: [i2c_mem_store_log_slot_addr](#).
- static uint32_t [i2c_mem_store_secret_slot_addr](#) (const [i2c_mem_store_t](#) *ctx, uint16_t slot)
Internal helper: [i2c_mem_store_secret_slot_addr](#).
- static [i2c_mem_store_status_t](#) [i2c_mem_store_read_raw](#) ([i2c_mem_store_t](#) *ctx, uint32_t addr, uint8_t *dst, uint16_t len)
Internal helper: [i2c_mem_store_read_raw](#).
- static [i2c_mem_store_status_t](#) [i2c_mem_store_write_raw](#) ([i2c_mem_store_t](#) *ctx, uint32_t addr, const uint8_t *src, uint16_t len)
Write raw bytes while respecting EEPROM page boundaries and write-cycle delay.
- static void [i2c_mem_store_meta_encode](#) (const [i2c_mem_store_t](#) *ctx, uint8_t out[I2C_MEM_STORE_↵ META_WIRE_SIZE])
Encode runtime metadata into the fixed little-endian on-memory layout.
- static bool [i2c_mem_store_meta_decode](#) ([i2c_mem_store_t](#) *ctx, const uint8_t in[I2C_MEM_STORE_↵ META_WIRE_SIZE])
Decode and validate persisted metadata from one metadata copy.
- static [i2c_mem_store_status_t](#) [i2c_mem_store_meta_save](#) ([i2c_mem_store_t](#) *ctx)
Internal helper: [i2c_mem_store_meta_save](#).
- static [i2c_mem_store_status_t](#) [i2c_mem_store_meta_load](#) ([i2c_mem_store_t](#) *ctx)
Internal helper: [i2c_mem_store_meta_load](#).
- static bool [i2c_mem_store_layout_valid](#) (const [i2c_mem_store_cfg_t](#) *cfg, uint16_t *out_slot_count, uint16_↵ _t *out_secret_slots)
Internal helper: [i2c_mem_store_layout_valid](#).
- void [i2c_mem_store_default_cfg](#) ([i2c_mem_store_cfg_t](#) *cfg, I2C_HandleTypeDef *hi2c)
Fill configuration with safe defaults for I2C1 memory.
- void [i2c_mem_store_default_cfg_m24c01r](#) ([i2c_mem_store_cfg_t](#) *cfg, I2C_HandleTypeDef *hi2c)
Fill configuration specifically for ST M24C01-R (1 Kbit EEPROM).

- `i2c_mem_store_status_t i2c_mem_store_init (i2c_mem_store_t *ctx, const i2c_mem_store_cfg_t *cfg, bool format_if_needed)`
Initialize library, load metadata and optionally format on first use.
- `i2c_mem_store_status_t i2c_mem_store_format (i2c_mem_store_t *ctx)`
Format metadata and reset log/secret allocators.
- `i2c_mem_store_status_t i2c_mem_store_append_message (i2c_mem_store_t *ctx, int16_t rssi_dbm, const uint8_t *payload, uint8_t payload_len)`
Append one received message to persistent ring log.
- `i2c_mem_store_status_t i2c_mem_store_read_message (i2c_mem_store_t *ctx, uint16_t index_from_latest, i2c_mem_store_log_record_t *out_record)`
Read one message from log (0 = latest, 1 = previous, ...).
- `i2c_mem_store_status_t i2c_mem_store_get_log_count (i2c_mem_store_t *ctx, uint16_t *out_count)`
Get number of valid records currently available in ring log.
- `i2c_mem_store_status_t i2c_mem_store_secret_write (i2c_mem_store_t *ctx, uint16_t slot_id, const uint8_t *data, uint8_t data_len)`
Store encrypted arbitrary payload in secret partition slot.
- `i2c_mem_store_status_t i2c_mem_store_secret_read (i2c_mem_store_t *ctx, uint16_t slot_id, uint8_t *data_out, uint8_t data_capacity, uint8_t *data_len)`
Read and decrypt payload from secret partition slot.
- `i2c_mem_store_status_t i2c_mem_store_secret_erase (i2c_mem_store_t *ctx, uint16_t slot_id)`
Erase one secret slot (invalidate entry marker).
- `i2c_mem_store_status_t i2c_mem_store_trusted_device_write (i2c_mem_store_t *ctx, uint16_t slot_id, const i2c_mem_store_trusted_device_t *trusted)`
Store trusted-device descriptor in encrypted secret slot.
- `i2c_mem_store_status_t i2c_mem_store_trusted_device_read (i2c_mem_store_t *ctx, uint16_t slot_id, i2c_mem_store_trusted_device_t *trusted)`
Read trusted-device descriptor from encrypted secret slot.

12.16.1 Opis szczegółowy

Implementation of external I2C memory storage helper.
Definicja w pliku `i2c_mem_store.c`.

12.16.2 Dokumentacja definicji

12.16.2.1 I2C_MEM_STORE_LOG_CRC_OFFSET

```
#define I2C_MEM_STORE_LOG_CRC_OFFSET (I2C_MEM_STORE_LOG_RECORD_SIZE - 2U)
```

Definicja w linii 23 pliku `i2c_mem_store.c`.

12.16.2.2 I2C_MEM_STORE_LOG_HEADER_SIZE

```
#define I2C_MEM_STORE_LOG_HEADER_SIZE 16U
```

Definicja w linii 22 pliku `i2c_mem_store.c`.

12.16.2.3 I2C_MEM_STORE_LOG_MAGIC

```
#define I2C_MEM_STORE_LOG_MAGIC 0xA55AU
```

Definicja w linii 19 pliku `i2c_mem_store.c`.

12.16.2.4 I2C_MEM_STORE_LOG_START_ADDR

```
#define I2C_MEM_STORE_LOG_START_ADDR 0x0030UL
```

Definicja w linii 15 pliku `i2c_mem_store.c`.

12.16.2.5 I2C_MEM_STORE_META_MAGIC

```
#define I2C_MEM_STORE_META_MAGIC 0x4D533331UL
```

Definicja w linii 17 pliku [i2c_mem_store.c](#).

12.16.2.6 I2C_MEM_STORE_META_MIRROR_ADDR

```
#define I2C_MEM_STORE_META_MIRROR_ADDR 0x0018UL
```

Definicja w linii 14 pliku [i2c_mem_store.c](#).

12.16.2.7 I2C_MEM_STORE_META_PRIMARY_ADDR

```
#define I2C_MEM_STORE_META_PRIMARY_ADDR 0x0000UL
```

Definicja w linii 13 pliku [i2c_mem_store.c](#).

12.16.2.8 I2C_MEM_STORE_META_VERSION

```
#define I2C_MEM_STORE_META_VERSION 0x0001U
```

Definicja w linii 18 pliku [i2c_mem_store.c](#).

12.16.2.9 I2C_MEM_STORE_META_WIRE_SIZE

```
#define I2C_MEM_STORE_META_WIRE_SIZE 24U
```

Definicja w linii 28 pliku [i2c_mem_store.c](#).

12.16.2.10 I2C_MEM_STORE_SECRET_CRC_OFFSET

```
#define I2C_MEM_STORE_SECRET_CRC_OFFSET (I2C_MEM_STORE_SECRET_SLOT_SIZE - 2U)
```

Definicja w linii 26 pliku [i2c_mem_store.c](#).

12.16.2.11 I2C_MEM_STORE_SECRET_HEADER_SIZE

```
#define I2C_MEM_STORE_SECRET_HEADER_SIZE 8U
```

Definicja w linii 25 pliku [i2c_mem_store.c](#).

12.16.2.12 I2C_MEM_STORE_SECRET_MAGIC

```
#define I2C_MEM_STORE_SECRET_MAGIC 0xC0DEU
```

Definicja w linii 20 pliku [i2c_mem_store.c](#).

12.16.3 Dokumentacja funkcji

12.16.3.1 i2c_mem_store_append_message()

```
i2c_mem_store_status_t i2c_mem_store_append_message (  
    i2c_mem_store_t * ctx,  
    int16_t rssi_dbm,  
    const uint8_t * payload,  
    uint8_t payload_len)
```

Append one received message to persistent ring log.

Parametry

<i>ctx</i>	Context object.
<i>rssi_dbm</i>	RSSI associated with payload.
<i>payload</i>	Message bytes.
<i>payload_len</i>	Message length.

Zwraca

Status code.

Definicja w linii 510 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_crc16_ccitt()`, `I2C_MEM_STORE_EFULL`, `I2C_MEM_STORE_EINVAL`, `i2c_mem_store_le16_write()`, `i2c_mem_store_le32_write()`, `i2c_mem_store_log_slot_addr()`, `i2c_mem_store_meta_save()`, `I2C_MEM_STORE_OK`, `i2c_mem_store_write_raw()`, `i2c_mem_store_t::initialized`, `i2c_mem_store_t::next_sequence`, `i2c_mem_store_t::slot_count`, `i2c_mem_store_t::valid_count` i `i2c_mem_store_t::write_slot`.

Odwołania w `radio_test_store_packet_in_i2c_memory()`.

12.16.3.2 i2c_mem_store_crc16_ccitt()

```
uint16_t i2c_mem_store_crc16_ccitt (
    const uint8_t * data,
    uint16_t len) [static]
```

Calculate CRC-16/CCITT used to protect metadata and stored records.

Definicja w linii 64 pliku `i2c_mem_store.c`.

Odwołania w `i2c_mem_store_append_message()`, `i2c_mem_store_meta_decode()`, `i2c_mem_store_meta_encode()`, `i2c_mem_store_read_message()`, `i2c_mem_store_secret_read()` i `i2c_mem_store_secret_write()`.

12.16.3.3 i2c_mem_store_default_cfg()

```
void i2c_mem_store_default_cfg (
    i2c_mem_store_cfg_t * cfg,
    I2C_HandleTypeDef * hi2c)
```

Fill configuration with safe defaults for I2C1 memory.

Parametry

<i>cfg</i>	[out] Configuration object.
<i>hi2c</i>	I2C handle (typically &hi2c1).

Definicja w linii 409 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_cfg_t::crypto_key`, `i2c_mem_store_cfg_t::hi2c`, `i2c_mem_store_cfg_t::i2c_addr_7bit`, `i2c_mem_store_cfg_t::mem_addr_size`, `i2c_mem_store_cfg_t::page_bytes`, `i2c_mem_store_cfg_t::secret_area_bytes`, `i2c_mem_store_cfg_t::total_bytes` i `i2c_mem_store_cfg_t::write_cycle_ms`.

Odwołania w `i2c_mem_store_default_cfg_m24c01r()`.

12.16.3.4 i2c_mem_store_default_cfg_m24c01r()

```
void i2c_mem_store_default_cfg_m24c01r (
    i2c_mem_store_cfg_t * cfg,
    I2C_HandleTypeDef * hi2c)
```

Fill configuration specifically for ST M24C01-R (1 Kbit EEPROM).

Parametry

<i>cfg</i>	[out] Configuration object.
<i>hi2c</i>	I2C handle (typically &hi2c1).

Definicja w linii 440 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_cfg_t::i2c_addr_7bit`, `i2c_mem_store_default_cfg()`, `i2c_mem_store_cfg_t::mem_addr_size`, `i2c_mem_store_cfg_t::page_bytes`, `i2c_mem_store_cfg_t::secret_area_bytes`, `i2c_mem_store_cfg_t::total_bytes` i `i2c_mem_store_cfg_t::write_cycle_ms`.

Odwołania w `radio_test_demo_init()`.

12.16.3.5 i2c_mem_store_format()

```
i2c_mem_store_status_t i2c_mem_store_format (  
    i2c_mem_store_t * ctx)
```

Format metadata and reset log/secret allocators.

Parametry

<i>ctx</i>	Context object.
------------	-----------------

Zwraca

Status code.

Definicja w linii 496 pliku [i2c_mem_store.c](#).

Odwołuje się do [I2C_MEM_STORE_ESTATE](#), [i2c_mem_store_meta_save\(\)](#), [i2c_mem_store_t::initialized](#), [i2c_mem_store_t::next_sequence](#), [i2c_mem_store_t::secret_counter](#), [i2c_mem_store_t::valid_count](#) i [i2c_mem_store_t::write_slot](#).
Odwołania w [i2c_mem_store_init\(\)](#).

12.16.3.6 i2c_mem_store_get_log_count()

```
i2c_mem_store_status_t i2c_mem_store_get_log_count (  
    i2c_mem_store_t * ctx,  
    uint16_t * out_count)
```

Get number of valid records currently available in ring log.

Parametry

<i>ctx</i>	Context object.
<i>out_count</i>	[out] Record count.

Zwraca

Status code.

Definicja w linii 624 pliku [i2c_mem_store.c](#).

Odwołuje się do [I2C_MEM_STORE_EINVAL](#), [I2C_MEM_STORE_OK](#), [i2c_mem_store_t::initialized](#) i [i2c_mem_store_t::valid_count](#).

12.16.3.7 i2c_mem_store_hal_to_status()

```
i2c_mem_store_status_t i2c_mem_store_hal_to_status (  
    HAL_StatusTypeDef st) [static]
```

Internal helper: [i2c_mem_store_hal_to_status](#).

Definicja w linii 50 pliku [i2c_mem_store.c](#).

Odwołuje się do [I2C_MEM_STORE_EHAL](#), [I2C_MEM_STORE_ETIMEOUT](#) i [I2C_MEM_STORE_OK](#).

Odwołania w [i2c_mem_store_read_raw\(\)](#) i [i2c_mem_store_write_raw\(\)](#).

12.16.3.8 i2c_mem_store_init()

```
i2c_mem_store_status_t i2c_mem_store_init (  
    i2c_mem_store_t * ctx,  
    const i2c_mem_store_cfg_t * cfg,  
    bool format_if_needed)
```

Initialize library, load metadata and optionally format on first use.

Parametry

<i>ctx</i>	Context object.
<i>cfg</i>	Runtime configuration.
<i>format_if_needed</i>	If <code>true</code> , metadata is created when invalid/missing.

Zwraca

Status code.

Definicja w linii 457 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_t::cfg`, `I2C_MEM_STORE_EINVAL`, `i2c_mem_store_format()`, `i2c_mem_store_layout_valid()`, `i2c_mem_store_meta_load()`, `I2C_MEM_STORE_OK`, `i2c_mem_store_t::initialized`, `i2c_mem_store_t::secret_slot_count` i `i2c_mem_store_t::slot_count`.

Odwołania w `radio_test_demo_init()`.

12.16.3.9 i2c_mem_store_layout_valid()

```
bool i2c_mem_store_layout_valid (
    const i2c_mem_store_cfg_t * cfg,
    uint16_t * out_slot_count,
    uint16_t * out_secret_slots) [static]
```

Internal helper: `i2c_mem_store_layout_valid`.

Definicja w linii 371 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_cfg_t::hi2c`, `i2c_mem_store_cfg_t::page_bytes`, `i2c_mem_store_cfg_t::secret_area_bytes` i `i2c_mem_store_cfg_t::total_bytes`.

Odwołania w `i2c_mem_store_init()`.

12.16.3.10 i2c_mem_store_le16_read()

```
uint16_t i2c_mem_store_le16_read (
    const uint8_t * src) [static]
```

Internal helper: `i2c_mem_store_le16_read`.

Definicja w linii 106 pliku `i2c_mem_store.c`.

Odwołania w `i2c_mem_store_meta_decode()`, `i2c_mem_store_read_message()` i `i2c_mem_store_secret_read()`.

12.16.3.11 i2c_mem_store_le16_write()

```
void i2c_mem_store_le16_write (
    uint8_t * dst,
    uint16_t value) [static]
```

Internal helper: `i2c_mem_store_le16_write`.

Definicja w linii 90 pliku `i2c_mem_store.c`.

Odwołania w `i2c_mem_store_append_message()`, `i2c_mem_store_meta_encode()` i `i2c_mem_store_secret_write()`.

12.16.3.12 i2c_mem_store_le32_read()

```
uint32_t i2c_mem_store_le32_read (
    const uint8_t * src) [static]
```

Internal helper: `i2c_mem_store_le32_read`.

Definicja w linii 112 pliku `i2c_mem_store.c`.

Odwołania w `i2c_mem_store_meta_decode()`, `i2c_mem_store_read_message()`, `i2c_mem_store_secret_read()` i `i2c_mem_store_xtea_ctr_crypt()`.

12.16.3.13 i2c_mem_store_le32_write()

```
void i2c_mem_store_le32_write (
    uint8_t * dst,
    uint32_t value) [static]
```

Internal helper: [i2c_mem_store_le32_write](#).

Definicja w linii 97 pliku [i2c_mem_store.c](#).

Odwołania w [i2c_mem_store_append_message\(\)](#), [i2c_mem_store_meta_encode\(\)](#), [i2c_mem_store_secret_write\(\)](#) i [i2c_mem_store_xtea_ctr_crypt\(\)](#).

12.16.3.14 i2c_mem_store_log_base_addr()

```
uint32_t i2c_mem_store_log_base_addr (
    const i2c_mem_store_t * ctx) [static]
```

Internal helper: [i2c_mem_store_log_base_addr](#).

Definicja w linii 176 pliku [i2c_mem_store.c](#).

Odwołania w [i2c_mem_store_log_slot_addr\(\)](#).

12.16.3.15 i2c_mem_store_log_slot_addr()

```
uint32_t i2c_mem_store_log_slot_addr (
    const i2c_mem_store_t * ctx,
    uint16_t slot) [static]
```

Internal helper: [i2c_mem_store_log_slot_addr](#).

Definicja w linii 188 pliku [i2c_mem_store.c](#).

Odwołuje się do [i2c_mem_store_log_base_addr\(\)](#).

Odwołania w [i2c_mem_store_append_message\(\)](#) i [i2c_mem_store_read_message\(\)](#).

12.16.3.16 i2c_mem_store_meta_decode()

```
bool i2c_mem_store_meta_decode (
    i2c_mem_store_t * ctx,
    const uint8_t in[I2C_MEM_STORE_META_WIRE_SIZE]) [static]
```

Decode and validate persisted metadata from one metadata copy.

Definicja w linii 293 pliku [i2c_mem_store.c](#).

Odwołuje się do [i2c_mem_store_crc16_ccitt\(\)](#), [i2c_mem_store_le16_read\(\)](#), [i2c_mem_store_le32_read\(\)](#), [i2c_mem_store_t::next_sequence](#), [i2c_mem_store_t::secret_counter](#), [i2c_mem_store_t::slot_count](#), [i2c_mem_store_t::valid_count](#) i [i2c_mem_store_t::write_slot](#).

Odwołania w [i2c_mem_store_meta_load\(\)](#).

12.16.3.17 i2c_mem_store_meta_encode()

```
void i2c_mem_store_meta_encode (
    const i2c_mem_store_t * ctx,
    uint8_t out[I2C_MEM_STORE_META_WIRE_SIZE]) [static]
```

Encode runtime metadata into the fixed little-endian on-memory layout.

Definicja w linii 266 pliku [i2c_mem_store.c](#).

Odwołuje się do [i2c_mem_store_crc16_ccitt\(\)](#), [i2c_mem_store_le16_write\(\)](#), [i2c_mem_store_le32_write\(\)](#), [i2c_mem_store_t::next_sequence](#), [i2c_mem_store_t::secret_counter](#), [i2c_mem_store_t::valid_count](#) i [i2c_mem_store_t::write_slot](#).

Odwołania w [i2c_mem_store_meta_save\(\)](#).

12.16.3.18 i2c_mem_store_meta_load()

```
i2c_mem_store_status_t i2c_mem_store_meta_load (
    i2c_mem_store_t * ctx) [static]
```

Internal helper: [i2c_mem_store_meta_load](#).

Definicja w linii 350 pliku [i2c_mem_store.c](#).

Odwołuje się do [I2C_MEM_STORE_ENOTFOUND](#), [i2c_mem_store_meta_decode\(\)](#), [I2C_MEM_STORE_OK](#) i [i2c_mem_store_read_raw\(\)](#).

Odwołania w [i2c_mem_store_init\(\)](#).

12.16.3.19 i2c_mem_store_meta_save()

```
i2c_mem_store_status_t i2c_mem_store_meta_save (
    i2c_mem_store_t * ctx) [static]
```

Internal helper: [i2c_mem_store_meta_save](#).

Definicja w linii 333 pliku [i2c_mem_store.c](#).

Odwołuje się do [i2c_mem_store_meta_encode\(\)](#), [I2C_MEM_STORE_OK](#) i [i2c_mem_store_write_raw\(\)](#).

Odwołania w [i2c_mem_store_append_message\(\)](#), [i2c_mem_store_format\(\)](#) i [i2c_mem_store_secret_write\(\)](#).

12.16.3.20 i2c_mem_store_min_u16()

```
uint16_t i2c_mem_store_min_u16 (
    uint16_t a,
    uint16_t b) [static]
```

Internal helper: [i2c_mem_store_min_u16](#).

Definicja w linii 44 pliku [i2c_mem_store.c](#).

Odwołania w [i2c_mem_store_write_raw\(\)](#).

12.16.3.21 i2c_mem_store_read_message()

```
i2c_mem_store_status_t i2c_mem_store_read_message (
    i2c_mem_store_t * ctx,
    uint16_t index_from_latest,
    i2c_mem_store_log_record_t * out_record)
```

Read one message from log (0 = latest, 1 = previous, ...).

Parametry

<i>ctx</i>	Context object.
<i>index_from_latest</i>	Relative index from latest entry.
<i>out_record</i>	[out] Decoded record.

Zwraca

Status code.

Definicja w linii 569 pliku [i2c_mem_store.c](#).

Odwołuje się do [i2c_mem_store_crc16_ccitt\(\)](#), [I2C_MEM_STORE_ECRC](#), [I2C_MEM_STORE_EINVAL](#), [I2C_MEM_STORE_ENOTFOUND](#), [i2c_mem_store_le16_read\(\)](#), [i2c_mem_store_le32_read\(\)](#), [i2c_mem_store_log_slot_addr\(\)](#), [I2C_MEM_STORE_OK](#), [i2c_mem_store_read_raw\(\)](#), [i2c_mem_store_t::initialized](#), [i2c_mem_store_log_record_t::length](#), [i2c_mem_store_log_record_t::payload](#), [i2c_mem_store_log_record_t::rssi_dbm](#), [i2c_mem_store_log_record_t::sequence](#), [i2c_mem_store_t::slot_count](#), [i2c_mem_store_log_record_t::timestamp_ms](#), [i2c_mem_store_t::valid_count](#) i [i2c_mem_store_t::write_slot](#).

12.16.3.22 i2c_mem_store_read_raw()

```
i2c_mem_store_status_t i2c_mem_store_read_raw (
    i2c_mem_store_t * ctx,
    uint32_t addr,
    uint8_t * dst,
    uint16_t len) [static]
```

Internal helper: [i2c_mem_store_read_raw](#).

Definicja w linii 200 pliku [i2c_mem_store.c](#).

Odwołuje się do [i2c_mem_store_t::cfg](#), [i2c_mem_store_cfg_t::hi2c](#), [i2c_mem_store_cfg_t::i2c_addr_7bit](#), [I2C_MEM_STORE_EINVAL](#), [i2c_mem_store_hal_to_status\(\)](#) i [i2c_mem_store_cfg_t::mem_addr_size](#).

Odwołania w [i2c_mem_store_meta_load\(\)](#), [i2c_mem_store_read_message\(\)](#) i [i2c_mem_store_secret_read\(\)](#).

12.16.3.23 i2c_mem_store_secret_base_addr()

```
uint32_t i2c_mem_store_secret_base_addr (
    const i2c_mem_store_t * ctx) [static]
```

Internal helper: [i2c_mem_store_secret_base_addr](#).

Definicja w linii 182 pliku [i2c_mem_store.c](#).

Odwołuje się do [i2c_mem_store_t::cfg](#), [i2c_mem_store_cfg_t::secret_area_bytes](#) i [i2c_mem_store_cfg_t::total_bytes](#).

Odwołania w [i2c_mem_store_secret_slot_addr\(\)](#).

12.16.3.24 i2c_mem_store_secret_erase()

```
i2c_mem_store_status_t i2c_mem_store_secret_erase (
    i2c_mem_store_t * ctx,
    uint16_t slot_id)
```

Erase one secret slot (invalidate entry marker).

Parametry

<i>ctx</i>	Context object.
<i>slot_id</i>	Slot index in secret partition.

Zwraca

Status code.

Definicja w linii 746 pliku [i2c_mem_store.c](#).

Odwołuje się do [I2C_MEM_STORE_EINVAL](#), [i2c_mem_store_secret_slot_addr\(\)](#), [i2c_mem_store_write_raw\(\)](#), [i2c_mem_store_t::initialized](#) i [i2c_mem_store_t::secret_slot_count](#).

12.16.3.25 i2c_mem_store_secret_read()

```
i2c_mem_store_status_t i2c_mem_store_secret_read (
    i2c_mem_store_t * ctx,
    uint16_t slot_id,
    uint8_t * data_out,
    uint8_t data_capacity,
    uint8_t * data_len)
```

Read and decrypt payload from secret partition slot.

Parametry

<i>ctx</i>	Context object.
<i>slot_id</i>	Slot index in secret partition.
<i>data_out</i>	Output buffer for decrypted payload.
<i>data_capacity</i>	Output buffer capacity.
<i>data_len</i>	[out] Returned payload length.

Zwraca

Status code.

Definicja w linii 683 pliku [i2c_mem_store.c](#).

Odwołuje się do [i2c_mem_store_t::cfg](#), [i2c_mem_store_cfg_t::crypto_key](#), [i2c_mem_store_crc16_ccitt\(\)](#), [I2C_MEM_STORE_ECRC](#), [I2C_MEM_STORE_EINVAL](#), [I2C_MEM_STORE_ENOTFOUND](#), [I2C_MEM_STORE_EOVERFLOW](#), [I2C_MEM_STORE_EPROTO](#), [i2c_mem_store_le16_read\(\)](#), [i2c_mem_store_le32_read\(\)](#), [I2C_MEM_STORE_OK](#), [i2c_mem_store_read_raw\(\)](#), [i2c_mem_store_secret_slot_addr\(\)](#), [i2c_mem_store_xtea_ctr_crypt\(\)](#), [i2c_mem_store_t::initialized](#) i [i2c_mem_store_t::secret_slot_count](#).

Odwołania w [i2c_mem_store_trusted_device_read\(\)](#).

12.16.3.26 i2c_mem_store_secret_slot_addr()

```
uint32_t i2c_mem_store_secret_slot_addr (
    const i2c_mem_store_t * ctx,
    uint16_t slot) [static]
```

Internal helper: [i2c_mem_store_secret_slot_addr](#).

Definicja w linii 194 pliku [i2c_mem_store.c](#).

Odwołuje się do [i2c_mem_store_secret_base_addr\(\)](#).

Odwołania w [i2c_mem_store_secret_erase\(\)](#), [i2c_mem_store_secret_read\(\)](#) i [i2c_mem_store_secret_write\(\)](#).

12.16.3.27 i2c_mem_store_secret_write()

```
i2c_mem_store_status_t i2c_mem_store_secret_write (
    i2c_mem_store_t * ctx,
    uint16_t slot_id,
    const uint8_t * data,
    uint8_t data_len)
```

Store encrypted arbitrary payload in secret partition slot.

Parametry

<i>ctx</i>	Context object.
<i>slot_id</i>	Slot index in secret partition.
<i>data</i>	Plain payload.
<i>data_len</i>	Payload length.

Zwraca

Status code.

Definicja w linii 635 pliku [i2c_mem_store.c](#).

Odwołuje się do [i2c_mem_store_t::cfg](#), [i2c_mem_store_cfg_t::crypto_key](#), [i2c_mem_store_crc16_ccitt\(\)](#), [I2C_MEM_STORE_EINVAL](#), [I2C_MEM_STORE_EOVERFLOW](#), [i2c_mem_store_le16_write\(\)](#), [i2c_mem_store_le32_write\(\)](#), [i2c_mem_store_meta_save\(\)](#), [I2C_MEM_STORE_OK](#), [i2c_mem_store_secret_slot_addr\(\)](#), [i2c_mem_store_write_raw\(\)](#), [i2c_mem_store_xtea_ctr_crypt\(\)](#), [i2c_mem_store_t::initialized](#), [i2c_mem_store_t::secret_counter](#) i [i2c_mem_store_t::secret_slot_count](#).

Odwołania w [i2c_mem_store_trusted_device_write\(\)](#).

12.16.3.28 i2c_mem_store_trusted_device_read()

```
i2c_mem_store_status_t i2c_mem_store_trusted_device_read (
    i2c_mem_store_t * ctx,
    uint16_t slot_id,
    i2c_mem_store_trusted_device_t * trusted)
```

Read trusted-device descriptor from encrypted secret slot.

Parametry

<i>ctx</i>	Context object.
<i>slot_id</i>	Slot index.
<i>trusted</i>	[out] Trusted-device object.

Zwraca

Status code.

Definicja w linii 787 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_trusted_device_t::code`, `i2c_mem_store_trusted_device_t::code_len`, `I2C_MEM_STORE_EINVAL`, `I2C_MEM_STORE_EPROTO`, `I2C_MEM_STORE_OK`, `i2c_mem_store_secret_read()`, `i2c_mem_store_trusted_device_t::id`, `i2c_mem_store_trusted_device_t::id_len` i `i2c_mem_store_t::initialized`.

12.16.3.29 i2c_mem_store_trusted_device_write()

```
i2c_mem_store_status_t i2c_mem_store_trusted_device_write (
    i2c_mem_store_t * ctx,
    uint16_t slot_id,
    const i2c_mem_store_trusted_device_t * trusted)
```

Store trusted-device descriptor in encrypted secret slot.

Parametry

<i>ctx</i>	Context object.
<i>slot_id</i>	Slot index.
<i>trusted</i>	Trusted-device object.

Zwraca

Status code.

Definicja w linii 758 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_trusted_device_t::code`, `i2c_mem_store_trusted_device_t::code_len`, `I2C_MEM_STORE_EINVAL`, `I2C_MEM_STORE_EOVERFLOW`, `i2c_mem_store_secret_write()`, `i2c_mem_store_trusted_device_t::id`, `i2c_mem_store_trusted_device_t::id_len` i `i2c_mem_store_t::initialized`.

12.16.3.30 i2c_mem_store_write_raw()

```
i2c_mem_store_status_t i2c_mem_store_write_raw (
    i2c_mem_store_t * ctx,
    uint32_t addr,
    const uint8_t * src,
    uint16_t len) [static]
```

Write raw bytes while respecting EEPROM page boundaries and write-cycle delay.

Definicja w linii 223 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_t::cfg`, `i2c_mem_store_cfg_t::hi2c`, `i2c_mem_store_cfg_t::i2c_addr_7bit`, `I2C_MEM_STORE_EINVAL`, `i2c_mem_store_hal_to_status()`, `i2c_mem_store_min_u16()`, `I2C_MEM_STORE_OK`, `i2c_mem_store_cfg_t::mem_addr_size`, `i2c_mem_store_cfg_t::page_bytes` i `i2c_mem_store_cfg_t::write_cycle_ms`.
Odwołania w `i2c_mem_store_append_message()`, `i2c_mem_store_meta_save()`, `i2c_mem_store_secret_erase()` i `i2c_mem_store_secret_write()`.

12.16.3.31 i2c_mem_store_xtea_ctr_crypt()

```
void i2c_mem_store_xtea_ctr_crypt (
    uint8_t * data,
    uint16_t len,
    const uint8_t key_bytes[16],
    uint32_t nonce) [static]
```

Apply XTEA-CTR in place to encrypt or decrypt secret-slot payload bytes.

Definicja w linii 141 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_le32_read()`, `i2c_mem_store_le32_write()` i `i2c_mem_store_xtea_encrypt_block()`.
Odwołania w `i2c_mem_store_secret_read()` i `i2c_mem_store_secret_write()`.

12.16.3.32 i2c_mem_store_xtea_encrypt_block()

```
void i2c_mem_store_xtea_encrypt_block (
    uint32_t v[2],
    const uint32_t key[4]) [static]
```

Encrypt one 64-bit block with XTEA for the software CTR keystream.

Definicja w linii 121 pliku [i2c_mem_store.c](#).

Odwolania w [i2c_mem_store_xtea_ctr_crypt\(\)](#).

12.17 i2c_mem_store.c

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #include "i2c_mem_store.h"
00007
00008 #include "../app.h"
00009 #include "app_delay.h"
00010
00011 #include <string.h>
00012
00013 #define I2C_MEM_STORE_META_PRIMARY_ADDR    0x0000UL
00014 #define I2C_MEM_STORE_META_MIRROR_ADDR    0x0018UL
00015 #define I2C_MEM_STORE_LOG_START_ADDR      0x0030UL
00016
00017 #define I2C_MEM_STORE_META_MAGIC          0x4D533331UL
00018 #define I2C_MEM_STORE_META_VERSION        0x0001U
00019 #define I2C_MEM_STORE_LOG_MAGIC           0xA55AU
00020 #define I2C_MEM_STORE_SECRET_MAGIC        0xC0DEU
00021
00022 #define I2C_MEM_STORE_LOG_HEADER_SIZE      16U
00023 #define I2C_MEM_STORE_LOG_CRC_OFFSET      (I2C_MEM_STORE_LOG_RECORD_SIZE - 2U)
00024
00025 #define I2C_MEM_STORE_SECRET_HEADER_SIZE   8U
00026 #define I2C_MEM_STORE_SECRET_CRC_OFFSET   (I2C_MEM_STORE_SECRET_SLOT_SIZE - 2U)
00027
00028 #define I2C_MEM_STORE_META_WIRE_SIZE       24U
00029
00030 typedef struct
00031 {
00032     uint32_t magic;
00033     uint16_t version;
00034     uint16_t write_slot;
00035     uint16_t valid_count;
00036     uint16_t reserved;
00037     uint32_t next_sequence;
00038     uint32_t secret_counter;
00039     uint16_t crc16;
00040     uint16_t reserved2;
00041 } i2c_mem_store_meta_wire_t;
00042
00044 static uint16_t i2c_mem_store_min_ul6(uint16_t a, uint16_t b)
00045 {
00046     return (a < b) ? a : b;
00047 }
00048
00050 static i2c_mem_store_status_t i2c_mem_store_hal_to_status(HAL_StatusTypeDef st)
00051 {
00052     if (st == HAL_OK)
00053     {
00054         return I2C_MEM_STORE_OK;
00055     }
00056     if (st == HAL_TIMEOUT)
00057     {
00058         return I2C_MEM_STORE_ETIMEOUT;
00059     }
00060     return I2C_MEM_STORE_EHAL;
00061 }
00062
00064 static uint16_t i2c_mem_store_crc16_ccitt(const uint8_t *data, uint16_t len)
00065 {
00066     uint16_t crc = 0xFFFFU;
00067     uint16_t i;
00068     uint8_t j;
00069
00070     for (i = 0U; i < len; i++)
00071     {
00072         crc ^= (uint16_t)((uint16_t)data[i] << 8);
00073         for (j = 0U; j < 8U; j++)
00074         {
00075             if ((crc & 0x8000U) != 0U)
```

```

00076         {
00077             crc = (uint16_t)((crc << 1) ^ 0x1021U);
00078         }
00079         else
00080         {
00081             crc <= 1;
00082         }
00083     }
00084 }
00085
00086     return crc;
00087 }
00088
00090 static void i2c_mem_store_le16_write(uint8_t *dst, uint16_t value)
00091 {
00092     dst[0] = (uint8_t)(value & 0xFFU);
00093     dst[1] = (uint8_t)(value >> 8);
00094 }
00095
00097 static void i2c_mem_store_le32_write(uint8_t *dst, uint32_t value)
00098 {
00099     dst[0] = (uint8_t)(value & 0xFFU);
00100     dst[1] = (uint8_t)((value >> 8) & 0xFFU);
00101     dst[2] = (uint8_t)((value >> 16) & 0xFFU);
00102     dst[3] = (uint8_t)((value >> 24) & 0xFFU);
00103 }
00104
00106 static uint16_t i2c_mem_store_le16_read(const uint8_t *src)
00107 {
00108     return (uint16_t)src[0] | ((uint16_t)src[1] << 8);
00109 }
00110
00112 static uint32_t i2c_mem_store_le32_read(const uint8_t *src)
00113 {
00114     return ((uint32_t)src[0]) |
00115            ((uint32_t)src[1] << 8) |
00116            ((uint32_t)src[2] << 16) |
00117            ((uint32_t)src[3] << 24);
00118 }
00119
00121 static void i2c_mem_store_xtea_encrypt_block(uint32_t v[2], const uint32_t key[4])
00122 {
00123     uint32_t i;
00124     uint32_t sum = 0U;
00125     uint32_t delta = 0x9E3779B9UL;
00126     uint32_t v0 = v[0];
00127     uint32_t v1 = v[1];
00128
00129     for (i = 0U; i < 32U; i++)
00130     {
00131         v0 += (((v1 << 4) ^ (v1 >> 5)) + v1) ^ (sum + key[sum & 3U]);
00132         sum += delta;
00133         v1 += (((v0 << 4) ^ (v0 >> 5)) + v0) ^ (sum + key[(sum >> 11) & 3U]);
00134     }
00135
00136     v[0] = v0;
00137     v[1] = v1;
00138 }
00139
00141 static void i2c_mem_store_xtea_ctr_crypt(uint8_t *data,
00142                                           uint16_t len,
00143                                           const uint8_t key_bytes[16],
00144                                           uint32_t nonce)
00145 {
00146     uint32_t key[4];
00147     uint32_t ctr = 0U;
00148     uint16_t offset = 0U;
00149     uint16_t i;
00150     uint8_t keystream[8];
00151
00152     key[0] = i2c_mem_store_le32_read(&key_bytes[0]);
00153     key[1] = i2c_mem_store_le32_read(&key_bytes[4]);
00154     key[2] = i2c_mem_store_le32_read(&key_bytes[8]);
00155     key[3] = i2c_mem_store_le32_read(&key_bytes[12]);
00156
00157     while (offset < len)
00158     {
00159         uint32_t block[2];
00160         block[0] = nonce;
00161         block[1] = ctr;
00162         i2c_mem_store_xtea_encrypt_block(block, key);
00163         i2c_mem_store_le32_write(&keystream[0], block[0]);
00164         i2c_mem_store_le32_write(&keystream[4], block[1]);
00165
00166         for (i = 0U; (i < 8U) && (offset < len); i++)
00167         {
00168             data[offset] ^= keystream[i];

```



```

00169         offset++;
00170     }
00171     ctr++;
00172 }
00173 }
00174
00176 static uint32_t i2c_mem_store_log_base_addr(const i2c_mem_store_t *ctx)
00177 {
00178     return I2C_MEM_STORE_LOG_START_ADDR;
00179 }
00180
00182 static uint32_t i2c_mem_store_secret_base_addr(const i2c_mem_store_t *ctx)
00183 {
00184     return ctx->cfg.total_bytes - ctx->cfg.secret_area_bytes;
00185 }
00186
00188 static uint32_t i2c_mem_store_log_slot_addr(const i2c_mem_store_t *ctx, uint16_t slot)
00189 {
00190     return i2c_mem_store_log_base_addr(ctx) + ((uint32_t)slot * I2C_MEM_STORE_LOG_RECORD_SIZE);
00191 }
00192
00194 static uint32_t i2c_mem_store_secret_slot_addr(const i2c_mem_store_t *ctx, uint16_t slot)
00195 {
00196     return i2c_mem_store_secret_base_addr(ctx) + ((uint32_t)slot * I2C_MEM_STORE_SECRET_SLOT_SIZE);
00197 }
00198
00200 static i2c_mem_store_status_t i2c_mem_store_read_raw(i2c_mem_store_t *ctx,
00201                                                       uint32_t addr,
00202                                                       uint8_t *dst,
00203                                                       uint16_t len)
00204 {
00205     HAL_StatusTypeDef st;
00206
00207     if ((ctx == NULL) || (dst == NULL) || (len == 0U))
00208     {
00209         return I2C_MEM_STORE_EINVAL;
00210     }
00211
00212     st = app_i2c_mem_read(ctx->cfg.hi2c,
00213                          (uint16_t)(ctx->cfg.i2c_addr_7bit << 1),
00214                          (uint16_t)addr,
00215                          ctx->cfg.mem_addr_size,
00216                          dst,
00217                          len,
00218                          100U);
00219     return i2c_mem_store_hal_to_status(st);
00220 }
00221
00223 static i2c_mem_store_status_t i2c_mem_store_write_raw(i2c_mem_store_t *ctx,
00224                                                       uint32_t addr,
00225                                                       const uint8_t *src,
00226                                                       uint16_t len)
00227 {
00228     uint32_t current_addr = addr;
00229     uint16_t remaining = len;
00230     uint16_t chunk;
00231     uint16_t page_off;
00232     HAL_StatusTypeDef st;
00233
00234     if ((ctx == NULL) || (src == NULL) || (len == 0U))
00235     {
00236         return I2C_MEM_STORE_EINVAL;
00237     }
00238
00239     while (remaining > 0U)
00240     {
00241         page_off = (uint16_t)(current_addr % ctx->cfg.page_bytes);
00242         chunk = i2c_mem_store_min_u16(remaining, (uint16_t)(ctx->cfg.page_bytes - page_off));
00243
00244         st = app_i2c_mem_write(ctx->cfg.hi2c,
00245                              (uint16_t)(ctx->cfg.i2c_addr_7bit << 1),
00246                              (uint16_t)current_addr,
00247                              ctx->cfg.mem_addr_size,
00248                              src,
00249                              chunk,
00250                              100U);
00251         if (st != HAL_OK)
00252         {
00253             return i2c_mem_store_hal_to_status(st);
00254         }
00255
00256         app_delay_ms(ctx->cfg.write_cycle_ms);
00257         current_addr += chunk;
00258         src += chunk;
00259         remaining = (uint16_t)(remaining - chunk);
00260     }
00261 }

```

```

00262     return I2C_MEM_STORE_OK;
00263 }
00264
00266 static void i2c_mem_store_meta_encode(const i2c_mem_store_t *ctx, uint8_t
    out[I2C_MEM_STORE_META_WIRE_SIZE])
00267 {
00268     i2c_mem_store_meta_wire_t w;
00269
00270     memset(&w, 0, sizeof(w));
00271     w.magic = I2C_MEM_STORE_META_MAGIC;
00272     w.version = I2C_MEM_STORE_META_VERSION;
00273     w.write_slot = ctx->write_slot;
00274     w.valid_count = ctx->valid_count;
00275     w.next_sequence = ctx->next_sequence;
00276     w.secret_counter = ctx->secret_counter;
00277
00278     i2c_mem_store_le32_write(&out[0], w.magic);
00279     i2c_mem_store_le16_write(&out[4], w.version);
00280     i2c_mem_store_le16_write(&out[6], w.write_slot);
00281     i2c_mem_store_le16_write(&out[8], w.valid_count);
00282     i2c_mem_store_le16_write(&out[10], 0U);
00283     i2c_mem_store_le32_write(&out[12], w.next_sequence);
00284     i2c_mem_store_le32_write(&out[16], w.secret_counter);
00285     i2c_mem_store_le16_write(&out[20], 0U);
00286     i2c_mem_store_le16_write(&out[22], 0U);
00287
00288     w.crc16 = i2c_mem_store_crc16_ccitt(out, 20U);
00289     i2c_mem_store_le16_write(&out[20], w.crc16);
00290 }
00291
00293 static bool i2c_mem_store_meta_decode(i2c_mem_store_t *ctx, const uint8_t
    in[I2C_MEM_STORE_META_WIRE_SIZE])
00294 {
00295     uint16_t crc_stored;
00296     uint16_t crc_calc;
00297     uint16_t write_slot;
00298     uint16_t valid_count;
00299
00300     crc_stored = i2c_mem_store_le16_read(&in[20]);
00301     crc_calc = i2c_mem_store_crc16_ccitt(in, 20U);
00302     if (crc_calc != crc_stored)
00303     {
00304         return false;
00305     }
00306
00307     if (i2c_mem_store_le32_read(&in[0]) != I2C_MEM_STORE_META_MAGIC)
00308     {
00309         return false;
00310     }
00311     if (i2c_mem_store_le16_read(&in[4]) != I2C_MEM_STORE_META_VERSION)
00312     {
00313         return false;
00314     }
00315
00316     write_slot = i2c_mem_store_le16_read(&in[6]);
00317     valid_count = i2c_mem_store_le16_read(&in[8]);
00318     if (((ctx->slot_count == 0U) && (write_slot != 0U)) ||
00319         ((ctx->slot_count > 0U) && (write_slot >= ctx->slot_count)) ||
00320         (valid_count > ctx->slot_count))
00321     {
00322         return false;
00323     }
00324
00325     ctx->write_slot = write_slot;
00326     ctx->valid_count = valid_count;
00327     ctx->next_sequence = i2c_mem_store_le32_read(&in[12]);
00328     ctx->secret_counter = i2c_mem_store_le32_read(&in[16]);
00329     return true;
00330 }
00331
00333 static i2c_mem_store_status_t i2c_mem_store_meta_save(i2c_mem_store_t *ctx)
00334 {
00335     uint8_t raw[I2C_MEM_STORE_META_WIRE_SIZE];
00336     i2c_mem_store_status_t rc;
00337
00338     i2c_mem_store_meta_encode(ctx, raw);
00339
00340     rc = i2c_mem_store_write_raw(ctx, I2C_MEM_STORE_META_PRIMARY_ADDR, raw, sizeof(raw));
00341     if (rc != I2C_MEM_STORE_OK)
00342     {
00343         return rc;
00344     }
00345     rc = i2c_mem_store_write_raw(ctx, I2C_MEM_STORE_META_MIRROR_ADDR, raw, sizeof(raw));
00346     return rc;
00347 }
00348
00350 static i2c_mem_store_status_t i2c_mem_store_meta_load(i2c_mem_store_t *ctx)

```

```

00351 {
00352     uint8_t raw[I2C_MEM_STORE_META_WIRE_SIZE];
00353     i2c_mem_store_status_t rc;
00354
00355     rc = i2c_mem_store_read_raw(ctx, I2C_MEM_STORE_META_PRIMARY_ADDR, raw, sizeof(raw));
00356     if ((rc == I2C_MEM_STORE_OK) && i2c_mem_store_meta_decode(ctx, raw))
00357     {
00358         return I2C_MEM_STORE_OK;
00359     }
00360
00361     rc = i2c_mem_store_read_raw(ctx, I2C_MEM_STORE_META_MIRROR_ADDR, raw, sizeof(raw));
00362     if ((rc == I2C_MEM_STORE_OK) && i2c_mem_store_meta_decode(ctx, raw))
00363     {
00364         return I2C_MEM_STORE_OK;
00365     }
00366
00367     return I2C_MEM_STORE_ENOTFOUND;
00368 }
00369
00371 static bool i2c_mem_store_layout_valid(const i2c_mem_store_cfg_t *cfg,
00372                                       uint16_t *out_slot_count,
00373                                       uint16_t *out_secret_slots)
00374 {
00375     uint32_t secret_base;
00376     uint32_t log_bytes;
00377     uint16_t slot_count;
00378     uint16_t secret_slots;
00379
00380     if ((cfg == NULL) || (cfg->hi2c == NULL))
00381     {
00382         return false;
00383     }
00384     if ((cfg->total_bytes < 128U) || (cfg->page_bytes == 0U) || (cfg->secret_area_bytes <
I2C_MEM_STORE_SECRET_SLOT_SIZE))
00385     {
00386         return false;
00387     }
00388
00389     secret_base = cfg->total_bytes - cfg->secret_area_bytes;
00390     if (secret_base <= I2C_MEM_STORE_LOG_START_ADDR)
00391     {
00392         return false;
00393     }
00394
00395     log_bytes = secret_base - I2C_MEM_STORE_LOG_START_ADDR;
00396     slot_count = (uint16_t)(log_bytes / I2C_MEM_STORE_LOG_RECORD_SIZE);
00397     secret_slots = (uint16_t)(cfg->secret_area_bytes / I2C_MEM_STORE_SECRET_SLOT_SIZE);
00398
00399     if (secret_slots == 0U)
00400     {
00401         return false;
00402     }
00403
00404     *out_slot_count = slot_count;
00405     *out_secret_slots = secret_slots;
00406     return true;
00407 }
00408
00409 void i2c_mem_store_default_cfg(i2c_mem_store_cfg_t *cfg, I2C_HandleTypeDef *hi2c)
00410 {
00411     uint8_t i;
00412     static const uint8_t default_key[16] =
00413     {
00414         0x2AU, 0x19U, 0x84U, 0xc7U,
00415         0x3DU, 0x55U, 0xABU, 0x10U,
00416         0xE1U, 0x7CU, 0x06U, 0x93U,
00417         0x44U, 0x5FU, 0xB2U, 0x0DU
00418     };
00419
00420     if (cfg == NULL)
00421     {
00422         return;
00423     }
00424
00425     memset(cfg, 0, sizeof(*cfg));
00426     cfg->hi2c = hi2c;
00427     cfg->i2c_addr_7bit = I2C_MEM_STORE_I2C_ADDR_DEFAULT;
00428     cfg->mem_addr_size = I2C_MEMADD_SIZE_8BIT;
00429     cfg->total_bytes = I2C_MEM_STORE_TOTAL_BYTES_DEFAULT;
00430     cfg->page_bytes = I2C_MEM_STORE_PAGE_BYTES_DEFAULT;
00431     cfg->write_cycle_ms = I2C_MEM_STORE_WRITE_CYCLE_MS_DEFAULT;
00432     cfg->secret_area_bytes = I2C_MEM_STORE_SECRET_AREA_BYTES_DEFAULT;
00433
00434     for (i = 0U; i < 16U; i++)
00435     {
00436         cfg->crypto_key[i] = default_key[i];
00437     }

```

```

00438 }
00439
00440 void i2c_mem_store_default_cfg_m24c01r(i2c_mem_store_cfg_t *cfg, I2C_HandleTypeDef *hi2c)
00441 {
00442     i2c_mem_store_default_cfg(cfg, hi2c);
00443
00444     if (cfg == NULL)
00445     {
00446         return;
00447     }
00448
00449     cfg->i2c_addr_7bit = I2C_MEM_STORE_I2C_ADDR_DEFAULT;
00450     cfg->mem_addr_size = I2C_MEMADD_SIZE_8BIT;
00451     cfg->total_bytes = 128U;
00452     cfg->page_bytes = 8U;
00453     cfg->write_cycle_ms = 5U;
00454     cfg->secret_area_bytes = I2C_MEM_STORE_SECRET_AREA_BYTES_DEFAULT;
00455 }
00456
00457 i2c_mem_store_status_t i2c_mem_store_init(i2c_mem_store_t *ctx,
00458                                           const i2c_mem_store_cfg_t *cfg,
00459                                           bool format_if_needed)
00460 {
00461     i2c_mem_store_status_t rc;
00462     uint16_t slot_count = 0U;
00463     uint16_t secret_slots = 0U;
00464
00465     if ((ctx == NULL) || (cfg == NULL))
00466     {
00467         return I2C_MEM_STORE_EINVAL;
00468     }
00469
00470     if (!i2c_mem_store_layout_valid(cfg, &slot_count, &secret_slots))
00471     {
00472         return I2C_MEM_STORE_EINVAL;
00473     }
00474
00475     memset(ctx, 0, sizeof(*ctx));
00476     ctx->cfg = *cfg;
00477     ctx->slot_count = slot_count;
00478     ctx->secret_slot_count = secret_slots;
00479     ctx->initialized = true;
00480
00481     rc = i2c_mem_store_meta_load(ctx);
00482     if (rc == I2C_MEM_STORE_OK)
00483     {
00484         return I2C_MEM_STORE_OK;
00485     }
00486
00487     if (!format_if_needed)
00488     {
00489         ctx->initialized = false;
00490         return rc;
00491     }
00492
00493     return i2c_mem_store_format(ctx);
00494 }
00495
00496 i2c_mem_store_status_t i2c_mem_store_format(i2c_mem_store_t *ctx)
00497 {
00498     if ((ctx == NULL) || (!ctx->initialized))
00499     {
00500         return I2C_MEM_STORE_ESTATE;
00501     }
00502
00503     ctx->write_slot = 0U;
00504     ctx->valid_count = 0U;
00505     ctx->next_sequence = 1UL;
00506     ctx->secret_counter = 1UL;
00507     return i2c_mem_store_meta_save(ctx);
00508 }
00509
00510 i2c_mem_store_status_t i2c_mem_store_append_message(i2c_mem_store_t *ctx,
00511                                                     int16_t rssi_dbm,
00512                                                     const uint8_t *payload,
00513                                                     uint8_t payload_len)
00514 {
00515     uint8_t rec[I2C_MEM_STORE_LOG_RECORD_SIZE];
00516     uint8_t effective_len;
00517     uint16_t crc;
00518     uint32_t addr;
00519     i2c_mem_store_status_t rc;
00520
00521     if ((ctx == NULL) || (!ctx->initialized) || ((payload == NULL) && (payload_len > 0U)))
00522     {
00523         return I2C_MEM_STORE_EINVAL;
00524     }

```

```

00525     if (ctx->slot_count == 0U)
00526     {
00527         return I2C_MEM_STORE_EFULL;
00528     }
00529
00530     effective_len = payload_len;
00531     if (effective_len > I2C_MEM_STORE_LOG_PAYLOAD_MAX)
00532     {
00533         effective_len = I2C_MEM_STORE_LOG_PAYLOAD_MAX;
00534     }
00535
00536     memset(rec, 0xFF, sizeof(rec));
00537     i2c_mem_store_le16_write(&rec[0], I2C_MEM_STORE_LOG_MAGIC);
00538     rec[2] = effective_len;
00539     rec[3] = 0U;
00540     i2c_mem_store_le32_write(&rec[4], ctx->next_sequence);
00541     i2c_mem_store_le32_write(&rec[8], HAL_GetTick());
00542     i2c_mem_store_le16_write(&rec[12], (uint16_t)rssi_dbm);
00543
00544     if (effective_len > 0U)
00545     {
00546         memcpy(&rec[I2C_MEM_STORE_LOG_HEADER_SIZE], payload, effective_len);
00547     }
00548
00549     crc = i2c_mem_store_crc16_ccitt(rec, I2C_MEM_STORE_LOG_CRC_OFFSET);
00550     i2c_mem_store_le16_write(&rec[I2C_MEM_STORE_LOG_CRC_OFFSET], crc);
00551
00552     addr = i2c_mem_store_log_slot_addr(ctx, ctx->write_slot);
00553     rc = i2c_mem_store_write_raw(ctx, addr, rec, sizeof(rec));
00554     if (rc != I2C_MEM_STORE_OK)
00555     {
00556         return rc;
00557     }
00558
00559     ctx->write_slot = (uint16_t)((ctx->write_slot + 1U) % ctx->slot_count);
00560     if (ctx->valid_count < ctx->slot_count)
00561     {
00562         ctx->valid_count++;
00563     }
00564     ctx->next_sequence++;
00565
00566     return i2c_mem_store_meta_save(ctx);
00567 }
00568
00569 i2c_mem_store_status_t i2c_mem_store_read_message(i2c_mem_store_t *ctx,
00570     uint16_t index_from_latest,
00571     i2c_mem_store_log_record_t *out_record)
00572 {
00573     uint16_t slot;
00574     uint8_t rec[I2C_MEM_STORE_LOG_RECORD_SIZE];
00575     uint16_t crc_stored;
00576     uint16_t crc_calc;
00577     i2c_mem_store_status_t rc;
00578
00579     if ((ctx == NULL) || (!ctx->initialized) || (out_record == NULL))
00580     {
00581         return I2C_MEM_STORE_EINVAL;
00582     }
00583     if (ctx->slot_count == 0U)
00584     {
00585         return I2C_MEM_STORE_ENOTFOUND;
00586     }
00587     if (index_from_latest >= ctx->valid_count)
00588     {
00589         return I2C_MEM_STORE_ENOTFOUND;
00590     }
00591
00592     slot = (uint16_t)((ctx->write_slot + ctx->slot_count - 1U - index_from_latest) % ctx->slot_count);
00593     rc = i2c_mem_store_read_raw(ctx, i2c_mem_store_log_slot_addr(ctx, slot), rec, sizeof(rec));
00594     if (rc != I2C_MEM_STORE_OK)
00595     {
00596         return rc;
00597     }
00598
00599     if (i2c_mem_store_le16_read(&rec[0]) != I2C_MEM_STORE_LOG_MAGIC)
00600     {
00601         return I2C_MEM_STORE_ENOTFOUND;
00602     }
00603
00604     crc_stored = i2c_mem_store_le16_read(&rec[I2C_MEM_STORE_LOG_CRC_OFFSET]);
00605     crc_calc = i2c_mem_store_crc16_ccitt(rec, I2C_MEM_STORE_LOG_CRC_OFFSET);
00606     if (crc_stored != crc_calc)
00607     {
00608         return I2C_MEM_STORE_ECRC;
00609     }
00610
00611     memset(out_record, 0, sizeof(*out_record));

```

```

00612     out_record->sequence = i2c_mem_store_le32_read(&rec[4]);
00613     out_record->timestamp_ms = i2c_mem_store_le32_read(&rec[8]);
00614     out_record->rssi_dbm = (int16_t)i2c_mem_store_le16_read(&rec[12]);
00615     out_record->length = rec[2];
00616     if (out_record->length > I2C_MEM_STORE_LOG_PAYLOAD_MAX)
00617     {
00618         out_record->length = I2C_MEM_STORE_LOG_PAYLOAD_MAX;
00619     }
00620     memcpy(out_record->payload, &rec[I2C_MEM_STORE_LOG_HEADER_SIZE], out_record->length);
00621     return I2C_MEM_STORE_OK;
00622 }
00623
00624 i2c_mem_store_status_t i2c_mem_store_get_log_count(i2c_mem_store_t *ctx, uint16_t *out_count)
00625 {
00626     if ((ctx == NULL) || (!ctx->initialized) || (out_count == NULL))
00627     {
00628         return I2C_MEM_STORE_EINVAL;
00629     }
00630
00631     *out_count = ctx->valid_count;
00632     return I2C_MEM_STORE_OK;
00633 }
00634
00635 i2c_mem_store_status_t i2c_mem_store_secret_write(i2c_mem_store_t *ctx,
00636                                                    uint16_t slot_id,
00637                                                    const uint8_t *data,
00638                                                    uint8_t data_len)
00639 {
00640     uint8_t rec[I2C_MEM_STORE_SECRET_SLOT_SIZE];
00641     uint8_t plain[I2C_MEM_STORE_SECRET_PAYLOAD_MAX];
00642     uint32_t nonce;
00643     uint16_t crc;
00644     i2c_mem_store_status_t rc;
00645
00646     if ((ctx == NULL) || (!ctx->initialized) || ((data == NULL) && (data_len > 0)))
00647     {
00648         return I2C_MEM_STORE_EINVAL;
00649     }
00650     if ((slot_id >= ctx->secret_slot_count) || (data_len > I2C_MEM_STORE_SECRET_PAYLOAD_MAX))
00651     {
00652         return I2C_MEM_STORE_EOVERFLOW;
00653     }
00654
00655     memset(rec, 0xFF, sizeof(rec));
00656     memset(plain, 0, sizeof(plain));
00657     if (data_len > 0U)
00658     {
00659         memcpy(plain, data, data_len);
00660     }
00661
00662     nonce = ctx->secret_counter++;
00663     i2c_mem_store_xtea_ctr_crypt(plain, sizeof(plain), ctx->cfg.crypto_key, nonce);
00664
00665     i2c_mem_store_le16_write(&rec[0], I2C_MEM_STORE_SECRET_MAGIC);
00666     rec[2] = data_len;
00667     rec[3] = (uint8_t)slot_id;
00668     i2c_mem_store_le32_write(&rec[4], nonce);
00669     memcpy(&rec[I2C_MEM_STORE_SECRET_HEADER_SIZE], plain, sizeof(plain));
00670
00671     crc = i2c_mem_store_crc16_ccitt(rec, I2C_MEM_STORE_SECRET_CRC_OFFSET);
00672     i2c_mem_store_le16_write(&rec[I2C_MEM_STORE_SECRET_CRC_OFFSET], crc);
00673
00674     rc = i2c_mem_store_write_raw(ctx, i2c_mem_store_secret_slot_addr(ctx, slot_id), rec, sizeof(rec));
00675     if (rc != I2C_MEM_STORE_OK)
00676     {
00677         return rc;
00678     }
00679
00680     return i2c_mem_store_meta_save(ctx);
00681 }
00682
00683 i2c_mem_store_status_t i2c_mem_store_secret_read(i2c_mem_store_t *ctx,
00684                                                  uint16_t slot_id,
00685                                                  uint8_t *data_out,
00686                                                  uint8_t data_capacity,
00687                                                  uint8_t *data_len)
00688 {
00689     uint8_t rec[I2C_MEM_STORE_SECRET_SLOT_SIZE];
00690     uint8_t plain[I2C_MEM_STORE_SECRET_PAYLOAD_MAX];
00691     uint16_t crc_stored;
00692     uint16_t crc_calc;
00693     uint8_t len;
00694     uint32_t nonce;
00695     i2c_mem_store_status_t rc;
00696
00697     if ((ctx == NULL) || (!ctx->initialized) || (data_out == NULL) || (data_len == NULL))
00698     {

```

```

00699         return I2C_MEM_STORE_EINVAL;
00700     }
00701     if (slot_id >= ctx->secret_slot_count)
00702     {
00703         return I2C_MEM_STORE_EINVAL;
00704     }
00705
00706     rc = i2c_mem_store_read_raw(ctx, i2c_mem_store_secret_slot_addr(ctx, slot_id), rec, sizeof(rec));
00707     if (rc != I2C_MEM_STORE_OK)
00708     {
00709         return rc;
00710     }
00711
00712     if (i2c_mem_store_le16_read(&rec[0]) != I2C_MEM_STORE_SECRET_MAGIC)
00713     {
00714         return I2C_MEM_STORE_ENOTFOUND;
00715     }
00716
00717     crc_stored = i2c_mem_store_le16_read(&rec[I2C_MEM_STORE_SECRET_CRC_OFFSET]);
00718     crc_calc = i2c_mem_store_crc16_ccitt(rec, I2C_MEM_STORE_SECRET_CRC_OFFSET);
00719     if (crc_stored != crc_calc)
00720     {
00721         return I2C_MEM_STORE_ECRC;
00722     }
00723
00724     len = rec[2];
00725     if (len > I2C_MEM_STORE_SECRET_PAYLOAD_MAX)
00726     {
00727         return I2C_MEM_STORE_EPROTO;
00728     }
00729     if (len > data_capacity)
00730     {
00731         return I2C_MEM_STORE_EOVERFLOW;
00732     }
00733
00734     memcpy(plain, &rec[I2C_MEM_STORE_SECRET_HEADER_SIZE], sizeof(plain));
00735     nonce = i2c_mem_store_le32_read(&rec[4]);
00736     i2c_mem_store_xtea_ctr_crypt(plain, sizeof(plain), ctx->cfg.crypto_key, nonce);
00737
00738     if (len > 0U)
00739     {
00740         memcpy(data_out, plain, len);
00741     }
00742     *data_len = len;
00743     return I2C_MEM_STORE_OK;
00744 }
00745
00746 i2c_mem_store_status_t i2c_mem_store_secret_erase(i2c_mem_store_t *ctx, uint16_t slot_id)
00747 {
00748     uint8_t marker[2] = { 0xFFU, 0xFFU };
00749
00750     if ((ctx == NULL) || (!ctx->initialized) || (slot_id >= ctx->secret_slot_count))
00751     {
00752         return I2C_MEM_STORE_EINVAL;
00753     }
00754
00755     return i2c_mem_store_write_raw(ctx, i2c_mem_store_secret_slot_addr(ctx, slot_id), marker,
sizeof(marker));
00756 }
00757
00758 i2c_mem_store_status_t i2c_mem_store_trusted_device_write(i2c_mem_store_t *ctx,
uint16_t slot_id,
const i2c_mem_store_trusted_device_t
*trusted)
00761 {
00762     uint8_t buf[I2C_MEM_STORE_SECRET_PAYLOAD_MAX];
00763     uint8_t total_len;
00764
00765     if ((ctx == NULL) || (!ctx->initialized) || (trusted == NULL))
00766     {
00767         return I2C_MEM_STORE_EINVAL;
00768     }
00769     if ((trusted->id_len > I2C_MEM_STORE_TRUSTED_ID_MAX) || (trusted->code_len >
I2C_MEM_STORE_TRUSTED_CODE_MAX))
00770     {
00771         return I2C_MEM_STORE_EOVERFLOW;
00772     }
00773
00774     total_len = (uint8_t)(2U + trusted->id_len + trusted->code_len);
00775     if (total_len > I2C_MEM_STORE_SECRET_PAYLOAD_MAX)
00776     {
00777         return I2C_MEM_STORE_EOVERFLOW;
00778     }
00779
00780     buf[0] = trusted->id_len;
00781     buf[1] = trusted->code_len;
00782     memcpy(&buf[2], trusted->id, trusted->id_len);

```

```

00783     memcpy(&buf[2U + trusted->id_len], trusted->code, trusted->code_len);
00784     return i2c_mem_store_secret_write(ctx, slot_id, buf, total_len);
00785 }
00786
00787 i2c_mem_store_status_t i2c_mem_store_trusted_device_read(i2c_mem_store_t *ctx,
00788                                                         uint16_t slot_id,
00789                                                         i2c_mem_store_trusted_device_t *trusted)
00790 {
00791     uint8_t buf[I2C_MEM_STORE_SECRET_PAYLOAD_MAX];
00792     uint8_t len = 0U;
00793     i2c_mem_store_status_t rc;
00794     uint8_t id_len;
00795     uint8_t code_len;
00796
00797     if ((ctx == NULL) || (!ctx->initialized) || (trusted == NULL))
00798     {
00799         return I2C_MEM_STORE_EINVAL;
00800     }
00801
00802     rc = i2c_mem_store_secret_read(ctx, slot_id, buf, sizeof(buf), &len);
00803     if (rc != I2C_MEM_STORE_OK)
00804     {
00805         return rc;
00806     }
00807     if (len < 2U)
00808     {
00809         return I2C_MEM_STORE_EPROTO;
00810     }
00811
00812     id_len = buf[0];
00813     code_len = buf[1];
00814     if ((id_len > I2C_MEM_STORE_TRUSTED_ID_MAX) || (code_len > I2C_MEM_STORE_TRUSTED_CODE_MAX))
00815     {
00816         return I2C_MEM_STORE_EPROTO;
00817     }
00818     if ((uint8_t)(2U + id_len + code_len) != len)
00819     {
00820         return I2C_MEM_STORE_EPROTO;
00821     }
00822
00823     memset(trusted, 0, sizeof(*trusted));
00824     trusted->id_len = id_len;
00825     trusted->code_len = code_len;
00826     memcpy(trusted->id, &buf[2], id_len);
00827     memcpy(trusted->code, &buf[2U + id_len], code_len);
00828     return I2C_MEM_STORE_OK;
00829 }

```

12.18 Dokumentacja pliku

Core/App/i2c_mem_store_lib/i2c_mem_store.h

I2C NVM storage library for received messages and encrypted service codes.

```

#include "stm32u5xx_hal.h"
#include <stdbool.h>
#include <stdint.h>

```

Struktury danych

- struct [i2c_mem_store_log_record_t](#)
One logged message record returned to caller.
- struct [i2c_mem_store_trusted_device_t](#)
Trusted-device object stored in encrypted partition.
- struct [i2c_mem_store_cfg_t](#)
Runtime configuration for external I2C memory.
- struct [i2c_mem_store_t](#)
Driver context.

Definicje

Defaults

- `#define I2C_MEM_STORE_I2C_ADDR_DEFAULT 0x50U`
- `#define I2C_MEM_STORE_TOTAL_BYTES_DEFAULT 128U`
- `#define I2C_MEM_STORE_PAGE_BYTES_DEFAULT 8U`
- `#define I2C_MEM_STORE_WRITE_CYCLE_MS_DEFAULT 5U`
- `#define I2C_MEM_STORE_SECRET_AREA_BYTES_DEFAULT 24U`
- `#define I2C_MEM_STORE_LOG_RECORD_SIZE 24U`
- `#define I2C_MEM_STORE_LOG_PAYLOAD_MAX 6U`
- `#define I2C_MEM_STORE_SECRET_SLOT_SIZE 24U`
- `#define I2C_MEM_STORE_SECRET_PAYLOAD_MAX 14U`
- `#define I2C_MEM_STORE_TRUSTED_ID_MAX 6U`
- `#define I2C_MEM_STORE_TRUSTED_CODE_MAX 8U`

Wyliczenia

- `enum i2c_mem_store_status_t {`
`I2C_MEM_STORE_OK = 0 , I2C_MEM_STORE_EINVAL , I2C_MEM_STORE_ESTATE , I2C_MEM_STORE_EHAL`
`,`
`I2C_MEM_STORE_ETIMEOUT , I2C_MEM_STORE_ECRC , I2C_MEM_STORE_EPROTO , I2C_MEM_STORE_ENOTFOUN`
`,`
`I2C_MEM_STORE_EOVERFLOW , I2C_MEM_STORE_EFULL }`

Return codes for I2C memory store operations.

Funkcje

- `void i2c_mem_store_default_cfg (i2c_mem_store_cfg_t *cfg, I2C_HandleTypeDef *hi2c)`
Fill configuration with safe defaults for I2C1 memory.
- `void i2c_mem_store_default_cfg_m24c01r (i2c_mem_store_cfg_t *cfg, I2C_HandleTypeDef *hi2c)`
Fill configuration specifically for ST M24C01-R (1 Kbit EEPROM).
- `i2c_mem_store_status_t i2c_mem_store_init (i2c_mem_store_t *ctx, const i2c_mem_store_cfg_t *cfg, bool format_if_needed)`
Initialize library, load metadata and optionally format on first use.
- `i2c_mem_store_status_t i2c_mem_store_format (i2c_mem_store_t *ctx)`
Format metadata and reset log/secret allocators.
- `i2c_mem_store_status_t i2c_mem_store_append_message (i2c_mem_store_t *ctx, int16_t rssi_dbm, const uint8_t *payload, uint8_t payload_len)`
Append one received message to persistent ring log.
- `i2c_mem_store_status_t i2c_mem_store_read_message (i2c_mem_store_t *ctx, uint16_t index_from_latest, i2c_mem_store_log_record_t *out_record)`
Read one message from log (0 = latest, 1 = previous, ...).
- `i2c_mem_store_status_t i2c_mem_store_get_log_count (i2c_mem_store_t *ctx, uint16_t *out_count)`
Get number of valid records currently available in ring log.
- `i2c_mem_store_status_t i2c_mem_store_secret_write (i2c_mem_store_t *ctx, uint16_t slot_id, const uint8_t *data, uint8_t data_len)`
Store encrypted arbitrary payload in secret partition slot.
- `i2c_mem_store_status_t i2c_mem_store_secret_read (i2c_mem_store_t *ctx, uint16_t slot_id, uint8_t *data_out, uint8_t data_capacity, uint8_t *data_len)`
Read and decrypt payload from secret partition slot.
- `i2c_mem_store_status_t i2c_mem_store_secret_erase (i2c_mem_store_t *ctx, uint16_t slot_id)`
Erase one secret slot (invalidate entry marker).
- `i2c_mem_store_status_t i2c_mem_store_trusted_device_write (i2c_mem_store_t *ctx, uint16_t slot_id, const i2c_mem_store_trusted_device_t *trusted)`
Store trusted-device descriptor in encrypted secret slot.
- `i2c_mem_store_status_t i2c_mem_store_trusted_device_read (i2c_mem_store_t *ctx, uint16_t slot_id, i2c_mem_store_trusted_device_t *trusted)`
Read trusted-device descriptor from encrypted secret slot.

12.18.1 Opis szczegółowy

I2C NVM storage library for received messages and encrypted service codes.

This library targets external I2C memory (EEPROM/FRAM family) connected to I2C1. It provides:

- append-only ring log for received radio messages,
- dedicated encrypted partition for trusted-device/service codes.

Encryption uses XTEA-CTR software cipher with a 128-bit key from configuration. For production security, derive/load this key from TPM-protected material.

Definicja w pliku [i2c_mem_store.h](#).

12.18.2 Dokumentacja definicji

12.18.2.1 I2C_MEM_STORE_I2C_ADDR_DEFAULT

```
#define I2C_MEM_STORE_I2C_ADDR_DEFAULT 0x50U
```

Definicja w linii 29 pliku [i2c_mem_store.h](#).

12.18.2.2 I2C_MEM_STORE_LOG_PAYLOAD_MAX

```
#define I2C_MEM_STORE_LOG_PAYLOAD_MAX 6U
```

Definicja w linii 35 pliku [i2c_mem_store.h](#).

12.18.2.3 I2C_MEM_STORE_LOG_RECORD_SIZE

```
#define I2C_MEM_STORE_LOG_RECORD_SIZE 24U
```

Definicja w linii 34 pliku [i2c_mem_store.h](#).

12.18.2.4 I2C_MEM_STORE_PAGE_BYTES_DEFAULT

```
#define I2C_MEM_STORE_PAGE_BYTES_DEFAULT 8U
```

Definicja w linii 31 pliku [i2c_mem_store.h](#).

12.18.2.5 I2C_MEM_STORE_SECRET_AREA_BYTES_DEFAULT

```
#define I2C_MEM_STORE_SECRET_AREA_BYTES_DEFAULT 24U
```

Definicja w linii 33 pliku [i2c_mem_store.h](#).

12.18.2.6 I2C_MEM_STORE_SECRET_PAYLOAD_MAX

```
#define I2C_MEM_STORE_SECRET_PAYLOAD_MAX 14U
```

Definicja w linii 37 pliku [i2c_mem_store.h](#).

12.18.2.7 I2C_MEM_STORE_SECRET_SLOT_SIZE

```
#define I2C_MEM_STORE_SECRET_SLOT_SIZE 24U
```

Definicja w linii 36 pliku [i2c_mem_store.h](#).

12.18.2.8 I2C_MEM_STORE_TOTAL_BYTES_DEFAULT

```
#define I2C_MEM_STORE_TOTAL_BYTES_DEFAULT 128U
```

Definicja w linii 30 pliku [i2c_mem_store.h](#).

12.18.2.9 I2C_MEM_STORE_TRUSTED_CODE_MAX

```
#define I2C_MEM_STORE_TRUSTED_CODE_MAX 8U
```

Definicja w linii 39 pliku [i2c_mem_store.h](#).

12.18.2.10 I2C_MEM_STORE_TRUSTED_ID_MAX

```
#define I2C_MEM_STORE_TRUSTED_ID_MAX 6U
```

Definicja w linii 38 pliku [i2c_mem_store.h](#).

12.18.2.11 I2C_MEM_STORE_WRITE_CYCLE_MS_DEFAULT

```
#define I2C_MEM_STORE_WRITE_CYCLE_MS_DEFAULT 5U
```

Definicja w linii 32 pliku [i2c_mem_store.h](#).

12.18.3 Dokumentacja typów wyliczanych**12.18.3.1 i2c_mem_store_status_t**

```
enum i2c_mem_store_status_t
```

Return codes for I2C memory store operations.

Wartości wyliczeń

I2C_MEM_STORE_OK	Success.
I2C_MEM_STORE_EINVAL	Invalid argument.
I2C_MEM_STORE_ESTATE	Invalid state or uninitialized context.
I2C_MEM_STORE_EHAL	HAL I2C operation failed.
I2C_MEM_STORE_ETIMEOUT	Timeout/error interpreted as timeout.
I2C_MEM_STORE_ECRC	CRC mismatch in persisted object.
I2C_MEM_STORE_EPROTO	Invalid record/protocol format in NVM.
I2C_MEM_STORE_ENOTFOUND	Requested object not found/empty.
I2C_MEM_STORE_EOVERFLOW	Buffer too small or payload too large.
I2C_MEM_STORE_EFULL	Partition full or layout cannot fit.

Definicja w linii 45 pliku [i2c_mem_store.h](#).

12.18.4 Dokumentacja funkcji**12.18.4.1 i2c_mem_store_append_message()**

```
i2c_mem_store_status_t i2c_mem_store_append_message (
    i2c_mem_store_t * ctx,
    int16_t rssi_dbm,
    const uint8_t * payload,
    uint8_t payload_len)
```

Append one received message to persistent ring log.

Parametry

<i>ctx</i>	Context object.
<i>rssi_dbm</i>	RSSI associated with payload.
<i>payload</i>	Message bytes.
<i>payload_len</i>	Message length.

Zwraca

Status code.

Definicja w linii 510 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_crc16_ccitt()`, `I2C_MEM_STORE_EFULL`, `I2C_MEM_STORE_EINVAL`, `i2c_mem_store_le16_write()`, `i2c_mem_store_le32_write()`, `i2c_mem_store_log_slot_addr()`, `i2c_mem_store_meta_save()`, `I2C_MEM_STORE_OK`, `i2c_mem_store_write_raw()`, `i2c_mem_store_t::initialized`, `i2c_mem_store_t::next_sequence`, `i2c_mem_store_t::slot_count`, `i2c_mem_store_t::valid_count` i `i2c_mem_store_t::write_slot`.

Odwołania w `radio_test_store_packet_in_i2c_memory()`.

12.18.4.2 i2c_mem_store_default_cfg()

```
void i2c_mem_store_default_cfg (
    i2c_mem_store_cfg_t * cfg,
    I2C_HandleTypeDef * hi2c)
```

Fill configuration with safe defaults for I2C1 memory.

Parametry

<i>cfg</i>	[out] Configuration object.
<i>hi2c</i>	I2C handle (typically <code>&hi2c1</code>).

Definicja w linii 409 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_cfg_t::crypto_key`, `i2c_mem_store_cfg_t::hi2c`, `i2c_mem_store_cfg_t::i2c_addr_7bit`, `i2c_mem_store_cfg_t::mem_addr_size`, `i2c_mem_store_cfg_t::page_bytes`, `i2c_mem_store_cfg_t::secret_area_bytes`, `i2c_mem_store_cfg_t::total_bytes` i `i2c_mem_store_cfg_t::write_cycle_ms`.

Odwołania w `i2c_mem_store_default_cfg_m24c01r()`.

12.18.4.3 i2c_mem_store_default_cfg_m24c01r()

```
void i2c_mem_store_default_cfg_m24c01r (
    i2c_mem_store_cfg_t * cfg,
    I2C_HandleTypeDef * hi2c)
```

Fill configuration specifically for ST M24C01-R (1 Kbit EEPROM).

Parametry

<i>cfg</i>	[out] Configuration object.
<i>hi2c</i>	I2C handle (typically <code>&hi2c1</code>).

Definicja w linii 440 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_cfg_t::i2c_addr_7bit`, `i2c_mem_store_default_cfg()`, `i2c_mem_store_cfg_t::mem_addr_size`, `i2c_mem_store_cfg_t::page_bytes`, `i2c_mem_store_cfg_t::secret_area_bytes`, `i2c_mem_store_cfg_t::total_bytes` i `i2c_mem_store_cfg_t::write_cycle_ms`.

Odwołania w `radio_test_demo_init()`.

12.18.4.4 i2c_mem_store_format()

```
i2c_mem_store_status_t i2c_mem_store_format (
    i2c_mem_store_t * ctx)
```

Format metadata and reset log/secret allocators.

Parametry

<i>ctx</i>	Context object.
------------	-----------------

Zwraca

Status code.

Definicja w linii 496 pliku `i2c_mem_store.c`.

Odwołuje się do `I2C_MEM_STORE_ESTATE`, `i2c_mem_store_meta_save()`, `i2c_mem_store_t::initialized`, `i2c_mem_store_t::next_sequence`, `i2c_mem_store_t::secret_counter`, `i2c_mem_store_t::valid_count` i `i2c_mem_store_t::write_slot`.
Odwołania w `i2c_mem_store_init()`.

12.18.4.5 i2c_mem_store_get_log_count()

```
i2c_mem_store_status_t i2c_mem_store_get_log_count (
    i2c_mem_store_t * ctx,
    uint16_t * out_count)
```

Get number of valid records currently available in ring log.

Parametry

<i>ctx</i>	Context object.
<i>out_count</i>	[out] Record count.

Zwraca

Status code.

Definicja w linii 624 pliku `i2c_mem_store.c`.

Odwołuje się do `I2C_MEM_STORE_EINVAL`, `I2C_MEM_STORE_OK`, `i2c_mem_store_t::initialized` i `i2c_mem_store_t::valid_count`.

12.18.4.6 i2c_mem_store_init()

```
i2c_mem_store_status_t i2c_mem_store_init (
    i2c_mem_store_t * ctx,
    const i2c_mem_store_cfg_t * cfg,
    bool format_if_needed)
```

Initialize library, load metadata and optionally format on first use.

Parametry

<i>ctx</i>	Context object.
<i>cfg</i>	Runtime configuration.
<i>format_if_needed</i>	If <code>true</code> , metadata is created when invalid/missing.

Zwraca

Status code.

Definicja w linii 457 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_t::cfg`, `I2C_MEM_STORE_EINVAL`, `i2c_mem_store_format()`, `i2c_mem_store_layout_valid()`, `i2c_mem_store_meta_load()`, `I2C_MEM_STORE_OK`, `i2c_mem_store_t::initialized`, `i2c_mem_store_t::secret_slot_count` i `i2c_mem_store_t::slot_count`.
Odwołania w `radio_test_demo_init()`.

12.18.4.7 i2c_mem_store_read_message()

```
i2c_mem_store_status_t i2c_mem_store_read_message (
    i2c_mem_store_t * ctx,
```

```
uint16_t index_from_latest,
i2c_mem_store_log_record_t * out_record)
```

Read one message from log (0 = latest, 1 = previous, ...).

Parametry

<i>ctx</i>	Context object.
<i>index_from_latest</i>	Relative index from latest entry.
<i>out_record</i>	[out] Decoded record.

Zwraca

Status code.

Definicja w linii 569 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_crc16_ccitt()`, `I2C_MEM_STORE_ECRC`, `I2C_MEM_STORE_EINVAL`, `I2C_MEM_STORE_ENOTFOUND`, `i2c_mem_store_le16_read()`, `i2c_mem_store_le32_read()`, `i2c_mem_store_log_slot_addr()`, `I2C_MEM_STORE_OK`, `i2c_mem_store_read_raw()`, `i2c_mem_store_t::initialized`, `i2c_mem_store_log_record_t::length`, `i2c_mem_store_log_record_t::payload`, `i2c_mem_store_log_record_t::rssi_dbm`, `i2c_mem_store_log_record_t::sequence`, `i2c_mem_store_t::slot_count`, `i2c_mem_store_log_record_t::timestamp_ms`, `i2c_mem_store_t::valid_count` i `i2c_mem_store_t::write_slot`.

12.18.4.8 i2c_mem_store_secret_erase()

```
i2c_mem_store_status_t i2c_mem_store_secret_erase (
    i2c_mem_store_t * ctx,
    uint16_t slot_id)
```

Erase one secret slot (invalidate entry marker).

Parametry

<i>ctx</i>	Context object.
<i>slot_id</i>	Slot index in secret partition.

Zwraca

Status code.

Definicja w linii 746 pliku `i2c_mem_store.c`.

Odwołuje się do `I2C_MEM_STORE_EINVAL`, `i2c_mem_store_secret_slot_addr()`, `i2c_mem_store_write_raw()`, `i2c_mem_store_t::initialized` i `i2c_mem_store_t::secret_slot_count`.

12.18.4.9 i2c_mem_store_secret_read()

```
i2c_mem_store_status_t i2c_mem_store_secret_read (
    i2c_mem_store_t * ctx,
    uint16_t slot_id,
    uint8_t * data_out,
    uint8_t data_capacity,
    uint8_t * data_len)
```

Read and decrypt payload from secret partition slot.

Parametry

<i>ctx</i>	Context object.
<i>slot_id</i>	Slot index in secret partition.
<i>data_out</i>	Output buffer for decrypted payload.
<i>data_capacity</i>	Output buffer capacity.
<i>data_len</i>	[out] Returned payload length.

Zwraca

Status code.

Definicja w linii 683 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_t::cfg`, `i2c_mem_store_cfg_t::crypto_key`, `i2c_mem_store_crc16_ccitt()`, `I2C_MEM_STORE_ECRC`, `I2C_MEM_STORE_EINVAL`, `I2C_MEM_STORE_ENOTFOUND`, `I2C_MEM_STORE_EOVERFLOW`, `I2C_MEM_STORE_EPROTO`, `i2c_mem_store_le16_read()`, `i2c_mem_store_le32_read()`, `I2C_MEM_STORE_OK`, `i2c_mem_store_read_raw()`, `i2c_mem_store_secret_slot_addr()`, `i2c_mem_store_xtea_ctr_crypt()`, `i2c_mem_store_t::initialized` i `i2c_mem_store_t::secret_slot_count`.

Odwołania w `i2c_mem_store_trusted_device_read()`.

12.18.4.10 i2c_mem_store_secret_write()

```
i2c_mem_store_status_t i2c_mem_store_secret_write (
    i2c_mem_store_t * ctx,
    uint16_t slot_id,
    const uint8_t * data,
    uint8_t data_len)
```

Store encrypted arbitrary payload in secret partition slot.

Parametry

<i>ctx</i>	Context object.
<i>slot_id</i>	Slot index in secret partition.
<i>data</i>	Plain payload.
<i>data_len</i>	Payload length.

Zwraca

Status code.

Definicja w linii 635 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_t::cfg`, `i2c_mem_store_cfg_t::crypto_key`, `i2c_mem_store_crc16_ccitt()`, `I2C_MEM_STORE_EINVAL`, `I2C_MEM_STORE_EOVERFLOW`, `i2c_mem_store_le16_write()`, `i2c_mem_store_le32_write()`, `i2c_mem_store_meta_save()`, `I2C_MEM_STORE_OK`, `i2c_mem_store_secret_slot_addr()`, `i2c_mem_store_write_raw()`, `i2c_mem_store_xtea_ctr_crypt()`, `i2c_mem_store_t::initialized`, `i2c_mem_store_t::secret_counter` i `i2c_mem_store_t::secret_slot_count`.
Odwołania w `i2c_mem_store_trusted_device_write()`.

12.18.4.11 i2c_mem_store_trusted_device_read()

```
i2c_mem_store_status_t i2c_mem_store_trusted_device_read (
    i2c_mem_store_t * ctx,
    uint16_t slot_id,
    i2c_mem_store_trusted_device_t * trusted)
```

Read trusted-device descriptor from encrypted secret slot.

Parametry

<i>ctx</i>	Context object.
<i>slot</i> ↔ <i>_id</i>	Slot index.
<i>trusted</i>	[out] Trusted-device object.

Zwraca

Status code.

Definicja w linii 787 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_trusted_device_t::code`, `i2c_mem_store_trusted_device_t::code_len`, `I2C_MEM_STORE_EINVAL`, `I2C_MEM_STORE_EPROTO`, `I2C_MEM_STORE_OK`, `i2c_mem_store_secret_read()`, `i2c_mem_store_trusted_device_t::id`, `i2c_mem_store_trusted_device_t::id_len` i `i2c_mem_store_t::initialized`.

12.18.4.12 i2c_mem_store_trusted_device_write()

```
i2c_mem_store_status_t i2c_mem_store_trusted_device_write (
    i2c_mem_store_t * ctx,
    uint16_t slot_id,
    const i2c_mem_store_trusted_device_t * trusted)
```

Store trusted-device descriptor in encrypted secret slot.

Parametry

<i>ctx</i>	Context object.
<i>slot</i> ↔ <i>_id</i>	Slot index.
<i>trusted</i>	Trusted-device object.

Zwraca

Status code.

Definicja w linii 758 pliku `i2c_mem_store.c`.

Odwołuje się do `i2c_mem_store_trusted_device_t::code`, `i2c_mem_store_trusted_device_t::code_len`, `I2C_MEM_STORE_EINVAL`, `I2C_MEM_STORE_EOVERFLOW`, `i2c_mem_store_secret_write()`, `i2c_mem_store_trusted_device_t::id`, `i2c_mem_store_trusted_device_t::id_len` i `i2c_mem_store_t::initialized`.

12.19 i2c_mem_store.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00013
00014 #ifndef APP_I2C_MEM_STORE_LIB_I2C_MEM_STORE_H_
00015 #define APP_I2C_MEM_STORE_LIB_I2C_MEM_STORE_H_
00016
00017 #include "stm32u5xx_hal.h"
00018
00019 #include <stdbool.h>
00020 #include <stdint.h>
00021
00022 #ifdef __cplusplus
00023 extern "C" {
00024 #endif
00025
00029 #define I2C_MEM_STORE_I2C_ADDR_DEFAULT      0x50U
00030 #define I2C_MEM_STORE_TOTAL_BYTES_DEFAULT   128U
00031 #define I2C_MEM_STORE_PAGE_BYTES_DEFAULT    8U
00032 #define I2C_MEM_STORE_WRITE_CYCLE_MS_DEFAULT 5U
```



```

00033 #define I2C_MEM_STORE_SECRET_AREA_BYTES_DEFAULT    24U
00034 #define I2C_MEM_STORE_LOG_RECORD_SIZE                24U
00035 #define I2C_MEM_STORE_LOG_PAYLOAD_MAX                6U
00036 #define I2C_MEM_STORE_SECRET_SLOT_SIZE              24U
00037 #define I2C_MEM_STORE_SECRET_PAYLOAD_MAX            14U
00038 #define I2C_MEM_STORE_TRUSTED_ID_MAX                 6U
00039 #define I2C_MEM_STORE_TRUSTED_CODE_MAX               8U
00041
00045 typedef enum
00046 {
00047     I2C_MEM_STORE_OK = 0,
00048     I2C_MEM_STORE_EINVAL,
00049     I2C_MEM_STORE_ESTATE,
00050     I2C_MEM_STORE_EHAL,
00051     I2C_MEM_STORE_ETIMEOUT,
00052     I2C_MEM_STORE_ECRC,
00053     I2C_MEM_STORE_EPROTO,
00054     I2C_MEM_STORE_ENOTFOUND,
00055     I2C_MEM_STORE_EOVERFLOW,
00056     I2C_MEM_STORE_EFULL
00057 } i2c_mem_store_status_t;
00058
00062 typedef struct
00063 {
00064     uint32_t sequence;
00065     uint32_t timestamp_ms;
00066     int16_t rssi_dbm;
00067     uint8_t length;
00068     uint8_t payload[I2C_MEM_STORE_LOG_PAYLOAD_MAX];
00069 } i2c_mem_store_log_record_t;
00070
00074 typedef struct
00075 {
00076     uint8_t id_len;
00077     uint8_t code_len;
00078     uint8_t id[I2C_MEM_STORE_TRUSTED_ID_MAX];
00079     uint8_t code[I2C_MEM_STORE_TRUSTED_CODE_MAX];
00080 } i2c_mem_store_trusted_device_t;
00081
00085 typedef struct
00086 {
00087     I2C_HandleTypeDef *hi2c;
00088     uint16_t i2c_addr_7bit;
00089     uint16_t mem_addr_size;
00090     uint32_t total_bytes;
00091     uint16_t page_bytes;
00092     uint16_t write_cycle_ms;
00093     uint32_t secret_area_bytes;
00094     uint8_t crypto_key[16];
00095 } i2c_mem_store_cfg_t;
00096
00100 typedef struct
00101 {
00102     i2c_mem_store_cfg_t cfg;
00103     uint16_t slot_count;
00104     uint16_t secret_slot_count;
00105     uint16_t write_slot;
00106     uint16_t valid_count;
00107     uint32_t next_sequence;
00108     uint32_t secret_counter;
00109     bool initialized;
00110 } i2c_mem_store_t;
00111
00117 void i2c_mem_store_default_cfg(i2c_mem_store_cfg_t *cfg, I2C_HandleTypeDef *hi2c);
00118
00124 void i2c_mem_store_default_cfg_m24c01r(i2c_mem_store_cfg_t *cfg, I2C_HandleTypeDef *hi2c);
00125
00133 i2c_mem_store_status_t i2c_mem_store_init(i2c_mem_store_t *ctx,
00134                                           const i2c_mem_store_cfg_t *cfg,
00135                                           bool format_if_needed);
00136
00142 i2c_mem_store_status_t i2c_mem_store_format(i2c_mem_store_t *ctx);
00143
00152 i2c_mem_store_status_t i2c_mem_store_append_message(i2c_mem_store_t *ctx,
00153                                                      int16_t rssi_dbm,
00154                                                      const uint8_t *payload,
00155                                                      uint8_t payload_len);
00156
00164 i2c_mem_store_status_t i2c_mem_store_read_message(i2c_mem_store_t *ctx,
00165                                                    uint16_t index_from_latest,
00166                                                    i2c_mem_store_log_record_t *out_record);
00167
00174 i2c_mem_store_status_t i2c_mem_store_get_log_count(i2c_mem_store_t *ctx, uint16_t *out_count);
00175
00184 i2c_mem_store_status_t i2c_mem_store_secret_write(i2c_mem_store_t *ctx,
00185                                                    uint16_t slot_id,
00186                                                    const uint8_t *data,

```

```

00187             uint8_t data_len);
00188
00198 i2c_mem_store_status_t i2c_mem_store_secret_read(i2c_mem_store_t *ctx,
00199             uint16_t slot_id,
00200             uint8_t *data_out,
00201             uint8_t data_capacity,
00202             uint8_t *data_len);
00203
00210 i2c_mem_store_status_t i2c_mem_store_secret_erase(i2c_mem_store_t *ctx, uint16_t slot_id);
00211
00219 i2c_mem_store_status_t i2c_mem_store_trusted_device_write(i2c_mem_store_t *ctx,
00220             uint16_t slot_id,
00221             const i2c_mem_store_trusted_device_t
00222             *trusted);
00223
00230 i2c_mem_store_status_t i2c_mem_store_trusted_device_read(i2c_mem_store_t *ctx,
00231             uint16_t slot_id,
00232             i2c_mem_store_trusted_device_t *trusted);
00233
00234 #ifdef __cplusplus
00235 }
00236 #endif
00237
00238 #endif /* APP_I2C_MEM_STORE_LIB_I2C_MEM_STORE_H */

```

12.20 Dokumentacja pliku Core/App/laviet_crypto.c

Portable AES-CTR and HMAC-SHA256 backend for LAVIET_FRAME_V1.

```

#include "laviet_crypto.h"
#include "cmsis_os2.h"
#include "stm32u5xx_hal.h"
#include <stdio.h>
#include <string.h>

```

Struktury danych

- struct [laviet_sha256_ctx_t](#)

Definicje

- #define [LAVIET_CRYPTO_TIMEOUT_MS](#) 1000U
- #define [LAVIET_TEST_FIXED_FRAME_MAC](#) 0U
- #define [LAVIET_TEST_FIXED_FRAME_MAC_BYTE](#) 0x01U

Funkcje

- static const char * [laviet_hal_status_text](#) (HAL_StatusTypeDef status)
Convert a HAL status code to a short diagnostic string.
- static bool [laviet_crypto_log_mismatch](#) (const char *stage, const uint8_t *expected, const uint8_t *actual, uint16_t len)
Print the first byte mismatch found during crypto self-test comparison.
- static uint32_t [laviet_rot32](#) (uint32_t value, uint8_t bits)
Rotate a 32-bit word right for SHA-256 round functions.
- static void [laviet_sha256_transform](#) ([laviet_sha256_ctx_t](#) *ctx, const uint8_t data[64])
Process one 512-bit SHA-256 block into the running digest state.
- static void [laviet_sha256_init](#) ([laviet_sha256_ctx_t](#) *ctx)
Initialize the software SHA-256 context with the standard IV.
- static void [laviet_sha256_update](#) ([laviet_sha256_ctx_t](#) *ctx, const uint8_t *data, uint16_t len)
Feed bytes into the software SHA-256 streaming context.
- static void [laviet_sha256_final](#) ([laviet_sha256_ctx_t](#) *ctx, uint8_t hash[LAVIET_SHA256_LEN])
Finalize SHA-256 padding and write the 32-byte digest.
- static uint8_t [laviet_aes_xtime](#) (uint8_t x)

- Internal helper: *laviet_aes_xtime*.

 - static void **laviet_aes_key_expansion** (const uint8_t key[16], uint8_t round_key[176])

Expand a 128-bit AES key into the AES-128 round-key schedule.
- static void **laviet_aes_add_round_key** (uint8_t state[16], const uint8_t *round_key)

Internal helper: *laviet_aes_add_round_key*.
- static void **laviet_aes_sub_bytes** (uint8_t state[16])

Internal helper: *laviet_aes_sub_bytes*.
- static void **laviet_aes_shift_rows** (uint8_t state[16])

Internal helper: *laviet_aes_shift_rows*.
- static void **laviet_aes_mix_columns** (uint8_t state[16])

Internal helper: *laviet_aes_mix_columns*.
- static void **laviet_aes_encrypt_block** (const uint8_t in[16], uint8_t out[16], const uint8_t round_key[176])

Encrypt one 16-byte block with the software AES-128 implementation.
- static void **laviet_make_counter_block** (const **laviet_frame_t** *frame, uint16_t block_index, uint8_t out[16])

Build the LAVIET-specific AES-CTR counter block from frame identity fields.
- static bool **laviet_crypto_lock** (void)

Internal helper: *laviet_crypto_lock*.
- static void **laviet_crypto_unlock** (void)

Internal helper: *laviet_crypto_unlock*.
- static bool **laviet_hmac_sha256_sw** (const uint8_t *key, uint16_t key_len, const uint8_t *data, uint16_t data_len, uint8_t out[LAVIET_SHA256_LEN])

Software HMAC-SHA256 implementation used as the portable fallback.
- static bool **laviet_hmac_sha256_hw_locked** (const uint8_t *key, uint16_t key_len, const uint8_t *data, uint16_t data_len, uint8_t out[LAVIET_SHA256_LEN])

Hardware HASH HMAC-SHA256 path; caller must hold the crypto mutex.
- bool **laviet_frame_hmac_sha256** (const **laviet_frame_t** *frame, const uint8_t key[LAVIET_HMAC_KEY_LEN], uint8_t out[LAVIET_MAC_TAG_LEN])

Calculate the LAVIET frame MAC over the serialized MAC input.
- static bool **laviet_aes_ctr_crypt_sw** (uint8_t *data, uint8_t len, const uint8_t key[LAVIET_AES_KEY_LEN], const **laviet_frame_t** *frame)

Software AES-CTR encrypt/decrypt routine for arbitrary LAVIET payload length.
- static bool **laviet_aes_ctr_crypt_hw_locked** (uint8_t *data, uint8_t len, const uint8_t key[LAVIET_AES_KEY_LEN], const **laviet_frame_t** *frame)

Hardware AES-CTR path for one LAVIET payload block; caller must hold the crypto mutex.
- static bool **laviet_crypto_self_test_locked** (void)

Compare hardware crypto output against software reference vectors under the crypto mutex.
- bool **laviet_crypto_init** (void)

Initialize crypto backends and run the optional hardware self-test.
- bool **laviet_hmac_sha256** (const uint8_t *key, uint16_t key_len, const uint8_t *data, uint16_t data_len, uint8_t out[LAVIET_SHA256_LEN])

Calculate HMAC-SHA256 over an arbitrary buffer.
- bool **laviet_aes_ctr_crypt** (uint8_t *data, uint8_t len, const uint8_t key[LAVIET_AES_KEY_LEN], const **laviet_frame_t** *frame)

Encrypt or decrypt a LAVIET payload in-place with AES-CTR.
- bool **laviet_mac_equal** (const uint8_t a[LAVIET_MAC_TAG_LEN], const uint8_t b[LAVIET_MAC_TAG_LEN])

Compare two MAC tags without data-dependent early exit.
- void **laviet_secure_zero** (void *ptr, size_t len)

Clear sensitive memory through a volatile byte pointer.

Zmienne

- static osMutexId_t [s_crypto_mutex](#) = NULL
- static bool [s_crypto_hw_ready](#) = false
- static const uint32_t [s_sha256_k](#) [64]
- static const uint8_t [s_aes_sbox](#) [256]
- static const uint8_t [s_aes_rcon](#) [11]

12.20.1 Opis szczegółowy

Portable AES-CTR and HMAC-SHA256 backend for LAVIET_FRAME_V1.
Definicja w pliku [laviet_crypto.c](#).

12.20.2 Dokumentacja definicji

12.20.2.1 LAVIET_CRYPTO_TIMEOUT_MS

```
#define LAVIET_CRYPTO_TIMEOUT_MS 1000U
```

Definicja w linii 21 pliku [laviet_crypto.c](#).

12.20.2.2 LAVIET_TEST_FIXED_FRAME_MAC

```
#define LAVIET_TEST_FIXED_FRAME_MAC 0U
```

Definicja w linii 22 pliku [laviet_crypto.c](#).

12.20.2.3 LAVIET_TEST_FIXED_FRAME_MAC_BYTE

```
#define LAVIET_TEST_FIXED_FRAME_MAC_BYTE 0x01U
```

Definicja w linii 23 pliku [laviet_crypto.c](#).

12.20.3 Dokumentacja funkcji

12.20.3.1 laviet_aes_add_round_key()

```
void laviet_aes_add_round_key (
    uint8_t state[16],
    const uint8_t * round_key) [static]
```

Internal helper: [laviet_aes_add_round_key](#).

Definicja w linii 469 pliku [laviet_crypto.c](#).

Odwołania w [laviet_aes_encrypt_block\(\)](#).

12.20.3.2 laviet_aes_ctr_crypt()

```
bool laviet_aes_ctr_crypt (
    uint8_t * data,
    uint8_t len,
    const uint8_t key[LAVIET_AES_KEY_LEN],
    const laviet_frame_t * frame)
```

Encrypt or decrypt a LAVIET payload in-place with AES-CTR.

Parametry

<i>data</i>	Payload buffer.
<i>len</i>	Payload length.
<i>key</i>	AES-128 key.
<i>frame</i>	Frame fields used to build the CTR nonce.

Zwraca

true when the operation succeeded.

Definicja w linii 1134 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_aes_ctr_crypt_hw_locked\(\)](#), [laviet_aes_ctr_crypt_sw\(\)](#), [LAVIET_AES_KEY_LEN](#), [laviet_crypto_lock\(\)](#), [laviet_crypto_unlock\(\)](#) i [laviet_secure_zero\(\)](#).

12.20.3.3 laviet_aes_ctr_crypt_hw_locked()

```
bool laviet_aes_ctr_crypt_hw_locked (
    uint8_t * data,
    uint8_t len,
    const uint8_t key[LAVIET_AES_KEY_LEN],
    const laviet_frame_t * frame) [static]
```

Hardware AES-CTR path for one LAVIET payload block; caller must hold the crypto mutex.

Definicja w linii 813 pliku [laviet_crypto.c](#).

Odwołuje się do [LAVIET_AES_KEY_LEN](#), [laviet_hal_status_text\(\)](#), [laviet_make_counter_block\(\)](#) i [laviet_secure_zero\(\)](#).

Odwołania w [laviet_aes_ctr_crypt\(\)](#) i [laviet_crypto_self_test_locked\(\)](#).

12.20.3.4 laviet_aes_ctr_crypt_sw()

```
bool laviet_aes_ctr_crypt_sw (
    uint8_t * data,
    uint8_t len,
    const uint8_t key[LAVIET_AES_KEY_LEN],
    const laviet_frame_t * frame) [static]
```

Software AES-CTR encrypt/decrypt routine for arbitrary LAVIET payload length.

Definicja w linii 775 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_aes_encrypt_block\(\)](#), [laviet_aes_key_expansion\(\)](#), [LAVIET_AES_KEY_LEN](#), [laviet_make_counter_block\(\)](#) i [laviet_secure_zero\(\)](#).

Odwołania w [laviet_aes_ctr_crypt\(\)](#) i [laviet_crypto_self_test_locked\(\)](#).

12.20.3.5 laviet_aes_encrypt_block()

```
void laviet_aes_encrypt_block (
    const uint8_t in[16],
    uint8_t out[16],
    const uint8_t round_key[176]) [static]
```

Encrypt one 16-byte block with the software AES-128 implementation.

Definicja w linii 543 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_aes_add_round_key\(\)](#), [laviet_aes_mix_columns\(\)](#), [laviet_aes_shift_rows\(\)](#), [laviet_aes_sub_bytes\(\)](#) i [laviet_secure_zero\(\)](#).

Odwołania w [laviet_aes_ctr_crypt_sw\(\)](#).

12.20.3.6 laviet_aes_key_expansion()

```
void laviet_aes_key_expansion (
    const uint8_t key[16],
    uint8_t round_key[176]) [static]
```

Expand a 128-bit AES key into the AES-128 round-key schedule.

Definicja w linii 438 pliku [laviet_crypto.c](#).

Odwołania w [laviet_aes_ctr_crypt_sw\(\)](#).

12.20.3.7 laviet_aes_mix_columns()

```
void laviet_aes_mix_columns (
    uint8_t state[16]) [static]
```

Internal helper: [laviet_aes_mix_columns](#).

Definicja w linii 516 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_aes_xtime\(\)](#).
Odwołania w [laviet_aes_encrypt_block\(\)](#).

12.20.3.8 laviet_aes_shift_rows()

```
void laviet_aes_shift_rows (
    uint8_t state[16]) [static]
```

Internal helper: [laviet_aes_shift_rows](#).
Definicja w linii 491 pliku [laviet_crypto.c](#).
Odwołania w [laviet_aes_encrypt_block\(\)](#).

12.20.3.9 laviet_aes_sub_bytes()

```
void laviet_aes_sub_bytes (
    uint8_t state[16]) [static]
```

Internal helper: [laviet_aes_sub_bytes](#).
Definicja w linii 480 pliku [laviet_crypto.c](#).
Odwołania w [laviet_aes_encrypt_block\(\)](#).

12.20.3.10 laviet_aes_xtime()

```
uint8_t laviet_aes_xtime (
    uint8_t x) [static]
```

Internal helper: [laviet_aes_xtime](#).
Definicja w linii 432 pliku [laviet_crypto.c](#).
Odwołania w [laviet_aes_mix_columns\(\)](#).

12.20.3.11 laviet_crypto_init()

```
bool laviet_crypto_init (
    void )
```

Initialize crypto backends and run the optional hardware self-test.

Zwraca

`true` when the software fallback is usable.

Definicja w linii 1049 pliku [laviet_crypto.c](#).
Odwołuje się do [laviet_crypto_lock\(\)](#), [laviet_crypto_self_test_locked\(\)](#) i [laviet_crypto_unlock\(\)](#).

12.20.3.12 laviet_crypto_lock()

```
bool laviet_crypto_lock (
    void ) [static]
```

Internal helper: [laviet_crypto_lock](#).
Definicja w linii 592 pliku [laviet_crypto.c](#).
Odwołania w [laviet_aes_ctr_crypt\(\)](#), [laviet_crypto_init\(\)](#) i [laviet_hmac_sha256\(\)](#).

12.20.3.13 laviet_crypto_log_mismatch()

```
bool laviet_crypto_log_mismatch (
    const char * stage,
    const uint8_t * expected,
    const uint8_t * actual,
    uint16_t len) [static]
```

Print the first byte mismatch found during crypto self-test comparison.
Definicja w linii 207 pliku [laviet_crypto.c](#).
Odwołania w [laviet_crypto_self_test_locked\(\)](#).

12.20.3.14 laviet_crypto_self_test_locked()

```
bool laviet_crypto_self_test_locked (
    void ) [static]
```

Compare hardware crypto output against software reference vectors under the crypto mutex.

Definicja w linii 934 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_aes_ctr_crypt_hw_locked\(\)](#), [laviet_aes_ctr_crypt_sw\(\)](#), [laviet_crypto_log_mismatch\(\)](#), [LAVIET_FLAG_ENCRYPTED](#), [laviet_hmac_sha256_hw_locked\(\)](#), [laviet_hmac_sha256_sw\(\)](#) i [LAVIET_SHA256_LEN](#).

Odwołania w [laviet_crypto_init\(\)](#).

12.20.3.15 laviet_crypto_unlock()

```
void laviet_crypto_unlock (
    void ) [static]
```

Internal helper: [laviet_crypto_unlock](#).

Definicja w linii 603 pliku [laviet_crypto.c](#).

Odwołania w [laviet_aes_ctr_crypt\(\)](#), [laviet_crypto_init\(\)](#) i [laviet_hmac_sha256\(\)](#).

12.20.3.16 laviet_frame_hmac_sha256()

```
bool laviet_frame_hmac_sha256 (
    const laviet_frame_t * frame,
    const uint8_t key[LAVIET_HMAC_KEY_LEN],
    uint8_t out[LAVIET_MAC_TAG_LEN])
```

Calculate the LAVIET frame MAC over the serialized MAC input.

Parametry

<i>frame</i>	Frame to authenticate.
<i>key</i>	HMAC key.
<i>out</i>	[out] MAC tag buffer.

Zwraca

`true` when the MAC tag was generated.

Definicja w linii 739 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_frame_build_mac_input\(\)](#), [LAVIET_FRAME_HEADER_LEN](#), [LAVIET_HMAC_KEY_LEN](#), [laviet_hmac_sha256\(\)](#), [LAVIET_MAC_TAG_LEN](#), [LAVIET_MAX_PAYLOAD](#) i [laviet_secure_zero\(\)](#).

12.20.3.17 laviet_hal_status_text()

```
const char * laviet_hal_status_text (
    HAL_StatusTypeDef status) [static]
```

Convert a HAL status code to a short diagnostic string.

Definicja w linii 116 pliku [laviet_crypto.c](#).

Odwołania w [laviet_aes_ctr_crypt_hw_locked\(\)](#) i [laviet_hmac_sha256_hw_locked\(\)](#).

12.20.3.18 laviet_hmac_sha256()

```
bool laviet_hmac_sha256 (
    const uint8_t * key,
    uint16_t key_len,
    const uint8_t * data,
    uint16_t data_len,
    uint8_t out[LAVIET_SHA256_LEN])
```

Calculate HMAC-SHA256 over an arbitrary buffer.

Parametry

<i>key</i>	HMAC key bytes.
<i>key_len</i>	HMAC key length.
<i>data</i>	Data buffer.
<i>data_len</i>	Data length.
<i>out</i>	[out] 32-byte digest output.

Zwraca

`true` when the digest was generated.

Definicja w linii 1109 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_crypto_lock\(\)](#), [laviet_crypto_unlock\(\)](#), [laviet_hmac_sha256_hw_locked\(\)](#), [laviet_hmac_sha256_sw\(\)](#) i [LAVIET_SHA256_LEN](#).

Odwołania w [laviet_frame_hmac_sha256\(\)](#).

12.20.3.19 laviet_hmac_sha256_hw_locked()

```
bool laviet_hmac_sha256_hw_locked (
    const uint8_t * key,
    uint16_t key_len,
    const uint8_t * data,
    uint16_t data_len,
    uint8_t out[LAVIET_SHA256_LEN]) [static]
```

Hardware HASH HMAC-SHA256 path; caller must hold the crypto mutex.

Definicja w linii 665 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_hal_status_text\(\)](#) i [LAVIET_SHA256_LEN](#).

Odwołania w [laviet_crypto_self_test_locked\(\)](#) i [laviet_hmac_sha256\(\)](#).

12.20.3.20 laviet_hmac_sha256_sw()

```
bool laviet_hmac_sha256_sw (
    const uint8_t * key,
    uint16_t key_len,
    const uint8_t * data,
    uint16_t data_len,
    uint8_t out[LAVIET_SHA256_LEN]) [static]
```

Software HMAC-SHA256 implementation used as the portable fallback.

Definicja w linii 612 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_secure_zero\(\)](#), [laviet_sha256_final\(\)](#), [laviet_sha256_init\(\)](#), [LAVIET_SHA256_LEN](#) i [laviet_sha256_update\(\)](#).

Odwołania w [laviet_crypto_self_test_locked\(\)](#) i [laviet_hmac_sha256\(\)](#).

12.20.3.21 laviet_mac_equal()

```
bool laviet_mac_equal (
    const uint8_t a[LAVIET_MAC_TAG_LEN],
    const uint8_t b[LAVIET_MAC_TAG_LEN])
```

Compare two MAC tags without data-dependent early exit.

Parametry

<i>a</i>	First MAC tag.
<i>b</i>	Second MAC tag.

Zwraca

true when tags are identical.

Definicja w linii 1172 pliku [laviet_crypto.c](#).

Odwołuje się do [LAVIET_MAC_TAG_LEN](#).

12.20.3.22 laviet_make_counter_block()

```
void laviet_make_counter_block (
    const laviet_frame_t * frame,
    uint16_t block_index,
    uint8_t out[16]) [static]
```

Build the LAVIET-specific AES-CTR counter block from frame identity fields.

Definicja w linii 569 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_frame_t::counter](#), [laviet_frame_t::dst_id](#), [laviet_frame_t::msg_id](#) i [laviet_frame_t::src_id](#).

Odwołania w [laviet_aes_ctr_crypt_hw_locked\(\)](#) i [laviet_aes_ctr_crypt_sw\(\)](#).

12.20.3.23 laviet_rotr32()

```
uint32_t laviet_rotr32 (
    uint32_t value,
    uint8_t bits) [static]
```

Rotate a 32-bit word right for SHA-256 round functions.

Definicja w linii 273 pliku [laviet_crypto.c](#).

Odwołania w [laviet_sha256_transform\(\)](#).

12.20.3.24 laviet_secure_zero()

```
void laviet_secure_zero (
    void * ptr,
    size_t len)
```

Clear sensitive memory through a volatile byte pointer.

Parametry

<i>ptr</i>	Buffer to wipe.
<i>len</i>	Number of bytes to clear.

Definicja w linii 1191 pliku [laviet_crypto.c](#).

Odwołania w [laviet_aes_ctr_crypt\(\)](#), [laviet_aes_ctr_crypt_hw_locked\(\)](#), [laviet_aes_ctr_crypt_sw\(\)](#), [laviet_aes_encrypt_block\(\)](#), [laviet_frame_hmac_sha256\(\)](#), [laviet_hmac_sha256_sw\(\)](#) i [security_main_get_peer_link_key\(\)](#).

12.20.3.25 laviet_sha256_final()

```
void laviet_sha256_final (
    laviet_sha256_ctx_t * ctx,
    uint8_t hash[LAVIET_SHA256_LEN]) [static]
```

Finalize SHA-256 padding and write the 32-byte digest.

Definicja w linii 387 pliku [laviet_crypto.c](#).

Odwołuje się do [LAVIET_SHA256_LEN](#) i [laviet_sha256_transform\(\)](#).

Odwołania w [laviet_hmac_sha256_sw\(\)](#).

12.20.3.26 laviet_sha256_init()

```
void laviet_sha256_init (
    laviet_sha256_ctx_t * ctx) [static]
```

Initialize the software SHA-256 context with the standard IV.

Definicja w linii 349 pliku [laviet_crypto.c](#).

Odwołania w [laviet_hmac_sha256_sw\(\)](#).

12.20.3.27 laviet_sha256_transform()

```
void laviet_sha256_transform (
    laviet_sha256_ctx_t * ctx,
    const uint8_t data[64]) [static]
```

Process one 512-bit SHA-256 block into the running digest state.

Definicja w linii 279 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_rot32\(\)](#).

Odwołania w [laviet_sha256_final\(\)](#) i [laviet_sha256_update\(\)](#).

12.20.3.28 laviet_sha256_update()

```
void laviet_sha256_update (
    laviet_sha256_ctx_t * ctx,
    const uint8_t * data,
    uint16_t len) [static]
```

Feed bytes into the software SHA-256 streaming context.

Definicja w linii 364 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_sha256_transform\(\)](#).

Odwołania w [laviet_hmac_sha256_sw\(\)](#).

12.20.4 Dokumentacja zmiennych

12.20.4.1 s_aes_rcon

```
const uint8_t s_aes_rcon[11] [static]
```

Wartość początkowa:

```
=
{
    0x00U, 0x01U, 0x02U, 0x04U, 0x08U, 0x10U, 0x20U, 0x40U, 0x80U, 0x1BU, 0x36U
}
```

Definicja w linii 110 pliku [laviet_crypto.c](#).

12.20.4.2 s_aes_sbox

```
const uint8_t s_aes_sbox[256] [static]
```

Definicja w linii 74 pliku [laviet_crypto.c](#).

12.20.4.3 s_crypto_hw_ready

```
bool s_crypto_hw_ready = false [static]
```

Definicja w linii 26 pliku [laviet_crypto.c](#).

12.20.4.4 s_crypto_mutex

```
osMutexId_t s_crypto_mutex = NULL [static]
```

Definicja w linii 25 pliku [laviet_crypto.c](#).

12.20.4.5 s_sha256_k

```
const uint32_t s_sha256_k[64] [static]
```

Wartość początkowa:

```
=
{
    0x428A2F98UL, 0x71374491UL, 0xB5C0FBCFUL, 0xE9B5DBA5UL,
    0x3956C25BUL, 0x59F111F1UL, 0x923F82A4UL, 0xAB1C5ED5UL,
    0xD807AA98UL, 0x12835B01UL, 0x243185BEUL, 0x550C7DC3UL,
    0x72BE5D74UL, 0x80DEB1FEUL, 0x9BDC06A7UL, 0xC19BF174UL,
    0xE49B69C1UL, 0xEFBE4786UL, 0x0FC19DC6UL, 0x240CA1CCUL,
    0x2DE92C6FUL, 0x4A7484AAUL, 0x5CB0A9DCUL, 0x76F988DAUL,
    0x983E5152UL, 0xA831C66DUL, 0xB00327C8UL, 0xBF597FC7UL,
    0xC6E00BF3UL, 0xD5A79147UL, 0x06CA6351UL, 0x14292967UL,
```

```

    0x27B70A85UL, 0x2E1B2138UL, 0x4D2C6DFCUL, 0x53380D13UL,
    0x650A7354UL, 0x766A0ABBUL, 0x81C2C92EUL, 0x92722C85UL,
    0xA2BFE8A1UL, 0xA81A664BUL, 0xC24B8B70UL, 0xC76C51A3UL,
    0xD192E819UL, 0xD6990624UL, 0xF40E3585UL, 0x106AA070UL,
    0x19A4C116UL, 0x1E376C08UL, 0x2748774CUL, 0x34B0BCB5UL,
    0x391C0CB3UL, 0x4ED8AA4AUL, 0x5B9CCA4FUL, 0x682E6FF3UL,
    0x748F82EEUL, 0x78A5636FUL, 0x84C87814UL, 0x8CC70208UL,
    0x90BEFFFAUL, 0xA4506CEBUL, 0xBEF9A3F7UL, 0xC67178F2UL
}

```

Definicja w linii 54 pliku `laviet_crypto.c`.

12.21 laviet_crypto.c

[Idź do dokumentacji tego pliku.](#)

```

00001
00005
00006 #include "laviet_crypto.h"
00007
00008 #include "cmsis_os2.h"
00009 #include "stm32u5xx_hal.h"
00010
00011 #include <stdio.h>
00012 #include <string.h>
00013
00014 #if defined(HAL_CRYPT_MODULE_ENABLED)
00015 extern CRYPT_HandleTypeDef hcryp;
00016 #endif
00017 #if defined(HAL_HASH_MODULE_ENABLED)
00018 extern HASH_HandleTypeDef hhash;
00019 #endif
00020
00021 #define LAVIET_CRYPT_TIMEOUT_MS 1000U
00022 #define LAVIET_TEST_FIXED_FRAME_MAC 0U
00023 #define LAVIET_TEST_FIXED_FRAME_MAC_BYTE 0x01U
00024
00025 static osMutexId_t s_crypto_mutex = NULL;
00026 static bool s_crypto_hw_ready = false;
00027
00028 static const char *laviet_hal_status_text(HAL_StatusTypeDef status);
00029 #if defined(HAL_CRYPT_MODULE_ENABLED)
00030 static const char *laviet_cryp_state_text(HAL_CRYPT_StateTypeDef state);
00031 static void laviet_crypto_log_cryp_context(const char *stage);
00032 #endif
00033 #if defined(HAL_HASH_MODULE_ENABLED)
00034 static const char *laviet_hash_state_text(HAL_HASH_StateTypeDef state);
00035 static void laviet_crypto_log_hash_context(const char *stage);
00036 #endif
00037 static bool laviet_crypto_log_mismatch(const char *stage,
00038                                       const uint8_t *expected,
00039                                       const uint8_t *actual,
00040                                       uint16_t len);
00041 #if defined(HAL_CRYPT_MODULE_ENABLED)
00042 static void laviet_crypto_pack_be_words(uint32_t words[4], const uint8_t bytes[16]);
00043 static void laviet_crypto_unpack_be_words(uint8_t bytes[16], const uint32_t words[4]);
00044 #endif
00045
00046 typedef struct
00047 {
00048     uint8_t data[64];
00049     uint32_t datalen;
00050     uint64_t bitlen;
00051     uint32_t state[8];
00052 } laviet_sha256_ctx_t;
00053
00054 static const uint32_t s_sha256_k[64] =
00055 {
00056     0x428A2F98UL, 0x71374491UL, 0xB5C0FBCFUL, 0xE9B5DBA5UL,
00057     0x3956C25BUL, 0x59F111F1UL, 0x923F82A4UL, 0xAB1C5ED5UL,
00058     0xD807AA98UL, 0x12835B01UL, 0x243185BEUL, 0x550C7DC3UL,
00059     0x72BE5D74UL, 0x80DEB1FEUL, 0x9BDC06A7UL, 0xC19BF174UL,
00060     0xE49B69C1UL, 0xEFBE4786UL, 0x0FC19DC6UL, 0x240CA1CCUL,
00061     0x2DE922CFUL, 0x4A7484AAUL, 0x5CB0A9DCUL, 0x76F988DAUL,
00062     0x983E5152UL, 0xA831C66DUL, 0xB00327C8UL, 0xBF597FC7UL,
00063     0xC6E00BF3UL, 0xD5A79147UL, 0x06CA6351UL, 0x14292967UL,
00064     0x27B70A85UL, 0x2E1B2138UL, 0x4D2C6DFCUL, 0x53380D13UL,
00065     0x650A7354UL, 0x766A0ABBUL, 0x81C2C92EUL, 0x92722C85UL,
00066     0xA2BFE8A1UL, 0xA81A664BUL, 0xC24B8B70UL, 0xC76C51A3UL,
00067     0xD192E819UL, 0xD6990624UL, 0xF40E3585UL, 0x106AA070UL,
00068     0x19A4C116UL, 0x1E376C08UL, 0x2748774CUL, 0x34B0BCB5UL,
00069     0x391C0CB3UL, 0x4ED8AA4AUL, 0x5B9CCA4FUL, 0x682E6FF3UL,
00070     0x748F82EEUL, 0x78A5636FUL, 0x84C87814UL, 0x8CC70208UL,
00071     0x90BEFFFAUL, 0xA4506CEBUL, 0xBEF9A3F7UL, 0xC67178F2UL
00072 };

```

```

00073
00074 static const uint8_t s_aes_sbox[256] =
00075 {
00076     0x63U, 0x7CU, 0x77U, 0x7BU, 0xF2U, 0x6BU, 0x6FU, 0xC5U,
00077     0x30U, 0x01U, 0x67U, 0x2BU, 0xFEU, 0xD7U, 0xABU, 0x76U,
00078     0xCAU, 0x82U, 0xC9U, 0x7DU, 0xFAU, 0x59U, 0x47U, 0xF0U,
00079     0xADU, 0xD4U, 0xA2U, 0xAFU, 0x9CU, 0xA4U, 0x72U, 0xC0U,
00080     0xB7U, 0xFDU, 0x93U, 0x26U, 0x36U, 0x3FU, 0xF7U, 0xCCU,
00081     0x34U, 0xA5U, 0xE5U, 0xF1U, 0x71U, 0xD8U, 0x31U, 0x15U,
00082     0x04U, 0xC7U, 0x23U, 0xC3U, 0x18U, 0x96U, 0x05U, 0x9AU,
00083     0x07U, 0x12U, 0x80U, 0xE2U, 0xEBU, 0x27U, 0xB2U, 0x75U,
00084     0x09U, 0x83U, 0x2CU, 0x1AU, 0x1BU, 0x6EU, 0x5AU, 0xA0U,
00085     0x52U, 0x3BU, 0xD6U, 0xB3U, 0x29U, 0xE3U, 0x2FU, 0x84U,
00086     0x53U, 0xD1U, 0x00U, 0xEDU, 0x20U, 0xFCU, 0xB1U, 0x5BU,
00087     0x6AU, 0xCBU, 0xBEU, 0x39U, 0x4AU, 0x4CU, 0x58U, 0xCFU,
00088     0xD0U, 0xEFU, 0xAAU, 0xFBU, 0x43U, 0x4DU, 0x33U, 0x85U,
00089     0x45U, 0xF9U, 0x02U, 0x7FU, 0x50U, 0x3CU, 0x9FU, 0xA8U,
00090     0x51U, 0xA3U, 0x40U, 0x8FU, 0x92U, 0x9DU, 0x38U, 0xF5U,
00091     0xBCU, 0xB6U, 0xDAU, 0x21U, 0x10U, 0xFFU, 0xF3U, 0xD2U,
00092     0xCDU, 0x0CU, 0x13U, 0xECU, 0x5FU, 0x97U, 0x44U, 0x17U,
00093     0xC4U, 0xA7U, 0x7EU, 0x3DU, 0x64U, 0x5DU, 0x19U, 0x73U,
00094     0x60U, 0x81U, 0x4FU, 0xDCU, 0x22U, 0x2AU, 0x90U, 0x88U,
00095     0x46U, 0xEEU, 0xB8U, 0x14U, 0xDEU, 0x5EU, 0x0BU, 0xDBU,
00096     0xE0U, 0x32U, 0x3AU, 0x0AU, 0x49U, 0x06U, 0x24U, 0x5CU,
00097     0xC2U, 0xD3U, 0xACU, 0x62U, 0x91U, 0x95U, 0xE4U, 0x79U,
00098     0xE7U, 0xC8U, 0x37U, 0x6DU, 0x8DU, 0xD5U, 0x4EU, 0xA9U,
00099     0x6CU, 0x56U, 0xF4U, 0xEAU, 0x65U, 0x7AU, 0xAEU, 0x08U,
00100     0xBAU, 0x78U, 0x25U, 0x2EU, 0x1CU, 0xA6U, 0xB4U, 0xC6U,
00101     0xE8U, 0xDDU, 0x74U, 0x1FU, 0x4BU, 0xBDU, 0x8BU, 0x8AU,
00102     0x70U, 0x3EU, 0xB5U, 0x66U, 0x48U, 0x03U, 0xF6U, 0x0EU,
00103     0x61U, 0x35U, 0x57U, 0xB9U, 0x86U, 0xC1U, 0x1DU, 0x9EU,
00104     0xE1U, 0xF8U, 0x98U, 0x11U, 0x69U, 0xD9U, 0x8EU, 0x94U,
00105     0x9BU, 0x1EU, 0x87U, 0xE9U, 0xCEU, 0x55U, 0x28U, 0xDFU,
00106     0x8CU, 0xA1U, 0x89U, 0x0DU, 0xBFU, 0xE6U, 0x42U, 0x68U,
00107     0x41U, 0x99U, 0x2DU, 0x0FU, 0xB0U, 0x54U, 0xBBU, 0x16U
00108 };
00109
00110 static const uint8_t s_aes_rcon[11] =
00111 {
00112     0x00U, 0x01U, 0x02U, 0x04U, 0x08U, 0x10U, 0x20U, 0x40U, 0x80U, 0x1BU, 0x36U
00113 };
00114
00116 static const char *laviet_hal_status_text(HAL_StatusTypeDef status)
00117 {
00118     switch (status)
00119     {
00120         case HAL_OK:
00121             return "OK";
00122         case HAL_ERROR:
00123             return "ERROR";
00124         case HAL_BUSY:
00125             return "BUSY";
00126         case HAL_TIMEOUT:
00127             return "TIMEOUT";
00128         default:
00129             return "?";
00130     }
00131 }
00132
00133 #if defined(HAL_CRYP_MODULE_ENABLED)
00135 static const char *laviet_cryp_state_text(HAL_CRYP_StateTypeDef state)
00136 {
00137     switch (state)
00138     {
00139         case HAL_CRYP_STATE_RESET:
00140             return "RESET";
00141         case HAL_CRYP_STATE_READY:
00142             return "READY";
00143         case HAL_CRYP_STATE_BUSY:
00144             return "BUSY";
00145         #if (USE_HAL_CRYP_SUSPEND_RESUME == 1U)
00146         case HAL_CRYP_STATE_SUSPENDED:
00147             return "SUSPENDED";
00148         #endif
00149         default:
00150             return "?";
00151     }
00152 }
00153
00155 static void laviet_crypto_log_cryp_context(const char *stage)
00156 {
00157     printf("CRYPTO: AES %s inst=%p state=%d(%s) err=0x%08lX alg=0x%08lX dtype=0x%08lX width=%lu\n",
00158           keysize=0x%08lX\r\n",
00159           (stage != NULL) ? stage : "state",
00160           (void *)hccryp.Instance,
00161           (int)HAL_CRYP_GetState(&hccryp),
00162           laviet_cryp_state_text(HAL_CRYP_GetState(&hccryp)),

```

```

00162         (unsigned long)HAL_Cryp_GetError(&hcrp),
00163         (unsigned long)hcrp.Init.Algorithm,
00164         (unsigned long)hcrp.Init.DataType,
00165         (unsigned long)hcrp.Init.DataWidthUnit,
00166         (unsigned long)hcrp.Init.KeySize);
00167     }
00168 #endif
00169
00170 #if defined(HAL_HASH_MODULE_ENABLED)
00172 static const char *laviet_hash_state_text (HAL_HASH_StateTypeDef state)
00173 {
00174     switch (state)
00175     {
00176         case HAL_HASH_STATE_RESET:
00177             return "RESET";
00178         case HAL_HASH_STATE_READY:
00179             return "READY";
00180         case HAL_HASH_STATE_BUSY:
00181             return "BUSY";
00182         case HAL_HASH_STATE_TIMEOUT:
00183             return "TIMEOUT";
00184         case HAL_HASH_STATE_ERROR:
00185             return "ERROR";
00186         case HAL_HASH_STATE_SUSPENDED:
00187             return "SUSPENDED";
00188         default:
00189             return "?";
00190     }
00191 }
00192
00194 static void laviet_crypto_log_hash_context(const char *stage)
00195 {
00196     printf("CRYPTO: HASH %s state=%d(%s) err=0x%08lX dtype=0x%08lX key_len=%lu\r\n",
00197         (stage != NULL) ? stage : "state",
00198         (int)HAL_HASH_GetState(&hhash),
00199         laviet_hash_state_text (HAL_HASH_GetState(&hhash)),
00200         (unsigned long)HAL_HASH_GetError(&hhash),
00201         (unsigned long)hhash.Init.DataType,
00202         (unsigned long)hhash.Init.KeySize);
00203 }
00204 #endif
00205
00207 static bool laviet_crypto_log_mismatch(const char *stage,
00208                                     const uint8_t *expected,
00209                                     const uint8_t *actual,
00210                                     uint16_t len)
00211 {
00212     uint16_t i;
00213
00214     if ((expected == NULL) || (actual == NULL))
00215     {
00216         printf("CRYPTO: self-test failed stage=%s reason=NULL_compare_buffer\r\n",
00217             (stage != NULL) ? stage : "?");
00218         return false;
00219     }
00220
00221     for (i = 0U; i < len; i++)
00222     {
00223         if (expected[i] != actual[i])
00224         {
00225             printf("CRYPTO: self-test mismatch stage=%s index=%u expected=0x%02X actual=0x%02X
len=%u\r\n",
00226                 (stage != NULL) ? stage : "?",
00227                 (unsigned int)i,
00228                 (unsigned int)expected[i],
00229                 (unsigned int)actual[i],
00230                 (unsigned int)len);
00231             return false;
00232         }
00233     }
00234
00235     return true;
00236 }
00237
00238 #if defined(HAL_Cryp_MODULE_ENABLED)
00240 static void laviet_crypto_pack_be_words(uint32_t words[4], const uint8_t bytes[16])
00241 {
00242     uint8_t i;
00243
00244     for (i = 0U; i < 4U; i++)
00245     {
00246         uint8_t offset = (uint8_t)(i * 4U);
00247
00248         words[i] = ((uint32_t)bytes[offset] << 24) |
00249             ((uint32_t)bytes[offset + 1U] << 16) |
00250             ((uint32_t)bytes[offset + 2U] << 8) |
00251             (uint32_t)bytes[offset + 3U];

```

```

00252     }
00253 }
00254
00256 static void laviet_crypto_unpack_be_words(uint8_t bytes[16], const uint32_t words[4])
00257 {
00258     uint8_t i;
00259
00260     for (i = 0U; i < 4U; i++)
00261     {
00262         uint8_t offset = (uint8_t)(i * 4U);
00263
00264         bytes[offset] = (uint8_t)(words[i] >> 24);
00265         bytes[offset + 1U] = (uint8_t)(words[i] >> 16);
00266         bytes[offset + 2U] = (uint8_t)(words[i] >> 8);
00267         bytes[offset + 3U] = (uint8_t)words[i];
00268     }
00269 }
00270 #endif
00271
00273 static uint32_t laviet_rotr32(uint32_t value, uint8_t bits)
00274 {
00275     return (value >> bits) | (value << (32U - bits));
00276 }
00277
00279 static void laviet_sha256_transform(laviet_sha256_ctx_t *ctx, const uint8_t data[64])
00280 {
00281     uint32_t m[64];
00282     uint32_t a;
00283     uint32_t b;
00284     uint32_t c;
00285     uint32_t d;
00286     uint32_t e;
00287     uint32_t f;
00288     uint32_t g;
00289     uint32_t h;
00290     uint32_t i;
00291
00292     for (i = 0U; i < 16U; i++)
00293     {
00294         m[i] = ((uint32_t)data[i * 4U] << 24) |
00295             ((uint32_t)data[(i * 4U) + 1U] << 16) |
00296             ((uint32_t)data[(i * 4U) + 2U] << 8) |
00297             (uint32_t)data[(i * 4U) + 3U];
00298     }
00299     for (i = 16U; i < 64U; i++)
00300     {
00301         uint32_t s0 = laviet_rotr32(m[i - 15U], 7U) ^
00302             laviet_rotr32(m[i - 15U], 18U) ^
00303             (m[i - 15U] >> 3);
00304         uint32_t s1 = laviet_rotr32(m[i - 2U], 17U) ^
00305             laviet_rotr32(m[i - 2U], 19U) ^
00306             (m[i - 2U] >> 10);
00307         m[i] = m[i - 16U] + s0 + m[i - 7U] + s1;
00308     }
00309
00310     a = ctx->state[0];
00311     b = ctx->state[1];
00312     c = ctx->state[2];
00313     d = ctx->state[3];
00314     e = ctx->state[4];
00315     f = ctx->state[5];
00316     g = ctx->state[6];
00317     h = ctx->state[7];
00318
00319     for (i = 0U; i < 64U; i++)
00320     {
00321         uint32_t s1 = laviet_rotr32(e, 6U) ^ laviet_rotr32(e, 11U) ^ laviet_rotr32(e, 25U);
00322         uint32_t ch = (e & f) ^ ((~e) & g);
00323         uint32_t temp1 = h + s1 + ch + s_sha256_k[i] + m[i];
00324         uint32_t s0 = laviet_rotr32(a, 2U) ^ laviet_rotr32(a, 13U) ^ laviet_rotr32(a, 22U);
00325         uint32_t maj = (a & b) ^ (a & c) ^ (b & c);
00326         uint32_t temp2 = s0 + maj;
00327
00328         h = g;
00329         g = f;
00330         f = e;
00331         e = d + temp1;
00332         d = c;
00333         c = b;
00334         b = a;
00335         a = temp1 + temp2;
00336     }
00337
00338     ctx->state[0] += a;
00339     ctx->state[1] += b;
00340     ctx->state[2] += c;
00341     ctx->state[3] += d;

```

```

00342     ctx->state[4] += e;
00343     ctx->state[5] += f;
00344     ctx->state[6] += g;
00345     ctx->state[7] += h;
00346 }
00347
00349 static void laviet_sha256_init(laviet_sha256_ctx_t *ctx)
00350 {
00351     ctx->datalen = 0U;
00352     ctx->bitlen = 0ULL;
00353     ctx->state[0] = 0x6A09E667UL;
00354     ctx->state[1] = 0xBB67AE85UL;
00355     ctx->state[2] = 0x3C6EF372UL;
00356     ctx->state[3] = 0xA54FF53AUL;
00357     ctx->state[4] = 0x510E527FUL;
00358     ctx->state[5] = 0x9B05688CUL;
00359     ctx->state[6] = 0x1F83D9ABUL;
00360     ctx->state[7] = 0x5BE0CD19UL;
00361 }
00362
00364 static void laviet_sha256_update(laviet_sha256_ctx_t *ctx, const uint8_t *data, uint16_t len)
00365 {
00366     uint16_t i;
00367
00368     if ((ctx == NULL) || ((data == NULL) && (len > 0U)))
00369     {
00370         return;
00371     }
00372
00373     for (i = 0U; i < len; i++)
00374     {
00375         ctx->data[ctx->datalen] = data[i];
00376         ctx->datalen++;
00377         if (ctx->datalen == 64U)
00378         {
00379             laviet_sha256_transform(ctx, ctx->data);
00380             ctx->bitlen += 512ULL;
00381             ctx->datalen = 0U;
00382         }
00383     }
00384 }
00385
00387 static void laviet_sha256_final(laviet_sha256_ctx_t *ctx, uint8_t hash[LAVIET_SHA256_LEN])
00388 {
00389     uint32_t i = ctx->datalen;
00390     uint8_t j;
00391
00392     if (ctx->datalen < 56U)
00393     {
00394         ctx->data[i++] = 0x80U;
00395         while (i < 56U)
00396         {
00397             ctx->data[i++] = 0x00U;
00398         }
00399     }
00400     else
00401     {
00402         ctx->data[i++] = 0x80U;
00403         while (i < 64U)
00404         {
00405             ctx->data[i++] = 0x00U;
00406         }
00407         laviet_sha256_transform(ctx, ctx->data);
00408         memset(ctx->data, 0, 56U);
00409     }
00410
00411     ctx->bitlen += ((uint64_t)ctx->datalen * 8ULL);
00412     ctx->data[63] = (uint8_t)ctx->bitlen;
00413     ctx->data[62] = (uint8_t)(ctx->bitlen >> 8);
00414     ctx->data[61] = (uint8_t)(ctx->bitlen >> 16);
00415     ctx->data[60] = (uint8_t)(ctx->bitlen >> 24);
00416     ctx->data[59] = (uint8_t)(ctx->bitlen >> 32);
00417     ctx->data[58] = (uint8_t)(ctx->bitlen >> 40);
00418     ctx->data[57] = (uint8_t)(ctx->bitlen >> 48);
00419     ctx->data[56] = (uint8_t)(ctx->bitlen >> 56);
00420     laviet_sha256_transform(ctx, ctx->data);
00421
00422     for (i = 0U; i < 8U; i++)
00423     {
00424         for (j = 0U; j < 4U; j++)
00425         {
00426             hash[(i * 4U) + j] = (uint8_t)(ctx->state[i] >> (24U - (j * 8U)));
00427         }
00428     }
00429 }
00430
00432 static uint8_t laviet_aes_xtime(uint8_t x)

```

```

00433 {
00434     return (uint8_t)((x << 1) ^ (((x >> 7) & 1U) * 0x1BU));
00435 }
00436
00438 static void laviet_aes_key_expansion(const uint8_t key[16], uint8_t round_key[176])
00439 {
00440     uint8_t i;
00441     uint8_t j;
00442     uint8_t temp[4];
00443
00444     memcpy(round_key, key, 16U);
00445     for (i = 4U; i < 44U; i++)
00446     {
00447         for (j = 0U; j < 4U; j++)
00448         {
00449             temp[j] = round_key[((i - 1U) * 4U) + j];
00450         }
00451
00452         if ((i % 4U) == 0U)
00453         {
00454             uint8_t t = temp[0];
00455             temp[0] = (uint8_t)(s_aes_sbox[temp[1]] ^ s_aes_rcon[i / 4U]);
00456             temp[1] = s_aes_sbox[temp[2]];
00457             temp[2] = s_aes_sbox[temp[3]];
00458             temp[3] = s_aes_sbox[t];
00459         }
00460
00461         for (j = 0U; j < 4U; j++)
00462         {
00463             round_key[(i * 4U) + j] = (uint8_t)(round_key[((i - 4U) * 4U) + j] ^ temp[j]);
00464         }
00465     }
00466 }
00467
00469 static void laviet_aes_add_round_key(uint8_t state[16], const uint8_t *round_key)
00470 {
00471     uint8_t i;
00472
00473     for (i = 0U; i < 16U; i++)
00474     {
00475         state[i] ^= round_key[i];
00476     }
00477 }
00478
00480 static void laviet_aes_sub_bytes(uint8_t state[16])
00481 {
00482     uint8_t i;
00483
00484     for (i = 0U; i < 16U; i++)
00485     {
00486         state[i] = s_aes_sbox[state[i]];
00487     }
00488 }
00489
00491 static void laviet_aes_shift_rows(uint8_t state[16])
00492 {
00493     uint8_t temp;
00494
00495     temp = state[1];
00496     state[1] = state[5];
00497     state[5] = state[9];
00498     state[9] = state[13];
00499     state[13] = temp;
00500
00501     temp = state[2];
00502     state[2] = state[10];
00503     state[10] = temp;
00504     temp = state[6];
00505     state[6] = state[14];
00506     state[14] = temp;
00507
00508     temp = state[15];
00509     state[15] = state[11];
00510     state[11] = state[7];
00511     state[7] = state[3];
00512     state[3] = temp;
00513 }
00514
00516 static void laviet_aes_mix_columns(uint8_t state[16])
00517 {
00518     uint8_t i;
00519
00520     for (i = 0U; i < 4U; i++)
00521     {
00522         uint8_t *col = &state[i * 4U];
00523         uint8_t t = (uint8_t)(col[0] ^ col[1] ^ col[2] ^ col[3]);
00524         uint8_t tmp = col[0];

```



```

00525     uint8_t tm;
00526
00527     tm = (uint8_t)(col[0] ^ col[1]);
00528     tm = laviet_aes_xtime(tm);
00529     col[0] ^= (uint8_t)(tm ^ t);
00530     tm = (uint8_t)(col[1] ^ col[2]);
00531     tm = laviet_aes_xtime(tm);
00532     col[1] ^= (uint8_t)(tm ^ t);
00533     tm = (uint8_t)(col[2] ^ col[3]);
00534     tm = laviet_aes_xtime(tm);
00535     col[2] ^= (uint8_t)(tm ^ t);
00536     tm = (uint8_t)(col[3] ^ tmp);
00537     tm = laviet_aes_xtime(tm);
00538     col[3] ^= (uint8_t)(tm ^ t);
00539 }
00540 }
00541
00543 static void laviet_aes_encrypt_block(const uint8_t in[16],
00544                                     uint8_t out[16],
00545                                     const uint8_t round_key[176])
00546 {
00547     uint8_t state[16];
00548     uint8_t round;
00549
00550     memcpy(state, in, 16U);
00551     laviet_aes_add_round_key(state, round_key);
00552
00553     for (round = 1U; round < 10U; round++)
00554     {
00555         laviet_aes_sub_bytes(state);
00556         laviet_aes_shift_rows(state);
00557         laviet_aes_mix_columns(state);
00558         laviet_aes_add_round_key(state, &round_key[round * 16U]);
00559     }
00560
00561     laviet_aes_sub_bytes(state);
00562     laviet_aes_shift_rows(state);
00563     laviet_aes_add_round_key(state, &round_key[160]);
00564     memcpy(out, state, 16U);
00565     laviet_secure_zero(state, sizeof(state));
00566 }
00567
00569 static void laviet_make_counter_block(const laviet_frame_t *frame,
00570                                     uint16_t block_index,
00571                                     uint8_t out[16])
00572 {
00573     out[0] = (uint8_t)'L';
00574     out[1] = (uint8_t)'V';
00575     out[2] = (uint8_t)'1';
00576     out[3] = 0U;
00577     out[4] = (uint8_t)(frame->src_id » 8);
00578     out[5] = (uint8_t)frame->src_id;
00579     out[6] = (uint8_t)(frame->dst_id » 8);
00580     out[7] = (uint8_t)frame->dst_id;
00581     out[8] = (uint8_t)(frame->msg_id » 8);
00582     out[9] = (uint8_t)frame->msg_id;
00583     out[10] = (uint8_t)(frame->counter » 24);
00584     out[11] = (uint8_t)(frame->counter » 16);
00585     out[12] = (uint8_t)(frame->counter » 8);
00586     out[13] = (uint8_t)frame->counter;
00587     out[14] = (uint8_t)(block_index » 8);
00588     out[15] = (uint8_t)block_index;
00589 }
00590
00592 static bool laviet_crypto_lock(void)
00593 {
00594     if (s_crypto_mutex == NULL)
00595     {
00596         return false;
00597     }
00598
00599     return (osMutexAcquire(s_crypto_mutex, LAVIET_CRYPTTO_TIMEOUT_MS) == osOK);
00600 }
00601
00603 static void laviet_crypto_unlock(void)
00604 {
00605     if (s_crypto_mutex != NULL)
00606     {
00607         (void)osMutexRelease(s_crypto_mutex);
00608     }
00609 }
00610
00612 static bool laviet_hmac_sha256_sw(const uint8_t *key,
00613                                   uint16_t key_len,
00614                                   const uint8_t *data,
00615                                   uint16_t data_len,
00616                                   uint8_t out[LAVIET_SHA256_LEN])

```

```

00617 {
00618     uint8_t key_block[64];
00619     uint8_t inner_hash[LAVIET_SHA256_LEN];
00620     laviet_sha256_ctx_t ctx;
00621     uint8_t i;
00622
00623     if ((key == NULL) || (out == NULL) || ((data == NULL) && (data_len > 0U)))
00624     {
00625         return false;
00626     }
00627
00628     memset(key_block, 0, sizeof(key_block));
00629     if (key_len > 64U)
00630     {
00631         laviet_sha256_init(&ctx);
00632         laviet_sha256_update(&ctx, key, key_len);
00633         laviet_sha256_final(&ctx, key_block);
00634     }
00635     else if (key_len > 0U)
00636     {
00637         memcpy(key_block, key, key_len);
00638     }
00639
00640     for (i = 0U; i < 64U; i++)
00641     {
00642         key_block[i] ^= 0x36U;
00643     }
00644     laviet_sha256_init(&ctx);
00645     laviet_sha256_update(&ctx, key_block, sizeof(key_block));
00646     laviet_sha256_update(&ctx, data, data_len);
00647     laviet_sha256_final(&ctx, inner_hash);
00648
00649     for (i = 0U; i < 64U; i++)
00650     {
00651         key_block[i] ^= (uint8_t)(0x36U ^ 0x5CU);
00652     }
00653     laviet_sha256_init(&ctx);
00654     laviet_sha256_update(&ctx, key_block, sizeof(key_block));
00655     laviet_sha256_update(&ctx, inner_hash, sizeof(inner_hash));
00656     laviet_sha256_final(&ctx, out);
00657
00658     laviet_secure_zero(&ctx, sizeof(ctx));
00659     laviet_secure_zero(inner_hash, sizeof(inner_hash));
00660     laviet_secure_zero(key_block, sizeof(key_block));
00661     return true;
00662 }
00663
00665 static bool laviet_hmac_sha256_hw_locked(const uint8_t *key,
00666                                         uint16_t key_len,
00667                                         const uint8_t *data,
00668                                         uint16_t data_len,
00669                                         uint8_t out[LAVIET_SHA256_LEN])
00670 {
00671     #if defined(HAL_HASH_MODULE_ENABLED)
00672     bool ok = false;
00673     HAL_StatusTypeDef deinit_status;
00674     HAL_StatusTypeDef init_status;
00675     HAL_StatusTypeDef hmac_status;
00676
00677     if ((key == NULL) ||
00678         (out == NULL) ||
00679         ((data == NULL) && (data_len > 0U)))
00680     {
00681         return false;
00682     }
00683
00684     deinit_status = HAL_HASH_DeInit(&hhash);
00685     if (deinit_status != HAL_OK)
00686     {
00687         printf("CRYPTO: HW HMAC HASH deinit failed st=%d(%s)\r\n",
00688              (int)deinit_status,
00689              laviet_hal_status_text(deinit_status));
00690         laviet_crypto_log_hash_context("after_deinit_fail");
00691         return false;
00692     }
00693     hhash.Init.DataType = HASH_DATATYPE_8B;
00694     hhash.Init.KeySize = key_len;
00695     hhash.Init.pKey = (uint8_t *)key;
00696     init_status = HAL_HASH_Init(&hhash);
00697     if (init_status != HAL_OK)
00698     {
00699         printf("CRYPTO: HW HMAC HASH init failed st=%d(%s) key_len=%u data_len=%u\r\n",
00700              (int)init_status,
00701              laviet_hal_status_text(init_status),
00702              (unsigned int)key_len,
00703              (unsigned int)data_len);
00704         laviet_crypto_log_hash_context("after_init_fail");

```

```

00705         return false;
00706     }
00707
00708     hmac_status = HAL_HMACEx_SHA256_Start(&hhash,
00709                                           data,
00710                                           data_len,
00711                                           out,
00712                                           LAVIET_CRYPTTO_TIMEOUT_MS);
00713     if (hmac_status == HAL_OK)
00714     {
00715         ok = true;
00716     }
00717     else
00718     {
00719         printf("CRYPTO: HW HMAC SHA256 failed st=%d(%s) key_len=%u data_len=%u timeout=%lu\r\n",
00720               (int)hmac_status,
00721               laviet_hal_status_text(hmac_status),
00722               (unsigned int)key_len,
00723               (unsigned int)data_len,
00724               (unsigned long)LAVIET_CRYPTTO_TIMEOUT_MS);
00725         laviet_crypto_log_hash_context("after_hmac_fail");
00726     }
00727
00728     return ok;
00729 #else
00730     (void)key;
00731     (void)key_len;
00732     (void)data;
00733     (void)data_len;
00734     (void)out;
00735     return false;
00736 #endif
00737 }
00738
00739 bool laviet_frame_hmac_sha256(const laviet_frame_t *frame,
00740                              const uint8_t key[LAVIET_HMAC_KEY_LEN],
00741                              uint8_t out[LAVIET_MAC_TAG_LEN])
00742 {
00743     #if (LAVIET_TEST_FIXED_FRAME_MAC == 1U)
00744         (void)frame;
00745         (void)key;
00746
00747         if (out == NULL)
00748         {
00749             return false;
00750         }
00751
00752         memset(out, LAVIET_TEST_FIXED_FRAME_MAC_BYTE, LAVIET_MAC_TAG_LEN);
00753         return true;
00754     #else
00755         uint8_t mac_input[LAVIET_FRAME_HEADER_LEN + LAVIET_MAX_PAYLOAD];
00756         uint8_t mac_input_len = 0U;
00757         bool ok;
00758
00759         if ((frame == NULL) || (key == NULL) || (out == NULL))
00760         {
00761             return false;
00762         }
00763         if (!laviet_frame_build_mac_input(frame, mac_input, sizeof(mac_input), &mac_input_len))
00764         {
00765             return false;
00766         }
00767
00768         ok = laviet_hmac_sha256(key, LAVIET_HMAC_KEY_LEN, mac_input, mac_input_len, out);
00769         laviet_secure_zero(mac_input, sizeof(mac_input));
00770         return ok;
00771     #endif
00772 }
00773
00775 static bool laviet_aes_ctr_crypt_sw(uint8_t *data,
00776                                     uint8_t len,
00777                                     const uint8_t key[LAVIET_AES_KEY_LEN],
00778                                     const laviet_frame_t *frame)
00779 {
00780     uint8_t round_key[176];
00781     uint8_t counter_block[16];
00782     uint8_t stream[16];
00783     uint8_t offset = 0U;
00784     uint16_t block_index = 0U;
00785
00786     if (((data == NULL) && (len > 0U)) || (key == NULL) || (frame == NULL))
00787     {
00788         return false;
00789     }
00790
00791     laviet_aes_key_expansion(key, round_key);
00792     while (offset < len)

```

```

00793     {
00794         uint8_t i;
00795
00796         laviet_make_counter_block(frame, block_index, counter_block);
00797         laviet_aes_encrypt_block(counter_block, stream, round_key);
00798         for (i = 0U; (i < 16U) && (offset < len); i++)
00799         {
00800             data[offset] ^= stream[i];
00801             offset++;
00802         }
00803         block_index++;
00804     }
00805
00806     laviet_secure_zero(round_key, sizeof(round_key));
00807     laviet_secure_zero(counter_block, sizeof(counter_block));
00808     laviet_secure_zero(stream, sizeof(stream));
00809     return true;
00810 }
00811
00812 static bool laviet_aes_ctr_crypt_hw_locked(uint8_t *data,
00813                                           uint8_t len,
00814                                           const uint8_t key[LAVIET_AES_KEY_LEN],
00815                                           const laviet_frame_t *frame)
00816 {
00817     #if defined(HAL_CRYPT_MODULE_ENABLED)
00818         uint8_t iv_bytes[16];
00819         uint8_t input_bytes[16];
00820         uint8_t output_bytes[16];
00821         uint32_t key_words[4];
00822         uint32_t iv_words[4];
00823         uint32_t input_words[4];
00824         uint32_t output_words[4];
00825         HAL_StatusTypeDef deinit_status;
00826         HAL_StatusTypeDef init_status;
00827         HAL_StatusTypeDef encrypt_status;
00828         bool ok = false;
00829
00830         if (((data == NULL) && (len > 0U)) ||
00831             (key == NULL) ||
00832             (frame == NULL) ||
00833             (hccryp.Instance != AES))
00834         {
00835             return false;
00836         }
00837         if (len == 0U)
00838         {
00839             return true;
00840         }
00841         if (len > sizeof(input_bytes))
00842         {
00843             return false;
00844         }
00845
00846         memset(iv_bytes, 0, sizeof(iv_bytes));
00847         memset(input_bytes, 0, sizeof(input_bytes));
00848         memset(output_bytes, 0, sizeof(output_bytes));
00849         memset(key_words, 0, sizeof(key_words));
00850         memset(iv_words, 0, sizeof(iv_words));
00851         memset(input_words, 0, sizeof(input_words));
00852         memset(output_words, 0, sizeof(output_words));
00853         laviet_crypto_pack_be_words(key_words, key);
00854         laviet_make_counter_block(frame, 0U, iv_bytes);
00855         laviet_crypto_pack_be_words(iv_words, iv_bytes);
00856         memcpy(input_bytes, data, len);
00857         laviet_crypto_pack_be_words(input_words, input_bytes);
00858
00859         deinit_status = HAL_CRYPT_DeInit(&hccryp);
00860         if (deinit_status != HAL_OK)
00861         {
00862             printf("CRYPTO: HW AES deinit failed st=%d(%s) payload_len=%u\r\n",
00863                   (int)deinit_status,
00864                   laviet_hal_status_text(deinit_status),
00865                   (unsigned int)len);
00866             laviet_crypto_log_cryp_context("after_deinit_fail");
00867             goto cleanup;
00868         }
00869         hccryp.Instance = AES;
00870         hccryp.Init.DataType = CRYPT_NO_SWAP;
00871         hccryp.Init.KeySize = CRYPT_KEYSIZE_128B;
00872         hccryp.Init.pKey = key_words;
00873         hccryp.Init.pInitVect = iv_words;
00874         hccryp.Init.Algorithm = CRYPT_AES_CTR;
00875         hccryp.Init.DataWidthUnit = CRYPT_DATAWIDTHUNIT_BYTE;
00876         hccryp.Init.HeaderWidthUnit = CRYPT_HEADERWIDTHUNIT_BYTE;
00877         hccryp.Init.KeyIVConfigSkip = CRYPT_KEYIVCONFIG_ALWAYS;
00878         hccryp.Init.KeyMode = CRYPT_KEYMODE_NORMAL;
00879
00880

```

```

00881     init_status = HAL_CRYPT_Init(&hcryp);
00882     if (init_status != HAL_OK)
00883     {
00884         printf("CRYPTO: HW AES init failed st=%d(%) payload_len=%u block_len=%u\r\n",
00885             (int)init_status,
00886             laviet_hal_status_text(init_status),
00887             (unsigned int)len,
00888             (unsigned int)sizeof(input_bytes));
00889         laviet_crypto_log_cryp_context("after_init_fail");
00890         goto cleanup;
00891     }
00892
00893     encrypt_status = HAL_CRYPT_Encrypt(&hcryp,
00894         input_words,
00895         sizeof(input_bytes),
00896         output_words,
00897         LAVIET_CRYPTO_TIMEOUT_MS);
00898     if (encrypt_status == HAL_OK)
00899     {
00900         laviet_crypto_unpack_be_words(output_bytes, output_words);
00901         memcpy(data, output_bytes, len);
00902         ok = true;
00903     }
00904     else
00905     {
00906         printf("CRYPTO: HW AES CTR encrypt failed st=%d(%) payload_len=%u block_len=%u
00907         timeout=%lu\r\n",
00908             (int)encrypt_status,
00909             laviet_hal_status_text(encrypt_status),
00910             (unsigned int)len,
00911             (unsigned int)sizeof(input_bytes),
00912             (unsigned long)LAVIET_CRYPTO_TIMEOUT_MS);
00913         laviet_crypto_log_cryp_context("after_encrypt_fail");
00914     }
00915 cleanup:
00916     laviet_secure_zero(iv_bytes, sizeof(iv_bytes));
00917     laviet_secure_zero(input_bytes, sizeof(input_bytes));
00918     laviet_secure_zero(output_bytes, sizeof(output_bytes));
00919     laviet_secure_zero(key_words, sizeof(key_words));
00920     laviet_secure_zero(iv_words, sizeof(iv_words));
00921     laviet_secure_zero(input_words, sizeof(input_words));
00922     laviet_secure_zero(output_words, sizeof(output_words));
00923     return ok;
00924 #else
00925     (void)data;
00926     (void)len;
00927     (void)key;
00928     (void)frame;
00929     return false;
00930 #endif
00931 }
00932
00933 static bool laviet_crypto_self_test_locked(void)
00934 {
00935     static const uint8_t s_test_key[32] =
00936     {
00937         0x00U, 0x11U, 0x22U, 0x33U, 0x44U, 0x55U, 0x66U, 0x77U,
00938         0x88U, 0x99U, 0xAAU, 0xBBU, 0xCCU, 0xDDU, 0xEEU, 0xFFU,
00939         0x10U, 0x21U, 0x32U, 0x43U, 0x54U, 0x65U, 0x76U, 0x87U,
00940         0x98U, 0xA9U, 0xBAU, 0xCBU, 0xDCU, 0xEDU, 0xFEU, 0x0FU
00941     };
00942     static const uint8_t s_test_data[19] =
00943     {
00944         (uint8_t)'L', (uint8_t)'A', (uint8_t)'V', (uint8_t)'I', (uint8_t)'E',
00945         (uint8_t)'T', (uint8_t)'\_', (uint8_t)'H', (uint8_t)'W', (uint8_t)'\_',
00946         (uint8_t)'T', (uint8_t)'E', (uint8_t)'S', (uint8_t)'T', 0x01U, 0x02U,
00947         0x03U, 0x04U, 0x05U
00948     };
00949     static const laviet_frame_t s_test_frame =
00950     {
00951         .ver_type = 0x11U,
00952         .flags = LAVIET_FLAG_ENCRYPTED,
00953         .src_id = 0x1234U,
00954         .dst_id = 0x0001U,
00955         .msg_id = 0x4567U,
00956         .counter = 0x89ABCDEFUL
00957     };
00958     uint8_t hmac_sw[LAVIET_SHA256_LEN];
00959     uint8_t hmac_hw[LAVIET_SHA256_LEN];
00960     uint8_t aes_sw_l[1] = { 0xA5U };
00961     uint8_t aes_hw_l[1] = { 0xA5U };
00962     uint8_t aes_sw_l6[16];
00963     uint8_t aes_hw_l6[16];
00964     uint8_t i;
00965     memset(hmac_sw, 0, sizeof(hmac_sw));

```

```

00968     memset(hmac_hw, 0, sizeof(hmac_hw));
00969     for (i = 0U; i < sizeof(aes_sw_16); i++)
00970     {
00971         aes_sw_16[i] = (uint8_t)(i * 7U);
00972         aes_hw_16[i] = aes_sw_16[i];
00973     }
00974
00975     if (!laviyet_hmac_sha256_sw(s_test_key, sizeof(s_test_key), s_test_data, sizeof(s_test_data),
hmac_sw))
00976     {
00977         printf("CRYPTO: self-test failed stage=hmac_sw\r\n");
00978         return false;
00979     }
00980     if (!laviyet_hmac_sha256_hw_locked(s_test_key, sizeof(s_test_key), s_test_data,
sizeof(s_test_data), hmac_hw))
00981     {
00982         printf("CRYPTO: self-test failed stage=hmac_hw\r\n");
00983         return false;
00984     }
00985     if (!laviyet_crypto_log_mismatch("hmac_compare", hmac_sw, hmac_hw, sizeof(hmac_sw)))
00986     {
00987         return false;
00988     }
00989
00990     if (!laviyet_aes_ctr_crypt_sw(NULL, 0U, s_test_key, &s_test_frame))
00991     {
00992         printf("CRYPTO: self-test failed stage=aes_sw_zero_len\r\n");
00993         return false;
00994     }
00995     if (!laviyet_aes_ctr_crypt_hw_locked(NULL, 0U, s_test_key, &s_test_frame))
00996     {
00997         printf("CRYPTO: self-test failed stage=aes_hw_zero_len\r\n");
00998         return false;
00999     }
01000
01001     if (!laviyet_aes_ctr_crypt_sw(aes_sw_1, sizeof(aes_sw_1), s_test_key, &s_test_frame))
01002     {
01003         printf("CRYPTO: self-test failed stage=aes_sw_1B_encrypt\r\n");
01004         return false;
01005     }
01006     if (!laviyet_aes_ctr_crypt_hw_locked(aes_hw_1, sizeof(aes_hw_1), s_test_key, &s_test_frame))
01007     {
01008         printf("CRYPTO: self-test failed stage=aes_hw_1B_encrypt\r\n");
01009         return false;
01010     }
01011     if (!laviyet_crypto_log_mismatch("aes_1B_compare", aes_sw_1, aes_hw_1, sizeof(aes_sw_1)))
01012     {
01013         return false;
01014     }
01015
01016     if (!laviyet_aes_ctr_crypt_sw(aes_sw_16, sizeof(aes_sw_16), s_test_key, &s_test_frame))
01017     {
01018         printf("CRYPTO: self-test failed stage=aes_sw_16B_encrypt\r\n");
01019         return false;
01020     }
01021     if (!laviyet_aes_ctr_crypt_hw_locked(aes_hw_16, sizeof(aes_hw_16), s_test_key, &s_test_frame))
01022     {
01023         printf("CRYPTO: self-test failed stage=aes_hw_16B_encrypt\r\n");
01024         return false;
01025     }
01026     if (!laviyet_crypto_log_mismatch("aes_16B_compare", aes_sw_16, aes_hw_16, sizeof(aes_sw_16)))
01027     {
01028         return false;
01029     }
01030
01031     if (!laviyet_aes_ctr_crypt_hw_locked(aes_hw_16, sizeof(aes_hw_16), s_test_key, &s_test_frame))
01032     {
01033         printf("CRYPTO: self-test failed stage=aes_hw_16B_decrypt\r\n");
01034         return false;
01035     }
01036     if (!laviyet_aes_ctr_crypt_sw(aes_sw_16, sizeof(aes_sw_16), s_test_key, &s_test_frame))
01037     {
01038         printf("CRYPTO: self-test failed stage=aes_sw_16B_decrypt\r\n");
01039         return false;
01040     }
01041     if (!laviyet_crypto_log_mismatch("aes_roundtrip_compare", aes_sw_16, aes_hw_16, sizeof(aes_sw_16)))
01042     {
01043         return false;
01044     }
01045
01046     return true;
01047 }
01048
01049 bool laviyet_crypto_init(void)
01050 {
01051     if (s_crypto_mutex == NULL)
01052     {

```

```

01053     s_crypto_mutex = osMutexNew(NULL);
01054     if (s_crypto_mutex == NULL)
01055     {
01056         printf("CRYPTO: mutex create failed, using software backend\r\n");
01057         s_crypto_hw_ready = false;
01058         return true;
01059     }
01060 }
01061
01062 s_crypto_hw_ready = false;
01063
01064 #if (LAVIET_TEST_FIXED_FRAME_MAC == 1U)
01065     printf("CRYPTO: TEST MODE fixed frame MAC enabled (tag=01 x 32)\r\n");
01066 #endif
01067
01068 #if defined(HAL_CRYPT_MODULE_ENABLED) && defined(HAL_HASH_MODULE_ENABLED)
01069     if ((hccryp.Instance != AES) ||
01070         (HAL_CRYPT_GetState(&hccryp) != HAL_CRYPT_STATE_READY) ||
01071         (HAL_HASH_GetState(&hhash) != HAL_HASH_STATE_READY))
01072     {
01073         printf("CRYPTO: hardware backend not ready, using software fallback\r\n");
01074         laviet_crypto_log_cryp_context("init_check");
01075         laviet_crypto_log_hash_context("init_check");
01076         return true;
01077     }
01078
01079     laviet_crypto_log_cryp_context("pre_self_test");
01080     laviet_crypto_log_hash_context("pre_self_test");
01081
01082     if (!laviet_crypto_lock())
01083     {
01084         printf("CRYPTO: hardware lock unavailable, using software fallback\r\n");
01085         return true;
01086     }
01087
01088     s_crypto_hw_ready = laviet_crypto_self_test_locked();
01089     laviet_crypto_unlock();
01090
01091     if (s_crypto_hw_ready)
01092     {
01093         printf("CRYPTO: hardware backend ready\r\n");
01094     }
01095     else
01096     {
01097         printf("CRYPTO: hardware self-test failed, using software fallback\r\n");
01098         laviet_crypto_log_cryp_context("post_self_test_fail");
01099         laviet_crypto_log_hash_context("post_self_test_fail");
01100     }
01101
01102     return true;
01103 #else
01104     printf("CRYPTO: hardware backend disabled, using software backend\r\n");
01105     return true;
01106 #endif
01107 }
01108
01109 bool laviet_hmac_sha256(const uint8_t *key,
01110                        uint16_t key_len,
01111                        const uint8_t *data,
01112                        uint16_t data_len,
01113                        uint8_t out[LAVIET_SHA256_LEN])
01114 {
01115     if ((key == NULL) || (out == NULL) || ((data == NULL) && (data_len > 0U)))
01116     {
01117         return false;
01118     }
01119
01120     if (s_crypto_hw_ready && laviet_crypto_lock())
01121     {
01122         bool ok = laviet_hmac_sha256_hw_locked(key, key_len, data, data_len, out);
01123
01124         laviet_crypto_unlock();
01125         if (ok)
01126         {
01127             return true;
01128         }
01129     }
01130
01131     return laviet_hmac_sha256_sw(key, key_len, data, data_len, out);
01132 }
01133
01134 bool laviet_aes_ctr_crypt(uint8_t *data,
01135                           uint8_t len,
01136                           const uint8_t key[LAVIET_AES_KEY_LEN],
01137                           const laviet_frame_t *frame)
01138 {
01139     uint8_t temp[16];

```

```

01140
01141     if (((data == NULL) && (len > 0U)) || (key == NULL) || (frame == NULL))
01142     {
01143         return false;
01144     }
01145
01146     if (s_crypto_hw_ready && (len <= sizeof(temp)) && laviet_crypto_lock())
01147     {
01148         bool ok;
01149
01150         memset(temp, 0, sizeof(temp));
01151         if (len > 0U)
01152         {
01153             memcpy(temp, data, len);
01154         }
01155         ok = laviet_aes_ctr_crypt_hw_locked(temp, len, key, frame);
01156         laviet_crypto_unlock();
01157         if (ok)
01158         {
01159             if (len > 0U)
01160             {
01161                 memcpy(data, temp, len);
01162             }
01163             laviet_secure_zero(temp, sizeof(temp));
01164             return true;
01165         }
01166         laviet_secure_zero(temp, sizeof(temp));
01167     }
01168
01169     return laviet_aes_ctr_crypt_sw(data, len, key, frame);
01170 }
01171
01172 bool laviet_mac_equal(const uint8_t a[LAVIET_MAC_TAG_LEN],
01173                      const uint8_t b[LAVIET_MAC_TAG_LEN])
01174 {
01175     uint8_t diff = 0U;
01176     uint8_t i;
01177
01178     if ((a == NULL) || (b == NULL))
01179     {
01180         return false;
01181     }
01182
01183     for (i = 0U; i < LAVIET_MAC_TAG_LEN; i++)
01184     {
01185         diff |= (uint8_t)(a[i] ^ b[i]);
01186     }
01187
01188     return diff == 0U;
01189 }
01190
01191 void laviet_secure_zero(void *ptr, size_t len)
01192 {
01193     volatile uint8_t *p = (volatile uint8_t *)ptr;
01194
01195     if (ptr == NULL)
01196     {
01197         return;
01198     }
01199     while (len > 0U)
01200     {
01201         *p = 0U;
01202         p++;
01203         len--;
01204     }
01205 }

```

12.22 Dokumentacja pliku Core/App/laviet_crypto.h

Crypto helpers for LAVIET_FRAME_V1.

```

#include "laviet_frame.h"
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

```

Definicje

- #define LAVIET_AES_KEY_LEN 16U

AES-128 key length used by LAVIET frame encryption.

- `#define LAVIET_HMAC_KEY_LEN 32U`

HMAC-SHA256 key length used by LAVIET frame authentication.

- `#define LAVIET_SHA256_LEN 32U`

SHA-256 digest length in bytes.

Funkcje

- `bool laviyet_crypto_init (void)`

Initialize crypto backends and run the optional hardware self-test.

- `bool laviyet_hmac_sha256 (const uint8_t *key, uint16_t key_len, const uint8_t *data, uint16_t data_len, uint8_t out[LAVIET_SHA256_LEN])`

Calculate HMAC-SHA256 over an arbitrary buffer.

- `bool laviyet_frame_hmac_sha256 (const laviyet_frame_t *frame, const uint8_t key[LAVIET_HMAC_KEY_LEN], uint8_t out[LAVIET_MAC_TAG_LEN])`

Calculate the LAVIET frame MAC over the serialized MAC input.

- `bool laviyet_aes_ctr_crypt (uint8_t *data, uint8_t len, const uint8_t key[LAVIET_AES_KEY_LEN], const laviyet_frame_t *frame)`

Encrypt or decrypt a LAVIET payload in-place with AES-CTR.

- `bool laviyet_mac_equal (const uint8_t a[LAVIET_MAC_TAG_LEN], const uint8_t b[LAVIET_MAC_TAG_LEN])`

Compare two MAC tags without data-dependent early exit.

- `void laviyet_secure_zero (void *ptr, size_t len)`

Clear sensitive memory through a volatile byte pointer.

12.22.1 Opis szczegółowy

Crypto helpers for LAVIET_FRAME_V1.

Definicja w pliku [laviyet_crypto.h](#).

12.22.2 Dokumentacja definicji

12.22.2.1 LAVIET_AES_KEY_LEN

```
#define LAVIET_AES_KEY_LEN 16U
```

AES-128 key length used by LAVIET frame encryption.

Definicja w linii 20 pliku [laviyet_crypto.h](#).

Odwołania w [laviyet_aes_ctr_crypt\(\)](#), [laviyet_aes_ctr_crypt_hw_locked\(\)](#) i [laviyet_aes_ctr_crypt_sw\(\)](#).

12.22.2.2 LAVIET_HMAC_KEY_LEN

```
#define LAVIET_HMAC_KEY_LEN 32U
```

HMAC-SHA256 key length used by LAVIET frame authentication.

Definicja w linii 22 pliku [laviyet_crypto.h](#).

Odwołania w [laviyet_frame_hmac_sha256\(\)](#).

12.22.2.3 LAVIET_SHA256_LEN

```
#define LAVIET_SHA256_LEN 32U
```

SHA-256 digest length in bytes.

Definicja w linii 24 pliku [laviyet_crypto.h](#).

Odwołania w [laviyet_crypto_self_test_locked\(\)](#), [laviyet_hmac_sha256\(\)](#), [laviyet_hmac_sha256_hw_locked\(\)](#), [laviyet_hmac_sha256_sw\(\)](#) i [laviyet_sha256_final\(\)](#).

12.22.3 Dokumentacja funkcji

12.22.3.1 `laviet_aes_ctr_crypt()`

```
bool laviet_aes_ctr_crypt (
    uint8_t * data,
    uint8_t len,
    const uint8_t key[LAVIET_AES_KEY_LEN],
    const laviet_frame_t * frame)
```

Encrypt or decrypt a LAVIET payload in-place with AES-CTR.

Parametry

<i>data</i>	Payload buffer.
<i>len</i>	Payload length.
<i>key</i>	AES-128 key.
<i>frame</i>	Frame fields used to build the CTR nonce.

Zwraca

`true` when the operation succeeded.

Definicja w linii 1134 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_aes_ctr_crypt_hw_locked\(\)](#), [laviet_aes_ctr_crypt_sw\(\)](#), [LAVIET_AES_KEY_LEN](#), [laviet_crypto_lock\(\)](#), [laviet_crypto_unlock\(\)](#) i [laviet_secure_zero\(\)](#).

12.22.3.2 `laviet_crypto_init()`

```
bool laviet_crypto_init (
    void )
```

Initialize crypto backends and run the optional hardware self-test.

Zwraca

`true` when the software fallback is usable.

Definicja w linii 1049 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_crypto_lock\(\)](#), [laviet_crypto_self_test_locked\(\)](#) i [laviet_crypto_unlock\(\)](#).

12.22.3.3 `laviet_frame_hmac_sha256()`

```
bool laviet_frame_hmac_sha256 (
    const laviet_frame_t * frame,
    const uint8_t key[LAVIET_HMAC_KEY_LEN],
    uint8_t out[LAVIET_MAC_TAG_LEN])
```

Calculate the LAVIET frame MAC over the serialized MAC input.

Parametry

<i>frame</i>	Frame to authenticate.
<i>key</i>	HMAC key.
<i>out</i>	[out] MAC tag buffer.

Zwraca

`true` when the MAC tag was generated.

Definicja w linii 739 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_frame_build_mac_input\(\)](#), [LAVIET_FRAME_HEADER_LEN](#), [LAVIET_HMAC_KEY_LEN](#), [laviet_hmac_sha256\(\)](#), [LAVIET_MAC_TAG_LEN](#), [LAVIET_MAX_PAYLOAD](#) i [laviet_secure_zero\(\)](#).

12.22.3.4 laviet_hmac_sha256()

```
bool laviet_hmac_sha256 (
    const uint8_t * key,
    uint16_t key_len,
    const uint8_t * data,
    uint16_t data_len,
    uint8_t out[LAVIET_SHA256_LEN])
```

Calculate HMAC-SHA256 over an arbitrary buffer.

Parametry

<i>key</i>	HMAC key bytes.
<i>key_len</i>	HMAC key length.
<i>data</i>	Data buffer.
<i>data_len</i>	Data length.
<i>out</i>	[out] 32-byte digest output.

Zwraca

`true` when the digest was generated.

Definicja w linii 1109 pliku [laviet_crypto.c](#).

Odwołuje się do [laviet_crypto_lock\(\)](#), [laviet_crypto_unlock\(\)](#), [laviet_hmac_sha256_hw_locked\(\)](#), [laviet_hmac_sha256_sw\(\)](#) i [LAVIET_SHA256_LEN](#).

Odwołania w [laviet_frame_hmac_sha256\(\)](#).

12.22.3.5 laviet_mac_equal()

```
bool laviet_mac_equal (
    const uint8_t a[LAVIET_MAC_TAG_LEN],
    const uint8_t b[LAVIET_MAC_TAG_LEN])
```

Compare two MAC tags without data-dependent early exit.

Parametry

<i>a</i>	First MAC tag.
<i>b</i>	Second MAC tag.

Zwraca

`true` when tags are identical.

Definicja w linii 1172 pliku [laviet_crypto.c](#).

Odwołuje się do [LAVIET_MAC_TAG_LEN](#).

12.22.3.6 laviet_secure_zero()

```
void laviet_secure_zero (
    void * ptr,
    size_t len)
```

Clear sensitive memory through a volatile byte pointer.

Parametry

<i>ptr</i>	Buffer to wipe.
<i>len</i>	Number of bytes to clear.

Definicja w linii 1191 pliku [laviet_crypto.c](#).

Odwolania w [laviet_aes_ctr_crypt\(\)](#), [laviet_aes_ctr_crypt_hw_locked\(\)](#), [laviet_aes_ctr_crypt_sw\(\)](#), [laviet_aes_encrypt_block\(\)](#), [laviet_frame_hmac_sha256\(\)](#), [laviet_hmac_sha256_sw\(\)](#) i [security_main_get_peer_link_key\(\)](#).

12.23 laviet_crypto.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_LAVIET_CRYPT_H_
00007 #define APP_LAVIET_CRYPT_H_
00008
00009 #include "laviet_frame.h"
00010
00011 #include <stdbool.h>
00012 #include <stddef.h>
00013 #include <stdint.h>
00014
00015 #ifdef __cplusplus
00016 extern "C" {
00017 #endif
00018
00020 #define LAVIET_AES_KEY_LEN      16U
00022 #define LAVIET_HMAC_KEY_LEN     32U
00024 #define LAVIET_SHA256_LEN      32U
00025
00030 bool laviet_crypto_init(void);
00031
00041 bool laviet_hmac_sha256(const uint8_t *key,
00042                        uint16_t key_len,
00043                        const uint8_t *data,
00044                        uint16_t data_len,
00045                        uint8_t out[LAVIET_SHA256_LEN]);
00046
00054 bool laviet_frame_hmac_sha256(const laviet_frame_t *frame,
00055                              const uint8_t key[LAVIET_HMAC_KEY_LEN],
00056                              uint8_t out[LAVIET_MAC_TAG_LEN]);
00057
00066 bool laviet_aes_ctr_crypt(uint8_t *data,
00067                          uint8_t len,
00068                          const uint8_t key[LAVIET_AES_KEY_LEN],
00069                          const laviet_frame_t *frame);
00070
00077 bool laviet_mac_equal(const uint8_t a[LAVIET_MAC_TAG_LEN],
00078                      const uint8_t b[LAVIET_MAC_TAG_LEN]);
00079
00085 void laviet_secure_zero(void *ptr, size_t len);
00086
00087 #ifdef __cplusplus
00088 }
00089 #endif
00090
00091 #endif /* APP_LAVIET_CRYPT_H_ */
```

12.24 Dokumentacja pliku Core/App/laviet_frame.c

LAVIET_FRAME_V1 parser, serializer and cheap validation.

```
#include "laviet_frame.h"
#include "stm32u5xx_hal.h"
```

```
#include <string.h>
```

Funkcje

- static void [laviet_be16_write](#) (uint8_t *dst, uint16_t value)
Internal helper: [laviet_be16_write](#).
- static void [laviet_be32_write](#) (uint8_t *dst, uint32_t value)
Internal helper: [laviet_be32_write](#).
- static uint16_t [laviet_be16_read](#) (const uint8_t *src)
Internal helper: [laviet_be16_read](#).
- static uint32_t [laviet_be32_read](#) (const uint8_t *src)
Internal helper: [laviet_be32_read](#).
- static uint8_t [laviet_frame_version_from_ver_type](#) (uint8_t ver_type)
Internal helper: [laviet_frame_version_from_ver_type](#).
- static bool [laviet_frame_type_known](#) (laviet_frame_type_t type)
Internal helper: [laviet_frame_type_known](#).
- uint8_t [laviet_frame_ver_type](#) (laviet_frame_type_t type)
Compose the version/type byte for a frame type.
- [laviet_frame_type_t](#) [laviet_frame_type](#) (const [laviet_frame_t](#) *frame)
Extract the low-nibble frame type from a parsed frame.
- bool [laviet_frame_is_broadcast](#) (const [laviet_frame_t](#) *frame)
Check whether a frame targets the broadcast address.
- uint16_t [laviet_local_node_id](#) (void)
Derive a stable local node identifier from STM32 unique ID words.
- [laviet_frame_status_t](#) [laviet_frame_validate_plain](#) (const [laviet_frame_t](#) *frame)
Validate parsed frame fields before encryption/MAC checks.
- [laviet_frame_status_t](#) [laviet_frame_encode](#) (const [laviet_frame_t](#) *frame, uint8_t *out, uint8_t out_capacity, uint8_t *out_len)
Serialize a parsed frame into wire format.
- [laviet_frame_status_t](#) [laviet_frame_decode](#) (const uint8_t *in, uint8_t in_len, [laviet_frame_t](#) *frame_out)
Parse a serialized wire frame into a frame structure.
- bool [laviet_frame_build_mac_input](#) (const [laviet_frame_t](#) *frame, uint8_t *out, uint8_t out_capacity, uint8_t *out_len)
Build the canonical byte sequence covered by the frame MAC.
- bool [laviet_frame_write_ack_payload](#) (uint16_t acked_msg_id, uint32_t acked_counter, uint8_t out[LAVIET_ACK_PAYLOAD_LEN])
Encode an ACK payload.
- bool [laviet_frame_read_ack_payload](#) (const uint8_t payload[LAVIET_ACK_PAYLOAD_LEN], uint16_t *acked_msg_id, uint32_t *acked_counter)
Decode an ACK payload.

12.24.1 Opis szczegółowy

LAVIET_FRAME_V1 parser, serializer and cheap validation.
Definicja w pliku [laviet_frame.c](#).

12.24.2 Dokumentacja funkcji

12.24.2.1 laviet_be16_read()

```
uint16_t laviet_be16_read (
    const uint8_t * src) [static]
```

Internal helper: [laviet_be16_read](#).

Definicja w linii 29 pliku [laviet_frame.c](#).

Odwwołania w [laviet_frame_decode\(\)](#) i [laviet_frame_read_ack_payload\(\)](#).

12.24.2.2 laviet_be16_write()

```
void laviet_be16_write (
    uint8_t * dst,
    uint16_t value) [static]
```

Internal helper: [laviet_be16_write](#).

Definicja w linii 13 pliku [laviet_frame.c](#).

Odwołania w [laviet_frame_build_mac_input\(\)](#), [laviet_frame_encode\(\)](#) i [laviet_frame_write_ack_payload\(\)](#).

12.24.2.3 laviet_be32_read()

```
uint32_t laviet_be32_read (
    const uint8_t * src) [static]
```

Internal helper: [laviet_be32_read](#).

Definicja w linii 35 pliku [laviet_frame.c](#).

Odwołania w [laviet_frame_decode\(\)](#) i [laviet_frame_read_ack_payload\(\)](#).

12.24.2.4 laviet_be32_write()

```
void laviet_be32_write (
    uint8_t * dst,
    uint32_t value) [static]
```

Internal helper: [laviet_be32_write](#).

Definicja w linii 20 pliku [laviet_frame.c](#).

Odwołania w [laviet_frame_build_mac_input\(\)](#), [laviet_frame_encode\(\)](#) i [laviet_frame_write_ack_payload\(\)](#).

12.24.2.5 laviet_frame_build_mac_input()

```
bool laviet_frame_build_mac_input (
    const laviet_frame_t * frame,
    uint8_t * out,
    uint8_t out_capacity,
    uint8_t * out_len)
```

Build the canonical byte sequence covered by the frame MAC.

Parametry

<i>frame</i>	Frame whose header and payload are used.
<i>out</i>	Output buffer.
<i>out_capacity</i>	Output buffer capacity.
<i>out_len</i>	[out] MAC input length.

Zwraca

`true` when the MAC input was built.

Definicja w linii 378 pliku [laviet_frame.c](#).

Odwołuje się do [laviet_frame_t::counter](#), [laviet_frame_t::dst_id](#), [laviet_frame_t::flags](#), [laviet_be16_write\(\)](#), [laviet_be32_write\(\)](#), [LAVIET_FRAME_HEADER_LEN](#), [LAVIET_MAX_PAYLOAD](#), [laviet_frame_t::msg_id](#), [laviet_frame_t::payload](#), [laviet_frame_t::payload_len](#), [laviet_frame_t::src_id](#) i [laviet_frame_t::ver_type](#).

Odwołania w [laviet_frame_hmac_sha256\(\)](#).

12.24.2.6 laviet_frame_decode()

```
laviet_frame_status_t laviet_frame_decode (
    const uint8_t * in,
    uint8_t in_len,
    laviet_frame_t * frame_out)
```

Parse a serialized wire frame into a frame structure.

Parametry

<i>in</i>	Serialized frame bytes.
<i>in_len</i>	Serialized frame length.
<i>frame_out</i>	[out] Parsed frame.

Zwraca

Decode status.

Definicja w linii 328 pliku [laviet_frame.c](#).

Odwołuje się do [laviet_frame_t::counter](#), [laviet_frame_t::dst_id](#), [laviet_frame_t::flags](#), [laviet_be16_read\(\)](#), [laviet_be32_read\(\)](#), [LAVIET_FRAME_HEADER_LEN](#), [LAVIET_FRAME_MAX_LEN](#), [LAVIET_FRAME_MIN_LEN](#), [laviet_frame_validate_plain\(\)](#), [LAVIET_MAC_TAG_LEN](#), [LAVIET_MAX_PAYLOAD](#), [LAVIET_STATUS_LENGTH](#), [LAVIET_STATUS_NULL](#), [LAVIET_STATUS_OK](#), [LAVIET_STATUS_PAYLOAD](#), [laviet_frame_t::mac_tag](#), [laviet_frame_t::msg_id](#), [laviet_frame_t::payload](#), [laviet_frame_t::payload_len](#), [laviet_frame_t::src_id](#) i [laviet_frame_t::ver_type](#).

12.24.2.7 laviet_frame_encode()

```
laviet_frame_status_t laviet_frame_encode (
    const laviet_frame_t * frame,
    uint8_t * out,
    uint8_t out_capacity,
    uint8_t * out_len)
```

Serialize a parsed frame into wire format.

Parametry

<i>frame</i>	Frame to serialize.
<i>out</i>	Output buffer.
<i>out_capacity</i>	Output buffer capacity.
<i>out_len</i>	[out] Serialized frame length.

Zwraca

Encode status.

Definicja w linii 287 pliku [laviet_frame.c](#).

Odwołuje się do [laviet_frame_t::counter](#), [laviet_frame_t::dst_id](#), [laviet_frame_t::flags](#), [laviet_be16_write\(\)](#), [laviet_be32_write\(\)](#), [LAVIET_FRAME_HEADER_LEN](#), [laviet_frame_validate_plain\(\)](#), [LAVIET_MAC_TAG_LEN](#), [LAVIET_STATUS_LENGTH](#), [LAVIET_STATUS_NULL](#), [LAVIET_STATUS_OK](#), [laviet_frame_t::mac_tag](#), [laviet_frame_t::msg_id](#), [laviet_frame_t::payload](#), [laviet_frame_t::payload_len](#), [laviet_frame_t::src_id](#) i [laviet_frame_t::ver_type](#).

12.24.2.8 laviet_frame_is_broadcast()

```
bool laviet_frame_is_broadcast (
    const laviet_frame_t * frame)
```

Check whether a frame targets the broadcast address.

Parametry

<i>frame</i>	Frame object.
--------------	---------------

Zwraca

`true` when `dst_id` equals `LAVIET_BROADCAST_ID`.

Definicja w linii 70 pliku `laviet_frame.c`.

Odwołuje się do `laviet_frame_t::dst_id` i `LAVIET_BROADCAST_ID`.

12.24.2.9 laviet_frame_read_ack_payload()

```
bool laviet_frame_read_ack_payload (
    const uint8_t payload[LAVIET_ACK_PAYLOAD_LEN],
    uint16_t * acked_msg_id,
    uint32_t * acked_counter)
```

Decode an ACK payload.

Parametry

<i>payload</i>	ACK payload bytes.
<i>acked_msg_id</i>	[out] Decoded message identifier.
<i>acked_counter</i>	[out] Decoded counter value.

Zwraca

`true` when the payload was decoded.

Definicja w linii 429 pliku `laviet_frame.c`.

Odwołuje się do `LAVIET_ACK_PAYLOAD_LEN`, `laviet_be16_read()` i `laviet_be32_read()`.

12.24.2.10 laviet_frame_type()

```
laviet_frame_type_t laviet_frame_type (
    const laviet_frame_t * frame)
```

Extract the low-nibble frame type from a parsed frame.

Parametry

<i>frame</i>	Frame object.
--------------	---------------

Zwraca

Frame type or `LAVIET_TYPE_NONE` for a null frame.

Definicja w linii 60 pliku `laviet_frame.c`.

Odwołuje się do `LAVIET_TYPE_NONE` i `laviet_frame_t::ver_type`.

Odwołania w `laviet_frame_validate_plain()`.

12.24.2.11 laviet_frame_type_known()

```
bool laviet_frame_type_known (
    laviet_frame_type_t type) [static]
```

Internal helper: `laviet_frame_type_known`.

Definicja w linii 50 pliku `laviet_frame.c`.

Odwołuje się do `LAVIET_TYPE_DATA` i `LAVIET_TYPE_ERROR`.

Odwołania w `laviet_frame_validate_plain()`.

12.24.2.12 laviet_frame_validate_plain()

```
laviet_frame_status_t laviet_frame_validate_plain (
    const laviet_frame_t * frame)
```

Validate parsed frame fields before encryption/MAC checks.

Parametry

<i>frame</i>	Frame object.
--------------	---------------

Zwraca

Validation status.

Definicja w linii 120 pliku [laviet_frame.c](#).

Odwołuje się do [laviet_frame_t::dst_id](#), [laviet_frame_t::flags](#), [LAVIET_ACK_PAYLOAD_LEN](#), [LAVIET_BROADCAST_ID](#), [LAVIET_COUNTER_SYNC_PAYLOAD_LEN](#), [LAVIET_ERROR_PAYLOAD_LEN](#), [LAVIET_FLAG_ACK_REQUIRED](#), [LAVIET_FLAG_BROADCAST](#), [LAVIET_FLAG_CONFIG_ACCESS](#), [LAVIET_FLAG_COUNTER_OVERRIDE](#), [LAVIET_FLAG_ENCRYPTED](#), [LAVIET_FLAG_IS_ACK](#), [LAVIET_FLAG_KEY_UPDATE](#), [LAVIET_FLAG_PAIRING](#), [laviet_frame_type\(\)](#), [laviet_frame_type_known\(\)](#), [LAVIET_FRAME_VERSION](#), [laviet_frame_version_from_ver_type\(\)](#), [LAVIET_GATEWAY_ID](#), [LAVIET_MAX_PAYLOAD](#), [LAVIET_PAIR_PAYLOAD_LEN](#), [LAVIET_STATUS_FLAGS](#), [LAVIET_STATUS_ID](#), [LAVIET_STATUS_NULL](#), [LAVIET_STATUS_OK](#), [LAVIET_STATUS_PAYLOAD](#), [LAVIET_STATUS_TYPE](#), [LAVIET_STATUS_VERSION](#), [LAVIET_TYPE_ACK](#), [LAVIET_TYPE_CFG](#), [LAVIET_TYPE_COUNTER_SYNC](#), [LAVIET_TYPE_DATA](#), [LAVIET_TYPE_ERROR](#), [LAVIET_TYPE_KEY_ROTATE](#), [LAVIET_TYPE_PAIR_REQ](#), [LAVIET_TYPE_PAIR_RESP](#), [LAVIET_TYPE_RESP](#), [laviet_frame_t::msg_id](#), [laviet_frame_t::payload_len](#), [laviet_frame_t::src_id](#) i [laviet_frame_t::ver_type](#).

Odwołania w [laviet_frame_decode\(\)](#) i [laviet_frame_encode\(\)](#).

12.24.2.13 laviet_frame_ver_type()

```
uint8_t laviet_frame_ver_type (
    laviet_frame_type_t type)
```

Compose the version/type byte for a frame type.

Parametry

<i>type</i>	LAVIET frame type.
-------------	--------------------

Zwraca

Encoded `ver_type` byte.

Definicja w linii 55 pliku [laviet_frame.c](#).

Odwołuje się do [LAVIET_FRAME_VERSION](#).

12.24.2.14 laviet_frame_version_from_ver_type()

```
uint8_t laviet_frame_version_from_ver_type (
    uint8_t ver_type) [static]
```

Internal helper: [laviet_frame_version_from_ver_type](#).

Definicja w linii 44 pliku [laviet_frame.c](#).

Odwołania w [laviet_frame_validate_plain\(\)](#).

12.24.2.15 laviet_frame_write_ack_payload()

```
bool laviet_frame_write_ack_payload (
    uint16_t acked_msg_id,
```

```
uint32_t acked_counter,
uint8_t out[LAVIET_ACK_PAYLOAD_LEN])
```

Encode an ACK payload.

Parametry

<i>acked_msg↔ _id</i>	Message identifier being acknowledged.
<i>acked_counter</i>	Counter value being acknowledged.
<i>out</i>	[out] Fixed-size ACK payload buffer.

Zwraca

true when the payload was written.

Definicja w linii 415 pliku [laviet_frame.c](#).

Odwołuje się do [LAVIET_ACK_PAYLOAD_LEN](#), [laviet_be16_write\(\)](#) i [laviet_be32_write\(\)](#).

12.24.2.16 laviet_local_node_id()

```
uint16_t laviet_local_node_id (
    void )
```

Derive a stable local node identifier from STM32 unique ID words.

Zwraca

Non-reserved 16-bit node identifier.

Definicja w linii 80 pliku [laviet_frame.c](#).

Odwołuje się do [LAVIET_BROADCAST_ID](#) i [LAVIET_GATEWAY_ID](#).

12.25 laviet_frame.c

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #include "laviet_frame.h"
00007
00008 #include "stm32u5xx_hal.h"
00009
00010 #include <string.h>
00011
00013 static void laviet_be16_write(uint8_t *dst, uint16_t value)
00014 {
00015     dst[0] = (uint8_t)(value >> 8);
00016     dst[1] = (uint8_t)value;
00017 }
00018
00020 static void laviet_be32_write(uint8_t *dst, uint32_t value)
00021 {
00022     dst[0] = (uint8_t)(value >> 24);
00023     dst[1] = (uint8_t)(value >> 16);
00024     dst[2] = (uint8_t)(value >> 8);
00025     dst[3] = (uint8_t)value;
00026 }
00027
00029 static uint16_t laviet_be16_read(const uint8_t *src)
00030 {
00031     return (uint16_t)((uint16_t)src[0] << 8) | src[1];
00032 }
00033
00035 static uint32_t laviet_be32_read(const uint8_t *src)
00036 {
00037     return ((uint32_t)src[0] << 24) |
00038           ((uint32_t)src[1] << 16) |
00039           ((uint32_t)src[2] << 8) |
00040           (uint32_t)src[3];
00041 }
00042
```

```

00044 static uint8_t laviet_frame_version_from_ver_type(uint8_t ver_type)
00045 {
00046     return (uint8_t)((ver_type » 4) & 0x0FU);
00047 }
00048
00050 static bool laviet_frame_type_known(laviet_frame_type_t type)
00051 {
00052     return (type >= LAVIET_TYPE_DATA) && (type <= LAVIET_TYPE_ERROR);
00053 }
00054
00055 uint8_t laviet_frame_ver_type(laviet_frame_type_t type)
00056 {
00057     return (uint8_t)(((uint8_t)LAVIET_FRAME_VERSION « 4) | ((uint8_t)type & 0x0FU));
00058 }
00059
00060 laviet_frame_type_t laviet_frame_type(const laviet_frame_t *frame)
00061 {
00062     if (frame == NULL)
00063     {
00064         return LAVIET_TYPE_NONE;
00065     }
00066     return (laviet_frame_type_t)(frame->ver_type & 0x0FU);
00067 }
00068
00069
00070 bool laviet_frame_is_broadcast(const laviet_frame_t *frame)
00071 {
00072     if (frame == NULL)
00073     {
00074         return false;
00075     }
00076     return (frame->dst_id == LAVIET_BROADCAST_ID);
00077 }
00078
00079
00080 uint16_t laviet_local_node_id(void)
00081 {
00082     uint32_t uid0 = HAL_GetUIDw0();
00083     uint32_t uid1 = HAL_GetUIDw1();
00084     uint32_t uid2 = HAL_GetUIDw2();
00085     uint32_t hash = 2166136261UL;
00086     uint8_t i;
00087     const uint8_t *uid_bytes = (const uint8_t *)&uid0;
00088
00089     for (i = 0U; i < 4U; i++)
00090     {
00091         hash ^= uid_bytes[i];
00092         hash *= 16777619UL;
00093     }
00094     uid_bytes = (const uint8_t *)&uid1;
00095     for (i = 0U; i < 4U; i++)
00096     {
00097         hash ^= uid_bytes[i];
00098         hash *= 16777619UL;
00099     }
00100     uid_bytes = (const uint8_t *)&uid2;
00101     for (i = 0U; i < 4U; i++)
00102     {
00103         hash ^= uid_bytes[i];
00104         hash *= 16777619UL;
00105     }
00106
00107     hash = (hash ^ (hash » 16)) & 0xFFFFUL;
00108     if ((hash == 0U) || (hash == LAVIET_GATEWAY_ID) || (hash == LAVIET_BROADCAST_ID))
00109     {
00110         hash = (uint32_t)(0x1000U | (hash & 0x0FFFU));
00111         if ((hash == 0U) || (hash == LAVIET_GATEWAY_ID) || (hash == LAVIET_BROADCAST_ID))
00112         {
00113             hash = 0x1002U;
00114         }
00115     }
00116
00117     return (uint16_t)hash;
00118 }
00119
00120 laviet_frame_status_t laviet_frame_validate_plain(const laviet_frame_t *frame)
00121 {
00122     laviet_frame_type_t type;
00123     bool broadcast;
00124
00125     if (frame == NULL)
00126     {
00127         return LAVIET_STATUS_NULL;
00128     }
00129
00130     if (laviet_frame_version_from_ver_type(frame->ver_type) != LAVIET_FRAME_VERSION)
00131     {

```

```

00132         return LAVIET_STATUS_VERSION;
00133     }
00134
00135     type = laviet_frame_type(frame);
00136     if (!laviet_frame_type_known(type))
00137     {
00138         return LAVIET_STATUS_TYPE;
00139     }
00140
00141     if ((frame->src_id == 0U) ||
00142         (frame->src_id == LAVIET_BROADCAST_ID) ||
00143         (frame->dst_id == 0U) ||
00144         (frame->msg_id == 0U))
00145     {
00146         return LAVIET_STATUS_ID;
00147     }
00148
00149     if (frame->payload_len > LAVIET_MAX_PAYLOAD)
00150     {
00151         return LAVIET_STATUS_PAYLOAD;
00152     }
00153
00154     broadcast = (frame->dst_id == LAVIET_BROADCAST_ID);
00155     if (broadcast)
00156     {
00157         if (((frame->flags & LAVIET_FLAG_BROADCAST) == 0U) ||
00158             ((frame->flags & LAVIET_FLAG_ACK_REQUIRED) != 0U))
00159         {
00160             return LAVIET_STATUS_FLAGS;
00161         }
00162         if ((frame->src_id != LAVIET_GATEWAY_ID) &&
00163             (type != LAVIET_TYPE_DATA) &&
00164             (type != LAVIET_TYPE_RESP) &&
00165             (type != LAVIET_TYPE_PAIR_REQ))
00166         {
00167             return LAVIET_STATUS_FLAGS;
00168         }
00169     }
00170     else if ((frame->flags & LAVIET_FLAG_BROADCAST) != 0U)
00171     {
00172         return LAVIET_STATUS_FLAGS;
00173     }
00174
00175     switch (type)
00176     {
00177     case LAVIET_TYPE_DATA:
00178         if ((frame->payload_len == 0U) || (frame->payload_len > LAVIET_MAX_PAYLOAD))
00179         {
00180             return LAVIET_STATUS_PAYLOAD;
00181         }
00182         if (((frame->flags & LAVIET_FLAG_IS_ACK) != 0U) ||
00183             ((frame->flags & LAVIET_FLAG_PAIRING) != 0U) ||
00184             ((frame->flags & LAVIET_FLAG_CONFIG_ACCESS) != 0U) ||
00185             ((frame->flags & LAVIET_FLAG_COUNTER_OVERRIDE) != 0U) ||
00186             ((frame->flags & LAVIET_FLAG_KEY_UPDATE) != 0U))
00187         {
00188             return LAVIET_STATUS_FLAGS;
00189         }
00190         break;
00191
00192     case LAVIET_TYPE_ACK:
00193         if (frame->payload_len != LAVIET_ACK_PAYLOAD_LEN)
00194         {
00195             return LAVIET_STATUS_PAYLOAD;
00196         }
00197         if (((frame->flags & LAVIET_FLAG_IS_ACK) == 0U) ||
00198             ((frame->flags & LAVIET_FLAG_ACK_REQUIRED) != 0U) ||
00199             ((frame->flags & LAVIET_FLAG_ENCRYPTED) != 0U))
00200         {
00201             return LAVIET_STATUS_FLAGS;
00202         }
00203         break;
00204
00205     case LAVIET_TYPE_RESP:
00206         if ((frame->flags & (LAVIET_FLAG_IS_ACK |
00207                             LAVIET_FLAG_PAIRING |
00208                             LAVIET_FLAG_CONFIG_ACCESS |
00209                             LAVIET_FLAG_COUNTER_OVERRIDE |
00210                             LAVIET_FLAG_KEY_UPDATE)) != 0U)
00211         {
00212             return LAVIET_STATUS_FLAGS;
00213         }
00214         break;
00215
00216     case LAVIET_TYPE_PAIR_REQ:
00217     case LAVIET_TYPE_PAIR_RESP:
00218         if (frame->payload_len != LAVIET_PAIR_PAYLOAD_LEN)

```

```

00219         {
00220             return LAVIET_STATUS_PAYLOAD;
00221         }
00222         if (((frame->flags & LAVIET_FLAG_PAIRING) == 0U) ||
00223             ((frame->flags & LAVIET_FLAG_CONFIG_ACCESS) != 0U) ||
00224             ((frame->flags & LAVIET_FLAG_COUNTER_OVERRIDE) != 0U) ||
00225             ((frame->flags & LAVIET_FLAG_KEY_UPDATE) != 0U))
00226         {
00227             return LAVIET_STATUS_FLAGS;
00228         }
00229         break;
00230
00231     case LAVIET_TYPE_CFG:
00232         if ((frame->payload_len < 2U) || (frame->payload_len > LAVIET_MAX_PAYLOAD))
00233         {
00234             return LAVIET_STATUS_PAYLOAD;
00235         }
00236         if (((frame->flags & LAVIET_FLAG_CONFIG_ACCESS) == 0U) ||
00237             ((frame->flags & LAVIET_FLAG_IS_ACK) != 0U) ||
00238             ((frame->flags & LAVIET_FLAG_PAIRING) != 0U))
00239         {
00240             return LAVIET_STATUS_FLAGS;
00241         }
00242         break;
00243
00244     case LAVIET_TYPE_COUNTER_SYNC:
00245         if (frame->payload_len != LAVIET_COUNTER_SYNC_PAYLOAD_LEN)
00246         {
00247             return LAVIET_STATUS_PAYLOAD;
00248         }
00249         if (((frame->flags & LAVIET_FLAG_COUNTER_OVERRIDE) == 0U) ||
00250             ((frame->flags & LAVIET_FLAG_IS_ACK) != 0U) ||
00251             (frame->src_id != LAVIET_GATEWAY_ID))
00252         {
00253             return LAVIET_STATUS_FLAGS;
00254         }
00255         break;
00256
00257     case LAVIET_TYPE_KEY_ROTATE:
00258         if ((frame->payload_len == 0U) || (frame->payload_len > LAVIET_MAX_PAYLOAD))
00259         {
00260             return LAVIET_STATUS_PAYLOAD;
00261         }
00262         if (((frame->flags & LAVIET_FLAG_KEY_UPDATE) == 0U) ||
00263             ((frame->flags & LAVIET_FLAG_IS_ACK) != 0U))
00264         {
00265             return LAVIET_STATUS_FLAGS;
00266         }
00267         break;
00268
00269     case LAVIET_TYPE_ERROR:
00270         if (frame->payload_len != LAVIET_ERROR_PAYLOAD_LEN)
00271         {
00272             return LAVIET_STATUS_PAYLOAD;
00273         }
00274         if ((frame->flags & LAVIET_FLAG_IS_ACK) != 0U)
00275         {
00276             return LAVIET_STATUS_FLAGS;
00277         }
00278         break;
00279
00280     default:
00281         return LAVIET_STATUS_TYPE;
00282 }
00283
00284 return LAVIET_STATUS_OK;
00285 }
00286
00287 laviet_frame_status_t laviet_frame_encode(const laviet_frame_t *frame,
00288                                           uint8_t *out,
00289                                           uint8_t out_capacity,
00290                                           uint8_t *out_len)
00291 {
00292     laviet_frame_status_t status;
00293     uint8_t total_len;
00294
00295     if ((frame == NULL) || (out == NULL) || (out_len == NULL))
00296     {
00297         return LAVIET_STATUS_NULL;
00298     }
00299
00300     status = laviet_frame_validate_plain(frame);
00301     if (status != LAVIET_STATUS_OK)
00302     {
00303         return status;
00304     }
00305 }

```

```

00306     total_len = (uint8_t)(LAVIET_FRAME_HEADER_LEN + frame->payload_len + LAVIET_MAC_TAG_LEN);
00307     if (out_capacity < total_len)
00308     {
00309         return LAVIET_STATUS_LENGTH;
00310     }
00311
00312     out[0] = frame->ver_type;
00313     out[1] = frame->flags;
00314     laviet_be16_write(&out[2], frame->src_id);
00315     laviet_be16_write(&out[4], frame->dst_id);
00316     laviet_be16_write(&out[6], frame->msg_id);
00317     laviet_be32_write(&out[8], frame->counter);
00318     out[12] = frame->payload_len;
00319     if (frame->payload_len > 0U)
00320     {
00321         memcpy(&out[LAVIET_FRAME_HEADER_LEN], frame->payload, frame->payload_len);
00322     }
00323     memcpy(&out[LAVIET_FRAME_HEADER_LEN + frame->payload_len], frame->mac_tag, LAVIET_MAC_TAG_LEN);
00324     *out_len = total_len;
00325     return LAVIET_STATUS_OK;
00326 }
00327
00328 laviet_frame_status_t laviet_frame_decode(const uint8_t *in,
00329                                           uint8_t in_len,
00330                                           laviet_frame_t *frame_out)
00331 {
00332     uint8_t payload_len;
00333     uint8_t expected_len;
00334     laviet_frame_status_t status;
00335
00336     if ((in == NULL) || (frame_out == NULL))
00337     {
00338         return LAVIET_STATUS_NULL;
00339     }
00340     if ((in_len < LAVIET_FRAME_MIN_LEN) || (in_len > LAVIET_FRAME_MAX_LEN))
00341     {
00342         return LAVIET_STATUS_LENGTH;
00343     }
00344
00345     payload_len = in[12];
00346     if (payload_len > LAVIET_MAX_PAYLOAD)
00347     {
00348         return LAVIET_STATUS_PAYLOAD;
00349     }
00350     expected_len = (uint8_t)(LAVIET_FRAME_HEADER_LEN + payload_len + LAVIET_MAC_TAG_LEN);
00351     if (expected_len != in_len)
00352     {
00353         return LAVIET_STATUS_LENGTH;
00354     }
00355
00356     memset(frame_out, 0, sizeof(*frame_out));
00357     frame_out->ver_type = in[0];
00358     frame_out->flags = in[1];
00359     frame_out->src_id = laviet_be16_read(&in[2]);
00360     frame_out->dst_id = laviet_be16_read(&in[4]);
00361     frame_out->msg_id = laviet_be16_read(&in[6]);
00362     frame_out->counter = laviet_be32_read(&in[8]);
00363     frame_out->payload_len = payload_len;
00364     if (payload_len > 0U)
00365     {
00366         memcpy(frame_out->payload, &in[LAVIET_FRAME_HEADER_LEN], payload_len);
00367     }
00368     memcpy(frame_out->mac_tag, &in[LAVIET_FRAME_HEADER_LEN + payload_len], LAVIET_MAC_TAG_LEN);
00369
00370     status = laviet_frame_validate_plain(frame_out);
00371     if (status != LAVIET_STATUS_OK)
00372     {
00373         memset(frame_out, 0, sizeof(*frame_out));
00374     }
00375     return status;
00376 }
00377
00378 bool laviet_frame_build_mac_input(const laviet_frame_t *frame,
00379                                  uint8_t *out,
00380                                  uint8_t out_capacity,
00381                                  uint8_t *out_len)
00382 {
00383     uint8_t len;
00384
00385     if ((frame == NULL) || (out == NULL) || (out_len == NULL))
00386     {
00387         return false;
00388     }
00389     if (frame->payload_len > LAVIET_MAX_PAYLOAD)
00390     {
00391         return false;
00392     }

```

```

00393
00394     len = (uint8_t)(LAVIET_FRAME_HEADER_LEN + frame->payload_len);
00395     if (out_capacity < len)
00396     {
00397         return false;
00398     }
00399
00400     out[0] = frame->ver_type;
00401     out[1] = frame->flags;
00402     laviet_be16_write(&out[2], frame->src_id);
00403     laviet_be16_write(&out[4], frame->dst_id);
00404     laviet_be16_write(&out[6], frame->msg_id);
00405     laviet_be32_write(&out[8], frame->counter);
00406     out[12] = frame->payload_len;
00407     if (frame->payload_len > 0U)
00408     {
00409         memcpy(&out[LAVIET_FRAME_HEADER_LEN], frame->payload, frame->payload_len);
00410     }
00411     *out_len = len;
00412     return true;
00413 }
00414
00415 bool laviet_frame_write_ack_payload(uint16_t acked_msg_id,
00416                                     uint32_t acked_counter,
00417                                     uint8_t out[LAVIET_ACK_PAYLOAD_LEN])
00418 {
00419     if ((out == NULL) || (acked_msg_id == 0U))
00420     {
00421         return false;
00422     }
00423
00424     laviet_be16_write(&out[0], acked_msg_id);
00425     laviet_be32_write(&out[2], acked_counter);
00426     return true;
00427 }
00428
00429 bool laviet_frame_read_ack_payload(const uint8_t payload[LAVIET_ACK_PAYLOAD_LEN],
00430                                     uint16_t *acked_msg_id,
00431                                     uint32_t *acked_counter)
00432 {
00433     if ((payload == NULL) || (acked_msg_id == NULL) || (acked_counter == NULL))
00434     {
00435         return false;
00436     }
00437
00438     *acked_msg_id = laviet_be16_read(&payload[0]);
00439     *acked_counter = laviet_be32_read(&payload[2]);
00440     return (*acked_msg_id != 0U);
00441 }

```

12.26 Dokumentacja pliku Core/App/laviet_frame.h

LAVIET_FRAME_V1 wire protocol helpers.

```
#include <stdbool.h>
```

```
#include <stdint.h>
```

Struktury danych

- struct `laviet_frame_t`
Parsed LAVIET frame representation.

Definicje

- #define `LAVIET_FRAME_VERSION` 1U
Wire format version encoded in the high nibble of ver_type.
- #define `LAVIET_MAX_PAYLOAD` 16U
Maximum payload bytes carried by one LAVIET frame.
- #define `LAVIET_MAC_TAG_LEN` 32U
Full HMAC-SHA256 tag length stored on the wire.
- #define `LAVIET_FRAME_HEADER_LEN` 13U
Serialized header length before payload and MAC.

- #define LAVIET_FRAME_MIN_LEN (LAVIET_FRAME_HEADER_LEN + LAVIET_MAC_TAG_LEN)
Minimum serialized frame length.
- #define LAVIET_FRAME_MAX_LEN (LAVIET_FRAME_HEADER_LEN + LAVIET_MAX_PAYLOAD + LAVIET_MAC_TAG_LEN)
Maximum serialized frame length.
- #define LAVIET_GATEWAY_ID 0x0001U
Reserved node identifier of the gateway.
- #define LAVIET_BROADCAST_ID 0xFFFFU
Broadcast destination identifier.
- #define LAVIET_FLAG_ENCRYPTED (1U << 0)
Payload is encrypted with the current frame key.
- #define LAVIET_FLAG_ACK_REQUIRED (1U << 1)
Sender requests an ACK frame.
- #define LAVIET_FLAG_IS_ACK (1U << 2)
Frame itself is an ACK.
- #define LAVIET_FLAG_PAIRING (1U << 3)
Frame belongs to pairing flow.
- #define LAVIET_FLAG_CONFIG_ACCESS (1U << 4)
Frame contains configuration/control payload.
- #define LAVIET_FLAG_BROADCAST (1U << 5)
Destination is the broadcast address.
- #define LAVIET_FLAG_COUNTER_OVERRIDE (1U << 6)
Payload carries a counter synchronization override.
- #define LAVIET_FLAG_KEY_UPDATE (1U << 7)
Payload carries key rotation material.
- #define LAVIET_PAIR_PAYLOAD_LEN 8U
Fixed payload length of pair request/response frames.
- #define LAVIET_ACK_PAYLOAD_LEN 6U
Fixed payload length of ACK frames.
- #define LAVIET_COUNTER_SYNC_PAYLOAD_LEN 4U
Fixed payload length of counter synchronization frames.
- #define LAVIET_ERROR_PAYLOAD_LEN 8U
Fixed payload length reserved for error frames.

Wyliczenia

- enum laviet_frame_type_t {
LAVIET_TYPE_NONE = 0x00U , LAVIET_TYPE_DATA = 0x01U , LAVIET_TYPE_ACK = 0x02U ,
LAVIET_TYPE_RESP = 0x03U ,
LAVIET_TYPE_PAIR_REQ = 0x04U , LAVIET_TYPE_PAIR_RESP = 0x05U , LAVIET_TYPE_CFG = 0x06U
, LAVIET_TYPE_COUNTER_SYNC = 0x07U ,
LAVIET_TYPE_KEY_ROTATE = 0x08U , LAVIET_TYPE_ERROR = 0x09U }
LAVIET frame type stored in the low nibble of ver_type.
- enum laviet_frame_status_t {
LAVIET_STATUS_OK = 0 , LAVIET_STATUS_NULL , LAVIET_STATUS_LENGTH , LAVIET_STATUS_VERSION
,
LAVIET_STATUS_TYPE , LAVIET_STATUS_ID , LAVIET_STATUS_FLAGS , LAVIET_STATUS_PAYLOAD }
Validation, encode and decode result codes.

Funkcje

- `uint8_t laviet_frame_ver_type (laviet_frame_type_t type)`
Compose the version/type byte for a frame type.
- `laviet_frame_type_t laviet_frame_type (const laviet_frame_t *frame)`
Extract the low-nibble frame type from a parsed frame.
- `bool laviet_frame_is_broadcast (const laviet_frame_t *frame)`
Check whether a frame targets the broadcast address.
- `uint16_t laviet_local_node_id (void)`
Derive a stable local node identifier from STM32 unique ID words.
- `laviet_frame_status_t laviet_frame_validate_plain (const laviet_frame_t *frame)`
Validate parsed frame fields before encryption/MAC checks.
- `laviet_frame_status_t laviet_frame_encode (const laviet_frame_t *frame, uint8_t *out, uint8_t out_capacity, uint8_t *out_len)`
Serialize a parsed frame into wire format.
- `laviet_frame_status_t laviet_frame_decode (const uint8_t *in, uint8_t in_len, laviet_frame_t *frame_out)`
Parse a serialized wire frame into a frame structure.
- `bool laviet_frame_build_mac_input (const laviet_frame_t *frame, uint8_t *out, uint8_t out_capacity, uint8_t *out_len)`
Build the canonical byte sequence covered by the frame MAC.
- `bool laviet_frame_write_ack_payload (uint16_t acked_msg_id, uint32_t acked_counter, uint8_t out[LAVIET_ACK_PAYLOAD_LEN])`
Encode an ACK payload.
- `bool laviet_frame_read_ack_payload (const uint8_t payload[LAVIET_ACK_PAYLOAD_LEN], uint16_t *acked_msg_id, uint32_t *acked_counter)`
Decode an ACK payload.

12.26.1 Opis szczegółowy

LAVIET_FRAME_V1 wire protocol helpers.
Definicja w pliku [laviet_frame.h](#).

12.26.2 Dokumentacja definicji

12.26.2.1 LAVIET_ACK_PAYLOAD_LEN

```
#define LAVIET_ACK_PAYLOAD_LEN 6U
```

Fixed payload length of ACK frames.

Definicja w linii 54 pliku [laviet_frame.h](#).

Odwołania w [laviet_frame_read_ack_payload\(\)](#), [laviet_frame_validate_plain\(\)](#) i [laviet_frame_write_ack_payload\(\)](#).

12.26.2.2 LAVIET_BROADCAST_ID

```
#define LAVIET_BROADCAST_ID 0xFFFFU
```

Broadcast destination identifier.

Definicja w linii 32 pliku [laviet_frame.h](#).

Odwołania w [laviet_frame_is_broadcast\(\)](#), [laviet_frame_validate_plain\(\)](#), [laviet_local_node_id\(\)](#) i [security_main_get_frame_keys\(\)](#).

12.26.2.3 LAVIET_COUNTER_SYNC_PAYLOAD_LEN

```
#define LAVIET_COUNTER_SYNC_PAYLOAD_LEN 4U
```

Fixed payload length of counter synchronization frames.

Definicja w linii 56 pliku [laviet_frame.h](#).

Odwołania w [laviet_frame_validate_plain\(\)](#).

12.26.2.4 LAVIET_ERROR_PAYLOAD_LEN

```
#define LAVIET_ERROR_PAYLOAD_LEN 8U
```

Fixed payload length reserved for error frames.
Definicja w linii 58 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_validate_plain\(\)](#).

12.26.2.5 LAVIET_FLAG_ACK_REQUIRED

```
#define LAVIET_FLAG_ACK_REQUIRED (1U << 1)
```

Sender requests an ACK frame.
Definicja w linii 37 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_validate_plain\(\)](#).

12.26.2.6 LAVIET_FLAG_BROADCAST

```
#define LAVIET_FLAG_BROADCAST (1U << 5)
```

Destination is the broadcast address.
Definicja w linii 45 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_validate_plain\(\)](#).

12.26.2.7 LAVIET_FLAG_CONFIG_ACCESS

```
#define LAVIET_FLAG_CONFIG_ACCESS (1U << 4)
```

Frame contains configuration/control payload.
Definicja w linii 43 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_validate_plain\(\)](#).

12.26.2.8 LAVIET_FLAG_COUNTER_OVERRIDE

```
#define LAVIET_FLAG_COUNTER_OVERRIDE (1U << 6)
```

Payload carries a counter synchronization override.
Definicja w linii 47 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_validate_plain\(\)](#).

12.26.2.9 LAVIET_FLAG_ENCRYPTED

```
#define LAVIET_FLAG_ENCRYPTED (1U << 0)
```

Payload is encrypted with the current frame key.
Definicja w linii 35 pliku [laviet_frame.h](#).
Odwołania w [laviet_crypto_self_test_locked\(\)](#) i [laviet_frame_validate_plain\(\)](#).

12.26.2.10 LAVIET_FLAG_IS_ACK

```
#define LAVIET_FLAG_IS_ACK (1U << 2)
```

Frame itself is an ACK.
Definicja w linii 39 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_validate_plain\(\)](#).

12.26.2.11 LAVIET_FLAG_KEY_UPDATE

```
#define LAVIET_FLAG_KEY_UPDATE (1U << 7)
```

Payload carries key rotation material.
Definicja w linii 49 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_validate_plain\(\)](#).

12.26.2.12 LAVIET_FLAG_PAIRING

```
#define LAVIET_FLAG_PAIRING (1U << 3)
```

Frame belongs to pairing flow.

Definicja w linii 41 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_validate_plain\(\)](#).

12.26.2.13 LAVIET_FRAME_HEADER_LEN

```
#define LAVIET_FRAME_HEADER_LEN 13U
```

Serialized header length before payload and MAC.
Definicja w linii 23 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_build_mac_input\(\)](#), [laviet_frame_decode\(\)](#), [laviet_frame_encode\(\)](#) i [laviet_frame_hmac_sha256\(\)](#).

12.26.2.14 LAVIET_FRAME_MAX_LEN

```
#define LAVIET_FRAME_MAX_LEN (LAVIET_FRAME_HEADER_LEN + LAVIET_MAX_PAYLOAD + LAVIET_MAC_TAG_LEN)
```

Maximum serialized frame length.
Definicja w linii 27 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_decode\(\)](#).

12.26.2.15 LAVIET_FRAME_MIN_LEN

```
#define LAVIET_FRAME_MIN_LEN (LAVIET_FRAME_HEADER_LEN + LAVIET_MAC_TAG_LEN)
```

Minimum serialized frame length.
Definicja w linii 25 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_decode\(\)](#).

12.26.2.16 LAVIET_FRAME_VERSION

```
#define LAVIET_FRAME_VERSION 1U
```

Wire format version encoded in the high nibble of `ver_type`.
Definicja w linii 17 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_validate_plain\(\)](#) i [laviet_frame_ver_type\(\)](#).

12.26.2.17 LAVIET_GATEWAY_ID

```
#define LAVIET_GATEWAY_ID 0x0001U
```

Reserved node identifier of the gateway.
Definicja w linii 30 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_validate_plain\(\)](#) i [laviet_local_node_id\(\)](#).

12.26.2.18 LAVIET_MAC_TAG_LEN

```
#define LAVIET_MAC_TAG_LEN 32U
```

Full HMAC-SHA256 tag length stored on the wire.
Definicja w linii 21 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_decode\(\)](#), [laviet_frame_encode\(\)](#), [laviet_frame_hmac_sha256\(\)](#) i [laviet_mac_equal\(\)](#).

12.26.2.19 LAVIET_MAX_PAYLOAD

```
#define LAVIET_MAX_PAYLOAD 16U
```

Maximum payload bytes carried by one LAVIET frame.
Definicja w linii 19 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_build_mac_input\(\)](#), [laviet_frame_decode\(\)](#), [laviet_frame_hmac_sha256\(\)](#), [laviet_frame_validate_plain\(\)](#) i [radio_main_cmd_send_user_text\(\)](#).

12.26.2.20 LAVIET_PAIR_PAYLOAD_LEN

```
#define LAVIET_PAIR_PAYLOAD_LEN 8U
```

Fixed payload length of pair request/response frames.
Definicja w linii 52 pliku [laviet_frame.h](#).
Odwołania w [laviet_frame_validate_plain\(\)](#).

12.26.3 Dokumentacja typów wyliczanych

12.26.3.1 `laviet_frame_status_t`

enum `laviet_frame_status_t`

Validation, encode and decode result codes.

Wartości wyliczeń

LAVIET_STATUS_OK	Operation completed successfully.
LAVIET_STATUS_NULL	Null pointer argument.
LAVIET_STATUS_LENGTH	Invalid serialized length or buffer capacity.
LAVIET_STATUS_VERSION	Unsupported frame version.
LAVIET_STATUS_TYPE	Unknown or invalid frame type.
LAVIET_STATUS_ID	Invalid source, destination or message identifier.
LAVIET_STATUS_FLAGS	Flag combination is invalid for the frame type.
LAVIET_STATUS_PAYLOAD	Payload length or shape is invalid.

Definicja w linii 80 pliku [laviet_frame.h](#).

12.26.3.2 `laviet_frame_type_t`

enum `laviet_frame_type_t`

LAVIET frame type stored in the low nibble of `ver_type`.

Wartości wyliczeń

LAVIET_TYPE_NONE	Invalid/empty type sentinel.
LAVIET_TYPE_DATA	User/application payload.
LAVIET_TYPE_ACK	Acknowledgement payload.
LAVIET_TYPE_RESP	Response/status payload.
LAVIET_TYPE_PAIR_REQ	Pairing request.
LAVIET_TYPE_PAIR_RESP	Pairing response.
LAVIET_TYPE_CFG	Configuration/control payload.
LAVIET_TYPE_COUNTER_SYNC	Gateway counter synchronization.
LAVIET_TYPE_KEY_ROTATE	Key rotation payload.
LAVIET_TYPE_ERROR	Error/status payload.

Definicja w linii 63 pliku [laviet_frame.h](#).

12.26.4 Dokumentacja funkcji

12.26.4.1 `laviet_frame_build_mac_input()`

```
bool laviet_frame_build_mac_input (
    const laviet_frame_t * frame,
    uint8_t * out,
    uint8_t out_capacity,
    uint8_t * out_len)
```

Build the canonical byte sequence covered by the frame MAC.

Parametry

<i>frame</i>	Frame whose header and payload are used.
<i>out</i>	Output buffer.
<i>out_capacity</i>	Output buffer capacity.
<i>out_len</i>	[out] MAC input length.

Zwraca

`true` when the MAC input was built.

Definicja w linii 378 pliku [laviet_frame.c](#).

Odwołuje się do [laviet_frame_t::counter](#), [laviet_frame_t::dst_id](#), [laviet_frame_t::flags](#), [laviet_be16_write\(\)](#), [laviet_be32_write\(\)](#), [LAVIET_FRAME_HEADER_LEN](#), [LAVIET_MAX_PAYLOAD](#), [laviet_frame_t::msg_id](#), [laviet_frame_t::payload](#), [laviet_frame_t::payload_len](#), [laviet_frame_t::src_id](#) i [laviet_frame_t::ver_type](#).

Odwołania w [laviet_frame_hmac_sha256\(\)](#).

12.26.4.2 laviet_frame_decode()

```
laviet_frame_status_t laviet_frame_decode (
    const uint8_t * in,
    uint8_t in_len,
    laviet_frame_t * frame_out)
```

Parse a serialized wire frame into a frame structure.

Parametry

<i>in</i>	Serialized frame bytes.
<i>in_len</i>	Serialized frame length.
<i>frame_out</i>	[out] Parsed frame.

Zwraca

Decode status.

Definicja w linii 328 pliku [laviet_frame.c](#).

Odwołuje się do [laviet_frame_t::counter](#), [laviet_frame_t::dst_id](#), [laviet_frame_t::flags](#), [laviet_be16_read\(\)](#), [laviet_be32_read\(\)](#), [LAVIET_FRAME_HEADER_LEN](#), [LAVIET_FRAME_MAX_LEN](#), [LAVIET_FRAME_MIN_LEN](#), [laviet_frame_validate_plain\(\)](#), [LAVIET_MAC_TAG_LEN](#), [LAVIET_MAX_PAYLOAD](#), [LAVIET_STATUS_LENGTH](#), [LAVIET_STATUS_NULL](#), [LAVIET_STATUS_OK](#), [LAVIET_STATUS_PAYLOAD](#), [laviet_frame_t::mac_tag](#), [laviet_frame_t::msg_id](#), [laviet_frame_t::payload](#), [laviet_frame_t::payload_len](#), [laviet_frame_t::src_id](#) i [laviet_frame_t::ver_type](#).

12.26.4.3 laviet_frame_encode()

```
laviet_frame_status_t laviet_frame_encode (
    const laviet_frame_t * frame,
    uint8_t * out,
    uint8_t out_capacity,
    uint8_t * out_len)
```

Serialize a parsed frame into wire format.

Parametry

<i>frame</i>	Frame to serialize.
--------------	---------------------

<i>out</i>	Output buffer.
<i>out_capacity</i>	Output buffer capacity.
<i>out_len</i>	[out] Serialized frame length.

Zwraca

Encode status.

Definicja w linii 287 pliku [laviet_frame.c](#).

Odwołuje się do [laviet_frame_t::counter](#), [laviet_frame_t::dst_id](#), [laviet_frame_t::flags](#), [laviet_be16_write\(\)](#), [laviet_be32_write\(\)](#), [LAVIET_FRAME_HEADER_LEN](#), [laviet_frame_validate_plain\(\)](#), [LAVIET_MAC_TAG_LEN](#), [LAVIET_STATUS_LENGTH](#), [LAVIET_STATUS_NULL](#), [LAVIET_STATUS_OK](#), [laviet_frame_t::mac_tag](#), [laviet_frame_t::msg_id](#), [laviet_frame_t::payload](#), [laviet_frame_t::payload_len](#), [laviet_frame_t::src_id](#) i [laviet_frame_t::ver_type](#).

12.26.4.4 laviet_frame_is_broadcast()

```
bool laviet_frame_is_broadcast (
    const laviet_frame_t * frame)
```

Check whether a frame targets the broadcast address.

Parametry

<i>frame</i>	Frame object.
--------------	---------------

Zwraca

true when [dst_id](#) equals [LAVIET_BROADCAST_ID](#).

Definicja w linii 70 pliku [laviet_frame.c](#).

Odwołuje się do [laviet_frame_t::dst_id](#) i [LAVIET_BROADCAST_ID](#).

12.26.4.5 laviet_frame_read_ack_payload()

```
bool laviet_frame_read_ack_payload (
    const uint8_t payload[LAVIET_ACK_PAYLOAD_LEN],
    uint16_t * acked_msg_id,
    uint32_t * acked_counter)
```

Decode an ACK payload.

Parametry

<i>payload</i>	ACK payload bytes.
<i>acked_msg_id</i>	[out] Decoded message identifier.
<i>acked_counter</i>	[out] Decoded counter value.

Zwraca

true when the payload was decoded.

Definicja w linii 429 pliku [laviet_frame.c](#).

Odwołuje się do [LAVIET_ACK_PAYLOAD_LEN](#), [laviet_be16_read\(\)](#) i [laviet_be32_read\(\)](#).

12.26.4.6 laviet_frame_type()

```
laviet_frame_type_t laviet_frame_type (
    const laviet_frame_t * frame)
```

Extract the low-nibble frame type from a parsed frame.

Parametry

<i>frame</i>	Frame object.
--------------	---------------

Zwraca

Frame type or `LAVIET_TYPE_NONE` for a null frame.

Definicja w linii 60 pliku `laviet_frame.c`.

Odwołuje się do `LAVIET_TYPE_NONE` i `laviet_frame_t::ver_type`.

Odwołania w `laviet_frame_validate_plain()`.

12.26.4.7 laviet_frame_validate_plain()

```
laviet_frame_status_t laviet_frame_validate_plain (
    const laviet_frame_t * frame)
```

Validate parsed frame fields before encryption/MAC checks.

Parametry

<i>frame</i>	Frame object.
--------------	---------------

Zwraca

Validation status.

Definicja w linii 120 pliku `laviet_frame.c`.

Odwołuje się do `laviet_frame_t::dst_id`, `laviet_frame_t::flags`, `LAVIET_ACK_PAYLOAD_LEN`, `LAVIET_BROADCAST_ID`, `LAVIET_COUNTER_SYNC_PAYLOAD_LEN`, `LAVIET_ERROR_PAYLOAD_LEN`, `LAVIET_FLAG_ACK_REQUIRED`, `LAVIET_FLAG_BROADCAST`, `LAVIET_FLAG_CONFIG_ACCESS`, `LAVIET_FLAG_COUNTER_OVERRIDE`, `LAVIET_FLAG_ENCRYPTED`, `LAVIET_FLAG_IS_ACK`, `LAVIET_FLAG_KEY_UPDATE`, `LAVIET_FLAG_PAIRING`, `laviet_frame_type()`, `laviet_frame_type_known()`, `LAVIET_FRAME_VERSION`, `laviet_frame_version_from_ver_type()`, `LAVIET_GATEWAY_ID`, `LAVIET_MAX_PAYLOAD`, `LAVIET_PAIR_PAYLOAD_LEN`, `LAVIET_STATUS_FLAGS`, `LAVIET_STATUS_ID`, `LAVIET_STATUS_NULL`, `LAVIET_STATUS_OK`, `LAVIET_STATUS_PAYLOAD`, `LAVIET_STATUS_TYPE`, `LAVIET_STATUS_VERSION`, `LAVIET_TYPE_ACK`, `LAVIET_TYPE_CFG`, `LAVIET_TYPE_COUNTER_SYNC`, `LAVIET_TYPE_DATA`, `LAVIET_TYPE_ERROR`, `LAVIET_TYPE_KEY_ROTATE`, `LAVIET_TYPE_PAIR_REQ`, `LAVIET_TYPE_PAIR_RESP`, `LAVIET_TYPE_RESP`, `laviet_frame_t::msg_id`, `laviet_frame_t::payload_len`, `laviet_frame_t::src_id` i `laviet_frame_t::ver_type`.

Odwołania w `laviet_frame_decode()` i `laviet_frame_encode()`.

12.26.4.8 laviet_frame_ver_type()

```
uint8_t laviet_frame_ver_type (
    laviet_frame_type_t type)
```

Compose the version/type byte for a frame type.

Parametry

<i>type</i>	LAVIET frame type.
-------------	--------------------

Zwraca

Encoded `ver_type` byte.

Definicja w linii 55 pliku [laviet_frame.c](#).

Odwołuje się do [LAVIET_FRAME_VERSION](#).

12.26.4.9 laviet_frame_write_ack_payload()

```
bool laviet_frame_write_ack_payload (
    uint16_t  acked_msg_id,
    uint32_t  acked_counter,
    uint8_t  out[LAVIET_ACK_PAYLOAD_LEN])
```

Encode an ACK payload.

Parametry

<i>acked_msg_id</i>	Message identifier being acknowledged.
<i>acked_counter</i>	Counter value being acknowledged.
<i>out</i>	[out] Fixed-size ACK payload buffer.

Zwraca

`true` when the payload was written.

Definicja w linii 415 pliku [laviet_frame.c](#).

Odwołuje się do [LAVIET_ACK_PAYLOAD_LEN](#), [laviet_be16_write\(\)](#) i [laviet_be32_write\(\)](#).

12.26.4.10 laviet_local_node_id()

```
uint16_t laviet_local_node_id (
    void )
```

Derive a stable local node identifier from STM32 unique ID words.

Zwraca

Non-reserved 16-bit node identifier.

Definicja w linii 80 pliku [laviet_frame.c](#).

Odwołuje się do [LAVIET_BROADCAST_ID](#) i [LAVIET_GATEWAY_ID](#).

12.27 laviet_frame.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_LAVIET_FRAME_H_
00007 #define APP_LAVIET_FRAME_H_
00008
00009 #include <stdbool.h>
00010 #include <stdint.h>
00011
00012 #ifdef __cplusplus
00013 extern "C" {
00014 #endif
00015
00017 #define LAVIET_FRAME_VERSION          1U
00019 #define LAVIET_MAX_PAYLOAD            16U
00021 #define LAVIET_MAC_TAG_LEN            32U
00023 #define LAVIET_FRAME_HEADER_LEN      13U
00025 #define LAVIET_FRAME_MIN_LEN          (LAVIET_FRAME_HEADER_LEN + LAVIET_MAC_TAG_LEN)
00027 #define LAVIET_FRAME_MAX_LEN          (LAVIET_FRAME_HEADER_LEN + LAVIET_MAX_PAYLOAD +
    LAVIET_MAC_TAG_LEN)
00028
```



```

00030 #define LAVIET_GATEWAY_ID          0x0001U
00032 #define LAVIET_BROADCAST_ID        0xFFFFU
00033
00035 #define LAVIET_FLAG_ENCRYPTED        (1U < 0)
00037 #define LAVIET_FLAG_ACK_REQUIRED    (1U < 1)
00039 #define LAVIET_FLAG_IS_ACK          (1U < 2)
00041 #define LAVIET_FLAG_PAIRING         (1U < 3)
00043 #define LAVIET_FLAG_CONFIG_ACCESS   (1U < 4)
00045 #define LAVIET_FLAG_BROADCAST       (1U < 5)
00047 #define LAVIET_FLAG_COUNTER_OVERRIDE (1U < 6)
00049 #define LAVIET_FLAG_KEY_UPDATE      (1U < 7)
00050
00052 #define LAVIET_PAIR_PAYLOAD_LEN      8U
00054 #define LAVIET_ACK_PAYLOAD_LEN       6U
00056 #define LAVIET_COUNTER_SYNC_PAYLOAD_LEN 4U
00058 #define LAVIET_ERROR_PAYLOAD_LEN     8U
00059
00063 typedef enum
00064 {
00065     LAVIET_TYPE_NONE = 0x00U,
00066     LAVIET_TYPE_DATA = 0x01U,
00067     LAVIET_TYPE_ACK = 0x02U,
00068     LAVIET_TYPE_RESP = 0x03U,
00069     LAVIET_TYPE_PAIR_REQ = 0x04U,
00070     LAVIET_TYPE_PAIR_RESP = 0x05U,
00071     LAVIET_TYPE_CFG = 0x06U,
00072     LAVIET_TYPE_COUNTER_SYNC = 0x07U,
00073     LAVIET_TYPE_KEY_ROTATE = 0x08U,
00074     LAVIET_TYPE_ERROR = 0x09U
00075 } laviet_frame_type_t;
00076
00080 typedef enum
00081 {
00082     LAVIET_STATUS_OK = 0,
00083     LAVIET_STATUS_NULL,
00084     LAVIET_STATUS_LENGTH,
00085     LAVIET_STATUS_VERSION,
00086     LAVIET_STATUS_TYPE,
00087     LAVIET_STATUS_ID,
00088     LAVIET_STATUS_FLAGS,
00089     LAVIET_STATUS_PAYLOAD
00090 } laviet_frame_status_t;
00091
00095 typedef struct
00096 {
00097     uint8_t ver_type;
00098     uint8_t flags;
00099     uint16_t src_id;
00100     uint16_t dst_id;
00101     uint16_t msg_id;
00102     uint32_t counter;
00103     uint8_t payload_len;
00104     uint8_t payload[LAVIET_MAX_PAYLOAD];
00105     uint8_t mac_tag[LAVIET_MAC_TAG_LEN];
00106 } laviet_frame_t;
00107
00113 uint8_t laviet_frame_ver_type(laviet_frame_type_t type);
00114
00120 laviet_frame_type_t laviet_frame_type(const laviet_frame_t *frame);
00121
00127 bool laviet_frame_is_broadcast(const laviet_frame_t *frame);
00128
00133 uint16_t laviet_local_node_id(void);
00134
00140 laviet_frame_status_t laviet_frame_validate_plain(const laviet_frame_t *frame);
00141
00150 laviet_frame_status_t laviet_frame_encode(const laviet_frame_t *frame,
00151     uint8_t *out,
00152     uint8_t out_capacity,
00153     uint8_t *out_len);
00154
00162 laviet_frame_status_t laviet_frame_decode(const uint8_t *in,
00163     uint8_t in_len,
00164     laviet_frame_t *frame_out);
00165
00174 bool laviet_frame_build_mac_input(const laviet_frame_t *frame,
00175     uint8_t *out,
00176     uint8_t out_capacity,
00177     uint8_t *out_len);
00178
00186 bool laviet_frame_write_ack_payload(uint16_t acked_msg_id,
00187     uint32_t acked_counter,
00188     uint8_t out[LAVIET_ACK_PAYLOAD_LEN]);
00189
00197 bool laviet_frame_read_ack_payload(const uint8_t payload[LAVIET_ACK_PAYLOAD_LEN],
00198     uint16_t *acked_msg_id,
00199     uint32_t *acked_counter);

```

```

00200
00201 #ifdef __cplusplus
00202 }
00203 #endif
00204
00205 #endif /* APP_LAVIET_FRAME_H_ */

```

12.28 lcd.c

```

00001 #include "lcd.h"
00002
00003 #include "../app.h"
00004 #include "app_delay.h"
00005
00006 #include <stdbool.h>
00007 #include <stddef.h>
00008 #include <stdio.h>
00009 #include <string.h>
00010
00011 #define LCD_I2C_ADDR_DEFAULT_7BIT 0x27U
00012 #define LCD_I2C_ADDR_ALT_7BIT 0x3FU
00013
00014 #define LCD_RS_MASK 0x01U
00015 #define LCD_EN_MASK 0x04U
00016 #define LCD_BL_MASK 0x08U
00017
00018 #define LCD_ROWS 4U
00019 #define LCD_COLS 20U
00020
00021 #define LCD_I2C_TIMEOUT_MS 20U
00022 #define LCD_INIT_RETRY_COUNT 3U
00023
00024 #define LCD_CMD_CLEAR 0x01U
00025 #define LCD_CMD_HOME 0x02U
00026 #define LCD_CMD_ENTRY_MODE 0x06U
00027 #define LCD_CMD_DISPLAY_ON 0x0CU
00028 #define LCD_CMD_DISPLAY_OFF 0x08U
00029 #define LCD_CMD_FUNCTION_SET 0x28U
00030 #define LCD_CMD_SET_DDRAM 0x80U
00031
00032 typedef struct
00033 {
00034     bool initialized;
00035     uint8_t backlight;
00036     uint8_t addr_7bit;
00037 } lcd_state_t;
00038
00039 extern I2C_HandleTypeDef hi2c1;
00040
00041 static lcd_state_t s_lcd_state =
00042 {
00043     .initialized = false,
00044     .backlight = LCD_BL_MASK,
00045     .addr_7bit = LCD_I2C_ADDR_DEFAULT_7BIT
00046 };
00047
00048 static bool s_lcd_ready_logged = false;
00049 static bool s_lcd_failure_logged = false;
00050
00051 static bool lcd_i2c_tx_locked(const uint8_t *data, uint16_t length);
00052 static bool lcd_write_nibble_locked(uint8_t nibble, bool rs);
00053 static bool lcd_write_byte_locked(uint8_t value, bool rs);
00054 static bool lcd_send_command_locked(uint8_t command);
00055 static bool lcd_send_data_locked(uint8_t data);
00056 static uint8_t lcd_ddram_base(uint8_t row);
00057 static bool lcd_init_locked(void);
00058 static bool lcd_ensure_ready(void);
00059 static void lcd_animation_clear_frame(char frame[LCD_ROWS][LCD_COLS + 1U]);
00060 static void lcd_animation_put_text(char frame[LCD_ROWS][LCD_COLS + 1U],
00061     uint8_t row,
00062     uint8_t col,
00063     const char *text);
00064 static void lcd_animation_put_char(char frame[LCD_ROWS][LCD_COLS + 1U],
00065     int8_t row,
00066     int8_t col,
00067     char ch);
00068 static void lcd_animation_render_frame(char frame[LCD_ROWS][LCD_COLS + 1U], uint32_t hold_ms);
00069
00071 static bool lcd_i2c_tx_locked(const uint8_t *data, uint16_t length)
00072 {
00073     if ((data == NULL) || (length == 0U))
00074     {
00075         return false;
00076     }

```

```

00077
00078     return (app_i2c_master_transmit(&hi2c1,
00079                                     (uint16_t)(s_lcd_state.addr_7bit << 1),
00080                                     data,
00081                                     length,
00082                                     LCD_I2C_TIMEOUT_MS) == HAL_OK);
00083 }
00084
00091 static bool lcd_write_nibble_locked(uint8_t nibble, bool rs)
00092 {
00093     uint8_t bus;
00094     uint8_t frame[3];
00095
00096     bus = (uint8_t)((nibble & 0x0FU) << 4);
00097     bus |= s_lcd_state.backlight;
00098     if (rs)
00099     {
00100         bus |= LCD_RS_MASK;
00101     }
00102
00103     frame[0] = bus;
00104     frame[1] = (uint8_t)(bus | LCD_EN_MASK);
00105     frame[2] = bus;
00106
00107     return lcd_i2c_tx_locked(frame, (uint16_t)sizeof(frame));
00108 }
00109
00111 static bool lcd_write_byte_locked(uint8_t value, bool rs)
00112 {
00113     if (!lcd_write_nibble_locked((uint8_t)(value >> 4), rs))
00114     {
00115         return false;
00116     }
00117
00118     if (!lcd_write_nibble_locked((uint8_t)(value & 0x0FU), rs))
00119     {
00120         return false;
00121     }
00122
00123     return true;
00124 }
00125
00127 static bool lcd_send_command_locked(uint8_t command)
00128 {
00129     if (!lcd_write_byte_locked(command, false))
00130     {
00131         return false;
00132     }
00133
00134     if ((command == LCD_CMD_CLEAR) || (command == LCD_CMD_HOME))
00135     {
00136         app_delay_ms(3U);
00137     }
00138     else
00139     {
00140         app_delay_ms(1U);
00141     }
00142
00143     return true;
00144 }
00145
00147 static bool lcd_send_data_locked(uint8_t data)
00148 {
00149     if (!lcd_write_byte_locked(data, true))
00150     {
00151         return false;
00152     }
00153
00154     app_delay_ms(1U);
00155     return true;
00156 }
00157
00159 static uint8_t lcd_ddram_base(uint8_t row)
00160 {
00161     switch (row)
00162     {
00163     case 0U:
00164         return 0x00U;
00165     case 1U:
00166         return 0x40U;
00167     case 2U:
00168         return 0x14U;
00169     case 3U:
00170         return 0x54U;
00171     default:
00172         return 0x00U;
00173     }
}

```

```
00174 }
00175
00182 static bool lcd_init_locked(void)
00183 {
00184     uint8_t candidates[3];
00185     uint32_t candidate_idx;
00186     uint32_t attempt;
00187
00188     candidates[0] = s_lcd_state.addr_7bit;
00189     candidates[1] = LCD_I2C_ADDR_DEFAULT_7BIT;
00190     candidates[2] = LCD_I2C_ADDR_ALT_7BIT;
00191
00192     for (candidate_idx = 0U; candidate_idx < (sizeof(candidates) / sizeof(candidates[0]));
00193         candidate_idx++)
00194     {
00195         uint32_t previous_idx;
00196         bool duplicate = false;
00197
00198         for (previous_idx = 0U; previous_idx < candidate_idx; previous_idx++)
00199         {
00200             if (candidates[previous_idx] == candidates[candidate_idx])
00201             {
00202                 duplicate = true;
00203                 break;
00204             }
00205             if (duplicate)
00206             {
00207                 continue;
00208             }
00209
00210             s_lcd_state.addr_7bit = candidates[candidate_idx];
00211
00212             for (attempt = 0U; attempt < LCD_INIT_RETRY_COUNT; attempt++)
00213             {
00214                 app_delay_ms(60U);
00215
00216                 if (!lcd_i2c_tx_locked(&s_lcd_state.backlight, 1U))
00217                 {
00218                     continue;
00219                 }
00220
00221                 app_delay_ms(5U);
00222
00223                 if (!lcd_write_nibble_locked(0x03U, false))
00224                 {
00225                     continue;
00226                 }
00227                 app_delay_ms(5U);
00228
00229                 if (!lcd_write_nibble_locked(0x03U, false))
00230                 {
00231                     continue;
00232                 }
00233                 app_delay_ms(5U);
00234
00235                 if (!lcd_write_nibble_locked(0x03U, false))
00236                 {
00237                     continue;
00238                 }
00239                 app_delay_ms(2U);
00240
00241                 if (!lcd_write_nibble_locked(0x02U, false))
00242                 {
00243                     continue;
00244                 }
00245                 app_delay_ms(2U);
00246
00247                 if (!lcd_send_command_locked(LCD_CMD_FUNCTION_SET))
00248                 {
00249                     continue;
00250                 }
00251
00252                 if (!lcd_send_command_locked(LCD_CMD_FUNCTION_SET))
00253                 {
00254                     continue;
00255                 }
00256
00257                 if (!lcd_send_command_locked(LCD_CMD_DISPLAY_OFF))
00258                 {
00259                     continue;
00260                 }
00261
00262                 if (!lcd_send_command_locked(LCD_CMD_CLEAR))
00263                 {
00264                     continue;
00265                 }
00266             }
00267         }
00268     }
```

```

00266
00267         if (!lcd_send_command_locked(LCD_CMD_ENTRY_MODE))
00268         {
00269             continue;
00270         }
00271
00272         if (!lcd_send_command_locked(LCD_CMD_DISPLAY_ON))
00273         {
00274             continue;
00275         }
00276
00277         if (!lcd_send_command_locked(LCD_CMD_HOME))
00278         {
00279             continue;
00280         }
00281
00282         s_lcd_state.initialized = true;
00283         return true;
00284     }
00285 }
00286
00287 s_lcd_state.initialized = false;
00288 return false;
00289 }
00290
00291 static bool lcd_ensure_ready(void)
00292 {
00293     bool ok;
00294
00295     if (s_lcd_state.initialized)
00296     {
00297         return true;
00298     }
00299
00300     ok = false;
00301
00302     if (app_i2c_lock(0U))
00303     {
00304         ok = lcd_init_locked();
00305         app_i2c_unlock();
00306     }
00307
00308     return ok;
00309 }
00310
00311 static void lcd_animation_clear_frame(char frame[LCD_ROWS][LCD_COLS + 1U])
00312 {
00313     uint8_t row;
00314     uint8_t col;
00315
00316     for (row = 0U; row < LCD_ROWS; row++)
00317     {
00318         for (col = 0U; col < LCD_COLS; col++)
00319         {
00320             frame[row][col] = ' ';
00321         }
00322         frame[row][LCD_COLS] = '\0';
00323     }
00324 }
00325
00326 static void lcd_animation_put_text(char frame[LCD_ROWS][LCD_COLS + 1U],
00327                                   uint8_t row,
00328                                   uint8_t col,
00329                                   const char *text)
00330 {
00331     uint8_t idx;
00332
00333     if ((text == NULL) || (row >= LCD_ROWS) || (col >= LCD_COLS))
00334     {
00335         return;
00336     }
00337
00338     idx = 0U;
00339     while ((text[idx] != '\0') && ((uint8_t)(col + idx) < LCD_COLS))
00340     {
00341         frame[row][col + idx] = text[idx];
00342         idx++;
00343     }
00344 }
00345
00346 static void lcd_animation_put_char(char frame[LCD_ROWS][LCD_COLS + 1U],
00347                                   int8_t row,
00348                                   int8_t col,
00349                                   char ch)
00350 {
00351     if ((row < 0) || (col < 0))
00352     {
00353

```

```

00362         return;
00363     }
00364
00365     if (((uint8_t)row >= LCD_ROWS) || ((uint8_t)col >= LCD_COLS))
00366     {
00367         return;
00368     }
00369     frame[(uint8_t)row][(uint8_t)col] = ch;
00371 }
00372
00374 static void lcd_animation_render_frame(char frame[LCD_ROWS][LCD_COLS + 1U], uint32_t hold_ms)
00375 {
00376     uint8_t row;
00377
00378     lcd_clear();
00379     for (row = 0U; row < LCD_ROWS; row++)
00380     {
00381         (void)lcd_write_line(row, frame[row], LCD_COLS);
00382     }
00383     app_delay_ms(hold_ms);
00384 }
00385
00386 void lcd_demo(void)
00387 {
00388     char number[12];
00389     int count;
00390
00391     lcd_init();
00392     lcd_backlight(1U);
00393
00394     count = 0;
00395     while (1)
00396     {
00397         (void)sprintf(number, sizeof(number), "%d", count);
00398         lcd_clear();
00399         (void)lcd_write_line(0U, "Hello BEKO studs", LCD_COLS);
00400         (void)lcd_write_line(1U, number, LCD_COLS);
00401         count++;
00402         app_delay_ms(100U);
00403     }
00404 }
00405
00406 void lcd_init(void)
00407 {
00408     bool ok;
00409
00410     s_lcd_state.initialized = false;
00411     ok = lcd_ensure_ready();
00412     if (ok)
00413     {
00414         if (!s_lcd_ready_logged)
00415         {
00416             printf("LCD: init OK addr=0x%02X\r\n", (unsigned int)s_lcd_state.addr_7bit);
00417             s_lcd_ready_logged = true;
00418         }
00419         s_lcd_failure_logged = false;
00420     }
00421     else if (!s_lcd_failure_logged)
00422     {
00423         printf("LCD: init failed addr=0x%02X err=0x%08lX\r\n",
00424             (unsigned int)s_lcd_state.addr_7bit,
00425             (unsigned long)HAL_I2C_GetError(&hi2c1));
00426         s_lcd_failure_logged = true;
00427     }
00428 }
00429
00430 void lcd_write_string(uint8_t *str)
00431 {
00432     if ((str == NULL) || (!lcd_ensure_ready()))
00433     {
00434         return;
00435     }
00436
00437     if (!app_i2c_lock(0U))
00438     {
00439         return;
00440     }
00441
00442     while (*str != '\0')
00443     {
00444         if (!lcd_send_data_locked(*str))
00445         {
00446             s_lcd_state.initialized = false;
00447             break;
00448         }
00449     }

```

```
00450         str++;
00451     }
00452
00453     app_i2c_unlock();
00454 }
00455
00456 void lcd_set_cursor(uint8_t row, uint8_t column)
00457 {
00458     uint8_t address;
00459
00460     if ((row >= LCD_ROWS) || (column >= LCD_COLS) || (!lcd_ensure_ready()))
00461     {
00462         return;
00463     }
00464
00465     address = (uint8_t)(lcd_ddram_base(row) + column);
00466
00467     if (!app_i2c_lock(0U))
00468     {
00469         return;
00470     }
00471
00472     if (!lcd_send_command_locked((uint8_t)(LCD_CMD_SET_DDRAM | address)))
00473     {
00474         s_lcd_state.initialized = false;
00475     }
00476
00477     app_i2c_unlock();
00478 }
00479
00480 void lcd_clear(void)
00481 {
00482     if (!lcd_ensure_ready())
00483     {
00484         return;
00485     }
00486
00487     if (!app_i2c_lock(0U))
00488     {
00489         return;
00490     }
00491
00492     if (!lcd_send_command_locked(LCD_CMD_CLEAR) ||
00493         !lcd_send_command_locked(LCD_CMD_HOME))
00494     {
00495         s_lcd_state.initialized = false;
00496     }
00497
00498     app_i2c_unlock();
00499 }
00500
00501 void lcd_backlight(uint8_t state)
00502 {
00503     s_lcd_state.backlight = (state != 0U) ? LCD_BL_MASK : 0U;
00504
00505     if (!app_i2c_lock(0U))
00506     {
00507         return;
00508     }
00509
00510     if (!lcd_i2c_tx_locked(&s_lcd_state.backlight, 1U))
00511     {
00512         s_lcd_state.initialized = false;
00513     }
00514
00515     app_i2c_unlock();
00516 }
00517
00524 bool lcd_write_line(uint8_t row, const char *text, uint8_t width)
00525 {
00526     uint8_t i;
00527
00528     if ((row >= LCD_ROWS) || (!lcd_ensure_ready()))
00529     {
00530         return false;
00531     }
00532
00533     if (width > LCD_COLS)
00534     {
00535         width = LCD_COLS;
00536     }
00537
00538     if (!app_i2c_lock(0U))
00539     {
00540         return false;
00541     }
00542 }
```

```

00543     if (!lcd_send_command_locked((uint8_t) (LCD_CMD_SET_DDRAM | lcd_ddram_base(row))))
00544     {
00545         s_lcd_state.initialized = false;
00546         app_i2c_unlock();
00547         return false;
00548     }
00549
00550     for (i = 0U; i < width; i++)
00551     {
00552         uint8_t ch;
00553
00554         ch = ' ';
00555         if ((text != NULL) && (text[i] != '\0'))
00556         {
00557             ch = (uint8_t)text[i];
00558         }
00559
00560         if (!lcd_send_data_locked(ch))
00561         {
00562             s_lcd_state.initialized = false;
00563             app_i2c_unlock();
00564             return false;
00565         }
00566     }
00567
00568     app_i2c_unlock();
00569     return true;
00570 }
00571
00572 void lcd_animation_hello_beko(void)
00573 {
00574     char frame[LCD_ROWS][LCD_COLS + 1U];
00575     static const char title[] = "HELLO BEKO";
00576     static const char subtitle[] = "RX monitor...";
00577     static const int8_t blast_offsets[][2] =
00578     {
00579         { 0, 0 }, { -1, 0 }, { 1, 0 }, { 0, -3 }, { 0, 3 },
00580         { -1, -5 }, { -1, 5 }, { 1, -5 }, { 1, 5 }, { -2, 0 },
00581         { 2, 0 }, { -2, -7 }, { -2, 7 }, { 2, -7 }, { 2, 7 }
00582     };
00583     uint8_t frame_idx;
00584     int8_t center_row = 1;
00585     int8_t center_col = 10;
00586
00587     /* 8 x 250 ms gives a simple 2 second boot animation without blocking too long. */
00588     for (frame_idx = 0U; frame_idx < 8U; frame_idx++)
00589     {
00590         uint8_t i;
00591         uint8_t active_count = (uint8_t)((frame_idx + 1U) * 2U);
00592         char spark = '.';
00593
00594         if (active_count > (sizeof(blast_offsets) / sizeof(blast_offsets[0])))
00595         {
00596             active_count = (uint8_t)(sizeof(blast_offsets) / sizeof(blast_offsets[0]));
00597         }
00598
00599         if (frame_idx >= 2U)
00600         {
00601             spark = '*';
00602         }
00603         if (frame_idx >= 5U)
00604         {
00605             spark = '+';
00606         }
00607
00608         lcd_animation_clear_frame(frame);
00609
00610         for (i = 0U; i < active_count; i++)
00611         {
00612             lcd_animation_put_char(frame,
00613                                     (int8_t)(center_row + blast_offsets[i][0]),
00614                                     (int8_t)(center_col + blast_offsets[i][1]),
00615                                     spark);
00616         }
00617
00618         if (frame_idx >= 1U)
00619         {
00620             lcd_animation_put_text(frame, 1U, 5U, title);
00621         }
00622
00623         if (frame_idx >= 6U)
00624         {
00625             lcd_animation_put_text(frame, 2U, 3U, subtitle);
00626         }
00627
00628         lcd_animation_render_frame(frame, 250U);
00629     }

```



```
00630 }
```

12.29 Dokumentacja pliku Core/App/lcd_library/lcd.h

HD44780-compatible 20x4 LCD driver over I2C expander.

```
#include <stdbool.h>
```

```
#include <stdint.h>
```

Funkcje

- void `lcd_demo` (void)
Run the original LCD demo routine.
- void `lcd_init` (void)
Initialize the LCD driver and display controller.
- void `lcd_write_string` (uint8_t *str)
Write a null-terminated string at the current cursor position.
- void `lcd_set_cursor` (uint8_t row, uint8_t column)
Move the LCD cursor.
- void `lcd_clear` (void)
Clear the LCD display.
- void `lcd_backlight` (uint8_t state)
Enable or disable LCD backlight.
- bool `lcd_write_line` (uint8_t row, const char *text, uint8_t width)
Write one padded/truncated text line.
- void `lcd_animation_hello_beko` (void)
Play the boot "HELLO BEKO" LCD animation.

12.29.1 Opis szczegółowy

HD44780-compatible 20x4 LCD driver over I2C expander.

Definicja w pliku [lcd.h](#).

12.29.2 Dokumentacja funkcji

12.29.2.1 `lcd_animation_hello_beko()`

```
void lcd_animation_hello_beko (
    void )
```

Play the boot "HELLO BEKO" LCD animation.

Definicja w linii 572 pliku [lcd.c](#).

12.29.2.2 `lcd_backlight()`

```
void lcd_backlight (
    uint8_t state)
```

Enable or disable LCD backlight.

Parametry

<i>state</i>	Non-zero to enable, zero to disable.
--------------	--------------------------------------

Definicja w linii 501 pliku [lcd.c](#).

Odwołuje się do [app_i2c_unlock\(\)](#).

12.29.2.3 lcd_clear()

```
void lcd_clear (
    void )
```

Clear the LCD display.

Definicja w linii 480 pliku [lcd.c](#).

Odwołuje się do [app_i2c_unlock\(\)](#).

Odwołania w [lcd_demo\(\)](#).

12.29.2.4 lcd_demo()

```
void lcd_demo (
    void )
```

Run the original LCD demo routine.

Definicja w linii 386 pliku [lcd.c](#).

Odwołuje się do [lcd_clear\(\)](#) i [lcd_init\(\)](#).

12.29.2.5 lcd_init()

```
void lcd_init (
    void )
```

Initialize the LCD driver and display controller.

Definicja w linii 406 pliku [lcd.c](#).

Odwołania w [lcd_demo\(\)](#).

12.29.2.6 lcd_set_cursor()

```
void lcd_set_cursor (
    uint8_t row,
    uint8_t column)
```

Move the LCD cursor.

Parametry

<i>row</i>	Zero-based row index.
<i>column</i>	Zero-based column index.

Definicja w linii 456 pliku [lcd.c](#).

Odwołuje się do [app_i2c_unlock\(\)](#).

12.29.2.7 lcd_write_line()

```
bool lcd_write_line (
    uint8_t row,
    const char * text,
    uint8_t width)
```

Write one padded/truncated text line.

Parametry

<i>row</i>	Zero-based row index.
<i>text</i>	Text to render.
<i>width</i>	Maximum display width.

Zwraca

true when the line was written.

Write one padded/truncated text line.

Writing the whole row every time avoids leftover characters when a shorter string replaces a longer one, which is especially visible in menu screens and popup overlays.

Definicja w linii 524 pliku `lcd.c`.

Odwołuje się do `app_i2c_unlock()`.

12.29.2.8 lcd_write_string()

```
void lcd_write_string (
    uint8_t * str)
```

Write a null-terminated string at the current cursor position.

Parametry

<i>str</i>	String bytes to send to the LCD.
------------	----------------------------------

Definicja w linii 430 pliku `lcd.c`.

Odwołuje się do `app_i2c_unlock()`.

12.30 lcd.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef INC_LCD_H_
00007 #define INC_LCD_H_
00008
00009 #include <stdbool.h>
00010 #include <stdint.h>
00011
00015 void lcd_demo(void);
00016
00020 void lcd_init(void);
00021
00026 void lcd_write_string(uint8_t *str);
00027
00033 void lcd_set_cursor(uint8_t row, uint8_t column);
00034
00038 void lcd_clear(void);
00039
00044 void lcd_backlight(uint8_t state);
00045
00053 bool lcd_write_line(uint8_t row, const char *text, uint8_t width);
00054
00058 void lcd_animation_hello_beko(void);
00059
00060
00061 #endif /* INC_LCD_H_ */
```

12.31 lcd_main.c

```
00001 #include "lcd_main.h"
00002
00003 #include "cmsis_os2.h"
00004 #include "FreeRTOS.h"
00005 #include "laviot_frame.h"
00006 #include "lcd_library/lcd.h"
00007 #include "task.h"
00008
00009 #include <ctype.h>
00010 #include <stdio.h>
00011 #include <string.h>
00012
00013 #define LCD_MAIN_ROWS          4U
00014 #define LCD_MAIN_COLS          20U
00015 #define LCD_MAIN_QUEUE_LENGTH 256U
```

```

00016 #define LCD_MAIN_MONITOR_HISTORY    64U
00017 #define LCD_TASK_STACK_SIZE          8192U
00018 #define LCD_TASK_STACK_WORDS         (LCD_TASK_STACK_SIZE / sizeof(StackType_t))
00019
00020 typedef enum
00021 {
00022     LCD_MAIN_MSG_SET_LINE = 0,
00023     LCD_MAIN_MSG_SET_LINES,
00024     LCD_MAIN_MSG_SHOW_MENU,
00025     LCD_MAIN_MSG_PUSH_MONITOR,
00026     LCD_MAIN_MSG_SET_MODE,
00027     LCD_MAIN_MSG_SHOW_POPUP,
00028     LCD_MAIN_MSG_SHOW_BOOT,
00029     LCD_MAIN_MSG_CLEAR,
00030     LCD_MAIN_MSG_MONITOR_SCROLL_UP,
00031     LCD_MAIN_MSG_MONITOR_SCROLL_DOWN
00032 } lcd_main_msg_type_t;
00033
00034 typedef struct
00035 {
00036     lcd_main_msg_type_t type;
00037     uint8_t line_index;
00038     lcd_main_mode_t mode;
00039     char text0[LCD_MAIN_COLS + 1U];
00040     char text1[LCD_MAIN_COLS + 1U];
00041     char text2[LCD_MAIN_COLS + 1U];
00042     char text3[LCD_MAIN_COLS + 1U];
00043 } lcd_main_msg_t;
00044
00045 static osThreadId_t s_lcd_task = NULL;
00046 static osMessageQueueId_t s_lcd_queue = NULL;
00047 static StaticTask_t s_lcd_task_cb;
00048 static StackType_t s_lcd_task_stack[LCD_TASK_STACK_WORDS];
00049
00050 static lcd_main_mode_t s_mode = LCD_MODE_MONITOR;
00051 static char s_monitor_lines[LCD_MAIN_ROWS][LCD_MAIN_COLS + 1U];
00052 static char s_monitor_history[LCD_MAIN_MONITOR_HISTORY][LCD_MAIN_COLS + 1U];
00053 static char s_ui_lines[LCD_MAIN_ROWS][LCD_MAIN_COLS + 1U];
00054 static char s_rendered_lines[LCD_MAIN_ROWS][LCD_MAIN_COLS + 1U];
00055 static uint8_t s_monitor_history_head = 0U;
00056 static uint8_t s_monitor_history_count = 0U;
00057 static uint8_t s_monitor_view_offset = 0U;
00058 static bool s_render_cache_valid = false;
00059
00060 static void lcd_main_task_fn(void *argument);
00061 static void lcd_main_fill_line(char *dst, const char *src);
00062 static void lcd_main_write_source_field(char *dst, uint32_t source_id);
00063 static void lcd_main_fill_line_from_payload(char *dst,
00064     int16_t rssi_dbm,
00065     uint32_t source_id,
00066     const uint8_t *data,
00067     uint32_t length);
00068 static void lcd_main_clear_lines(char lines[LCD_MAIN_ROWS][LCD_MAIN_COLS + 1U]);
00069 static void lcd_main_monitor_history_append(const char *line);
00070 static void lcd_main_monitor_rebuild_lines(void);
00071 static void lcd_main_render_mode(void);
00072 static void lcd_main_render_lines(char lines[LCD_MAIN_ROWS][LCD_MAIN_COLS + 1U]);
00073 static void lcd_main_invalidate_render_cache(void);
00074 static bool lcd_main_is_monitor_message_type(lcd_main_msg_type_t type);
00075 static void lcd_main_drop_pending_messages(void);
00076 static bool lcd_main_post_message(const lcd_main_msg_t *msg);
00077
00078 static const osThreadAttr_t s_lcd_task_attr =
00079 {
00080     .name = "lcd_task",
00081     .priority = (osPriority_t)osPriorityLow,
00082     .stack_mem = s_lcd_task_stack,
00083     .stack_size = sizeof(s_lcd_task_stack),
00084     .cb_mem = &s_lcd_task_cb,
00085     .cb_size = sizeof(s_lcd_task_cb)
00086 };
00087
00088 void lcd_main_create_task(void)
00089 {
00090     if (s_lcd_queue == NULL)
00091     {
00092         s_lcd_queue = osMessageQueueNew(LCD_MAIN_QUEUE_LENGTH, sizeof(lcd_main_msg_t), NULL);
00093         if (s_lcd_queue == NULL)
00094         {
00095             printf("LCD: queue create failed\r\n");
00096             return;
00097         }
00098     }
00099
00100     if (s_lcd_task == NULL)
00101     {
00102         s_lcd_task = osThreadNew(lcd_main_task_fn, NULL, &s_lcd_task_attr);

```

```

00103         if (s_lcd_task == NULL)
00104         {
00105             printf("LCD: task create failed\r\n");
00106         }
00107     }
00108 }
00109
00110 bool lcd_main_set_line(uint8_t line_index, const char *text)
00111 {
00112     lcd_main_msg_t msg;
00113
00114     if (line_index >= LCD_MAIN_ROWS)
00115     {
00116         return false;
00117     }
00118
00119     memset(&msg, 0, sizeof(msg));
00120     msg.type = LCD_MAIN_MSG_SET_LINE;
00121     msg.line_index = line_index;
00122     lcd_main_fill_line(msg.text0, text);
00123     return lcd_main_post_message(&msg);
00124 }
00125
00126 bool lcd_main_set_lines(const char *line0, const char *line1)
00127 {
00128     lcd_main_msg_t msg;
00129
00130     memset(&msg, 0, sizeof(msg));
00131     msg.type = LCD_MAIN_MSG_SET_LINES;
00132     lcd_main_fill_line(msg.text0, line0);
00133     lcd_main_fill_line(msg.text1, line1);
00134     return lcd_main_post_message(&msg);
00135 }
00136
00143 bool lcd_main_show_menu(const char *l0, const char *l1, const char *l2, const char *l3)
00144 {
00145     lcd_main_msg_t msg;
00146
00147     memset(&msg, 0, sizeof(msg));
00148     msg.type = LCD_MAIN_MSG_SHOW_MENU;
00149     lcd_main_fill_line(msg.text0, l0);
00150     lcd_main_fill_line(msg.text1, l1);
00151     lcd_main_fill_line(msg.text2, l2);
00152     lcd_main_fill_line(msg.text3, l3);
00153     return lcd_main_post_message(&msg);
00154 }
00155
00156 bool lcd_main_push_message(int16_t rssi_dbm, const uint8_t *data, uint32_t length)
00157 {
00158     return lcd_main_push_message_from(rssi_dbm, 0U, data, length);
00159 }
00160
00161 bool lcd_main_push_message_from(int16_t rssi_dbm, uint32_t source_id, const uint8_t *data, uint32_t
length)
00162 {
00163     lcd_main_msg_t msg;
00164
00165     if ((data == NULL) || (length == 0U))
00166     {
00167         return false;
00168     }
00169
00170     memset(&msg, 0, sizeof(msg));
00171     msg.type = LCD_MAIN_MSG_PUSH_MONITOR;
00172     lcd_main_fill_line_from_payload(msg.text0, rssi_dbm, source_id, data, length);
00173     return lcd_main_post_message(&msg);
00174 }
00175
00176 bool lcd_main_set_mode(lcd_main_mode_t mode)
00177 {
00178     lcd_main_msg_t msg;
00179
00180     memset(&msg, 0, sizeof(msg));
00181     msg.type = LCD_MAIN_MSG_SET_MODE;
00182     msg.mode = mode;
00183     return lcd_main_post_message(&msg);
00184 }
00185
00186 bool lcd_main_show_popup(const char *l0, const char *l1, const char *l2, const char *l3)
00187 {
00188     lcd_main_msg_t msg;
00189
00190     memset(&msg, 0, sizeof(msg));
00191     msg.type = LCD_MAIN_MSG_SHOW_POPUP;
00192     lcd_main_fill_line(msg.text0, l0);
00193     lcd_main_fill_line(msg.text1, l1);
00194     lcd_main_fill_line(msg.text2, l2);

```

```

00195     lcd_main_fill_line(msg.text3, 13);
00196     return lcd_main_post_message(&msg);
00197 }
00198
00199 bool lcd_main_show_boot_hello(void)
00200 {
00201     lcd_main_msg_t msg;
00202
00203     memset(&msg, 0, sizeof(msg));
00204     msg.type = LCD_MAIN_MSG_SHOW_BOOT;
00205     return lcd_main_post_message(&msg);
00206 }
00207
00208 bool lcd_main_monitor_scroll_up(void)
00209 {
00210     lcd_main_msg_t msg;
00211
00212     memset(&msg, 0, sizeof(msg));
00213     msg.type = LCD_MAIN_MSG_MONITOR_SCROLL_UP;
00214     return lcd_main_post_message(&msg);
00215 }
00216
00217 bool lcd_main_monitor_scroll_down(void)
00218 {
00219     lcd_main_msg_t msg;
00220
00221     memset(&msg, 0, sizeof(msg));
00222     msg.type = LCD_MAIN_MSG_MONITOR_SCROLL_DOWN;
00223     return lcd_main_post_message(&msg);
00224 }
00225
00227 static void lcd_main_task_fn(void *argument)
00228 {
00229     lcd_main_msg_t msg;
00230     bool render_required;
00231
00232     (void)argument;
00233
00234     lcd_init();
00235     lcd_backlight(1U);
00236     lcd_clear();
00237
00238     lcd_main_clear_lines(s_monitor_lines);
00239     lcd_main_clear_lines(s_monitor_history);
00240     lcd_main_clear_lines(s_ui_lines);
00241     lcd_main_clear_lines(s_rendered_lines);
00242     s_monitor_history_head = 0U;
00243     s_monitor_history_count = 0U;
00244     s_monitor_view_offset = 0U;
00245     s_render_cache_valid = false;
00246
00247     lcd_animation_hello_beko();
00248     lcd_main_clear_lines(s_monitor_lines);
00249     s_mode = LCD_MODE_MONITOR;
00250     lcd_main_render_mode();
00251
00252     for (;;)
00253     {
00254         if (osMessageQueueGet(s_lcd_queue, &msg, NULL, osWaitForever) != osOK)
00255         {
00256             continue;
00257         }
00258
00259         render_required = false;
00260
00261         do
00262         {
00263             switch (msg.type)
00264             {
00265                 case LCD_MAIN_MSG_SET_LINE:
00266                     if (msg.line_index < LCD_MAIN_ROWS)
00267                     {
00268                         lcd_main_fill_line(s_ui_lines[msg.line_index], msg.text0);
00269                         render_required = true;
00270                     }
00271                     break;
00272
00273                 case LCD_MAIN_MSG_SET_LINES:
00274                     lcd_main_clear_lines(s_ui_lines);
00275                     lcd_main_fill_line(s_ui_lines[0], msg.text0);
00276                     lcd_main_fill_line(s_ui_lines[1], msg.text1);
00277                     lcd_main_invalidate_render_cache();
00278                     render_required = true;
00279                     break;
00280
00281                 case LCD_MAIN_MSG_SHOW_MENU:
00282                     lcd_main_fill_line(s_ui_lines[0], msg.text0);

```

```

00283         lcd_main_fill_line(s_ui_lines[1], msg.text1);
00284         lcd_main_fill_line(s_ui_lines[2], msg.text2);
00285         lcd_main_fill_line(s_ui_lines[3], msg.text3);
00286         s_mode = LCD_MODE_MENU;
00287         lcd_main_invalidate_render_cache();
00288         render_required = true;
00289         break;
00290
00291     case LCD_MAIN_MSG_PUSH_MONITOR:
00292         lcd_main_monitor_history_append(msg.text0);
00293         if (s_mode == LCD_MODE_MONITOR)
00294         {
00295             render_required = true;
00296         }
00297         break;
00298
00299     case LCD_MAIN_MSG_SET_MODE:
00300         if ((msg.mode == LCD_MODE_MENU) || (msg.mode == LCD_MODE_POPUP))
00301         {
00302             lcd_main_clear_lines(s_ui_lines);
00303         }
00304         s_mode = msg.mode;
00305         if (msg.mode == LCD_MODE_MONITOR)
00306         {
00307             s_monitor_view_offset = 0U;
00308             lcd_main_monitor_rebuild_lines();
00309         }
00310         lcd_main_invalidate_render_cache();
00311         render_required = true;
00312         break;
00313
00314     case LCD_MAIN_MSG_SHOW_POPUP:
00315         lcd_main_clear_lines(s_ui_lines);
00316         lcd_main_fill_line(s_ui_lines[0], msg.text0);
00317         lcd_main_fill_line(s_ui_lines[1], msg.text1);
00318         lcd_main_fill_line(s_ui_lines[2], msg.text2);
00319         lcd_main_fill_line(s_ui_lines[3], msg.text3);
00320         s_mode = LCD_MODE_POPUP;
00321         lcd_main_invalidate_render_cache();
00322         render_required = true;
00323         break;
00324
00325     case LCD_MAIN_MSG_SHOW_BOOT:
00326         lcd_animation_hello_beko();
00327         s_mode = LCD_MODE_MONITOR;
00328         s_monitor_view_offset = 0U;
00329         lcd_main_monitor_rebuild_lines();
00330         lcd_main_invalidate_render_cache();
00331         render_required = true;
00332         break;
00333
00334     case LCD_MAIN_MSG_CLEAR:
00335         lcd_main_clear_lines(s_monitor_lines);
00336         lcd_main_clear_lines(s_monitor_history);
00337         lcd_main_clear_lines(s_ui_lines);
00338         s_monitor_history_head = 0U;
00339         s_monitor_history_count = 0U;
00340         s_monitor_view_offset = 0U;
00341         lcd_main_monitor_rebuild_lines();
00342         lcd_main_invalidate_render_cache();
00343         render_required = true;
00344         break;
00345
00346     case LCD_MAIN_MSG_MONITOR_SCROLL_UP:
00347         if (s_monitor_history_count > LCD_MAIN_ROWS)
00348         {
00349             uint8_t max_offset = (uint8_t)(s_monitor_history_count - LCD_MAIN_ROWS);
00350
00351             if (s_monitor_view_offset < max_offset)
00352             {
00353                 s_monitor_view_offset++;
00354                 lcd_main_monitor_rebuild_lines();
00355                 render_required = (s_mode == LCD_MODE_MONITOR);
00356             }
00357         }
00358         break;
00359
00360     case LCD_MAIN_MSG_MONITOR_SCROLL_DOWN:
00361         if (s_monitor_view_offset > 0U)
00362         {
00363             s_monitor_view_offset--;
00364             lcd_main_monitor_rebuild_lines();
00365             render_required = (s_mode == LCD_MODE_MONITOR);
00366         }
00367         break;
00368
00369     default:

```

```

00370             break;
00371         }
00372     } while (osMessageQueueGet(s_lcd_queue, &msg, NULL, 0U) == osOK);
00373     if (render_required)
00374     {
00375         lcd_main_render_mode();
00376     }
00377 }
00378 }
00379 }
00380
00382 static void lcd_main_fill_line(char *dst, const char *src)
00383 {
00384     uint8_t i;
00385
00386     if (dst == NULL)
00387     {
00388         return;
00389     }
00390
00391     for (i = 0U; i < LCD_MAIN_COLS; i++)
00392     {
00393         if ((src != NULL) && (src[i] != '\0'))
00394         {
00395             dst[i] = src[i];
00396         }
00397         else
00398         {
00399             dst[i] = ' ';
00400         }
00401     }
00402     dst[LCD_MAIN_COLS] = '\0';
00403 }
00404
00406 static void lcd_main_write_source_field(char *dst, uint32_t source_id)
00407 {
00408     static const char hex[] = "0123456789ABCDEF";
00409     uint16_t source16;
00410
00411     if (dst == NULL)
00412     {
00413         return;
00414     }
00415
00416     if (source_id == LAVIET_GATEWAY_ID)
00417     {
00418         dst[0] = 'G';
00419         dst[1] = 'A';
00420         dst[2] = 'T';
00421         dst[3] = 'E';
00422         return;
00423     }
00424     if (source_id == LAVIET_BROADCAST_ID)
00425     {
00426         dst[0] = 'B';
00427         dst[1] = 'C';
00428         dst[2] = 'S';
00429         dst[3] = 'T';
00430         return;
00431     }
00432     if ((source_id == 0UL) || (source_id > 0xFFFFUL))
00433     {
00434         dst[0] = '-';
00435         dst[1] = '-';
00436         dst[2] = '-';
00437         dst[3] = '-';
00438         return;
00439     }
00440
00441     source16 = (uint16_t)source_id;
00442     dst[0] = hex[(source16 >> 12) & 0x0FU];
00443     dst[1] = hex[(source16 >> 8) & 0x0FU];
00444     dst[2] = hex[(source16 >> 4) & 0x0FU];
00445     dst[3] = hex[source16 & 0x0FU];
00446 }
00447
00449 static void lcd_main_fill_line_from_payload(char *dst,
00450                                             int16_t rssi_dbm,
00451                                             uint32_t source_id,
00452                                             const uint8_t *data,
00453                                             uint32_t length)
00454 {
00455     uint32_t i;
00456     uint32_t msg_start = 5U;
00457     uint32_t msg_max_len = (LCD_MAIN_COLS - msg_start);
00458
00459     if (dst == NULL)

```



```

00460     {
00461         return;
00462     }
00463
00464     (void) rssi_dbm;
00465     lcd_main_fill_line(dst, NULL);
00466     lcd_main_write_source_field(dst, source_id);
00467     dst[4] = ':';
00468
00469     for (i = 0U; i < msg_max_len; i++)
00470     {
00471         uint32_t src_idx = i;
00472         uint32_t dst_idx = i + msg_start;
00473         if ((src_idx < length) && (data != NULL))
00474         {
00475             char c = (char) data[src_idx];
00476             dst[dst_idx] = isprint((unsigned char) c) ? c : '.';
00477         }
00478         else
00479         {
00480             dst[dst_idx] = ' ';
00481         }
00482     }
00483
00484     dst[LCD_MAIN_COLS] = '\0';
00485 }
00486
00487 static void lcd_main_clear_lines(char lines[LCD_MAIN_ROWS][LCD_MAIN_COLS + 1U])
00488 {
00489     uint8_t row;
00490
00491     for (row = 0U; row < LCD_MAIN_ROWS; row++)
00492     {
00493         lcd_main_fill_line(lines[row], NULL);
00494     }
00495 }
00496
00497 static void lcd_main_monitor_history_append(const char *line)
00498 {
00499     uint8_t old_count = s_monitor_history_count;
00500     uint8_t max_offset;
00501     uint8_t idx = s_monitor_history_head;
00502
00503     lcd_main_fill_line(s_monitor_history[idx], line);
00504     s_monitor_history_head = (uint8_t)((s_monitor_history_head + 1U) % LCD_MAIN_MONITOR_HISTORY);
00505     if (s_monitor_history_count < LCD_MAIN_MONITOR_HISTORY)
00506     {
00507         s_monitor_history_count++;
00508     }
00509
00510     if ((old_count >= LCD_MAIN_ROWS) && (s_monitor_view_offset > 0U))
00511     {
00512         max_offset = (s_monitor_history_count > LCD_MAIN_ROWS) ?
00513                     (uint8_t)(s_monitor_history_count - LCD_MAIN_ROWS) : 0U;
00514         if (s_monitor_view_offset < max_offset)
00515         {
00516             s_monitor_view_offset++;
00517         }
00518     }
00519
00520     lcd_main_monitor_rebuild_lines();
00521 }
00522
00523 static void lcd_main_monitor_rebuild_lines(void)
00524 {
00525     uint8_t oldest_idx;
00526     uint8_t start;
00527     uint8_t visible;
00528     uint8_t row_base;
00529     uint8_t i;
00530     uint8_t max_offset;
00531
00532     lcd_main_clear_lines(s_monitor_lines);
00533     if (s_monitor_history_count == 0U)
00534     {
00535         return;
00536     }
00537
00538     max_offset = (s_monitor_history_count > LCD_MAIN_ROWS) ?
00539                 (uint8_t)(s_monitor_history_count - LCD_MAIN_ROWS) : 0U;
00540     if (s_monitor_view_offset > max_offset)
00541     {
00542         s_monitor_view_offset = max_offset;
00543     }
00544
00545     if (s_monitor_history_count > LCD_MAIN_ROWS)
00546     {

```

```

00550         start = (uint8_t)(s_monitor_history_count - LCD_MAIN_ROWS - s_monitor_view_offset);
00551         visible = LCD_MAIN_ROWS;
00552         row_base = 0U;
00553     }
00554     else
00555     {
00556         start = 0U;
00557         visible = s_monitor_history_count;
00558         row_base = (uint8_t)(LCD_MAIN_ROWS - visible);
00559     }
00560
00561     oldest_idx = (uint8_t)((s_monitor_history_head + LCD_MAIN_MONITOR_HISTORY -
s_monitor_history_count) %
00562                             LCD_MAIN_MONITOR_HISTORY);
00563     for (i = 0U; i < visible; i++)
00564     {
00565         uint8_t history_idx = (uint8_t)((oldest_idx + start + i) % LCD_MAIN_MONITOR_HISTORY);
00566
00567         memcpy(s_monitor_lines[row_base + i],
00568             s_monitor_history[history_idx],
00569             (LCD_MAIN_COLS + 1U));
00570     }
00571 }
00572
00573 static void lcd_main_render_mode(void)
00574 {
00575     if (s_mode == LCD_MODE_MONITOR)
00576     {
00577         lcd_main_render_lines(s_monitor_lines);
00578     }
00579     else
00580     {
00581         lcd_main_render_lines(s_ui_lines);
00582     }
00583 }
00584 }
00585
00586 static void lcd_main_render_lines(char lines[LCD_MAIN_ROWS][LCD_MAIN_COLS + 1U])
00587 {
00588     uint8_t row;
00589     bool all_ok = true;
00590
00591     for (row = 0U; row < LCD_MAIN_ROWS; row++)
00592     {
00593         if ((!s_render_cache_valid) ||
00594             (memcmp(lines[row], s_rendered_lines[row], (LCD_MAIN_COLS + 1U)) != 0))
00595         {
00596             if (lcd_write_line(row, lines[row], LCD_MAIN_COLS))
00597             {
00598                 memcpy(s_rendered_lines[row], lines[row], (LCD_MAIN_COLS + 1U));
00599             }
00600             else
00601             {
00602                 all_ok = false;
00603             }
00604         }
00605     }
00606 }
00607
00608 s_render_cache_valid = all_ok;
00609 }
00610
00611 static void lcd_main_invalidate_render_cache(void)
00612 {
00613     s_render_cache_valid = false;
00614 }
00615 }
00616
00617 static bool lcd_main_is_monitor_message_type(lcd_main_msg_type_t type)
00618 {
00619     return (type == LCD_MAIN_MSG_PUSH_MONITOR);
00620 }
00621 }
00622
00623 /*
00624 * Full-screen UI updates must win over any stale backlog. Dropping pending LCD messages here is
00625 * cheaper than letting old popups arrive late and overwrite the current menu state.
00626 */
00627 static void lcd_main_drop_pending_messages(void)
00628 {
00629     lcd_main_msg_t dropped;
00630
00631     if (s_lcd_queue == NULL)
00632     {
00633         return;
00634     }
00635
00636     while (osMessageQueueGet(s_lcd_queue, &dropped, NULL, 0U) == osOK)
00637     {
00638     }
00639 }

```

```

00640
00642 static bool lcd_main_post_message(const lcd_main_msg_t *msg)
00643 {
00644     osStatus_t st;
00645
00646     if ((msg == NULL) || (s_lcd_queue == NULL))
00647     {
00648         return false;
00649     }
00650
00651     st = osMessageQueuePut(s_lcd_queue, msg, 0U, 0U);
00652     if (st == osOK)
00653     {
00654         return true;
00655     }
00656
00657     if (st == osErrorResource)
00658     {
00659         if (lcd_main_is_monitor_message_type(msg->type))
00660         {
00661             return false;
00662         }
00663
00664         lcd_main_drop_pending_messages();
00665         st = osMessageQueuePut(s_lcd_queue, msg, 0U, 5U);
00666         if (st == osOK)
00667         {
00668             return true;
00669         }
00670     }
00671
00672     return false;
00673 }

```

12.32 Dokumentacja pliku Core/App/lcd_main.h

RTOS-facing LCD rendering task API.

```
#include <stdbool.h>
```

```
#include <stdint.h>
```

Wyliczenia

- enum `lcd_main_mode_t` { `LCD_MODE_MONITOR` = 0 , `LCD_MODE_MENU` , `LCD_MODE_POPUP` }

High-level ownership mode for the LCD task.

Funkcje

- void `lcd_main_create_task` (void)
Create the LCD task and message queue.
- bool `lcd_main_set_line` (uint8_t line_index, const char *text)
Update one monitor line.
- bool `lcd_main_set_lines` (const char *line0, const char *line1)
Update the first two monitor lines.
- bool `lcd_main_show_menu` (const char *l0, const char *l1, const char *l2, const char *l3)
Replace the menu screen with four LCD lines.
- bool `lcd_main_push_message` (int16_t rssi_dbm, const uint8_t *data, uint32_t length)
Append an RX message to the monitor history.
- bool `lcd_main_push_message_from` (int16_t rssi_dbm, uint32_t source_id, const uint8_t *data, uint32_t length)
Append an RX message with source identifier to the monitor history.
- bool `lcd_main_set_mode` (lcd_main_mode_t mode)
Switch LCD ownership mode.
- bool `lcd_main_show_popup` (const char *l0, const char *l1, const char *l2, const char *l3)
Show a four-line popup screen.
- bool `lcd_main_show_boot_hello` (void)

Queue the boot welcome animation.

- bool `lcd_main_monitor_scroll_up` (void)

Scroll monitor history one line toward older entries.

- bool `lcd_main_monitor_scroll_down` (void)

Scroll monitor history one line toward newer entries.

12.32.1 Opis szczegółowy

RTOS-facing LCD rendering task API.

Definicja w pliku `lcd_main.h`.

12.32.2 Dokumentacja typów wyliczanych

12.32.2.1 `lcd_main_mode_t`

enum `lcd_main_mode_t`

High-level ownership mode for the LCD task.

Wartości wyliczeń

LCD_MODE_MONITOR	Show radio monitor history.
LCD_MODE_MENU	Show menu-provided 4-line screen.
LCD_MODE_POPUP	Show modal/popup content.

Definicja w linii 15 pliku `lcd_main.h`.

12.32.3 Dokumentacja funkcji

12.32.3.1 `lcd_main_create_task()`

```
void lcd_main_create_task (
    void )
```

Create the LCD task and message queue.

Definicja w linii 88 pliku `lcd_main.c`.

Odwolania w `app_freertos_init()`.

12.32.3.2 `lcd_main_monitor_scroll_down()`

```
bool lcd_main_monitor_scroll_down (
    void )
```

Scroll monitor history one line toward newer entries.

Zwraca

`true` when the request was queued.

Definicja w linii 217 pliku `lcd_main.c`.

12.32.3.3 `lcd_main_monitor_scroll_up()`

```
bool lcd_main_monitor_scroll_up (
    void )
```

Scroll monitor history one line toward older entries.

Zwraca

`true` when the request was queued.

Definicja w linii 208 pliku `lcd_main.c`.

12.32.3.4 lcd_main_push_message()

```
bool lcd_main_push_message (
    int16_t rssi_dbm,
    const uint8_t * data,
    uint32_t length)
```

Append an RX message to the monitor history.

Parametry

<i>rssi_dbm</i>	RSSI associated with the payload.
<i>data</i>	Payload bytes.
<i>length</i>	Payload length.

Zwraca

`true` when the message was queued.

Definicja w linii 156 pliku [lcd_main.c](#).

12.32.3.5 lcd_main_push_message_from()

```
bool lcd_main_push_message_from (
    int16_t rssi_dbm,
    uint32_t source_id,
    const uint8_t * data,
    uint32_t length)
```

Append an RX message with source identifier to the monitor history.

Parametry

<i>rssi_dbm</i>	RSSI associated with the payload.
<i>source_id</i>	Source node/device identifier.
<i>data</i>	Payload bytes.
<i>length</i>	Payload length.

Zwraca

`true` when the message was queued.

Definicja w linii 161 pliku [lcd_main.c](#).

12.32.3.6 lcd_main_set_line()

```
bool lcd_main_set_line (
    uint8_t line_index,
    const char * text)
```

Update one monitor line.

Parametry

<i>line_index</i>	Zero-based LCD line index.
<i>text</i>	Text copied and padded to display width.

Zwraca

`true` when the update request was queued.

Definicja w linii 110 pliku [lcd_main.c](#).

12.32.3.7 lcd_main_set_lines()

```
bool lcd_main_set_lines (
    const char * line0,
    const char * line1)
```

Update the first two monitor lines.

Parametry

<i>line0</i>	First line text.
<i>line1</i>	Second line text.

Zwraca

`true` when the update request was queued.

Definicja w linii 126 pliku [lcd_main.c](#).

12.32.3.8 lcd_main_set_mode()

```
bool lcd_main_set_mode (
    lcd_main_mode_t mode)
```

Switch LCD ownership mode.

Parametry

<i>mode</i>	New LCD mode.
-------------	---------------

Zwraca

`true` when the mode change was queued.

Definicja w linii 176 pliku [lcd_main.c](#).

12.32.3.9 lcd_main_show_boot_hello()

```
bool lcd_main_show_boot_hello (
    void )
```

Queue the boot welcome animation.

Zwraca

`true` when the request was queued.

Definicja w linii 199 pliku [lcd_main.c](#).

12.32.3.10 lcd_main_show_menu()

```
bool lcd_main_show_menu (
    const char * i0,
    const char * i1,
    const char * i2,
    const char * i3)
```

Replace the menu screen with four LCD lines.

Parametry

<i>l0</i>	First line.
<i>l1</i>	Second line.
<i>l2</i>	Third line.
<i>l3</i>	Fourth line.

Zwraca

`true` when the screen was queued.

Replace the menu screen with four LCD lines.

Menu navigation used to enqueue each line separately, which could leave stale rows on screen when the LCD queue was busy. Sending the whole frame atomically keeps the UI coherent.

Definicja w linii 143 pliku `lcd_main.c`.

12.32.3.11 lcd_main_show_popup()

```
bool lcd_main_show_popup (
    const char * l0,
    const char * l1,
    const char * l2,
    const char * l3)
```

Show a four-line popup screen.

Parametry

<i>l0</i>	First line.
<i>l1</i>	Second line.
<i>l2</i>	Third line.
<i>l3</i>	Fourth line.

Zwraca

`true` when the popup was queued.

Definicja w linii 186 pliku `lcd_main.c`.

12.33 lcd_main.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_LCD_MAIN_H_
00007 #define APP_LCD_MAIN_H_
00008
00009 #include <stdbool.h>
00010 #include <stdint.h>
00011
00015 typedef enum
00016 {
00017     LCD_MODE_MONITOR = 0,
00018     LCD_MODE_MENU,
00019     LCD_MODE_POPUP
00020 } lcd_main_mode_t;
00021
00025 void lcd_main_create_task(void);
00026
```

```

00033 bool lcd_main_set_line(uint8_t line_index, const char *text);
00034
00041 bool lcd_main_set_lines(const char *line0, const char *line1);
00042
00051 bool lcd_main_show_menu(const char *l0, const char *l1, const char *l2, const char *l3);
00052
00060 bool lcd_main_push_message(int16_t rssi_dbm, const uint8_t *data, uint32_t length);
00061
00070 bool lcd_main_push_message_from(int16_t rssi_dbm, uint32_t source_id, const uint8_t *data, uint32_t
length);
00071
00077 bool lcd_main_set_mode(lcd_main_mode_t mode);
00078
00087 bool lcd_main_show_popup(const char *l0, const char *l1, const char *l2, const char *l3);
00088
00093 bool lcd_main_show_boot_hello(void);
00094
00099 bool lcd_main_monitor_scroll_up(void);
00100
00105 bool lcd_main_monitor_scroll_down(void);
00106
00107 #endif /* APP_LCD_MAIN_H */

```

12.34 Dokumentacja pliku Core/App/led_array_lib/led_array_lib.c

LED array driver implementation.

```

#include "led_array_lib.h"
#include "cmsis_os2.h"
#include <string.h>

```

Struktury danych

- struct [led_array_ctx_t](#)

Definicje

- #define [LED_ARRAY_COUNT](#) 5U
- #define [LED_ARRAY_VALID_MASK](#) LED_ARRAY_LED_ALL
- #define [LED_ARRAY_RAINBOW_PHASE_INC](#) 8U
- #define [LED_ARRAY_PWM_FREQ_HZ](#) 20000U
- #define [LED_ARRAY_PWM_STEPS](#) 100U

Funkcje

- static uint32_t [led_array_now_ms](#) (void)
Internal helper: [led_array_now_ms](#).
- static uint32_t [led_array_get_tim_clock](#) (TIM_TypeDef *tim)
Internal helper: [led_array_get_tim_clock](#).
- static void [led_array_timer_base_init](#) (TIM_TypeDef *tim)
Internal helper: [led_array_timer_base_init](#).
- static void [led_array_timer_pwm_ch_init](#) (TIM_TypeDef *tim, uint8_t channel)
Internal helper: [led_array_timer_pwm_ch_init](#).
- static void [led_array_hw_init](#) (void)
Internal helper: [led_array_hw_init](#).
- static void [led_array_set_hw_duty](#) (uint8_t index, uint8_t pct)
Internal helper: [led_array_set_hw_duty](#).
- static bool [led_array_mask_valid](#) (uint8_t mask)
Internal helper: [led_array_mask_valid](#).
- static uint8_t [led_array_clamp_pct](#) (uint8_t value)
Internal helper: [led_array_clamp_pct](#).
- static void [led_array_write_pin](#) (uint8_t index, GPIO_PinState state)

- Internal helper: [led_array_write_pin](#).*
- static void [led_array_apply_outputs](#) (void)
 - Internal helper: [led_array_apply_outputs](#).*
- static uint8_t [led_array_tri8](#) (uint8_t phase)
 - Internal helper: [led_array_tri8](#).*
- static void [led_array_effect_none](#) (void)
 - Internal helper: [led_array_effect_none](#).*
- static void [led_array_effect_fade_update](#) (uint32_t delta_ms)
 - Internal helper: [led_array_effect_fade_update](#).*
- static void [led_array_effect_breath_update](#) (uint32_t delta_ms)
 - Internal helper: [led_array_effect_breath_update](#).*
- static void [led_array_effect_rainbow_update](#) (uint32_t delta_ms)
 - Internal helper: [led_array_effect_rainbow_update](#).*
- static void [led_array_tick_core](#) (uint32_t delta_ms)
 - Internal helper: [led_array_tick_core](#).*
- [led_array_status_t](#) [led_array_init](#) (void)
 - Initialize the LED array driver.*
- [led_array_status_t](#) [led_array_deinit](#) (void)
 - Deinitialize the LED array driver and turn off all LEDs.*
- [led_array_status_t](#) [led_array_timer_init](#) (uint32_t tick_period_ms)
 - Configure internal scheduler tick period.*
- [led_array_status_t](#) [led_array_timer_deinit](#) (void)
 - Disable the scheduler updates.*
- void [led_array_timer_tick](#) (void)
 - Tick function for ISR-driven scheduling.*
- void [led_array_process](#) (void)
 - Polling scheduler update (HAL_GetTick based).*
- [led_array_status_t](#) [led_array_on](#) (uint8_t mask)
 - Turn LEDs on at 100% brightness.*
- [led_array_status_t](#) [led_array_off](#) (uint8_t mask)
 - Turn LEDs off.*
- [led_array_status_t](#) [led_array_set_brightness](#) (uint8_t mask, uint8_t brightness_pct)
 - Set LED brightness in range 0..100%.*
- [led_array_status_t](#) [led_array_fire_fade_out](#) (uint8_t mask, uint16_t fade_ms)
 - Trigger one-shot fade-out from 100% to 0%.*
- [led_array_status_t](#) [led_array_start_breath](#) (uint8_t mask, uint16_t period_ms, uint8_t min_pct, uint8_t max_pct)
 - Start breathing effect on selected LEDs.*
- [led_array_status_t](#) [led_array_start_rainbow](#) (uint16_t step_ms, uint8_t min_pct, uint8_t max_pct)
 - Start circulating rainbow-like wave on all LEDs.*
- [led_array_status_t](#) [led_array_stop_effect](#) (void)
 - Stop the active effect and keep current manual brightness values.*
- [led_array_effect_t](#) [led_array_get_effect](#) (void)
 - Read currently active global effect.*
- bool [led_array_is_initialized](#) (void)
 - Read initialization state.*

Zmienne

- static [led_array_ctx_t](#) [s_led](#)

12.34.1 Opis szczegółowy

LED array driver implementation.
Definicja w pliku [led_array_lib.c](#).

12.34.2 Dokumentacja definicji

12.34.2.1 LED_ARRAY_COUNT

```
#define LED_ARRAY_COUNT 5U
```

Definicja w linii 12 pliku [led_array_lib.c](#).

12.34.2.2 LED_ARRAY_PWM_FREQ_HZ

```
#define LED_ARRAY_PWM_FREQ_HZ 20000U
```

Definicja w linii 15 pliku [led_array_lib.c](#).

12.34.2.3 LED_ARRAY_PWM_STEPS

```
#define LED_ARRAY_PWM_STEPS 100U
```

Definicja w linii 16 pliku [led_array_lib.c](#).

12.34.2.4 LED_ARRAY_RAINBOW_PHASE_INC

```
#define LED_ARRAY_RAINBOW_PHASE_INC 8U
```

Definicja w linii 14 pliku [led_array_lib.c](#).

12.34.2.5 LED_ARRAY_VALID_MASK

```
#define LED_ARRAY_VALID_MASK LED_ARRAY_LED_ALL
```

Definicja w linii 13 pliku [led_array_lib.c](#).

12.34.3 Dokumentacja funkcji

12.34.3.1 led_array_apply_outputs()

```
void led_array_apply_outputs (
    void ) [static]
```

Internal helper: [led_array_apply_outputs](#).
Definicja w linii 271 pliku [led_array_lib.c](#).
Odwołuje się do [led_array_set_hw_duty\(\)](#).
Odwołania w [led_array_tick_core\(\)](#).

12.34.3.2 led_array_clamp_pct()

```
uint8_t led_array_clamp_pct (
    uint8_t value) [static]
```

Internal helper: [led_array_clamp_pct](#).
Definicja w linii 252 pliku [led_array_lib.c](#).
Odwołania w [led_array_set_brightness\(\)](#), [led_array_start_breath\(\)](#) i [led_array_start_rainbow\(\)](#).

12.34.3.3 led_array_deinit()

```
led_array_status_t led_array_deinit (
    void )
```

Deinitialize the LED array driver and turn off all LEDs.

Zwraca

Driver status.

Definicja w linii 471 pliku [led_array_lib.c](#).
Odwołuje się do [led_array_write_pin\(\)](#).

12.34.3.4 led_array_effect_breath_update()

```
void led_array_effect_breath_update (
    uint32_t delta_ms) [static]
```

Internal helper: [led_array_effect_breath_update](#).

Definicja w linii 350 pliku [led_array_lib.c](#).

Odwołania w [led_array_tick_core\(\)](#).

12.34.3.5 led_array_effect_fade_update()

```
void led_array_effect_fade_update (
    uint32_t delta_ms) [static]
```

Internal helper: [led_array_effect_fade_update](#).

Definicja w linii 300 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_effect_none\(\)](#).

Odwołania w [led_array_tick_core\(\)](#).

12.34.3.6 led_array_effect_none()

```
void led_array_effect_none (
    void ) [static]
```

Internal helper: [led_array_effect_none](#).

Definicja w linii 294 pliku [led_array_lib.c](#).

Odwołania w [led_array_effect_fade_update\(\)](#) i [led_array_tick_core\(\)](#).

12.34.3.7 led_array_effect_rainbow_update()

```
void led_array_effect_rainbow_update (
    uint32_t delta_ms) [static]
```

Internal helper: [led_array_effect_rainbow_update](#).

Definicja w linii 403 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_tri8\(\)](#).

Odwołania w [led_array_tick_core\(\)](#).

12.34.3.8 led_array_fire_fade_out()

```
led_array_status_t led_array_fire_fade_out (
    uint8_t mask,
    uint16_t fade_ms)
```

Trigger one-shot fade-out from 100% to 0%.

Parametry

<i>mask</i>	LED mask.
<i>fade_ms</i>	Fade duration in milliseconds.

Zwraca

Driver status.

Definicja w linii 617 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_mask_valid\(\)](#).

12.34.3.9 led_array_get_effect()

```
led_array_effect_t led_array_get_effect (
    void )
```

Read currently active global effect.

Zwraca

Active effect.

Definicja w linii 716 pliku [led_array_lib.c](#).

12.34.3.10 led_array_get_tim_clock()

```
uint32_t led_array_get_tim_clock (
    TIM_TypeDef * tim) [static]
```

Internal helper: [led_array_get_tim_clock](#).

Definicja w linii 76 pliku [led_array_lib.c](#).

Odwwołania w [led_array_timer_base_init\(\)](#).

12.34.3.11 led_array_hw_init()

```
void led_array_hw_init (
    void ) [static]
```

Internal helper: [led_array_hw_init](#).

Definicja w linii 167 pliku [led_array_lib.c](#).

Odwwołuje się do [led_array_timer_base_init\(\)](#) i [led_array_timer_pwm_ch_init\(\)](#).

Odwwołania w [led_array_init\(\)](#).

12.34.3.12 led_array_init()

```
led_array_status_t led_array_init (
    void )
```

Initialize the LED array driver.

Zwraca

Driver status.

Definicja w linii 457 pliku [led_array_lib.c](#).

Odwwołuje się do [led_array_hw_init\(\)](#), [LED_ARRAY_LIB_DEFAULT_TICK_MS](#) i [led_array_now_ms\(\)](#).

12.34.3.13 led_array_is_initialized()

```
bool led_array_is_initialized (
    void )
```

Read initialization state.

Zwraca

true if initialized.

Definicja w linii 725 pliku [led_array_lib.c](#).

12.34.3.14 led_array_mask_valid()

```
bool led_array_mask_valid (
    uint8_t mask) [static]
```

Internal helper: [led_array_mask_valid](#).

Definicja w linii 242 pliku [led_array_lib.c](#).

Odwwołania w [led_array_fire_fade_out\(\)](#), [led_array_off\(\)](#), [led_array_on\(\)](#), [led_array_set_brightness\(\)](#) i [led_array_start_breath\(\)](#).

12.34.3.15 led_array_now_ms()

```
uint32_t led_array_now_ms (
    void ) [static]
```

Internal helper: [led_array_now_ms](#).

Definicja w linii 54 pliku [led_array_lib.c](#).

Odwwołania w [led_array_init\(\)](#), [led_array_process\(\)](#) i [led_array_timer_init\(\)](#).

12.34.3.16 led_array_off()

```
led_array_status_t led_array_off (
    uint8_t mask)
```

Turn LEDs off.

Parametry

<i>mask</i>	LED mask.
-------------	-----------

Zwraca

Driver status.

Definicja w linii 568 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_mask_valid\(\)](#).

12.34.3.17 led_array_on()

```
led_array_status_t led_array_on (
    uint8_t mask)
```

Turn LEDs on at 100% brightness.

Parametry

<i>mask</i>	LED mask.
-------------	-----------

Zwraca

Driver status.

Definicja w linii 544 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_mask_valid\(\)](#).

12.34.3.18 led_array_process()

```
void led_array_process (
    void )
```

Polling scheduler update (HAL_GetTick based).

Call this from the main loop when not using ISR-driven ticks.

Definicja w linii 527 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_now_ms\(\)](#) i [led_array_tick_core\(\)](#).

12.34.3.19 led_array_set_brightness()

```
led_array_status_t led_array_set_brightness (
    uint8_t mask,
    uint8_t brightness_pct)
```

Set LED brightness in range 0..100%.

Parametry

<i>mask</i>	LED mask.
<i>brightness_pct</i>	Brightness in percent.

Zwraca

Driver status.

Definicja w linii 592 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_clamp_pct\(\)](#) i [led_array_mask_valid\(\)](#).

12.34.3.20 led_array_set_hw_duty()

```
void led_array_set_hw_duty (
    uint8_t index,
    uint8_t pct) [static]
```

Internal helper: [led_array_set_hw_duty](#).

Definicja w linii 220 pliku [led_array_lib.c](#).

Odwołania w [led_array_apply_outputs\(\)](#) i [led_array_write_pin\(\)](#).

12.34.3.21 led_array_start_breath()

```
led_array_status_t led_array_start_breath (
    uint8_t mask,
    uint16_t period_ms,
    uint8_t min_pct,
    uint8_t max_pct)
```

Start breathing effect on selected LEDs.

Parametry

<i>mask</i>	LED mask.
<i>period_ms</i>	Full breath period (up + down) in milliseconds.
<i>min_pct</i>	Minimum brightness (0..100).
<i>max_pct</i>	Maximum brightness (0..100, must be $\geq$ min_pct).

Zwraca

Driver status.

Definicja w linii 646 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_clamp_pct\(\)](#) i [led_array_mask_valid\(\)](#).

12.34.3.22 led_array_start_rainbow()

```
led_array_status_t led_array_start_rainbow (
    uint16_t step_ms,
    uint8_t min_pct,
    uint8_t max_pct)
```

Start circulating rainbow-like wave on all LEDs.

For a monochrome LED strip this effect is implemented as a moving brightness wave across LED1..LED5.

Parametry

<i>step_ms</i>	Phase step period in milliseconds.
<i>min_pct</i>	Minimum brightness (0..100).
<i>max_pct</i>	Maximum brightness (0..100, must be $\geq$ min_pct).

Zwraca

Driver status.

Definicja w linii 676 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_clamp_pct\(\)](#).

12.34.3.23 led_array_stop_effect()

```
led_array_status_t led_array_stop_effect (
    void )
```

Stop the active effect and keep current manual brightness values.

Zwraca

Driver status.

Definicja w linii 705 pliku [led_array_lib.c](#).

12.34.3.24 led_array_tick_core()

```
void led_array_tick_core (
    uint32_t delta_ms) [static]
```

Internal helper: [led_array_tick_core](#).

Definicja w linii 435 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_apply_outputs\(\)](#), [led_array_effect_breath_update\(\)](#), [led_array_effect_fade_update\(\)](#), [led_array_effect_none\(\)](#) i [led_array_effect_rainbow_update\(\)](#).

Odwołania w [led_array_process\(\)](#) i [led_array_timer_tick\(\)](#).

12.34.3.25 led_array_timer_base_init()

```
void led_array_timer_base_init (
    TIM_TypeDef * tim) [static]
```

Internal helper: [led_array_timer_base_init](#).

Definicja w linii 114 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_get_tim_clock\(\)](#).

Odwołania w [led_array_hw_init\(\)](#).

12.34.3.26 led_array_timer_deinit()

```
led_array_status_t led_array_timer_deinit (
    void )
```

Disable the scheduler updates.

Zwraca

Driver status.

Definicja w linii 506 pliku [led_array_lib.c](#).

12.34.3.27 led_array_timer_init()

```
led_array_status_t led_array_timer_init (
    uint32_t tick_period_ms)
```

Configure internal scheduler tick period.

This controls effect update and brightness modulation step.

Parametry

<code>tick_period_ms</code>	Tick period in milliseconds (≥ 1).
-----------------------------	---

Zwraca

Driver status.

Definicja w linii 489 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_now_ms\(\)](#).

12.34.3.28 led_array_timer_pwm_ch_init()

```
void led_array_timer_pwm_ch_init (
    TIM_TypeDef * tim,
    uint8_t channel) [static]
```

Internal helper: [led_array_timer_pwm_ch_init](#).

Definicja w linii 141 pliku [led_array_lib.c](#).

Odwołania w [led_array_hw_init\(\)](#).

12.34.3.29 led_array_timer_tick()

```
void led_array_timer_tick (
    void )
```

Tick function for ISR-driven scheduling.

Call this from SysTick or a hardware timer ISR if you want deterministic timing independent of the main loop.

Definicja w linii 517 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_tick_core\(\)](#).

12.34.3.30 led_array_tri8()

```
uint8_t led_array_tri8 (
    uint8_t phase) [static]
```

Internal helper: [led_array_tri8](#).

Definicja w linii 284 pliku [led_array_lib.c](#).

Odwołania w [led_array_effect_rainbow_update\(\)](#).

12.34.3.31 led_array_write_pin()

```
void led_array_write_pin (
    uint8_t index,
    GPIO_PinState state) [static]
```

Internal helper: [led_array_write_pin](#).

Definicja w linii 258 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_set_hw_duty\(\)](#).

Odwołania w [led_array_deinit\(\)](#).

12.34.4 Dokumentacja zmiennych**12.34.4.1 s_led**

```
led_array_ctx_t s_led [static]
```

Definicja w linii 51 pliku [led_array_lib.c](#).

12.35 led_array_lib.c

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #include "led_array_lib.h"
00007
00008 #include "cmsis_os2.h"
00009
00010 #include <string.h>
00011
00012 #define LED_ARRAY_COUNT          50
```



```

00013 #define LED_ARRAY_VALID_MASK      LED_ARRAY_LED_ALL
00014 #define LED_ARRAY_RAINBOW_PHASE_INC 8U
00015 #define LED_ARRAY_PWM_FREQ_HZ      20000U
00016 #define LED_ARRAY_PWM_STEPS        100U
00017
00018 typedef struct
00019 {
00020     bool initialized;
00021     bool timer_running;
00022     uint32_t tick_period_ms;
00023     uint32_t last_tick_ms;
00024     led_array_effect_t effect;
00025     uint8_t manual_brightness[LED_ARRAY_COUNT];
00026     uint8_t active_brightness[LED_ARRAY_COUNT];
00027     struct
00028     {
00029         uint8_t mask;
00030         uint16_t duration_ms;
00031         uint32_t elapsed_ms;
00032     } fade;
00033     struct
00034     {
00035         uint8_t mask;
00036         uint16_t period_ms;
00037         uint8_t min_pct;
00038         uint8_t max_pct;
00039         uint32_t elapsed_ms;
00040     } breath;
00041     struct
00042     {
00043         uint16_t step_ms;
00044         uint8_t min_pct;
00045         uint8_t max_pct;
00046         uint16_t phase_accum_ms;
00047         uint8_t phase_base;
00048     } rainbow;
00049 } led_array_ctx_t;
00050
00051 static led_array_ctx_t s_led;
00052
00054 static uint32_t led_array_now_ms(void)
00055 {
00056     osKernelState_t state;
00057     uint32_t tick_hz;
00058     uint32_t tick_count;
00059
00060     state = osKernelGetState();
00061     if ((state == osKernelRunning) || (state == osKernelLocked))
00062     {
00063         tick_hz = osKernelGetTickFreq();
00064         tick_count = osKernelGetTickCount();
00065         if (tick_hz == 0U)
00066         {
00067             return tick_count;
00068         }
00069         return (uint32_t)((uint64_t)tick_count * 1000ULL) / (uint64_t)tick_hz;
00070     }
00071
00072     return HAL_GetTick();
00073 }
00074
00076 static uint32_t led_array_get_tim_clock(TIM_TypeDef *tim)
00077 {
00078     uint32_t pclk;
00079     uint32_t timer_clock;
00080     RCC_ClkInitTypeDef clk_cfg;
00081     uint32_t flash_latency;
00082
00083     HAL_RCC_GetClockConfig(&clk_cfg, &flash_latency);
00084
00085     if ((tim == TIM2) || (tim == TIM3))
00086     {
00087         pclk = HAL_RCC_GetPCLK1Freq();
00088         if (clk_cfg.APB1CLKDivider == RCC_HCLK_DIV1)
00089         {
00090             timer_clock = pclk;
00091         }
00092         else
00093         {
00094             timer_clock = pclk * 2U;
00095         }
00096     }
00097     else
00098     {
00099         pclk = HAL_RCC_GetPCLK2Freq();
00100         if (clk_cfg.APB2CLKDivider == RCC_HCLK_DIV1)
00101         {

```

```

00102         timer_clock = pclk;
00103     }
00104     else
00105     {
00106         timer_clock = pclk * 2U;
00107     }
00108 }
00109
00110 return timer_clock;
00111 }
00112
00114 static void led_array_timer_base_init(TIM_TypeDef *tim)
00115 {
00116     uint32_t tim_clk;
00117     uint32_t ticks_per_period;
00118     uint32_t psc;
00119
00120     tim_clk = led_array_get_tim_clock(tim);
00121     ticks_per_period = LED_ARRAY_PWM_FREQ_HZ * LED_ARRAY_PWM_STEPS;
00122     if ((ticks_per_period == 0U) || (tim_clk == 0U))
00123     {
00124         return;
00125     }
00126
00127     psc = tim_clk / ticks_per_period;
00128     if (psc == 0U)
00129     {
00130         psc = 1U;
00131     }
00132
00133     tim->CR1 = 0U;
00134     tim->PSC = (uint16_t)(psc - 1U);
00135     tim->ARR = (uint32_t)(LED_ARRAY_PWM_STEPS - 1U);
00136     tim->EGR = TIM_EGR_UG;
00137     tim->CR1 |= TIM_CR1_ARPE;
00138 }
00139
00141 static void led_array_timer_pwm_ch_init(TIM_TypeDef *tim, uint8_t channel)
00142 {
00143     if (channel == 1U)
00144     {
00145         tim->CCMR1 &= ~(TIM_CCMR1_OC1M_Msk | TIM_CCMR1_CC1S_Msk);
00146         tim->CCMR1 |= (6U << TIM_CCMR1_OC1M_Pos) | TIM_CCMR1_OC1PE;
00147         tim->CCER |= TIM_CCER_CC1E;
00148         tim->CCR1 = 0U;
00149     }
00150     else if (channel == 2U)
00151     {
00152         tim->CCMR1 &= ~(TIM_CCMR1_OC2M_Msk | TIM_CCMR1_CC2S_Msk);
00153         tim->CCMR1 |= (6U << TIM_CCMR1_OC2M_Pos) | TIM_CCMR1_OC2PE;
00154         tim->CCER |= TIM_CCER_CC2E;
00155         tim->CCR2 = 0U;
00156     }
00157     else if (channel == 3U)
00158     {
00159         tim->CCMR2 &= ~(TIM_CCMR2_OC3M_Msk | TIM_CCMR2_CC3S_Msk);
00160         tim->CCMR2 |= (6U << TIM_CCMR2_OC3M_Pos) | TIM_CCMR2_OC3PE;
00161         tim->CCER |= TIM_CCER_CC3E;
00162         tim->CCR3 = 0U;
00163     }
00164 }
00165
00167 static void led_array_hw_init(void)
00168 {
00169     GPIO_InitTypeDef gpio = {0};
00170
00171     __HAL_RCC_GPIOA_CLK_ENABLE();
00172     __HAL_RCC_GPIOB_CLK_ENABLE();
00173     __HAL_RCC_GPIOC_CLK_ENABLE();
00174     __HAL_RCC_TIM1_CLK_ENABLE();
00175     __HAL_RCC_TIM2_CLK_ENABLE();
00176     __HAL_RCC_TIM3_CLK_ENABLE();
00177     __HAL_RCC_TIM8_CLK_ENABLE();
00178
00179     gpio.Mode = GPIO_MODE_AF_PP;
00180     gpio.Pull = GPIO_NOPULL;
00181     gpio.Speed = GPIO_SPEED_FREQ_HIGH;
00182
00183     gpio.Pin = GPIO_PIN_7;
00184     gpio.Alternate = GPIO_AF3_TIM8;
00185     HAL_GPIO_Init(GPIOC, &gpio);
00186
00187     gpio.Pin = GPIO_PIN_8;
00188     gpio.Alternate = GPIO_AF1_TIM1;
00189     HAL_GPIO_Init(GPIOA, &gpio);
00190
00191     gpio.Pin = GPIO_PIN_10;

```

```

00192     gpio.Alternate = GPIO_AF1_TIM2;
00193     HAL_GPIO_Init(GPIOB, &gpio);
00194
00195     gpio.Pin = GPIO_PIN_4 | GPIO_PIN_5;
00196     gpio.Alternate = GPIO_AF2_TIM3;
00197     HAL_GPIO_Init(GPIOB, &gpio);
00198
00199     led_array_timer_base_init(TIM1);
00200     led_array_timer_base_init(TIM2);
00201     led_array_timer_base_init(TIM3);
00202     led_array_timer_base_init(TIM8);
00203
00204     led_array_timer_pwm_ch_init(TIM8, 2U); /* PC7 */
00205     led_array_timer_pwm_ch_init(TIM1, 1U); /* PA8 */
00206     led_array_timer_pwm_ch_init(TIM2, 3U); /* PB10 */
00207     led_array_timer_pwm_ch_init(TIM3, 1U); /* PB4 */
00208     led_array_timer_pwm_ch_init(TIM3, 2U); /* PB5 */
00209
00210     TIM1->BDTR |= TIM_BDTR_MOE;
00211     TIM8->BDTR |= TIM_BDTR_MOE;
00212
00213     TIM1->CR1 |= TIM_CR1_CEN;
00214     TIM2->CR1 |= TIM_CR1_CEN;
00215     TIM3->CR1 |= TIM_CR1_CEN;
00216     TIM8->CR1 |= TIM_CR1_CEN;
00217 }
00218
00220 static void led_array_set_hw_duty(uint8_t index, uint8_t pct)
00221 {
00222     uint32_t ccr;
00223
00224     if (pct > 100U)
00225     {
00226         pct = 100U;
00227     }
00228     ccr = ((uint32_t)pct * (LED_ARRAY_PWM_STEPS - 1U)) / 100U;
00229
00230     switch (index)
00231     {
00232     case 0U: TIM8->CCR2 = ccr; break;
00233     case 1U: TIM1->CCR1 = ccr; break;
00234     case 2U: TIM2->CCR3 = ccr; break;
00235     case 3U: TIM3->CCR1 = ccr; break;
00236     case 4U: TIM3->CCR2 = ccr; break;
00237     default: break;
00238     }
00239 }
00240
00242 static bool led_array_mask_valid(uint8_t mask)
00243 {
00244     if ((mask == 0U) || ((mask & (uint8_t)(~LED_ARRAY_VALID_MASK)) != 0U))
00245     {
00246         return false;
00247     }
00248     return true;
00249 }
00250
00252 static uint8_t led_array_clamp_pct(uint8_t value)
00253 {
00254     return (value > 100U) ? 100U : value;
00255 }
00256
00258 static void led_array_write_pin(uint8_t index, GPIO_PinState state)
00259 {
00260     if (state == GPIO_PIN_SET)
00261     {
00262         led_array_set_hw_duty(index, 100U);
00263     }
00264     else
00265     {
00266         led_array_set_hw_duty(index, 0U);
00267     }
00268 }
00269
00271 static void led_array_apply_outputs(void)
00272 {
00273     uint8_t i;
00274     uint8_t b;
00275
00276     for (i = 0U; i < LED_ARRAY_COUNT; i++)
00277     {
00278         b = s_led.active_brightness[i];
00279         led_array_set_hw_duty(i, b);
00280     }
00281 }
00282
00284 static uint8_t led_array_tri8(uint8_t phase)

```

```

00285 {
00286     if (phase < 128U)
00287     {
00288         return (uint8_t)(phase « 1);
00289     }
00290     return (uint8_t)((255U - phase) « 1);
00291 }
00292
00294 static void led_array_effect_none(void)
00295 {
00296     memcpy(s_led.active_brightness, s_led.manual_brightness, sizeof(s_led.active_brightness));
00297 }
00298
00300 static void led_array_effect_fade_update(uint32_t delta_ms)
00301 {
00302     uint8_t i;
00303     uint32_t remain;
00304     uint32_t level;
00305
00306     if (s_led.fade.duration_ms == 0U)
00307     {
00308         s_led.effect = LED_ARRAY_EFFECT_NONE;
00309         led_array_effect_none();
00310         return;
00311     }
00312
00313     s_led.fade.elapsed_ms += delta_ms;
00314     if (s_led.fade.elapsed_ms >= s_led.fade.duration_ms)
00315     {
00316         for (i = 0U; i < LED_ARRAY_COUNT; i++)
00317         {
00318             if ((s_led.fade.mask & (uint8_t)(1U « i)) != 0U)
00319             {
00320                 s_led.manual_brightness[i] = 0U;
00321                 s_led.active_brightness[i] = 0U;
00322             }
00323             else
00324             {
00325                 s_led.active_brightness[i] = s_led.manual_brightness[i];
00326             }
00327         }
00328
00329         s_led.effect = LED_ARRAY_EFFECT_NONE;
00330         return;
00331     }
00332
00333     remain = (uint32_t)s_led.fade.duration_ms - s_led.fade.elapsed_ms;
00334
00335     for (i = 0U; i < LED_ARRAY_COUNT; i++)
00336     {
00337         if ((s_led.fade.mask & (uint8_t)(1U « i)) != 0U)
00338         {
00339             level = (100U * remain) / (uint32_t)s_led.fade.duration_ms;
00340             s_led.active_brightness[i] = (uint8_t)level;
00341         }
00342         else
00343         {
00344             s_led.active_brightness[i] = s_led.manual_brightness[i];
00345         }
00346     }
00347 }
00348
00350 static void led_array_effect_breath_update(uint32_t delta_ms)
00351 {
00352     uint8_t i;
00353     uint32_t half;
00354     uint32_t phase;
00355     uint32_t tri;
00356     uint32_t range;
00357     uint8_t level;
00358
00359     if (s_led.breath.period_ms < 2U)
00360     {
00361         s_led.breath.period_ms = 2U;
00362     }
00363
00364     s_led.breath.elapsed_ms = (s_led.breath.elapsed_ms + delta_ms) % s_led.breath.period_ms;
00365     half = s_led.breath.period_ms / 2U;
00366     if (half == 0U)
00367     {
00368         half = 1U;
00369     }
00370
00371     if (s_led.breath.elapsed_ms < half)
00372     {
00373         phase = s_led.breath.elapsed_ms;
00374     }

```

```

00375     else
00376     {
00377         phase = (uint32_t)s_led.breath.period_ms - s_led.breath.elapsed_ms;
00378     }
00379
00380     tri = (phase * 100U) / half;
00381     if (tri > 100U)
00382     {
00383         tri = 100U;
00384     }
00385
00386     range = (uint32_t)s_led.breath.max_pct - s_led.breath.min_pct;
00387     level = (uint8_t)(s_led.breath.min_pct + ((range * tri) / 100U));
00388
00389     for (i = 0U; i < LED_ARRAY_COUNT; i++)
00390     {
00391         if ((s_led.breath.mask & (uint8_t)(1U << i)) != 0U)
00392         {
00393             s_led.active_brightness[i] = level;
00394         }
00395         else
00396         {
00397             s_led.active_brightness[i] = s_led.manual_brightness[i];
00398         }
00399     }
00400 }
00401
00403 static void led_array_effect_rainbow_update(uint32_t delta_ms)
00404 {
00405     uint8_t i;
00406     uint8_t phase;
00407     uint8_t tri;
00408     uint32_t range;
00409     uint8_t level;
00410     const uint8_t phase_step = (uint8_t)(256U / LED_ARRAY_COUNT);
00411
00412     if (s_led.rainbow.step_ms == 0U)
00413     {
00414         s_led.rainbow.step_ms = 1U;
00415     }
00416
00417     s_led.rainbow.phase_accum_ms = (uint16_t)(s_led.rainbow.phase_accum_ms + delta_ms);
00418     while (s_led.rainbow.phase_accum_ms >= s_led.rainbow.step_ms)
00419     {
00420         s_led.rainbow.phase_accum_ms = (uint16_t)(s_led.rainbow.phase_accum_ms -
s_led.rainbow.step_ms);
00421         s_led.rainbow.phase_base = (uint8_t)(s_led.rainbow.phase_base + LED_ARRAY_RAINBOW_PHASE_INC);
00422     }
00423
00424     range = (uint32_t)s_led.rainbow.max_pct - s_led.rainbow.min_pct;
00425     for (i = 0U; i < LED_ARRAY_COUNT; i++)
00426     {
00427         phase = (uint8_t)(s_led.rainbow.phase_base + (uint8_t)(phase_step * i));
00428         tri = led_array_tri8(phase);
00429         level = (uint8_t)(s_led.rainbow.min_pct + ((range * tri) / 254U));
00430         s_led.active_brightness[i] = level;
00431     }
00432 }
00433
00435 static void led_array_tick_core(uint32_t delta_ms)
00436 {
00437     switch (s_led.effect)
00438     {
00439     case LED_ARRAY_EFFECT_FADE_OUT:
00440         led_array_effect_fade_update(delta_ms);
00441         break;
00442     case LED_ARRAY_EFFECT_BREATH:
00443         led_array_effect_breath_update(delta_ms);
00444         break;
00445     case LED_ARRAY_EFFECT_RAINBOW:
00446         led_array_effect_rainbow_update(delta_ms);
00447         break;
00448     case LED_ARRAY_EFFECT_NONE:
00449     default:
00450         led_array_effect_none();
00451         break;
00452     }
00453
00454     led_array_apply_outputs();
00455 }
00456
00457 led_array_status_t led_array_init(void)
00458 {
00459     led_array_hw_init();
00460
00461     memset(&s_led, 0, sizeof(s_led));
00462     s_led.tick_period_ms = LED_ARRAY_LIB_DEFAULT_TICK_MS;

```

```
00463     s_led.last_tick_ms = led_array_now_ms();
00464     s_led.timer_running = true;
00465     s_led.initialized = true;
00466     s_led.effect = LED_ARRAY_EFFECT_NONE;
00467
00468     return LED_ARRAY_OK;
00469 }
00470
00471 led_array_status_t led_array_deinit(void)
00472 {
00473     uint8_t i;
00474
00475     if (!s_led.initialized)
00476     {
00477         return LED_ARRAY_ESTATE;
00478     }
00479
00480     for (i = 0U; i < LED_ARRAY_COUNT; i++)
00481     {
00482         led_array_write_pin(i, GPIO_PIN_RESET);
00483     }
00484
00485     memset(&s_led, 0, sizeof(s_led));
00486     return LED_ARRAY_OK;
00487 }
00488
00489 led_array_status_t led_array_timer_init(uint32_t tick_period_ms)
00490 {
00491     if (!s_led.initialized)
00492     {
00493         return LED_ARRAY_ESTATE;
00494     }
00495     if (tick_period_ms == 0U)
00496     {
00497         return LED_ARRAY_EINVAL;
00498     }
00499
00500     s_led.tick_period_ms = tick_period_ms;
00501     s_led.last_tick_ms = led_array_now_ms();
00502     s_led.timer_running = true;
00503     return LED_ARRAY_OK;
00504 }
00505
00506 led_array_status_t led_array_timer_deinit(void)
00507 {
00508     if (!s_led.initialized)
00509     {
00510         return LED_ARRAY_ESTATE;
00511     }
00512
00513     s_led.timer_running = false;
00514     return LED_ARRAY_OK;
00515 }
00516
00517 void led_array_timer_tick(void)
00518 {
00519     if (!s_led.initialized || !s_led.timer_running)
00520     {
00521         return;
00522     }
00523
00524     led_array_tick_core(s_led.tick_period_ms);
00525 }
00526
00527 void led_array_process(void)
00528 {
00529     uint32_t now;
00530
00531     if (!s_led.initialized || !s_led.timer_running)
00532     {
00533         return;
00534     }
00535
00536     now = led_array_now_ms();
00537     while ((now - s_led.last_tick_ms) >= s_led.tick_period_ms)
00538     {
00539         s_led.last_tick_ms += s_led.tick_period_ms;
00540         led_array_tick_core(s_led.tick_period_ms);
00541     }
00542 }
00543
00544 led_array_status_t led_array_on(uint8_t mask)
00545 {
00546     uint8_t i;
00547
00548     if (!s_led.initialized)
00549     {
```

```

00550         return LED_ARRAY_ESTATE;
00551     }
00552     if (!led_array_mask_valid(mask))
00553     {
00554         return LED_ARRAY_EINVAL;
00555     }
00556     s_led.effect = LED_ARRAY_EFFECT_NONE;
00557     for (i = 0U; i < LED_ARRAY_COUNT; i++)
00558     {
00559         if ((mask & (uint8_t)(1U << i)) != 0U)
00560         {
00561             s_led.manual_brightness[i] = 100U;
00562         }
00563     }
00564     return LED_ARRAY_OK;
00565 }
00566
00567 led_array_status_t led_array_off(uint8_t mask)
00568 {
00569     uint8_t i;
00570
00571     if (!s_led.initialized)
00572     {
00573         return LED_ARRAY_ESTATE;
00574     }
00575     if (!led_array_mask_valid(mask))
00576     {
00577         return LED_ARRAY_EINVAL;
00578     }
00579
00580     s_led.effect = LED_ARRAY_EFFECT_NONE;
00581     for (i = 0U; i < LED_ARRAY_COUNT; i++)
00582     {
00583         if ((mask & (uint8_t)(1U << i)) != 0U)
00584         {
00585             s_led.manual_brightness[i] = 0U;
00586         }
00587     }
00588     return LED_ARRAY_OK;
00589 }
00590
00591 led_array_status_t led_array_set_brightness(uint8_t mask, uint8_t brightness_pct)
00592 {
00593     uint8_t i;
00594
00595     if (!s_led.initialized)
00596     {
00597         return LED_ARRAY_ESTATE;
00598     }
00599     if (!led_array_mask_valid(mask))
00600     {
00601         return LED_ARRAY_EINVAL;
00602     }
00603
00604     brightness_pct = led_array_clamp_pct(brightness_pct);
00605     s_led.effect = LED_ARRAY_EFFECT_NONE;
00606     for (i = 0U; i < LED_ARRAY_COUNT; i++)
00607     {
00608         if ((mask & (uint8_t)(1U << i)) != 0U)
00609         {
00610             s_led.manual_brightness[i] = brightness_pct;
00611         }
00612     }
00613     return LED_ARRAY_OK;
00614 }
00615
00616 led_array_status_t led_array_fire_fade_out(uint8_t mask, uint16_t fade_ms)
00617 {
00618     uint8_t i;
00619
00620     if (!s_led.initialized)
00621     {
00622         return LED_ARRAY_ESTATE;
00623     }
00624     if (!led_array_mask_valid(mask) || (fade_ms == 0U))
00625     {
00626         return LED_ARRAY_EINVAL;
00627     }
00628
00629     for (i = 0U; i < LED_ARRAY_COUNT; i++)
00630     {
00631         if ((mask & (uint8_t)(1U << i)) != 0U)
00632         {
00633             s_led.manual_brightness[i] = 100U;
00634             s_led.active_brightness[i] = 100U;
00635         }
00636     }

```

```

00637     }
00638
00639     s_led.effect = LED_ARRAY_EFFECT_FADE_OUT;
00640     s_led.fade.mask = (uint8_t)(mask & LED_ARRAY_VALID_MASK);
00641     s_led.fade.duration_ms = fade_ms;
00642     s_led.fade.elapsed_ms = 0U;
00643     return LED_ARRAY_OK;
00644 }
00645
00646 led_array_status_t led_array_start_breath(uint8_t mask,
00647                                           uint16_t period_ms,
00648                                           uint8_t min_pct,
00649                                           uint8_t max_pct)
00650 {
00651     if (!s_led.initialized)
00652     {
00653         return LED_ARRAY_ESTATE;
00654     }
00655     if (!led_array_mask_valid(mask) || (period_ms < 2U))
00656     {
00657         return LED_ARRAY_EINVAL;
00658     }
00659
00660     min_pct = led_array_clamp_pct(min_pct);
00661     max_pct = led_array_clamp_pct(max_pct);
00662     if (max_pct < min_pct)
00663     {
00664         return LED_ARRAY_EINVAL;
00665     }
00666
00667     s_led.effect = LED_ARRAY_EFFECT_BREATH;
00668     s_led.breath.mask = (uint8_t)(mask & LED_ARRAY_VALID_MASK);
00669     s_led.breath.period_ms = period_ms;
00670     s_led.breath.min_pct = min_pct;
00671     s_led.breath.max_pct = max_pct;
00672     s_led.breath.elapsed_ms = 0U;
00673     return LED_ARRAY_OK;
00674 }
00675
00676 led_array_status_t led_array_start_rainbow(uint16_t step_ms,
00677                                           uint8_t min_pct,
00678                                           uint8_t max_pct)
00679 {
00680     if (!s_led.initialized)
00681     {
00682         return LED_ARRAY_ESTATE;
00683     }
00684     if (step_ms == 0U)
00685     {
00686         return LED_ARRAY_EINVAL;
00687     }
00688
00689     min_pct = led_array_clamp_pct(min_pct);
00690     max_pct = led_array_clamp_pct(max_pct);
00691     if (max_pct < min_pct)
00692     {
00693         return LED_ARRAY_EINVAL;
00694     }
00695
00696     s_led.effect = LED_ARRAY_EFFECT_RAINBOW;
00697     s_led.rainbow.step_ms = step_ms;
00698     s_led.rainbow.min_pct = min_pct;
00699     s_led.rainbow.max_pct = max_pct;
00700     s_led.rainbow.phase_accum_ms = 0U;
00701     s_led.rainbow.phase_base = 0U;
00702     return LED_ARRAY_OK;
00703 }
00704
00705 led_array_status_t led_array_stop_effect(void)
00706 {
00707     if (!s_led.initialized)
00708     {
00709         return LED_ARRAY_ESTATE;
00710     }
00711
00712     s_led.effect = LED_ARRAY_EFFECT_NONE;
00713     return LED_ARRAY_OK;
00714 }
00715
00716 led_array_effect_t led_array_get_effect(void)
00717 {
00718     if (!s_led.initialized)
00719     {
00720         return LED_ARRAY_EFFECT_NONE;
00721     }
00722     return s_led.effect;
00723 }

```



```

00724
00725 bool led_array_is_initialized(void)
00726 {
00727     return s_led.initialized;
00728 }
00729
00730 #if LED_ARRAY_LIB_OWNS_HAL_SYSTICK_CALLBACK
00731 void HAL_SYSTICK_Callback(void)
00732 {
00733     led_array_timer_tick();
00734 }
00735 #endif

```

12.36 Dokumentacja pliku Core/App/led_array_lib/led_array_lib.h

LED array driver for 5 LEDs (PC7, PA8, PB10, PB4, PB5).

```

#include "led_array_lib_config.h"
#include "stm32u5xx_hal.h"
#include <stdbool.h>
#include <stdint.h>

```

Wyliczenia

- enum `led_array_status_t` { `LED_ARRAY_OK` = 0 , `LED_ARRAY_EINVAL` = -1 , `LED_ARRAY_ESTATE` = -2 }

Driver status codes.

- enum `led_array_led_mask_t` {
`LED_ARRAY_LED_1` = (1U << 0) , `LED_ARRAY_LED_2` = (1U << 1) , `LED_ARRAY_LED_3` = (1U << 2)
, `LED_ARRAY_LED_4` = (1U << 3) ,
`LED_ARRAY_LED_5` = (1U << 4) , `LED_ARRAY_LED_ALL` = 0x1FU }

LED mask bits.

- enum `led_array_effect_t` { `LED_ARRAY_EFFECT_NONE` = 0 , `LED_ARRAY_EFFECT_FADE_OUT` , `LED_ARRAY_EFFECT_BREATH` , `LED_ARRAY_EFFECT_RAINBOW` }

Global effect mode.

Funkcje

- `led_array_status_t led_array_init` (void)
Initialize the LED array driver.
- `led_array_status_t led_array_deinit` (void)
Deinitialize the LED array driver and turn off all LEDs.
- `led_array_status_t led_array_timer_init` (uint32_t tick_period_ms)
Configure internal scheduler tick period.
- `led_array_status_t led_array_timer_deinit` (void)
Disable the scheduler updates.
- void `led_array_timer_tick` (void)
Tick function for ISR-driven scheduling.
- void `led_array_process` (void)
Polling scheduler update (HAL_GetTick based).
- `led_array_status_t led_array_on` (uint8_t mask)
Turn LEDs on at 100% brightness.
- `led_array_status_t led_array_off` (uint8_t mask)
Turn LEDs off.
- `led_array_status_t led_array_set_brightness` (uint8_t mask, uint8_t brightness_pct)
Set LED brightness in range 0..100%.
- `led_array_status_t led_array_fire_fade_out` (uint8_t mask, uint16_t fade_ms)
Trigger one-shot fade-out from 100% to 0%.

- `led_array_status_t led_array_start_breath` (uint8_t mask, uint16_t period_ms, uint8_t min_pct, uint8_t max_pct)
Start breathing effect on selected LEDs.
- `led_array_status_t led_array_start_rainbow` (uint16_t step_ms, uint8_t min_pct, uint8_t max_pct)
Start circulating rainbow-like wave on all LEDs.
- `led_array_status_t led_array_stop_effect` (void)
Stop the active effect and keep current manual brightness values.
- `led_array_effect_t led_array_get_effect` (void)
Read currently active global effect.
- `bool led_array_is_initialized` (void)
Read initialization state.

12.36.1 Opis szczegółowy

LED array driver for 5 LEDs (PC7, PA8, PB10, PB4, PB5).
Definicja w pliku `led_array_lib.h`.

12.36.2 Dokumentacja typów wyliczanych

12.36.2.1 led_array_effect_t

enum `led_array_effect_t`

Global effect mode.

Definicja w linii 39 pliku `led_array_lib.h`.

12.36.2.2 led_array_led_mask_t

enum `led_array_led_mask_t`

LED mask bits.

Wartości wyliczeń

<code>LED_ARRAY_LED↔ _1</code>	PC7.
<code>LED_ARRAY_LED↔ _2</code>	PA8.
<code>LED_ARRAY_LED↔ _3</code>	PB10.
<code>LED_ARRAY_LED↔ _4</code>	PB4.
<code>LED_ARRAY_LED↔ _5</code>	PB5.

Definicja w linii 28 pliku `led_array_lib.h`.

12.36.2.3 led_array_status_t

enum `led_array_status_t`

Driver status codes.

Definicja w linii 20 pliku `led_array_lib.h`.

12.36.3 Dokumentacja funkcji

12.36.3.1 led_array_deinit()

```
led_array_status_t led_array_deinit (
    void )
```

Deinitialize the LED array driver and turn off all LEDs.

Zwraca

Driver status.

Definicja w linii 471 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_write_pin\(\)](#).

12.36.3.2 led_array_fire_fade_out()

```
led_array_status_t led_array_fire_fade_out (
    uint8_t mask,
    uint16_t fade_ms)
```

Trigger one-shot fade-out from 100% to 0%.

Parametry

<i>mask</i>	LED mask.
<i>fade_ms</i>	Fade duration in milliseconds.

Zwraca

Driver status.

Definicja w linii 617 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_mask_valid\(\)](#).

12.36.3.3 led_array_get_effect()

```
led_array_effect_t led_array_get_effect (
    void )
```

Read currently active global effect.

Zwraca

Active effect.

Definicja w linii 716 pliku [led_array_lib.c](#).

12.36.3.4 led_array_init()

```
led_array_status_t led_array_init (
    void )
```

Initialize the LED array driver.

Zwraca

Driver status.

Definicja w linii 457 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_hw_init\(\)](#), [LED_ARRAY_LIB_DEFAULT_TICK_MS](#) i [led_array_now_ms\(\)](#).

12.36.3.5 led_array_is_initialized()

```
bool led_array_is_initialized (
    void )
```

Read initialization state.

Zwraca

true if initialized.

Definicja w linii 725 pliku [led_array_lib.c](#).

12.36.3.6 led_array_off()

```
led_array_status_t led_array_off (
    uint8_t mask)
```

Turn LEDs off.

Parametry

<i>mask</i>	LED mask.
-------------	-----------

Zwraca

Driver status.

Definicja w linii 568 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_mask_valid\(\)](#).

12.36.3.7 led_array_on()

```
led_array_status_t led_array_on (
    uint8_t mask)
```

Turn LEDs on at 100% brightness.

Parametry

<i>mask</i>	LED mask.
-------------	-----------

Zwraca

Driver status.

Definicja w linii 544 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_mask_valid\(\)](#).

12.36.3.8 led_array_process()

```
void led_array_process (
    void )
```

Polling scheduler update (HAL_GetTick based).

Call this from the main loop when not using ISR-driven ticks.

Definicja w linii 527 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_now_ms\(\)](#) i [led_array_tick_core\(\)](#).

12.36.3.9 led_array_set_brightness()

```
led_array_status_t led_array_set_brightness (
    uint8_t mask,
    uint8_t brightness_pct)
```

Set LED brightness in range 0..100%.

Parametry

<i>mask</i>	LED mask.
<i>brightness_pct</i>	Brightness in percent.

Zwraca

Driver status.

Definicja w linii 592 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_clamp_pct\(\)](#) i [led_array_mask_valid\(\)](#).

12.36.3.10 led_array_start_breath()

```
led_array_status_t led_array_start_breath (
    uint8_t mask,
    uint16_t period_ms,
    uint8_t min_pct,
    uint8_t max_pct)
```

Start breathing effect on selected LEDs.

Parametry

<i>mask</i>	LED mask.
<i>period_ms</i>	Full breath period (up + down) in milliseconds.
<i>min_pct</i>	Minimum brightness (0..100).
<i>max_pct</i>	Maximum brightness (0..100, must be $\geq$ min_pct).

Zwraca

Driver status.

Definicja w linii 646 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_clamp_pct\(\)](#) i [led_array_mask_valid\(\)](#).

12.36.3.11 led_array_start_rainbow()

```
led_array_status_t led_array_start_rainbow (
    uint16_t step_ms,
    uint8_t min_pct,
    uint8_t max_pct)
```

Start circulating rainbow-like wave on all LEDs.

For a monochrome LED strip this effect is implemented as a moving brightness wave across LED1..LED5.

Parametry

<i>step_ms</i>	Phase step period in milliseconds.
<i>min_pct</i>	Minimum brightness (0..100).
<i>max_pct</i>	Maximum brightness (0..100, must be $\geq$ min_pct).

Zwraca

Driver status.

Definicja w linii 676 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_clamp_pct\(\)](#).

12.36.3.12 led_array_stop_effect()

```
led_array_status_t led_array_stop_effect (
    void )
```

Stop the active effect and keep current manual brightness values.

Zwraca

Driver status.

Definicja w linii 705 pliku [led_array_lib.c](#).

12.36.3.13 led_array_timer_deinit()

```
led_array_status_t led_array_timer_deinit (
    void )
```

Disable the scheduler updates.

Zwraca

Driver status.

Definicja w linii 506 pliku [led_array_lib.c](#).

12.36.3.14 led_array_timer_init()

```
led_array_status_t led_array_timer_init (
    uint32_t tick_period_ms)
```

Configure internal scheduler tick period.

This controls effect update and brightness modulation step.

Parametry

<i>tick_period_ms</i>	Tick period in milliseconds (≥ 1).
-----------------------	---

Zwraca

Driver status.

Definicja w linii 489 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_now_ms\(\)](#).

12.36.3.15 led_array_timer_tick()

```
void led_array_timer_tick (
    void )
```

Tick function for ISR-driven scheduling.

Call this from SysTick or a hardware timer ISR if you want deterministic timing independent of the main loop.

Definicja w linii 517 pliku [led_array_lib.c](#).

Odwołuje się do [led_array_tick_core\(\)](#).

12.37 led_array_lib.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_LED_ARRAY_LIB_LED_ARRAY_LIB_H_
00007 #define APP_LED_ARRAY_LIB_LED_ARRAY_LIB_H_
00008
00009 #include "led_array_lib_config.h"
00010 #include "stm32u5xx_hal.h"
00011
00012 #include <stdbool.h>
00013 #include <stdint.h>
00014
00015 #ifdef __cplusplus
00016 extern "C" {
00017 #endif
00018
00020 typedef enum
00021 {
```

```

00022     LED_ARRAY_OK = 0,
00023     LED_ARRAY_EINVAL = -1,
00024     LED_ARRAY_ESTATE = -2
00025 } led_array_status_t;
00026
00028 typedef enum
00029 {
00030     LED_ARRAY_LED_1 = (1U << 0),
00031     LED_ARRAY_LED_2 = (1U << 1),
00032     LED_ARRAY_LED_3 = (1U << 2),
00033     LED_ARRAY_LED_4 = (1U << 3),
00034     LED_ARRAY_LED_5 = (1U << 4),
00035     LED_ARRAY_LED_ALL = 0x1FU
00036 } led_array_led_mask_t;
00037
00039 typedef enum
00040 {
00041     LED_ARRAY_EFFECT_NONE = 0,
00042     LED_ARRAY_EFFECT_FADE_OUT,
00043     LED_ARRAY_EFFECT_BREATH,
00044     LED_ARRAY_EFFECT_RAINBOW
00045 } led_array_effect_t;
00046
00051 led_array_status_t led_array_init(void);
00052
00057 led_array_status_t led_array_deinit(void);
00058
00067 led_array_status_t led_array_timer_init(uint32_t tick_period_ms);
00068
00073 led_array_status_t led_array_timer_deinit(void);
00074
00081 void led_array_timer_tick(void);
00082
00088 void led_array_process(void);
00089
00095 led_array_status_t led_array_on(uint8_t mask);
00096
00102 led_array_status_t led_array_off(uint8_t mask);
00103
00110 led_array_status_t led_array_set_brightness(uint8_t mask, uint8_t brightness_pct);
00111
00118 led_array_status_t led_array_fire_fade_out(uint8_t mask, uint16_t fade_ms);
00119
00128 led_array_status_t led_array_start_breath(uint8_t mask,
00129                                           uint16_t period_ms,
00130                                           uint8_t min_pct,
00131                                           uint8_t max_pct);
00132
00144 led_array_status_t led_array_start_rainbow(uint16_t step_ms,
00145                                           uint8_t min_pct,
00146                                           uint8_t max_pct);
00147
00152 led_array_status_t led_array_stop_effect(void);
00153
00158 led_array_effect_t led_array_get_effect(void);
00159
00164 bool led_array_is_initialized(void);
00165
00166 #ifdef __cplusplus
00167 }
00168 #endif
00169
00170 #endif /* APP_LED_ARRAY_LIB_LED_ARRAY_LIB_H_ */
00171

```

12.38 Dokumentacja pliku Core/App/led_array_lib/led_array_lib_config.h

Compile-time configuration for led_array_lib.

Definicje

- `#define LED_ARRAY_LIB_DEFAULT_TICK_MS 1U`
Default scheduler tick period in milliseconds.
- `#define LED_ARRAY_LIB_OWNS_HAL_SYSTICK_CALLBACK 0`
Own HAL_SYSTICK_Callback in the library.

12.38.1 Opis szczegółowy

Compile-time configuration for `led_array_lib`.

Definicja w pliku [led_array_lib_config.h](#).

12.38.2 Dokumentacja definicji

12.38.2.1 LED_ARRAY_LIB_DEFAULT_TICK_MS

```
#define LED_ARRAY_LIB_DEFAULT_TICK_MS 1U
```

Default scheduler tick period in milliseconds.

For smooth dimming effects keep this at 1 ms.

Definicja w linii 15 pliku [led_array_lib_config.h](#).

Odwołania w [led_array_init\(\)](#).

12.38.2.2 LED_ARRAY_LIB_OWNS_HAL_SYSTICK_CALLBACK

```
#define LED_ARRAY_LIB_OWNS_HAL_SYSTICK_CALLBACK 0
```

Own `HAL_SYSTICK_Callback` in the library.

If set to 1, the library provides `HAL_SYSTICK_Callback()` and calls [led_array_timer_tick\(\)](#) every SysTick interrupt.

Keep set to 0 if your project already defines `HAL_SYSTICK_Callback`.

Definicja w linii 27 pliku [led_array_lib_config.h](#).

12.39 led_array_lib_config.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_LED_ARRAY_LIB_LED_ARRAY_LIB_CONFIG_H_
00007 #define APP_LED_ARRAY_LIB_LED_ARRAY_LIB_CONFIG_H_
00008
00014 #ifndef LED_ARRAY_LIB_DEFAULT_TICK_MS
00015 #define LED_ARRAY_LIB_DEFAULT_TICK_MS 1U
00016 #endif
00017
00026 #ifndef LED_ARRAY_LIB_OWNS_HAL_SYSTICK_CALLBACK
00027 #define LED_ARRAY_LIB_OWNS_HAL_SYSTICK_CALLBACK 0
00028 #endif
00029
00030 #endif /* APP_LED_ARRAY_LIB_LED_ARRAY_LIB_CONFIG_H_ */
00031
```

12.40 led_array_main.c

```
00001 #include "led_array_main.h"
00002
00003 #include "cmsis_os2.h"
00004 #include "FreeRTOS.h"
00005 #include "task.h"
00006 #include "led_array_lib/led_array_lib.h"
00007
00008 #include <stdio.h>
00009
00010 static osThreadId_t s_led_array_task = NULL;
00011 static StaticTask_t s_led_array_task_cb;
00012 #define LED_TASK_STACK_SIZE 2048U
00013 #define LED_TASK_STACK_WORDS (LED_TASK_STACK_SIZE / sizeof(StackType_t))
00014 static StackType_t s_led_array_task_stack[LED_TASK_STACK_WORDS];
00015
00016 static void led_array_main_task_fn(void *argument);
00017
00018 static const osThreadAttr_t s_led_array_task_attr =
00019 {
00020     .name = "led_task",
00021     .priority = (osPriority_t)osPriorityBelowNormal,
00022     .stack_mem = s_led_array_task_stack,
00023     .stack_size = sizeof(s_led_array_task_stack),
00024     .cb_mem = &s_led_array_task_cb,
00025     .cb_size = sizeof(s_led_array_task_cb)
00026 };
00027
```



```

00028 void led_array_main_create_task(void)
00029 {
00030     if (s_led_array_task == NULL)
00031     {
00032         s_led_array_task = osThreadNew(led_array_main_task_fn, NULL, &s_led_array_task_attr);
00033         if (s_led_array_task == NULL)
00034         {
00035             printf("LED task: create failed (heap/stack)\r\n");
00036         }
00037         else
00038         {
00039             printf("LED task: created\r\n");
00040         }
00041     }
00042 }
00043
00045 static void led_array_main_task_fn(void *argument)
00046 {
00047     (void)argument;
00048
00049     led_array_init();
00050     led_array_timer_init(10U);
00051     led_array_start_rainbow(15U, 5U, 100U);
00052     printf("LED task: running rainbow\r\n");
00053
00054     for (;;)
00055     {
00056         led_array_process();
00057         osDelay(5U);
00058     }
00059 }

```

12.41 Dokumentacja pliku Core/App/led_array_main.h

RTOS task facade for LED array status effects.

Funkcje

- void `led_array_main_create_task` (void)
Create the LED array demo/status task.

12.41.1 Opis szczegółowy

RTOS task facade for LED array status effects.

Definicja w pliku `led_array_main.h`.

12.41.2 Dokumentacja funkcji

12.41.2.1 `led_array_main_create_task()`

```
void led_array_main_create_task (
    void )
```

Create the LED array demo/status task.

Definicja w linii 28 pliku `led_array_main.c`.

Odwwołania w `app_freertos_init()`.

12.42 `led_array_main.h`

[Idź do dokumentacji tego pliku.](#)

```

00001
00005
00006 #ifndef APP_LED_ARRAY_MAIN_H_
00007 #define APP_LED_ARRAY_MAIN_H_
00008
00012 void led_array_main_create_task(void);
00013
00014 #endif /* APP_LED_ARRAY_MAIN_H_ */

```

12.43 menu_main.c

```

00001 #include "menu_main.h"
00002
00003 #include "laviet_frame.h"
00004 #include "bmp280_main.h"
00005 #include "button_main.h"
00006 #include "cmsis_os2.h"
00007 #include "FreeRTOS.h"
00008 #include "lcd_main.h"
00009 #include "led_array_lib/led_array_lib.h"
00010 #include "radio_main.h"
00011 #include "radio_lib/radio_lib.h"
00012 #include "security_main.h"
00013 #include "task.h"
00014 #include "tof_main.h"
00015
00016 #include <stdio.h>
00017 #include <string.h>
00018
00019 #define MENU_TASK_STACK_SIZE 8192U
00020 #define MENU_TASK_STACK_WORDS (MENU_TASK_STACK_SIZE / sizeof(StackType_t))
00021 #define MENU_NOTIFY_QUEUE_DEPTH 64U
00022 #define MENU_INACTIVITY_TIMEOUT_MS 60000UL
00023 #define MENU_DISPLAY_ROWS 4U
00024 #define MENU_ITEMS_VISIBLE 3U
00025 #define MENU_LINE_CHARS 20U
00026 #define MENU_LINE_BUF_SIZE (MENU_LINE_CHARS + 1U)
00027 #define MENU_SETTINGS_PIN_LEN 4U
00028 #define MENU_SETTINGS_PIN_UNLOCK_MS 60000UL
00029 #define MENU_PAIRING_WINDOW_MS 300000UL
00030 #define MENU_TRUSTED_DEVICE_SLOTS 16U
00031 #define MENU_TRUSTED_NODE_SLOTS 15U
00032 #define MENU_DEVICE_SLOT_INVALID 0xFFU
00033
00034 typedef enum
00035 {
00036     MENU_PAGE_NONE = 0,
00037     MENU_PAGE_PAGER,
00038     MENU_PAGE_RADIO_SETTINGS,
00039     MENU_PAGE_SEND_OPTIONS,
00040     MENU_PAGE_SEND_DIRECT_LIST,
00041     MENU_PAGE_SEND_TARGET_LIST,
00042     MENU_PAGE_MSG_GROUPS,
00043     MENU_PAGE_GROUP_ALERT,
00044     MENU_PAGE_GROUP_STATUS,
00045     MENU_PAGE_GROUP_SERVICE,
00046     MENU_PAGE_GROUP_QUICK,
00047     MENU_PAGE_MAIN,
00048     MENU_PAGE_DEVICES,
00049     MENU_PAGE_DEVICE_DELETE_LIST,
00050     MENU_PAGE_DEVICE_DELETE_ACTION,
00051     MENU_PAGE_PIN_SETTINGS,
00052     MENU_PAGE_SECURITY,
00053     MENU_PAGE_SECURITY_FH,
00054     MENU_PAGE_SECURITY_CODING,
00055     MENU_PAGE_SECURITY_NOTIFY,
00056     MENU_PAGE_SECURITY_AUTOPING,
00057     MENU_PAGE_HARDWARE,
00058     MENU_PAGE_HARDWARE_LED,
00059     MENU_PAGE_MODULATION,
00060     MENU_PAGE_MOD_LORA,
00061     MENU_PAGE_MOD_LORA_FREQ,
00062     MENU_PAGE_MOD_LORA_BW,
00063     MENU_PAGE_MOD_LORA_SF,
00064     MENU_PAGE_MOD_LORA_CR,
00065     MENU_PAGE_MOD_LORA_POWER,
00066     MENU_PAGE_MOD_LORA_CRC,
00067     MENU_PAGE_MOD_LORA_PREAMBLE,
00068     MENU_PAGE_MOD_LORA_HEADER,
00069     MENU_PAGE_MOD_LORA_IQ,
00070     MENU_PAGE_MOD_LORA_SYNC,
00071     MENU_PAGE_MOD_FSK,
00072     MENU_PAGE_MOD_FSK_SHAPING,
00073     MENU_PAGE_MOD_FSK_FREQ,
00074     MENU_PAGE_MOD_FSK_BITRATE,
00075     MENU_PAGE_MOD_FSK_BW,
00076     MENU_PAGE_MOD_FSK_FILTER,
00077     MENU_PAGE_MOD_FSK_POWER,
00078     MENU_PAGE_MOD_FSK_PREAMBLE,
00079     MENU_PAGE_MOD_FSK_SYNC_LEN,
00080     MENU_PAGE_MOD_FSK_SYNC_WORD,
00081     MENU_PAGE_MOD_FSK_ADDR,
00082     MENU_PAGE_MOD_FSK_CRC,
00083     MENU_PAGE_MOD_FSK_WHITEN,
00084     MENU_PAGE_MOD_OOK,

```

```
00085     MENU_PAGE_MOD_OOK_FREQ,
00086     MENU_PAGE_MOD_OOK_BITRATE,
00087     MENU_PAGE_MOD_OOK_POWER,
00088     MENU_PAGE_MOD_OOK_BW,
00089     MENU_PAGE_MOD_OOK_PREAMBLE,
00090     MENU_PAGE_MOD_OOK_SYNC_LEN,
00091     MENU_PAGE_MOD_OOK_SYNC_WORD,
00092     MENU_PAGE_MOD_OOK_THRESH_TYPE,
00093     MENU_PAGE_MOD_OOK_THRESH_VALUE,
00094     MENU_PAGE_INFO
00095 } menu_page_id_t;
00096
00097 typedef enum
00098 {
00099     MENU_ACTION_NONE = 0,
00100     MENU_ACTION_BACK,
00101     MENU_ACTION_EXIT_TO_MONITOR,
00102     MENU_ACTION_SEND_DEFAULT,
00103     MENU_ACTION_SEND_DIRECT_DEFAULT,
00104     MENU_ACTION_SEND_ALERT_FIRE,
00105     MENU_ACTION_SEND_ALERT_INTR,
00106     MENU_ACTION_SEND_ALERT_LOWBATT,
00107     MENU_ACTION_SEND_STATUS_OK,
00108     MENU_ACTION_SEND_STATUS_BUSY,
00109     MENU_ACTION_SEND_STATUS_IDLE,
00110     MENU_ACTION_SEND_SERVICE_PING,
00111     MENU_ACTION_SEND_SERVICE_RESET,
00112     MENU_ACTION_SEND_SERVICE_SYNC,
00113     MENU_ACTION_SEND_ASK_DONE,
00114     MENU_ACTION_SEND_ACT_COME_OVER,
00115     MENU_ACTION_SEND_ACT_STOP,
00116     MENU_ACTION_SEND_ASK_READY,
00117     MENU_ACTION_DEVICE_ADD,
00118     MENU_ACTION_DEVICE_ADD_NETWORK,
00119     MENU_ACTION_DEVICE_DELETE,
00120     MENU_ACTION_DEVICE_DELETE_CONFIRM,
00121     MENU_ACTION_DEVICE_INFO,
00122     MENU_ACTION_PIN_TOGGLE,
00123     MENU_ACTION_PIN_SET_OPERATOR,
00124     MENU_ACTION_PIN_SET_ADMIN,
00125     MENU_ACTION_SEC_TOGGLE_FH,
00126     MENU_ACTION_SEC_FH_ENABLE,
00127     MENU_ACTION_SEC_FH_DISABLE,
00128     MENU_ACTION_SEC_FH_PERIOD_1000,
00129     MENU_ACTION_SEC_FH_PERIOD_2000,
00130     MENU_ACTION_SEC_FH_PERIOD_5000,
00131     MENU_ACTION_SEC_FH_PERIOD_10000,
00132     MENU_ACTION_SEC_TPM_INFO,
00133     MENU_ACTION_SEC_ROTATE_KEYS,
00134     MENU_ACTION_SEC_CODING_ON,
00135     MENU_ACTION_SEC_CODING_OFF,
00136     MENU_ACTION_SEC_NOTIFY_POPUP,
00137     MENU_ACTION_SEC_NOTIFY_BADGE,
00138     MENU_ACTION_SEC_AUTOPING_ON,
00139     MENU_ACTION_SEC_AUTOPING_OFF,
00140     MENU_ACTION_SEC_AUTOPING_PERIOD_1,
00141     MENU_ACTION_SEC_AUTOPING_PERIOD_10,
00142     MENU_ACTION_SEC_AUTOPING_PERIOD_100,
00143     MENU_ACTION_SEC_AUTOPING_PERIOD_1000,
00144     MENU_ACTION_SEC_AUTOPING_PERIOD_10000,
00145     MENU_ACTION_SEC_AUTOPING_FRAME,
00146     MENU_ACTION_SEC_AUTOPING_RAW,
00147     MENU_ACTION_SEC_TOGGLE_CODING,
00148     MENU_ACTION_SEC_TOGGLE_NOTIFY_MODE,
00149     MENU_ACTION_HW_MEASURE_DIST,
00151     MENU_ACTION_HW_MEASURE_TEMP,
00152     MENU_ACTION_HW_MEASURE_PRESS,
00153     MENU_ACTION_HW_LED_RAINBOW,
00154     MENU_ACTION_HW_LED_BREATH,
00155     MENU_ACTION_HW_LED_OFF,
00156     MENU_ACTION_HW_LED_MODE,
00157     MENU_ACTION_HW_RADIO_RESET,
00158     MENU_ACTION_MOD_LORA_ENABLE,
00159     MENU_ACTION_MOD_LORA_STD,
00160     MENU_ACTION_MOD_LORA_RANGE,
00161     MENU_ACTION_MOD_LORA_FAST,
00162     MENU_ACTION_MOD_LORA_FREQ_8680,
00163     MENU_ACTION_MOD_LORA_FREQ_8681,
00164     MENU_ACTION_MOD_LORA_FREQ_8683,
00165     MENU_ACTION_MOD_LORA_FREQ_8685,
00166     MENU_ACTION_MOD_LORA_FREQ_8688,
00167     MENU_ACTION_MOD_LORA_FREQ_86905,
00168     MENU_ACTION_MOD_LORA_FREQ_869525,
00169     MENU_ACTION_MOD_LORA_BW_7_8,
00170     MENU_ACTION_MOD_LORA_BW_10_4,
00171     MENU_ACTION_MOD_LORA_BW_15_6,
```

```
00172 MENU_ACTION_MOD_LORA_BW_20_8,
00173 MENU_ACTION_MOD_LORA_BW_31_25,
00174 MENU_ACTION_MOD_LORA_BW_41_7,
00175 MENU_ACTION_MOD_LORA_BW_62_5,
00176 MENU_ACTION_MOD_LORA_BW_125,
00177 MENU_ACTION_MOD_LORA_BW_250,
00178 MENU_ACTION_MOD_LORA_BW_500,
00179 MENU_ACTION_MOD_LORA_SF_6,
00180 MENU_ACTION_MOD_LORA_SF_7,
00181 MENU_ACTION_MOD_LORA_SF_8,
00182 MENU_ACTION_MOD_LORA_SF_9,
00183 MENU_ACTION_MOD_LORA_SF_10,
00184 MENU_ACTION_MOD_LORA_SF_11,
00185 MENU_ACTION_MOD_LORA_SF_12,
00186 MENU_ACTION_MOD_LORA_CR_45,
00187 MENU_ACTION_MOD_LORA_CR_46,
00188 MENU_ACTION_MOD_LORA_CR_47,
00189 MENU_ACTION_MOD_LORA_CR_48,
00190 MENU_ACTION_MOD_LORA_PWR_2,
00191 MENU_ACTION_MOD_LORA_PWR_5,
00192 MENU_ACTION_MOD_LORA_PWR_8,
00193 MENU_ACTION_MOD_LORA_PWR_11,
00194 MENU_ACTION_MOD_LORA_PWR_14,
00195 MENU_ACTION_MOD_LORA_PWR_17,
00196 MENU_ACTION_MOD_LORA_PWR_20,
00197 MENU_ACTION_MOD_LORA_CRC_OFF,
00198 MENU_ACTION_MOD_LORA_CRC_ON,
00199 MENU_ACTION_MOD_LORA_PREAMBLE_6,
00200 MENU_ACTION_MOD_LORA_PREAMBLE_8,
00201 MENU_ACTION_MOD_LORA_PREAMBLE_12,
00202 MENU_ACTION_MOD_LORA_PREAMBLE_16,
00203 MENU_ACTION_MOD_LORA_PREAMBLE_24,
00204 MENU_ACTION_MOD_LORA_PREAMBLE_32,
00205 MENU_ACTION_MOD_LORA_HEADER_EXPLICIT,
00206 MENU_ACTION_MOD_LORA_HEADER_IMPLICIT,
00207 MENU_ACTION_MOD_LORA_IQ_NORMAL,
00208 MENU_ACTION_MOD_LORA_IQ_INVERT,
00209 MENU_ACTION_MOD_LORA_SYNC_12,
00210 MENU_ACTION_MOD_LORA_SYNC_34,
00211 MENU_ACTION_MOD_LORA_SYNC_56,
00212 MENU_ACTION_MOD_LORA_SYNC_A5,
00213 MENU_ACTION_MOD_LORA_RESET,
00214 MENU_ACTION_MOD_FSK_ENABLE,
00215 MENU_ACTION_MOD_FSK_SHAPING_FSK,
00216 MENU_ACTION_MOD_FSK_SHAPING_GFSK,
00217 MENU_ACTION_MOD_FSK_SHAPING_MSK,
00218 MENU_ACTION_MOD_FSK_SHAPING_GMSK,
00219 MENU_ACTION_MOD_FSK_FREQ_8680,
00220 MENU_ACTION_MOD_FSK_FREQ_8681,
00221 MENU_ACTION_MOD_FSK_FREQ_8683,
00222 MENU_ACTION_MOD_FSK_FREQ_8685,
00223 MENU_ACTION_MOD_FSK_FREQ_8688,
00224 MENU_ACTION_MOD_FSK_FREQ_86905,
00225 MENU_ACTION_MOD_FSK_FREQ_869525,
00226 MENU_ACTION_MOD_FSK_BITRATE_1200,
00227 MENU_ACTION_MOD_FSK_BITRATE_2400,
00228 MENU_ACTION_MOD_FSK_BITRATE_4800,
00229 MENU_ACTION_MOD_FSK_BITRATE_9600,
00230 MENU_ACTION_MOD_FSK_BITRATE_19200,
00231 MENU_ACTION_MOD_FSK_BITRATE_38400,
00232 MENU_ACTION_MOD_FSK_BITRATE_50000,
00233 MENU_ACTION_MOD_FSK_BITRATE_100000,
00234 MENU_ACTION_MOD_FSK_BW_31_25,
00235 MENU_ACTION_MOD_FSK_BW_62_5,
00236 MENU_ACTION_MOD_FSK_BW_125,
00237 MENU_ACTION_MOD_FSK_BW_250,
00238 MENU_ACTION_MOD_FSK_BW_500,
00239 MENU_ACTION_MOD_FSK_FILTER_NONE,
00240 MENU_ACTION_MOD_FSK_FILTER_BT10,
00241 MENU_ACTION_MOD_FSK_FILTER_BT07,
00242 MENU_ACTION_MOD_FSK_FILTER_BT05,
00243 MENU_ACTION_MOD_FSK_FILTER_BT03,
00244 MENU_ACTION_MOD_FSK_PWR_2,
00245 MENU_ACTION_MOD_FSK_PWR_5,
00246 MENU_ACTION_MOD_FSK_PWR_8,
00247 MENU_ACTION_MOD_FSK_PWR_11,
00248 MENU_ACTION_MOD_FSK_PWR_14,
00249 MENU_ACTION_MOD_FSK_PWR_17,
00250 MENU_ACTION_MOD_FSK_PWR_20,
00251 MENU_ACTION_MOD_FSK_PREAMBLE_4,
00252 MENU_ACTION_MOD_FSK_PREAMBLE_8,
00253 MENU_ACTION_MOD_FSK_PREAMBLE_12,
00254 MENU_ACTION_MOD_FSK_PREAMBLE_16,
00255 MENU_ACTION_MOD_FSK_PREAMBLE_24,
00256 MENU_ACTION_MOD_FSK_PREAMBLE_32,
00257 MENU_ACTION_MOD_FSK_SYNC_LEN_0,
00258 MENU_ACTION_MOD_FSK_SYNC_LEN_1,
```

```

00259     MENU_ACTION_MOD_FSK_SYNC_LEN_2,
00260     MENU_ACTION_MOD_FSK_SYNC_LEN_3,
00261     MENU_ACTION_MOD_FSK_SYNC_LEN_4,
00262     MENU_ACTION_MOD_FSK_SYNC_WORD_55AA,
00263     MENU_ACTION_MOD_FSK_SYNC_WORD_2DD4,
00264     MENU_ACTION_MOD_FSK_SYNC_WORD_A55A,
00265     MENU_ACTION_MOD_FSK_SYNC_WORD_C194C1,
00266     MENU_ACTION_MOD_FSK_SYNC_WORD_1ACFFC1D,
00267     MENU_ACTION_MOD_FSK_ADDR_NONE,
00268     MENU_ACTION_MOD_FSK_ADDR_NODE,
00269     MENU_ACTION_MOD_FSK_ADDR_NODE_BC,
00270     MENU_ACTION_MOD_FSK_CRC_OFF,
00271     MENU_ACTION_MOD_FSK_CRC_IBM,
00272     MENU_ACTION_MOD_FSK_CRC_CCITT,
00273     MENU_ACTION_MOD_FSK_WHITEN_OFF,
00274     MENU_ACTION_MOD_FSK_WHITEN_ON,
00275     MENU_ACTION_MOD_FSK_RESET,
00276     MENU_ACTION_MOD_OOK_ENABLE,
00277     MENU_ACTION_MOD_OOK_FREQ_8680,
00278     MENU_ACTION_MOD_OOK_FREQ_8681,
00279     MENU_ACTION_MOD_OOK_FREQ_8683,
00280     MENU_ACTION_MOD_OOK_FREQ_8685,
00281     MENU_ACTION_MOD_OOK_FREQ_8688,
00282     MENU_ACTION_MOD_OOK_FREQ_86905,
00283     MENU_ACTION_MOD_OOK_FREQ_869525,
00284     MENU_ACTION_MOD_OOK_BITRATE_1200,
00285     MENU_ACTION_MOD_OOK_BITRATE_2400,
00286     MENU_ACTION_MOD_OOK_BITRATE_4800,
00287     MENU_ACTION_MOD_OOK_BITRATE_9600,
00288     MENU_ACTION_MOD_OOK_BITRATE_19200,
00289     MENU_ACTION_MOD_OOK_BITRATE_38400,
00290     MENU_ACTION_MOD_OOK_BITRATE_50000,
00291     MENU_ACTION_MOD_OOK_BITRATE_100000,
00292     MENU_ACTION_MOD_OOK_PWR_2,
00293     MENU_ACTION_MOD_OOK_PWR_5,
00294     MENU_ACTION_MOD_OOK_PWR_8,
00295     MENU_ACTION_MOD_OOK_PWR_11,
00296     MENU_ACTION_MOD_OOK_PWR_14,
00297     MENU_ACTION_MOD_OOK_PWR_17,
00298     MENU_ACTION_MOD_OOK_PWR_20,
00299     MENU_ACTION_MOD_OOK_BW_31_25,
00300     MENU_ACTION_MOD_OOK_BW_62_5,
00301     MENU_ACTION_MOD_OOK_BW_125,
00302     MENU_ACTION_MOD_OOK_BW_250,
00303     MENU_ACTION_MOD_OOK_BW_500,
00304     MENU_ACTION_MOD_OOK_PREAMBLE_4,
00305     MENU_ACTION_MOD_OOK_PREAMBLE_8,
00306     MENU_ACTION_MOD_OOK_PREAMBLE_12,
00307     MENU_ACTION_MOD_OOK_PREAMBLE_16,
00308     MENU_ACTION_MOD_OOK_PREAMBLE_24,
00309     MENU_ACTION_MOD_OOK_PREAMBLE_32,
00310     MENU_ACTION_MOD_OOK_SYNC_LEN_0,
00311     MENU_ACTION_MOD_OOK_SYNC_LEN_1,
00312     MENU_ACTION_MOD_OOK_SYNC_LEN_2,
00313     MENU_ACTION_MOD_OOK_SYNC_LEN_3,
00314     MENU_ACTION_MOD_OOK_SYNC_LEN_4,
00315     MENU_ACTION_MOD_OOK_SYNC_WORD_55AA,
00316     MENU_ACTION_MOD_OOK_SYNC_WORD_2DD4,
00317     MENU_ACTION_MOD_OOK_SYNC_WORD_A55A,
00318     MENU_ACTION_MOD_OOK_SYNC_WORD_C194C1,
00319     MENU_ACTION_MOD_OOK_SYNC_WORD_1ACFFC1D,
00320     MENU_ACTION_MOD_OOK_THRESH_FIXED,
00321     MENU_ACTION_MOD_OOK_THRESH_PEAK,
00322     MENU_ACTION_MOD_OOK_THRESH_AVG,
00323     MENU_ACTION_MOD_OOK_THRESH_4,
00324     MENU_ACTION_MOD_OOK_THRESH_8,
00325     MENU_ACTION_MOD_OOK_THRESH_12,
00326     MENU_ACTION_MOD_OOK_THRESH_16,
00327     MENU_ACTION_MOD_OOK_THRESH_24,
00328     MENU_ACTION_MOD_OOK_THRESH_32,
00329     MENU_ACTION_MOD_OOK_THRESH_48,
00330     MENU_ACTION_MOD_OOK_THRESH_64,
00331     MENU_ACTION_MOD_OOK_RESET,
00332     MENU_ACTION_MOD_FSK,
00333     MENU_ACTION_MOD_OOK,
00334     MENU_ACTION_INFO_SHOW
00335 } menu_action_t;
00336
00337 typedef struct
00338 {
00339     const char *label;
00340     menu_page_id_t child_page;
00341     menu_action_t action;
00342 } menu_item_t;
00343
00344 typedef struct
00345 {

```

```
00346     const char *title;
00347     menu_page_id_t parent;
00348     const menu_item_t *items;
00349     uint8_t item_count;
00350 } menu_page_t;
00351
00352 typedef enum
00353 {
00354     MENU_MODAL_NONE = 0,
00355     MENU_MODAL_INFO,
00356     MENU_MODAL_PAIR_REQUEST,
00357     MENU_MODAL_PAIR_SETUP,
00358     MENU_MODAL_SEND_CONFIRM,
00359     MENU_MODAL_QUICK_REPLY,
00360     MENU_MODAL_PIN
00361 } menu_modal_t;
00362
00363 typedef enum
00364 {
00365     MENU_AUTH_NONE = 0,
00366     MENU_AUTH_OPERATOR,
00367     MENU_AUTH_ADMIN
00368 } menu_auth_level_t;
00369
00370 typedef struct
00371 {
00372     menu_page_id_t current_page;
00373     uint8_t selected_idx;
00374     uint8_t led_mode;
00375     uint8_t selected_device_slot;
00376     uint32_t pending_target_node_id;
00377     menu_action_t send_target_action;
00378     menu_page_id_t transient_parent_page;
00379     bool pairing_network_mode;
00380     uint32_t quick_reply_src_id;
00381     uint32_t quick_reply_deadline_ms;
00382     uint32_t quick_reply_last_seconds;
00383     char quick_reply_text[MENU_LINE_BUF_SIZE];
00384     bool tx_ack_waiting;
00385     char tx_sent_text[MENU_LINE_BUF_SIZE];
00386     menu_page_id_t pending_auth_page;
00387     menu_auth_level_t pending_auth_level;
00388     uint8_t pin_digits[MENU_SETTINGS_PIN_LEN];
00389     uint8_t pin_index;
00390     uint32_t operator_unlock_until_ms;
00391     uint32_t admin_unlock_until_ms;
00392     bool popup_enabled;
00393     uint32_t last_input_ms;
00394     menu_modal_t modal;
00395     menu_action_t pending_action;
00396 } menu_state_t;
00397
00398 typedef struct
00399 {
00400     menu_action_t action;
00401     bool set_modulation;
00402     radio_main_modulation_t modulation;
00403     bool set_option;
00404     radio_main_option_t option;
00405     uint32_t value;
00406     bool persist_lora_preset;
00407     const char *ok_text;
00408     const char *err_text;
00409 } menu_radio_action_binding_t;
00410
00411 static osThreadId_t s_menu_task = NULL;
00412 static osMessageQueueId_t s_menu_notify_queue = NULL;
00413 static StaticTask_t s_menu_task_cb;
00414 static StackType_t s_menu_task_stack[MENU_TASK_STACK_WORDS];
00415 static uint8_t s_operator_pin[MENU_SETTINGS_PIN_LEN] = { 0U, 0U, 0U, 0U };
00416 static uint8_t s_admin_pin[MENU_SETTINGS_PIN_LEN] = { 9U, 9U, 9U, 9U };
00417 static bool s_pin_enabled = true;
00418
00419 static void menu_main_task_fn(void *argument);
00420 static const menu_page_t *menu_get_page(menu_page_id_t page_id);
00421 static void menu_render(menu_state_t *st);
00422 static void menu_render_popup(const char *l0, const char *l1, const char *l2, const char *l3);
00423 static void menu_render_quick_reply(menu_state_t *st);
00424 static void menu_render_pin(menu_state_t *st);
00425 static void menu_enter_monitor(menu_state_t *st);
00426 static void menu_open_page(menu_state_t *st, menu_page_id_t page_id);
00427 static void menu_open_info_modal(menu_state_t *st,
00428                                 const char *l0,
00429                                 const char *l1,
00430                                 const char *l2,
00431                                 const char *l3);
00432 static void menu_close_modal(menu_state_t *st);
```

```

00433 static void menu_clear_tx_ack_state(menu_state_t *st);
00434 static void menu_handle_button(menu_state_t *st, button_event_t evt);
00435 static void menu_handle_modal_button(menu_state_t *st, button_event_t evt);
00436 static void menu_handle_pin_button(menu_state_t *st, button_event_t evt);
00437 static void menu_handle_notification(menu_state_t *st, const menu_notification_t *n);
00438 static void menu_execute_action(menu_state_t *st, menu_action_t action);
00439 static void menu_show_action_result(menu_state_t *st, menu_notification_type_t type, const char
*text);
00440 static void menu_show_ok_or_error(menu_state_t *st, bool ok, const char *ok_text, const char
*err_text);
00441 static void menu_show_send_result(menu_state_t *st, bool ok, const char *sent_text);
00442 static bool menu_execute_radio_action(menu_state_t *st, menu_action_t action);
00443 static bool menu_is_send_action(menu_action_t action);
00444 static bool menu_is_send_target_allowed(uint32_t node_id);
00445 static bool menu_item_is_selectable(const menu_state_t *st, const menu_page_t *page, uint8_t
item_idx);
00446 static void menu_open_send_prompt(menu_state_t *st, menu_action_t action, const char *label);
00447 static void menu_open_send_target_page(menu_state_t *st, menu_action_t action);
00448 static bool menu_start_pairing_session(menu_state_t *st, bool send_pair_req, bool network_mode);
00449 static void menu_open_device_delete_action(menu_state_t *st, uint8_t slot);
00450 static bool menu_get_trusted_slot(uint8_t slot, trusted_info_t *info_out);
00451 static bool menu_gateway_slot_in_use(void);
00452 static uint8_t menu_line_append_device_slot_name(char *dst, uint8_t offset, uint8_t slot);
00453 static void menu_build_trusted_slot_label(uint8_t slot, const trusted_info_t *info, char *dst);
00454 static void menu_build_empty_trusted_slot_label(uint8_t slot, char *dst);
00455 static bool menu_should_open_quick_reply(const char *text);
00456 static bool menu_modal_is_preemptible(menu_modal_t modal);
00457 static menu_auth_level_t menu_page_auth_level(menu_page_id_t page_id);
00458 static bool menu_auth_unlocked(const menu_state_t *st, menu_auth_level_t auth_level);
00459 static bool menu_pin_matches(const menu_state_t *st);
00460 static void menu_request_pin(menu_state_t *st, menu_page_id_t target_page, menu_auth_level_t
auth_level);
00461 static void menu_start_pin_change(menu_state_t *st, menu_action_t action);
00462 static bool menu_is_pin_change_action(menu_action_t action);
00463 static void menu_clear_pin_state(menu_state_t *st);
00464 static void menu_line_clear(char *dst);
00465 static uint8_t menu_line_copy(char *dst, uint8_t offset, const char *src, uint8_t max_chars);
00466 static uint8_t menu_line_append_u32(char *dst, uint8_t offset, uint32_t value);
00467 static uint8_t menu_line_append_i32(char *dst, uint8_t offset, int32_t value);
00468 static uint8_t menu_line_append_hex32(char *dst, uint8_t offset, uint32_t value);
00469 static uint8_t menu_line_append_freq_mhz(char *dst, uint8_t offset, uint32_t freq_hz);
00470 static void menu_line_format_u32(char *dst, const char *prefix, uint32_t value, const char *suffix);
00471 static void menu_line_format_i32(char *dst, const char *prefix, int32_t value, const char *suffix);
00472 static void menu_line_format_hex32(char *dst, const char *prefix, uint32_t value);
00473 static void menu_line_format_device_ref(char *dst, const char *prefix, uint32_t device_code);
00474 static void menu_line_format_source(char *dst, uint32_t device_code);
00475 static void menu_line_format_freq(char *dst, const char *prefix, uint32_t freq_hz);
00476 static void menu_line_format_fixed2(char *dst, const char *prefix, float value, const char *suffix);
00477 static void menu_line_copy_or_default(char *dst, const char *text, const char *fallback);
00478 static const char *menu_pair_code_text(const char *text, uint8_t prefix_len);
00479 static const char *menu_lora_bw_text(radio_lora_bw_t bw);
00480 static const char *menu_lora_cr_text(uint8_t denominator);
00481 static const char *menu_fsk_shape_text(radio_main_fsk_shaping_t shaping);
00482 static const char *menu_filter_text(radio_main_filter_t filter);
00483 static const char *menu_crc_text(radio_main_crc_type_t crc_type);
00484 static const char *menu_ook_threshold_text(radio_main_ook_threshold_t threshold);
00485 static const char *menu_address_filter_text(radio_main_address_filter_t filter);
00486 static void menu_build_page_title(const menu_state_t *st, const menu_page_t *page, char *dst);
00487 static void menu_build_item_label(const menu_state_t *st,
const menu_page_t *page,
uint8_t item_idx,
char *dst);
00491 static uint8_t menu_radio_settings_item_count(const radio_main_runtime_cfg_t *cfg);
00492 static void menu_build_radio_settings_item(const radio_main_runtime_cfg_t *cfg,
uint8_t item_idx,
char *dst);
00495
00496 static const menu_item_t s_page_pager_items[] =
00497 {
00498     { "Radio settings", MENU_PAGE_RADIO_SETTINGS, MENU_ACTION_NONE },
00499     { "Send message", MENU_PAGE_SEND_OPTIONS, MENU_ACTION_NONE },
00500     { "Message groups", MENU_PAGE_MSG_GROUPS, MENU_ACTION_NONE },
00501     { "Main menu", MENU_PAGE_MAIN, MENU_ACTION_NONE },
00502     { "Exit", MENU_PAGE_NONE, MENU_ACTION_EXIT_TO_MONITOR }
00503 };
00504
00505 static const menu_item_t s_page_radio_settings_items[] =
00506 {
00507     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00508     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00509     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00510     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00511     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00512     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00513     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00514     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00515     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },

```

```
00516     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00517     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00518     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00519     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00520     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00521 };
00522
00523 static const menu_item_t s_page_send_options_items[] =
00524 {
00525     { "STS:OK", MENU_PAGE_NONE, MENU_ACTION_SEND_DEFAULT },
00526     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00527 };
00528
00529 static const menu_item_t s_page_send_direct_list_items[] =
00530 {
00531     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00532     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00533     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00534     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00535     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00536     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00537     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00538     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00539     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00540     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00541     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00542     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00543     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00544     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00545     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00546     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00547     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00548 };
00549
00550 static const menu_item_t s_page_send_target_list_items[] =
00551 {
00552     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00553     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00554     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00555     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00556     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00557     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00558     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00559     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00560     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00561     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00562     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00563     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00564     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00565     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00566     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00567     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00568     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00569 };
00570
00571 static const menu_item_t s_page_msg_groups_items[] =
00572 {
00573     { "ALERT", MENU_PAGE_GROUP_ALERT, MENU_ACTION_NONE },
00574     { "STATUS", MENU_PAGE_GROUP_STATUS, MENU_ACTION_NONE },
00575     { "SERVICE", MENU_PAGE_GROUP_SERVICE, MENU_ACTION_NONE },
00576     { "ASK/ACT", MENU_PAGE_GROUP_QUICK, MENU_ACTION_NONE },
00577     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00578 };
00579
00580 static const menu_item_t s_page_group_alert_items[] =
00581 {
00582     { "ALR:FIRE", MENU_PAGE_NONE, MENU_ACTION_SEND_ALERT_FIRE },
00583     { "ALR:INTRUSION", MENU_PAGE_NONE, MENU_ACTION_SEND_ALERT_INTR },
00584     { "ALR:LOWBATT", MENU_PAGE_NONE, MENU_ACTION_SEND_ALERT_LOWBATT },
00585     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00586 };
00587
00588 static const menu_item_t s_page_group_status_items[] =
00589 {
00590     { "STS:OK", MENU_PAGE_NONE, MENU_ACTION_SEND_STATUS_OK },
00591     { "STS:BUSY", MENU_PAGE_NONE, MENU_ACTION_SEND_STATUS_BUSY },
00592     { "STS:IDLE", MENU_PAGE_NONE, MENU_ACTION_SEND_STATUS_IDLE },
00593     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00594 };
00595
00596 static const menu_item_t s_page_group_service_items[] =
00597 {
00598     { "SRV:PING", MENU_PAGE_NONE, MENU_ACTION_SEND_SERVICE_PING },
00599     { "SRV:RESET", MENU_PAGE_NONE, MENU_ACTION_SEND_SERVICE_RESET },
00600     { "SRV:SYNC", MENU_PAGE_NONE, MENU_ACTION_SEND_SERVICE_SYNC },
00601     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00602 };
```



```

00603
00604 static const menu_item_t s_page_group_quick_items[] =
00605 {
00606     { "ASK: Is done?", MENU_PAGE_NONE, MENU_ACTION_SEND_ASK_DONE },
00607     { "ACT: Come over.", MENU_PAGE_NONE, MENU_ACTION_SEND_ACT_COME_OVER },
00608     { "ACT: Stop!", MENU_PAGE_NONE, MENU_ACTION_SEND_ACT_STOP },
00609     { "ASK: Is ready?", MENU_PAGE_NONE, MENU_ACTION_SEND_ASK_READY },
00610     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00611 };
00612
00613 static const menu_item_t s_page_main_items[] =
00614 {
00615     { "Devices", MENU_PAGE_DEVICES, MENU_ACTION_NONE },
00616     { "Security", MENU_PAGE_SECURITY, MENU_ACTION_NONE },
00617     { "Hardware", MENU_PAGE_HARDWARE, MENU_ACTION_NONE },
00618     { "Modulation", MENU_PAGE_MODULATION, MENU_ACTION_NONE },
00619     { "Info", MENU_PAGE_INFO, MENU_ACTION_NONE },
00620     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00621 };
00622
00623 static const menu_item_t s_page_pin_settings_items[] =
00624 {
00625     { "Toggle PIN", MENU_PAGE_NONE, MENU_ACTION_PIN_TOGGLE },
00626     { "Change operator PIN", MENU_PAGE_NONE, MENU_ACTION_PIN_SET_OPERATOR },
00627     { "Change admin PIN", MENU_PAGE_NONE, MENU_ACTION_PIN_SET_ADMIN },
00628     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00629 };
00630
00631 static const menu_item_t s_page_devices_items[] =
00632 {
00633     { "Add new device", MENU_PAGE_NONE, MENU_ACTION_DEVICE_ADD },
00634     { "Pair with network", MENU_PAGE_NONE, MENU_ACTION_DEVICE_ADD_NETWORK },
00635     { "Delete device", MENU_PAGE_DEVICE_DELETE_LIST, MENU_ACTION_NONE },
00636     { "Connection info", MENU_PAGE_NONE, MENU_ACTION_DEVICE_INFO },
00637     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00638 };
00639
00640 static const menu_item_t s_page_device_delete_list_items[] =
00641 {
00642     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00643     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00644     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00645     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00646     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00647     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00648     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00649     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00650     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00651     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00652     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00653     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00654     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00655     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00656     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00657     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00658     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00659 };
00660
00661 static const menu_item_t s_page_device_delete_action_items[] =
00662 {
00663     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00664     { "Wyrzuc", MENU_PAGE_NONE, MENU_ACTION_DEVICE_DELETE_CONFIRM },
00665     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00666 };
00667
00668 static const menu_item_t s_page_security_items[] =
00669 {
00670     { "PIN settings", MENU_PAGE_PIN_SETTINGS, MENU_ACTION_NONE },
00671     { "Frequency hopping", MENU_PAGE_SECURITY_FH, MENU_ACTION_NONE },
00672     { "TPM", MENU_PAGE_NONE, MENU_ACTION_SEC_TPM_INFO },
00673     { "Keys", MENU_PAGE_NONE, MENU_ACTION_SEC_ROTATE_KEYS },
00674     { "Coding", MENU_PAGE_SECURITY_CODING, MENU_ACTION_NONE },
00675     { "Notif mode", MENU_PAGE_SECURITY_NOTIFY, MENU_ACTION_NONE },
00676     { "Auto ping", MENU_PAGE_SECURITY_AUTOPING, MENU_ACTION_NONE },
00677     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00678 };
00679
00680 static const menu_item_t s_page_security_fh_items[] =
00681 {
00682     { "Enable FH", MENU_PAGE_NONE, MENU_ACTION_SEC_FH_ENABLE },
00683     { "Disable FH", MENU_PAGE_NONE, MENU_ACTION_SEC_FH_DISABLE },
00684     { "Cycle 1 sec", MENU_PAGE_NONE, MENU_ACTION_SEC_FH_PERIOD_1000 },
00685     { "Cycle 2 sec", MENU_PAGE_NONE, MENU_ACTION_SEC_FH_PERIOD_2000 },
00686     { "Cycle 5 sec", MENU_PAGE_NONE, MENU_ACTION_SEC_FH_PERIOD_5000 },
00687     { "Cycle 10 sec", MENU_PAGE_NONE, MENU_ACTION_SEC_FH_PERIOD_10000 },
00688     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00689 };

```

```

00690
00691 static const menu_item_t s_page_security_coding_items[] =
00692 {
00693     { "Coding ON", MENU_PAGE_NONE, MENU_ACTION_SEC_CODING_ON },
00694     { "Coding OFF", MENU_PAGE_NONE, MENU_ACTION_SEC_CODING_OFF },
00695     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00696 };
00697
00698 static const menu_item_t s_page_security_notify_items[] =
00699 {
00700     { "Popup", MENU_PAGE_NONE, MENU_ACTION_SEC_NOTIFY_POPUP },
00701     { "Badge", MENU_PAGE_NONE, MENU_ACTION_SEC_NOTIFY_BADGE },
00702     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00703 };
00704
00705 static const menu_item_t s_page_security_autoping_items[] =
00706 {
00707     { "", MENU_PAGE_NONE, MENU_ACTION_NONE },
00708     { "Auto ping ON", MENU_PAGE_NONE, MENU_ACTION_SEC_AUTOPING_ON },
00709     { "Auto ping OFF", MENU_PAGE_NONE, MENU_ACTION_SEC_AUTOPING_OFF },
00710     { "Every 1 ms", MENU_PAGE_NONE, MENU_ACTION_SEC_AUTOPING_PERIOD_1 },
00711     { "Every 10 ms", MENU_PAGE_NONE, MENU_ACTION_SEC_AUTOPING_PERIOD_10 },
00712     { "Every 100 ms", MENU_PAGE_NONE, MENU_ACTION_SEC_AUTOPING_PERIOD_100 },
00713     { "Every 1 sec", MENU_PAGE_NONE, MENU_ACTION_SEC_AUTOPING_PERIOD_1000 },
00714     { "Every 10 sec", MENU_PAGE_NONE, MENU_ACTION_SEC_AUTOPING_PERIOD_10000 },
00715     { "LAVIET frame", MENU_PAGE_NONE, MENU_ACTION_SEC_AUTOPING_FRAME },
00716     { "Raw PING", MENU_PAGE_NONE, MENU_ACTION_SEC_AUTOPING_RAW },
00717     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00718 };
00719
00720 static const menu_item_t s_page_hardware_items[] =
00721 {
00722     { "Dist measure", MENU_PAGE_NONE, MENU_ACTION_HW_MEASURE_DIST },
00723     { "Temperature", MENU_PAGE_NONE, MENU_ACTION_HW_MEASURE_TEMP },
00724     { "Pressure", MENU_PAGE_NONE, MENU_ACTION_HW_MEASURE_PRESS },
00725     { "Led", MENU_PAGE_HARDWARE_LED, MENU_ACTION_NONE },
00726     { "Reset SX1276", MENU_PAGE_NONE, MENU_ACTION_HW_RADIO_RESET },
00727     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00728 };
00729
00730 static const menu_item_t s_page_hardware_led_items[] =
00731 {
00732     { "Rainbow", MENU_PAGE_NONE, MENU_ACTION_HW_LED_RAINBOW },
00733     { "Breath", MENU_PAGE_NONE, MENU_ACTION_HW_LED_BREATH },
00734     { "Off", MENU_PAGE_NONE, MENU_ACTION_HW_LED_OFF },
00735     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00736 };
00737
00738 static const menu_item_t s_page_modulation_items[] =
00739 {
00740     { "LoRa", MENU_PAGE_MOD_LORA, MENU_ACTION_NONE },
00741     { "FSK/GxSK", MENU_PAGE_MOD_FSK, MENU_ACTION_NONE },
00742     { "OOK", MENU_PAGE_MOD_OOK, MENU_ACTION_NONE },
00743     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00744 };
00745
00746 static const menu_item_t s_page_mod_lora_items[] =
00747 {
00748     { "Use LoRa", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_ENABLE },
00749     { "Reset defaults", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_RESET },
00750     { "Frequency", MENU_PAGE_MOD_LORA_FREQ, MENU_ACTION_NONE },
00751     { "Bandwidth", MENU_PAGE_MOD_LORA_BW, MENU_ACTION_NONE },
00752     { "Spread factor", MENU_PAGE_MOD_LORA_SF, MENU_ACTION_NONE },
00753     { "Coding rate", MENU_PAGE_MOD_LORA_CR, MENU_ACTION_NONE },
00754     { "TX power", MENU_PAGE_MOD_LORA_POWER, MENU_ACTION_NONE },
00755     { "CRC", MENU_PAGE_MOD_LORA_CRC, MENU_ACTION_NONE },
00756     { "Preamble", MENU_PAGE_MOD_LORA_PREAMBLE, MENU_ACTION_NONE },
00757     { "Header mode", MENU_PAGE_MOD_LORA_HEADER, MENU_ACTION_NONE },
00758     { "I/Q invert", MENU_PAGE_MOD_LORA_IQ, MENU_ACTION_NONE },
00759     { "Sync word", MENU_PAGE_MOD_LORA_SYNC, MENU_ACTION_NONE },
00760     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00761 };
00762
00763 static const menu_item_t s_page_mod_lora_freq_items[] =
00764 {
00765     { "868.0 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_FREQ_8680 },
00766     { "868.1 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_FREQ_8681 },
00767     { "868.3 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_FREQ_8683 },
00768     { "868.5 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_FREQ_8685 },
00769     { "868.8 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_FREQ_8688 },
00770     { "869.05 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_FREQ_86905 },
00771     { "869.525 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_FREQ_869525 },
00772     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00773 };
00774
00775 static const menu_item_t s_page_mod_lora_bw_items[] =
00776 {

```

```
00777     { "BW 7.8 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_BW_7_8 },
00778     { "BW 10.4 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_BW_10_4 },
00779     { "BW 15.6 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_BW_15_6 },
00780     { "BW 20.8 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_BW_20_8 },
00781     { "BW 31.25 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_BW_31_25 },
00782     { "BW 41.7 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_BW_41_7 },
00783     { "BW 62.5 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_BW_62_5 },
00784     { "BW 125 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_BW_125 },
00785     { "BW 250 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_BW_250 },
00786     { "BW 500 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_BW_500 },
00787     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00788 };
00789
00790 static const menu_item_t s_page_mod_lora_sf_items[] =
00791 {
00792     { "SF6", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_SF_6 },
00793     { "SF7", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_SF_7 },
00794     { "SF8", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_SF_8 },
00795     { "SF9", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_SF_9 },
00796     { "SF10", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_SF_10 },
00797     { "SF11", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_SF_11 },
00798     { "SF12", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_SF_12 },
00799     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00800 };
00801
00802 static const menu_item_t s_page_mod_lora_cr_items[] =
00803 {
00804     { "CR 4/5", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_CR_45 },
00805     { "CR 4/6", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_CR_46 },
00806     { "CR 4/7", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_CR_47 },
00807     { "CR 4/8", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_CR_48 },
00808     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00809 };
00810
00811 static const menu_item_t s_page_mod_lora_power_items[] =
00812 {
00813     { "2 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PWR_2 },
00814     { "5 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PWR_5 },
00815     { "8 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PWR_8 },
00816     { "11 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PWR_11 },
00817     { "14 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PWR_14 },
00818     { "17 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PWR_17 },
00819     { "20 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PWR_20 },
00820     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00821 };
00822
00823 static const menu_item_t s_page_mod_lora_crc_items[] =
00824 {
00825     { "CRC OFF", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_CRC_OFF },
00826     { "CRC ON", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_CRC_ON },
00827     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00828 };
00829
00830 static const menu_item_t s_page_mod_lora_preamble_items[] =
00831 {
00832     { "6 sym", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PREAMBLE_6 },
00833     { "8 sym", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PREAMBLE_8 },
00834     { "12 sym", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PREAMBLE_12 },
00835     { "16 sym", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PREAMBLE_16 },
00836     { "24 sym", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PREAMBLE_24 },
00837     { "32 sym", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_PREAMBLE_32 },
00838     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00839 };
00840
00841 static const menu_item_t s_page_mod_lora_header_items[] =
00842 {
00843     { "Explicit", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_HEADER_EXPLICIT },
00844     { "Implicit", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_HEADER_IMPLICIT },
00845     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00846 };
00847
00848 static const menu_item_t s_page_mod_lora_iq_items[] =
00849 {
00850     { "Normal", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_IQ_NORMAL },
00851     { "Invert", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_IQ_INVERT },
00852     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00853 };
00854
00855 static const menu_item_t s_page_mod_lora_sync_items[] =
00856 {
00857     { "0x12 private", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_SYNC_12 },
00858     { "0x34 public", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_SYNC_34 },
00859     { "0x56 custom", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_SYNC_56 },
00860     { "0xA5 custom", MENU_PAGE_NONE, MENU_ACTION_MOD_LORA_SYNC_A5 },
00861     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00862 };
00863
```

```
00864 static const menu_item_t s_page_mod_fsk_items[] =
00865 {
00866     { "Use FSK/GxSK", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_ENABLE },
00867     { "Reset defaults", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_RESET },
00868     { "Shaping", MENU_PAGE_MOD_FSK_SHAPING, MENU_ACTION_NONE },
00869     { "Frequency", MENU_PAGE_MOD_FSK_FREQ, MENU_ACTION_NONE },
00870     { "Bitrate", MENU_PAGE_MOD_FSK_BITRATE, MENU_ACTION_NONE },
00871     { "RX bandwidth", MENU_PAGE_MOD_FSK_BW, MENU_ACTION_NONE },
00872     { "Filter / BT", MENU_PAGE_MOD_FSK_FILTER, MENU_ACTION_NONE },
00873     { "TX power", MENU_PAGE_MOD_FSK_POWER, MENU_ACTION_NONE },
00874     { "Preamble", MENU_PAGE_MOD_FSK_PREAMBLE, MENU_ACTION_NONE },
00875     { "Sync length", MENU_PAGE_MOD_FSK_SYNC_LEN, MENU_ACTION_NONE },
00876     { "Sync word", MENU_PAGE_MOD_FSK_SYNC_WORD, MENU_ACTION_NONE },
00877     { "Address mode", MENU_PAGE_MOD_FSK_ADDR, MENU_ACTION_NONE },
00878     { "CRC", MENU_PAGE_MOD_FSK_CRC, MENU_ACTION_NONE },
00879     { "Whitening", MENU_PAGE_MOD_FSK_WHITEN, MENU_ACTION_NONE },
00880     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00881 };
00882
00883 static const menu_item_t s_page_mod_fsk_shaping_items[] =
00884 {
00885     { "FSK", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SHAPING_FSK },
00886     { "GFSK", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SHAPING_GFSK },
00887     { "MSK", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SHAPING_MSK },
00888     { "GMSK", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SHAPING_GMSK },
00889     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00890 };
00891
00892 static const menu_item_t s_page_mod_fsk_freq_items[] =
00893 {
00894     { "868.0 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_FREQ_8680 },
00895     { "868.1 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_FREQ_8681 },
00896     { "868.3 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_FREQ_8683 },
00897     { "868.5 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_FREQ_8685 },
00898     { "868.8 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_FREQ_8688 },
00899     { "869.05 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_FREQ_86905 },
00900     { "869.525 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_FREQ_869525 },
00901     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00902 };
00903
00904 static const menu_item_t s_page_mod_fsk_bitrate_items[] =
00905 {
00906     { "1.2 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BITRATE_1200 },
00907     { "2.4 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BITRATE_2400 },
00908     { "4.8 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BITRATE_4800 },
00909     { "9.6 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BITRATE_9600 },
00910     { "19.2 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BITRATE_19200 },
00911     { "38.4 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BITRATE_38400 },
00912     { "50.0 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BITRATE_50000 },
00913     { "100 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BITRATE_100000 },
00914     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00915 };
00916
00917 static const menu_item_t s_page_mod_fsk_bw_items[] =
00918 {
00919     { "BW 31.25 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BW_31_25 },
00920     { "BW 62.5 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BW_62_5 },
00921     { "BW 125 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BW_125 },
00922     { "BW 250 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BW_250 },
00923     { "BW 500 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_BW_500 },
00924     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00925 };
00926
00927 static const menu_item_t s_page_mod_fsk_filter_items[] =
00928 {
00929     { "No filter", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_FILTER_NONE },
00930     { "BT = 1.0", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_FILTER_BT10 },
00931     { "BT = 0.7", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_FILTER_BT07 },
00932     { "BT = 0.5", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_FILTER_BT05 },
00933     { "BT = 0.3", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_FILTER_BT03 },
00934     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00935 };
00936
00937 static const menu_item_t s_page_mod_fsk_power_items[] =
00938 {
00939     { "2 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PWR_2 },
00940     { "5 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PWR_5 },
00941     { "8 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PWR_8 },
00942     { "11 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PWR_11 },
00943     { "14 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PWR_14 },
00944     { "17 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PWR_17 },
00945     { "20 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PWR_20 },
00946     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00947 };
00948
00949 static const menu_item_t s_page_mod_fsk_preamble_items[] =
00950 {
```

```

00951     { "4 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PREAMBLE_4 },
00952     { "8 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PREAMBLE_8 },
00953     { "12 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PREAMBLE_12 },
00954     { "16 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PREAMBLE_16 },
00955     { "24 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PREAMBLE_24 },
00956     { "32 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_PREAMBLE_32 },
00957     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00958 };
00959
00960 static const menu_item_t s_page_mod_fsk_sync_len_items[] =
00961 {
00962     { "0 byte", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SYNC_LEN_0 },
00963     { "1 byte", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SYNC_LEN_1 },
00964     { "2 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SYNC_LEN_2 },
00965     { "3 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SYNC_LEN_3 },
00966     { "4 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SYNC_LEN_4 },
00967     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00968 };
00969
00970 static const menu_item_t s_page_mod_fsk_sync_word_items[] =
00971 {
00972     { "0x55AA", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SYNC_WORD_55AA },
00973     { "0x2DD4", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SYNC_WORD_2DD4 },
00974     { "0xA55A", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SYNC_WORD_A55A },
00975     { "0xC194C1", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SYNC_WORD_C194C1 },
00976     { "0x1ACFFC1D", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_SYNC_WORD_1ACFFC1D },
00977     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00978 };
00979
00980 static const menu_item_t s_page_mod_fsk_addr_items[] =
00981 {
00982     { "No filter", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_ADDR_NONE },
00983     { "Node only", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_ADDR_NODE },
00984     { "Node + BC", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_ADDR_NODE_BC },
00985     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00986 };
00987
00988 static const menu_item_t s_page_mod_fsk_crc_items[] =
00989 {
00990     { "CRC OFF", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_CRC_OFF },
00991     { "CRC IBM", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_CRC_IBM },
00992     { "CRC CCITT", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_CRC_CCITT },
00993     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
00994 };
00995
00996 static const menu_item_t s_page_mod_fsk_whiten_items[] =
00997 {
00998     { "Whitening OFF", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_WHITEN_OFF },
00999     { "Whitening ON", MENU_PAGE_NONE, MENU_ACTION_MOD_FSK_WHITEN_ON },
01000     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
01001 };
01002
01003 static const menu_item_t s_page_mod_ook_items[] =
01004 {
01005     { "Use OOK mode", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_ENABLE },
01006     { "Reset defaults", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_RESET },
01007     { "Frequency", MENU_PAGE_MOD_OOK_FREQ, MENU_ACTION_NONE },
01008     { "Bitrate", MENU_PAGE_MOD_OOK_BITRATE, MENU_ACTION_NONE },
01009     { "RX bandwidth", MENU_PAGE_MOD_OOK_BW, MENU_ACTION_NONE },
01010     { "TX power", MENU_PAGE_MOD_OOK_POWER, MENU_ACTION_NONE },
01011     { "Preamble", MENU_PAGE_MOD_OOK_PREAMBLE, MENU_ACTION_NONE },
01012     { "Sync len", MENU_PAGE_MOD_OOK_SYNC_LEN, MENU_ACTION_NONE },
01013     { "Sync word", MENU_PAGE_MOD_OOK_SYNC_WORD, MENU_ACTION_NONE },
01014     { "Threshold type", MENU_PAGE_MOD_OOK_THRESH_TYPE, MENU_ACTION_NONE },
01015     { "Threshold val", MENU_PAGE_MOD_OOK_THRESH_VALUE, MENU_ACTION_NONE },
01016     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
01017 };
01018
01019 static const menu_item_t s_page_mod_ook_freq_items[] =
01020 {
01021     { "868.0 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_FREQ_8680 },
01022     { "868.1 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_FREQ_8681 },
01023     { "868.3 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_FREQ_8683 },
01024     { "868.5 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_FREQ_8685 },
01025     { "868.8 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_FREQ_8688 },
01026     { "869.05 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_FREQ_86905 },
01027     { "869.525 MHz", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_FREQ_869525 },
01028     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
01029 };
01030
01031 static const menu_item_t s_page_mod_ook_bitrate_items[] =
01032 {
01033     { "1.2 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BITRATE_1200 },
01034     { "2.4 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BITRATE_2400 },
01035     { "4.8 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BITRATE_4800 },
01036     { "9.6 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BITRATE_9600 },
01037     { "19.2 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BITRATE_19200 },

```

```
01038     { "38.4 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BITRATE_38400 },
01039     { "50.0 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BITRATE_50000 },
01040     { "100 kbps", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BITRATE_100000 },
01041     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
01042 };
01043
01044 static const menu_item_t s_page_mod_ook_power_items[] =
01045 {
01046     { "2 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PWR_2 },
01047     { "5 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PWR_5 },
01048     { "8 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PWR_8 },
01049     { "11 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PWR_11 },
01050     { "14 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PWR_14 },
01051     { "17 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PWR_17 },
01052     { "20 dBm", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PWR_20 },
01053     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
01054 };
01055
01056 static const menu_item_t s_page_mod_ook_bw_items[] =
01057 {
01058     { "BW 31.25 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BW_31_25 },
01059     { "BW 62.5 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BW_62_5 },
01060     { "BW 125 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BW_125 },
01061     { "BW 250 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BW_250 },
01062     { "BW 500 kHz", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_BW_500 },
01063     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
01064 };
01065
01066 static const menu_item_t s_page_mod_ook_preamble_items[] =
01067 {
01068     { "4 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PREAMBLE_4 },
01069     { "8 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PREAMBLE_8 },
01070     { "12 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PREAMBLE_12 },
01071     { "16 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PREAMBLE_16 },
01072     { "24 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PREAMBLE_24 },
01073     { "32 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_PREAMBLE_32 },
01074     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
01075 };
01076
01077 static const menu_item_t s_page_mod_ook_sync_len_items[] =
01078 {
01079     { "0 byte", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_SYNC_LEN_0 },
01080     { "1 byte", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_SYNC_LEN_1 },
01081     { "2 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_SYNC_LEN_2 },
01082     { "3 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_SYNC_LEN_3 },
01083     { "4 bytes", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_SYNC_LEN_4 },
01084     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
01085 };
01086
01087 static const menu_item_t s_page_mod_ook_sync_word_items[] =
01088 {
01089     { "0x55AA", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_SYNC_WORD_55AA },
01090     { "0x2DD4", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_SYNC_WORD_2DD4 },
01091     { "0xA55A", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_SYNC_WORD_A55A },
01092     { "0xC194C1", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_SYNC_WORD_C194C1 },
01093     { "0x1ACFFC1D", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_SYNC_WORD_1ACFFC1D },
01094     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
01095 };
01096
01097 static const menu_item_t s_page_mod_ook_thresh_type_items[] =
01098 {
01099     { "Fixed", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_THRESH_FIXED },
01100     { "Peak", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_THRESH_PEAK },
01101     { "Average", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_THRESH_AVG },
01102     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
01103 };
01104
01105 static const menu_item_t s_page_mod_ook_thresh_value_items[] =
01106 {
01107     { "Thr = 4", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_THRESH_4 },
01108     { "Thr = 8", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_THRESH_8 },
01109     { "Thr = 12", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_THRESH_12 },
01110     { "Thr = 16", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_THRESH_16 },
01111     { "Thr = 24", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_THRESH_24 },
01112     { "Thr = 32", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_THRESH_32 },
01113     { "Thr = 48", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_THRESH_48 },
01114     { "Thr = 64", MENU_PAGE_NONE, MENU_ACTION_MOD_OOK_THRESH_64 },
01115     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
01116 };
01117
01118 static const menu_item_t s_page_info_items[] =
01119 {
01120     { "Show info", MENU_PAGE_NONE, MENU_ACTION_INFO_SHOW },
01121     { "Back", MENU_PAGE_NONE, MENU_ACTION_BACK }
01122 };
01123
01124 static const menu_page_t s_pages[] =
```



```

01125 {
01126     { "PAGER", MENU_PAGE_NONE, s_page_pager_items, (uint8_t)(sizeof(s_page_pager_items) /
01127         sizeof(s_page_pager_items[0])) },
01127     { "RADIO SETTINGS", MENU_PAGE_PAGER, s_page_radio_settings_items,
01128         (uint8_t)(sizeof(s_page_radio_settings_items) / sizeof(s_page_radio_settings_items[0])) },
01128     { "SEND", MENU_PAGE_PAGER, s_page_send_options_items, (uint8_t)(sizeof(s_page_send_options_items)
01129         / sizeof(s_page_send_options_items[0])) },
01129     { "SEND DIRECT", MENU_PAGE_SEND_OPTIONS, s_page_send_direct_list_items,
01130         (uint8_t)(sizeof(s_page_send_direct_list_items) / sizeof(s_page_send_direct_list_items[0])) },
01130     { "SEND TO", MENU_PAGE_SEND_OPTIONS, s_page_send_target_list_items,
01131         (uint8_t)(sizeof(s_page_send_target_list_items) / sizeof(s_page_send_target_list_items[0])) },
01131     { "MSG GROUPS", MENU_PAGE_PAGER, s_page_msg_groups_items,
01132         (uint8_t)(sizeof(s_page_msg_groups_items) / sizeof(s_page_msg_groups_items[0])) },
01132     { "ALERT", MENU_PAGE_MSG_GROUPS, s_page_group_alert_items,
01133         (uint8_t)(sizeof(s_page_group_alert_items) / sizeof(s_page_group_alert_items[0])) },
01133     { "STATUS", MENU_PAGE_MSG_GROUPS, s_page_group_status_items,
01134         (uint8_t)(sizeof(s_page_group_status_items) / sizeof(s_page_group_status_items[0])) },
01134     { "SERVICE", MENU_PAGE_MSG_GROUPS, s_page_group_service_items,
01135         (uint8_t)(sizeof(s_page_group_service_items) / sizeof(s_page_group_service_items[0])) },
01135     { "ASK/ACT", MENU_PAGE_MSG_GROUPS, s_page_group_quick_items,
01136         (uint8_t)(sizeof(s_page_group_quick_items) / sizeof(s_page_group_quick_items[0])) },
01136     { "MAIN MENU", MENU_PAGE_PAGER, s_page_main_items, (uint8_t)(sizeof(s_page_main_items) /
01137         sizeof(s_page_main_items[0])) },
01137     { "DEVICES", MENU_PAGE_MAIN, s_page_devices_items, (uint8_t)(sizeof(s_page_devices_items) /
01138         sizeof(s_page_devices_items[0])) },
01138     { "DELETE DEV", MENU_PAGE_DEVICES, s_page_device_delete_list_items,
01139         (uint8_t)(sizeof(s_page_device_delete_list_items) / sizeof(s_page_device_delete_list_items[0])) },
01139     { "DELETE DEV", MENU_PAGE_DEVICE_DELETE_LIST, s_page_device_delete_action_items,
01140         (uint8_t)(sizeof(s_page_device_delete_action_items) / sizeof(s_page_device_delete_action_items[0])) },
01140     { "PIN SETTINGS", MENU_PAGE_SECURITY, s_page_pin_settings_items,
01141         (uint8_t)(sizeof(s_page_pin_settings_items) / sizeof(s_page_pin_settings_items[0])) },
01141     { "SECURITY", MENU_PAGE_MAIN, s_page_security_items, (uint8_t)(sizeof(s_page_security_items) /
01142         sizeof(s_page_security_items[0])) },
01142     { "SEC FH", MENU_PAGE_SECURITY, s_page_security_fh_items,
01143         (uint8_t)(sizeof(s_page_security_fh_items) / sizeof(s_page_security_fh_items[0])) },
01143     { "CODING", MENU_PAGE_SECURITY, s_page_security_coding_items,
01144         (uint8_t)(sizeof(s_page_security_coding_items) / sizeof(s_page_security_coding_items[0])) },
01144     { "NOTIFY", MENU_PAGE_SECURITY, s_page_security_notify_items,
01145         (uint8_t)(sizeof(s_page_security_notify_items) / sizeof(s_page_security_notify_items[0])) },
01145     { "AUTOPING", MENU_PAGE_SECURITY, s_page_security_autoping_items,
01146         (uint8_t)(sizeof(s_page_security_autoping_items) / sizeof(s_page_security_autoping_items[0])) },
01146     { "HARDWARE", MENU_PAGE_MAIN, s_page_hardware_items, (uint8_t)(sizeof(s_page_hardware_items) /
01147         sizeof(s_page_hardware_items[0])) },
01147     { "LED", MENU_PAGE_HARDWARE, s_page_hardware_led_items,
01148         (uint8_t)(sizeof(s_page_hardware_led_items) / sizeof(s_page_hardware_led_items[0])) },
01148     { "MODULATION", MENU_PAGE_MAIN, s_page_modulation_items, (uint8_t)(sizeof(s_page_modulation_items)
01149         / sizeof(s_page_modulation_items[0])) },
01149     { "LORA", MENU_PAGE_MODULATION, s_page_mod_lora_items, (uint8_t)(sizeof(s_page_mod_lora_items) /
01150         sizeof(s_page_mod_lora_items[0])) },
01150     { "LORA FREQ", MENU_PAGE_MOD_LORA, s_page_mod_lora_freq_items,
01151         (uint8_t)(sizeof(s_page_mod_lora_freq_items) / sizeof(s_page_mod_lora_freq_items[0])) },
01151     { "LORA BW", MENU_PAGE_MOD_LORA, s_page_mod_lora_bw_items,
01152         (uint8_t)(sizeof(s_page_mod_lora_bw_items) / sizeof(s_page_mod_lora_bw_items[0])) },
01152     { "LORA SF", MENU_PAGE_MOD_LORA, s_page_mod_lora_sf_items,
01153         (uint8_t)(sizeof(s_page_mod_lora_sf_items) / sizeof(s_page_mod_lora_sf_items[0])) },
01153     { "LORA CR", MENU_PAGE_MOD_LORA, s_page_mod_lora_cr_items,
01154         (uint8_t)(sizeof(s_page_mod_lora_cr_items) / sizeof(s_page_mod_lora_cr_items[0])) },
01154     { "LORA PWR", MENU_PAGE_MOD_LORA, s_page_mod_lora_power_items,
01155         (uint8_t)(sizeof(s_page_mod_lora_power_items) / sizeof(s_page_mod_lora_power_items[0])) },
01155     { "LORA CRC", MENU_PAGE_MOD_LORA, s_page_mod_lora_crc_items,
01156         (uint8_t)(sizeof(s_page_mod_lora_crc_items) / sizeof(s_page_mod_lora_crc_items[0])) },
01156     { "LORA PREAM", MENU_PAGE_MOD_LORA, s_page_mod_lora_preamble_items,
01157         (uint8_t)(sizeof(s_page_mod_lora_preamble_items) / sizeof(s_page_mod_lora_preamble_items[0])) },
01157     { "LORA HEADER", MENU_PAGE_MOD_LORA, s_page_mod_lora_header_items,
01158         (uint8_t)(sizeof(s_page_mod_lora_header_items) / sizeof(s_page_mod_lora_header_items[0])) },
01158     { "LORA IQ", MENU_PAGE_MOD_LORA, s_page_mod_lora_iq_items,
01159         (uint8_t)(sizeof(s_page_mod_lora_iq_items) / sizeof(s_page_mod_lora_iq_items[0])) },
01159     { "LORA SYNC", MENU_PAGE_MOD_LORA, s_page_mod_lora_sync_items,
01160         (uint8_t)(sizeof(s_page_mod_lora_sync_items) / sizeof(s_page_mod_lora_sync_items[0])) },
01160     { "FSK", MENU_PAGE_MODULATION, s_page_mod_fsk_items, (uint8_t)(sizeof(s_page_mod_fsk_items) /
01161         sizeof(s_page_mod_fsk_items[0])) },
01161     { "FSK SHAPE", MENU_PAGE_MOD_FSK, s_page_mod_fsk_shaping_items,
01162         (uint8_t)(sizeof(s_page_mod_fsk_shaping_items) / sizeof(s_page_mod_fsk_shaping_items[0])) },
01162     { "FSK FREQ", MENU_PAGE_MOD_FSK, s_page_mod_fsk_freq_items,
01163         (uint8_t)(sizeof(s_page_mod_fsk_freq_items) / sizeof(s_page_mod_fsk_freq_items[0])) },
01163     { "FSK BITR", MENU_PAGE_MOD_FSK, s_page_mod_fsk_bitrate_items,
01164         (uint8_t)(sizeof(s_page_mod_fsk_bitrate_items) / sizeof(s_page_mod_fsk_bitrate_items[0])) },
01164     { "FSK BW", MENU_PAGE_MOD_FSK, s_page_mod_fsk_bw_items, (uint8_t)(sizeof(s_page_mod_fsk_bw_items)
01165         / sizeof(s_page_mod_fsk_bw_items[0])) },
01165     { "FSK FILTER", MENU_PAGE_MOD_FSK, s_page_mod_fsk_filter_items,
01166         (uint8_t)(sizeof(s_page_mod_fsk_filter_items) / sizeof(s_page_mod_fsk_filter_items[0])) },
01166     { "FSK PWR", MENU_PAGE_MOD_FSK, s_page_mod_fsk_power_items,
01167         (uint8_t)(sizeof(s_page_mod_fsk_power_items) / sizeof(s_page_mod_fsk_power_items[0])) },
01167     { "FSK PREAM", MENU_PAGE_MOD_FSK, s_page_mod_fsk_preamble_items,
01168         (uint8_t)(sizeof(s_page_mod_fsk_preamble_items) / sizeof(s_page_mod_fsk_preamble_items[0])) },
01168     { "FSK SLEN", MENU_PAGE_MOD_FSK, s_page_mod_fsk_sync_len_items,
01169         (uint8_t)(sizeof(s_page_mod_fsk_sync_len_items) / sizeof(s_page_mod_fsk_sync_len_items[0])) },

```

```

01169     { "FSK SYNC", MENU_PAGE_MOD_FSK, s_page_mod_fsk_sync_word_items,
(uint8_t)(sizeof(s_page_mod_fsk_sync_word_items) / sizeof(s_page_mod_fsk_sync_word_items[0])) },
01170     { "FSK ADDR", MENU_PAGE_MOD_FSK, s_page_mod_fsk_addr_items,
(uint8_t)(sizeof(s_page_mod_fsk_addr_items) / sizeof(s_page_mod_fsk_addr_items[0])) },
01171     { "FSK CRC", MENU_PAGE_MOD_FSK, s_page_mod_fsk_crc_items,
(uint8_t)(sizeof(s_page_mod_fsk_crc_items) / sizeof(s_page_mod_fsk_crc_items[0])) },
01172     { "FSK WHITE", MENU_PAGE_MOD_FSK, s_page_mod_fsk_whiten_items,
(uint8_t)(sizeof(s_page_mod_fsk_whiten_items) / sizeof(s_page_mod_fsk_whiten_items[0])) },
01173     { "OOK", MENU_PAGE_MODULATION, s_page_mod_ook_items, (uint8_t)(sizeof(s_page_mod_ook_items) /
sizeof(s_page_mod_ook_items[0])) },
01174     { "OOK FREQ", MENU_PAGE_MOD_OOK, s_page_mod_ook_freq_items,
(uint8_t)(sizeof(s_page_mod_ook_freq_items) / sizeof(s_page_mod_ook_freq_items[0])) },
01175     { "OOK BITR", MENU_PAGE_MOD_OOK, s_page_mod_ook_bitrate_items,
(uint8_t)(sizeof(s_page_mod_ook_bitrate_items) / sizeof(s_page_mod_ook_bitrate_items[0])) },
01176     { "OOK PWR", MENU_PAGE_MOD_OOK, s_page_mod_ook_power_items,
(uint8_t)(sizeof(s_page_mod_ook_power_items) / sizeof(s_page_mod_ook_power_items[0])) },
01177     { "OOK BW", MENU_PAGE_MOD_OOK, s_page_mod_ook_bw_items, (uint8_t)(sizeof(s_page_mod_ook_bw_items)
/ sizeof(s_page_mod_ook_bw_items[0])) },
01178     { "OOK PRE", MENU_PAGE_MOD_OOK, s_page_mod_ook_preamble_items,
(uint8_t)(sizeof(s_page_mod_ook_preamble_items) / sizeof(s_page_mod_ook_preamble_items[0])) },
01179     { "OOK SYNL", MENU_PAGE_MOD_OOK, s_page_mod_ook_sync_len_items,
(uint8_t)(sizeof(s_page_mod_ook_sync_len_items) / sizeof(s_page_mod_ook_sync_len_items[0])) },
01180     { "OOK SYNW", MENU_PAGE_MOD_OOK, s_page_mod_ook_sync_word_items,
(uint8_t)(sizeof(s_page_mod_ook_sync_word_items) / sizeof(s_page_mod_ook_sync_word_items[0])) },
01181     { "OOK THR T", MENU_PAGE_MOD_OOK, s_page_mod_ook_thresh_type_items,
(uint8_t)(sizeof(s_page_mod_ook_thresh_type_items) / sizeof(s_page_mod_ook_thresh_type_items[0])) },
01182     { "OOK THR V", MENU_PAGE_MOD_OOK, s_page_mod_ook_thresh_value_items,
(uint8_t)(sizeof(s_page_mod_ook_thresh_value_items) / sizeof(s_page_mod_ook_thresh_value_items[0])) },
01183     { "INFO", MENU_PAGE_MAIN, s_page_info_items, (uint8_t)(sizeof(s_page_info_items) /
sizeof(s_page_info_items[0])) }
01184 };
01185
01186 #define MENU_RADIO_ACTIVATE(_action, _mod, _ok, _err) \
01187     { (_action), true, (_mod), false, RADIO_MAIN_OPTION_LORA_FREQ, 0UL, false, (_ok), (_err) }
01188
01189 #define MENU_RADIO_SET(_action, _mod, _option, _value, _ok, _err) \
01190     { (_action), true, (_mod), true, (_option), (_value), false, (_ok), (_err) }
01191
01192 #define MENU_RADIO_PRESET(_action, _preset, _ok, _err) \
01193     { (_action), true, RADIO_MAIN_MODULATION_LORA, true, RADIO_MAIN_OPTION_LORA_PRESET, (_preset),
true, (_ok), (_err) }
01194
01195 static const menu_radio_action_binding_t s_radio_action_bindings[] =
01196 {
01197     MENU_RADIO_ACTIVATE(MENU_ACTION_MOD_LORA_ENABLE, RADIO_MAIN_MODULATION_LORA, "LoRa mode", "LoRa
set failed"),
01198     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_RESET, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_RESET_DEFAULTS, 0UL, "LoRa reset", "LoRa reset failed"),
01199     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_FREQ_8680, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_FREQ, 868000000UL, "LoRa F=868.0", "LoRa freq failed"),
01200     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_FREQ_8681, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_FREQ, 868100000UL, "LoRa F=868.1", "LoRa freq failed"),
01201     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_FREQ_8683, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_FREQ, 868300000UL, "LoRa F=868.3", "LoRa freq failed"),
01202     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_FREQ_8685, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_FREQ, 868500000UL, "LoRa F=868.5", "LoRa freq failed"),
01203     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_FREQ_8688, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_FREQ, 868800000UL, "LoRa F=868.8", "LoRa freq failed"),
01204     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_FREQ_86905, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_FREQ, 869050000UL, "LoRa F=869.05", "LoRa freq failed"),
01205     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_FREQ_869525, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_FREQ, 869525000UL, "LoRa F=869.525", "LoRa freq failed"),
01206     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_BW_7_8, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_BW,
(uint32_t)RADIO_LORA_BW_7_8_KHZ, "LoRa BW7.8", "LoRa BW failed"),
01207     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_BW_10_4, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_BW, (uint32_t)RADIO_LORA_BW_10_4_KHZ, "LoRa BW10.4", "LoRa BW failed"),
01208     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_BW_15_6, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_BW, (uint32_t)RADIO_LORA_BW_15_6_KHZ, "LoRa BW15.6", "LoRa BW failed"),
01209     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_BW_20_8, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_BW, (uint32_t)RADIO_LORA_BW_20_8_KHZ, "LoRa BW20.8", "LoRa BW failed"),
01210     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_BW_31_25, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_BW, (uint32_t)RADIO_LORA_BW_31_25_KHZ, "LoRa BW31.2", "LoRa BW failed"),
01211     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_BW_41_7, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_BW, (uint32_t)RADIO_LORA_BW_41_7_KHZ, "LoRa BW41.7", "LoRa BW failed"),
01212     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_BW_62_5, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_BW, (uint32_t)RADIO_LORA_BW_62_5_KHZ, "LoRa BW62.5", "LoRa BW failed"),
01213     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_BW_125, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_BW,
(uint32_t)RADIO_LORA_BW_125_KHZ, "LoRa BW125", "LoRa BW failed"),
01214     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_BW_250, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_BW,
(uint32_t)RADIO_LORA_BW_250_KHZ, "LoRa BW250", "LoRa BW failed"),
01215     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_BW_500, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_BW,
(uint32_t)RADIO_LORA_BW_500_KHZ, "LoRa BW500", "LoRa BW failed"),
01216     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_SF_6, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_SF,
6UL, "LoRa SF6", "LoRa SF failed"),
01217     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_SF_7, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_SF,
7UL, "LoRa SF7", "LoRa SF failed"),
01218     MENU_RADIO_SET(MENU_ACTION_MOD_LORA_SF_8, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_SF,

```



```

8UL, "LoRa SF8", "LoRa SF failed"),
01219 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_SF_9, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_SF,
9UL, "LoRa SF9", "LoRa SF failed"),
01220 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_SF_10, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_SF,
10UL, "LoRa SF10", "LoRa SF failed"),
01221 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_SF_11, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_SF,
11UL, "LoRa SF11", "LoRa SF failed"),
01222 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_SF_12, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_SF,
12UL, "LoRa SF12", "LoRa SF failed"),
01223 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_CR_45, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_CR,
5UL, "LoRa CR4/5", "LoRa CR failed"),
01224 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_CR_46, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_CR,
6UL, "LoRa CR4/6", "LoRa CR failed"),
01225 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_CR_47, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_CR,
7UL, "LoRa CR4/7", "LoRa CR failed"),
01226 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_CR_48, RADIO_MAIN_MODULATION_LORA, RADIO_MAIN_OPTION_LORA_CR,
8UL, "LoRa CR4/8", "LoRa CR failed"),
01227 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PWR_2, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_TX_POWER, 2UL, "LoRa P=2", "LoRa power failed"),
01228 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PWR_5, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_TX_POWER, 5UL, "LoRa P=5", "LoRa power failed"),
01229 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PWR_8, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_TX_POWER, 8UL, "LoRa P=8", "LoRa power failed"),
01230 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PWR_11, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_TX_POWER, 11UL, "LoRa P=11", "LoRa power failed"),
01231 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PWR_14, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_TX_POWER, 14UL, "LoRa P=14", "LoRa power failed"),
01232 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PWR_17, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_TX_POWER, 17UL, "LoRa P=17", "LoRa power failed"),
01233 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PWR_20, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_TX_POWER, 20UL, "LoRa P=20", "LoRa power failed"),
01234 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_CRC_OFF, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_CRC, (uint32_t)RADIO_MAIN_CRC_OFF, "LoRa CRC OFF", "LoRa CRC failed"),
01235 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_CRC_ON, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_CRC, (uint32_t)RADIO_MAIN_CRC_SX1276, "LoRa CRC ON", "LoRa CRC failed"),
01236 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PREAMBLE_6, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_PREAMBLE, 6UL, "LoRa PRE=6", "LoRa preamble failed"),
01237 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PREAMBLE_8, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_PREAMBLE, 8UL, "LoRa PRE=8", "LoRa preamble failed"),
01238 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PREAMBLE_12, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_PREAMBLE, 12UL, "LoRa PRE=12", "LoRa preamble failed"),
01239 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PREAMBLE_16, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_PREAMBLE, 16UL, "LoRa PRE=16", "LoRa preamble failed"),
01240 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PREAMBLE_24, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_PREAMBLE, 24UL, "LoRa PRE=24", "LoRa preamble failed"),
01241 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_PREAMBLE_32, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_PREAMBLE, 32UL, "LoRa PRE=32", "LoRa preamble failed"),
01242 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_HEADER_EXPLICIT, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_HEADER_MODE, (uint32_t)RADIO_MAIN_HEADER_EXPLICIT, "Hdr explicit", "Header
failed"),
01243 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_HEADER_IMPLICIT, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_HEADER_MODE, (uint32_t)RADIO_MAIN_HEADER_IMPLICIT, "Hdr implicit", "Header
failed"),
01244 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_IQ_NORMAL, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_IQ_INVERT, 0UL, "IQ normal", "IQ set failed"),
01245 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_IQ_INVERT, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_IQ_INVERT, 1UL, "IQ invert", "IQ set failed"),
01246 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_SYNC_12, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_SYNC_WORD, 0x12UL, "Sync 0x12", "Sync set failed"),
01247 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_SYNC_34, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_SYNC_WORD, 0x34UL, "Sync 0x34", "Sync set failed"),
01248 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_SYNC_56, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_SYNC_WORD, 0x56UL, "Sync 0x56", "Sync set failed"),
01249 MENU_RADIO_SET(MENU_ACTION_MOD_LORA_SYNC_A5, RADIO_MAIN_MODULATION_LORA,
RADIO_MAIN_OPTION_LORA_SYNC_WORD, 0xA5UL, "Sync 0xA5", "Sync set failed"),
01250
01251 MENU_RADIO_ACTIVATE(MENU_ACTION_MOD_FSK_ENABLE, RADIO_MAIN_MODULATION_FSK, "FSK mode", "FSK set
failed"),
01252 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SHAPING_FSK, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SHAPING, (uint32_t)RADIO_MAIN_FSK_SHAPING_FSK, "Shape FSK", "Shape failed"),
01253 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SHAPING_GFSK, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SHAPING, (uint32_t)RADIO_MAIN_FSK_SHAPING_GFSK, "Shape GFSK", "Shape failed"),
01254 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SHAPING_MSK, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SHAPING, (uint32_t)RADIO_MAIN_FSK_SHAPING_MSK, "Shape MSK", "Shape failed"),
01255 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SHAPING_GMSK, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SHAPING, (uint32_t)RADIO_MAIN_FSK_SHAPING_GMSK, "Shape GMSK", "Shape failed"),
01256 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_FREQ_8680, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_FREQ, 868000000UL, "FSK F=868.0", "FSK freq failed"),
01257 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_FREQ_8681, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_FREQ, 868100000UL, "FSK F=868.1", "FSK freq failed"),
01258 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_FREQ_8683, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_FREQ, 868300000UL, "FSK F=868.3", "FSK freq failed"),
01259 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_FREQ_8685, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_FREQ, 868500000UL, "FSK F=868.5", "FSK freq failed"),
01260 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_FREQ_8688, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_FREQ, 868800000UL, "FSK F=868.8", "FSK freq failed"),
01261 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_FREQ_86905, RADIO_MAIN_MODULATION_FSK,

```

```
RADIO_MAIN_OPTION_FSK_FREQ, 869050000UL, "FSK F=869.05", "FSK freq failed"),
01262 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_FREQ_869525, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_FREQ, 869525000UL, "FSK F=869.525", "FSK freq failed"),
01263 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BITRATE_1200, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_BITRATE, 1200UL, "FSK BR1.2", "FSK bitrate failed"),
01264 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BITRATE_2400, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_BITRATE, 2400UL, "FSK BR2.4", "FSK bitrate failed"),
01265 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BITRATE_4800, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_BITRATE, 4800UL, "FSK BR4.8", "FSK bitrate failed"),
01266 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BITRATE_9600, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_BITRATE, 9600UL, "FSK BR9.6", "FSK bitrate failed"),
01267 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BITRATE_19200, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_BITRATE, 19200UL, "FSK BR19.2", "FSK bitrate failed"),
01268 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BITRATE_38400, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_BITRATE, 38400UL, "FSK BR38.4", "FSK bitrate failed"),
01269 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BITRATE_50000, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_BITRATE, 50000UL, "FSK BR50", "FSK bitrate failed"),
01270 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BITRATE_100000, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_BITRATE, 100000UL, "FSK BR100", "FSK bitrate failed"),
01271 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BW_31_25, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_RX_BW, (uint32_t)RADIO_LORA_BW_31_25_KHZ, "FSK BW31.2", "FSK BW failed"),
01272 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BW_62_5, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_RX_BW, (uint32_t)RADIO_LORA_BW_62_5_KHZ, "FSK BW62.5", "FSK BW failed"),
01273 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BW_125, RADIO_MAIN_MODULATION_FSK, RADIO_MAIN_OPTION_FSK_RX_BW,
(uint32_t)RADIO_LORA_BW_125_KHZ, "FSK BW125", "FSK BW failed"),
01274 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BW_250, RADIO_MAIN_MODULATION_FSK, RADIO_MAIN_OPTION_FSK_RX_BW,
(uint32_t)RADIO_LORA_BW_250_KHZ, "FSK BW250", "FSK BW failed"),
01275 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_BW_500, RADIO_MAIN_MODULATION_FSK, RADIO_MAIN_OPTION_FSK_RX_BW,
(uint32_t)RADIO_LORA_BW_500_KHZ, "FSK BW500", "FSK BW failed"),
01276 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_FILTER_NONE, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_FILTER, (uint32_t)RADIO_MAIN_FILTER_NONE, "Filt OFF", "Filter failed"),
01277 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_FILTER_BT10, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_FILTER, (uint32_t)RADIO_MAIN_FILTER_BT_10, "Filt BT1.0", "Filter failed"),
01278 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_FILTER_BT07, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_FILTER, (uint32_t)RADIO_MAIN_FILTER_BT_07, "Filt BT0.7", "Filter failed"),
01279 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_FILTER_BT05, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_FILTER, (uint32_t)RADIO_MAIN_FILTER_BT_05, "Filt BT0.5", "Filter failed"),
01280 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_FILTER_BT03, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_FILTER, (uint32_t)RADIO_MAIN_FILTER_BT_03, "Filt BT0.3", "Filter failed"),
01281 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PWR_2, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_TX_POWER, 2UL, "FSK P=2", "FSK power failed"),
01282 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PWR_5, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_TX_POWER, 5UL, "FSK P=5", "FSK power failed"),
01283 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PWR_8, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_TX_POWER, 8UL, "FSK P=8", "FSK power failed"),
01284 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PWR_11, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_TX_POWER, 11UL, "FSK P=11", "FSK power failed"),
01285 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PWR_14, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_TX_POWER, 14UL, "FSK P=14", "FSK power failed"),
01286 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PWR_17, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_TX_POWER, 17UL, "FSK P=17", "FSK power failed"),
01287 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PWR_20, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_TX_POWER, 20UL, "FSK P=20", "FSK power failed"),
01288 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PREAMBLE_4, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_PREAMBLE, 4UL, "FSK PRE4", "FSK preamble failed"),
01289 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PREAMBLE_8, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_PREAMBLE, 8UL, "FSK PRE8", "FSK preamble failed"),
01290 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PREAMBLE_12, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_PREAMBLE, 12UL, "FSK PRE12", "FSK preamble failed"),
01291 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PREAMBLE_16, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_PREAMBLE, 16UL, "FSK PRE16", "FSK preamble failed"),
01292 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PREAMBLE_24, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_PREAMBLE, 24UL, "FSK PRE24", "FSK preamble failed"),
01293 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_PREAMBLE_32, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_PREAMBLE, 32UL, "FSK PRE32", "FSK preamble failed"),
01294 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SYNC_LEN_0, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SYNC_LEN, 0UL, "SyncLen 0", "SyncLen failed"),
01295 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SYNC_LEN_1, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SYNC_LEN, 1UL, "SyncLen 1", "SyncLen failed"),
01296 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SYNC_LEN_2, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SYNC_LEN, 2UL, "SyncLen 2", "SyncLen failed"),
01297 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SYNC_LEN_3, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SYNC_LEN, 3UL, "SyncLen 3", "SyncLen failed"),
01298 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SYNC_LEN_4, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SYNC_LEN, 4UL, "SyncLen 4", "SyncLen failed"),
01299 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SYNC_WORD_55AA, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SYNC_WORD, 0x55AAUL, "Sync 55AA", "Sync word failed"),
01300 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SYNC_WORD_2DD4, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SYNC_WORD, 0x2DD4UL, "Sync 2DD4", "Sync word failed"),
01301 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SYNC_WORD_A55A, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SYNC_WORD, 0xA55AUL, "Sync A55A", "Sync word failed"),
01302 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SYNC_WORD_C194C1, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SYNC_WORD, 0x00C194C1UL, "Sync C194C1", "Sync word failed"),
01303 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_SYNC_WORD_1ACFFC1D, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_SYNC_WORD, 0x1ACFFC1DUL, "Sync 1ACFFC1D", "Sync word failed"),
01304 MENU_RADIO_SET(MENU_ACTION_MOD_FSK_ADDR_NONE, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_ADDRESS_FILTER, (uint32_t)RADIO_MAIN_ADDRESS_FILTER_NONE, "Addr OFF", "Addr
```

```

    failed"),
01305     MENU_RADIO_SET(MENU_ACTION_MOD_FSK_ADDR_NODE, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_ADDRESS_FILTER, (uint32_t)RADIO_MAIN_ADDRESS_FILTER_NODE, "Addr NODE", "Addr
failed"),
01306     MENU_RADIO_SET(MENU_ACTION_MOD_FSK_ADDR_NODE_BC, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_ADDRESS_FILTER, (uint32_t)RADIO_MAIN_ADDRESS_FILTER_NODE_BROADCAST, "Addr N+BC",
"Addr failed"),
01307     MENU_RADIO_SET(MENU_ACTION_MOD_FSK_CRC_OFF, RADIO_MAIN_MODULATION_FSK, RADIO_MAIN_OPTION_FSK_CRC,
(uint32_t)RADIO_MAIN_CRC_OFF, "FSK CRC OFF", "FSK CRC failed"),
01308     MENU_RADIO_SET(MENU_ACTION_MOD_FSK_CRC_IBM, RADIO_MAIN_MODULATION_FSK, RADIO_MAIN_OPTION_FSK_CRC,
(uint32_t)RADIO_MAIN_CRC_IBM, "FSK CRC IBM", "FSK CRC failed"),
01309     MENU_RADIO_SET(MENU_ACTION_MOD_FSK_CRC_CCITT, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_CRC, (uint32_t)RADIO_MAIN_CRC_CCITT, "FSK CRC CCT", "FSK CRC failed"),
01310     MENU_RADIO_SET(MENU_ACTION_MOD_FSK_WHITEN_OFF, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_WHITENING, 0UL, "White OFF", "Whitening failed"),
01311     MENU_RADIO_SET(MENU_ACTION_MOD_FSK_WHITEN_ON, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_WHITENING, 1UL, "White ON", "Whitening failed"),
01312     MENU_RADIO_SET(MENU_ACTION_MOD_FSK_RESET, RADIO_MAIN_MODULATION_FSK,
RADIO_MAIN_OPTION_FSK_RESET_DEFAULTS, 0UL, "FSK reset", "FSK reset failed"),
01313
01314     MENU_RADIO_ACTIVATE(MENU_ACTION_MOD_OOK_ENABLE, RADIO_MAIN_MODULATION_OOK, "OOK mode", "OOK set
failed"),
01315     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_FREQ_8680, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_FREQ, 868000000UL, "OOK F=868.0", "OOK freq failed"),
01316     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_FREQ_8681, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_FREQ, 868100000UL, "OOK F=868.1", "OOK freq failed"),
01317     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_FREQ_8683, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_FREQ, 868300000UL, "OOK F=868.3", "OOK freq failed"),
01318     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_FREQ_8685, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_FREQ, 868500000UL, "OOK F=868.5", "OOK freq failed"),
01319     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_FREQ_8688, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_FREQ, 868800000UL, "OOK F=868.8", "OOK freq failed"),
01320     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_FREQ_86905, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_FREQ, 869050000UL, "OOK F=869.05", "OOK freq failed"),
01321     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_FREQ_869525, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_FREQ, 869525000UL, "OOK F=869.525", "OOK freq failed"),
01322     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BITRATE_1200, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_BITRATE, 1200UL, "OOK BR1.2", "OOK bitrate failed"),
01323     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BITRATE_2400, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_BITRATE, 2400UL, "OOK BR2.4", "OOK bitrate failed"),
01324     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BITRATE_4800, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_BITRATE, 4800UL, "OOK BR4.8", "OOK bitrate failed"),
01325     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BITRATE_9600, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_BITRATE, 9600UL, "OOK BR9.6", "OOK bitrate failed"),
01326     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BITRATE_19200, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_BITRATE, 19200UL, "OOK BR19.2", "OOK bitrate failed"),
01327     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BITRATE_38400, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_BITRATE, 38400UL, "OOK BR38.4", "OOK bitrate failed"),
01328     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BITRATE_50000, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_BITRATE, 50000UL, "OOK BR50", "OOK bitrate failed"),
01329     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BITRATE_100000, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_BITRATE, 100000UL, "OOK BR100", "OOK bitrate failed"),
01330     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PWR_2, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_TX_POWER, 2UL, "OOK P=2", "OOK power failed"),
01331     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PWR_5, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_TX_POWER, 5UL, "OOK P=5", "OOK power failed"),
01332     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PWR_8, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_TX_POWER, 8UL, "OOK P=8", "OOK power failed"),
01333     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PWR_11, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_TX_POWER, 11UL, "OOK P=11", "OOK power failed"),
01334     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PWR_14, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_TX_POWER, 14UL, "OOK P=14", "OOK power failed"),
01335     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PWR_17, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_TX_POWER, 17UL, "OOK P=17", "OOK power failed"),
01336     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PWR_20, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_TX_POWER, 20UL, "OOK P=20", "OOK power failed"),
01337     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BW_31_25, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_RX_BW, (uint32_t)RADIO_LORA_BW_31_25_KHZ, "OOK BW31.2", "OOK BW failed"),
01338     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BW_62_5, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_RX_BW, (uint32_t)RADIO_LORA_BW_62_5_KHZ, "OOK BW62.5", "OOK BW failed"),
01339     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BW_125, RADIO_MAIN_MODULATION_OOK, RADIO_MAIN_OPTION_OOK_RX_BW,
(uint32_t)RADIO_LORA_BW_125_KHZ, "OOK BW125", "OOK BW failed"),
01340     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BW_250, RADIO_MAIN_MODULATION_OOK, RADIO_MAIN_OPTION_OOK_RX_BW,
(uint32_t)RADIO_LORA_BW_250_KHZ, "OOK BW250", "OOK BW failed"),
01341     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_BW_500, RADIO_MAIN_MODULATION_OOK, RADIO_MAIN_OPTION_OOK_RX_BW,
(uint32_t)RADIO_LORA_BW_500_KHZ, "OOK BW500", "OOK BW failed"),
01342     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PREAMBLE_4, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_PREAMBLE, 4UL, "OOK PRE4", "OOK preamble failed"),
01343     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PREAMBLE_8, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_PREAMBLE, 8UL, "OOK PRE8", "OOK preamble failed"),
01344     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PREAMBLE_12, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_PREAMBLE, 12UL, "OOK PRE12", "OOK preamble failed"),
01345     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PREAMBLE_16, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_PREAMBLE, 16UL, "OOK PRE16", "OOK preamble failed"),
01346     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PREAMBLE_24, RADIO_MAIN_MODULATION_OOK,
RADIO_MAIN_OPTION_OOK_PREAMBLE, 24UL, "OOK PRE24", "OOK preamble failed"),
01347     MENU_RADIO_SET(MENU_ACTION_MOD_OOK_PREAMBLE_32, RADIO_MAIN_MODULATION_OOK,

```

```

    RADIO_MAIN_OPTION_OOK_PREAMBLE, 32UL, "OOK PRE32", "OOK preamble failed"),
01348 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_SYNC_LEN_0, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_SYNC_LEN, 0UL, "OSyncL 0", "OOK sync len failed"),
01349 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_SYNC_LEN_1, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_SYNC_LEN, 1UL, "OSyncL 1", "OOK sync len failed"),
01350 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_SYNC_LEN_2, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_SYNC_LEN, 2UL, "OSyncL 2", "OOK sync len failed"),
01351 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_SYNC_LEN_3, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_SYNC_LEN, 3UL, "OSyncL 3", "OOK sync len failed"),
01352 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_SYNC_LEN_4, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_SYNC_LEN, 4UL, "OSyncL 4", "OOK sync len failed"),
01353 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_SYNC_WORD_55AA, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_SYNC_WORD, 0x55AAUL, "OSync 55AA", "OOK sync word failed"),
01354 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_SYNC_WORD_2DD4, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_SYNC_WORD, 0x2DD4UL, "OSync 2DD4", "OOK sync word failed"),
01355 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_SYNC_WORD_A55A, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_SYNC_WORD, 0xA55AUL, "OSync A55A", "OOK sync word failed"),
01356 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_SYNC_WORD_C194C1, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_SYNC_WORD, 0x00C194C1UL, "OSync C194", "OOK sync word failed"),
01357 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_SYNC_WORD_1ACFFC1D, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_SYNC_WORD, 0x1ACFFC1DUL, "OSync 1ACF", "OOK sync word failed"),
01358 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_THRESH_FIXED, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_THRESHOLD_TYPE, (uint32_t)RADIO_MAIN_OOK_THRESHOLD_FIXED, "Thr fixed", "Thr type
failed"),
01359 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_THRESH_PEAK, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_THRESHOLD_TYPE, (uint32_t)RADIO_MAIN_OOK_THRESHOLD_PEAK, "Thr peak", "Thr type
failed"),
01360 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_THRESH_AVG, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_THRESHOLD_TYPE, (uint32_t)RADIO_MAIN_OOK_THRESHOLD_AVERAGE, "Thr avg", "Thr type
failed"),
01361 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_THRESH_4, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE, 4UL, "Thr=4", "Thr value failed"),
01362 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_THRESH_8, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE, 8UL, "Thr=8", "Thr value failed"),
01363 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_THRESH_12, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE, 12UL, "Thr=12", "Thr value failed"),
01364 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_THRESH_16, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE, 16UL, "Thr=16", "Thr value failed"),
01365 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_THRESH_24, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE, 24UL, "Thr=24", "Thr value failed"),
01366 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_THRESH_32, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE, 32UL, "Thr=32", "Thr value failed"),
01367 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_THRESH_48, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE, 48UL, "Thr=48", "Thr value failed"),
01368 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_THRESH_64, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE, 64UL, "Thr=64", "Thr value failed"),
01369 MENU_RADIO_SET(MENU_ACTION_MOD_OOK_RESET, RADIO_MAIN_MODULATION_OOK,
    RADIO_MAIN_OPTION_OOK_RESET_DEFAULTS, 0UL, "OOK reset", "OOK reset failed")
01370 };
01371
01372 #undef MENU_RADIO_ACTIVATE
01373 #undef MENU_RADIO_SET
01374 #undef MENU_RADIO_PRESET
01375
01376 static const osThreadAttr_t s_menu_task_attr =
01377 {
01378     .name = "menu_task",
01379     .priority = (osPriority_t)osPriorityLow,
01380     .stack_mem = s_menu_task_stack,
01381     .stack_size = sizeof(s_menu_task_stack),
01382     .cb_mem = &s_menu_task_cb,
01383     .cb_size = sizeof(s_menu_task_cb)
01384 };
01385
01386 void menu_main_create_task(void)
01387 {
01388     if (s_menu_notify_queue == NULL)
01389     {
01390         s_menu_notify_queue = osMessageQueueNew(MENU_NOTIFY_QUEUE_DEPTH, sizeof(menu_notification_t),
01391         NULL);
01392         if (s_menu_notify_queue == NULL)
01393         {
01394             printf("MENU: notify queue create failed\r\n");
01395             return;
01396         }
01397     }
01398     if (s_menu_task == NULL)
01399     {
01400         s_menu_task = osThreadNew(menu_main_task_fn, NULL, &s_menu_task_attr);
01401         if (s_menu_task == NULL)
01402         {
01403             printf("MENU: task create failed\r\n");
01404         }
01405     }
01406 }
01407

```

```

01408 bool menu_main_post_notification(const menu_notification_t *n)
01409 {
01410     menu_notification_t dropped;
01411
01412     if ((n == NULL) || (s_menu_notify_queue == NULL))
01413     {
01414         return false;
01415     }
01416
01417     if (osMessageQueuePut(s_menu_notify_queue, n, 0U, 0U) == osOK)
01418     {
01419         return true;
01420     }
01421
01422     if (osMessageQueueGet(s_menu_notify_queue, &dropped, NULL, 0U) == osOK)
01423     {
01424         return (osMessageQueuePut(s_menu_notify_queue, n, 0U, 0U) == osOK);
01425     }
01426
01427     return false;
01428 }
01429
01431 static void menu_main_task_fn(void *argument)
01432 {
01433     menu_state_t st;
01434     button_event_t evt;
01435     menu_notification_t notify;
01436     security_runtime_cfg_t cfg;
01437
01438     (void)argument;
01439     memset(&st, 0, sizeof(st));
01440     st.popup_enabled = true;
01441     st.selected_device_slot = MENU_DEVICE_SLOT_INVALID;
01442     st.last_input_ms = HAL_GetTick();
01443
01444     if (security_main_cmd_get_runtime_cfg(&cfg))
01445     {
01446         st.popup_enabled = (cfg.notify_mode == SECURITY_NOTIFY_POPUP);
01447     }
01448
01449     menu_enter_monitor(&st);
01450     printf("MENU: task ready\r\n");
01451
01452     for (;;)
01453     {
01454         if (button_main_get_event(&evt, 25U))
01455         {
01456             st.last_input_ms = HAL_GetTick();
01457             menu_handle_button(&st, evt);
01458         }
01459
01460         while ((s_menu_notify_queue != NULL) &&
01461             (osMessageQueueGet(s_menu_notify_queue, &notify, NULL, 0U) == osOK))
01462         {
01463             menu_handle_notification(&st, &notify);
01464         }
01465
01466         if (st.modal == MENU_MODAL_QUICK_REPLY)
01467         {
01468             uint32_t now_ms = HAL_GetTick();
01469             uint32_t remaining_ms = (st.quick_reply_deadline_ms > now_ms) ?
01470                 (st.quick_reply_deadline_ms - now_ms) : 0U;
01471             uint32_t remaining_seconds = (remaining_ms + 999U) / 1000U;
01472
01473             if (remaining_ms == 0U)
01474             {
01475                 menu_show_action_result(&st, MENU_NOTIFICATION_WARNING, "Reply timeout");
01476             }
01477             else if (remaining_seconds != st.quick_reply_last_seconds)
01478             {
01479                 st.quick_reply_last_seconds = remaining_seconds;
01480                 menu_render_quick_reply(&st);
01481             }
01482         }
01483
01484         if ((st.current_page != MENU_PAGE_NONE) &&
01485             (st.modal == MENU_MODAL_NONE) &&
01486             ((HAL_GetTick() - st.last_input_ms) >= MENU_INACTIVITY_TIMEOUT_MS))
01487         {
01488             menu_enter_monitor(&st);
01489         }
01490     }
01491 }
01492
01493 /* Returns the UI to passive monitor mode and clears transient menu state. */
01494 static void menu_enter_monitor(menu_state_t *st)
01495 {

```

```

01496     if (st == NULL)
01497     {
01498         return;
01499     }
01500
01501     st->current_page = MENU_PAGE_NONE;
01502     st->selected_idx = 0U;
01503     st->modal = MENU_MODAL_NONE;
01504     st->pending_action = MENU_ACTION_NONE;
01505     st->selected_device_slot = MENU_DEVICE_SLOT_INVALID;
01506     st->pending_target_node_id = 0U;
01507     st->send_target_action = MENU_ACTION_NONE;
01508     st->transient_parent_page = MENU_PAGE_NONE;
01509     st->pairing_network_mode = false;
01510     st->quick_reply_src_id = 0U;
01511     st->quick_reply_deadline_ms = 0U;
01512     st->quick_reply_last_seconds = 0U;
01513     st->operator_unlock_until_ms = 0UL;
01514     st->admin_unlock_until_ms = 0UL;
01515     menu_clear_tx_ack_state(st);
01516     menu_clear_pin_state(st);
01517     menu_line_clear(st->quick_reply_text);
01518     (void)lcd_main_set_mode(LCD_MODE_MONITOR);
01519 }
01520
01521 /* Opens a normal page and resets selection/modal context. */
01522 static void menu_open_page(menu_state_t *st, menu_page_id_t page_id)
01523 {
01524     const menu_page_t *page;
01525
01526     if (st == NULL)
01527     {
01528         return;
01529     }
01530
01531     st->current_page = page_id;
01532     st->selected_idx = 0U;
01533     st->modal = MENU_MODAL_NONE;
01534     st->pending_action = MENU_ACTION_NONE;
01535     st->pending_target_node_id = 0U;
01536     st->send_target_action = MENU_ACTION_NONE;
01537     st->transient_parent_page = MENU_PAGE_NONE;
01538     st->pairing_network_mode = false;
01539     st->quick_reply_src_id = 0U;
01540     st->quick_reply_deadline_ms = 0U;
01541     st->quick_reply_last_seconds = 0U;
01542     menu_clear_tx_ack_state(st);
01543     menu_line_clear(st->quick_reply_text);
01544     page = menu_get_page(page_id);
01545     if (page != NULL)
01546     {
01547         while ((st->selected_idx < page->item_count) &&
01548                !menu_item_is_selectable(st, page, st->selected_idx))
01549         {
01550             st->selected_idx++;
01551         }
01552         if ((page->item_count > 0U) && (st->selected_idx >= page->item_count))
01553         {
01554             st->selected_idx = 0U;
01555         }
01556     }
01557     menu_render(st);
01558 }
01559
01560 /* Shows an informational modal without changing the underlying page. */
01561 static void menu_open_info_modal(menu_state_t *st,
01562                                 const char *l0,
01563                                 const char *l1,
01564                                 const char *l2,
01565                                 const char *l3)
01566 {
01567     if (st == NULL)
01568     {
01569         return;
01570     }
01571
01572     menu_clear_tx_ack_state(st);
01573     st->modal = MENU_MODAL_INFO;
01574     st->pending_action = MENU_ACTION_NONE;
01575     menu_render_popup(l0, l1, l2, l3);
01576 }
01577
01579 static void menu_clear_tx_ack_state(menu_state_t *st)
01580 {
01581     if (st == NULL)
01582     {
01583         return;

```



```

01584     }
01585
01586     st->tx_ack_waiting = false;
01587     menu_line_clear(st->tx_sent_text);
01588 }
01589
01590 /* Closes the active modal and redraws the underlying page or monitor. */
01591 static void menu_close_modal(menu_state_t *st)
01592 {
01593     if (st == NULL)
01594     {
01595         return;
01596     }
01597
01598     if (st->modal == MENU_MODAL_PIN)
01599     {
01600         menu_clear_pin_state(st);
01601     }
01602     if (st->modal == MENU_MODAL_INFO)
01603     {
01604         menu_clear_tx_ack_state(st);
01605     }
01606     st->modal = MENU_MODAL_NONE;
01607     st->pending_action = MENU_ACTION_NONE;
01608
01609     if (st->current_page == MENU_PAGE_NONE)
01610     {
01611         (void)lcd_main_set_mode(LCD_MODE_MONITOR);
01612     }
01613     else
01614     {
01615         menu_render(st);
01616     }
01617 }
01618
01620 static void menu_line_clear(char *dst)
01621 {
01622     uint8_t i;
01623
01624     if (dst == NULL)
01625     {
01626         return;
01627     }
01628
01629     for (i = 0U; i < MENU_LINE_CHARS; i++)
01630     {
01631         dst[i] = ' ';
01632     }
01633     dst[MENU_LINE_CHARS] = '\0';
01634 }
01635
01637 static uint8_t menu_line_copy(char *dst, uint8_t offset, const char *src, uint8_t max_chars)
01638 {
01639     uint8_t copied = 0U;
01640
01641     if ((dst == NULL) || (src == NULL) || (offset >= MENU_LINE_CHARS))
01642     {
01643         return offset;
01644     }
01645
01646     while ((offset < MENU_LINE_CHARS) &&
01647           (copied < max_chars) &&
01648           (src[copied] != '\0'))
01649     {
01650         dst[offset++] = src[copied++];
01651     }
01652
01653     return offset;
01654 }
01655
01657 static uint8_t menu_line_append_u32(char *dst, uint8_t offset, uint32_t value)
01658 {
01659     char digits[10];
01660     uint8_t count = 0U;
01661
01662     if ((dst == NULL) || (offset >= MENU_LINE_CHARS))
01663     {
01664         return offset;
01665     }
01666
01667     do
01668     {
01669         digits[count++] = (char)('0' + (value % 10UL));
01670         value /= 10UL;
01671     } while ((value != 0UL) && (count < sizeof(digits)));
01672
01673     while ((count > 0U) && (offset < MENU_LINE_CHARS))

```

```

01674     {
01675         dst[offset++] = digits[--count];
01676     }
01677     return offset;
01678 }
01679
01680 static uint8_t menu_line_append_i32(char *dst, uint8_t offset, int32_t value)
01681 {
01682     uint32_t magnitude;
01683     if ((dst == NULL) || (offset >= MENU_LINE_CHARS))
01684     {
01685         return offset;
01686     }
01687     if (value < 0)
01688     {
01689         dst[offset++] = '-';
01690         magnitude = (uint32_t)(-(int64_t)value);
01691     }
01692     else
01693     {
01694         magnitude = (uint32_t)value;
01695     }
01696     return menu_line_append_u32(dst, offset, magnitude);
01697 }
01698
01699 static uint8_t menu_line_append_hex32(char *dst, uint8_t offset, uint32_t value)
01700 {
01701     static const char hex[] = "0123456789ABCDEF";
01702     int8_t nibble;
01703     if ((dst == NULL) || (offset >= MENU_LINE_CHARS))
01704     {
01705         return offset;
01706     }
01707     for (nibble = 7; (nibble >= 0) && (offset < MENU_LINE_CHARS); nibble--)
01708     {
01709         dst[offset++] = hex[(value >> ((uint32_t)nibble * 4UL)) & 0x0FU];
01710     }
01711     return offset;
01712 }
01713
01714 static uint8_t menu_line_append_freq_mhz(char *dst, uint8_t offset, uint32_t freq_hz)
01715 {
01716     uint32_t mhz;
01717     uint32_t frac_hz;
01718     char frac_digits[6];
01719     uint8_t frac_len = 6U;
01720     int8_t idx;
01721     if ((dst == NULL) || (offset >= MENU_LINE_CHARS))
01722     {
01723         return offset;
01724     }
01725     mhz = freq_hz / 1000000UL;
01726     frac_hz = freq_hz % 1000000UL;
01727     offset = menu_line_append_u32(dst, offset, mhz);
01728     if (frac_hz == 0UL)
01729     {
01730         return offset;
01731     }
01732     if (offset < MENU_LINE_CHARS)
01733     {
01734         dst[offset++] = '.';
01735     }
01736     for (idx = 5; idx >= 0; idx--)
01737     {
01738         frac_digits[idx] = (char)('0' + (frac_hz % 10UL));
01739         frac_hz /= 10UL;
01740     }
01741     while ((frac_len > 1U) && (frac_digits[frac_len - 1U] == '0'))
01742     {
01743         frac_len--;
01744     }
01745     for (idx = 0; (idx < (int8_t)frac_len) && (offset < MENU_LINE_CHARS); idx++)
01746     {

```



```

01764         dst[offset++] = frac_digits[idx];
01765     }
01766
01767     return offset;
01768 }
01769
01771 static void menu_line_format_u32(char *dst, const char *prefix, uint32_t value, const char *suffix)
01772 {
01773     uint8_t offset = 0U;
01774
01775     menu_line_clear(dst);
01776     offset = menu_line_copy(dst, offset, prefix, MENU_LINE_CHARS);
01777     offset = menu_line_append_u32(dst, offset, value);
01778     (void)menu_line_copy(dst, offset, suffix, MENU_LINE_CHARS);
01779 }
01780
01782 static void menu_line_format_i32(char *dst, const char *prefix, int32_t value, const char *suffix)
01783 {
01784     uint8_t offset = 0U;
01785
01786     menu_line_clear(dst);
01787     offset = menu_line_copy(dst, offset, prefix, MENU_LINE_CHARS);
01788     offset = menu_line_append_i32(dst, offset, value);
01789     (void)menu_line_copy(dst, offset, suffix, MENU_LINE_CHARS);
01790 }
01791
01793 static void menu_line_format_hex32(char *dst, const char *prefix, uint32_t value)
01794 {
01795     uint8_t offset = 0U;
01796
01797     menu_line_clear(dst);
01798     offset = menu_line_copy(dst, offset, prefix, MENU_LINE_CHARS);
01799     offset = menu_line_copy(dst, offset, "0x", 2U);
01800     (void)menu_line_append_hex32(dst, offset, value);
01801 }
01802
01804 static void menu_line_format_device_ref(char *dst, const char *prefix, uint32_t device_code)
01805 {
01806     uint8_t offset = 0U;
01807
01808     menu_line_clear(dst);
01809     offset = menu_line_copy(dst, offset, prefix, MENU_LINE_CHARS);
01810     if (device_code == LAVIET_GATEWAY_ID)
01811     {
01812         (void)menu_line_copy(dst, offset, "gateway", MENU_LINE_CHARS);
01813         return;
01814     }
01815     if (device_code == LAVIET_BROADCAST_ID)
01816     {
01817         (void)menu_line_copy(dst, offset, "broadcast", MENU_LINE_CHARS);
01818         return;
01819     }
01820
01821     offset = menu_line_copy(dst, offset, "0x", 2U);
01822     (void)menu_line_append_hex32(dst, offset, device_code);
01823 }
01824
01826 static void menu_line_format_source(char *dst, uint32_t device_code)
01827 {
01828     menu_line_format_device_ref(dst, "From ", device_code);
01829 }
01830
01832 static void menu_line_format_freq(char *dst, const char *prefix, uint32_t freq_hz)
01833 {
01834     uint8_t offset = 0U;
01835
01836     menu_line_clear(dst);
01837     offset = menu_line_copy(dst, offset, prefix, MENU_LINE_CHARS);
01838     offset = menu_line_append_freq_mhz(dst, offset, freq_hz);
01839     (void)menu_line_copy(dst, offset, " MHz", 4U);
01840 }
01841
01843 static void menu_line_format_fixed2(char *dst, const char *prefix, float value, const char *suffix)
01844 {
01845     int32_t scaled;
01846     uint32_t magnitude;
01847     uint8_t offset = 0U;
01848
01849     menu_line_clear(dst);
01850     offset = menu_line_copy(dst, offset, prefix, MENU_LINE_CHARS);
01851
01852     if (value >= 0.0f)
01853     {
01854         scaled = (int32_t)((value * 100.0f) + 0.5f);
01855     }
01856     else
01857     {

```

```

01858         scaled = (int32_t)((value * 100.0f) - 0.5f);
01859     }
01860
01861     if (scaled < 0)
01862     {
01863         magnitude = (uint32_t)(-(int64_t)scaled);
01864     }
01865     else
01866     {
01867         magnitude = (uint32_t)scaled;
01868     }
01869
01870     if (scaled < 0)
01871     {
01872         offset = menu_line_copy(dst, offset, "-", 1U);
01873     }
01874
01875     offset = menu_line_append_u32(dst, offset, magnitude / 100UL);
01876     offset = menu_line_copy(dst, offset, ".", 1U);
01877     if ((magnitude % 100UL) < 10UL)
01878     {
01879         offset = menu_line_copy(dst, offset, "0", 1U);
01880     }
01881     offset = menu_line_append_u32(dst, offset, magnitude % 100UL);
01882     (void)menu_line_copy(dst, offset, suffix, MENU_LINE_CHARS);
01883 }
01884
01886 static void menu_line_copy_or_default(char *dst, const char *text, const char *fallback)
01887 {
01888     if (dst == NULL)
01889     {
01890         return;
01891     }
01892
01893     menu_line_clear(dst);
01894
01895     if (text != NULL)
01896     {
01897         (void)menu_line_copy(dst, 0U, text, MENU_LINE_CHARS);
01898         return;
01899     }
01900
01901     if (fallback != NULL)
01902     {
01903         (void)menu_line_copy(dst, 0U, fallback, MENU_LINE_CHARS);
01904     }
01905 }
01906
01908 static const char *menu_pair_code_text(const char *text, uint8_t prefix_len)
01909 {
01910     if ((text != NULL) && (text[prefix_len] == ' '))
01911     {
01912         return &text[prefix_len + 1U];
01913     }
01914
01915     return "----";
01916 }
01917
01919 static bool menu_get_trusted_slot(uint8_t slot, trusted_info_t *info_out)
01920 {
01921     if (info_out == NULL)
01922     {
01923         return false;
01924     }
01925
01926     memset(info_out, 0, sizeof(*info_out));
01927     return (slot < MENU_TRUSTED_DEVICE_SLOTS) &&
01928         security_main_cmd_get_device(slot, info_out) &&
01929         info_out->in_use;
01930 }
01931
01933 static bool menu_gateway_slot_in_use(void)
01934 {
01935     trusted_info_t info;
01936
01937     return menu_get_trusted_slot(0U, &info) &&
01938         (info.node_id == LAVIET_GATEWAY_ID);
01939 }
01940
01942 static uint8_t menu_line_append_device_slot_name(char *dst, uint8_t offset, uint8_t slot)
01943 {
01944     if ((dst == NULL) || (offset >= MENU_LINE_CHARS))
01945     {
01946         return offset;
01947     }
01948
01949     if (slot == 0U)

```

```

01950     {
01951         return menu_line_copy(dst, offset, "G", 1U);
01952     }
01953
01954     offset = menu_line_copy(dst, offset, "D", 1U);
01955     return menu_line_append_u32(dst, offset, slot);
01956 }
01957
01959 static void menu_build_trusted_slot_label(uint8_t slot, const trusted_info_t *info, char *dst)
01960 {
01961     uint8_t offset = 0U;
01962
01963     if ((info == NULL) || (dst == NULL))
01964     {
01965         return;
01966     }
01967
01968     menu_line_clear(dst);
01969     offset = menu_line_append_device_slot_name(dst, offset, slot);
01970     offset = menu_line_copy(dst, offset, " ", 1U);
01971     if (info->node_id == LAVIET_GATEWAY_ID)
01972     {
01973         (void)menu_line_copy(dst, offset, "gateway", 7U);
01974     }
01975     else if (info->node_id == LAVIET_BROADCAST_ID)
01976     {
01977         (void)menu_line_copy(dst, offset, "broadcast", 9U);
01978     }
01979     else
01980     {
01981         offset = menu_line_copy(dst, offset, "0x", 2U);
01982         (void)menu_line_append_hex32(dst, offset, info->node_id);
01983     }
01984 }
01985
01987 static void menu_build_empty_trusted_slot_label(uint8_t slot, char *dst)
01988 {
01989     uint8_t offset = 0U;
01990
01991     if (dst == NULL)
01992     {
01993         return;
01994     }
01995
01996     menu_line_clear(dst);
01997     offset = menu_line_append_device_slot_name(dst, offset, slot);
01998     (void)menu_line_copy(dst, offset, " (empty)", 8U);
01999 }
02000
02002 static const char *menu_lora_bw_text(radio_lora_bw_t bw)
02003 {
02004     if (bw == RADIO_LORA_BW_7_8_KHZ)
02005     {
02006         return "7.8";
02007     }
02008     if (bw == RADIO_LORA_BW_10_4_KHZ)
02009     {
02010         return "10.4";
02011     }
02012     if (bw == RADIO_LORA_BW_15_6_KHZ)
02013     {
02014         return "15.6";
02015     }
02016     if (bw == RADIO_LORA_BW_20_8_KHZ)
02017     {
02018         return "20.8";
02019     }
02020     if (bw == RADIO_LORA_BW_31_25_KHZ)
02021     {
02022         return "31.25";
02023     }
02024     if (bw == RADIO_LORA_BW_41_7_KHZ)
02025     {
02026         return "41.7";
02027     }
02028     if (bw == RADIO_LORA_BW_62_5_KHZ)
02029     {
02030         return "62.5";
02031     }
02032     if (bw == RADIO_LORA_BW_125_KHZ)
02033     {
02034         return "125";
02035     }
02036     if (bw == RADIO_LORA_BW_250_KHZ)
02037     {
02038         return "250";
02039     }

```

```
02040     if (bw == RADIO_LORA_BW_500_KHZ)
02041     {
02042         return "500";
02043     }
02044     return "?";
02045 }
02046
02047 static const char *menu_lora_cr_text(uint8_t denominator)
02048 {
02049     if (denominator == 5U)
02050     {
02051         return "4/5";
02052     }
02053     if (denominator == 6U)
02054     {
02055         return "4/6";
02056     }
02057     if (denominator == 7U)
02058     {
02059         return "4/7";
02060     }
02061     if (denominator == 8U)
02062     {
02063         return "4/8";
02064     }
02065     return "?";
02066 }
02067
02068 static const char *menu_fsk_shape_text(radio_main_fsk_shaping_t shaping)
02069 {
02070     if (shaping == RADIO_MAIN_FSK_SHAPING_GFSK)
02071     {
02072         return "GFSK";
02073     }
02074     if (shaping == RADIO_MAIN_FSK_SHAPING_MSK)
02075     {
02076         return "MSK";
02077     }
02078     if (shaping == RADIO_MAIN_FSK_SHAPING_GMSK)
02079     {
02080         return "GMSK";
02081     }
02082     return "FSK";
02083 }
02084
02085 static const char *menu_filter_text(radio_main_filter_t filter)
02086 {
02087     if (filter == RADIO_MAIN_FILTER_BT_10)
02088     {
02089         return "BT1.0";
02090     }
02091     if (filter == RADIO_MAIN_FILTER_BT_07)
02092     {
02093         return "BT0.7";
02094     }
02095     if (filter == RADIO_MAIN_FILTER_BT_05)
02096     {
02097         return "BT0.5";
02098     }
02099     if (filter == RADIO_MAIN_FILTER_BT_03)
02100     {
02101         return "BT0.3";
02102     }
02103     return "OFF";
02104 }
02105
02106 static const char *menu_crc_text(radio_main_crc_type_t crc_type)
02107 {
02108     if (crc_type == RADIO_MAIN_CRC_SX1276)
02109     {
02110         return "SX";
02111     }
02112     if (crc_type == RADIO_MAIN_CRC_IBM)
02113     {
02114         return "IBM";
02115     }
02116     if (crc_type == RADIO_MAIN_CRC_CCITT)
02117     {
02118         return "CCITT";
02119     }
02120     return "OFF";
02121 }
```

```

02131
02133 static const char *menu_ook_threshold_text(radio_main_ook_threshold_t threshold)
02134 {
02135     if (threshold == RADIO_MAIN_OOK_THRESHOLD_PEAK)
02136     {
02137         return "Peak";
02138     }
02139     if (threshold == RADIO_MAIN_OOK_THRESHOLD_AVERAGE)
02140     {
02141         return "Avg";
02142     }
02143     return "Fix";
02144 }
02145
02148 static const char *menu_address_filter_text(radio_main_address_filter_t filter)
02149 {
02150     if (filter == RADIO_MAIN_ADDRESS_FILTER_NODE)
02151     {
02152         return "NODE";
02153     }
02154     if (filter == RADIO_MAIN_ADDRESS_FILTER_NODE_BROADCAST)
02155     {
02156         return "NODE+BC";
02157     }
02158     return "OFF";
02159 }
02160
02163 static uint8_t menu_radio_settings_item_count(const radio_main_runtime_cfg_t *cfg)
02164 {
02165     if (cfg == NULL)
02166     {
02167         return 1U;
02168     }
02169     if (cfg->active_modulation == RADIO_MAIN_MODULATION_FSK)
02170     {
02171         return 14U;
02172     }
02173     if (cfg->active_modulation == RADIO_MAIN_MODULATION_OOK)
02174     {
02175         return 12U;
02176     }
02177     return 13U;
02178 }
02179
02184 static void menu_build_radio_settings_item(const radio_main_runtime_cfg_t *cfg,
02185                                           uint8_t item_idx,
02186                                           char *dst)
02187 {
02188     uint8_t offset = 0U;
02189     menu_line_clear(dst);
02190     if ((cfg == NULL) || (dst == NULL))
02191     {
02192         return;
02193     }
02194     if (cfg->active_modulation == RADIO_MAIN_MODULATION_FSK)
02195     {
02196         if (item_idx == 0U)
02197         {
02198             (void)menu_line_copy(dst, 0U, "Mode FSK/GxSK", MENU_LINE_CHARS);
02199         }
02200         else if (item_idx == 1U)
02201         {
02202             menu_line_format_freq(dst, "Freq ", cfg->fsk.frequency_hz);
02203         }
02204         else if (item_idx == 2U)
02205         {
02206             menu_line_format_i32(dst, "Power ", cfg->fsk.tx_power_dbm, " dBm");
02207         }
02208         else if (item_idx == 3U)
02209         {
02210             offset = menu_line_copy(dst, 0U, "Shape ", MENU_LINE_CHARS);
02211             (void)menu_line_copy(dst, offset, menu_fsk_shape_text(cfg->fsk.shaping), 10U);
02212         }
02213         else if (item_idx == 4U)
02214         {
02215             menu_line_format_u32(dst, "Bitrate ", cfg->fsk.bitrate_bps, " bps");
02216         }
02217         else if (item_idx == 5U)
02218         {
02219             offset = menu_line_copy(dst, 0U, "RX BW ", MENU_LINE_CHARS);
02220         }
02221     }

```

```

02222         offset = menu_line_copy(dst, offset, menu_lora_bw_text(cfg->fsk.rx_bandwidth), 8U);
02223         (void)menu_line_copy(dst, offset, " kHz", 4U);
02224     }
02225     else if (item_idx == 6U)
02226     {
02227         offset = menu_line_copy(dst, 0U, "Filter ", MENU_LINE_CHARS);
02228         (void)menu_line_copy(dst, offset, menu_filter_text(cfg->fsk.filter), 8U);
02229     }
02230     else if (item_idx == 7U)
02231     {
02232         menu_line_format_u32(dst, "Preamble ", cfg->fsk.preamble_len, " B");
02233     }
02234     else if (item_idx == 8U)
02235     {
02236         menu_line_format_u32(dst, "Sync len ", cfg->fsk.sync_word_len, " B");
02237     }
02238     else if (item_idx == 9U)
02239     {
02240         menu_line_format_hex32(dst, "Sync ", (uint32_t)cfg->fsk.sync_word);
02241     }
02242     else if (item_idx == 10U)
02243     {
02244         offset = menu_line_copy(dst, 0U, "Addr ", MENU_LINE_CHARS);
02245         (void)menu_line_copy(dst, offset, menu_address_filter_text(cfg->fsk.address_filter), 10U);
02246     }
02247     else if (item_idx == 11U)
02248     {
02249         offset = menu_line_copy(dst, 0U, "CRC ", MENU_LINE_CHARS);
02250         (void)menu_line_copy(dst, offset, menu_crc_text(cfg->fsk.crc_type), 10U);
02251     }
02252     else if (item_idx == 12U)
02253     {
02254         offset = menu_line_copy(dst, 0U, "Whitening ", MENU_LINE_CHARS);
02255         (void)menu_line_copy(dst, offset, cfg->fsk.data_whitening ? "ON" : "OFF", 3U);
02256     }
02257     return;
02258 }
02259
02260 if (cfg->active_modulation == RADIO_MAIN_MODULATION_OOK)
02261 {
02262     if (item_idx == 0U)
02263     {
02264         (void)menu_line_copy(dst, 0U, "Mode OOK", MENU_LINE_CHARS);
02265     }
02266     else if (item_idx == 1U)
02267     {
02268         menu_line_format_freq(dst, "Freq ", cfg->ook.frequency_hz);
02269     }
02270     else if (item_idx == 2U)
02271     {
02272         menu_line_format_i32(dst, "Power ", cfg->ook.tx_power_dbm, " dBm");
02273     }
02274     else if (item_idx == 3U)
02275     {
02276         menu_line_format_u32(dst, "Bitrate ", cfg->ook.bitrate_bps, " bps");
02277     }
02278     else if (item_idx == 4U)
02279     {
02280         offset = menu_line_copy(dst, 0U, "RX BW ", MENU_LINE_CHARS);
02281         offset = menu_line_copy(dst, offset, menu_lora_bw_text(cfg->ook.rx_bandwidth), 8U);
02282         (void)menu_line_copy(dst, offset, " kHz", 4U);
02283     }
02284     else if (item_idx == 5U)
02285     {
02286         menu_line_format_u32(dst, "Preamble ", cfg->ook.preamble_len, " B");
02287     }
02288     else if (item_idx == 6U)
02289     {
02290         menu_line_format_u32(dst, "Sync len ", cfg->ook.sync_word_len, " B");
02291     }
02292     else if (item_idx == 7U)
02293     {
02294         menu_line_format_hex32(dst, "Sync ", cfg->ook.sync_word);
02295     }
02296     else if (item_idx == 8U)
02297     {
02298         offset = menu_line_copy(dst, 0U, "Thresh ", MENU_LINE_CHARS);
02299         (void)menu_line_copy(dst, offset, menu_ook_threshold_text(cfg->ook.threshold), 8U);
02300     }
02301     else if (item_idx == 9U)
02302     {
02303         menu_line_format_u32(dst, "Thr value ", cfg->ook.threshold_value, "");
02304     }
02305     else if (item_idx == 10U)
02306     {
02307         offset = menu_line_copy(dst, 0U, "Coding ", MENU_LINE_CHARS);
02308         (void)menu_line_copy(dst, offset, cfg->coding_enabled ? "ON" : "OFF", 3U);

```

```

02309     }
02310     return;
02311 }
02312
02313 if (item_idx == 0U)
02314 {
02315     (void)menu_line_copy(dst, 0U, "Mode LoRa", MENU_LINE_CHARS);
02316 }
02317 else if (item_idx == 1U)
02318 {
02319     menu_line_format_freq(dst, "Freq ", cfg->lora.frequency_hz);
02320 }
02321 else if (item_idx == 2U)
02322 {
02323     menu_line_format_i32(dst, "Power ", cfg->lora.tx_power_dbm, " dBm");
02324 }
02325 else if (item_idx == 3U)
02326 {
02327     offset = menu_line_copy(dst, 0U, "BW ", MENU_LINE_CHARS);
02328     offset = menu_line_copy(dst, offset, menu_lora_bw_text(cfg->lora.bandwidth), 8U);
02329     (void)menu_line_copy(dst, offset, " kHz", 4U);
02330 }
02331 else if (item_idx == 4U)
02332 {
02333     menu_line_format_u32(dst, "SF ", cfg->lora.spreading_factor, "");
02334 }
02335 else if (item_idx == 5U)
02336 {
02337     offset = menu_line_copy(dst, 0U, "CR ", MENU_LINE_CHARS);
02338     (void)menu_line_copy(dst, offset, menu_lora_cr_text(cfg->lora.coding_rate), 6U);
02339 }
02340 else if (item_idx == 6U)
02341 {
02342     offset = menu_line_copy(dst, 0U, "CRC ", MENU_LINE_CHARS);
02343     (void)menu_line_copy(dst, offset, cfg->lora.crc_on ? "ON" : "OFF", 3U);
02344 }
02345 else if (item_idx == 7U)
02346 {
02347     menu_line_format_u32(dst, "Preamble ", cfg->lora.preamble_len, " sym");
02348 }
02349 else if (item_idx == 8U)
02350 {
02351     offset = menu_line_copy(dst, 0U, "Header ", MENU_LINE_CHARS);
02352     (void)menu_line_copy(dst, offset,
02353                          cfg->lora.implicit_header ? "Implicit" : "Explicit",
02354                          8U);
02355 }
02356 else if (item_idx == 9U)
02357 {
02358     offset = menu_line_copy(dst, 0U, "I/Q ", MENU_LINE_CHARS);
02359     (void)menu_line_copy(dst, offset, cfg->lora.invert_iq ? "Invert" : "Normal", 6U);
02360 }
02361 else if (item_idx == 10U)
02362 {
02363     menu_line_format_hex32(dst, "Sync ", cfg->lora.sync_word);
02364 }
02365 else if (item_idx == 11U)
02366 {
02367     offset = menu_line_copy(dst, 0U, "Code ", MENU_LINE_CHARS);
02368     offset = menu_line_copy(dst, offset, cfg->coding_enabled ? "ON" : "OFF", 3U);
02369     offset = menu_line_copy(dst, offset, " FH ", 4U);
02370     (void)menu_line_copy(dst, offset, cfg->fh_enabled ? "ON" : "OFF", 3U);
02371 }
02372 }
02373
02375 static void menu_build_page_title(const menu_state_t *st, const menu_page_t *page, char *dst)
02376 {
02377     menu_line_clear(dst);
02378     if ((st == NULL) || (page == NULL) || (dst == NULL))
02379     {
02380         return;
02381     }
02382     (void)menu_line_copy(dst, 0U, page->title, MENU_LINE_CHARS);
02383 }
02384
02387 static void menu_build_item_label(const menu_state_t *st,
02388                                  const menu_page_t *page,
02389                                  uint8_t item_idx,
02390                                  char *dst)
02391 {
02392     radio_main_runtime_cfg_t radio_cfg;
02393     trusted_info_t info;
02394     uint32_t period_ms;
02395     uint8_t offset = 0U;
02396
02397     menu_line_clear(dst);

```

```
02398     if ((st == NULL) || (page == NULL) || (dst == NULL))
02399     {
02400         return;
02401     }
02402
02403     if ((st->current_page == MENU_PAGE_DEVICE_DELETE_LIST) &&
02404         (item_idx < MENU_TRUSTED_DEVICE_SLOTS))
02405     {
02406         memset(&info, 0, sizeof(info));
02407         if (menu_get_trusted_slot(item_idx, &info))
02408         {
02409             menu_build_trusted_slot_label(item_idx, &info, dst);
02410         }
02411         else
02412         {
02413             menu_build_empty_trusted_slot_label(item_idx, dst);
02414         }
02415         return;
02416     }
02417
02418     if ((st->current_page == MENU_PAGE_SEND_DIRECT_LIST) &&
02419         (item_idx < MENU_TRUSTED_DEVICE_SLOTS))
02420     {
02421         memset(&info, 0, sizeof(info));
02422         if (menu_get_trusted_slot(item_idx, &info))
02423         {
02424             menu_build_trusted_slot_label(item_idx, &info, dst);
02425         }
02426         else
02427         {
02428             menu_build_empty_trusted_slot_label(item_idx, dst);
02429         }
02430         return;
02431     }
02432
02433     if ((st->current_page == MENU_PAGE_SEND_TARGET_LIST) &&
02434         (item_idx < MENU_TRUSTED_DEVICE_SLOTS))
02435     {
02436         memset(&info, 0, sizeof(info));
02437         if (menu_get_trusted_slot(item_idx, &info))
02438         {
02439             menu_build_trusted_slot_label(item_idx, &info, dst);
02440         }
02441         else
02442         {
02443             menu_build_empty_trusted_slot_label(item_idx, dst);
02444         }
02445         return;
02446     }
02447
02448     if ((st->current_page == MENU_PAGE_PIN_SETTINGS) && (item_idx == 0U))
02449     {
02450         (void)menu_line_copy(dst, 0U, s_pin_enabled ? "PIN ON" : "PIN OFF", MENU_LINE_CHARS);
02451         return;
02452     }
02453
02454     if ((st->current_page == MENU_PAGE_RADIO_SETTINGS) &&
02455         radio_main_get_runtime_cfg(&radio_cfg))
02456     {
02457         uint8_t item_count = menu_radio_settings_item_count(&radio_cfg);
02458
02459         if (item_idx < item_count)
02460         {
02461             if (item_idx == (uint8_t)(item_count - 1U))
02462             {
02463                 (void)menu_line_copy(dst, 0U, "Back", MENU_LINE_CHARS);
02464             }
02465             else
02466             {
02467                 menu_build_radio_settings_item(&radio_cfg, item_idx, dst);
02468             }
02469         }
02470         return;
02471     }
02472
02473     if ((st->current_page == MENU_PAGE_DEVICE_DELETE_ACTION) &&
02474         (item_idx == 0U))
02475     {
02476         memset(&info, 0, sizeof(info));
02477         if ((st->selected_device_slot != MENU_DEVICE_SLOT_INVALID) &&
02478             menu_get_trusted_slot(st->selected_device_slot, &info))
02479         {
02480             menu_build_trusted_slot_label(st->selected_device_slot, &info, dst);
02481         }
02482         else
02483         {
02484             (void)menu_line_copy(dst, offset, "No device selected", 18U);
```



```

02485     }
02486     return;
02487 }
02488
02489 if ((st->current_page == MENU_PAGE_SECURITY_AUTOPING) &&
02490     (item_idx == 0U))
02491 {
02492     if (radio_main_get_runtime_cfg(&radio_cfg))
02493     {
02494         period_ms = radio_cfg.auto_ping_period_ms;
02495         offset = menu_line_copy(dst, offset, "Period ", 7U);
02496         offset = menu_line_append_u32(dst, offset, period_ms);
02497         offset = menu_line_copy(dst, offset, " ms ", 4U);
02498         (void)menu_line_copy(dst,
02499                             offset,
02500                             (radio_cfg.auto_ping_mode == RADIO_MAIN_AUTO_PING_RAW) ? "RAW" :
02501                             "FRM",
02502                             3U);
02503     }
02504     else
02505     {
02506         (void)menu_line_copy(dst, offset, "Period unavailable", 18U);
02507     }
02508     return;
02509 }
02510
02511 if ((item_idx < page->item_count) && (page->items[item_idx].label != NULL))
02512 {
02513     (void)menu_line_copy(dst, 0U, page->items[item_idx].label, MENU_LINE_CHARS);
02514 }
02515 }
02516
02517 static void menu_render(menu_state_t *st)
02518 {
02519     char screen[MENU_DISPLAY_ROWS][MENU_LINE_BUF_SIZE];
02520     char label[MENU_LINE_BUF_SIZE];
02521     const menu_page_t *page;
02522     uint8_t row;
02523     uint8_t start_idx;
02524
02525     if (st == NULL)
02526     {
02527         return;
02528     }
02529
02530     page = menu_get_page(st->current_page);
02531     if (page == NULL)
02532     {
02533         return;
02534     }
02535
02536     for (row = 0U; row < MENU_DISPLAY_ROWS; row++)
02537     {
02538         menu_line_clear(screen[row]);
02539     }
02540
02541     menu_build_page_title(st, page, screen[0]);
02542
02543     start_idx = (uint8_t)((st->selected_idx / MENU_ITEMS_VISIBLE) * MENU_ITEMS_VISIBLE);
02544     for (row = 0U; row < MENU_ITEMS_VISIBLE; row++)
02545     {
02546         uint8_t item_idx = (uint8_t)(start_idx + row);
02547         uint8_t dst_row = (uint8_t)(row + 1U);
02548
02549         if (item_idx < page->item_count)
02550         {
02551             screen[dst_row][0] = ' ';
02552             if (item_idx == st->selected_idx)
02553             {
02554                 screen[dst_row][0] = '>';
02555             }
02556
02557             menu_build_item_label(st, page, item_idx, label);
02558
02559             (void)menu_line_copy(screen[dst_row],
02560                                 1U,
02561                                 label,
02562                                 (uint8_t)(MENU_LINE_CHARS - 1U));
02563         }
02564     }
02565
02566     (void)lcd_main_show_menu(screen[0], screen[1], screen[2], screen[3]);
02567 }
02568
02569 static void menu_render_popup(const char *l0, const char *l1, const char *l2, const char *l3)
02570 {
02571     char line0[MENU_LINE_BUF_SIZE];
02572

```

```

02573     char line1[MENU_LINE_BUF_SIZE];
02574     char line2[MENU_LINE_BUF_SIZE];
02575     char line3[MENU_LINE_BUF_SIZE];
02576
02577     menu_line_copy_or_default(line0, 10, "");
02578     menu_line_copy_or_default(line1, 11, "");
02579     menu_line_copy_or_default(line2, 12, "");
02580     menu_line_copy_or_default(line3, 13, "");
02581
02582     (void)lcd_main_show_popup(line0, line1, line2, line3);
02583 }
02584
02586 static void menu_render_quick_reply(menu_state_t *st)
02587 {
02588     char line1[MENU_LINE_BUF_SIZE];
02589     char line2[MENU_LINE_BUF_SIZE];
02590     char line3[MENU_LINE_BUF_SIZE];
02591
02592     if (st == NULL)
02593     {
02594         return;
02595     }
02596
02597     menu_line_format_source(line1, st->quick_reply_src_id);
02598     menu_line_copy_or_default(line2, "UP=no OK=ok DN=yes", "");
02599     menu_line_format_u32(line3, "Reply in ", st->quick_reply_last_seconds, " s");
02600     menu_render_popup(st->quick_reply_text, line1, line2, line3);
02601 }
02602
02604 static void menu_render_pin(menu_state_t *st)
02605 {
02606     char line1[MENU_LINE_BUF_SIZE];
02607     const char *title = "OPERATOR PIN";
02608     uint8_t offset;
02609     uint8_t i;
02610
02611     if (st == NULL)
02612     {
02613         return;
02614     }
02615
02616     if (st->pending_auth_level == MENU_AUTH_ADMIN)
02617     {
02618         title = "ADMIN PIN";
02619     }
02620     if (st->pending_action == MENU_ACTION_PIN_SET_OPERATOR)
02621     {
02622         title = "NEW OPERATOR PIN";
02623     }
02624     else if (st->pending_action == MENU_ACTION_PIN_SET_ADMIN)
02625     {
02626         title = "NEW ADMIN PIN";
02627     }
02628
02629     menu_line_clear(line1);
02630     offset = menu_line_copy(line1, 0U, "PIN: ", 5U);
02631     for (i = 0U; i < MENU_SETTINGS_PIN_LEN; i++)
02632     {
02633         char digit = '_';
02634
02635         if (i < st->pin_index)
02636         {
02637             digit = '*';
02638         }
02639         else if (i == st->pin_index)
02640         {
02641             digit = (char)('0' + st->pin_digits[i]);
02642         }
02643
02644         if (offset < MENU_LINE_CHARS)
02645         {
02646             line1[offset++] = digit;
02647         }
02648         if (offset < MENU_LINE_CHARS)
02649         {
02650             line1[offset++] = ' ';
02651         }
02652     }
02653
02654     menu_render_popup(title, line1, "UP/DN digit", "OK=next HOLD=cancel");
02655 }
02656
02658 static menu_auth_level_t menu_page_auth_level(menu_page_id_t page_id)
02659 {
02660     switch (page_id)
02661     {
02662         case MENU_PAGE_PAGER:

```

```

02663         case MENU_PAGE_RADIO_SETTINGS:
02664         case MENU_PAGE_SEND_OPTIONS:
02665         case MENU_PAGE_SEND_DIRECT_LIST:
02666         case MENU_PAGE_SEND_TARGET_LIST:
02667         case MENU_PAGE_MSG_GROUPS:
02668         case MENU_PAGE_GROUP_ALERT:
02669         case MENU_PAGE_GROUP_STATUS:
02670         case MENU_PAGE_GROUP_SERVICE:
02671         case MENU_PAGE_GROUP_QUICK:
02672             return MENU_AUTH_OPERATOR;
02673
02674         case MENU_PAGE_MAIN:
02675         case MENU_PAGE_DEVICES:
02676         case MENU_PAGE_DEVICE_DELETE_LIST:
02677         case MENU_PAGE_DEVICE_DELETE_ACTION:
02678         case MENU_PAGE_PIN_SETTINGS:
02679         case MENU_PAGE_SECURITY:
02680         case MENU_PAGE_SECURITY_FH:
02681         case MENU_PAGE_SECURITY_CODING:
02682         case MENU_PAGE_SECURITY_NOTIFY:
02683         case MENU_PAGE_SECURITY_AUTOPING:
02684         case MENU_PAGE_HARDWARE:
02685         case MENU_PAGE_HARDWARE_LED:
02686         case MENU_PAGE_MODULATION:
02687         case MENU_PAGE_MOD_LORA:
02688         case MENU_PAGE_MOD_LORA_FREQ:
02689         case MENU_PAGE_MOD_LORA_BW:
02690         case MENU_PAGE_MOD_LORA_SF:
02691         case MENU_PAGE_MOD_LORA_CR:
02692         case MENU_PAGE_MOD_LORA_POWER:
02693         case MENU_PAGE_MOD_LORA_CRC:
02694         case MENU_PAGE_MOD_LORA_PREAMBLE:
02695         case MENU_PAGE_MOD_LORA_HEADER:
02696         case MENU_PAGE_MOD_LORA_IQ:
02697         case MENU_PAGE_MOD_LORA_SYNC:
02698         case MENU_PAGE_MOD_FSK:
02699         case MENU_PAGE_MOD_FSK_SHAPING:
02700         case MENU_PAGE_MOD_FSK_FREQ:
02701         case MENU_PAGE_MOD_FSK_BITRATE:
02702         case MENU_PAGE_MOD_FSK_BW:
02703         case MENU_PAGE_MOD_FSK_FILTER:
02704         case MENU_PAGE_MOD_FSK_POWER:
02705         case MENU_PAGE_MOD_FSK_PREAMBLE:
02706         case MENU_PAGE_MOD_FSK_SYNC_LEN:
02707         case MENU_PAGE_MOD_FSK_SYNC_WORD:
02708         case MENU_PAGE_MOD_FSK_ADDR:
02709         case MENU_PAGE_MOD_FSK_CRC:
02710         case MENU_PAGE_MOD_FSK_WHITEN:
02711         case MENU_PAGE_MOD_OOK:
02712         case MENU_PAGE_MOD_OOK_FREQ:
02713         case MENU_PAGE_MOD_OOK_BITRATE:
02714         case MENU_PAGE_MOD_OOK_POWER:
02715         case MENU_PAGE_MOD_OOK_BW:
02716         case MENU_PAGE_MOD_OOK_PREAMBLE:
02717         case MENU_PAGE_MOD_OOK_SYNC_LEN:
02718         case MENU_PAGE_MOD_OOK_SYNC_WORD:
02719         case MENU_PAGE_MOD_OOK_THRESH_TYPE:
02720         case MENU_PAGE_MOD_OOK_THRESH_VALUE:
02721         case MENU_PAGE_INFO:
02722             return MENU_AUTH_ADMIN;
02723
02724         default:
02725             return MENU_AUTH_NONE;
02726     }
02727 }
02728
02730 static bool menu_auth_unlocked(const menu_state_t *st, menu_auth_level_t auth_level)
02731 {
02732     uint32_t until_ms = 0UL;
02733
02734     if (!s_pin_enabled)
02735     {
02736         return true;
02737     }
02738
02739     if ((st == NULL) || (auth_level == MENU_AUTH_NONE))
02740     {
02741         return (auth_level == MENU_AUTH_NONE);
02742     }
02743
02744     if (auth_level == MENU_AUTH_ADMIN)
02745     {
02746         until_ms = st->admin_unlock_until_ms;
02747     }
02748     else
02749     {
02750         until_ms = st->operator_unlock_until_ms;

```

```
02751     }
02752
02753     if (until_ms == 0UL)
02754     {
02755         return false;
02756     }
02757
02758     return ((int32_t)(until_ms - HAL_GetTick()) > 0);
02759 }
02760
02761 static bool menu_pin_matches(const menu_state_t *st)
02762 {
02763     const uint8_t *pin_ref = s_operator_pin;
02764     uint8_t diff = 0U;
02765     uint8_t i;
02766
02767     if (st == NULL)
02768     {
02769         return false;
02770     }
02771
02772     if (st->pending_auth_level == MENU_AUTH_ADMIN)
02773     {
02774         pin_ref = s_admin_pin;
02775     }
02776
02777     for (i = 0U; i < MENU_SETTINGS_PIN_LEN; i++)
02778     {
02779         diff |= (uint8_t)(st->pin_digits[i] ^ pin_ref[i]);
02780     }
02781
02782     return (diff == 0U);
02783 }
02784
02785 static bool menu_is_pin_change_action(menu_action_t action)
02786 {
02787     return ((action == MENU_ACTION_PIN_SET_OPERATOR) ||
02788             (action == MENU_ACTION_PIN_SET_ADMIN));
02789 }
02790
02791 static void menu_clear_pin_state(menu_state_t *st)
02792 {
02793     if (st == NULL)
02794     {
02795         return;
02796     }
02797
02798     memset(st->pin_digits, 0, sizeof(st->pin_digits));
02799     st->pin_index = 0U;
02800     st->pending_auth_page = MENU_PAGE_NONE;
02801     st->pending_auth_level = MENU_AUTH_NONE;
02802 }
02803
02804 static void menu_start_pin_change(menu_state_t *st, menu_action_t action)
02805 {
02806     if ((st == NULL) || !menu_is_pin_change_action(action))
02807     {
02808         return;
02809     }
02810
02811     st->modal = MENU_MODAL_PIN;
02812     st->pending_action = action;
02813     st->pending_auth_page = MENU_PAGE_NONE;
02814     st->pending_auth_level = MENU_AUTH_NONE;
02815     memset(st->pin_digits, 0, sizeof(st->pin_digits));
02816     st->pin_index = 0U;
02817     menu_render_pin(st);
02818 }
02819
02820 static void menu_request_pin(menu_state_t *st, menu_page_id_t target_page, menu_auth_level_t
auth_level)
02821 {
02822     if (st == NULL)
02823     {
02824         return;
02825     }
02826
02827     st->modal = MENU_MODAL_PIN;
02828     st->pending_auth_page = target_page;
02829     st->pending_auth_level = auth_level;
02830     memset(st->pin_digits, 0, sizeof(st->pin_digits));
02831     st->pin_index = 0U;
02832     menu_render_pin(st);
02833 }
02834
02835 static bool menu_should_open_quick_reply(const char *text)
02836 {
```

```

02843     uint8_t len;
02844     char last;
02845
02846     if (text == NULL)
02847     {
02848         return false;
02849     }
02850
02851     len = 0U;
02852     while ((len < MENU_LINE_CHARS) && (text[len] != '\0'))
02853     {
02854         len++;
02855     }
02856
02857     while ((len > 0U) && (text[len - 1U] == ' '))
02858     {
02859         len--;
02860     }
02861
02862     if (len == 0U)
02863     {
02864         return false;
02865     }
02866
02867     last = text[len - 1U];
02868     return ((last == '.') || (last == '?') || (last == '!'));
02869 }
02870
02871 static bool menu_modal_is_preemptible(menu_modal_t modal)
02872 {
02873     if (modal == MENU_MODAL_INFO)
02874     {
02875         return true;
02876     }
02877
02878     if (modal == MENU_MODAL_QUICK_REPLY)
02879     {
02880         return true;
02881     }
02882
02883     return false;
02884 }
02885
02886 static void menu_show_action_result(menu_state_t *st, menu_notification_type_t type, const char *text)
02887 {
02888     const char *title = "NOTICE";
02889     char body[MENU_LINE_BUF_SIZE];
02890
02891     if (type == MENU_NOTIFICATION_WARNING)
02892     {
02893         title = "WARNING";
02894     }
02895     else if (type == MENU_NOTIFICATION_ERROR)
02896     {
02897         title = "ERROR";
02898     }
02899     else if (type == MENU_NOTIFICATION_SECURITY)
02900     {
02901         title = "SECURITY";
02902     }
02903     else if (type == MENU_NOTIFICATION_DELIVERY)
02904     {
02905         title = "DELIVERED";
02906     }
02907     else if (type == MENU_NOTIFICATION_PAIRING)
02908     {
02909         title = "PAIRING";
02910     }
02911
02912     menu_line_copy_or_default(body, text, "(empty)");
02913     menu_open_info_modal(st, title, body, "Any key=back", "");
02914 }
02915
02916 static void menu_show_ok_or_error(menu_state_t *st, bool ok, const char *ok_text, const char
*err_text)
02917 {
02918     char selected[MENU_LINE_BUF_SIZE];
02919
02920     if (ok)
02921     {
02922         menu_line_copy_or_default(selected, ok_text, "");
02923         menu_show_action_result(st, MENU_NOTIFICATION_SECURITY, selected);
02924     }
02925
02926     menu_line_copy_or_default(selected, err_text, "");
02927     menu_show_action_result(st, MENU_NOTIFICATION_ERROR, selected);
02928 }

```

```

02932 }
02933
02935 static void menu_show_send_result(menu_state_t *st, bool ok, const char *sent_text)
02936 {
02937     char line0[MENU_LINE_BUF_SIZE];
02938     char line1[MENU_LINE_BUF_SIZE];
02939     char radio_error[MENU_LINE_BUF_SIZE];
02940
02941     if (!ok)
02942     {
02943         menu_line_clear(radio_error);
02944         if (radio_main_get_last_error_text(radio_error, (uint8_t)sizeof(radio_error)) &&
02945             (radio_error[0] != '\0'))
02946         {
02947             menu_show_action_result(st, MENU_NOTIFICATION_ERROR, radio_error);
02948         }
02949         else
02950         {
02951             menu_show_action_result(st, MENU_NOTIFICATION_ERROR, "Send failed");
02952         }
02953         return;
02954     }
02955
02956     menu_line_copy_or_default(line0, sent_text, "Message sent");
02957     menu_line_copy_or_default(line1, "Waiting for ACK", "");
02958     menu_open_info_modal(st, "TX SENT", line0, line1, "Any key=back");
02959     menu_line_copy_or_default(st->tx_sent_text, line0, "");
02960     st->tx_ack_waiting = true;
02961 }
02962
02964 static void menu_handle_notification(menu_state_t *st, const menu_notification_t *n)
02965 {
02966     char code_line[MENU_LINE_BUF_SIZE];
02967     char source_line[MENU_LINE_BUF_SIZE];
02968     char text_safe[MENU_LINE_BUF_SIZE];
02969     bool has_text;
02970
02971     if ((st == NULL) || (n == NULL))
02972     {
02973         return;
02974     }
02975     if ((n->type == MENU_NOTIFICATION_DELIVERY) &&
02976         st->tx_ack_waiting &&
02977         (st->modal == MENU_MODAL_INFO))
02978     {
02979         st->tx_ack_waiting = false;
02980         menu_render_popup("TX SENT", st->tx_sent_text, "Delivery ACK", "Any key=back");
02981         return;
02982     }
02983     if (st->modal != MENU_MODAL_NONE)
02984     {
02985         if (menu_modal_is_preemptible(st->modal))
02986         {
02987             st->modal = MENU_MODAL_NONE;
02988         }
02989         else
02990         {
02991             return;
02992         }
02993     }
02994
02995     memset(text_safe, 0, sizeof(text_safe));
02996     (void)menu_line_copy(text_safe, 0U, n->text, MENU_LINE_CHARS);
02997     has_text = (text_safe[0] != '\0');
02998     if (!has_text && (n->type != MENU_NOTIFICATION_PAIRING))
02999     {
03000         return;
03001     }
03002
03003     if (n->type == MENU_NOTIFICATION_RX)
03004     {
03005         uint32_t source_id = (n->reply_device_code != 0UL) ? n->reply_device_code : n->device_code;
03006
03007         if (menu_should_open_quick_reply(has_text ? text_safe : NULL))
03008         {
03009             st->modal = MENU_MODAL_QUICK_REPLY;
03010             st->quick_reply_src_id = source_id;
03011             st->quick_reply_deadline_ms = HAL_GetTick() + 30000UL;
03012             st->quick_reply_last_seconds = 30U;
03013             menu_line_copy_or_default(st->quick_reply_text, text_safe, "");
03014             menu_render_quick_reply(st);
03015             return;
03016         }
03017
03018         menu_line_format_source(source_line, source_id);
03019         menu_open_info_modal(st, "RX MESSAGE", has_text ? text_safe : "(empty)", source_line, "Any
key=back");

```

```

03020     return;
03021 }
03022
03023 if ((!st->popup_enabled) && (n->type != MENU_NOTIFICATION_PAIRING))
03024 {
03025     return;
03026 }
03027
03028 if ((n->type == MENU_NOTIFICATION_PAIRING) &&
03029     (strcmp(text_safe, "PAIR_REQ", 8U) == 0))
03030 {
03031     st->modal = MENU_MODAL_PAIR_REQUEST;
03032     menu_line_clear(code_line);
03033     (void)menu_line_copy(code_line, 0U, "Code: ", 6U);
03034     (void)menu_line_copy(code_line, 6U, menu_pair_code_text(text_safe, 8U), 11U);
03035     menu_render_popup("PAIR REQUEST", code_line, "OK=accept", "Hold OK=reject");
03036     return;
03037 }
03038
03039 if ((n->type == MENU_NOTIFICATION_PAIRING) &&
03040     (strcmp(text_safe, "PAIR_SENT", 9U) == 0))
03041 {
03042     menu_line_clear(code_line);
03043     (void)menu_line_copy(code_line, 0U, "Code: ", 6U);
03044     (void)menu_line_copy(code_line, 6U, menu_pair_code_text(text_safe, 9U), 11U);
03045     menu_open_info_modal(st, "PAIR REQ SENT", code_line, "Wait for PAIR_OK", "");
03046     return;
03047 }
03048
03049 if ((n->type == MENU_NOTIFICATION_PAIRING) &&
03050     (strcmp(text_safe, "PAIR_OK", 7U) == 0))
03051 {
03052     menu_line_clear(code_line);
03053     (void)menu_line_copy(code_line, 0U, "Code: ", 6U);
03054     (void)menu_line_copy(code_line, 6U, menu_pair_code_text(text_safe, 7U), 11U);
03055     menu_open_info_modal(st, "PAIRING OK", code_line, "Device trusted", "");
03056     return;
03057 }
03058
03059 if ((n->type == MENU_NOTIFICATION_PAIRING) &&
03060     (strcmp(text_safe, "PAIR_ERROR", 10U) == 0))
03061 {
03062     menu_open_info_modal(st, "PAIRING", "PAIR rejected", "Any key=back", "");
03063     return;
03064 }
03065
03066 if (has_text)
03067 {
03068     menu_show_action_result(st, n->type, text_safe);
03069 }
03070 else
03071 {
03072     menu_show_action_result(st, n->type, "(empty)");
03073 }
03074 }
03075
03076 static void menu_handle_pin_button(menu_state_t *st, button_event_t evt)
03077 {
03078     menu_page_id_t target_page;
03079     menu_action_t pin_action;
03080
03081     if ((st == NULL) || (evt == BUTTON_EVENT_NONE))
03082     {
03083         return;
03084     }
03085
03086     if (evt == BUTTON_EVENT_UP_SHORT)
03087     {
03088         st->pin_digits[st->pin_index] = (uint8_t)((st->pin_digits[st->pin_index] + 1U) % 10U);
03089         menu_render_pin(st);
03090         return;
03091     }
03092
03093     if (evt == BUTTON_EVENT_DOWN_SHORT)
03094     {
03095         st->pin_digits[st->pin_index] = (st->pin_digits[st->pin_index] == 0U) ?
03096             9U :
03097             (uint8_t)(st->pin_digits[st->pin_index] - 1U);
03098         menu_render_pin(st);
03099         return;
03100     }
03101
03102     if (evt == BUTTON_EVENT_OK_LONG)
03103     {
03104         menu_close_modal(st);
03105         return;
03106     }
03107 }

```

```

03108
03109     if (evt != BUTTON_EVENT_OK_SHORT)
03110     {
03111         return;
03112     }
03113
03114     if ((uint8_t)(st->pin_index + 1U) < MENU_SETTINGS_PIN_LEN)
03115     {
03116         st->pin_index++;
03117         menu_render_pin(st);
03118         return;
03119     }
03120
03121     pin_action = st->pending_action;
03122     if (menu_is_pin_change_action(pin_action))
03123     {
03124         if (pin_action == MENU_ACTION_PIN_SET_ADMIN)
03125         {
03126             memcpy(s_admin_pin, st->pin_digits, sizeof(s_admin_pin));
03127             st->admin_unlock_until_ms = 0UL;
03128         }
03129         else
03130         {
03131             memcpy(s_operator_pin, st->pin_digits, sizeof(s_operator_pin));
03132             st->operator_unlock_until_ms = 0UL;
03133         }
03134         st->modal = MENU_MODAL_NONE;
03135         st->pending_action = MENU_ACTION_NONE;
03136         menu_clear_pin_state(st);
03137         menu_show_action_result(st, MENU_NOTIFICATION_SECURITY, "PIN changed");
03138         return;
03139     }
03140
03141     if (!menu_pin_matches(st))
03142     {
03143         memset(st->pin_digits, 0, sizeof(st->pin_digits));
03144         st->pin_index = 0U;
03145         menu_render_popup("BAD PIN", "Try again", "UP/DN digit", "OK=next HOLD=cancel");
03146         return;
03147     }
03148
03149     target_page = st->pending_auth_page;
03150     if (st->pending_auth_level == MENU_AUTH_ADMIN)
03151     {
03152         st->admin_unlock_until_ms = HAL_GetTick() + MENU_SETTINGS_PIN_UNLOCK_MS;
03153         st->operator_unlock_until_ms = st->admin_unlock_until_ms;
03154     }
03155     else if (st->pending_auth_level == MENU_AUTH_OPERATOR)
03156     {
03157         st->operator_unlock_until_ms = HAL_GetTick() + MENU_SETTINGS_PIN_UNLOCK_MS;
03158     }
03159     st->modal = MENU_MODAL_NONE;
03160     menu_clear_pin_state(st);
03161
03162     if (target_page == MENU_PAGE_NONE)
03163     {
03164         if (st->current_page == MENU_PAGE_NONE)
03165         {
03166             (void)lcd_main_set_mode(LCD_MODE_MONITOR);
03167         }
03168         else
03169         {
03170             menu_render(st);
03171         }
03172         return;
03173     }
03174
03175     menu_open_page(st, target_page);
03176 }
03177
03179 static void menu_handle_modal_button(menu_state_t *st, button_event_t evt)
03180 {
03181     menu_action_t action;
03182     bool ok;
03183     bool send_pair_req;
03184
03185     if ((st == NULL) || (evt == BUTTON_EVENT_NONE))
03186     {
03187         return;
03188     }
03189
03190     switch (st->modal)
03191     {
03192     case MENU_MODAL_PIN:
03193         menu_handle_pin_button(st, evt);
03194         break;
03195

```



```

03196     case MENU_MODAL_PAIR_REQUEST:
03197         if (evt == BUTTON_EVENT_OK_SHORT)
03198         {
03199             (void)radio_main_cmd_pairing_accept(true);
03200             menu_close_modal(st);
03201         }
03202         else if (evt == BUTTON_EVENT_OK_LONG)
03203         {
03204             (void)radio_main_cmd_pairing_accept(false);
03205             menu_close_modal(st);
03206         }
03207         break;
03208
03209     case MENU_MODAL_PAIR_SETUP:
03210         send_pair_req = (!st->pairing_network_mode && (evt == BUTTON_EVENT_OK_LONG));
03211         if (st->pairing_network_mode)
03212         {
03213             if (evt != BUTTON_EVENT_OK_SHORT)
03214             {
03215                 menu_close_modal(st);
03216                 return;
03217             }
03218         }
03219         else if ((evt != BUTTON_EVENT_OK_SHORT) && (evt != BUTTON_EVENT_OK_LONG))
03220         {
03221             menu_close_modal(st);
03222             return;
03223         }
03224
03225         ok = menu_start_pairing_session(st, send_pair_req, st->pairing_network_mode);
03226         if (!ok)
03227         {
03228             menu_show_action_result(st, MENU_NOTIFICATION_ERROR, "Pairing start failed");
03229         }
03230         break;
03231
03232     case MENU_MODAL_SEND_CONFIRM:
03233         action = st->pending_action;
03234         menu_close_modal(st);
03235         if (evt == BUTTON_EVENT_OK_SHORT)
03236         {
03237             menu_execute_action(st, action);
03238         }
03239         break;
03240
03241     case MENU_MODAL_QUICK_REPLY:
03242         if (evt == BUTTON_EVENT_UP_SHORT)
03243         {
03244             ok = radio_main_cmd_send_user_text("NO", st->quick_reply_src_id);
03245             menu_show_send_result(st, ok, "Sent NO");
03246         }
03247         else if (evt == BUTTON_EVENT_OK_SHORT)
03248         {
03249             ok = radio_main_cmd_send_user_text("OK", st->quick_reply_src_id);
03250             menu_show_send_result(st, ok, "Sent OK");
03251         }
03252         else if (evt == BUTTON_EVENT_DOWN_SHORT)
03253         {
03254             ok = radio_main_cmd_send_user_text("YES", st->quick_reply_src_id);
03255             menu_show_send_result(st, ok, "Sent YES");
03256         }
03257         else
03258         {
03259             menu_close_modal(st);
03260         }
03261         break;
03262
03263     case MENU_MODAL_INFO:
03264     default:
03265         menu_close_modal(st);
03266         break;
03267 }
03268 }
03269
03271 static void menu_handle_button(menu_state_t *st, button_event_t evt)
03272 {
03273     const menu_page_t *page;
03274     radio_main_runtime_cfg_t radio_cfg;
03275
03276     if ((st == NULL) || (evt == BUTTON_EVENT_NONE))
03277     {
03278         return;
03279     }
03280
03281     if (st->modal != MENU_MODAL_NONE)
03282     {
03283         menu_handle_modal_button(st, evt);

```

```

03284     return;
03285 }
03286
03287 if (st->current_page == MENU_PAGE_NONE)
03288 {
03289     if (evt == BUTTON_EVENT_UP_SHORT)
03290     {
03291         (void)lcd_main_monitor_scroll_up();
03292     }
03293     else if (evt == BUTTON_EVENT_DOWN_SHORT)
03294     {
03295         (void)lcd_main_monitor_scroll_down();
03296     }
03297     else if ((evt == BUTTON_EVENT_OK_SHORT) || (evt == BUTTON_EVENT_OK_LONG))
03298     {
03299         if (menu_auth_unlocked(st, MENU_AUTH_OPERATOR))
03300         {
03301             menu_open_page(st, MENU_PAGE_PAGER);
03302         }
03303         else
03304         {
03305             menu_request_pin(st, MENU_PAGE_PAGER, MENU_AUTH_OPERATOR);
03306         }
03307     }
03308     return;
03309 }
03310
03311 page = menu_get_page(st->current_page);
03312 if (page == NULL)
03313 {
03314     menu_enter_monitor(st);
03315     return;
03316 }
03317
03318 if ((menu_page_auth_level(st->current_page) != MENU_AUTH_NONE) &&
03319     !menu_auth_unlocked(st, menu_page_auth_level(st->current_page)) &&
03320     (evt != BUTTON_EVENT_OK_LONG))
03321 {
03322     menu_request_pin(st, st->current_page, menu_page_auth_level(st->current_page));
03323     return;
03324 }
03325
03326 switch (evt)
03327 {
03328     case BUTTON_EVENT_UP_SHORT:
03329         if (page->item_count > 0U)
03330         {
03331             uint8_t start = st->selected_idx;
03332
03333             do
03334             {
03335                 if (st->selected_idx == 0U)
03336                 {
03337                     st->selected_idx = (uint8_t)(page->item_count - 1U);
03338                 }
03339                 else
03340                 {
03341                     st->selected_idx = (uint8_t)(st->selected_idx - 1U);
03342                 }
03343             } while ((st->selected_idx != start) &&
03344                 !menu_item_is_selectable(st, page, st->selected_idx));
03345             menu_render(st);
03346         }
03347         break;
03348
03349     case BUTTON_EVENT_DOWN_SHORT:
03350         if (page->item_count > 0U)
03351         {
03352             uint8_t start = st->selected_idx;
03353
03354             do
03355             {
03356                 st->selected_idx = (uint8_t)((st->selected_idx + 1U) % page->item_count);
03357             } while ((st->selected_idx != start) &&
03358                 !menu_item_is_selectable(st, page, st->selected_idx));
03359             menu_render(st);
03360         }
03361         break;
03362
03363     case BUTTON_EVENT_OK_SHORT:
03364         if (st->selected_idx < page->item_count)
03365         {
03366             const menu_item_t *item = &page->items[st->selected_idx];
03367
03368             if (!menu_item_is_selectable(st, page, st->selected_idx))
03369             {
03370                 break;

```

```

03371     }
03372     if ((st->current_page == MENU_PAGE_RADIO_SETTINGS) &&
03373         radio_main_get_runtime_cfg(&radio_cfg))
03374     {
03375         if (st->selected_idx == (uint8_t)(menu_radio_settings_item_count(&radio_cfg) -
1U))
03376         {
03377             menu_execute_action(st, MENU_ACTION_BACK);
03378             break;
03379         }
03380     }
03381     if ((st->current_page == MENU_PAGE_DEVICE_DELETE_LIST) &&
03382         (st->selected_idx < MENU_TRUSTED_DEVICE_SLOTS))
03383     {
03384         trusted_info_t info;
03385
03386         memset(&info, 0, sizeof(info));
03387         if (menu_get_trusted_slot(st->selected_idx, &info))
03388         {
03389             menu_open_device_delete_action(st, st->selected_idx);
03390         }
03391         else
03392         {
03393             menu_show_action_result(st, MENU_NOTIFICATION_WARNING, "Empty slot");
03394         }
03395     }
03396     else if (st->current_page == MENU_PAGE_SEND_TARGET_LIST)
03397     {
03398         if (st->selected_idx < MENU_TRUSTED_DEVICE_SLOTS)
03399         {
03400             trusted_info_t info;
03401             char label[MENU_LINE_BUF_SIZE];
03402
03403             memset(&info, 0, sizeof(info));
03404             if (menu_get_trusted_slot(st->selected_idx, &info) &&
03405                 menu_is_send_target_allowed(info.node_id))
03406             {
03407                 st->pending_target_node_id = info.node_id;
03408                 menu_build_trusted_slot_label(st->selected_idx, &info, label);
03409                 menu_open_send_prompt(st, st->send_target_action, label);
03410             }
03411             else
03412             {
03413                 menu_show_action_result(st, MENU_NOTIFICATION_WARNING, "Empty slot");
03414             }
03415         }
03416         else
03417         {
03418             menu_execute_action(st, MENU_ACTION_BACK);
03419         }
03420     }
03421     else if ((st->current_page == MENU_PAGE_SEND_DIRECT_LIST) &&
03422         (st->selected_idx < MENU_TRUSTED_DEVICE_SLOTS))
03423     {
03424         trusted_info_t info;
03425         char label[MENU_LINE_BUF_SIZE];
03426
03427         memset(&info, 0, sizeof(info));
03428         if (menu_get_trusted_slot(st->selected_idx, &info) &&
03429             menu_is_send_target_allowed(info.node_id))
03430         {
03431             st->pending_target_node_id = info.node_id;
03432             menu_build_trusted_slot_label(st->selected_idx, &info, label);
03433             menu_open_send_prompt(st, MENU_ACTION_SEND_DIRECT_DEFAULT, label);
03434         }
03435         else
03436         {
03437             menu_show_action_result(st, MENU_NOTIFICATION_WARNING, "Empty slot");
03438         }
03439     }
03440     else if (item->child_page != MENU_PAGE_NONE)
03441     {
03442         if ((menu_page_auth_level(item->child_page) != MENU_AUTH_NONE) &&
03443             !menu_auth_unlocked(st, menu_page_auth_level(item->child_page)))
03444         {
03445             menu_request_pin(st, item->child_page,
menu_page_auth_level(item->child_page));
03446             break;
03447         }
03448         menu_open_page(st, item->child_page);
03449     }
03450     else if (menu_is_send_action(item->action))
03451     {
03452         menu_open_send_target_page(st, item->action);
03453     }
03454     else
03455     {

```

```

03456         menu_execute_action(st, item->action);
03457     }
03458 }
03459 break;
03460
03461 case BUTTON_EVENT_OK_LONG:
03462     if ((st->current_page == MENU_PAGE_SEND_TARGET_LIST) &&
03463         (st->transient_parent_page != MENU_PAGE_NONE))
03464     {
03465         menu_open_page(st, st->transient_parent_page);
03466         break;
03467     }
03468     if (page->parent == MENU_PAGE_NONE)
03469     {
03470         menu_enter_monitor(st);
03471     }
03472     else
03473     {
03474         menu_open_page(st, page->parent);
03475     }
03476     break;
03477
03478 default:
03479     break;
03480 }
03481 }
03482
03484 static void menu_execute_action(menu_state_t *st, menu_action_t action)
03485 {
03486     char line0[MENU_LINE_BUF_SIZE];
03487     char line1[MENU_LINE_BUF_SIZE];
03488     security_runtime_cfg_t cfg;
03489     radio_main_runtime_cfg_t radio_cfg;
03490     trusted_info_t info;
03491     bmp280_api_data_t bmp;
03492     bool tpm_ready;
03493     bool send_ok;
03494     bool ok;
03495     bool have_radio_cfg;
03496     uint8_t idx;
03497     uint8_t count;
03498     uint32_t auto_ping_ms;
03499     int32_t distance_mm;
03500     uint32_t dst_id;
03501
03502     menu_line_clear(line0);
03503     menu_line_clear(line1);
03504     memset(&cfg, 0, sizeof(cfg));
03505     memset(&radio_cfg, 0, sizeof(radio_cfg));
03506     memset(&info, 0, sizeof(info));
03507     have_radio_cfg = radio_main_get_runtime_cfg(&radio_cfg);
03508
03509     if (menu_execute_radio_action(st, action))
03510     {
03511         return;
03512     }
03513
03514     if (menu_is_send_action(action) &&
03515         !menu_is_send_target_allowed(st->pending_target_node_id))
03516     {
03517         st->send_target_action = MENU_ACTION_NONE;
03518         menu_show_action_result(st, MENU_NOTIFICATION_WARNING, "No target");
03519         return;
03520     }
03521
03522     switch (action)
03523     {
03524     case MENU_ACTION_BACK:
03525         if ((st->current_page == MENU_PAGE_SEND_TARGET_LIST) &&
03526             (st->transient_parent_page != MENU_PAGE_NONE))
03527         {
03528             menu_open_page(st, st->transient_parent_page);
03529             break;
03530         }
03531         {
03532             const menu_page_t *page = menu_get_page(st->current_page);
03533
03534             if ((page == NULL) || (page->parent == MENU_PAGE_NONE))
03535             {
03536                 menu_enter_monitor(st);
03537             }
03538             else
03539             {
03540                 menu_open_page(st, page->parent);
03541             }
03542         }
03543         break;

```

```
03544
03545     case MENU_ACTION_EXIT_TO_MONITOR:
03546         menu_enter_monitor(st);
03547         break;
03548
03549     case MENU_ACTION_SEND_DEFAULT:
03550         dst_id = st->pending_target_node_id;
03551         send_ok = radio_main_cmd_send_template(1U, 0U, dst_id);
03552         menu_show_send_result(st, send_ok, "Sent STS:OK");
03553         st->pending_target_node_id = 0U;
03554         st->send_target_action = MENU_ACTION_NONE;
03555         break;
03556
03557     case MENU_ACTION_SEND_DIRECT_DEFAULT:
03558         if (st->pending_target_node_id == 0U)
03559         {
03560             menu_show_action_result(st, MENU_NOTIFICATION_WARNING, "No target");
03561             break;
03562         }
03563         send_ok = radio_main_cmd_send_template(1U, 0U, st->pending_target_node_id);
03564         menu_line_format_device_ref(line0, "Sent to ", st->pending_target_node_id);
03565         menu_show_send_result(st, send_ok, line0);
03566         st->pending_target_node_id = 0U;
03567         st->send_target_action = MENU_ACTION_NONE;
03568         break;
03569
03570     case MENU_ACTION_SEND_ALERT_FIRE:
03571         dst_id = st->pending_target_node_id;
03572         send_ok = radio_main_cmd_send_template(0U, 0U, dst_id);
03573         menu_show_send_result(st, send_ok, "Sent ALR:FIRE");
03574         st->pending_target_node_id = 0U;
03575         st->send_target_action = MENU_ACTION_NONE;
03576         break;
03577
03578     case MENU_ACTION_SEND_ALERT_INTR:
03579         dst_id = st->pending_target_node_id;
03580         send_ok = radio_main_cmd_send_template(0U, 1U, dst_id);
03581         menu_show_send_result(st, send_ok, "Sent ALR:INTR");
03582         st->pending_target_node_id = 0U;
03583         st->send_target_action = MENU_ACTION_NONE;
03584         break;
03585
03586     case MENU_ACTION_SEND_ALERT_LOWBATT:
03587         dst_id = st->pending_target_node_id;
03588         send_ok = radio_main_cmd_send_template(0U, 2U, dst_id);
03589         menu_show_send_result(st, send_ok, "Sent ALR:LOW");
03590         st->pending_target_node_id = 0U;
03591         st->send_target_action = MENU_ACTION_NONE;
03592         break;
03593
03594     case MENU_ACTION_SEND_STATUS_OK:
03595         dst_id = st->pending_target_node_id;
03596         send_ok = radio_main_cmd_send_template(1U, 0U, dst_id);
03597         menu_show_send_result(st, send_ok, "Sent STS:OK");
03598         st->pending_target_node_id = 0U;
03599         st->send_target_action = MENU_ACTION_NONE;
03600         break;
03601
03602     case MENU_ACTION_SEND_STATUS_BUSY:
03603         dst_id = st->pending_target_node_id;
03604         send_ok = radio_main_cmd_send_template(1U, 1U, dst_id);
03605         menu_show_send_result(st, send_ok, "Sent STS:BUSY");
03606         st->pending_target_node_id = 0U;
03607         st->send_target_action = MENU_ACTION_NONE;
03608         break;
03609
03610     case MENU_ACTION_SEND_STATUS_IDLE:
03611         dst_id = st->pending_target_node_id;
03612         send_ok = radio_main_cmd_send_template(1U, 2U, dst_id);
03613         menu_show_send_result(st, send_ok, "Sent STS:IDLE");
03614         st->pending_target_node_id = 0U;
03615         st->send_target_action = MENU_ACTION_NONE;
03616         break;
03617
03618     case MENU_ACTION_SEND_SERVICE_PING:
03619         dst_id = st->pending_target_node_id;
03620         send_ok = radio_main_cmd_send_template(2U, 0U, dst_id);
03621         menu_show_send_result(st, send_ok, "Sent SRV:PING");
03622         st->pending_target_node_id = 0U;
03623         st->send_target_action = MENU_ACTION_NONE;
03624         break;
03625
03626     case MENU_ACTION_SEND_SERVICE_RESET:
03627         dst_id = st->pending_target_node_id;
03628         send_ok = radio_main_cmd_send_template(2U, 1U, dst_id);
03629         menu_show_send_result(st, send_ok, "Sent SRV:RESET");
03630         st->pending_target_node_id = 0U;
```

```

03631         st->send_target_action = MENU_ACTION_NONE;
03632         break;
03633
03634     case MENU_ACTION_SEND_SERVICE_SYNC:
03635         dst_id = st->pending_target_node_id;
03636         send_ok = radio_main_cmd_send_template(2U, 2U, dst_id);
03637         menu_show_send_result(st, send_ok, "Sent SRV:SYNC");
03638         st->pending_target_node_id = 0U;
03639         st->send_target_action = MENU_ACTION_NONE;
03640         break;
03641
03642     case MENU_ACTION_SEND_ASK_DONE:
03643         dst_id = st->pending_target_node_id;
03644         send_ok = radio_main_cmd_send_user_text("ASK: Is done?", dst_id);
03645         menu_show_send_result(st, send_ok, "Sent ASK:DONE");
03646         st->pending_target_node_id = 0U;
03647         st->send_target_action = MENU_ACTION_NONE;
03648         break;
03649
03650     case MENU_ACTION_SEND_ACT_COME_OVER:
03651         dst_id = st->pending_target_node_id;
03652         send_ok = radio_main_cmd_send_user_text("ACT Come over.", dst_id);
03653         menu_show_send_result(st, send_ok, "Sent ACT:COME");
03654         st->pending_target_node_id = 0U;
03655         st->send_target_action = MENU_ACTION_NONE;
03656         break;
03657
03658     case MENU_ACTION_SEND_ACT_STOP:
03659         dst_id = st->pending_target_node_id;
03660         send_ok = radio_main_cmd_send_user_text("ACT: Stop!", dst_id);
03661         menu_show_send_result(st, send_ok, "Sent ACT:STOP");
03662         st->pending_target_node_id = 0U;
03663         st->send_target_action = MENU_ACTION_NONE;
03664         break;
03665
03666     case MENU_ACTION_SEND_ASK_READY:
03667         dst_id = st->pending_target_node_id;
03668         send_ok = radio_main_cmd_send_user_text("ASK: Is ready?", dst_id);
03669         menu_show_send_result(st, send_ok, "Sent ASK:READY");
03670         st->pending_target_node_id = 0U;
03671         st->send_target_action = MENU_ACTION_NONE;
03672         break;
03673
03674     case MENU_ACTION_DEVICE_ADD:
03675         st->pairing_network_mode = false;
03676         st->modal = MENU_MODAL_PAIR_SETUP;
03677         st->pending_action = MENU_ACTION_NONE;
03678         menu_render_popup("PAIR MODE 5 min", "OK=listen", "Hold OK=PAIR_REQ", "Any key=cancel");
03679         break;
03680
03681     case MENU_ACTION_DEVICE_ADD_NETWORK:
03682         if (menu_gateway_slot_in_use())
03683         {
03684             menu_show_action_result(st, MENU_NOTIFICATION_WARNING, "Remove G first");
03685             break;
03686         }
03687         st->pairing_network_mode = true;
03688         st->modal = MENU_MODAL_PAIR_SETUP;
03689         st->pending_action = MENU_ACTION_NONE;
03690         menu_render_popup("NET PAIR 5 min", "OK=listen", "GW sends PAIR_REQ", "Any key=cancel");
03691         break;
03692
03693     case MENU_ACTION_DEVICE_DELETE:
03694         menu_open_page(st, MENU_PAGE_DEVICE_DELETE_LIST);
03695         break;
03696
03697     case MENU_ACTION_DEVICE_DELETE_CONFIRM:
03698         if (st->selected_device_slot >= MENU_TRUSTED_DEVICE_SLOTS)
03699         {
03700             menu_show_action_result(st, MENU_NOTIFICATION_WARNING, "No device");
03701             break;
03702         }
03703         if (!security_main_cmd_get_device(st->selected_device_slot, &info) || !info.in_use)
03704         {
03705             st->current_page = MENU_PAGE_DEVICE_DELETE_LIST;
03706             st->selected_idx = 0U;
03707             menu_show_action_result(st, MENU_NOTIFICATION_WARNING, "Device gone");
03708             break;
03709         }
03710
03711         ok = security_main_cmd_delete_device(info.node_id);
03712         if (ok)
03713         {
03714             send_ok = radio_main_cmd_send_pair_error(info.node_id);
03715         }
03716         else
03717         {

```

```

03718         send_ok = false;
03719     }
03720
03721     st->current_page = MENU_PAGE_DEVICE_DELETE_LIST;
03722     st->selected_idx = 0U;
03723     st->selected_device_slot = MENU_DEVICE_SLOT_INVALID;
03724
03725     if (!ok)
03726     {
03727         menu_show_action_result(st, MENU_NOTIFICATION_ERROR, "Delete failed");
03728     }
03729     else if (send_ok)
03730     {
03731         menu_show_action_result(st, MENU_NOTIFICATION_SECURITY, "Device removed");
03732     }
03733     else
03734     {
03735         menu_show_action_result(st, MENU_NOTIFICATION_WARNING, "Removed local only");
03736     }
03737     break;
03738
03739     case MENU_ACTION_DEVICE_INFO:
03740         count = 0U;
03741         for (idx = 1U; idx <= MENU_TRUSTED_NODE_SLOTS; idx++)
03742         {
03743             if (security_main_cmd_get_device(idx, &info) &&
03744                 info.in_use &&
03745                 (info.node_id != LAVIET_GATEWAY_ID) &&
03746                 (info.node_id != LAVIET_BROADCAST_ID) &&
03747                 (info.node_id != 0U))
03748             {
03749                 count++;
03750             }
03751         }
03752         menu_line_copy_or_default(line0,
03753             menu_gateway_slot_in_use() ? "Gateway connected" : "Gateway
missing",
03754             "");
03755         menu_line_format_u32(line1, "Nodes ", count, "/15");
03756         menu_open_info_modal(st, "CONNECTION INFO", line0, line1, "Any key=back");
03757         break;
03758
03759     case MENU_ACTION_PIN_TOGGLE:
03760         s_pin_enabled = !s_pin_enabled;
03761         if (!s_pin_enabled)
03762         {
03763             st->operator_unlock_until_ms = 0UL;
03764             st->admin_unlock_until_ms = 0UL;
03765         }
03766         menu_show_action_result(st,
03767             MENU_NOTIFICATION_SECURITY,
03768             s_pin_enabled ? "PIN enabled" : "PIN disabled");
03769         break;
03770
03771     case MENU_ACTION_PIN_SET_OPERATOR:
03772     case MENU_ACTION_PIN_SET_ADMIN:
03773         menu_start_pin_change(st, action);
03774         break;
03775
03776     case MENU_ACTION_SEC_TOGGLE_FH:
03777         if (security_main_cmd_get_runtime_cfg(&cfg))
03778         {
03779             const char *state_text = "FH OFF";
03780
03781             bool new_state = !cfg.fh_enabled;
03782             ok = security_main_cmd_set_fh(new_state) && radio_main_cmd_set_fh(new_state);
03783             if (new_state)
03784             {
03785                 state_text = "FH ON";
03786             }
03787             menu_show_ok_or_error(st, ok, state_text, "FH set failed");
03788         }
03789         break;
03790
03791     case MENU_ACTION_SEC_FH_ENABLE:
03792         if (have_radio_cfg && (radio_cfg.active_modulation != RADIO_MAIN_MODULATION_LORA))
03793         {
03794             menu_show_action_result(st, MENU_NOTIFICATION_WARNING, "FH needs LoRa mode");
03795             break;
03796         }
03797         ok = security_main_cmd_set_fh(true) && radio_main_cmd_set_fh(true);
03798         menu_show_ok_or_error(st, ok, "FH enabled", "FH enable failed");
03799         break;
03800
03801     case MENU_ACTION_SEC_FH_DISABLE:
03802         ok = security_main_cmd_set_fh(false) && radio_main_cmd_set_fh(false);
03803         menu_show_ok_or_error(st, ok, "FH disabled", "FH disable failed");

```

```
03804         break;
03805
03806     case MENU_ACTION_SEC_FH_PERIOD_1000:
03807         ok = security_main_cmd_set_fh_period(1000UL) && radio_main_cmd_set_fh_period(1000UL);
03808         menu_show_ok_or_error(st, ok, "FH cycle 1 sec", "FH cycle failed");
03809         break;
03810
03811     case MENU_ACTION_SEC_FH_PERIOD_2000:
03812         ok = security_main_cmd_set_fh_period(2000UL) && radio_main_cmd_set_fh_period(2000UL);
03813         menu_show_ok_or_error(st, ok, "FH cycle 2 sec", "FH cycle failed");
03814         break;
03815
03816     case MENU_ACTION_SEC_FH_PERIOD_5000:
03817         ok = security_main_cmd_set_fh_period(5000UL) && radio_main_cmd_set_fh_period(5000UL);
03818         menu_show_ok_or_error(st, ok, "FH cycle 5 sec", "FH cycle failed");
03819         break;
03820
03821     case MENU_ACTION_SEC_FH_PERIOD_10000:
03822         ok = security_main_cmd_set_fh_period(10000UL) && radio_main_cmd_set_fh_period(10000UL);
03823         menu_show_ok_or_error(st, ok, "FH cycle 10 sec", "FH cycle failed");
03824         break;
03825
03826     case MENU_ACTION_SEC_TPM_INFO:
03827         if (security_main_get_tpm_ready(&tpm_ready))
03828         {
03829             if (tpm_ready)
03830             {
03831                 menu_open_info_modal(st, "TPM", "Ready", "Any key=back", "");
03832             }
03833             else
03834             {
03835                 menu_open_info_modal(st, "TPM", "Not ready", "Any key=back", "");
03836             }
03837         }
03838         else
03839         {
03840             menu_show_action_result(st, MENU_NOTIFICATION_ERROR, "TPM query failed");
03841         }
03842         break;
03843
03844     case MENU_ACTION_SEC_ROTATE_KEYS:
03845         ok = security_main_cmd_rotate_key();
03846         if (ok)
03847         {
03848             menu_show_action_result(st, MENU_NOTIFICATION_SECURITY, "Key rotated");
03849         }
03850         else if (security_main_get_tpm_ready(&tpm_ready) && tpm_ready)
03851         {
03852             menu_show_action_result(st, MENU_NOTIFICATION_ERROR, "Hold TPM PP button");
03853         }
03854         else
03855         {
03856             menu_show_action_result(st, MENU_NOTIFICATION_ERROR, "Keys: not ready");
03857         }
03858         break;
03859
03860     case MENU_ACTION_SEC_CODING_ON:
03861         ok = security_main_cmd_set_coding(true) && radio_main_cmd_set_coding(true);
03862         menu_show_ok_or_error(st, ok, "Coding ON", "Coding set failed");
03863         break;
03864
03865     case MENU_ACTION_SEC_CODING_OFF:
03866         ok = security_main_cmd_set_coding(false) && radio_main_cmd_set_coding(false);
03867         menu_show_ok_or_error(st, ok, "Coding OFF", "Coding set failed");
03868         break;
03869
03870     case MENU_ACTION_SEC_NOTIFY_POPUP:
03871         ok = security_main_cmd_set_notify_mode(SEcurity_NOTIFY_POPUP);
03872         if (ok)
03873         {
03874             st->popup_enabled = true;
03875         }
03876         menu_show_ok_or_error(st, ok, "Notif POPUP", "Notif set failed");
03877         break;
03878
03879     case MENU_ACTION_SEC_NOTIFY_BADGE:
03880         ok = security_main_cmd_set_notify_mode(SEcurity_NOTIFY_BADGE);
03881         if (ok)
03882         {
03883             st->popup_enabled = false;
03884         }
03885         menu_show_ok_or_error(st, ok, "Notif BADGE", "Notif set failed");
03886         break;
03887
03888     case MENU_ACTION_SEC_AUTOPING_ON:
03889         ok = security_main_cmd_set_auto_ping(true) && radio_main_cmd_set_auto_ping(true);
03890         if (ok && radio_main_get_auto_ping_period_ms(&auto_ping_ms))
```



```

03891         {
03892             menu_line_format_u32(line1, "Period ", auto_ping_ms, " ms");
03893             menu_open_info_modal(st, "AUTOPING ON", line1, "Any key=back", "");
03894         }
03895     else
03896     {
03897         menu_show_ok_or_error(st, ok, "AutoPing ON", "AutoPing failed");
03898     }
03899     break;
03900
03901     case MENU_ACTION_SEC_AUTOPING_OFF:
03902         ok = security_main_cmd_set_auto_ping(false) && radio_main_cmd_set_auto_ping(false);
03903         menu_show_ok_or_error(st, ok, "AutoPing OFF", "AutoPing failed");
03904         break;
03905
03906     case MENU_ACTION_SEC_AUTOPING_PERIOD_1:
03907         ok = security_main_cmd_set_auto_ping_period(1UL) &&
03908             radio_main_cmd_set_auto_ping_period(1UL);
03909         menu_show_ok_or_error(st, ok, "Ping 1 ms", "Period failed");
03910         break;
03911
03912     case MENU_ACTION_SEC_AUTOPING_PERIOD_10:
03913         ok = security_main_cmd_set_auto_ping_period(10UL) &&
03914             radio_main_cmd_set_auto_ping_period(10UL);
03915         menu_show_ok_or_error(st, ok, "Ping 10 ms", "Period failed");
03916         break;
03917
03918     case MENU_ACTION_SEC_AUTOPING_PERIOD_100:
03919         ok = security_main_cmd_set_auto_ping_period(100UL) &&
03920             radio_main_cmd_set_auto_ping_period(100UL);
03921         menu_show_ok_or_error(st, ok, "Ping 100 ms", "Period failed");
03922         break;
03923
03924     case MENU_ACTION_SEC_AUTOPING_PERIOD_1000:
03925         ok = security_main_cmd_set_auto_ping_period(1000UL) &&
03926             radio_main_cmd_set_auto_ping_period(1000UL);
03927         menu_show_ok_or_error(st, ok, "Ping 1 sec", "Period failed");
03928         break;
03929
03930     case MENU_ACTION_SEC_AUTOPING_PERIOD_10000:
03931         ok = security_main_cmd_set_auto_ping_period(10000UL) &&
03932             radio_main_cmd_set_auto_ping_period(10000UL);
03933         menu_show_ok_or_error(st, ok, "Ping 10 sec", "Period failed");
03934         break;
03935
03936     case MENU_ACTION_SEC_AUTOPING_FRAME:
03937         ok = security_main_cmd_set_auto_ping_mode(RADIO_MAIN_AUTO_PING_FRAME) &&
03938             radio_main_cmd_set_auto_ping_mode(RADIO_MAIN_AUTO_PING_FRAME);
03939         menu_show_ok_or_error(st, ok, "Ping LAVIET", "Mode failed");
03940         break;
03941
03942     case MENU_ACTION_SEC_AUTOPING_RAW:
03943         ok = security_main_cmd_set_auto_ping_mode(RADIO_MAIN_AUTO_PING_RAW) &&
03944             radio_main_cmd_set_auto_ping_mode(RADIO_MAIN_AUTO_PING_RAW);
03945         menu_show_ok_or_error(st, ok, "Ping RAW", "Mode failed");
03946         break;
03947
03948     case MENU_ACTION_SEC_TOGGLE_CODING:
03949         if (security_main_cmd_get_runtime_cfg(&cfg))
03950         {
03951             const char *state_text = "Coding OFF";
03952
03953             bool new_state = !cfg.coding_enabled;
03954             ok = security_main_cmd_set_coding(new_state) && radio_main_cmd_set_coding(new_state);
03955             if (new_state)
03956             {
03957                 state_text = "Coding ON";
03958             }
03959             menu_show_ok_or_error(st, ok, state_text, "Coding set failed");
03960         }
03961         break;
03962
03963     case MENU_ACTION_SEC_TOGGLE_NOTIFY_MODE:
03964         if (security_main_cmd_get_runtime_cfg(&cfg))
03965         {
03966             security_notify_mode_t mode = SECURITY_NOTIFY_POPUP;
03967
03968             if (cfg.notify_mode == SECURITY_NOTIFY_POPUP)
03969             {
03970                 mode = SECURITY_NOTIFY_BADGE;
03971             }
03972             ok = security_main_cmd_set_notify_mode(mode);
03973             if (ok)
03974             {
03975                 if (mode == SECURITY_NOTIFY_POPUP)
03976                 {
03977                     st->popup_enabled = true;
03978                 }
03979             }
03980         }
03981     }
03982 }

```

```

03973         }
03974         else
03975         {
03976             st->popup_enabled = false;
03977         }
03978     }
03979     if (ok)
03980     {
03981         if (st->popup_enabled)
03982         {
03983             menu_show_action_result(st, MENU_NOTIFICATION_SECURITY, "Notif POPUP");
03984         }
03985         else
03986         {
03987             menu_show_action_result(st, MENU_NOTIFICATION_SECURITY, "Notif BADGE");
03988         }
03989     }
03990     else
03991     {
03992         menu_show_action_result(st, MENU_NOTIFICATION_ERROR, "Notif set failed");
03993     }
03994 }
03995 break;
03996
03997 case MENU_ACTION_SEC_TOGGLE_AUTOPING:
03998     if (security_main_cmd_get_runtime_cfg(&cfg))
03999     {
04000         const char *state_text = "AutoPing OFF";
04001
04002         bool new_state = !cfg.auto_ping_enabled;
04003         ok = security_main_cmd_set_auto_ping(new_state) &&
04004 radio_main_cmd_set_auto_ping(new_state);
04005         if (new_state)
04006         {
04007             state_text = "AutoPing ON";
04008         }
04009         menu_show_ok_or_error(st, ok, state_text, "AutoPing failed");
04010     }
04011     break;
04012
04013 case MENU_ACTION_HW_MEASURE_DIST:
04014     distance_mm = tof_main_get_last_distance();
04015     menu_line_format_i32(line1, "", distance_mm, " mm");
04016     menu_open_info_modal(st, "Distance", line1, "Any key=back", "");
04017     break;
04018
04019 case MENU_ACTION_HW_MEASURE_TEMP:
04020     if (bmp280_main_get_last(&bmp))
04021     {
04022         menu_line_format_fixed2(line0, "Temp ", bmp.temperature_c, " C");
04023         menu_open_info_modal(st, "BMP280", line0, "Any key=back", "");
04024     }
04025     else
04026     {
04027         menu_open_info_modal(st, "BMP280", "No data", "Any key=back", "");
04028     }
04029     break;
04030
04031 case MENU_ACTION_HW_MEASURE_PRESS:
04032     if (bmp280_main_get_last(&bmp))
04033     {
04034         menu_line_format_fixed2(line0, "Press ", bmp.pressure_hpa, " hPa");
04035         menu_open_info_modal(st, "BMP280", line0, "Any key=back", "");
04036     }
04037     else
04038     {
04039         menu_open_info_modal(st, "BMP280", "No data", "Any key=back", "");
04040     }
04041     break;
04042
04043 case MENU_ACTION_HW_LED_RAINBOW:
04044     st->led_mode = 0U;
04045     (void)led_array_start_rainbow(15U, 5U, 100U);
04046     menu_show_action_result(st, MENU_NOTIFICATION_SECURITY, "LED rainbow");
04047     break;
04048
04049 case MENU_ACTION_HW_LED_BREATH:
04050     st->led_mode = 1U;
04051     (void)led_array_start_breath(LED_ARRAY_LED_ALL, 1200U, 5U, 100U);
04052     menu_show_action_result(st, MENU_NOTIFICATION_SECURITY, "LED breath");
04053     break;
04054
04055 case MENU_ACTION_HW_LED_OFF:
04056     st->led_mode = 2U;
04057     (void)led_array_stop_effect();
04058     (void)led_array_off(LED_ARRAY_LED_ALL);

```

```

04059         menu_show_action_result(st, MENU_NOTIFICATION_SECURITY, "LED off");
04060         break;
04061
04062     case MENU_ACTION_HW_LED_MODE:
04063         st->led_mode = (uint8_t)((st->led_mode + 1U) % 3U);
04064         if (st->led_mode == 0U)
04065         {
04066             (void)led_array_start_rainbow(15U, 5U, 100U);
04067             menu_show_action_result(st, MENU_NOTIFICATION_SECURITY, "LED rainbow");
04068         }
04069         else if (st->led_mode == 1U)
04070         {
04071             (void)led_array_start_breath(LED_ARRAY_LED_ALL, 1200U, 5U, 100U);
04072             menu_show_action_result(st, MENU_NOTIFICATION_SECURITY, "LED breath");
04073         }
04074         else
04075         {
04076             (void)led_array_stop_effect();
04077             (void)led_array_off(LED_ARRAY_LED_ALL);
04078             menu_show_action_result(st, MENU_NOTIFICATION_SECURITY, "LED off");
04079         }
04080         break;
04081
04082     case MENU_ACTION_HW_RADIO_RESET:
04083         ok = radio_main_cmd_reset_module();
04084         menu_show_ok_or_error(st, ok, "SX1276 reset OK", "SX1276 reset fail");
04085         break;
04086
04087     case MENU_ACTION_MOD_LORA_STD:
04088         ok = radio_main_cmd_set_modulation((uint8_t)RADIO_MAIN_MODULATION_LORA) &&
04089             radio_main_cmd_set_lora_preset(0U) &&
04090             security_main_cmd_set_lora_preset(0U);
04091         menu_show_ok_or_error(st, ok, "LoRa STD", "LoRa preset failed");
04092         break;
04093
04094     case MENU_ACTION_MOD_LORA_RANGE:
04095         ok = radio_main_cmd_set_modulation((uint8_t)RADIO_MAIN_MODULATION_LORA) &&
04096             radio_main_cmd_set_lora_preset(1U) &&
04097             security_main_cmd_set_lora_preset(1U);
04098         menu_show_ok_or_error(st, ok, "LoRa RANGE", "LoRa preset failed");
04099         break;
04100
04101     case MENU_ACTION_MOD_LORA_FAST:
04102         ok = radio_main_cmd_set_modulation((uint8_t)RADIO_MAIN_MODULATION_LORA) &&
04103             radio_main_cmd_set_lora_preset(2U) &&
04104             security_main_cmd_set_lora_preset(2U);
04105         menu_show_ok_or_error(st, ok, "LoRa FAST", "LoRa preset failed");
04106         break;
04107
04108     case MENU_ACTION_MOD_FSK:
04109         ok = radio_main_cmd_set_modulation((uint8_t)RADIO_MAIN_MODULATION_FSK);
04110         menu_show_ok_or_error(st, ok, "FSK mode", "FSK set failed");
04111         break;
04112
04113     case MENU_ACTION_MOD_OOK:
04114         ok = radio_main_cmd_set_modulation((uint8_t)RADIO_MAIN_MODULATION_OOK);
04115         menu_show_ok_or_error(st, ok, "OOK mode", "OOK set failed");
04116         break;
04117
04118     case MENU_ACTION_INFO_SHOW:
04119         menu_line_format_hex32(line0, "Node ", radio_main_get_node_id());
04120         menu_open_info_modal(st, "BEKO W1", line0, "Menu/Sec active", "Any key=back");
04121         break;
04122
04123     default:
04124         break;
04125 }
04126 }
04127
04128 static bool menu_execute_radio_action(menu_state_t *st, menu_action_t action)
04129 {
04130     size_t idx;
04131     for (idx = 0U; idx < (sizeof(s_radio_action_bindings) / sizeof(s_radio_action_bindings[0]));
04132         idx++)
04133     {
04134         const menu_radio_action_binding_t *binding = &s_radio_action_bindings[idx];
04135         radio_main_runtime_cfg_t runtime_cfg;
04136         bool ok = true;
04137
04138         if (binding->action != action)
04139         {
04140             continue;
04141         }
04142
04143         memset(&runtime_cfg, 0, sizeof(runtime_cfg));
04144         if (binding->set_modulation)
04145 
```

```

04146     {
04147         if (!radio_main_get_runtime_cfg(&runtime_cfg) ||
04148             (runtime_cfg.active_modulation != binding->modulation))
04149         {
04150             ok = radio_main_cmd_set_modulation((uint8_t)binding->modulation);
04151         }
04152     }
04153
04154     if (ok && binding->set_option)
04155     {
04156         ok = radio_main_cmd_set_option(binding->option, binding->value);
04157     }
04158
04159     if (ok && binding->persist_lora_preset)
04160     {
04161         ok = security_main_cmd_set_lora_preset((uint8_t)binding->value);
04162     }
04163
04164     if (ok && radio_main_get_runtime_cfg(&runtime_cfg))
04165     {
04166         ok = security_main_cmd_set_radio_runtime_cfg(&runtime_cfg);
04167     }
04168
04169     menu_show_ok_or_error(st, ok, binding->ok_text, binding->err_text);
04170     return true;
04171 }
04172
04173 return false;
04174 }
04175
04177 static const menu_page_t *menu_get_page(menu_page_id_t page_id)
04178 {
04179     size_t idx;
04180
04181     if (page_id == MENU_PAGE_NONE)
04182     {
04183         return NULL;
04184     }
04185
04186     idx = (size_t)page_id - 1U;
04187     if (idx >= (sizeof(s_pages) / sizeof(s_pages[0])))
04188     {
04189         return NULL;
04190     }
04191
04192     return &s_pages[idx];
04193 }
04194
04195 /* Only send-type actions require a confirmation modal. */
04196 static bool menu_is_send_action(menu_action_t action)
04197 {
04198     switch (action)
04199     {
04200         case MENU_ACTION_SEND_DEFAULT:
04201         case MENU_ACTION_SEND_ALERT_FIRE:
04202         case MENU_ACTION_SEND_ALERT_INTR:
04203         case MENU_ACTION_SEND_ALERT_LOWBATT:
04204         case MENU_ACTION_SEND_STATUS_OK:
04205         case MENU_ACTION_SEND_STATUS_BUSY:
04206         case MENU_ACTION_SEND_STATUS_IDLE:
04207         case MENU_ACTION_SEND_SERVICE_PING:
04208         case MENU_ACTION_SEND_SERVICE_RESET:
04209         case MENU_ACTION_SEND_SERVICE_SYNC:
04210         case MENU_ACTION_SEND_ASK_DONE:
04211         case MENU_ACTION_SEND_ACT_COME_OVER:
04212         case MENU_ACTION_SEND_ACT_STOP:
04213         case MENU_ACTION_SEND_ASK_READY:
04214             return true;
04215         default:
04216             return false;
04217     }
04218 }
04219
04221 static bool menu_is_send_target_allowed(uint32_t node_id)
04222 {
04223     return ((node_id != 0UL) && (node_id != LAVIET_BROADCAST_ID));
04224 }
04225
04227 static bool menu_item_is_selectable(const menu_state_t *st, const menu_page_t *page, uint8_t item_idx)
04228 {
04229     radio_main_runtime_cfg_t radio_cfg;
04230
04231     if ((st == NULL) || (page == NULL) || (item_idx >= page->item_count))
04232     {
04233         return false;
04234     }
04235 }

```

```

04236     if ((st->current_page == MENU_PAGE_RADIO_SETTINGS) &&
04237         radio_main_get_runtime_cfg(&radio_cfg))
04238     {
04239         return (item_idx < menu_radio_settings_item_count(&radio_cfg));
04240     }
04241
04242     if (st->current_page == MENU_PAGE_SEND_TARGET_LIST)
04243     {
04244         if (item_idx == (uint8_t)(page->item_count - 1U))
04245         {
04246             return true;
04247         }
04248
04249         if (item_idx < MENU_TRUSTED_DEVICE_SLOTS)
04250         {
04251             return true;
04252         }
04253
04254         return false;
04255     }
04256
04257     if (st->current_page == MENU_PAGE_SEND_DIRECT_LIST)
04258     {
04259         if (item_idx == (uint8_t)(page->item_count - 1U))
04260         {
04261             return true;
04262         }
04263
04264         if (item_idx < MENU_TRUSTED_DEVICE_SLOTS)
04265         {
04266             return true;
04267         }
04268
04269         return false;
04270     }
04271
04272     if ((st->current_page == MENU_PAGE_SECURITY_AUTOPING) &&
04273         (item_idx == 0U))
04274     {
04275         return false;
04276     }
04277
04278     if ((st->current_page == MENU_PAGE_DEVICE_DELETE_ACTION) &&
04279         (item_idx == 0U))
04280     {
04281         return false;
04282     }
04283
04284     return true;
04285 }
04286
04288 static void menu_open_send_prompt(menu_state_t *st, menu_action_t action, const char *label)
04289 {
04290     char msg[MENU_LINE_BUF_SIZE];
04291
04292     if (st == NULL)
04293     {
04294         return;
04295     }
04296
04297     menu_line_clear(msg);
04298     if (label != NULL)
04299     {
04300         (void)menu_line_copy(msg, 0U, label, MENU_LINE_CHARS);
04301     }
04302     else
04303     {
04304         (void)menu_line_copy(msg, 0U, "Message", MENU_LINE_CHARS);
04305     }
04306
04307     st->modal = MENU_MODAL_SEND_CONFIRM;
04308     st->pending_action = action;
04309     menu_render_popup("SEND MESSAGE?", msg, "OK=send", "Any key=back");
04310 }
04311
04313 static void menu_open_send_target_page(menu_state_t *st, menu_action_t action)
04314 {
04315     menu_page_id_t previous_page;
04316     const menu_page_t *page;
04317
04318     if (st == NULL)
04319     {
04320         return;
04321     }
04322
04323     previous_page = st->current_page;
04324     st->current_page = MENU_PAGE_SEND_TARGET_LIST;

```

```

04325     st->selected_idx = 0U;
04326     st->modal = MENU_MODAL_NONE;
04327     st->pending_action = MENU_ACTION_NONE;
04328     st->pending_target_node_id = 0U;
04329     st->send_target_action = action;
04330     st->transient_parent_page = previous_page;
04331     page = menu_get_page(MENU_PAGE_SEND_TARGET_LIST);
04332     if (page != NULL)
04333     {
04334         while ((st->selected_idx < page->item_count) &&
04335             !menu_item_is_selectable(st, page, st->selected_idx))
04336         {
04337             st->selected_idx++;
04338         }
04339         if ((page->item_count > 0U) && (st->selected_idx >= page->item_count))
04340         {
04341             st->selected_idx = (uint8_t)(page->item_count - 1U);
04342         }
04343     }
04344     menu_render(st);
04345 }
04346
04347 static void menu_open_device_delete_action(menu_state_t *st, uint8_t slot)
04348 {
04349     if (st == NULL)
04350     {
04351         return;
04352     }
04353
04354     st->current_page = MENU_PAGE_DEVICE_DELETE_ACTION;
04355     st->selected_idx = 1U;
04356     st->modal = MENU_MODAL_NONE;
04357     st->pending_action = MENU_ACTION_NONE;
04358     st->selected_device_slot = slot;
04359     menu_render(st);
04360 }
04361
04362 /* Starts pairing and reports the mode/result through a modal message. */
04363 static bool menu_start_pairing_session(menu_state_t *st, bool send_pair_req, bool network_mode)
04364 {
04365     bool send_ok = true;
04366
04367     if (st == NULL)
04368     {
04369         return false;
04370     }
04371
04372     if (network_mode && menu_gateway_slot_in_use())
04373     {
04374         return false;
04375     }
04376     if (network_mode)
04377     {
04378         if (!radio_main_cmd_start_network_pairing(MENU_PAIRING_WINDOW_MS))
04379         {
04380             return false;
04381         }
04382         send_pair_req = false;
04383     }
04384     else if (!radio_main_cmd_start_pairing(MENU_PAIRING_WINDOW_MS))
04385     {
04386         return false;
04387     }
04388
04389     if (send_pair_req)
04390     {
04391         if (network_mode)
04392         {
04393             send_ok = radio_main_cmd_send_network_pair_req();
04394         }
04395         else
04396         {
04397             send_ok = radio_main_cmd_send_pair_req();
04398         }
04399     }
04400
04401     if (send_pair_req)
04402     {
04403         const char *status_line = "PAIR_REQ failed";
04404
04405         if (send_ok)
04406         {
04407             status_line = "PAIR_REQ sent";
04408         }
04409
04410         menu_open_info_modal(st,
04411             network_mode ? "NET PAIR" : "PAIR MODE",
04412             "5 min active",

```

```

04413             status_line,
04414             "Any key=close");
04415     }
04416     else
04417     {
04418         menu_open_info_modal(st,
04419             network_mode ? "NET PAIR" : "PAIR MODE",
04420             "5 min active",
04421             network_mode ? "Wait PAIR_REQ GW" : "Listening...",
04422             "Any key=close");
04423     }
04424
04425     return true;
04426 }

```

12.44 Dokumentacja pliku Core/App/menu_main.h

LCD menu controller task.

```

#include <stdbool.h>
#include <stdint.h>

```

Struktury danych

- struct [menu_notification_t](#)
Message passed from application subsystems to the menu task.

Wyliczenia

- enum [menu_notification_type_t](#) {
MENU_NOTIFICATION_RX = 0 , MENU_NOTIFICATION_WARNING , MENU_NOTIFICATION_ERROR ,
MENU_NOTIFICATION_PAIRING ,
MENU_NOTIFICATION_SECURITY , MENU_NOTIFICATION_DELIVERY }
Notification classes accepted by the menu task.

Funkcje

- void [menu_main_create_task](#) (void)
Create the LCD menu task and notification queue.
- bool [menu_main_post_notification](#) (const [menu_notification_t](#) *n)
Queue a notification for menu/UI handling.

12.44.1 Opis szczegółowy

LCD menu controller task.

Definicja w pliku [menu_main.h](#).

12.44.2 Dokumentacja typów wyliczanych

12.44.2.1 menu_notification_type_t

enum [menu_notification_type_t](#)

Notification classes accepted by the menu task.

Wartości wyliczeń

MENU_NOTIFICATION_RX	Received user/radio payload.
MENU_NOTIFICATION_WARNING	Non-fatal warning for the UI.
MENU_NOTIFICATION_ERROR	Error popup or status message.
MENU_NOTIFICATION_PAIRING	Pairing workflow update.

MENU_NOTIFICATION_SECURITY	Security subsystem update.
MENU_NOTIFICATION_DELIVERY	Delivery/ACK status update.

Definicja w linii 15 pliku [menu_main.h](#).

12.44.3 Dokumentacja funkcji

12.44.3.1 menu_main_create_task()

```
void menu_main_create_task (
    void )
```

Create the LCD menu task and notification queue.

Definicja w linii 1386 pliku [menu_main.c](#).

Odwolania w [app_freertos_init\(\)](#).

12.44.3.2 menu_main_post_notification()

```
bool menu_main_post_notification (
    const menu_notification_t * n)
```

Queue a notification for menu/UI handling.

Parametry

<i>n</i>	Notification object.
----------	----------------------

Zwraca

`true` when the notification was queued.

Definicja w linii 1408 pliku [menu_main.c](#).

12.45 menu_main.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_MENU_MAIN_H_
00007 #define APP_MENU_MAIN_H_
00008
00009 #include <stdbool.h>
00010 #include <stdint.h>
00011
00015 typedef enum
00016 {
00017     MENU_NOTIFICATION_RX = 0,
00018     MENU_NOTIFICATION_WARNING,
00019     MENU_NOTIFICATION_ERROR,
00020     MENU_NOTIFICATION_PAIRING,
00021     MENU_NOTIFICATION_SECURITY,
00022     MENU_NOTIFICATION_DELIVERY
00023 } menu_notification_type_t;
00024
00028 typedef struct
00029 {
00030     menu_notification_type_t type;
00031     int16_t rssi_dbm;
00032     uint32_t device_code;
00033     uint32_t reply_device_code;
00034     char text[21];
00035 } menu_notification_t;
00036
00040 void menu_main_create_task(void);
00041
00047 bool menu_main_post_notification(const menu_notification_t *n);
00048
00049 #endif /* APP_MENU_MAIN_H_ */
```


12.46 Dokumentacja pliku Core/App/pager_config.h

Application-level build and board configuration switches.

Definicje

- `#define PAGER_CONFIG_UART_SENSITIVE_LOGS 0`
Enable verbose UART logs for sensitive secure/control payloads.
- `#define SERVICE_UART 1`
Build the UART service console task.
- `#define PAGER_CONFIG_BMP280_MOUNTED 0`
Board-level presence flag for BMP280/BME280.

12.46.1 Opis szczegółowy

Application-level build and board configuration switches.

Definicja w pliku [pager_config.h](#).

12.46.2 Dokumentacja definicji

12.46.2.1 PAGER_CONFIG_BMP280_MOUNTED

```
#define PAGER_CONFIG_BMP280_MOUNTED 0
```

Board-level presence flag for BMP280/BME280.

Current board variant has no sensor mounted. Keep the driver code in tree for future variants, but do not start the RTOS task on this hardware.

Definicja w linii 33 pliku [pager_config.h](#).

12.46.2.2 PAGER_CONFIG_UART_SENSITIVE_LOGS

```
#define PAGER_CONFIG_UART_SENSITIVE_LOGS 0
```

Enable verbose UART logs for sensitive secure/control payloads.

Keep this disabled in production because these logs can expose pairing data and protected LAVIET_FRAME_V1 payloads.

Definicja w linii 15 pliku [pager_config.h](#).

12.46.2.3 SERVICE_UART

```
#define SERVICE_UART 1
```

Build the UART service console task.

Set to 0 to remove the command service while leaving normal printf diagnostics intact.

Definicja w linii 24 pliku [pager_config.h](#).

12.47 pager_config.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_PAGER_CONFIG_H_
00007 #define APP_PAGER_CONFIG_H_
00008
00015 #define PAGER_CONFIG_UART_SENSITIVE_LOGS 0
00016
00023 #ifndef SERVICE_UART
00024 #define SERVICE_UART 1
00025 #endif
00026
00033 #define PAGER_CONFIG_BMP280_MOUNTED 0
00034
00035 #endif /* APP_PAGER_CONFIG_H_ */
```

12.48 Dokumentacja pliku

Core/App/radio_lib/common/sx1276/radio_sx1276.c

Implementacja niskopoziomowego dostępu do rejestrów SX1276.

```
#include "radio_sx1276.h"
#include "radio_sx1276_regs.h"
```

Funkcje

- static bool `sx1276_select` (const `sx1276_bus_t` *bus)
Internal helper: `sx1276_select`.
- static void `sx1276_deselect` (const `sx1276_bus_t` *bus)
Internal helper: `sx1276_deselect`.
- bool `sx1276_read_reg` (const `sx1276_bus_t` *bus, uint8_t reg, uint8_t *value)
Odczytuje pojedynczy rejestr SX1276.
- bool `sx1276_write_reg` (const `sx1276_bus_t` *bus, uint8_t reg, uint8_t value)
Zapisuje pojedynczy rejestr SX1276.
- bool `sx1276_read_burst` (const `sx1276_bus_t` *bus, uint8_t reg, uint8_t *data, uint8_t len)
Odczytuje sekwencję bajtów od rejestru startowego.
- bool `sx1276_write_burst` (const `sx1276_bus_t` *bus, uint8_t reg, const uint8_t *data, uint8_t len)
Zapisuje sekwencję bajtów od rejestru startowego.
- bool `sx1276_get_version` (const `sx1276_bus_t` *bus, uint8_t *version)
Odczytuje identyfikator wersji układu SX1276.
- bool `sx1276_set_op_mode` (const `sx1276_bus_t` *bus, uint8_t op_mode)
Ustawia rejestr trybu pracy `RegOpMode`.
- bool `sx1276_set_frequency` (const `sx1276_bus_t` *bus, uint32_t frequency_hz)
Ustawia częstotliwość nośną RF.
- bool `sx1276_set_pa_output_power` (const `sx1276_bus_t` *bus, int8_t tx_power_dbm)
Ustawia moc nadajnika dla toru `PA_BOOST`.
- bool `sx1276_set_lora_modem` (const `sx1276_bus_t` *bus, uint8_t bandwidth_code, uint8_t spreading_factor, uint8_t coding_rate_denominator, bool implicit_header, bool crc_on, bool low_datarate_optimize)
Konfiguruje parametry modemu LoRa.
- bool `sx1276_set_symbol_timeout` (const `sx1276_bus_t` *bus, uint16_t symbol_timeout)
Ustawia timeout symbolowy dla odbioru pojedynczego.
- bool `sx1276_set_preamble` (const `sx1276_bus_t` *bus, uint16_t preamble_len)
Ustawia długość preambuły LoRa.
- bool `sx1276_set_sync_word` (const `sx1276_bus_t` *bus, uint8_t sync_word)
Ustawia sync word LoRa.
- bool `sx1276_set_invert_iq` (const `sx1276_bus_t` *bus, bool invert_iq)
Włącza lub wyłącza inwersję IQ.
- bool `sx1276_set_fifo_base_addrs` (const `sx1276_bus_t` *bus, uint8_t tx_base, uint8_t rx_base)
Ustawia bazowe adresy FIFO dla TX i RX.
- bool `sx1276_set_fifo_addr_ptr` (const `sx1276_bus_t` *bus, uint8_t addr)
Ustawia wskaźnik bieżącego adresu FIFO.
- bool `sx1276_write_fifo` (const `sx1276_bus_t` *bus, const uint8_t *data, uint8_t len)
Zapisuje payload do FIFO.
- bool `sx1276_read_fifo` (const `sx1276_bus_t` *bus, uint8_t *data, uint8_t len)
Odczytuje payload z FIFO.
- bool `sx1276_set_payload_length` (const `sx1276_bus_t` *bus, uint8_t len)
Ustawia długość payloadu LoRa.
- bool `sx1276_get_rx_current_addr` (const `sx1276_bus_t` *bus, uint8_t *addr)

- Odczytuje adres początku aktualnego pakietu RX w FIFO.*
- bool `sx1276_get_rx_nb_bytes` (const `sx1276_bus_t` *bus, uint8_t *len)
Odczytuje liczbę odebranych bajtów pakietu.
- bool `sx1276_get_packet_snr_raw` (const `sx1276_bus_t` *bus, int8_t *snr_raw)
Odczytuje surowy rejestr SNR pakietu.
- bool `sx1276_get_packet_rssi_raw` (const `sx1276_bus_t` *bus, uint8_t *rssi_raw)
Odczytuje surowy rejestr RSSI pakietu.
- bool `sx1276_get_irq_flags` (const `sx1276_bus_t` *bus, uint8_t *flags)
Odczytuje flagi IRQ LoRa.
- bool `sx1276_clear_irq_flags` (const `sx1276_bus_t` *bus, uint8_t flags_to_clear)
Czyści wskazane flagi IRQ LoRa.
- bool `sx1276_map_dio` (const `sx1276_bus_t` *bus, uint8_t dio_mapping1, uint8_t dio_mapping2)
Ustawia mapowanie funkcji dla linii DIO.

12.48.1 Opis szczegółowy

Implementacja niskopoziomowego dostępu do rejestrów SX1276.

Definicja w pliku [radio_sx1276.c](#).

12.48.2 Dokumentacja funkcji

12.48.2.1 sx1276_clear_irq_flags()

```
bool sx1276_clear_irq_flags (
    const sx1276_bus_t * bus,
    uint8_t flags_to_clear)
```

Czyści wskazane flagi IRQ LoRa.

Definicja w linii 352 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_lora_init\(\)](#), [radio_lora_process\(\)](#), [radio_lora_send_async\(\)](#), [radio_lora_start_rx_continuous\(\)](#) i [radio_lora_start_rx_single\(\)](#).

12.48.2.2 sx1276_deselect()

```
void sx1276_deselect (
    const sx1276_bus_t * bus) [static]
```

Internal helper: [sx1276_deselect](#).

Definicja w linii 22 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_bus_t::nss_pin](#) i [sx1276_bus_t::nss_port](#).

Odwołania w [sx1276_read_burst\(\)](#), [sx1276_read_reg\(\)](#), [sx1276_write_burst\(\)](#) i [sx1276_write_reg\(\)](#).

12.48.2.3 sx1276_get_irq_flags()

```
bool sx1276_get_irq_flags (
    const sx1276_bus_t * bus,
    uint8_t * flags)
```

Odczytuje flagi IRQ LoRa.

Definicja w linii 347 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#).

Odwołania w [radio_read_irq_flags_retry\(\)](#) i [radio_wait_for_tx_enter_or_done\(\)](#).

12.48.2.4 sx1276_get_packet_rssi_raw()

```
bool sx1276_get_packet_rssi_raw (
    const sx1276_bus_t * bus,
    uint8_t * rssi_raw)
```

Odczytuje surowy rejestr RSSI pakietu.

Definicja w linii 342 pliku [radio_sx1276.c](#).
Odwołuje się do [sx1276_read_reg\(\)](#).
Odwołania w [radio_read_rx_packet\(\)](#).

12.48.2.5 [sx1276_get_packet_snr_raw\(\)](#)

```
bool sx1276_get_packet_snr_raw (  
    const sx1276\_bus\_t * bus,  
    int8_t * snr_raw)
```

Odczytuje surowy rejestr SNR pakietu.
Definicja w linii 329 pliku [radio_sx1276.c](#).
Odwołuje się do [sx1276_read_reg\(\)](#).
Odwołania w [radio_read_rx_packet\(\)](#).

12.48.2.6 [sx1276_get_rx_current_addr\(\)](#)

```
bool sx1276_get_rx_current_addr (  
    const sx1276\_bus\_t * bus,  
    uint8_t * addr)
```

Odczytuje adres początku aktualnego pakietu RX w FIFO.
Definicja w linii 319 pliku [radio_sx1276.c](#).
Odwołuje się do [sx1276_read_reg\(\)](#).
Odwołania w [radio_read_rx_packet\(\)](#).

12.48.2.7 [sx1276_get_rx_nb_bytes\(\)](#)

```
bool sx1276_get_rx_nb_bytes (  
    const sx1276\_bus\_t * bus,  
    uint8_t * len)
```

Odczytuje liczbę odebranych bajtów pakietu.
Definicja w linii 324 pliku [radio_sx1276.c](#).
Odwołuje się do [sx1276_read_reg\(\)](#).
Odwołania w [radio_read_rx_packet\(\)](#).

12.48.2.8 [sx1276_get_version\(\)](#)

```
bool sx1276_get_version (  
    const sx1276\_bus\_t * bus,  
    uint8_t * version)
```

Odczytuje identyfikator wersji układu SX1276.
Definicja w linii 125 pliku [radio_sx1276.c](#).
Odwołuje się do [sx1276_read_reg\(\)](#).
Odwołania w [radio_fsk_init\(\)](#), [radio_lora_init\(\)](#) i [radio_ook_init\(\)](#).

12.48.2.9 [sx1276_map_dio\(\)](#)

```
bool sx1276_map_dio (  
    const sx1276\_bus\_t * bus,  
    uint8_t dio_mapping1,  
    uint8_t dio_mapping2)
```

Ustawia mapowanie funkcji dla linii DIO.
Definicja w linii 357 pliku [radio_sx1276.c](#).
Odwołuje się do [sx1276_write_reg\(\)](#).
Odwołania w [radio_config_dio_for_rx\(\)](#) i [radio_config_dio_for_tx\(\)](#).

12.48.2.10 [sx1276_read_burst\(\)](#)

```
bool sx1276_read_burst (  
    const sx1276\_bus\_t * bus,
```

```
uint8_t reg,
uint8_t * data,
uint8_t len)
```

Odczytuje sekwencję bajtów od rejestru startowego.

Parametry

<i>bus</i>	Kontekst magistrali SPI.
<i>reg</i>	Adres startowy.
<i>data</i>	[out] Bufor danych.
<i>len</i>	Liczba bajtów do odczytu.

Zwraca

`true` gdy transfer się powiódł.

Definicja w linii 73 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_bus_t::hspi](#), [sx1276_bus_t::spi_timeout_ms](#), [sx1276_deselect\(\)](#) i [sx1276_select\(\)](#).

Odwołania w [radio_fsk_raw_read_burst\(\)](#), [radio_lora_raw_read_burst\(\)](#), [radio_ook_raw_read_burst\(\)](#), [radio_read_rx_packet\(\)](#), [radio_read_rx_packet\(\)](#) i [sx1276_read_fifo\(\)](#).

12.48.2.11 sx1276_read_fifo()

```
bool sx1276_read_fifo (
    const sx1276_bus_t * bus,
    uint8_t * data,
    uint8_t len)
```

Odczytuje payload z FIFO.

Definicja w linii 309 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_burst\(\)](#).

Odwołania w [radio_read_rx_packet\(\)](#).

12.48.2.12 sx1276_read_reg()

```
bool sx1276_read_reg (
    const sx1276_bus_t * bus,
    uint8_t reg,
    uint8_t * value)
```

Odczytuje pojedynczy rejestr SX1276.

Parametry

<i>bus</i>	Kontekst magistrali SPI.
<i>reg</i>	Adres rejestru.
<i>value</i>	[out] Odczytana wartość.

Zwraca

`true` gdy transfer się powiódł.

Definicja w linii 27 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_bus_t::hspi](#), [sx1276_bus_t::spi_timeout_ms](#), [sx1276_deselect\(\)](#) i [sx1276_select\(\)](#).

Odwołania w [radio_fsk_process\(\)](#), [radio_fsk_raw_read_reg\(\)](#), [radio_lora_process\(\)](#), [radio_lora_raw_read_reg\(\)](#), [radio_ook_process\(\)](#), [radio_ook_raw_read_reg\(\)](#), [radio_read_rx_packet\(\)](#), [radio_read_rx_packet\(\)](#), [radio_wait_for_tx_enter_or_done\(\)](#), [sx1276_get_irq_flags\(\)](#), [sx1276_get_packet_rssi_raw\(\)](#), [sx1276_get_packet_snr_raw\(\)](#), [sx1276_get_rx_current_addr\(\)](#), [sx1276_get_rx_nb_bytes\(\)](#), [sx1276_get_version\(\)](#), [sx1276_set_invert_iq\(\)](#), [sx1276_set_lora_modem\(\)](#) i [sx1276_set_symbol_timeout\(\)](#).

12.48.2.13 `sx1276_select()`

```
bool sx1276_select (
    const sx1276\_bus\_t * bus) [static]
```

Internal helper: [sx1276_select](#).

Definicja w linii 10 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_bus_t::hspi](#), [sx1276_bus_t::nss_pin](#) i [sx1276_bus_t::nss_port](#).

Odwołania w [sx1276_read_burst\(\)](#), [sx1276_read_reg\(\)](#), [sx1276_write_burst\(\)](#) i [sx1276_write_reg\(\)](#).

12.48.2.14 `sx1276_set_fifo_addr_ptr()`

```
bool sx1276_set_fifo_addr_ptr (
    const sx1276\_bus\_t * bus,
    uint8_t addr)
```

Ustawia wskaźnik bieżącego adresu FIFO.

Definicja w linii 299 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_lora_init\(\)](#), [radio_lora_send_async\(\)](#) i [radio_read_rx_packet\(\)](#).

12.48.2.15 `sx1276_set_fifo_base_addrs()`

```
bool sx1276_set_fifo_base_addrs (
    const sx1276\_bus\_t * bus,
    uint8_t tx_base,
    uint8_t rx_base)
```

Ustawia bazowe adresy FIFO dla TX i RX.

Definicja w linii 293 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#).

12.48.2.16 `sx1276_set_frequency()`

```
bool sx1276_set_frequency (
    const sx1276\_bus\_t * bus,
    uint32_t frequency_hz)
```

Ustawia częstotliwość nośną RF.

Definicja w linii 135 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_fsk_config\(\)](#), [radio_apply_lora_config\(\)](#) i [radio_apply_ook_config\(\)](#).

12.48.2.17 `sx1276_set_invert_iq()`

```
bool sx1276_set_invert_iq (
    const sx1276\_bus\_t * bus,
    bool invert_iq)
```

Włącza lub wyłącza inwersję IQ.

Definicja w linii 268 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#).

12.48.2.18 `sx1276_set_lora_modem()`

```
bool sx1276_set_lora_modem (
    const sx1276\_bus\_t * bus,
    uint8_t bandwidth_code,
    uint8_t spreading_factor,
    uint8_t coding_rate_denominator,
    bool implicit_header,
    bool crc_on,
    bool low_datarate_optimize)
```

Konfiguruje parametry modemu LoRa.

Definicja w linii 184 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#).

12.48.2.19 [sx1276_set_op_mode\(\)](#)

```
bool sx1276_set_op_mode (  
    const sx1276\_bus\_t * bus,  
    uint8_t op_mode)
```

Ustawia rejestr trybu pracy RegOpMode.

Definicja w linii 130 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_fsk_send_async\(\)](#), [radio_fsk_start_rx_continuous\(\)](#), [radio_fsk_start_rx_single\(\)](#), [radio_lora_init\(\)](#), [radio_lora_send_async\(\)](#), [radio_lora_sleep\(\)](#), [radio_lora_standby\(\)](#), [radio_lora_start_rx_continuous\(\)](#), [radio_lora_start_rx_single\(\)](#), [radio_ook_send_async\(\)](#), [radio_ook_start_rx_continuous\(\)](#), [radio_ook_start_rx_single\(\)](#), [radio_resume_after_tx\(\)](#) i [radio_resume_after_tx\(\)](#).

12.48.2.20 [sx1276_set_pa_output_power\(\)](#)

```
bool sx1276_set_pa_output_power (  
    const sx1276\_bus\_t * bus,  
    int8_t tx_power_dbm)
```

Ustawia moc nadajnika dla toru PA_BOOST.

Definicja w linii 152 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_fsk_config\(\)](#), [radio_apply_lora_config\(\)](#) i [radio_apply_ook_config\(\)](#).

12.48.2.21 [sx1276_set_payload_length\(\)](#)

```
bool sx1276_set_payload_length (  
    const sx1276\_bus\_t * bus,  
    uint8_t len)
```

Ustawia długość payloadu LoRa.

Definicja w linii 314 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#) i [radio_lora_send_async\(\)](#).

12.48.2.22 [sx1276_set_preamble\(\)](#)

```
bool sx1276_set_preamble (  
    const sx1276\_bus\_t * bus,  
    uint16_t preamble_len)
```

Ustawia długość preambuły LoRa.

Definicja w linii 257 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#).

12.48.2.23 [sx1276_set_symbol_timeout\(\)](#)

```
bool sx1276_set_symbol_timeout (  
    const sx1276\_bus\_t * bus,  
    uint16_t symbol_timeout)
```

Ustawia timeout symbolowy dla odbioru pojedynczego.

Definicja w linii 241 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_lora_start_rx_continuous\(\)](#) i [radio_lora_start_rx_single\(\)](#).

12.48.2.24 sx1276_set_sync_word()

```
bool sx1276_set_sync_word (
    const sx1276\_bus\_t * bus,
    uint8_t sync_word)
```

Ustawia sync word LoRa.

Definicja w linii 263 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#).

12.48.2.25 sx1276_write_burst()

```
bool sx1276_write_burst (
    const sx1276\_bus\_t * bus,
    uint8_t reg,
    const uint8_t * data,
    uint8_t len)
```

Zapisuje sekwencję bajtów od rejestru startowego.

Parametry

<i>bus</i>	Kontekst magistrali SPI.
<i>reg</i>	Adres startowy.
<i>data</i>	Bufor danych do zapisu.
<i>len</i>	Liczba bajtów do zapisu.

Zwraca

`true` gdy transfer się powiódł.

Definicja w linii 99 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_bus_t::hspi](#), [sx1276_bus_t::spi_timeout_ms](#), [sx1276_deselect\(\)](#) i [sx1276_select\(\)](#).

Odwołania w [radio_fsk_raw_write_burst\(\)](#), [radio_fsk_send_async\(\)](#), [radio_lora_raw_write_burst\(\)](#), [radio_ook_raw_write_burst\(\)](#), [radio_ook_send_async\(\)](#), [radio_write_sync_word\(\)](#), [radio_write_sync_word\(\)](#) i [sx1276_write_fifo\(\)](#).

12.48.2.26 sx1276_write_fifo()

```
bool sx1276_write_fifo (
    const sx1276\_bus\_t * bus,
    const uint8_t * data,
    uint8_t len)
```

Zapisuje payload do FIFO.

Definicja w linii 304 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_burst\(\)](#).

Odwołania w [radio_lora_send_async\(\)](#).

12.48.2.27 sx1276_write_reg()

```
bool sx1276_write_reg (
    const sx1276\_bus\_t * bus,
    uint8_t reg,
    uint8_t value)
```

Zapisuje pojedynczy rejestr SX1276.

Parametry

<i>bus</i>	Kontekst magistrali SPI.
<i>reg</i>	Adres rejestru.
<i>value</i>	Wartość do zapisu.

Zwraca

true gdy transfer się powiódł.

Definicja w linii 51 pliku `radio_sx1276.c`.

Odwołuje się do `sx1276_bus_t::hspi`, `sx1276_bus_t::spi_timeout_ms`, `sx1276_deselect()` i `sx1276_select()`.

Odwołania w `radio_apply_fsk_config()`, `radio_apply_lora_config()`, `radio_apply_ook_config()`, `radio_apply_threshold()`, `radio_fsk_raw_write_reg()`, `radio_fsk_send_async()`, `radio_fsk_start_rx_continuous()`, `radio_fsk_start_rx_single()`, `radio_lora_raw_write_reg()`, `radio_ook_raw_write_reg()`, `radio_ook_send_async()`, `radio_ook_start_rx_continuous()`, `radio_ook_start_rx_single()`, `radio_write_sync_word()`, `radio_write_sync_word()`, `sx1276_clear_irq_flags()`, `sx1276_map_dio()`, `sx1276_set_fifo_addr_ptr()`, `sx1276_set_fifo_base_addrs()`, `sx1276_set_frequency()`, `sx1276_set_invert_iq()`, `sx1276_set_lora_modem()`, `sx1276_set_op_mode()`, `sx1276_set_pa_output_power()`, `sx1276_set_payload_length()`, `sx1276_set_preamble()`, `sx1276_set_symbol_timeout()` i `sx1276_set_sync_word()`.

12.49 radio_sx1276.c

[Idź do dokumentacji tego pliku.](#)

```

00001
00005
00006 #include "radio_sx1276.h"
00007 #include "radio_sx1276_regs.h"
00008
00010 static bool sx1276_select(const sx1276_bus_t *bus)
00011 {
00012     if ((bus == NULL) || (bus->hspi == NULL) || (bus->nss_port == NULL))
00013     {
00014         return false;
00015     }
00016
00017     HAL_GPIO_WritePin(bus->nss_port, bus->nss_pin, GPIO_PIN_RESET);
00018     return true;
00019 }
00020
00022 static void sx1276_deselect(const sx1276_bus_t *bus)
00023 {
00024     HAL_GPIO_WritePin(bus->nss_port, bus->nss_pin, GPIO_PIN_SET);
00025 }
00026
00027 bool sx1276_read_reg(const sx1276_bus_t *bus, uint8_t reg, uint8_t *value)
00028 {
00029     uint8_t tx_buf[2];
00030     uint8_t rx_buf[2];
00031
00032     if ((value == NULL) || !sx1276_select(bus))
00033     {
00034         return false;
00035     }
00036
00037     tx_buf[0] = (uint8_t)(reg & SX1276_SPI_READ_MASK);
00038     tx_buf[1] = 0x00U;
00039
00040     if (HAL_SPI_TransmitReceive(bus->hspi, tx_buf, rx_buf, 2U, bus->spi_timeout_ms) != HAL_OK)
00041     {
00042         sx1276_deselect(bus);
00043         return false;
00044     }
00045
00046     sx1276_deselect(bus);
00047     *value = rx_buf[1];
00048     return true;
00049 }
00050
00051 bool sx1276_write_reg(const sx1276_bus_t *bus, uint8_t reg, uint8_t value)
00052 {
00053     uint8_t tx_buf[2];
00054
00055     if (!sx1276_select(bus))
00056     {
00057         return false;
00058     }

```

```

00059
00060     tx_buf[0] = (uint8_t)(reg | SX1276_SPI_WRITE_MASK);
00061     tx_buf[1] = value;
00062
00063     if (HAL_SPI_Transmit(bus->hspi, tx_buf, 2U, bus->spi_timeout_ms) != HAL_OK)
00064     {
00065         sx1276_deselect(bus);
00066         return false;
00067     }
00068
00069     sx1276_deselect(bus);
00070     return true;
00071 }
00072
00073 bool sx1276_read_burst(const sx1276_bus_t *bus, uint8_t reg, uint8_t *data, uint8_t len)
00074 {
00075     uint8_t addr;
00076
00077     if ((data == NULL) || (len == 0U) || !sx1276_select(bus))
00078     {
00079         return false;
00080     }
00081
00082     addr = (uint8_t)(reg & SX1276_SPI_READ_MASK);
00083     if (HAL_SPI_Transmit(bus->hspi, &addr, 1U, bus->spi_timeout_ms) != HAL_OK)
00084     {
00085         sx1276_deselect(bus);
00086         return false;
00087     }
00088
00089     if (HAL_SPI_Receive(bus->hspi, data, len, bus->spi_timeout_ms) != HAL_OK)
00090     {
00091         sx1276_deselect(bus);
00092         return false;
00093     }
00094
00095     sx1276_deselect(bus);
00096     return true;
00097 }
00098
00099 bool sx1276_write_burst(const sx1276_bus_t *bus, uint8_t reg, const uint8_t *data, uint8_t len)
00100 {
00101     uint8_t addr;
00102
00103     if ((data == NULL) || (len == 0U) || !sx1276_select(bus))
00104     {
00105         return false;
00106     }
00107
00108     addr = (uint8_t)(reg | SX1276_SPI_WRITE_MASK);
00109     if (HAL_SPI_Transmit(bus->hspi, &addr, 1U, bus->spi_timeout_ms) != HAL_OK)
00110     {
00111         sx1276_deselect(bus);
00112         return false;
00113     }
00114
00115     if (HAL_SPI_Transmit(bus->hspi, (uint8_t *)data, len, bus->spi_timeout_ms) != HAL_OK)
00116     {
00117         sx1276_deselect(bus);
00118         return false;
00119     }
00120
00121     sx1276_deselect(bus);
00122     return true;
00123 }
00124
00125 bool sx1276_get_version(const sx1276_bus_t *bus, uint8_t *version)
00126 {
00127     return sx1276_read_reg(bus, SX1276_REG_VERSION, version);
00128 }
00129
00130 bool sx1276_set_op_mode(const sx1276_bus_t *bus, uint8_t op_mode)
00131 {
00132     return sx1276_write_reg(bus, SX1276_REG_OP_MODE, op_mode);
00133 }
00134
00135 bool sx1276_set_frequency(const sx1276_bus_t *bus, uint32_t frequency_hz)
00136 {
00137     uint64_t frf;
00138     uint8_t frf_msb;
00139     uint8_t frf_mid;
00140     uint8_t frf_lsb;
00141
00142     frf = (((uint64_t)frequency_hz) << 19) / 32000000ULL;
00143     frf_msb = (uint8_t)((frf >> 16) & 0xFFU);
00144     frf_mid = (uint8_t)((frf >> 8) & 0xFFU);
00145     frf_lsb = (uint8_t)(frf & 0xFFU);

```

```

00146
00147     return sx1276_write_reg(bus, SX1276_REG_FRF_MSB, frf_msb) &&
00148         sx1276_write_reg(bus, SX1276_REG_FRF_MID, frf_mid) &&
00149         sx1276_write_reg(bus, SX1276_REG_FRF_LSB, frf_lsb);
00150 }
00151
00152 bool sx1276_set_pa_output_power(const sx1276_bus_t *bus, int8_t tx_power_dbm)
00153 {
00154     uint8_t pa_config;
00155     uint8_t pa_dac;
00156     int8_t out_power;
00157
00158     if (tx_power_dbm < 2)
00159     {
00160         tx_power_dbm = 2;
00161     }
00162     else if (tx_power_dbm > 20)
00163     {
00164         tx_power_dbm = 20;
00165     }
00166
00167     if (tx_power_dbm > 17)
00168     {
00169         pa_dac = SX1276_PA_DAC_ENABLE_20DBM;
00170         out_power = (int8_t)(tx_power_dbm - 5);
00171     }
00172     else
00173     {
00174         pa_dac = SX1276_PA_DAC_DISABLE_17DBM;
00175         out_power = (int8_t)(tx_power_dbm - 2);
00176     }
00177
00178     pa_config = (uint8_t)(0x80U | ((uint8_t)out_power & 0x0FU));
00179
00180     return sx1276_write_reg(bus, SX1276_REG_PA_DAC, pa_dac) &&
00181         sx1276_write_reg(bus, SX1276_REG_PA_CONFIG, pa_config);
00182 }
00183
00184 bool sx1276_set_lora_modem(const sx1276_bus_t *bus,
00185     uint8_t bandwidth_code,
00186     uint8_t spreading_factor,
00187     uint8_t coding_rate_denominator,
00188     bool implicit_header,
00189     bool crc_on,
00190     bool low_datarate_optimize)
00191 {
00192     uint8_t modem_config_1;
00193     uint8_t modem_config_2;
00194     uint8_t modem_config_3;
00195     uint8_t coding_rate_field;
00196     uint8_t detect_optimize;
00197
00198     if ((bandwidth_code > 9U) ||
00199         (spreading_factor < 6U) || (spreading_factor > 12U) ||
00200         (coding_rate_denominator < 5U) || (coding_rate_denominator > 8U))
00201     {
00202         return false;
00203     }
00204
00205     coding_rate_field = (uint8_t)(coding_rate_denominator - 4U);
00206
00207     modem_config_1 = (uint8_t)((bandwidth_code << 4) |
00208         (coding_rate_field << 1) |
00209         (implicit_header ? 1U : 0U));
00210
00211     modem_config_2 = (uint8_t)((spreading_factor << 4) |
00212         (crc_on ? 0x04U : 0x00U));
00213
00214     modem_config_3 = (uint8_t)((low_datarate_optimize ? 0x08U : 0x00U) | 0x04U);
00215
00216     if (!sx1276_write_reg(bus, SX1276_REG_MODEM_CONFIG_1, modem_config_1) ||
00217         !sx1276_write_reg(bus, SX1276_REG_MODEM_CONFIG_2, modem_config_2) ||
00218         !sx1276_write_reg(bus, SX1276_REG_MODEM_CONFIG_3, modem_config_3))
00219     {
00220         return false;
00221     }
00222
00223     if (!sx1276_read_reg(bus, SX1276_REG_DETECTION_OPTIMIZE, &detect_optimize))
00224     {
00225         return false;
00226     }
00227
00228     detect_optimize = (uint8_t)(detect_optimize & 0xF8U);
00229     if (spreading_factor == 6U)
00230     {
00231         detect_optimize |= 0x05U;
00232         return sx1276_write_reg(bus, SX1276_REG_DETECTION_OPTIMIZE, detect_optimize) &&

```

```

00233         sx1276_write_reg(bus, SX1276_REG_DETECTION_THRESHOLD, 0x0CU);
00234     }
00235
00236     detect_optimize |= 0x03U;
00237     return sx1276_write_reg(bus, SX1276_REG_DETECTION_OPTIMIZE, detect_optimize) &&
00238         sx1276_write_reg(bus, SX1276_REG_DETECTION_THRESHOLD, 0x0AU);
00239 }
00240
00241 bool sx1276_set_symbol_timeout(const sx1276_bus_t *bus, uint16_t symbol_timeout)
00242 {
00243     uint8_t modem_config_2;
00244
00245     if (!sx1276_read_reg(bus, SX1276_REG_MODEM_CONFIG_2, &modem_config_2))
00246     {
00247         return false;
00248     }
00249
00250     modem_config_2 &= (uint8_t)~0x03U;
00251     modem_config_2 |= (uint8_t)((symbol_timeout >> 8) & 0x03U);
00252
00253     return sx1276_write_reg(bus, SX1276_REG_MODEM_CONFIG_2, modem_config_2) &&
00254         sx1276_write_reg(bus, SX1276_REG_SYMB_TIMEOUT_LSB, (uint8_t)(symbol_timeout & 0xFFU));
00255 }
00256
00257 bool sx1276_set_preamble(const sx1276_bus_t *bus, uint16_t preamble_len)
00258 {
00259     return sx1276_write_reg(bus, SX1276_REG_PREAMBLE_MSB, (uint8_t)((preamble_len >> 8) & 0xFFU)) &&
00260         sx1276_write_reg(bus, SX1276_REG_PREAMBLE_LSB, (uint8_t)(preamble_len & 0xFFU));
00261 }
00262
00263 bool sx1276_set_sync_word(const sx1276_bus_t *bus, uint8_t sync_word)
00264 {
00265     return sx1276_write_reg(bus, SX1276_REG_SYNC_WORD, sync_word);
00266 }
00267
00268 bool sx1276_set_invert_iq(const sx1276_bus_t *bus, bool invert_iq)
00269 {
00270     uint8_t invert_iq_reg;
00271     uint8_t invert_iq2_reg;
00272
00273     if (!sx1276_read_reg(bus, SX1276_REG_INVERT_IQ, &invert_iq_reg))
00274     {
00275         return false;
00276     }
00277
00278     invert_iq_reg &= (uint8_t)~0x40U;
00279     if (invert_iq)
00280     {
00281         invert_iq_reg = (uint8_t)(invert_iq_reg | 0x40U);
00282         invert_iq2_reg = 0x19U;
00283     }
00284     else
00285     {
00286         invert_iq2_reg = 0x1DU;
00287     }
00288
00289     return sx1276_write_reg(bus, SX1276_REG_INVERT_IQ, invert_iq_reg) &&
00290         sx1276_write_reg(bus, SX1276_REG_INVERT_IQ2, invert_iq2_reg);
00291 }
00292
00293 bool sx1276_set_fifo_base_addrs(const sx1276_bus_t *bus, uint8_t tx_base, uint8_t rx_base)
00294 {
00295     return sx1276_write_reg(bus, SX1276_REG_FIFO_TX_BASE_ADDR, tx_base) &&
00296         sx1276_write_reg(bus, SX1276_REG_FIFO_RX_BASE_ADDR, rx_base);
00297 }
00298
00299 bool sx1276_set_fifo_addr_ptr(const sx1276_bus_t *bus, uint8_t addr)
00300 {
00301     return sx1276_write_reg(bus, SX1276_REG_FIFO_ADDR_PTR, addr);
00302 }
00303
00304 bool sx1276_write_fifo(const sx1276_bus_t *bus, const uint8_t *data, uint8_t len)
00305 {
00306     return sx1276_write_burst(bus, SX1276_REG_FIFO, data, len);
00307 }
00308
00309 bool sx1276_read_fifo(const sx1276_bus_t *bus, uint8_t *data, uint8_t len)
00310 {
00311     return sx1276_read_burst(bus, SX1276_REG_FIFO, data, len);
00312 }
00313
00314 bool sx1276_set_payload_length(const sx1276_bus_t *bus, uint8_t len)
00315 {
00316     return sx1276_write_reg(bus, SX1276_REG_PAYLOAD_LENGTH, len);
00317 }
00318
00319 bool sx1276_get_rx_current_addr(const sx1276_bus_t *bus, uint8_t *addr)

```

```

00320 {
00321     return sx1276_read_reg(bus, SX1276_REG_FIFO_RX_CURRENT_ADDR, addr);
00322 }
00323
00324 bool sx1276_get_rx_nb_bytes(const sx1276_bus_t *bus, uint8_t *len)
00325 {
00326     return sx1276_read_reg(bus, SX1276_REG_RX_NB_BYTES, len);
00327 }
00328
00329 bool sx1276_get_packet_snr_raw(const sx1276_bus_t *bus, int8_t *snr_raw)
00330 {
00331     uint8_t value;
00332     if ((snr_raw == NULL) || !sx1276_read_reg(bus, SX1276_REG_PKT_SNR_VALUE, &value))
00333     {
00334         return false;
00335     }
00336     *snr_raw = (int8_t)value;
00337     return true;
00338 }
00339
00340 bool sx1276_get_packet_rssi_raw(const sx1276_bus_t *bus, uint8_t *rssi_raw)
00341 {
00342     return sx1276_read_reg(bus, SX1276_REG_PKT_RSSI_VALUE, rssi_raw);
00343 }
00344
00345 bool sx1276_get_irq_flags(const sx1276_bus_t *bus, uint8_t *flags)
00346 {
00347     return sx1276_read_reg(bus, SX1276_REG_IRQ_FLAGS, flags);
00348 }
00349
00350 bool sx1276_clear_irq_flags(const sx1276_bus_t *bus, uint8_t flags_to_clear)
00351 {
00352     return sx1276_write_reg(bus, SX1276_REG_IRQ_FLAGS, flags_to_clear);
00353 }
00354
00355 bool sx1276_map_dio(const sx1276_bus_t *bus, uint8_t dio_mapping1, uint8_t dio_mapping2)
00356 {
00357     return sx1276_write_reg(bus, SX1276_REG_DIO_MAPPING_1, dio_mapping1) &&
00358         sx1276_write_reg(bus, SX1276_REG_DIO_MAPPING_2, dio_mapping2);
00359 }
00360
00361 }

```

12.50 Dokumentacja pliku Core/App/radio_lib/common/sx1276/radio_sx1276.h

Niskopoziomowe API dostępu do rejestrów SX1276.

```

#include "stm32u5xx_hal.h"
#include <stdbool.h>
#include <stdint.h>

```

Struktury danych

- struct [sx1276_bus_t](#)
Kontekst magistrali SPI dla układu SX1276.

Funkcje

- bool [sx1276_read_reg](#) (const [sx1276_bus_t](#) *bus, uint8_t reg, uint8_t *value)
Odczytuje pojedynczy rejestr SX1276.
- bool [sx1276_write_reg](#) (const [sx1276_bus_t](#) *bus, uint8_t reg, uint8_t value)
Zapisuje pojedynczy rejestr SX1276.
- bool [sx1276_read_burst](#) (const [sx1276_bus_t](#) *bus, uint8_t reg, uint8_t *data, uint8_t len)
Odczytuje sekwencję bajtów od rejestru startowego.
- bool [sx1276_write_burst](#) (const [sx1276_bus_t](#) *bus, uint8_t reg, const uint8_t *data, uint8_t len)
Zapisuje sekwencję bajtów od rejestru startowego.

Pomocnicze operacje konfiguracyjne

- bool `sx1276_get_version` (const `sx1276_bus_t` *bus, uint8_t *version)
Odczytuje identyfikator wersji układu SX1276.
- bool `sx1276_set_op_mode` (const `sx1276_bus_t` *bus, uint8_t op_mode)
Ustawia rejestr trybu pracy `RegOpMode`.
- bool `sx1276_set_frequency` (const `sx1276_bus_t` *bus, uint32_t frequency_hz)
Ustawia częstotliwość nośną RF.
- bool `sx1276_set_pa_output_power` (const `sx1276_bus_t` *bus, int8_t tx_power_dbm)
Ustawia moc nadajnika dla toru `PA_BOOST`.
- bool `sx1276_set_lora_modem` (const `sx1276_bus_t` *bus, uint8_t bandwidth_code, uint8_t spreading_factor, uint8_t coding_rate_denominator, bool implicit_header, bool crc_on, bool low_datarate_optimize)
Konfiguruje parametry modemu LoRa.
- bool `sx1276_set_symbol_timeout` (const `sx1276_bus_t` *bus, uint16_t symbol_timeout)
Ustawia timeout symbolowy dla odbioru pojedynczego.
- bool `sx1276_set_preamble` (const `sx1276_bus_t` *bus, uint16_t preamble_len)
Ustawia długość preambuły LoRa.
- bool `sx1276_set_sync_word` (const `sx1276_bus_t` *bus, uint8_t sync_word)
Ustawia sync word LoRa.
- bool `sx1276_set_invert_iq` (const `sx1276_bus_t` *bus, bool invert_iq)
Włącza lub wyłącza inwersję IQ.

FIFO i pakiet

- bool `sx1276_set_fifo_base_addrs` (const `sx1276_bus_t` *bus, uint8_t tx_base, uint8_t rx_base)
Ustawia bazowe adresy FIFO dla TX i RX.
- bool `sx1276_set_fifo_addr_ptr` (const `sx1276_bus_t` *bus, uint8_t addr)
Ustawia wskaźnik bieżącego adresu FIFO.
- bool `sx1276_write_fifo` (const `sx1276_bus_t` *bus, const uint8_t *data, uint8_t len)
Zapisuje payload do FIFO.
- bool `sx1276_read_fifo` (const `sx1276_bus_t` *bus, uint8_t *data, uint8_t len)
Odczytuje payload z FIFO.
- bool `sx1276_set_payload_length` (const `sx1276_bus_t` *bus, uint8_t len)
Ustawia długość payloadu LoRa.
- bool `sx1276_get_rx_current_addr` (const `sx1276_bus_t` *bus, uint8_t *addr)
Odczytuje adres początku aktualnego pakietu RX w FIFO.
- bool `sx1276_get_rx_nb_bytes` (const `sx1276_bus_t` *bus, uint8_t *len)
Odczytuje liczbę odebranych bajtów pakietu.
- bool `sx1276_get_packet_snr_raw` (const `sx1276_bus_t` *bus, int8_t *snr_raw)
Odczytuje surowy rejestr SNR pakietu.
- bool `sx1276_get_packet_rssi_raw` (const `sx1276_bus_t` *bus, uint8_t *rssi_raw)
Odczytuje surowy rejestr RSSI pakietu.

IRQ i mapowanie DIO

- bool `sx1276_get_irq_flags` (const `sx1276_bus_t` *bus, uint8_t *flags)
Odczytuje flagi IRQ LoRa.
- bool `sx1276_clear_irq_flags` (const `sx1276_bus_t` *bus, uint8_t flags_to_clear)
Czyści wskazane flagi IRQ LoRa.
- bool `sx1276_map_dio` (const `sx1276_bus_t` *bus, uint8_t dio_mapping1, uint8_t dio_mapping2)
Ustawia mapowanie funkcji dla linii DIO.

12.50.1 Opis szczegółowy

Niskopoziomowe API dostępu do rejestrów SX1276.

Definicja w pliku `radio_sx1276.h`.

12.50.2 Dokumentacja funkcji

12.50.2.1 sx1276_clear_irq_flags()

```
bool sx1276_clear_irq_flags (
    const sx1276_bus_t * bus,
    uint8_t flags_to_clear)
```

Czyści wskazane flagi IRQ LoRa.

Definicja w linii 352 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_lora_init\(\)](#), [radio_lora_process\(\)](#), [radio_lora_send_async\(\)](#), [radio_lora_start_rx_continuous\(\)](#) i [radio_lora_start_rx_single\(\)](#).

12.50.2.2 sx1276_get_irq_flags()

```
bool sx1276_get_irq_flags (
    const sx1276_bus_t * bus,
    uint8_t * flags)
```

Odczytuje flagi IRQ LoRa.

Definicja w linii 347 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#).

Odwołania w [radio_read_irq_flags_retry\(\)](#) i [radio_wait_for_tx_enter_or_done\(\)](#).

12.50.2.3 sx1276_get_packet_rssi_raw()

```
bool sx1276_get_packet_rssi_raw (
    const sx1276_bus_t * bus,
    uint8_t * rssi_raw)
```

Odczytuje surowy rejestr RSSI pakietu.

Definicja w linii 342 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#).

Odwołania w [radio_read_rx_packet\(\)](#).

12.50.2.4 sx1276_get_packet_snr_raw()

```
bool sx1276_get_packet_snr_raw (
    const sx1276_bus_t * bus,
    int8_t * snr_raw)
```

Odczytuje surowy rejestr SNR pakietu.

Definicja w linii 329 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#).

Odwołania w [radio_read_rx_packet\(\)](#).

12.50.2.5 sx1276_get_rx_current_addr()

```
bool sx1276_get_rx_current_addr (
    const sx1276_bus_t * bus,
    uint8_t * addr)
```

Odczytuje adres początku aktualnego pakietu RX w FIFO.

Definicja w linii 319 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#).

Odwołania w [radio_read_rx_packet\(\)](#).

12.50.2.6 sx1276_get_rx_nb_bytes()

```
bool sx1276_get_rx_nb_bytes (
    const sx1276_bus_t * bus,
    uint8_t * len)
```

Odczytuje liczbę odebranych bajtów pakietu.

Definicja w linii 324 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#).
 Odwołania w [radio_read_rx_packet\(\)](#).

12.50.2.7 sx1276_get_version()

```
bool sx1276_get_version (
    const sx1276\_bus\_t * bus,
    uint8_t * version)
```

Odczytuje identyfikator wersji układu SX1276.

Definicja w linii 125 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#).

Odwołania w [radio_fsk_init\(\)](#), [radio_lora_init\(\)](#) i [radio_ook_init\(\)](#).

12.50.2.8 sx1276_map_dio()

```
bool sx1276_map_dio (
    const sx1276\_bus\_t * bus,
    uint8_t dio_mapping1,
    uint8_t dio_mapping2)
```

Ustawia mapowanie funkcji dla linii DIO.

Definicja w linii 357 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_config_dio_for_rx\(\)](#) i [radio_config_dio_for_tx\(\)](#).

12.50.2.9 sx1276_read_burst()

```
bool sx1276_read_burst (
    const sx1276\_bus\_t * bus,
    uint8_t reg,
    uint8_t * data,
    uint8_t len)
```

Odczytuje sekwencję bajtów od rejestru startowego.

Parametry

<i>bus</i>	Kontekst magistrali SPI.
<i>reg</i>	Adres startowy.
<i>data</i>	[out] Bufor danych.
<i>len</i>	Liczba bajtów do odczytu.

Zwraca

`true` gdy transfer się powiódł.

Definicja w linii 73 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_bus_t::hspi](#), [sx1276_bus_t::spi_timeout_ms](#), [sx1276_deselect\(\)](#) i [sx1276_select\(\)](#).

Odwołania w [radio_fsk_raw_read_burst\(\)](#), [radio_lora_raw_read_burst\(\)](#), [radio_ook_raw_read_burst\(\)](#), [radio_read_rx_packet\(\)](#), [radio_read_rx_packet\(\)](#) i [sx1276_read_fifo\(\)](#).

12.50.2.10 sx1276_read_fifo()

```
bool sx1276_read_fifo (
    const sx1276\_bus\_t * bus,
    uint8_t * data,
    uint8_t len)
```

Odczytuje payload z FIFO.

Definicja w linii 309 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_burst\(\)](#).
 Odwołania w [radio_read_rx_packet\(\)](#).

12.50.2.11 sx1276_read_reg()

```
bool sx1276_read_reg (
    const sx1276\_bus\_t * bus,
    uint8_t reg,
    uint8_t * value)
```

Odczytuje pojedynczy rejestr SX1276.

Parametry

<i>bus</i>	Kontekst magistrali SPI.
<i>reg</i>	Adres rejestru.
<i>value</i>	[out] Odczytana wartość.

Zwraca

`true` gdy transfer się powiódł.

Definicja w linii 27 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_bus_t::hspi](#), [sx1276_bus_t::spi_timeout_ms](#), [sx1276_deselect\(\)](#) i [sx1276_select\(\)](#).

Odwołania w [radio_fsk_process\(\)](#), [radio_fsk_raw_read_reg\(\)](#), [radio_lora_process\(\)](#), [radio_lora_raw_read_reg\(\)](#), [radio_ook_process\(\)](#), [radio_ook_raw_read_reg\(\)](#), [radio_read_rx_packet\(\)](#), [radio_read_rx_packet\(\)](#), [radio_wait_for_tx_enter_or_done\(\)](#), [sx1276_get_irq_flags\(\)](#), [sx1276_get_packet_rssi_raw\(\)](#), [sx1276_get_packet_snr_raw\(\)](#), [sx1276_get_rx_current_addr\(\)](#), [sx1276_get_rx_nb_bytes\(\)](#), [sx1276_get_version\(\)](#), [sx1276_set_invert_iq\(\)](#), [sx1276_set_lora_modem\(\)](#) i [sx1276_set_symbol_timeout\(\)](#).

12.50.2.12 sx1276_set_fifo_addr_ptr()

```
bool sx1276_set_fifo_addr_ptr (
    const sx1276\_bus\_t * bus,
    uint8_t addr)
```

Ustawia wskaźnik bieżącego adresu FIFO.

Definicja w linii 299 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_lora_init\(\)](#), [radio_lora_send_async\(\)](#) i [radio_read_rx_packet\(\)](#).

12.50.2.13 sx1276_set_fifo_base_addrs()

```
bool sx1276_set_fifo_base_addrs (
    const sx1276\_bus\_t * bus,
    uint8_t tx_base,
    uint8_t rx_base)
```

Ustawia bazowe adresy FIFO dla TX i RX.

Definicja w linii 293 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#).

12.50.2.14 sx1276_set_frequency()

```
bool sx1276_set_frequency (
    const sx1276\_bus\_t * bus,
    uint32_t frequency_hz)
```

Ustawia częstotliwość nośną RF.

Definicja w linii 135 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_fsk_config\(\)](#), [radio_apply_lora_config\(\)](#) i [radio_apply_ook_config\(\)](#).

12.50.2.15 `sx1276_set_invert_iq()`

```
bool sx1276_set_invert_iq (  
    const sx1276\_bus\_t * bus,  
    bool invert_iq)
```

Włącza lub wyłącza inwersję IQ.

Definicja w linii 268 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#).

12.50.2.16 `sx1276_set_lora_modem()`

```
bool sx1276_set_lora_modem (  
    const sx1276\_bus\_t * bus,  
    uint8_t bandwidth_code,  
    uint8_t spreading_factor,  
    uint8_t coding_rate_denominator,  
    bool implicit_header,  
    bool crc_on,  
    bool low_datarate_optimize)
```

Konfiguruje parametry modemu LoRa.

Definicja w linii 184 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#).

12.50.2.17 `sx1276_set_op_mode()`

```
bool sx1276_set_op_mode (  
    const sx1276\_bus\_t * bus,  
    uint8_t op_mode)
```

Ustawia rejestr trybu pracy RegOpMode.

Definicja w linii 130 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_fsk_send_async\(\)](#), [radio_fsk_start_rx_continuous\(\)](#), [radio_fsk_start_rx_single\(\)](#), [radio_lora_init\(\)](#), [radio_lora_send_async\(\)](#), [radio_lora_sleep\(\)](#), [radio_lora_standby\(\)](#), [radio_lora_start_rx_continuous\(\)](#), [radio_lora_start_rx_single\(\)](#), [radio_ook_send_async\(\)](#), [radio_ook_start_rx_continuous\(\)](#), [radio_ook_start_rx_single\(\)](#), [radio_resume_after_tx\(\)](#) i [radio_resume_after_tx\(\)](#).

12.50.2.18 `sx1276_set_pa_output_power()`

```
bool sx1276_set_pa_output_power (  
    const sx1276\_bus\_t * bus,  
    int8_t tx_power_dbm)
```

Ustawia moc nadajnika dla toru PA_BOOST.

Definicja w linii 152 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_fsk_config\(\)](#), [radio_apply_lora_config\(\)](#) i [radio_apply_ook_config\(\)](#).

12.50.2.19 `sx1276_set_payload_length()`

```
bool sx1276_set_payload_length (  
    const sx1276\_bus\_t * bus,  
    uint8_t len)
```

Ustawia długość payloadu LoRa.

Definicja w linii 314 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#) i [radio_lora_send_async\(\)](#).

12.50.2.20 sx1276_set_preamble()

```
bool sx1276_set_preamble (
    const sx1276_bus_t * bus,
    uint16_t preamble_len)
```

Ustawia długość preambuły LoRa.

Definicja w linii 257 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#).

12.50.2.21 sx1276_set_symbol_timeout()

```
bool sx1276_set_symbol_timeout (
    const sx1276_bus_t * bus,
    uint16_t symbol_timeout)
```

Ustawia timeout symbolowy dla odbioru pojedynczego.

Definicja w linii 241 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_read_reg\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#), [radio_lora_start_rx_continuous\(\)](#) i [radio_lora_start_rx_single\(\)](#).

12.50.2.22 sx1276_set_sync_word()

```
bool sx1276_set_sync_word (
    const sx1276_bus_t * bus,
    uint8_t sync_word)
```

Ustawia sync word LoRa.

Definicja w linii 263 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#).

Odwołania w [radio_apply_lora_config\(\)](#).

12.50.2.23 sx1276_write_burst()

```
bool sx1276_write_burst (
    const sx1276_bus_t * bus,
    uint8_t reg,
    const uint8_t * data,
    uint8_t len)
```

Zapisuje sekwencję bajtów od rejestru startowego.

Parametry

<i>bus</i>	Kontekst magistrali SPI.
<i>reg</i>	Adres startowy.
<i>data</i>	Bufor danych do zapisu.
<i>len</i>	Liczba bajtów do zapisu.

Zwraca

`true` gdy transfer się powiódł.

Definicja w linii 99 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_bus_t::hspi](#), [sx1276_bus_t::spi_timeout_ms](#), [sx1276_deselect\(\)](#) i [sx1276_select\(\)](#).

Odwołania w [radio_fsk_raw_write_burst\(\)](#), [radio_fsk_send_async\(\)](#), [radio_lora_raw_write_burst\(\)](#), [radio_ook_raw_write_burst\(\)](#), [radio_ook_send_async\(\)](#), [radio_write_sync_word\(\)](#), [radio_write_sync_word\(\)](#) i [sx1276_write_fifo\(\)](#).

12.50.2.24 sx1276_write_fifo()

```
bool sx1276_write_fifo (
```

```
const sx1276_bus_t * bus,
const uint8_t * data,
uint8_t len)
```

Zapisuje payload do FIFO.

Definicja w linii 304 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_write_burst\(\)](#).

Odwołania w [radio_lora_send_async\(\)](#).

12.50.2.25 sx1276_write_reg()

```
bool sx1276_write_reg (
    const sx1276_bus_t * bus,
    uint8_t reg,
    uint8_t value)
```

Zapisuje pojedynczy rejestr SX1276.

Parametry

<i>bus</i>	Kontekst magistrali SPI.
<i>reg</i>	Adres rejestru.
<i>value</i>	Wartość do zapisu.

Zwraca

true gdy transfer się powiódł.

Definicja w linii 51 pliku [radio_sx1276.c](#).

Odwołuje się do [sx1276_bus_t::hspi](#), [sx1276_bus_t::spi_timeout_ms](#), [sx1276_deselect\(\)](#) i [sx1276_select\(\)](#).

Odwołania w [radio_apply_fsk_config\(\)](#), [radio_apply_lora_config\(\)](#), [radio_apply_ook_config\(\)](#), [radio_apply_threshold\(\)](#), [radio_fsk_raw_write_reg\(\)](#), [radio_fsk_send_async\(\)](#), [radio_fsk_start_rx_continuous\(\)](#), [radio_fsk_start_rx_single\(\)](#), [radio_lora_raw_write_reg\(\)](#), [radio_ook_raw_write_reg\(\)](#), [radio_ook_send_async\(\)](#), [radio_ook_start_rx_continuous\(\)](#), [radio_ook_start_rx_single\(\)](#), [radio_write_sync_word\(\)](#), [radio_write_sync_word\(\)](#), [sx1276_clear_irq_flags\(\)](#), [sx1276_map_dio\(\)](#), [sx1276_set_fifo_addr_ptr\(\)](#), [sx1276_set_fifo_base_addrs\(\)](#), [sx1276_set_frequency\(\)](#), [sx1276_set_invert_iq\(\)](#), [sx1276_set_lora_modem\(\)](#), [sx1276_set_op_mode\(\)](#), [sx1276_set_pa_output_power\(\)](#), [sx1276_set_payload_length\(\)](#), [sx1276_set_preamble\(\)](#), [sx1276_set_symbol_timeout\(\)](#) i [sx1276_set_sync_word\(\)](#).

12.51 radio_sx1276.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_RADIO_LIB_COMMON_SX1276_RADIO_SX1276_H_
00007 #define APP_RADIO_LIB_COMMON_SX1276_RADIO_SX1276_H_
00008
00009 #include "stm32u5xx_hal.h"
00010
00011 #include <stdbool.h>
00012 #include <stdint.h>
00013
00017 typedef struct
00018 {
00019     SPI_HandleTypeDef *hspi;
00020     GPIO_TypeDef *nss_port;
00021     uint16_t nss_pin;
00022     uint32_t spi_timeout_ms;
00023 } sx1276_bus_t;
00024
00032 bool sx1276_read_reg(const sx1276_bus_t *bus, uint8_t reg, uint8_t *value);
00033
00041 bool sx1276_write_reg(const sx1276_bus_t *bus, uint8_t reg, uint8_t value);
00042
00051 bool sx1276_read_burst(const sx1276_bus_t *bus, uint8_t reg, uint8_t *data, uint8_t len);
00052
00061 bool sx1276_write_burst(const sx1276_bus_t *bus, uint8_t reg, const uint8_t *data, uint8_t len);
```

```

00062
00068 bool sx1276_get_version(const sx1276_bus_t *bus, uint8_t *version);
00070 bool sx1276_set_op_mode(const sx1276_bus_t *bus, uint8_t op_mode);
00072 bool sx1276_set_frequency(const sx1276_bus_t *bus, uint32_t frequency_hz);
00074 bool sx1276_set_pa_output_power(const sx1276_bus_t *bus, int8_t tx_power_dbm);
00076 bool sx1276_set_lora_modem(const sx1276_bus_t *bus,
00077                             uint8_t bandwidth_code,
00078                             uint8_t spreading_factor,
00079                             uint8_t coding_rate_denominator,
00080                             bool implicit_header,
00081                             bool crc_on,
00082                             bool low_data_rate_optimize);
00084 bool sx1276_set_symbol_timeout(const sx1276_bus_t *bus, uint16_t symbol_timeout);
00086 bool sx1276_set_preamble(const sx1276_bus_t *bus, uint16_t preamble_len);
00088 bool sx1276_set_sync_word(const sx1276_bus_t *bus, uint8_t sync_word);
00090 bool sx1276_set_invert_iq(const sx1276_bus_t *bus, bool invert_iq);
00092
00098 bool sx1276_set_fifo_base_addrs(const sx1276_bus_t *bus, uint8_t tx_base, uint8_t rx_base);
00100 bool sx1276_set_fifo_addr_ptr(const sx1276_bus_t *bus, uint8_t addr);
00102 bool sx1276_write_fifo(const sx1276_bus_t *bus, const uint8_t *data, uint8_t len);
00104 bool sx1276_read_fifo(const sx1276_bus_t *bus, uint8_t *data, uint8_t len);
00106 bool sx1276_set_payload_length(const sx1276_bus_t *bus, uint8_t len);
00108 bool sx1276_get_rx_current_addr(const sx1276_bus_t *bus, uint8_t *addr);
00110 bool sx1276_get_rx_nb_bytes(const sx1276_bus_t *bus, uint8_t *len);
00112 bool sx1276_get_packet_snr_raw(const sx1276_bus_t *bus, int8_t *snr_raw);
00114 bool sx1276_get_packet_rssi_raw(const sx1276_bus_t *bus, uint8_t *rssi_raw);
00116
00122 bool sx1276_get_irq_flags(const sx1276_bus_t *bus, uint8_t *flags);
00124 bool sx1276_clear_irq_flags(const sx1276_bus_t *bus, uint8_t flags_to_clear);
00126 bool sx1276_map_dio(const sx1276_bus_t *bus, uint8_t dio_mapping1, uint8_t dio_mapping2);
00128
00129 #endif /* APP_RADIO_LIB_COMMON_SX1276_RADIO_SX1276_H */

```

12.52 Dokumentacja pliku Core/App/radio_lib/common/sx1276/radio_sx1276_regs.h

Mapa rejestrów SX1276 i definicje bitów używane w bibliotece.

Definicje

- #define SX1276_REG_FIFO 0x00U
- #define SX1276_REG_OP_MODE 0x01U
- #define SX1276_REG_BITRATE_MSB 0x02U
- #define SX1276_REG_BITRATE_LSB 0x03U
- #define SX1276_REG_FDEV_MSB 0x04U
- #define SX1276_REG_FDEV_LSB 0x05U
- #define SX1276_REG_FRF_MSB 0x06U
- #define SX1276_REG_FRF_MID 0x07U
- #define SX1276_REG_FRF_LSB 0x08U
- #define SX1276_REG_PA_CONFIG 0x09U
- #define SX1276_REG_LNA 0x0CU
- #define SX1276_REG_RX_CONFIG 0x0DU
- #define SX1276_REG_RSSI_CONFIG 0x0EU
- #define SX1276_REG_RSSI_VALUE_FSK 0x11U
- #define SX1276_REG_RX_BW 0x12U
- #define SX1276_REG_AFC_BW 0x13U
- #define SX1276_REG_OOK_PEAK 0x14U
- #define SX1276_REG_OOK_FIX 0x15U
- #define SX1276_REG_OOK_AVG 0x16U
- #define SX1276_REG_FIFO_ADDR_PTR 0x0DU
- #define SX1276_REG_FIFO_TX_BASE_ADDR 0x0EU
- #define SX1276_REG_FIFO_RX_BASE_ADDR 0x0FU
- #define SX1276_REG_FIFO_RX_CURRENT_ADDR 0x10U
- #define SX1276_REG_IRQ_FLAGS_MASK 0x11U
- #define SX1276_REG_IRQ_FLAGS 0x12U
- #define SX1276_REG_RX_NB_BYTES 0x13U

- #define SX1276_REG_PKT_SNR_VALUE 0x19U
- #define SX1276_REG_PKT_RSSI_VALUE 0x1AU
- #define SX1276_REG_PREAMBLE_DETECT 0x1FU
- #define SX1276_REG_MODEM_CONFIG_1 0x1DU
- #define SX1276_REG_MODEM_CONFIG_2 0x1EU
- #define SX1276_REG_SYMB_TIMEOUT_LSB 0x1FU
- #define SX1276_REG_PREAMBLE_MSB 0x20U
- #define SX1276_REG_PREAMBLE_LSB 0x21U
- #define SX1276_REG_PAYLOAD_LENGTH 0x22U
- #define SX1276_REG_PREAMBLE_MSB_FSK 0x25U
- #define SX1276_REG_PREAMBLE_LSB_FSK 0x26U
- #define SX1276_REG_SYNC_CONFIG 0x27U
- #define SX1276_REG_SYNC_VALUE_1 0x28U
- #define SX1276_REG_MODEM_CONFIG_3 0x26U
- #define SX1276_REG_PACKET_CONFIG_1 0x30U
- #define SX1276_REG_PACKET_CONFIG_2 0x31U
- #define SX1276_REG_PAYLOAD_LENGTH_FSK 0x32U
- #define SX1276_REG_NODE_ADDRESS 0x33U
- #define SX1276_REG_BROADCAST_ADDRESS 0x34U
- #define SX1276_REG_FIFO_THRESH 0x35U
- #define SX1276_REG_DETECTION_OPTIMIZE 0x31U
- #define SX1276_REG_INVERT_IQ 0x33U
- #define SX1276_REG_DETECTION_THRESHOLD 0x37U
- #define SX1276_REG_SYNC_WORD 0x39U
- #define SX1276_REG_IRQ_FLAGS_1 0x3EU
- #define SX1276_REG_IRQ_FLAGS_2 0x3FU
- #define SX1276_REG_INVERT_IQ2 0x3BU
- #define SX1276_REG_DIO_MAPPING_1 0x40U
- #define SX1276_REG_DIO_MAPPING_2 0x41U
- #define SX1276_REG_VERSION 0x42U
- #define SX1276_REG_PA_DAC 0x4DU
- #define SX1276_SPI_WRITE_MASK 0x80U
- #define SX1276_SPI_READ_MASK 0x7FU
- #define SX1276_VERSION_ID 0x12U
- #define SX1276_OPMODE_LONG_RANGE_MODE 0x80U
- #define SX1276_OPMODE_ACCESS_SHARED_REG 0x40U
- #define SX1276_OPMODE_MODULATION_MASK 0x60U
- #define SX1276_OPMODE_MODULATION_FSK 0x00U
- #define SX1276_OPMODE_MODULATION_OOK 0x20U
- #define SX1276_OPMODE_SHAPING_MASK 0x18U
- #define SX1276_OPMODE_SHAPING_NONE 0x00U
- #define SX1276_OPMODE_SHAPING_FSK_BT10 0x08U
- #define SX1276_OPMODE_SHAPING_FSK_BT05 0x10U
- #define SX1276_OPMODE_SHAPING_FSK_BT03 0x18U
- #define SX1276_OPMODE_SHAPING_OOK_FILTER 0x08U
- #define SX1276_OPMODE_LOW_FREQ_MODE_ON 0x08U
- #define SX1276_OPMODE_MODE_MASK 0x07U
- #define SX1276_MODE_SLEEP 0x00U
- #define SX1276_MODE_STDBY 0x01U
- #define SX1276_MODE_FSTX 0x02U
- #define SX1276_MODE_TX 0x03U
- #define SX1276_MODE_FSRX 0x04U
- #define SX1276_MODE_RXCONTINUOUS 0x05U
- #define SX1276_MODE_RXSINGLE 0x06U
- #define SX1276_MODE_CAD 0x07U

- #define SX1276_IRQ_RX_TIMEOUT 0x80U
- #define SX1276_IRQ_RX_DONE 0x40U
- #define SX1276_IRQ_PAYLOAD_CRC_ERROR 0x20U
- #define SX1276_IRQ_VALID_HEADER 0x10U
- #define SX1276_IRQ_TX_DONE 0x08U
- #define SX1276_IRQ_CAD_DONE 0x04U
- #define SX1276_IRQ_FHSS_CHANGE_CHANNEL 0x02U
- #define SX1276_IRQ_CAD_DETECTED 0x01U
- #define SX1276_DIO0_MAP_RX_DONE 0x00U
- #define SX1276_DIO0_MAP_TX_DONE 0x40U
- #define SX1276_DIO0_MAP_CAD_DONE 0x80U
- #define SX1276_DIO1_MAP_RX_TIMEOUT 0x00U
- #define SX1276_DIO1_MAP_FHSS_CHANGE 0x10U
- #define SX1276_DIO1_MAP_CAD_DETECTED 0x20U
- #define SX1276_DIO0_MAP_FSK_PAYLOAD_READY 0x00U
- #define SX1276_DIO0_MAP_FSK_PACKET_SENT 0x00U
- #define SX1276_PACKET_FORMAT_FIXED 0x00U
- #define SX1276_PACKET_FORMAT_VARIABLE 0x80U
- #define SX1276_PACKET_DC_FREE_NONE 0x00U
- #define SX1276_PACKET_DC_FREE_WHITENING 0x40U
- #define SX1276_PACKET_CRC_ON 0x10U
- #define SX1276_PACKET_CRC_OFF 0x00U
- #define SX1276_PACKET_CRC_WHITENING_IBM 0x01U
- #define SX1276_PACKET_CRC_WHITENING_CCITT 0x00U
- #define SX1276_PACKET_ADDR_FILTER_OFF 0x00U
- #define SX1276_PACKET_ADDR_FILTER_NODE 0x02U
- #define SX1276_PACKET_ADDR_FILTER_NODE_BC 0x04U
- #define SX1276_SYNC_AUTO_RESTART_OFF 0x00U
- #define SX1276_SYNC_AUTO_RESTART_NO_PLL 0x40U
- #define SX1276_SYNC_AUTO_RESTART_PLL 0x80U
- #define SX1276_SYNC_PREAMBLE_POLARITY_AA 0x00U
- #define SX1276_SYNC_PREAMBLE_POLARITY_55 0x20U
- #define SX1276_SYNC_ON 0x10U
- #define SX1276_SYNC_OFF 0x00U
- #define SX1276_SYNC_FIFOFILL_AUTO 0x00U
- #define SX1276_SYNC_FIFOFILL_MANUAL 0x08U
- #define SX1276_PACKET_DATA_MODE_CONTINUOUS 0x00U
- #define SX1276_PACKET_DATA_MODE_PACKET 0x40U
- #define SX1276_RXCONFIG_RESTART_RX_NO_PLL_LOCK 0x40U
- #define SX1276_RXCONFIG_AFC_AUTO_ON 0x10U
- #define SX1276_RXCONFIG_AGC_AUTO_ON 0x08U
- #define SX1276_RXCONFIG_TRIGGER_RSSI 0x00U
- #define SX1276_FIFO_THRESH_TX_START_NOT_EMPTY 0x80U
- #define SX1276_IRQ1_MODE_READY 0x80U
- #define SX1276_IRQ1_RX_READY 0x40U
- #define SX1276_IRQ1_TX_READY 0x20U
- #define SX1276_IRQ1_PLL_LOCK 0x10U
- #define SX1276_IRQ1_RSSI 0x08U
- #define SX1276_IRQ1_TIMEOUT 0x04U
- #define SX1276_IRQ1_PREAMBLE_DETECT 0x02U
- #define SX1276_IRQ1_SYNC_ADDRESS_MATCH 0x01U
- #define SX1276_IRQ2_FIFO_FULL 0x80U
- #define SX1276_IRQ2_FIFO_NOT_EMPTY 0x40U
- #define SX1276_IRQ2_FIFO_LEVEL 0x20U
- #define SX1276_IRQ2_FIFO_OVERRUN 0x10U

- `#define SX1276_IRQ2_PACKET_SENT 0x08U`
- `#define SX1276_IRQ2_PAYLOAD_READY 0x04U`
- `#define SX1276_IRQ2_CRC_OK 0x02U`
- `#define SX1276_IRQ2_LOW_BAT 0x01U`
- `#define SX1276_PA_DAC_ENABLE_20DBM 0x87U`
- `#define SX1276_PA_DAC_DISABLE_17DBM 0x84U`

12.52.1 Opis szczegółowy

Mapa rejestrów SX1276 i definicje bitów używane w bibliotece.
Definicja w pliku [radio_sx1276_regs.h](#).

12.52.2 Dokumentacja definicji

12.52.2.1 SX1276_DIO0_MAP_CAD_DONE

```
#define SX1276_DIO0_MAP_CAD_DONE 0x80U
```

Definicja w linii 111 pliku [radio_sx1276_regs.h](#).

12.52.2.2 SX1276_DIO0_MAP_FSK_PACKET_SENT

```
#define SX1276_DIO0_MAP_FSK_PACKET_SENT 0x00U
```

Definicja w linii 119 pliku [radio_sx1276_regs.h](#).

12.52.2.3 SX1276_DIO0_MAP_FSK_PAYLOAD_READY

```
#define SX1276_DIO0_MAP_FSK_PAYLOAD_READY 0x00U
```

Definicja w linii 118 pliku [radio_sx1276_regs.h](#).

12.52.2.4 SX1276_DIO0_MAP_RX_DONE

```
#define SX1276_DIO0_MAP_RX_DONE 0x00U
```

Definicja w linii 109 pliku [radio_sx1276_regs.h](#).

12.52.2.5 SX1276_DIO0_MAP_TX_DONE

```
#define SX1276_DIO0_MAP_TX_DONE 0x40U
```

Definicja w linii 110 pliku [radio_sx1276_regs.h](#).

12.52.2.6 SX1276_DIO1_MAP_CAD_DETECTED

```
#define SX1276_DIO1_MAP_CAD_DETECTED 0x20U
```

Definicja w linii 115 pliku [radio_sx1276_regs.h](#).

12.52.2.7 SX1276_DIO1_MAP_FHSS_CHANGE

```
#define SX1276_DIO1_MAP_FHSS_CHANGE 0x10U
```

Definicja w linii 114 pliku [radio_sx1276_regs.h](#).

12.52.2.8 SX1276_DIO1_MAP_RX_TIMEOUT

```
#define SX1276_DIO1_MAP_RX_TIMEOUT 0x00U
```

Definicja w linii 113 pliku [radio_sx1276_regs.h](#).

12.52.2.9 SX1276_FIFO_THRESH_TX_START_NOT_EMPTY

```
#define SX1276_FIFO_THRESH_TX_START_NOT_EMPTY 0x80U
```

Definicja w linii 156 pliku [radio_sx1276_regs.h](#).

12.52.2.10 SX1276_IRQ1_MODE_READY

```
#define SX1276_IRQ1_MODE_READY 0x80U
```

Definicja w linii 159 pliku [radio_sx1276_regs.h](#).

12.52.2.11 SX1276_IRQ1_PLL_LOCK

```
#define SX1276_IRQ1_PLL_LOCK 0x10U
```

Definicja w linii 162 pliku [radio_sx1276_regs.h](#).

12.52.2.12 SX1276_IRQ1_PREAMBLE_DETECT

```
#define SX1276_IRQ1_PREAMBLE_DETECT 0x02U
```

Definicja w linii 165 pliku [radio_sx1276_regs.h](#).

12.52.2.13 SX1276_IRQ1_RSSI

```
#define SX1276_IRQ1_RSSI 0x08U
```

Definicja w linii 163 pliku [radio_sx1276_regs.h](#).

12.52.2.14 SX1276_IRQ1_RX_READY

```
#define SX1276_IRQ1_RX_READY 0x40U
```

Definicja w linii 160 pliku [radio_sx1276_regs.h](#).

12.52.2.15 SX1276_IRQ1_SYNC_ADDRESS_MATCH

```
#define SX1276_IRQ1_SYNC_ADDRESS_MATCH 0x01U
```

Definicja w linii 166 pliku [radio_sx1276_regs.h](#).

12.52.2.16 SX1276_IRQ1_TIMEOUT

```
#define SX1276_IRQ1_TIMEOUT 0x04U
```

Definicja w linii 164 pliku [radio_sx1276_regs.h](#).

12.52.2.17 SX1276_IRQ1_TX_READY

```
#define SX1276_IRQ1_TX_READY 0x20U
```

Definicja w linii 161 pliku [radio_sx1276_regs.h](#).

12.52.2.18 SX1276_IRQ2_CRC_OK

```
#define SX1276_IRQ2_CRC_OK 0x02U
```

Definicja w linii 175 pliku [radio_sx1276_regs.h](#).

12.52.2.19 SX1276_IRQ2_FIFO_FULL

```
#define SX1276_IRQ2_FIFO_FULL 0x80U
```

Definicja w linii 169 pliku [radio_sx1276_regs.h](#).

12.52.2.20 SX1276_IRQ2_FIFO_LEVEL

```
#define SX1276_IRQ2_FIFO_LEVEL 0x20U
```

Definicja w linii 171 pliku [radio_sx1276_regs.h](#).

12.52.2.21 SX1276_IRQ2_FIFO_NOT_EMPTY

```
#define SX1276_IRQ2_FIFO_NOT_EMPTY 0x40U
```

Definicja w linii 170 pliku [radio_sx1276_regs.h](#).

12.52.2.22 SX1276_IRQ2_FIFO_OVERRUN

```
#define SX1276_IRQ2_FIFO_OVERRUN 0x10U
```

Definicja w linii 172 pliku [radio_sx1276_regs.h](#).

12.52.2.23 SX1276_IRQ2_LOW_BAT

```
#define SX1276_IRQ2_LOW_BAT 0x01U
```

Definicja w linii 176 pliku [radio_sx1276_regs.h](#).

12.52.2.24 SX1276_IRQ2_PACKET_SENT

```
#define SX1276_IRQ2_PACKET_SENT 0x08U
```

Definicja w linii 173 pliku [radio_sx1276_regs.h](#).

12.52.2.25 SX1276_IRQ2_PAYLOAD_READY

```
#define SX1276_IRQ2_PAYLOAD_READY 0x04U
```

Definicja w linii 174 pliku [radio_sx1276_regs.h](#).

12.52.2.26 SX1276_IRQ_CAD_DETECTED

```
#define SX1276_IRQ_CAD_DETECTED 0x01U
```

Definicja w linii 106 pliku [radio_sx1276_regs.h](#).

12.52.2.27 SX1276_IRQ_CAD_DONE

```
#define SX1276_IRQ_CAD_DONE 0x04U
```

Definicja w linii 104 pliku [radio_sx1276_regs.h](#).

12.52.2.28 SX1276_IRQ_FHSS_CHANGE_CHANNEL

```
#define SX1276_IRQ_FHSS_CHANGE_CHANNEL 0x02U
```

Definicja w linii 105 pliku [radio_sx1276_regs.h](#).

12.52.2.29 SX1276_IRQ_PAYLOAD_CRC_ERROR

```
#define SX1276_IRQ_PAYLOAD_CRC_ERROR 0x20U
```

Definicja w linii 101 pliku [radio_sx1276_regs.h](#).

12.52.2.30 SX1276_IRQ_RX_DONE

```
#define SX1276_IRQ_RX_DONE 0x40U
```

Definicja w linii 100 pliku [radio_sx1276_regs.h](#).

12.52.2.31 SX1276_IRQ_RX_TIMEOUT

```
#define SX1276_IRQ_RX_TIMEOUT 0x80U
```

Definicja w linii 99 pliku [radio_sx1276_regs.h](#).

12.52.2.32 SX1276_IRQ_TX_DONE

```
#define SX1276_IRQ_TX_DONE 0x08U
```

Definicja w linii 103 pliku [radio_sx1276_regs.h](#).

12.52.2.33 SX1276_IRQ_VALID_HEADER

```
#define SX1276_IRQ_VALID_HEADER 0x10U
```

Definicja w linii 102 pliku [radio_sx1276_regs.h](#).

12.52.2.34 SX1276_MODE_CAD

```
#define SX1276_MODE_CAD 0x07U
```

Definicja w linii 96 pliku [radio_sx1276_regs.h](#).

12.52.2.35 SX1276_MODE_FSRX

```
#define SX1276_MODE_FSRX 0x04U
```

Definicja w linii 93 pliku [radio_sx1276_regs.h](#).

12.52.2.36 SX1276_MODE_FSTX

```
#define SX1276_MODE_FSTX 0x02U
```

Definicja w linii 91 pliku [radio_sx1276_regs.h](#).

12.52.2.37 SX1276_MODE_RXCONTINUOUS

```
#define SX1276_MODE_RXCONTINUOUS 0x05U
```

Definicja w linii 94 pliku [radio_sx1276_regs.h](#).

12.52.2.38 SX1276_MODE_RXSINGLE

```
#define SX1276_MODE_RXSINGLE 0x06U
```

Definicja w linii 95 pliku [radio_sx1276_regs.h](#).

12.52.2.39 SX1276_MODE_SLEEP

```
#define SX1276_MODE_SLEEP 0x00U
```

Definicja w linii 89 pliku [radio_sx1276_regs.h](#).

12.52.2.40 SX1276_MODE_STDBY

```
#define SX1276_MODE_STDBY 0x01U
```

Definicja w linii 90 pliku [radio_sx1276_regs.h](#).

12.52.2.41 SX1276_MODE_TX

```
#define SX1276_MODE_TX 0x03U
```

Definicja w linii 92 pliku [radio_sx1276_regs.h](#).

12.52.2.42 SX1276_OPMODE_ACCESS_SHARED_REG

```
#define SX1276_OPMODE_ACCESS_SHARED_REG 0x40U
```

Definicja w linii 77 pliku [radio_sx1276_regs.h](#).

12.52.2.43 SX1276_OPMODE_LONG_RANGE_MODE

```
#define SX1276_OPMODE_LONG_RANGE_MODE 0x80U
```

Definicja w linii 76 pliku [radio_sx1276_regs.h](#).

12.52.2.44 SX1276_OPMODE_LOW_FREQ_MODE_ON

```
#define SX1276_OPMODE_LOW_FREQ_MODE_ON 0x08U
```

Definicja w linii 87 pliku [radio_sx1276_regs.h](#).

12.52.2.45 SX1276_OPMODE_MODE_MASK

```
#define SX1276_OPMODE_MODE_MASK 0x07U
```

Definicja w linii 88 pliku [radio_sx1276_regs.h](#).

12.52.2.46 SX1276_OPMODE_MODULATION_FSK

```
#define SX1276_OPMODE_MODULATION_FSK 0x00U
```

Definicja w linii 79 pliku [radio_sx1276_regs.h](#).

12.52.2.47 SX1276_OPMODE_MODULATION_MASK

```
#define SX1276_OPMODE_MODULATION_MASK 0x60U
```

Definicja w linii 78 pliku [radio_sx1276_regs.h](#).

12.52.2.48 SX1276_OPMODE_MODULATION_OOK

```
#define SX1276_OPMODE_MODULATION_OOK 0x20U
```

Definicja w linii 80 pliku [radio_sx1276_regs.h](#).

12.52.2.49 SX1276_OPMODE_SHAPING_FSK_BT03

```
#define SX1276_OPMODE_SHAPING_FSK_BT03 0x18U
```

Definicja w linii 85 pliku [radio_sx1276_regs.h](#).

12.52.2.50 SX1276_OPMODE_SHAPING_FSK_BT05

```
#define SX1276_OPMODE_SHAPING_FSK_BT05 0x10U
```

Definicja w linii 84 pliku [radio_sx1276_regs.h](#).

12.52.2.51 SX1276_OPMODE_SHAPING_FSK_BT10

```
#define SX1276_OPMODE_SHAPING_FSK_BT10 0x08U
```

Definicja w linii 83 pliku [radio_sx1276_regs.h](#).

12.52.2.52 SX1276_OPMODE_SHAPING_MASK

```
#define SX1276_OPMODE_SHAPING_MASK 0x18U
```

Definicja w linii 81 pliku [radio_sx1276_regs.h](#).

12.52.2.53 SX1276_OPMODE_SHAPING_NONE

```
#define SX1276_OPMODE_SHAPING_NONE 0x00U
```

Definicja w linii 82 pliku [radio_sx1276_regs.h](#).

12.52.2.54 SX1276_OPMODE_SHAPING_OOK_FILTER

```
#define SX1276_OPMODE_SHAPING_OOK_FILTER 0x08U
```

Definicja w linii 86 pliku [radio_sx1276_regs.h](#).

12.52.2.55 SX1276_PA_DAC_DISABLE_17DBM

```
#define SX1276_PA_DAC_DISABLE_17DBM 0x84U
```

Definicja w linii 180 pliku [radio_sx1276_regs.h](#).

12.52.2.56 SX1276_PA_DAC_ENABLE_20DBM

```
#define SX1276_PA_DAC_ENABLE_20DBM 0x87U
```

Definicja w linii 179 pliku [radio_sx1276_regs.h](#).

12.52.2.57 SX1276_PACKET_ADDR_FILTER_NODE

```
#define SX1276_PACKET_ADDR_FILTER_NODE 0x02U
```

Definicja w linii 131 pliku [radio_sx1276_regs.h](#).

12.52.2.58 SX1276_PACKET_ADDR_FILTER_NODE_BC

```
#define SX1276_PACKET_ADDR_FILTER_NODE_BC 0x04U
```

Definicja w linii 132 pliku [radio_sx1276_regs.h](#).

12.52.2.59 SX1276_PACKET_ADDR_FILTER_OFF

```
#define SX1276_PACKET_ADDR_FILTER_OFF 0x00U
```

Definicja w linii 130 pliku [radio_sx1276_regs.h](#).

12.52.2.60 SX1276_PACKET_CRC_OFF

```
#define SX1276_PACKET_CRC_OFF 0x00U
```

Definicja w linii 127 pliku [radio_sx1276_regs.h](#).

12.52.2.61 SX1276_PACKET_CRC_ON

```
#define SX1276_PACKET_CRC_ON 0x10U
```

Definicja w linii 126 pliku [radio_sx1276_regs.h](#).

12.52.2.62 SX1276_PACKET_CRC_WHITENING_CCITT

```
#define SX1276_PACKET_CRC_WHITENING_CCITT 0x00U
```

Definicja w linii 129 pliku [radio_sx1276_regs.h](#).

12.52.2.63 SX1276_PACKET_CRC_WHITENING_IBM

```
#define SX1276_PACKET_CRC_WHITENING_IBM 0x01U
```

Definicja w linii 128 pliku [radio_sx1276_regs.h](#).

12.52.2.64 SX1276_PACKET_DATA_MODE_CONTINUOUS

```
#define SX1276_PACKET_DATA_MODE_CONTINUOUS 0x00U
```

Definicja w linii 146 pliku [radio_sx1276_regs.h](#).

12.52.2.65 SX1276_PACKET_DATA_MODE_PACKET

```
#define SX1276_PACKET_DATA_MODE_PACKET 0x40U
```

Definicja w linii 147 pliku [radio_sx1276_regs.h](#).

12.52.2.66 SX1276_PACKET_DC_FREE_NONE

```
#define SX1276_PACKET_DC_FREE_NONE 0x00U
```

Definicja w linii 124 pliku [radio_sx1276_regs.h](#).

12.52.2.67 SX1276_PACKET_DC_FREE_WHITENING

```
#define SX1276_PACKET_DC_FREE_WHITENING 0x40U
```

Definicja w linii 125 pliku [radio_sx1276_regs.h](#).

12.52.2.68 SX1276_PACKET_FORMAT_FIXED

```
#define SX1276_PACKET_FORMAT_FIXED 0x00U
```

Definicja w linii 122 pliku [radio_sx1276_regs.h](#).

12.52.2.69 SX1276_PACKET_FORMAT_VARIABLE

```
#define SX1276_PACKET_FORMAT_VARIABLE 0x80U
```

Definicja w linii 123 pliku [radio_sx1276_regs.h](#).

12.52.2.70 SX1276_REG_AFC_BW

```
#define SX1276_REG_AFC_BW 0x13U
```

Definicja w linii 25 pliku [radio_sx1276_regs.h](#).

12.52.2.71 SX1276_REG_BITRATE_LSB

```
#define SX1276_REG_BITRATE_LSB 0x03U
```

Definicja w linii 13 pliku [radio_sx1276_regs.h](#).

12.52.2.72 SX1276_REG_BITRATE_MSB

```
#define SX1276_REG_BITRATE_MSB 0x02U
```

Definicja w linii 12 pliku [radio_sx1276_regs.h](#).

12.52.2.73 SX1276_REG_BROADCAST_ADDRESS

```
#define SX1276_REG_BROADCAST_ADDRESS 0x34U
```

Definicja w linii 54 pliku [radio_sx1276_regs.h](#).

12.52.2.74 SX1276_REG_DETECTION_OPTIMIZE

```
#define SX1276_REG_DETECTION_OPTIMIZE 0x31U
```

Definicja w linii 56 pliku [radio_sx1276_regs.h](#).

12.52.2.75 SX1276_REG_DETECTION_THRESHOLD

```
#define SX1276_REG_DETECTION_THRESHOLD 0x37U
```

Definicja w linii 58 pliku [radio_sx1276_regs.h](#).

12.52.2.76 SX1276_REG_DIO_MAPPING_1

```
#define SX1276_REG_DIO_MAPPING_1 0x40U
```

Definicja w linii 63 pliku [radio_sx1276_regs.h](#).

12.52.2.77 SX1276_REG_DIO_MAPPING_2

```
#define SX1276_REG_DIO_MAPPING_2 0x41U
```

Definicja w linii 64 pliku [radio_sx1276_regs.h](#).

12.52.2.78 SX1276_REG_FDEV_LSB

```
#define SX1276_REG_FDEV_LSB 0x05U
```

Definicja w linii 15 pliku [radio_sx1276_regs.h](#).

12.52.2.79 SX1276_REG_FDEV_MSB

```
#define SX1276_REG_FDEV_MSB 0x04U
```

Definicja w linii 14 pliku [radio_sx1276_regs.h](#).

12.52.2.80 SX1276_REG_FIFO

```
#define SX1276_REG_FIFO 0x00U
```

Definicja w linii 10 pliku [radio_sx1276_regs.h](#).

12.52.2.81 SX1276_REG_FIFO_ADDR_PTR

```
#define SX1276_REG_FIFO_ADDR_PTR 0x0DU
```

Definicja w linii 29 pliku [radio_sx1276_regs.h](#).

12.52.2.82 SX1276_REG_FIFO_RX_BASE_ADDR

```
#define SX1276_REG_FIFO_RX_BASE_ADDR 0x0FU
```

Definicja w linii 31 pliku [radio_sx1276_regs.h](#).

12.52.2.83 SX1276_REG_FIFO_RX_CURRENT_ADDR

```
#define SX1276_REG_FIFO_RX_CURRENT_ADDR 0x10U
```

Definicja w linii 32 pliku [radio_sx1276_regs.h](#).

12.52.2.84 SX1276_REG_FIFO_THRESH

```
#define SX1276_REG_FIFO_THRESH 0x35U
```

Definicja w linii 55 pliku [radio_sx1276_regs.h](#).

12.52.2.85 SX1276_REG_FIFO_TX_BASE_ADDR

```
#define SX1276_REG_FIFO_TX_BASE_ADDR 0x0EU
```

Definicja w linii 30 pliku [radio_sx1276_regs.h](#).

12.52.2.86 SX1276_REG_FRF_LSB

```
#define SX1276_REG_FRF_LSB 0x08U
```

Definicja w linii 18 pliku [radio_sx1276_regs.h](#).

12.52.2.87 SX1276_REG_FRF_MID

```
#define SX1276_REG_FRF_MID 0x07U
```

Definicja w linii 17 pliku [radio_sx1276_regs.h](#).

12.52.2.88 SX1276_REG_FRF_MSB

```
#define SX1276_REG_FRF_MSB 0x06U
```

Definicja w linii 16 pliku [radio_sx1276_regs.h](#).

12.52.2.89 SX1276_REG_INVERT_IQ

```
#define SX1276_REG_INVERT_IQ 0x33U
```

Definicja w linii 57 pliku [radio_sx1276_regs.h](#).

12.52.2.90 SX1276_REG_INVERT_IQ2

```
#define SX1276_REG_INVERT_IQ2 0x3BU
```

Definicja w linii 62 pliku [radio_sx1276_regs.h](#).

12.52.2.91 SX1276_REG_IRQ_FLAGS

```
#define SX1276_REG_IRQ_FLAGS 0x12U
```

Definicja w linii 34 pliku [radio_sx1276_regs.h](#).

12.52.2.92 SX1276_REG_IRQ_FLAGS_1

```
#define SX1276_REG_IRQ_FLAGS_1 0x3EU
```

Definicja w linii 60 pliku [radio_sx1276_regs.h](#).

12.52.2.93 SX1276_REG_IRQ_FLAGS_2

```
#define SX1276_REG_IRQ_FLAGS_2 0x3FU
```

Definicja w linii 61 pliku [radio_sx1276_regs.h](#).

12.52.2.94 SX1276_REG_IRQ_FLAGS_MASK

```
#define SX1276_REG_IRQ_FLAGS_MASK 0x11U
```

Definicja w linii 33 pliku [radio_sx1276_regs.h](#).

12.52.2.95 SX1276_REG_LNA

```
#define SX1276_REG_LNA 0x0CU
```

Definicja w linii 20 pliku [radio_sx1276_regs.h](#).

12.52.2.96 SX1276_REG_MODEM_CONFIG_1

```
#define SX1276_REG_MODEM_CONFIG_1 0x1DU
```

Definicja w linii 39 pliku [radio_sx1276_regs.h](#).

12.52.2.97 SX1276_REG_MODEM_CONFIG_2

```
#define SX1276_REG_MODEM_CONFIG_2 0x1EU
```

Definicja w linii 40 pliku [radio_sx1276_regs.h](#).

12.52.2.98 SX1276_REG_MODEM_CONFIG_3

```
#define SX1276_REG_MODEM_CONFIG_3 0x26U
```

Definicja w linii 49 pliku [radio_sx1276_regs.h](#).

12.52.2.99 SX1276_REG_NODE_ADDRESS

```
#define SX1276_REG_NODE_ADDRESS 0x33U
```

Definicja w linii 53 pliku [radio_sx1276_regs.h](#).

12.52.2.100 SX1276_REG_OOK_AVG

```
#define SX1276_REG_OOK_AVG 0x16U
```

Definicja w linii 28 pliku [radio_sx1276_regs.h](#).

12.52.2.101 SX1276_REG_OOK_FIX

```
#define SX1276_REG_OOK_FIX 0x15U
```

Definicja w linii 27 pliku [radio_sx1276_regs.h](#).

12.52.2.102 SX1276_REG_OOK_PEAK

```
#define SX1276_REG_OOK_PEAK 0x14U
```

Definicja w linii 26 pliku [radio_sx1276_regs.h](#).

12.52.2.103 SX1276_REG_OP_MODE

```
#define SX1276_REG_OP_MODE 0x01U
```

Definicja w linii 11 pliku [radio_sx1276_regs.h](#).

12.52.2.104 SX1276_REG_PA_CONFIG

```
#define SX1276_REG_PA_CONFIG 0x09U
```

Definicja w linii 19 pliku [radio_sx1276_regs.h](#).

12.52.2.105 SX1276_REG_PA_DAC

```
#define SX1276_REG_PA_DAC 0x4DU
```

Definicja w linii 66 pliku [radio_sx1276_regs.h](#).

12.52.2.106 SX1276_REG_PACKET_CONFIG_1

```
#define SX1276_REG_PACKET_CONFIG_1 0x30U
```

Definicja w linii 50 pliku [radio_sx1276_regs.h](#).

12.52.2.107 SX1276_REG_PACKET_CONFIG_2

```
#define SX1276_REG_PACKET_CONFIG_2 0x31U
```

Definicja w linii 51 pliku [radio_sx1276_regs.h](#).

12.52.2.108 SX1276_REG_PAYLOAD_LENGTH

```
#define SX1276_REG_PAYLOAD_LENGTH 0x22U
```

Definicja w linii 44 pliku [radio_sx1276_regs.h](#).

12.52.2.109 SX1276_REG_PAYLOAD_LENGTH_FSK

```
#define SX1276_REG_PAYLOAD_LENGTH_FSK 0x32U
```

Definicja w linii 52 pliku [radio_sx1276_regs.h](#).

12.52.2.110 SX1276_REG_PKT_RSSI_VALUE

```
#define SX1276_REG_PKT_RSSI_VALUE 0x1AU
```

Definicja w linii 37 pliku [radio_sx1276_regs.h](#).

12.52.2.111 SX1276_REG_PKT_SNR_VALUE

```
#define SX1276_REG_PKT_SNR_VALUE 0x19U
```

Definicja w linii 36 pliku [radio_sx1276_regs.h](#).

12.52.2.112 SX1276_REG_PREAMBLE_DETECT

```
#define SX1276_REG_PREAMBLE_DETECT 0x1FU
```

Definicja w linii 38 pliku [radio_sx1276_regs.h](#).

12.52.2.113 SX1276_REG_PREAMBLE_LSB

```
#define SX1276_REG_PREAMBLE_LSB 0x21U
```

Definicja w linii 43 pliku [radio_sx1276_regs.h](#).

12.52.2.114 SX1276_REG_PREAMBLE_LSB_FSK

```
#define SX1276_REG_PREAMBLE_LSB_FSK 0x26U
```

Definicja w linii 46 pliku [radio_sx1276_regs.h](#).

12.52.2.115 SX1276_REG_PREAMBLE_MSB

```
#define SX1276_REG_PREAMBLE_MSB 0x20U
```

Definicja w linii 42 pliku [radio_sx1276_regs.h](#).

12.52.2.116 SX1276_REG_PREAMBLE_MSB_FSK

```
#define SX1276_REG_PREAMBLE_MSB_FSK 0x25U
```

Definicja w linii 45 pliku [radio_sx1276_regs.h](#).

12.52.2.117 SX1276_REG_RSSI_CONFIG

```
#define SX1276_REG_RSSI_CONFIG 0x0EU
```

Definicja w linii 22 pliku [radio_sx1276_regs.h](#).

12.52.2.118 SX1276_REG_RSSI_VALUE_FSK

```
#define SX1276_REG_RSSI_VALUE_FSK 0x11U
```

Definicja w linii 23 pliku [radio_sx1276_regs.h](#).

12.52.2.119 SX1276_REG_RX_BW

```
#define SX1276_REG_RX_BW 0x12U
```

Definicja w linii 24 pliku [radio_sx1276_regs.h](#).

12.52.2.120 SX1276_REG_RX_CONFIG

```
#define SX1276_REG_RX_CONFIG 0x0DU
```

Definicja w linii 21 pliku [radio_sx1276_regs.h](#).

12.52.2.121 SX1276_REG_RX_NB_BYTES

```
#define SX1276_REG_RX_NB_BYTES 0x13U
```

Definicja w linii 35 pliku [radio_sx1276_regs.h](#).

12.52.2.122 SX1276_REG_SYMB_TIMEOUT_LSB

```
#define SX1276_REG_SYMB_TIMEOUT_LSB 0x1FU
```

Definicja w linii 41 pliku [radio_sx1276_regs.h](#).

12.52.2.123 SX1276_REG_SYNC_CONFIG

```
#define SX1276_REG_SYNC_CONFIG 0x27U
```

Definicja w linii 47 pliku [radio_sx1276_regs.h](#).

12.52.2.124 SX1276_REG_SYNC_VALUE_1

```
#define SX1276_REG_SYNC_VALUE_1 0x28U
```

Definicja w linii 48 pliku [radio_sx1276_regs.h](#).

12.52.2.125 SX1276_REG_SYNC_WORD

```
#define SX1276_REG_SYNC_WORD 0x39U
```

Definicja w linii 59 pliku [radio_sx1276_regs.h](#).

12.52.2.126 SX1276_REG_VERSION

```
#define SX1276_REG_VERSION 0x42U
```

Definicja w linii 65 pliku [radio_sx1276_regs.h](#).

12.52.2.127 SX1276_RXCONFIG_AFC_AUTO_ON

```
#define SX1276_RXCONFIG_AFC_AUTO_ON 0x10U
```

Definicja w linii 151 pliku [radio_sx1276_regs.h](#).

12.52.2.128 SX1276_RXCONFIG_AGC_AUTO_ON

```
#define SX1276_RXCONFIG_AGC_AUTO_ON 0x08U
```

Definicja w linii 152 pliku [radio_sx1276_regs.h](#).

12.52.2.129 SX1276_RXCONFIG_RESTART_RX_NO_PLL_LOCK

```
#define SX1276_RXCONFIG_RESTART_RX_NO_PLL_LOCK 0x40U
```

Definicja w linii 150 pliku [radio_sx1276_regs.h](#).

12.52.2.130 SX1276_RXCONFIG_TRIGGER_RSSI

```
#define SX1276_RXCONFIG_TRIGGER_RSSI 0x00U
```

Definicja w linii 153 pliku [radio_sx1276_regs.h](#).

12.52.2.131 SX1276_SPI_READ_MASK

```
#define SX1276_SPI_READ_MASK 0x7FU
```

Definicja w linii 70 pliku [radio_sx1276_regs.h](#).

12.52.2.132 SX1276_SPI_WRITE_MASK

```
#define SX1276_SPI_WRITE_MASK 0x80U
```

Definicja w linii 69 pliku [radio_sx1276_regs.h](#).

12.52.2.133 SX1276_SYNC_AUTO_RESTART_NO_PLL

```
#define SX1276_SYNC_AUTO_RESTART_NO_PLL 0x40U
```

Definicja w linii 136 pliku [radio_sx1276_regs.h](#).

12.52.2.134 SX1276_SYNC_AUTO_RESTART_OFF

```
#define SX1276_SYNC_AUTO_RESTART_OFF 0x00U
```

Definicja w linii 135 pliku [radio_sx1276_regs.h](#).

12.52.2.135 SX1276_SYNC_AUTO_RESTART_PLL

```
#define SX1276_SYNC_AUTO_RESTART_PLL 0x80U
```

Definicja w linii 137 pliku [radio_sx1276_regs.h](#).

12.52.2.136 SX1276_SYNC_FIFOFILL_AUTO

```
#define SX1276_SYNC_FIFOFILL_AUTO 0x00U
```

Definicja w linii 142 pliku [radio_sx1276_regs.h](#).

12.52.2.137 SX1276_SYNC_FIFOFILL_MANUAL

```
#define SX1276_SYNC_FIFOFILL_MANUAL 0x08U
```

Definicja w linii 143 pliku [radio_sx1276_regs.h](#).

12.52.2.138 SX1276_SYNC_OFF

```
#define SX1276_SYNC_OFF 0x00U
```

Definicja w linii 141 pliku [radio_sx1276_regs.h](#).

12.52.2.139 SX1276_SYNC_ON

```
#define SX1276_SYNC_ON 0x10U
```

Definicja w linii 140 pliku [radio_sx1276_regs.h](#).

12.52.2.140 SX1276_SYNC_PREAMBLE_POLARITY_55

```
#define SX1276_SYNC_PREAMBLE_POLARITY_55 0x20U
```

Definicja w linii 139 pliku [radio_sx1276_regs.h](#).

12.52.2.141 SX1276_SYNC_PREAMBLE_POLARITY_AA

```
#define SX1276_SYNC_PREAMBLE_POLARITY_AA 0x00U
```

Definicja w linii 138 pliku [radio_sx1276_regs.h](#).

12.52.2.142 SX1276_VERSION_ID

```
#define SX1276_VERSION_ID 0x12U
```

Definicja w linii 73 pliku [radio_sx1276_regs.h](#).

12.53 radio_sx1276_regs.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_RADIO_LIB_COMMON_SX1276_RADIO_SX1276_REGS_H_
00007 #define APP_RADIO_LIB_COMMON_SX1276_RADIO_SX1276_REGS_H_
00008
00009 /* Rejestry wspólne */
00010 #define SX1276_REG_FIFO 0x00U
00011 #define SX1276_REG_OP_MODE 0x01U
00012 #define SX1276_REG_BITRATE_MSB 0x02U
00013 #define SX1276_REG_BITRATE_LSB 0x03U
00014 #define SX1276_REG_FDEV_MSB 0x04U
00015 #define SX1276_REG_FDEV_LSB 0x05U
00016 #define SX1276_REG_FRF_MSB 0x06U
00017 #define SX1276_REG_FRF_MID 0x07U
00018 #define SX1276_REG_FRF_LSB 0x08U
00019 #define SX1276_REG_PA_CONFIG 0x09U
00020 #define SX1276_REG_LNA 0x0CU
00021 #define SX1276_REG_RX_CONFIG 0x0DU
00022 #define SX1276_REG_RSSI_CONFIG 0x0EU
00023 #define SX1276_REG_RSSI_VALUE_FSK 0x11U
00024 #define SX1276_REG_RX_BW 0x12U
00025 #define SX1276_REG_AFC_BW 0x13U
00026 #define SX1276_REG_OOK_PEAK 0x14U
00027 #define SX1276_REG_OOK_FIX 0x15U
00028 #define SX1276_REG_OOK_AVG 0x16U
00029 #define SX1276_REG_FIFO_ADDR_PTR 0x0DU
00030 #define SX1276_REG_FIFO_TX_BASE_ADDR 0x0EU
00031 #define SX1276_REG_FIFO_RX_BASE_ADDR 0x0FU
00032 #define SX1276_REG_FIFO_RX_CURRENT_ADDR 0x10U
00033 #define SX1276_REG_IRQ_FLAGS_MASK 0x11U
00034 #define SX1276_REG_IRQ_FLAGS 0x12U
00035 #define SX1276_REG_RX_NB_BYTES 0x13U
00036 #define SX1276_REG_PKT_SNR_VALUE 0x19U
00037 #define SX1276_REG_PKT_RSSI_VALUE 0x1AU
00038 #define SX1276_REG_PREAMBLE_DETECT 0x1FU
00039 #define SX1276_REG_MODEM_CONFIG_1 0x1DU
00040 #define SX1276_REG_MODEM_CONFIG_2 0x1EU
00041 #define SX1276_REG_SYMB_TIMEOUT_LSB 0x1FU
00042 #define SX1276_REG_PREAMBLE_MSB 0x20U
00043 #define SX1276_REG_PREAMBLE_LSB 0x21U
00044 #define SX1276_REG_PAYLOAD_LENGTH 0x22U
00045 #define SX1276_REG_PREAMBLE_MSB_FSK 0x25U
00046 #define SX1276_REG_PREAMBLE_LSB_FSK 0x26U
00047 #define SX1276_REG_SYNC_CONFIG 0x27U
00048 #define SX1276_REG_SYNC_VALUE_1 0x28U
00049 #define SX1276_REG_MODEM_CONFIG_3 0x26U
00050 #define SX1276_REG_PACKET_CONFIG_1 0x30U
00051 #define SX1276_REG_PACKET_CONFIG_2 0x31U
00052 #define SX1276_REG_PAYLOAD_LENGTH_FSK 0x32U
00053 #define SX1276_REG_NODE_ADDRESS 0x33U
00054 #define SX1276_REG_BROADCAST_ADDRESS 0x34U
00055 #define SX1276_REG_FIFO_THRESH 0x35U
00056 #define SX1276_REG_DETECTION_OPTIMIZE 0x31U
00057 #define SX1276_REG_INVERT_IQ 0x33U
00058 #define SX1276_REG_DETECTION_THRESHOLD 0x37U
00059 #define SX1276_REG_SYNC_WORD 0x39U
00060 #define SX1276_REG_IRQ_FLAGS_1 0x3EU
00061 #define SX1276_REG_IRQ_FLAGS_2 0x3FU
00062 #define SX1276_REG_INVERT_IQ2 0x3BU
00063 #define SX1276_REG_DIO_MAPPING_1 0x40U
00064 #define SX1276_REG_DIO_MAPPING_2 0x41U
00065 #define SX1276_REG_VERSION 0x42U
00066 #define SX1276_REG_PA_DAC 0x4DU
00067
00068 /* Maski odczytu/zapisu SPI */
00069 #define SX1276_SPI_WRITE_MASK 0x80U
00070 #define SX1276_SPI_READ_MASK 0x7FU
00071
00072 /* Identyfikator wersji */
00073 #define SX1276_VERSION_ID 0x12U
00074
00075 /* RegOpMode */
00076 #define SX1276_OPMODE_LONG_RANGE_MODE 0x80U
00077 #define SX1276_OPMODE_ACCESS_SHARED_REG 0x40U
00078 #define SX1276_OPMODE_MODULATION_MASK 0x60U
```

```

00079 #define SX1276_OPMODE_MODULATION_FSK      0x00U
00080 #define SX1276_OPMODE_MODULATION_OOK      0x20U
00081 #define SX1276_OPMODE_SHAPING_MASK        0x18U
00082 #define SX1276_OPMODE_SHAPING_NONE        0x00U
00083 #define SX1276_OPMODE_SHAPING_FSK_BT10    0x08U
00084 #define SX1276_OPMODE_SHAPING_FSK_BT05    0x10U
00085 #define SX1276_OPMODE_SHAPING_FSK_BT03    0x18U
00086 #define SX1276_OPMODE_SHAPING_OOK_FILTER  0x08U
00087 #define SX1276_OPMODE_LOW_FREQ_MODE_ON    0x08U
00088 #define SX1276_OPMODE_MODE_MASK           0x07U
00089 #define SX1276_MODE_SLEEP                  0x00U
00090 #define SX1276_MODE_STDBY                  0x01U
00091 #define SX1276_MODE_FSTX                   0x02U
00092 #define SX1276_MODE_TX                     0x03U
00093 #define SX1276_MODE_FSRX                   0x04U
00094 #define SX1276_MODE_RXCONTINUOUS           0x05U
00095 #define SX1276_MODE_RXSINGLE                0x06U
00096 #define SX1276_MODE_CAD                    0x07U
00097
00098 /* RegIrqFlags */
00099 #define SX1276_IRQ_RX_TIMEOUT              0x80U
00100 #define SX1276_IRQ_RX_DONE                0x40U
00101 #define SX1276_IRQ_PAYLOAD_CRC_ERROR      0x20U
00102 #define SX1276_IRQ_VALID_HEADER           0x10U
00103 #define SX1276_IRQ_TX_DONE                0x08U
00104 #define SX1276_IRQ_CAD_DONE               0x04U
00105 #define SX1276_IRQ_FHSS_CHANGE_CHANNEL    0x02U
00106 #define SX1276_IRQ_CAD_DETECTED           0x01U
00107
00108 /* RegDioMapping1 (tryb LoRa) */
00109 #define SX1276_DIO0_MAP_RX_DONE           0x00U
00110 #define SX1276_DIO0_MAP_TX_DONE           0x40U
00111 #define SX1276_DIO0_MAP_CAD_DONE          0x80U
00112
00113 #define SX1276_DIO1_MAP_RX_TIMEOUT        0x00U
00114 #define SX1276_DIO1_MAP_FHSS_CHANGE      0x10U
00115 #define SX1276_DIO1_MAP_CAD_DETECTED      0x20U
00116
00117 /* RegDioMapping1 (tryb FSK/OOK) */
00118 #define SX1276_DIO0_MAP_FSK_PAYLOAD_READY 0x00U
00119 #define SX1276_DIO0_MAP_FSK_PACKET_SENT   0x00U
00120
00121 /* RegPacketConfig1 */
00122 #define SX1276_PACKET_FORMAT_FIXED        0x00U
00123 #define SX1276_PACKET_FORMAT_VARIABLE     0x80U
00124 #define SX1276_PACKET_DC_FREE_NONE        0x00U
00125 #define SX1276_PACKET_DC_FREE_WHITENING   0x40U
00126 #define SX1276_PACKET_CRC_ON              0x10U
00127 #define SX1276_PACKET_CRC_OFF             0x00U
00128 #define SX1276_PACKET_CRC_WHITENING_IBM   0x01U
00129 #define SX1276_PACKET_CRC_WHITENING_CCITT 0x00U
00130 #define SX1276_PACKET_ADDR_FILTER_OFF     0x00U
00131 #define SX1276_PACKET_ADDR_FILTER_NODE    0x02U
00132 #define SX1276_PACKET_ADDR_FILTER_NODE_BC 0x04U
00133
00134 /* RegSyncConfig */
00135 #define SX1276_SYNC_AUTO_RESTART_OFF      0x00U
00136 #define SX1276_SYNC_AUTO_RESTART_NO_PLL   0x40U
00137 #define SX1276_SYNC_AUTO_RESTART_PLL     0x80U
00138 #define SX1276_SYNC_PREAMBLE_POLARITY_AA  0x00U
00139 #define SX1276_SYNC_PREAMBLE_POLARITY_55  0x20U
00140 #define SX1276_SYNC_ON                    0x10U
00141 #define SX1276_SYNC_OFF                   0x00U
00142 #define SX1276_SYNC_FIFO_FILL_AUTO        0x00U
00143 #define SX1276_SYNC_FIFO_FILL_MANUAL      0x08U
00144
00145 /* RegPacketConfig2 */
00146 #define SX1276_PACKET_DATA_MODE_CONTINUOUS 0x00U
00147 #define SX1276_PACKET_DATA_MODE_PACKET    0x40U
00148
00149 /* RegRxConfig */
00150 #define SX1276_RXCONFIG_RESTART_RX_NO_PLL_LOCK 0x40U
00151 #define SX1276_RXCONFIG_AFC_AUTO_ON         0x10U
00152 #define SX1276_RXCONFIG_AGC_AUTO_ON         0x08U
00153 #define SX1276_RXCONFIG_TRIGGER_RSSI        0x00U
00154
00155 /* RegFifoThresh */
00156 #define SX1276_FIFO_THRESH_TX_START_NOT_EMPTY 0x80U
00157
00158 /* RegIrqFlags1 */
00159 #define SX1276_IRQ1_MODE_READY             0x80U
00160 #define SX1276_IRQ1_RX_READY              0x40U
00161 #define SX1276_IRQ1_TX_READY              0x20U
00162 #define SX1276_IRQ1_PLL_LOCK              0x10U
00163 #define SX1276_IRQ1_RSSI                  0x08U
00164 #define SX1276_IRQ1_TIMEOUT               0x04U
00165 #define SX1276_IRQ1_PREAMBLE_DETECT       0x02U

```

```

00166 #define SX1276_IRQ1_SYNC_ADDRESS_MATCH      0x01U
00167
00168 /* RegIrqFlags2 */
00169 #define SX1276_IRQ2_FIFO_FULL                  0x80U
00170 #define SX1276_IRQ2_FIFO_NOT_EMPTY            0x40U
00171 #define SX1276_IRQ2_FIFO_LEVEL                0x20U
00172 #define SX1276_IRQ2_FIFO_OVERRUN              0x10U
00173 #define SX1276_IRQ2_PACKET_SENT               0x08U
00174 #define SX1276_IRQ2_PAYLOAD_READY             0x04U
00175 #define SX1276_IRQ2_CRC_OK                    0x02U
00176 #define SX1276_IRQ2_LOW_BAT                   0x01U
00177
00178 /* RegPaDac */
00179 #define SX1276_PA_DAC_ENABLE_20DBM            0x87U
00180 #define SX1276_PA_DAC_DISABLE_17DBM          0x84U
00181
00182 #endif /* APP_RADIO_LIB_COMMON_SX1276_RADIO_SX1276_REGS_H_ */

```

12.54 Dokumentacja pliku

Core/App/radio_lib/modulations/fsk/radio_fsk.c

Backend modulacji FSK/GFSK/MSK/GMSK dla SX1276.

```

#include "radio_fsk.h"
#include "app_delay.h"
#include "main.h"
#include "../common/sx1276/radio_sx1276.h"
#include "../common/sx1276/radio_sx1276_regs.h"
#include "../lora/radio_lora.h"
#include <string.h>

```

Struktury danych

- struct [radio_fsk_context_t](#)

Definicje

- #define [RADIO_FSK_FIFO_THRESH_LEVEL](#) 31U
- #define [RADIO_FSK_RESET_LOW_DELAY_MS](#) 2U
- #define [RADIO_FSK_RESET_HIGH_DELAY_MS](#) 10U
- #define [RADIO_FSK_NODE_ADDRESS_DEFAULT](#) 0x01U
- #define [RADIO_FSK_BROADCAST_ADDRESS](#) 0xFFU
- #define [RADIO_FSK_TX_GUARD_MS](#) 40UL
- #define [RADIO_FSK_FIFO_MAX_BYTES](#) 64U

Funkcje

- static uint32_t [radio_irq_save](#) (void)
Internal helper: [radio_irq_save](#).
- static void [radio_irq_restore](#) (uint32_t primask)
Internal helper: [radio_irq_restore](#).
- static void [radio_set_event_flags](#) (uint32_t events)
Internal helper: [radio_set_event_flags](#).
- static bool [radio_cfg_valid](#) (const [radio_fsk_cfg_t](#) *cfg)
Internal helper: [radio_cfg_valid](#).
- static void [radio_hw_reset](#) (void)
Internal helper: [radio_hw_reset](#).
- static uint8_t [radio_get_rx_bw_reg_value](#) ([radio_lora_bw_t](#) bw)
Internal helper: [radio_get_rx_bw_reg_value](#).
- static uint8_t [radio_get_shaping_bits](#) (const [radio_fsk_cfg_t](#) *cfg)

- Internal helper: *radio_get_shaping_bits*.

 - static uint32_t **radio_get_frequency_deviation** (const **radio_fsk_cfg_t** *cfg)

Internal helper: *radio_get_frequency_deviation*.
- static bool **radio_set_op_mode_ready** (uint8_t op_mode)
 - static uint8_t **radio_get_packet_config_1** (const **radio_fsk_cfg_t** *cfg)

Internal helper: *radio_get_packet_config_1*.
- static bool **radio_write_sync_word** (const **radio_fsk_cfg_t** *cfg)

Internal helper: *radio_write_sync_word*.
- static bool **radio_apply_fsk_config** (const **radio_fsk_cfg_t** *cfg)

Internal helper: *radio_apply_fsk_config*.
- static void **radio_set_state** (**radio_state_t** state)

Internal helper: *radio_set_state*.
- static void **radio_resume_after_tx** (void)

Internal helper: *radio_resume_after_tx*.
- static uint32_t **radio_tx_timeout_ms** (uint8_t payload_len)

Internal helper: *radio_tx_timeout_ms*.
- static uint32_t **radio_rx_single_timeout_ms** (uint32_t symbol_timeout)

Internal helper: *radio_rx_single_timeout_ms*.
- static void **radio_handle_exti_pin** (uint16_t pin)

Internal helper: *radio_handle_exti_pin*.
- static **radio_status_t** **radio_read_rx_packet** (void)

Internal helper: *radio_read_rx_packet*.
- void **radio_fsk_default_hw_cfg** (**radio_hw_cfg_t** *cfg, SPI_HandleTypeDef *hspi)

Domyślna konfiguracja sprzętowa backendu FSK.
- void **radio_fsk_default_lora_cfg** (**radio_lora_cfg_t** *cfg)

Domyślna konfiguracja parametrów backendu FSK.
- **radio_status_t** **radio_fsk_set_runtime_cfg** (const **radio_fsk_cfg_t** *cfg)

Ustawia runtime konfigurację właściwą dla backendu FSK.
- **radio_status_t** **radio_fsk_init** (const **radio_hw_cfg_t** *hw, const **radio_lora_cfg_t** *cfg, **radio_event_cb_t** cb, void *user_ctx)

Inicjalizacja backendu FSK.
- **radio_status_t** **radio_fsk_deinit** (void)

Deinicjalizacja backendu FSK.
- **radio_status_t** **radio_fsk_start_rx_continuous** (void)

Uruchomienie odbioru ciągłego FSK.
- **radio_status_t** **radio_fsk_start_rx_single** (uint16_t symbol_timeout)

Uruchomienie odbioru pojedynczego FSK.
- **radio_status_t** **radio_fsk_send_async** (const uint8_t *data, uint8_t len)

Uruchomienie nadawania asynchronicznego FSK.
- **radio_status_t** **radio_fsk_standby** (void)

Przejdźcie do standby backendu FSK.
- **radio_status_t** **radio_fsk_sleep** (void)

Przejdźcie do sleep backendu FSK.
- void **radio_fsk_process** (void)

Przetwarzanie logiki odroczonej backendu FSK.
- uint32_t **radio_fsk_take_events** (void)

Zwrócenie i wyczyszczenie flag zdarzeń backendu FSK.
- bool **radio_fsk_get_last_packet** (**radio_packet_t** *pkt)

Odczyt ostatniego pakietu backendu FSK.
- **radio_state_t** **radio_fsk_get_state** (void)

Odczyt stanu backendu FSK.

- [radio_status_t radio_fsk_raw_read_reg](#) (uint8_t reg, uint8_t *value)
Odczyt pojedynczego rejestru (backend FSK).
- [radio_status_t radio_fsk_raw_write_reg](#) (uint8_t reg, uint8_t value)
Zapis pojedynczego rejestru (backend FSK).
- [radio_status_t radio_fsk_raw_read_burst](#) (uint8_t reg, uint8_t *data, uint8_t len)
Odczyt sekwencji rejestrów (backend FSK).
- [radio_status_t radio_fsk_raw_write_burst](#) (uint8_t reg, const uint8_t *data, uint8_t len)
Zapis sekwencji rejestrów (backend FSK).
- void [radio_fsk_on_exti](#) (uint16_t gpio_pin)
Przekazanie pinu EXTI do backendu FSK.

Zmienne

- static [radio_fsk_context_t s_radio](#)
- static [radio_fsk_cfg_t s_runtime_cfg](#)
- static bool [s_runtime_cfg_ready](#) = false

12.54.1 Opis szczegółowy

Backend modulacji FSK/GFSK/MSK/GMSK dla SX1276.
Definicja w pliku [radio_fsk.c](#).

12.54.2 Dokumentacja definicji

12.54.2.1 RADIO_FSK_BROADCAST_ADDRESS

```
#define RADIO_FSK_BROADCAST_ADDRESS 0xFFU
```

Definicja w linii 20 pliku [radio_fsk.c](#).

12.54.2.2 RADIO_FSK_FIFO_MAX_BYTES

```
#define RADIO_FSK_FIFO_MAX_BYTES 64U
```

Definicja w linii 22 pliku [radio_fsk.c](#).

12.54.2.3 RADIO_FSK_FIFO_THRESH_LEVEL

```
#define RADIO_FSK_FIFO_THRESH_LEVEL 31U
```

Definicja w linii 16 pliku [radio_fsk.c](#).

12.54.2.4 RADIO_FSK_NODE_ADDRESS_DEFAULT

```
#define RADIO_FSK_NODE_ADDRESS_DEFAULT 0x01U
```

Definicja w linii 19 pliku [radio_fsk.c](#).

12.54.2.5 RADIO_FSK_RESET_HIGH_DELAY_MS

```
#define RADIO_FSK_RESET_HIGH_DELAY_MS 10U
```

Definicja w linii 18 pliku [radio_fsk.c](#).

12.54.2.6 RADIO_FSK_RESET_LOW_DELAY_MS

```
#define RADIO_FSK_RESET_LOW_DELAY_MS 2U
```

Definicja w linii 17 pliku [radio_fsk.c](#).

12.54.2.7 RADIO_FSK_TX_GUARD_MS

```
#define RADIO_FSK_TX_GUARD_MS 40UL
```

Definicja w linii 21 pliku [radio_fsk.c](#).

12.54.3 Dokumentacja funkcji

12.54.3.1 radio_apply_fsk_config()

```
bool radio_apply_fsk_config (
    const radio_fsk_cfg_t * cfg) [static]
```

Internal helper: [radio_apply_fsk_config](#).

Definicja w linii 271 pliku [radio_fsk.c](#).

Odwołuje się do [radio_fsk_cfg_t::bitrate_bps](#), [radio_fsk_cfg_t::frequency_hz](#), [radio_fsk_cfg_t::preamble_len](#), [radio_get_frequency_deviation\(\)](#), [radio_get_packet_config_1\(\)](#), [radio_get_rx_bw_reg_value\(\)](#), [radio_get_shaping_bits\(\)](#), [RADIO_LIB_MAX_PAYLOAD](#), [radio_write_sync_word\(\)](#), [radio_fsk_cfg_t::rx_bandwidth](#), [sx1276_set_frequency\(\)](#), [sx1276_set_pa_output_power\(\)](#), [sx1276_write_reg\(\)](#) i [radio_fsk_cfg_t::tx_power_dbm](#).

Odwołania w [radio_fsk_init\(\)](#).

12.54.3.2 radio_cfg_valid()

```
bool radio_cfg_valid (
    const radio_fsk_cfg_t * cfg) [static]
```

Internal helper: [radio_cfg_valid](#).

Definicja w linii 71 pliku [radio_fsk.c](#).

Odwołuje się do [radio_fsk_cfg_t::address_filter](#), [radio_fsk_cfg_t::bitrate_bps](#), [radio_fsk_cfg_t::crc_type](#), [radio_fsk_cfg_t::filter](#), [radio_fsk_cfg_t::frequency_hz](#), [radio_fsk_cfg_t::preamble_len](#), [RADIO_FSK_SHAPING_GMSK](#), [radio_fsk_cfg_t::rx_bandwidth](#), [radio_fsk_cfg_t::shaping](#) i [radio_fsk_cfg_t::sync_word_len](#).

Odwołania w [radio_fsk_init\(\)](#), [radio_fsk_set_runtime_cfg\(\)](#) i [radio_lora_init\(\)](#).

12.54.3.3 radio_fsk_default_hw_cfg()

```
void radio_fsk_default_hw_cfg (
    radio_hw_cfg_t * cfg,
    SPI_HandleTypeDef * hspi)
```

Domyślna konfiguracja sprzętowa backendu FSK.

Definicja w linii 467 pliku [radio_fsk.c](#).

Odwołuje się do [radio_lora_default_hw_cfg\(\)](#).

12.54.3.4 radio_fsk_default_lora_cfg()

```
void radio_fsk_default_lora_cfg (
    radio_lora_cfg_t * cfg)
```

Domyślna konfiguracja parametrów backendu FSK.

Definicja w linii 472 pliku [radio_fsk.c](#).

Odwołuje się do [radio_lora_default_lora_cfg\(\)](#).

12.54.3.5 radio_fsk_deinit()

```
radio_status_t radio_fsk_deinit (
    void )
```

Deinicjalizacja backendu FSK.

Definicja w linii 550 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_ESTATE](#), [radio_fsk_sleep\(\)](#), [RADIO_OK](#) i [RADIO_STATE_UNINIT](#).

Odwołania w [radio_deinit\(\)](#).

12.54.3.6 radio_fsk_get_last_packet()

```
bool radio_fsk_get_last_packet (
    radio_packet_t * pkt)
```

Odczyt ostatniego pakietu backendu FSK.

Definicja w linii 834 pliku [radio_fsk.c](#).

Odwołania w [radio_get_last_packet\(\)](#).

12.54.3.7 radio_fsk_get_state()

```
radio_state_t radio_fsk_get_state (
    void )
```

Odczyt stanu backendu FSK.

Definicja w linii 845 pliku [radio_fsk.c](#).

Odwołania w [radio_get_state\(\)](#).

12.54.3.8 radio_fsk_init()

```
radio_status_t radio_fsk_init (
    const radio_hw_cfg_t * hw,
    const radio_lora_cfg_t * cfg,
    radio_event_cb_t cb,
    void * user_ctx)
```

Inicjalizacja backendu FSK.

Definicja w linii 489 pliku [radio_fsk.c](#).

Odwołuje się do [radio_hw_cfg_t::hspi](#), [radio_hw_cfg_t::nss](#), [radio_gpio_t::pin](#), [radio_gpio_t::port](#), [radio_apply_fsk_config\(\)](#), [radio_cfg_valid\(\)](#), [radio_default_fsk_cfg\(\)](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [radio_hw_reset\(\)](#), [RADIO_OK](#), [RADIO_STATE_STANDBY](#), [RADIO_STATE_UNINIT](#), [radio_hw_cfg_t::reset](#), [radio_hw_cfg_t::spi_timeout_ms](#) i [sx1276_get_version\(\)](#).

Odwołania w [radio_init\(\)](#).

12.54.3.9 radio_fsk_on_exti()

```
void radio_fsk_on_exti (
    uint16_t gpio_pin)
```

Przekazanie pinu EXTI do backendu FSK.

Definicja w linii 900 pliku [radio_fsk.c](#).

Odwołuje się do [radio_handle_exti_pin\(\)](#).

Odwołania w [radio_on_exti\(\)](#).

12.54.3.10 radio_fsk_process()

```
void radio_fsk_process (
    void )
```

Przetwarzanie logiki odroczonej backendu FSK.

Definicja w linii 705 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EVENT_CRC_ERR](#), [RADIO_EVENT_FIFO_OVERRUN](#), [RADIO_EVENT_HW_ERROR](#), [RADIO_EVENT_NONE](#), [RADIO_EVENT_RX_DONE](#), [RADIO_EVENT_RX_TIMEOUT](#), [RADIO_EVENT_TX_DONE](#), [radio_fsk_standby\(\)](#), [radio_irq_restore\(\)](#), [radio_irq_save\(\)](#), [RADIO_OK](#), [radio_read_rx_packet\(\)](#), [radio_resume_after_tx\(\)](#), [radio_set_event_flags\(\)](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_TX](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_process\(\)](#).

12.54.3.11 radio_fsk_raw_read_burst()

```
radio_status_t radio_fsk_raw_read_burst (
    uint8_t reg,
    uint8_t * data,
    uint8_t len)
```

Odczyt sekwencji rejestrów (backend FSK).

Definicja w linii 870 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_read_burst\(\)](#).

Odwołania w [radio_raw_read_burst\(\)](#).

12.54.3.12 radio_fsk_raw_read_reg()

```
radio_status_t radio_fsk_raw_read_reg (
    uint8_t reg,
    uint8_t * value)
```

Odczyt pojedynczego rejestru (backend FSK).

Definicja w linii 850 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_raw_read_reg\(\)](#).

12.54.3.13 radio_fsk_raw_write_burst()

```
radio_status_t radio_fsk_raw_write_burst (  
    uint8_t reg,  
    const uint8_t * data,  
    uint8_t len)
```

Zapis sekwencji rejestrów (backend FSK).

Definicja w linii 885 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_write_burst\(\)](#).

Odwołania w [radio_raw_write_burst\(\)](#).

12.54.3.14 radio_fsk_raw_write_reg()

```
radio_status_t radio_fsk_raw_write_reg (  
    uint8_t reg,  
    uint8_t value)
```

Zapis pojedynczego rejestru (backend FSK).

Definicja w linii 860 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_raw_write_reg\(\)](#).

12.54.3.15 radio_fsk_send_async()

```
radio_status_t radio_fsk_send_async (  
    const uint8_t * data,  
    uint8_t len)
```

Uruchomienie nadawania asynchronicznego FSK.

Definicja w linii 618 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [radio_get_shaping_bits\(\)](#), [radio_irq_restore\(\)](#), [radio_irq_save\(\)](#), [RADIO_LIB_MAX_PAYLOAD](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_TX](#), [radio_tx_timeout_ms\(\)](#), [sx1276_set_op_mode\(\)](#), [sx1276_write_burst\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_send_async\(\)](#).

12.54.3.16 radio_fsk_set_runtime_cfg()

```
radio_status_t radio_fsk_set_runtime_cfg (  
    const radio_fsk_cfg_t * cfg)
```

Ustawia runtime konfigurację właściwą dla backendu FSK.

Definicja w linii 477 pliku [radio_fsk.c](#).

Odwołuje się do [radio_cfg_valid\(\)](#), [RADIO_EINVAL](#) i [RADIO_OK](#).

Odwołania w [radio_set_fsk_cfg\(\)](#).

12.54.3.17 radio_fsk_sleep()

```
radio_status_t radio_fsk_sleep (  
    void )
```

Przejsie do sleep backendu FSK.

Definicja w linii 687 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [radio_get_shaping_bits\(\)](#), [RADIO_OK](#), [radio_set_state\(\)](#) i [RADIO_STATE_STANDBY](#).

Odwołania w [radio_fsk_deinit\(\)](#) i [radio_sleep\(\)](#).

12.54.3.18 radio_fsk_standby()

```
radio_status_t radio_fsk_standby (
    void )
```

Przejsie do standby backendu FSK.

Definicja w linii 669 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [radio_get_shaping_bits\(\)](#), [RADIO_OK](#), [radio_set_state\(\)](#) i [RADIO_STATE_STANDBY](#).

Odwołania w [radio_fsk_process\(\)](#) i [radio_standby\(\)](#).

12.54.3.19 radio_fsk_start_rx_continuous()

```
radio_status_t radio_fsk_start_rx_continuous (
    void )
```

Uruchomienie odbioru ciągłego FSK.

Definicja w linii 563 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_ESTATE](#), [radio_get_shaping_bits\(\)](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_CONT](#), [RADIO_STATE_TX](#), [sx1276_set_op_mode\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_start_rx_continuous\(\)](#).

12.54.3.20 radio_fsk_start_rx_single()

```
radio_status_t radio_fsk_start_rx_single (
    uint16_t symbol_timeout)
```

Uruchomienie odbioru pojedynczego FSK.

Definicja w linii 588 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [radio_get_shaping_bits\(\)](#), [RADIO_OK](#), [radio_rx_single_timeout_ms\(\)](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_TX](#), [sx1276_set_op_mode\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_start_rx_single\(\)](#).

12.54.3.21 radio_fsk_take_events()

```
uint32_t radio_fsk_take_events (
    void )
```

Zwrócenie i wyczyszczenie flag zdarzeń backendu FSK.

Definicja w linii 823 pliku [radio_fsk.c](#).

Odwołuje się do [radio_irq_restore\(\)](#) i [radio_irq_save\(\)](#).

Odwołania w [radio_take_events\(\)](#).

12.54.3.22 radio_get_frequency_deviation()

```
uint32_t radio_get_frequency_deviation (
    const radio_fsk_cfg_t * cfg) [static]
```

Internal helper: [radio_get_frequency_deviation](#).

Definicja w linii 162 pliku [radio_fsk.c](#).

Odwołuje się do [radio_fsk_cfg_t::bitrate_bps](#), [RADIO_FSK_SHAPING_GMSK](#), [RADIO_FSK_SHAPING_MSK](#) i [radio_fsk_cfg_t::shaping](#).

Odwołania w [radio_apply_fsk_config\(\)](#).

12.54.3.23 radio_get_packet_config_1()

```
uint8_t radio_get_packet_config_1 (
    const radio_fsk_cfg_t * cfg) [static]
```

Internal helper: [radio_get_packet_config_1](#).

Definicja w linii 208 pliku [radio_fsk.c](#).

Odwołuje się do [radio_fsk_cfg_t::address_filter](#), [radio_fsk_cfg_t::crc_type](#) i [radio_fsk_cfg_t::data_whitening](#).

Odwołania w [radio_apply_fsk_config\(\)](#).

12.54.3.24 radio_get_rx_bw_reg_value()

```
uint8_t radio_get_rx_bw_reg_value (
    radio_lora_bw_t bw) [static]
```

Internal helper: [radio_get_rx_bw_reg_value](#).

Definicja w linii 102 pliku [radio_fsk.c](#).

Odwołania w [radio_apply_fsk_config\(\)](#).

12.54.3.25 radio_get_shaping_bits()

```
uint8_t radio_get_shaping_bits (
    const radio_fsk_cfg_t * cfg) [static]
```

Internal helper: [radio_get_shaping_bits](#).

Definicja w linii 131 pliku [radio_fsk.c](#).

Odwołuje się do [radio_fsk_cfg_t::filter](#), [RADIO_FSK_SHAPING_FSK](#), [RADIO_FSK_SHAPING_MSK](#) i [radio_fsk_cfg_t::shaping](#).

Odwołania w [radio_apply_fsk_config\(\)](#), [radio_fsk_send_async\(\)](#), [radio_fsk_sleep\(\)](#), [radio_fsk_standby\(\)](#), [radio_fsk_start_rx_continuous\(\)](#), [radio_fsk_start_rx_single\(\)](#) i [radio_resume_after_tx\(\)](#).

12.54.3.26 radio_handle_exti_pin()

```
void radio_handle_exti_pin (
    uint16_t pin) [static]
```

Internal helper: [radio_handle_exti_pin](#).

Definicja w linii 410 pliku [radio_fsk.c](#).

Odwołuje się do [radio_irq_restore\(\)](#) i [radio_irq_save\(\)](#).

Odwołania w [radio_fsk_on_exti\(\)](#).

12.54.3.27 radio_hw_reset()

```
void radio_hw_reset (
    void ) [static]
```

Internal helper: [radio_hw_reset](#).

Definicja w linii 92 pliku [radio_fsk.c](#).

Odwołania w [radio_fsk_init\(\)](#).

12.54.3.28 radio_irq_restore()

```
void radio_irq_restore (
    uint32_t primask) [static]
```

Internal helper: [radio_irq_restore](#).

Definicja w linii 54 pliku [radio_fsk.c](#).

Odwołania w [radio_fsk_process\(\)](#), [radio_fsk_send_async\(\)](#), [radio_fsk_take_events\(\)](#), [radio_handle_exti_pin\(\)](#) i [radio_set_event_flags\(\)](#).

12.54.3.29 radio_irq_save()

```
uint32_t radio_irq_save (
    void ) [static]
```

Internal helper: [radio_irq_save](#).

Definicja w linii 46 pliku [radio_fsk.c](#).

Odwołania w [radio_fsk_process\(\)](#), [radio_fsk_send_async\(\)](#), [radio_fsk_take_events\(\)](#), [radio_handle_exti_pin\(\)](#) i [radio_set_event_flags\(\)](#).

12.54.3.30 radio_read_rx_packet()

```
radio_status_t radio_read_rx_packet (
    void ) [static]
```

Internal helper: [radio_read_rx_packet](#).

Definicja w linii 432 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_LIB_MAX_PAYLOAD](#), [RADIO_OK](#), [sx1276_read_burst\(\)](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_fsk_process\(\)](#).

12.54.3.31 [radio_resume_after_tx\(\)](#)

```
void radio_resume_after_tx (  
    void ) [static]
```

Internal helper: [radio_resume_after_tx](#).

Definicja w linii 324 pliku [radio_fsk.c](#).

Odwołuje się do [radio_get_shaping_bits\(\)](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_CONT](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_STANDBY](#) i [sx1276_set_op_mode\(\)](#).

Odwołania w [radio_fsk_process\(\)](#).

12.54.3.32 [radio_rx_single_timeout_ms\(\)](#)

```
uint32_t radio_rx_single_timeout_ms (  
    uint32_t symbol_timeout) [static]
```

Internal helper: [radio_rx_single_timeout_ms](#).

Definicja w linii 385 pliku [radio_fsk.c](#).

Odwołania w [radio_fsk_start_rx_single\(\)](#).

12.54.3.33 [radio_set_event_flags\(\)](#)

```
void radio_set_event_flags (  
    uint32_t events) [static]
```

Internal helper: [radio_set_event_flags](#).

Definicja w linii 63 pliku [radio_fsk.c](#).

Odwołuje się do [radio_irq_restore\(\)](#) i [radio_irq_save\(\)](#).

Odwołania w [radio_fsk_process\(\)](#).

12.54.3.34 [radio_set_op_mode_ready\(\)](#)

```
bool radio_set_op_mode_ready (  
    uint8_t op_mode) [static]
```

Definicja w linii 178 pliku [radio_fsk.c](#).

12.54.3.35 [radio_set_state\(\)](#)

```
void radio_set_state (  
    radio_state_t state) [static]
```

Internal helper: [radio_set_state](#).

Definicja w linii 318 pliku [radio_fsk.c](#).

Odwołania w [radio_fsk_send_async\(\)](#), [radio_fsk_sleep\(\)](#), [radio_fsk_standby\(\)](#), [radio_fsk_start_rx_continuous\(\)](#), [radio_fsk_start_rx_single\(\)](#) i [radio_resume_after_tx\(\)](#).

12.54.3.36 [radio_tx_timeout_ms\(\)](#)

```
uint32_t radio_tx_timeout_ms (  
    uint8_t payload_len) [static]
```

Internal helper: [radio_tx_timeout_ms](#).

Definicja w linii 353 pliku [radio_fsk.c](#).

Odwołania w [radio_fsk_send_async\(\)](#).

12.54.3.37 [radio_write_sync_word\(\)](#)

```
bool radio_write_sync_word (  
    const radio_fsk_cfg_t * cfg) [static]
```

Internal helper: [radio_write_sync_word](#).

Definicja w linii 250 pliku `radio_fsk.c`.

Odwołuje się do `sx1276_write_burst()`, `sx1276_write_reg()`, `radio_fsk_cfg_t::sync_word` i `radio_fsk_cfg_t::sync_word_len`.

Odwołania w `radio_apply_fsk_config()`.

12.54.4 Dokumentacja zmiennych

12.54.4.1 s_radio

```
radio_fsk_context_t s_radio [static]
```

Definicja w linii 41 pliku `radio_fsk.c`.

12.54.4.2 s_runtime_cfg

```
radio_fsk_cfg_t s_runtime_cfg [static]
```

Definicja w linii 42 pliku `radio_fsk.c`.

12.54.4.3 s_runtime_cfg_ready

```
bool s_runtime_cfg_ready = false [static]
```

Definicja w linii 43 pliku `radio_fsk.c`.

12.55 radio_fsk.c

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #include "radio_fsk.h"
00007
00008 #include "app_delay.h"
00009 #include "main.h"
00010 #include "../common/sx1276/radio_sx1276.h"
00011 #include "../common/sx1276/radio_sx1276_regs.h"
00012 #include "../lora/radio_lora.h"
00013
00014 #include <string.h>
00015
00016 #define RADIO_FSK_FIFO_THRESH_LEVEL      31U
00017 #define RADIO_FSK_RESET_LOW_DELAY_MS    2U
00018 #define RADIO_FSK_RESET_HIGH_DELAY_MS   10U
00019 #define RADIO_FSK_NODE_ADDRESS_DEFAULT  0x01U
00020 #define RADIO_FSK_BROADCAST_ADDRESS     0xFFU
00021 #define RADIO_FSK_TX_GUARD_MS           40UL
00022 #define RADIO_FSK_FIFO_MAX_BYTES        64U
00023
00024 typedef struct
00025 {
00026     bool initialized;
00027     sx1276_bus_t bus;
00028     radio_hw_cfg_t hw;
00029     radio_fsk_cfg_t cfg;
00030     radio_event_cb_t callback;
00031     void *callback_ctx;
00032     volatile uint8_t dio_pending_mask;
00033     volatile uint32_t event_flags;
00034     radio_state_t state;
00035     radio_state_t tx_resume_state;
00036     uint32_t rx_single_deadline_ms;
00037     uint32_t tx_deadline_ms;
00038     radio_packet_t last_packet;
00039 } radio_fsk_context_t;
00040
00041 static radio_fsk_context_t s_radio;
00042 static radio_fsk_cfg_t s_runtime_cfg;
00043 static bool s_runtime_cfg_ready = false;
00044
00046 static uint32_t radio_irq_save(void)
00047 {
00048     uint32_t primask = __get_PRIMASK();
00049     __disable_irq();
00050     return primask;
00051 }
00052
00054 static void radio_irq_restore(uint32_t primask)
00055 {
00056     if (primask == 0U)
```

```

00057     {
00058         __enable_irq();
00059     }
00060 }
00061
00063 static void radio_set_event_flags(uint32_t events)
00064 {
00065     uint32_t key = radio_irq_save();
00066     s_radio.event_flags |= events;
00067     radio_irq_restore(key);
00068 }
00069
00071 static bool radio_cfg_valid(const radio_fsk_cfg_t *cfg)
00072 {
00073     if ((cfg == NULL) ||
00074         (cfg->frequency_hz < 137000000UL) || (cfg->frequency_hz > 1020000000UL) ||
00075         (cfg->bitrate_bps < 600UL) || (cfg->bitrate_bps > 300000UL) ||
00076         ((uint8_t)cfg->rx_bandwidth > (uint8_t)RADIO_LORA_BW_500_KHZ) ||
00077         ((uint8_t)cfg->shaping > (uint8_t)RADIO_FSK_SHAPING_GMSK) ||
00078         ((uint8_t)cfg->filter > (uint8_t)RADIO_FSK_FILTER_BT_03) ||
00079         (cfg->preamble_len == 0U) ||
00080         (cfg->sync_word_len == 0U) ||
00081         (cfg->sync_word_len > 4U) ||
00082         ((uint8_t)cfg->address_filter > (uint8_t)RADIO_ADDRESS_FILTER_NODE_BROADCAST) ||
00083         ((uint8_t)cfg->crc_type > (uint8_t)RADIO_PACKET_CRC_IBM))
00084     {
00085         return false;
00086     }
00087
00088     return true;
00089 }
00090
00092 static void radio_hw_reset(void)
00093 {
00094     HAL_GPIO_WritePin(s_radio.hw.nss.port, s_radio.hw.nss.pin, GPIO_PIN_SET);
00095     HAL_GPIO_WritePin(s_radio.hw.reset.port, s_radio.hw.reset.pin, GPIO_PIN_RESET);
00096     app_delay_ms(RADIO_FSK_RESET_LOW_DELAY_MS);
00097     HAL_GPIO_WritePin(s_radio.hw.reset.port, s_radio.hw.reset.pin, GPIO_PIN_SET);
00098     app_delay_ms(RADIO_FSK_RESET_HIGH_DELAY_MS);
00099 }
00100
00102 static uint8_t radio_get_rx_bw_reg_value(radio_lora_bw_t bw)
00103 {
00104     switch (bw)
00105     {
00106         case RADIO_LORA_BW_7_8_KHZ:
00107             return 0x06U;
00108         case RADIO_LORA_BW_10_4_KHZ:
00109             return 0x15U;
00110         case RADIO_LORA_BW_15_6_KHZ:
00111             return 0x05U;
00112         case RADIO_LORA_BW_20_8_KHZ:
00113             return 0x14U;
00114         case RADIO_LORA_BW_31_25_KHZ:
00115             return 0x04U;
00116         case RADIO_LORA_BW_41_7_KHZ:
00117             return 0x13U;
00118         case RADIO_LORA_BW_62_5_KHZ:
00119             return 0x03U;
00120         case RADIO_LORA_BW_125_KHZ:
00121             return 0x02U;
00122         case RADIO_LORA_BW_250_KHZ:
00123             return 0x01U;
00124         case RADIO_LORA_BW_500_KHZ:
00125             default:
00126                 return 0x00U;
00127     }
00128 }
00129
00131 static uint8_t radio_get_shaping_bits(const radio_fsk_cfg_t *cfg)
00132 {
00133     if (cfg->shaping == RADIO_FSK_SHAPING_FSK)
00134     {
00135         return SX1276_OPMODE_SHAPING_NONE;
00136     }
00137
00138     if (cfg->shaping == RADIO_FSK_SHAPING_MSK)
00139     {
00140         return SX1276_OPMODE_SHAPING_FSK_BT10;
00141     }
00142
00143     switch (cfg->filter)
00144     {
00145         case RADIO_FSK_FILTER_BT_10:
00146             return SX1276_OPMODE_SHAPING_FSK_BT10;
00147
00148         case RADIO_FSK_FILTER_BT_07:

```



```

00149         case RADIO_FSK_FILTER_BT_05:
00150             return SX1276_OPMODE_SHAPING_FSK_BT05;
00151
00152         case RADIO_FSK_FILTER_BT_03:
00153             return SX1276_OPMODE_SHAPING_FSK_BT03;
00154
00155         case RADIO_FSK_FILTER_NONE:
00156             default:
00157                 return SX1276_OPMODE_SHAPING_NONE;
00158     }
00159 }
00160
00162 static uint32_t radio_get_frequency_deviation(const radio_fsk_cfg_t *cfg)
00163 {
00164     if ((cfg->shaping == RADIO_FSK_SHAPING_MSK) ||
00165         (cfg->shaping == RADIO_FSK_SHAPING_GMSK))
00166     {
00167         return (cfg->bitrate_bps / 4UL);
00168     }
00169
00170     return (cfg->bitrate_bps / 2UL);
00171 }
00172
00173 /*
00174  * Waits until the FSK modem reports ModeReady after leaving RX.
00175  * Semtech fixed a known SX127x issue around FSK transmissions started while the
00176  * radio was still in RX, so this helper makes the RX->STDBY transition explicit.
00177  */
00178 static bool radio_set_op_mode_ready(uint8_t op_mode)
00179 {
00180     uint8_t irq1;
00181     uint8_t retry;
00182
00183     if (!sx1276_set_op_mode(&s_radio.bus, op_mode))
00184     {
00185         return false;
00186     }
00187
00188     if ((op_mode & SX1276_OPMODE_MODE_MASK) == SX1276_MODE_SLEEP)
00189     {
00190         app_delay_ms(1U);
00191         return true;
00192     }
00193
00194     for (retry = 0U; retry < 5U; retry++)
00195     {
00196         if (sx1276_read_reg(&s_radio.bus, SX1276_REG_IRQ_FLAGS_1, &irq1) &&
00197             ((irq1 & SX1276_IRQ1_MODE_READY) != 0U))
00198         {
00199             return true;
00200         }
00201         app_delay_ms(1U);
00202     }
00203
00204     return false;
00205 }
00206
00208 static uint8_t radio_get_packet_config_1(const radio_fsk_cfg_t *cfg)
00209 {
00210     uint8_t value = SX1276_PACKET_FORMAT_VARIABLE;
00211
00212     if (cfg->data_whitening)
00213     {
00214         value |= SX1276_PACKET_DC_FREE_WHITENING;
00215     }
00216
00217     if (cfg->crc_type != RADIO_PACKET_CRC_OFF)
00218     {
00219         value |= SX1276_PACKET_CRC_ON;
00220         if (cfg->crc_type == RADIO_PACKET_CRC_IBM)
00221         {
00222             value |= SX1276_PACKET_CRC_WHITENING_IBM;
00223         }
00224         else
00225         {
00226             value |= SX1276_PACKET_CRC_WHITENING_CCITT;
00227         }
00228     }
00229
00230     switch (cfg->address_filter)
00231     {
00232         case RADIO_ADDRESS_FILTER_NODE:
00233             value |= SX1276_PACKET_ADDR_FILTER_NODE;
00234             break;
00235
00236         case RADIO_ADDRESS_FILTER_NODE_BROADCAST:
00237             value |= SX1276_PACKET_ADDR_FILTER_NODE_BC;

```

```

00238         break;
00239
00240     case RADIO_ADDRESS_FILTER_OFF:
00241     default:
00242         value |= SX1276_PACKET_ADDR_FILTER_OFF;
00243         break;
00244     }
00245
00246     return value;
00247 }
00248
00250 static bool radio_write_sync_word(const radio_fsk_cfg_t *cfg)
00251 {
00252     uint8_t sync_config;
00253     uint8_t sync_bytes[4];
00254     uint8_t i;
00255
00256     sync_config = (uint8_t) (SX1276_SYNC_AUTO_RESTART_PLL |
00257                             SX1276_SYNC_ON |
00258                             SX1276_SYNC_FIFO_FILL_AUTO |
00259                             ((cfg->sync_word_len - 1U) & 0x07U));
00260     for (i = 0U; i < cfg->sync_word_len; i++)
00261     {
00262         uint8_t shift = (uint8_t) ((cfg->sync_word_len - 1U - i) * 8U) & 0x1FU;
00263         sync_bytes[i] = (uint8_t) ((cfg->sync_word >> shift) & 0xFFU);
00264     }
00265
00266     return sx1276_write_reg(&s_radio.bus, SX1276_REG_SYNC_CONFIG, sync_config) &&
00267         sx1276_write_burst(&s_radio.bus, SX1276_REG_SYNC_VALUE_1, sync_bytes, cfg->sync_word_len);
00268 }
00269
00271 static bool radio_apply_fsk_config(const radio_fsk_cfg_t *cfg)
00272 {
00273     uint32_t bitrate_reg;
00274     uint32_t fdev_reg;
00275     uint8_t op_mode;
00276     uint8_t rx_bw;
00277
00278     bitrate_reg = (32000000UL + (cfg->bitrate_bps / 2UL)) / cfg->bitrate_bps;
00279     fdev_reg = (uint32_t) (((uint64_t) radio_get_frequency_deviation(cfg)) << 19) / 32000000UL;
00280     rx_bw = radio_get_rx_bw_reg_value(cfg->rx_bandwidth);
00281     op_mode = (uint8_t) (SX1276_OPMODE_MODULATION_FSK |
00282                         radio_get_shaping_bits(cfg) |
00283                         SX1276_MODE_SLEEP);
00284
00285     return radio_set_op_mode_ready(op_mode) &&
00286         sx1276_set_frequency(&s_radio.bus, cfg->frequency_hz) &&
00287         sx1276_set_pa_output_power(&s_radio.bus, cfg->tx_power_dbm) &&
00288         sx1276_write_reg(&s_radio.bus, SX1276_REG_BITRATE_MSB, (uint8_t) ((bitrate_reg >> 8) &
00289     0xFFU)) &&
00289         sx1276_write_reg(&s_radio.bus, SX1276_REG_BITRATE_LSB, (uint8_t) (bitrate_reg & 0xFFU)) &&
00290         sx1276_write_reg(&s_radio.bus, SX1276_REG_FDEV_MSB, (uint8_t) ((fdev_reg >> 8) & 0x3FU)) &&
00291         sx1276_write_reg(&s_radio.bus, SX1276_REG_FDEV_LSB, (uint8_t) (fdev_reg & 0xFFU)) &&
00292         sx1276_write_reg(&s_radio.bus, SX1276_REG_RX_BW, rx_bw) &&
00293         sx1276_write_reg(&s_radio.bus, SX1276_REG_AFC_BW, rx_bw) &&
00294         sx1276_write_reg(&s_radio.bus, SX1276_REG_PREAMBLE_DETECT, 0xAAU) &&
00295         sx1276_write_reg(&s_radio.bus, SX1276_REG_RX_CONFIG,
00296             (uint8_t) (SX1276_RXCONFIG_AFC_AUTO_ON |
00297                     SX1276_RXCONFIG_AGC_AUTO_ON |
00298                     SX1276_RXCONFIG_TRIGGER_RSSI)) &&
00299         sx1276_write_reg(&s_radio.bus, SX1276_REG_PREAMBLE_MSB_FSK, (uint8_t) ((cfg->preamble_len >>
00300     8) & 0xFFU)) &&
00300         sx1276_write_reg(&s_radio.bus, SX1276_REG_PREAMBLE_LSB_FSK, (uint8_t) (cfg->preamble_len &
00301     0xFFU)) &&
00301         radio_write_sync_word(cfg) &&
00302         sx1276_write_reg(&s_radio.bus, SX1276_REG_PACKET_CONFIG_1, radio_get_packet_config_1(cfg))
00303     &&
00303         sx1276_write_reg(&s_radio.bus, SX1276_REG_PACKET_CONFIG_2, SX1276_PACKET_DATA_MODE_PACKET)
00304     &&
00304         sx1276_write_reg(&s_radio.bus, SX1276_REG_PAYLOAD_LENGTH_FSK, RADIO_LIB_MAX_PAYLOAD) &&
00305         sx1276_write_reg(&s_radio.bus, SX1276_REG_NODE_ADDRESS, RADIO_FSK_NODE_ADDRESS_DEFAULT) &&
00306         sx1276_write_reg(&s_radio.bus, SX1276_REG_BROADCAST_ADDRESS, RADIO_FSK_BROADCAST_ADDRESS)
00307     &&
00307         sx1276_write_reg(&s_radio.bus, SX1276_REG_FIFO_THRESH,
00308             (uint8_t) (SX1276_FIFO_THRESH_TX_START_NOT_EMPTY |
00309                     (RADIO_FSK_FIFO_THRESH_LEVEL & 0x3FU))) &&
00309         sx1276_write_reg(&s_radio.bus, SX1276_REG_DIO_MAPPING_1, 0x00U) &&
00310         sx1276_write_reg(&s_radio.bus, SX1276_REG_DIO_MAPPING_2, 0x00U) &&
00311         radio_set_op_mode_ready((uint8_t) (SX1276_OPMODE_MODULATION_FSK |
00312             radio_get_shaping_bits(cfg) |
00313             SX1276_MODE_STDBY));
00314 }
00315 }
00316
00318 static void radio_set_state(radio_state_t state)
00319 {
00320     s_radio.state = state;
00321 }

```

```

00322
00324 static void radio_resume_after_tx(void)
00325 {
00326     if (s_radio.tx_resume_state == RADIO_STATE_RX_CONT)
00327     {
00328         (void)sx1276_set_op_mode(&s_radio.bus,
00329                                 (uint8_t)(SX1276_OPMODE_MODULATION_FSK |
00330                                             radio_get_shaping_bits(&s_radio.cfg) |
00331                                             SX1276_MODE_RXCONTINUOUS));
00332         radio_set_state(RADIO_STATE_RX_CONT);
00333     }
00334     else if (s_radio.tx_resume_state == RADIO_STATE_RX_SINGLE)
00335     {
00336         (void)sx1276_set_op_mode(&s_radio.bus,
00337                                 (uint8_t)(SX1276_OPMODE_MODULATION_FSK |
00338                                             radio_get_shaping_bits(&s_radio.cfg) |
00339                                             SX1276_MODE_RXSINGLE));
00340         radio_set_state(RADIO_STATE_RX_SINGLE);
00341     }
00342     else
00343     {
00344         (void)sx1276_set_op_mode(&s_radio.bus,
00345                                 (uint8_t)(SX1276_OPMODE_MODULATION_FSK |
00346                                             radio_get_shaping_bits(&s_radio.cfg) |
00347                                             SX1276_MODE_STDBY));
00348         radio_set_state(RADIO_STATE_STANDBY);
00349     }
00350 }
00351
00353 static uint32_t radio_tx_timeout_ms(uint8_t payload_len)
00354 {
00355     uint64_t timeout_ms;
00356     uint32_t total_bytes;
00357     uint32_t crc_len = (s_radio.cfg.crc_type == RADIO_PACKET_CRC_OFF) ? 0UL : 2UL;
00358
00359     total_bytes = (uint32_t)s_radio.cfg.preamble_len +
00360                  (uint32_t)s_radio.cfg.sync_word_len +
00361                  (uint32_t)payload_len +
00362                  crc_len +
00363                  1UL + /* variable-length byte */
00364                  2UL; /* guard for packet engine overhead */
00365     timeout_ms = ((uint64_t)total_bytes * 8ULL * 1000ULL) / s_radio.cfg.bitrate_bps;
00366     if (((uint64_t)total_bytes * 8ULL * 1000ULL) % s_radio.cfg.bitrate_bps) != 0ULL)
00367     {
00368         timeout_ms++;
00369     }
00370
00371     timeout_ms += RADIO_FSK_TX_GUARD_MS;
00372     if (timeout_ms < RADIO_FSK_TX_GUARD_MS)
00373     {
00374         timeout_ms = RADIO_FSK_TX_GUARD_MS;
00375     }
00376     if (timeout_ms > 0xFFFFFFFFULL)
00377     {
00378         return 0xFFFFFFFFFUL;
00379     }
00380
00381     return (uint32_t)timeout_ms;
00382 }
00383
00385 static uint32_t radio_rx_single_timeout_ms(uint32_t symbol_timeout)
00386 {
00387     uint64_t timeout_ms;
00388
00389     timeout_ms = ((uint64_t)symbol_timeout * 1000ULL) / s_radio.cfg.bitrate_bps;
00390     if (((uint64_t)symbol_timeout * 1000ULL) % s_radio.cfg.bitrate_bps) != 0ULL)
00391     {
00392         timeout_ms++;
00393     }
00394
00395     timeout_ms += 2ULL;
00396     if (timeout_ms == 0ULL)
00397     {
00398         timeout_ms = 1ULL;
00399     }
00400
00401     if (timeout_ms > 0xFFFFFFFFULL)
00402     {
00403         return 0xFFFFFFFFFUL;
00404     }
00405
00406     return (uint32_t)timeout_ms;
00407 }
00408
00410 static void radio_handle_exti_pin(uint16_t pin)
00411 {
00412     uint8_t i;

```

```

00413
00414     if (!s_radio.initialized)
00415     {
00416         return;
00417     }
00418
00419     for (i = 0U; i < 6U; i++)
00420     {
00421         if (s_radio.hw.dio[i].pin == pin)
00422         {
00423             uint32_t key = radio_irq_save();
00424             s_radio.dio_pending_mask |= (uint8_t)(1U << i);
00425             radio_irq_restore(key);
00426             break;
00427         }
00428     }
00429 }
00430
00432 static radio_status_t radio_read_rx_packet(void)
00433 {
00434     uint8_t packet_len;
00435     uint8_t rssi_raw;
00436
00437     if (!sx1276_read_burst(&s_radio.bus, SX1276_REG_FIFO, &packet_len, 1U))
00438     {
00439         return RADIO_EHW;
00440     }
00441
00442     if (packet_len > RADIO_LIB_MAX_PAYLOAD)
00443     {
00444         s_radio.last_packet.valid = false;
00445         return RADIO_EBUS;
00446     }
00447
00448     if ((packet_len > 0U) &&
00449         !sx1276_read_burst(&s_radio.bus, SX1276_REG_FIFO, s_radio.last_packet.data, packet_len))
00450     {
00451         return RADIO_EHW;
00452     }
00453
00454     if (!sx1276_read_reg(&s_radio.bus, SX1276_REG_RSSI_VALUE_FSK, &rssi_raw))
00455     {
00456         return RADIO_EHW;
00457     }
00458
00459     s_radio.last_packet.length = packet_len;
00460     s_radio.last_packet.rssi_dbm = (int16_t)(-(int16_t)rssi_raw / 2);
00461     s_radio.last_packet.snr_db = 0;
00462     s_radio.last_packet.timestamp_ms = HAL_GetTick();
00463     s_radio.last_packet.valid = true;
00464     return RADIO_OK;
00465 }
00466
00467 void radio_fsk_default_hw_cfg(radio_hw_cfg_t *cfg, SPI_HandleTypeDef *hspl)
00468 {
00469     radio_lora_default_hw_cfg(cfg, hspl);
00470 }
00471
00472 void radio_fsk_default_lora_cfg(radio_lora_cfg_t *cfg)
00473 {
00474     radio_lora_default_lora_cfg(cfg);
00475 }
00476
00477 radio_status_t radio_fsk_set_runtime_cfg(const radio_fsk_cfg_t *cfg)
00478 {
00479     if (!radio_cfg_valid(cfg))
00480     {
00481         return RADIO_EINVAL;
00482     }
00483
00484     s_runtime_cfg = *cfg;
00485     s_runtime_cfg_ready = true;
00486     return RADIO_OK;
00487 }
00488
00489 radio_status_t radio_fsk_init(const radio_hw_cfg_t *hw,
00490                             const radio_lora_cfg_t *cfg,
00491                             radio_event_cb_t cb,
00492                             void *user_ctx)
00493 {
00494     uint8_t version;
00495
00496     (void)cfg;
00497
00498     if ((hw == NULL) || (hw->hspl == NULL) ||
00499         (hw->nss.port == NULL) || (hw->reset.port == NULL))
00500     {

```

```

00501     return RADIO_EINVAL;
00502 }
00503
00504 if (!s_runtime_cfg_ready)
00505 {
00506     radio_default_fsk_cfg(&s_runtime_cfg);
00507     s_runtime_cfg_ready = true;
00508 }
00509
00510 if (!radio_cfg_valid(&s_runtime_cfg))
00511 {
00512     return RADIO_EINVAL;
00513 }
00514
00515 memset(&s_radio, 0, sizeof(s_radio));
00516 s_radio.hw = *hw;
00517 s_radio.cfg = s_runtime_cfg;
00518 s_radio.callback = cb;
00519 s_radio.callback_ctx = user_ctx;
00520 s_radio.bus.hspi = hw->hspi;
00521 s_radio.bus.nss_port = hw->nss.port;
00522 s_radio.bus.nss_pin = hw->nss.pin;
00523 s_radio.bus.spi_timeout_ms = hw->spi_timeout_ms;
00524 s_radio.state = RADIO_STATE_UNINIT;
00525
00526 radio_hw_reset();
00527
00528 if (!radio_set_op_mode_ready((uint8_t) (SX1276_OPMODE_MODULATION_FSK | SX1276_MODE_SLEEP)))
00529 {
00530     return RADIO_EHW;
00531 }
00532
00533 app_delay_ms(1U);
00534
00535 if (!sx1276_get_version(&s_radio.bus, &version) || (version != SX1276_VERSION_ID))
00536 {
00537     return RADIO_EHW;
00538 }
00539
00540 if (!radio_apply_fsk_config(&s_radio.cfg))
00541 {
00542     return RADIO_EHW;
00543 }
00544
00545 s_radio.initialized = true;
00546 s_radio.state = RADIO_STATE_STANDBY;
00547 return RADIO_OK;
00548 }
00549
00550 radio_status_t radio_fsk_deinit(void)
00551 {
00552     if (!s_radio.initialized)
00553     {
00554         return RADIO_ESTATE;
00555     }
00556
00557     (void)radio_fsk_sleep();
00558     memset(&s_radio, 0, sizeof(s_radio));
00559     s_radio.state = RADIO_STATE_UNINIT;
00560     return RADIO_OK;
00561 }
00562
00563 radio_status_t radio_fsk_start_rx_continuous(void)
00564 {
00565     if (!s_radio.initialized)
00566     {
00567         return RADIO_ESTATE;
00568     }
00569
00570     if (s_radio.state == RADIO_STATE_TX)
00571     {
00572         return RADIO_EBUS;
00573     }
00574
00575     if (!sx1276_write_reg(&s_radio.bus, SX1276_REG_DIO_MAPPING_1, 0x00U) ||
00576         !sx1276_set_op_mode(&s_radio.bus,
00577             (uint8_t) (SX1276_OPMODE_MODULATION_FSK |
00578                 radio_get_shaping_bits(&s_radio.cfg) |
00579                 SX1276_MODE_RXCONTINUOUS)))
00580     {
00581         return RADIO_EHW;
00582     }
00583
00584     radio_set_state(RADIO_STATE_RX_CONT);
00585     return RADIO_OK;
00586 }
00587

```

```

00588 radio_status_t radio_fsk_start_rx_single(uint16_t symbol_timeout)
00589 {
00590     if (!s_radio.initialized)
00591     {
00592         return RADIO_ESTATE;
00593     }
00594     if (s_radio.state == RADIO_STATE_TX)
00595     {
00596         return RADIO_EBUS;
00597     }
00598     if (symbol_timeout == 0U)
00599     {
00600         return RADIO_EINVAL;
00601     }
00602     if (!sx1276_write_reg(&s_radio.bus, SX1276_REG_DIO_MAPPING_1, 0x00U) ||
00603         !sx1276_set_op_mode(&s_radio.bus,
00604             (uint8_t)(SX1276_OPMODE_MODULATION_FSK |
00605                 radio_get_shaping_bits(&s_radio.cfg) |
00606                 SX1276_MODE_RXSINGLE)))
00607     {
00608         return RADIO_EHW;
00609     }
00610     s_radio.rx_single_deadline_ms = HAL_GetTick() + radio_rx_single_timeout_ms(symbol_timeout);
00611     radio_set_state(RADIO_STATE_RX_SINGLE);
00612     return RADIO_OK;
00613 }
00614
00615 radio_status_t radio_fsk_send_async(const uint8_t *data, uint8_t len)
00616 {
00617     uint8_t fifo_buf[RADIO_LIB_MAX_PAYLOAD + 1U];
00618     if (!s_radio.initialized)
00619     {
00620         return RADIO_ESTATE;
00621     }
00622     if ((data == NULL) || (len == 0U) || (len > RADIO_LIB_MAX_PAYLOAD))
00623     {
00624         return RADIO_EINVAL;
00625     }
00626     if (((uint16_t)len + 1U) > RADIO_FSK_FIFO_MAX_BYTES)
00627     {
00628         /* Chunked FIFO TX is not implemented in this backend yet. */
00629         return RADIO_EINVAL;
00630     }
00631     if (s_radio.state == RADIO_STATE_TX)
00632     {
00633         return RADIO_EBUS;
00634     }
00635     fifo_buf[0] = len;
00636     memcpy(&fifo_buf[1], data, len);
00637     s_radio.tx_resume_state = s_radio.state;
00638     if (!radio_set_op_mode_ready((uint8_t)(SX1276_OPMODE_MODULATION_FSK |
00639         radio_get_shaping_bits(&s_radio.cfg) |
00640         SX1276_MODE_STDBY) ||
00641         !sx1276_write_reg(&s_radio.bus, SX1276_REG_DIO_MAPPING_1, 0x00U) ||
00642         !sx1276_write_burst(&s_radio.bus, SX1276_REG_FIFO, fifo_buf, (uint8_t)(len + 1U)) ||
00643         !sx1276_set_op_mode(&s_radio.bus,
00644             (uint8_t)(SX1276_OPMODE_MODULATION_FSK |
00645                 radio_get_shaping_bits(&s_radio.cfg) |
00646                 SX1276_MODE_TX)))
00647     {
00648         return RADIO_EHW;
00649     }
00650     uint32_t key = radio_irq_save();
00651     s_radio.dio_pending_mask = 0U;
00652     radio_irq_restore(key);
00653     radio_set_state(RADIO_STATE_TX);
00654     s_radio.tx_deadline_ms = HAL_GetTick() + radio_tx_timeout_ms(len);
00655     return RADIO_OK;
00656 }
00657
00658 radio_status_t radio_fsk_standby(void)
00659 {
00660     if (!s_radio.initialized)
00661     {
00662         return RADIO_ESTATE;
00663     }

```

```

00675
00676     if (!radio_set_op_mode_ready((uint8_t) (SX1276_OPMODE_MODULATION_FSK |
00677                                     radio_get_shaping_bits(&s_radio.cfg) |
00678                                     SX1276_MODE_STDBY)))
00679     {
00680         return RADIO_EHW;
00681     }
00682
00683     radio_set_state(RADIO_STATE_STANDBY);
00684     return RADIO_OK;
00685 }
00686
00687 radio_status_t radio_fsk_sleep(void)
00688 {
00689     if (!s_radio.initialized)
00690     {
00691         return RADIO_ESTATE;
00692     }
00693
00694     if (!radio_set_op_mode_ready((uint8_t) (SX1276_OPMODE_MODULATION_FSK |
00695                                     radio_get_shaping_bits(&s_radio.cfg) |
00696                                     SX1276_MODE_SLEEP)))
00697     {
00698         return RADIO_EHW;
00699     }
00700
00701     radio_set_state(RADIO_STATE_STANDBY);
00702     return RADIO_OK;
00703 }
00704
00705 void radio_fsk_process(void)
00706 {
00707     uint8_t pending;
00708     uint8_t irq1 = 0U;
00709     uint8_t irq2 = 0U;
00710     uint32_t events = RADIO_EVENT_NONE;
00711     uint32_t key;
00712
00713     if (!s_radio.initialized)
00714     {
00715         return;
00716     }
00717
00718     key = radio_irq_save();
00719     pending = s_radio.dio_pending_mask;
00720     s_radio.dio_pending_mask = 0U;
00721     radio_irq_restore(key);
00722
00723     if ((pending == 0U) &&
00724         (s_radio.hw.dio[0].port != NULL) &&
00725         (HAL_GPIO_ReadPin(s_radio.hw.dio[0].port, s_radio.hw.dio[0].pin) == GPIO_PIN_SET))
00726     {
00727         pending |= 0x01U;
00728     }
00729
00730     if ((pending == 0U) && (s_radio.state != RADIO_STATE_TX))
00731     {
00732         if ((s_radio.state == RADIO_STATE_RX_SINGLE) &&
00733             ((int32_t) (HAL_GetTick() - s_radio.rx_single_deadline_ms) >= 0))
00734         {
00735             events |= RADIO_EVENT_RX_TIMEOUT;
00736             (void)radio_fsk_standby();
00737         }
00738         else
00739         {
00740             return;
00741         }
00742     }
00743
00744     if ((pending == 0U) &&
00745         (s_radio.state == RADIO_STATE_TX) &&
00746         ((int32_t) (HAL_GetTick() - s_radio.tx_deadline_ms) >= 0))
00747     {
00748         (void)radio_fsk_standby();
00749         radio_resume_after_tx();
00750         events |= RADIO_EVENT_HW_ERROR;
00751         goto done;
00752     }
00753
00754     if (!sx1276_read_reg(&s_radio.bus, SX1276_REG_IRQ_FLAGS_1, &irq1) ||
00755         !sx1276_read_reg(&s_radio.bus, SX1276_REG_IRQ_FLAGS_2, &irq2))
00756     {
00757         events |= RADIO_EVENT_HW_ERROR;
00758         goto done;
00759     }
00760
00761     if ((irq2 & SX1276_IRQ2_FIFO_OVERRUN) != 0U)

```

```

00762     {
00763         events |= RADIO_EVENT_FIFO_OVERRUN;
00764     }
00765
00766     if ((irq1 & SX1276_IRQ1_TIMEOUT) != 0U)
00767     {
00768         events |= RADIO_EVENT_RX_TIMEOUT;
00769         if (s_radio.state == RADIO_STATE_RX_SINGLE)
00770         {
00771             (void)radio_fsk_standby();
00772         }
00773     }
00774
00775     if ((irq2 & SX1276_IRQ2_PAYLOAD_READY) != 0U)
00776     {
00777         if ((s_radio.cfg.crc_type != RADIO_PACKET_CRC_OFF) &&
00778             ((irq2 & SX1276_IRQ2_CRC_OK) == 0U))
00779         {
00780             events |= RADIO_EVENT_CRC_ERR;
00781         }
00782         else
00783         {
00784             radio_status_t status = radio_read_rx_packet();
00785             if (status == RADIO_OK)
00786             {
00787                 events |= RADIO_EVENT_RX_DONE;
00788             }
00789             else if (status == RADIO_EBUS)
00790             {
00791                 events |= RADIO_EVENT_FIFO_OVERRUN;
00792             }
00793             else
00794             {
00795                 events |= RADIO_EVENT_HW_ERROR;
00796             }
00797         }
00798
00799         if (s_radio.state == RADIO_STATE_RX_SINGLE)
00800         {
00801             (void)radio_fsk_standby();
00802         }
00803     }
00804
00805     if ((irq2 & SX1276_IRQ2_PACKET_SENT) != 0U)
00806     {
00807         events |= RADIO_EVENT_TX_DONE;
00808         radio_resume_after_tx();
00809         s_radio.tx_deadline_ms = 0U;
00810     }
00811
00812 done:
00813     if (events != RADIO_EVENT_NONE)
00814     {
00815         radio_set_event_flags(events);
00816         if (s_radio.callback != NULL)
00817         {
00818             s_radio.callback(events, s_radio.callback_ctx);
00819         }
00820     }
00821 }
00822
00823 uint32_t radio_fsk_take_events(void)
00824 {
00825     uint32_t events;
00826     uint32_t key = radio_irq_save();
00827
00828     events = s_radio.event_flags;
00829     s_radio.event_flags = 0U;
00830     radio_irq_restore(key);
00831     return events;
00832 }
00833
00834 bool radio_fsk_get_last_packet(radio_packet_t *pkt)
00835 {
00836     if ((pkt == NULL) || !s_radio.last_packet.valid)
00837     {
00838         return false;
00839     }
00840
00841     *pkt = s_radio.last_packet;
00842     return true;
00843 }
00844
00845 radio_state_t radio_fsk_get_state(void)
00846 {
00847     return s_radio.state;
00848 }

```



```

00849
00850 radio_status_t radio_fsk_raw_read_reg(uint8_t reg, uint8_t *value)
00851 {
00852     if (!s_radio.initialized)
00853     {
00854         return RADIO_ESTATE;
00855     }
00856     return sx1276_read_reg(&s_radio.bus, reg, value) ? RADIO_OK : RADIO_EHW;
00857 }
00858
00859
00860 radio_status_t radio_fsk_raw_write_reg(uint8_t reg, uint8_t value)
00861 {
00862     if (!s_radio.initialized)
00863     {
00864         return RADIO_ESTATE;
00865     }
00866     return sx1276_write_reg(&s_radio.bus, reg, value) ? RADIO_OK : RADIO_EHW;
00867 }
00868
00869
00870 radio_status_t radio_fsk_raw_read_burst(uint8_t reg, uint8_t *data, uint8_t len)
00871 {
00872     if (!s_radio.initialized)
00873     {
00874         return RADIO_ESTATE;
00875     }
00876     if ((data == NULL) || (len == 0U))
00877     {
00878         return RADIO_EINVAL;
00879     }
00880     return sx1276_read_burst(&s_radio.bus, reg, data, len) ? RADIO_OK : RADIO_EHW;
00881 }
00882
00883
00884
00885 radio_status_t radio_fsk_raw_write_burst(uint8_t reg, const uint8_t *data, uint8_t len)
00886 {
00887     if (!s_radio.initialized)
00888     {
00889         return RADIO_ESTATE;
00890     }
00891     if ((data == NULL) || (len == 0U))
00892     {
00893         return RADIO_EINVAL;
00894     }
00895     return sx1276_write_burst(&s_radio.bus, reg, data, len) ? RADIO_OK : RADIO_EHW;
00896 }
00897
00898
00899
00900 void radio_fsk_on_exti(uint16_t gpio_pin)
00901 {
00902     radio_handle_exti_pin(gpio_pin);
00903 }

```

12.56 Dokumentacja pliku Core/App/radio_lib/modulations/fsk/radio_fsk.h

Interfejs backendu modulacji FSK dla SX1276.

```
#include "../radio_lib.h"
```

Funkcje

- void `radio_fsk_default_hw_cfg` (`radio_hw_cfg_t` *cfg, `SPI_HandleTypeDef` *hspi)
Domyślna konfiguracja sprzętowa backendu FSK.
- void `radio_fsk_default_lora_cfg` (`radio_lora_cfg_t` *cfg)
Domyślna konfiguracja parametrów backendu FSK.
- `radio_status_t` `radio_fsk_set_runtime_cfg` (const `radio_fsk_cfg_t` *cfg)
Ustawia runtime konfigurację właściwą dla backendu FSK.
- `radio_status_t` `radio_fsk_init` (const `radio_hw_cfg_t` *hw, const `radio_lora_cfg_t` *cfg, `radio_event_cb_t` cb, void *user_ctx)
Inicjalizacja backendu FSK.

- [radio_status_t radio_fsk_deinit](#) (void)
Deinicjalizacja backendu FSK.
- [radio_status_t radio_fsk_start_rx_continuous](#) (void)
Uruchomienie odbioru ciągłego FSK.
- [radio_status_t radio_fsk_start_rx_single](#) (uint16_t symbol_timeout)
Uruchomienie odbioru pojedynczego FSK.
- [radio_status_t radio_fsk_send_async](#) (const uint8_t *data, uint8_t len)
Uruchomienie nadawania asynchronicznego FSK.
- [radio_status_t radio_fsk_standby](#) (void)
Przejsie do standby backendu FSK.
- [radio_status_t radio_fsk_sleep](#) (void)
Przejsie do sleep backendu FSK.
- void [radio_fsk_process](#) (void)
Przetwarzanie logiki odroczonej backendu FSK.
- uint32_t [radio_fsk_take_events](#) (void)
Zwrócenie i wyczyszczenie flag zdarzeń backendu FSK.
- bool [radio_fsk_get_last_packet](#) ([radio_packet_t](#) *pkt)
Odczyt ostatniego pakietu backendu FSK.
- [radio_state_t radio_fsk_get_state](#) (void)
Odczyt stanu backendu FSK.
- [radio_status_t radio_fsk_raw_read_reg](#) (uint8_t reg, uint8_t *value)
Odczyt pojedynczego rejestru (backend FSK).
- [radio_status_t radio_fsk_raw_write_reg](#) (uint8_t reg, uint8_t value)
Zapis pojedynczego rejestru (backend FSK).
- [radio_status_t radio_fsk_raw_read_burst](#) (uint8_t reg, uint8_t *data, uint8_t len)
Odczyt sekwencji rejestrów (backend FSK).
- [radio_status_t radio_fsk_raw_write_burst](#) (uint8_t reg, const uint8_t *data, uint8_t len)
Zapis sekwencji rejestrów (backend FSK).
- void [radio_fsk_on_exti](#) (uint16_t gpio_pin)
Przekazanie pinu EXTI do backendu FSK.

12.56.1 Opis szczegółowy

Interfejs backendu modulacji FSK dla SX1276.
Definicja w pliku [radio_fsk.h](#).

12.56.2 Dokumentacja funkcji

12.56.2.1 radio_fsk_default_hw_cfg()

```
void radio_fsk_default_hw_cfg (
    radio_hw_cfg_t * cfg,
    SPI_HandleTypeDef * hspi)
```

Domyślna konfiguracja sprzętowa backendu FSK.
Definicja w linii 467 pliku [radio_fsk.c](#).
Odwołuje się do [radio_lora_default_hw_cfg\(\)](#).

12.56.2.2 radio_fsk_default_lora_cfg()

```
void radio_fsk_default_lora_cfg (
    radio_lora_cfg_t * cfg)
```

Domyślna konfiguracja parametrów backendu FSK.
Definicja w linii 472 pliku [radio_fsk.c](#).
Odwołuje się do [radio_lora_default_lora_cfg\(\)](#).

12.56.2.3 radio_fsk_deinit()

```
radio_status_t radio_fsk_deinit (
    void )
```

Deinicjalizacja backendu FSK.

Definicja w linii 550 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_ESTATE](#), [radio_fsk_sleep\(\)](#), [RADIO_OK](#) i [RADIO_STATE_UNINIT](#).

Odwołania w [radio_deinit\(\)](#).

12.56.2.4 radio_fsk_get_last_packet()

```
bool radio_fsk_get_last_packet (
    radio_packet_t * pkt)
```

Odczyt ostatniego pakietu backendu FSK.

Definicja w linii 834 pliku [radio_fsk.c](#).

Odwołania w [radio_get_last_packet\(\)](#).

12.56.2.5 radio_fsk_get_state()

```
radio_state_t radio_fsk_get_state (
    void )
```

Odczyt stanu backendu FSK.

Definicja w linii 845 pliku [radio_fsk.c](#).

Odwołania w [radio_get_state\(\)](#).

12.56.2.6 radio_fsk_init()

```
radio_status_t radio_fsk_init (
    const radio_hw_cfg_t * hw,
    const radio_lora_cfg_t * cfg,
    radio_event_cb_t cb,
    void * user_ctx)
```

Inicjalizacja backendu FSK.

Definicja w linii 489 pliku [radio_fsk.c](#).

Odwołuje się do [radio_hw_cfg_t::hspi](#), [radio_hw_cfg_t::nss](#), [radio_gpio_t::pin](#), [radio_gpio_t::port](#), [radio_apply_fsk_config\(\)](#), [radio_cfg_valid\(\)](#), [radio_default_fsk_cfg\(\)](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [radio_hw_reset\(\)](#), [RADIO_OK](#), [RADIO_STATE_STANDBY](#), [RADIO_STATE_UNINIT](#), [radio_hw_cfg_t::reset](#), [radio_hw_cfg_t::spi_timeout_ms](#) i [sx1276_get_version\(\)](#).

Odwołania w [radio_init\(\)](#).

12.56.2.7 radio_fsk_on_exti()

```
void radio_fsk_on_exti (
    uint16_t gpio_pin)
```

Przekazanie pinu EXTI do backendu FSK.

Definicja w linii 900 pliku [radio_fsk.c](#).

Odwołuje się do [radio_handle_exti_pin\(\)](#).

Odwołania w [radio_on_exti\(\)](#).

12.56.2.8 radio_fsk_process()

```
void radio_fsk_process (
    void )
```

Przetwarzanie logiki odroczonej backendu FSK.

Definicja w linii 705 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EVENT_CRC_ERR](#), [RADIO_EVENT_FIFO_OVERRUN](#), [RADIO_EVENT_HW_ERROR](#), [RADIO_EVENT_NONE](#), [RADIO_EVENT_RX_DONE](#), [RADIO_EVENT_RX_TIMEOUT](#), [RADIO_EVENT_TX_DONE](#), [radio_fsk_standby\(\)](#), [radio_irq_restore\(\)](#), [radio_irq_save\(\)](#), [RADIO_OK](#), [radio_read_rx_packet\(\)](#), [radio_resume_after_tx\(\)](#), [radio_set_event_flags\(\)](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_TX](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_process\(\)](#).

12.56.2.9 radio_fsk_raw_read_burst()

```
radio_status_t radio_fsk_raw_read_burst (
    uint8_t reg,
    uint8_t * data,
    uint8_t len)
```

Odczyt sekwencji rejestrów (backend FSK).

Definicja w linii 870 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_read_burst\(\)](#).

Odwołania w [radio_raw_read_burst\(\)](#).

12.56.2.10 radio_fsk_raw_read_reg()

```
radio_status_t radio_fsk_raw_read_reg (
    uint8_t reg,
    uint8_t * value)
```

Odczyt pojedynczego rejestru (backend FSK).

Definicja w linii 850 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_raw_read_reg\(\)](#).

12.56.2.11 radio_fsk_raw_write_burst()

```
radio_status_t radio_fsk_raw_write_burst (
    uint8_t reg,
    const uint8_t * data,
    uint8_t len)
```

Zapis sekwencji rejestrów (backend FSK).

Definicja w linii 885 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_write_burst\(\)](#).

Odwołania w [radio_raw_write_burst\(\)](#).

12.56.2.12 radio_fsk_raw_write_reg()

```
radio_status_t radio_fsk_raw_write_reg (
    uint8_t reg,
    uint8_t value)
```

Zapis pojedynczego rejestru (backend FSK).

Definicja w linii 860 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_raw_write_reg\(\)](#).

12.56.2.13 radio_fsk_send_async()

```
radio_status_t radio_fsk_send_async (
    const uint8_t * data,
    uint8_t len)
```

Uruchomienie nadawania asynchronicznego FSK.

Definicja w linii 618 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [radio_get_shaping_bits\(\)](#), [radio_irq_restore\(\)](#), [radio_irq_save\(\)](#), [RADIO_LIB_MAX_PAYLOAD](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_TX](#), [radio_tx_timeout_ms\(\)](#), [sx1276_set_op_mode\(\)](#), [sx1276_write_burst\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_send_async\(\)](#).

12.56.2.14 radio_fsk_set_runtime_cfg()

```
radio_status_t radio_fsk_set_runtime_cfg (
    const radio_fsk_cfg_t * cfg)
```

Ustawia runtime konfigurację właściwą dla backendu FSK.

Definicja w linii 477 pliku [radio_fsk.c](#).

Odwołuje się do [radio_cfg_valid\(\)](#), [RADIO_EINVAL](#) i [RADIO_OK](#).

Odwołania w [radio_set_fsk_cfg\(\)](#).

12.56.2.15 radio_fsk_sleep()

```
radio_status_t radio_fsk_sleep (
    void )
```

Przejsie do sleep backendu FSK.

Definicja w linii 687 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [radio_get_shaping_bits\(\)](#), [RADIO_OK](#), [radio_set_state\(\)](#) i [RADIO_STATE_STANDBY](#).

Odwołania w [radio_fsk_deinit\(\)](#) i [radio_sleep\(\)](#).

12.56.2.16 radio_fsk_standby()

```
radio_status_t radio_fsk_standby (
    void )
```

Przejsie do standby backendu FSK.

Definicja w linii 669 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [radio_get_shaping_bits\(\)](#), [RADIO_OK](#), [radio_set_state\(\)](#) i [RADIO_STATE_STANDBY](#).

Odwołania w [radio_fsk_process\(\)](#) i [radio_standby\(\)](#).

12.56.2.17 radio_fsk_start_rx_continuous()

```
radio_status_t radio_fsk_start_rx_continuous (
    void )
```

Uruchomienie odbioru ciągłego FSK.

Definicja w linii 563 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_ESTATE](#), [radio_get_shaping_bits\(\)](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_CONT](#), [RADIO_STATE_TX](#), [sx1276_set_op_mode\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_start_rx_continuous\(\)](#).

12.56.2.18 radio_fsk_start_rx_single()

```
radio_status_t radio_fsk_start_rx_single (
    uint16_t symbol_timeout)
```

Uruchomienie odbioru pojedynczego FSK.

Definicja w linii 588 pliku [radio_fsk.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [radio_get_shaping_bits\(\)](#), [RADIO_OK](#), [radio_rx_single_timeout_ms\(\)](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_TX](#), [sx1276_set_op_mode\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_start_rx_single\(\)](#).

12.56.2.19 radio_fsk_take_events()

```
uint32_t radio_fsk_take_events (
    void )
```

Zwrócenie i wyczyszczenie flag zdarzeń backendu FSK.

Definicja w linii 823 pliku [radio_fsk.c](#).

Odwołuje się do [radio_irq_restore\(\)](#) i [radio_irq_save\(\)](#).

Odwołania w [radio_take_events\(\)](#).

12.57 radio_fsk.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
```

```

00006 #ifndef APP_RADIO_LIB_MODULATIONS_FSK_RADIO_FSK_H_
00007 #define APP_RADIO_LIB_MODULATIONS_FSK_RADIO_FSK_H_
00008
00009 #include "../radio_lib.h"
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00016 void radio_fsk_default_hw_cfg(radio\_hw\_cfg\_t *cfg, SPI\_HandleTypeDef *hspi);
00018 void radio_fsk_default_lora_cfg(radio\_lora\_cfg\_t *cfg);
00020 radio\_status\_t radio_fsk_set_runtime_cfg(const radio\_fsk\_cfg\_t *cfg);
00022 radio\_status\_t radio_fsk_init(const radio\_hw\_cfg\_t *hw,
00023                               const radio\_lora\_cfg\_t *cfg,
00024                               radio\_event\_cb\_t cb,
00025                               void *user_ctx);
00027 radio\_status\_t radio_fsk_deinit(void);
00029 radio\_status\_t radio_fsk_start_rx_continuous(void);
00031 radio\_status\_t radio_fsk_start_rx_single(uint16\_t symbol_timeout);
00033 radio\_status\_t radio_fsk_send_async(const uint8\_t *data, uint8\_t len);
00035 radio\_status\_t radio_fsk_standby(void);
00037 radio\_status\_t radio_fsk_sleep(void);
00039 void radio_fsk_process(void);
00041 uint32\_t radio_fsk_take_events(void);
00043 bool radio_fsk_get_last_packet(radio\_packet\_t *pkt);
00045 radio\_state\_t radio_fsk_get_state(void);
00047 radio\_status\_t radio_fsk_raw_read_reg(uint8\_t reg, uint8\_t *value);
00049 radio\_status\_t radio_fsk_raw_write_reg(uint8\_t reg, uint8\_t value);
00051 radio\_status\_t radio_fsk_raw_read_burst(uint8\_t reg, uint8\_t *data, uint8\_t len);
00053 radio\_status\_t radio_fsk_raw_write_burst(uint8\_t reg, const uint8\_t *data, uint8\_t len);
00055 void radio_fsk_on_exti(uint16\_t gpio_pin);
00056
00057 #ifdef __cplusplus
00058 }
00059 #endif
00060
00061 #endif /* APP_RADIO_LIB_MODULATIONS_FSK_RADIO_FSK_H_ */

```

12.58 Dokumentacja pliku

Core/App/radio_lib/modulations/lora/radio_lora.c

Implementacja backendu LoRa (SX1276/RFM95).

```

#include "radio_lora.h"
#include "app_delay.h"
#include "main.h"
#include "../common/sx1276/radio_sx1276.h"
#include "../common/sx1276/radio_sx1276_regs.h"
#include <string.h>

```

Struktury danych

- [struct radio_context_t](#)

Definicje

- [#define RADIO_SX1276_FIFO_TX_BASE_ADDR](#) 0x00U
- [#define RADIO_SX1276_FIFO_RX_BASE_ADDR](#) 0x00U
- [#define RADIO_DEFAULT_RX_TIMEOUT_SYMBOLS](#) 0x03FFU
- [#define RADIO_RESET_LOW_DELAY_MS](#) 2U
- [#define RADIO_RESET_HIGH_DELAY_MS](#) 10U

Funkcje

- [static uint32_t radio_irq_save](#) ([void](#))
Internal helper: [radio_irq_save](#).
- [static void radio_irq_restore](#) ([uint32_t](#) primask)
Internal helper: [radio_irq_restore](#).
- [static void radio_set_event_flags](#) ([uint32_t](#) events)

- Internal helper: *radio_set_event_flags*.
 - static bool `radio_read_irq_flags_retry` (uint8_t *irq_flags)
- Internal helper: *radio_read_irq_flags_retry*.
 - static bool `radio_cfg_valid` (const `radio_lora_cfg_t` *cfg)
- Internal helper: *radio_cfg_valid*.
 - static void `radio_hw_reset` (void)
- Internal helper: *radio_hw_reset*.
 - static bool `radio_low_data_rate_optimize_required` (const `radio_lora_cfg_t` *cfg)
- Internal helper: *radio_low_data_rate_optimize_required*.
 - static bool `radio_apply_lora_config` (const `radio_lora_cfg_t` *cfg)
- Internal helper: *radio_apply_lora_config*.
 - static bool `radio_config_dio_for_rx` (void)
- Internal helper: *radio_config_dio_for_rx*.
 - static bool `radio_config_dio_for_tx` (void)
- Internal helper: *radio_config_dio_for_tx*.
 - static void `radio_set_state` (`radio_state_t` state)
- Internal helper: *radio_set_state*.
 - static bool `radio_wait_for_tx_enter_or_done` (uint8_t *op_mode_snapshot)
- Internal helper: *radio_wait_for_tx_enter_or_done*.
 - static void `radio_handle_exti_pin` (uint16_t pin)
- Internal helper: *radio_handle_exti_pin*.
 - static `radio_status_t` `radio_read_rx_packet` (void)
- Internal helper: *radio_read_rx_packet*.
 - static void `radio_resume_after_tx` (void)
- Internal helper: *radio_resume_after_tx*.
 - void `radio_lora_default_hw_cfg` (`radio_hw_cfg_t` *cfg, SPI_HandleTypeDef *hspi)
- Uzupełnia domyślną konfigurację sprzętową LoRa.
 - void `radio_lora_default_lora_cfg` (`radio_lora_cfg_t` *cfg)
- Uzupełnia domyślną konfigurację parametrów LoRa.
 - `radio_status_t` `radio_lora_init` (const `radio_hw_cfg_t` *hw, const `radio_lora_cfg_t` *cfg, `radio_event_cb_t` cb, void *user_ctx)
- Inicjalizuje backend LoRa.
 - `radio_status_t` `radio_lora_deinit` (void)
- Deinicjalizuje backend LoRa.
 - `radio_status_t` `radio_lora_start_rx_continuous` (void)
- Uruchamia odbiór ciągły LoRa.
 - `radio_status_t` `radio_lora_start_rx_single` (uint16_t symbol_timeout)
- Uruchamia odbiór pojedynczy LoRa.
 - `radio_status_t` `radio_lora_send_async` (const uint8_t *data, uint8_t len)
- Rozpoczyna asynchroniczne nadawanie LoRa.
 - `radio_status_t` `radio_lora_standby` (void)
- Przełącza radio LoRa do standby.
 - `radio_status_t` `radio_lora_sleep` (void)
- Przełącza radio LoRa do sleep.
 - void `radio_lora_process` (void)
- Obsługuje logikę odroczonej po IRQ (wywoływać w pętli głównej).
 - uint32_t `radio_lora_take_events` (void)
- Zwraca i czyści maskę zdarzeń LoRa.
 - bool `radio_lora_get_last_packet` (`radio_packet_t` *pkt)
- Odczytuje ostatnio odebrany pakiet LoRa.
 - `radio_state_t` `radio_lora_get_state` (void)

- *Zwraca stan backendu LoRa.*
• `radio_status_t radio_lora_raw_read_reg` (`uint8_t reg`, `uint8_t *value`)
Odczyt pojedynczego rejestru transceivera.
- `radio_status_t radio_lora_raw_write_reg` (`uint8_t reg`, `uint8_t value`)
Zapis pojedynczego rejestru transceivera.
- `radio_status_t radio_lora_raw_read_burst` (`uint8_t reg`, `uint8_t *data`, `uint8_t len`)
Odczyt sekwencji rejestrów.
- `radio_status_t radio_lora_raw_write_burst` (`uint8_t reg`, `const uint8_t *data`, `uint8_t len`)
Zapis sekwencji rejestrów.
- `void radio_lora_on_exti` (`uint16_t gpio_pin`)
Przekazuje pin EXTI do backendu LoRa.

Zmienne

- `static radio_context_t s_radio`

12.58.1 Opis szczegółowy

Implementacja backendu LoRa (SX1276/RFM95).

Definicja w pliku `radio_lora.c`.

12.58.2 Dokumentacja definicji

12.58.2.1 RADIO_DEFAULT_RX_TIMEOUT_SYMBOLS

```
#define RADIO_DEFAULT_RX_TIMEOUT_SYMBOLS 0x03FFU
```

Definicja w linii 17 pliku `radio_lora.c`.

12.58.2.2 RADIO_RESET_HIGH_DELAY_MS

```
#define RADIO_RESET_HIGH_DELAY_MS 10U
```

Definicja w linii 19 pliku `radio_lora.c`.

12.58.2.3 RADIO_RESET_LOW_DELAY_MS

```
#define RADIO_RESET_LOW_DELAY_MS 2U
```

Definicja w linii 18 pliku `radio_lora.c`.

12.58.2.4 RADIO_SX1276_FIFO_RX_BASE_ADDR

```
#define RADIO_SX1276_FIFO_RX_BASE_ADDR 0x00U
```

Definicja w linii 16 pliku `radio_lora.c`.

12.58.2.5 RADIO_SX1276_FIFO_TX_BASE_ADDR

```
#define RADIO_SX1276_FIFO_TX_BASE_ADDR 0x00U
```

Definicja w linii 15 pliku `radio_lora.c`.

12.58.3 Dokumentacja funkcji

12.58.3.1 radio_apply_lora_config()

```
bool radio_apply_lora_config (
    const radio_lora_cfg_t * cfg) [static]
```

Internal helper: `radio_apply_lora_config`.

Definicja w linii 127 pliku `radio_lora.c`.

Odwołuje się do `radio_lora_cfg_t::bandwidth`, `radio_lora_cfg_t::coding_rate`, `radio_lora_cfg_t::crc_on`, `radio_lora_cfg_t::frequency_hz`, `radio_lora_cfg_t::implicit_header`, `radio_lora_cfg_t::invert_iq`, `radio_lora_cfg_t::payload_len`, `radio_lora_cfg_t::preamble_len`,

[radio_low_data_rate_optimize_required\(\)](#), [radio_lora_cfg_t::spreading_factor](#), [sx1276_set_fifo_base_addrs\(\)](#), [sx1276_set_frequency\(\)](#), [sx1276_set_invert_iq\(\)](#), [sx1276_set_lora_modem\(\)](#), [sx1276_set_pa_output_power\(\)](#), [sx1276_set_payload_length\(\)](#), [sx1276_set_preamble\(\)](#), [sx1276_set_symbol_timeout\(\)](#), [sx1276_set_sync_word\(\)](#), [sx1276_write_reg\(\)](#), [radio_lora_cfg_t::sync_word](#) i [radio_lora_cfg_t::tx_power_dbm](#).

Odwołania w [radio_lora_init\(\)](#).

12.58.3.2 radio_cfg_valid()

```
bool radio_cfg_valid (
    const radio_lora_cfg_t * cfg) [static]
```

Internal helper: [radio_cfg_valid](#).

Definicja w linii 79 pliku [radio_lora.c](#).

Odwołuje się do [radio_lora_cfg_t::bandwidth](#), [radio_lora_cfg_t::coding_rate](#), [radio_lora_cfg_t::frequency_hz](#), [radio_lora_cfg_t::implicit_header](#), [radio_lora_cfg_t::payload_len](#), [radio_lora_cfg_t::preamble_len](#) i [radio_lora_cfg_t::spreading_factor](#).

12.58.3.3 radio_config_dio_for_rx()

```
bool radio_config_dio_for_rx (
    void ) [static]
```

Internal helper: [radio_config_dio_for_rx](#).

Definicja w linii 155 pliku [radio_lora.c](#).

Odwołuje się do [sx1276_map_dio\(\)](#).

Odwołania w [radio_lora_start_rx_continuous\(\)](#) i [radio_lora_start_rx_single\(\)](#).

12.58.3.4 radio_config_dio_for_tx()

```
bool radio_config_dio_for_tx (
    void ) [static]
```

Internal helper: [radio_config_dio_for_tx](#).

Definicja w linii 164 pliku [radio_lora.c](#).

Odwołuje się do [sx1276_map_dio\(\)](#).

Odwołania w [radio_lora_send_async\(\)](#).

12.58.3.5 radio_handle_exti_pin()

```
void radio_handle_exti_pin (
    uint16_t pin) [static]
```

Internal helper: [radio_handle_exti_pin](#).

Definicja w linii 216 pliku [radio_lora.c](#).

Odwołuje się do [radio_irq_restore\(\)](#) i [radio_irq_save\(\)](#).

Odwołania w [radio_lora_on_exti\(\)](#).

12.58.3.6 radio_hw_reset()

```
void radio_hw_reset (
    void ) [static]
```

Internal helper: [radio_hw_reset](#).

Definicja w linii 105 pliku [radio_lora.c](#).

Odwołania w [radio_lora_init\(\)](#).

12.58.3.7 radio_irq_restore()

```
void radio_irq_restore (
    uint32_t primask) [static]
```

Internal helper: [radio_irq_restore](#).

Definicja w linii 48 pliku [radio_lora.c](#).

Odwołania w [radio_handle_exti_pin\(\)](#), [radio_lora_process\(\)](#), [radio_lora_take_events\(\)](#) i [radio_set_event_flags\(\)](#).

12.58.3.8 radio_irq_save()

```
uint32_t radio_irq_save (
    void ) [static]
```

Internal helper: [radio_irq_save](#).

Definicja w linii 40 pliku [radio_lora.c](#).

Odwołania w [radio_handle_exti_pin\(\)](#), [radio_lora_process\(\)](#), [radio_lora_take_events\(\)](#) i [radio_set_event_flags\(\)](#).

12.58.3.9 radio_lora_default_hw_cfg()

```
void radio_lora_default_hw_cfg (
    radio_hw_cfg_t * cfg,
    SPI_HandleTypeDef * hspi)
```

Uzupełnia domyślną konfigurację sprzętową LoRa.

Parametry

<i>cfg</i>	[out] Konfiguracja sprzętowa.
<i>hspi</i>	Uchwyt SPI.

Definicja w linii 317 pliku [radio_lora.c](#).

Odwołuje się do [radio_hw_cfg_t::dio](#), [radio_hw_cfg_t::hspi](#), [radio_hw_cfg_t::nss](#), [radio_gpio_t::pin](#), [radio_gpio_t::port](#), [RADIO_LIB_SPI_TIMEOUT_MS](#), [radio_hw_cfg_t::reset](#) i [radio_hw_cfg_t::spi_timeout_ms](#).

Odwołania w [radio_default_hw_cfg\(\)](#), [radio_fsk_default_hw_cfg\(\)](#) i [radio_ook_default_hw_cfg\(\)](#).

12.58.3.10 radio_lora_default_lora_cfg()

```
void radio_lora_default_lora_cfg (
    radio_lora_cfg_t * cfg)
```

Uzupełnia domyślną konfigurację parametrów LoRa.

Parametry

<i>cfg</i>	[out] Konfiguracja LoRa.
------------	--------------------------

Definicja w linii 347 pliku [radio_lora.c](#).

Odwołuje się do [radio_lora_cfg_t::bandwidth](#), [radio_lora_cfg_t::coding_rate](#), [radio_lora_cfg_t::crc_on](#), [radio_lora_cfg_t::frequency_hz](#), [radio_lora_cfg_t::implicit_header](#), [radio_lora_cfg_t::invert_iq](#), [radio_lora_cfg_t::payload_len](#), [radio_lora_cfg_t::preamble_len](#), [RADIO_LIB_DEFAULT_FREQ_HZ](#), [RADIO_LIB_DEFAULT_SYNC_WORD](#), [radio_lora_cfg_t::spreading_factor](#), [radio_lora_cfg_t::sync_word](#) i [radio_lora_cfg_t::tx_power_dbm](#).

Odwołania w [radio_default_lora_cfg\(\)](#), [radio_fsk_default_lora_cfg\(\)](#) i [radio_ook_default_lora_cfg\(\)](#).

12.58.3.11 radio_lora_deinit()

```
radio_status_t radio_lora_deinit (
    void )
```

Deinicjalizuje backend LoRa.

Zwraca

Kod statusu.

Definicja w linii 439 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_ESTATE](#), [radio_lora_sleep\(\)](#), [RADIO_OK](#) i [RADIO_STATE_UNINIT](#).

Odwołania w [radio_deinit\(\)](#).

12.58.3.12 radio_lora_get_last_packet()

```
bool radio_lora_get_last_packet (
    radio_packet_t * pkt)
```

Odczytuje ostatnio odebrany pakiet LoRa.

Parametry

<i>pkt</i>	[out] Struktura pakietu.
------------	--------------------------

Zwraca

`true` jeśli pakiet jest poprawny.

Definicja w linii 724 pliku [radio_lora.c](#).

Odwołania w [radio_get_last_packet\(\)](#).

12.58.3.13 radio_lora_get_state()

```
radio_state_t radio_lora_get_state (
    void )
```

Zwraca stan backendu LoRa.

Zwraca

Stan pracy.

Definicja w linii 735 pliku [radio_lora.c](#).

Odwołania w [radio_get_state\(\)](#).

12.58.3.14 radio_lora_init()

```
radio_status_t radio_lora_init (
    const radio_hw_cfg_t * hw,
    const radio_lora_cfg_t * cfg,
    radio_event_cb_t cb,
    void * user_ctx)
```

Inicjalizuje backend LoRa.

Parametry

<i>hw</i>	Konfiguracja sprzętowa.
<i>cfg</i>	Konfiguracja LoRa.
<i>cb</i>	Callback zdarzeń (opcjonalny).
<i>user_ctx</i>	Kontekst użytkownika callbacku.

Zwraca

Kod statusu.

Definicja w linii 368 pliku [radio_lora.c](#).

Odwołuje się do [radio_hw_cfg_t::hspi](#), [radio_hw_cfg_t::nss](#), [radio_gpio_t::pin](#), [radio_gpio_t::port](#), [radio_apply_lora_config\(\)](#), [radio_cfg_valid\(\)](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [radio_hw_reset\(\)](#), [RADIO_OK](#), [RADIO_STATE_STANDBY](#), [RADIO_STATE_UNINIT](#), [radio_hw_cfg_t::reset](#), [radio_hw_cfg_t::spi_timeout_ms](#), [sx1276_clear_irq_flags\(\)](#), [sx1276_get_version\(\)](#), [sx1276_set_fifo_addr_ptr\(\)](#) i [sx1276_set_op_mode\(\)](#).

Odwołania w [radio_init\(\)](#).

12.58.3.15 radio_lora_on_exti()

```
void radio_lora_on_exti (
    uint16_t gpio_pin)
```

Przekazuje pin EXTI do backendu LoRa.

Parametry

<i>gpio_pin</i>	Pin zgłoszony przez HAL.
-----------------	--------------------------

Definicja w linii 790 pliku [radio_lora.c](#).

Odwołuje się do [radio_handle_exti_pin\(\)](#).

Odwołania w [radio_on_exti\(\)](#).

12.58.3.16 radio_lora_process()

```
void radio_lora_process (
    void )
```

Obsługuje logikę odroczonej po IRQ (wywoływać w pętli głównej).

Definicja w linii 580 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EVENT_CAD_DETECTED](#), [RADIO_EVENT_CAD_DONE](#), [RADIO_EVENT_CRC_ERR](#), [RADIO_EVENT_FIFO_OVERRUN](#), [RADIO_EVENT_HW_ERROR](#), [RADIO_EVENT_NONE](#), [RADIO_EVENT_RX_DONE](#), [RADIO_EVENT_RX_TIMEOUT](#), [RADIO_EVENT_TX_DONE](#), [radio_irq_restore\(\)](#), [radio_irq_save\(\)](#), [radio_lora_standby\(\)](#), [RADIO_OK](#), [radio_read_irq_flags_retry\(\)](#), [radio_read_rx_packet\(\)](#), [radio_resume_after_tx\(\)](#), [radio_set_event_flags\(\)](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_TX](#), [sx1276_clear_irq_flags\(\)](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_process\(\)](#).

12.58.3.17 radio_lora_raw_read_burst()

```
radio_status_t radio_lora_raw_read_burst (
    uint8_t reg,
    uint8_t * data,
    uint8_t len)
```

Odczyt sekwencji rejestrów.

Parametry

<i>reg</i>	Adres startowy.
<i>data</i>	[out] Bufor danych.
<i>len</i>	Długość transferu.

Zwraca

Kod statusu.

Definicja w linii 760 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_read_burst\(\)](#).

Odwołania w [radio_raw_read_burst\(\)](#).

12.58.3.18 radio_lora_raw_read_reg()

```
radio_status_t radio_lora_raw_read_reg (
    uint8_t reg,
    uint8_t * value)
```

Odczyt pojedynczego rejestru transceivera.

Parametry

<i>reg</i>	Adres rejestru.
<i>value</i>	[out] Odczytana wartość.

Zwraca

Kod statusu.

Definicja w linii 740 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_raw_read_reg\(\)](#).

12.58.3.19 radio_lora_raw_write_burst()

```
radio_status_t radio_lora_raw_write_burst (  
    uint8_t reg,  
    const uint8_t * data,  
    uint8_t len)
```

Zapis sekwencji rejestrów.

Parametry

<i>reg</i>	Adres startowy.
<i>data</i>	Bufor danych.
<i>len</i>	Długość transferu.

Zwraca

Kod statusu.

Definicja w linii 775 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_write_burst\(\)](#).

Odwołania w [radio_raw_write_burst\(\)](#).

12.58.3.20 radio_lora_raw_write_reg()

```
radio_status_t radio_lora_raw_write_reg (  
    uint8_t reg,  
    uint8_t value)
```

Zapis pojedynczego rejestru transceivera.

Parametry

<i>reg</i>	Adres rejestru.
<i>value</i>	Wartość do zapisu.

Zwraca

Kod statusu.

Definicja w linii 750 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_raw_write_reg\(\)](#).

12.58.3.21 radio_lora_send_async()

```
radio_status_t radio_lora_send_async (
    const uint8_t * data,
    uint8_t len)
```

Rozpoczyna asynchroniczne nadawanie LoRa.

Parametry

<i>data</i>	Payload.
<i>len</i>	Długość payloadu.

Zwraca

Kod statusu.

Definicja w linii 507 pliku [radio_lora.c](#).

Odwołuje się do [radio_config_dio_for_tx\(\)](#), [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_LIB_MAX_PAYLOAD](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_TX](#), [radio_wait_for_tx_enter_or_done\(\)](#), [sx1276_clear_irq_flags\(\)](#), [sx1276_set_fifo_addr_ptr\(\)](#), [sx1276_set_op_mode\(\)](#), [sx1276_set_payload_length\(\)](#) i [sx1276_write_fifo\(\)](#).

Odwołania w [radio_send_async\(\)](#).

12.58.3.22 radio_lora_sleep()

```
radio_status_t radio_lora_sleep (
    void )
```

Przełącza radio LoRa do sleep.

Zwraca

Kod statusu.

Definicja w linii 564 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_STANDBY](#) i [sx1276_set_op_mode\(\)](#).

Odwołania w [radio_lora_deinit\(\)](#) i [radio_sleep\(\)](#).

12.58.3.23 radio_lora_standby()

```
radio_status_t radio_lora_standby (
    void )
```

Przełącza radio LoRa do standby.

Zwraca

Kod statusu.

Definicja w linii 548 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_STANDBY](#) i [sx1276_set_op_mode\(\)](#).

Odwołania w [radio_lora_process\(\)](#), [radio_resume_after_tx\(\)](#) i [radio_standby\(\)](#).

12.58.3.24 radio_lora_start_rx_continuous()

```
radio_status_t radio_lora_start_rx_continuous (
    void )
```

Uruchamia odbiór ciągły LoRa.

Zwraca

Kod statusu.

Definicja w linii 452 pliku [radio_lora.c](#).

Odwołuje się do [radio_config_dio_for_rx\(\)](#), [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_CONT](#), [RADIO_STATE_TX](#), [sx1276_clear_irq_flags\(\)](#), [sx1276_set_op_mode\(\)](#) i [sx1276_set_symbol_timeout\(\)](#).

Odwołania w [radio_resume_after_tx\(\)](#) i [radio_start_rx_continuous\(\)](#).

12.58.3.25 radio_lora_start_rx_single()

```
radio_status_t radio_lora_start_rx_single (
    uint16_t symbol_timeout)
```

Uruchamia odbiór pojedynczy LoRa.

Parametry

<i>symbol_timeout</i>	Timeout w symbolach.
-----------------------	----------------------

Zwraca

Kod statusu.

Definicja w linii 476 pliku [radio_lora.c](#).

Odwołuje się do [radio_config_dio_for_rx\(\)](#), [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_TX](#), [sx1276_clear_irq_flags\(\)](#), [sx1276_set_op_mode\(\)](#) i [sx1276_set_symbol_timeout\(\)](#).

Odwołania w [radio_resume_after_tx\(\)](#) i [radio_start_rx_single\(\)](#).

12.58.3.26 radio_lora_take_events()

```
uint32_t radio_lora_take_events (
    void )
```

Zwraca i czyści maskę zdarzeń LoRa.

Zwraca

Maska zdarzeń.

Definicja w linii 711 pliku [radio_lora.c](#).

Odwołuje się do [radio_irq_restore\(\)](#) i [radio_irq_save\(\)](#).

Odwołania w [radio_take_events\(\)](#).

12.58.3.27 radio_low_data_rate_optimize_required()

```
bool radio_low_data_rate_optimize_required (
    const radio_lora_cfg_t * cfg) [static]
```

Internal helper: [radio_low_data_rate_optimize_required](#).

Definicja w linii 115 pliku [radio_lora.c](#).

Odwołuje się do [radio_lora_cfg_t::bandwidth](#) i [radio_lora_cfg_t::spreading_factor](#).

Odwołania w [radio_apply_lora_config\(\)](#).

12.58.3.28 radio_read_irq_flags_retry()

```
bool radio_read_irq_flags_retry (
    uint8_t * irq_flags) [static]
```

Internal helper: [radio_read_irq_flags_retry](#).

Definicja w linii 67 pliku [radio_lora.c](#).

Odwołuje się do [sx1276_get_irq_flags\(\)](#).

Odwołania w [radio_lora_process\(\)](#).

12.58.3.29 radio_read_rx_packet()

```
radio_status_t radio_read_rx_packet (  
    void ) [static]
```

Internal helper: [radio_read_rx_packet](#).

Definicja w linii 255 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_LIB_MAX_PAYLOAD](#), [RADIO_OK](#), [sx1276_get_packet_rssi_raw\(\)](#), [sx1276_get_packet_snr_raw\(\)](#), [sx1276_get_rx_current_addr\(\)](#), [sx1276_get_rx_nb_bytes\(\)](#), [sx1276_read_fifo\(\)](#) i [sx1276_set_fifo_addr_ptr\(\)](#).

Odwołania w [radio_lora_process\(\)](#).

12.58.3.30 radio_resume_after_tx()

```
void radio_resume_after_tx (  
    void ) [static]
```

Internal helper: [radio_resume_after_tx](#).

Definicja w linii 298 pliku [radio_lora.c](#).

Odwołuje się do [radio_lora_standby\(\)](#), [radio_lora_start_rx_continuous\(\)](#), [radio_lora_start_rx_single\(\)](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_CONT](#), [RADIO_STATE_RX_SINGLE](#) i [RADIO_STATE_STANDBY](#).

Odwołania w [radio_lora_process\(\)](#).

12.58.3.31 radio_set_event_flags()

```
void radio_set_event_flags (  
    uint32_t events) [static]
```

Internal helper: [radio_set_event_flags](#).

Definicja w linii 57 pliku [radio_lora.c](#).

Odwołuje się do [radio_irq_restore\(\)](#) i [radio_irq_save\(\)](#).

Odwołania w [radio_lora_process\(\)](#).

12.58.3.32 radio_set_state()

```
void radio_set_state (  
    radio_state_t state) [static]
```

Internal helper: [radio_set_state](#).

Definicja w linii 173 pliku [radio_lora.c](#).

Odwołania w [radio_lora_send_async\(\)](#), [radio_lora_sleep\(\)](#), [radio_lora_standby\(\)](#), [radio_lora_start_rx_continuous\(\)](#), [radio_lora_start_rx_single\(\)](#) i [radio_resume_after_tx\(\)](#).

12.58.3.33 radio_wait_for_tx_enter_or_done()

```
bool radio_wait_for_tx_enter_or_done (  
    uint8_t * op_mode_snapshot) [static]
```

Internal helper: [radio_wait_for_tx_enter_or_done](#).

Definicja w linii 179 pliku [radio_lora.c](#).

Odwołuje się do [sx1276_get_irq_flags\(\)](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_lora_send_async\(\)](#).

12.58.4 Dokumentacja zmiennych

12.58.4.1 s_radio

```
radio_context_t s_radio [static]
```

Definicja w linii 37 pliku [radio_lora.c](#).

12.59 radio_lora.c

[Idź do dokumentacji tego pliku.](#)

00001


```

00005
00006 #include "radio_lora.h"
00007
00008 #include "app_delay.h"
00009 #include "main.h"
00010 #include "../common/sx1276/radio_sx1276.h"
00011 #include "../common/sx1276/radio_sx1276_regs.h"
00012
00013 #include <string.h>
00014
00015 #define RADIO_SX1276_FIFO_TX_BASE_ADDR 0x00U
00016 #define RADIO_SX1276_FIFO_RX_BASE_ADDR 0x00U
00017 #define RADIO_DEFAULT_RX_TIMEOUT_SYMBOLS 0x03FFU
00018 #define RADIO_RESET_LOW_DELAY_MS 2U
00019 #define RADIO_RESET_HIGH_DELAY_MS 10U
00020
00021 typedef struct
00022 {
00023     bool initialized;
00024     sx1276_bus_t bus;
00025     radio_hw_cfg_t hw;
00026     radio_lora_cfg_t lora;
00027     radio_event_cb_t callback;
00028     void *callback_ctx;
00029     volatile uint8_t dio_pending_mask;
00030     volatile uint32_t event_flags;
00031     radio_state_t state;
00032     radio_state_t tx_resume_state;
00033     uint16_t rx_single_timeout_symbols;
00034     radio_packet_t last_packet;
00035 } radio_context_t;
00036
00037 static radio_context_t s_radio;
00038
00040 static uint32_t radio_irq_save(void)
00041 {
00042     uint32_t primask = __get_PRIMASK();
00043     __disable_irq();
00044     return primask;
00045 }
00046
00048 static void radio_irq_restore(uint32_t primask)
00049 {
00050     if (primask == 0U)
00051     {
00052         __enable_irq();
00053     }
00054 }
00055
00057 static void radio_set_event_flags(uint32_t events)
00058 {
00059     uint32_t key;
00060
00061     key = radio_irq_save();
00062     s_radio.event_flags |= events;
00063     radio_irq_restore(key);
00064 }
00065
00067 static bool radio_read_irq_flags_retry(uint8_t *irq_flags)
00068 {
00069     if (sx1276_get_irq_flags(&s_radio.bus, irq_flags))
00070     {
00071         return true;
00072     }
00073
00074     /* Retry once to handle occasional SPI transient errors. */
00075     return sx1276_get_irq_flags(&s_radio.bus, irq_flags);
00076 }
00077
00079 static bool radio_cfg_valid(const radio_lora_cfg_t *cfg)
00080 {
00081     if ((cfg == NULL) ||
00082         (cfg->frequency_hz < 137000000UL) || (cfg->frequency_hz > 1020000000UL) ||
00083         ((uint8_t)cfg->bandwidth > (uint8_t)RADIO_LORA_BW_500_KHZ) ||
00084         (cfg->spreading_factor < 6U) || (cfg->spreading_factor > 12U) ||
00085         (cfg->coding_rate < 5U) || (cfg->coding_rate > 8U) ||
00086         (cfg->preamble_len == 0U))
00087     {
00088         return false;
00089     }
00090
00091     if (cfg->implicit_header && (cfg->payload_len == 0U))
00092     {
00093         return false;
00094     }
00095
00096     if ((cfg->spreading_factor == 6U) && !cfg->implicit_header)

```

```

00097     {
00098         return false;
00099     }
00100
00101     return true;
00102 }
00103
00105 static void radio_hw_reset(void)
00106 {
00107     HAL_GPIO_WritePin(s_radio.hw.nss.port, s_radio.hw.nss.pin, GPIO_PIN_SET);
00108     HAL_GPIO_WritePin(s_radio.hw.reset.port, s_radio.hw.reset.pin, GPIO_PIN_RESET);
00109     app_delay_ms(RADIO_RESET_LOW_DELAY_MS);
00110     HAL_GPIO_WritePin(s_radio.hw.reset.port, s_radio.hw.reset.pin, GPIO_PIN_SET);
00111     app_delay_ms(RADIO_RESET_HIGH_DELAY_MS);
00112 }
00113
00115 static bool radio_low_data_rate_optimize_required(const radio_lora_cfg_t *cfg)
00116 {
00117     if ((cfg->bandwidth <= RADIO_LORA_BW_125_KHZ) &&
00118         (cfg->spreading_factor >= 11U))
00119     {
00120         return true;
00121     }
00122
00123     return false;
00124 }
00125
00127 static bool radio_apply_lora_config(const radio_lora_cfg_t *cfg)
00128 {
00129     bool ldo_required;
00130
00131     ldo_required = radio_low_data_rate_optimize_required(cfg);
00132
00133     return sx1276_set_frequency(&s_radio.bus, cfg->frequency_hz) &&
00134         sx1276_set_pa_output_power(&s_radio.bus, cfg->tx_power_dbm) &&
00135         sx1276_set_lora_modem(&s_radio.bus,
00136             (uint8_t)cfg->bandwidth,
00137             cfg->spreading_factor,
00138             cfg->coding_rate,
00139             cfg->implicit_header,
00140             cfg->crc_on,
00141             ldo_required) &&
00142         sx1276_set_symbol_timeout(&s_radio.bus, RADIO_DEFAULT_RX_TIMEOUT_SYMBOLS) &&
00143         sx1276_set_preamble(&s_radio.bus, cfg->preamble_len) &&
00144         sx1276_set_sync_word(&s_radio.bus, cfg->sync_word) &&
00145         sx1276_set_invert_iq(&s_radio.bus, cfg->invert_iq) &&
00146         sx1276_set_fifo_base_addrs(&s_radio.bus,
00147             RADIO_SX1276_FIFO_TX_BASE_ADDR,
00148             RADIO_SX1276_FIFO_RX_BASE_ADDR) &&
00149         sx1276_set_payload_length(&s_radio.bus, cfg->payload_len) &&
00150         sx1276_write_reg(&s_radio.bus, SX1276_REG_LNA, 0x23U) &&
00151         sx1276_write_reg(&s_radio.bus, SX1276_REG_IRQ_FLAGS_MASK, 0x00U);
00152 }
00153
00155 static bool radio_config_dio_for_rx(void)
00156 {
00157     const uint8_t dio_mapping1 = (uint8_t)(SX1276_DIO0_MAP_RX_DONE |
00158         SX1276_DIO1_MAP_RX_TIMEOUT);
00159
00160     return sx1276_map_dio(&s_radio.bus, dio_mapping1, 0x00U);
00161 }
00162
00164 static bool radio_config_dio_for_tx(void)
00165 {
00166     const uint8_t dio_mapping1 = (uint8_t)(SX1276_DIO0_MAP_TX_DONE |
00167         SX1276_DIO1_MAP_RX_TIMEOUT);
00168
00169     return sx1276_map_dio(&s_radio.bus, dio_mapping1, 0x00U);
00170 }
00171
00173 static void radio_set_state(radio_state_t state)
00174 {
00175     s_radio.state = state;
00176 }
00177
00179 static bool radio_wait_for_tx_enter_or_done(uint8_t *op_mode_snapshot)
00180 {
00181     uint8_t op_mode;
00182     uint8_t irq_flags;
00183     uint8_t retry;
00184
00185     for (retry = 0U; retry < 5U; retry++)
00186     {
00187         if (sx1276_read_reg(&s_radio.bus, SX1276_REG_OP_MODE, &op_mode))
00188         {
00189             if (op_mode_snapshot != NULL)
00190             {

```

```

00191         *op_mode_snapshot = op_mode;
00192     }
00193
00194     op_mode &= SX1276_OPMODE_MODE_MASK;
00195     if ((op_mode == SX1276_MODE_TX) || (op_mode == SX1276_MODE_FSTX))
00196     {
00197         return true;
00198     }
00199 }
00200
00201 if (sx1276_get_irq_flags(&s_radio.bus, &irq_flags))
00202 {
00203     if ((irq_flags & SX1276_IRQ_TX_DONE) != 0U)
00204     {
00205         return true;
00206     }
00207 }
00208
00209 app_delay_ms(1U);
00210 }
00211
00212 return false;
00213 }
00214
00216 static void radio_handle_exti_pin(uint16_t pin)
00217 {
00218     uint8_t i;
00219
00220     if (!s_radio.initialized)
00221     {
00222         return;
00223     }
00224
00225     for (i = 0U; i < 6U; i++)
00226     {
00227         if ((i == 2U) && (RADIO_LIB_ENABLE_DIO2_EXTI == 0))
00228         {
00229             continue;
00230         }
00231         if ((i == 3U) && (RADIO_LIB_ENABLE_DIO3_EXTI == 0))
00232         {
00233             continue;
00234         }
00235         if ((i == 4U) && (RADIO_LIB_ENABLE_DIO4_EXTI == 0))
00236         {
00237             continue;
00238         }
00239         if ((i == 5U) && (RADIO_LIB_ENABLE_DIO5_EXTI == 0))
00240         {
00241             continue;
00242         }
00243
00244         if (s_radio.hw.dio[i].pin == pin)
00245         {
00246             uint32_t key = radio_irq_save();
00247             s_radio.dio_pending_mask |= (uint8_t)(1U << i);
00248             radio_irq_restore(key);
00249             break;
00250         }
00251     }
00252 }
00253
00255 static radio_status_t radio_read_rx_packet(void)
00256 {
00257     uint8_t current_addr;
00258     uint8_t rx_len;
00259     int8_t snr_raw;
00260     uint8_t rssi_raw;
00261     bool rf_ok;
00262
00263     rf_ok = sx1276_get_rx_current_addr(&s_radio.bus, &current_addr) &&
00264             sx1276_get_rx_nb_bytes(&s_radio.bus, &rx_len) &&
00265             sx1276_set_fifo_addr_ptr(&s_radio.bus, current_addr);
00266     if (!rf_ok)
00267     {
00268         return RADIO_EHW;
00269     }
00270
00271     if (rx_len > RADIO_LIB_MAX_PAYLOAD)
00272     {
00273         s_radio.last_packet.valid = false;
00274         return RADIO_EBUS;
00275     }
00276
00277     if (!sx1276_read_fifo(&s_radio.bus, s_radio.last_packet.data, rx_len))
00278     {
00279         return RADIO_EHW;

```

```

00280     }
00281
00282     if (!sx1276_get_packet_snr_raw(&s_radio.bus, &snr_raw) ||
00283         !sx1276_get_packet_rssi_raw(&s_radio.bus, &rssi_raw))
00284     {
00285         return RADIO_EHW;
00286     }
00287
00288     s_radio.last_packet.length = rx_len;
00289     s_radio.last_packet.snr_db = (int8_t)(snr_raw / 4);
00290     s_radio.last_packet.rssi_dbm = (int16_t)((int16_t)rssi_raw - 157);
00291     s_radio.last_packet.timestamp_ms = HAL_GetTick();
00292     s_radio.last_packet.valid = true;
00293
00294     return RADIO_OK;
00295 }
00296
00297 static void radio_resume_after_tx(void)
00298 {
00299     /* Po wejściu tutaj TX jest już zakończony. */
00300     radio_set_state(RADIO_STATE_STANDBY);
00301
00302     if (s_radio.tx_resume_state == RADIO_STATE_RX_CONT)
00303     {
00304         (void)radio_lora_start_rx_continuous();
00305     }
00306     else if (s_radio.tx_resume_state == RADIO_STATE_RX_SINGLE)
00307     {
00308         (void)radio_lora_start_rx_single(s_radio.rx_single_timeout_symbols);
00309     }
00310     else
00311     {
00312         (void)radio_lora_standby();
00313     }
00314 }
00315
00316 void radio_lora_default_hw_cfg(radio_hw_cfg_t *cfg, SPI_HandleTypeDef *hspi)
00317 {
00318     if (cfg == NULL)
00319     {
00320         return;
00321     }
00322
00323     memset(cfg, 0, sizeof(*cfg));
00324
00325     cfg->hspi = hspi;
00326     cfg->nss.port = SPI_CS_GPIO_Port;
00327     cfg->nss.pin = SPI_CS_Pin;
00328     cfg->reset.port = RMF_RST_GPIO_Port;
00329     cfg->reset.pin = RMF_RST_Pin;
00330
00331     cfg->dio[0].port = RFM_DIO0_EXIT_2_GPIO_Port;
00332     cfg->dio[0].pin = RFM_DIO0_EXIT_2_Pin;
00333     cfg->dio[1].port = RFM_DIO1_EXIT_1_GPIO_Port;
00334     cfg->dio[1].pin = RFM_DIO1_EXIT_1_Pin;
00335     cfg->dio[2].port = RFM_DIO2_EXIT_15_GPIO_Port;
00336     cfg->dio[2].pin = RFM_DIO2_EXIT_15_Pin;
00337     cfg->dio[3].port = RFM_DIO3_EXIT_14_GPIO_Port;
00338     cfg->dio[3].pin = RFM_DIO3_EXIT_14_Pin;
00339     cfg->dio[4].port = RFM_DIO4_EXIT_13_GPIO_Port;
00340     cfg->dio[4].pin = RFM_DIO4_EXIT_13_Pin;
00341     cfg->dio[5].port = RFM_DIO5_EXIT_8_GPIO_Port;
00342     cfg->dio[5].pin = RFM_DIO5_EXIT_8_Pin;
00343     cfg->spi_timeout_ms = RADIO_LIB_SPI_TIMEOUT_MS;
00344 }
00345
00346 void radio_lora_default_lora_cfg(radio_lora_cfg_t *cfg)
00347 {
00348     if (cfg == NULL)
00349     {
00350         return;
00351     }
00352
00353     memset(cfg, 0, sizeof(*cfg));
00354     cfg->frequency_hz = RADIO_LIB_DEFAULT_FREQ_HZ;
00355     cfg->bandwidth = RADIO_LORA_BW_125_KHZ;
00356     cfg->spreading_factor = 7U;
00357     cfg->coding_rate = 5U;
00358     cfg->preamble_len = 8U;
00359     cfg->sync_word = RADIO_LIB_DEFAULT_SYNC_WORD;
00360     cfg->crc_on = true;
00361     cfg->invert_iq = false;
00362     cfg->tx_power_dbm = 14;
00363     cfg->implicit_header = false;
00364     cfg->payload_len = 0U;
00365 }
00366
00367

```

```

00368 radio_status_t radio_lora_init(const radio_hw_cfg_t *hw,
00369                                const radio_lora_cfg_t *cfg,
00370                                radio_event_cb_t cb,
00371                                void *user_ctx)
00372 {
00373     uint8_t version;
00374
00375     if ((hw == NULL) || (hw->hspi == NULL) ||
00376         (hw->nss.port == NULL) || (hw->reset.port == NULL) ||
00377         !radio_cfg_valid(cfg))
00378     {
00379         return RADIO_EINVAL;
00380     }
00381
00382     memset(&s_radio, 0, sizeof(s_radio));
00383
00384     s_radio.hw = *hw;
00385     s_radio.lora = *cfg;
00386     s_radio.callback = cb;
00387     s_radio.callback_ctx = user_ctx;
00388     s_radio.bus.hspi = hw->hspi;
00389     s_radio.bus.nss_port = hw->nss.port;
00390     s_radio.bus.nss_pin = hw->nss.pin;
00391     s_radio.bus.spi_timeout_ms = hw->spi_timeout_ms;
00392     s_radio.rx_single_timeout_symbols = RADIO_DEFAULT_RX_TIMEOUT_SYMBOLS;
00393     s_radio.state = RADIO_STATE_UNINIT;
00394
00395     radio_hw_reset();
00396
00397     if (!sx1276_set_op_mode(&s_radio.bus, (uint8_t)(SX1276_OPMODE_LONG_RANGE_MODE |
SX1276_MODE_SLEEP)))
00398     {
00399         return RADIO_EHW;
00400     }
00401
00402     app_delay_ms(1U);
00403
00404     if (!sx1276_get_version(&s_radio.bus, &version) || (version != SX1276_VERSION_ID))
00405     {
00406         return RADIO_EHW;
00407     }
00408
00409     if (!radio_apply_lora_config(&s_radio.lora) ||
00410         !sx1276_set_fifo_addr_ptr(&s_radio.bus, RADIO_SX1276_FIFO_TX_BASE_ADDR) ||
00411         !sx1276_clear_irq_flags(&s_radio.bus, 0xFFU) ||
00412         !sx1276_set_op_mode(&s_radio.bus, (uint8_t)(SX1276_OPMODE_LONG_RANGE_MODE |
SX1276_MODE_STDBY)))
00413     {
00414         return RADIO_EHW;
00415     }
00416
00417 #if RADIO_LIB_ENABLE_DIO5_EXTI == 0
00418     __HAL_GPIO_EXTI_CLEAR_IT(RFM_DIO5_EXIT_8_Pin);
00419     HAL_NVIC_DisableIRQ(RFM_DIO5_EXIT_8_EXTI_IRQn);
00420 #endif
00421 #if RADIO_LIB_ENABLE_DIO4_EXTI == 0
00422     __HAL_GPIO_EXTI_CLEAR_IT(RFM_DIO4_EXIT_13_Pin);
00423     HAL_NVIC_DisableIRQ(RFM_DIO4_EXIT_13_EXTI_IRQn);
00424 #endif
00425 #if RADIO_LIB_ENABLE_DIO3_EXTI == 0
00426     __HAL_GPIO_EXTI_CLEAR_IT(RFM_DIO3_EXIT_14_Pin);
00427     HAL_NVIC_DisableIRQ(RFM_DIO3_EXIT_14_EXTI_IRQn);
00428 #endif
00429 #if RADIO_LIB_ENABLE_DIO2_EXTI == 0
00430     __HAL_GPIO_EXTI_CLEAR_IT(RFM_DIO2_EXIT_15_Pin);
00431     HAL_NVIC_DisableIRQ(RFM_DIO2_EXIT_15_EXTI_IRQn);
00432 #endif
00433
00434     s_radio.initialized = true;
00435     s_radio.state = RADIO_STATE_STANDBY;
00436     return RADIO_OK;
00437 }
00438
00439 radio_status_t radio_lora_deinit(void)
00440 {
00441     if (!s_radio.initialized)
00442     {
00443         return RADIO_ESTATE;
00444     }
00445
00446     (void)radio_lora_sleep();
00447     memset(&s_radio, 0, sizeof(s_radio));
00448     s_radio.state = RADIO_STATE_UNINIT;
00449     return RADIO_OK;
00450 }
00451
00452 radio_status_t radio_lora_start_rx_continuous(void)

```

```

00453 {
00454     if (!s_radio.initialized)
00455     {
00456         return RADIO_ESTATE;
00457     }
00458
00459     if (s_radio.state == RADIO_STATE_TX)
00460     {
00461         return RADIO_EBUS;
00462     }
00463
00464     if (!radio_config_dio_for_rx() ||
00465         !sx1276_set_symbol_timeout(&s_radio.bus, RADIO_DEFAULT_RX_TIMEOUT_SYMBOLS) ||
00466         !sx1276_clear_irq_flags(&s_radio.bus, 0xFFU) ||
00467         !sx1276_set_op_mode(&s_radio.bus, (uint8_t)(SX1276_OPMODE_LONG_RANGE_MODE |
SX1276_MODE_RXCONTINUOUS)))
00468     {
00469         return RADIO_EHW;
00470     }
00471
00472     radio_set_state(RADIO_STATE_RX_CONT);
00473     return RADIO_OK;
00474 }
00475
00476 radio_status_t radio_lora_start_rx_single(uint16_t symbol_timeout)
00477 {
00478     if (!s_radio.initialized)
00479     {
00480         return RADIO_ESTATE;
00481     }
00482
00483     if (s_radio.state == RADIO_STATE_TX)
00484     {
00485         return RADIO_EBUS;
00486     }
00487
00488     if (symbol_timeout == 0U)
00489     {
00490         return RADIO_EINVAL;
00491     }
00492
00493     s_radio.rx_single_timeout_symbols = symbol_timeout;
00494
00495     if (!radio_config_dio_for_rx() ||
00496         !sx1276_set_symbol_timeout(&s_radio.bus, symbol_timeout) ||
00497         !sx1276_clear_irq_flags(&s_radio.bus, 0xFFU) ||
00498         !sx1276_set_op_mode(&s_radio.bus, (uint8_t)(SX1276_OPMODE_LONG_RANGE_MODE |
SX1276_MODE_RXSINGLE)))
00499     {
00500         return RADIO_EHW;
00501     }
00502
00503     radio_set_state(RADIO_STATE_RX_SINGLE);
00504     return RADIO_OK;
00505 }
00506
00507 radio_status_t radio_lora_send_async(const uint8_t *data, uint8_t len)
00508 {
00509     uint8_t op_mode = 0U;
00510
00511     if (!s_radio.initialized)
00512     {
00513         return RADIO_ESTATE;
00514     }
00515
00516     if ((data == NULL) || (len == 0U) || (len > RADIO_LIB_MAX_PAYLOAD))
00517     {
00518         return RADIO_EINVAL;
00519     }
00520
00521     if (s_radio.state == RADIO_STATE_TX)
00522     {
00523         return RADIO_EBUS;
00524     }
00525
00526     s_radio.tx_resume_state = s_radio.state;
00527
00528     if (!sx1276_set_op_mode(&s_radio.bus, (uint8_t)(SX1276_OPMODE_LONG_RANGE_MODE |
SX1276_MODE_STDBY)) ||
00529         !radio_config_dio_for_tx() ||
00530         !sx1276_clear_irq_flags(&s_radio.bus, 0xFFU) ||
00531         !sx1276_set_fifo_addr_ptr(&s_radio.bus, RADIO_SX1276_FIFO_TX_BASE_ADDR) ||
00532         !sx1276_set_payload_length(&s_radio.bus, len) ||
00533         !sx1276_write_fifo(&s_radio.bus, data, len) ||
00534         !sx1276_set_op_mode(&s_radio.bus, (uint8_t)(SX1276_OPMODE_LONG_RANGE_MODE | SX1276_MODE_TX)))
00535     {
00536         return RADIO_EHW;

```

```

00537     }
00538
00539     if (!radio_wait_for_tx_enter_or_done(&op_mode))
00540     {
00541         return RADIO_EHW;
00542     }
00543
00544     radio_set_state(RADIO_STATE_TX);
00545     return RADIO_OK;
00546 }
00547
00548 radio_status_t radio_lora_standby(void)
00549 {
00550     if (!s_radio.initialized)
00551     {
00552         return RADIO_ESTATE;
00553     }
00554
00555     if (!sx1276_set_op_mode(&s_radio.bus, (uint8_t)(SX1276_OPMODE_LONG_RANGE_MODE |
SX1276_MODE_STDBY)))
00556     {
00557         return RADIO_EHW;
00558     }
00559
00560     radio_set_state(RADIO_STATE_STANDBY);
00561     return RADIO_OK;
00562 }
00563
00564 radio_status_t radio_lora_sleep(void)
00565 {
00566     if (!s_radio.initialized)
00567     {
00568         return RADIO_ESTATE;
00569     }
00570
00571     if (!sx1276_set_op_mode(&s_radio.bus, (uint8_t)(SX1276_OPMODE_LONG_RANGE_MODE |
SX1276_MODE_SLEEP)))
00572     {
00573         return RADIO_EHW;
00574     }
00575
00576     radio_set_state(RADIO_STATE_STANDBY);
00577     return RADIO_OK;
00578 }
00579
00580 void radio_lora_process(void)
00581 {
00582     uint8_t pending;
00583     uint8_t irq_flags;
00584     uint8_t op_mode;
00585     uint32_t events = RADIO_EVENT_NONE;
00586     uint32_t key;
00587
00588     if (!s_radio.initialized)
00589     {
00590         return;
00591     }
00592
00593     key = radio_irq_save();
00594     pending = s_radio.dio_pending_mask;
00595     s_radio.dio_pending_mask = 0U;
00596     radio_irq_restore(key);
00597
00598     /*
00599     * Główna ścieżka: zdarzenia z EXTI (pending != 0).
00600     * Fallback: podczas TX sprawdzaj flagi IRQ pollingiem, jeśli zbocze DIO0
00601     * zostało pominięte.
00602     */
00603     if ((pending == 0U) &&
00604         (s_radio.hw.dio[0].port != NULL) &&
00605         (HAL_GPIO_ReadPin(s_radio.hw.dio[0].port, s_radio.hw.dio[0].pin) == GPIO_PIN_SET))
00606     {
00607         pending |= 0x01U;
00608     }
00609
00610     if ((pending == 0U) && (s_radio.state != RADIO_STATE_TX))
00611     {
00612         return;
00613     }
00614
00615     if (!radio_read_irq_flags_retry(&irq_flags))
00616     {
00617         events |= RADIO_EVENT_HW_ERROR;
00618         goto done;
00619     }
00620
00621     if (irq_flags == 0U)

```

```

00622     {
00623         /*
00624          * Ścieżka recovery: jeśli zbocze TX_DONE zostało pominięte, układ może
00625          * być już poza TX, mimo że stan software nadal wskazuje TX.
00626          */
00627         if ((s_radio.state == RADIO_STATE_TX) &&
00628             sx1276_read_reg(&s_radio.bus, SX1276_REG_OP_MODE, &op_mode))
00629         {
00630             op_mode &= SX1276_OPMODE_MODE_MASK;
00631             if ((op_mode != SX1276_MODE_TX) && (op_mode != SX1276_MODE_FSTX))
00632             {
00633                 events |= RADIO_EVENT_TX_DONE;
00634                 radio_resume_after_tx();
00635             }
00636         }
00637         goto done;
00638     }
00639
00640     if (!sx1276_clear_irq_flags(&s_radio.bus, irq_flags))
00641     {
00642         events |= RADIO_EVENT_HW_ERROR;
00643     }
00644
00645     if ((irq_flags & SX1276_IRQ_PAYLOAD_CRC_ERROR) != 0U)
00646     {
00647         events |= RADIO_EVENT_CRC_ERR;
00648     }
00649
00650     if ((irq_flags & SX1276_IRQ_RX_TIMEOUT) != 0U)
00651     {
00652         events |= RADIO_EVENT_RX_TIMEOUT;
00653         if (s_radio.state == RADIO_STATE_RX_SINGLE)
00654         {
00655             (void)radio_lora_standby();
00656         }
00657     }
00658
00659     if ((irq_flags & SX1276_IRQ_RX_DONE) != 0U)
00660     {
00661         if ((irq_flags & SX1276_IRQ_PAYLOAD_CRC_ERROR) == 0U)
00662         {
00663             radio_status_t read_status = radio_read_rx_packet();
00664             if (read_status == RADIO_OK)
00665             {
00666                 events |= RADIO_EVENT_RX_DONE;
00667             }
00668             else if (read_status == RADIO_EBUS)
00669             {
00670                 events |= RADIO_EVENT_FIFO_OVERRUN;
00671             }
00672             else
00673             {
00674                 events |= RADIO_EVENT_HW_ERROR;
00675             }
00676         }
00677
00678         if (s_radio.state == RADIO_STATE_RX_SINGLE)
00679         {
00680             (void)radio_lora_standby();
00681         }
00682     }
00683
00684     if ((irq_flags & SX1276_IRQ_TX_DONE) != 0U)
00685     {
00686         events |= RADIO_EVENT_TX_DONE;
00687         radio_resume_after_tx();
00688     }
00689
00690     if ((irq_flags & SX1276_IRQ_CAD_DONE) != 0U)
00691     {
00692         events |= RADIO_EVENT_CAD_DONE;
00693     }
00694
00695     if ((irq_flags & SX1276_IRQ_CAD_DETECTED) != 0U)
00696     {
00697         events |= RADIO_EVENT_CAD_DETECTED;
00698     }
00699
00700 done:
00701     if (events != RADIO_EVENT_NONE)
00702     {
00703         radio_set_event_flags(events);
00704         if (s_radio.callback != NULL)
00705         {
00706             s_radio.callback(events, s_radio.callback_ctx);
00707         }
00708     }

```



```
00709 }
00710
00711 uint32_t radio_lora_take_events(void)
00712 {
00713     uint32_t events;
00714     uint32_t key;
00715
00716     key = radio_irq_save();
00717     events = s_radio.event_flags;
00718     s_radio.event_flags = 0U;
00719     radio_irq_restore(key);
00720
00721     return events;
00722 }
00723
00724 bool radio_lora_get_last_packet(radio_packet_t *pkt)
00725 {
00726     if ((pkt == NULL) || !s_radio.last_packet.valid)
00727     {
00728         return false;
00729     }
00730
00731     *pkt = s_radio.last_packet;
00732     return true;
00733 }
00734
00735 radio_state_t radio_lora_get_state(void)
00736 {
00737     return s_radio.state;
00738 }
00739
00740 radio_status_t radio_lora_raw_read_reg(uint8_t reg, uint8_t *value)
00741 {
00742     if (!s_radio.initialized)
00743     {
00744         return RADIO_ESTATE;
00745     }
00746
00747     return sx1276_read_reg(&s_radio.bus, reg, value) ? RADIO_OK : RADIO_EHW;
00748 }
00749
00750 radio_status_t radio_lora_raw_write_reg(uint8_t reg, uint8_t value)
00751 {
00752     if (!s_radio.initialized)
00753     {
00754         return RADIO_ESTATE;
00755     }
00756
00757     return sx1276_write_reg(&s_radio.bus, reg, value) ? RADIO_OK : RADIO_EHW;
00758 }
00759
00760 radio_status_t radio_lora_raw_read_burst(uint8_t reg, uint8_t *data, uint8_t len)
00761 {
00762     if (!s_radio.initialized)
00763     {
00764         return RADIO_ESTATE;
00765     }
00766
00767     if ((data == NULL) || (len == 0U))
00768     {
00769         return RADIO_EINVAL;
00770     }
00771
00772     return sx1276_read_burst(&s_radio.bus, reg, data, len) ? RADIO_OK : RADIO_EHW;
00773 }
00774
00775 radio_status_t radio_lora_raw_write_burst(uint8_t reg, const uint8_t *data, uint8_t len)
00776 {
00777     if (!s_radio.initialized)
00778     {
00779         return RADIO_ESTATE;
00780     }
00781
00782     if ((data == NULL) || (len == 0U))
00783     {
00784         return RADIO_EINVAL;
00785     }
00786
00787     return sx1276_write_burst(&s_radio.bus, reg, data, len) ? RADIO_OK : RADIO_EHW;
00788 }
00789
00790 void radio_lora_on_exti(uint16_t gpio_pin)
00791 {
00792     radio_handle_exti_pin(gpio_pin);
00793 }
00794
```

12.60 Dokumentacja pliku

Core/App/radio_lib/modulations/lora/radio_lora.h

Backend modulacji LoRa dla SX1276/RFM95.

```
#include "../radio_lib.h"
```

Funkcje

- void [radio_lora_default_hw_cfg](#) ([radio_hw_cfg_t](#) *cfg, SPI_HandleTypeDef *hspi)
Uzupełnia domyślną konfigurację sprzętową LoRa.
- void [radio_lora_default_lora_cfg](#) ([radio_lora_cfg_t](#) *cfg)
Uzupełnia domyślną konfigurację parametrów LoRa.
- [radio_status_t](#) [radio_lora_init](#) (const [radio_hw_cfg_t](#) *hw, const [radio_lora_cfg_t](#) *cfg, [radio_event_cb_t](#) cb, void *user_ctx)
Inicjalizuje backend LoRa.
- [radio_status_t](#) [radio_lora_deinit](#) (void)
Deinicjalizuje backend LoRa.
- [radio_status_t](#) [radio_lora_start_rx_continuous](#) (void)
Uruchamia odbiór ciągły LoRa.
- [radio_status_t](#) [radio_lora_start_rx_single](#) (uint16_t symbol_timeout)
Uruchamia odbiór pojedynczy LoRa.
- [radio_status_t](#) [radio_lora_send_async](#) (const uint8_t *data, uint8_t len)
Rozpoczyna asynchroniczne nadawanie LoRa.
- [radio_status_t](#) [radio_lora_standby](#) (void)
Przełącza radio LoRa do standby.
- [radio_status_t](#) [radio_lora_sleep](#) (void)
Przełącza radio LoRa do sleep.
- void [radio_lora_process](#) (void)
Obsługuje logikę odroczonej po IRQ (wywoływać w pętli głównej).
- uint32_t [radio_lora_take_events](#) (void)
Zwraca i czyści maskę zdarzeń LoRa.
- bool [radio_lora_get_last_packet](#) ([radio_packet_t](#) *pkt)
Odczytuje ostatnio odebrany pakiet LoRa.
- [radio_state_t](#) [radio_lora_get_state](#) (void)
Zwraca stan backendu LoRa.
- [radio_status_t](#) [radio_lora_raw_read_reg](#) (uint8_t reg, uint8_t *value)
Odczyt pojedynczego rejestru transceivera.
- [radio_status_t](#) [radio_lora_raw_write_reg](#) (uint8_t reg, uint8_t value)
Zapis pojedynczego rejestru transceivera.
- [radio_status_t](#) [radio_lora_raw_read_burst](#) (uint8_t reg, uint8_t *data, uint8_t len)
Odczyt sekwencji rejestrów.
- [radio_status_t](#) [radio_lora_raw_write_burst](#) (uint8_t reg, const uint8_t *data, uint8_t len)
Zapis sekwencji rejestrów.
- void [radio_lora_on_exti](#) (uint16_t gpio_pin)
Przekazuje pin EXTI do backendu LoRa.

12.60.1 Opis szczegółowy

Backend modulacji LoRa dla SX1276/RFM95.

Definicja w pliku [radio_lora.h](#).

12.60.2 Dokumentacja funkcji

12.60.2.1 radio_lora_default_hw_cfg()

```
void radio_lora_default_hw_cfg (
    radio_hw_cfg_t * cfg,
    SPI_HandleTypeDef * hspi)
```

Uzupełnia domyślną konfigurację sprzętową LoRa.

Parametry

<i>cfg</i>	[out] Konfiguracja sprzętowa.
<i>hspi</i>	Uchwyt SPI.

Definicja w linii 317 pliku [radio_lora.c](#).

Odwołuje się do [radio_hw_cfg_t::dio](#), [radio_hw_cfg_t::hspi](#), [radio_hw_cfg_t::nss](#), [radio_gpio_t::pin](#), [radio_gpio_t::port](#), [RADIO_LIB_SPI_TIMEOUT_MS](#), [radio_hw_cfg_t::reset](#) i [radio_hw_cfg_t::spi_timeout_ms](#).

Odwołania w [radio_default_hw_cfg\(\)](#), [radio_fsk_default_hw_cfg\(\)](#) i [radio_ook_default_hw_cfg\(\)](#).

12.60.2.2 radio_lora_default_lora_cfg()

```
void radio_lora_default_lora_cfg (
    radio_lora_cfg_t * cfg)
```

Uzupełnia domyślną konfigurację parametrów LoRa.

Parametry

<i>cfg</i>	[out] Konfiguracja LoRa.
------------	--------------------------

Definicja w linii 347 pliku [radio_lora.c](#).

Odwołuje się do [radio_lora_cfg_t::bandwidth](#), [radio_lora_cfg_t::coding_rate](#), [radio_lora_cfg_t::crc_on](#), [radio_lora_cfg_t::frequency_hz](#), [radio_lora_cfg_t::implicit_header](#), [radio_lora_cfg_t::invert_iq](#), [radio_lora_cfg_t::payload_len](#), [radio_lora_cfg_t::preamble_len](#), [RADIO_LIB_DEFAULT_FREQ_HZ](#), [RADIO_LIB_DEFAULT_SYNC_WORD](#), [radio_lora_cfg_t::spreading_factor](#), [radio_lora_cfg_t::sync_word](#) i [radio_lora_cfg_t::tx_power_dbm](#).

Odwołania w [radio_default_lora_cfg\(\)](#), [radio_fsk_default_lora_cfg\(\)](#) i [radio_ook_default_lora_cfg\(\)](#).

12.60.2.3 radio_lora_deinit()

```
radio_status_t radio_lora_deinit (
    void )
```

Deinicjalizuje backend LoRa.

Zwraca

Kod statusu.

Definicja w linii 439 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_ESTATE](#), [radio_lora_sleep\(\)](#), [RADIO_OK](#) i [RADIO_STATE_UNINIT](#).

Odwołania w [radio_deinit\(\)](#).

12.60.2.4 radio_lora_get_last_packet()

```
bool radio_lora_get_last_packet (
    radio_packet_t * pkt)
```

Odczytuje ostatnio odebrany pakiet LoRa.

Parametry

<i>pkt</i>	[out] Struktura pakietu.
------------	--------------------------

Zwraca

`true` jeśli pakiet jest poprawny.

Definicja w linii 724 pliku [radio_lora.c](#).

Odwołania w [radio_get_last_packet\(\)](#).

12.60.2.5 radio_lora_get_state()

```
radio_state_t radio_lora_get_state (
    void )
```

Zwraca stan backendu LoRa.

Zwraca

Stan pracy.

Definicja w linii 735 pliku [radio_lora.c](#).

Odwołania w [radio_get_state\(\)](#).

12.60.2.6 radio_lora_init()

```
radio_status_t radio_lora_init (
    const radio_hw_cfg_t * hw,
    const radio_lora_cfg_t * cfg,
    radio_event_cb_t cb,
    void * user_ctx)
```

Inicjalizuje backend LoRa.

Parametry

<i>hw</i>	Konfiguracja sprzętowa.
<i>cfg</i>	Konfiguracja LoRa.
<i>cb</i>	Callback zdarzeń (opcjonalny).
<i>user_ctx</i>	Kontekst użytkownika callbacku.

Zwraca

Kod statusu.

Definicja w linii 368 pliku [radio_lora.c](#).

Odwołuje się do [radio_hw_cfg_t::hspi](#), [radio_hw_cfg_t::nss](#), [radio_gpio_t::pin](#), [radio_gpio_t::port](#), [radio_apply_lora_config\(\)](#), [radio_cfg_valid\(\)](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [radio_hw_reset\(\)](#), [RADIO_OK](#), [RADIO_STATE_STANDBY](#), [RADIO_STATE_UNINIT](#), [radio_hw_cfg_t::reset](#), [radio_hw_cfg_t::spi_timeout_ms](#), [sx1276_clear_irq_flags\(\)](#), [sx1276_get_version\(\)](#), [sx1276_set_fifo_addr_ptr\(\)](#) i [sx1276_set_op_mode\(\)](#).

Odwołania w [radio_init\(\)](#).

12.60.2.7 radio_lora_on_exti()

```
void radio_lora_on_exti (
    uint16_t gpio_pin)
```

Przekazuje pin EXTI do backendu LoRa.

Parametry

<i>gpio_pin</i>	Pin zgłoszony przez HAL.
-----------------	--------------------------

Definicja w linii 790 pliku [radio_lora.c](#).

Odwołuje się do [radio_handle_exti_pin\(\)](#).

Odwołania w [radio_on_exti\(\)](#).

12.60.2.8 radio_lora_process()

```
void radio_lora_process (
    void )
```

Obsługuje logikę odroczonej po IRQ (wywoływać w pętli głównej).

Definicja w linii 580 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EVENT_CAD_DETECTED](#), [RADIO_EVENT_CAD_DONE](#), [RADIO_EVENT_CRC_ERR](#), [RADIO_EVENT_FIFO_OVERRUN](#), [RADIO_EVENT_HW_ERROR](#), [RADIO_EVENT_NONE](#), [RADIO_EVENT_RX_DONE](#), [RADIO_EVENT_RX_TIMEOUT](#), [RADIO_EVENT_TX_DONE](#), [radio_irq_restore\(\)](#), [radio_irq_save\(\)](#), [radio_lora_standby\(\)](#), [RADIO_OK](#), [radio_read_irq_flags_retry\(\)](#), [radio_read_rx_packet\(\)](#), [radio_resume_after_tx\(\)](#), [radio_set_event_flags\(\)](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_TX](#), [sx1276_clear_irq_flags\(\)](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_process\(\)](#).

12.60.2.9 radio_lora_raw_read_burst()

```
radio_status_t radio_lora_raw_read_burst (
    uint8_t reg,
    uint8_t * data,
    uint8_t len)
```

Odczyt sekwencji rejestrów.

Parametry

<i>reg</i>	Adres startowy.
<i>data</i>	[out] Bufor danych.
<i>len</i>	Długość transferu.

Zwraca

Kod statusu.

Definicja w linii 760 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_read_burst\(\)](#).

Odwołania w [radio_raw_read_burst\(\)](#).

12.60.2.10 radio_lora_raw_read_reg()

```
radio_status_t radio_lora_raw_read_reg (
    uint8_t reg,
    uint8_t * value)
```

Odczyt pojedynczego rejestru transceivera.

Parametry

<i>reg</i>	Adres rejestru.
<i>value</i>	[out] Odczytana wartość.

Zwraca

Kod statusu.

Definicja w linii 740 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_raw_read_reg\(\)](#).

12.60.2.11 radio_lora_raw_write_burst()

```
radio_status_t radio_lora_raw_write_burst (  
    uint8_t reg,  
    const uint8_t * data,  
    uint8_t len)
```

Zapis sekwencji rejestrów.

Parametry

<i>reg</i>	Adres startowy.
<i>data</i>	Bufor danych.
<i>len</i>	Długość transferu.

Zwraca

Kod statusu.

Definicja w linii 775 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_write_burst\(\)](#).

Odwołania w [radio_raw_write_burst\(\)](#).

12.60.2.12 radio_lora_raw_write_reg()

```
radio_status_t radio_lora_raw_write_reg (  
    uint8_t reg,  
    uint8_t value)
```

Zapis pojedynczego rejestru transceivera.

Parametry

<i>reg</i>	Adres rejestru.
<i>value</i>	Wartość do zapisu.

Zwraca

Kod statusu.

Definicja w linii 750 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_raw_write_reg\(\)](#).

12.60.2.13 radio_lora_send_async()

```
radio_status_t radio_lora_send_async (  
    const uint8_t * data,  
    uint8_t len)
```

Rozpoczyna asynchroniczne nadawanie LoRa.

Parametry

<i>data</i>	Payload.
<i>len</i>	Długość payloadu.

Zwraca

Kod statusu.

Definicja w linii 507 pliku [radio_lora.c](#).

Odwołuje się do [radio_config_dio_for_tx\(\)](#), [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_LIB_MAX_PAYLOAD](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_TX](#), [radio_wait_for_tx_enter_or_done\(\)](#), [sx1276_clear_irq_flags\(\)](#), [sx1276_set_fifo_addr_ptr\(\)](#), [sx1276_set_op_mode\(\)](#), [sx1276_set_payload_length\(\)](#) i [sx1276_write_fifo\(\)](#).

Odwołania w [radio_send_async\(\)](#).

12.60.2.14 radio_lora_sleep()

```
radio_status_t radio_lora_sleep (  
    void )
```

Przełącza radio LoRa do sleep.

Zwraca

Kod statusu.

Definicja w linii 564 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_STANDBY](#) i [sx1276_set_op_mode\(\)](#).

Odwołania w [radio_lora_deinit\(\)](#) i [radio_sleep\(\)](#).

12.60.2.15 radio_lora_standby()

```
radio_status_t radio_lora_standby (  
    void )
```

Przełącza radio LoRa do standby.

Zwraca

Kod statusu.

Definicja w linii 548 pliku [radio_lora.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_STANDBY](#) i [sx1276_set_op_mode\(\)](#).

Odwołania w [radio_lora_process\(\)](#), [radio_resume_after_tx\(\)](#) i [radio_standby\(\)](#).

12.60.2.16 radio_lora_start_rx_continuous()

```
radio_status_t radio_lora_start_rx_continuous (  
    void )
```

Uruchamia odbiór ciągły LoRa.

Zwraca

Kod statusu.

Definicja w linii 452 pliku [radio_lora.c](#).

Odwołuje się do [radio_config_dio_for_rx\(\)](#), [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_CONT](#), [RADIO_STATE_TX](#), [sx1276_clear_irq_flags\(\)](#), [sx1276_set_op_mode\(\)](#) i [sx1276_set_symbol_timeout\(\)](#).

Odwołania w [radio_resume_after_tx\(\)](#) i [radio_start_rx_continuous\(\)](#).

12.60.2.17 radio_lora_start_rx_single()

```
radio_status_t radio_lora_start_rx_single (
    uint16_t symbol_timeout)
```

Uruchamia odbiór pojedynczy LoRa.

Parametry

<i>symbol_timeout</i>	Timeout w symbolach.
-----------------------	----------------------

Zwraca

Kod statusu.

Definicja w linii 476 pliku [radio_lora.c](#).

Odwołuje się do [radio_config_dio_for_rx\(\)](#), [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_TX](#), [sx1276_clear_irq_flags\(\)](#), [sx1276_set_op_mode\(\)](#) i [sx1276_set_symbol_timeout\(\)](#).

Odwołania w [radio_resume_after_tx\(\)](#) i [radio_start_rx_single\(\)](#).

12.60.2.18 radio_lora_take_events()

```
uint32_t radio_lora_take_events (
    void )
```

Zwraca i czyści maskę zdarzeń LoRa.

Zwraca

Maska zdarzeń.

Definicja w linii 711 pliku [radio_lora.c](#).

Odwołuje się do [radio_irq_restore\(\)](#) i [radio_irq_save\(\)](#).

Odwołania w [radio_take_events\(\)](#).

12.61 radio_lora.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_RADIO_LIB_MODULATIONS_LORA_RADIO_LORA_H_
00007 #define APP_RADIO_LIB_MODULATIONS_LORA_RADIO_LORA_H_
00008
00009 #include "../radio_lib.h"
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00020 void radio_lora_default_hw_cfg(radio_hw_cfg_t *cfg, SPI_HandleTypeDef *hspi);
00021
00026 void radio_lora_default_lora_cfg(radio_lora_cfg_t *cfg);
00027
00036 radio_status_t radio_lora_init(const radio_hw_cfg_t *hw,
00037                               const radio_lora_cfg_t *cfg,
00038                               radio_event_cb_t cb,
00039                               void *user_ctx);
00040
00045 radio_status_t radio_lora_deinit(void);
00046
00051 radio_status_t radio_lora_start_rx_continuous(void);
00052
00058 radio_status_t radio_lora_start_rx_single(uint16_t symbol_timeout);
00059
00066 radio_status_t radio_lora_send_async(const uint8_t *data, uint8_t len);
00067
00072 radio_status_t radio_lora_standby(void);
00073
00078 radio_status_t radio_lora_sleep(void);
00079
```



```

00083 void radio_lora_process(void);
00084
00089 uint32_t radio_lora_take_events(void);
00090
00096 bool radio_lora_get_last_packet(radio_packet_t *pkt);
00097
00102 radio_state_t radio_lora_get_state(void);
00103
00110 radio_status_t radio_lora_raw_read_reg(uint8_t reg, uint8_t *value);
00111
00118 radio_status_t radio_lora_raw_write_reg(uint8_t reg, uint8_t value);
00119
00127 radio_status_t radio_lora_raw_read_burst(uint8_t reg, uint8_t *data, uint8_t len);
00128
00136 radio_status_t radio_lora_raw_write_burst(uint8_t reg, const uint8_t *data, uint8_t len);
00137
00142 void radio_lora_on_exti(uint16_t gpio_pin);
00143
00144 #ifdef __cplusplus
00145 }
00146 #endif
00147
00148 #endif /* APP_RADIO_LIB_MODULATIONS_LORA_RADIO_LORA_H_ */

```

12.62 Dokumentacja pliku Core/App/radio_lib/modulations/ook/radio_ook.c

Backend modulacji OOK dla SX1276.

```

#include "radio_ook.h"
#include "app_delay.h"
#include "main.h"
#include "../common/sx1276/radio_sx1276.h"
#include "../common/sx1276/radio_sx1276_regs.h"
#include "../lora/radio_lora.h"
#include <string.h>

```

Struktury danych

- struct [radio_ook_context_t](#)

Definicje

- #define [RADIO_OOK_FIFO_THRESH_LEVEL](#) 31U
- #define [RADIO_OOK_RESET_LOW_DELAY_MS](#) 2U
- #define [RADIO_OOK_RESET_HIGH_DELAY_MS](#) 10U
- #define [RADIO_OOK_TX_GUARD_MS](#) 40UL
- #define [RADIO_OOK_FIFO_MAX_BYTES](#) 64U

Funkcje

- static uint32_t [radio_irq_save](#) (void)
Internal helper: [radio_irq_save](#).
- static void [radio_irq_restore](#) (uint32_t primask)
Internal helper: [radio_irq_restore](#).
- static void [radio_set_event_flags](#) (uint32_t events)
Internal helper: [radio_set_event_flags](#).
- static bool [radio_cfg_valid](#) (const [radio_ook_cfg_t](#) *cfg)
Internal helper: [radio_cfg_valid](#).
- static void [radio_hw_reset](#) (void)
Internal helper: [radio_hw_reset](#).
- static uint8_t [radio_get_rx_bw_reg_value](#) ([radio_lora_bw_t](#) bw)
Internal helper: [radio_get_rx_bw_reg_value](#).

- static bool [radio_write_sync_word](#) (const [radio_ook_cfg_t](#) *cfg)
Internal helper: [radio_write_sync_word](#).
- static bool [radio_set_op_mode_ready](#) (uint8_t op_mode)
- static bool [radio_apply_threshold](#) (const [radio_ook_cfg_t](#) *cfg)
Internal helper: [radio_apply_threshold](#).
- static bool [radio_apply_ook_config](#) (const [radio_ook_cfg_t](#) *cfg)
Internal helper: [radio_apply_ook_config](#).
- static void [radio_set_state](#) ([radio_state_t](#) state)
Internal helper: [radio_set_state](#).
- static void [radio_resume_after_tx](#) (void)
Internal helper: [radio_resume_after_tx](#).
- static uint32_t [radio_tx_timeout_ms](#) (uint8_t payload_len)
Internal helper: [radio_tx_timeout_ms](#).
- static uint32_t [radio_rx_single_timeout_ms](#) (uint32_t symbol_timeout)
Internal helper: [radio_rx_single_timeout_ms](#).
- static void [radio_handle_exti_pin](#) (uint16_t pin)
Internal helper: [radio_handle_exti_pin](#).
- static [radio_status_t](#) [radio_read_rx_packet](#) (void)
Internal helper: [radio_read_rx_packet](#).
- void [radio_ook_default_hw_cfg](#) ([radio_hw_cfg_t](#) *cfg, SPI_HandleTypeDef *hspi)
Domyślna konfiguracja sprzętowa backendu OOK.
- void [radio_ook_default_lora_cfg](#) ([radio_lora_cfg_t](#) *cfg)
Domyślna konfiguracja parametrów backendu OOK.
- [radio_status_t](#) [radio_ook_set_runtime_cfg](#) (const [radio_ook_cfg_t](#) *cfg)
Ustawia runtime konfigurację właściwą dla backendu OOK.
- [radio_status_t](#) [radio_ook_init](#) (const [radio_hw_cfg_t](#) *hw, const [radio_lora_cfg_t](#) *cfg, [radio_event_cb_t](#) cb, void *user_ctx)
Inicjalizacja backendu OOK.
- [radio_status_t](#) [radio_ook_deinit](#) (void)
Deinicjalizacja backendu OOK.
- [radio_status_t](#) [radio_ook_start_rx_continuous](#) (void)
Uruchomienie odbioru ciągłego OOK.
- [radio_status_t](#) [radio_ook_start_rx_single](#) (uint16_t symbol_timeout)
Uruchomienie odbioru pojedynczego OOK.
- [radio_status_t](#) [radio_ook_send_async](#) (const uint8_t *data, uint8_t len)
Uruchomienie nadawania asynchronicznego OOK.
- [radio_status_t](#) [radio_ook_standby](#) (void)
Przejsie do standby backendu OOK.
- [radio_status_t](#) [radio_ook_sleep](#) (void)
Przejsie do sleep backendu OOK.
- void [radio_ook_process](#) (void)
Przetwarzanie logiki odroczonej backendu OOK.
- uint32_t [radio_ook_take_events](#) (void)
Zwrócenie i wyczyszczenie flag zdarzeń backendu OOK.
- bool [radio_ook_get_last_packet](#) ([radio_packet_t](#) *pkt)
Odczyt ostatniego pakietu backendu OOK.
- [radio_state_t](#) [radio_ook_get_state](#) (void)
Odczyt stanu backendu OOK.
- [radio_status_t](#) [radio_ook_raw_read_reg](#) (uint8_t reg, uint8_t *value)
Odczyt pojedynczego rejestru (backend OOK).
- [radio_status_t](#) [radio_ook_raw_write_reg](#) (uint8_t reg, uint8_t value)

Zapis pojedynczego rejestru (backend OOK).

- [radio_status_t radio_ook_raw_read_burst](#) (uint8_t reg, uint8_t *data, uint8_t len)

Odczyt sekwencji rejestrów (backend OOK).

- [radio_status_t radio_ook_raw_write_burst](#) (uint8_t reg, const uint8_t *data, uint8_t len)

Zapis sekwencji rejestrów (backend OOK).

- void [radio_ook_on_exti](#) (uint16_t gpio_pin)

Przekazanie pinu EXTI do backendu OOK.

Zmienne

- static [radio_ook_context_t s_radio](#)
- static [radio_ook_cfg_t s_runtime_cfg](#)
- static bool [s_runtime_cfg_ready](#) = false

12.62.1 Opis szczegółowy

Backend modulacji OOK dla SX1276.

Definicja w pliku [radio_ook.c](#).

12.62.2 Dokumentacja definicji

12.62.2.1 RADIO_OOK_FIFO_MAX_BYTES

```
#define RADIO_OOK_FIFO_MAX_BYTES 64U
```

Definicja w linii 20 pliku [radio_ook.c](#).

12.62.2.2 RADIO_OOK_FIFO_THRESH_LEVEL

```
#define RADIO_OOK_FIFO_THRESH_LEVEL 31U
```

Definicja w linii 16 pliku [radio_ook.c](#).

12.62.2.3 RADIO_OOK_RESET_HIGH_DELAY_MS

```
#define RADIO_OOK_RESET_HIGH_DELAY_MS 10U
```

Definicja w linii 18 pliku [radio_ook.c](#).

12.62.2.4 RADIO_OOK_RESET_LOW_DELAY_MS

```
#define RADIO_OOK_RESET_LOW_DELAY_MS 2U
```

Definicja w linii 17 pliku [radio_ook.c](#).

12.62.2.5 RADIO_OOK_TX_GUARD_MS

```
#define RADIO_OOK_TX_GUARD_MS 40UL
```

Definicja w linii 19 pliku [radio_ook.c](#).

12.62.3 Dokumentacja funkcji

12.62.3.1 radio_apply_ook_config()

```
bool radio_apply_ook_config (
    const radio\_ook\_cfg\_t * cfg) [static]
```

Internal helper: [radio_apply_ook_config](#).

Definicja w linii 197 pliku [radio_ook.c](#).

Odwwołuje się do [radio_ook_cfg_t::bitrate_bps](#), [radio_ook_cfg_t::frequency_hz](#), [radio_ook_cfg_t::preamble_len](#), [radio_apply_threshold\(\)](#), [radio_get_rx_bw_reg_value\(\)](#), [RADIO_LIB_MAX_PAYLOAD](#), [radio_write_sync_word\(\)](#), [radio_ook_cfg_t::rx_bandwidth](#), [sx1276_set_frequency\(\)](#), [sx1276_set_pa_output_power\(\)](#), [sx1276_write_reg\(\)](#) i [radio_ook_cfg_t::tx_power_dbm](#).

Odwwołania w [radio_ook_init\(\)](#).

12.62.3.2 radio_apply_threshold()

```
bool radio_apply_threshold (
    const radio_ook_cfg_t * cfg) [static]
```

Internal helper: [radio_apply_threshold](#).

Definicja w linii 178 pliku [radio_ook.c](#).

Odwołuje się do [sx1276_write_reg\(\)](#), [radio_ook_cfg_t::threshold](#) i [radio_ook_cfg_t::threshold_value](#).

Odwołania w [radio_apply_ook_config\(\)](#).

12.62.3.3 radio_cfg_valid()

```
bool radio_cfg_valid (
    const radio_ook_cfg_t * cfg) [static]
```

Internal helper: [radio_cfg_valid](#).

Definicja w linii 68 pliku [radio_ook.c](#).

Odwołuje się do [radio_ook_cfg_t::bitrate_bps](#), [radio_ook_cfg_t::frequency_hz](#), [radio_ook_cfg_t::preamble_len](#), [radio_ook_cfg_t::rx_bandwidth](#), [radio_ook_cfg_t::sync_word_len](#) i [radio_ook_cfg_t::threshold](#).

Odwołania w [radio_ook_init\(\)](#) i [radio_ook_set_runtime_cfg\(\)](#).

12.62.3.4 radio_get_rx_bw_reg_value()

```
uint8_t radio_get_rx_bw_reg_value (
    radio_lora_bw_t bw) [static]
```

Internal helper: [radio_get_rx_bw_reg_value](#).

Definicja w linii 96 pliku [radio_ook.c](#).

Odwołania w [radio_apply_ook_config\(\)](#).

12.62.3.5 radio_handle_exti_pin()

```
void radio_handle_exti_pin (
    uint16_t pin) [static]
```

Internal helper: [radio_handle_exti_pin](#).

Definicja w linii 327 pliku [radio_ook.c](#).

Odwołuje się do [radio_irq_restore\(\)](#) i [radio_irq_save\(\)](#).

Odwołania w [radio_ook_on_exti\(\)](#).

12.62.3.6 radio_hw_reset()

```
void radio_hw_reset (
    void ) [static]
```

Internal helper: [radio_hw_reset](#).

Definicja w linii 86 pliku [radio_ook.c](#).

Odwołania w [radio_ook_init\(\)](#).

12.62.3.7 radio_irq_restore()

```
void radio_irq_restore (
    uint32_t primask) [static]
```

Internal helper: [radio_irq_restore](#).

Definicja w linii 51 pliku [radio_ook.c](#).

Odwołania w [radio_handle_exti_pin\(\)](#), [radio_ook_process\(\)](#), [radio_ook_send_async\(\)](#), [radio_ook_take_events\(\)](#) i [radio_set_event_flags\(\)](#).

12.62.3.8 radio_irq_save()

```
uint32_t radio_irq_save (
    void ) [static]
```

Internal helper: [radio_irq_save](#).

Definicja w linii 43 pliku [radio_ook.c](#).

Odwołania w [radio_handle_exti_pin\(\)](#), [radio_ook_process\(\)](#), [radio_ook_send_async\(\)](#), [radio_ook_take_events\(\)](#) i [radio_set_event_flags\(\)](#).

12.62.3.9 radio_ook_default_hw_cfg()

```
void radio_ook_default_hw_cfg (
    radio_hw_cfg_t * cfg,
    SPI_HandleTypeDef * hspi)
```

Domyślna konfiguracja sprzętowa backendu OOK.

Definicja w linii 384 pliku [radio_ook.c](#).

Odwołuje się do [radio_lora_default_hw_cfg\(\)](#).

12.62.3.10 radio_ook_default_lora_cfg()

```
void radio_ook_default_lora_cfg (
    radio_lora_cfg_t * cfg)
```

Domyślna konfiguracja parametrów backendu OOK.

Definicja w linii 389 pliku [radio_ook.c](#).

Odwołuje się do [radio_lora_default_lora_cfg\(\)](#).

12.62.3.11 radio_ook_deinit()

```
radio_status_t radio_ook_deinit (
    void )
```

Deinicjalizacja backendu OOK.

Definicja w linii 467 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_ESTATE](#), [RADIO_OK](#), [radio_ook_sleep\(\)](#) i [RADIO_STATE_UNINIT](#).

Odwołania w [radio_deinit\(\)](#).

12.62.3.12 radio_ook_get_last_packet()

```
bool radio_ook_get_last_packet (
    radio_packet_t * pkt)
```

Odczyt ostatniego pakietu backendu OOK.

Definicja w linii 743 pliku [radio_ook.c](#).

Odwołania w [radio_get_last_packet\(\)](#).

12.62.3.13 radio_ook_get_state()

```
radio_state_t radio_ook_get_state (
    void )
```

Odczyt stanu backendu OOK.

Definicja w linii 754 pliku [radio_ook.c](#).

Odwołania w [radio_get_state\(\)](#).

12.62.3.14 radio_ook_init()

```
radio_status_t radio_ook_init (
    const radio_hw_cfg_t * hw,
    const radio_lora_cfg_t * cfg,
    radio_event_cb_t cb,
    void * user_ctx)
```

Inicjalizacja backendu OOK.

Definicja w linii 406 pliku [radio_ook.c](#).

Odwołuje się do [radio_hw_cfg_t::hspi](#), [radio_hw_cfg_t::nss](#), [radio_gpio_t::pin](#), [radio_gpio_t::port](#), [radio_apply_ook_config\(\)](#), [radio_cfg_valid\(\)](#), [radio_default_ook_cfg\(\)](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [radio_hw_reset\(\)](#), [RADIO_OK](#), [RADIO_STATE_STANDBY](#), [RADIO_STATE_UNINIT](#), [radio_hw_cfg_t::reset](#), [radio_hw_cfg_t::spi_timeout_ms](#) i [sx1276_get_version\(\)](#).

Odwołania w [radio_init\(\)](#).

12.62.3.15 radio_ook_on_exti()

```
void radio_ook_on_exti (
    uint16_t gpio_pin)
```

Przekazanie pinu EXTI do backendu OOK.

Definicja w linii 809 pliku [radio_ook.c](#).

Odwołuje się do [radio_handle_exti_pin\(\)](#).

Odwołania w [radio_on_exti\(\)](#).

12.62.3.16 radio_ook_process()

```
void radio_ook_process (
    void )
```

Przetwarzanie logiki odroczonej backendu OOK.

Definicja w linii 622 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EVENT_FIFO_OVERRUN](#), [RADIO_EVENT_HW_ERROR](#), [RADIO_EVENT_NONE](#), [RADIO_EVENT_RX_DONE](#), [RADIO_EVENT_RX_TIMEOUT](#), [RADIO_EVENT_TX_DONE](#), [radio_irq_restore\(\)](#), [radio_irq_save\(\)](#), [RADIO_OK](#), [radio_ook_standby\(\)](#), [radio_read_rx_packet\(\)](#), [radio_resume_after_tx\(\)](#), [radio_set_event_flags\(\)](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_TX](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_process\(\)](#).

12.62.3.17 radio_ook_raw_read_burst()

```
radio_status_t radio_ook_raw_read_burst (
    uint8_t reg,
    uint8_t * data,
    uint8_t len)
```

Odczyt sekwencji rejestrów (backend OOK).

Definicja w linii 779 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_read_burst\(\)](#).

Odwołania w [radio_raw_read_burst\(\)](#).

12.62.3.18 radio_ook_raw_read_reg()

```
radio_status_t radio_ook_raw_read_reg (
    uint8_t reg,
    uint8_t * value)
```

Odczyt pojedynczego rejestru (backend OOK).

Definicja w linii 759 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_raw_read_reg\(\)](#).

12.62.3.19 radio_ook_raw_write_burst()

```
radio_status_t radio_ook_raw_write_burst (
    uint8_t reg,
    const uint8_t * data,
    uint8_t len)
```

Zapis sekwencji rejestrów (backend OOK).

Definicja w linii 794 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_write_burst\(\)](#).

Odwołania w [radio_raw_write_burst\(\)](#).

12.62.3.20 radio_ook_raw_write_reg()

```
radio_status_t radio_ook_raw_write_reg (
    uint8_t reg,
    uint8_t value)
```

Zapis pojedynczego rejestru (backend OOK).

Definicja w linii 769 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_raw_write_reg\(\)](#).

12.62.3.21 radio_ook_send_async()

```
radio_status_t radio_ook_send_async (
    const uint8_t * data,
    uint8_t len)
```

Uruchomienie nadawania asynchronicznego OOK.

Definicja w linii 535 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [radio_irq_restore\(\)](#), [radio_irq_save\(\)](#), [RADIO_LIB_MAX_PAYLOAD](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_TX](#), [radio_tx_timeout_ms\(\)](#), [sx1276_set_op_mode\(\)](#), [sx1276_write_burst\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_send_async\(\)](#).

12.62.3.22 radio_ook_set_runtime_cfg()

```
radio_status_t radio_ook_set_runtime_cfg (
    const radio_ook_cfg_t * cfg)
```

Ustawia runtime konfigurację właściwą dla backendu OOK.

Definicja w linii 394 pliku [radio_ook.c](#).

Odwołuje się do [radio_cfg_valid\(\)](#), [RADIO_EINVAL](#) i [RADIO_OK](#).

Odwołania w [radio_set_ook_cfg\(\)](#).

12.62.3.23 radio_ook_sleep()

```
radio_status_t radio_ook_sleep (
    void )
```

Przejsie do sleep backendu OOK.

Definicja w linii 604 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#) i [RADIO_STATE_STANDBY](#).

Odwołania w [radio_ook_deinit\(\)](#) i [radio_sleep\(\)](#).

12.62.3.24 radio_ook_standby()

```
radio_status_t radio_ook_standby (
    void )
```

Przejsie do standby backendu OOK.

Definicja w linii 586 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#) i [RADIO_STATE_STANDBY](#).

Odwołania w [radio_ook_process\(\)](#) i [radio_standby\(\)](#).

12.62.3.25 radio_ook_start_rx_continuous()

```
radio_status_t radio_ook_start_rx_continuous (
    void )
```

Uruchomienie odbioru ciągłego OOK.

Definicja w linii 480 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_CONT](#), [RADIO_STATE_TX](#), [sx1276_set_op_mode\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_start_rx_continuous\(\)](#).

12.62.3.26 radio_ook_start_rx_single()

```
radio_status_t radio_ook_start_rx_single (
    uint16_t symbol_timeout)
```

Uruchomienie odbioru pojedynczego OOK.

Definicja w linii 505 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_rx_single_timeout_ms\(\)](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_TX](#), [sx1276_set_op_mode\(\)](#) i [sx1276_write_reg\(\)](#).
 Odwołania w [radio_start_rx_single\(\)](#).

12.62.3.27 radio_ook_take_events()

```
uint32_t radio_ook_take_events (
    void )
```

Zwrócenie i wyczyszczenie flag zdarzeń backendu OOK.

Definicja w linii 732 pliku [radio_ook.c](#).

Odwołuje się do [radio_irq_restore\(\)](#) i [radio_irq_save\(\)](#).

Odwołania w [radio_take_events\(\)](#).

12.62.3.28 radio_read_rx_packet()

```
radio_status_t radio_read_rx_packet (
    void ) [static]
```

Internal helper: [radio_read_rx_packet](#).

Definicja w linii 349 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_LIB_MAX_PAYLOAD](#), [RADIO_OK](#), [sx1276_read_burst\(\)](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_ook_process\(\)](#).

12.62.3.29 radio_resume_after_tx()

```
void radio_resume_after_tx (
    void ) [static]
```

Internal helper: [radio_resume_after_tx](#).

Definicja w linii 243 pliku [radio_ook.c](#).

Odwołuje się do [radio_set_state\(\)](#), [RADIO_STATE_RX_CONT](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_STANDBY](#) i [sx1276_set_op_mode\(\)](#).

Odwołania w [radio_ook_process\(\)](#).

12.62.3.30 radio_rx_single_timeout_ms()

```
uint32_t radio_rx_single_timeout_ms (
    uint32_t symbol_timeout) [static]
```

Internal helper: [radio_rx_single_timeout_ms](#).

Definicja w linii 302 pliku [radio_ook.c](#).

Odwołania w [radio_ook_start_rx_single\(\)](#).

12.62.3.31 radio_set_event_flags()

```
void radio_set_event_flags (
    uint32_t events) [static]
```

Internal helper: [radio_set_event_flags](#).

Definicja w linii 60 pliku [radio_ook.c](#).

Odwołuje się do [radio_irq_restore\(\)](#) i [radio_irq_save\(\)](#).

Odwołania w [radio_ook_process\(\)](#).

12.62.3.32 radio_set_op_mode_ready()

```
bool radio_set_op_mode_ready (
    uint8_t op_mode) [static]
```

Definicja w linii 148 pliku [radio_ook.c](#).

12.62.3.33 radio_set_state()

```
void radio_set_state (
    radio_state_t state) [static]
```


Internal helper: [radio_set_state](#).

Definicja w linii 237 pliku [radio_ook.c](#).

Odwołania w [radio_ook_send_async\(\)](#), [radio_ook_sleep\(\)](#), [radio_ook_standby\(\)](#), [radio_ook_start_rx_continuous\(\)](#), [radio_ook_start_rx_single\(\)](#) i [radio_resume_after_tx\(\)](#).

12.62.3.34 radio_tx_timeout_ms()

```
uint32_t radio_tx_timeout_ms (
    uint8_t payload_len) [static]
```

Internal helper: [radio_tx_timeout_ms](#).

Definicja w linii 272 pliku [radio_ook.c](#).

Odwołania w [radio_ook_send_async\(\)](#).

12.62.3.35 radio_write_sync_word()

```
bool radio_write_sync_word (
    const radio_ook_cfg_t * cfg) [static]
```

Internal helper: [radio_write_sync_word](#).

Definicja w linii 125 pliku [radio_ook.c](#).

Odwołuje się do [sx1276_write_burst\(\)](#), [sx1276_write_reg\(\)](#), [radio_ook_cfg_t::sync_word](#) i [radio_ook_cfg_t::sync_word_len](#).

Odwołania w [radio_apply_ook_config\(\)](#).

12.62.4 Dokumentacja zmiennych

12.62.4.1 s_radio

```
radio_ook_context_t s_radio [static]
```

Definicja w linii 38 pliku [radio_ook.c](#).

12.62.4.2 s_runtime_cfg

```
radio_ook_cfg_t s_runtime_cfg [static]
```

Definicja w linii 39 pliku [radio_ook.c](#).

12.62.4.3 s_runtime_cfg_ready

```
bool s_runtime_cfg_ready = false [static]
```

Definicja w linii 40 pliku [radio_ook.c](#).

12.63 radio_ook.c

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #include "radio_ook.h"
00007
00008 #include "app_delay.h"
00009 #include "main.h"
00010 #include "../common/sx1276/radio_sx1276.h"
00011 #include "../common/sx1276/radio_sx1276_regs.h"
00012 #include "../lorawan/radio_lora.h"
00013
00014 #include <string.h>
00015
00016 #define RADIO_OOK_FIFO_THRESH_LEVEL      31U
00017 #define RADIO_OOK_RESET_LOW_DELAY_MS    2U
00018 #define RADIO_OOK_RESET_HIGH_DELAY_MS   10U
00019 #define RADIO_OOK_TX_GUARD_MS            40UL
00020 #define RADIO_OOK_FIFO_MAX_BYTES        64U
00021 typedef struct
00022 {
00023     bool initialized;
00024     sx1276_bus_t bus;
00025     radio_hw_cfg_t hw;
00026     radio_ook_cfg_t cfg;
00027     radio_event_cb_t callback;
00028     void *callback_ctx;
```

```

00029     volatile uint8_t dio_pending_mask;
00030     volatile uint32_t event_flags;
00031     radio_state_t state;
00032     radio_state_t tx_resume_state;
00033     uint32_t rx_single_deadline_ms;
00034     uint32_t tx_deadline_ms;
00035     radio_packet_t last_packet;
00036 } radio_ook_context_t;
00037
00038 static radio_ook_context_t s_radio;
00039 static radio_ook_cfg_t s_runtime_cfg;
00040 static bool s_runtime_cfg_ready = false;
00041
00043 static uint32_t radio_irq_save(void)
00044 {
00045     uint32_t primask = __get_PRIMASK();
00046     __disable_irq();
00047     return primask;
00048 }
00049
00051 static void radio_irq_restore(uint32_t primask)
00052 {
00053     if (primask == 0U)
00054     {
00055         __enable_irq();
00056     }
00057 }
00058
00060 static void radio_set_event_flags(uint32_t events)
00061 {
00062     uint32_t key = radio_irq_save();
00063     s_radio.event_flags |= events;
00064     radio_irq_restore(key);
00065 }
00066
00068 static bool radio_cfg_valid(const radio_ook_cfg_t *cfg)
00069 {
00070     if ((cfg == NULL) ||
00071         (cfg->frequency_hz < 137000000UL) || (cfg->frequency_hz > 1020000000UL) ||
00072         (cfg->bitrate_bps < 600UL) || (cfg->bitrate_bps > 300000UL) ||
00073         ((uint8_t)cfg->rx_bandwidth > (uint8_t)RADIO_LORA_BW_500_KHZ) ||
00074         (cfg->preamble_len == 0U) ||
00075         (cfg->sync_word_len == 0U) ||
00076         (cfg->sync_word_len > 4U) ||
00077         ((uint8_t)cfg->threshold > (uint8_t)RADIO_OOK_THRESHOLD_AVERAGE))
00078     {
00079         return false;
00080     }
00081
00082     return true;
00083 }
00084
00086 static void radio_hw_reset(void)
00087 {
00088     HAL_GPIO_WritePin(s_radio.hw.nss.port, s_radio.hw.nss.pin, GPIO_PIN_SET);
00089     HAL_GPIO_WritePin(s_radio.hw.reset.port, s_radio.hw.reset.pin, GPIO_PIN_RESET);
00090     app_delay_ms(RADIO_OOK_RESET_LOW_DELAY_MS);
00091     HAL_GPIO_WritePin(s_radio.hw.reset.port, s_radio.hw.reset.pin, GPIO_PIN_SET);
00092     app_delay_ms(RADIO_OOK_RESET_HIGH_DELAY_MS);
00093 }
00094
00096 static uint8_t radio_get_rx_bw_reg_value(radio_lora_bw_t bw)
00097 {
00098     switch (bw)
00099     {
00100         case RADIO_LORA_BW_7_8_KHZ:
00101             return 0x06U;
00102         case RADIO_LORA_BW_10_4_KHZ:
00103             return 0x15U;
00104         case RADIO_LORA_BW_15_6_KHZ:
00105             return 0x05U;
00106         case RADIO_LORA_BW_20_8_KHZ:
00107             return 0x14U;
00108         case RADIO_LORA_BW_31_25_KHZ:
00109             return 0x04U;
00110         case RADIO_LORA_BW_41_7_KHZ:
00111             return 0x13U;
00112         case RADIO_LORA_BW_62_5_KHZ:
00113             return 0x03U;
00114         case RADIO_LORA_BW_125_KHZ:
00115             return 0x02U;
00116         case RADIO_LORA_BW_250_KHZ:
00117             return 0x01U;
00118         case RADIO_LORA_BW_500_KHZ:
00119             default:
00120                 return 0x00U;
00121     }

```

```

00122 }
00123
00125 static bool radio_write_sync_word(const radio_ook_cfg_t *cfg)
00126 {
00127     uint8_t sync_bytes[4];
00128     uint8_t i;
00129
00130     for (i = 0U; i < cfg->sync_word_len; i++)
00131     {
00132         uint8_t shift = (uint8_t)((cfg->sync_word_len - 1U - i) * 8U) & 0x1FU);
00133         sync_bytes[i] = (uint8_t)((cfg->sync_word » shift) & 0xFFU);
00134     }
00135
00136     return sx1276_write_reg(&s_radio.bus, SX1276_REG_SYNC_CONFIG,
00137                             (uint8_t)(SX1276_SYNC_AUTO_RESTART_PLL |
00138                                         SX1276_SYNC_ON |
00139                                         SX1276_SYNC_FIFO_FILL_AUTO |
00140                                         ((cfg->sync_word_len - 1U) & 0x07U))) &&
00141         sx1276_write_burst(&s_radio.bus, SX1276_REG_SYNC_VALUE_1, sync_bytes, cfg->sync_word_len);
00142 }
00143
00144 /*
00145  * Waits for ModeReady after OOK/FSK-mode transitions. This keeps the packet
00146  * engine from being reconfigured while the modem still exits RX.
00147  */
00148 static bool radio_set_op_mode_ready(uint8_t op_mode)
00149 {
00150     uint8_t irq1;
00151     uint8_t retry;
00152
00153     if (!sx1276_set_op_mode(&s_radio.bus, op_mode))
00154     {
00155         return false;
00156     }
00157
00158     if ((op_mode & SX1276_OPMODE_MODE_MASK) == SX1276_MODE_SLEEP)
00159     {
00160         app_delay_ms(1U);
00161         return true;
00162     }
00163
00164     for (retry = 0U; retry < 5U; retry++)
00165     {
00166         if (sx1276_read_reg(&s_radio.bus, SX1276_REG_IRQ_FLAGS_1, &irq1) &&
00167             ((irq1 & SX1276_IRQ1_MODE_READY) != 0U))
00168         {
00169             return true;
00170         }
00171         app_delay_ms(1U);
00172     }
00173
00174     return false;
00175 }
00176
00178 static bool radio_apply_threshold(const radio_ook_cfg_t *cfg)
00179 {
00180     switch (cfg->threshold)
00181     {
00182         case RADIO_OOK_THRESHOLD_FIXED:
00183             return sx1276_write_reg(&s_radio.bus, SX1276_REG_OOK_FIX, cfg->threshold_value);
00184
00185         case RADIO_OOK_THRESHOLD_AVERAGE:
00186             return sx1276_write_reg(&s_radio.bus, SX1276_REG_OOK_AVG,
00187                                     (uint8_t)(0x80U | (cfg->threshold_value & 0x7FU)));
00188
00189         case RADIO_OOK_THRESHOLD_PEAK:
00190             default:
00191                 return sx1276_write_reg(&s_radio.bus, SX1276_REG_OOK_PEAK,
00192                                         (uint8_t)(0x40U | (cfg->threshold_value & 0x3FU)));
00193     }
00194 }
00195
00197 static bool radio_apply_ook_config(const radio_ook_cfg_t *cfg)
00198 {
00199     uint32_t bitrate_reg;
00200     uint8_t rx_bw;
00201     uint8_t op_mode;
00202
00203     bitrate_reg = (32000000UL + (cfg->bitrate_bps / 2UL)) / cfg->bitrate_bps;
00204     rx_bw = radio_get_rx_bw_reg_value(cfg->rx_bandwidth);
00205     op_mode = (uint8_t)(SX1276_OPMODE_MODULATION_OOK | SX1276_OPMODE_SHAPING_NONE |
00206                         SX1276_MODE_SLEEP);
00207
00208     return radio_set_op_mode_ready(op_mode) &&
00209         sx1276_set_frequency(&s_radio.bus, cfg->frequency_hz) &&
00210         sx1276_set_pa_output_power(&s_radio.bus, cfg->tx_power_dbm) &&
00211         sx1276_write_reg(&s_radio.bus, SX1276_REG_BITRATE_MSB, (uint8_t)((bitrate_reg » 8) &

```

```

0xFFU)) &&
00211     sx1276_write_reg(&s_radio.bus, SX1276_REG_BITRATE_LSB, (uint8_t)(bitrate_reg & 0xFFU)) &&
00212     sx1276_write_reg(&s_radio.bus, SX1276_REG_RX_BW, rx_bw) &&
00213     sx1276_write_reg(&s_radio.bus, SX1276_REG_AFC_BW, rx_bw) &&
00214     sx1276_write_reg(&s_radio.bus, SX1276_REG_PREAMBLE_DETECT, 0xAAU) &&
00215     sx1276_write_reg(&s_radio.bus, SX1276_REG_RX_CONFIG,
00216                     (uint8_t)(SX1276_RXCONFIG_AFC_AUTO_ON |
00217                               SX1276_RXCONFIG_AGC_AUTO_ON |
00218                               SX1276_RXCONFIG_TRIGGER_RSSI)) &&
00219     sx1276_write_reg(&s_radio.bus, SX1276_REG_PREAMBLE_MSB_FSK, (uint8_t)((cfg->preamble_len »
00220 8) & 0xFFU)) &&
00220     sx1276_write_reg(&s_radio.bus, SX1276_REG_PREAMBLE_LSB_FSK, (uint8_t)(cfg->preamble_len &
00221 0xFFU)) &&
00221     radio_write_sync_word(cfg) &&
00222     radio_apply_threshold(cfg) &&
00223     sx1276_write_reg(&s_radio.bus, SX1276_REG_PACKET_CONFIG_1, SX1276_PACKET_FORMAT_VARIABLE)
00224     &&
00224     sx1276_write_reg(&s_radio.bus, SX1276_REG_PACKET_CONFIG_2, SX1276_PACKET_DATA_MODE_PACKET)
00225     &&
00225     sx1276_write_reg(&s_radio.bus, SX1276_REG_PAYLOAD_LENGTH_FSK, RADIO_LIB_MAX_PAYLOAD) &&
00226     sx1276_write_reg(&s_radio.bus, SX1276_REG_FIFO_THRESH,
00227                     (uint8_t)(SX1276_FIFO_THRESH_TX_START_NOT_EMPTY |
00228                               (RADIO_OOK_FIFO_THRESH_LEVEL & 0x3FU))) &&
00229     sx1276_write_reg(&s_radio.bus, SX1276_REG_DIO_MAPPING_1, 0x00U) &&
00230     sx1276_write_reg(&s_radio.bus, SX1276_REG_DIO_MAPPING_2, 0x00U) &&
00231     radio_set_op_mode_ready((uint8_t)(SX1276_OPMODE_MODULATION_OOK |
00232                                     SX1276_OPMODE_SHAPING_NONE |
00233                                     SX1276_MODE_STDBY));
00234 }
00235
00237 static void radio_set_state(radio_state_t state)
00238 {
00239     s_radio.state = state;
00240 }
00241
00243 static void radio_resume_after_tx(void)
00244 {
00245     if (s_radio.tx_resume_state == RADIO_STATE_RX_CONT)
00246     {
00247         (void)sx1276_set_op_mode(&s_radio.bus,
00248                                 (uint8_t)(SX1276_OPMODE_MODULATION_OOK |
00249                                             SX1276_OPMODE_SHAPING_NONE |
00250                                             SX1276_MODE_RXCONTINUOUS));
00251         radio_set_state(RADIO_STATE_RX_CONT);
00252     }
00253     else if (s_radio.tx_resume_state == RADIO_STATE_RX_SINGLE)
00254     {
00255         (void)sx1276_set_op_mode(&s_radio.bus,
00256                                 (uint8_t)(SX1276_OPMODE_MODULATION_OOK |
00257                                             SX1276_OPMODE_SHAPING_NONE |
00258                                             SX1276_MODE_RXSINGLE));
00259         radio_set_state(RADIO_STATE_RX_SINGLE);
00260     }
00261     else
00262     {
00263         (void)sx1276_set_op_mode(&s_radio.bus,
00264                                 (uint8_t)(SX1276_OPMODE_MODULATION_OOK |
00265                                             SX1276_OPMODE_SHAPING_NONE |
00266                                             SX1276_MODE_STDBY));
00267         radio_set_state(RADIO_STATE_STANDBY);
00268     }
00269 }
00270
00272 static uint32_t radio_tx_timeout_ms(uint8_t payload_len)
00273 {
00274     uint64_t timeout_ms;
00275     uint32_t total_bytes;
00276
00277     total_bytes = (uint32_t)s_radio.cfg.preamble_len +
00278                  (uint32_t)s_radio.cfg.sync_word_len +
00279                  (uint32_t)payload_len +
00280                  1UL + /* variable-length byte */
00281                  2UL; /* guard for packet engine overhead */
00282     timeout_ms = ((uint64_t)total_bytes * 8ULL * 1000ULL) / s_radio.cfg.bitrate_bps;
00283     if (((uint64_t)total_bytes * 8ULL * 1000ULL) % s_radio.cfg.bitrate_bps) != 0ULL)
00284     {
00285         timeout_ms++;
00286     }
00287
00288     timeout_ms += RADIO_OOK_TX_GUARD_MS;
00289     if (timeout_ms < RADIO_OOK_TX_GUARD_MS)
00290     {
00291         timeout_ms = RADIO_OOK_TX_GUARD_MS;
00292     }
00293     if (timeout_ms > 0xFFFFFFFFULL)
00294     {
00295         return 0xFFFFFFFFUL;

```

```

00296     }
00297
00298     return (uint32_t)timeout_ms;
00299 }
00300
00302 static uint32_t radio_rx_single_timeout_ms(uint32_t symbol_timeout)
00303 {
00304     uint64_t timeout_ms;
00305
00306     timeout_ms = ((uint64_t)symbol_timeout * 1000ULL) / s_radio.cfg.bitrate_bps;
00307     if (((uint64_t)symbol_timeout * 1000ULL) % s_radio.cfg.bitrate_bps) != 0ULL)
00308     {
00309         timeout_ms++;
00310     }
00311
00312     timeout_ms += 2ULL;
00313     if (timeout_ms == 0ULL)
00314     {
00315         timeout_ms = 1ULL;
00316     }
00317
00318     if (timeout_ms > 0xFFFFFFFFULL)
00319     {
00320         return 0xFFFFFFFFUL;
00321     }
00322
00323     return (uint32_t)timeout_ms;
00324 }
00325
00327 static void radio_handle_exti_pin(uint16_t pin)
00328 {
00329     uint8_t i;
00330
00331     if (!s_radio.initialized)
00332     {
00333         return;
00334     }
00335
00336     for (i = 0U; i < 6U; i++)
00337     {
00338         if (s_radio.hw.dio[i].pin == pin)
00339         {
00340             uint32_t key = radio_irq_save();
00341             s_radio.dio_pending_mask |= (uint8_t)(1U << i);
00342             radio_irq_restore(key);
00343             break;
00344         }
00345     }
00346 }
00347
00349 static radio_status_t radio_read_rx_packet(void)
00350 {
00351     uint8_t packet_len;
00352     uint8_t rssi_raw;
00353
00354     if (!sx1276_read_burst(&s_radio.bus, SX1276_REG_FIFO, &packet_len, 1U))
00355     {
00356         return RADIO_EHW;
00357     }
00358
00359     if (packet_len > RADIO_LIB_MAX_PAYLOAD)
00360     {
00361         s_radio.last_packet.valid = false;
00362         return RADIO_EBUS;
00363     }
00364
00365     if ((packet_len > 0U) &&
00366         !sx1276_read_burst(&s_radio.bus, SX1276_REG_FIFO, s_radio.last_packet.data, packet_len))
00367     {
00368         return RADIO_EHW;
00369     }
00370
00371     if (!sx1276_read_reg(&s_radio.bus, SX1276_REG_RSSI_VALUE_FSK, &rssi_raw))
00372     {
00373         return RADIO_EHW;
00374     }
00375
00376     s_radio.last_packet.length = packet_len;
00377     s_radio.last_packet.rssi_dbm = (int16_t)(-(int16_t)rssi_raw / 2);
00378     s_radio.last_packet.snr_db = 0;
00379     s_radio.last_packet.timestamp_ms = HAL_GetTick();
00380     s_radio.last_packet.valid = true;
00381     return RADIO_OK;
00382 }
00383
00384 void radio_ook_default_hw_cfg(radio_hw_cfg_t *cfg, SPI_HandleTypeDef *hspi)
00385 {

```

```

00386     radio_lora_default_hw_cfg(cfg, hspi);
00387 }
00388
00389 void radio_ook_default_lora_cfg(radio_lora_cfg_t *cfg)
00390 {
00391     radio_lora_default_lora_cfg(cfg);
00392 }
00393
00394 radio_status_t radio_ook_set_runtime_cfg(const radio_ook_cfg_t *cfg)
00395 {
00396     if (!radio_cfg_valid(cfg))
00397     {
00398         return RADIO_EINVAL;
00399     }
00400
00401     s_runtime_cfg = *cfg;
00402     s_runtime_cfg_ready = true;
00403     return RADIO_OK;
00404 }
00405
00406 radio_status_t radio_ook_init(const radio_hw_cfg_t *hw,
00407                             const radio_lora_cfg_t *cfg,
00408                             radio_event_cb_t cb,
00409                             void *user_ctx)
00410 {
00411     uint8_t version;
00412
00413     (void)cfg;
00414
00415     if ((hw == NULL) || (hw->hspi == NULL) ||
00416         (hw->nss.port == NULL) || (hw->reset.port == NULL))
00417     {
00418         return RADIO_EINVAL;
00419     }
00420
00421     if (!s_runtime_cfg_ready)
00422     {
00423         radio_default_ook_cfg(&s_runtime_cfg);
00424         s_runtime_cfg_ready = true;
00425     }
00426
00427     if (!radio_cfg_valid(&s_runtime_cfg))
00428     {
00429         return RADIO_EINVAL;
00430     }
00431
00432     memset(&s_radio, 0, sizeof(s_radio));
00433     s_radio.hw = *hw;
00434     s_radio.cfg = s_runtime_cfg;
00435     s_radio.callback = cb;
00436     s_radio.callback_ctx = user_ctx;
00437     s_radio.bus.hspi = hw->hspi;
00438     s_radio.bus.nss_port = hw->nss.port;
00439     s_radio.bus.nss_pin = hw->nss.pin;
00440     s_radio.bus.spi_timeout_ms = hw->spi_timeout_ms;
00441     s_radio.state = RADIO_STATE_UNINIT;
00442
00443     radio_hw_reset();
00444
00445     if (!radio_set_op_mode_ready((uint8_t) (SX1276_OPMODE_MODULATION_OOK | SX1276_MODE_SLEEP)))
00446     {
00447         return RADIO_EHW;
00448     }
00449
00450     app_delay_ms(1U);
00451
00452     if (!sx1276_get_version(&s_radio.bus, &version) || (version != SX1276_VERSION_ID))
00453     {
00454         return RADIO_EHW;
00455     }
00456
00457     if (!radio_apply_ook_config(&s_radio.cfg))
00458     {
00459         return RADIO_EHW;
00460     }
00461
00462     s_radio.initialized = true;
00463     s_radio.state = RADIO_STATE_STANDBY;
00464     return RADIO_OK;
00465 }
00466
00467 radio_status_t radio_ook_deinit(void)
00468 {
00469     if (!s_radio.initialized)
00470     {
00471         return RADIO_ESTATE;
00472     }

```

```

00473
00474     (void)radio_ook_sleep();
00475     memset(&s_radio, 0, sizeof(s_radio));
00476     s_radio.state = RADIO_STATE_UNINIT;
00477     return RADIO_OK;
00478 }
00479
00480 radio_status_t radio_ook_start_rx_continuous(void)
00481 {
00482     if (!s_radio.initialized)
00483     {
00484         return RADIO_ESTATE;
00485     }
00486
00487     if (s_radio.state == RADIO_STATE_TX)
00488     {
00489         return RADIO_EBUS;
00490     }
00491
00492     if (!sx1276_write_reg(&s_radio.bus, SX1276_REG_DIO_MAPPING_1, 0x00U) ||
00493         !sx1276_set_op_mode(&s_radio.bus,
00494                             (uint8_t)(SX1276_OPMODE_MODULATION_OOK |
00495                                         SX1276_OPMODE_SHAPING_NONE |
00496                                         SX1276_MODE_RXCONTINUOUS)))
00497     {
00498         return RADIO_EHW;
00499     }
00500
00501     radio_set_state(RADIO_STATE_RX_CONT);
00502     return RADIO_OK;
00503 }
00504
00505 radio_status_t radio_ook_start_rx_single(uint16_t symbol_timeout)
00506 {
00507     if (!s_radio.initialized)
00508     {
00509         return RADIO_ESTATE;
00510     }
00511
00512     if (s_radio.state == RADIO_STATE_TX)
00513     {
00514         return RADIO_EBUS;
00515     }
00516
00517     if (symbol_timeout == 0U)
00518     {
00519         return RADIO_EINVAL;
00520     }
00521
00522     if (!sx1276_write_reg(&s_radio.bus, SX1276_REG_DIO_MAPPING_1, 0x00U) ||
00523         !sx1276_set_op_mode(&s_radio.bus,
00524                             (uint8_t)(SX1276_OPMODE_MODULATION_OOK |
00525                                         SX1276_OPMODE_SHAPING_NONE |
00526                                         SX1276_MODE_RXSINGLE)))
00527     {
00528         return RADIO_EHW;
00529     }
00530
00531     s_radio.rx_single_deadline_ms = HAL_GetTick() + radio_rx_single_timeout_ms(symbol_timeout);
00532     radio_set_state(RADIO_STATE_RX_SINGLE);
00533     return RADIO_OK;
00534 }
00535
00536 radio_status_t radio_ook_send_async(const uint8_t *data, uint8_t len)
00537 {
00538     uint8_t fifo_buf[RADIO_LIB_MAX_PAYLOAD + 1U];
00539
00540     if (!s_radio.initialized)
00541     {
00542         return RADIO_ESTATE;
00543     }
00544
00545     if ((data == NULL) || (len == 0U) || (len > RADIO_LIB_MAX_PAYLOAD))
00546     {
00547         return RADIO_EINVAL;
00548     }
00549
00550     if (((uint16_t)len + 1U) > RADIO_OOK_FIFO_MAX_BYTES)
00551     {
00552         /* Chunked FIFO TX is not implemented in this backend yet. */
00553         return RADIO_EINVAL;
00554     }
00555
00556     if (s_radio.state == RADIO_STATE_TX)
00557     {
00558         return RADIO_EBUS;
00559     }
00560
00561     fifo_buf[0] = len;

```

```

00560     memcpy(&fifo_buf[1], data, len);
00561     s_radio.tx_resume_state = s_radio.state;
00562
00563     if (!radio_set_op_mode_ready((uint8_t) (SX1276_OPMODE_MODULATION_OOK |
00564                                         SX1276_OPMODE_SHAPING_NONE |
00565                                         SX1276_MODE_STDBY))) ||
00566         !sx1276_write_reg(&s_radio.bus, SX1276_REG_DIO_MAPPING_1, 0x00U) ||
00567         !sx1276_write_burst(&s_radio.bus, SX1276_REG_FIFO, fifo_buf, (uint8_t)(len + 1U)) ||
00568         !sx1276_set_op_mode(&s_radio.bus,
00569                             (uint8_t) (SX1276_OPMODE_MODULATION_OOK |
00570                                         SX1276_OPMODE_SHAPING_NONE |
00571                                         SX1276_MODE_TX)))
00572     {
00573         return RADIO_EHW;
00574     }
00575
00576     {
00577         uint32_t key = radio_irq_save();
00578         s_radio.dio_pending_mask = 0U;
00579         radio_irq_restore(key);
00580     }
00581     radio_set_state(RADIO_STATE_TX);
00582     s_radio.tx_deadline_ms = HAL_GetTick() + radio_tx_timeout_ms(len);
00583     return RADIO_OK;
00584 }
00585
00586 radio_status_t radio_ook_standby(void)
00587 {
00588     if (!s_radio.initialized)
00589     {
00590         return RADIO_ESTATE;
00591     }
00592
00593     if (!radio_set_op_mode_ready((uint8_t) (SX1276_OPMODE_MODULATION_OOK |
00594                                         SX1276_OPMODE_SHAPING_NONE |
00595                                         SX1276_MODE_STDBY)))
00596     {
00597         return RADIO_EHW;
00598     }
00599
00600     radio_set_state(RADIO_STATE_STANDBY);
00601     return RADIO_OK;
00602 }
00603
00604 radio_status_t radio_ook_sleep(void)
00605 {
00606     if (!s_radio.initialized)
00607     {
00608         return RADIO_ESTATE;
00609     }
00610
00611     if (!radio_set_op_mode_ready((uint8_t) (SX1276_OPMODE_MODULATION_OOK |
00612                                         SX1276_OPMODE_SHAPING_NONE |
00613                                         SX1276_MODE_SLEEP)))
00614     {
00615         return RADIO_EHW;
00616     }
00617
00618     radio_set_state(RADIO_STATE_STANDBY);
00619     return RADIO_OK;
00620 }
00621
00622 void radio_ook_process(void)
00623 {
00624     uint8_t pending;
00625     uint8_t irq1 = 0U;
00626     uint8_t irq2 = 0U;
00627     uint32_t events = RADIO_EVENT_NONE;
00628     uint32_t key;
00629
00630     if (!s_radio.initialized)
00631     {
00632         return;
00633     }
00634
00635     key = radio_irq_save();
00636     pending = s_radio.dio_pending_mask;
00637     s_radio.dio_pending_mask = 0U;
00638     radio_irq_restore(key);
00639
00640     if ((pending == 0U) &&
00641         (s_radio.hw.dio[0].port != NULL) &&
00642         (HAL_GPIO_ReadPin(s_radio.hw.dio[0].port, s_radio.hw.dio[0].pin) == GPIO_PIN_SET))
00643     {
00644         pending |= 0x01U;
00645     }
00646

```



```

00647     if ((pending == 0U) && (s_radio.state != RADIO_STATE_TX))
00648     {
00649         if ((s_radio.state == RADIO_STATE_RX_SINGLE) &&
00650             ((int32_t)(HAL_GetTick() - s_radio.rx_single_deadline_ms) >= 0))
00651         {
00652             events |= RADIO_EVENT_RX_TIMEOUT;
00653             (void)radio_ook_standby();
00654         }
00655         else
00656         {
00657             return;
00658         }
00659     }
00660
00661     if ((pending == 0U) &&
00662         (s_radio.state == RADIO_STATE_TX) &&
00663         ((int32_t)(HAL_GetTick() - s_radio.tx_deadline_ms) >= 0))
00664     {
00665         (void)radio_ook_standby();
00666         radio_resume_after_tx();
00667         events |= RADIO_EVENT_HW_ERROR;
00668         goto done;
00669     }
00670
00671     if (!sx1276_read_reg(&s_radio.bus, SX1276_REG_IRQ_FLAGS_1, &irq1) ||
00672         !sx1276_read_reg(&s_radio.bus, SX1276_REG_IRQ_FLAGS_2, &irq2))
00673     {
00674         events |= RADIO_EVENT_HW_ERROR;
00675         goto done;
00676     }
00677
00678     if ((irq2 & SX1276_IRQ2_FIFO_OVERRUN) != 0U)
00679     {
00680         events |= RADIO_EVENT_FIFO_OVERRUN;
00681     }
00682
00683     if ((irq1 & SX1276_IRQ1_TIMEOUT) != 0U)
00684     {
00685         events |= RADIO_EVENT_RX_TIMEOUT;
00686         if (s_radio.state == RADIO_STATE_RX_SINGLE)
00687         {
00688             (void)radio_ook_standby();
00689         }
00690     }
00691
00692     if ((irq2 & SX1276_IRQ2_PAYLOAD_READY) != 0U)
00693     {
00694         radio_status_t status = radio_read_rx_packet();
00695         if (status == RADIO_OK)
00696         {
00697             events |= RADIO_EVENT_RX_DONE;
00698         }
00699         else if (status == RADIO_EBUS)
00700         {
00701             events |= RADIO_EVENT_FIFO_OVERRUN;
00702         }
00703         else
00704         {
00705             events |= RADIO_EVENT_HW_ERROR;
00706         }
00707
00708         if (s_radio.state == RADIO_STATE_RX_SINGLE)
00709         {
00710             (void)radio_ook_standby();
00711         }
00712     }
00713
00714     if ((irq2 & SX1276_IRQ2_PACKET_SENT) != 0U)
00715     {
00716         events |= RADIO_EVENT_TX_DONE;
00717         radio_resume_after_tx();
00718         s_radio.tx_deadline_ms = 0U;
00719     }
00720
00721 done:
00722     if (events != RADIO_EVENT_NONE)
00723     {
00724         radio_set_event_flags(events);
00725         if (s_radio.callback != NULL)
00726         {
00727             s_radio.callback(events, s_radio.callback_ctx);
00728         }
00729     }
00730 }
00731
00732 uint32_t radio_ook_take_events(void)
00733 {

```

```
00734     uint32_t events;
00735     uint32_t key = radio_irq_save();
00736
00737     events = s_radio.event_flags;
00738     s_radio.event_flags = 0U;
00739     radio_irq_restore(key);
00740     return events;
00741 }
00742
00743 bool radio_ook_get_last_packet(radio_packet_t *pkt)
00744 {
00745     if ((pkt == NULL) || !s_radio.last_packet.valid)
00746     {
00747         return false;
00748     }
00749
00750     *pkt = s_radio.last_packet;
00751     return true;
00752 }
00753
00754 radio_state_t radio_ook_get_state(void)
00755 {
00756     return s_radio.state;
00757 }
00758
00759 radio_status_t radio_ook_raw_read_reg(uint8_t reg, uint8_t *value)
00760 {
00761     if (!s_radio.initialized)
00762     {
00763         return RADIO_ESTATE;
00764     }
00765
00766     return sx1276_read_reg(&s_radio.bus, reg, value) ? RADIO_OK : RADIO_EHW;
00767 }
00768
00769 radio_status_t radio_ook_raw_write_reg(uint8_t reg, uint8_t value)
00770 {
00771     if (!s_radio.initialized)
00772     {
00773         return RADIO_ESTATE;
00774     }
00775
00776     return sx1276_write_reg(&s_radio.bus, reg, value) ? RADIO_OK : RADIO_EHW;
00777 }
00778
00779 radio_status_t radio_ook_raw_read_burst(uint8_t reg, uint8_t *data, uint8_t len)
00780 {
00781     if (!s_radio.initialized)
00782     {
00783         return RADIO_ESTATE;
00784     }
00785
00786     if ((data == NULL) || (len == 0U))
00787     {
00788         return RADIO_EINVAL;
00789     }
00790
00791     return sx1276_read_burst(&s_radio.bus, reg, data, len) ? RADIO_OK : RADIO_EHW;
00792 }
00793
00794 radio_status_t radio_ook_raw_write_burst(uint8_t reg, const uint8_t *data, uint8_t len)
00795 {
00796     if (!s_radio.initialized)
00797     {
00798         return RADIO_ESTATE;
00799     }
00800
00801     if ((data == NULL) || (len == 0U))
00802     {
00803         return RADIO_EINVAL;
00804     }
00805
00806     return sx1276_write_burst(&s_radio.bus, reg, data, len) ? RADIO_OK : RADIO_EHW;
00807 }
00808
00809 void radio_ook_on_exti(uint16_t gpio_pin)
00810 {
00811     radio_handle_exti_pin(gpio_pin);
00812 }
```

12.64 Dokumentacja pliku

Core/App/radio_lib/modulations/ook/radio_ook.h

Interfejs backendu modulacji OOK dla SX1276.

```
#include "../..../radio_lib.h"
```

Funkcje

- void [radio_ook_default_hw_cfg](#) ([radio_hw_cfg_t](#) *cfg, SPI_HandleTypeDef *hspi)
Domyślna konfiguracja sprzętowa backendu OOK.
- void [radio_ook_default_lora_cfg](#) ([radio_lora_cfg_t](#) *cfg)
Domyślna konfiguracja parametrów backendu OOK.
- [radio_status_t](#) [radio_ook_set_runtime_cfg](#) (const [radio_ook_cfg_t](#) *cfg)
Ustawia runtime konfigurację właściwą dla backendu OOK.
- [radio_status_t](#) [radio_ook_init](#) (const [radio_hw_cfg_t](#) *hw, const [radio_lora_cfg_t](#) *cfg, [radio_event_cb_t](#) cb, void *user_ctx)
Inicjalizacja backendu OOK.
- [radio_status_t](#) [radio_ook_deinit](#) (void)
Deinicjalizacja backendu OOK.
- [radio_status_t](#) [radio_ook_start_rx_continuous](#) (void)
Uruchomienie odbioru ciągłego OOK.
- [radio_status_t](#) [radio_ook_start_rx_single](#) (uint16_t symbol_timeout)
Uruchomienie odbioru pojedynczego OOK.
- [radio_status_t](#) [radio_ook_send_async](#) (const uint8_t *data, uint8_t len)
Uruchomienie nadawania asynchronicznego OOK.
- [radio_status_t](#) [radio_ook_standby](#) (void)
Przejsie do standby backendu OOK.
- [radio_status_t](#) [radio_ook_sleep](#) (void)
Przejsie do sleep backendu OOK.
- void [radio_ook_process](#) (void)
Przetwarzanie logiki odroczonej backendu OOK.
- uint32_t [radio_ook_take_events](#) (void)
Zwrócenie i wyczyszczenie flag zdarzeń backendu OOK.
- bool [radio_ook_get_last_packet](#) ([radio_packet_t](#) *pkt)
Odczyt ostatniego pakietu backendu OOK.
- [radio_state_t](#) [radio_ook_get_state](#) (void)
Odczyt stanu backendu OOK.
- [radio_status_t](#) [radio_ook_raw_read_reg](#) (uint8_t reg, uint8_t *value)
Odczyt pojedynczego rejestru (backend OOK).
- [radio_status_t](#) [radio_ook_raw_write_reg](#) (uint8_t reg, uint8_t value)
Zapis pojedynczego rejestru (backend OOK).
- [radio_status_t](#) [radio_ook_raw_read_burst](#) (uint8_t reg, uint8_t *data, uint8_t len)
Odczyt sekwencji rejestrów (backend OOK).
- [radio_status_t](#) [radio_ook_raw_write_burst](#) (uint8_t reg, const uint8_t *data, uint8_t len)
Zapis sekwencji rejestrów (backend OOK).
- void [radio_ook_on_exti](#) (uint16_t gpio_pin)
Przekazanie pinu EXTI do backendu OOK.

12.64.1 Opis szczegółowy

Interfejs backendu modulacji OOK dla SX1276.

Definicja w pliku [radio_ook.h](#).

12.64.2 Dokumentacja funkcji

12.64.2.1 radio_ook_default_hw_cfg()

```
void radio_ook_default_hw_cfg (
    radio_hw_cfg_t * cfg,
    SPI_HandleTypeDef * hspi)
```

Domyślna konfiguracja sprzętowa backendu OOK.

Definicja w linii 384 pliku [radio_ook.c](#).

Odwołuje się do [radio_lora_default_hw_cfg\(\)](#).

12.64.2.2 radio_ook_default_lora_cfg()

```
void radio_ook_default_lora_cfg (
    radio_lora_cfg_t * cfg)
```

Domyślna konfiguracja parametrów backendu OOK.

Definicja w linii 389 pliku [radio_ook.c](#).

Odwołuje się do [radio_lora_default_lora_cfg\(\)](#).

12.64.2.3 radio_ook_deinit()

```
radio_status_t radio_ook_deinit (
    void )
```

Deinicjalizacja backendu OOK.

Definicja w linii 467 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_ESTATE](#), [RADIO_OK](#), [radio_ook_sleep\(\)](#) i [RADIO_STATE_UNINIT](#).

Odwołania w [radio_deinit\(\)](#).

12.64.2.4 radio_ook_get_last_packet()

```
bool radio_ook_get_last_packet (
    radio_packet_t * pkt)
```

Odczyt ostatniego pakietu backendu OOK.

Definicja w linii 743 pliku [radio_ook.c](#).

Odwołania w [radio_get_last_packet\(\)](#).

12.64.2.5 radio_ook_get_state()

```
radio_state_t radio_ook_get_state (
    void )
```

Odczyt stanu backendu OOK.

Definicja w linii 754 pliku [radio_ook.c](#).

Odwołania w [radio_get_state\(\)](#).

12.64.2.6 radio_ook_init()

```
radio_status_t radio_ook_init (
    const radio_hw_cfg_t * hw,
    const radio_lora_cfg_t * cfg,
    radio_event_cb_t cb,
    void * user_ctx)
```

Inicjalizacja backendu OOK.

Definicja w linii 406 pliku [radio_ook.c](#).

Odwołuje się do [radio_hw_cfg_t::hspi](#), [radio_hw_cfg_t::nss](#), [radio_gpio_t::pin](#), [radio_gpio_t::port](#), [radio_apply_ook_config\(\)](#), [radio_cfg_valid\(\)](#), [radio_default_ook_cfg\(\)](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [radio_hw_reset\(\)](#), [RADIO_OK](#), [RADIO_STATE_STANDBY](#), [RADIO_STATE_UNINIT](#), [radio_hw_cfg_t::reset](#), [radio_hw_cfg_t::spi_timeout_ms](#) i [sx1276_get_version\(\)](#).

Odwołania w [radio_init\(\)](#).

12.64.2.7 radio_ook_on_exti()

```
void radio_ook_on_exti (
    uint16_t gpio_pin)
```

Przekazanie pinu EXTI do backendu OOK.

Definicja w linii 809 pliku [radio_ook.c](#).

Odwołuje się do [radio_handle_exti_pin\(\)](#).

Odwołania w [radio_on_exti\(\)](#).

12.64.2.8 radio_ook_process()

```
void radio_ook_process (
    void )
```

Przetwarzanie logiki odroczonej backendu OOK.

Definicja w linii 622 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EVENT_FIFO_OVERRUN](#), [RADIO_EVENT_HW_ERROR](#), [RADIO_EVENT_NONE](#), [RADIO_EVENT_RX_DONE](#), [RADIO_EVENT_RX_TIMEOUT](#), [RADIO_EVENT_TX_DONE](#), [radio_irq_restore\(\)](#), [radio_irq_save\(\)](#), [RADIO_OK](#), [radio_ook_standby\(\)](#), [radio_read_rx_packet\(\)](#), [radio_resume_after_tx\(\)](#), [radio_set_event_flags\(\)](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_TX](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_process\(\)](#).

12.64.2.9 radio_ook_raw_read_burst()

```
radio_status_t radio_ook_raw_read_burst (
    uint8_t reg,
    uint8_t * data,
    uint8_t len)
```

Odczyt sekwencji rejestrów (backend OOK).

Definicja w linii 779 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_read_burst\(\)](#).

Odwołania w [radio_raw_read_burst\(\)](#).

12.64.2.10 radio_ook_raw_read_reg()

```
radio_status_t radio_ook_raw_read_reg (
    uint8_t reg,
    uint8_t * value)
```

Odczyt pojedynczego rejestru (backend OOK).

Definicja w linii 759 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_read_reg\(\)](#).

Odwołania w [radio_raw_read_reg\(\)](#).

12.64.2.11 radio_ook_raw_write_burst()

```
radio_status_t radio_ook_raw_write_burst (
    uint8_t reg,
    const uint8_t * data,
    uint8_t len)
```

Zapis sekwencji rejestrów (backend OOK).

Definicja w linii 794 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_write_burst\(\)](#).

Odwołania w [radio_raw_write_burst\(\)](#).

12.64.2.12 radio_ook_raw_write_reg()

```
radio_status_t radio_ook_raw_write_reg (
    uint8_t reg,
    uint8_t value)
```

Zapis pojedynczego rejestru (backend OOK).

Definicja w linii 769 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_raw_write_reg\(\)](#).

12.64.2.13 [radio_ook_send_async\(\)](#)

```
radio_status_t radio_ook_send_async (  
    const uint8_t * data,  
    uint8_t len)
```

Uruchomienie nadawania asynchronicznego OOK.

Definicja w linii 535 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [radio_irq_restore\(\)](#), [radio_irq_save\(\)](#), [RADIO_LIB_MAX_PAYLOAD](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_TX](#), [radio_tx_timeout_ms\(\)](#), [sx1276_set_op_mode\(\)](#), [sx1276_write_burst\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_send_async\(\)](#).

12.64.2.14 [radio_ook_set_runtime_cfg\(\)](#)

```
radio_status_t radio_ook_set_runtime_cfg (  
    const radio_ook_cfg_t * cfg)
```

Ustawia runtime konfigurację właściwą dla backendu OOK.

Definicja w linii 394 pliku [radio_ook.c](#).

Odwołuje się do [radio_cfg_valid\(\)](#), [RADIO_EINVAL](#) i [RADIO_OK](#).

Odwołania w [radio_set_ook_cfg\(\)](#).

12.64.2.15 [radio_ook_sleep\(\)](#)

```
radio_status_t radio_ook_sleep (  
    void )
```

Przejsie do sleep backendu OOK.

Definicja w linii 604 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#) i [RADIO_STATE_STANDBY](#).

Odwołania w [radio_ook_deinit\(\)](#) i [radio_sleep\(\)](#).

12.64.2.16 [radio_ook_standby\(\)](#)

```
radio_status_t radio_ook_standby (  
    void )
```

Przejsie do standby backendu OOK.

Definicja w linii 586 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#) i [RADIO_STATE_STANDBY](#).

Odwołania w [radio_ook_process\(\)](#) i [radio_standby\(\)](#).

12.64.2.17 [radio_ook_start_rx_continuous\(\)](#)

```
radio_status_t radio_ook_start_rx_continuous (  
    void )
```

Uruchomienie odbioru ciągłego OOK.

Definicja w linii 480 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_CONT](#), [RADIO_STATE_TX](#), [sx1276_set_op_mode\(\)](#) i [sx1276_write_reg\(\)](#).

Odwołania w [radio_start_rx_continuous\(\)](#).

12.64.2.18 [radio_ook_start_rx_single\(\)](#)

```
radio_status_t radio_ook_start_rx_single (  
    uint16_t symbol_timeout)
```

Uruchomienie odbioru pojedynczego OOK.

Definicja w linii 505 pliku [radio_ook.c](#).

Odwołuje się do [RADIO_EBUS](#), [RADIO_EHW](#), [RADIO_EINVAL](#), [RADIO_ESTATE](#), [RADIO_OK](#), [radio_rx_single_timeout_ms\(\)](#), [radio_set_state\(\)](#), [RADIO_STATE_RX_SINGLE](#), [RADIO_STATE_TX](#), [sx1276_set_op_mode\(\)](#) i [sx1276_write_reg\(\)](#).
Odwołania w [radio_start_rx_single\(\)](#).

12.64.2.19 radio_ook_take_events()

```
uint32_t radio_ook_take_events (
    void )
```

Zwrócenie i wyczyszczenie flag zdarzeń backendu OOK.

Definicja w linii 732 pliku [radio_ook.c](#).

Odwołuje się do [radio_irq_restore\(\)](#) i [radio_irq_save\(\)](#).

Odwołania w [radio_take_events\(\)](#).

12.65 radio_ook.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_RADIO_LIB_MODULATIONS_OOK_RADIO_OOK_H_
00007 #define APP_RADIO_LIB_MODULATIONS_OOK_RADIO_OOK_H_
00008
00009 #include "../radio_lib.h"
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00016 void radio_ook_default_hw_cfg(radio_hw_cfg_t *cfg, SPI_HandleTypeDef *hspl);
00018 void radio_ook_default_lora_cfg(radio_lora_cfg_t *cfg);
00020 radio_status_t radio_ook_set_runtime_cfg(const radio_ook_cfg_t *cfg);
00022 radio_status_t radio_ook_init(const radio_hw_cfg_t *hw,
00023                             const radio_lora_cfg_t *cfg,
00024                             radio_event_cb_t cb,
00025                             void *user_ctx);
00027 radio_status_t radio_ook_deinit(void);
00029 radio_status_t radio_ook_start_rx_continuous(void);
00031 radio_status_t radio_ook_start_rx_single(uint16_t symbol_timeout);
00033 radio_status_t radio_ook_send_async(const uint8_t *data, uint8_t len);
00035 radio_status_t radio_ook_standby(void);
00037 radio_status_t radio_ook_sleep(void);
00039 void radio_ook_process(void);
00041 uint32_t radio_ook_take_events(void);
00043 bool radio_ook_get_last_packet(radio_packet_t *pkt);
00045 radio_state_t radio_ook_get_state(void);
00047 radio_status_t radio_ook_raw_read_reg(uint8_t reg, uint8_t *value);
00049 radio_status_t radio_ook_raw_write_reg(uint8_t reg, uint8_t value);
00051 radio_status_t radio_ook_raw_read_burst(uint8_t reg, uint8_t *data, uint8_t len);
00053 radio_status_t radio_ook_raw_write_burst(uint8_t reg, const uint8_t *data, uint8_t len);
00055 void radio_ook_on_exti(uint16_t gpio_pin);
00056
00057 #ifdef __cplusplus
00058 }
00059 #endif
00060
00061 #endif /* APP_RADIO_LIB_MODULATIONS_OOK_RADIO_OOK_H_ */
```

12.66 Dokumentacja pliku Core/App/radio_lib/radio_lib.c

Fasada biblioteki radia delegująca wywołania do backendu wybranego w runtime.

```
#include "radio_lib.h"
#include "modulations/fsk/radio_fsk.h"
#include "modulations/lora/radio_lora.h"
#include "modulations/ook/radio_ook.h"
#include <string.h>
```

Funkcje

- static uint8_t [radio_normalize_backend](#) (uint8_t modulation)
Internal helper: [radio_normalize_backend](#).

- void [radio_default_hw_cfg](#) ([radio_hw_cfg_t](#) *cfg, SPI_HandleTypeDef *hspi)
Wypełnia domyślną konfigurację sprzętową dla aktywnej modulacji.
- void [radio_default_lora_cfg](#) ([radio_lora_cfg_t](#) *cfg)
Wypełnia domyślny profil LoRa.
- void [radio_default_fsk_cfg](#) ([radio_fsk_cfg_t](#) *cfg)
Wypełnia domyślny profil FSK/GFSK/MSK/GMSK.
- void [radio_default_ook_cfg](#) ([radio_ook_cfg_t](#) *cfg)
Wypełnia domyślny profil OOK.
- void [radio_select_backend](#) (uint8_t modulation)
Wybiera backend modulacji używany przez wspólne API w runtime.
- uint8_t [radio_get_backend](#) (void)
Zwraca backend aktualnie wybrany do dispatchu runtime.
- [radio_status_t](#) [radio_set_fsk_cfg](#) (const [radio_fsk_cfg_t](#) *cfg)
Przekazuje konfigurację runtime dla backendu FSK.
- [radio_status_t](#) [radio_set_ook_cfg](#) (const [radio_ook_cfg_t](#) *cfg)
Przekazuje konfigurację runtime dla backendu OOK.
- [radio_status_t](#) [radio_init](#) (const [radio_hw_cfg_t](#) *hw, const [radio_lora_cfg_t](#) *cfg, [radio_event_cb_t](#) cb, void *user_ctx)
Inicjalizuje backend radiowy.
- [radio_status_t](#) [radio_deinit](#) (void)
Deinicjalizuje backend radiowy.
- [radio_status_t](#) [radio_start_rx_continuous](#) (void)
Uruchamia odbiór ciągły.
- [radio_status_t](#) [radio_start_rx_single](#) (uint16_t symbol_timeout)
Uruchamia odbiór pojedynczy z timeoutem symbolowym.
- [radio_status_t](#) [radio_send_async](#) (const uint8_t *data, uint8_t len)
Rozpoczyna nadawanie asynchroniczne.
- [radio_status_t](#) [radio_standby](#) (void)
Przełącza radio do trybu standby.
- [radio_status_t](#) [radio_sleep](#) (void)
Przełącza radio do trybu sleep.
- void [radio_process](#) (void)
Obsługuje odroczone logikę IRQ w kontekście pętli głównej.
- uint32_t [radio_take_events](#) (void)
Zwraca i czyści aktualną maskę zdarzeń.
- bool [radio_get_last_packet](#) ([radio_packet_t](#) *pkt)
Odczytuje ostatnio odebraną ramkę.
- [radio_state_t](#) [radio_get_state](#) (void)
Zwraca aktualny stan backendu.
- [radio_status_t](#) [radio_raw_read_reg](#) (uint8_t reg, uint8_t *value)
Odczytuje pojedynczy rejestr transceivera.
- [radio_status_t](#) [radio_raw_write_reg](#) (uint8_t reg, uint8_t value)
Zapisuje pojedynczy rejestr transceivera.
- [radio_status_t](#) [radio_raw_read_burst](#) (uint8_t reg, uint8_t *data, uint8_t len)
Odczytuje ciąg bajtów z rejestrów transceivera.
- [radio_status_t](#) [radio_raw_write_burst](#) (uint8_t reg, const uint8_t *data, uint8_t len)
Zapisuje ciąg bajtów do rejestrów transceivera.
- void [radio_on_exti](#) (uint16_t gpio_pin)
Przekazuje numer pinu EXTI do backendu radiowego.
- void [HAL_GPIO_EXTI_Callback](#) (uint16_t GPIO_Pin)

Zmienne

- static uint8_t `s_active_backend` = `RADIO_LIB_ACTIVE_MODULATION`

12.66.1 Opis szczegółowy

Fasada biblioteki radia delegująca wywołania do backendu wybranego w runtime.
Definicja w pliku [radio_lib.c](#).

12.66.2 Dokumentacja funkcji

12.66.2.1 HAL_GPIO_EXTI_Callback()

```
void HAL_GPIO_EXTI_Callback (
    uint16_t GPIO_Pin)
```

Definicja w linii 366 pliku [radio_lib.c](#).

12.66.2.2 radio_default_fsk_cfg()

```
void radio_default_fsk_cfg (
    radio_fsk_cfg_t * cfg)
```

Wypełnia domyślny profil FSK/GFSK/MSK/GMSK.

Parametry

<i>cfg</i>	[out] Struktura konfiguracji FSK do uzupełnienia.
------------	---

Definicja w linii 39 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_cfg_t::address_filter](#), [radio_fsk_cfg_t::bitrate_bps](#), [radio_fsk_cfg_t::crc_type](#), [radio_fsk_cfg_t::data_whitening](#), [radio_fsk_cfg_t::filter](#), [radio_fsk_cfg_t::frequency_hz](#), [radio_fsk_cfg_t::preamble_len](#), [RADIO_FSK_SHAPING_GFSK](#), [radio_fsk_cfg_t::rx_bandwidth](#), [radio_fsk_cfg_t::shaping](#), [radio_fsk_cfg_t::sync_word](#), [radio_fsk_cfg_t::sync_word_len](#) i [radio_fsk_cfg_t::tx_power_dbm](#).

Odwołania w [radio_fsk_init\(\)](#).

12.66.2.3 radio_default_hw_cfg()

```
void radio_default_hw_cfg (
    radio_hw_cfg_t * cfg,
    SPI_HandleTypeDef * hspi)
```

Wypełnia domyślną konfigurację sprzętową dla aktywnej modulacji.

Parametry

<i>cfg</i>	[out] Struktura konfiguracji do uzupełnienia.
<i>hspi</i>	Uchwyt SPI używany przez radio.

Definicja w linii 29 pliku [radio_lib.c](#).

Odwołuje się do [radio_lora_default_hw_cfg\(\)](#).

Odwołania w [radio_test_demo_init\(\)](#).

12.66.2.4 radio_default_lora_cfg()

```
void radio_default_lora_cfg (
    radio_lora_cfg_t * cfg)
```

Wypełnia domyślny profil LoRa.

Parametry

<i>cfg</i>	[out] Struktura konfiguracji LoRa do uzupełnienia.
------------	--

Definicja w linii 34 pliku [radio_lib.c](#).

Odwołuje się do [radio_lora_default_lora_cfg\(\)](#).

Odwołania w [radio_test_demo_init\(\)](#).

12.66.2.5 radio_default_ook_cfg()

```
void radio_default_ook_cfg (
    radio_ook_cfg_t * cfg)
```

Wypełnia domyślny profil OOK.

Parametry

<i>cfg</i>	[out] Struktura konfiguracji OOK do uzupełnienia.
------------	---

Definicja w linii 61 pliku [radio_lib.c](#).

Odwołuje się do [radio_ook_cfg_t::bitrate_bps](#), [radio_ook_cfg_t::frequency_hz](#), [radio_ook_cfg_t::preamble_len](#), [radio_ook_cfg_t::rx_bandwidth](#), [radio_ook_cfg_t::sync_word](#), [radio_ook_cfg_t::sync_word_len](#), [radio_ook_cfg_t::threshold](#), [radio_ook_cfg_t::threshold_value](#) i [radio_ook_cfg_t::tx_power_dbm](#).

Odwołania w [radio_ook_init\(\)](#).

12.66.2.6 radio_deinit()

```
radio_status_t radio_deinit (
    void )
```

Deinicjalizuje backend radiowy.

Zwraca

Kod statusu.

Definicja w linii 119 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_deinit\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_deinit\(\)](#) i [radio_ook_deinit\(\)](#).

12.66.2.7 radio_get_backend()

```
uint8_t radio_get_backend (
    void )
```

Zwraca backend aktualnie wybrany do dispatchu runtime.

Zwraca

Jedna ze stałych [RADIO_LIB_MODULATION_*](#).

Definicja w linii 85 pliku [radio_lib.c](#).

12.66.2.8 radio_get_last_packet()

```
bool radio_get_last_packet (
    radio_packet_t * pkt)
```

Odczytuje ostatnio odebraną ramkę.

Parametry

<i>pkt</i>	[out] Struktura wyjściowa pakietu.
------------	------------------------------------

Zwraca

`true` jeśli bufor zawiera poprawne dane.

Definicja w linii 250 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_get_last_packet()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_get_last_packet()` i `radio_ook_get_last_packet()`.

Odwołania w `radio_test_demo_process()`.

12.66.2.9 radio_get_state()

```
radio_state_t radio_get_state (
    void )
```

Zwraca aktualny stan backendu.

Zwraca

Stan pracy.

Definicja w linii 266 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_get_state()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_get_state()` i `radio_ook_get_state()`.

Odwołania w `radio_test_demo_process()` i `radio_test_dump_tx_debug()`.

12.66.2.10 radio_init()

```
radio_status_t radio_init (
    const radio_hw_cfg_t * hw,
    const radio_lora_cfg_t * cfg,
    radio_event_cb_t cb,
    void * user_ctx)
```

Inicjalizuje backend radiowy.

Parametry

<i>hw</i>	Konfiguracja sprzętowa.
<i>cfg</i>	Konfiguracja profilu LoRa.
<i>cb</i>	Callback zdarzeń (opcjonalny, może być NULL).
<i>user_ctx</i>	Kontekst użytkownika przekazywany do callbacku.

Zwraca

Kod statusu.

Definicja w linii 100 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_init()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_init()` i `radio_ook_init()`.

Odwołania w `radio_test_demo_init()`.

12.66.2.11 radio_normalize_backend()

```
uint8_t radio_normalize_backend (
    uint8_t modulation) [static]
```

Internal helper: `radio_normalize_backend`.

Definicja w linii 17 pliku `radio_lib.c`.

Odwołuje się do `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA` i `RADIO_LIB_MODULATION_OOK`.

Odwołania w `radio_select_backend()`.

12.66.2.12 radio_on_exti()

```
void radio_on_exti (
    uint16_t gpio_pin)
```

Przekazuje numer pinu EXTI do backendu radiowego.

Użyj tej funkcji, gdy `RADIO_LIB_OWNS_HAL_EXTI_CALLBACK == 0` i callback `HAL_GPIO_EXTI_↔` Callback jest implementowany poza biblioteką.

Parametry

<i>gpio_pin</i>	Numer/maska pinu przekazana przez HAL.
-----------------	--

Definicja w linii 346 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_on_exti()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_on_exti()` i `radio_ook_on_exti()`.

12.66.2.13 radio_process()

```
void radio_process (
    void )
```

Obsługuje odroczoną logikę IRQ w kontekście pętli głównej.

Definicja w linii 215 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_process()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_process()` i `radio_ook_process()`.

Odwołania w `radio_test_demo_process()`.

12.66.2.14 radio_raw_read_burst()

```
radio_status_t radio_raw_read_burst (
    uint8_t reg,
    uint8_t * data,
    uint8_t len)
```

Odczytuje ciąg bajtów z rejestrów transceivera.

Parametry

<i>reg</i>	Adres startowy.
<i>data</i>	[out] Bufor danych.
<i>len</i>	Liczba bajtów.

Zwraca

Kod statusu.

Definicja w linii 314 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_raw_read_burst()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_raw_read_burst()` i `radio_ook_raw_read_burst()`.

12.66.2.15 radio_raw_read_reg()

```
radio_status_t radio_raw_read_reg (
    uint8_t reg,
    uint8_t * value)
```

Odczytuje pojedynczy rejestr transceivera.

Parametry

<i>reg</i>	Adres rejestru.
<i>value</i>	[out] Odczytana wartość.

Zwraca

Kod statusu.

Definicja w linii 282 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_raw_read_reg\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_raw_read_reg\(\)](#) i [radio_ook_raw_read_reg\(\)](#).

Odwołania w [radio_test_dump_basic\(\)](#), [radio_test_dump_tx_debug\(\)](#) i [radio_test_probe\(\)](#).

12.66.2.16 radio_raw_write_burst()

```
radio_status_t radio_raw_write_burst (  
    uint8_t reg,  
    const uint8_t * data,  
    uint8_t len)
```

Zapisuje ciąg bajtów do rejestrów transceivera.

Parametry

<i>reg</i>	Adres startowy.
<i>data</i>	Bufor danych.
<i>len</i>	Liczba bajtów.

Zwraca

Kod statusu.

Definicja w linii 330 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_raw_write_burst\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_raw_write_burst\(\)](#) i [radio_ook_raw_write_burst\(\)](#).

12.66.2.17 radio_raw_write_reg()

```
radio_status_t radio_raw_write_reg (  
    uint8_t reg,  
    uint8_t value)
```

Zapisuje pojedynczy rejestr transceivera.

Parametry

<i>reg</i>	Adres rejestru.
<i>value</i>	Wartość do zapisu.

Zwraca

Kod statusu.

Definicja w linii 298 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_raw_write_reg\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_raw_write_reg\(\)](#) i [radio_ook_raw_write_reg\(\)](#).

12.66.2.18 radio_select_backend()

```
void radio_select_backend (
    uint8_t modulation)
```

Wybiera backend modulacji używany przez wspólne API w runtime.

Parametry

<i>modulation</i>	Jedna ze stałych <code>RADIO_LIB_MODULATION_*</code> .
-------------------	--

Definicja w linii 80 pliku `radio_lib.c`.

Odwołuje się do `radio_normalize_backend()`.

12.66.2.19 radio_send_async()

```
radio_status_t radio_send_async (
    const uint8_t * data,
    uint8_t len)
```

Rozpoczyna nadawanie asynchroniczne.

Parametry

<i>data</i>	Wskaźnik na payload.
<i>len</i>	Długość payloadu.

Zwraca

Kod statusu.

Definicja w linii 167 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_send_async()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_send_async()` i `radio_ook_send_async()`.

Odwołania w `radio_test_send_ping()`.

12.66.2.20 radio_set_fsk_cfg()

```
radio_status_t radio_set_fsk_cfg (
    const radio_fsk_cfg_t * cfg)
```

Przekazuje konfigurację runtime dla backendu FSK.

Parametry

<i>cfg</i>	Konfiguracja FSK do zapamiętania przed <code>radio_init(...)</code> .
------------	---

Zwraca

Kod statusu.

Definicja w linii 90 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_set_runtime_cfg()`.

12.66.2.21 radio_set_ook_cfg()

```
radio_status_t radio_set_ook_cfg (
    const radio_ook_cfg_t * cfg)
```

Przekazuje konfigurację runtime dla backendu OOK.

Parametry

<code>cfg</code>	Konfiguracja OOK do zapamiętania przed <code>radio_init(...)</code> .
------------------	---

Zwraca

Kod statusu.

Definicja w linii 95 pliku `radio_lib.c`.

Odwołuje się do `radio_ook_set_runtime_cfg()`.

12.66.2.22 `radio_sleep()`

```
radio_status_t radio_sleep (
    void )
```

Przełącza radio do trybu sleep.

Zwraca

Kod statusu.

Definicja w linii 199 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_sleep()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_sleep()` i `radio_ook_sleep()`.

12.66.2.23 `radio_standby()`

```
radio_status_t radio_standby (
    void )
```

Przełącza radio do trybu standby.

Zwraca

Kod statusu.

Definicja w linii 183 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_standby()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_standby()` i `radio_ook_standby()`.

Odwołania w `radio_test_demo_process()`.

12.66.2.24 `radio_start_rx_continuous()`

```
radio_status_t radio_start_rx_continuous (
    void )
```

Uruchamia odbiór ciągły.

Zwraca

Kod statusu.

Definicja w linii 135 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_start_rx_continuous()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_start_rx_continuous()` i `radio_ook_start_rx_continuous()`.

Odwołania w `radio_test_demo_init()` i `radio_test_demo_process()`.

12.66.2.25 `radio_start_rx_single()`

```
radio_status_t radio_start_rx_single (
    uint16_t symbol_timeout)
```

Uruchamia odbiór pojedynczy z timeoutem symbolowym.

Parametry

<i>symbol_timeout</i>	Timeout w symbolach.
-----------------------	----------------------

Zwraca

Kod statusu.

Definicja w linii 151 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_start_rx_single\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_start_rx_single\(\)](#) i [radio_ook_start_rx_single\(\)](#).

12.66.2.26 radio_take_events()

```
uint32_t radio_take_events (
    void )
```

Zwraca i czyści aktualną maskę zdarzeń.

Zwraca

Maska zdarzeń.

Definicja w linii 234 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_take_events\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_take_events\(\)](#) i [radio_ook_take_events\(\)](#).

Odwołania w [radio_test_demo_process\(\)](#).

12.66.3 Dokumentacja zmiennych**12.66.3.1 s_active_backend**

```
uint8_t s_active_backend = RADIO_LIB_ACTIVE_MODULATION [static]
```

Definicja w linii 14 pliku [radio_lib.c](#).

12.67 radio_lib.c

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #include "radio_lib.h"
00007
00008 #include "modulations/fsk/radio_fsk.h"
00009 #include "modulations/lora/radio_lora.h"
00010 #include "modulations/ook/radio_ook.h"
00011
00012 #include <string.h>
00013
00014 static uint8_t s_active_backend = RADIO_LIB_ACTIVE_MODULATION;
00015
00017 static uint8_t radio_normalize_backend(uint8_t modulation)
00018 {
00019     if ((modulation == RADIO_LIB_MODULATION_LORA) ||
00020         (modulation == RADIO_LIB_MODULATION_FSK) ||
00021         (modulation == RADIO_LIB_MODULATION_OOK))
00022     {
00023         return modulation;
00024     }
00025     return RADIO_LIB_MODULATION_LORA;
00026 }
00027
00028
00029 void radio_default_hw_cfg(radio_hw_cfg_t *cfg, SPI_HandleTypeDef *hspi)
00030 {
00031     radio_lora_default_hw_cfg(cfg, hspi);
00032 }
00033
00034 void radio_default_lora_cfg(radio_lora_cfg_t *cfg)
00035 {
00036     radio_lora_default_lora_cfg(cfg);
```



```

00037 }
00038
00039 void radio_default_fsk_cfg(radio_fsk_cfg_t *cfg)
00040 {
00041     if (cfg == NULL)
00042     {
00043         return;
00044     }
00045
00046     memset(cfg, 0, sizeof(*cfg));
00047     cfg->frequency_hz = 868300000UL;
00048     cfg->bitrate_bps = 4800UL;
00049     cfg->rx_bandwidth = RADIO_LORA_BW_125_KHZ;
00050     cfg->shaping = RADIO_FSK_SHAPING_GFSK;
00051     cfg->filter = RADIO_FSK_FILTER_BT_05;
00052     cfg->tx_power_dbm = 14;
00053     cfg->preamble_len = 8U;
00054     cfg->sync_word_len = 2U;
00055     cfg->sync_word = 0x00002DD4UL;
00056     cfg->address_filter = RADIO_ADDRESS_FILTER_OFF;
00057     cfg->crc_type = RADIO_PACKET_CRC_CCITT;
00058     cfg->data_whitening = true;
00059 }
00060
00061 void radio_default_ook_cfg(radio_ook_cfg_t *cfg)
00062 {
00063     if (cfg == NULL)
00064     {
00065         return;
00066     }
00067
00068     memset(cfg, 0, sizeof(*cfg));
00069     cfg->frequency_hz = 868500000UL;
00070     cfg->bitrate_bps = 4800UL;
00071     cfg->rx_bandwidth = RADIO_LORA_BW_125_KHZ;
00072     cfg->tx_power_dbm = 10;
00073     cfg->preamble_len = 8U;
00074     cfg->sync_word_len = 2U;
00075     cfg->sync_word = 0x00002DD4UL;
00076     cfg->threshold = RADIO_OOK_THRESHOLD_PEAK;
00077     cfg->threshold_value = 12U;
00078 }
00079
00080 void radio_select_backend(uint8_t modulation)
00081 {
00082     s_active_backend = radio_normalize_backend(modulation);
00083 }
00084
00085 uint8_t radio_get_backend(void)
00086 {
00087     return s_active_backend;
00088 }
00089
00090 radio_status_t radio_set_fsk_cfg(const radio_fsk_cfg_t *cfg)
00091 {
00092     return radio_fsk_set_runtime_cfg(cfg);
00093 }
00094
00095 radio_status_t radio_set_ook_cfg(const radio_ook_cfg_t *cfg)
00096 {
00097     return radio_ook_set_runtime_cfg(cfg);
00098 }
00099
00100 radio_status_t radio_init(const radio_hw_cfg_t *hw,
00101                          const radio_lora_cfg_t *cfg,
00102                          radio_event_cb_t cb,
00103                          void *user_ctx)
00104 {
00105     switch (s_active_backend)
00106     {
00107         case RADIO_LIB_MODULATION_FSK:
00108             return radio_fsk_init(hw, cfg, cb, user_ctx);
00109
00110         case RADIO_LIB_MODULATION_OOK:
00111             return radio_ook_init(hw, cfg, cb, user_ctx);
00112
00113         case RADIO_LIB_MODULATION_LORA:
00114             default:
00115                 return radio_lora_init(hw, cfg, cb, user_ctx);
00116     }
00117 }
00118
00119 radio_status_t radio_deinit(void)
00120 {
00121     switch (s_active_backend)
00122     {
00123         case RADIO_LIB_MODULATION_FSK:

```

```
00124         return radio_fsk_deinit();
00125
00126     case RADIO_LIB_MODULATION_OOK:
00127         return radio_ook_deinit();
00128
00129     case RADIO_LIB_MODULATION_LORA:
00130     default:
00131         return radio_lora_deinit();
00132     }
00133 }
00134
00135 radio_status_t radio_start_rx_continuous(void)
00136 {
00137     switch (s_active_backend)
00138     {
00139         case RADIO_LIB_MODULATION_FSK:
00140             return radio_fsk_start_rx_continuous();
00141
00142         case RADIO_LIB_MODULATION_OOK:
00143             return radio_ook_start_rx_continuous();
00144
00145         case RADIO_LIB_MODULATION_LORA:
00146         default:
00147             return radio_lora_start_rx_continuous();
00148     }
00149 }
00150
00151 radio_status_t radio_start_rx_single(uint16_t symbol_timeout)
00152 {
00153     switch (s_active_backend)
00154     {
00155         case RADIO_LIB_MODULATION_FSK:
00156             return radio_fsk_start_rx_single(symbol_timeout);
00157
00158         case RADIO_LIB_MODULATION_OOK:
00159             return radio_ook_start_rx_single(symbol_timeout);
00160
00161         case RADIO_LIB_MODULATION_LORA:
00162         default:
00163             return radio_lora_start_rx_single(symbol_timeout);
00164     }
00165 }
00166
00167 radio_status_t radio_send_async(const uint8_t *data, uint8_t len)
00168 {
00169     switch (s_active_backend)
00170     {
00171         case RADIO_LIB_MODULATION_FSK:
00172             return radio_fsk_send_async(data, len);
00173
00174         case RADIO_LIB_MODULATION_OOK:
00175             return radio_ook_send_async(data, len);
00176
00177         case RADIO_LIB_MODULATION_LORA:
00178         default:
00179             return radio_lora_send_async(data, len);
00180     }
00181 }
00182
00183 radio_status_t radio_standby(void)
00184 {
00185     switch (s_active_backend)
00186     {
00187         case RADIO_LIB_MODULATION_FSK:
00188             return radio_fsk_standby();
00189
00190         case RADIO_LIB_MODULATION_OOK:
00191             return radio_ook_standby();
00192
00193         case RADIO_LIB_MODULATION_LORA:
00194         default:
00195             return radio_lora_standby();
00196     }
00197 }
00198
00199 radio_status_t radio_sleep(void)
00200 {
00201     switch (s_active_backend)
00202     {
00203         case RADIO_LIB_MODULATION_FSK:
00204             return radio_fsk_sleep();
00205
00206         case RADIO_LIB_MODULATION_OOK:
00207             return radio_ook_sleep();
00208
00209         case RADIO_LIB_MODULATION_LORA:
00210         default:
```

```
00211         return radio_lora_sleep();
00212     }
00213 }
00214
00215 void radio_process(void)
00216 {
00217     switch (s_active_backend)
00218     {
00219         case RADIO_LIB_MODULATION_FSK:
00220             radio_fsk_process();
00221             break;
00222
00223         case RADIO_LIB_MODULATION_OOK:
00224             radio_ook_process();
00225             break;
00226
00227         case RADIO_LIB_MODULATION_LORA:
00228         default:
00229             radio_lora_process();
00230             break;
00231     }
00232 }
00233
00234 uint32_t radio_take_events(void)
00235 {
00236     switch (s_active_backend)
00237     {
00238         case RADIO_LIB_MODULATION_FSK:
00239             return radio_fsk_take_events();
00240
00241         case RADIO_LIB_MODULATION_OOK:
00242             return radio_ook_take_events();
00243
00244         case RADIO_LIB_MODULATION_LORA:
00245         default:
00246             return radio_lora_take_events();
00247     }
00248 }
00249
00250 bool radio_get_last_packet(radio_packet_t *pkt)
00251 {
00252     switch (s_active_backend)
00253     {
00254         case RADIO_LIB_MODULATION_FSK:
00255             return radio_fsk_get_last_packet(pkt);
00256
00257         case RADIO_LIB_MODULATION_OOK:
00258             return radio_ook_get_last_packet(pkt);
00259
00260         case RADIO_LIB_MODULATION_LORA:
00261         default:
00262             return radio_lora_get_last_packet(pkt);
00263     }
00264 }
00265
00266 radio_state_t radio_get_state(void)
00267 {
00268     switch (s_active_backend)
00269     {
00270         case RADIO_LIB_MODULATION_FSK:
00271             return radio_fsk_get_state();
00272
00273         case RADIO_LIB_MODULATION_OOK:
00274             return radio_ook_get_state();
00275
00276         case RADIO_LIB_MODULATION_LORA:
00277         default:
00278             return radio_lora_get_state();
00279     }
00280 }
00281
00282 radio_status_t radio_raw_read_reg(uint8_t reg, uint8_t *value)
00283 {
00284     switch (s_active_backend)
00285     {
00286         case RADIO_LIB_MODULATION_FSK:
00287             return radio_fsk_raw_read_reg(reg, value);
00288
00289         case RADIO_LIB_MODULATION_OOK:
00290             return radio_ook_raw_read_reg(reg, value);
00291
00292         case RADIO_LIB_MODULATION_LORA:
00293         default:
00294             return radio_lora_raw_read_reg(reg, value);
00295     }
00296 }
00297
```

```

00298 radio_status_t radio_raw_write_reg(uint8_t reg, uint8_t value)
00299 {
00300     switch (s_active_backend)
00301     {
00302         case RADIO_LIB_MODULATION_FSK:
00303             return radio_fsk_raw_write_reg(reg, value);
00304
00305         case RADIO_LIB_MODULATION_OOK:
00306             return radio_ook_raw_write_reg(reg, value);
00307
00308         case RADIO_LIB_MODULATION_LORA:
00309         default:
00310             return radio_lora_raw_write_reg(reg, value);
00311     }
00312 }
00313
00314 radio_status_t radio_raw_read_burst(uint8_t reg, uint8_t *data, uint8_t len)
00315 {
00316     switch (s_active_backend)
00317     {
00318         case RADIO_LIB_MODULATION_FSK:
00319             return radio_fsk_raw_read_burst(reg, data, len);
00320
00321         case RADIO_LIB_MODULATION_OOK:
00322             return radio_ook_raw_read_burst(reg, data, len);
00323
00324         case RADIO_LIB_MODULATION_LORA:
00325         default:
00326             return radio_lora_raw_read_burst(reg, data, len);
00327     }
00328 }
00329
00330 radio_status_t radio_raw_write_burst(uint8_t reg, const uint8_t *data, uint8_t len)
00331 {
00332     switch (s_active_backend)
00333     {
00334         case RADIO_LIB_MODULATION_FSK:
00335             return radio_fsk_raw_write_burst(reg, data, len);
00336
00337         case RADIO_LIB_MODULATION_OOK:
00338             return radio_ook_raw_write_burst(reg, data, len);
00339
00340         case RADIO_LIB_MODULATION_LORA:
00341         default:
00342             return radio_lora_raw_write_burst(reg, data, len);
00343     }
00344 }
00345
00346 void radio_on_exti(uint16_t gpio_pin)
00347 {
00348     switch (s_active_backend)
00349     {
00350         case RADIO_LIB_MODULATION_FSK:
00351             radio_fsk_on_exti(gpio_pin);
00352             break;
00353
00354         case RADIO_LIB_MODULATION_OOK:
00355             radio_ook_on_exti(gpio_pin);
00356             break;
00357
00358         case RADIO_LIB_MODULATION_LORA:
00359         default:
00360             radio_lora_on_exti(gpio_pin);
00361             break;
00362     }
00363 }
00364
00365 #if RADIO_LIB_OWNS_HAL_EXTI_CALLBACK
00366 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
00367 {
00368     radio_on_exti(GPIO_Pin);
00369 }
00370 #endif

```

12.68 Dokumentacja pliku Core/App/radio_lib/radio_lib.h

Główne API biblioteki radiowej.

```

#include "radio_lib_config.h"
#include "stm32u5xx_hal.h"
#include <stdbool.h>
#include <stdint.h>

```

Struktury danych

- struct [radio_gpio_t](#)
Opis pojedynczego pinu GPIO.
- struct [radio_hw_cfg_t](#)
Konfiguracja sprzętowa modułu radiowego.
- struct [radio_lora_cfg_t](#)
Konfiguracja profilu LoRa.
- struct [radio_fsk_cfg_t](#)
Konfiguracja backendu FSK/GFSK/MSK/GMSK.
- struct [radio_ook_cfg_t](#)
Konfiguracja backendu OOK.
- struct [radio_packet_t](#)
Bufor ostatnio odebranej ramki.

Definicje typów

- typedef void(* [radio_event_cb_t](#)) (uint32_t events, void *user_ctx)
Typ funkcji callback dla zdarzeń radiowych.

Wyliczenia

- enum [radio_status_t](#) {
 [RADIO_OK](#) = 0 , [RADIO_EINVAL](#) = -1 , [RADIO_EBUS](#) = -2 , [RADIO_ETIMEOUT](#) = -3 ,
 [RADIO_ESTATE](#) = -4 , [RADIO_EHW](#) = -5 }
Kody statusu zwracane przez API radia.
- enum [radio_state_t](#) {
 [RADIO_STATE_UNINIT](#) = 0 , [RADIO_STATE_STANDBY](#) , [RADIO_STATE_RX_CONT](#) , [RADIO_STATE_RX_SINGLE](#)
 , [RADIO_STATE_TX](#) }
Aktualny stan pracy backendu radiowego.
- enum [radio_event_mask_t](#) {
 [RADIO_EVENT_NONE](#) = 0U , [RADIO_EVENT_TX_DONE](#) = (1UL << 0) , [RADIO_EVENT_RX_DONE](#) =
 (1UL << 1) , [RADIO_EVENT_RX_TIMEOUT](#) = (1UL << 2) ,
 [RADIO_EVENT_CRC_ERR](#) = (1UL << 3) , [RADIO_EVENT_CAD_DONE](#) = (1UL << 4) , [RADIO_EVENT_CAD_DETECTED](#)
 = (1UL << 5) , [RADIO_EVENT_FIFO_OVERRUN](#) = (1UL << 6) ,
 [RADIO_EVENT_HW_ERROR](#) = (1UL << 7) }
Maska zdarzeń generowanych przez backend.
- enum [radio_lora_bw_t](#) {
 [RADIO_LORA_BW_7_8_KHZ](#) = 0U , [RADIO_LORA_BW_10_4_KHZ](#) = 1U , [RADIO_LORA_BW_15_6_KHZ](#) = 2U , [RADIO_LORA_BW_20_8_KHZ](#) = 3U ,
 [RADIO_LORA_BW_31_25_KHZ](#) = 4U , [RADIO_LORA_BW_41_7_KHZ](#) = 5U , [RADIO_LORA_BW_62_5_KHZ](#) = 6U , [RADIO_LORA_BW_125_KHZ](#) = 7U ,
 [RADIO_LORA_BW_250_KHZ](#) = 8U , [RADIO_LORA_BW_500_KHZ](#) = 9U }
Kody szerokości pasma LoRa.
- enum [radio_fsk_shaping_t](#) { [RADIO_FSK_SHAPING_FSK](#) = 0U , [RADIO_FSK_SHAPING_GFSK](#) ,
 [RADIO_FSK_SHAPING_MSK](#) , [RADIO_FSK_SHAPING_GMSK](#) }
Kształtowanie widma dla rodziny FSK.
- enum [radio_fsk_filter_t](#) {
 [RADIO_FSK_FILTER_NONE](#) = 0U , [RADIO_FSK_FILTER_BT_10](#) , [RADIO_FSK_FILTER_BT_07](#) ,
 [RADIO_FSK_FILTER_BT_05](#) ,
 [RADIO_FSK_FILTER_BT_03](#) }
Ustawienie filtra BT dla wariantów gaussowskich FSK.
- enum [radio_packet_crc_t](#) { [RADIO_PACKET_CRC_OFF](#) = 0U , [RADIO_PACKET_CRC_CCITT](#) , [RADIO_PACKET_CRC_IBM](#) }

Typ CRC pakietowego dla backendów packet-mode.

- enum `radio_address_filter_t` { `RADIO_ADDRESS_FILTER_OFF` = 0U , `RADIO_ADDRESS_FILTER_NODE` , `RADIO_ADDRESS_FILTER_NODE_BROADCAST` }

Tryb filtrowania adresowego dla packet engine FSK/OOK.

- enum `radio_ook_threshold_t` { `RADIO_OOK_THRESHOLD_FIXED` = 0U , `RADIO_OOK_THRESHOLD_↵` `PEAK` , `RADIO_OOK_THRESHOLD_AVERAGE` }

Tryb progu detekcji OOK.

Funkcje

- void `radio_default_hw_cfg` (`radio_hw_cfg_t` *cfg, `SPI_HandleTypeDef` *hspi)
Wypełnia domyślną konfigurację sprzętową dla aktywnej modulacji.
- void `radio_default_lora_cfg` (`radio_lora_cfg_t` *cfg)
Wypełnia domyślny profil LoRa.
- void `radio_default_fsk_cfg` (`radio_fsk_cfg_t` *cfg)
Wypełnia domyślny profil FSK/GFSK/MSK/GMSK.
- void `radio_default_ook_cfg` (`radio_ook_cfg_t` *cfg)
Wypełnia domyślny profil OOK.
- void `radio_select_backend` (`uint8_t` modulation)
Wybiera backend modulacji używany przez wspólne API w runtime.
- `uint8_t` `radio_get_backend` (void)
Zwraca backend aktualnie wybrany do dispatchu runtime.
- `radio_status_t` `radio_set_fsk_cfg` (const `radio_fsk_cfg_t` *cfg)
Przekazuje konfigurację runtime dla backendu FSK.
- `radio_status_t` `radio_set_ook_cfg` (const `radio_ook_cfg_t` *cfg)
Przekazuje konfigurację runtime dla backendu OOK.
- `radio_status_t` `radio_init` (const `radio_hw_cfg_t` *hw, const `radio_lora_cfg_t` *cfg, `radio_event_cb_t` cb, void *user_ctx)
Inicjalizuje backend radiowy.
- `radio_status_t` `radio_deinit` (void)
Deinicjalizuje backend radiowy.
- `radio_status_t` `radio_start_rx_continuous` (void)
Uruchamia odbiór ciągły.
- `radio_status_t` `radio_start_rx_single` (`uint16_t` symbol_timeout)
Uruchamia odbiór pojedynczy z timeoutem symbolowym.
- `radio_status_t` `radio_send_async` (const `uint8_t` *data, `uint8_t` len)
Rozpoczyna nadawanie asynchroniczne.
- `radio_status_t` `radio_standby` (void)
Przełącza radio do trybu standby.
- `radio_status_t` `radio_sleep` (void)
Przełącza radio do trybu sleep.
- void `radio_process` (void)
Obsługuje odroczoną logikę IRQ w kontekście pętli głównej.
- `uint32_t` `radio_take_events` (void)
Zwraca i czyści aktualną maskę zdarzeń.
- bool `radio_get_last_packet` (`radio_packet_t` *pkt)
Odczytuje ostatnio odebraną ramkę.
- `radio_state_t` `radio_get_state` (void)
Zwraca aktualny stan backendu.
- `radio_status_t` `radio_raw_read_reg` (`uint8_t` reg, `uint8_t` *value)
Odczytuje pojedynczy rejestr transceivera.
- `radio_status_t` `radio_raw_write_reg` (`uint8_t` reg, `uint8_t` value)

Zapisuje pojedynczy rejestr transceivera.

- `radio_status_t radio_raw_read_burst` (`uint8_t reg`, `uint8_t *data`, `uint8_t len`)

Odczytuje ciąg bajtów z rejestrów transceivera.

- `radio_status_t radio_raw_write_burst` (`uint8_t reg`, `const uint8_t *data`, `uint8_t len`)

Zapisuje ciąg bajtów do rejestrów transceivera.

- `void radio_on_exti` (`uint16_t gpio_pin`)

Przekazuje numer pinu EXTI do backendu radiowego.

12.68.1 Opis szczegółowy

Główne API biblioteki radiowej.

Ten plik udostępnia jednolite API dla aplikacji. Domyślny backend nadal może być wskazany kompilacyjnie przez `radio_lib_config.h`, ale warstwa aplikacji może go również przełączyć w runtime przez `radio_select_backend(...)`.

Definicja w pliku `radio_lib.h`.

12.68.2 Dokumentacja definicji typów

12.68.2.1 radio_event_cb_t

```
typedef void(* radio_event_cb_t) (uint32_t events, void *user_ctx)
```

Typ funkcji callback dla zdarzeń radiowych.

Parametry

<code>events</code>	Maska zdarzeń (<code>radio_event_mask_t</code>).
<code>user_ctx</code>	Wskaźnik kontekstu użytkownika przekazany w <code>radio_init</code> .

Definicja w linii 227 pliku `radio_lib.h`.

12.68.3 Dokumentacja typów wyliczanych

12.68.3.1 radio_address_filter_t

```
enum radio_address_filter_t
```

Tryb filtrowania adresowego dla packet engine FSK/OOK.

Definicja w linii 157 pliku `radio_lib.h`.

12.68.3.2 radio_event_mask_t

```
enum radio_event_mask_t
```

Maska zdarzeń generowanych przez backend.

Wartości wyliczeń

<code>RADIO_EVENT_NONE</code>	Brak zdarzeń.
<code>RADIO_EVENT_TX_DONE</code>	Zakończono nadawanie.
<code>RADIO_EVENT_RX_DONE</code>	Odebrano ramkę.
<code>RADIO_EVENT_RX_TIMEOUT</code>	Timeout odbioru.
<code>RADIO_EVENT_CRC_ERR</code>	Błąd CRC ramki.
<code>RADIO_EVENT_CAD_DONE</code>	Zakończono CAD.
<code>RADIO_EVENT_CAD_DETECTED</code>	CAD wykrył aktywność.
<code>RADIO_EVENT_FIFO_OVERRUN</code>	Przepełnienie/niepoprawna długość FIFO.

RADIO_EVENT_HW_ERROR	Błąd sprzętowy backendu.
----------------------	--------------------------

Definicja w linii 52 pliku [radio_lib.h](#).

12.68.3.3 radio_fsk_filter_t

enum [radio_fsk_filter_t](#)

Ustawienie filtra BT dla wariantów gaussowskich FSK.

Definicja w linii 135 pliku [radio_lib.h](#).

12.68.3.4 radio_fsk_shaping_t

enum [radio_fsk_shaping_t](#)

Kształtowanie widma dla rodziny FSK.

Wartości wyliczeń

RADIO_FSK_SHAPING_FSK	Klasyczne FSK bez filtra gaussowskiego.
RADIO_FSK_SHAPING_GFSK	GFSK z filtrem gaussowskim.
RADIO_FSK_SHAPING_MSK	MSK.
RADIO_FSK_SHAPING_GMSK	GMSK.

Definicja w linii 124 pliku [radio_lib.h](#).

12.68.3.5 radio_lora_bw_t

enum [radio_lora_bw_t](#)

Kody szerokości pasma LoRa.

Definicja w linii 68 pliku [radio_lib.h](#).

12.68.3.6 radio_ook_threshold_t

enum [radio_ook_threshold_t](#)

Tryb progu detekcji OOK.

Definicja w linii 167 pliku [radio_lib.h](#).

12.68.3.7 radio_packet_crc_t

enum [radio_packet_crc_t](#)

Typ CRC pakietowego dla backendów packet-mode.

Definicja w linii 147 pliku [radio_lib.h](#).

12.68.3.8 radio_state_t

enum [radio_state_t](#)

Aktualny stan pracy backendu radiowego.

Wartości wyliczeń

RADIO_STATE_UNINIT	Backend nie został zainicjalizowany.
RADIO_STATE_STANDBY	Tryb gotowości.
RADIO_STATE_RX_CONT	Odbiór ciągły aktywny.
RADIO_STATE_RX_SINGLE	Odbiór pojedynczy aktywny.
RADIO_STATE_TX	Nadawanie aktywne.

Definicja w linii 40 pliku [radio_lib.h](#).

12.68.3.9 radio_status_t

enum [radio_status_t](#)

Kody statusu zwracane przez API radia.

Wartości wyliczeń

RADIO_OK	Operacja zakończona poprawnie.
RADIO_EINVAL	Niepoprawny argument wejściowy.
RADIO_EBUS	Zasób zajęty lub niepoprawny stan wykonania.
RADIO_ETIMEOUT	Przekroczono limit czasu operacji.
RADIO_ESTATE	Niepoprawny stan cyklu życia (np. brak init).
RADIO_EHW	Błąd warstwy sprzętowej (SPI/GPIO/rejestry).

Definicja w linii 27 pliku [radio_lib.h](#).

12.68.4 Dokumentacja funkcji

12.68.4.1 radio_default_fsk_cfg()

```
void radio_default_fsk_cfg (
    radio_fsk_cfg_t * cfg)
```

Wypełnia domyślny profil FSK/GFSK/MSK/GMSK.

Parametry

<i>cfg</i>	[out] Struktura konfiguracji FSK do uzupełnienia.
------------	---

Definicja w linii 39 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_cfg_t::address_filter](#), [radio_fsk_cfg_t::bitrate_bps](#), [radio_fsk_cfg_t::crc_type](#), [radio_fsk_cfg_t::data_whiteni](#), [radio_fsk_cfg_t::filter](#), [radio_fsk_cfg_t::frequency_hz](#), [radio_fsk_cfg_t::preamble_len](#), [RADIO_FSK_SHAPING_GFSK](#), [radio_fsk_cfg_t::rx_bandwidth](#), [radio_fsk_cfg_t::shaping](#), [radio_fsk_cfg_t::sync_word](#), [radio_fsk_cfg_t::sync_word_len](#) i [radio_fsk_cfg_t::tx_power_dbm](#).

Odwołania w [radio_fsk_init\(\)](#).

12.68.4.2 radio_default_hw_cfg()

```
void radio_default_hw_cfg (
    radio_hw_cfg_t * cfg,
    SPI_HandleTypeDef * hspi)
```

Wypełnia domyślną konfigurację sprzętową dla aktywnej modulacji.

Parametry

<i>cfg</i>	[out] Struktura konfiguracji do uzupełnienia.
<i>hspi</i>	Uchwyt SPI używany przez radio.

Definicja w linii 29 pliku [radio_lib.c](#).

Odwołuje się do [radio_lora_default_hw_cfg\(\)](#).

Odwołania w [radio_test_demo_init\(\)](#).

12.68.4.3 radio_default_lora_cfg()

```
void radio_default_lora_cfg (
    radio_lora_cfg_t * cfg)
```

Wypełnia domyślny profil LoRa.

Parametry

<i>cfg</i>	[out] Struktura konfiguracji LoRa do uzupełnienia.
------------	--

Definicja w linii 34 pliku [radio_lib.c](#).

Odwołuje się do [radio_lora_default_lora_cfg\(\)](#).

Odwołania w [radio_test_demo_init\(\)](#).

12.68.4.4 radio_default_ook_cfg()

```
void radio_default_ook_cfg (
    radio_ook_cfg_t * cfg)
```

Wypełnia domyślny profil OOK.

Parametry

<i>cfg</i>	[out] Struktura konfiguracji OOK do uzupełnienia.
------------	---

Definicja w linii 61 pliku [radio_lib.c](#).

Odwołuje się do [radio_ook_cfg_t::bitrate_bps](#), [radio_ook_cfg_t::frequency_hz](#), [radio_ook_cfg_t::preamble_len](#), [radio_ook_cfg_t::rx_bandwidth](#), [radio_ook_cfg_t::sync_word](#), [radio_ook_cfg_t::sync_word_len](#), [radio_ook_cfg_t::threshold](#), [radio_ook_cfg_t::threshold_value](#) i [radio_ook_cfg_t::tx_power_dbm](#).

Odwołania w [radio_ook_init\(\)](#).

12.68.4.5 radio_deinit()

```
radio_status_t radio_deinit (
    void )
```

Deinicjalizuje backend radiowy.

Zwraca

Kod statusu.

Definicja w linii 119 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_deinit\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_deinit\(\)](#) i [radio_ook_deinit\(\)](#).

12.68.4.6 radio_get_backend()

```
uint8_t radio_get_backend (
    void )
```

Zwraca backend aktualnie wybrany do dispatchu runtime.

Zwraca

Jedna ze stałych [RADIO_LIB_MODULATION_*](#).

Definicja w linii 85 pliku [radio_lib.c](#).

12.68.4.7 radio_get_last_packet()

```
bool radio_get_last_packet (
    radio_packet_t * pkt)
```

Odczytuje ostatnio odebraną ramkę.

Parametry

<i>pkt</i>	[out] Struktura wyjściowa pakietu.
------------	------------------------------------

Zwraca

`true` jeśli bufor zawiera poprawne dane.

Definicja w linii 250 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_get_last_packet\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_get_last_packet\(\)](#) i [radio_ook_get_last_packet\(\)](#).

Odwołania w [radio_test_demo_process\(\)](#).

12.68.4.8 radio_get_state()

```
radio_state_t radio_get_state (
    void )
```

Zwraca aktualny stan backendu.

Zwraca

Stan pracy.

Definicja w linii 266 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_get_state\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_get_state\(\)](#) i [radio_ook_get_state\(\)](#).

Odwołania w [radio_test_demo_process\(\)](#) i [radio_test_dump_tx_debug\(\)](#).

12.68.4.9 radio_init()

```
radio_status_t radio_init (
    const radio_hw_cfg_t * hw,
    const radio_lora_cfg_t * cfg,
    radio_event_cb_t cb,
    void * user_ctx)
```

Inicjalizuje backend radiowy.

Parametry

<i>hw</i>	Konfiguracja sprzętowa.
<i>cfg</i>	Konfiguracja profilu LoRa.
<i>cb</i>	Callback zdarzeń (opcjonalny, może być NULL).
<i>user_ctx</i>	Kontekst użytkownika przekazywany do callbacku.

Zwraca

Kod statusu.

Definicja w linii 100 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_init\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_init\(\)](#) i [radio_ook_init\(\)](#).

Odwołania w [radio_test_demo_init\(\)](#).

12.68.4.10 radio_on_exti()

```
void radio_on_exti (
    uint16_t gpio_pin)
```

Przekazuje numer pinu EXTI do backendu radiowego.

Użyj tej funkcji, gdy `RADIO_LIB_OWNS_HAL_EXTI_CALLBACK == 0` i `callback HAL_GPIO_EXTI_↵` Callback jest implementowany poza biblioteką.

Parametry

<i>gpio_pin</i>	Numer/maska pinu przekazana przez HAL.
-----------------	--

Definicja w linii 346 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_on_exti()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_on_exti()` i `radio_ook_on_exti()`.

12.68.4.11 radio_process()

```
void radio_process (
    void )
```

Obsługuje odroczoną logikę IRQ w kontekście pętli głównej.

Definicja w linii 215 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_process()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_process()` i `radio_ook_process()`.

Odwołania w `radio_test_demo_process()`.

12.68.4.12 radio_raw_read_burst()

```
radio_status_t radio_raw_read_burst (
    uint8_t reg,
    uint8_t * data,
    uint8_t len)
```

Odczytuje ciąg bajtów z rejestrów transceivera.

Parametry

<i>reg</i>	Adres startowy.
<i>data</i>	[out] Bufor danych.
<i>len</i>	Liczba bajtów.

Zwraca

Kod statusu.

Definicja w linii 314 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_raw_read_burst()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_raw_read_burst()` i `radio_ook_raw_read_burst()`.

12.68.4.13 radio_raw_read_reg()

```
radio_status_t radio_raw_read_reg (
    uint8_t reg,
    uint8_t * value)
```

Odczytuje pojedynczy rejestr transceivera.

Parametry

<i>reg</i>	Adres rejestru.
<i>value</i>	[out] Odczytana wartość.

Zwraca

Kod statusu.

Definicja w linii 282 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_raw_read_reg\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_raw_read_reg\(\)](#) i [radio_ook_raw_read_reg\(\)](#).

Odwołania w [radio_test_dump_basic\(\)](#), [radio_test_dump_tx_debug\(\)](#) i [radio_test_probe\(\)](#).

12.68.4.14 radio_raw_write_burst()

```
radio_status_t radio_raw_write_burst (  
    uint8_t reg,  
    const uint8_t * data,  
    uint8_t len)
```

Zapisuje ciąg bajtów do rejestrów transceivera.

Parametry

<i>reg</i>	Adres startowy.
<i>data</i>	Bufor danych.
<i>len</i>	Liczba bajtów.

Zwraca

Kod statusu.

Definicja w linii 330 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_raw_write_burst\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_raw_write_burst\(\)](#) i [radio_ook_raw_write_burst\(\)](#).

12.68.4.15 radio_raw_write_reg()

```
radio_status_t radio_raw_write_reg (  
    uint8_t reg,  
    uint8_t value)
```

Zapisuje pojedynczy rejestr transceivera.

Parametry

<i>reg</i>	Adres rejestru.
<i>value</i>	Wartość do zapisu.

Zwraca

Kod statusu.

Definicja w linii 298 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_raw_write_reg\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_raw_write_reg\(\)](#) i [radio_ook_raw_write_reg\(\)](#).

12.68.4.16 radio_select_backend()

```
void radio_select_backend (  
    uint8_t modulation)
```

Wybiera backend modulacji używany przez wspólne API w runtime.

Parametry

<i>modulation</i>	Jedna ze stałych <code>RADIO_LIB_MODULATION_*</code> .
-------------------	--

Definicja w linii 80 pliku `radio_lib.c`.

Odwołuje się do `radio_normalize_backend()`.

12.68.4.17 `radio_send_async()`

```
radio_status_t radio_send_async (  
    const uint8_t * data,  
    uint8_t len)
```

Rozpoczyna nadawanie asynchroniczne.

Parametry

<i>data</i>	Wskaźnik na payload.
<i>len</i>	Długość payloadu.

Zwraca

Kod statusu.

Definicja w linii 167 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_send_async()`, `RADIO_LIB_MODULATION_FSK`, `RADIO_LIB_MODULATION_LORA`, `RADIO_LIB_MODULATION_OOK`, `radio_lora_send_async()` i `radio_ook_send_async()`.

Odwołania w `radio_test_send_ping()`.

12.68.4.18 `radio_set_fsk_cfg()`

```
radio_status_t radio_set_fsk_cfg (  
    const radio_fsk_cfg_t * cfg)
```

Przekazuje konfigurację runtime dla backendu FSK.

Parametry

<i>cfg</i>	Konfiguracja FSK do zapamiętania przed <code>radio_init(...)</code> .
------------	---

Zwraca

Kod statusu.

Definicja w linii 90 pliku `radio_lib.c`.

Odwołuje się do `radio_fsk_set_runtime_cfg()`.

12.68.4.19 `radio_set_ook_cfg()`

```
radio_status_t radio_set_ook_cfg (  
    const radio_ook_cfg_t * cfg)
```

Przekazuje konfigurację runtime dla backendu OOK.

Parametry

<i>cfg</i>	Konfiguracja OOK do zapamiętania przed <code>radio_init(...)</code> .
------------	---

Zwraca

Kod statusu.

Definicja w linii 95 pliku [radio_lib.c](#).

Odwołuje się do [radio_ook_set_runtime_cfg\(\)](#).

12.68.4.20 radio_sleep()

```
radio_status_t radio_sleep (
    void )
```

Przełącza radio do trybu sleep.

Zwraca

Kod statusu.

Definicja w linii 199 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_sleep\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_sleep\(\)](#) i [radio_ook_sleep\(\)](#).

12.68.4.21 radio_standby()

```
radio_status_t radio_standby (
    void )
```

Przełącza radio do trybu standby.

Zwraca

Kod statusu.

Definicja w linii 183 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_standby\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_standby\(\)](#) i [radio_ook_standby\(\)](#).

Odwołania w [radio_test_demo_process\(\)](#).

12.68.4.22 radio_start_rx_continuous()

```
radio_status_t radio_start_rx_continuous (
    void )
```

Uruchamia odbiór ciągły.

Zwraca

Kod statusu.

Definicja w linii 135 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_start_rx_continuous\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_start_rx_continuous\(\)](#) i [radio_ook_start_rx_continuous\(\)](#).

Odwołania w [radio_test_demo_init\(\)](#) i [radio_test_demo_process\(\)](#).

12.68.4.23 radio_start_rx_single()

```
radio_status_t radio_start_rx_single (
    uint16_t symbol_timeout)
```

Uruchamia odbiór pojedynczy z timeoutem symbolowym.

Parametry

<i>symbol_timeout</i>	Timeout w symbolach.
-----------------------	----------------------

Zwraca

Kod statusu.

Definicja w linii 151 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_start_rx_single\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_start_rx_single\(\)](#) i [radio_ook_start_rx_single\(\)](#).

12.68.4.24 radio_take_events()

```
uint32_t radio_take_events (
    void )
```

Zwraca i czyści aktualną maskę zdarzeń.

Zwraca

Maska zdarzeń.

Definicja w linii 234 pliku [radio_lib.c](#).

Odwołuje się do [radio_fsk_take_events\(\)](#), [RADIO_LIB_MODULATION_FSK](#), [RADIO_LIB_MODULATION_LORA](#), [RADIO_LIB_MODULATION_OOK](#), [radio_lora_take_events\(\)](#) i [radio_ook_take_events\(\)](#).

Odwołania w [radio_test_demo_process\(\)](#).

12.69 radio_lib.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00009
00010 #ifndef APP_RADIO_LIB_RADIO_LIB_H_
00011 #define APP_RADIO_LIB_RADIO_LIB_H_
00012
00013 #include "radio_lib_config.h"
00014
00015 #include "stm32u5xx_hal.h"
00016
00017 #include <stdbool.h>
00018 #include <stdint.h>
00019
00020 #ifdef __cplusplus
00021 extern "C" {
00022 #endif
00023
00027 typedef enum
00028 {
00029     RADIO_OK = 0,
00030     RADIO_EINVAL = -1,
00031     RADIO_EBUS = -2,
00032     RADIO_ETIMEOUT = -3,
00033     RADIO_ESTATE = -4,
00034     RADIO_EHW = -5
00035 } radio_status_t;
00036
00040 typedef enum
00041 {
00042     RADIO_STATE_UNINIT = 0,
00043     RADIO_STATE_STANDBY,
00044     RADIO_STATE_RX_CONT,
00045     RADIO_STATE_RX_SINGLE,
00046     RADIO_STATE_TX
00047 } radio_state_t;
00048
00052 typedef enum
00053 {
00054     RADIO_EVENT_NONE = 0U,
00055     RADIO_EVENT_TX_DONE = (1UL < 0),
00056     RADIO_EVENT_RX_DONE = (1UL < 1),
00057     RADIO_EVENT_RX_TIMEOUT = (1UL < 2),
00058     RADIO_EVENT_CRC_ERR = (1UL < 3),
00059     RADIO_EVENT_CAD_DONE = (1UL < 4),
00060     RADIO_EVENT_CAD_DETECTED = (1UL < 5),
00061     RADIO_EVENT_FIFO_OVERRUN = (1UL < 6),
00062     RADIO_EVENT_HW_ERROR = (1UL < 7)
00063 } radio_event_mask_t;
00064
00068 typedef enum
00069 {
00070     RADIO_LORA_BW_7_8_KHZ = 0U,
```



```

00071     RADIO_LORA_BW_10_4_KHZ = 1U,
00072     RADIO_LORA_BW_15_6_KHZ = 2U,
00073     RADIO_LORA_BW_20_8_KHZ = 3U,
00074     RADIO_LORA_BW_31_25_KHZ = 4U,
00075     RADIO_LORA_BW_41_7_KHZ = 5U,
00076     RADIO_LORA_BW_62_5_KHZ = 6U,
00077     RADIO_LORA_BW_125_KHZ = 7U,
00078     RADIO_LORA_BW_250_KHZ = 8U,
00079     RADIO_LORA_BW_500_KHZ = 9U
00080 } radio_lora_bw_t;
00081
00085 typedef struct
00086 {
00087     GPIO_TypeDef *port;
00088     uint16_t pin;
00089 } radio_gpio_t;
00090
00094 typedef struct
00095 {
00096     SPI_HandleTypeDef *hsapi;
00097     radio_gpio_t nss;
00098     radio_gpio_t reset;
00099     radio_gpio_t dio[6];
00100     uint32_t spi_timeout_ms;
00101 } radio_hw_cfg_t;
00102
00106 typedef struct
00107 {
00108     uint32_t frequency_hz;
00109     radio_lora_bw_t bandwidth;
00110     uint8_t spreading_factor;
00111     uint8_t coding_rate;
00112     uint16_t preamble_len;
00113     uint8_t sync_word;
00114     bool crc_on;
00115     bool invert_iq;
00116     int8_t tx_power_dbm;
00117     bool implicit_header;
00118     uint8_t payload_len;
00119 } radio_lora_cfg_t;
00120
00124 typedef enum
00125 {
00126     RADIO_FSK_SHAPING_FSK = 0U,
00127     RADIO_FSK_SHAPING_GFSK,
00128     RADIO_FSK_SHAPING_MSK,
00129     RADIO_FSK_SHAPING_GMSK
00130 } radio_fsk_shaping_t;
00131
00135 typedef enum
00136 {
00137     RADIO_FSK_FILTER_NONE = 0U,
00138     RADIO_FSK_FILTER_BT_10,
00139     RADIO_FSK_FILTER_BT_07,
00140     RADIO_FSK_FILTER_BT_05,
00141     RADIO_FSK_FILTER_BT_03
00142 } radio_fsk_filter_t;
00143
00147 typedef enum
00148 {
00149     RADIO_PACKET_CRC_OFF = 0U,
00150     RADIO_PACKET_CRC_CCITT,
00151     RADIO_PACKET_CRC_IBM
00152 } radio_packet_crc_t;
00153
00157 typedef enum
00158 {
00159     RADIO_ADDRESS_FILTER_OFF = 0U,
00160     RADIO_ADDRESS_FILTER_NODE,
00161     RADIO_ADDRESS_FILTER_NODE_BROADCAST
00162 } radio_address_filter_t;
00163
00167 typedef enum
00168 {
00169     RADIO_OOK_THRESHOLD_FIXED = 0U,
00170     RADIO_OOK_THRESHOLD_PEAK,
00171     RADIO_OOK_THRESHOLD_AVERAGE
00172 } radio_ook_threshold_t;
00173
00177 typedef struct
00178 {
00179     uint32_t frequency_hz;
00180     uint32_t bitrate_bps;
00181     radio_lora_bw_t rx_bandwidth;
00182     radio_fsk_shaping_t shaping;
00183     radio_fsk_filter_t filter;
00184     int8_t tx_power_dbm;

```

```
00185     uint16_t preamble_len;
00186     uint8_t sync_word_len;
00187     uint32_t sync_word;
00188     radio_address_filter_t address_filter;
00189     radio_packet_crc_t crc_type;
00190     bool data_whitening;
00191 } radio_fsk_cfg_t;
00192
00196 typedef struct
00197 {
00198     uint32_t frequency_hz;
00199     uint32_t bitrate_bps;
00200     radio_lora_bw_t rx_bandwidth;
00201     int8_t tx_power_dbm;
00202     uint16_t preamble_len;
00203     uint8_t sync_word_len;
00204     uint32_t sync_word;
00205     radio_ook_threshold_t threshold;
00206     uint8_t threshold_value;
00207 } radio_ook_cfg_t;
00208
00212 typedef struct
00213 {
00214     uint8_t data[RADIO_LIB_MAX_PAYLOAD];
00215     uint8_t length;
00216     int16_t rssi_dbm;
00217     int8_t snr_db;
00218     uint32_t timestamp_ms;
00219     bool valid;
00220 } radio_packet_t;
00221
00227 typedef void (*radio_event_cb_t)(uint32_t events, void *user_ctx);
00228
00234 void radio_default_hw_cfg(radio_hw_cfg_t *cfg, SPI_HandleTypeDef *hspl);
00235
00240 void radio_default_lora_cfg(radio_lora_cfg_t *cfg);
00241
00246 void radio_default_fsk_cfg(radio_fsk_cfg_t *cfg);
00247
00252 void radio_default_ook_cfg(radio_ook_cfg_t *cfg);
00253
00258 void radio_select_backend(uint8_t modulation);
00259
00264 uint8_t radio_get_backend(void);
00265
00271 radio_status_t radio_set_fsk_cfg(const radio_fsk_cfg_t *cfg);
00272
00278 radio_status_t radio_set_ook_cfg(const radio_ook_cfg_t *cfg);
00279
00288 radio_status_t radio_init(const radio_hw_cfg_t *hw,
00289                           const radio_lora_cfg_t *cfg,
00290                           radio_event_cb_t cb,
00291                           void *user_ctx);
00292
00297 radio_status_t radio_deinit(void);
00298
00303 radio_status_t radio_start_rx_continuous(void);
00304
00310 radio_status_t radio_start_rx_single(uint16_t symbol_timeout);
00311
00318 radio_status_t radio_send_async(const uint8_t *data, uint8_t len);
00319
00324 radio_status_t radio_standby(void);
00325
00330 radio_status_t radio_sleep(void);
00331
00335 void radio_process(void);
00336
00341 uint32_t radio_take_events(void);
00342
00348 bool radio_get_last_packet(radio_packet_t *pkt);
00349
00354 radio_state_t radio_get_state(void);
00355
00362 radio_status_t radio_raw_read_reg(uint8_t reg, uint8_t *value);
00363
00370 radio_status_t radio_raw_write_reg(uint8_t reg, uint8_t value);
00371
00379 radio_status_t radio_raw_read_burst(uint8_t reg, uint8_t *data, uint8_t len);
00380
00388 radio_status_t radio_raw_write_burst(uint8_t reg, const uint8_t *data, uint8_t len);
00389
00398 void radio_on_exti(uint16_t gpio_pin);
00399
00400 #ifdef __cplusplus
00401 }
00402 #endif
```

```
00403
00404 #endif /* APP_RADIO_LIB_RADIO_LIB_H_ */
```

12.70 Dokumentacja pliku Core/App/radio_lib/radio_lib_config.h

Konfiguracja kompilacyjna biblioteki radio_lib.

```
#include <stdint.h>
```

Definicje

- #define [RADIO_LIB_ACTIVE_MODULATION](#) [RADIO_LIB_MODULATION_LORA](#)
Aktywny backend modulacji wybierany na etapie kompilacji.
- #define [RADIO_LIB_OWNS_HAL_EXTI_CALLBACK](#) 1
Własność callbacku EXTI.
- #define [RADIO_LIB_SPI_TIMEOUT_MS](#) 100U
Timeout blokujących transferów SPI (ms).
- #define [RADIO_LIB_MAX_PAYLOAD](#) 255U
Maksymalna długość payloadu bufora pakietu.
- #define [RADIO_LIB_DEFAULT_FREQ_HZ](#) 868500000UL
Domyślna częstotliwość LoRa (Hz).
- #define [RADIO_LIB_DEFAULT_SYNC_WORD](#) 0x34U
Domyślny sync word LoRa.

Identyfikatory backendów modulacji

Stałe używane przez [RADIO_LIB_ACTIVE_MODULATION](#).

- #define [RADIO_LIB_MODULATION_LORA](#) 0U
Backend LoRa.
- #define [RADIO_LIB_MODULATION_FSK](#) 1U
Backend FSK.
- #define [RADIO_LIB_MODULATION_OOK](#) 2U
Backend OOK.

Filtry EXTI DIO (LoRa)

Pozwala wyłączyć obsługę mniej używanych linii DIO.

- #define [RADIO_LIB_ENABLE_DIO2_EXTI](#) 0
- #define [RADIO_LIB_ENABLE_DIO3_EXTI](#) 0
- #define [RADIO_LIB_ENABLE_DIO4_EXTI](#) 0
- #define [RADIO_LIB_ENABLE_DIO5_EXTI](#) 0

12.70.1 Opis szczegółowy

Konfiguracja kompilacyjna biblioteki radio_lib.

Definicja w pliku [radio_lib_config.h](#).

12.70.2 Dokumentacja definicji

12.70.2.1 RADIO_LIB_ACTIVE_MODULATION

```
#define RADIO_LIB_ACTIVE_MODULATION RADIO\_LIB\_MODULATION\_LORA
```

Aktywny backend modulacji wybierany na etapie kompilacji.

Domyślnie aktywny jest backend LoRa.

Definicja w linii 27 pliku [radio_lib_config.h](#).

12.70.2.2 RADIO_LIB_DEFAULT_FREQ_HZ

```
#define RADIO_LIB_DEFAULT_FREQ_HZ 868500000UL
```

Domyślna częstotliwość LoRa (Hz).

Definicja w linii 58 pliku [radio_lib_config.h](#).

Odwołania w [radio_lora_default_lora_cfg\(\)](#).

12.70.2.3 RADIO_LIB_DEFAULT_SYNC_WORD

```
#define RADIO_LIB_DEFAULT_SYNC_WORD 0x34U
```

Domyślny sync word LoRa.

Definicja w linii 65 pliku [radio_lib_config.h](#).

Odwołania w [radio_lora_default_lora_cfg\(\)](#).

12.70.2.4 RADIO_LIB_ENABLE_DIO2_EXTI

```
#define RADIO_LIB_ENABLE_DIO2_EXTI 0
```

Definicja w linii 74 pliku [radio_lib_config.h](#).

12.70.2.5 RADIO_LIB_ENABLE_DIO3_EXTI

```
#define RADIO_LIB_ENABLE_DIO3_EXTI 0
```

Definicja w linii 78 pliku [radio_lib_config.h](#).

12.70.2.6 RADIO_LIB_ENABLE_DIO4_EXTI

```
#define RADIO_LIB_ENABLE_DIO4_EXTI 0
```

Definicja w linii 82 pliku [radio_lib_config.h](#).

12.70.2.7 RADIO_LIB_ENABLE_DIO5_EXTI

```
#define RADIO_LIB_ENABLE_DIO5_EXTI 0
```

Definicja w linii 86 pliku [radio_lib_config.h](#).

12.70.2.8 RADIO_LIB_MAX_PAYLOAD

```
#define RADIO_LIB_MAX_PAYLOAD 255U
```

Maksymalna długość payloadu bufora pakietu.

Definicja w linii 51 pliku [radio_lib_config.h](#).

Odwołania w [radio_apply_fsk_config\(\)](#), [radio_apply_ook_config\(\)](#), [radio_fsk_send_async\(\)](#), [radio_lora_send_async\(\)](#), [radio_ook_send_async\(\)](#), [radio_read_rx_packet\(\)](#), [radio_read_rx_packet\(\)](#) i [radio_read_rx_packet\(\)](#).

12.70.2.9 RADIO_LIB_MODULATION_FSK

```
#define RADIO_LIB_MODULATION_FSK 1U
```

Backend FSK.

Definicja w linii 17 pliku [radio_lib_config.h](#).

Odwołania w [radio_deinit\(\)](#), [radio_get_last_packet\(\)](#), [radio_get_state\(\)](#), [radio_init\(\)](#), [radio_normalize_backend\(\)](#), [radio_on_exti\(\)](#), [radio_process\(\)](#), [radio_raw_read_burst\(\)](#), [radio_raw_read_reg\(\)](#), [radio_raw_write_burst\(\)](#), [radio_raw_write_reg\(\)](#), [radio_send_async\(\)](#), [radio_sleep\(\)](#), [radio_standby\(\)](#), [radio_start_rx_continuous\(\)](#), [radio_start_rx_single\(\)](#) i [radio_take_events\(\)](#).

12.70.2.10 RADIO_LIB_MODULATION_LORA

```
#define RADIO_LIB_MODULATION_LORA 0U
```

Backend LoRa.

Definicja w linii 16 pliku [radio_lib_config.h](#).

Odwołania w [radio_deinit\(\)](#), [radio_get_last_packet\(\)](#), [radio_get_state\(\)](#), [radio_init\(\)](#), [radio_normalize_backend\(\)](#), [radio_on_exti\(\)](#), [radio_process\(\)](#), [radio_raw_read_burst\(\)](#), [radio_raw_read_reg\(\)](#), [radio_raw_write_burst\(\)](#),

[radio_raw_write_reg\(\)](#), [radio_send_async\(\)](#), [radio_sleep\(\)](#), [radio_standby\(\)](#), [radio_start_rx_continuous\(\)](#), [radio_start_rx_single\(\)](#) i [radio_take_events\(\)](#).

12.70.2.11 RADIO_LIB_MODULATION_OOK

```
#define RADIO_LIB_MODULATION_OOK 2U
```

Backend OOK.

Definicja w linii 18 pliku [radio_lib_config.h](#).

Odwołania w [radio_deinit\(\)](#), [radio_get_last_packet\(\)](#), [radio_get_state\(\)](#), [radio_init\(\)](#), [radio_normalize_backend\(\)](#), [radio_on_exti\(\)](#), [radio_process\(\)](#), [radio_raw_read_burst\(\)](#), [radio_raw_read_reg\(\)](#), [radio_raw_write_burst\(\)](#), [radio_raw_write_reg\(\)](#), [radio_send_async\(\)](#), [radio_sleep\(\)](#), [radio_standby\(\)](#), [radio_start_rx_continuous\(\)](#), [radio_start_rx_single\(\)](#) i [radio_take_events\(\)](#).

12.70.2.12 RADIO_LIB_OWNS_HAL_EXTI_CALLBACK

```
#define RADIO_LIB_OWNS_HAL_EXTI_CALLBACK 1
```

Własność callbacku EXTI.

Gdy ustawione na 1, biblioteka dostarcza HAL_GPIO_EXTI_Callback. Gdy ustawione na 0, użytkownik musi sam przekazać pin do [radio_on_exti](#).

Definicja w linii 37 pliku [radio_lib_config.h](#).

12.70.2.13 RADIO_LIB_SPI_TIMEOUT_MS

```
#define RADIO_LIB_SPI_TIMEOUT_MS 100U
```

Timeout blokujących transferów SPI (ms).

Definicja w linii 44 pliku [radio_lib_config.h](#).

Odwołania w [radio_lora_default_hw_cfg\(\)](#).

12.71 radio_lib_config.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_RADIO_LIB_RADIO_LIB_CONFIG_H_
00007 #define APP_RADIO_LIB_RADIO_LIB_CONFIG_H_
00008
00009 #include <stdint.h>
00010
00016 #define RADIO_LIB_MODULATION_LORA 0U
00017 #define RADIO_LIB_MODULATION_FSK 1U
00018 #define RADIO_LIB_MODULATION_OOK 2U
00020
00026 #ifndef RADIO_LIB_ACTIVE_MODULATION
00027 #define RADIO_LIB_ACTIVE_MODULATION RADIO_LIB_MODULATION_LORA
00028 #endif
00029
00036 #ifndef RADIO_LIB_OWNS_HAL_EXTI_CALLBACK
00037 #define RADIO_LIB_OWNS_HAL_EXTI_CALLBACK 1
00038 #endif
00039
00043 #ifndef RADIO_LIB_SPI_TIMEOUT_MS
00044 #define RADIO_LIB_SPI_TIMEOUT_MS 100U
00045 #endif
00046
00050 #ifndef RADIO_LIB_MAX_PAYLOAD
00051 #define RADIO_LIB_MAX_PAYLOAD 255U
00052 #endif
00053
00057 #ifndef RADIO_LIB_DEFAULT_FREQ_HZ
00058 #define RADIO_LIB_DEFAULT_FREQ_HZ 868500000UL
00059 #endif
00060
00064 #ifndef RADIO_LIB_DEFAULT_SYNC_WORD
00065 #define RADIO_LIB_DEFAULT_SYNC_WORD 0x34U
00066 #endif
00067
00073 #ifndef RADIO_LIB_ENABLE_DIO2_EXTI
00074 #define RADIO_LIB_ENABLE_DIO2_EXTI 0
00075 #endif
00076
00077 #ifndef RADIO_LIB_ENABLE_DIO3_EXTI
```

```

00078 #define RADIO_LIB_ENABLE_DIO3_EXTI 0
00079 #endif
00080
00081 #ifndef RADIO_LIB_ENABLE_DIO4_EXTI
00082 #define RADIO_LIB_ENABLE_DIO4_EXTI 0
00083 #endif
00084
00085 #ifndef RADIO_LIB_ENABLE_DIO5_EXTI
00086 #define RADIO_LIB_ENABLE_DIO5_EXTI 0
00087 #endif
00088
00090 #endif /* APP_RADIO_LIB_RADIO_LIB_CONFIG_H */

```

12.72 Dokumentacja pliku Core/App/radio_lib/test/radio_test.c

Implementacja narzędzi testowych i diagnostycznych radio_lib.

```

#include "radio_test.h"
#include "../common/sx1276/radio_sx1276_regs.h"
#include "../././lcd_main.h"
#include "../././i2c_mem_store_lib/i2c_mem_store.h"
#include "cmsis_os2.h"
#include <ctype.h>
#include <stdio.h>

```

Funkcje

- static uint32_t [radio_test_irq_save](#) (void)
Internal helper: [radio_test_irq_save](#).
- static void [radio_test_irq_restore](#) (uint32_t primask)
Internal helper: [radio_test_irq_restore](#).
- static uint32_t [radio_test_now_ms](#) (void)
Internal helper: [radio_test_now_ms](#).
- static void [radio_test_event_cb](#) (uint32_t events, void *user_ctx)
Callback zdarzeń radiowych używany w scenariuszu demo.
- static void [radio_test_print_packet](#) (const [radio_packet_t](#) *pkt)
Drukuje szczegóły odebranego pakietu (HEX + ASCII).
- static void [radio_test_print_packet_text](#) (const [radio_packet_t](#) *pkt)
Drukuje payload odebranego pakietu jako tekst terminalowy.
- static void [radio_test_publish_packet_to_lcd](#) (const [radio_packet_t](#) *pkt)
Internal helper: [radio_test_publish_packet_to_lcd](#).
- static void [radio_test_store_packet_in_i2c_memory](#) (const [radio_packet_t](#) *pkt)
Internal helper: [radio_test_store_packet_in_i2c_memory](#).
- static void [radio_test_dump_tx_debug](#) (void)
Drukuje podstawowe rejestry diagnostyczne ścieżki TX.
- bool [radio_test_probe](#) (uint8_t *version)
Sprawdza wersję układu i porównuje z oczekiwanym ID SX1276.
- void [radio_test_dump_basic](#) (void)
Drukuje skrócony zrzut podstawowych rejestrów radiowych.
- [radio_status_t](#) [radio_test_send_ping](#) (void)
Wysyła testowy pakiet "PING".
- void [radio_test_demo_init](#) (SPI_HandleTypeDef *hsapi)
Inicjalizuje demonstracyjny scenariusz pracy radia.
- void [radio_test_demo_process](#) (void)
Obsługuje logikę demo: zdarzenia, watchdog TX i okresowe PING.

Zmienne

- static volatile uint32_t [s_radio_cb_events](#) = 0U
- static uint32_t [s_radio_last_ping_ms](#) = 0U
- static uint32_t [s_radio_last_tx_start_ms](#) = 0U
- static bool [s_demo_initialized](#) = false
- static [i2c_mem_store_t](#) [s_msg_store](#)
- static bool [s_msg_store_ready](#) = false
- I2C_HandleTypeDef [hi2c1](#)

12.72.1 Opis szczegółowy

Implementacja narzędzi testowych i diagnostycznych [radio_lib](#).
Definicja w pliku [radio_test.c](#).

12.72.2 Dokumentacja funkcji

12.72.2.1 radio_test_demo_init()

```
void radio_test_demo_init (
    SPI_HandleTypeDef * hspi)
```

Inicjalizuje demonstracyjny scenariusz pracy radia.

Parametry

<i>hspi</i>	Uchwyt SPI przekazany z aplikacji.
-------------	------------------------------------

Definicja w linii 239 pliku [radio_test.c](#).

Odwołuje się do [i2c_mem_store_default_cfg_m24c01r\(\)](#), [i2c_mem_store_init\(\)](#), [I2C_MEM_STORE_OK](#), [radio_default_hw_cfg\(\)](#), [radio_default_lora_cfg\(\)](#), [radio_init\(\)](#), [RADIO_OK](#), [radio_start_rx_continuous\(\)](#), [radio_test_dump_basic\(\)](#), [radio_test_event_cb\(\)](#), [radio_test_now_ms\(\)](#) i [radio_test_probe\(\)](#).

12.72.2.2 radio_test_demo_process()

```
void radio_test_demo_process (
    void )
```

Obsługuje logikę demo: zdarzenia, watchdog TX i okresowe PING.

Obsługuje pętlę demona testowego (zdarzenia, logi, cykliczny ping).

Definicja w linii 303 pliku [radio_test.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EVENT_CAD_DETECTED](#), [RADIO_EVENT_CAD_DONE](#), [RADIO_EVENT_CRC_ERR](#), [RADIO_EVENT_FIFO_OVERRUN](#), [RADIO_EVENT_HW_ERROR](#), [RADIO_EVENT_RX_DONE](#), [RADIO_EVENT_RX_TIMEOUT](#), [RADIO_EVENT_TX_DONE](#), [radio_get_last_packet\(\)](#), [radio_get_state\(\)](#), [RADIO_OK](#), [radio_process\(\)](#), [radio_standby\(\)](#), [radio_start_rx_continuous\(\)](#), [RADIO_STATE_RX_CONT](#), [RADIO_STATE_TX](#), [radio_take_events\(\)](#), [radio_test_dump_tx_debug\(\)](#), [radio_test_irq_restore\(\)](#), [radio_test_irq_save\(\)](#), [radio_test_now_ms\(\)](#), [radio_test_print_packet\(\)](#), [radio_test_print_packet_text\(\)](#), [radio_test_publish_packet_to_lcd\(\)](#), [radio_test_send_ping\(\)](#) i [radio_test_store_packet_in_i2c_memory\(\)](#).

12.72.2.3 radio_test_dump_basic()

```
void radio_test_dump_basic (
    void )
```

Drukuje skrócony zrzut podstawowych rejestrów radiowych.

Drukuje podstawowy zrzut rejestrów przez `printf`.

Definicja w linii 199 pliku [radio_test.c](#).

Odwołuje się do [RADIO_OK](#) i [radio_raw_read_reg\(\)](#).

Odwołania w [radio_test_demo_init\(\)](#).

12.72.2.4 `radio_test_dump_tx_debug()`

```
void radio_test_dump_tx_debug (
    void ) [static]
```

Drukuje podstawowe rejestry diagnostyczne ścieżki TX.

Definicja w linii 157 pliku `radio_test.c`.

Odwołuje się do `radio_get_state()`, `RADIO_OK` i `radio_raw_read_reg()`.

Odwołania w `radio_test_demo_process()`.

12.72.2.5 `radio_test_event_cb()`

```
void radio_test_event_cb (
    uint32_t events,
    void * user_ctx) [static]
```

Callback zdarzeń radiowych używany w scenariuszu demo.

Parametry

<i>events</i>	Maska zdarzeń z <code>radio_lib</code> .
<i>user_ctx</i>	Nieużywany kontekst użytkownika.

Definicja w linii 69 pliku `radio_test.c`.

Odwołuje się do `radio_test_irq_restore()` i `radio_test_irq_save()`.

Odwołania w `radio_test_demo_init()`.

12.72.2.6 `radio_test_irq_restore()`

```
void radio_test_irq_restore (
    uint32_t primask) [static]
```

Internal helper: `radio_test_irq_restore`.

Definicja w linii 35 pliku `radio_test.c`.

Odwołania w `radio_test_demo_process()` i `radio_test_event_cb()`.

12.72.2.7 `radio_test_irq_save()`

```
uint32_t radio_test_irq_save (
    void ) [static]
```

Internal helper: `radio_test_irq_save`.

Definicja w linii 27 pliku `radio_test.c`.

Odwołania w `radio_test_demo_process()` i `radio_test_event_cb()`.

12.72.2.8 `radio_test_now_ms()`

```
uint32_t radio_test_now_ms (
    void ) [static]
```

Internal helper: `radio_test_now_ms`.

Definicja w linii 44 pliku `radio_test.c`.

Odwołania w `radio_test_demo_init()` i `radio_test_demo_process()`.

12.72.2.9 `radio_test_print_packet()`

```
void radio_test_print_packet (
    const radio_packet_t * pkt) [static]
```

Drukuje szczegóły odebranego pakietu (HEX + ASCII).

Parametry

<i>pkt</i>	Wskaźnik na pakiet do wydruku.
------------	--------------------------------

Definicja w linii 84 pliku [radio_test.c](#).

Odwołuje się do [radio_packet_t::data](#), [radio_packet_t::length](#), [radio_packet_t::rssi_dbm](#) i [radio_packet_t::snr_db](#).

Odwołania w [radio_test_demo_process\(\)](#).

12.72.2.10 radio_test_print_packet_text()

```
void radio_test_print_packet_text (
    const radio\_packet\_t * pkt) [static]
```

Drukuje payload odebranego pakietu jako tekst terminalowy.

Parametry

<i>pkt</i>	Wskaźnik na pakiet do wydruku.
------------	--------------------------------

Definicja w linii 110 pliku [radio_test.c](#).

Odwołuje się do [radio_packet_t::data](#) i [radio_packet_t::length](#).

Odwołania w [radio_test_demo_process\(\)](#).

12.72.2.11 radio_test_probe()

```
bool radio_test_probe (
    uint8_t * version)
```

Sprawdza wersję układu i porównuje z oczekiwanym ID SX1276.

Parametry

<i>version</i>	[out] Opcjonalny odczyt surowego rejestru wersji.
----------------	---

Zwraca

`true` gdy układ odpowiada poprawnym identyfikatorem.

Definicja w linii 179 pliku [radio_test.c](#).

Odwołuje się do [RADIO_OK](#) i [radio_raw_read_reg\(\)](#).

Odwołania w [radio_test_demo_init\(\)](#).

12.72.2.12 radio_test_publish_packet_to_lcd()

```
void radio_test_publish_packet_to_lcd (
    const radio\_packet\_t * pkt) [static]
```

Internal helper: [radio_test_publish_packet_to_lcd](#).

Definicja w linii 124 pliku [radio_test.c](#).

Odwołuje się do [radio_packet_t::data](#), [radio_packet_t::length](#) i [radio_packet_t::rssi_dbm](#).

Odwołania w [radio_test_demo_process\(\)](#).

12.72.2.13 radio_test_send_ping()

```
radio\_status\_t radio_test_send_ping (
    void )
```

Wysyła testowy pakiet "PING".

Wysyła krótką ramkę testową "PING".

Zwraca

Kod statusu z [radio_send_async](#).

Definicja w linii 232 pliku [radio_test.c](#).

Odwołuje się do [radio_send_async\(\)](#).

Odwołania w [radio_test_demo_process\(\)](#).

12.72.2.14 radio_test_store_packet_in_i2c_memory()

```
void radio_test_store_packet_in_i2c_memory (
    const radio_packet_t * pkt) [static]
```

Internal helper: [radio_test_store_packet_in_i2c_memory](#).

Definicja w linii 138 pliku [radio_test.c](#).

Odwołuje się do [radio_packet_t::data](#), [i2c_mem_store_append_message\(\)](#), [I2C_MEM_STORE_OK](#), [radio_packet_t::length](#) i [radio_packet_t::rssi_dbm](#).

Odwołania w [radio_test_demo_process\(\)](#).

12.72.3 Dokumentacja zmiennych**12.72.3.1 s_demo_initialized**

```
bool s_demo_initialized = false [static]
```

Definicja w linii 20 pliku [radio_test.c](#).

12.72.3.2 s_msg_store

```
i2c_mem_store_t s_msg_store [static]
```

Definicja w linii 21 pliku [radio_test.c](#).

12.72.3.3 s_msg_store_ready

```
bool s_msg_store_ready = false [static]
```

Definicja w linii 22 pliku [radio_test.c](#).

12.72.3.4 s_radio_cb_events

```
volatile uint32_t s_radio_cb_events = 0U [static]
```

Definicja w linii 17 pliku [radio_test.c](#).

12.72.3.5 s_radio_last_ping_ms

```
uint32_t s_radio_last_ping_ms = 0U [static]
```

Definicja w linii 18 pliku [radio_test.c](#).

12.72.3.6 s_radio_last_tx_start_ms

```
uint32_t s_radio_last_tx_start_ms = 0U [static]
```

Definicja w linii 19 pliku [radio_test.c](#).

12.73 radio_test.c

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #include "radio_test.h"
00007
00008 #include "../common/sx1276/radio_sx1276_regs.h"
00009 #include "../..//lcd_main.h"
00010 #include "../..//i2c_mem_store_lib/i2c_mem_store.h"
00011
00012 #include "cmsis_os2.h"
00013
```

```

00014 #include <ctype.h>
00015 #include <stdio.h>
00016
00017 static volatile uint32_t s_radio_cb_events = 0U;
00018 static uint32_t s_radio_last_ping_ms = 0U;
00019 static uint32_t s_radio_last_tx_start_ms = 0U;
00020 static bool s_demo_initialized = false;
00021 static i2c_mem_store_t s_msg_store;
00022 static bool s_msg_store_ready = false;
00023
00024 extern I2C_HandleTypeDef hi2c1;
00025
00027 static uint32_t radio_test_irq_save(void)
00028 {
00029     uint32_t primask = __get_PRIMASK();
00030     __disable_irq();
00031     return primask;
00032 }
00033
00035 static void radio_test_irq_restore(uint32_t primask)
00036 {
00037     if (primask == 0U)
00038     {
00039         __enable_irq();
00040     }
00041 }
00042
00044 static uint32_t radio_test_now_ms(void)
00045 {
00046     osKernelState_t state = osKernelGetState();
00047     uint32_t tick_hz;
00048     uint32_t tick_count;
00049
00050     if ((state == osKernelRunning) || (state == osKernelLocked))
00051     {
00052         tick_hz = osKernelGetTickFreq();
00053         tick_count = osKernelGetTickCount();
00054         if (tick_hz == 0U)
00055         {
00056             return tick_count;
00057         }
00058         return (uint32_t)((uint64_t)tick_count * 1000ULL) / (uint64_t)tick_hz;
00059     }
00060     return HAL_GetTick();
00061 }
00062
00063
00069 static void radio_test_event_cb(uint32_t events, void *user_ctx)
00070 {
00071     uint32_t key;
00072
00073     (void)user_ctx;
00074
00075     key = radio_test_irq_save();
00076     s_radio_cb_events |= events;
00077     radio_test_irq_restore(key);
00078 }
00079
00084 static void radio_test_print_packet(const radio_packet_t *pkt)
00085 {
00086     uint8_t i;
00087
00088     printf("RADIO RX len=%u RSSI=%d dBm SNR=%d dB hex:",
00089           pkt->length,
00090           pkt->rssi_dbm,
00091           pkt->snr_db);
00092     for (i = 0U; i < pkt->length; i++)
00093     {
00094         printf(" %02X", pkt->data[i]);
00095     }
00096
00097     printf(" ascii:");
00098     for (i = 0U; i < pkt->length; i++)
00099     {
00100         char c = (char)pkt->data[i];
00101         printf("%c", isprint((unsigned char)c) ? c : '.');
00102     }
00103     printf("\r\n");
00104 }
00105
00110 static void radio_test_print_packet_text(const radio_packet_t *pkt)
00111 {
00112     uint8_t i;
00113
00114     printf("RADIO RX TEXT: ");
00115     for (i = 0U; i < pkt->length; i++)
00116     {

```

```

00117         char c = (char)pkt->data[i];
00118         printf("%c", isprint((unsigned char)c) ? c : '.');
00119     }
00120     printf("\r\n");
00121 }
00122
00124 static void radio_test_publish_packet_to_lcd(const radio_packet_t *pkt)
00125 {
00126     if ((pkt == NULL) || (pkt->length == 0U))
00127     {
00128         return;
00129     }
00130
00131     if (!lcd_main_push_message(pkt->rssi_dbm, pkt->data, pkt->length))
00132     {
00133         printf("LCD queue: drop\r\n");
00134     }
00135 }
00136
00138 static void radio_test_store_packet_in_i2c_memory(const radio_packet_t *pkt)
00139 {
00140     i2c_mem_store_status_t rc;
00141
00142     if ((pkt == NULL) || (!s_msg_store_ready))
00143     {
00144         return;
00145     }
00146
00147     rc = i2c_mem_store_append_message(&s_msg_store, pkt->rssi_dbm, pkt->data, pkt->length);
00148     if (rc != I2C_MEM_STORE_OK)
00149     {
00150         printf("MEM LOG write failed: %d\r\n", (int)rc);
00151     }
00152 }
00153
00157 static void radio_test_dump_tx_debug(void)
00158 {
00159     uint8_t irq_flags = 0U;
00160     uint8_t dio_map1 = 0U;
00161     uint8_t op_mode = 0U;
00162
00163     if ((radio_raw_read_reg(SX1276_REG_IRQ_FLAGS, &irq_flags) != RADIO_OK) ||
00164         (radio_raw_read_reg(SX1276_REG_DIO_MAPPING_1, &dio_map1) != RADIO_OK) ||
00165         (radio_raw_read_reg(SX1276_REG_OP_MODE, &op_mode) != RADIO_OK))
00166     {
00167         printf("RADIO DBG: reg read failed\r\n");
00168         return;
00169     }
00170
00171     printf("RADIO DBG: irq=0x%02X dio1=0x%02X op=0x%02X state=%d\r\n",
00172           irq_flags,
00173           dio_map1,
00174           op_mode,
00175           (int)radio_get_state());
00176 }
00177
00178 /* Public API documented in radio_test.h. */
00179 bool radio_test_probe(uint8_t *version)
00180 {
00181     uint8_t reg_version = 0U;
00182
00183     if (radio_raw_read_reg(SX1276_REG_VERSION, &reg_version) != RADIO_OK)
00184     {
00185         return false;
00186     }
00187
00188     if (version != NULL)
00189     {
00190         *version = reg_version;
00191     }
00192
00193     return (reg_version == SX1276_VERSION_ID);
00194 }
00195
00199 void radio_test_dump_basic(void)
00200 {
00201     uint8_t reg_version = 0U;
00202     uint8_t reg_op_mode = 0U;
00203     uint8_t reg_irq_flags = 0U;
00204     uint8_t reg_modem_cfg_1 = 0U;
00205     uint8_t reg_modem_cfg_2 = 0U;
00206     uint8_t reg_modem_cfg_3 = 0U;
00207
00208     if ((radio_raw_read_reg(SX1276_REG_VERSION, &reg_version) != RADIO_OK) ||
00209         (radio_raw_read_reg(SX1276_REG_OP_MODE, &reg_op_mode) != RADIO_OK) ||
00210         (radio_raw_read_reg(SX1276_REG_IRQ_FLAGS, &reg_irq_flags) != RADIO_OK) ||
00211         (radio_raw_read_reg(SX1276_REG_MODEM_CONFIG_1, &reg_modem_cfg_1) != RADIO_OK) ||

```

```

00212         (radio_raw_read_reg(SX1276_REG_MODEM_CONFIG_2, &reg_modem_cfg_2) != RADIO_OK) ||
00213         (radio_raw_read_reg(SX1276_REG_MODEM_CONFIG_3, &reg_modem_cfg_3) != RADIO_OK))
00214     {
00215         printf("RADIO: dump read failed\r\n");
00216         return;
00217     }
00218
00219     printf("RADIO REG version=0x%02X opmode=0x%02X irq=0x%02X mc1=0x%02X mc2=0x%02X mc3=0x%02X\r\n",
00220           reg_version,
00221           reg_op_mode,
00222           reg_irq_flags,
00223           reg_modem_cfg_1,
00224           reg_modem_cfg_2,
00225           reg_modem_cfg_3);
00226 }
00227
00232 radio_status_t radio_test_send_ping(void)
00233 {
00234     static const uint8_t ping_payload[] = { 'P', 'I', 'N', 'G' };
00235     return radio_send_async(ping_payload, (uint8_t)sizeof(ping_payload));
00236 }
00237
00238 /* Public API documented in radio_test.h. */
00239 void radio_test_demo_init(SPI_HandleTypeDef *hspl)
00240 {
00241     radio_hw_cfg_t radio_hw;
00242     radio_lora_cfg_t radio_cfg;
00243     radio_status_t radio_status;
00244     i2c_mem_store_cfg_t mem_cfg;
00245     i2c_mem_store_status_t mem_rc;
00246     uint8_t radio_version = 0U;
00247
00248     if (hspl == NULL)
00249     {
00250         printf("RADIO init failed: null SPI\r\n");
00251         s_demo_initialized = false;
00252         return;
00253     }
00254
00255     printf("RADIO init\r\n");
00256
00257     i2c_mem_store_default_cfg_m24c01r(&mem_cfg, &hi2c1);
00258     mem_rc = i2c_mem_store_init(&s_msg_store, &mem_cfg, true);
00259     if (mem_rc == I2C_MEM_STORE_OK)
00260     {
00261         s_msg_store_ready = true;
00262         printf("MEM LOG init OK\r\n");
00263     }
00264     else
00265     {
00266         s_msg_store_ready = false;
00267         printf("MEM LOG init failed: %d\r\n", (int)mem_rc);
00268     }
00269
00270     radio_default_hw_cfg(&radio_hw, hspl);
00271     radio_default_lora_cfg(&radio_cfg);
00272     radio_status = radio_init(&radio_hw, &radio_cfg, radio_test_event_cb, NULL);
00273     if (radio_status != RADIO_OK)
00274     {
00275         printf("RADIO init failed: %d\r\n", (int)radio_status);
00276         s_demo_initialized = false;
00277         return;
00278     }
00279
00280     if (radio_test_probe(&radio_version))
00281     {
00282         printf("RADIO probe OK, version=0x%02X\r\n", radio_version);
00283     }
00284     else
00285     {
00286         printf("RADIO probe failed\r\n");
00287     }
00288
00289     radio_test_dump_basic();
00290
00291     radio_status = radio_start_rx_continuous();
00292     printf("RADIO RX start: %s\r\n", (radio_status == RADIO_OK) ? "OK" : "FAIL");
00293
00294     s_radio_cb_events = 0U;
00295     s_radio_last_ping_ms = radio_test_now_ms();
00296     s_radio_last_tx_start_ms = 0U;
00297     s_demo_initialized = (radio_status == RADIO_OK);
00298 }
00299
00303 void radio_test_demo_process(void)
00304 {
00305     uint32_t events;

```

```

00306     uint32_t cb_events;
00307     uint32_t key;
00308     radio_packet_t pkt;
00309     uint32_t now;
00310     radio_status_t tx_status;
00311
00312     if (!s_demo_initialized)
00313     {
00314         return;
00315     }
00316
00317     radio_process();
00318
00319     events = radio_take_events();
00320     key = radio_test_irq_save();
00321     cb_events = s_radio_cb_events;
00322     s_radio_cb_events = 0U;
00323     radio_test_irq_restore(key);
00324     events |= cb_events;
00325
00326     if ((events & RADIO_EVENT_TX_DONE) != 0U)
00327     {
00328         printf("RADIO EVT: TX_DONE\r\n");
00329         s_radio_last_tx_start_ms = 0U;
00330     }
00331     if ((events & RADIO_EVENT_RX_DONE) != 0U)
00332     {
00333         printf("RADIO EVT: RX_DONE\r\n");
00334         if (radio_get_last_packet(&pkt))
00335         {
00336             radio_test_print_packet(&pkt);
00337             radio_test_print_packet_text(&pkt);
00338             radio_test_publish_packet_to_lcd(&pkt);
00339             radio_test_store_packet_in_i2c_memory(&pkt);
00340         }
00341         else
00342         {
00343             printf("RADIO RX: no packet data\r\n");
00344         }
00345
00346         if (radio_get_state() != RADIO_STATE_RX_CONT)
00347         {
00348             (void)radio_start_rx_continuous();
00349         }
00350     }
00351     if ((events & RADIO_EVENT_RX_TIMEOUT) != 0U)
00352     {
00353         printf("RADIO EVT: RX_TIMEOUT\r\n");
00354     }
00355     if ((events & RADIO_EVENT_CRC_ERR) != 0U)
00356     {
00357         printf("RADIO EVT: CRC_ERR\r\n");
00358     }
00359     if ((events & RADIO_EVENT_FIFO_OVERRUN) != 0U)
00360     {
00361         printf("RADIO EVT: FIFO_OVERRUN\r\n");
00362     }
00363     if ((events & RADIO_EVENT_CAD_DONE) != 0U)
00364     {
00365         printf("RADIO EVT: CAD_DONE\r\n");
00366     }
00367     if ((events & RADIO_EVENT_CAD_DETECTED) != 0U)
00368     {
00369         printf("RADIO EVT: CAD_DETECTED\r\n");
00370     }
00371     if ((events & RADIO_EVENT_HW_ERROR) != 0U)
00372     {
00373         printf("RADIO EVT: HW_ERROR\r\n");
00374     }
00375
00376     now = radio_test_now_ms();
00377     if ((radio_get_state() == RADIO_STATE_TX) &&
00378         (s_radio_last_tx_start_ms != 0U) &&
00379         ((now - s_radio_last_tx_start_ms) > 2000U))
00380     {
00381         printf("RADIO WARN: TX stuck, recovering\r\n");
00382         radio_test_dump_tx_debug();
00383         (void)radio_standby();
00384         (void)radio_start_rx_continuous();
00385         s_radio_last_tx_start_ms = 0U;
00386     }
00387
00388     if ((now - s_radio_last_ping_ms) >= 5000U)
00389     {
00390         if (radio_get_state() == RADIO_STATE_TX)
00391         {
00392             printf("RADIO TX busy, skip ping\r\n");

```

```

00393         radio_test_dump_tx_debug();
00394     }
00395     else
00396     {
00397         tx_status = radio_test_send_ping();
00398         if (tx_status == RADIO_OK)
00399         {
00400             printf("RADIO TX: PING\r\n");
00401             s_radio_last_tx_start_ms = now;
00402         }
00403         else
00404         {
00405             printf("RADIO TX failed: %d\r\n", (int)tx_status);
00406             if (tx_status == RADIO_EHW)
00407             {
00408                 printf("RADIO TX recovery: standby+rx_cont\r\n");
00409                 (void)radio_standby();
00410                 (void)radio_start_rx_continuous();
00411             }
00412         }
00413     }
00414     s_radio_last_ping_ms = now;
00415 }
00416 }

```

12.74 Dokumentacja pliku Core/App/radio_lib/test/radio_test.h

Narzędzia testowe i diagnostyczne dla radio_lib.

```

#include "../radio_lib.h"
#include <stdbool.h>
#include <stdint.h>

```

Funkcje

- bool [radio_test_probe](#) (uint8_t *version)
Sprawdza wersję układu i porównuje z oczekiwanym ID SX1276.
- void [radio_test_dump_basic](#) (void)
Drukuje podstawowy zrzut rejestrów przez printf.
- [radio_status_t](#) [radio_test_send_ping](#) (void)
Wysyła krótką ramkę testową "PING".
- void [radio_test_demo_init](#) (SPI_HandleTypeDef *hspi)
Inicjalizuje demonstracyjny scenariusz pracy radia.
- void [radio_test_demo_process](#) (void)
Obsługuje pętlę demona testowego (zdarzenia, logi, cykliczny ping).

12.74.1 Opis szczegółowy

Narzędzia testowe i diagnostyczne dla radio_lib.

Definicja w pliku [radio_test.h](#).

12.74.2 Dokumentacja funkcji

12.74.2.1 radio_test_demo_init()

```

void radio_test_demo_init (
    SPI_HandleTypeDef * hspi)

```

Inicjalizuje demonstracyjny scenariusz pracy radia.

Parametry

<i>hspi</i>	Uchwyt SPI przekazany z aplikacji.
-------------	------------------------------------

Definicja w linii 239 pliku [radio_test.c](#).

Odwołuje się do [i2c_mem_store_default_cfg_m24c01r\(\)](#), [i2c_mem_store_init\(\)](#), [I2C_MEM_STORE_OK](#), [radio_default_hw_cfg\(\)](#), [radio_default_lora_cfg\(\)](#), [radio_init\(\)](#), [RADIO_OK](#), [radio_start_rx_continuous\(\)](#), [radio_test_dump_basic\(\)](#), [radio_test_event_cb\(\)](#), [radio_test_now_ms\(\)](#) i [radio_test_probe\(\)](#).

12.74.2.2 radio_test_demo_process()

```
void radio_test_demo_process (
    void )
```

Obsługuje pętlę demona testowego (zdarzenia, logi, cykliczny ping).

Obsługuje pętlę demona testowego (zdarzenia, logi, cykliczny ping).

Definicja w linii 303 pliku [radio_test.c](#).

Odwołuje się do [RADIO_EHW](#), [RADIO_EVENT_CAD_DETECTED](#), [RADIO_EVENT_CAD_DONE](#), [RADIO_EVENT_CRC_ERR](#), [RADIO_EVENT_FIFO_OVERRUN](#), [RADIO_EVENT_HW_ERROR](#), [RADIO_EVENT_RX_DONE](#), [RADIO_EVENT_RX_TIMEOUT](#), [RADIO_EVENT_TX_DONE](#), [radio_get_last_packet\(\)](#), [radio_get_state\(\)](#), [RADIO_OK](#), [radio_process\(\)](#), [radio_standby\(\)](#), [radio_start_rx_continuous\(\)](#), [RADIO_STATE_RX_CONT](#), [RADIO_STATE_TX](#), [radio_take_events\(\)](#), [radio_test_dump_tx_debug\(\)](#), [radio_test_irq_restore\(\)](#), [radio_test_irq_save\(\)](#), [radio_test_now_ms\(\)](#), [radio_test_print_packet\(\)](#), [radio_test_print_packet_text\(\)](#), [radio_test_publish_packet_to_lcd\(\)](#), [radio_test_send_ping\(\)](#) i [radio_test_store_packet_in_i2c_memory\(\)](#).

12.74.2.3 radio_test_dump_basic()

```
void radio_test_dump_basic (
    void )
```

Drukuje podstawowy zrzut rejestrów przez `printf`.

Drukuje podstawowy zrzut rejestrów przez `printf`.

Definicja w linii 199 pliku [radio_test.c](#).

Odwołuje się do [RADIO_OK](#) i [radio_raw_read_reg\(\)](#).

Odwołania w [radio_test_demo_init\(\)](#).

12.74.2.4 radio_test_probe()

```
bool radio_test_probe (
    uint8_t * version)
```

Sprawdza wersję układu i porównuje z oczekiwanym ID SX1276.

Parametry

<i>version</i>	[out] Opcjonalny odczyt surowego rejestru wersji.
----------------	---

Zwraca

`true` gdy układ odpowiada poprawnym identyfikatorem.

Definicja w linii 179 pliku [radio_test.c](#).

Odwołuje się do [RADIO_OK](#) i [radio_raw_read_reg\(\)](#).

Odwołania w [radio_test_demo_init\(\)](#).

12.74.2.5 radio_test_send_ping()

```
radio_status_t radio_test_send_ping (
    void )
```

Wysyła krótką ramkę testową "PING".

Zwraca

Kod statusu.

Wysyła krótką ramkę testową "PING".

Zwraca

Kod statusu z `radio_send_async`.

Definicja w linii 232 pliku `radio_test.c`.

Odwołuje się do `radio_send_async()`.

Odwołania w `radio_test_demo_process()`.

12.75 radio_test.h

[Idź do dokumentacji tego pliku.](#)

```

00001
00005
00006 #ifndef APP_RADIO_LIB_TEST_RADIO_TEST_H_
00007 #define APP_RADIO_LIB_TEST_RADIO_TEST_H_
00008
00009 #include "../radio_lib.h"
00010
00011 #include <stdbool.h>
00012 #include <stdint.h>
00013
00014 #ifdef __cplusplus
00015 extern "C" {
00016 #endif
00017
00023 bool radio_test_probe(uint8_t *version);
00024
00028 void radio_test_dump_basic(void);
00029
00034 radio_status_t radio_test_send_ping(void);
00035
00040 void radio_test_demo_init(SPI_HandleTypeDef *hspi);
00041
00045 void radio_test_demo_process(void);
00046
00047 #ifdef __cplusplus
00048 }
00049 #endif
00050
00051 #endif /* APP_RADIO_LIB_TEST_RADIO_TEST_H_ */

```

12.76 radio_main.c

```

00001 #include "radio_main.h"
00002
00003 #include "cmsis_os2.h"
00004 #include "FreeRTOS.h"
00005 #include "laviet_crypto.h"
00006 #include "laviet_frame.h"
00007 #include "lcd_main.h"
00008 #include "menu_main.h"
00009 #include "radio_lib/radio_lib.h"
00010 #include "radio_lib/common/sx1276/radio_sx1276_regs.h"
00011 #include "security_main.h"
00012 #include "task.h"
00013
00014 #include <ctype.h>
00015 #include <stdio.h>
00016 #include <string.h>
00017
00018 #define RADIO_TASK_STACK_SIZE 6144U
00019 #define RADIO_TASK_STACK_WORDS (RADIO_TASK_STACK_SIZE / sizeof(StackType_t))
00020 #define RADIO_CMD_QUEUE_DEPTH 16U
00021 #define RADIO_CMD_WAIT_MS 1500U
00022 #define RADIO_CMD_POLL_MS 5U
00023 #define RADIO_MSG_BUF_MAX 255U
00024 #define RADIO_HOP_PERIOD_DEFAULT_MS 10000UL
00025 #define RADIO_PAIR_CODE_LEN LAVIET_PAIR_PAYLOAD_LEN
00026 #define RADIO_RECOVERY_RESET_PULSE_MS 2U
00027 #define RADIO_RECOVERY_RESET_BOOT_MS 10U
00028 #define RADIO_RECOVERY_RETRY_COUNT 2U
00029 #define RADIO_TX_GUARD_MIN_MS 200UL
00030 #define RADIO_TX_GUARD_LORA_MS 20000UL
00031 #define RADIO_TX_GUARD_MARGIN_MS 64UL
00032 #define RADIO_ACK_TIMEOUT_MIN_MS 6000UL
00033 #define RADIO_ACK_TIMEOUT_MARGIN_MS 1500UL
00034 #define RADIO_ACK_ACTIVE_POLL_MS 1U
00035 #define RADIO_ACK_TX_DELAY_MS 100UL
00036 #define RADIO_IDLE_POLL_MS 10U

```

```

00037 #define RADIO_ACK_CLOSED_TTL_MS          10000UL
00038 #define RADIO_ACK_RETRY_LIMIT              2U
00039 #define RADIO_AUTO_PING_MIN_PERIOD_MS      250UL
00040 #define RADIO_AUTO_PING_PERIOD_DEFAULT_MS  1000UL
00041 #define RADIO_AUTO_PING_RAW_LEN            4U
00042 #define RADIO_NETWORK_PAIR_RSSI_MIN_DBM    (-20)
00043 #define RADIO_TRUSTED_DEVICE_SLOTS         16U
00044 #define RADIO_BCAST_CTRL_MAGIC             0xB7U
00045 #define RADIO_BCAST_CTRL_INSTALL_FRAGMENT  1U
00046 #define RADIO_BCAST_CTRL_ACTIVATE          2U
00047 #define RADIO_BCAST_GROUP_KEY_LEN          16U
00048 #define RADIO_BCAST_GROUP_FRAGMENT_LEN     8U
00049 #define RADIO_BCAST_GROUP_FRAGMENT_COUNT   2U
00050 #define RADIO_LORA_BW_HZ_7_8               7800UL
00051 #define RADIO_LORA_BW_HZ_10_4             10400UL
00052 #define RADIO_LORA_BW_HZ_15_6             15600UL
00053 #define RADIO_LORA_BW_HZ_20_8             20800UL
00054 #define RADIO_LORA_BW_HZ_31_25            31250UL
00055 #define RADIO_LORA_BW_HZ_41_7             41700UL
00056 #define RADIO_LORA_BW_HZ_62_5             62500UL
00057 #define RADIO_LORA_BW_HZ_125              125000UL
00058 #define RADIO_LORA_BW_HZ_250              250000UL
00059 #define RADIO_LORA_BW_HZ_500              500000UL
00060
00061 typedef enum
00062 {
00063     RADIO_MAIN_CMD_NONE = 0,
00064     RADIO_MAIN_CMD_SEND_TEMPLATE,
00065     RADIO_MAIN_CMD_SEND_USER_TEXT,
00066     RADIO_MAIN_CMD_SEND_RAW,
00067     RADIO_MAIN_CMD_SET_PRESET,
00068     RADIO_MAIN_CMD_SET_MODULATION,
00069     RADIO_MAIN_CMD_SET_MOD_FREQ,
00070     RADIO_MAIN_CMD_SET_MOD_BW,
00071     RADIO_MAIN_CMD_SET_OPTION,
00072     RADIO_MAIN_CMD_SET_FH,
00073     RADIO_MAIN_CMD_SET_FH_PERIOD,
00074     RADIO_MAIN_CMD_SET_CODING,
00075     RADIO_MAIN_CMD_SET_AUTO_PING,
00076     RADIO_MAIN_CMD_SET_AUTO_PING_PERIOD,
00077     RADIO_MAIN_CMD_SET_AUTO_PING_MODE,
00078     RADIO_MAIN_CMD_RESET_MODULE,
00079     RADIO_MAIN_CMD_START_PAIRING,
00080     RADIO_MAIN_CMD_START_NETWORK_PAIRING,
00081     RADIO_MAIN_CMD_PAIRING_ACCEPT,
00082     RADIO_MAIN_CMD_SEND_PAIR_REQ,
00083     RADIO_MAIN_CMD_SEND_NETWORK_PAIR_REQ,
00084     RADIO_MAIN_CMD_SEND_PAIR_ERROR
00085 } radio_main_cmd_id_t;
00086
00087 typedef struct
00088 {
00089     volatile bool done;
00090     bool result;
00091     radio_main_runtime_cfg_t runtime_cfg;
00092 } radio_main_cmd_sync_t;
00093
00094 typedef struct
00095 {
00096     radio_main_cmd_id_t id;
00097     radio_main_cmd_sync_t *sync;
00098     union
00099     {
00100         struct
00101         {
00102             uint8_t group_id;
00103             uint8_t msg_id;
00104             uint32_t dst_id;
00105         } send_template;
00106         struct
00107         {
00108             uint32_t dst_id;
00109             uint8_t len;
00110             char text[21];
00111         } send_text;
00112         struct
00113         {
00114             const uint8_t *data;
00115             uint8_t len;
00116         } send_raw;
00117         struct
00118         {
00119             uint8_t value;
00120         } set_u8;
00121         struct
00122         {
00123             uint32_t value;

```

```

00124         } set_u32;
00125     struct
00126     {
00127         radio_main_option_t option;
00128         uint32_t value;
00129     } set_option;
00130     struct
00131     {
00132         bool enabled;
00133     } set_bool;
00134     struct
00135     {
00136         uint32_t timeout_ms;
00137         bool network_mode;
00138     } pairing;
00139     struct
00140     {
00141         uint32_t dst_id;
00142     } to_node;
00143     } u;
00144 } radio_main_cmd_t;
00145
00146 typedef struct
00147 {
00148     radio_hw_cfg_t hw;
00149     radio_lora_cfg_t backend_cfg;
00150     radio_lora_cfg_t lora_cfg;
00151     radio_main_modulation_t modulation_id;
00152     radio_main_modulation_t backend_modulation_id;
00153     bool initialized;
00154     bool fh_enabled;
00155     uint32_t fh_period_ms;
00156     bool coding_enabled;
00157     bool auto_ping_enabled;
00158     uint32_t auto_ping_period_ms;
00159     radio_main_auto_ping_mode_t auto_ping_mode;
00160     bool crypto_ready;
00161     uint8_t lora_preset;
00162     radio_main_fsk_cfg_t fsk_cfg;
00163     radio_main_ook_cfg_t ook_cfg;
00164     uint8_t hop_idx;
00165     uint32_t last_hop_ms;
00166     uint32_t last_ping_ms;
00167     uint16_t node_id;
00168     uint16_t next_msg_id;
00169     uint32_t gateway_rx_counter;
00170     uint32_t gateway_tx_counter;
00171     bool gateway_key_mode_known;
00172     security_frame_key_mode_t gateway_key_mode;
00173     bool gateway_pair_code_valid;
00174     uint8_t gateway_pair_code_len;
00175     uint8_t gateway_pair_code[8];
00176     bool broadcast_group_key_valid;
00177     uint32_t broadcast_group_epoch;
00178     uint8_t broadcast_group_aes_key[16];
00179     uint8_t broadcast_group_hmac_key[32];
00180     bool broadcast_group_pending_valid;
00181     uint32_t broadcast_group_pending_epoch;
00182     uint8_t broadcast_group_pending_mask;
00183     uint8_t broadcast_group_pending_key[RADIO_BCAST_GROUP_KEY_LEN];
00184     bool pairing_active;
00185     bool pairing_network_mode;
00186     uint32_t pairing_until_ms;
00187     bool pairing_pending;
00188     uint32_t pairing_pending_node;
00189     uint8_t pairing_pending_code[8];
00190     uint8_t pairing_pending_code_len;
00191     bool pairing_pending_network;
00192     bool pairing_outgoing_pending;
00193     uint8_t pairing_outgoing_code[8];
00194     uint8_t pairing_outgoing_code_len;
00195     bool pairing_outgoing_network;
00196     bool tx_in_progress;
00197     bool tx_silent;
00198     uint32_t tx_deadline_ms;
00199     char last_error_text[21];
00200     struct
00201     {
00202         bool active;
00203         uint16_t peer_id;
00204         uint16_t msg_id;
00205         uint32_t counter;
00206         uint8_t raw_len;
00207         uint8_t raw[LAVIET_FRAME_MAX_LEN];
00208         uint8_t retries_done;
00209         uint32_t deadline_ms;
00210         uint32_t wait_ms;

```

```
00211     } ack_pending;
00212     struct
00213     {
00214         bool valid;
00215         bool timed_out;
00216         uint16_t peer_id;
00217         uint16_t msg_id;
00218         uint32_t counter;
00219         uint32_t expires_ms;
00220     } ack_recent;
00221 } radio_main_ctx_t;
00222
00223 static osThreadId_t s_radio_task = NULL;
00224 static osMessageQueueId_t s_radio_cmd_queue = NULL;
00225 static osMutexId_t s_radio_state_mutex = NULL;
00226 static StaticTask_t s_radio_task_cb;
00227 static StackType_t s_radio_task_stack[RADIO_TASK_STACK_WORDS];
00228 static radio_main_ctx_t s_ctx;
00229
00230 extern SPI_HandleTypeDef hspil;
00231
00232 static void radio_main_task_fn(void *argument);
00233 static bool radio_main_enqueue_sync(const radio_main_cmd_t *cmd, radio_main_cmd_sync_t *sync);
00234 static bool radio_main_wait_sync(radio_main_cmd_sync_t *sync, uint32_t timeout_ms);
00235 static bool radio_main_radio_init_and_start(void);
00236 static void radio_main_apply_preset_cfg(uint8_t preset_id, radio_lora_cfg_t *cfg);
00237 static void radio_main_load_default_profiles(void);
00238 static void radio_main_apply_modulation_cfg(void);
00239 static bool radio_main_switch_backend(radio_main_modulation_t modulation);
00240 static void radio_main_load_default_profile(radio_main_modulation_t modulation);
00241 static void radio_main_sync_snapshot(radio_main_runtime_cfg_t *cfg);
00242 static bool radio_main_apply_option(radio_main_option_t option, uint32_t value);
00243 static radio_packet_crc_t radio_main_map_fsk_crc(radio_main_crc_type_t crc_type);
00244 static radio_address_filter_t radio_main_map_fsk_address_filter(radio_main_address_filter_t filter);
00245 static radio_ook_threshold_t radio_main_map_ook_threshold(radio_main_ook_threshold_t threshold);
00246 static bool radio_main_push_backend_cfg(void);
00247 static bool radio_main_validate_frequency(uint32_t frequency_hz);
00248 static bool radio_main_validate_tx_power(int32_t tx_power_dbm);
00249 static bool radio_main_validate_bitrate(uint32_t bitrate_bps);
00250 static bool radio_main_validate_preamble(uint32_t preamble_len);
00251 static bool radio_main_is_supported_bw(uint8_t bw_code);
00252 static bool radio_main_validate_hop_period(uint32_t period_ms);
00253 static bool radio_main_validate_auto_ping_period(uint32_t period_ms);
00254 static bool radio_main_validate_auto_ping_mode(radio_main_auto_ping_mode_t mode);
00255 static bool radio_main_reconfigure_radio(void);
00256 static bool radio_main_reset_module_internal(void);
00257 static bool radio_main_force_recover_radio(const char *reason);
00258 static bool radio_main_send_system_frame(laviet_frame_type_t type,
00259     uint32_t dst_id,
00260     const uint8_t *payload,
00261     uint16_t payload_len);
00262 static bool radio_main_send_system_frame_ex(laviet_frame_type_t type,
00263     uint32_t dst_id,
00264     const uint8_t *payload,
00265     uint16_t payload_len,
00266     bool request_ack,
00267     bool silent);
00268 static bool radio_main_send_ack(const laviet_frame_t *frame);
00269 static bool radio_main_send_current_backend_with_retry_ex(const uint8_t *data, uint8_t len, bool
    silent);
00270 static bool radio_main_send_raw_with_retry(const uint8_t *data, uint8_t len);
00271 static bool radio_main_send_raw_with_retry_ex(const uint8_t *data, uint8_t len, bool silent);
00272 static bool radio_main_send_template_internal(uint8_t group_id, uint8_t msg_id, uint32_t dst_id);
00273 static void radio_main_post_rx_notification(int16_t rssi_dbm,
00274     uint16_t src_id,
00275     uint16_t dst_id,
00276     const uint8_t *payload,
00277     uint8_t payload_len,
00278     laviet_frame_type_t frame_type);
00279 static uint32_t radio_main_ack_timeout_ms(uint8_t raw_len);
00280 static bool radio_main_ack_track_start(const laviet_frame_t *frame, const uint8_t *raw, uint8_t
    raw_len);
00281 static void radio_main_ack_track_close(bool timed_out);
00282 static void radio_main_ack_recent_expire(void);
00283 static void radio_main_handle_ack_frame(uint16_t src_id, uint16_t acked_msg_id, uint32_t
    acked_counter);
00284 static void radio_main_handle_ack_timeout(void);
00285 static void radio_main_handle_events(void);
00286 static void radio_main_handle_rx_packet(const radio_packet_t *pkt);
00287 static bool radio_main_source_is_trusted(uint16_t src_id);
00288 static bool radio_main_source_is_allowed_for_data(const laviet_frame_t *frame);
00289 static void radio_main_handle_hopping(void);
00290 static void radio_main_handle_auto_ping(void);
00291 static void radio_main_ensure_rx_continuous(void);
00292 static void radio_main_watchdog_tx(void);
00293 static void radio_main_notify(menu_notification_type_t type, const char *text);
00294 static void radio_main_clear_gateway_pair_code(void);
```

```

00295 static void radio_main_set_gateway_pair_code(const uint8_t *code, uint8_t len);
00296 static void radio_main_load_gateway_pair_code_from_security(void);
00297 static bool radio_main_get_gateway_cached_frame_keys(security_frame_key_mode_t mode,
00298                                                     uint8_t enc_key_out[16],
00299                                                     uint8_t hmac_key_out[32]);
00300 static void radio_main_clear_broadcast_group_state(void);
00301 static uint32_t radio_main_be32_read(const uint8_t *src);
00302 static bool radio_main_derive_broadcast_group_frame_keys(const uint8_t
00303                                                         group_key[RADIO_BCAST_GROUP_KEY_LEN],
00304                                                         uint32_t epoch,
00305                                                         uint8_t enc_key_out[16],
00306                                                         uint8_t hmac_key_out[32]);
00307 static bool radio_main_try_broadcast_group_rx(const laviet_frame_t *frame, uint8_t enc_key[16]);
00308 static bool radio_main_handle_key_rotate_frame(const laviet_frame_t *frame);
00309 static uint8_t radio_main_format_payload_text(const uint8_t *payload,
00310                                              uint8_t payload_len,
00311                                              char *out,
00312                                              uint8_t out_size);
00313 static bool radio_main_push_payload_to_monitor(int16_t rssi_dbm,
00314                                              uint16_t src_id,
00315                                              const uint8_t *payload,
00316                                              uint8_t payload_len);
00317 static void radio_main_log_gateway_rx_frame(const laviet_frame_t *raw_frame,
00318                                             const laviet_frame_t *decoded_frame,
00319                                             int16_t rssi_dbm,
00320                                             int8_t snr_db);
00321 static const char *radio_main_frame_type_text(laviet_frame_type_t type);
00322 static const char *radio_main_key_mode_text(security_frame_key_mode_t mode);
00323 static void radio_main_print_rx_ascii(const uint8_t *data, uint8_t len);
00324 static void radio_main_print_tx_ascii(const uint8_t *data, uint8_t len);
00325 static void radio_main_print_hex_bytes(const char *label, const uint8_t *data, uint16_t len);
00326 static void radio_main_log_hmac_debug(const char *prefix,
00327                                     const laviet_frame_t *frame,
00328                                     security_frame_key_mode_t mode,
00329                                     const uint8_t *pair_code,
00330                                     uint8_t pair_code_len,
00331                                     const uint8_t hmac_key[32],
00332                                     const uint8_t expected_mac[LAVIET_MAC_TAG_LEN]);
00333 static void radio_main_print_generated_pattern(const char *label, uint8_t value, uint16_t len);
00334 static void radio_main_print_fsk_sync_word(uint64_t sync_word, uint8_t sync_len, const char *label);
00335 static void radio_main_print_ook_sync_word(uint32_t sync_word, uint8_t sync_len, const char *label);
00336 static void radio_main_print_tx_frame(const uint8_t *data, uint8_t len, bool retry_attempt);
00337 static uint32_t radio_main_frf_to_hz(uint8_t msb, uint8_t mid, uint8_t lsb);
00338 static void radio_main_log_runtime_profile(const char *reason);
00339 static void radio_main_log_hw_registers(const char *reason);
00340 static bool radio_main_finish_pairing(bool accept);
00341 static bool radio_main_send_pair_request_internal(void);
00342 static bool radio_main_pair_payload_parse(const uint8_t *payload,
00343                                         uint8_t payload_len,
00344                                         const uint8_t **code_out,
00345                                         uint8_t *code_len_out);
00346 static uint8_t radio_main_pair_payload_build(bool network_mode,
00347                                             const uint8_t *code,
00348                                             uint8_t code_len,
00349                                             uint8_t *payload_out,
00350                                             uint8_t payload_capacity);
00351 static bool radio_main_laviet_verify_rx(const laviet_frame_t *frame, uint8_t enc_key[16]);
00352 static void radio_main_make_pair_code(uint8_t *code_out, uint8_t len);
00353 static void radio_main_pair_code_to_text(const uint8_t *code, uint8_t len, char *out, uint8_t
00354                                         out_size);
00355 static uint32_t radio_main_now_ms(void);
00356 static uint32_t radio_main_tx_timeout_ms(uint16_t payload_len);
00357 static void radio_main_tx_mark_started_ex(uint16_t payload_len, bool silent);
00358 static void radio_main_tx_clear(void);
00359 static bool radio_main_tx_timed_out(void);
00360 static void radio_main_set_last_error(const char *text);
00361 static void radio_main_clear_last_error(void);
00362 static uint32_t radio_main_current_frequency_hz(void);
00363 static uint16_t radio_main_current_duty_cycle_permille(uint32_t frequency_hz);
00364 static uint32_t radio_main_lora_bw_hz(radio_lora_bw_t bw);
00365 static uint32_t radio_main_auto_ping_airtime_ms(void);
00366 static uint32_t radio_main_estimate_lora_airtime_ms(uint16_t payload_len);
00367 static uint32_t radio_main_estimate_fsk_ook_airtime_ms(uint16_t payload_len, bool ook_mode);
00368 static uint32_t radio_main_auto_ping_period_ms(void);
00369 static const uint32_t s_hop_channels_hz[3] =
00370 {
00371     868100000UL,
00372     868300000UL,
00373     868500000UL
00374 };
00375 static const char *s_template_groups[3][3] =
00376 {
00377     { "ALR:FIRE", "ALR:INTRUSION", "ALR:LOWBATT" },
00378     { "STS:OK", "STS:BUSY", "STS:IDLE" },
00379     { "SRV:PING", "SRV:RESET", "SRV:SYNC" }
00380 }

```

```
00380 };
00381 static const uint8_t s_auto_ping_raw[RADIO_AUTO_PING_RAW_LEN] =
00382 {
00383     (uint8_t)'P', (uint8_t)'I', (uint8_t)'N', (uint8_t)'G'
00384 };
00385
00386 static const osThreadAttr_t s_radio_task_attr =
00387 {
00388     .name = "radio_task",
00389     .priority = (osPriority_t)osPriorityNormal,
00390     .stack_mem = s_radio_task_stack,
00391     .stack_size = sizeof(s_radio_task_stack),
00392     .cb_mem = &s_radio_task_cb,
00393     .cb_size = sizeof(s_radio_task_cb)
00394 };
00395
00396 void radio_main_create_task(void)
00397 {
00398     if (s_radio_state_mutex == NULL)
00399     {
00400         s_radio_state_mutex = osMutexNew(NULL);
00401         if (s_radio_state_mutex == NULL)
00402         {
00403             printf("RADIO: state mutex create failed\r\n");
00404             return;
00405         }
00406     }
00407
00408     if (s_radio_cmd_queue == NULL)
00409     {
00410         s_radio_cmd_queue = osMessageQueueNew(RADIO_CMD_QUEUE_DEPTH, sizeof(radio_main_cmd_t), NULL);
00411         if (s_radio_cmd_queue == NULL)
00412         {
00413             printf("RADIO: cmd queue create failed\r\n");
00414             return;
00415         }
00416     }
00417
00418     if (s_radio_task == NULL)
00419     {
00420         s_radio_task = osThreadNew(radio_main_task_fn, NULL, &s_radio_task_attr);
00421         if (s_radio_task == NULL)
00422         {
00423             printf("RADIO: task create failed\r\n");
00424         }
00425     }
00426 }
00427
00428 bool radio_main_cmd_send_template(uint8_t group_id, uint8_t msg_id, uint32_t dst_id)
00429 {
00430     radio_main_cmd_t cmd;
00431     radio_main_cmd_sync_t sync;
00432
00433     memset(&cmd, 0, sizeof(cmd));
00434     cmd.id = RADIO_MAIN_CMD_SEND_TEMPLATE;
00435     cmd.u.send_template.group_id = group_id;
00436     cmd.u.send_template.msg_id = msg_id;
00437     cmd.u.send_template.dst_id = dst_id;
00438
00439     return radio_main_enqueue_sync(&cmd, &sync);
00440 }
00441
00442 bool radio_main_cmd_send_user_text(const char *text, uint32_t dst_id)
00443 {
00444     radio_main_cmd_t cmd;
00445     radio_main_cmd_sync_t sync;
00446     size_t len;
00447
00448     if (text == NULL)
00449     {
00450         return false;
00451     }
00452
00453     memset(&cmd, 0, sizeof(cmd));
00454     cmd.id = RADIO_MAIN_CMD_SEND_USER_TEXT;
00455     cmd.u.send_text.dst_id = dst_id;
00456     len = strlen(text);
00457     if ((len == 0) || (len > LAVIET_MAX_PAYLOAD))
00458     {
00459         return false;
00460     }
00461     cmd.u.send_text.len = (uint8_t)len;
00462     memcpy(cmd.u.send_text.text, text, len);
00463     cmd.u.send_text.text[len] = '\0';
00464
00465     return radio_main_enqueue_sync(&cmd, &sync);
00466 }
```

```
00467
00468 bool radio_main_cmd_send_raw(const uint8_t *data, uint8_t len)
00469 {
00470     radio_main_cmd_t cmd;
00471     radio_main_cmd_sync_t sync;
00472
00473     if ((data == NULL) || (len == 0U))
00474     {
00475         return false;
00476     }
00477
00478     memset(&cmd, 0, sizeof(cmd));
00479     cmd.id = RADIO_MAIN_CMD_SEND_RAW;
00480     cmd.u.send_raw.data = data;
00481     cmd.u.send_raw.len = len;
00482
00483     return radio_main_enqueue_sync(&cmd, &sync);
00484 }
00485
00486 bool radio_main_cmd_set_lora_preset(uint8_t preset_id)
00487 {
00488     radio_main_cmd_t cmd;
00489     radio_main_cmd_sync_t sync;
00490
00491     memset(&cmd, 0, sizeof(cmd));
00492     cmd.id = RADIO_MAIN_CMD_SET_PRESET;
00493     cmd.u.set_u8.value = preset_id;
00494
00495     return radio_main_enqueue_sync(&cmd, &sync);
00496 }
00497
00498 bool radio_main_cmd_set_modulation(uint8_t modulation_id)
00499 {
00500     radio_main_cmd_t cmd;
00501     radio_main_cmd_sync_t sync;
00502
00503     memset(&cmd, 0, sizeof(cmd));
00504     cmd.id = RADIO_MAIN_CMD_SET_MODULATION;
00505     cmd.u.set_u8.value = modulation_id;
00506
00507     return radio_main_enqueue_sync(&cmd, &sync);
00508 }
00509
00510 bool radio_main_cmd_set_modulation_freq(uint32_t frequency_hz)
00511 {
00512     radio_main_cmd_t cmd;
00513     radio_main_cmd_sync_t sync;
00514
00515     memset(&cmd, 0, sizeof(cmd));
00516     cmd.id = RADIO_MAIN_CMD_SET_MOD_FREQ;
00517     cmd.u.set_u32.value = frequency_hz;
00518
00519     return radio_main_enqueue_sync(&cmd, &sync);
00520 }
00521
00522 bool radio_main_cmd_set_modulation_bw(uint8_t bandwidth_code)
00523 {
00524     radio_main_cmd_t cmd;
00525     radio_main_cmd_sync_t sync;
00526
00527     memset(&cmd, 0, sizeof(cmd));
00528     cmd.id = RADIO_MAIN_CMD_SET_MOD_BW;
00529     cmd.u.set_u8.value = bandwidth_code;
00530
00531     return radio_main_enqueue_sync(&cmd, &sync);
00532 }
00533
00534 bool radio_main_cmd_set_option(radio_main_option_t option, uint32_t value)
00535 {
00536     radio_main_cmd_t cmd;
00537     radio_main_cmd_sync_t sync;
00538
00539     memset(&cmd, 0, sizeof(cmd));
00540     cmd.id = RADIO_MAIN_CMD_SET_OPTION;
00541     cmd.u.set_option.option = option;
00542     cmd.u.set_option.value = value;
00543
00544     return radio_main_enqueue_sync(&cmd, &sync);
00545 }
00546
00547 bool radio_main_cmd_set_fh(bool enabled)
00548 {
00549     radio_main_cmd_t cmd;
00550     radio_main_cmd_sync_t sync;
00551
00552     memset(&cmd, 0, sizeof(cmd));
00553     cmd.id = RADIO_MAIN_CMD_SET_FH;
```

```
00554     cmd.u.set_bool.enabled = enabled;
00555     return radio_main_enqueue_sync(&cmd, &sync);
00556 }
00557
00558 bool radio_main_cmd_set_fh_period(uint32_t period_ms)
00559 {
00560     radio_main_cmd_t cmd;
00561     radio_main_cmd_sync_t sync;
00562
00563     memset(&cmd, 0, sizeof(cmd));
00564     cmd.id = RADIO_MAIN_CMD_SET_FH_PERIOD;
00565     cmd.u.set_u32.value = period_ms;
00566     return radio_main_enqueue_sync(&cmd, &sync);
00567 }
00568
00569 bool radio_main_cmd_set_coding(bool enabled)
00570 {
00571     radio_main_cmd_t cmd;
00572     radio_main_cmd_sync_t sync;
00573
00574     memset(&cmd, 0, sizeof(cmd));
00575     cmd.id = RADIO_MAIN_CMD_SET_CODING;
00576     cmd.u.set_bool.enabled = enabled;
00577     return radio_main_enqueue_sync(&cmd, &sync);
00578 }
00579
00580 bool radio_main_cmd_set_auto_ping(bool enabled)
00581 {
00582     radio_main_cmd_t cmd;
00583     radio_main_cmd_sync_t sync;
00584
00585     memset(&cmd, 0, sizeof(cmd));
00586     cmd.id = RADIO_MAIN_CMD_SET_AUTO_PING;
00587     cmd.u.set_bool.enabled = enabled;
00588     return radio_main_enqueue_sync(&cmd, &sync);
00589 }
00590
00591 bool radio_main_cmd_set_auto_ping_period(uint32_t period_ms)
00592 {
00593     radio_main_cmd_t cmd;
00594     radio_main_cmd_sync_t sync;
00595
00596     memset(&cmd, 0, sizeof(cmd));
00597     cmd.id = RADIO_MAIN_CMD_SET_AUTO_PING_PERIOD;
00598     cmd.u.set_u32.value = period_ms;
00599     return radio_main_enqueue_sync(&cmd, &sync);
00600 }
00601
00602 bool radio_main_cmd_set_auto_ping_mode(radio_main_auto_ping_mode_t mode)
00603 {
00604     radio_main_cmd_t cmd;
00605     radio_main_cmd_sync_t sync;
00606
00607     memset(&cmd, 0, sizeof(cmd));
00608     cmd.id = RADIO_MAIN_CMD_SET_AUTO_PING_MODE;
00609     cmd.u.set_u32.value = (uint32_t)mode;
00610     return radio_main_enqueue_sync(&cmd, &sync);
00611 }
00612
00613 bool radio_main_cmd_reset_module(void)
00614 {
00615     radio_main_cmd_t cmd;
00616     radio_main_cmd_sync_t sync;
00617
00618     memset(&cmd, 0, sizeof(cmd));
00619     cmd.id = RADIO_MAIN_CMD_RESET_MODULE;
00620     return radio_main_enqueue_sync(&cmd, &sync);
00621 }
00622
00623 bool radio_main_cmd_start_pairing(uint32_t timeout_ms)
00624 {
00625     radio_main_cmd_t cmd;
00626     radio_main_cmd_sync_t sync;
00627
00628     memset(&cmd, 0, sizeof(cmd));
00629     cmd.id = RADIO_MAIN_CMD_START_PAIRING;
00630     cmd.u.pairing.timeout_ms = timeout_ms;
00631     cmd.u.pairing.network_mode = false;
00632     return radio_main_enqueue_sync(&cmd, &sync);
00633 }
00634
00635 bool radio_main_cmd_start_network_pairing(uint32_t timeout_ms)
00636 {
00637     radio_main_cmd_t cmd;
00638     radio_main_cmd_sync_t sync;
00639
00640     memset(&cmd, 0, sizeof(cmd));
```



```

00641     cmd.id = RADIO_MAIN_CMD_START_NETWORK_PAIRING;
00642     cmd.u.pairing.timeout_ms = timeout_ms;
00643     cmd.u.pairing.network_mode = true;
00644     return radio_main_enqueue_sync(&cmd, &sync);
00645 }
00646
00647 bool radio_main_cmd_pairing_accept(bool accept)
00648 {
00649     radio_main_cmd_t cmd;
00650     radio_main_cmd_sync_t sync;
00651
00652     memset(&cmd, 0, sizeof(cmd));
00653     cmd.id = RADIO_MAIN_CMD_PAIRING_ACCEPT;
00654     cmd.u.set_bool.enabled = accept;
00655     return radio_main_enqueue_sync(&cmd, &sync);
00656 }
00657
00658 bool radio_main_cmd_send_pair_req(void)
00659 {
00660     radio_main_cmd_t cmd;
00661     radio_main_cmd_sync_t sync;
00662
00663     memset(&cmd, 0, sizeof(cmd));
00664     cmd.id = RADIO_MAIN_CMD_SEND_PAIR_REQ;
00665     return radio_main_enqueue_sync(&cmd, &sync);
00666 }
00667
00668 bool radio_main_cmd_send_network_pair_req(void)
00669 {
00670     printf("RADIO: node-originated network PAIR_REQ disabled\r\n");
00671     return false;
00672 }
00673
00674 bool radio_main_cmd_send_pair_error(uint32_t dst_id)
00675 {
00676     radio_main_cmd_t cmd;
00677     radio_main_cmd_sync_t sync;
00678
00679     memset(&cmd, 0, sizeof(cmd));
00680     cmd.id = RADIO_MAIN_CMD_SEND_PAIR_ERROR;
00681     cmd.u.to_node.dst_id = dst_id;
00682     return radio_main_enqueue_sync(&cmd, &sync);
00683 }
00684
00685 bool radio_main_get_runtime_cfg(radio_main_runtime_cfg_t *cfg_out)
00686 {
00687     bool ok = false;
00688
00689     if ((cfg_out == NULL) || (s_radio_state_mutex == NULL))
00690     {
00691         return false;
00692     }
00693
00694     if (osMutexAcquire(s_radio_state_mutex, 100U) == osOK)
00695     {
00696         radio_main_sync_snapshot(cfg_out);
00697         (void)osMutexRelease(s_radio_state_mutex);
00698         ok = true;
00699     }
00700
00701     return ok;
00702 }
00703
00704 bool radio_main_get_auto_ping_period_ms(uint32_t *period_ms_out)
00705 {
00706     bool ok = false;
00707
00708     if (period_ms_out == NULL)
00709     {
00710         return false;
00711     }
00712
00713     if ((s_radio_state_mutex != NULL) &&
00714         (osMutexAcquire(s_radio_state_mutex, 100U) == osOK))
00715     {
00716         *period_ms_out = radio_main_auto_ping_period_ms();
00717         (void)osMutexRelease(s_radio_state_mutex);
00718         ok = true;
00719     }
00720
00721     return ok;
00722 }
00723
00724 bool radio_main_get_last_error_text(char *out, uint8_t out_size)
00725 {
00726     bool ok = false;
00727

```

```

00728     if ((out == NULL) || (out_size == 0U) || (s_radio_state_mutex == NULL))
00729     {
00730         return false;
00731     }
00732
00733     if (osMutexAcquire(s_radio_state_mutex, 100U) == osOK)
00734     {
00735         strncpy(out, s_ctx.last_error_text, out_size - 1U);
00736         out[out_size - 1U] = '\0';
00737         (void)osMutexRelease(s_radio_state_mutex);
00738         ok = true;
00739     }
00740
00741     return ok;
00742 }
00743
00744 uint32_t radio_main_get_node_id(void)
00745 {
00746     uint32_t node_id = 0U;
00747
00748     if (s_radio_state_mutex == NULL)
00749     {
00750         return 0U;
00751     }
00752     if (osMutexAcquire(s_radio_state_mutex, 100U) != osOK)
00753     {
00754         return 0U;
00755     }
00756
00757     node_id = s_ctx.node_id;
00758     (void)osMutexRelease(s_radio_state_mutex);
00759     return node_id;
00760 }
00761
00762 static void radio_main_task_fn(void *argument)
00763 {
00764     radio_main_cmd_t cmd;
00765     security_runtime_cfg_t sec_cfg;
00766
00767     (void)argument;
00768     memset(&s_ctx, 0, sizeof(s_ctx));
00769     memset(&sec_cfg, 0, sizeof(sec_cfg));
00770     s_ctx.node_id = laviet_local_node_id();
00771     s_ctx.next_msg_id = 1U;
00772     s_ctx.crypto_ready = laviet_crypto_init();
00773     radio_main_clear_gateway_pair_code();
00774     radio_main_clear_broadcast_group_state();
00775     (void)security_main_get_gateway_counter(&s_ctx.gateway_rx_counter, &s_ctx.gateway_tx_counter);
00776     s_ctx.modulation_id = RADIO_MAIN_MODULATION_LORA;
00777     s_ctx.backend_modulation_id = s_ctx.modulation_id;
00778     s_ctx.lora_preset = 2U;
00779     radio_main_clear_last_error();
00780
00781     radio_default_hw_cfg(&s_ctx.hw, &hspi1);
00782     radio_default_lora_cfg(&s_ctx.lora_cfg);
00783     s_ctx.backend_cfg = s_ctx.lora_cfg;
00784     radio_main_load_default_profiles();
00785
00786     if (security_main_cmd_get_runtime_cfg(&sec_cfg))
00787     {
00788         s_ctx.coding_enabled = sec_cfg.coding_enabled;
00789         s_ctx.fh_enabled = sec_cfg.fh_enabled;
00790         s_ctx.fh_period_ms = sec_cfg.fh_period_ms;
00791         s_ctx.auto_ping_enabled = sec_cfg.auto_ping_enabled;
00792         s_ctx.auto_ping_period_ms = sec_cfg.auto_ping_period_ms;
00793         s_ctx.auto_ping_mode = sec_cfg.auto_ping_mode;
00794         s_ctx.lora_preset = sec_cfg.lora_preset;
00795         if (sec_cfg.radio_profiles_persisted)
00796         {
00797             s_ctx.lora_cfg = sec_cfg.lora;
00798             s_ctx.fsk_cfg = sec_cfg.fsk;
00799             s_ctx.ook_cfg = sec_cfg.ook;
00800         }
00801     }
00802     else
00803     {
00804         s_ctx.fh_enabled = false;
00805         s_ctx.fh_period_ms = RADIO_HOP_PERIOD_DEFAULT_MS;
00806         s_ctx.coding_enabled = false;
00807         s_ctx.auto_ping_enabled = false;
00808         s_ctx.auto_ping_period_ms = RADIO_AUTO_PING_PERIOD_DEFAULT_MS;
00809         s_ctx.auto_ping_mode = RADIO_MAIN_AUTO_PING_FRAME;
00810         s_ctx.lora_preset = 2U;
00811     }
00812
00813     radio_main_load_gateway_pair_code_from_security();
00814
00815

```

```

00816     if (!radio_main_validate_hop_period(s_ctx.fh_period_ms))
00817     {
00818         s_ctx.fh_period_ms = RADIO_HOP_PERIOD_DEFAULT_MS;
00819     }
00820     if (!radio_main_validate_auto_ping_period(s_ctx.auto_ping_period_ms))
00821     {
00822         s_ctx.auto_ping_period_ms = RADIO_AUTO_PING_PERIOD_DEFAULT_MS;
00823     }
00824     if (!radio_main_validate_auto_ping_mode(s_ctx.auto_ping_mode))
00825     {
00826         s_ctx.auto_ping_mode = RADIO_MAIN_AUTO_PING_FRAME;
00827     }
00828     if (!sec_cfg.radio_profiles_persisted)
00829     {
00830         radio_main_apply_preset_cfg(s_ctx.lora_preset, &s_ctx.lora_cfg);
00831     }
00832     s_ctx.backend_modulation_id = s_ctx.modulation_id;
00833     radio_main_apply_modulation_cfg();
00834     s_ctx.last_hop_ms = radio_main_now_ms();
00835     s_ctx.last_ping_ms = radio_main_now_ms();
00836     s_ctx.hop_idx = 0U;
00837     if (radio_main_radio_init_and_start())
00838     {
00839         printf("RADIO: init OK laviet_node=0x%04X\r\n", (unsigned int)s_ctx.node_id);
00840     }
00841     else
00842     {
00843         printf("RADIO: init failed, fallback to LoRa defaults\r\n");
00844         s_ctx.modulation_id = RADIO_MAIN_MODULATION_LORA;
00845         s_ctx.backend_modulation_id = RADIO_MAIN_MODULATION_LORA;
00846         s_ctx.lora_preset = 2U;
00847         radio_main_apply_preset_cfg(s_ctx.lora_preset, &s_ctx.lora_cfg);
00848         radio_main_load_default_fsk_profile(&s_ctx.fsk_cfg);
00849         radio_main_load_default_ook_profile(&s_ctx.ook_cfg);
00850         radio_main_apply_modulation_cfg();
00851         if (radio_main_radio_init_and_start())
00852         {
00853             printf("RADIO: fallback init OK laviet_node=0x%04X\r\n", (unsigned int)s_ctx.node_id);
00854         }
00855         else
00856         {
00857             printf("RADIO: fallback init failed\r\n");
00858         }
00859     }
00860     for (;;)
00861     {
00862         while (osMessageQueueGet(s_radio_cmd_queue, &cmd, NULL, 0U) == osOK)
00863         {
00864             bool cmd_result = false;
00865             switch (cmd.id)
00866             {
00867                 case RADIO_MAIN_CMD_SEND_TEMPLATE:
00868                     radio_main_clear_last_error();
00869                     cmd_result = radio_main_send_template_internal(cmd.u.send_template.group_id,
00870                                                                     cmd.u.send_template.msg_id,
00871                                                                     cmd.u.send_template.dst_id);
00872                     break;
00873                 case RADIO_MAIN_CMD_SEND_USER_TEXT:
00874                     radio_main_clear_last_error();
00875                     cmd_result = radio_main_send_system_frame(LAVIET_TYPE_DATA,
00876                                                                cmd.u.send_text.dst_id,
00877                                                                (const uint8_t *)cmd.u.send_text.text,
00878                                                                cmd.u.send_text.len);
00879                     break;
00880                 case RADIO_MAIN_CMD_SEND_RAW:
00881                     radio_main_clear_last_error();
00882                     cmd_result = radio_main_send_raw_with_retry(cmd.u.send_raw.data,
00883                                                                cmd.u.send_raw.len);
00884                     if (!cmd_result && (s_ctx.last_error_text[0] == '\0'))
00885                     {
00886                         radio_main_set_last_error("Raw TX failed");
00887                     }
00888                     break;
00889                 case RADIO_MAIN_CMD_SET_PRESET:
00890                     if (cmd.u.set_u8.value <= 2U)
00891                     {
00892                         s_ctx.lora_preset = cmd.u.set_u8.value;
00893                         radio_main_apply_preset_cfg(s_ctx.lora_preset, &s_ctx.lora_cfg);
00894                         if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA)
00895                             radio_main_apply_modulation_cfg();
00896                     }
00897             }
00898         }
00899     }

```

```
00903         {
00904             cmd_result = radio_main_reconfigure_radio();
00905         }
00906         else
00907         {
00908             cmd_result = true;
00909         }
00910     }
00911     break;
00912
00913     case RADIO_MAIN_CMD_SET_MODULATION:
00914         if (cmd.u.set_u8.value <= 2U)
00915         {
00916             radio_main_modulation_t previous_modulation = s_ctx.modulation_id;
00917             radio_main_modulation_t requested =
00918                 (radio_main_modulation_t) cmd.u.set_u8.value;
00919
00920             s_ctx.modulation_id = requested;
00921             cmd_result = radio_main_switch_backend(requested);
00922             if (!cmd_result)
00923             {
00924                 radio_main_load_default_profile(requested);
00925                 cmd_result = radio_main_switch_backend(requested);
00926             }
00927             if (!cmd_result)
00928             {
00929                 s_ctx.modulation_id = previous_modulation;
00930                 (void)radio_main_switch_backend(previous_modulation);
00931             }
00932         }
00933         break;
00934
00935     case RADIO_MAIN_CMD_SET_MOD_FREQ:
00936         if (s_ctx.modulation_id == RADIO_MAIN_MODULATION_FSK)
00937         {
00938             cmd_result = radio_main_apply_option(RADIO_MAIN_OPTION_FSK_FREQ,
00939                                                 cmd.u.set_u32.value);
00940         }
00941         else if (s_ctx.modulation_id == RADIO_MAIN_MODULATION_OOK)
00942         {
00943             cmd_result = radio_main_apply_option(RADIO_MAIN_OPTION_OOK_FREQ,
00944                                                 cmd.u.set_u32.value);
00945         }
00946         else
00947         {
00948             cmd_result = radio_main_apply_option(RADIO_MAIN_OPTION_LORA_FREQ,
00949                                                 cmd.u.set_u32.value);
00950         }
00951         break;
00952
00953     case RADIO_MAIN_CMD_SET_MOD_BW:
00954         if (s_ctx.modulation_id == RADIO_MAIN_MODULATION_FSK)
00955         {
00956             cmd_result = radio_main_apply_option(RADIO_MAIN_OPTION_FSK_RX_BW,
00957                                                 cmd.u.set_u8.value);
00958         }
00959         else if (s_ctx.modulation_id == RADIO_MAIN_MODULATION_OOK)
00960         {
00961             cmd_result = radio_main_apply_option(RADIO_MAIN_OPTION_OOK_RX_BW,
00962                                                 cmd.u.set_u8.value);
00963         }
00964         else
00965         {
00966             cmd_result = radio_main_apply_option(RADIO_MAIN_OPTION_LORA_BW,
00967                                                 cmd.u.set_u8.value);
00968         }
00969         break;
00970
00971     case RADIO_MAIN_CMD_SET_OPTION:
00972         cmd_result = radio_main_apply_option(cmd.u.set_option.option,
00973                                             cmd.u.set_option.value);
00974         break;
00975
00976     case RADIO_MAIN_CMD_SET_FH:
00977         s_ctx.fh_enabled = cmd.u.set_bool.enabled;
00978         s_ctx.last_hop_ms = radio_main_now_ms();
00979         cmd_result = true;
00980         break;
00981
00982     case RADIO_MAIN_CMD_SET_FH_PERIOD:
00983         if (radio_main_validate_hop_period(cmd.u.set_u32.value))
00984         {
00985             s_ctx.fh_period_ms = cmd.u.set_u32.value;
00986             s_ctx.last_hop_ms = radio_main_now_ms();
00987             cmd_result = true;
00988         }
00989         break;
```

```

00989
00990         case RADIO_MAIN_CMD_SET_CODING:
00991             s_ctx.coding_enabled = cmd.u.set_bool.enabled;
00992             cmd_result = true;
00993             break;
00994
00995         case RADIO_MAIN_CMD_SET_AUTO_PING:
00996             s_ctx.auto_ping_enabled = cmd.u.set_bool.enabled;
00997             s_ctx.last_ping_ms = radio_main_now_ms();
00998             cmd_result = true;
00999             break;
01000
01001         case RADIO_MAIN_CMD_SET_AUTO_PING_PERIOD:
01002             if (radio_main_validate_auto_ping_period(cmd.u.set_u32.value))
01003             {
01004                 s_ctx.auto_ping_period_ms = cmd.u.set_u32.value;
01005                 s_ctx.last_ping_ms = radio_main_now_ms();
01006                 cmd_result = true;
01007             }
01008             break;
01009
01010         case RADIO_MAIN_CMD_SET_AUTO_PING_MODE:
01011             if (radio_main_validate_auto_ping_mode((radio_main_auto_ping_mode_t)cmd.u.set_u32.value))
01012             {
01013                 s_ctx.auto_ping_mode = (radio_main_auto_ping_mode_t)cmd.u.set_u32.value;
01014                 s_ctx.last_ping_ms = radio_main_now_ms();
01015                 cmd_result = true;
01016             }
01017             break;
01018
01019         case RADIO_MAIN_CMD_RESET_MODULE:
01020             cmd_result = radio_main_reset_module_internal();
01021             break;
01022
01023         case RADIO_MAIN_CMD_START_PAIRING:
01024         case RADIO_MAIN_CMD_START_NETWORK_PAIRING:
01025             cmd_result = s_ctx.initialized || radio_main_force_recover_radio("start pairing");
01026             if (cmd_result && cmd.u.pairing.network_mode)
01027             {
01028                 trusted_info_t gateway_info;
01029
01030                 memset(&gateway_info, 0, sizeof(gateway_info));
01031                 if (security_main_cmd_get_device(0U, &gateway_info) && gateway_info.in_use)
01032                 {
01033                     printf("RADIO: network pairing blocked, gateway slot already in use\r\n");
01034                     cmd_result = false;
01035                 }
01036             }
01037             if (cmd_result)
01038             {
01039                 s_ctx.pairing_active = true;
01040                 s_ctx.pairing_network_mode = cmd.u.pairing.network_mode;
01041                 if (cmd.u.pairing.network_mode)
01042                 {
01043                     s_ctx.gateway_key_mode_known = false;
01044                 }
01045                 s_ctx.pairing_pending = false;
01046                 s_ctx.pairing_pending_network = false;
01047                 s_ctx.pairing_outgoing_pending = false;
01048                 s_ctx.pairing_outgoing_network = false;
01049                 s_ctx.pairing_until_ms = radio_main_now_ms() + cmd.u.pairing.timeout_ms;
01050                 if (s_ctx.initialized &&
01051                     (radio_get_state() != RADIO_STATE_TX))
01052                 {
01053                     (void)radio_start_rx_continuous();
01054                 }
01055             }
01056             break;
01057
01058         case RADIO_MAIN_CMD_PAIRING_ACCEPT:
01059             if (s_ctx.pairing_pending)
01060             {
01061                 cmd_result = radio_main_finish_pairing(cmd.u.set_bool.enabled);
01062             }
01063             break;
01064
01065         case RADIO_MAIN_CMD_SEND_PAIR_REQ:
01066             s_ctx.pairing_network_mode = false;
01067             cmd_result = radio_main_send_pair_request_internal();
01068             break;
01069
01070         case RADIO_MAIN_CMD_SEND_NETWORK_PAIR_REQ:
01071             printf("RADIO: network PAIR_REQ command rejected on node\r\n");
01072             cmd_result = false;
01073             break;
01074

```

```

01075         case RADIO_MAIN_CMD_SEND_PAIR_ERROR:
01076             cmd_result = radio_main_send_system_frame(LAVIET_TYPE_ERROR,
01077                                                         cmd.u.to_node.dst_id,
01078                                                         (const uint8_t
01079 *)"\x04\x00\x00\x00\x00\x00\x00\x00",
01079                                                         LAVIET_ERROR_PAYLOAD_LEN);
01080             break;
01081         default:
01082             break;
01083     }
01084
01085     if (cmd.sync != NULL)
01086     {
01087         cmd.sync->result = cmd_result;
01088         cmd.sync->done = true;
01089     }
01090 }
01091
01092 if (s_ctx.initialized)
01093 {
01094     radio_process();
01095     radio_main_handle_events();
01096     radio_main_watchdog_tx();
01097     radio_main_handle_ack_timeout();
01098     radio_main_ack_recent_expire();
01099     radio_main_handle_hopping();
01100     radio_main_handle_auto_ping();
01101     radio_main_ensure_rx_continuous();
01102 }
01103
01104 if (s_ctx.pairing_active && (radio_main_now_ms() >= s_ctx.pairing_until_ms))
01105 {
01106     s_ctx.pairing_active = false;
01107     s_ctx.pairing_network_mode = false;
01108     s_ctx.pairing_pending = false;
01109     s_ctx.pairing_pending_network = false;
01110     s_ctx.pairing_outgoing_pending = false;
01111     s_ctx.pairing_outgoing_network = false;
01112     radio_main_notify(MENU_NOTIFICATION_PAIRING, "Pairing timeout");
01113 }
01114
01115 {
01116     uint32_t poll_ms = (s_ctx.tx_in_progress || s_ctx.ack_pending.active) ?
01117         RADIO_ACK_ACTIVE_POLL_MS : RADIO_IDLE_POLL_MS;
01118     uint32_t auto_ping_period_ms = radio_main_auto_ping_period_ms();
01119
01120     if (s_ctx.auto_ping_enabled && (auto_ping_period_ms < poll_ms))
01121     {
01122         poll_ms = auto_ping_period_ms;
01123     }
01124     if (poll_ms == 0UL)
01125     {
01126         poll_ms = 1UL;
01127     }
01128     osDelay(poll_ms);
01129 }
01130 }
01131 }
01132 }
01133
01134 static bool radio_main_enqueue_sync(const radio_main_cmd_t *cmd, radio_main_cmd_sync_t *sync)
01135 {
01136     radio_main_cmd_t local;
01137
01138     if ((cmd == NULL) || (sync == NULL) || (s_radio_cmd_queue == NULL))
01139     {
01140         return false;
01141     }
01142
01143     memset(sync, 0, sizeof(*sync));
01144     local = *cmd;
01145     local.sync = sync;
01146
01147     if (osMessageQueuePut(s_radio_cmd_queue, &local, 0U, 100U) != osOK)
01148     {
01149         return false;
01150     }
01151
01152     return radio_main_wait_sync(sync, RADIO_CMD_WAIT_MS);
01153 }
01154
01155 static bool radio_main_wait_sync(radio_main_cmd_sync_t *sync, uint32_t timeout_ms)
01156 {
01157     uint32_t start = radio_main_now_ms();
01158
01159     if (sync == NULL)
01160     {
01161         return false;
01162     }

```

```

01163         return false;
01164     }
01165
01166     while (!sync->done)
01167     {
01168         if ((radio_main_now_ms() - start) > timeout_ms)
01169         {
01170             return false;
01171         }
01172         osDelay(RADIO_CMD_POLL_MS);
01173     }
01174
01175     return sync->result;
01176 }
01177
01178 /*
01179  * Computes a conservative TX watchdog window for the active modulation.
01180  * FSK/OOK use bitrate-derived timing, while LoRa uses a generous fixed window
01181  * because the airtime depends on SF/BW/CR/header mode and should not be
01182  * cut short by the safety watchdog.
01183  */
01184 static uint32_t radio_main_tx_timeout_ms(uint16_t payload_len)
01185 {
01186     uint64_t timeout_ms = RADIO_TX_GUARD_LORA_MS;
01187     uint32_t bitrate_bps = 0U;
01188     uint32_t total_bytes = (payload_len == 0U) ? 1UL : (uint32_t)payload_len;
01189
01190     if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK)
01191     {
01192         bitrate_bps = s_ctx.fsk_cfg.bitrate_bps;
01193         total_bytes++;
01194     }
01195     else if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK)
01196     {
01197         bitrate_bps = s_ctx.ook_cfg.bitrate_bps;
01198         total_bytes++;
01199     }
01200
01201     if (bitrate_bps > 0UL)
01202     {
01203         timeout_ms = ((uint64_t)total_bytes * 8ULL * 1000ULL) / bitrate_bps;
01204         if (((uint64_t)total_bytes * 8ULL * 1000ULL) % bitrate_bps) != 0ULL)
01205         {
01206             timeout_ms++;
01207         }
01208         timeout_ms += RADIO_TX_GUARD_MARGIN_MS;
01209         if (timeout_ms < RADIO_TX_GUARD_MIN_MS)
01210         {
01211             timeout_ms = RADIO_TX_GUARD_MIN_MS;
01212         }
01213     }
01214
01215     if (timeout_ms > 0xFFFFFFFFULL)
01216     {
01217         return 0xFFFFFFFFUL;
01218     }
01219
01220     return (uint32_t)timeout_ms;
01221 }
01222
01223 /*
01224  * Marks a locally initiated TX so the application layer can recover even if the
01225  * backend reports HW_ERROR but keeps exposing RADIO_STATE_TX for a short time.
01226  */
01227 static void radio_main_tx_mark_started_ex(uint16_t payload_len, bool silent)
01228 {
01229     s_ctx.tx_in_progress = true;
01230     s_ctx.tx_silent = silent;
01231     s_ctx.tx_deadline_ms = radio_main_now_ms() + radio_main_tx_timeout_ms(payload_len);
01232 }
01233
01234 static void radio_main_tx_clear(void)
01235 {
01236     s_ctx.tx_in_progress = false;
01237     s_ctx.tx_silent = false;
01238     s_ctx.tx_deadline_ms = 0U;
01239 }
01240
01241 static bool radio_main_tx_timed_out(void)
01242 {
01243     return (s_ctx.tx_in_progress &&
01244         ((int32_t)(radio_main_now_ms() - s_ctx.tx_deadline_ms) >= 0));
01245 }
01246
01247 static uint32_t radio_main_ack_timeout_ms(uint8_t raw_len)
01248 {
01249     uint32_t tx_airtime_ms;

```

```

01253     uint32_t ack_airtime_ms;
01254     uint32_t wait_ms;
01255     uint8_t ack_raw_len = (uint8_t)(LAVIET_FRAME_MIN_LEN + LAVIET_ACK_PAYLOAD_LEN);
01256
01257     if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK)
01258     {
01259         tx_airtime_ms = radio_main_estimate_fsk_ook_airtime_ms(raw_len, false);
01260         ack_airtime_ms = radio_main_estimate_fsk_ook_airtime_ms(ack_raw_len, false);
01261     }
01262     else if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK)
01263     {
01264         tx_airtime_ms = radio_main_estimate_fsk_ook_airtime_ms(raw_len, true);
01265         ack_airtime_ms = radio_main_estimate_fsk_ook_airtime_ms(ack_raw_len, true);
01266     }
01267     else
01268     {
01269         tx_airtime_ms = radio_main_estimate_lora_airtime_ms(raw_len);
01270         ack_airtime_ms = radio_main_estimate_lora_airtime_ms(ack_raw_len);
01271     }
01272
01273     wait_ms = tx_airtime_ms + ack_airtime_ms + RADIO_ACK_TIMEOUT_MARGIN_MS;
01274     if (wait_ms < RADIO_ACK_TIMEOUT_MIN_MS)
01275     {
01276         wait_ms = RADIO_ACK_TIMEOUT_MIN_MS;
01277     }
01278
01279     return wait_ms;
01280 }
01281
01282 static bool radio_main_ack_track_start(const laviet_frame_t *frame, const uint8_t *raw, uint8_t
raw_len)
01283 {
01284     if ((frame == NULL) || (raw == NULL) || (raw_len == 0U))
01285     {
01286         return false;
01287     }
01288     if ((frame->dst_id == LAVIET_BROADCAST_ID) ||
01289         ((frame->flags & LAVIET_FLAG_ACK_REQUIRED) == 0U) ||
01290         (laviet_frame_type(frame) == LAVIET_TYPE_ACK))
01291     {
01292         return true;
01293     }
01294     if (s_ctx.ack_pending.active)
01295     {
01296         radio_main_set_last_error("ACK pending");
01297         return false;
01298     }
01299
01300     memset(&s_ctx.ack_pending, 0, sizeof(s_ctx.ack_pending));
01301     s_ctx.ack_pending.active = true;
01302     s_ctx.ack_pending.peer_id = frame->dst_id;
01303     s_ctx.ack_pending.msg_id = frame->msg_id;
01304     s_ctx.ack_pending.counter = frame->counter;
01305     s_ctx.ack_pending.raw_len = raw_len;
01306     memcpy(s_ctx.ack_pending.raw, raw, raw_len);
01307     s_ctx.ack_pending.wait_ms = radio_main_ack_timeout_ms(raw_len);
01308     s_ctx.ack_pending.deadline_ms = radio_main_now_ms() + s_ctx.ack_pending.wait_ms;
01309     s_ctx.ack_pending.retries_done = 0U;
01310     printf("RADIO ACK wait dst=0x%04X msg=0x%04X counter=%lu timeout=%lu ms\r\n",
01311         (unsigned int)s_ctx.ack_pending.peer_id,
01312         (unsigned int)s_ctx.ack_pending.msg_id,
01313         (unsigned long)s_ctx.ack_pending.counter,
01314         (unsigned long)s_ctx.ack_pending.wait_ms);
01315     return true;
01316 }
01317
01318 static void radio_main_ack_track_close(bool timed_out)
01319 {
01320     if (!s_ctx.ack_pending.active)
01321     {
01322         return;
01323     }
01324
01325     s_ctx.ack_recent.valid = true;
01326     s_ctx.ack_recent.timed_out = timed_out;
01327     s_ctx.ack_recent.peer_id = s_ctx.ack_pending.peer_id;
01328     s_ctx.ack_recent.msg_id = s_ctx.ack_pending.msg_id;
01329     s_ctx.ack_recent.counter = s_ctx.ack_pending.counter;
01330     s_ctx.ack_recent.expires_ms = radio_main_now_ms() + RADIO_ACK_CLOSED_TTL_MS;
01331     memset(&s_ctx.ack_pending, 0, sizeof(s_ctx.ack_pending));
01332 }
01333
01334 static void radio_main_ack_recent_expire(void)
01335 {
01336     if (s_ctx.ack_recent.valid &&
01337         ((int32_t)(radio_main_now_ms() - s_ctx.ack_recent.expires_ms) >= 0))
01338     {

```



```

01342     memset(&s_ctx.ack_recent, 0, sizeof(s_ctx.ack_recent));
01343 }
01344 }
01345
01347 static void radio_main_handle_ack_frame(uint16_t src_id, uint16_t acked_msg_id, uint32_t
acked_counter)
01348 {
01349     if (s_ctx.ack_pending.active &&
01350         (src_id == s_ctx.ack_pending.peer_id) &&
01351         (acked_msg_id == s_ctx.ack_pending.msg_id) &&
01352         (acked_counter == s_ctx.ack_pending.counter))
01353     {
01354         printf("RADIO ACK delivered src=0x%04X msg=0x%04X counter=%lu retries=%u\r\n",
01355             (unsigned int)src_id,
01356             (unsigned int)acked_msg_id,
01357             (unsigned long)acked_counter,
01358             (unsigned int)s_ctx.ack_pending.retries_done);
01359         radio_main_notify(MENU_NOTIFICATION_DELIVERY, "Delivery ACK");
01360         radio_main_ack_track_close(false);
01361         return;
01362     }
01363
01364     if (s_ctx.ack_pending.active && (src_id == s_ctx.ack_pending.peer_id))
01365     {
01366         printf("RADIO ACK bad src=0x%04X ack_msg=0x%04X ack_counter=%lu expected_msg=0x%04X
expected_counter=%lu\r\n",
01367             (unsigned int)src_id,
01368             (unsigned int)acked_msg_id,
01369             (unsigned long)acked_counter,
01370             (unsigned int)s_ctx.ack_pending.msg_id,
01371             (unsigned long)s_ctx.ack_pending.counter);
01372         return;
01373     }
01374
01375     if (s_ctx.ack_recent.valid &&
01376         (src_id == s_ctx.ack_recent.peer_id) &&
01377         (acked_msg_id == s_ctx.ack_recent.msg_id) &&
01378         (acked_counter == s_ctx.ack_recent.counter))
01379     {
01380         printf("RADIO ACK %s src=0x%04X ack_msg=0x%04X ack_counter=%lu\r\n",
01381             s_ctx.ack_recent.timed_out ? "late" : "duplicate",
01382             (unsigned int)src_id,
01383             (unsigned int)acked_msg_id,
01384             (unsigned long)acked_counter);
01385         return;
01386     }
01387
01388     printf("RADIO ACK unexpected src=0x%04X ack_msg=0x%04X ack_counter=%lu\r\n",
01389         (unsigned int)src_id,
01390         (unsigned int)acked_msg_id,
01391         (unsigned long)acked_counter);
01392 }
01393
01395 static void radio_main_handle_ack_timeout(void)
01396 {
01397     if (!s_ctx.ack_pending.active || s_ctx.tx_in_progress)
01398     {
01399         return;
01400     }
01401     if ((int32_t)(radio_main_now_ms() - s_ctx.ack_pending.deadline_ms) < 0)
01402     {
01403         return;
01404     }
01405
01406     if (s_ctx.ack_pending.retries_done < RADIO_ACK_RETRY_LIMIT)
01407     {
01408         printf("RADIO ACK retry %u/%u dst=0x%04X msg=0x%04X counter=%lu\r\n",
01409             (unsigned int)(s_ctx.ack_pending.retries_done + 1U),
01410             (unsigned int)RADIO_ACK_RETRY_LIMIT,
01411             (unsigned int)s_ctx.ack_pending.peer_id,
01412             (unsigned int)s_ctx.ack_pending.msg_id,
01413             (unsigned long)s_ctx.ack_pending.counter);
01414
01415         if (radio_main_send_raw_with_retry(s_ctx.ack_pending.raw, s_ctx.ack_pending.raw_len))
01416         {
01417             s_ctx.ack_pending.retries_done++;
01418             s_ctx.ack_pending.deadline_ms = radio_main_now_ms() + s_ctx.ack_pending.wait_ms;
01419             return;
01420         }
01421
01422         printf("RADIO ACK retry send failed dst=0x%04X msg=0x%04X\r\n",
01423             (unsigned int)s_ctx.ack_pending.peer_id,
01424             (unsigned int)s_ctx.ack_pending.msg_id);
01425         s_ctx.ack_pending.deadline_ms = radio_main_now_ms() + 50UL;
01426         return;
01427     }
01428 }

```

```

01429     printf("RADIO ACK timeout dst=0x%04X msg=0x%04X counter=%lu retries=%u\r\n",
01430            (unsigned int)s_ctx.ack_pending.peer_id,
01431            (unsigned int)s_ctx.ack_pending.msg_id,
01432            (unsigned long)s_ctx.ack_pending.counter,
01433            (unsigned int)s_ctx.ack_pending.retries_done);
01434     radio_main_set_last_error("ACK timeout");
01435     radio_main_notify(MENU_NOTIFICATION_WARNING, "ACK timeout");
01436     radio_main_ack_track_close(true);
01437 }
01438
01440 static void radio_main_set_last_error(const char *text)
01441 {
01442     if (text == NULL)
01443     {
01444         s_ctx.last_error_text[0] = '\0';
01445         return;
01446     }
01447
01448     strncpy(s_ctx.last_error_text, text, sizeof(s_ctx.last_error_text) - 1U);
01449     s_ctx.last_error_text[sizeof(s_ctx.last_error_text) - 1U] = '\0';
01450 }
01451
01453 static void radio_main_clear_last_error(void)
01454 {
01455     s_ctx.last_error_text[0] = '\0';
01456 }
01457
01459 static uint32_t radio_main_current_frequency_hz(void)
01460 {
01461     if (s_ctx.modulation_id == RADIO_MAIN_MODULATION_FSK)
01462     {
01463         return s_ctx.fsk_cfg.frequency_hz;
01464     }
01465     if (s_ctx.modulation_id == RADIO_MAIN_MODULATION_OOK)
01466     {
01467         return s_ctx.oook_cfg.frequency_hz;
01468     }
01469
01470     return s_ctx.lora_cfg.frequency_hz;
01471 }
01472
01473 /*
01474  * Returns the ETSI-style duty-cycle limit for the current ISM sub-band in
01475  * permille. The auto-ping scheduler later uses half of that duty cycle to keep
01476  * a safety margin instead of driving the channel at the legal maximum.
01477  */
01478 static uint16_t radio_main_current_duty_cycle_permille(uint32_t frequency_hz)
01479 {
01480     if ((frequency_hz >= 868700000UL) && (frequency_hz <= 869200000UL))
01481     {
01482         return 1U; /* 0.1% */
01483     }
01484     if ((frequency_hz >= 869400000UL) && (frequency_hz <= 869650000UL))
01485     {
01486         return 100U; /* 10% */
01487     }
01488     if ((frequency_hz >= 863000000UL) && (frequency_hz < 868700000UL))
01489     {
01490         return 10U; /* 1% */
01491     }
01492     if ((frequency_hz > 869650000UL) && (frequency_hz <= 870000000UL))
01493     {
01494         return 10U; /* 1% */
01495     }
01496
01497     return 10U;
01498 }
01499
01501 static uint32_t radio_main_lora_bw_hz(radio_lora_bw_t bw)
01502 {
01503     switch (bw)
01504     {
01505         case RADIO_LORA_BW_7_8_KHZ:
01506             return RADIO_LORA_BW_HZ_7_8;
01507         case RADIO_LORA_BW_10_4_KHZ:
01508             return RADIO_LORA_BW_HZ_10_4;
01509         case RADIO_LORA_BW_15_6_KHZ:
01510             return RADIO_LORA_BW_HZ_15_6;
01511         case RADIO_LORA_BW_20_8_KHZ:
01512             return RADIO_LORA_BW_HZ_20_8;
01513         case RADIO_LORA_BW_31_25_KHZ:
01514             return RADIO_LORA_BW_HZ_31_25;
01515         case RADIO_LORA_BW_41_7_KHZ:
01516             return RADIO_LORA_BW_HZ_41_7;
01517         case RADIO_LORA_BW_62_5_KHZ:
01518             return RADIO_LORA_BW_HZ_62_5;
01519         case RADIO_LORA_BW_250_KHZ:

```

```

01520         return RADIO_LORA_BW_HZ_250;
01521     case RADIO_LORA_BW_500_KHZ:
01522         return RADIO_LORA_BW_HZ_500;
01523     case RADIO_LORA_BW_125_KHZ:
01524     default:
01525         return RADIO_LORA_BW_HZ_125;
01526     }
01527 }
01528
01529 /*
01530  * Estimates LoRa packet airtime using the standard SX127x packet formula.
01531  * The result is intentionally conservative, because auto-ping should stay
01532  * clearly below the band occupancy limit.
01533  */
01534 static uint32_t radio_main_estimate_lora_airtime_ms(uint16_t payload_len)
01535 {
01536     uint32_t bw_hz = radio_main_lora_bw_hz(s_ctx.lora_cfg.bandwidth);
01537     uint32_t sf = s_ctx.lora_cfg.spreading_factor;
01538     uint32_t cr = (uint32_t)(s_ctx.lora_cfg.coding_rate - 4U);
01539     uint32_t de = ((bw_hz <= RADIO_LORA_BW_HZ_125) && (sf >= 11U)) ? 1U : 0U;
01540     uint32_t ih = s_ctx.lora_cfg.implicit_header ? 1U : 0U;
01541     uint32_t crc = s_ctx.lora_cfg.crc_on ? 1U : 0U;
01542     uint64_t tsym_us;
01543     uint64_t preamble_us;
01544     int32_t numerator;
01545     int32_t denominator;
01546     int32_t ceil_term = 0;
01547     uint32_t payload_symbols;
01548     uint64_t payload_us;
01549     uint64_t total_ms;
01550
01551     if ((bw_hz == 0U) || (sf < 6U))
01552     {
01553         return RADIO_TX_GUARD_LORA_MS;
01554     }
01555
01556     tsym_us = (((uint64_t)1ULL << sf) * 1000000ULL) / (uint64_t)bw_hz;
01557     if (((uint64_t)1ULL << sf) * 1000000ULL % (uint64_t)bw_hz)
01558     {
01559         tsym_us++;
01560     }
01561
01562     preamble_us = ((uint64_t)s_ctx.lora_cfg.preamble_len * tsym_us) +
01563                   ((17ULL * tsym_us) / 4ULL);
01564     if (((17ULL * tsym_us) % 4ULL) != 0ULL)
01565     {
01566         preamble_us++;
01567     }
01568
01569     numerator = (int32_t)(8UL * payload_len) - (int32_t)(4UL * sf) + 28 +
01570                (int32_t)(16UL * crc) - (int32_t)(20UL * ih);
01571     denominator = (int32_t)(4UL * (sf - (2UL * de)));
01572     if ((numerator > 0) && (denominator > 0))
01573     {
01574         ceil_term = (numerator + denominator - 1) / denominator;
01575     }
01576
01577     payload_symbols = 8U;
01578     if (ceil_term > 0)
01579     {
01580         payload_symbols += (uint32_t)ceil_term * (cr + 4U);
01581     }
01582
01583     payload_us = (uint64_t)payload_symbols * tsym_us;
01584     total_ms = (preamble_us + payload_us + 999ULL) / 1000ULL;
01585     if (total_ms == 0ULL)
01586     {
01587         total_ms = 1ULL;
01588     }
01589     if (total_ms > 0xFFFFFFFFULL)
01590     {
01591         return 0xFFFFFFFFFUL;
01592     }
01593
01594     return (uint32_t)total_ms;
01595 }
01596
01597 /*
01598  * Estimates on-air time for FSK/OOK by accounting for preamble, sync word,
01599  * packet-length byte, optional CRC and a small extra guard for packet-engine
01600  * framing overhead. This is conservative on purpose for auto-ping throttling.
01601  */
01602 static uint32_t radio_main_estimate_fsk_ook_airtime_ms(uint16_t payload_len, bool ook_mode)
01603 {
01604     uint32_t bitrate_bps;
01605     uint32_t preamble_len;
01606     uint32_t sync_len;

```

```

01607     uint32_t crc_len = 0U;
01608     uint32_t total_bytes;
01609     uint64_t total_ms;
01610
01611     if (ook_mode)
01612     {
01613         bitrate_bps = s_ctx.ook_cfg.bitrate_bps;
01614         preamble_len = s_ctx.ook_cfg.preamble_len;
01615         sync_len = s_ctx.ook_cfg.sync_word_len;
01616     }
01617     else
01618     {
01619         bitrate_bps = s_ctx.fsk_cfg.bitrate_bps;
01620         preamble_len = s_ctx.fsk_cfg.preamble_len;
01621         sync_len = s_ctx.fsk_cfg.sync_word_len;
01622         if (s_ctx.fsk_cfg.crc_type != RADIO_MAIN_CRC_OFF)
01623         {
01624             crc_len = 2U;
01625         }
01626     }
01627
01628     if (bitrate_bps == 0UL)
01629     {
01630         return RADIO_TX_GUARD_MIN_MS;
01631     }
01632
01633     total_bytes = preamble_len + sync_len + 1UL + payload_len + crc_len + 2UL;
01634     total_ms = ((uint64_t)total_bytes * 8ULL * 1000ULL) / bitrate_bps;
01635     if (((uint64_t)total_bytes * 8ULL * 1000ULL) % bitrate_bps != 0ULL)
01636     {
01637         total_ms++;
01638     }
01639     total_ms += RADIO_TX_GUARD_MARGIN_MS;
01640
01641     if (total_ms < RADIO_TX_GUARD_MIN_MS)
01642     {
01643         total_ms = RADIO_TX_GUARD_MIN_MS;
01644     }
01645     if (total_ms > 0xFFFFFFFFULL)
01646     {
01647         return 0xFFFFFFFFUL;
01648     }
01649
01650     return (uint32_t)total_ms;
01651 }
01652
01654 static uint32_t radio_main_auto_ping_airtime_ms(void)
01655 {
01656     const char *msg = s_template_groups[2][0];
01657     uint16_t payload_len = (uint16_t)strlen(msg);
01658     uint16_t raw_len;
01659
01660     if (s_ctx.auto_ping_mode == RADIO_MAIN_AUTO_PING_RAW)
01661     {
01662         raw_len = RADIO_AUTO_PING_RAW_LEN;
01663     }
01664     else
01665     {
01666         if (payload_len > LAVIET_MAX_PAYLOAD)
01667         {
01668             payload_len = LAVIET_MAX_PAYLOAD;
01669         }
01670         raw_len = (uint16_t)(LAVIET_FRAME_HEADER_LEN + payload_len + LAVIET_MAC_TAG_LEN);
01671     }
01672
01673     if (s_ctx.modulation_id == RADIO_MAIN_MODULATION_FSK)
01674     {
01675         return radio_main_estimate_fsk_ook_airtime_ms(raw_len, false);
01676     }
01677     if (s_ctx.modulation_id == RADIO_MAIN_MODULATION_OOK)
01678     {
01679         return radio_main_estimate_fsk_ook_airtime_ms(raw_len, true);
01680     }
01681
01682     return radio_main_estimate_lora_airtime_ms(raw_len);
01683 }
01684
01686 static uint32_t radio_main_auto_ping_period_ms(void)
01687 {
01688     uint16_t duty_permille;
01689     uint32_t airtime_ms;
01690     uint64_t period_ms;
01691
01692     if (radio_main_validate_auto_ping_period(s_ctx.auto_ping_period_ms))
01693     {
01694         return s_ctx.auto_ping_period_ms;
01695     }

```

```

01696
01697     duty_permille = radio_main_current_duty_cycle_permille(radio_main_current_frequency_hz());
01698     airtime_ms = radio_main_auto_ping_airtime_ms();
01699     if (duty_permille == 0U)
01700     {
01701         return RADIO_AUTO_PING_MIN_PERIOD_MS;
01702     }
01703
01704     period_ms = ((uint64_t)airtime_ms * 2000ULL) / duty_permille;
01705     if (((uint64_t)airtime_ms * 2000ULL) % duty_permille) != 0ULL)
01706     {
01707         period_ms++;
01708     }
01709
01710     if (period_ms < RADIO_AUTO_PING_MIN_PERIOD_MS)
01711     {
01712         period_ms = RADIO_AUTO_PING_MIN_PERIOD_MS;
01713     }
01714     if (period_ms > 0xFFFFFFFFULL)
01715     {
01716         return 0xFFFFFFFFUL;
01717     }
01718
01719     return (uint32_t)period_ms;
01720 }
01721
01722 static bool radio_main_radio_init_and_start(void)
01723 {
01724     radio_status_t st;
01725
01726     if (!radio_main_push_backend_cfg())
01727     {
01728         return false;
01729     }
01730
01731     st = radio_init(&s_ctx.hw, &s_ctx.backend_cfg, NULL, NULL);
01732     if (st != RADIO_OK)
01733     {
01734         return false;
01735     }
01736
01737     st = radio_start_rx_continuous();
01738     if (st != RADIO_OK)
01739     {
01740         (void)radio_deinit();
01741         return false;
01742     }
01743
01744     s_ctx.initialized = true;
01745     radio_main_tx_clear();
01746     radio_main_log_runtime_profile("radio init");
01747     radio_main_log_hw_registers("radio init");
01748     return true;
01749 }
01750
01751 void radio_main_load_default_lora_preset(uint8_t preset_id, radio_lora_cfg_t *cfg)
01752 {
01753     if (cfg == NULL)
01754     {
01755         return;
01756     }
01757
01758     switch (preset_id)
01759     {
01760     {
01761         case 0U: /* STD */
01762             cfg->frequency_hz = 868100000UL;
01763             cfg->bandwidth = RADIO_LORA_BW_125_KHZ;
01764             cfg->spreading_factor = 7U;
01765             cfg->coding_rate = 5U;
01766             cfg->preamble_len = 8U;
01767             cfg->sync_word = 0x34U;
01768             cfg->crc_on = true;
01769             cfg->invert_iq = false;
01770             cfg->tx_power_dbm = 14;
01771             cfg->implicit_header = false;
01772             cfg->payload_len = 0U;
01773             break;
01774
01775         case 1U: /* RANGE */
01776             cfg->frequency_hz = 868300000UL;
01777             cfg->bandwidth = RADIO_LORA_BW_125_KHZ;
01778             cfg->spreading_factor = 12U;
01779             cfg->coding_rate = 5U;
01780             cfg->preamble_len = 12U;
01781             cfg->sync_word = 0x34U;
01782             cfg->crc_on = true;
01783             cfg->invert_iq = false;

```

```

01784         cfg->tx_power_dbm = 14;
01785         cfg->implicit_header = false;
01786         cfg->payload_len = 0U;
01787         break;
01788
01789         case 2U: /* FAST */
01790             cfg->frequency_hz = 868500000UL;
01791             cfg->bandwidth = RADIO_LORA_BW_500_KHZ;
01792             cfg->spreading_factor = 7U;
01793             cfg->coding_rate = 5U;
01794             cfg->preamble_len = 8U;
01795             cfg->sync_word = 0x34U;
01796             cfg->crc_on = true;
01797             cfg->invert_iq = false;
01798             cfg->tx_power_dbm = 17;
01799             cfg->implicit_header = false;
01800             cfg->payload_len = 0U;
01801             break;
01802
01803         default:
01804             break;
01805     }
01806 }
01807
01809 static void radio_main_apply_preset_cfg(uint8_t preset_id, radio_lora_cfg_t *cfg)
01810 {
01811     radio_main_load_default_lora_preset(preset_id, cfg);
01812 }
01813
01821 static void radio_main_load_default_profiles(void)
01822 {
01823     radio_main_load_default_fsk_profile(&s_ctx.fsk_cfg);
01824     radio_main_load_default_ook_profile(&s_ctx.ook_cfg);
01825 }
01826
01827 void radio_main_load_default_fsk_profile(radio_main_fsk_cfg_t *cfg)
01828 {
01829     if (cfg == NULL)
01830     {
01831         return;
01832     }
01833
01834     memset(cfg, 0, sizeof(*cfg));
01835     cfg->shaping = RADIO_MAIN_FSK_SHAPING_GFSK;
01836     cfg->frequency_hz = 868300000UL;
01837     cfg->bitrate_bps = 4800UL;
01838     cfg->rx_bandwidth = RADIO_LORA_BW_125_KHZ;
01839     cfg->filter = RADIO_MAIN_FILTER_BT_05;
01840     cfg->tx_power_dbm = 14;
01841     cfg->preamble_len = 8U;
01842     cfg->sync_word_len = 2U;
01843     cfg->sync_word = 0x00002DD4UL;
01844     cfg->address_filter = RADIO_MAIN_ADDRESS_FILTER_NONE;
01845     cfg->crc_type = RADIO_MAIN_CRC_CCITT;
01846     cfg->data_whitening = true;
01847 }
01848
01849 void radio_main_load_default_ook_profile(radio_main_ook_cfg_t *cfg)
01850 {
01851     if (cfg == NULL)
01852     {
01853         return;
01854     }
01855
01856     memset(cfg, 0, sizeof(*cfg));
01857     cfg->frequency_hz = 868500000UL;
01858     cfg->bitrate_bps = 4800UL;
01859     cfg->rx_bandwidth = RADIO_LORA_BW_125_KHZ;
01860     cfg->tx_power_dbm = 10;
01861     cfg->preamble_len = 8U;
01862     cfg->sync_word_len = 2U;
01863     cfg->sync_word = 0x00002DD4UL;
01864     cfg->threshold = RADIO_MAIN_OOK_THRESHOLD_PEAK;
01865     cfg->threshold_value = 12U;
01866 }
01867
01869 static void radio_main_sync_snapshot(radio_main_runtime_cfg_t *cfg)
01870 {
01871     if (cfg == NULL)
01872     {
01873         return;
01874     }
01875
01876     memset(cfg, 0, sizeof(*cfg));
01877     cfg->active_modulation = s_ctx.modulation_id;
01878     cfg->lora = s_ctx.lora_cfg;
01879     cfg->fsk = s_ctx.fsk_cfg;

```

```

01880     cfg->ook = s_ctx.ook_cfg;
01881     cfg->fh_enabled = s_ctx.fh_enabled;
01882     cfg->fh_period_ms = s_ctx.fh_period_ms;
01883     cfg->coding_enabled = s_ctx.coding_enabled;
01884     cfg->auto_ping_enabled = s_ctx.auto_ping_enabled;
01885     cfg->auto_ping_period_ms = radio_main_auto_ping_period_ms();
01886     cfg->auto_ping_mode = s_ctx.auto_ping_mode;
01887 }
01888
01897 static void radio_main_apply_modulation_cfg(void)
01898 {
01899     s_ctx.backend_cfg = s_ctx.lora_cfg;
01900
01901     if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA)
01902     {
01903         if (s_ctx.backend_cfg.implicit_header && (s_ctx.backend_cfg.payload_len == 0U))
01904         {
01905             /*
01906              * BEKO wysyła ramki o zmiennej długości, więc implicit header traktujemy
01907              * jako tryb eksperymentalny. Wypełniamy `payload_len`, aby backend LoRa
01908              * zaakceptował konfigurację przy starcie.
01909              */
01910             s_ctx.backend_cfg.payload_len = LAVIET_FRAME_MAX_LEN;
01911         }
01912         return;
01913     }
01914 }
01915
01916 static bool radio_main_switch_backend(radio_main_modulation_t modulation)
01917 {
01918     radio_main_modulation_t previous_modulation = s_ctx.backend_modulation_id;
01919     bool was_initialized = s_ctx.initialized;
01920     bool ok;
01921
01922     s_ctx.backend_modulation_id = modulation;
01923     radio_main_apply_modulation_cfg();
01924     ok = s_ctx.initialized ? radio_main_reconfigure_radio() : radio_main_radio_init_and_start();
01925     if (ok)
01926     {
01927         return true;
01928     }
01929
01930     printf("RADIO: backend switch to %u failed, restore %u\r\n",
01931           (unsigned int)modulation,
01932           (unsigned int)previous_modulation);
01933     s_ctx.backend_modulation_id = previous_modulation;
01934     radio_main_apply_modulation_cfg();
01935     if (was_initialized && !s_ctx.initialized)
01936     {
01937         (void)radio_main_radio_init_and_start();
01938     }
01939     return false;
01940 }
01941
01942 static void radio_main_load_default_profile(radio_main_modulation_t modulation)
01943 {
01944     switch (modulation)
01945     {
01946     case RADIO_MAIN_MODULATION_FSK:
01947         radio_main_load_default_fsk_profile(&s_ctx.fsk_cfg);
01948         break;
01949     case RADIO_MAIN_MODULATION_OOK:
01950         radio_main_load_default_ook_profile(&s_ctx.ook_cfg);
01951         break;
01952     case RADIO_MAIN_MODULATION_LORA:
01953     default:
01954         radio_main_apply_preset_cfg(s_ctx.lora_preset, &s_ctx.lora_cfg);
01955         break;
01956     }
01957     radio_main_apply_modulation_cfg();
01958 }
01959
01968 static radio_packet_crc_t radio_main_map_fsk_crc(radio_main_crc_type_t crc_type)
01969 {
01970     switch (crc_type)
01971     {
01972     case RADIO_MAIN_CRC_IBM:
01973         return RADIO_PACKET_CRC_IBM;
01974     case RADIO_MAIN_CRC_CCITT:
01975         return RADIO_PACKET_CRC_CCITT;
01976     }
01977 }

```

```

01978         case RADIO_MAIN_CRC_OFF:
01979         case RADIO_MAIN_CRC_SX1276:
01980         default:
01981             return RADIO_PACKET_CRC_OFF;
01982     }
01983 }
01984
01986 static radio_address_filter_t radio_main_map_fsk_address_filter(radio_main_address_filter_t filter)
01987 {
01988     switch (filter)
01989     {
01990         case RADIO_MAIN_ADDRESS_FILTER_NODE:
01991             return RADIO_ADDRESS_FILTER_NODE;
01992
01993         case RADIO_MAIN_ADDRESS_FILTER_NODE_BROADCAST:
01994             return RADIO_ADDRESS_FILTER_NODE_BROADCAST;
01995
01996         case RADIO_MAIN_ADDRESS_FILTER_NONE:
01997         default:
01998             return RADIO_ADDRESS_FILTER_OFF;
01999     }
02000 }
02001
02003 static radio_ook_threshold_t radio_main_map_ook_threshold(radio_main_ook_threshold_t threshold)
02004 {
02005     switch (threshold)
02006     {
02007         case RADIO_MAIN_OOK_THRESHOLD_FIXED:
02008             return RADIO_OOK_THRESHOLD_FIXED;
02009
02010         case RADIO_MAIN_OOK_THRESHOLD_AVERAGE:
02011             return RADIO_OOK_THRESHOLD_AVERAGE;
02012
02013         case RADIO_MAIN_OOK_THRESHOLD_PEAK:
02014         default:
02015             return RADIO_OOK_THRESHOLD_PEAK;
02016     }
02017 }
02018
02026 static bool radio_main_push_backend_cfg(void)
02027 {
02028     radio_status_t st;
02029
02030     if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK)
02031     {
02032         radio_fsk_cfg_t cfg;
02033
02034         memset(&cfg, 0, sizeof(cfg));
02035         cfg.frequency_hz = s_ctx.fsk_cfg.frequency_hz;
02036         cfg.bitrate_bps = s_ctx.fsk_cfg.bitrate_bps;
02037         cfg.rx_bandwidth = s_ctx.fsk_cfg.rx_bandwidth;
02038         cfg.shaping = (radio_fsk_shaping_t)s_ctx.fsk_cfg.shaping;
02039         cfg.filter = (radio_fsk_filter_t)s_ctx.fsk_cfg.filter;
02040         cfg.tx_power_dbm = s_ctx.fsk_cfg.tx_power_dbm;
02041         cfg.preamble_len = s_ctx.fsk_cfg.preamble_len;
02042         cfg.sync_word_len = s_ctx.fsk_cfg.sync_word_len;
02043         cfg.sync_word = (uint32_t)(s_ctx.fsk_cfg.sync_word & 0xFFFFFFFFFUL);
02044         cfg.address_filter = radio_main_map_fsk_address_filter(s_ctx.fsk_cfg.address_filter);
02045         cfg.crc_type = radio_main_map_fsk_crc(s_ctx.fsk_cfg.crc_type);
02046         cfg.data_whitening = s_ctx.fsk_cfg.data_whitening;
02047
02048         radio_select_backend(RADIO_LIB_MODULATION_FSK);
02049         st = radio_set_fsk_cfg(&cfg);
02050         if (st != RADIO_OK)
02051         {
02052             printf("RADIO: FSK cfg reject st=%d\r\n", (int)st);
02053         }
02054         return (st == RADIO_OK);
02055     }
02056
02057     if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK)
02058     {
02059         radio_ook_cfg_t cfg;
02060
02061         memset(&cfg, 0, sizeof(cfg));
02062         cfg.frequency_hz = s_ctx.ook_cfg.frequency_hz;
02063         cfg.bitrate_bps = s_ctx.ook_cfg.bitrate_bps;
02064         cfg.rx_bandwidth = s_ctx.ook_cfg.rx_bandwidth;
02065         cfg.tx_power_dbm = s_ctx.ook_cfg.tx_power_dbm;
02066         cfg.preamble_len = s_ctx.ook_cfg.preamble_len;
02067         cfg.sync_word_len = s_ctx.ook_cfg.sync_word_len;
02068         cfg.sync_word = s_ctx.ook_cfg.sync_word;
02069         cfg.threshold = radio_main_map_ook_threshold(s_ctx.ook_cfg.threshold);
02070         cfg.threshold_value = s_ctx.ook_cfg.threshold_value;
02071
02072         radio_select_backend(RADIO_LIB_MODULATION_OOK);
02073         st = radio_set_ook_cfg(&cfg);

```



```
02074         if (st != RADIO_OK)
02075         {
02076             printf("RADIO: OOK cfg reject st=%d\r\n", (int)st);
02077         }
02078         return (st == RADIO_OK);
02079     }
02080
02081     radio_select_backend(RADIO_LIB_MODULATION_LORA);
02082     return true;
02083 }
02084
02086 static bool radio_main_validate_frequency(uint32_t frequency_hz)
02087 {
02088     return ((frequency_hz >= 863000000UL) &&
02089             (frequency_hz <= 8700000000UL));
02090 }
02091
02093 static bool radio_main_validate_tx_power(int32_t tx_power_dbm)
02094 {
02095     return ((tx_power_dbm >= 2) && (tx_power_dbm <= 20));
02096 }
02097
02099 static bool radio_main_validate_bitrate(uint32_t bitrate_bps)
02100 {
02101     return ((bitrate_bps >= 600UL) &&
02102             (bitrate_bps <= 300000UL));
02103 }
02104
02106 static bool radio_main_validate_preamble(uint32_t preamble_len)
02107 {
02108     return ((preamble_len >= 1UL) &&
02109             (preamble_len <= 65535UL));
02110 }
02111
02113 static bool radio_main_validate_hop_period(uint32_t period_ms)
02114 {
02115     return ((period_ms >= 250UL) && (period_ms <= 60000UL));
02116 }
02117
02119 static bool radio_main_validate_auto_ping_period(uint32_t period_ms)
02120 {
02121     return ((period_ms == 1UL) ||
02122             (period_ms == 10UL) ||
02123             (period_ms == 100UL) ||
02124             (period_ms == 1000UL) ||
02125             (period_ms == 10000UL));
02126 }
02127
02129 static bool radio_main_validate_auto_ping_mode(radio_main_auto_ping_mode_t mode)
02130 {
02131     return ((mode == RADIO_MAIN_AUTO_PING_FRAME) ||
02132             (mode == RADIO_MAIN_AUTO_PING_RAW));
02133 }
02134
02136 static bool radio_main_apply_option(radio_main_option_t option, uint32_t value)
02137 {
02138     radio_main_ctx_t saved_ctx;
02139     bool reconfigure_now = false;
02140     bool was_initialized;
02141
02142     saved_ctx = s_ctx;
02143     was_initialized = s_ctx.initialized;
02144
02145     switch (option)
02146     {
02147     case RADIO_MAIN_OPTION_LORA_PRESET:
02148         if (value > 2UL)
02149         {
02150             return false;
02151         }
02152         s_ctx.lora_preset = (uint8_t)value;
02153         radio_main_apply_preset_cfg(s_ctx.lora_preset, &s_ctx.lora_cfg);
02154         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA);
02155         break;
02156
02157     case RADIO_MAIN_OPTION_LORA_FREQ:
02158         if (!radio_main_validate_frequency(value))
02159         {
02160             return false;
02161         }
02162         s_ctx.lora_cfg.frequency_hz = value;
02163         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA);
02164         break;
02165
02166     case RADIO_MAIN_OPTION_LORA_BW:
02167         if (!radio_main_is_supported_bw((uint8_t)value))
02168         {
```

```
02169         return false;
02170     }
02171     s_ctx.lora_cfg.bandwidth = (radio_lora_bw_t)value;
02172     reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA);
02173     break;
02174
02175 case RADIO_MAIN_OPTION_LORA_SF:
02176     if ((value < 6UL) || (value > 12UL))
02177     {
02178         return false;
02179     }
02180     s_ctx.lora_cfg.spreading_factor = (uint8_t)value;
02181     if (value == 6UL)
02182     {
02183         s_ctx.lora_cfg.implicit_header = true;
02184         s_ctx.lora_cfg.payload_len = LAVIET_FRAME_MAX_LEN;
02185     }
02186     reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA);
02187     break;
02188
02189 case RADIO_MAIN_OPTION_LORA_CR:
02190     if ((value < 5UL) || (value > 8UL))
02191     {
02192         return false;
02193     }
02194     s_ctx.lora_cfg.coding_rate = (uint8_t)value;
02195     reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA);
02196     break;
02197
02198 case RADIO_MAIN_OPTION_LORA_TX_POWER:
02199     if (!radio_main_validate_tx_power((int32_t)value))
02200     {
02201         return false;
02202     }
02203     s_ctx.lora_cfg.tx_power_dbm = (int8_t)value;
02204     reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA);
02205     break;
02206
02207 case RADIO_MAIN_OPTION_LORA_CRC:
02208     if ((value != (uint32_t)RADIO_MAIN_CRC_OFF) &&
02209         (value != (uint32_t)RADIO_MAIN_CRC_SX1276))
02210     {
02211         return false;
02212     }
02213     s_ctx.lora_cfg.crc_on = (value != (uint32_t)RADIO_MAIN_CRC_OFF);
02214     reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA);
02215     break;
02216
02217 case RADIO_MAIN_OPTION_LORA_PREAMBLE:
02218     if (!radio_main_validate_preamble(value))
02219     {
02220         return false;
02221     }
02222     s_ctx.lora_cfg.preamble_len = (uint16_t)value;
02223     reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA);
02224     break;
02225
02226 case RADIO_MAIN_OPTION_LORA_HEADER_MODE:
02227     if (value > (uint32_t)RADIO_MAIN_HEADER_IMPLICIT)
02228     {
02229         return false;
02230     }
02231     if ((value == (uint32_t)RADIO_MAIN_HEADER_EXPLICIT) &&
02232         (s_ctx.lora_cfg.spreading_factor == 6U))
02233     {
02234         return false;
02235     }
02236     s_ctx.lora_cfg.implicit_header = (value == (uint32_t)RADIO_MAIN_HEADER_IMPLICIT);
02237     if (s_ctx.lora_cfg.implicit_header)
02238     {
02239         s_ctx.lora_cfg.payload_len = LAVIET_FRAME_MAX_LEN;
02240     }
02241     else
02242     {
02243         s_ctx.lora_cfg.payload_len = 0U;
02244     }
02245     reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA);
02246     break;
02247
02248 case RADIO_MAIN_OPTION_LORA_IQ_INVERT:
02249     if (value > 1UL)
02250     {
02251         return false;
02252     }
02253     s_ctx.lora_cfg.invert_iq = (value != 0UL);
02254     reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA);
02255     break;
```

```
02256
02257     case RADIO_MAIN_OPTION_LORA_SYNC_WORD:
02258         s_ctx.lora_cfg.sync_word = (uint8_t)(value & 0xFFU);
02259         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA);
02260         break;
02261
02262     case RADIO_MAIN_OPTION_LORA_RESET_DEFAULTS:
02263         s_ctx.lora_preset = 2U;
02264         radio_main_apply_preset_cfg(s_ctx.lora_preset, &s_ctx.lora_cfg);
02265         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA);
02266         break;
02267
02268     case RADIO_MAIN_OPTION_FSK_SHAPING:
02269         if (value > (uint32_t)RADIO_MAIN_FSK_SHAPING_GMSK)
02270         {
02271             return false;
02272         }
02273         s_ctx.fsk_cfg.shaping = (radio_main_fsk_shaping_t)value;
02274         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
02275         break;
02276
02277     case RADIO_MAIN_OPTION_FSK_FREQ:
02278         if (!radio_main_validate_frequency(value))
02279         {
02280             return false;
02281         }
02282         s_ctx.fsk_cfg.frequency_hz = value;
02283         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
02284         break;
02285
02286     case RADIO_MAIN_OPTION_FSK_BITRATE:
02287         if (!radio_main_validate_bitrate(value))
02288         {
02289             return false;
02290         }
02291         s_ctx.fsk_cfg.bitrate_bps = value;
02292         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
02293         break;
02294
02295     case RADIO_MAIN_OPTION_FSK_RX_BW:
02296         if (!radio_main_is_supported_bw((uint8_t)value))
02297         {
02298             return false;
02299         }
02300         s_ctx.fsk_cfg.rx_bandwidth = (radio_lora_bw_t)value;
02301         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
02302         break;
02303
02304     case RADIO_MAIN_OPTION_FSK_FILTER:
02305         if (value > (uint32_t)RADIO_MAIN_FILTER_BT_03)
02306         {
02307             return false;
02308         }
02309         s_ctx.fsk_cfg.filter = (radio_main_filter_t)value;
02310         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
02311         break;
02312
02313     case RADIO_MAIN_OPTION_FSK_TX_POWER:
02314         if (!radio_main_validate_tx_power((int32_t)value))
02315         {
02316             return false;
02317         }
02318         s_ctx.fsk_cfg.tx_power_dbm = (int8_t)value;
02319         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
02320         break;
02321
02322     case RADIO_MAIN_OPTION_FSK_PREAMBLE:
02323         if (!radio_main_validate_preamble(value))
02324         {
02325             return false;
02326         }
02327         s_ctx.fsk_cfg.preamble_len = (uint16_t)value;
02328         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
02329         break;
02330
02331     case RADIO_MAIN_OPTION_FSK_SYNC_LEN:
02332         if (value > 4UL)
02333         {
02334             return false;
02335         }
02336         s_ctx.fsk_cfg.sync_word_len = (uint8_t)value;
02337         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
02338         break;
02339
02340     case RADIO_MAIN_OPTION_FSK_SYNC_WORD:
02341         s_ctx.fsk_cfg.sync_word = value;
02342         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
```

```
02343         break;
02344
02345     case RADIO_MAIN_OPTION_FSK_ADDRESS_FILTER:
02346         if (value > (uint32_t)RADIO_MAIN_ADDRESS_FILTER_NODE_BROADCAST)
02347         {
02348             return false;
02349         }
02350         s_ctx.fsk_cfg.address_filter = (radio_main_address_filter_t)value;
02351         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
02352         break;
02353
02354     case RADIO_MAIN_OPTION_FSK_CRC:
02355         if (value > (uint32_t)RADIO_MAIN_CRC_CCITT)
02356         {
02357             return false;
02358         }
02359         s_ctx.fsk_cfg.crc_type = (radio_main_crc_type_t)value;
02360         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
02361         break;
02362
02363     case RADIO_MAIN_OPTION_FSK_WHITENING:
02364         if (value > 1UL)
02365         {
02366             return false;
02367         }
02368         s_ctx.fsk_cfg.data_whitening = (value != 0UL);
02369         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
02370         break;
02371
02372     case RADIO_MAIN_OPTION_FSK_RESET_DEFAULTS:
02373         radio_main_load_default_fsk_profile(&s_ctx.fsk_cfg);
02374         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK);
02375         break;
02376
02377     case RADIO_MAIN_OPTION_OOK_FREQ:
02378         if (!radio_main_validate_frequency(value))
02379         {
02380             return false;
02381         }
02382         s_ctx.ook_cfg.frequency_hz = value;
02383         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK);
02384         break;
02385
02386     case RADIO_MAIN_OPTION_OOK_BITRATE:
02387         if (!radio_main_validate_bitrate(value))
02388         {
02389             return false;
02390         }
02391         s_ctx.ook_cfg.bitrate_bps = value;
02392         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK);
02393         break;
02394
02395     case RADIO_MAIN_OPTION_OOK_TX_POWER:
02396         if (!radio_main_validate_tx_power((int32_t)value))
02397         {
02398             return false;
02399         }
02400         s_ctx.ook_cfg.tx_power_dbm = (int8_t)value;
02401         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK);
02402         break;
02403
02404     case RADIO_MAIN_OPTION_OOK_RX_BW:
02405         if (!radio_main_is_supported_bw((uint8_t)value))
02406         {
02407             return false;
02408         }
02409         s_ctx.ook_cfg.rx_bandwidth = (radio_lora_bw_t)value;
02410         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK);
02411         break;
02412
02413     case RADIO_MAIN_OPTION_OOK_PREAMBLE:
02414         if (!radio_main_validate_preamble(value))
02415         {
02416             return false;
02417         }
02418         s_ctx.ook_cfg.preamble_len = (uint16_t)value;
02419         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK);
02420         break;
02421
02422     case RADIO_MAIN_OPTION_OOK_SYNC_LEN:
02423         if (value > 4UL)
02424         {
02425             return false;
02426         }
02427         s_ctx.ook_cfg.sync_word_len = (uint8_t)value;
02428         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK);
02429         break;
```

```

02430
02431     case RADIO_MAIN_OPTION_OOK_SYNC_WORD:
02432         s_ctx.ook_cfg.sync_word = value;
02433         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK);
02434         break;
02435
02436     case RADIO_MAIN_OPTION_OOK_THRESHOLD_TYPE:
02437         if (value > (uint32_t)RADIO_MAIN_OOK_THRESHOLD_AVERAGE)
02438         {
02439             return false;
02440         }
02441         s_ctx.ook_cfg.threshold = (radio_main_ook_threshold_t)value;
02442         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK);
02443         break;
02444
02445     case RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE:
02446         if (value > 255UL)
02447         {
02448             return false;
02449         }
02450         s_ctx.ook_cfg.threshold_value = (uint8_t)value;
02451         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK);
02452         break;
02453
02454     case RADIO_MAIN_OPTION_OOK_RESET_DEFAULTS:
02455         radio_main_load_default_ook_profile(&s_ctx.ook_cfg);
02456         reconfigure_now = (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK);
02457         break;
02458
02459     default:
02460         return false;
02461 }
02462
02463 radio_main_apply_modulation_cfg();
02464 if (!was_initialized || !reconfigure_now)
02465 {
02466     return true;
02467 }
02468
02469 if (radio_main_reconfigure_radio())
02470 {
02471     return true;
02472 }
02473
02474 /*
02475  * Roll back the in-memory profile when the hardware rejects the new setting.
02476  * Without this, one bad combination leaves the app with a broken runtime config
02477  * and every later send/change keeps failing until a manual reset.
02478  */
02479 s_ctx = saved_ctx;
02480 radio_main_apply_modulation_cfg();
02481 s_ctx.initialized = false;
02482 (void)radio_main_radio_init_and_start();
02483 return false;
02484 }
02485
02486 static bool radio_main_is_supported_bw(uint8_t bw_code)
02487 {
02488     return ((bw_code == (uint8_t)RADIO_LORA_BW_7_8_KHZ) ||
02489             (bw_code == (uint8_t)RADIO_LORA_BW_10_4_KHZ) ||
02490             (bw_code == (uint8_t)RADIO_LORA_BW_15_6_KHZ) ||
02491             (bw_code == (uint8_t)RADIO_LORA_BW_20_8_KHZ) ||
02492             (bw_code == (uint8_t)RADIO_LORA_BW_31_25_KHZ) ||
02493             (bw_code == (uint8_t)RADIO_LORA_BW_41_7_KHZ) ||
02494             (bw_code == (uint8_t)RADIO_LORA_BW_62_5_KHZ) ||
02495             (bw_code == (uint8_t)RADIO_LORA_BW_125_KHZ) ||
02496             (bw_code == (uint8_t)RADIO_LORA_BW_250_KHZ) ||
02497             (bw_code == (uint8_t)RADIO_LORA_BW_500_KHZ));
02498 }
02499 }
02500
02501 static bool radio_main_reconfigure_radio(void)
02502 {
02503     if (!s_ctx.initialized)
02504     {
02505         return false;
02506     }
02507     if (radio_get_state() == RADIO_STATE_TX)
02508     {
02509         if (!radio_main_tx_timed_out())
02510         {
02511             return false;
02512         }
02513         return radio_main_force_recover_radio("reconfigure while TX stuck");
02514     }
02515 }
02516
02517 radio_main_tx_clear();
02518 (void)radio_standby();

```

```

02519     (void)radio_deinit();
02520     s_ctx.initialized = false;
02521     return radio_main_radio_init_and_start();
02522 }
02523
02524 /*
02525  * Forces a full backend teardown and optional hardware reset of SX1276.
02526  * This path is used when TX appears stuck, so it deliberately ignores the
02527  * normal "don't touch radio during TX" guard and rebuilds the runtime state
02528  * from the currently selected profile.
02529  */
02530 static bool radio_main_force_recover_radio(const char *reason)
02531 {
02532     uint8_t attempt;
02533
02534     printf("RADIO: recovery start (%s) state=%d backend=%u\r\n",
02535           (reason != NULL) ? reason : "unknown",
02536           (int)radio_get_state(),
02537           (unsigned int)radio_get_backend());
02538
02539     radio_main_tx_clear();
02540
02541     for (attempt = 0U; attempt < RADIO_RECOVERY_RETRY_COUNT; attempt++)
02542     {
02543         (void)radio_sleep();
02544         (void)radio_standby();
02545         (void)radio_deinit();
02546         s_ctx.initialized = false;
02547
02548         if ((s_ctx.hw.reset.port != NULL) && (s_ctx.hw.reset.pin != 0U))
02549         {
02550             HAL_GPIO_WritePin(s_ctx.hw.reset.port, s_ctx.hw.reset.pin, GPIO_PIN_RESET);
02551             osDelay(RADIO_RECOVERY_RESET_PULSE_MS);
02552             HAL_GPIO_WritePin(s_ctx.hw.reset.port, s_ctx.hw.reset.pin, GPIO_PIN_SET);
02553             osDelay(RADIO_RECOVERY_RESET_BOOT_MS);
02554         }
02555
02556         radio_main_apply_modulation_cfg();
02557         if (radio_main_radio_init_and_start())
02558         {
02559             printf("RADIO: recovery OK attempt=%u\r\n", (unsigned int)(attempt + 1U));
02560             return true;
02561         }
02562     }
02563
02564     printf("RADIO: recovery FAILED\r\n");
02565     return false;
02566 }
02567
02568 /*
02569  * Performs an explicit SX1276 hardware reset through the RESET pin and then restores
02570  * the current runtime profile. This gives the hardware menu a true module reboot path.
02571  */
02572 static bool radio_main_reset_module_internal(void)
02573 {
02574     return radio_main_force_recover_radio("manual reset");
02575 }
02576
02577 static bool radio_main_send_system_frame(laviet_frame_type_t type,
02578                                         uint32_t dst_id,
02579                                         const uint8_t *payload,
02580                                         uint16_t payload_len)
02581 {
02582     return radio_main_send_system_frame_ex(type, dst_id, payload, payload_len, true, false);
02583 }
02584
02585 static bool radio_main_send_system_frame_ex(laviet_frame_type_t type,
02586                                             uint32_t dst_id,
02587                                             const uint8_t *payload,
02588                                             uint16_t payload_len,
02589                                             bool request_ack,
02590                                             bool silent)
02591 {
02592     laviet_frame_t frame;
02593     uint8_t raw[LAVIET_FRAME_MAX_LEN];
02594     uint8_t raw_len = 0U;
02595     uint8_t enc_key[16];
02596     uint8_t hmac_key[32];
02597     uint16_t dst16;
02598     uint16_t peer_id;
02599     uint32_t rx_counter;
02600     uint32_t tx_counter;
02601     bool encrypted = false;
02602     bool use_pair_link;
02603     security_frame_key_mode_t key_mode = SECURITY_FRAME_KEY_MODE_SHARED;
02604     bool keys_ok = false;

```

```

02608     if (!s_ctx.initialized)
02609     {
02610         return false;
02611     }
02612     if (!s_ctx.crypto_ready)
02613     {
02614         radio_main_set_last_error("Crypto not ready");
02615         return false;
02616     }
02617     if (((payload == NULL) && (payload_len > 0U)) || (payload_len > LAVIET_MAX_PAYLOAD))
02618     {
02619         radio_main_set_last_error("Payload >16B");
02620         return false;
02621     }
02622     if (dst_id == 0xFFFFFFFFUL)
02623     {
02624         dst16 = LAVIET_BROADCAST_ID;
02625     }
02626     else if (dst_id > 0xFFFFUL)
02627     {
02628         radio_main_set_last_error("Bad dst_id");
02629         return false;
02630     }
02631     else
02632     {
02633         dst16 = (uint16_t)dst_id;
02634     }
02635     if (dst16 == 0U)
02636     {
02637         radio_main_set_last_error("Bad dst_id");
02638         return false;
02639     }
02640     if (((type == LAVIET_TYPE_DATA) || (type == LAVIET_TYPE_RESP)) &&
02641         (dst16 == LAVIET_BROADCAST_ID))
02642     {
02643         radio_main_set_last_error("Broadcast disabled");
02644         return false;
02645     }
02646
02647     if (request_ack &&
02648         s_ctx.ack_pending.active &&
02649         (dst_id != 0xFFFFFFFFUL) &&
02650         ((type == LAVIET_TYPE_DATA) || (type == LAVIET_TYPE_RESP)))
02651     {
02652         radio_main_set_last_error("ACK pending");
02653         return false;
02654     }
02655
02656     memset(&frame, 0, sizeof(frame));
02657     frame.ver_type = laviet_frame_ver_type(type);
02658     frame.flags = 0U;
02659     frame.src_id = s_ctx.node_id;
02660     frame.dst_id = dst16;
02661     frame.msg_id = s_ctx.next_msg_id;
02662     s_ctx.next_msg_id++;
02663     if (s_ctx.next_msg_id == 0U)
02664     {
02665         s_ctx.next_msg_id = 1U;
02666     }
02667
02668     if (!security_main_get_gateway_counter(&rx_counter, &tx_counter))
02669     {
02670         rx_counter = s_ctx.gateway_rx_counter;
02671         tx_counter = s_ctx.gateway_tx_counter;
02672     }
02673     tx_counter++;
02674     if (tx_counter == 0UL)
02675     {
02676         tx_counter = 1UL;
02677     }
02678     frame.counter = tx_counter;
02679     frame.payload_len = (uint8_t)payload_len;
02680     if ((payload_len > 0U) && (payload != NULL))
02681     {
02682         memcpy(frame.payload, payload, payload_len);
02683     }
02684
02685     if (dst16 == LAVIET_BROADCAST_ID)
02686     {
02687         frame.flags |= LAVIET_FLAG_BROADCAST;
02688     }
02689     else if (request_ack &&
02690         ((type == LAVIET_TYPE_DATA) || (type == LAVIET_TYPE_RESP)))
02691     {
02692         frame.flags |= LAVIET_FLAG_ACK_REQUIRED;
02693     }
02694

```

```

02695     switch (type)
02696     {
02697         case LAVIET_TYPE_DATA:
02698         case LAVIET_TYPE_RESP:
02699             encrypted = s_ctx.coding_enabled && (dst16 != LAVIET_BROADCAST_ID);
02700             break;
02701         case LAVIET_TYPE_CFG:
02702             frame.flags |= LAVIET_FLAG_CONFIG_ACCESS;
02703             encrypted = true;
02704             break;
02705         case LAVIET_TYPE_COUNTER_SYNC:
02706             frame.flags |= LAVIET_FLAG_COUNTER_OVERRIDE;
02707             encrypted = true;
02708             break;
02709         case LAVIET_TYPE_KEY_ROTATE:
02710             frame.flags |= LAVIET_FLAG_KEY_UPDATE;
02711             encrypted = true;
02712             break;
02713         case LAVIET_TYPE_ACK:
02714             frame.flags |= LAVIET_FLAG_IS_ACK;
02715             break;
02716         case LAVIET_TYPE_PAIR_REQ:
02717         case LAVIET_TYPE_PAIR_RESP:
02718             frame.flags |= LAVIET_FLAG_PAIRING;
02719             break;
02720         case LAVIET_TYPE_ERROR:
02721         default:
02722             break;
02723     }
02724     if (encrypted)
02725     {
02726         frame.flags |= LAVIET_FLAG_ENCRYPTED;
02727     }
02728
02729     peer_id = (dst16 == LAVIET_BROADCAST_ID) ? LAVIET_BROADCAST_ID : dst16;
02730     use_pair_link = ((type != LAVIET_TYPE_PAIR_REQ) &&
02731                     (type != LAVIET_TYPE_PAIR_RESP) &&
02732                     (type != LAVIET_TYPE_ERROR) &&
02733                     (dst16 != LAVIET_BROADCAST_ID));
02734     if (use_pair_link)
02735     {
02736         key_mode = SECURITY_FRAME_KEY_MODE_PAIR_V1_32;
02737     }
02738     if ((dst16 == LAVIET_GATEWAY_ID) &&
02739         use_pair_link &&
02740         radio_main_get_gateway_cached_frame_keys(key_mode, enc_key, hmac_key))
02741     {
02742         keys_ok = true;
02743     }
02744     else
02745     {
02746         keys_ok = security_main_get_frame_keys_mode(s_ctx.node_id, peer_id, key_mode, enc_key,
02747             hmac_key);
02748     }
02749     if (!keys_ok)
02750     {
02751         laviet_secure_zero(enc_key, sizeof(enc_key));
02752         laviet_secure_zero(hmac_key, sizeof(hmac_key));
02753         radio_main_set_last_error("No frame key");
02754         return false;
02755     }
02756     if (encrypted && !laviet_aes_ctr_crypt(frame.payload, frame.payload_len, enc_key, &frame))
02757     {
02758         laviet_secure_zero(enc_key, sizeof(enc_key));
02759         laviet_secure_zero(hmac_key, sizeof(hmac_key));
02760         radio_main_set_last_error("AES failed");
02761         return false;
02762     }
02763     if (!laviet_frame_hmac_sha256(&frame, hmac_key, frame.mac_tag))
02764     {
02765         laviet_secure_zero(enc_key, sizeof(enc_key));
02766         laviet_secure_zero(hmac_key, sizeof(hmac_key));
02767         radio_main_set_last_error("HMAC failed");
02768         return false;
02769     }
02770     if (!silent && (dst16 == LAVIET_GATEWAY_ID) && use_pair_link)
02771     {
02772         radio_main_log_hmac_debug("RADIO TX HMAC DBG",
02773             &frame,
02774             key_mode,
02775             s_ctx.gateway_pair_code_valid ? s_ctx.gateway_pair_code : NULL,
02776             s_ctx.gateway_pair_code_valid ? s_ctx.gateway_pair_code_len : 0U,
02777             hmac_key,
02778             frame.mac_tag);
02779     }
02780     if (laviet_frame_encode(&frame, raw, sizeof(raw), &raw_len) != LAVIET_STATUS_OK)

```



```

02781         laviet_secure_zero(enc_key, sizeof(enc_key));
02782         laviet_secure_zero(hmac_key, sizeof(hmac_key));
02783         radio_main_set_last_error("Frame encode failed");
02784         return false;
02785     }
02786
02787     if (!radio_main_send_raw_with_retry_ex(raw, raw_len, silent))
02788     {
02789         laviet_secure_zero(enc_key, sizeof(enc_key));
02790         laviet_secure_zero(hmac_key, sizeof(hmac_key));
02791         return false;
02792     }
02793     if (request_ack && !radio_main_ack_track_start(&frame, raw, raw_len))
02794     {
02795         laviet_secure_zero(enc_key, sizeof(enc_key));
02796         laviet_secure_zero(hmac_key, sizeof(hmac_key));
02797         return false;
02798     }
02799
02800     s_ctx.gateway_rx_counter = rx_counter;
02801     s_ctx.gateway_tx_counter = tx_counter;
02802     (void)security_main_commit_gateway_counter(rx_counter, tx_counter);
02803     laviet_secure_zero(enc_key, sizeof(enc_key));
02804     laviet_secure_zero(hmac_key, sizeof(hmac_key));
02805     return true;
02806 }
02807
02808 static bool radio_main_send_ack(const laviet_frame_t *frame)
02809 {
02810     uint8_t payload[LAVIET_ACK_PAYLOAD_LEN];
02811     bool sent;
02812
02813     if (frame == NULL)
02814     {
02815         printf("RADIO ACK skip reason=null-frame\r\n");
02816         return false;
02817     }
02818
02819     if (frame->dst_id != s_ctx.node_id)
02820     {
02821         printf("RADIO ACK skip reason=dst src=0x%04X dst=0x%04X node=0x%04X msg=0x%04X\n",
02822             flags=0x%02X\r\n",
02823             (unsigned int)frame->src_id,
02824             (unsigned int)frame->dst_id,
02825             (unsigned int)s_ctx.node_id,
02826             (unsigned int)frame->msg_id,
02827             (unsigned int)frame->flags);
02828         return false;
02829     }
02830
02831     if (frame->dst_id == LAVIET_BROADCAST_ID)
02832     {
02833         printf("RADIO ACK skip reason=broadcast src=0x%04X msg=0x%04X flags=0x%02X\r\n",
02834             (unsigned int)frame->src_id,
02835             (unsigned int)frame->msg_id,
02836             (unsigned int)frame->flags);
02837         return false;
02838     }
02839
02840     if ((frame->flags & LAVIET_FLAG_ACK_REQUIRED) == 0U)
02841     {
02842         printf("RADIO ACK skip reason=no-ack-required src=0x%04X dst=0x%04X msg=0x%04X\n",
02843             flags=0x%02X\r\n",
02844             (unsigned int)frame->src_id,
02845             (unsigned int)frame->dst_id,
02846             (unsigned int)frame->msg_id,
02847             (unsigned int)frame->flags);
02848         return false;
02849     }
02850
02851     if (laviet_frame_type(frame) == LAVIET_TYPE_ACK)
02852     {
02853         printf("RADIO ACK skip reason=ack-frame src=0x%04X msg=0x%04X flags=0x%02X\r\n",
02854             (unsigned int)frame->src_id,
02855             (unsigned int)frame->msg_id,
02856             (unsigned int)frame->flags);
02857         return false;
02858     }
02859
02860     if (!laviet_frame_write_ack_payload(frame->msg_id, frame->counter, payload))
02861     {
02862         printf("RADIO ACK build failed src=0x%04X msg=0x%04X counter=%lu\r\n",
02863             (unsigned int)frame->src_id,
02864             (unsigned int)frame->msg_id,
02865             (unsigned long)frame->counter);
02866         return false;
02867     }

```

```

02867
02868     printf("RADIO ACK tx delay=%lu ms dst=0x%04X ack_msg=0x%04X ack_counter=%lu\r\n",
02869           (unsigned long)RADIO_ACK_TX_DELAY_MS,
02870           (unsigned int)frame->src_id,
02871           (unsigned int)frame->msg_id,
02872           (unsigned long)frame->counter);
02873     osDelay(RADIO_ACK_TX_DELAY_MS);
02874     sent = radio_main_send_system_frame(LAVIET_TYPE_ACK, frame->src_id, payload, sizeof(payload));
02875     printf("RADIO ACK tx %s dst=0x%04X ack_msg=0x%04X ack_counter=%lu\r\n",
02876           sent ? "ok" : "failed",
02877           (unsigned int)frame->src_id,
02878           (unsigned int)frame->msg_id,
02879           (unsigned long)frame->counter);
02880
02881     return sent;
02882 }
02883
02884 /*
02885  * Starts TX and retries once after a forced radio recovery if the backend looks
02886  * wedged. This protects the user-facing send path from leaving SX1276 stuck in TX.
02887  */
02888 static bool radio_main_send_current_backend_with_retry_ex(const uint8_t *data, uint8_t len, bool
02889                  silent)
02889 {
02890     radio_status_t st;
02891
02892     if ((data == NULL) || (len == 0U))
02893     {
02894         return false;
02895     }
02896
02897     if ((s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK) &&
02898         ((uint16_t)len + 1U) > 64U)
02899     {
02900         printf("RADIO FSK TX ERROR: message too long (%u B raw, max 63 B payload FIFO path)\r\n",
02901               (unsigned int)len);
02902         radio_main_set_last_error("FSK msg too long");
02903         radio_main_notify(MENU_NOTIFICATION_ERROR, "FSK msg too long");
02904         return false;
02905     }
02906
02907     if ((s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK) &&
02908         ((uint16_t)len + 1U) > 64U)
02909     {
02910         printf("RADIO OOK TX ERROR: message too long (%u B raw, max 63 B payload FIFO path)\r\n",
02911               (unsigned int)len);
02912         radio_main_set_last_error("OOK msg too long");
02913         radio_main_notify(MENU_NOTIFICATION_ERROR, "OOK msg too long");
02914         return false;
02915     }
02916
02917     if (!s_ctx.initialized && !radio_main_force_recover_radio("send while uninitialized"))
02918     {
02919         return false;
02920     }
02921
02922     st = radio_send_async(data, len);
02923     if (st == RADIO_OK)
02924     {
02925         if (!silent)
02926         {
02927             radio_main_print_tx_frame(data, len, false);
02928         }
02929         radio_main_tx_mark_started_ex(len, silent);
02930         return true;
02931     }
02932
02933     if ((st == RADIO_EBUS) &&
02934         (radio_get_state() == RADIO_STATE_TX) &&
02935         !radio_main_tx_timed_out())
02936     {
02937         return false;
02938     }
02939
02940     if ((st != RADIO_EBUS) && (st != RADIO_EHW) && (st != RADIO_ESTATE))
02941     {
02942         return false;
02943     }
02944
02945     if (!radio_main_force_recover_radio("tx start failed"))
02946     {
02947         return false;
02948     }
02949
02950     st = radio_send_async(data, len);
02951     if (st == RADIO_OK)
02952     {

```

```

02953         if (!silent)
02954         {
02955             radio_main_print_tx_frame(data, len, true);
02956         }
02957         radio_main_tx_mark_started_ex(len, silent);
02958         return true;
02959     }
02960
02961     return false;
02962 }
02963
02964 static bool radio_main_send_raw_with_retry(const uint8_t *data, uint8_t len)
02965 {
02966     return radio_main_send_raw_with_retry_ex(data, len, false);
02967 }
02968
02969 static bool radio_main_send_raw_with_retry_ex(const uint8_t *data, uint8_t len, bool silent)
02970 {
02971     if ((data == NULL) || (len == 0U))
02972     {
02973         return false;
02974     }
02975     if (s_ctx.tx_in_progress)
02976     {
02977         return false;
02978     }
02979     if (!radio_main_switch_backend(s_ctx.modulation_id))
02980     {
02981         return false;
02982     }
02983     return radio_main_send_current_backend_with_retry_ex(data, len, silent);
02984 }
02985
02986 static bool radio_main_send_template_internal(uint8_t group_id, uint8_t msg_id, uint32_t dst_id)
02987 {
02988     const char *msg;
02989     uint16_t len;
02990
02991     if ((group_id >= 3U) || (msg_id >= 3U))
02992     {
02993         return false;
02994     }
02995
02996     msg = s_template_groups[group_id][msg_id];
02997     len = (uint16_t)strlen(msg);
02998     if ((len == 0U) || (len > LAVIET_MAX_PAYLOAD))
02999     {
03000         radio_main_set_last_error("Payload >16B");
03001         return false;
03002     }
03003
03004     return radio_main_send_system_frame(LAVIET_TYPE_DATA, dst_id, (const uint8_t *)msg, len);
03005 }
03006
03007 static void radio_main_post_rx_notification(int16_t rssi_dbm,
03008     uint16_t src_id,
03009     uint16_t dst_id,
03010     const uint8_t *payload,
03011     uint8_t payload_len,
03012     laviet_frame_type_t frame_type)
03013 {
03014     char text_buf[21];
03015     uint8_t text_len;
03016     menu_notification_t n;
03017
03018     memset(&n, 0, sizeof(n));
03019     n.type = MENU_NOTIFICATION_RX;
03020     n.rssi_dbm = rssi_dbm;
03021     n.device_code = (dst_id == LAVIET_BROADCAST_ID) ? LAVIET_BROADCAST_ID : src_id;
03022     n.reply_device_code = src_id;
03023
03024     text_len = radio_main_format_payload_text(payload, payload_len, text_buf, sizeof(text_buf));
03025     if (text_len > 0U)
03026     {
03027         memcpy(n.text, text_buf, text_len);
03028         n.text[text_len] = '\0';
03029     }
03030     else
03031     {
03032         (void)sprintf(n.text, sizeof(n.text), "TYPE:%u", (unsigned int)frame_type);
03033     }
03034
03035     printf("RADIO RX NOTIFY type=%s src=0x%04X dst=0x%04X rssi=%d text=\"%s\"\\r\\n",
03036         radio_main_frame_type_text(frame_type),
03037         (unsigned int)src_id,
03038         (unsigned int)dst_id,

```

```

03044         (int) rssi_dbm,
03045         n.text);
03046     (void) menu_main_post_notification(&n);
03047 }
03048
03050 static const char *radio_main_frame_type_text(laviet_frame_type_t type)
03051 {
03052     switch (type)
03053     {
03054         case LAVIET_TYPE_DATA:
03055             return "DATA";
03056         case LAVIET_TYPE_ACK:
03057             return "ACK";
03058         case LAVIET_TYPE_RESP:
03059             return "RESP";
03060         case LAVIET_TYPE_PAIR_REQ:
03061             return "PAIR_REQ";
03062         case LAVIET_TYPE_PAIR_RESP:
03063             return "PAIR_RESP";
03064         case LAVIET_TYPE_CFG:
03065             return "CFG";
03066         case LAVIET_TYPE_COUNTER_SYNC:
03067             return "COUNTER_SYNC";
03068         case LAVIET_TYPE_KEY_ROTATE:
03069             return "KEY_ROTATE";
03070         case LAVIET_TYPE_ERROR:
03071             return "ERROR";
03072         default:
03073             return "UNKNOWN";
03074     }
03075 }
03076
03078 static const char *radio_main_key_mode_text(security_frame_key_mode_t mode)
03079 {
03080     switch (mode)
03081     {
03082         case SECURITY_FRAME_KEY_MODE_PAIR_V1_32:
03083             return "PAIR32";
03084         case SECURITY_FRAME_KEY_MODE_SHARED:
03085             return "SHARED";
03086     }
03087 }
03088
03089 static bool radio_main_laviet_verify_rx(const laviet_frame_t *frame, uint8_t enc_key[16])
03090 {
03091     security_frame_key_mode_t candidates[2];
03092     security_frame_key_mode_t matched_mode = SECURITY_FRAME_KEY_MODE_SHARED;
03093     uint8_t trial_enc[16];
03094     uint8_t hmac_key[32];
03095     uint8_t expected[LAVIET_MAC_TAG_LEN];
03096     laviet_frame_type_t frame_type;
03097     uint16_t peer_id;
03098     bool use_pair_link;
03099     bool ok = false;
03100     uint8_t candidate_count = 0;
03101     uint8_t idx;
03102
03103     if ((frame == NULL) || (enc_key == NULL) || !s_ctx.crypto_ready)
03104     {
03105         return false;
03106     }
03107
03108     memset(enc_key, 0, 16);
03109     memset(trial_enc, 0, sizeof(trial_enc));
03110     memset(hmac_key, 0, sizeof(hmac_key));
03111     memset(expected, 0, sizeof(expected));
03112     frame_type = laviet_frame_type(frame);
03113     peer_id = laviet_frame_is_broadcast(frame) ? LAVIET_BROADCAST_ID : frame->src_id;
03114     use_pair_link = ((frame_type != LAVIET_TYPE_PAIR_REQ) &&
03115                     (frame_type != LAVIET_TYPE_PAIR_RESP) &&
03116                     (frame_type != LAVIET_TYPE_ERROR) &&
03117                     (peer_id != LAVIET_BROADCAST_ID));
03118
03119     if (radio_main_try_broadcast_group_rx(frame, enc_key))
03120     {
03121         laviet_secure_zero(trial_enc, sizeof(trial_enc));
03122         laviet_secure_zero(hmac_key, sizeof(hmac_key));
03123         laviet_secure_zero(expected, sizeof(expected));
03124         return true;
03125     }
03126
03127     if (!use_pair_link)
03128     {
03129         candidates[candidate_count++] = SECURITY_FRAME_KEY_MODE_SHARED;
03130     }
03131     else

```

```

03134     {
03135         candidates[candidate_count++] = SECURITY_FRAME_KEY_MODE_PAIR_V1_32;
03136     }
03137
03138     for (idx = 0U; idx < candidate_count; idx++)
03139     {
03140         bool candidate_ok = false;
03141         bool mac_match = false;
03142
03143         memset(trial_enc, 0, sizeof(trial_enc));
03144         memset(hmac_key, 0, sizeof(hmac_key));
03145         memset(expected, 0, sizeof(expected));
03146
03147         if ((peer_id == LAVIET_GATEWAY_ID) &&
03148             use_pair_link &&
03149             radio_main_get_gateway_cached_frame_keys(candidates[idx], trial_enc, hmac_key))
03150         {
03151             candidate_ok = true;
03152         }
03153         else if (security_main_get_frame_keys_mode(s_ctx.node_id, peer_id, candidates[idx], trial_enc,
03154             hmac_key))
03155         {
03156             candidate_ok = true;
03157         }
03158         else if (frame->src_id == LAVIET_GATEWAY_ID)
03159         {
03160             trusted_info_t gateway_info;
03161
03162             memset(&gateway_info, 0, sizeof(gateway_info));
03163             if (security_main_cmd_get_device(0U, &gateway_info) && gateway_info.in_use)
03164             {
03165                 printf("RADIO RX HMAC no-key mode=%s src=0x%04X dst=0x%04X peer=0x%04X slot0=0x%041X
03166 code_len=%u\r\n",
03167                     radio_main_key_mode_text(candidates[idx]),
03168                     (unsigned int)frame->src_id,
03169                     (unsigned int)frame->dst_id,
03170                     (unsigned int)peer_id,
03171                     (unsigned long)gateway_info.node_id,
03172                     (unsigned int)gateway_info.code_len);
03173             }
03174             else
03175             {
03176                 printf("RADIO RX HMAC no-key mode=%s src=0x%04X dst=0x%04X peer=0x%04X
03177 slot0=<empty>\r\n",
03178                     radio_main_key_mode_text(candidates[idx]),
03179                     (unsigned int)frame->src_id,
03180                     (unsigned int)frame->dst_id,
03181                     (unsigned int)peer_id);
03182             }
03183         }
03184
03185         if (candidate_ok &&
03186             laviet_frame_hmac_sha256(frame, hmac_key, expected))
03187         {
03188             if (frame->src_id == LAVIET_GATEWAY_ID)
03189             {
03190                 radio_main_log_hmac_debug("RADIO RX HMAC DBG",
03191                     frame,
03192                     candidates[idx],
03193                     ((peer_id == LAVIET_GATEWAY_ID) &&
03194                     s_ctx.gateway_pair_code_valid) ?
03195                     s_ctx.gateway_pair_code : NULL,
03196                     ((peer_id == LAVIET_GATEWAY_ID) &&
03197                     s_ctx.gateway_pair_code_valid) ?
03198                     s_ctx.gateway_pair_code_len : 0U,
03199                     hmac_key,
03200                     expected);
03201             }
03202             mac_match = laviet_mac_equal(expected, frame->mac_tag);
03203         }
03204
03205         if (candidate_ok && mac_match)
03206         {
03207             memcpy(enc_key, trial_enc, sizeof(trial_enc));
03208             matched_mode = candidates[idx];
03209             ok = true;
03210             break;
03211         }
03212     }
03213
03214     if (ok && use_pair_link && (peer_id == LAVIET_GATEWAY_ID))
03215     {
03216         if (!s_ctx.gateway_key_mode_known || (s_ctx.gateway_key_mode != matched_mode))
03217         {
03218             printf("RADIO RX gateway key mode=%s\r\n", radio_main_key_mode_text(matched_mode));
03219         }
03220         s_ctx.gateway_key_mode_known = true;
03221     }

```

```

03216     s_ctx.gateway_key_mode = matched_mode;
03217 }
03218
03219 laviet_secure_zero(trial_enc, sizeof(trial_enc));
03220 laviet_secure_zero(hmac_key, sizeof(hmac_key));
03221 laviet_secure_zero(expected, sizeof(expected));
03222 if (!ok)
03223 {
03224     laviet_secure_zero(enc_key, 16U);
03225 }
03226 return ok;
03227 }
03228
03229 static void radio_main_handle_events(void)
03230 {
03231     uint32_t events = radio_take_events();
03232     radio_packet_t pkt;
03233
03234     if ((events & RADIO_EVENT_RX_DONE) != 0U)
03235     {
03236         if (radio_get_last_packet(&pkt))
03237         {
03238             radio_main_handle_rx_packet(&pkt);
03239         }
03240     }
03241
03242     if ((events & RADIO_EVENT_TX_DONE) != 0U)
03243     {
03244         if (!s_ctx.tx_silent)
03245         {
03246             printf("RADIO EVT: TX_DONE\r\n");
03247         }
03248         radio_main_tx_clear();
03249         radio_main_ensure_rx_continuous();
03250     }
03251
03252     if ((events & RADIO_EVENT_CRC_ERR) != 0U)
03253     {
03254         printf("RADIO EVT: CRC_ERR\r\n");
03255     }
03256
03257     if ((events & RADIO_EVENT_RX_TIMEOUT) != 0U)
03258     {
03259         printf("RADIO EVT: RX_TIMEOUT\r\n");
03260     }
03261
03262     if ((events & RADIO_EVENT_HW_ERROR) != 0U)
03263     {
03264         printf("RADIO EVT: HW_ERROR\r\n");
03265         if (!radio_main_force_recover_radio("backend HW error"))
03266         {
03267             radio_main_notify(MENU_NOTIFICATION_ERROR, "Radio recovery failed");
03268         }
03269     }
03270 }
03271
03272 static bool radio_main_source_is_trusted(uint16_t src_id)
03273 {
03274     trusted_info_t info;
03275     uint8_t idx;
03276
03277     if ((src_id == 0U) || (src_id == LAVIET_BROADCAST_ID))
03278     {
03279         return false;
03280     }
03281
03282     for (idx = 0U; idx < RADIO_TRUSTED_DEVICE_SLOTS; idx++)
03283     {
03284         memset(&info, 0, sizeof(info));
03285         if (security_main_cmd_get_device(idx, &info) &&
03286             info.in_use &&
03287             (info.node_id == src_id))
03288         {
03289             return true;
03290         }
03291     }
03292
03293     return false;
03294 }
03295
03296 static bool radio_main_source_is_allowed_for_data(const laviet_frame_t *frame)
03297 {
03298     if (frame == NULL)
03299     {
03300         return false;
03301     }
03302
03303     if ((frame->src_id == LAVIET_GATEWAY_ID) &&
03304         (frame->dst_id == LAVIET_BROADCAST_ID))
03305     {

```

```

03306         return true;
03307     }
03308
03309     return radio_main_source_is_trusted(frame->src_id);
03310 }
03311
03313 static void radio_main_handle_rx_packet(const radio_packet_t *pkt)
03314 {
03315     laviet_frame_t frame;
03316     laviet_frame_t frame_decoded;
03317     laviet_frame_status_t frame_status;
03318     bool for_me;
03319     uint8_t enc_key[16];
03320     uint32_t rx_counter;
03321     uint32_t tx_counter;
03322     laviet_frame_type_t frame_type;
03323
03324     if ((pkt == NULL) || (pkt->length == 0U))
03325     {
03326         return;
03327     }
03328
03329     printf("RADIO RX len=%u RSSI=%d SNR=%d\r\n", pkt->length, pkt->rss_i_dbm, pkt->snr_db);
03330     radio_main_print_rx_ascii(pkt->data, pkt->length);
03331     radio_main_print_hex_bytes("RADIO RX HEX: ", pkt->data, pkt->length);
03332     (void)security_main_log_message(pkt->rss_i_dbm, pkt->data, pkt->length);
03333
03334     frame_status = laviet_frame_decode(pkt->data, pkt->length, &frame);
03335     if (frame_status != LAVIET_STATUS_OK)
03336     {
03337         printf("RADIO RX non-LAVIET drop status=%d\r\n", (int)frame_status);
03338         return;
03339     }
03340
03341     if (!radio_main_laviet_verify_rx(&frame, enc_key))
03342     {
03343         if (frame.src_id == LAVIET_GATEWAY_ID)
03344         {
03345             radio_main_log_gateway_rx_frame(&frame, &frame, pkt->rss_i_dbm, pkt->snr_db);
03346             printf("RADIO RX GATEWAY NOTE HMAC failed; PAYLOAD DEC fields below are still raw
03347 ciphertext\r\n");
03348             printf("RADIO RX HMAC drop src=0x%04X dst=0x%04X msg=0x%04X\r\n",
03349                 (unsigned int)frame.src_id,
03350                 (unsigned int)frame.dst_id,
03351                 (unsigned int)frame.msg_id);
03352             return;
03353         }
03354
03355         if (!security_main_get_gateway_counter(&rx_counter, &tx_counter))
03356         {
03357             rx_counter = s_ctx.gateway_rx_counter;
03358             tx_counter = s_ctx.gateway_tx_counter;
03359         }
03360
03361         frame_type = laviet_frame_type(&frame);
03362         if ((frame.src_id == LAVIET_GATEWAY_ID) &&
03363             (frame_type != LAVIET_TYPE_COUNTER_SYNC))
03364         {
03365             if ((frame.counter == 0UL) && (rx_counter == 0UL))
03366             {
03367                 printf("RADIO RX compat allow src=0x%04X counter=0 while last=0\r\n",
03368                     (unsigned int)frame.src_id);
03369             }
03370             else if (frame.counter <= rx_counter)
03371             {
03372                 printf("RADIO RX REPLAY bypass src=0x%04X counter=%lu last=%lu\r\n",
03373                     (unsigned int)frame.src_id,
03374                     (unsigned long)frame.counter,
03375                     (unsigned long)rx_counter);
03376             }
03377         }
03378
03379         frame_decoded = frame;
03380         if ((frame_decoded.flags & LAVIET_FLAG_ENCRYPTED) != 0U)
03381         {
03382             if (!laviet_aes_ctr_crypt(frame_decoded.payload,
03383                                     frame_decoded.payload_len,
03384                                     enc_key,
03385                                     &frame_decoded))
03386             {
03387                 printf("RADIO RX AES drop src=0x%04X\r\n", (unsigned int)frame_decoded.src_id);
03388                 laviet_secure_zero(enc_key, 16U);
03389                 return;
03390             }
03391         }
03392         laviet_secure_zero(enc_key, 16U);

```

```

03393
03394     for_me = (frame_decoded.dst_id == s_ctx.node_id) ||
03395             (frame_decoded.dst_id == LAVIET_BROADCAST_ID);
03396     if (!for_me)
03397     {
03398         printf("RADIO RX not-for-me src=0x%04X dst=0x%04X\r\n",
03399             (unsigned int)frame_decoded.src_id,
03400             (unsigned int)frame_decoded.dst_id);
03401         return;
03402     }
03403     if (frame_decoded.src_id == LAVIET_GATEWAY_ID)
03404     {
03405         radio_main_log_gateway_rx_frame(&frame, &frame_decoded, pkt->rssi_dbm, pkt->snr_db);
03406     }
03407     if (((frame_type == LAVIET_TYPE_DATA) || (frame_type == LAVIET_TYPE_RESP)) &&
03408         !radio_main_source_is_allowed_for_data(&frame_decoded))
03409     {
03410         printf("RADIO RX outside-network drop src=0x%04X dst=0x%04X type=%s\r\n",
03411             (unsigned int)frame_decoded.src_id,
03412             (unsigned int)frame_decoded.dst_id,
03413             radio_main_frame_type_text(frame_type));
03414         return;
03415     }
03416
03417     printf("RADIO RX OK type=%s src=0x%04X dst=0x%04X msg=0x%04X counter=%lu flags=0x%02X len=%u
03418 rssi=%d snr=%d\r\n",
03419         radio_main_frame_type_text(frame_type),
03420         (unsigned int)frame_decoded.src_id,
03421         (unsigned int)frame_decoded.dst_id,
03422         (unsigned int)frame_decoded.msg_id,
03423         (unsigned long)frame_decoded.counter,
03424         (unsigned int)frame_decoded.flags,
03425         (unsigned int)frame_decoded.payload_len,
03426         (int)pkt->rssi_dbm,
03427         (int)pkt->snr_db);
03428
03429     if (frame_decoded.payload_len > 0U)
03430     {
03431         char text_buf[21];
03432
03433         radio_main_print_hex_bytes("RADIO RX DEC HEX: ", frame_decoded.payload,
03434             frame_decoded.payload_len);
03435         if (radio_main_format_payload_text(frame_decoded.payload,
03436             frame_decoded.payload_len,
03437             text_buf,
03438             sizeof(text_buf)) > 0U)
03439         {
03440             printf("RADIO RX DEC TXT: \"%s\"\r\n", text_buf);
03441             if (((frame_type == LAVIET_TYPE_DATA) || (frame_type == LAVIET_TYPE_RESP)) &&
03442                 ((frame_decoded.flags & LAVIET_FLAG_ENCRYPTED) != 0U) &&
03443                 (strncmp(text_buf, "HEX:", 4U) == 0))
03444             {
03445                 printf("RADIO RX WARN non-printable payload after decrypt; check gateway nonce fields
03446 src/dst/msg/counter\r\n");
03447             }
03448         }
03449     }
03450     if (for_me)
03451     {
03452         if (frame_type == LAVIET_TYPE_DATA)
03453         {
03454             if (frame_decoded.dst_id == s_ctx.node_id)
03455             {
03456                 bool ack_ok = radio_main_send_ack(&frame_decoded);
03457
03458                 if (((frame_decoded.flags & LAVIET_FLAG_ACK_REQUIRED) != 0U) && !ack_ok)
03459                 {
03460                     printf("RADIO RX DATA ACK failed src=0x%04X msg=0x%04X counter=%lu
03461 flags=0x%02X\r\n",
03462                         (unsigned int)frame_decoded.src_id,
03463                         (unsigned int)frame_decoded.msg_id,
03464                         (unsigned long)frame_decoded.counter,
03465                         (unsigned int)frame_decoded.flags);
03466                 }
03467                 (void)security_main_get_gateway_counter(&rx_counter, &tx_counter);
03468             }
03469             if (frame_decoded.payload_len > 0U)
03470             {
03471                 bool stored = radio_main_push_payload_to_monitor(pkt->rssi_dbm,
03472                     frame_decoded.src_id,
03473                     frame_decoded.payload,
03474                     frame_decoded.payload_len);
03475
03476                 printf("RADIO RX MONITOR type=DATA stored=%u src=0x%04X len=%u\r\n",
03477                     stored ? 1U : 0U,
03478                     (unsigned int)frame_decoded.src_id,

```



```

03476         (unsigned int)frame_decoded.payload_len);
03477     }
03478     radio_main_post_rx_notification(pkt->rssi_dbm,
03479         frame_decoded.src_id,
03480         frame_decoded.dst_id,
03481         frame_decoded.payload,
03482         frame_decoded.payload_len,
03483         frame_type);
03484 }
03485 else if (frame_type == LAVIET_TYPE_RESP)
03486 {
03487     if ((frame_decoded.dst_id == s_ctx.node_id) &&
03488         ((frame_decoded.flags & LAVIET_FLAG_ACK_REQUIRED) != 0U))
03489     {
03490         bool ack_ok = radio_main_send_ack(&frame_decoded);
03491         if (!ack_ok)
03492         {
03493             printf("RADIO RX RESP ACK failed src=0x%04X msg=0x%04X counter=%lu
03494 flags=0x%02X\r\n",
03495                 (unsigned int)frame_decoded.src_id,
03496                 (unsigned int)frame_decoded.msg_id,
03497                 (unsigned long)frame_decoded.counter,
03498                 (unsigned int)frame_decoded.flags);
03499         }
03500     }
03501     if (frame_decoded.payload_len > 0U)
03502     {
03503         bool stored = radio_main_push_payload_to_monitor(pkt->rssi_dbm,
03504             frame_decoded.src_id,
03505             frame_decoded.payload,
03506             frame_decoded.payload_len);
03507
03508         printf("RADIO RX MONITOR type=RESP stored=%u src=0x%04X len=%u\r\n",
03509             stored ? 1U : 0U,
03510             (unsigned int)frame_decoded.src_id,
03511             (unsigned int)frame_decoded.payload_len);
03512     }
03513     radio_main_post_rx_notification(pkt->rssi_dbm,
03514         frame_decoded.src_id,
03515         frame_decoded.dst_id,
03516         frame_decoded.payload,
03517         frame_decoded.payload_len,
03518         frame_type);
03519 }
03520 else if (frame_type == LAVIET_TYPE_COUNTER_SYNC)
03521 {
03522     uint32_t new_counter;
03523
03524     if ((frame_decoded.src_id != LAVIET_GATEWAY_ID) ||
03525         (frame_decoded.payload_len != LAVIET_COUNTER_SYNC_PAYLOAD_LEN))
03526     {
03527         printf("RADIO RX bad COUNTER_SYNC\r\n");
03528         return;
03529     }
03530     new_counter = ((uint32_t)frame_decoded.payload[0] << 24) |
03531         ((uint32_t)frame_decoded.payload[1] << 16) |
03532         ((uint32_t)frame_decoded.payload[2] << 8) |
03533         (uint32_t)frame_decoded.payload[3];
03534     s_ctx.gateway_rx_counter = new_counter;
03535     s_ctx.gateway_tx_counter = tx_counter;
03536     (void)security_main_commit_gateway_counter(s_ctx.gateway_rx_counter,
03537         s_ctx.gateway_tx_counter);
03538     (void)radio_main_send_ack(&frame_decoded);
03539     radio_main_notify(MENU_NOTIFICATION_WARNING, "Counter sync");
03540 }
03541 else if (frame_type == LAVIET_TYPE_PAIR_REQ)
03542 {
03543     if (s_ctx.pairing_active && !s_ctx.pairing_pending)
03544     {
03545         char pair_note[21];
03546         char code_text[12];
03547         const uint8_t *pair_code = NULL;
03548         uint8_t pair_code_len = 0U;
03549         trusted_info_t gateway_info;
03550
03551         if (!radio_main_pair_payload_parse(frame_decoded.payload,
03552             (uint8_t)frame_decoded.payload_len,
03553             &pair_code,
03554             &pair_code_len))
03555         {
03556             return;
03557         }
03558
03559         if (s_ctx.pairing_network_mode && (frame_decoded.src_id != LAVIET_GATEWAY_ID))
03560         {
03561             printf("RADIO RX non-gateway PAIR_REQ drop src=0x%04X\r\n",

```

```

03562         (unsigned int)frame_decoded.src_id);
03563         return;
03564     }
03565
03566     memset(&gateway_info, 0, sizeof(gateway_info));
03567     if (s_ctx.pairing_network_mode &&
03568         security_main_cmd_get_device(0U, &gateway_info) &&
03569         gateway_info.in_use)
03570     {
03571         printf("RADIO RX network PAIR_REQ drop, gateway slot already in use\r\n");
03572         return;
03573     }
03574
03575     if (s_ctx.pairing_network_mode &&
03576         (pkt->rssi_dbm < RADIO_NETWORK_PAIR_RSSI_MIN_DBM))
03577     {
03578         printf("RADIO RX network PAIR_REQ drop, gateway RSSI too low rssi=%d min=%d\r\n",
03579             (int)pkt->rssi_dbm,
03580             (int)RADIO_NETWORK_PAIR_RSSI_MIN_DBM);
03581         radio_main_notify(MENU_NOTIFICATION_WARNING, "Gateway RSSI low");
03582         return;
03583     }
03584
03585     s_ctx.pairing_pending = true;
03586     s_ctx.pairing_pending_network = s_ctx.pairing_network_mode;
03587     s_ctx.pairing_pending_node = frame_decoded.src_id;
03588     s_ctx.pairing_pending_code_len = (pair_code_len > sizeof(s_ctx.pairing_pending_code))
?         sizeof(s_ctx.pairing_pending_code) : pair_code_len;
03589     memcpy(s_ctx.pairing_pending_code, pair_code, s_ctx.pairing_pending_code_len);
03590
03591     radio_main_pair_code_to_text(s_ctx.pairing_pending_code,
03592                                 s_ctx.pairing_pending_code_len,
03593                                 code_text,
03594                                 (uint8_t)sizeof(code_text));
03595     snprintf(pair_note, sizeof(pair_note), "PAIR_REQ %s", code_text);
03596     radio_main_notify(MENU_NOTIFICATION_PAIRING, pair_note);
03597 }
03598
03599 }
03600 else if (frame_type == LAVIET_TYPE_PAIR_RESP)
03601 {
03602     if (s_ctx.pairing_active && s_ctx.pairing_outgoing_pending)
03603     {
03604         bool code_match = false;
03605         char code_text[12];
03606         const uint8_t *pair_code = NULL;
03607         uint8_t pair_code_len = 0U;
03608
03609         if (!radio_main_pair_payload_parse(frame_decoded.payload,
03610             (uint8_t)frame_decoded.payload_len,
03611             &pair_code,
03612             &pair_code_len))
03613         {
03614             return;
03615         }
03616
03617         if (s_ctx.pairing_outgoing_network &&
03618             (pkt->rssi_dbm < RADIO_NETWORK_PAIR_RSSI_MIN_DBM))
03619         {
03620             printf("RADIO RX network PAIR_RESP drop, gateway RSSI too low rssi=%d min=%d\r\n",
03621                 (int)pkt->rssi_dbm,
03622                 (int)RADIO_NETWORK_PAIR_RSSI_MIN_DBM);
03623             radio_main_notify(MENU_NOTIFICATION_WARNING, "Gateway RSSI low");
03624             s_ctx.pairing_outgoing_pending = false;
03625             s_ctx.pairing_outgoing_network = false;
03626             return;
03627         }
03628
03629         if ((pair_code_len == s_ctx.pairing_outgoing_code_len) &&
03630             (pair_code_len > 0U) &&
03631             (memcmp(pair_code, s_ctx.pairing_outgoing_code, pair_code_len) == 0))
03632         {
03633             code_match = true;
03634         }
03635
03636         radio_main_pair_code_to_text(s_ctx.pairing_outgoing_code,
03637                                     s_ctx.pairing_outgoing_code_len,
03638                                     code_text,
03639                                     (uint8_t)sizeof(code_text));
03640
03641         if (code_match)
03642         {
03643             bool saved;
03644
03645             if (s_ctx.pairing_outgoing_network)
03646             {
03647                 saved = security_main_cmd_add_gateway(frame_decoded.src_id,

```

```

03648                                     s_ctx.pairing_outgoing_code,
03649                                     s_ctx.pairing_outgoing_code_len);
03650     }
03651     else
03652     {
03653         saved = security_main_cmd_add_device(frame_decoded.src_id,
03654                                             s_ctx.pairing_outgoing_code,
03655                                             s_ctx.pairing_outgoing_code_len);
03656     }
03657     if (saved)
03658     {
03659         char pair_note[21];
03660         if (s_ctx.pairing_outgoing_network)
03661         {
03662             radio_main_set_gateway_pair_code(s_ctx.pairing_outgoing_code,
03663                                             s_ctx.pairing_outgoing_code_len);
03664         }
03665         snprintf(pair_note, sizeof(pair_note), "PAIR_OK %s", code_text);
03666         radio_main_notify(MENU_NOTIFICATION_PAIRING, pair_note);
03667         s_ctx.pairing_active = false;
03668     }
03669     else
03670     {
03671         radio_main_notify(MENU_NOTIFICATION_ERROR, "Pairing save failed");
03672     }
03673 }
03674 }
03675 else
03676 {
03677     radio_main_notify(MENU_NOTIFICATION_ERROR, "PAIR code mismatch");
03678 }
03679
03680 s_ctx.pairing_outgoing_pending = false;
03681 s_ctx.pairing_outgoing_network = false;
03682 }
03683 }
03684 else if (frame_type == LAVIET_TYPE_KEY_ROTATE)
03685 {
03686     bool key_ok = radio_main_handle_key_rotate_frame(&frame_decoded);
03687
03688     if (!key_ok)
03689     {
03690         printf("RADIO RX KEY_ROTATE ignored\r\n");
03691     }
03692     if ((frame_decoded.dst_id == s_ctx.node_id) &&
03693         ((frame_decoded.flags & LAVIET_FLAG_ACK_REQUIRED) != 0U))
03694     {
03695         (void)radio_main_send_ack(&frame_decoded);
03696     }
03697 }
03698 else if (frame_type == LAVIET_TYPE_ERROR)
03699 {
03700     if (s_ctx.pairing_active && s_ctx.pairing_outgoing_pending)
03701     {
03702         radio_main_notify(MENU_NOTIFICATION_PAIRING, "PAIR_ERROR");
03703         s_ctx.pairing_outgoing_pending = false;
03704     }
03705     else
03706     {
03707         radio_main_notify(MENU_NOTIFICATION_WARNING, "LAVIET error");
03708     }
03709     if (frame_decoded.dst_id == s_ctx.node_id)
03710     {
03711         (void)radio_main_send_ack(&frame_decoded);
03712     }
03713 }
03714 else if (frame_type == LAVIET_TYPE_ACK)
03715 {
03716     uint16_t acked_msg_id = 0U;
03717     uint32_t acked_counter = 0UL;
03718
03719     if (laviet_frame_read_ack_payload(frame_decoded.payload,
03720                                     &acked_msg_id,
03721                                     &acked_counter))
03722     {
03723         printf("RADIO RX ACK src=0x%04X ack_msg=0x%04X ack_counter=%lu\r\n",
03724               (unsigned int)frame_decoded.src_id,
03725               (unsigned int)acked_msg_id,
03726               (unsigned long)acked_counter);
03727         radio_main_handle_ack_frame(frame_decoded.src_id, acked_msg_id, acked_counter);
03728     }
03729     else
03730     {
03731         printf("RADIO RX ACK bad payload src=0x%04X\r\n",
03732               (unsigned int)frame_decoded.src_id);
03733     }
03734 }

```

```

03735     else
03736     {
03737         radio_main_post_rx_notification(pkt->rss_i_dbm,
03738                                         frame_decoded.src_id,
03739                                         frame_decoded.dst_id,
03740                                         frame_decoded.payload,
03741                                         frame_decoded.payload_len,
03742                                         frame_type);
03743         if ((frame_decoded.dst_id == s_ctx.node_id) &&
03744             ((frame_decoded.flags & LAVIET_FLAG_ACK_REQUIRED) != 0U) &&
03745             (frame_type != LAVIET_TYPE_ACK))
03746         {
03747             (void)radio_main_send_ack(&frame_decoded);
03748         }
03749     }
03750 }
03751
03752 if ((frame_decoded.src_id == LAVIET_GATEWAY_ID) &&
03753     (frame_type != LAVIET_TYPE_COUNTER_SYNC) &&
03754     (frame_decoded.counter > s_ctx.gateway_rx_counter))
03755 {
03756     uint32_t latest_rx;
03757     uint32_t latest_tx;
03758
03759     latest_rx = s_ctx.gateway_rx_counter;
03760     latest_tx = s_ctx.gateway_tx_counter;
03761     if (security_main_get_gateway_counter(&latest_rx, &latest_tx))
03762     {
03763         s_ctx.gateway_tx_counter = latest_tx;
03764     }
03765     s_ctx.gateway_rx_counter = frame_decoded.counter;
03766     (void)security_main_commit_gateway_counter(s_ctx.gateway_rx_counter,
03767                                                s_ctx.gateway_tx_counter);
03768 }
03769 }
03770
03772 static void radio_main_handle_hopping(void)
03773 {
03774     uint32_t now;
03775
03776     if (!s_ctx.fh_enabled)
03777     {
03778         return;
03779     }
03780     if (!s_ctx.initialized)
03781     {
03782         return;
03783     }
03784     if (s_ctx.ack_pending.active)
03785     {
03786         return;
03787     }
03788
03789     now = radio_main_now_ms();
03790     if ((now - s_ctx.last_hop_ms) < s_ctx.fh_period_ms)
03791     {
03792         return;
03793     }
03794     if (radio_get_state() == RADIO_STATE_TX)
03795     {
03796         return;
03797     }
03798
03799     s_ctx.last_hop_ms = now;
03800     s_ctx.hop_idx = (uint8_t)((s_ctx.hop_idx + 1U) % 3U);
03801     s_ctx.lora_cfg.frequency_hz = s_hop_channels_hz[s_ctx.hop_idx];
03802     if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_LORA)
03803     {
03804         (void)radio_main_reconfigure_radio();
03805     }
03806 }
03807
03809 static void radio_main_handle_auto_ping(void)
03810 {
03811     uint32_t now;
03812     uint32_t period_ms;
03813
03814     if (!s_ctx.auto_ping_enabled)
03815     {
03816         return;
03817     }
03818     if (!s_ctx.initialized)
03819     {
03820         return;
03821     }
03822
03823     now = radio_main_now_ms();

```

```

03824     period_ms = radio_main_auto_ping_period_ms();
03825     if ((now - s_ctx.last_ping_ms) < period_ms)
03826     {
03827         return;
03828     }
03829     s_ctx.last_ping_ms = now;
03830
03831     if (radio_get_state() != RADIO_STATE_TX)
03832     {
03833         if (s_ctx.auto_ping_mode == RADIO_MAIN_AUTO_PING_RAW)
03834         {
03835             (void)radio_main_send_raw_with_retry_ex(s_auto_ping_raw, RADIO_AUTO_PING_RAW_LEN, true);
03836         }
03837         else
03838         {
03839             const char *msg = s_template_groups[2][0];
03840             (void)radio_main_send_system_frame_ex(LAVIET_TYPE_DATA,
03841                                                  LAVIET_GATEWAY_ID,
03842                                                  (const uint8_t *)msg,
03843                                                  (uint16_t)strlen(msg),
03844                                                  false,
03845                                                  true);
03846         }
03847     }
03848 }
03849
03850 /*
03851  * Secondary application-layer watchdog. Backend watchdogs should catch TX stalls
03852  * first, but this guard keeps the app recoverable if the backend state machine
03853  * or IRQ path still leaves the radio reported as TX.
03854  */
03855 static void radio_main_watchdog_tx(void)
03856 {
03857     if (!s_ctx.initialized)
03858     {
03859         return;
03860     }
03861     if (!radio_main_tx_timed_out())
03862     {
03863         return;
03864     }
03865
03866     if (!radio_main_force_recover_radio("application TX watchdog"))
03867     {
03868         radio_main_notify(MENU_NOTIFICATION_ERROR, "TX watchdog failed");
03869     }
03870 }
03871
03872 static void radio_main_ensure_rx_continuous(void)
03873 {
03874     if (!s_ctx.initialized)
03875     {
03876         return;
03877     }
03878
03879     if (radio_get_state() == RADIO_STATE_TX)
03880     {
03881         if (radio_main_tx_timed_out() &&
03882             !radio_main_force_recover_radio("ensure RX while TX stuck"))
03883         {
03884             radio_main_notify(MENU_NOTIFICATION_ERROR, "Radio stuck in TX");
03885         }
03886         return;
03887     }
03888
03889     if (radio_get_state() == RADIO_STATE_RX_CONT)
03890     {
03891         return;
03892     }
03893
03894     if (radio_start_rx_continuous() != RADIO_OK)
03895     {
03896         if (!radio_main_force_recover_radio("restart RX failed"))
03897         {
03898             radio_main_notify(MENU_NOTIFICATION_ERROR, "RX restart failed");
03899         }
03900     }
03901 }
03902
03903 static void radio_main_notify(menu_notification_type_t type, const char *text)
03904 {
03905     menu_notification_t n;
03906
03907     memset(&n, 0, sizeof(n));
03908     n.type = type;
03909     if (text != NULL)
03910     {
03911         snprintf(n.text, sizeof(n.text), "%s", text);
03912     }

```

```

03913     (void)menu_main_post_notification(&n);
03914 }
03915
03917 static void radio_main_clear_gateway_pair_code(void)
03918 {
03919     memset(s_ctx.gateway_pair_code, 0, sizeof(s_ctx.gateway_pair_code));
03920     s_ctx.gateway_pair_code_len = 0U;
03921     s_ctx.gateway_pair_code_valid = false;
03922 }
03923
03925 static void radio_main_set_gateway_pair_code(const uint8_t *code, uint8_t len)
03926 {
03927     radio_main_clear_gateway_pair_code();
03928     if ((code == NULL) || (len == 0U))
03929     {
03930         return;
03931     }
03932
03933     s_ctx.gateway_pair_code_len = (len > sizeof(s_ctx.gateway_pair_code)) ?
03934         (uint8_t)sizeof(s_ctx.gateway_pair_code) :
03935         len;
03936     memcpy(s_ctx.gateway_pair_code, code, s_ctx.gateway_pair_code_len);
03937     s_ctx.gateway_pair_code_valid = true;
03938     printf("RADIO: gateway pair code cache updated len=%u\r\n",
03939         (unsigned int)s_ctx.gateway_pair_code_len);
03940     radio_main_print_hex_bytes("RADIO: gateway pair code cache=", s_ctx.gateway_pair_code,
03941         s_ctx.gateway_pair_code_len);
03942 }
03943
03944 static void radio_main_load_gateway_pair_code_from_security(void)
03945 {
03946     trusted_info_t gateway_info;
03947
03948     memset(&gateway_info, 0, sizeof(gateway_info));
03949     if (security_main_cmd_get_device(0U, &gateway_info) &&
03950         gateway_info.in_use &&
03951         (gateway_info.node_id == LAVIET_GATEWAY_ID) &&
03952         (gateway_info.code_len > 0U))
03953     {
03954         radio_main_set_gateway_pair_code(gateway_info.code, gateway_info.code_len);
03955     }
03956 }
03957
03959 static bool radio_main_get_gateway_cached_frame_keys(security_frame_key_mode_t mode,
03960     uint8_t enc_key_out[16],
03961     uint8_t hmac_key_out[32])
03962 {
03963     if (!s_ctx.gateway_pair_code_valid)
03964     {
03965         radio_main_load_gateway_pair_code_from_security();
03966     }
03967
03968     if (!s_ctx.gateway_pair_code_valid)
03969     {
03970         return false;
03971     }
03972
03973     return security_main_get_frame_keys_for_code(s_ctx.node_id,
03974         LAVIET_GATEWAY_ID,
03975         mode,
03976         s_ctx.gateway_pair_code,
03977         s_ctx.gateway_pair_code_len,
03978         enc_key_out,
03979         hmac_key_out);
03980 }
03981
03983 static void radio_main_clear_broadcast_group_state(void)
03984 {
03985     s_ctx.broadcast_group_key_valid = false;
03986     s_ctx.broadcast_group_epoch = 0UL;
03987     laviet_secure_zero(s_ctx.broadcast_group_aes_key, sizeof(s_ctx.broadcast_group_aes_key));
03988     laviet_secure_zero(s_ctx.broadcast_group_hmac_key, sizeof(s_ctx.broadcast_group_hmac_key));
03989     s_ctx.broadcast_group_pending_valid = false;
03990     s_ctx.broadcast_group_pending_epoch = 0UL;
03991     s_ctx.broadcast_group_pending_mask = 0U;
03992     laviet_secure_zero(s_ctx.broadcast_group_pending_key, sizeof(s_ctx.broadcast_group_pending_key));
03993 }
03994
03996 static uint32_t radio_main_be32_read(const uint8_t *src)
03997 {
03998     if (src == NULL)
03999     {
04000         return 0UL;
04001     }
04002
04003     return ((uint32_t)src[0] << 24) |
04004         ((uint32_t)src[1] << 16) |

```

```

04005         ((uint32_t)src[2] << 8) |
04006         (uint32_t)src[3];
04007     }
04008
04010     static bool radio_main_derive_broadcast_group_frame_keys(const uint8_t
        group_key[RADIO_BCAST_GROUP_KEY_LEN],
                                uint32_t epoch,
                                uint8_t enc_key_out[16],
                                uint8_t hmac_key_out[32])
04011     {
04012
04013         static const uint8_t group_info_label[] = "LAVIET:BCAST:GROUP:V1";
04014         uint8_t group_info[(sizeof(group_info_label) - 1U) + 6U];
04015         uint8_t digest[32];
04016         uint8_t base_key[16];
04017         uint8_t key_info[8];
04018         uint8_t off = 0U;
04019         bool ok;
04020
04021         if ((group_key == NULL) || (enc_key_out == NULL) || (hmac_key_out == NULL) || (epoch == 0U))
04022         {
04023             return false;
04024         }
04025
04026         memset(group_info, 0, sizeof(group_info));
04027         memset(digest, 0, sizeof(digest));
04028         memset(base_key, 0, sizeof(base_key));
04029         memcpy(&group_info[off], group_info_label, sizeof(group_info_label) - 1U);
04030         off = (uint8_t)(off + (uint8_t)(sizeof(group_info_label) - 1U));
04031         group_info[off++] = (uint8_t)(epoch >> 24);
04032         group_info[off++] = (uint8_t)(epoch >> 16);
04033         group_info[off++] = (uint8_t)(epoch >> 8);
04034         group_info[off++] = (uint8_t)epoch;
04035         group_info[off++] = (uint8_t)(LAVIET_BROADCAST_ID >> 8);
04036         group_info[off++] = (uint8_t)LAVIET_BROADCAST_ID;
04037
04038         ok = laviet_hmac_sha256(group_key,
                                RADIO_BCAST_GROUP_KEY_LEN,
                                group_info,
                                sizeof(group_info),
                                digest);
04039
04040         if (ok)
04041         {
04042             memcpy(base_key, digest, sizeof(base_key));
04043             key_info[0] = (uint8_t)'L';
04044             key_info[1] = (uint8_t)'V';
04045             key_info[2] = (uint8_t)'1';
04046             key_info[3] = (uint8_t)'K';
04047             key_info[4] = (uint8_t)(LAVIET_BROADCAST_ID >> 8);
04048             key_info[5] = (uint8_t)LAVIET_BROADCAST_ID;
04049             key_info[6] = 0U;
04050             key_info[7] = 1U;
04051             ok = laviet_hmac_sha256(base_key, sizeof(base_key), key_info, sizeof(key_info), digest);
04052             if (ok)
04053             {
04054                 memcpy(enc_key_out, digest, 16U);
04055                 key_info[7] = 2U;
04056                 ok = laviet_hmac_sha256(base_key, sizeof(base_key), key_info, sizeof(key_info),
04057                                         hmac_key_out);
04058             }
04059         }
04060
04061         laviet_secure_zero(group_info, sizeof(group_info));
04062         laviet_secure_zero(digest, sizeof(digest));
04063         laviet_secure_zero(base_key, sizeof(base_key));
04064         laviet_secure_zero(key_info, sizeof(key_info));
04065         if (!ok)
04066         {
04067             laviet_secure_zero(enc_key_out, 16U);
04068             laviet_secure_zero(hmac_key_out, 32U);
04069         }
04070         return ok;
04071     }
04072
04073     static bool radio_main_try_broadcast_group_rx(const laviet_frame_t *frame, uint8_t enc_key[16])
04074     {
04075         uint8_t expected[LAVIET_MAC_TAG_LEN];
04076         bool mac_match = false;
04077         bool ok = false;
04078
04079         if ((frame == NULL) || (enc_key == NULL) ||
04080             !s_ctx.broadcast_group_key_valid ||
04081             (frame->src_id != LAVIET_GATEWAY_ID) ||
04082             (frame->dst_id != LAVIET_BROADCAST_ID) ||
04083             ((frame->flags & LAVIET_FLAG_ENCRYPTED) == 0U))
04084         {
04085             return false;
04086         }
04087     }
04088
04089     }
04090
04091     }

```

```

04092
04093     memset(expected, 0, sizeof(expected));
04094     if (!laviyet_frame_hmac_sha256(frame, s_ctx.broadcast_group_hmac_key, expected))
04095     {
04096         printf("RADIO RX HMAC DBG mode=BCAST_GROUP epoch=%lu src=0x%04X dst=0x%04X msg=0x%04X
counter=%lu flags=0x%02X payload_len=%u\r\n",
04097             (unsigned long)s_ctx.broadcast_group_epoch,
04098             (unsigned int)frame->src_id,
04099             (unsigned int)frame->dst_id,
04100             (unsigned int)frame->msg_id,
04101             (unsigned long)frame->counter,
04102             (unsigned int)frame->flags,
04103             (unsigned int)frame->payload_len);
04104         mac_match = laviyet_mac_equal(expected, frame->mac_tag);
04105     }
04106     if (mac_match)
04107     {
04108         memcpy(enc_key, s_ctx.broadcast_group_aes_key, 16U);
04109         ok = true;
04110     }
04111
04112     laviyet_secure_zero(expected, sizeof(expected));
04113     return ok;
04114 }
04115
04117 static bool radio_main_handle_key_rotate_frame(const laviyet_frame_t *frame)
04118 {
04119     uint32_t epoch;
04120     uint8_t command;
04121
04122     if ((frame == NULL) ||
04123         (frame->src_id != LAVIET_GATEWAY_ID) ||
04124         (frame->dst_id != s_ctx.node_id) ||
04125         (frame->payload_len != LAVIET_MAX_PAYLOAD) ||
04126         (frame->payload[0] != RADIO_BCAST_CTRL_MAGIC))
04127     {
04128         printf("RADIO RX KEY_ROTATE unsupported\r\n");
04129         return false;
04130     }
04131
04132     command = frame->payload[1];
04133     epoch = radio_main_be32_read(&frame->payload[2]);
04134     if (epoch == 0UL)
04135     {
04136         printf("RADIO RX BCAST KEY bad epoch\r\n");
04137         return false;
04138     }
04139
04140     if (command == RADIO_BCAST_CTRL_INSTALL_FRAGMENT)
04141     {
04142         uint8_t fragment_index = frame->payload[6];
04143         uint8_t fragment_count = frame->payload[7];
04144
04145         if ((fragment_count != RADIO_BCAST_GROUP_FRAGMENT_COUNT) ||
04146             (fragment_index >= RADIO_BCAST_GROUP_FRAGMENT_COUNT))
04147         {
04148             printf("RADIO RX BCAST KEY bad fragment idx=%u count=%u\r\n",
04149                 (unsigned int)fragment_index,
04150                 (unsigned int)fragment_count);
04151             return false;
04152         }
04153
04154         if (!s_ctx.broadcast_group_pending_valid) ||
04155             (s_ctx.broadcast_group_pending_epoch != epoch)
04156         {
04157             s_ctx.broadcast_group_pending_valid = true;
04158             s_ctx.broadcast_group_pending_epoch = epoch;
04159             s_ctx.broadcast_group_pending_mask = 0U;
04160             laviyet_secure_zero(s_ctx.broadcast_group_pending_key,
04161                 sizeof(s_ctx.broadcast_group_pending_key));
04162         }
04163
04164         memcpy(&s_ctx.broadcast_group_pending_key[fragment_index * RADIO_BCAST_GROUP_FRAGMENT_LEN],
04165             &frame->payload[8],
04166             RADIO_BCAST_GROUP_FRAGMENT_LEN);
04167         s_ctx.broadcast_group_pending_mask |= (uint8_t)(1U << fragment_index);
04168         printf("RADIO RX BCAST KEY fragment epoch=%lu idx=%u mask=0x%02X\r\n",
04169             (unsigned long)epoch,
04170             (unsigned int)fragment_index,
04171             (unsigned int)s_ctx.broadcast_group_pending_mask);
04172         return true;
04173     }
04174
04175     if (command == RADIO_BCAST_CTRL_ACTIVATE)
04176     {
04177         uint8_t enc_key[16];
04178         uint8_t hmac_key[32];

```



```

04179     uint8_t full_mask = (uint8_t)((1U < RADIO_BCAST_GROUP_FRAGMENT_COUNT) - 1U);
04180
04181     memset(enc_key, 0, sizeof(enc_key));
04182     memset(hmac_key, 0, sizeof(hmac_key));
04183     if ((!s_ctx.broadcast_group_pending_valid) ||
04184         (s_ctx.broadcast_group_pending_epoch != epoch) ||
04185         ((s_ctx.broadcast_group_pending_mask & full_mask) != full_mask))
04186     {
04187         printf("RADIO RX BCAST KEY activate missing epoch=%lu pending_epoch=%lu mask=0x%02X\r\n",
04188             (unsigned long)epoch,
04189             (unsigned long)s_ctx.broadcast_group_pending_epoch,
04190             (unsigned int)s_ctx.broadcast_group_pending_mask);
04191         return false;
04192     }
04193
04194     if (!radio_main_derive_broadcast_group_frame_keys(s_ctx.broadcast_group_pending_key,
04195                                                     epoch,
04196                                                     enc_key,
04197                                                     hmac_key))
04198     {
04199         printf("RADIO RX BCAST KEY derive failed epoch=%lu\r\n", (unsigned long)epoch);
04200         return false;
04201     }
04202
04203     memcpy(s_ctx.broadcast_group_aes_key, enc_key, sizeof(s_ctx.broadcast_group_aes_key));
04204     memcpy(s_ctx.broadcast_group_hmac_key, hmac_key, sizeof(s_ctx.broadcast_group_hmac_key));
04205     s_ctx.broadcast_group_epoch = epoch;
04206     s_ctx.broadcast_group_key_valid = true;
04207     s_ctx.broadcast_group_pending_valid = false;
04208     s_ctx.broadcast_group_pending_epoch = 0UL;
04209     s_ctx.broadcast_group_pending_mask = 0U;
04210     laviet_secure_zero(s_ctx.broadcast_group_pending_key,
04211                       sizeof(s_ctx.broadcast_group_pending_key));
04212     laviet_secure_zero(enc_key, sizeof(enc_key));
04213     laviet_secure_zero(hmac_key, sizeof(hmac_key));
04214     printf("RADIO RX BCAST KEY active epoch=%lu\r\n", (unsigned long)epoch);
04215     return true;
04216 }
04217
04218 printf("RADIO RX BCAST KEY unknown cmd=%u epoch=%lu\r\n",
04219       (unsigned int)command,
04220       (unsigned long)epoch);
04221 return false;
04222 }
04223
04225 static uint8_t radio_main_format_payload_text(const uint8_t *payload,
04226                                              uint8_t payload_len,
04227                                              char *out,
04228                                              uint8_t out_size)
04229 {
04230     static const char hex_table[] = "0123456789ABCDEF";
04231     uint8_t i;
04232     uint8_t out_idx = 0U;
04233     bool all_printable = true;
04234
04235     if ((out == NULL) || (out_size == 0U))
04236     {
04237         return 0U;
04238     }
04239
04240     out[0] = '\0';
04241     if ((payload == NULL) || (payload_len == 0U))
04242     {
04243         return 0U;
04244     }
04245
04246     for (i = 0U; i < payload_len; i++)
04247     {
04248         if (!isprint((unsigned char)payload[i]))
04249         {
04250             all_printable = false;
04251             break;
04252         }
04253     }
04254
04255     if (all_printable)
04256     {
04257         uint8_t copy_len = (payload_len >= (out_size - 1U)) ? (uint8_t)(out_size - 1U) : payload_len;
04258
04259         for (i = 0U; i < copy_len; i++)
04260         {
04261             out[i] = (char)payload[i];
04262         }
04263         out[copy_len] = '\0';
04264         return copy_len;
04265     }
04266 }

```

```

04267     if (out_size <= 5U)
04268     {
04269         return 0U;
04270     }
04271
04272     out[out_idx++] = 'H';
04273     out[out_idx++] = 'E';
04274     out[out_idx++] = 'X';
04275     out[out_idx++] = ':';
04276
04277     for (i = 0U; i < payload_len; i++)
04278     {
04279         if ((out_idx + 2U) >= out_size)
04280         {
04281             break;
04282         }
04283         out[out_idx++] = hex_table[payload[i] >> 4];
04284         out[out_idx++] = hex_table[payload[i] & 0x0F];
04285     }
04286
04287     out[out_idx] = '\0';
04288     return out_idx;
04289 }
04290
04292 static bool radio_main_push_payload_to_monitor(int16_t rssi_dbm,
04293                                               uint16_t src_id,
04294                                               const uint8_t *payload,
04295                                               uint8_t payload_len)
04296 {
04297     char text_buf[21];
04298     uint8_t text_len;
04299
04300     text_len = radio_main_format_payload_text(payload, payload_len, text_buf, sizeof(text_buf));
04301     if (text_len == 0U)
04302     {
04303         return false;
04304     }
04305
04306     return lcd_main_push_message_from(rssi_dbm, src_id, (const uint8_t *)text_buf, text_len);
04307 }
04308
04310 static void radio_main_log_gateway_rx_frame(const laviet_frame_t *raw_frame,
04311                                             const laviet_frame_t *decoded_frame,
04312                                             int16_t rssi_dbm,
04313                                             int8_t snr_db)
04314 {
04315     char raw_text[21];
04316     char dec_text[21];
04317     laviet_frame_type_t type;
04318     uint8_t version;
04319
04320     if ((raw_frame == NULL) || (decoded_frame == NULL) || (decoded_frame->src_id !=
LAVIET_GATEWAY_ID))
04321     {
04322         return;
04323     }
04324
04325     type = laviet_frame_type(decoded_frame);
04326     version = (uint8_t)((decoded_frame->ver_type >> 4) & 0x0FU);
04327
04328     printf("RADIO RX GATEWAY version=%u ver_type=0x%02X type=%s flags=0x%02X src=0x%04X dst=0x%04X
msg=0x%04X counter=%lu payload_len=%u rssi=%d snr=%d\r\n",
04329           (unsigned int)version,
04330           (unsigned int)decoded_frame->ver_type,
04331           radio_main_frame_type_text(type),
04332           (unsigned int)decoded_frame->flags,
04333           (unsigned int)decoded_frame->src_id,
04334           (unsigned int)decoded_frame->dst_id,
04335           (unsigned int)decoded_frame->msg_id,
04336           (unsigned long)decoded_frame->counter,
04337           (unsigned int)decoded_frame->payload_len,
04338           (int)rssi_dbm,
04339           (int)snr_db);
04340
04341     printf("RADIO RX GATEWAY FLAGS enc=%u ack_req=%u is_ack=%u pairing=%u cfg=%u bcast=%u
ctr_override=%u key_update=%u\r\n",
04342           ((decoded_frame->flags & LAVIET_FLAG_ENCRYPTED) != 0U) ? 1U : 0U,
04343           ((decoded_frame->flags & LAVIET_FLAG_ACK_REQUIRED) != 0U) ? 1U : 0U,
04344           ((decoded_frame->flags & LAVIET_FLAG_IS_ACK) != 0U) ? 1U : 0U,
04345           ((decoded_frame->flags & LAVIET_FLAG_PAIRING) != 0U) ? 1U : 0U,
04346           ((decoded_frame->flags & LAVIET_FLAG_CONFIG_ACCESS) != 0U) ? 1U : 0U,
04347           ((decoded_frame->flags & LAVIET_FLAG_BROADCAST) != 0U) ? 1U : 0U,
04348           ((decoded_frame->flags & LAVIET_FLAG_COUNTER_OVERRIDE) != 0U) ? 1U : 0U,
04349           ((decoded_frame->flags & LAVIET_FLAG_KEY_UPDATE) != 0U) ? 1U : 0U);
04350
04351     radio_main_print_hex_bytes("RADIO RX GATEWAY MAC: ", decoded_frame->mac_tag, LAVIET_MAC_TAG_LEN);
04352     radio_main_print_hex_bytes("RADIO RX GATEWAY PAYLOAD RAW HEX: ", raw_frame->payload,

```

```

        raw_frame->payload_len);
04353     if (radio_main_format_payload_text(raw_frame->payload, raw_frame->payload_len, raw_text,
sizeof(raw_text)) > 0U)
04354     {
04355         printf("RADIO RX GATEWAY PAYLOAD RAW TXT: \"%s\"\\r\\n", raw_text);
04356     }
04357
04358     radio_main_print_hex_bytes("RADIO RX GATEWAY PAYLOAD DEC HEX: ", decoded_frame->payload,
decoded_frame->payload_len);
04359     if (radio_main_format_payload_text(decoded_frame->payload, decoded_frame->payload_len, dec_text,
sizeof(dec_text)) > 0U)
04360     {
04361         printf("RADIO RX GATEWAY PAYLOAD DEC TXT: \"%s\"\\r\\n", dec_text);
04362     }
04363 }
04364
04366 static void radio_main_print_hex_bytes(const char *label, const uint8_t *data, uint16_t len)
04367 {
04368     uint16_t i;
04369
04370     printf("%s", (label != NULL) ? label : "");
04371     for (i = 0U; i < len; i++)
04372     {
04373         printf("%02X", data[i]);
04374         if ((uint16_t)(i + 1U) < len)
04375         {
04376             printf(" ");
04377         }
04378     }
04379     printf("\\r\\n");
04380 }
04381
04383 static void radio_main_log_hmac_debug(const char *prefix,
const laviet_frame_t *frame,
security_frame_key_mode_t mode,
const uint8_t *pair_code,
uint8_t pair_code_len,
const uint8_t hmac_key[32],
const uint8_t expected_mac[LAVIET_MAC_TAG_LEN])
04390 {
04391     uint8_t mac_input[LAVIET_FRAME_HEADER_LEN + LAVIET_MAX_PAYLOAD];
04392     uint8_t mac_input_len = 0U;
04393
04394     if ((prefix == NULL) || (frame == NULL) || (hmac_key == NULL) || (expected_mac == NULL))
04395     {
04396         return;
04397     }
04398
04399     printf("%s mode=%s src=0x%04X dst=0x%04X msg=0x%04X counter=%lu flags=0x%02X payload_len=%u\\r\\n",
prefix,
radio_main_key_mode_text(mode),
(unsigned int)frame->src_id,
(unsigned int)frame->dst_id,
(unsigned int)frame->msg_id,
(unsigned long)frame->counter,
(unsigned int)frame->flags,
(unsigned int)frame->payload_len);
04408     if ((pair_code != NULL) && (pair_code_len > 0U))
04409     {
04410         radio_main_print_hex_bytes(" pair_code=", pair_code, pair_code_len);
04411     }
04412     else
04413     {
04414         printf(" pair_code=<none>\\r\\n");
04415     }
04416     radio_main_print_hex_bytes(" hmac_key=", hmac_key, LAVIET_HMAC_KEY_LEN);
04417     if (laviet_frame_build_mac_input(frame, mac_input, sizeof(mac_input), &mac_input_len)
04418     {
04419         radio_main_print_hex_bytes(" mac_input=", mac_input, mac_input_len);
04420     }
04421     else
04422     {
04423         printf(" mac_input=<build_failed>\\r\\n");
04424     }
04425     radio_main_print_hex_bytes(" expected_mac=", expected_mac, LAVIET_MAC_TAG_LEN);
04426     radio_main_print_hex_bytes(" frame_mac=", frame->mac_tag, LAVIET_MAC_TAG_LEN);
04427 }
04428
04430 static void radio_main_print_generated_pattern(const char *label, uint8_t value, uint16_t len)
04431 {
04432     uint16_t i;
04433
04434     printf("%s", (label != NULL) ? label : "");
04435     for (i = 0U; i < len; i++)
04436     {
04437         printf("%02X", value);
04438         if ((uint16_t)(i + 1U) < len)

```

```

04439     {
04440         printf(" ");
04441     }
04442 }
04443 printf("\r\n");
04444 }
04445
04447 static void radio_main_print_fsk_sync_word(uint64_t sync_word, uint8_t sync_len, const char *label)
04448 {
04449     uint8_t bytes[8];
04450     uint8_t i;
04451
04452     if (sync_len == 0U)
04453     {
04454         printf("%sOFF\r\n", (label != NULL) ? label : "");
04455         return;
04456     }
04457
04458     for (i = 0U; i < sync_len; i++)
04459     {
04460         uint8_t shift = (uint8_t)((sync_len - 1U - i) * 8U) & 0x3FU;
04461         bytes[i] = (uint8_t)((sync_word » shift) & 0xFFU);
04462     }
04463     radio_main_print_hex_bytes(label, bytes, sync_len);
04464 }
04465
04467 static void radio_main_print_ook_sync_word(uint32_t sync_word, uint8_t sync_len, const char *label)
04468 {
04469     uint8_t bytes[4];
04470     uint8_t i;
04471
04472     if (sync_len == 0U)
04473     {
04474         printf("%sOFF\r\n", (label != NULL) ? label : "");
04475         return;
04476     }
04477
04478     for (i = 0U; i < sync_len; i++)
04479     {
04480         uint8_t shift = (uint8_t)((sync_len - 1U - i) * 8U) & 0x1FU;
04481         bytes[i] = (uint8_t)((sync_word » shift) & 0xFFU);
04482     }
04483     radio_main_print_hex_bytes(label, bytes, sync_len);
04484 }
04485
04487 static void radio_main_print_rx_ascii(const uint8_t *data, uint8_t len)
04488 {
04489     uint8_t i;
04490
04491     printf("RADIO RX RAW: ");
04492     for (i = 0U; i < len; i++)
04493     {
04494         char c = (char)data[i];
04495         printf("%c", isprint((unsigned char)c) ? c : '.');
04496     }
04497     printf("\r\n");
04498 }
04499
04501 static void radio_main_print_tx_ascii(const uint8_t *data, uint8_t len)
04502 {
04503     uint8_t i;
04504
04505     printf("RADIO TX TEXT: ");
04506     for (i = 0U; i < len; i++)
04507     {
04508         char c = (char)data[i];
04509         printf("%c", isprint((unsigned char)c) ? c : '.');
04510     }
04511     printf("\r\n");
04512 }
04513
04514 /*
04515  * Prints a terminal-side view of the radio packet as it is emitted by the
04516  * current backend. Hardware-generated elements such as preamble, sync or CRC
04517  * are logged explicitly from the active configuration so diagnostics include
04518  * more than just the BEKO payload bytes.
04519  */
04520 static void radio_main_print_tx_frame(const uint8_t *data, uint8_t len, bool retry_attempt)
04521 {
04522     const char *prefix = retry_attempt ? "RADIO TX RETRY" : "RADIO TX";
04523
04524     if ((data == NULL) || (len == 0U))
04525     {
04526         return;
04527     }
04528
04529     if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK)

```

```

04530     {
04531         uint8_t length_byte = len;
04532
04533         printf("%s FSK freq=%luHz bitrate=%lubps power=%ddBm bw=%u\r\n",
04534             prefix,
04535             (unsigned long)s_ctx.fsk_cfg.frequency_hz,
04536             (unsigned long)s_ctx.fsk_cfg.bitrate_bps,
04537             (int)s_ctx.fsk_cfg.tx_power_dbm,
04538             (unsigned int)s_ctx.fsk_cfg.rx_bandwidth);
04539         printf("%s FSK PREAMBLE(%uB HW): ",
04540             prefix,
04541             (unsigned int)s_ctx.fsk_cfg.preamble_len);
04542         radio_main_print_generated_pattern(" ", 0xAAU, s_ctx.fsk_cfg.preamble_len);
04543         printf("%s FSK SYNC(%uB HW): ", prefix, (unsigned int)s_ctx.fsk_cfg.sync_word_len);
04544         radio_main_print_fsk_sync_word(s_ctx.fsk_cfg.sync_word, s_ctx.fsk_cfg.sync_word_len, "");
04545         printf("%s FSK LEN(HW): %02X\r\n", prefix, length_byte);
04546         radio_main_print_hex_bytes("RADIO TX FSK PAYLOAD: ", data, len);
04547         printf("%s FSK CRC(HW): %s\r\n",
04548             prefix,
04549             (s_ctx.fsk_cfg.crc_type == RADIO_MAIN_CRC_OFF) ? "OFF" :
04550             ((s_ctx.fsk_cfg.crc_type == RADIO_MAIN_CRC_IBM) ? "IBM" : "CCITT"));
04551         radio_main_print_tx_ascii(data, len);
04552         return;
04553     }
04554
04555     if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK)
04556     {
04557         uint8_t length_byte = len;
04558
04559         printf("%s OOK freq=%luHz bitrate=%lubps power=%ddBm bw=%u thr=%u/%u\r\n",
04560             prefix,
04561             (unsigned long)s_ctx.ook_cfg.frequency_hz,
04562             (unsigned long)s_ctx.ook_cfg.bitrate_bps,
04563             (int)s_ctx.ook_cfg.tx_power_dbm,
04564             (unsigned int)s_ctx.ook_cfg.rx_bandwidth,
04565             (unsigned int)s_ctx.ook_cfg.threshold,
04566             (unsigned int)s_ctx.ook_cfg.threshold_value);
04567         printf("%s OOK PREAMBLE(%uB HW): ",
04568             prefix,
04569             (unsigned int)s_ctx.ook_cfg.preamble_len);
04570         radio_main_print_generated_pattern(" ", 0xAAU, s_ctx.ook_cfg.preamble_len);
04571         printf("%s OOK SYNC(%uB HW): ", prefix, (unsigned int)s_ctx.ook_cfg.sync_word_len);
04572         radio_main_print_ook_sync_word(s_ctx.ook_cfg.sync_word, s_ctx.ook_cfg.sync_word_len, "");
04573         printf("%s OOK LEN(HW): %02X\r\n", prefix, length_byte);
04574         radio_main_print_hex_bytes("RADIO TX OOK PAYLOAD: ", data, len);
04575         printf("%s OOK CRC(HW): OFF\r\n", prefix);
04576         radio_main_print_tx_ascii(data, len);
04577         return;
04578     }
04579
04580     printf("%s LORA freq=%luHz bw=%u sf=%u cr=4/%u preamble=%u sym sync=%02X crc=%s hdr=%s iq=%s\r\n",
04581         prefix,
04582         (unsigned long)s_ctx.lora_cfg.frequency_hz,
04583         (unsigned int)s_ctx.lora_cfg.bandwidth,
04584         (unsigned int)s_ctx.lora_cfg.spreading_factor,
04585         (unsigned int)s_ctx.lora_cfg.coding_rate,
04586         (unsigned int)s_ctx.lora_cfg.preamble_len,
04587         (unsigned int)s_ctx.lora_cfg.sync_word,
04588         s_ctx.lora_cfg.crc_on ? "ON" : "OFF",
04589         s_ctx.lora_cfg.implicit_header ? "implicit" : "explicit",
04590         s_ctx.lora_cfg.invert_iq ? "invert" : "normal");
04591     radio_main_print_hex_bytes("RADIO TX LORA PAYLOAD: ", data, len);
04592     radio_main_print_tx_ascii(data, len);
04593 }
04594
04596 static uint32_t radio_main_frf_to_hz(uint8_t msb, uint8_t mid, uint8_t lsb)
04597 {
04598     uint32_t frf = ((uint32_t)msb << 16) |
04599         ((uint32_t)mid << 8) |
04600         (uint32_t)lsb;
04601
04602     return (uint32_t)((uint64_t)frf * 32000000ULL >> 19);
04603 }
04604
04606 static void radio_main_log_runtime_profile(const char *reason)
04607 {
04608     const char *tag = (reason != NULL) ? reason : "state";
04609
04610     printf("RADIO CFG [%s]: sel=%u backend=%u lib=%u state=%d init=%u\r\n",
04611         tag,
04612         (unsigned int)s_ctx.modulation_id,
04613         (unsigned int)s_ctx.backend_modulation_id,
04614         (unsigned int)radio_get_backend(),
04615         (int)radio_get_state(),
04616         s_ctx.initialized ? 1U : 0U);
04617
04618     if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK)

```

```

04619     {
04620         printf("RADIO CFG [%s] FSK: freq=%luHz br=%lubps bw=%u shape=%u filter=%u pre=%u sync_len=%u
04621             crc=%u white=%u addr=%u\r\n",
04622             tag,
04623             (unsigned long)s_ctx.fsk_cfg.frequency_hz,
04624             (unsigned long)s_ctx.fsk_cfg.bitrate_bps,
04625             (unsigned int)s_ctx.fsk_cfg.rx_bandwidth,
04626             (unsigned int)s_ctx.fsk_cfg.shaping,
04627             (unsigned int)s_ctx.fsk_cfg.filter,
04628             (unsigned int)s_ctx.fsk_cfg.preamble_len,
04629             (unsigned int)s_ctx.fsk_cfg.sync_word_len,
04630             (unsigned int)s_ctx.fsk_cfg.crc_type,
04631             s_ctx.fsk_cfg.data_whitening ? 1U : 0U,
04632             (unsigned int)s_ctx.fsk_cfg.address_filter);
04633         radio_main_print_fsk_sync_word(s_ctx.fsk_cfg.sync_word,
04634             s_ctx.fsk_cfg.sync_word_len,
04635             "RADIO CFG FSK SYNC: ");
04636     }
04637     return;
04638 }
04639 if (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK)
04640 {
04641     printf("RADIO CFG [%s] OOK: freq=%luHz br=%lubps bw=%u pre=%u sync_len=%u thr=%u
04642         thr_v=%u\r\n",
04643         tag,
04644         (unsigned long)s_ctx.ook_cfg.frequency_hz,
04645         (unsigned long)s_ctx.ook_cfg.bitrate_bps,
04646         (unsigned int)s_ctx.ook_cfg.rx_bandwidth,
04647         (unsigned int)s_ctx.ook_cfg.preamble_len,
04648         (unsigned int)s_ctx.ook_cfg.sync_word_len,
04649         (unsigned int)s_ctx.ook_cfg.threshold,
04650         (unsigned int)s_ctx.ook_cfg.threshold_value);
04651         radio_main_print_ook_sync_word(s_ctx.ook_cfg.sync_word,
04652             s_ctx.ook_cfg.sync_word_len,
04653             "RADIO CFG OOK SYNC: ");
04654     }
04655     return;
04656 }
04657 printf("RADIO CFG [%s] LORA: freq=%luHz bw=%u sf=%u cr=4/%u pre=%u sync=%02X crc=%u iq=%u
04658     hdr=%u\r\n",
04659     tag,
04660     (unsigned long)s_ctx.lora_cfg.frequency_hz,
04661     (unsigned int)s_ctx.lora_cfg.bandwidth,
04662     (unsigned int)s_ctx.lora_cfg.spreading_factor,
04663     (unsigned int)s_ctx.lora_cfg.coding_rate,
04664     (unsigned int)s_ctx.lora_cfg.preamble_len,
04665     (unsigned int)s_ctx.lora_cfg.sync_word,
04666     s_ctx.lora_cfg.crc_on ? 1U : 0U,
04667     s_ctx.lora_cfg.invert_iq ? 1U : 0U,
04668     s_ctx.lora_cfg.implicit_header ? 1U : 0U);
04669 }
04670 static void radio_main_log_hw_registers(const char *reason)
04671 {
04672     uint8_t op_mode = 0U;
04673     uint8_t frf_msb = 0U;
04674     uint8_t frf_mid = 0U;
04675     uint8_t frf_lsb = 0U;
04676     uint8_t rx_bw = 0U;
04677     uint8_t bitrate_msb = 0U;
04678     uint8_t bitrate_lsb = 0U;
04679     uint8_t sync_cfg = 0U;
04680     uint8_t packet_cfg2 = 0U;
04681     const char *tag = (reason != NULL) ? reason : "state";
04682     bool ok;
04683     if (!s_ctx.initialized)
04684     {
04685         return;
04686     }
04687     ok = (radio_raw_read_reg(SX1276_REG_OP_MODE, &op_mode) == RADIO_OK) &&
04688         (radio_raw_read_reg(SX1276_REG_FRF_MSB, &frf_msb) == RADIO_OK) &&
04689         (radio_raw_read_reg(SX1276_REG_FRF_MID, &frf_mid) == RADIO_OK) &&
04690         (radio_raw_read_reg(SX1276_REG_FRF_LSB, &frf_lsb) == RADIO_OK) &&
04691         (radio_raw_read_reg(SX1276_REG_RX_BW, &rx_bw) == RADIO_OK);
04692     if (!ok)
04693     {
04694         printf("RADIO HW [%s]: reg read failed\r\n", tag);
04695         return;
04696     }
04697     printf("RADIO HW [%s]: opmode=0x%02X mode=%u mod=0x%02X frf=0x%02X%02X%02X -> %luHz
04698         rx_bw=0x%02X\r\n",
04699         tag,
04700         (unsigned int)op_mode,
04701         (unsigned int)(op_mode & SX1276_OPMODE_MODE_MASK),

```

```

04703         (unsigned int)(op_mode & SX1276_OPMODE_MODULATION_MASK),
04704         (unsigned int)frf_msb,
04705         (unsigned int)frf_mid,
04706         (unsigned int)frf_lsb,
04707         (unsigned long)radio_main_frf_to_hz(frf_msb, frf_mid, frf_lsb),
04708         (unsigned int)rx_bw);
04709
04710     if ((s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_FSK) ||
04711         (s_ctx.backend_modulation_id == RADIO_MAIN_MODULATION_OOK))
04712     {
04713         ok = (radio_raw_read_reg(SX1276_REG_BITRATE_MSB, &bitrate_msb) == RADIO_OK) &&
04714             (radio_raw_read_reg(SX1276_REG_BITRATE_LSB, &bitrate_lsb) == RADIO_OK) &&
04715             (radio_raw_read_reg(SX1276_REG_SYNC_CONFIG, &sync_cfg) == RADIO_OK) &&
04716             (radio_raw_read_reg(SX1276_REG_PACKET_CONFIG_2, &packet_cfg2) == RADIO_OK);
04717         if (ok)
04718         {
04719             uint16_t bitrate_reg = ((uint16_t)bitrate_msb << 8) | bitrate_lsb;
04720             uint32_t bitrate_hz = (bitrate_reg != 0U) ? (32000000U / (uint32_t)bitrate_reg) : 0U;
04721
04722             printf("RADIO HW [%s]: bitrate_reg=0x%04X -> %lubps sync_cfg=0x%02X
04723 packet_cfg2=0x%02X\r\n",
04724                 tag,
04725                 (unsigned int)bitrate_reg,
04726                 (unsigned long)bitrate_hz,
04727                 (unsigned int)sync_cfg,
04728                 (unsigned int)packet_cfg2);
04729         }
04730     }
04731
04732     static bool radio_main_finish_pairing(bool accept)
04733     {
04734         bool ok;
04735         menu_notification_t n;
04736         uint32_t node_id;
04737         uint8_t code_len;
04738         const uint8_t *code;
04739         char code_text[12];
04740         uint8_t payload[16];
04741         uint8_t payload_len;
04742
04743         if (!s_ctx.pairing_pending)
04744         {
04745             return false;
04746         }
04747
04748         node_id = s_ctx.pairing_pending_node;
04749         code = s_ctx.pairing_pending_code;
04750         code_len = s_ctx.pairing_pending_code_len;
04751
04752         if (!accept)
04753         {
04754             uint8_t err_payload[LAVIET_ERROR_PAYLOAD_LEN] = { 1U, LAVIET_TYPE_PAIR_REQ, 0U, 0U, 0U, 0U,
04755 0U, 0U };
04756
04757             (void)radio_main_send_system_frame(LAVIET_TYPE_ERROR,
04758                 node_id,
04759                 err_payload,
04760                 sizeof(err_payload));
04761
04762             memset(&n, 0, sizeof(n));
04763             n.type = MENU_NOTIFICATION_PAIRING;
04764             snprintf(n.text, sizeof(n.text), "Device rejected");
04765             (void)menu_main_post_notification(&n);
04766             s_ctx.pairing_pending = false;
04767             s_ctx.pairing_pending_network = false;
04768             return true;
04769         }
04770
04771         if (s_ctx.pairing_pending_network)
04772         {
04773             ok = security_main_cmd_add_gateway(node_id, code, code_len);
04774         }
04775         else
04776         {
04777             ok = security_main_cmd_add_device(node_id, code, code_len);
04778         }
04779         if (!ok)
04780         {
04781             radio_main_notify(MENU_NOTIFICATION_ERROR, "Pairing save failed");
04782             s_ctx.pairing_pending = false;
04783             s_ctx.pairing_pending_network = false;
04784             return false;
04785         }
04786         if (s_ctx.pairing_pending_network)
04787         {
04788             radio_main_set_gateway_pair_code(code, code_len);
04789         }
04790     }

```

```

04789
04790     payload_len = radio_main_pair_payload_build(s_ctx.pairing_pending_network,
04791                                                code,
04792                                                code_len,
04793                                                payload,
04794                                                (uint8_t)sizeof(payload));
04795     if (payload_len == 0U)
04796     {
04797         s_ctx.pairing_pending = false;
04798         s_ctx.pairing_pending_network = false;
04799         return false;
04800     }
04801
04802     (void)radio_main_send_system_frame(LAVIET_TYPE_PAIR_RESP, node_id, payload, payload_len);
04803
04804     radio_main_pair_code_to_text(code, code_len, code_text, (uint8_t)sizeof(code_text));
04805
04806     memset(&n, 0, sizeof(n));
04807     n.type = MENU_NOTIFICATION_PAIRING;
04808     snprintf(n.text, sizeof(n.text), "PAIR_OK %s", code_text);
04809     (void)menu_main_post_notification(&n);
04810     if (s_ctx.pairing_pending_network)
04811     {
04812         s_ctx.gateway_key_mode_known = false;
04813     }
04814     s_ctx.pairing_pending = false;
04815     s_ctx.pairing_pending_network = false;
04816     s_ctx.pairing_active = false;
04817     s_ctx.pairing_network_mode = false;
04818     return true;
04819 }
04820
04822 static bool radio_main_send_pair_request_internal(void)
04823 {
04824     char note[21];
04825     char code_text[12];
04826     uint8_t payload[16];
04827     uint8_t payload_len;
04828     uint32_t dst_id;
04829
04830     if (!s_ctx.pairing_active)
04831     {
04832         return false;
04833     }
04834     s_ctx.pairing_outgoing_code_len = RADIO_PAIR_CODE_LEN;
04835     radio_main_make_pair_code(s_ctx.pairing_outgoing_code, s_ctx.pairing_outgoing_code_len);
04836     payload_len = radio_main_pair_payload_build(s_ctx.pairing_network_mode,
04837                                                s_ctx.pairing_outgoing_code,
04838                                                s_ctx.pairing_outgoing_code_len,
04839                                                payload,
04840                                                (uint8_t)sizeof(payload));
04841     if (payload_len == 0U)
04842     {
04843         return false;
04844     }
04845
04846     dst_id = s_ctx.pairing_network_mode ? LAVIET_GATEWAY_ID : LAVIET_BROADCAST_ID;
04847     if (!radio_main_send_system_frame(LAVIET_TYPE_PAIR_REQ,
04848                                     dst_id,
04849                                     payload,
04850                                     payload_len))
04851     {
04852         return false;
04853     }
04854
04855     s_ctx.pairing_outgoing_pending = true;
04856     s_ctx.pairing_outgoing_network = s_ctx.pairing_network_mode;
04857     radio_main_pair_code_to_text(s_ctx.pairing_outgoing_code,
04858                                s_ctx.pairing_outgoing_code_len,
04859                                code_text,
04860                                (uint8_t)sizeof(code_text));
04861     snprintf(note, sizeof(note), "PAIR_SENT %.10s", code_text);
04862     radio_main_notify(MENU_NOTIFICATION_PAIRING, note);
04863     return true;
04864 }
04865
04867 static void radio_main_make_pair_code(uint8_t *code_out, uint8_t len)
04868 {
04869     uint8_t i;
04870     uint32_t x;
04871
04872     if ((code_out == NULL) || (len == 0U))
04873     {
04874         return;
04875     }
04876
04877     x = radio_main_now_ms() ^ s_ctx.node_id ^ (s_ctx.next_msg_id * 2654435761UL);

```



```

04878     for (i = 0U; i < len; i++)
04879     {
04880         x = (1103515245UL * x) + 12345UL;
04881         code_out[i] = (uint8_t)('0' + ((x >> 16) % 10UL));
04882     }
04883 }
04884
04886 static void radio_main_pair_code_to_text(const uint8_t *code, uint8_t len, char *out, uint8_t
    out_size)
04887 {
04888     uint8_t i;
04889     uint8_t max_copy;
04890
04891     if ((out == NULL) || (out_size == 0U))
04892     {
04893         return;
04894     }
04895
04896     out[0] = '\0';
04897     if ((code == NULL) || (len == 0U))
04898     {
04899         return;
04900     }
04901
04902     max_copy = (len >= (out_size - 1U)) ? (uint8_t)(out_size - 1U) : len;
04903     for (i = 0U; i < max_copy; i++)
04904     {
04905         char c = (char)code[i];
04906         out[i] = isprint((unsigned char)c) ? c : '.';
04907     }
04908     out[max_copy] = '\0';
04909 }
04910
04912 static bool radio_main_pair_payload_parse(const uint8_t *payload,
    uint8_t payload_len,
04913     const uint8_t **code_out,
04914     uint8_t *code_len_out)
04915 {
04916     if ((code_out == NULL) || (code_len_out == NULL))
04917     {
04918         return false;
04919     }
04920
04921     *code_out = NULL;
04922     *code_len_out = 0U;
04923
04924     if ((payload == NULL) || (payload_len != RADIO_PAIR_CODE_LEN))
04925     {
04926         return false;
04927     }
04928
04929     *code_out = payload;
04930     *code_len_out = payload_len;
04931     return true;
04932 }
04933
04936 static uint8_t radio_main_pair_payload_build(bool network_mode,
    const uint8_t *code,
04937     uint8_t code_len,
04938     uint8_t *payload_out,
04939     uint8_t payload_capacity)
04940 {
04941     (void)network_mode;
04942
04943     if ((payload_out == NULL) || (code == NULL) || (code_len == 0U))
04944     {
04945         return 0U;
04946     }
04947
04948     if ((code_len != RADIO_PAIR_CODE_LEN) || (payload_capacity < RADIO_PAIR_CODE_LEN))
04949     {
04950         return 0U;
04951     }
04952
04953     memcpy(payload_out, code, code_len);
04954     return code_len;
04955 }
04956
04957 static uint32_t radio_main_now_ms(void)
04958 {
04959     osKernelState_t state = osKernelGetState();
04960     uint32_t tick_hz;
04961     uint32_t ticks;
04962
04963     if ((state == osKernelRunning) || (state == osKernelLocked))
04964     {
04965         ticks = osKernelGetTickCount();
04966     }

```

```

04968         tick_hz = osKernelGetTickFreq();
04969         if (tick_hz == 0U)
04970         {
04971             return ticks;
04972         }
04973         return (uint32_t)((uint64_t)ticks * 1000ULL) / tick_hz;
04974     }
04975
04976     return HAL_GetTick();
04977 }

```

12.77 Dokumentacja pliku Core/App/radio_main.h

Application radio task, runtime profile and command API.

```

#include "radio_lib/radio_lib.h"
#include <stdbool.h>
#include <stdint.h>

```

Struktury danych

- struct [radio_main_fsk_cfg_t](#)
Profil rodziny FSK/GFSK/MSK/GMSK trzymany przez warstwę aplikacji.
- struct [radio_main_ook_cfg_t](#)
Profil rodziny OOK przechowywany przez warstwę aplikacji.
- struct [radio_main_runtime_cfg_t](#)
Snapshot pełnej konfiguracji radiowej widzianej przez aplikację.

Wyliczenia

- enum [radio_main_modulation_t](#) { [RADIO_MAIN_MODULATION_LORA](#) = 0U , [RADIO_MAIN_MODULATION_FSK](#) = 1U , [RADIO_MAIN_MODULATION_OOK](#) = 2U }
- Rodziny modulacji zarządzane przez warstwę aplikacyjną.*
- enum [radio_main_auto_ping_mode_t](#) { [RADIO_MAIN_AUTO_PING_FRAME](#) = 0U , [RADIO_MAIN_AUTO_PING_RAW](#) = 1U }
- Encoding used for the periodic automatic ping feature.*
- enum [radio_main_fsk_shaping_t](#) { [RADIO_MAIN_FSK_SHAPING_FSK](#) = 0U , [RADIO_MAIN_FSK_SHAPING_GFSK](#) = 1U , [RADIO_MAIN_FSK_SHAPING_MSK](#) = 2U , [RADIO_MAIN_FSK_SHAPING_GMSK](#) = 3U }
- Typ kształtowania sygnału dla rodziny FSK.*
- enum [radio_main_filter_t](#) { [RADIO_MAIN_FILTER_NONE](#) = 0U , [RADIO_MAIN_FILTER_BT_10](#) = 1U , [RADIO_MAIN_FILTER_BT_07](#) = 2U , [RADIO_MAIN_FILTER_BT_05](#) = 3U , [RADIO_MAIN_FILTER_BT_03](#) = 4U }
- Typ filtra i współczynnik BT dla FSK/GFSK/GMSK.*
- enum [radio_main_crc_type_t](#) { [RADIO_MAIN_CRC_OFF](#) = 0U , [RADIO_MAIN_CRC_SX1276](#) = 1U , [RADIO_MAIN_CRC_IBM](#) = 2U , [RADIO_MAIN_CRC_CCITT](#) = 3U }
- Rodzaj CRC pakietowego zapisanego w profilu aplikacyjnym.*
- enum [radio_main_header_mode_t](#) { [RADIO_MAIN_HEADER_EXPLICIT](#) = 0U , [RADIO_MAIN_HEADER_IMPLICIT](#) = 1U }
- Tryb nagłówka pakietu.*
- enum [radio_main_address_filter_t](#) { [RADIO_MAIN_ADDRESS_FILTER_NONE](#) = 0U , [RADIO_MAIN_ADDRESS_FILTER_NODE_BROADCAST](#) = 1U , [RADIO_MAIN_ADDRESS_FILTER_NODE_BROADCAST](#) = 2U }
- Tryb filtrowania adresu dla trybów pakietowych FSK.*
- enum [radio_main_ook_threshold_t](#) { [RADIO_MAIN_OOK_THRESHOLD_FIXED](#) = 0U , [RADIO_MAIN_OOK_THRESHOLD_PE](#) = 1U , [RADIO_MAIN_OOK_THRESHOLD_AVERAGE](#) = 2U }
- Tryb detekcji progu OOK.*

- enum `radio_main_option_t` {
`RADIO_MAIN_OPTION_LORA_PRESET` = 0U, `RADIO_MAIN_OPTION_LORA_FREQ`, `RADIO_MAIN_OPTION_LORA_BW`,
`RADIO_MAIN_OPTION_LORA_SF`,
`RADIO_MAIN_OPTION_LORA_CR`, `RADIO_MAIN_OPTION_LORA_TX_POWER`, `RADIO_MAIN_OPTION_LORA_CRC`,
`RADIO_MAIN_OPTION_LORA_PREAMBLE`,
`RADIO_MAIN_OPTION_LORA_HEADER_MODE`, `RADIO_MAIN_OPTION_LORA_IQ_INVERT`, `RADIO_MAIN_OPTION_LORA_RESET_DEFAULTS`,
`RADIO_MAIN_OPTION_FSK_SHAPING`, `RADIO_MAIN_OPTION_FSK_FREQ`, `RADIO_MAIN_OPTION_FSK_BITRATE`,
`RADIO_MAIN_OPTION_FSK_RX_BW`,
`RADIO_MAIN_OPTION_FSK_FILTER`, `RADIO_MAIN_OPTION_FSK_TX_POWER`, `RADIO_MAIN_OPTION_FSK_PREAMBLE`,
`RADIO_MAIN_OPTION_FSK_SYNC_LEN`,
`RADIO_MAIN_OPTION_FSK_SYNC_WORD`, `RADIO_MAIN_OPTION_FSK_ADDRESS_FILTER`,
`RADIO_MAIN_OPTION_FSK_CRC`, `RADIO_MAIN_OPTION_FSK_WHITENING`,
`RADIO_MAIN_OPTION_FSK_RESET_DEFAULTS`, `RADIO_MAIN_OPTION_OOK_FREQ`, `RADIO_MAIN_OPTION_OOK_BITRATE`,
`RADIO_MAIN_OPTION_OOK_TX_POWER`,
`RADIO_MAIN_OPTION_OOK_RX_BW`, `RADIO_MAIN_OPTION_OOK_PREAMBLE`, `RADIO_MAIN_OPTION_OOK_SYNC_LEN`,
`RADIO_MAIN_OPTION_OOK_SYNC_WORD`,
`RADIO_MAIN_OPTION_OOK_THRESHOLD_TYPE`, `RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE`,
`RADIO_MAIN_OPTION_OOK_RESET_DEFAULTS` }

Identyfikator pojedynczej opcji konfiguracyjnej.

Funkcje

- void `radio_main_load_default_lora_preset` (uint8_t preset_id, `radio_lora_cfg_t` *cfg)
Load one predefined LoRa preset into a profile object.
- void `radio_main_load_default_fsk_profile` (`radio_main_fsk_cfg_t` *cfg)
Load the default FSK/GFSK/MSK/GMSK profile.
- void `radio_main_load_default_ook_profile` (`radio_main_ook_cfg_t` *cfg)
Load the default OOK profile.
- void `radio_main_create_task` (void)
Create the radio task and command queue.
- bool `radio_main_cmd_send_template` (uint8_t group_id, uint8_t msg_id, uint32_t dst_id)
Queue transmission of a built-in message template.
- bool `radio_main_cmd_send_user_text` (const char *text, uint32_t dst_id)
Queue transmission of user-entered text.
- bool `radio_main_cmd_send_raw` (const uint8_t *data, uint8_t len)
Queue transmission of raw payload bytes.
- bool `radio_main_cmd_set_lora_preset` (uint8_t preset_id)
Select and apply a LoRa preset.
- bool `radio_main_cmd_set_modulation` (uint8_t modulation_id)
Switch active modulation family.
- bool `radio_main_cmd_set_modulation_freq` (uint32_t frequency_hz)
Set carrier frequency for the active modulation profile.
- bool `radio_main_cmd_set_modulation_bw` (uint8_t bandwidth_code)
Set bandwidth code for the active modulation profile.
- bool `radio_main_cmd_set_option` (`radio_main_option_t` option, uint32_t value)
Set one runtime radio option.
- bool `radio_main_cmd_set_fh` (bool enabled)
Enable or disable frequency hopping.
- bool `radio_main_cmd_set_fh_period` (uint32_t period_ms)
Set frequency-hopping period.
- bool `radio_main_cmd_set_coding` (bool enabled)
Enable or disable protected/coded traffic mode.
- bool `radio_main_cmd_set_auto_ping` (bool enabled)

- *Enable or disable automatic ping.*
 • bool `radio_main_cmd_set_auto_ping_period` (uint32_t period_ms)
Set automatic ping period.
- bool `radio_main_cmd_set_auto_ping_mode` (radio_main_auto_ping_mode_t mode)
Select automatic ping encoding mode.
- bool `radio_main_cmd_reset_module` (void)
Reset and reinitialize the radio module.
- bool `radio_main_cmd_start_pairing` (uint32_t timeout_ms)
Start local device pairing window.
- bool `radio_main_cmd_start_network_pairing` (uint32_t timeout_ms)
Start gateway/network pairing window.
- bool `radio_main_cmd_pairing_accept` (bool accept)
Accept or reject the current pairing candidate.
- bool `radio_main_cmd_send_pair_req` (void)
Send a device pair request using the current pairing code.
- bool `radio_main_cmd_send_network_pair_req` (void)
Send a network/gateway pair request.
- bool `radio_main_cmd_send_pair_error` (uint32_t dst_id)
Send a pairing error frame to a destination.
- bool `radio_main_get_runtime_cfg` (radio_main_runtime_cfg_t *cfg_out)
Read a snapshot of the radio runtime configuration.
- bool `radio_main_get_auto_ping_period_ms` (uint32_t *period_ms_out)
Read the effective automatic ping period.
- bool `radio_main_get_last_error_text` (char *out, uint8_t out_size)
Read the last short radio error message.
- uint32_t `radio_main_get_node_id` (void)
Return this node's application-level radio identifier.

12.77.1 Opis szczegółowy

Application radio task, runtime profile and command API.

Definicja w pliku `radio_main.h`.

12.77.2 Dokumentacja typów wyliczanych

12.77.2.1 radio_main_address_filter_t

enum `radio_main_address_filter_t`

Tryb filtrowania adresu dla trybów pakietowych FSK.

Wartości wyliczeń

RADIO_MAIN_ADDRESS_FILTER_NONE	Brak filtrowania adresowego.
RADIO_MAIN_ADDRESS_FILTER_NODE	Akceptuj tylko adres lokalny.
RADIO_MAIN_ADDRESS_FILTER_NODE_BROADCAST	Akceptuj adres lokalny i broadcast.

Definicja w linii 86 pliku `radio_main.h`.

12.77.2.2 radio_main_auto_ping_mode_tenum `radio_main_auto_ping_mode_t`

Encoding used for the periodic automatic ping feature.

Wartości wyliczeń

RADIO_MAIN_AUTO_PING_FRAME	Auto ping jako normalny <code>laviet_frame</code> .
RADIO_MAIN_AUTO_PING_RAW	Auto ping jako surowy payload ASCII <code>PING</code> .

Definicja w linii 30 pliku `radio_main.h`.**12.77.2.3 radio_main_crc_type_t**enum `radio_main_crc_type_t`

Rodzaj CRC pakietowego zapisanego w profilu aplikacyjnym.

LoRa na SX1276 wspiera tylko własny, wbudowany CRC payloadu ON/OFF. Pozostałe typy są przechowywane w profilach FSK/OOK na potrzeby menu, dokumentacji i przyszłego backendu FSK/OOK.

Wartości wyliczeń

RADIO_MAIN_CRC_OFF	CRC wyłączone.
RADIO_MAIN_CRC_SX1276	Wbudowane CRC payloadu SX1276/LoRa.
RADIO_MAIN_CRC_IBM	CRC-16 IBM/ANSI.
RADIO_MAIN_CRC_CCITT	CRC-16 CCITT.

Definicja w linii 66 pliku `radio_main.h`.**12.77.2.4 radio_main_filter_t**enum `radio_main_filter_t`

Typ filtra i współczynnik BT dla FSK/GFSK/GMSK.

Wartości wyliczeń

RADIO_MAIN_FILTER_NONE	Brak dodatkowego filtrowania gaussowskiego.
RADIO_MAIN_FILTER_BT_10	BT = 1.0.
RADIO_MAIN_FILTER_BT_07	BT = 0.7.
RADIO_MAIN_FILTER_BT_05	BT = 0.5.
RADIO_MAIN_FILTER_BT_03	BT = 0.3.

Definicja w linii 50 pliku `radio_main.h`.**12.77.2.5 radio_main_fsk_shaping_t**enum `radio_main_fsk_shaping_t`

Typ kształtowania sygnału dla rodziny FSK.

Wartości wyliczeń

RADIO_MAIN_FSK_SHAPING_FSK	Czyste FSK bez filtra gaussowskiego.
RADIO_MAIN_FSK_SHAPING_GFSK	GFSK z filtrem gaussowskim.
RADIO_MAIN_FSK_SHAPING_MSK	Minimal shift keying.
RADIO_MAIN_FSK_SHAPING_GMSK	GMSK z filtrem gaussowskim.

Definicja w linii 39 pliku [radio_main.h](#).

12.77.2.6 radio_main_header_mode_t

enum [radio_main_header_mode_t](#)

Tryb nagłówka pakietu.

Wartości wyliczeń

RADIO_MAIN_HEADER_EXPLICIT	Długość payloadu niesiona w nagłówku.
RADIO_MAIN_HEADER_IMPLICIT	Stała długość payloadu po obu stronach.

Definicja w linii 77 pliku [radio_main.h](#).

12.77.2.7 radio_main_modulation_t

enum [radio_main_modulation_t](#)

Rodziny modulacji zarządzane przez warstwę aplikacyjną.

Wartości tych identyfikatorów są używane zarówno przez `radio_main`, jak i przez menu LCD. Dzięki temu UI nie operuje na "gołych" liczbach.

Wartości wyliczeń

RADIO_MAIN_MODULATION_LORA	Profil LoRa.
RADIO_MAIN_MODULATION_FSK	Profil FSK/GFSK/MSK/GMSK.
RADIO_MAIN_MODULATION_OOK	Profil OOK.

Definicja w linii 20 pliku [radio_main.h](#).

12.77.2.8 radio_main_ook_threshold_t

enum [radio_main_ook_threshold_t](#)

Tryb detekcji progu OOK.

Wartości wyliczeń

RADIO_MAIN_OOK_THRESHOLD_FIXED	Stały próg RSSI/OOK.
RADIO_MAIN_OOK_THRESHOLD_PEAK	Próg śledzący szczyt sygnału.
RADIO_MAIN_OOK_THRESHOLD_AVERAGE	Próg wyznaczany z uśredniania.

Definicja w linii 96 pliku [radio_main.h](#).

12.77.2.9 radio_main_option_t

enum [radio_main_option_t](#)

Identyfikator pojedynczej opcji konfiguracyjnej.

Menu korzysta z tego enum-a do wysyłania jednej, spójnej komendy `radio_main_cmd_set_option(...)`, a `radio_main` wykonuje walidację i ewentualną rekonfigurację radia w jednym miejscu.

Wartości wyliczeń

RADIO_MAIN_OPTION_LORA_PRESET	Select one of the predefined LoRa presets.
RADIO_MAIN_OPTION_LORA_FREQ	Set LoRa carrier frequency.
RADIO_MAIN_OPTION_LORA_BW	Set LoRa bandwidth code.
RADIO_MAIN_OPTION_LORA_SF	Set LoRa spreading factor.
RADIO_MAIN_OPTION_LORA_CR	Set LoRa coding-rate denominator.
RADIO_MAIN_OPTION_LORA_TX_POWER	Set LoRa TX power.
RADIO_MAIN_OPTION_LORA_CRC	Enable or disable LoRa payload CRC.
RADIO_MAIN_OPTION_LORA_PREAMBLE	Set LoRa preamble length.
RADIO_MAIN_OPTION_LORA_HEADER_MODE	Select explicit or implicit LoRa header mode.
RADIO_MAIN_OPTION_LORA_IQ_INVERT	Enable or disable LoRa IQ inversion.
RADIO_MAIN_OPTION_LORA_SYNC_WORD	Set LoRa sync word.
RADIO_MAIN_OPTION_LORA_RESET_DEFAULTS	Restore default LoRa profile.
RADIO_MAIN_OPTION_FSK_SHAPING	Select FSK/GFSK/MSK/GMSK shaping.
RADIO_MAIN_OPTION_FSK_FREQ	Set FSK carrier frequency.
RADIO_MAIN_OPTION_FSK_BITRATE	Set FSK bitrate.
RADIO_MAIN_OPTION_FSK_RX_BW	Set FSK RX bandwidth code.
RADIO_MAIN_OPTION_FSK_FILTER	Select Gaussian filter BT.
RADIO_MAIN_OPTION_FSK_TX_POWER	Set FSK TX power.
RADIO_MAIN_OPTION_FSK_PREAMBLE	Set FSK preamble length.
RADIO_MAIN_OPTION_FSK_SYNC_LEN	Set FSK sync word length.
RADIO_MAIN_OPTION_FSK_SYNC_WORD	Set FSK sync word pattern.
RADIO_MAIN_OPTION_FSK_ADDRESS_FILTER	Set FSK address filtering mode.
RADIO_MAIN_OPTION_FSK_CRC	Set FSK packet CRC type.
RADIO_MAIN_OPTION_FSK_WHITENING	Enable or disable FSK data whitening.
RADIO_MAIN_OPTION_FSK_RESET_DEFAULTS	Restore default FSK profile.
RADIO_MAIN_OPTION_OOK_FREQ	Set OOK carrier frequency.
RADIO_MAIN_OPTION_OOK_BITRATE	Set OOK bitrate.
RADIO_MAIN_OPTION_OOK_TX_POWER	Set OOK TX power.
RADIO_MAIN_OPTION_OOK_RX_BW	Set OOK RX bandwidth code.
RADIO_MAIN_OPTION_OOK_PREAMBLE	Set OOK preamble length.
RADIO_MAIN_OPTION_OOK_SYNC_LEN	Set OOK sync word length.
RADIO_MAIN_OPTION_OOK_SYNC_WORD	Set OOK sync word pattern.
RADIO_MAIN_OPTION_OOK_THRESHOLD_TYPE	Select OOK threshold algorithm.
RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE	Set OOK raw threshold value.
RADIO_MAIN_OPTION_OOK_RESET_DEFAULTS	Restore default OOK profile.

Definicja w linii 110 pliku [radio_main.h](#).

12.77.3 Dokumentacja funkcji**12.77.3.1 radio_main_cmd_pairing_accept()**

```
bool radio_main_cmd_pairing_accept (
    bool accept)
```

Accept or reject the current pairing candidate.

Parametry

<i>accept</i>	true to accept, false to reject.
---------------	----------------------------------

Zwraca

true when the command was accepted.

Definicja w linii 647 pliku [radio_main.c](#).

12.77.3.2 radio_main_cmd_reset_module()

```
bool radio_main_cmd_reset_module (
    void )
```

Reset and reinitialize the radio module.

Zwraca

true when the command was accepted.

Definicja w linii 613 pliku [radio_main.c](#).

12.77.3.3 radio_main_cmd_send_network_pair_req()

```
bool radio_main_cmd_send_network_pair_req (
    void )
```

Send a network/gateway pair request.

Zwraca

true when the command was accepted.

Definicja w linii 668 pliku [radio_main.c](#).

12.77.3.4 radio_main_cmd_send_pair_error()

```
bool radio_main_cmd_send_pair_error (
    uint32_t dst_id)
```

Send a pairing error frame to a destination.

Parametry

<i>dst_id</i>	Destination node/device identifier.
---------------	-------------------------------------

Zwraca

true when the command was accepted.

Definicja w linii 674 pliku [radio_main.c](#).

12.77.3.5 radio_main_cmd_send_pair_req()

```
bool radio_main_cmd_send_pair_req (
    void )
```

Send a device pair request using the current pairing code.

Zwraca

true when the command was accepted.

Definicja w linii 658 pliku [radio_main.c](#).

12.77.3.6 radio_main_cmd_send_raw()

```
bool radio_main_cmd_send_raw (
    const uint8_t * data,
    uint8_t len)
```

Queue transmission of raw payload bytes.

Parametry

<i>data</i>	Payload bytes.
<i>len</i>	Payload length.

Zwraca

`true` when the command was accepted.

Definicja w linii 468 pliku [radio_main.c](#).

12.77.3.7 radio_main_cmd_send_template()

```
bool radio_main_cmd_send_template (
    uint8_t group_id,
    uint8_t msg_id,
    uint32_t dst_id)
```

Queue transmission of a built-in message template.

Parametry

<i>group↔ _id</i>	Template group identifier.
<i>msg_id</i>	Template message identifier.
<i>dst_id</i>	Destination node/device identifier.

Zwraca

`true` when the command was accepted.

Definicja w linii 428 pliku [radio_main.c](#).

12.77.3.8 radio_main_cmd_send_user_text()

```
bool radio_main_cmd_send_user_text (
    const char * text,
    uint32_t dst_id)
```

Queue transmission of user-entered text.

Parametry

<i>text</i>	Null-terminated text payload.
<i>dst↔ _id</i>	Destination node/device identifier.

Zwraca

`true` when the command was accepted.

Definicja w linii 442 pliku [radio_main.c](#).

Odwołuje się do [LAVIET_MAX_PAYLOAD](#).

12.77.3.9 radio_main_cmd_set_auto_ping()

```
bool radio_main_cmd_set_auto_ping (  
    bool enabled)
```

Enable or disable automatic ping.

Parametry

<i>enabled</i>	Desired auto-ping state.
----------------	--------------------------

Zwraca

`true` when the command was accepted.

Definicja w linii 580 pliku [radio_main.c](#).

12.77.3.10 radio_main_cmd_set_auto_ping_mode()

```
bool radio_main_cmd_set_auto_ping_mode (  
    radio\_main\_auto\_ping\_mode\_t mode)
```

Select automatic ping encoding mode.

Parametry

<i>mode</i>	Auto-ping encoding mode.
-------------	--------------------------

Zwraca

`true` when the command was accepted.

Definicja w linii 602 pliku [radio_main.c](#).

12.77.3.11 radio_main_cmd_set_auto_ping_period()

```
bool radio_main_cmd_set_auto_ping_period (  
    uint32_t period_ms)
```

Set automatic ping period.

Parametry

<i>period_ms</i>	Period in milliseconds.
------------------	-------------------------

Zwraca

`true` when the command was accepted.

Definicja w linii 591 pliku [radio_main.c](#).

12.77.3.12 radio_main_cmd_set_coding()

```
bool radio_main_cmd_set_coding (  
    bool enabled)
```

Enable or disable protected/coded traffic mode.

Parametry

<i>enabled</i>	Desired coding state.
----------------	-----------------------

Zwraca

`true` when the command was accepted.

Definicja w linii 569 pliku [radio_main.c](#).

12.77.3.13 radio_main_cmd_set_fh()

```
bool radio_main_cmd_set_fh (  
    bool enabled)
```

Enable or disable frequency hopping.

Parametry

<i>enabled</i>	Desired FH state.
----------------	-------------------

Zwraca

`true` when the command was accepted.

Definicja w linii 547 pliku [radio_main.c](#).

12.77.3.14 radio_main_cmd_set_fh_period()

```
bool radio_main_cmd_set_fh_period (  
    uint32_t period_ms)
```

Set frequency-hopping period.

Parametry

<i>period_ms</i>	Hop period in milliseconds.
------------------	-----------------------------

Zwraca

`true` when the command was accepted.

Definicja w linii 558 pliku [radio_main.c](#).

12.77.3.15 radio_main_cmd_set_lora_preset()

```
bool radio_main_cmd_set_lora_preset (  
    uint8_t preset_id)
```

Select and apply a LoRa preset.

Parametry

<i>preset↔ _id</i>	Preset index.
------------------------	---------------

Zwraca

`true` when the command was accepted and applied.

Definicja w linii 486 pliku [radio_main.c](#).

12.77.3.16 radio_main_cmd_set_modulation()

```
bool radio_main_cmd_set_modulation (  
    uint8_t modulation_id)
```

Switch active modulation family.

Parametry

<i>modulation↔ _id</i>	One of radio_main_modulation_t .
----------------------------	--

Zwraca

`true` when the command was accepted and applied.

Definicja w linii 498 pliku [radio_main.c](#).

12.77.3.17 radio_main_cmd_set_modulation_bw()

```
bool radio_main_cmd_set_modulation_bw (  
    uint8_t bandwidth_code)
```

Set bandwidth code for the active modulation profile.

Parametry

<i>bandwidth_code</i>	Bandwidth enum/code value.
-----------------------	----------------------------

Zwraca

`true` when the command was accepted and applied.

Definicja w linii 522 pliku [radio_main.c](#).

12.77.3.18 radio_main_cmd_set_modulation_freq()

```
bool radio_main_cmd_set_modulation_freq (  
    uint32_t frequency_hz)
```

Set carrier frequency for the active modulation profile.

Parametry

<i>frequency_hz</i>	Frequency in Hz.
---------------------	------------------

Zwraca

`true` when the command was accepted and applied.

Definicja w linii 510 pliku [radio_main.c](#).

12.77.3.19 radio_main_cmd_set_option()

```
bool radio_main_cmd_set_option (
    radio_main_option_t option,
    uint32_t value)
```

Set one runtime radio option.

Parametry

<i>option</i>	Option identifier.
<i>value</i>	Raw option value.

Zwraca

`true` when the command was accepted and applied.

Definicja w linii 534 pliku [radio_main.c](#).

12.77.3.20 radio_main_cmd_start_network_pairing()

```
bool radio_main_cmd_start_network_pairing (
    uint32_t timeout_ms)
```

Start gateway/network pairing window.

Parametry

<i>timeout_ms</i>	Pairing timeout in milliseconds.
-------------------	----------------------------------

Zwraca

`true` when the command was accepted.

Definicja w linii 635 pliku [radio_main.c](#).

12.77.3.21 radio_main_cmd_start_pairing()

```
bool radio_main_cmd_start_pairing (
    uint32_t timeout_ms)
```

Start local device pairing window.

Parametry

<i>timeout_ms</i>	Pairing timeout in milliseconds.
-------------------	----------------------------------

Zwraca

`true` when the command was accepted.

Definicja w linii 623 pliku [radio_main.c](#).

12.77.3.22 radio_main_create_task()

```
void radio_main_create_task (
    void )
```

Create the radio task and command queue.

Definicja w linii 396 pliku [radio_main.c](#).

Odwołania w [app_freertos_init\(\)](#).

12.77.3.23 `radio_main_get_auto_ping_period_ms()`

```
bool radio_main_get_auto_ping_period_ms (
    uint32_t * period_ms_out)
```

Read the effective automatic ping period.

Parametry

<i>period_ms_out</i>	[out] Period in milliseconds.
----------------------	-------------------------------

Zwraca

`true` when the value was copied.

Definicja w linii 704 pliku [radio_main.c](#).

12.77.3.24 `radio_main_get_last_error_text()`

```
bool radio_main_get_last_error_text (
    char * out,
    uint8_t out_size)
```

Read the last short radio error message.

Parametry

<i>out</i>	Destination text buffer.
<i>out_size</i>	Destination buffer size.

Zwraca

`true` when an error string was copied.

Definicja w linii 724 pliku [radio_main.c](#).

12.77.3.25 `radio_main_get_node_id()`

```
uint32_t radio_main_get_node_id (
    void )
```

Return this node's application-level radio identifier.

Zwraca

Node identifier.

Definicja w linii 744 pliku [radio_main.c](#).

12.77.3.26 `radio_main_get_runtime_cfg()`

```
bool radio_main_get_runtime_cfg (
    radio\_main\_runtime\_cfg\_t * cfg_out)
```

Read a snapshot of the radio runtime configuration.

Parametry

<i>cfg_out</i>	[out] Destination configuration snapshot.
----------------	---

Zwraca

`true` when a snapshot was copied.

Definicja w linii 685 pliku [radio_main.c](#).

12.77.3.27 radio_main_load_default_fsk_profile()

```
void radio_main_load_default_fsk_profile (
    radio_main_fsk_cfg_t * cfg)
```

Load the default FSK/GFSK/MSK/GMSK profile.

Parametry

<i>cfg</i>	[out] Destination FSK profile.
------------	--------------------------------

Definicja w linii 1827 pliku [radio_main.c](#).

Odwołuje się do [radio_main_fsk_cfg_t::address_filter](#), [radio_main_fsk_cfg_t::bitrate_bps](#), [radio_main_fsk_cfg_t::crc_type](#), [radio_main_fsk_cfg_t::data_whitening](#), [radio_main_fsk_cfg_t::filter](#), [radio_main_fsk_cfg_t::frequency_hz](#), [radio_main_fsk_cfg_t::preamble_len](#), [RADIO_MAIN_ADDRESS_FILTER_NONE](#), [RADIO_MAIN_CRC_CCITT](#), [RADIO_MAIN_FILTER_BT_05](#), [RADIO_MAIN_FSK_SHAPING](#), [radio_main_fsk_cfg_t::rx_bandwidth](#), [radio_main_fsk_cfg_t::shaping](#), [radio_main_fsk_cfg_t::sync_word](#), [radio_main_fsk_cfg_t::sync_word_len](#) i [radio_main_fsk_cfg_t::tx_power_dbm](#).

12.77.3.28 radio_main_load_default_lora_preset()

```
void radio_main_load_default_lora_preset (
    uint8_t preset_id,
    radio_lora_cfg_t * cfg)
```

Load one predefined LoRa preset into a profile object.

Parametry

<i>presets_id</i>	Preset index.
<i>cfg</i>	[out] Destination LoRa profile.

Definicja w linii 1752 pliku [radio_main.c](#).

Odwołuje się do [radio_lora_cfg_t::bandwidth](#), [radio_lora_cfg_t::coding_rate](#), [radio_lora_cfg_t::crc_on](#), [radio_lora_cfg_t::frequency_hz](#), [radio_lora_cfg_t::implicit_header](#), [radio_lora_cfg_t::invert_iq](#), [radio_lora_cfg_t::payload_len](#), [radio_lora_cfg_t::preamble_len](#), [radio_lora_cfg_t::spreading_factor](#), [radio_lora_cfg_t::sync_word](#) i [radio_lora_cfg_t::tx_power_dbm](#).

12.77.3.29 radio_main_load_default_ook_profile()

```
void radio_main_load_default_ook_profile (
    radio_main_ook_cfg_t * cfg)
```

Load the default OOK profile.

Parametry

<i>cfg</i>	[out] Destination OOK profile.
------------	--------------------------------

Definicja w linii 1849 pliku [radio_main.c](#).

Odwołuje się do [radio_main_ook_cfg_t::bitrate_bps](#), [radio_main_ook_cfg_t::frequency_hz](#), [radio_main_ook_cfg_t::preamble_len](#), [RADIO_MAIN_OOK_THRESHOLD_PEAK](#), [radio_main_ook_cfg_t::rx_bandwidth](#), [radio_main_ook_cfg_t::sync_word](#), [radio_main_ook_cfg_t::sync_word_len](#), [radio_main_ook_cfg_t::threshold](#), [radio_main_ook_cfg_t::threshold_value](#) i [radio_main_ook_cfg_t::tx_power_dbm](#).

12.78 radio_main.h

[Idź do dokumentacji tego pliku.](#)

```

00001
00005
00006 #ifndef APP_RADIO_MAIN_H_
00007 #define APP_RADIO_MAIN_H_
00008
00009 #include "radio_lib/radio_lib.h"
00010
00011 #include <stdbool.h>
00012 #include <stdint.h>
00013
00020 typedef enum
00021 {
00022     RADIO_MAIN_MODULATION_LORA = 0U,
00023     RADIO_MAIN_MODULATION_FSK = 1U,
00024     RADIO_MAIN_MODULATION_OOK = 2U
00025 } radio_main_modulation_t;
00026
00030 typedef enum
00031 {
00032     RADIO_MAIN_AUTO_PING_FRAME = 0U,
00033     RADIO_MAIN_AUTO_PING_RAW = 1U
00034 } radio_main_auto_ping_mode_t;
00035
00039 typedef enum
00040 {
00041     RADIO_MAIN_FSK_SHAPING_FSK = 0U,
00042     RADIO_MAIN_FSK_SHAPING_GFSK = 1U,
00043     RADIO_MAIN_FSK_SHAPING_MSK = 2U,
00044     RADIO_MAIN_FSK_SHAPING_GMSK = 3U
00045 } radio_main_fsk_shaping_t;
00046
00050 typedef enum
00051 {
00052     RADIO_MAIN_FILTER_NONE = 0U,
00053     RADIO_MAIN_FILTER_BT_10 = 1U,
00054     RADIO_MAIN_FILTER_BT_07 = 2U,
00055     RADIO_MAIN_FILTER_BT_05 = 3U,
00056     RADIO_MAIN_FILTER_BT_03 = 4U
00057 } radio_main_filter_t;
00058
00066 typedef enum
00067 {
00068     RADIO_MAIN_CRC_OFF = 0U,
00069     RADIO_MAIN_CRC_SX1276 = 1U,
00070     RADIO_MAIN_CRC_IBM = 2U,
00071     RADIO_MAIN_CRC_CCITT = 3U
00072 } radio_main_crc_type_t;
00073
00077 typedef enum
00078 {
00079     RADIO_MAIN_HEADER_EXPLICIT = 0U,
00080     RADIO_MAIN_HEADER_IMPLICIT = 1U
00081 } radio_main_header_mode_t;
00082
00086 typedef enum
00087 {
00088     RADIO_MAIN_ADDRESS_FILTER_NONE = 0U,
00089     RADIO_MAIN_ADDRESS_FILTER_NODE = 1U,
00090     RADIO_MAIN_ADDRESS_FILTER_NODE_BROADCAST = 2U
00091 } radio_main_address_filter_t;
00092
00096 typedef enum
00097 {
00098     RADIO_MAIN_OOK_THRESHOLD_FIXED = 0U,
00099     RADIO_MAIN_OOK_THRESHOLD_PEAK = 1U,
00100     RADIO_MAIN_OOK_THRESHOLD_AVERAGE = 2U
00101 } radio_main_ook_threshold_t;
00102
00110 typedef enum
00111 {
00112     RADIO_MAIN_OPTION_LORA_PRESET = 0U,
00113     RADIO_MAIN_OPTION_LORA_FREQ,
00114     RADIO_MAIN_OPTION_LORA_BW,
00115     RADIO_MAIN_OPTION_LORA_SF,
00116     RADIO_MAIN_OPTION_LORA_CR,
00117     RADIO_MAIN_OPTION_LORA_TX_POWER,
00118     RADIO_MAIN_OPTION_LORA_CRC,
00119     RADIO_MAIN_OPTION_LORA_PREAMBLE,
00120     RADIO_MAIN_OPTION_LORA_HEADER_MODE,
00121     RADIO_MAIN_OPTION_LORA_IO_INVERT,
00122     RADIO_MAIN_OPTION_LORA_SYNC_WORD,
00123     RADIO_MAIN_OPTION_LORA_RESET_DEFAULTS,
00124     RADIO_MAIN_OPTION_FSK_SHAPING,

```



```
00125     RADIO_MAIN_OPTION_FSK_FREQ,
00126     RADIO_MAIN_OPTION_FSK_BITRATE,
00127     RADIO_MAIN_OPTION_FSK_RX_BW,
00128     RADIO_MAIN_OPTION_FSK_FILTER,
00129     RADIO_MAIN_OPTION_FSK_TX_POWER,
00130     RADIO_MAIN_OPTION_FSK_PREAMBLE,
00131     RADIO_MAIN_OPTION_FSK_SYNC_LEN,
00132     RADIO_MAIN_OPTION_FSK_SYNC_WORD,
00133     RADIO_MAIN_OPTION_FSK_ADDRESS_FILTER,
00134     RADIO_MAIN_OPTION_FSK_CRC,
00135     RADIO_MAIN_OPTION_FSK_WHITENING,
00136     RADIO_MAIN_OPTION_FSK_RESET_DEFAULTS,
00137     RADIO_MAIN_OPTION_OOK_FREQ,
00138     RADIO_MAIN_OPTION_OOK_BITRATE,
00139     RADIO_MAIN_OPTION_OOK_TX_POWER,
00140     RADIO_MAIN_OPTION_OOK_RX_BW,
00141     RADIO_MAIN_OPTION_OOK_PREAMBLE,
00142     RADIO_MAIN_OPTION_OOK_SYNC_LEN,
00143     RADIO_MAIN_OPTION_OOK_SYNC_WORD,
00144     RADIO_MAIN_OPTION_OOK_THRESHOLD_TYPE,
00145     RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE,
00146     RADIO_MAIN_OPTION_OOK_RESET_DEFAULTS
00147 } radio_main_option_t;
00148
00152 typedef struct
00153 {
00154     radio_main_fsk_shaping_t shaping;
00155     uint32_t frequency_hz;
00156     uint32_t bitrate_bps;
00157     radio_lora_bw_t rx_bandwidth;
00158     radio_main_filter_t filter;
00159     int8_t tx_power_dbm;
00160     uint16_t preamble_len;
00161     uint8_t sync_word_len;
00162     uint64_t sync_word;
00163     radio_main_address_filter_t address_filter;
00164     radio_main_crc_type_t crc_type;
00165     bool data_whitening;
00166 } radio_main_fsk_cfg_t;
00167
00171 typedef struct
00172 {
00173     uint32_t frequency_hz;
00174     uint32_t bitrate_bps;
00175     radio_lora_bw_t rx_bandwidth;
00176     int8_t tx_power_dbm;
00177     uint16_t preamble_len;
00178     uint8_t sync_word_len;
00179     uint32_t sync_word;
00180     radio_main_ook_threshold_t threshold;
00181     uint8_t threshold_value;
00182 } radio_main_ook_cfg_t;
00183
00187 typedef struct
00188 {
00189     radio_main_modulation_t active_modulation;
00190     radio_lora_cfg_t lora;
00191     radio_main_fsk_cfg_t fsk;
00192     radio_main_ook_cfg_t ook;
00193     bool fh_enabled;
00194     uint32_t fh_period_ms;
00195     bool coding_enabled;
00196     bool auto_ping_enabled;
00197     uint32_t auto_ping_period_ms;
00198     radio_main_auto_ping_mode_t auto_ping_mode;
00199 } radio_main_runtime_cfg_t;
00200
00206 void radio_main_load_default_lora_preset(uint8_t preset_id, radio_lora_cfg_t *cfg);
00207
00212 void radio_main_load_default_fsk_profile(radio_main_fsk_cfg_t *cfg);
00213
00218 void radio_main_load_default_ook_profile(radio_main_ook_cfg_t *cfg);
00219
00223 void radio_main_create_task(void);
00224
00232 bool radio_main_cmd_send_template(uint8_t group_id, uint8_t msg_id, uint32_t dst_id);
00233
00240 bool radio_main_cmd_send_user_text(const char *text, uint32_t dst_id);
00241
00248 bool radio_main_cmd_send_raw(const uint8_t *data, uint8_t len);
00249
00255 bool radio_main_cmd_set_lora_preset(uint8_t preset_id);
00256
00262 bool radio_main_cmd_set_modulation(uint8_t modulation_id);
00263
00269 bool radio_main_cmd_set_modulation_freq(uint32_t frequency_hz);
00270
```

```

00276 bool radio_main_cmd_set_modulation_bw(uint8_t bandwidth_code);
00277
00284 bool radio_main_cmd_set_option(radio_main_option_t option, uint32_t value);
00285
00291 bool radio_main_cmd_set_fh(bool enabled);
00292
00298 bool radio_main_cmd_set_fh_period(uint32_t period_ms);
00299
00305 bool radio_main_cmd_set_coding(bool enabled);
00306
00312 bool radio_main_cmd_set_auto_ping(bool enabled);
00313
00319 bool radio_main_cmd_set_auto_ping_period(uint32_t period_ms);
00320
00326 bool radio_main_cmd_set_auto_ping_mode(radio_main_auto_ping_mode_t mode);
00327
00332 bool radio_main_cmd_reset_module(void);
00333
00339 bool radio_main_cmd_start_pairing(uint32_t timeout_ms);
00340
00346 bool radio_main_cmd_start_network_pairing(uint32_t timeout_ms);
00347
00353 bool radio_main_cmd_pairing_accept(bool accept);
00354
00359 bool radio_main_cmd_send_pair_req(void);
00360
00365 bool radio_main_cmd_send_network_pair_req(void);
00366
00372 bool radio_main_cmd_send_pair_error(uint32_t dst_id);
00373
00379 bool radio_main_get_runtime_cfg(radio_main_runtime_cfg_t *cfg_out);
00380
00386 bool radio_main_get_auto_ping_period_ms(uint32_t *period_ms_out);
00387
00394 bool radio_main_get_last_error_text(char *out, uint8_t out_size);
00395
00400 uint32_t radio_main_get_node_id(void);
00401
00402 #endif /* APP_RADIO_MAIN_H_ */

```

12.79 security_main.c

```

00001 #include "security_main.h"
00002
00003 #include "app_delay.h"
00004 #include "cmsis_os2.h"
00005 #include "FreeRTOS.h"
00006 #include "i2c_mem_store_lib/i2c_mem_store.h"
00007 #include "laviet_crypto.h"
00008 #include "laviet_frame.h"
00009 #include "main.h"
00010 #include "st33ktpm2x_lib/st33ktpm2x.h"
00011 #include "task.h"
00012
00013 #include <stdio.h>
00014 #include <string.h>
00015
00016 #define SECURITY_TASK_STACK_SIZE 15360U
00017 #define SECURITY_TASK_STACK_WORDS (SECURITY_TASK_STACK_SIZE / sizeof(StackType_t))
00018 #define SECURITY_CMD_QUEUE_DEPTH 16U
00019 #define SECURITY_CMD_WAIT_MS 3000U
00020 #define SECURITY_CMD_POLL_MS 5U
00021 #define SECURITY_TRUSTED_MAX 16U
00022 #define SECURITY_TRUSTED_SLOT_BASE 0U
00023 #define SECURITY_GATEWAY_COUNTER_SLOT 2U
00024 #define SECURITY_GATEWAY_COUNTER_MAGIC 0xC7U
00025 #define SECURITY_GATEWAY_COUNTER_VERSION 1U
00026 #define SECURITY_GATEWAY_COUNTER_STORE_LEN 10U
00027 #define SECURITY_STORE_MAGIC 0xA5U
00028 #define SECURITY_STORE_VERSION 4U
00029 #define SECURITY_STORE_FLAG_CODING 0x01U
00030 #define SECURITY_STORE_FLAG_FH 0x02U
00031 #define SECURITY_STORE_FLAG_AUTO_PING 0x04U
00032 #define SECURITY_STORE_AUTOPING_SHIFT 3U
00033 #define SECURITY_STORE_AUTOPING_MASK 0x38U
00034 #define SECURITY_STORE_FLAG_AUTOPING_RAW 0x40U
00035 #define SECURITY_AUTOPING_DEFAULT_MS 1000UL
00036 #define SECURITY_LEGACY_SETTINGS_SLOT 0U
00037 #define SECURITY_LEGACY_KEY_SEED_SLOT 1U
00038 #define SECURITY_SETTINGS_MAGIC 0x5345U
00039 #define SECURITY_SETTINGS_VERSION 1U
00040 #define SECURITY_KEY_MAGIC 0x4B59U
00041 #define SECURITY_KEY_SEED_BYTES 8U
00042 #define SECURITY_CODE_MAX I2C_MEM_STORE_TRUSTED_CODE_MAX

```

```

00043 #define SECURITY_TRUSTED_ID_LEN          4U
00044 #define SECURITY_TRUSTED_STORE_RETRIES     3U
00045 #define SECURITY_TRUSTED_STORE_RETRY_MS    10U
00046 #define SECURITY_TPM_ROOT_NV_INDEX         0x01C10101UL
00047 #define SECURITY_TPM_ROOT_LEGACY_NV_INDEX  0x01C10100UL
00048 #define SECURITY_TPM_NV_PPWRITE            0x00000001UL
00049 #define SECURITY_TPM_NV_OWNERWRITE         0x00000002UL
00050 #define SECURITY_TPM_NV_AUTHWRITE          0x00000004UL
00051 #define SECURITY_TPM_NV_OWNERREAD          0x00020000UL
00052 #define SECURITY_TPM_NV_AUTHREAD           0x00040000UL
00053 #define SECURITY_TPM_NV_NO_DA              0x02000000UL
00054 #define SECURITY_TPM_ROOT_NV_ATTRS         (SECURITY_TPM_NV_PPWRITE | \
00055                                             SECURITY_TPM_NV_OWNERREAD | \
00056                                             SECURITY_TPM_NV_NO_DA)
00057 #define SECURITY_TPM_REG_LOC_SEL            0x00U
00058 #define SECURITY_TPM_REG_ACCESS             0x04U
00059 #define SECURITY_TPM_REG_STS                0x18U
00060 #define SECURITY_TPM_REG_DATA_FIFO          0x24U
00061 #define SECURITY_TPM_REG_IF_CAP             0x30U
00062 #define SECURITY_TPM_REG_DID_VID            0x48U
00063 #define SECURITY_TPM_REG_RID                0x4CU
00064 #define SECURITY_TPM_DIAG_TIMEOUT_MS        20U
00065
00066 #if defined(TPM_INIT_LOG)
00067 #define SECURITY_TPM_INIT_LOG(...)          printf(__VA_ARGS__)
00068 #else
00069 #define SECURITY_TPM_INIT_LOG(...)          do { if (0) { printf(__VA_ARGS__); } } while (0)
00070 #endif
00071
00072 typedef enum
00073 {
00074     SECURITY_CMD_NONE = 0,
00075     SECURITY_CMD_ADD_DEVICE,
00076     SECURITY_CMD_DELETE_DEVICE,
00077     SECURITY_CMD_GET_DEVICE,
00078     SECURITY_CMD_SET_CODING,
00079     SECURITY_CMD_SET_FH,
00080     SECURITY_CMD_SET_FH_PERIOD,
00081     SECURITY_CMD_ROTATE_KEY,
00082     SECURITY_CMD_SET_NOTIFY,
00083     SECURITY_CMD_SET_PRESET,
00084     SECURITY_CMD_SET_AUTO_PING,
00085     SECURITY_CMD_SET_AUTO_PING_PERIOD,
00086     SECURITY_CMD_SET_AUTO_PING_MODE,
00087     SECURITY_CMD_SET_RADIO_RUNTIME,
00088     SECURITY_CMD_GET_RUNTIME,
00089     SECURITY_CMD_LOG_MESSAGE
00090 } security_cmd_id_t;
00091
00092 typedef struct
00093 {
00094     volatile bool done;
00095     bool result;
00096     trusted_info_t trusted_out;
00097     security_runtime_cfg_t runtime_out;
00098 } security_cmd_sync_t;
00099
00100 typedef struct
00101 {
00102     security_cmd_id_t id;
00103     security_cmd_sync_t *sync;
00104     union
00105     {
00106         struct
00107         {
00108             uint32_t node_id;
00109             uint8_t code_len;
00110             uint8_t code[SECURITY_CODE_MAX];
00111             bool gateway_slot;
00112         } add_device;
00113         struct
00114         {
00115             uint32_t node_id;
00116         } del_device;
00117         struct
00118         {
00119             uint8_t index;
00120         } get_device;
00121         struct
00122         {
00123             bool enabled;
00124         } set_bool;
00125         struct
00126         {
00127             uint32_t value;
00128         } set_u32;
00129     }

```

```

00130     {
00131         security_notify_mode_t mode;
00132     } set_notify;
00133     struct
00134     {
00135         uint8_t preset_id;
00136     } set_preset;
00137     struct
00138     {
00139         radio_main_runtime_cfg_t cfg;
00140     } set_radio;
00141     struct
00142     {
00143         int16_t rssi_dbm;
00144         uint8_t len;
00145         uint8_t payload[I2C_MEM_STORE_LOG_PAYLOAD_MAX];
00146     } log_msg;
00147     } u;
00148 } security_cmd_t;
00149
00150 typedef struct
00151 {
00152     bool in_use;
00153     bool is_master;
00154     uint32_t node_id;
00155     uint8_t code_len;
00156     uint8_t code[SECURITY_CODE_MAX];
00157 } security_trusted_entry_t;
00158
00159 typedef struct
00160 {
00161     uint8_t magic;
00162     uint8_t version;
00163     uint8_t flags;
00164     uint8_t notify_mode;
00165     uint8_t lora_preset;
00166     uint8_t modulation_fh;
00167     uint8_t seed[SECURITY_KEY_SEED_BYTES];
00168 } security_store_wire_t;
00169
00170 typedef struct
00171 {
00172     uint8_t magic;
00173     uint8_t version;
00174     uint8_t packed[11];
00175 } security_radio_store_wire_t;
00176
00177 typedef struct
00178 {
00179     uint16_t magic;
00180     uint8_t version;
00181     uint8_t flags;
00182     uint8_t notify_mode;
00183     uint8_t lora_preset;
00184     uint8_t reserved0;
00185     uint8_t reserved1;
00186     uint16_t crc;
00187 } security_settings_wire_t;
00188
00189 typedef struct
00190 {
00191     uint16_t magic;
00192     uint8_t seed[SECURITY_KEY_SEED_BYTES];
00193     uint16_t crc;
00194 } security_key_wire_t;
00195
00196 static osThreadId_t s_security_task = NULL;
00197 static osMessageQueueId_t s_security_cmd_queue = NULL;
00198 static osMutexId_t s_security_mutex = NULL;
00199 static StaticTask_t s_security_task_cb;
00200 static StackType_t s_security_task_stack[SECURITY_TASK_STACK_WORDS];
00201
00202 static bool s_security_initialized = false;
00203 static bool s_tpm_ready = false;
00204 static i2c_mem_store_t s_mem_store;
00205 static bool s_mem_ready = false;
00206 static bool s_runtime_shadow_valid = false;
00207 static security_store_wire_t s_runtime_shadow_store;
00208 static security_radio_store_wire_t s_runtime_shadow_radio;
00209 static st33ktpm2x_t s_tpm;
00210 static security_trusted_entry_t s_trusted[SECURITY_TRUSTED_MAX];
00211 static security_runtime_cfg_t s_runtime_cfg =
00212 {
00213     .coding_enabled = true,
00214     .fh_enabled = false,
00215     .fh_period_ms = 2000UL,
00216     .auto_ping_enabled = false,

```

```

00217     .auto_ping_period_ms = SECURITY_AUTOPING_DEFAULT_MS,
00218     .auto_ping_mode = RADIO_MAIN_AUTO_PING_FRAME,
00219     .notify_mode = SECURITY_NOTIFY_POPUP,
00220     .loras_preset = 2U,
00221     .active_modulation = RADIO_MAIN_MODULATION_LORA,
00222     .radio_profiles_persisted = false
00223 };
00224 static uint8_t s_network_key[16];
00225 static bool s_key_seed_tpm_backed = false;
00226 static bool s_key_seed_tpm_pp_backed = false;
00227 static const uint8_t s_shared_frame_root_key[16] =
00228 {
00229     (uint8_t)'L', (uint8_t)'A', (uint8_t)'V', (uint8_t)'I',
00230     (uint8_t)'E', (uint8_t)'T', (uint8_t)'_', (uint8_t)'S',
00231     (uint8_t)'H', (uint8_t)'A', (uint8_t)'R', (uint8_t)'E',
00232     (uint8_t)'D', (uint8_t)'_', (uint8_t)'V', (uint8_t)'I'
00233 };
00234 static uint8_t s_key_seed_cached[SECURITY_KEY_SEED_BYTES];
00235 static uint32_t s_gateway_rx_counter = 0UL;
00236 static uint32_t s_gateway_tx_counter = 0UL;
00237
00238 extern I2C_HandleTypeDef hi2c1;
00239 extern I2C_HandleTypeDef hi2c3;
00240 extern RNG_HandleTypeDef hrng;
00241
00242 static const uint32_t s_freq_options_hz[] =
00243 {
00244     868000000UL, 868100000UL, 868300000UL, 868500000UL, 868800000UL, 869050000UL, 869525000UL
00245 };
00246 static const uint32_t s_bitrate_options_bps[] =
00247 {
00248     1200UL, 2400UL, 4800UL, 9600UL, 19200UL, 38400UL, 50000UL, 100000UL
00249 };
00250 static const uint16_t s_preamble_options[] =
00251 {
00252     4U, 6U, 8U, 12U, 16U, 24U, 32U
00253 };
00254 static const int8_t s_power_options_dbm[] =
00255 {
00256     2, 5, 8, 11, 14, 17, 20
00257 };
00258 static const uint32_t s_sync_word_options[] =
00259 {
00260     0x12UL, 0x34UL, 0x56UL, 0xA5UL, 0x55AAUL, 0x2DD4UL, 0xA55AUL, 0x00C194C1UL, 0x1ACFFC1DUL
00261 };
00262 static const uint8_t s_sync_len_options[] =
00263 {
00264     0U, 1U, 2U, 3U, 4U
00265 };
00266 static const uint8_t s_ook_threshold_options[] =
00267 {
00268     4U, 8U, 12U, 16U, 24U, 32U, 48U, 64U
00269 };
00270 static const uint32_t s_fh_period_options_ms[] =
00271 {
00272     1000UL, 2000UL, 5000UL, 10000UL
00273 };
00274 static const uint32_t s_auto_ping_period_options_ms[] =
00275 {
00276     1UL, 10UL, 100UL, 1000UL, 10000UL
00277 };
00278
00279 static void security_main_task_fn(void *argument);
00280 static void security_log_stack_high_water(const char *tag);
00281 static bool security_main_wait_sync(security_cmd_sync_t *sync, uint32_t timeout_ms);
00282 static bool security_main_enqueue_sync(const security_cmd_t *cmd, security_cmd_sync_t *sync);
00283
00284 static void security_key_seed_to_key(const uint8_t seed[SECURITY_KEY_SEED_BYTES], uint8_t
key_out[16]);
00285 static void security_key_seed_to_store_key(const uint8_t seed[SECURITY_KEY_SEED_BYTES], uint8_t
key_out[16]);
00286 static bool security_seed_has_data(const uint8_t seed[SECURITY_KEY_SEED_BYTES]);
00287 static void security_peer_link_key_derive(uint32_t local_node_id,
uint32_t peer_node_id,
const uint8_t *code,
uint8_t code_len,
uint8_t key_out[16]);
00292 static bool security_load_runtime_and_seed_from_store(bool *seed_loaded_out);
00293 static bool security_save_runtime_and_seed_to_store(void);
00294 static bool security_commit_runtime_cfg_soft(void);
00295 static void security_load_default_radio_profiles(security_runtime_cfg_t *cfg);
00296 static uint8_t security_pack_bits(uint8_t *buf, uint8_t bit_pos, uint32_t value, uint8_t width);
00297 static uint8_t security_unpack_bits(const uint8_t *buf, uint8_t bit_pos, uint8_t width, uint32_t
*value_out);
00298 static uint8_t security_index_from_u32(uint32_t value, const uint32_t *table, uint8_t count, uint8_t
fallback);
00299 static bool security_auto_ping_period_is_valid(uint32_t period_ms);

```

```

00300 static bool security_auto_ping_mode_is_valid(radio_main_auto_ping_mode_t mode);
00301 static uint8_t security_index_from_u16(uint16_t value, const uint16_t *table, uint8_t count, uint8_t
    fallback);
00302 static uint8_t security_index_from_i8(int8_t value, const int8_t *table, uint8_t count, uint8_t
    fallback);
00303 static uint8_t security_index_from_u8(uint8_t value, const uint8_t *table, uint8_t count, uint8_t
    fallback);
00304 static uint8_t security_pack_modulation_fh(radio_main_modulation_t modulation, uint8_t fh_period_idx);
00305 static radio_main_modulation_t security_unpack_modulation(uint8_t modulation_fh);
00306 static uint8_t security_unpack_fh_period_idx(uint8_t modulation_fh);
00307 static uint32_t security_u32_from_index(uint8_t idx, const uint32_t *table, uint8_t count, uint32_t
    fallback);
00308 static uint16_t security_u16_from_index(uint8_t idx, const uint16_t *table, uint8_t count, uint16_t
    fallback);
00309 static int8_t security_i8_from_index(uint8_t idx, const int8_t *table, uint8_t count, int8_t
    fallback);
00310 static uint8_t security_u8_from_index(uint8_t idx, const uint8_t *table, uint8_t count, uint8_t
    fallback);
00311 static bool security_save_radio_profiles_to_store(void);
00312 static bool security_load_radio_profiles_from_store(void);
00313 static void security_migrate_trusted_slot_v1_to_v2(void);
00314 static bool security_load_settings_legacy_from_store(void);
00315 static bool security_load_key_seed_legacy_from_store(uint8_t seed[SECURITY_KEY_SEED_BYTES]);
00316 static bool security_rotate_key_internal(void);
00317 static bool security_add_device_internal(uint32_t node_id, const uint8_t *code, uint8_t len, bool
    gateway_slot);
00318 static bool security_delete_device_internal(uint32_t node_id);
00319 static bool security_get_device_internal(uint8_t idx, trusted_info_t *out);
00320 static bool security_lookup_peer_code(uint32_t peer_node_id,
    uint8_t code_out[SECURITY_CODE_MAX],
    uint8_t *code_len_out);
00321 static uint8_t security_trusted_store_capacity(void);
00322 static uint16_t security_trusted_store_slot(uint8_t idx);
00323 static void security_node_id_to_bytes(uint32_t node_id, uint8_t out[SECURITY_TRUSTED_ID_LEN]);
00324 static uint32_t security_node_id_from_bytes(const uint8_t in[SECURITY_TRUSTED_ID_LEN]);
00325 static void security_be32_write(uint8_t *dst, uint32_t value);
00326 static uint32_t security_be32_read(const uint8_t *src);
00327 static bool security_trusted_record_matches(const i2c_mem_store_trusted_device_t *rec,
    const security_trusted_entry_t *entry);
00328 static bool security_write_trusted_entry_to_store(uint8_t idx,
    const security_trusted_entry_t *entry);
00329 static bool security_store_trusted_slot(uint8_t idx);
00330 static bool security_erase_trusted_slot(uint8_t idx);
00331 static uint8_t security_load_trusted_from_store(void);
00332 static bool security_migrate_secrets_from_default_store_key(const i2c_mem_store_cfg_t *tpm_cfg);
00333 static bool security_load_gateway_counter_from_store(void);
00334 static bool security_store_gateway_counter_to_store(uint32_t rx_counter, uint32_t tx_counter);
00335 static bool security_tpm_load_root_seed(uint8_t seed[SECURITY_KEY_SEED_BYTES]);
00336 static bool security_tpm_store_root_seed(const uint8_t seed[SECURITY_KEY_SEED_BYTES]);
00337 static bool security_tpm_define_root_seed(void);
00338 static bool security_get_entropy_bytes(uint8_t *out, uint8_t len);
00339 static const char *security_tpm_status_text(st33ktpm2x_status_t rc);
00340 static uint32_t security_le32_read(const uint8_t *src);
00341 static void security_tpm_log_i2c_state(const char *tag, I2C_HandleTypeDef *hi2c);
00342 static void security_tpm_log_lines(const char *tag, const st33ktpm2x_cfg_t *cfg);
00343 static void security_tpm_scan_i2c3(const st33ktpm2x_cfg_t *cfg);
00344 static void security_tpm_dump_reg(const st33ktpm2x_cfg_t *cfg,
    const char *name,
    uint16_t reg,
    uint16_t mem_addr_size,
    uint8_t len);
00345 static void security_tpm_dump_raw_regs(const st33ktpm2x_cfg_t *cfg);
00346 static void security_bootstrap_tpm(void);
00347 static void security_bootstrap_store(void);
00348
00349 static const osThreadAttr_t s_security_task_attr =
00350 {
00351     .name = "security_task",
00352     .priority = (osPriority_t)osPriorityBelowNormal,
00353     .stack_mem = s_security_task_stack,
00354     .stack_size = sizeof(s_security_task_stack),
00355     .cb_mem = &s_security_task_cb,
00356     .cb_size = sizeof(s_security_task_cb)
00357 };
00358
00359 void security_main_create_task(void)
00360 {
00361     if (s_security_mutex == NULL)
00362     {
00363         s_security_mutex = osMutexNew(NULL);
00364         if (s_security_mutex == NULL)
00365         {
00366             printf("SEC: mutex create failed\r\n");
00367             return;
00368         }
00369     }
00370 }
00371
00372
00373

```

```

00379     if (s_security_cmd_queue == NULL)
00380     {
00381         s_security_cmd_queue = osMessageQueueNew(SEcurity_CMD_QUEUE_DEPTH, sizeof(security_cmd_t),
NULL);
00382     }
00383     if (s_security_cmd_queue == NULL)
00384     {
00385         printf("SEC: queue create failed\r\n");
00386         return;
00387     }
00388
00389     if (s_security_task == NULL)
00390     {
00391         memset(&s_security_task_cb, 0, sizeof(s_security_task_cb));
00392         memset(s_security_task_stack, 0, sizeof(s_security_task_stack));
00393         printf("SEC: task memory cb=%p stack=%p size=%lu\r\n",
00394             (void *)&s_security_task_cb,
00395             (void *)s_security_task_stack,
00396             (unsigned long)sizeof(s_security_task_stack));
00397         s_security_task = osThreadNew(security_main_task_fn, NULL, &s_security_task_attr);
00398         if (s_security_task == NULL)
00399         {
00400             printf("SEC: task create failed\r\n");
00401         }
00402     }
00403 }
00404
00405 bool security_main_cmd_add_device(uint32_t node_id, const uint8_t *code, uint8_t len)
00406 {
00407     security_cmd_t cmd;
00408     security_cmd_sync_t sync;
00409
00410     memset(&cmd, 0, sizeof(cmd));
00411     cmd.id = SECURITY_CMD_ADD_DEVICE;
00412     cmd.u.add_device.node_id = node_id;
00413     cmd.u.add_device.code_len = (len > SECURITY_CODE_MAX) ? SECURITY_CODE_MAX : len;
00414     cmd.u.add_device.gateway_slot = false;
00415     if ((cmd.u.add_device.code_len > 0U) && (code != NULL))
00416     {
00417         memcpy(cmd.u.add_device.code, code, cmd.u.add_device.code_len);
00418     }
00419
00420     return security_main_enqueue_sync(&cmd, &sync);
00421 }
00422
00423 bool security_main_cmd_add_gateway(uint32_t node_id, const uint8_t *code, uint8_t len)
00424 {
00425     security_cmd_t cmd;
00426     security_cmd_sync_t sync;
00427
00428     memset(&cmd, 0, sizeof(cmd));
00429     cmd.id = SECURITY_CMD_ADD_DEVICE;
00430     cmd.u.add_device.node_id = node_id;
00431     cmd.u.add_device.code_len = (len > SECURITY_CODE_MAX) ? SECURITY_CODE_MAX : len;
00432     cmd.u.add_device.gateway_slot = true;
00433     if ((cmd.u.add_device.code_len > 0U) && (code != NULL))
00434     {
00435         memcpy(cmd.u.add_device.code, code, cmd.u.add_device.code_len);
00436     }
00437
00438     return security_main_enqueue_sync(&cmd, &sync);
00439 }
00440
00441 bool security_main_cmd_delete_device(uint32_t node_id)
00442 {
00443     security_cmd_t cmd;
00444     security_cmd_sync_t sync;
00445
00446     memset(&cmd, 0, sizeof(cmd));
00447     cmd.id = SECURITY_CMD_DELETE_DEVICE;
00448     cmd.u.del_device.node_id = node_id;
00449
00450     return security_main_enqueue_sync(&cmd, &sync);
00451 }
00452
00453 bool security_main_cmd_get_device(uint8_t idx, trusted_info_t *out)
00454 {
00455     security_cmd_t cmd;
00456     security_cmd_sync_t sync;
00457     bool ok;
00458
00459     if (out == NULL)
00460     {
00461         return false;
00462     }
00463
00464     memset(&cmd, 0, sizeof(cmd));

```

```
00465     cmd.id = SECURITY_CMD_GET_DEVICE;
00466     cmd.u.get_device.index = idx;
00467
00468     ok = security_main_enqueue_sync(&cmd, &sync);
00469     if (ok)
00470     {
00471         *out = sync.trusted_out;
00472     }
00473     return ok;
00474 }
00475
00476 bool security_main_cmd_set_coding(bool enabled)
00477 {
00478     security_cmd_t cmd;
00479     security_cmd_sync_t sync;
00480
00481     memset(&cmd, 0, sizeof(cmd));
00482     cmd.id = SECURITY_CMD_SET_CODING;
00483     cmd.u.set_bool.enabled = enabled;
00484     return security_main_enqueue_sync(&cmd, &sync);
00485 }
00486
00487 bool security_main_cmd_rotate_key(void)
00488 {
00489     security_cmd_t cmd;
00490     security_cmd_sync_t sync;
00491
00492     memset(&cmd, 0, sizeof(cmd));
00493     cmd.id = SECURITY_CMD_ROTATE_KEY;
00494     return security_main_enqueue_sync(&cmd, &sync);
00495 }
00496
00497 bool security_main_cmd_set_fh(bool enabled)
00498 {
00499     security_cmd_t cmd;
00500     security_cmd_sync_t sync;
00501
00502     memset(&cmd, 0, sizeof(cmd));
00503     cmd.id = SECURITY_CMD_SET_FH;
00504     cmd.u.set_bool.enabled = enabled;
00505     return security_main_enqueue_sync(&cmd, &sync);
00506 }
00507
00508 bool security_main_cmd_set_fh_period(uint32_t period_ms)
00509 {
00510     security_cmd_t cmd;
00511     security_cmd_sync_t sync;
00512
00513     memset(&cmd, 0, sizeof(cmd));
00514     cmd.id = SECURITY_CMD_SET_FH_PERIOD;
00515     cmd.u.set_u32.value = period_ms;
00516     return security_main_enqueue_sync(&cmd, &sync);
00517 }
00518
00519 bool security_main_cmd_set_notify_mode(security_notify_mode_t mode)
00520 {
00521     security_cmd_t cmd;
00522     security_cmd_sync_t sync;
00523
00524     memset(&cmd, 0, sizeof(cmd));
00525     cmd.id = SECURITY_CMD_SET_NOTIFY;
00526     cmd.u.set_notify.mode = mode;
00527     return security_main_enqueue_sync(&cmd, &sync);
00528 }
00529
00530 bool security_main_cmd_set_lora_preset(uint8_t preset_id)
00531 {
00532     security_cmd_t cmd;
00533     security_cmd_sync_t sync;
00534
00535     memset(&cmd, 0, sizeof(cmd));
00536     cmd.id = SECURITY_CMD_SET_PRESET;
00537     cmd.u.set_preset.preset_id = preset_id;
00538     return security_main_enqueue_sync(&cmd, &sync);
00539 }
00540
00541 bool security_main_cmd_set_auto_ping(bool enabled)
00542 {
00543     security_cmd_t cmd;
00544     security_cmd_sync_t sync;
00545
00546     memset(&cmd, 0, sizeof(cmd));
00547     cmd.id = SECURITY_CMD_SET_AUTO_PING;
00548     cmd.u.set_bool.enabled = enabled;
00549     return security_main_enqueue_sync(&cmd, &sync);
00550 }
00551
```



```

00552 bool security_main_cmd_set_auto_ping_period(uint32_t period_ms)
00553 {
00554     security_cmd_t cmd;
00555     security_cmd_sync_t sync;
00556
00557     memset(&cmd, 0, sizeof(cmd));
00558     cmd.id = SECURITY_CMD_SET_AUTO_PING_PERIOD;
00559     cmd.u.set_u32.value = period_ms;
00560     return security_main_enqueue_sync(&cmd, &sync);
00561 }
00562
00563 bool security_main_cmd_set_auto_ping_mode(radio_main_auto_ping_mode_t mode)
00564 {
00565     security_cmd_t cmd;
00566     security_cmd_sync_t sync;
00567
00568     memset(&cmd, 0, sizeof(cmd));
00569     cmd.id = SECURITY_CMD_SET_AUTO_PING_MODE;
00570     cmd.u.set_u32.value = (uint32_t)mode;
00571     return security_main_enqueue_sync(&cmd, &sync);
00572 }
00573
00574 bool security_main_cmd_set_radio_runtime_cfg(const radio_main_runtime_cfg_t *cfg)
00575 {
00576     security_cmd_t cmd;
00577     security_cmd_sync_t sync;
00578
00579     if (cfg == NULL)
00580     {
00581         return false;
00582     }
00583
00584     memset(&cmd, 0, sizeof(cmd));
00585     cmd.id = SECURITY_CMD_SET_RADIO_RUNTIME;
00586     cmd.u.set_radio.cfg = *cfg;
00587     return security_main_enqueue_sync(&cmd, &sync);
00588 }
00589
00590 bool security_main_cmd_get_runtime_cfg(security_runtime_cfg_t *cfg_out)
00591 {
00592     security_cmd_t cmd;
00593     security_cmd_sync_t sync;
00594     bool ok;
00595
00596     if (cfg_out == NULL)
00597     {
00598         return false;
00599     }
00600
00601     memset(&cmd, 0, sizeof(cmd));
00602     cmd.id = SECURITY_CMD_GET_RUNTIME;
00603
00604     ok = security_main_enqueue_sync(&cmd, &sync);
00605     if (ok)
00606     {
00607         *cfg_out = sync.runtime_out;
00608     }
00609     return ok;
00610 }
00611
00612 bool security_main_get_network_key(uint8_t key_out[16])
00613 {
00614     bool ok = false;
00615
00616     if (key_out == NULL)
00617     {
00618         return false;
00619     }
00620     if (s_security_mutex == NULL)
00621     {
00622         return false;
00623     }
00624
00625     if (osMutexAcquire(s_security_mutex, 100U) == osOK)
00626     {
00627         memcpy(key_out, s_network_key, 16U);
00628         ok = s_security_initialized;
00629         (void)osMutexRelease(s_security_mutex);
00630     }
00631
00632     return ok;
00633 }
00634
00635 static bool security_lookup_peer_code(uint32_t peer_node_id,
00636                                     uint8_t code_out[SECURITY_CODE_MAX],
00637                                     uint8_t *code_len_out)
00638 {
00639

```

```

00640     uint8_t i;
00641
00642     if ((code_out == NULL) || (code_len_out == NULL) || (peer_node_id == 0U) || (s_security_mutex ==
NULL))
00643     {
00644         return false;
00645     }
00646
00647     memset(code_out, 0, SECURITY_CODE_MAX);
00648     *code_len_out = 0U;
00649     if (osMutexAcquire(s_security_mutex, 100U) == osOK)
00650     {
00651         if (s_security_initialized)
00652         {
00653             for (i = 0U; i < SECURITY_TRUSTED_MAX; i++)
00654             {
00655                 if (s_trusted[i].in_use && (s_trusted[i].node_id == peer_node_id))
00656                 {
00657                     *code_len_out = s_trusted[i].code_len;
00658                     if (*code_len_out > SECURITY_CODE_MAX)
00659                     {
00660                         *code_len_out = SECURITY_CODE_MAX;
00661                     }
00662                     if (*code_len_out > 0U)
00663                     {
00664                         memcpy(code_out, s_trusted[i].code, *code_len_out);
00665                     }
00666                     break;
00667                 }
00668             }
00669             (void)osMutexRelease(s_security_mutex);
00670         }
00671     }
00672
00673     return (*code_len_out > 0U);
00674 }
00675
00676 bool security_main_get_peer_link_key(uint32_t local_node_id, uint32_t peer_node_id, uint8_t
key_out[16])
00677 {
00678     bool ok = false;
00679     uint8_t code[SECURITY_CODE_MAX];
00680     uint8_t code_len = 0U;
00681
00682     if ((key_out == NULL) || (peer_node_id == 0U))
00683     {
00684         return false;
00685     }
00686
00687     if (!security_lookup_peer_code(peer_node_id, code, &code_len))
00688     {
00689         return false;
00690     }
00691
00692     security_peer_link_key_derive(local_node_id, peer_node_id, code, code_len, key_out);
00693     laviet_secure_zero(code, sizeof(code));
00694     ok = true;
00695     return ok;
00696 }
00697
00698 static bool security_get_frame_keys_mode_internal(uint16_t local_id,
uint16_t peer_id,
security_frame_key_mode_t mode,
const uint8_t *code_in,
uint8_t code_in_len,
bool use_explicit_code,
uint8_t enc_key_out[16],
uint8_t hmac_key_out[32])
00707 {
00708     uint8_t base_key[16];
00709     uint8_t digest[32];
00710     uint8_t info[8];
00711     uint8_t code[SECURITY_CODE_MAX];
00712     uint8_t code_len = 0U;
00713     uint16_t domain_id;
00714     bool ok = false;
00715
00716     if ((local_id == 0U) || (peer_id == 0U) || (enc_key_out == NULL) || (hmac_key_out == NULL))
00717     {
00718         return false;
00719     }
00720
00721     memset(base_key, 0, sizeof(base_key));
00722     memset(code, 0, sizeof(code));
00723     switch (mode)
00724     {
00725         case SECURITY_FRAME_KEY_MODE_PAIR_V1_32:

```

```

00726         if (use_explicit_code)
00727         {
00728             if ((code_in == NULL) || (code_in_len == 0U))
00729             {
00730                 return false;
00731             }
00732             code_len = (code_in_len > SECURITY_CODE_MAX) ? SECURITY_CODE_MAX : code_in_len;
00733             memcpy(code, code_in, code_len);
00734         }
00735         else if (!security_lookup_peer_code(peer_id, code, &code_len))
00736         {
00737             return false;
00738         }
00739         security_peer_link_key_derive(local_id, peer_id, code, code_len, base_key);
00740         ok = true;
00741         break;
00742     }
00743     case SECURITY_FRAME_KEY_MODE_SHARED:
00744     default:
00745         memcpy(base_key, s_shared_frame_root_key, sizeof(base_key));
00746         ok = true;
00747         break;
00748     }
00749 }
00750 if (!ok)
00751 {
00752     laviet_secure_zero(code, sizeof(code));
00753     return false;
00754 }
00755
00756 domain_id = (peer_id == LAVIET_BROADCAST_ID) ?
00757             LAVIET_BROADCAST_ID :
00758             ((local_id < peer_id) ? local_id : peer_id);
00759 info[0] = (uint8_t)'L';
00760 info[1] = (uint8_t)'V';
00761 info[2] = (uint8_t)'I';
00762 info[3] = (uint8_t)'K';
00763 info[4] = (uint8_t)(domain_id » 8);
00764 info[5] = (uint8_t)domain_id;
00765 info[6] = 0U;
00766 info[7] = 1U;
00767 ok = laviet_hmac_sha256(base_key, sizeof(base_key), info, sizeof(info), digest);
00768 if (ok)
00769 {
00770     memcpy(enc_key_out, digest, 16U);
00771     info[7] = 2U;
00772     ok = laviet_hmac_sha256(base_key, sizeof(base_key), info, sizeof(info), hmac_key_out);
00773 }
00774
00775 laviet_secure_zero(base_key, sizeof(base_key));
00776 laviet_secure_zero(digest, sizeof(digest));
00777 laviet_secure_zero(info, sizeof(info));
00778 laviet_secure_zero(code, sizeof(code));
00779 return ok;
00780 }
00781
00782 bool security_main_get_frame_keys_mode(uint16_t local_id,
00783                                       uint16_t peer_id,
00784                                       security_frame_key_mode_t mode,
00785                                       uint8_t enc_key_out[16],
00786                                       uint8_t hmac_key_out[32])
00787 {
00788     return security_get_frame_keys_mode_internal(local_id,
00789                                                peer_id,
00790                                                mode,
00791                                                NULL,
00792                                                0U,
00793                                                false,
00794                                                enc_key_out,
00795                                                hmac_key_out);
00796 }
00797
00798 bool security_main_get_frame_keys_for_code(uint16_t local_id,
00799                                           uint16_t peer_id,
00800                                           security_frame_key_mode_t mode,
00801                                           const uint8_t *code,
00802                                           uint8_t code_len,
00803                                           uint8_t enc_key_out[16],
00804                                           uint8_t hmac_key_out[32])
00805 {
00806     return security_get_frame_keys_mode_internal(local_id,
00807                                                peer_id,
00808                                                mode,
00809                                                code,
00810                                                code_len,
00811                                                true,
00812                                                enc_key_out,

```

```

00813                                     hmac_key_out);
00814 }
00815
00816 bool security_main_get_frame_keys(uint16_t local_id,
00817                                   uint16_t peer_id,
00818                                   bool use_pair_link,
00819                                   uint8_t enc_key_out[16],
00820                                   uint8_t hmac_key_out[32])
00821 {
00822     security_frame_key_mode_t mode = SECURITY_FRAME_KEY_MODE_SHARED;
00823
00824     if (use_pair_link && (peer_id != LAVIET_BROADCAST_ID))
00825     {
00826         mode = SECURITY_FRAME_KEY_MODE_PAIR_V1_32;
00827     }
00828
00829     return security_main_get_frame_keys_mode(local_id, peer_id, mode, enc_key_out, hmac_key_out);
00830 }
00831
00832 bool security_main_get_gateway_counter(uint32_t *rx_counter_out, uint32_t *tx_counter_out)
00833 {
00834     bool ok = false;
00835
00836     if ((rx_counter_out == NULL) || (tx_counter_out == NULL) || (s_security_mutex == NULL))
00837     {
00838         return false;
00839     }
00840
00841     if (osMutexAcquire(s_security_mutex, 100U) == osOK)
00842     {
00843         *rx_counter_out = s_gateway_rx_counter;
00844         *tx_counter_out = s_gateway_tx_counter;
00845         ok = s_security_initialized;
00846         (void)osMutexRelease(s_security_mutex);
00847     }
00848
00849     return ok;
00850 }
00851
00852 bool security_main_commit_gateway_counter(uint32_t rx_counter, uint32_t tx_counter)
00853 {
00854     bool ok = false;
00855
00856     if (s_security_mutex == NULL)
00857     {
00858         return false;
00859     }
00860
00861     if (osMutexAcquire(s_security_mutex, 100U) == osOK)
00862     {
00863         if (s_security_initialized)
00864         {
00865             s_gateway_rx_counter = rx_counter;
00866             s_gateway_tx_counter = tx_counter;
00867             if (!security_store_gateway_counter_to_store(rx_counter, tx_counter))
00868             {
00869                 printf("SEC: gateway counter persist failed\r\n");
00870             }
00871             ok = true;
00872         }
00873         (void)osMutexRelease(s_security_mutex);
00874     }
00875
00876     return ok;
00877 }
00878
00879 bool security_main_log_message(int16_t rssi_dbm, const uint8_t *payload, uint8_t payload_len)
00880 {
00881     security_cmd_t cmd;
00882
00883     if ((payload == NULL) && (payload_len > 0U))
00884     {
00885         return false;
00886     }
00887     if (s_security_cmd_queue == NULL)
00888     {
00889         return false;
00890     }
00891
00892     memset(&cmd, 0, sizeof(cmd));
00893     cmd.id = SECURITY_CMD_LOG_MESSAGE;
00894     cmd.u.log_msg.rssi_dbm = rssi_dbm;
00895     cmd.u.log_msg.len = payload_len;
00896     if (cmd.u.log_msg.len > I2C_MEM_STORE_LOG_PAYLOAD_MAX)
00897     {
00898         cmd.u.log_msg.len = I2C_MEM_STORE_LOG_PAYLOAD_MAX;
00899     }

```

```

00900     if ((cmd.u.log_msg.len > 0U) && (payload != NULL))
00901     {
00902         memcpy(cmd.u.log_msg.payload, payload, cmd.u.log_msg.len);
00903     }
00904
00905     return (osMessageQueuePut(s_security_cmd_queue, &cmd, 0U, 0U) == osOK);
00906 }
00907
00908 bool security_main_get_tpm_ready(bool *ready_out)
00909 {
00910     bool ok = false;
00911
00912     if ((ready_out == NULL) || (s_security_mutex == NULL))
00913     {
00914         return false;
00915     }
00916
00917     if (osMutexAcquire(s_security_mutex, 100U) == osOK)
00918     {
00919         *ready_out = s_tpm_ready;
00920         ok = true;
00921         (void)osMutexRelease(s_security_mutex);
00922     }
00923
00924     return ok;
00925 }
00926
00927 static void security_main_task_fn(void *argument)
00928 {
00929     security_cmd_t cmd;
00930
00931     (void)argument;
00932     memset(s_trusted, 0, sizeof(s_trusted));
00933
00934     security_bootstrap_tpm();
00935     security_log_stack_high_water("after_tpm");
00936     security_bootstrap_store();
00937     security_log_stack_high_water("after_store");
00938
00939     if (osMutexAcquire(s_security_mutex, 1000U) == osOK)
00940     {
00941         s_security_initialized = true;
00942         (void)osMutexRelease(s_security_mutex);
00943     }
00944
00945     printf("SEC: task ready\r\n");
00946
00947     for (;;)
00948     {
00949         if (osMessageQueueGet(s_security_cmd_queue, &cmd, NULL, osWaitForever) != osOK)
00950         {
00951             continue;
00952         }
00953
00954         if (cmd.id == SECURITY_CMD_LOG_MESSAGE)
00955         {
00956             if (s_mem_ready)
00957             {
00958                 (void)i2c_mem_store_append_message(&s_mem_store,
00959                                                     cmd.u.log_msg.rssi_dbm,
00960                                                     cmd.u.log_msg.payload,
00961                                                     cmd.u.log_msg.len);
00962             }
00963             continue;
00964         }
00965
00966         if (cmd.sync != NULL)
00967         {
00968             cmd.sync->result = false;
00969             memset(&cmd.sync->trusted_out, 0, sizeof(cmd.sync->trusted_out));
00970             memset(&cmd.sync->runtime_out, 0, sizeof(cmd.sync->runtime_out));
00971         }
00972
00973         if (osMutexAcquire(s_security_mutex, 1000U) == osOK)
00974         {
00975             switch (cmd.id)
00976             {
00977                 case SECURITY_CMD_ADD_DEVICE:
00978                     if (s_security_initialized)
00979                     {
00980                         cmd.sync->result = security_add_device_internal(cmd.u.add_device.node_id,
00981                                                                           cmd.u.add_device.code,
00982                                                                           cmd.u.add_device.code_len,
00983                                                                           cmd.u.add_device.gateway_slot);
00984                     }
00985                 break;

```

```
00987
00988     case SECURITY_CMD_DELETE_DEVICE:
00989         if (s_security_initialized)
00990         {
00991             cmd.sync->result = security_delete_device_internal(cmd.u.del_device.node_id);
00992         }
00993         break;
00994
00995     case SECURITY_CMD_GET_DEVICE:
00996         if (s_security_initialized)
00997         {
00998             cmd.sync->result = security_get_device_internal(cmd.u.get_device.index,
00999                                                             &cmd.sync->trusted_out);
01000         }
01001         break;
01002
01003     case SECURITY_CMD_SET_CODING:
01004         s_runtime_cfg.coding_enabled = cmd.u.set_bool.enabled;
01005         cmd.sync->result = security_commit_runtime_cfg_soft();
01006         break;
01007
01008     case SECURITY_CMD_SET_FH:
01009         s_runtime_cfg.fh_enabled = cmd.u.set_bool.enabled;
01010         cmd.sync->result = security_commit_runtime_cfg_soft();
01011         break;
01012
01013     case SECURITY_CMD_SET_FH_PERIOD:
01014         s_runtime_cfg.fh_period_ms = cmd.u.set_u32.value;
01015         cmd.sync->result = security_commit_runtime_cfg_soft();
01016         break;
01017
01018     case SECURITY_CMD_ROTATE_KEY:
01019         cmd.sync->result = security_rotate_key_internal();
01020         break;
01021
01022     case SECURITY_CMD_SET_NOTIFY:
01023         s_runtime_cfg.notify_mode = cmd.u.set_notify.mode;
01024         cmd.sync->result = security_commit_runtime_cfg_soft();
01025         break;
01026
01027     case SECURITY_CMD_SET_PRESET:
01028         s_runtime_cfg.lora_preset = cmd.u.set_preset.preset_id;
01029         cmd.sync->result = security_commit_runtime_cfg_soft();
01030         break;
01031
01032     case SECURITY_CMD_SET_AUTO_PING:
01033         s_runtime_cfg.auto_ping_enabled = cmd.u.set_bool.enabled;
01034         cmd.sync->result = security_commit_runtime_cfg_soft();
01035         break;
01036
01037     case SECURITY_CMD_SET_AUTO_PING_PERIOD:
01038         if (security_auto_ping_period_is_valid(cmd.u.set_u32.value))
01039         {
01040             s_runtime_cfg.auto_ping_period_ms = cmd.u.set_u32.value;
01041             cmd.sync->result = security_commit_runtime_cfg_soft();
01042         }
01043         break;
01044
01045     case SECURITY_CMD_SET_AUTO_PING_MODE:
01046         if
01047         (security_auto_ping_mode_is_valid((radio_main_auto_ping_mode_t)cmd.u.set_u32.value))
01048         {
01049             s_runtime_cfg.auto_ping_mode =
01050             (radio_main_auto_ping_mode_t)cmd.u.set_u32.value;
01051             cmd.sync->result = security_commit_runtime_cfg_soft();
01052         }
01053         break;
01054
01055     case SECURITY_CMD_SET_RADIO_RUNTIME:
01056         s_runtime_cfg.active_modulation = cmd.u.set_radio.cfg.active_modulation;
01057         s_runtime_cfg.lora = cmd.u.set_radio.cfg.lora;
01058         s_runtime_cfg.fsk = cmd.u.set_radio.cfg.fsk;
01059         s_runtime_cfg.ook = cmd.u.set_radio.cfg.ook;
01060         if (security_auto_ping_period_is_valid(cmd.u.set_radio.cfg.auto_ping_period_ms))
01061         {
01062             s_runtime_cfg.auto_ping_period_ms = cmd.u.set_radio.cfg.auto_ping_period_ms;
01063         }
01064         if (security_auto_ping_mode_is_valid(cmd.u.set_radio.cfg.auto_ping_mode))
01065         {
01066             s_runtime_cfg.auto_ping_mode = cmd.u.set_radio.cfg.auto_ping_mode;
01067         }
01068         s_runtime_cfg.radio_profiles_persisted = false;
01069         cmd.sync->result = security_commit_runtime_cfg_soft();
01070         break;
01071
01072     case SECURITY_CMD_GET_RUNTIME:
01073         cmd.sync->runtime_out = s_runtime_cfg;
```

```

01072             cmd.sync->result = true;
01073             break;
01074
01075             default:
01076                 break;
01077         }
01078
01079         (void)osMutexRelease(s_security_mutex);
01080     }
01081
01082     if (cmd.sync != NULL)
01083     {
01084         cmd.sync->done = true;
01085     }
01086 }
01087 }
01088
01090 static void security_log_stack_high_water(const char *tag)
01091 {
01092     UBaseType_t words;
01093
01094     words = uxTaskGetStackHighWaterMark(NULL);
01095     printf("SEC: stack %s high-water=%lu words (%lu B)\r\n",
01096         (tag != NULL) ? tag : "now",
01097         (unsigned long)words,
01098         (unsigned long)(words * sizeof(StackType_t)));
01099 }
01100
01102 static bool security_main_enqueue_sync(const security_cmd_t *cmd, security_cmd_sync_t *sync)
01103 {
01104     security_cmd_t local;
01105
01106     if ((cmd == NULL) || (sync == NULL) || (s_security_cmd_queue == NULL))
01107     {
01108         return false;
01109     }
01110
01111     memset(sync, 0, sizeof(*sync));
01112     local = *cmd;
01113     local.sync = sync;
01114
01115     if (osMessageQueuePut(s_security_cmd_queue, &local, 0U, 100U) != osOK)
01116     {
01117         return false;
01118     }
01119
01120     return security_main_wait_sync(sync, SECURITY_CMD_WAIT_MS);
01121 }
01122
01124 static bool security_main_wait_sync(security_cmd_sync_t *sync, uint32_t timeout_ms)
01125 {
01126     uint32_t start;
01127
01128     if (sync == NULL)
01129     {
01130         return false;
01131     }
01132
01133     start = HAL_GetTick();
01134     while (!sync->done)
01135     {
01136         if ((HAL_GetTick() - start) > timeout_ms)
01137         {
01138             return false;
01139         }
01140         osDelay(SECURITY_CMD_POLL_MS);
01141     }
01142
01143     return sync->result;
01144 }
01145
01147 static void security_key_seed_to_key(const uint8_t seed[SECURITY_KEY_SEED_BYTES], uint8_t key_out[16])
01148 {
01149     static const uint8_t label[] = "SEC:NET:ROOT";
01150     uint8_t digest[LAVIET_SHA256_LEN];
01151
01152     if ((seed == NULL) || (key_out == NULL))
01153     {
01154         return;
01155     }
01156
01157     memset(digest, 0, sizeof(digest));
01158     if (laviyet_hmac_sha256(seed,
01159         SECURITY_KEY_SEED_BYTES,
01160         label,
01161         (uint16_t)(sizeof(label) - 1U),
01162         digest))

```

```

01163     {
01164         memcpy(key_out, digest, 16U);
01165     }
01166     else
01167     {
01168         memset(key_out, 0, 16U);
01169     }
01170     laviet_secure_zero(digest, sizeof(digest));
01171 }
01172
01173 static void security_key_seed_to_store_key(const uint8_t seed[SECURITY_KEY_SEED_BYTES], uint8_t
key_out[16])
01174 {
01175     static const uint8_t label[] = "SEC:EEPROM:KEY";
01176     uint8_t digest[LAVIET_SHA256_LEN];
01177
01178     if ((seed == NULL) || (key_out == NULL))
01179     {
01180         return;
01181     }
01182
01183     memset(digest, 0, sizeof(digest));
01184     if (laviet_hmac_sha256(seed,
SECURITY_KEY_SEED_BYTES,
01185                             label,
01186                             (uint16_t)(sizeof(label) - 1U),
01187                             digest))
01188     {
01189         memcpy(key_out, digest, 16U);
01190     }
01191     laviet_secure_zero(digest, sizeof(digest));
01192 }
01193
01194 static bool security_seed_has_data(const uint8_t seed[SECURITY_KEY_SEED_BYTES])
01195 {
01196     uint8_t i;
01197
01198     if (seed == NULL)
01199     {
01200         return false;
01201     }
01202
01203     for (i = 0U; i < SECURITY_KEY_SEED_BYTES; i++)
01204     {
01205         if (seed[i] != 0U)
01206         {
01207             return true;
01208         }
01209     }
01210
01211     return false;
01212 }
01213
01214 static void security_load_default_radio_profiles(security_runtime_cfg_t *cfg)
01215 {
01216     if (cfg == NULL)
01217     {
01218         return;
01219     }
01220
01221     radio_main_load_default_lora_preset(2U, &cfg->lora);
01222     radio_main_load_default_fsk_profile(&cfg->fsk);
01223     radio_main_load_default_ook_profile(&cfg->ook);
01224     cfg->fh_period_ms = 2000UL;
01225     cfg->auto_ping_period_ms = SECURITY_AUTOPING_DEFAULT_MS;
01226     cfg->auto_ping_mode = RADIO_MAIN_AUTO_PING_FRAME;
01227     cfg->active_modulation = RADIO_MAIN_MODULATION_LORA;
01228     cfg->radio_profiles_persisted = false;
01229 }
01230
01231 static uint8_t security_pack_bits(uint8_t *buf, uint8_t bit_pos, uint32_t value, uint8_t width)
01232 {
01233     uint8_t bit;
01234
01235     for (bit = 0U; bit < width; bit++)
01236     {
01237         uint8_t dst_bit = (uint8_t)(bit_pos + bit);
01238         uint8_t dst_byte = (uint8_t)(dst_bit / 8U);
01239         uint8_t dst_mask = (uint8_t)(1U << (dst_bit % 8U));
01240
01241         if ((value >> bit) & 0x01UL) != 0UL)
01242         {
01243             buf[dst_byte] |= dst_mask;
01244         }
01245         else
01246         {
01247             buf[dst_byte] &= (uint8_t)~dst_mask;
01248         }
01249     }
01250 }

```



```

01253     }
01254 }
01255
01256     return (uint8_t)(bit_pos + width);
01257 }
01258
01260 static uint8_t security_unpack_bits(const uint8_t *buf, uint8_t bit_pos, uint8_t width, uint32_t
    *value_out)
01261 {
01262     uint32_t value = 0UL;
01263     uint8_t bit;
01264
01265     if (value_out == NULL)
01266     {
01267         return bit_pos;
01268     }
01269
01270     for (bit = 0U; bit < width; bit++)
01271     {
01272         uint8_t src_bit = (uint8_t)(bit_pos + bit);
01273         uint8_t src_byte = (uint8_t)(src_bit / 8U);
01274         uint8_t src_mask = (uint8_t)(1U << (src_bit % 8U));
01275
01276         if ((buf[src_byte] & src_mask) != 0U)
01277         {
01278             value |= (1UL << bit);
01279         }
01280     }
01281
01282     *value_out = value;
01283     return (uint8_t)(bit_pos + width);
01284 }
01285
01287 static uint8_t security_index_from_u32(uint32_t value, const uint32_t *table, uint8_t count, uint8_t
    fallback)
01288 {
01289     uint8_t i;
01290
01291     for (i = 0U; i < count; i++)
01292     {
01293         if (table[i] == value)
01294         {
01295             return i;
01296         }
01297     }
01298
01299     return fallback;
01300 }
01301
01303 static bool security_auto_ping_period_is_valid(uint32_t period_ms)
01304 {
01305     return security_index_from_u32(period_ms,
01306                                     s_auto_ping_period_options_ms,
01307                                     (uint8_t)(sizeof(s_auto_ping_period_options_ms) /
01308                                                 sizeof(s_auto_ping_period_options_ms[0])),
01309                                     0xFFU) != 0xFFU;
01310 }
01311
01313 static bool security_auto_ping_mode_is_valid(radio_main_auto_ping_mode_t mode)
01314 {
01315     return ((mode == RADIO_MAIN_AUTO_PING_FRAME) ||
01316            (mode == RADIO_MAIN_AUTO_PING_RAW));
01317 }
01318
01320 static uint8_t security_index_from_u16(uint16_t value, const uint16_t *table, uint8_t count, uint8_t
    fallback)
01321 {
01322     uint8_t i;
01323
01324     for (i = 0U; i < count; i++)
01325     {
01326         if (table[i] == value)
01327         {
01328             return i;
01329         }
01330     }
01331
01332     return fallback;
01333 }
01334
01336 static uint8_t security_index_from_i8(int8_t value, const int8_t *table, uint8_t count, uint8_t
    fallback)
01337 {
01338     uint8_t i;
01339
01340     for (i = 0U; i < count; i++)
01341     {

```

```

01342         if (table[i] == value)
01343         {
01344             return i;
01345         }
01346     }
01347
01348     return fallback;
01349 }
01350
01352 static uint8_t security_index_from_u8(uint8_t value, const uint8_t *table, uint8_t count, uint8_t
    fallback)
01353 {
01354     uint8_t i;
01355
01356     for (i = 0U; i < count; i++)
01357     {
01358         if (table[i] == value)
01359         {
01360             return i;
01361         }
01362     }
01363
01364     return fallback;
01365 }
01366
01368 static uint8_t security_pack_modulation_fh(radio_main_modulation_t modulation, uint8_t fh_period_idx)
01369 {
01370     return (uint8_t) (((uint8_t) modulation) & 0x03U |
01371                     ((fh_period_idx & 0x03U) << 2));
01372 }
01373
01375 static radio_main_modulation_t security_unpack_modulation(uint8_t modulation_fh)
01376 {
01377     uint8_t modulation = (uint8_t) (modulation_fh & 0x03U);
01378
01379     if (modulation > (uint8_t) RADIO_MAIN_MODULATION_OOK)
01380     {
01381         modulation = (uint8_t) RADIO_MAIN_MODULATION_LORA;
01382     }
01383
01384     return (radio_main_modulation_t) modulation;
01385 }
01386
01388 static uint8_t security_unpack_fh_period_idx(uint8_t modulation_fh)
01389 {
01390     return (uint8_t) ((modulation_fh >> 2) & 0x03U);
01391 }
01392
01394 static uint32_t security_u32_from_index(uint8_t idx, const uint32_t *table, uint8_t count, uint32_t
    fallback)
01395 {
01396     return (idx < count) ? table[idx] : fallback;
01397 }
01398
01400 static uint16_t security_u16_from_index(uint8_t idx, const uint16_t *table, uint8_t count, uint16_t
    fallback)
01401 {
01402     return (idx < count) ? table[idx] : fallback;
01403 }
01404
01406 static int8_t security_i8_from_index(uint8_t idx, const int8_t *table, uint8_t count, int8_t fallback)
01407 {
01408     return (idx < count) ? table[idx] : fallback;
01409 }
01410
01412 static uint8_t security_u8_from_index(uint8_t idx, const uint8_t *table, uint8_t count, uint8_t
    fallback)
01413 {
01414     return (idx < count) ? table[idx] : fallback;
01415 }
01416
01418 static void security_peer_link_key_derive(uint32_t local_node_id,
01419                                           uint32_t peer_node_id,
01420                                           const uint8_t *code,
01421                                           uint8_t code_len,
01422                                           uint8_t key_out[16])
01423 {
01424     static const uint8_t label[] = "SEC:PAIR:V1";
01425     uint8_t info[sizeof(label) - 1U + 8U];
01426     uint8_t digest[LAVIET_SHA256_LEN];
01427     uint32_t lo;
01428     uint32_t hi;
01429
01430     if ((key_out == NULL) || (code == NULL) || (code_len == 0U))
01431     {
01432         return;
01433     }

```

```

01434
01435     lo = (local_node_id < peer_node_id) ? local_node_id : peer_node_id;
01436     hi = (local_node_id < peer_node_id) ? peer_node_id : local_node_id;
01437     memset(info, 0, sizeof(info));
01438     memset(digest, 0, sizeof(digest));
01439     memcpy(info, label, sizeof(label) - 1U);
01440     security_be32_write(&info[sizeof(label) - 1U], lo);
01441     security_be32_write(&info[sizeof(label) - 1U + 4U], hi);
01442
01443     if (laviet_hmac_sha256(code, code_len, info, (uint16_t)sizeof(info), digest))
01444     {
01445         memcpy(key_out, digest, 16U);
01446     }
01447     else
01448     {
01449         memset(key_out, 0, 16U);
01450     }
01451
01452     laviet_secure_zero(info, sizeof(info));
01453     laviet_secure_zero(digest, sizeof(digest));
01454 }
01455
01456 static bool security_load_runtime_and_seed_from_store(bool *seed_loaded_out)
01457 {
01458     security_store_wire_t w;
01459
01460     if (seed_loaded_out != NULL)
01461     {
01462         {
01463             *seed_loaded_out = false;
01464         }
01465
01466         if (!s_runtime_shadow_valid)
01467         {
01468             return false;
01469         }
01470
01471         w = s_runtime_shadow_store;
01472         if ((w.magic != SECURITY_STORE_MAGIC) ||
01473             (w.version < 1U) ||
01474             (w.version > SECURITY_STORE_VERSION))
01475         {
01476             return false;
01477         }
01478
01479         s_runtime_cfg.coding_enabled = ((w.flags & SECURITY_STORE_FLAG_CODING) != 0U);
01480         s_runtime_cfg.fh_enabled = ((w.flags & SECURITY_STORE_FLAG_FH) != 0U);
01481         s_runtime_cfg.auto_ping_enabled = ((w.flags & SECURITY_STORE_FLAG_AUTO_PING) != 0U);
01482         s_runtime_cfg.auto_ping_period_ms = (w.version >= SECURITY_STORE_VERSION) ?
01483             security_u32_from_index((uint8_t)((w.flags &
01484 SECURITY_STORE_AUTOPING_MASK) »
01485 SECURITY_STORE_AUTOPING_SHIFT),
01486                                     s_auto_ping_period_options_ms,
01487                                     (uint8_t)(sizeof(s_auto_ping_period_options_ms) /
01488 sizeof(s_auto_ping_period_options_ms[0])),
01489                                     SECURITY_AUTOPING_DEFAULT_MS) :
01490             SECURITY_AUTOPING_DEFAULT_MS;
01491         s_runtime_cfg.auto_ping_mode = ((w.version >= SECURITY_STORE_VERSION) &&
01492             ((w.flags & SECURITY_STORE_FLAG_AUTOPING_RAW) != 0U)) ?
01493             RADIO_MAIN_AUTO_PING_RAW : RADIO_MAIN_AUTO_PING_FRAME;
01494         s_runtime_cfg.notify_mode = (w.notify_mode == (uint8_t)SECURITY_NOTIFY_BADGE) ?
01495             SECURITY_NOTIFY_BADGE : SECURITY_NOTIFY_POPUP;
01496         s_runtime_cfg.lora_preset = w.lora_preset;
01497         s_runtime_cfg.active_modulation = (w.version >= 3U) ?
01498             security_unpack_modulation(w.modulation_fh) :
01499             ((w.version >= 2U) ?
01500             (radio_main_modulation_t)w.modulation_fh :
01501             RADIO_MAIN_MODULATION_LORA);
01502         s_runtime_cfg.fh_period_ms = (w.version >= 3U) ?
01503             security_u32_from_index(security_unpack_fh_period_idx(w.modulation_fh),
01504                                     s_fh_period_options_ms,
01505                                     (uint8_t)(sizeof(s_fh_period_options_ms) /
01506                                     sizeof(s_fh_period_options_ms[0])),
01507                                     2000UL) :
01508             2000UL;
01509         s_runtime_cfg.radio_profiles_persisted = false;
01510         if (security_seed_has_data(w.seed))
01511         {
01512             memcpy(s_key_seed_cached, w.seed, SECURITY_KEY_SEED_BYTES);
01513             if (seed_loaded_out != NULL)
01514             {
01515                 *seed_loaded_out = true;
01516             }
01517         }
01518     }

```

```

01517     }
01518
01519     if (w.version >= 2U)
01520     {
01521         s_runtime_cfg.radio_profiles_persisted = security_load_radio_profiles_from_store();
01522     }
01523
01524     return true;
01525 }
01526
01527 static bool security_save_runtime_and_seed_to_store(void)
01528 {
01529     security_store_wire_t w;
01530
01531     memset(&w, 0, sizeof(w));
01532     w.magic = SECURITY_STORE_MAGIC;
01533     w.version = SECURITY_STORE_VERSION;
01534     w.flags = 0U;
01535     if (s_runtime_cfg.coding_enabled)
01536     {
01537         w.flags |= SECURITY_STORE_FLAG_CODING;
01538     }
01539     if (s_runtime_cfg.fh_enabled)
01540     {
01541         w.flags |= SECURITY_STORE_FLAG_FH;
01542     }
01543     if (s_runtime_cfg.auto_ping_enabled)
01544     {
01545         w.flags |= SECURITY_STORE_FLAG_AUTO_PING;
01546     }
01547     w.flags |= (uint8_t)((security_index_from_u32(s_runtime_cfg.auto_ping_period_ms,
01548         s_auto_ping_period_options_ms,
01549         (uint8_t)(sizeof(s_auto_ping_period_options_ms) /
01550             sizeof(s_auto_ping_period_options_ms[0])),
01551         3U) << SECURITY_STORE_AUTOPING_SHIFT) &
01552         SECURITY_STORE_AUTOPING_MASK);
01553     if (s_runtime_cfg.auto_ping_mode == RADIO_MAIN_AUTO_PING_RAW)
01554     {
01555         w.flags |= SECURITY_STORE_FLAG_AUTOPING_RAW;
01556     }
01557     w.notify_mode = (uint8_t)s_runtime_cfg.notify_mode;
01558     w.lora_preset = s_runtime_cfg.lora_preset;
01559     w.modulation_fh = security_pack_modulation_fh(s_runtime_cfg.active_modulation,
01560         security_index_from_u32(s_runtime_cfg.fh_period_ms,
01561             s_fh_period_options_ms,
01562             (uint8_t)(sizeof(s_fh_period_options_ms) /
01563                 sizeof(s_fh_period_options_ms[0])),
01564             1U));
01565     if (!s_key_seed_tpm_backed)
01566     {
01567         memcpy(w.seed, s_key_seed_cached, SECURITY_KEY_SEED_BYTES);
01568     }
01569     s_runtime_shadow_store = w;
01570     s_runtime_shadow_valid = true;
01571
01572     if (!security_save_radio_profiles_to_store())
01573     {
01574         return false;
01575     }
01576
01577     return true;
01578 }
01579
01580 static bool security_commit_runtime_cfg_soft(void)
01581 {
01582     if (!security_save_runtime_and_seed_to_store())
01583     {
01584         printf("SEC: runtime cfg applied but not persisted\r\n");
01585     }
01586
01587     return true;
01588 }
01589
01590 static bool security_save_radio_profiles_to_store(void)
01591 {
01592     security_radio_store_wire_t w;
01593     uint8_t bit_pos = 0U;
01594
01595     memset(&w, 0, sizeof(w));
01596     w.magic = SECURITY_STORE_MAGIC;
01597     w.version = SECURITY_STORE_VERSION;
01598
01599     bit_pos = security_pack_bits(w.packed, bit_pos,
01600         security_index_from_u32(s_runtime_cfg.lora.frequency_hz,
01601             s_freq_options_hz,

```

```

01605         (uint8_t) (sizeof(s_freq_options_hz) /
sizeof(s_freq_options_hz[0])),
01606                                     1U), 3U);
01607     bit_pos = security_pack_bits(w.packed, bit_pos, (uint32_t)s_runtime_cfg.lora.bandwidth, 4U);
01608     bit_pos = security_pack_bits(w.packed, bit_pos, (uint32_t)(s_runtime_cfg.lora.spreading_factor -
6U), 3U);
01609     bit_pos = security_pack_bits(w.packed, bit_pos, (uint32_t)(s_runtime_cfg.lora.coding_rate - 5U),
2U);
01610     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_i8(s_runtime_cfg.lora.tx_power_dbm,
01611                             s_power_options_dbm,
01612                             (uint8_t) (sizeof(s_power_options_dbm) /
01613 sizeof(s_power_options_dbm[0])),
4U), 3U);
01614     bit_pos = security_pack_bits(w.packed, bit_pos, s_runtime_cfg.lora.crc_on ? 1UL : 0UL, 1U);
01615     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_u16(s_runtime_cfg.lora.preamble_len,
01616                             &s_preamble_options[1],
01617                             6U, 1U), 3U);
01618     bit_pos = security_pack_bits(w.packed, bit_pos, s_runtime_cfg.lora.implicit_header ? 1UL : 0UL,
1U);
01619     bit_pos = security_pack_bits(w.packed, bit_pos, s_runtime_cfg.lora.invert_iq ? 1UL : 0UL, 1U);
01620     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_u32(s_runtime_cfg.lora.sync_word,
01621                             s_sync_word_options,
01622                             4U, 1U), 2U);
01623     bit_pos = security_pack_bits(w.packed, bit_pos, (uint32_t)s_runtime_cfg.fsk.shaping, 2U);
01624     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_u32(s_runtime_cfg.fsk.frequency_hz,
01625                             s_freq_options_hz,
01626                             (uint8_t) (sizeof(s_freq_options_hz) /
01627 sizeof(s_freq_options_hz[0])),
2U), 3U);
01628     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_u32(s_runtime_cfg.fsk.bitrate_bps,
01629                             s_bitrate_options_bps,
01630                             (uint8_t) (sizeof(s_bitrate_options_bps) /
01631 sizeof(s_bitrate_options_bps[0])),
2U), 3U);
01632     bit_pos = security_pack_bits(w.packed, bit_pos, (uint32_t)s_runtime_cfg.fsk.rx_bandwidth - 4UL,
3U);
01633     bit_pos = security_pack_bits(w.packed, bit_pos, (uint32_t)s_runtime_cfg.fsk.filter, 3U);
01634     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_i8(s_runtime_cfg.fsk.tx_power_dbm,
01635                             s_power_options_dbm,
01636                             (uint8_t) (sizeof(s_power_options_dbm) /
01637 sizeof(s_power_options_dbm[0])),
4U), 3U);
01638     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_u16(s_runtime_cfg.fsk.preamble_len,
01639                             s_preamble_options,
01640                             (uint8_t) (sizeof(s_preamble_options) /
01641 sizeof(s_preamble_options[0])),
2U), 3U);
01642     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_u8(s_runtime_cfg.fsk.sync_word_len,
01643                             s_sync_len_options,
01644                             5U, 2U), 3U);
01645     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_u32((uint32_t)s_runtime_cfg.fsk.sync_word,
01646                             &s_sync_word_options[4],
01647                             5U, 1U), 3U);
01648     bit_pos = security_pack_bits(w.packed, bit_pos, (uint32_t)s_runtime_cfg.fsk.address_filter, 2U);
01649     bit_pos = security_pack_bits(w.packed, bit_pos,
(s_runtime_cfg.fsk.crc_type == RADIO_MAIN_CRC_IBM) ? 1UL :
01650     ((s_runtime_cfg.fsk.crc_type == RADIO_MAIN_CRC_CCITT) ? 2UL : 0UL),
2U);
01651     bit_pos = security_pack_bits(w.packed, bit_pos, s_runtime_cfg.fsk.data_whitening ? 1UL : 0UL, 1U);
01652     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_u32(s_runtime_cfg.ook.frequency_hz,
01653                             s_freq_options_hz,
01654                             (uint8_t) (sizeof(s_freq_options_hz) /
01655 sizeof(s_freq_options_hz[0])),
3U), 3U);
01656     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_u32(s_runtime_cfg.ook.bitrate_bps,
01657                             s_bitrate_options_bps,
01658                             (uint8_t) (sizeof(s_bitrate_options_bps) /
01659 sizeof(s_bitrate_options_bps[0])),
2U), 3U);
01660     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_i8(s_runtime_cfg.ook.tx_power_dbm,
01661                             s_power_options_dbm,
01662                             (uint8_t) (sizeof(s_power_options_dbm) /
01663 sizeof(s_power_options_dbm[0])),
2U), 3U);
01664     bit_pos = security_pack_bits(w.packed, bit_pos,
security_index_from_i8(s_runtime_cfg.ook.tx_power_dbm,
01665                             s_power_options_dbm,
01666                             (uint8_t) (sizeof(s_power_options_dbm) /
01667 sizeof(s_power_options_dbm[0])),
2U), 3U);

```

```

01678                                     2U), 3U);
01679     bit_pos = security_pack_bits(w.packed, bit_pos, (uint32_t)s_runtime_cfg.ook.rx_bandwidth - 4UL,
01680     3U);
01681     bit_pos = security_pack_bits(w.packed, bit_pos,
01682     security_index_from_u16(s_runtime_cfg.ook.preamble_len,
01683     s_preamble_options,
01684     sizeof(s_preamble_options[0])),
01685     2U), 3U);
01686     bit_pos = security_pack_bits(w.packed, bit_pos,
01687     security_index_from_u8(s_runtime_cfg.ook.sync_word_len,
01688     s_sync_len_options,
01689     5U, 2U), 3U);
01690     bit_pos = security_pack_bits(w.packed, bit_pos,
01691     security_index_from_u32(s_runtime_cfg.ook.sync_word,
01692     &s_sync_word_options[4],
01693     5U, 1U), 3U);
01694     bit_pos = security_pack_bits(w.packed, bit_pos, (uint32_t)s_runtime_cfg.ook.threshold, 2U);
01695     (void)security_pack_bits(w.packed, bit_pos,
01696     security_index_from_u8(s_runtime_cfg.ook.threshold_value,
01697     s_ook_threshold_options,
01698     8U, 2U), 3U);
01699     s_runtime_shadow_radio = w;
01700     s_runtime_shadow_valid = true;
01701     return true;
01702 }
01703
01704 static bool security_load_radio_profiles_from_store(void)
01705 {
01706     security_radio_store_wire_t w;
01707     uint8_t bit_pos = 0U;
01708     uint32_t value = 0UL;
01709
01710     if (!s_runtime_shadow_valid)
01711     {
01712         return false;
01713     }
01714     w = s_runtime_shadow_radio;
01715     if ((w.magic != SECURITY_STORE_MAGIC) ||
01716         (w.version < 3U) ||
01717         (w.version > SECURITY_STORE_VERSION))
01718     {
01719         return false;
01720     }
01721
01722     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01723     s_runtime_cfg.lora.frequency_hz = security_u32_from_index((uint8_t)value, s_freq_options_hz, 7U,
01724     868500000UL);
01725     bit_pos = security_unpack_bits(w.packed, bit_pos, 4U, &value);
01726     s_runtime_cfg.lora.bandwidth = (radio_lora_bw_t)value;
01727     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01728     s_runtime_cfg.lora.spreading_factor = (uint8_t)(value + 6U);
01729     bit_pos = security_unpack_bits(w.packed, bit_pos, 2U, &value);
01730     s_runtime_cfg.lora.coding_rate = (uint8_t)(value + 5U);
01731     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01732     s_runtime_cfg.lora.tx_power_dbm = security_i8_from_index((uint8_t)value, s_power_options_dbm, 7U,
01733     17);
01734     bit_pos = security_unpack_bits(w.packed, bit_pos, 1U, &value);
01735     s_runtime_cfg.lora.crc_on = (value != 0UL);
01736     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01737     s_runtime_cfg.lora.preamble_len = security_u16_from_index((uint8_t)value, &s_preamble_options[1],
01738     6U, 8U);
01739     bit_pos = security_unpack_bits(w.packed, bit_pos, 1U, &value);
01740     s_runtime_cfg.lora.implicit_header = (value != 0UL);
01741     s_runtime_cfg.lora.payload_len = s_runtime_cfg.lora.implicit_header ? LAVIET_FRAME_MAX_LEN : 0U;
01742     bit_pos = security_unpack_bits(w.packed, bit_pos, 1U, &value);
01743     s_runtime_cfg.lora.invert_iq = (value != 0UL);
01744     bit_pos = security_unpack_bits(w.packed, bit_pos, 2U, &value);
01745     s_runtime_cfg.lora.sync_word = (uint8_t)security_u32_from_index((uint8_t)value,
01746     s_sync_word_options, 4U, 0x34U);
01747
01748     bit_pos = security_unpack_bits(w.packed, bit_pos, 2U, &value);
01749     s_runtime_cfg.fsk.shaping = (radio_main_fsk_shaping_t)value;
01750     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01751     s_runtime_cfg.fsk.frequency_hz = security_u32_from_index((uint8_t)value, s_freq_options_hz, 7U,
01752     868300000UL);
01753     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01754     s_runtime_cfg.fsk.bitrate_bps = security_u32_from_index((uint8_t)value, s_bitrate_options_bps, 8U,
01755     4800UL);
01756     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01757     s_runtime_cfg.fsk.rx_bandwidth = (radio_lora_bw_t)(value + 4U);
01758     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01759     s_runtime_cfg.fsk.filter = (radio_main_filter_t)value;
01760     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01761     s_runtime_cfg.fsk.tx_power_dbm = security_i8_from_index((uint8_t)value, s_power_options_dbm, 7U,
01762     14);
01763     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);

```

```

01757     s_runtime_cfg.fsk.preamble_len = security_u16_from_index((uint8_t)value, s_preamble_options, 7U,
01758     8U);
01758     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01759     s_runtime_cfg.fsk.sync_word_len = security_u8_from_index((uint8_t)value, s_sync_len_options, 5U,
01760     2U);
01760     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01761     s_runtime_cfg.fsk.sync_word = security_u32_from_index((uint8_t)value, &s_sync_word_options[4], 5U,
01762     0x2DD4UL);
01762     bit_pos = security_unpack_bits(w.packed, bit_pos, 2U, &value);
01763     s_runtime_cfg.fsk.address_filter = (radio_main_address_filter_t)value;
01764     bit_pos = security_unpack_bits(w.packed, bit_pos, 2U, &value);
01765     s_runtime_cfg.fsk.crc_type = (value == 1UL) ? RADIO_MAIN_CRC_IBM :
01766     ((value == 2UL) ? RADIO_MAIN_CRC_CCITT : RADIO_MAIN_CRC_OFF);
01767     bit_pos = security_unpack_bits(w.packed, bit_pos, 1U, &value);
01768     s_runtime_cfg.fsk.data_whitening = (value != 0UL);
01769
01770     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01771     s_runtime_cfg.ook.frequency_hz = security_u32_from_index((uint8_t)value, s_freq_options_hz, 7U,
01772     86850000UL);
01772     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01773     s_runtime_cfg.ook.bitrate_bps = security_u32_from_index((uint8_t)value, s_bitrate_options_bps, 8U,
01774     4800UL);
01774     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01775     s_runtime_cfg.ook.tx_power_dbm = security_i8_from_index((uint8_t)value, s_power_options_dbm, 7U,
01776     10);
01776     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01777     s_runtime_cfg.ook.rx_bandwidth = (radio_lora_bw_t)(value + 4U);
01778     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01779     s_runtime_cfg.ook.preamble_len = security_u16_from_index((uint8_t)value, s_preamble_options, 7U,
01780     8U);
01780     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01781     s_runtime_cfg.ook.sync_word_len = security_u8_from_index((uint8_t)value, s_sync_len_options, 5U,
01782     2U);
01782     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01783     s_runtime_cfg.ook.sync_word = security_u32_from_index((uint8_t)value, &s_sync_word_options[4], 5U,
01784     0x2DD4UL);
01784     bit_pos = security_unpack_bits(w.packed, bit_pos, 2U, &value);
01785     s_runtime_cfg.ook.threshold = (radio_main_ook_threshold_t)value;
01786     bit_pos = security_unpack_bits(w.packed, bit_pos, 3U, &value);
01787     s_runtime_cfg.ook.threshold_value = security_u8_from_index((uint8_t)value,
01788     s_ook_threshold_options, 8U, 12U);
01788
01789     return true;
01790 }
01791
01793 static bool security_load_settings_legacy_from_store(void)
01794 {
01795     return false;
01796 }
01797
01799 static bool security_load_key_seed_legacy_from_store(uint8_t seed[SECURITY_KEY_SEED_BYTES])
01800 {
01801     (void)seed;
01802     return false;
01803 }
01804
01806 static void security_migrate_trusted_slot_v1_to_v2(void)
01807 {
01808     i2c_mem_store_trusted_device_t trusted;
01809
01810     if (!s_mem_ready)
01811     {
01812         return;
01813     }
01814
01815     if (i2c_mem_store_trusted_device_read(&s_mem_store, 1U, &trusted) != I2C_MEM_STORE_OK)
01816     {
01817         return;
01818     }
01819
01820     if (i2c_mem_store_trusted_device_write(&s_mem_store, SECURITY_TRUSTED_SLOT_BASE, &trusted) ==
01821     I2C_MEM_STORE_OK)
01822     {
01823         (void)i2c_mem_store_secret_erase(&s_mem_store, 1U);
01824     }
01825
01827 static bool security_tpm_load_root_seed(uint8_t seed[SECURITY_KEY_SEED_BYTES])
01828 {
01829     uint16_t out_len = 0U;
01830     uint32_t tpm_rc = 0UL;
01831     uint32_t nv_index;
01832     st33ktpm2x_status_t rc;
01833     uint8_t attempt;
01834
01835     if ((seed == NULL) || !s_tpm_ready)
01836     {

```

```

01837     return false;
01838 }
01839
01840 for (attempt = 0U; attempt < 2U; attempt++)
01841 {
01842     nv_index = (attempt == 0U) ? SECURITY_TPM_ROOT_NV_INDEX : SECURITY_TPM_ROOT_LEGACY_NV_INDEX;
01843     memset(seed, 0, SECURITY_KEY_SEED_BYTES);
01844     out_len = 0U;
01845     tpm_rc = 0UL;
01846
01847     rc = st33ktpm2x_tpm2_nv_read(&s_tpm,
01848                                 nv_index,
01849                                 0U,
01850                                 seed,
01851                                 SECURITY_KEY_SEED_BYTES,
01852                                 &out_len,
01853                                 &tpm_rc);
01854     printf("SEC: TPM NV read idx=0x%08lX rc=%d(%s) tpm_rc=0x%08lX len=%u nonzero=%u\r\n",
01855           (unsigned long)nv_index,
01856           (int)rc,
01857           security_tpm_status_text(rc),
01858           (unsigned long)tpm_rc,
01859           (unsigned int)out_len,
01860           security_seed_has_data(seed) ? 1U : 0U);
01861     if ((rc == ST33KTPM2X_OK) &&
01862         (out_len == SECURITY_KEY_SEED_BYTES) &&
01863         security_seed_has_data(seed))
01864     {
01865         s_key_seed_tpm_pp_backed = (nv_index == SECURITY_TPM_ROOT_NV_INDEX);
01866         if (nv_index == SECURITY_TPM_ROOT_LEGACY_NV_INDEX)
01867         {
01868             printf("SEC: legacy TPM root seed loaded; hold PP during key rotation to
01869 migrate\r\n");
01870         }
01871         return true;
01872     }
01873
01874     if ((rc != ST33KTPM2X_ENOTSUP) && (rc != ST33KTPM2X_ETPM_RC))
01875     {
01876         printf("SEC: TPM root seed read failed rc=%d(%s) tpm_rc=0x%08lX\r\n",
01877               (int)rc,
01878               security_tpm_status_text(rc),
01879               (unsigned long)tpm_rc);
01880     }
01881
01882     return false;
01883 }
01884
01886 static bool security_tpm_define_root_seed(void)
01887 {
01888     uint32_t tpm_rc = 0UL;
01889     st33ktpm2x_status_t rc;
01890
01891     if (!s_tpm_ready)
01892     {
01893         return false;
01894     }
01895
01896     rc = st33ktpm2x_tpm2_nv_define(&s_tpm,
01897                                    SECURITY_TPM_ROOT_NV_INDEX,
01898                                    SECURITY_KEY_SEED_BYTES,
01899                                    SECURITY_TPM_ROOT_NV_ATTRS,
01900                                    &tpm_rc);
01901     printf("SEC: TPM NV define idx=0x%08lX attrs=0x%08lX rc=%d(%s) tpm_rc=0x%08lX\r\n",
01902           (unsigned long)SECURITY_TPM_ROOT_NV_INDEX,
01903           (unsigned long)SECURITY_TPM_ROOT_NV_ATTRS,
01904           (int)rc,
01905           security_tpm_status_text(rc),
01906           (unsigned long)tpm_rc);
01907     if (rc == ST33KTPM2X_OK)
01908     {
01909         return true;
01910     }
01911
01912     if ((rc != ST33KTPM2X_ENOTSUP) && (rc != ST33KTPM2X_ETPM_RC))
01913     {
01914         printf("SEC: TPM root seed define failed rc=%d(%s) tpm_rc=0x%08lX\r\n",
01915               (int)rc,
01916               security_tpm_status_text(rc),
01917               (unsigned long)tpm_rc);
01918     }
01919
01920     return false;
01921 }
01922
01924 static bool security_tpm_store_root_seed(const uint8_t seed[SECURITY_KEY_SEED_BYTES])

```



```

01925 {
01926     uint32_t tpm_rc = 0UL;
01927     st33ktpm2x_status_t rc;
01928
01929     if ((seed == NULL) || !s_tpm_ready || !security_seed_has_data(seed))
01930     {
01931         return false;
01932     }
01933
01934     rc = st33ktpm2x_tpm2_nv_write_platform_pp(&s_tpm,
01935                                              SECURITY_TPM_ROOT_NV_INDEX,
01936                                              0U,
01937                                              seed,
01938                                              SECURITY_KEY_SEED_BYTES,
01939                                              &tpm_rc);
01940     printf("SEC: TPM NV write PP idx=0x%08lX rc=%d(%s) tpm_rc=0x%08lX\r\n",
01941           (unsigned long)SECURITY_TPM_ROOT_NV_INDEX,
01942           (int)rc,
01943           security_tpm_status_text(rc),
01944           (unsigned long)tpm_rc);
01945     if (rc == ST33KTPM2X_OK)
01946     {
01947         s_key_seed_tpm_pp_backed = true;
01948         return true;
01949     }
01950
01951     if ((rc == ST33KTPM2X_ETPM_RC) && security_tpm_define_root_seed())
01952     {
01953         tpm_rc = 0UL;
01954         rc = st33ktpm2x_tpm2_nv_write_platform_pp(&s_tpm,
01955                                                  SECURITY_TPM_ROOT_NV_INDEX,
01956                                                  0U,
01957                                                  seed,
01958                                                  SECURITY_KEY_SEED_BYTES,
01959                                                  &tpm_rc);
01960         printf("SEC: TPM NV write PP retry idx=0x%08lX rc=%d(%s) tpm_rc=0x%08lX\r\n",
01961               (unsigned long)SECURITY_TPM_ROOT_NV_INDEX,
01962               (int)rc,
01963               security_tpm_status_text(rc),
01964               (unsigned long)tpm_rc);
01965         if (rc == ST33KTPM2X_OK)
01966         {
01967             s_key_seed_tpm_pp_backed = true;
01968             return true;
01969         }
01970     }
01971
01972     if ((rc != ST33KTPM2X_ENOTSUP) && (rc != ST33KTPM2X_ETPM_RC))
01973     {
01974         printf("SEC: TPM root seed write failed rc=%d(%s) tpm_rc=0x%08lX\r\n",
01975               (int)rc,
01976               security_tpm_status_text(rc),
01977               (unsigned long)tpm_rc);
01978     }
01979
01980     return false;
01981 }
01982
01983 static bool security_get_entropy_bytes(uint8_t *out, uint8_t len)
01984 {
01985     uint16_t out_len = 0U;
01986     uint32_t tpm_rc = 0UL;
01987     st33ktpm2x_status_t tpm_st;
01988     uint8_t i;
01989
01990     if ((out == NULL) || (len == 0U))
01991     {
01992         return false;
01993     }
01994
01995     if (s_tpm_ready)
01996     {
01997         tpm_st = st33ktpm2x_tpm2_get_random(&s_tpm,
01998                                           len,
01999                                           out,
02000                                           len,
02001                                           &out_len,
02002                                           &tpm_rc);
02003
02004         if ((tpm_st == ST33KTPM2X_OK) && (out_len >= len))
02005         {
02006             return true;
02007         }
02008
02009         printf("SEC: TPM random unavailable rc=%d(%s) tpm_rc=0x%08lX len=%u\r\n",
02010               (int)tpm_st,
02011               security_tpm_status_text(tpm_st),
02012               (unsigned long)tpm_rc,

```

```

02013         (unsigned int)out_len);
02014     s_tpm_ready = false;
02015 }
02016
02017 for (i = 0U; i < len; i += 4U)
02018 {
02019     uint32_t random_word = 0UL;
02020     uint8_t chunk = ((uint8_t)(len - i) >= 4U) ? 4U : (uint8_t)(len - i);
02021     uint8_t j;
02022
02023     if ((HAL_RNG_GetState(&hrng) != HAL_RNG_STATE_READY) ||
02024         (HAL_RNG_GenerateRandomNumber(&hrng, &random_word) != HAL_OK))
02025     {
02026         printf("SEC: HAL RNG unavailable\r\n");
02027         return false;
02028     }
02029
02030     for (j = 0U; j < chunk; j++)
02031     {
02032         out[i + j] = (uint8_t)(random_word >> (24U - (j * 8U)));
02033     }
02034 }
02035
02036 return true;
02037 }
02038
02040 static bool security_rotate_key_internal(void)
02041 {
02042     uint8_t seed[SECURITY_KEY_SEED_BYTES];
02043
02044     memset(seed, 0, sizeof(seed));
02045     if (!security_get_entropy_bytes(seed, sizeof(seed)))
02046     {
02047         laviet_secure_zero(seed, sizeof(seed));
02048         return false;
02049     }
02050
02051     if (s_tpm_ready)
02052     {
02053         if (!security_tpm_store_root_seed(seed))
02054         {
02055             printf("SEC: root seed write requires TPM PP button\r\n");
02056             laviet_secure_zero(seed, sizeof(seed));
02057             return false;
02058         }
02059         s_key_seed_tpm_backed = true;
02060         s_key_seed_tpm_pp_backed = true;
02061     }
02062     else
02063     {
02064         s_key_seed_tpm_backed = false;
02065         s_key_seed_tpm_pp_backed = false;
02066     }
02067
02068     memcpy(s_key_seed_cached, seed, SECURITY_KEY_SEED_BYTES);
02069     security_key_seed_to_key(seed, s_network_key);
02070     if (s_mem_ready)
02071     {
02072         (void)security_save_runtime_and_seed_to_store();
02073     }
02074
02075     laviet_secure_zero(seed, sizeof(seed));
02076     return true;
02077 }
02078
02080 static uint8_t security_trusted_store_capacity(void)
02081 {
02082     uint16_t available;
02083     uint16_t reserved_offset;
02084
02085     if (!s_mem_ready)
02086     {
02087         return 0U;
02088     }
02089     if (s_mem_store.secret_slot_count <= SECURITY_TRUSTED_SLOT_BASE)
02090     {
02091         return 0U;
02092     }
02093
02094     available = (uint16_t)(s_mem_store.secret_slot_count - SECURITY_TRUSTED_SLOT_BASE);
02095     if (SECURITY_GATEWAY_COUNTER_SLOT >= SECURITY_TRUSTED_SLOT_BASE)
02096     {
02097         reserved_offset = (uint16_t)(SECURITY_GATEWAY_COUNTER_SLOT - SECURITY_TRUSTED_SLOT_BASE);
02098         if (available > reserved_offset)
02099         {
02100             available = reserved_offset;
02101         }
02102     }

```

```

02102     }
02103     if (available > SECURITY_TRUSTED_MAX)
02104     {
02105         available = SECURITY_TRUSTED_MAX;
02106     }
02107
02108     return (uint8_t)available;
02109 }
02110
02111 static uint16_t security_trusted_store_slot(uint8_t idx)
02112 {
02113     return (uint16_t)(SECURITY_TRUSTED_SLOT_BASE + idx);
02114 }
02115
02116 static void security_node_id_to_bytes(uint32_t node_id, uint8_t out[SECURITY_TRUSTED_ID_LEN])
02117 {
02118     if (out == NULL)
02119     {
02120         return;
02121     }
02122
02123     out[0] = (uint8_t)((node_id >> 24) & 0xFFU);
02124     out[1] = (uint8_t)((node_id >> 16) & 0xFFU);
02125     out[2] = (uint8_t)((node_id >> 8) & 0xFFU);
02126     out[3] = (uint8_t)(node_id & 0xFFU);
02127 }
02128
02129 static uint32_t security_node_id_from_bytes(const uint8_t in[SECURITY_TRUSTED_ID_LEN])
02130 {
02131     if (in == NULL)
02132     {
02133         return 0U;
02134     }
02135
02136     return ((uint32_t)in[0] << 24) |
02137            ((uint32_t)in[1] << 16) |
02138            ((uint32_t)in[2] << 8) |
02139            (uint32_t)in[3];
02140 }
02141
02142 static void security_be32_write(uint8_t *dst, uint32_t value)
02143 {
02144     if (dst == NULL)
02145     {
02146         return;
02147     }
02148
02149     dst[0] = (uint8_t)(value >> 24);
02150     dst[1] = (uint8_t)(value >> 16);
02151     dst[2] = (uint8_t)(value >> 8);
02152     dst[3] = (uint8_t)value;
02153 }
02154
02155 static uint32_t security_be32_read(const uint8_t *src)
02156 {
02157     if (src == NULL)
02158     {
02159         return 0U;
02160     }
02161
02162     return ((uint32_t)src[0] << 24) |
02163            ((uint32_t)src[1] << 16) |
02164            ((uint32_t)src[2] << 8) |
02165            (uint32_t)src[3];
02166 }
02167
02168 static bool security_load_gateway_counter_from_store(void)
02169 {
02170     uint8_t data[I2C_MEM_STORE_SECRET_PAYLOAD_MAX];
02171     uint8_t len = 0U;
02172     i2c_mem_store_status_t rc;
02173
02174     if (!s_mem_ready || (s_mem_store.secret_slot_count <= SECURITY_GATEWAY_COUNTER_SLOT))
02175     {
02176         return false;
02177     }
02178
02179     memset(data, 0, sizeof(data));
02180     rc = i2c_mem_store_secret_read(&s_mem_store,
02181                                   SECURITY_GATEWAY_COUNTER_SLOT,
02182                                   data,
02183                                   sizeof(data),
02184                                   &len);
02185
02186     if ((rc != I2C_MEM_STORE_OK) ||
02187         (len != SECURITY_GATEWAY_COUNTER_STORE_LEN) ||
02188         (data[0] != SECURITY_GATEWAY_COUNTER_MAGIC) ||
02189         (data[1] != SECURITY_GATEWAY_COUNTER_VERSION))
02190     {

```

```

02195     {
02196         laviet_secure_zero(data, sizeof(data));
02197         return false;
02198     }
02199
02200     s_gateway_rx_counter = security_be32_read(&data[2]);
02201     s_gateway_tx_counter = security_be32_read(&data[6]);
02202     laviet_secure_zero(data, sizeof(data));
02203     return true;
02204 }
02205
02207 static bool security_store_gateway_counter_to_store(uint32_t rx_counter, uint32_t tx_counter)
02208 {
02209     uint8_t data[SECURITY_GATEWAY_COUNTER_STORE_LEN];
02210     i2c_mem_store_status_t rc;
02211
02212     if (!s_mem_ready)
02213     {
02214         return true;
02215     }
02216     if (s_mem_store.secret_slot_count <= SECURITY_GATEWAY_COUNTER_SLOT)
02217     {
02218         return false;
02219     }
02220
02221     memset(data, 0, sizeof(data));
02222     data[0] = SECURITY_GATEWAY_COUNTER_MAGIC;
02223     data[1] = SECURITY_GATEWAY_COUNTER_VERSION;
02224     security_be32_write(&data[2], rx_counter);
02225     security_be32_write(&data[6], tx_counter);
02226
02227     rc = i2c_mem_store_secret_write(&s_mem_store,
02228                                     SECURITY_GATEWAY_COUNTER_SLOT,
02229                                     data,
02230                                     sizeof(data));
02231     laviet_secure_zero(data, sizeof(data));
02232     return (rc == I2C_MEM_STORE_OK);
02233 }
02234
02236 static bool security_trusted_record_matches(const i2c_mem_store_trusted_device_t *rec,
02237                                             const security_trusted_entry_t *entry)
02238 {
02239     uint8_t id[SECURITY_TRUSTED_ID_LEN];
02240     uint8_t code_len;
02241
02242     if ((rec == NULL) || (entry == NULL) || !entry->in_use)
02243     {
02244         return false;
02245     }
02246     if (rec->id_len != SECURITY_TRUSTED_ID_LEN)
02247     {
02248         return false;
02249     }
02250     code_len = entry->code_len;
02251     if (code_len > SECURITY_CODE_MAX)
02252     {
02253         code_len = SECURITY_CODE_MAX;
02254     }
02255     if (rec->code_len != code_len)
02256     {
02257         return false;
02258     }
02259
02260     security_node_id_to_bytes(entry->node_id, id);
02261     if (memcmp(rec->id, id, SECURITY_TRUSTED_ID_LEN) != 0)
02262     {
02263         return false;
02264     }
02265
02266     if ((code_len > 0U) && (memcmp(rec->code, entry->code, code_len) != 0))
02267     {
02268         return false;
02269     }
02270
02271     return true;
02272 }
02273
02275 static bool security_write_trusted_entry_to_store(uint8_t idx,
02276                                                  const security_trusted_entry_t *entry)
02277 {
02278     i2c_mem_store_trusted_device_t rec;
02279     i2c_mem_store_trusted_device_t verify;
02280     i2c_mem_store_status_t rc = I2C_MEM_STORE_ESTATE;
02281     i2c_mem_store_status_t verify_rc = I2C_MEM_STORE_ESTATE;
02282     uint8_t capacity = security_trusted_store_capacity();
02283     uint8_t attempt;
02284

```

```

02285     if (!s_mem_ready)
02286     {
02287         printf("SEC: trusted store unavailable idx=%u\r\n", (unsigned int)idx);
02288         return false;
02289     }
02290     if ((idx >= capacity) || (entry == NULL) || !entry->in_use)
02291     {
02292         printf("SEC: trusted store no persistent slot idx=%u cap=%u secret_slots=%u\r\n",
02293             (unsigned int)idx,
02294             (unsigned int)capacity,
02295             (unsigned int)s_mem_store.secret_slot_count);
02296         return false;
02297     }
02298
02299     memset(&rec, 0, sizeof(rec));
02300     rec.id_len = SECURITY_TRUSTED_ID_LEN;
02301     rec.code_len = entry->code_len;
02302     if (rec.code_len > I2C_MEM_STORE_TRUSTED_CODE_MAX)
02303     {
02304         rec.code_len = I2C_MEM_STORE_TRUSTED_CODE_MAX;
02305     }
02306
02307     security_node_id_to_bytes(entry->node_id, rec.id);
02308     if (rec.code_len > 0U)
02309     {
02310         memcpy(rec.code, entry->code, rec.code_len);
02311     }
02312
02313     for (attempt = 0U; attempt < SECURITY_TRUSTED_STORE_RETRIES; attempt++)
02314     {
02315         rc = i2c_mem_store_trusted_device_write(&s_mem_store,
02316             security_trusted_store_slot(idx),
02317             &rec);
02318
02319         if (rc == I2C_MEM_STORE_OK)
02320         {
02321             memset(&verify, 0, sizeof(verify));
02322             verify_rc = i2c_mem_store_trusted_device_read(&s_mem_store,
02323                 security_trusted_store_slot(idx),
02324                 &verify);
02325
02326             if ((verify_rc == I2C_MEM_STORE_OK) &&
02327                 security_trusted_record_matches(&verify, entry))
02328             {
02329                 printf("SEC: trusted stored idx=%u slot=%u node=0x%08lX code_len=%u\r\n",
02330                     (unsigned int)idx,
02331                     (unsigned int)security_trusted_store_slot(idx),
02332                     (unsigned long)entry->node_id,
02333                     (unsigned int)rec.code_len);
02334                 laviet_secure_zero(&verify, sizeof(verify));
02335                 laviet_secure_zero(&rec, sizeof(rec));
02336                 return true;
02337             }
02338         }
02339
02340         printf("SEC: trusted store retry idx=%u slot=%u attempt=%u rc=%d verify_rc=%d cap=%u
02341 secret_slots=%u code_len=%u\r\n",
02342             (unsigned int)idx,
02343             (unsigned int)security_trusted_store_slot(idx),
02344             (unsigned int)(attempt + 1U),
02345             (int)rc,
02346             (int)verify_rc,
02347             (unsigned int)capacity,
02348             (unsigned int)s_mem_store.secret_slot_count,
02349             (unsigned int)rec.code_len);
02350         app_delay_ms(SECURITY_TRUSTED_STORE_RETRY_MS);
02351     }
02352
02353     laviet_secure_zero(&verify, sizeof(verify));
02354     laviet_secure_zero(&rec, sizeof(rec));
02355     return false;
02356 }
02357
02358 static bool security_store_trusted_slot(uint8_t idx)
02359 {
02360     uint8_t capacity = security_trusted_store_capacity();
02361
02362     if (!s_mem_ready)
02363     {
02364         printf("SEC: trusted store unavailable idx=%u\r\n", (unsigned int)idx);
02365         return false;
02366     }
02367
02368     if (idx >= capacity)
02369     {
02370         printf("SEC: trusted store no persistent slot idx=%u cap=%u secret_slots=%u\r\n",
02371             (unsigned int)idx,
02372             (unsigned int)capacity,
02373             (unsigned int)s_mem_store.secret_slot_count);
02374         return false;
02375     }

```

```

02372     }
02373     if (!s_trusted[idx].in_use)
02374     {
02375         return security_erase_trusted_slot(idx);
02376     }
02377
02378     return security_write_trusted_entry_to_store(idx, &s_trusted[idx]);
02379 }
02380
02382 static bool security_erase_trusted_slot(uint8_t idx)
02383 {
02384     uint8_t capacity = security_trusted_store_capacity();
02385     i2c_mem_store_status_t rc = I2C_MEM_STORE_ESTATE;
02386     i2c_mem_store_status_t verify_rc = I2C_MEM_STORE_ESTATE;
02387     i2c_mem_store_trusted_device_t verify;
02388     uint8_t attempt;
02389
02390     if (!s_mem_ready)
02391     {
02392         printf("SEC: trusted erase store unavailable idx=%u\r\n", (unsigned int)idx);
02393         return false;
02394     }
02395     if (idx >= capacity)
02396     {
02397         printf("SEC: trusted erase no persistent slot idx=%u cap=%u secret_slots=%u\r\n",
02398             (unsigned int)idx,
02399             (unsigned int)capacity,
02400             (unsigned int)s_mem_store.secret_slot_count);
02401         return false;
02402     }
02403
02404     for (attempt = 0U; attempt < SECURITY_TRUSTED_STORE_RETRIES; attempt++)
02405     {
02406         rc = i2c_mem_store_secret_erase(&s_mem_store, security_trusted_store_slot(idx));
02407         if (rc == I2C_MEM_STORE_OK)
02408         {
02409             memset(&verify, 0, sizeof(verify));
02410             verify_rc = i2c_mem_store_trusted_device_read(&s_mem_store,
02411                                                         security_trusted_store_slot(idx),
02412                                                         &verify);
02413             if (verify_rc == I2C_MEM_STORE_ENOTFOUND)
02414             {
02415                 laviet_secure_zero(&verify, sizeof(verify));
02416                 return true;
02417             }
02418             laviet_secure_zero(&verify, sizeof(verify));
02419         }
02420
02421         printf("SEC: trusted erase retry idx=%u slot=%u attempt=%u rc=%d verify_rc=%d\r\n",
02422             (unsigned int)idx,
02423             (unsigned int)security_trusted_store_slot(idx),
02424             (unsigned int)(attempt + 1U),
02425             (int)rc,
02426             (int)verify_rc);
02427         app_delay_ms(SECURITY_TRUSTED_STORE_RETRY_MS);
02428     }
02429
02430     laviet_secure_zero(&verify, sizeof(verify));
02431     return false;
02432 }
02433
02435 static uint8_t security_load_trusted_from_store(void)
02436 {
02437     uint8_t idx;
02438     uint8_t capacity = security_trusted_store_capacity();
02439     uint8_t loaded = 0U;
02440
02441     if (!s_mem_ready)
02442     {
02443         return 0U;
02444     }
02445
02446     for (idx = 0U; idx < capacity; idx++)
02447     {
02448         i2c_mem_store_trusted_device_t rec;
02449         i2c_mem_store_status_t rc;
02450         uint32_t node_id;
02451
02452         memset(&rec, 0, sizeof(rec));
02453         rc = i2c_mem_store_trusted_device_read(&s_mem_store, security_trusted_store_slot(idx), &rec);
02454         if (rc != I2C_MEM_STORE_OK)
02455         {
02456             continue;
02457         }
02458         if (rec.id_len != SECURITY_TRUSTED_ID_LEN)
02459         {
02460             continue;

```

```

02461     }
02462
02463     node_id = security_node_id_from_bytes(rec.id);
02464     if (node_id == 0U)
02465     {
02466         continue;
02467     }
02468
02469     s_trusted[idx].in_use = true;
02470     s_trusted[idx].is_master = (idx == 0U);
02471     s_trusted[idx].node_id = node_id;
02472     s_trusted[idx].code_len = rec.code_len;
02473     if (s_trusted[idx].code_len > SECURITY_CODE_MAX)
02474     {
02475         s_trusted[idx].code_len = SECURITY_CODE_MAX;
02476     }
02477     if (s_trusted[idx].code_len > 0U)
02478     {
02479         memcpy(s_trusted[idx].code, rec.code, s_trusted[idx].code_len);
02480     }
02481     loaded++;
02482 }
02483
02484 printf("SEC: trusted slots persisted=%u loaded=%u\r\n", capacity, loaded);
02485 return loaded;
02486 }
02487
02488 static bool security_migrate_secrets_from_default_store_key(const i2c_mem_store_cfg_t *tpm_cfg)
02489 {
02490     i2c_mem_store_cfg_t legacy_cfg;
02491     uint8_t loaded;
02492     uint8_t idx;
02493     bool counter_loaded;
02494
02495     if ((tpm_cfg == NULL) || !s_key_seed_tpm_backed)
02496     {
02497         return false;
02498     }
02499
02500     i2c_mem_store_default_cfg_m24c01r(&legacy_cfg, &hi2c1);
02501     legacy_cfg.secret_area_bytes = tpm_cfg->secret_area_bytes;
02502
02503     if (i2c_mem_store_init(&s_mem_store, &legacy_cfg, false) != I2C_MEM_STORE_OK)
02504     {
02505         (void)i2c_mem_store_init(&s_mem_store, tpm_cfg, false);
02506         return false;
02507     }
02508
02509     memset(s_trusted, 0, sizeof(s_trusted));
02510     counter_loaded = security_load_gateway_counter_from_store();
02511     loaded = security_load_trusted_from_store();
02512     if ((loaded == 0U) && !counter_loaded)
02513     {
02514         (void)i2c_mem_store_init(&s_mem_store, tpm_cfg, false);
02515         return false;
02516     }
02517
02518     if (i2c_mem_store_init(&s_mem_store, tpm_cfg, false) != I2C_MEM_STORE_OK)
02519     {
02520         printf("SEC: trusted TPM-key reinit failed after legacy load\r\n");
02521         memset(s_trusted, 0, sizeof(s_trusted));
02522         return false;
02523     }
02524
02525     if (counter_loaded && !security_store_gateway_counter_to_store(s_gateway_rx_counter,
02526         s_gateway_tx_counter))
02527     {
02528         printf("SEC: gateway counter TPM-key migration failed\r\n");
02529         return false;
02530     }
02531
02532     for (idx = 0U; idx < SECURITY_TRUSTED_MAX; idx++)
02533     {
02534         if (s_trusted[idx].in_use && !security_store_trusted_slot(idx))
02535         {
02536             printf("SEC: trusted TPM-key migration failed idx=%u\r\n", idx);
02537             return false;
02538         }
02539     }
02540
02541     printf("SEC: EEPROM secrets migrated to TPM-derived key\r\n");
02542     return true;
02543 }
02544
02545 static bool security_add_device_internal(uint32_t node_id, const uint8_t *code, uint8_t len, bool
02546     gateway_slot)
02547 {

```

```

02548     uint8_t i;
02549     uint8_t free_idx = 0xFFU;
02550     uint8_t max_slots = SECURITY_TRUSTED_MAX;
02551     security_trusted_entry_t new_entry;
02552
02553     if (node_id == 0U)
02554     {
02555         return false;
02556     }
02557     if (len > SECURITY_CODE_MAX)
02558     {
02559         len = SECURITY_CODE_MAX;
02560     }
02561     if (!s_mem_ready)
02562     {
02563         printf("SEC: trusted add failed; MEM store unavailable node=0x%08lX\r\n",
02564             (unsigned long)node_id);
02565         return false;
02566     }
02567
02568     max_slots = security_trusted_store_capacity();
02569     if (max_slots == 0U)
02570     {
02571         printf("SEC: trusted add failed; no persistent slots node=0x%08lX secret_slots=%u\r\n",
02572             (unsigned long)node_id,
02573             (unsigned int)s_mem_store.secret_slot_count);
02574         return false;
02575     }
02576
02577     if (gateway_slot)
02578     {
02579         if (s_trusted[0].in_use &&
02580             (s_trusted[0].node_id != 0U) &&
02581             (s_trusted[0].node_id != node_id))
02582         {
02583             printf("SEC: replacing slot0 0x%08lX with gateway 0x%08lX\r\n",
02584                 (unsigned long)s_trusted[0].node_id,
02585                 (unsigned long)node_id);
02586         }
02587
02588         memset(&new_entry, 0, sizeof(new_entry));
02589         new_entry.in_use = true;
02590         new_entry.is_master = true;
02591         new_entry.node_id = node_id;
02592         new_entry.code_len = len;
02593         if ((len > 0U) && (code != NULL))
02594         {
02595             memcpy(new_entry.code, code, len);
02596         }
02597
02598         if (!security_write_trusted_entry_to_store(0U, &new_entry))
02599         {
02600             printf("SEC: trusted gateway persist failed idx=0\r\n");
02601             return false;
02602         }
02603         s_trusted[0] = new_entry;
02604
02605         for (i = 1U; i < max_slots; i++)
02606         {
02607             if (s_trusted[i].in_use && (s_trusted[i].node_id == node_id))
02608             {
02609                 if (!security_erase_trusted_slot(i))
02610                 {
02611                     printf("SEC: trusted duplicate erase failed idx=%u\r\n", i);
02612                 }
02613                 else
02614                 {
02615                     memset(&s_trusted[i], 0, sizeof(s_trusted[i]));
02616                 }
02617             }
02618         }
02619
02620         return true;
02621     }
02622
02623     for (i = 0U; i < max_slots; i++)
02624     {
02625         if (s_trusted[i].in_use && (s_trusted[i].node_id == node_id))
02626         {
02627             new_entry = s_trusted[i];
02628             new_entry.code_len = len;
02629             memset(new_entry.code, 0, sizeof(new_entry.code));
02630             if ((len > 0U) && (code != NULL))
02631             {
02632                 memcpy(new_entry.code, code, len);
02633             }
02634             if (!security_write_trusted_entry_to_store(i, &new_entry))

```



```

02635         {
02636             printf("SEC: trusted update persist failed idx=%u\r\n", i);
02637             return false;
02638         }
02639         s_trusted[i] = new_entry;
02640         return true;
02641     }
02642 }
02643
02644 for (i = 1U; i < max_slots; i++)
02645 {
02646     if (!s_trusted[i].in_use)
02647     {
02648         free_idx = i;
02649         break;
02650     }
02651 }
02652
02653 if (free_idx == 0xFFU)
02654 {
02655     return false;
02656 }
02657
02658 memset(&new_entry, 0, sizeof(new_entry));
02659 new_entry.in_use = true;
02660 new_entry.is_master = gateway_slot;
02661 new_entry.node_id = node_id;
02662 new_entry.code_len = len;
02663 if ((len > 0U) && (code != NULL))
02664 {
02665     memcpy(new_entry.code, code, len);
02666 }
02667
02668 if (!security_write_trusted_entry_to_store(free_idx, &new_entry))
02669 {
02670     printf("SEC: trusted add persist failed idx=%u\r\n", free_idx);
02671     return false;
02672 }
02673 s_trusted[free_idx] = new_entry;
02674 return true;
02675 }
02676
02677 static bool security_delete_device_internal(uint32_t node_id)
02678 {
02679     uint8_t i;
02680
02681     for (i = 0U; i < SECURITY_TRUSTED_MAX; i++)
02682     {
02683         if (s_trusted[i].in_use && (s_trusted[i].node_id == node_id))
02684         {
02685             if (!security_erase_trusted_slot(i))
02686             {
02687                 printf("SEC: trusted erase persist failed idx=%u\r\n", i);
02688                 return false;
02689             }
02690             memset(&s_trusted[i], 0, sizeof(s_trusted[i]));
02691             return true;
02692         }
02693     }
02694 }
02695
02696 return false;
02697 }
02698
02699 static bool security_get_device_internal(uint8_t idx, trusted_info_t *out)
02700 {
02701     if ((idx >= SECURITY_TRUSTED_MAX) || (out == NULL))
02702     {
02703         return false;
02704     }
02705
02706     memset(out, 0, sizeof(*out));
02707     if (!s_trusted[idx].in_use)
02708     {
02709         out->slot = idx;
02710         out->is_master = (idx == 0U);
02711         out->in_use = false;
02712         return true;
02713     }
02714
02715     out->in_use = true;
02716     out->is_master = s_trusted[idx].is_master;
02717     out->slot = idx;
02718     out->node_id = s_trusted[idx].node_id;
02719     out->code_len = s_trusted[idx].code_len;
02720     memcpy(out->code, s_trusted[idx].code, out->code_len);
02721     return true;
02722 }
02723 }

```

```

02724
02726 static const char *security_tpm_status_text(st33ktpm2x_status_t rc)
02727 {
02728     switch (rc)
02729     {
02730         case ST33KTPM2X_OK:
02731             return "OK";
02732         case ST33KTPM2X_EINVAL:
02733             return "EINVAL";
02734         case ST33KTPM2X_ESTATE:
02735             return "ESTATE";
02736         case ST33KTPM2X_EHAL:
02737             return "EHAL";
02738         case ST33KTPM2X_ETIMEOUT:
02739             return "ETIMEOUT";
02740         case ST33KTPM2X_EOVERFLOW:
02741             return "EOVERFLOW";
02742         case ST33KTPM2X_EPROTO:
02743             return "EPROTO";
02744         case ST33KTPM2X_ETPM_RC:
02745             return "ETPM_RC";
02746         case ST33KTPM2X_ENOTSUP:
02747             return "ENOTSUP";
02748         default:
02749             return "?";
02750     }
02751 }
02752
02754 static uint32_t security_le32_read(const uint8_t *src)
02755 {
02756     return ((uint32_t)src[3] << 24) |
02757            ((uint32_t)src[2] << 16) |
02758            ((uint32_t)src[1] << 8) |
02759            (uint32_t)src[0];
02760 }
02761
02763 static void security_tpm_log_i2c_state(const char *tag, I2C_HandleTypeDef *hi2c)
02764 {
02765     if (hi2c == NULL)
02766     {
02767         SECURITY_TPM_INIT_LOG("SEC: TPM I2C %s handle=NULL\r\n", (tag != NULL) ? tag : "state");
02768         return;
02769     }
02770
02771     SECURITY_TPM_INIT_LOG("SEC: TPM I2C %s inst=%p state=%d mode=%d err=0x%08lX timing=0x%08lX\r\n",
02772                          (tag != NULL) ? tag : "state",
02773                          (void *)hi2c->Instance,
02774                          (int)HAL_I2C_GetState(hi2c),
02775                          (int)HAL_I2C_GetMode(hi2c),
02776                          (unsigned long)HAL_I2C_GetError(hi2c),
02777                          (unsigned long)hi2c->Init.Timing);
02778 }
02779
02781 static void security_tpm_log_lines(const char *tag, const st33ktpm2x_cfg_t *cfg)
02782 {
02783     GPIO_PinState reset_state = GPIO_PIN_RESET;
02784     GPIO_PinState davint_state = GPIO_PIN_RESET;
02785     GPIO_PinState scl_state;
02786     GPIO_PinState sda_state;
02787
02788     scl_state = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_0);
02789     sda_state = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_1);
02790     if ((cfg != NULL) && (cfg->reset_port != NULL) && (cfg->reset_pin != 0U))
02791     {
02792         reset_state = HAL_GPIO_ReadPin(cfg->reset_port, cfg->reset_pin);
02793     }
02794     if ((cfg != NULL) && (cfg->davint_port != NULL) && (cfg->davint_pin != 0U))
02795     {
02796         davint_state = HAL_GPIO_ReadPin(cfg->davint_port, cfg->davint_pin);
02797     }
02798
02799     SECURITY_TPM_INIT_LOG("SEC: TPM lines %s SCL_PC0=%u SDA_PC1=%u RESET#=%u DAVINT#=%u\r\n",
02800                          (tag != NULL) ? tag : "now",
02801                          (unsigned int)scl_state,
02802                          (unsigned int)sda_state,
02803                          (unsigned int)reset_state,
02804                          (unsigned int)davint_state);
02805 }
02806
02808 static void security_tpm_scan_i2c3(const st33ktpm2x_cfg_t *cfg)
02809 {
02810     uint8_t addr;
02811     uint8_t hits = 0U;
02812
02813     if ((cfg == NULL) || (cfg->hi2c == NULL))
02814     {
02815         return;

```

```

02816     }
02817
02818     for (addr = 0x08U; addr <= 0x77U; addr++)
02819     {
02820         HAL_StatusTypeDef st = HAL_I2C_IsDeviceReady(cfg->hi2c, (uint16_t)(addr << 1), 1U, 2U);
02821         if (st == HAL_OK)
02822         {
02823             hits++;
02824             SECURITY_TPM_INIT_LOG("SEC: TPM I2C3 scan hit addr=0x%02X\r\n", (unsigned int)addr);
02825         }
02826     }
02827
02828     if (hits == 0U)
02829     {
02830         SECURITY_TPM_INIT_LOG("SEC: TPM I2C3 scan no devices\r\n");
02831     }
02832     security_tpm_log_i2c_state("after_scan", cfg->hi2c);
02833 }
02834
02835 static void security_tpm_dump_reg(const st33ktpm2x_cfg_t *cfg,
02836                                  const char *name,
02837                                  uint16_t reg,
02838                                  uint16_t mem_addr_size,
02839                                  uint8_t len)
02840 {
02841     uint8_t buf[8];
02842     HAL_StatusTypeDef st;
02843     uint8_t i;
02844
02845     if ((cfg == NULL) || (cfg->hi2c == NULL) || (len == 0U) || (len > sizeof(buf)))
02846     {
02847         return;
02848     }
02849
02850     memset(buf, 0xEE, sizeof(buf));
02851     if (mem_addr_size == I2C_MEMADD_SIZE_8BIT)
02852     {
02853         uint8_t reg8 = (uint8_t)reg;
02854
02855         st = HAL_I2C_Master_Transmit(cfg->hi2c,
02856                                     (uint16_t)(cfg->i2c_addr_7bit << 1),
02857                                     &reg8,
02858                                     1U,
02859                                     SECURITY_TPM_DIAG_TIMEOUT_MS);
02860
02861         if (st == HAL_OK)
02862         {
02863             app_delay_ms(1U);
02864             st = HAL_I2C_Master_Receive(cfg->hi2c,
02865                                       (uint16_t)(cfg->i2c_addr_7bit << 1),
02866                                       buf,
02867                                       len,
02868                                       SECURITY_TPM_DIAG_TIMEOUT_MS);
02869         }
02870     }
02871     else
02872     {
02873         st = HAL_I2C_Mem_Read(cfg->hi2c,
02874                              (uint16_t)(cfg->i2c_addr_7bit << 1),
02875                              reg,
02876                              mem_addr_size,
02877                              buf,
02878                              len,
02879                              SECURITY_TPM_DIAG_TIMEOUT_MS);
02880     }
02881
02882     SECURITY_TPM_INIT_LOG("SEC: TPM reg%s %s addr=0x%02X reg=0x%04X len=%u st=%d err=0x%08lX data=",
02883                          (mem_addr_size == I2C_MEMADD_SIZE_8BIT) ? "8" : "16",
02884                          (name != NULL) ? name : "?",
02885                          (unsigned int)cfg->i2c_addr_7bit,
02886                          (unsigned int)reg,
02887                          (unsigned int)len,
02888                          (int)st,
02889                          (unsigned long)HAL_I2C_GetError(cfg->hi2c));
02890     for (i = 0U; i < len; i++)
02891     {
02892         SECURITY_TPM_INIT_LOG("%02X", (unsigned int)buf[i]);
02893     }
02894     if ((st == HAL_OK) && (len == 4U))
02895     {
02896         SECURITY_TPM_INIT_LOG(" le=0x%08lX be=0x%08lX",
02897                              (unsigned long)security_le32_read(buf),
02898                              (unsigned long)security_be32_read(buf));
02899     }
02900     SECURITY_TPM_INIT_LOG("\r\n");
02901 }
02902
02903 static void security_tpm_dump_raw_regs(const st33ktpm2x_cfg_t *cfg)

```

```

02905 {
02906     security_tpm_dump_reg(cfg, "LOCSEL", SECURITY_TPM_REG_LOC_SEL, I2C_MEMADD_SIZE_8BIT, 1U);
02907     security_tpm_dump_reg(cfg, "ACCESS", SECURITY_TPM_REG_ACCESS, I2C_MEMADD_SIZE_8BIT, 1U);
02908     security_tpm_dump_reg(cfg, "STS", SECURITY_TPM_REG_STS, I2C_MEMADD_SIZE_8BIT, 3U);
02909     security_tpm_dump_reg(cfg, "IFCAP", SECURITY_TPM_REG_IF_CAP, I2C_MEMADD_SIZE_8BIT, 4U);
02910     security_tpm_dump_reg(cfg, "DIDVID", SECURITY_TPM_REG_DID VID, I2C_MEMADD_SIZE_8BIT, 4U);
02911     security_tpm_dump_reg(cfg, "RID", SECURITY_TPM_REG_RID, I2C_MEMADD_SIZE_8BIT, 1U);
02912     security_tpm_dump_reg(cfg, "DIDVID", SECURITY_TPM_REG_DID VID, I2C_MEMADD_SIZE_16BIT, 4U);
02913     security_tpm_dump_reg(cfg, "RID", SECURITY_TPM_REG_RID, I2C_MEMADD_SIZE_16BIT, 1U);
02914 }
02915
02917 static void security_bootstrap_tpm(void)
02918 {
02919     st33ktpm2x_cfg_t cfg;
02920     st33ktpm2x_status_t rc;
02921     HAL_StatusTypeDef i2c_probe;
02922     uint32_t tpm_rc = 0UL;
02923     uint32_t did_vid = 0UL;
02924     uint8_t rid = 0U;
02925     bool davint_asserted = false;
02926
02927     st33ktpm2x_default_cfg(&cfg, &hi2c3);
02928     SECURITY_TPM_INIT_LOG("SEC: TPM cfg hi2c=%p inst=%p addr7=0x%02X io=%lu locality=%lu burst=%lu
reset_port=%p reset_pin=0x%04X davint_port=%p davint_pin=0x%04X\r\n",
02929                          (void *)cfg.hi2c,
02930                          (cfg.hi2c != NULL) ? (void *)cfg.hi2c->Instance : NULL,
02931                          (unsigned int)cfg.i2c_addr_7bit,
02932                          (unsigned long)cfg.io_timeout_ms,
02933                          (unsigned long)cfg.locality_timeout_ms,
02934                          (unsigned long)cfg.burst_timeout_ms,
02935                          (void *)cfg.reset_port,
02936                          (unsigned int)cfg.reset_pin,
02937                          (void *)cfg.davint_port,
02938                          (unsigned int)cfg.davint_pin);
02939     security_tpm_log_i2c_state("before_init", cfg.hi2c);
02940     security_tpm_log_lines("before_reset", &cfg);
02941
02942     rc = st33ktpm2x_init(&s_tpm, &cfg);
02943     if (rc != ST33KTPM2X_OK)
02944     {
02945         printf("SEC: TPM init failed rc=%d(%s)\r\n",
02946               (int)rc,
02947               security_tpm_status_text(rc));
02948         s_tpm_ready = false;
02949         return;
02950     }
02951
02952     SECURITY_TPM_INIT_LOG("SEC: TPM hard reset pulse=%lu recovery=%lu\r\n",
02953                          (unsigned long)cfg.reset_pulse_ms,
02954                          (unsigned long)cfg.reset_recovery_ms);
02955     rc = st33ktpm2x_hard_reset(&s_tpm);
02956     SECURITY_TPM_INIT_LOG("SEC: TPM hard reset rc=%d(%s)\r\n",
02957                          (int)rc,
02958                          security_tpm_status_text(rc));
02959     security_tpm_log_lines("after_reset", &cfg);
02960     security_tpm_log_i2c_state("after_reset", cfg.hi2c);
02961     if (st33ktpm2x_davint_is_asserted(&s_tpm, &davint_asserted) == ST33KTPM2X_OK)
02962     {
02963         SECURITY_TPM_INIT_LOG("SEC: TPM DAVINT# %s\r\n", davint_asserted ? "asserted" : "idle");
02964     }
02965
02966     security_tpm_scan_i2c3(&cfg);
02967
02968     i2c_probe = HAL_I2C_IsDeviceReady(cfg.hi2c,
02969                                     (uint16_t)(cfg.i2c_addr_7bit << 1),
02970                                     3U,
02971                                     cfg.io_timeout_ms);
02972     SECURITY_TPM_INIT_LOG("SEC: TPM I2C probe addr=0x%02X status=%d err=0x%081X\r\n",
02973                          (unsigned int)cfg.i2c_addr_7bit,
02974                          (int)i2c_probe,
02975                          (unsigned long)HAL_I2C_GetError(cfg.hi2c));
02976     security_tpm_dump_raw_regs(&cfg);
02977
02978     rc = st33ktpm2x_read_identity(&s_tpm, &did_vid, &rid);
02979     if (rc != ST33KTPM2X_OK)
02980     {
02981         s_tpm_ready = false;
02982         printf("SEC: TPM identity invalid rc=%d(%s) DIDVID=0x%081X RID=0x%02X\r\n",
02983               (int)rc,
02984               security_tpm_status_text(rc),
02985               (unsigned long)did_vid,
02986               rid);
02987         security_tpm_log_i2c_state("identity_failed", cfg.hi2c);
02988         return;
02989     }
02990     SECURITY_TPM_INIT_LOG("SEC: TPM identity OK DIDVID=0x%081X RID=0x%02X\r\n",
02991                          (unsigned long)did_vid,

```

```

02992         rid);
02993
02994     rc = st33ktpm2x_tpm2_startup(&s_tpm, ST33KTPM2X_TPM2_SU_CLEAR, &tpm_rc);
02995     SECURITY_TPM_INIT_LOG("SEC: TPM startup rc=%d(%s) tpm_rc=0x%08lX\r\n",
02996         (int)rc,
02997         security_tpm_status_text(rc),
02998         (unsigned long)tpm_rc);
02999     if ((rc != ST33KTPM2X_OK) && (rc != ST33KTPM2X_ETPM_RC))
03000     {
03001         s_tpm_ready = false;
03002         printf("SEC: TPM startup failed rc=%d(%s) tpm_rc=0x%08lX; define TPM_INIT_LOG for bus
03003         trace\r\n",
03004             (int)rc,
03005             security_tpm_status_text(rc),
03006             (unsigned long)tpm_rc);
03007         security_tpm_log_i2c_state("startup_failed", cfg.hi2c);
03008         security_tpm_dump_raw_regs(&cfg);
03009         return;
03010     }
03011     rc = st33ktpm2x_tpm2_self_test(&s_tpm, false, &tpm_rc);
03012     SECURITY_TPM_INIT_LOG("SEC: TPM self-test rc=%d(%s) tpm_rc=0x%08lX\r\n",
03013         (int)rc,
03014         security_tpm_status_text(rc),
03015         (unsigned long)tpm_rc);
03016     if ((rc != ST33KTPM2X_OK) && (rc != ST33KTPM2X_ETPM_RC))
03017     {
03018         s_tpm_ready = false;
03019         printf("SEC: TPM self-test failed rc=%d(%s) tpm_rc=0x%08lX; define TPM_INIT_LOG for bus
03020         trace\r\n",
03021             (int)rc,
03022             security_tpm_status_text(rc),
03023             (unsigned long)tpm_rc);
03024         security_tpm_log_i2c_state("self_test_failed", cfg.hi2c);
03025         security_tpm_dump_raw_regs(&cfg);
03026         return;
03027     }
03028     s_tpm_ready = true;
03029     if ((rc == ST33KTPM2X_ETPM_RC) && (tpm_rc != ST33KTPM2X_TPM2_RC_SUCCESS))
03030     {
03031         printf("SEC: TPM ready DIDVID=0x%08lX RID=0x%02X self_test_tpm_rc=0x%08lX\r\n",
03032             (unsigned long)did_vid,
03033             rid,
03034             (unsigned long)tpm_rc);
03035     }
03036     else
03037     {
03038         printf("SEC: TPM ready DIDVID=0x%08lX RID=0x%02X\r\n", (unsigned long)did_vid, rid);
03039     }
03040 }
03041
03042 static void security_bootstrap_store(void)
03043 {
03044     i2c_mem_store_cfg_t mem_cfg;
03045     bool loaded = false;
03046     bool loaded_v1 = false;
03047     bool runtime_loaded = false;
03048     bool store_seed_loaded = false;
03049     bool counter_loaded = false;
03050     uint8_t trusted_loaded = 0U;
03051
03052     i2c_mem_store_default_cfg_m24c01r(&mem_cfg, &hi2c1);
03053     /* M24C01-R has only 128 B, so keep EEPROM only for trusted devices. */
03054     mem_cfg.secret_area_bytes = 72U;
03055     memset(s_key_seed_cached, 0, sizeof(s_key_seed_cached));
03056     s_key_seed_tpm_backed = false;
03057     s_key_seed_tpm_pp_backed = false;
03058     security_load_default_radio_profiles(&s_runtime_cfg);
03059
03060     if (security_tpm_load_root_seed(s_key_seed_cached))
03061     {
03062         s_key_seed_tpm_backed = true;
03063         loaded = true;
03064         printf("SEC: root seed loaded from TPM\r\n");
03065         if (!s_key_seed_tpm_pp_backed)
03066         {
03067             if (security_tpm_store_root_seed(s_key_seed_cached))
03068             {
03069                 s_key_seed_tpm_pp_backed = true;
03070                 printf("SEC: root seed migrated to TPM PP-protected NV index\r\n");
03071             }
03072             else
03073             {
03074                 printf("SEC: root seed PP migration pending; hold TPM PP during next boot or key
03075                 rotation\r\n");
03076             }

```

```

03077     }
03078 }
03079 else if (s_tpm_ready)
03080 {
03081     loaded = security_rotate_key_internal();
03082     if (loaded && s_key_seed_tpm_backed)
03083     {
03084         printf("SEC: root seed generated and stored in TPM\r\n");
03085     }
03086 }
03087
03088 if (s_key_seed_tpm_backed && security_seed_has_data(s_key_seed_cached))
03089 {
03090     security_key_seed_to_store_key(s_key_seed_cached, mem_cfg.crypto_key);
03091     printf("SEC: MEM secret key derived from TPM root seed\r\n");
03092 }
03093
03094 if (i2c_mem_store_init(&s_mem_store, &mem_cfg, true) == I2C_MEM_STORE_OK)
03095 {
03096     s_mem_ready = true;
03097     printf("SEC: MEM store ready log_slots=%u secret_slots=%u\r\n",
03098           (unsigned int)s_mem_store.slot_count,
03099           (unsigned int)s_mem_store.secret_slot_count);
03100     if (s_mem_store.slot_count == 0U)
03101     {
03102         printf("SEC: MEM log disabled, storage reserved for trusted devices\r\n");
03103     }
03104 }
03105 else
03106 {
03107     s_mem_ready = false;
03108     printf("SEC: MEM store unavailable\r\n");
03109 }
03110
03111 if (s_mem_ready)
03112 {
03113     runtime_loaded = security_load_runtime_and_seed_from_store(&store_seed_loaded);
03114     loaded_v1 = runtime_loaded && !s_runtime_cfg.radio_profiles_persisted;
03115     if (!runtime_loaded)
03116     {
03117         if (security_load_settings_legacy_from_store() &&
03118             security_load_key_seed_legacy_from_store(s_key_seed_cached))
03119         {
03120             runtime_loaded = true;
03121             store_seed_loaded = security_seed_has_data(s_key_seed_cached);
03122         }
03123     }
03124     if (!loaded && runtime_loaded && store_seed_loaded)
03125     {
03126         if (s_tpm_ready)
03127         {
03128             if (security_tpm_store_root_seed(s_key_seed_cached))
03129             {
03130                 s_key_seed_tpm_backed = true;
03131                 s_key_seed_tpm_pp_backed = true;
03132                 loaded = true;
03133                 security_key_seed_to_store_key(s_key_seed_cached, mem_cfg.crypto_key);
03134                 (void)i2c_mem_store_init(&s_mem_store, &mem_cfg, false);
03135                 printf("SEC: legacy root seed migrated to TPM; EEPROM key updated\r\n");
03136             }
03137             else
03138             {
03139                 printf("SEC: legacy root seed ignored until TPM PP button is held\r\n");
03140             }
03141         }
03142         else
03143         {
03144             loaded = true;
03145         }
03146     }
03147 }
03148
03149 if (loaded && security_seed_has_data(s_key_seed_cached))
03150 {
03151     security_key_seed_to_key(s_key_seed_cached, s_network_key);
03152 }
03153 else
03154 {
03155     if (!security_rotate_key_internal())
03156     {
03157         printf("SEC: root key init failed\r\n");
03158         memset(s_network_key, 0, sizeof(s_network_key));
03159     }
03160 }
03161
03162 if (s_mem_ready)
03163 {

```

```

03164         if (loaded_v1)
03165         {
03166             security_migrate_trusted_slot_v1_to_v2();
03167         }
03168         (void)security_save_runtime_and_seed_to_store();
03169         counter_loaded = security_load_gateway_counter_from_store();
03170         if (counter_loaded)
03171         {
03172             printf("SEC: gateway counters rx=%lu tx=%lu\r\n",
03173                 (unsigned long)s_gateway_rx_counter,
03174                 (unsigned long)s_gateway_tx_counter);
03175         }
03176         trusted_loaded = security_load_trusted_from_store();
03177         if (s_key_seed_tpm_backed && (trusted_loaded == 0U) && !counter_loaded)
03178         {
03179             (void)security_migrate_secrets_from_default_store_key(&mem_cfg);
03180         }
03181     }
03182 }

```

12.80 Dokumentacja pliku Core/App/security_main.h

Security domain task and command API.

```

#include "radio_main.h"
#include <stdbool.h>
#include <stdint.h>

```

Struktury danych

- struct [trusted_info_t](#)
Public descriptor of one trusted pairing entry.
- struct [security_runtime_cfg_t](#)
Persistent and runtime security settings snapshot.

Wyliczenia

- enum [security_notify_mode_t](#) { [SECURITY_NOTIFY_POPUP](#) = 0 , [SECURITY_NOTIFY_BADGE](#) = 1 }
UI delivery mode for security notifications.
- enum [security_frame_key_mode_t](#) { [SECURITY_FRAME_KEY_MODE_SHARED](#) = 0 , [SECURITY_FRAME_KEY_MODE_PAIRING](#) = 1 }
Key derivation mode used for LAVIET frame encryption and MAC keys.

Funkcje

- void [security_main_create_task](#) (void)
Create the security task and command queue.
- bool [security_main_cmd_add_device](#) (uint32_t node_id, const uint8_t *code, uint8_t len)
Add or update a trusted device entry.
- bool [security_main_cmd_add_gateway](#) (uint32_t node_id, const uint8_t *code, uint8_t len)
Add or update the gateway/master trusted entry.
- bool [security_main_cmd_delete_device](#) (uint32_t node_id)
Delete a trusted device entry by node identifier.
- bool [security_main_cmd_get_device](#) (uint8_t idx, [trusted_info_t](#) *out)
Read one trusted device entry.
- bool [security_main_cmd_set_coding](#) (bool enabled)
Persist and apply protected/coded traffic setting.
- bool [security_main_cmd_rotate_key](#) (void)
Rotate the current network key seed.
- bool [security_main_cmd_set_fh](#) (bool enabled)

- Persist and apply frequency hopping setting.*
- bool `security_main_cmd_set_fh_period` (uint32_t period_ms)
Persist the frequency hopping period.
- bool `security_main_cmd_set_notify_mode` (security_notify_mode_t mode)
Persist the preferred security notification mode.
- bool `security_main_cmd_set_lora_preset` (uint8_t preset_id)
Persist the selected LoRa preset.
- bool `security_main_cmd_set_auto_ping` (bool enabled)
Persist and apply automatic ping enable state.
- bool `security_main_cmd_set_auto_ping_period` (uint32_t period_ms)
Persist the automatic ping period.
- bool `security_main_cmd_set_auto_ping_mode` (radio_main_auto_ping_mode_t mode)
Persist the automatic ping payload mode.
- bool `security_main_cmd_set_radio_runtime_cfg` (const radio_main_runtime_cfg_t *cfg)
Persist a full radio runtime configuration snapshot.
- bool `security_main_cmd_get_runtime_cfg` (security_runtime_cfg_t *cfg_out)
Read the current security runtime configuration.
- bool `security_main_get_network_key` (uint8_t key_out[16])
Copy the active 128-bit network key.
- bool `security_main_get_peer_link_key` (uint32_t local_node_id, uint32_t peer_node_id, uint8_t key_out[16])
Derive a peer link key for two node identifiers.
- bool `security_main_get_frame_keys_mode` (uint16_t local_id, uint16_t peer_id, security_frame_key_mode_t mode, uint8_t enc_key_out[16], uint8_t hmac_key_out[32])
Derive frame encryption and HMAC keys using a selected mode.
- bool `security_main_get_frame_keys_for_code` (uint16_t local_id, uint16_t peer_id, security_frame_key_mode_t mode, const uint8_t *code, uint8_t code_len, uint8_t enc_key_out[16], uint8_t hmac_key_out[32])
Derive frame keys from an explicit pairing code.
- bool `security_main_get_frame_keys` (uint16_t local_id, uint16_t peer_id, bool use_pair_link, uint8_t enc_key_out[16], uint8_t hmac_key_out[32])
Derive frame keys using either shared or pair-link material.
- bool `security_main_get_gateway_counter` (uint32_t *rx_counter_out, uint32_t *tx_counter_out)
Read persisted gateway RX/TX counters.
- bool `security_main_commit_gateway_counter` (uint32_t rx_counter, uint32_t tx_counter)
Persist gateway RX/TX counters after successful frame processing.
- bool `security_main_log_message` (int16_t rssi_dbm, const uint8_t *payload, uint8_t payload_len)
Append a received message to the protected message log.
- bool `security_main_get_tpm_ready` (bool *ready_out)
Read whether TPM-backed services are ready.

12.80.1 Opis szczegółowy

Security domain task and command API.
Definicja w pliku `security_main.h`.

12.80.2 Dokumentacja typów wyliczanych

12.80.2.1 security_frame_key_mode_t

enum `security_frame_key_mode_t`

Key derivation mode used for LAVIET frame encryption and MAC keys.

Wartości wyliczeń

SECURITY_FRAME_KEY_MODE_SHARED	Use the shared frame root key.
SECURITY_FRAME_KEY_MODE_PAIR_V1_32	Derive keys from a 32-bit pair code.

Definicja w linii 39 pliku [security_main.h](#).

12.80.2.2 security_notify_mode_t

enum [security_notify_mode_t](#)

UI delivery mode for security notifications.

Wartości wyliczeń

SECURITY_NOTIFY_POPUP	Show notifications as popups.
SECURITY_NOTIFY_BADGE	Prefer compact badge/status notifications.

Definicja w linii 30 pliku [security_main.h](#).

12.80.3 Dokumentacja funkcji**12.80.3.1 security_main_cmd_add_device()**

```
bool security_main_cmd_add_device (
    uint32_t node_id,
    const uint8_t * code,
    uint8_t len)
```

Add or update a trusted device entry.

Parametry

<i>node_id</i>	Trusted node identifier.
<i>code</i>	Pairing code bytes.
<i>len</i>	Pairing code length.

Zwraca

`true` when the command completed successfully.

Definicja w linii 405 pliku [security_main.c](#).

12.80.3.2 security_main_cmd_add_gateway()

```
bool security_main_cmd_add_gateway (
    uint32_t node_id,
    const uint8_t * code,
    uint8_t len)
```

Add or update the gateway/master trusted entry.

Parametry

<i>node_id</i>	Gateway node identifier.
----------------	--------------------------

<i>code</i>	Pairing code bytes.
<i>len</i>	Pairing code length.

Zwraca

`true` when the command completed successfully.

Definicja w linii 423 pliku [security_main.c](#).

12.80.3.3 security_main_cmd_delete_device()

```
bool security_main_cmd_delete_device (  
    uint32_t node_id)
```

Delete a trusted device entry by node identifier.

Parametry

<i>node_id</i>	Node identifier to remove.
----------------	----------------------------

Zwraca

`true` when the command completed successfully.

Definicja w linii 441 pliku [security_main.c](#).

12.80.3.4 security_main_cmd_get_device()

```
bool security_main_cmd_get_device (  
    uint8_t idx,  
    trusted_info_t * out)
```

Read one trusted device entry.

Parametry

<i>idx</i>	Logical trusted-entry index.
<i>out</i>	[out] Destination descriptor.

Zwraca

`true` when a descriptor was copied.

Definicja w linii 453 pliku [security_main.c](#).

12.80.3.5 security_main_cmd_get_runtime_cfg()

```
bool security_main_cmd_get_runtime_cfg (  
    security_runtime_cfg_t * cfg_out)
```

Read the current security runtime configuration.

Parametry

<i>cfg_out</i>	[out] Destination configuration snapshot.
----------------	---

Zwraca

`true` when the snapshot was copied.

Definicja w linii 590 pliku [security_main.c](#).

12.80.3.6 security_main_cmd_rotate_key()

```
bool security_main_cmd_rotate_key (  
    void )
```

Rotate the current network key seed.

Zwraca

`true` when key rotation completed successfully.

Definicja w linii 487 pliku [security_main.c](#).

12.80.3.7 security_main_cmd_set_auto_ping()

```
bool security_main_cmd_set_auto_ping (  
    bool enabled)
```

Persist and apply automatic ping enable state.

Parametry

<i>enabled</i>	Desired auto-ping state.
----------------	--------------------------

Zwraca

`true` when the command completed successfully.

Definicja w linii 541 pliku [security_main.c](#).

12.80.3.8 security_main_cmd_set_auto_ping_mode()

```
bool security_main_cmd_set_auto_ping_mode (  
    radio_main_auto_ping_mode_t mode)
```

Persist the automatic ping payload mode.

Parametry

<i>mode</i>	Payload mode.
-------------	---------------

Zwraca

`true` when the command completed successfully.

Definicja w linii 563 pliku [security_main.c](#).

12.80.3.9 security_main_cmd_set_auto_ping_period()

```
bool security_main_cmd_set_auto_ping_period (  
    uint32_t period_ms)
```

Persist the automatic ping period.

Parametry

<i>period_ms</i>	Period in milliseconds.
------------------	-------------------------

Zwraca

`true` when the command completed successfully.

Definicja w linii 552 pliku [security_main.c](#).

12.80.3.10 security_main_cmd_set_coding()

```
bool security_main_cmd_set_coding (
    bool enabled)
```

Persist and apply protected/coded traffic setting.

Parametry

<i>enabled</i>	Desired coding state.
----------------	-----------------------

Zwraca

`true` when the command completed successfully.

Definicja w linii 476 pliku [security_main.c](#).

12.80.3.11 security_main_cmd_set_fh()

```
bool security_main_cmd_set_fh (
    bool enabled)
```

Persist and apply frequency hopping setting.

Parametry

<i>enabled</i>	Desired FH state.
----------------	-------------------

Zwraca

`true` when the command completed successfully.

Definicja w linii 497 pliku [security_main.c](#).

12.80.3.12 security_main_cmd_set_fh_period()

```
bool security_main_cmd_set_fh_period (
    uint32_t period_ms)
```

Persist the frequency hopping period.

Parametry

<i>period_ms</i>	Period in milliseconds.
------------------	-------------------------

Zwraca

`true` when the command completed successfully.

Definicja w linii 508 pliku [security_main.c](#).

12.80.3.13 security_main_cmd_set_lora_preset()

```
bool security_main_cmd_set_lora_preset (  
    uint8_t preset_id)
```

Persist the selected LoRa preset.

Parametry

<i>preset_id</i>	Preset index.
------------------	---------------

Zwraca

`true` when the command completed successfully.

Definicja w linii 530 pliku [security_main.c](#).

12.80.3.14 security_main_cmd_set_notify_mode()

```
bool security_main_cmd_set_notify_mode (  
    security_notify_mode_t mode)
```

Persist the preferred security notification mode.

Parametry

<i>mode</i>	Notification display mode.
-------------	----------------------------

Zwraca

`true` when the command completed successfully.

Definicja w linii 519 pliku [security_main.c](#).

12.80.3.15 security_main_cmd_set_radio_runtime_cfg()

```
bool security_main_cmd_set_radio_runtime_cfg (  
    const radio_main_runtime_cfg_t * cfg)
```

Persist a full radio runtime configuration snapshot.

Parametry

<i>cfg</i>	Runtime radio snapshot.
------------	-------------------------

Zwraca

`true` when the command completed successfully.

Definicja w linii 574 pliku [security_main.c](#).

12.80.3.16 security_main_commit_gateway_counter()

```
bool security_main_commit_gateway_counter (  
    uint32_t rx_counter,  
    uint32_t tx_counter)
```

Persist gateway RX/TX counters after successful frame processing.

Parametry

<i>rx_counter</i>	RX counter to store.
<i>tx_counter</i>	TX counter to store.

Zwraca

`true` when counters were stored.

Definicja w linii 852 pliku [security_main.c](#).

12.80.3.17 security_main_create_task()

```
void security_main_create_task (
    void )
```

Create the security task and command queue.

Definicja w linii 367 pliku [security_main.c](#).

Odwołania w [app_freertos_init\(\)](#).

12.80.3.18 security_main_get_frame_keys()

```
bool security_main_get_frame_keys (
    uint16_t local_id,
    uint16_t peer_id,
    bool use_pair_link,
    uint8_t enc_key_out[16],
    uint8_t hmac_key_out[32])
```

Derive frame keys using either shared or pair-link material.

Parametry

<i>local_id</i>	Local frame node identifier.
<i>peer_id</i>	Peer frame node identifier.
<i>use_pair_link</i>	Use peer link material instead of the shared root mode.
<i>enc_key_out</i>	[out] 128-bit encryption key.
<i>hmac_key_out</i>	[out] 256-bit HMAC key.

Zwraca

`true` when both keys were derived.

Definicja w linii 816 pliku [security_main.c](#).

Odwołuje się do [LAVIET_BROADCAST_ID](#), [SECURITY_FRAME_KEY_MODE_PAIR_V1_32](#) i [SECURITY_FRAME_KEY_MODE_SH](#).

12.80.3.19 security_main_get_frame_keys_for_code()

```
bool security_main_get_frame_keys_for_code (
    uint16_t local_id,
    uint16_t peer_id,
    security_frame_key_mode_t mode,
    const uint8_t * code,
    uint8_t code_len,
    uint8_t enc_key_out[16],
    uint8_t hmac_key_out[32])
```

Derive frame keys from an explicit pairing code.

Parametry

<i>local_id</i>	Local frame node identifier.
<i>peer_id</i>	Peer frame node identifier.
<i>mode</i>	Key derivation mode.
<i>code</i>	Pairing code bytes.
<i>code_len</i>	Pairing code length.
<i>enc_key_out</i>	[out] 128-bit encryption key.
<i>hmac_key_out</i>	[out] 256-bit HMAC key.

Zwraca

`true` when both keys were derived.

Definicja w linii 798 pliku [security_main.c](#).

12.80.3.20 security_main_get_frame_keys_mode()

```
bool security_main_get_frame_keys_mode (
    uint16_t local_id,
    uint16_t peer_id,
    security_frame_key_mode_t mode,
    uint8_t enc_key_out[16],
    uint8_t hmac_key_out[32])
```

Derive frame encryption and HMAC keys using a selected mode.

Parametry

<i>local_id</i>	Local frame node identifier.
<i>peer_id</i>	Peer frame node identifier.
<i>mode</i>	Key derivation mode.
<i>enc_key_out</i>	[out] 128-bit encryption key.
<i>hmac_key_out</i>	[out] 256-bit HMAC key.

Zwraca

`true` when both keys were derived.

Definicja w linii 782 pliku [security_main.c](#).

12.80.3.21 security_main_get_gateway_counter()

```
bool security_main_get_gateway_counter (
    uint32_t * rx_counter_out,
    uint32_t * tx_counter_out)
```

Read persisted gateway RX/TX counters.

Parametry

<i>rx_counter_out</i>	[out] Last RX counter.
<i>tx_counter_out</i>	[out] Last TX counter.

Zwraca

`true` when counters were copied.

Definicja w linii 832 pliku [security_main.c](#).

12.80.3.22 security_main_get_network_key()

```
bool security_main_get_network_key (
    uint8_t key_out[16])
```

Copy the active 128-bit network key.

Parametry

<i>key_out</i>	[out] Destination key buffer.
----------------	-------------------------------

Zwraca

`true` when the key was copied.

Definicja w linii 612 pliku [security_main.c](#).

12.80.3.23 security_main_get_peer_link_key()

```
bool security_main_get_peer_link_key (
    uint32_t local_node_id,
    uint32_t peer_node_id,
    uint8_t key_out[16])
```

Derive a peer link key for two node identifiers.

Parametry

<i>local_node_id</i>	Local node identifier.
<i>peer_node_id</i>	Peer node identifier.
<i>key_out</i>	[out] Destination 128-bit key.

Zwraca

`true` when the key was derived.

Definicja w linii 676 pliku [security_main.c](#).

Odwołuje się do [laviat_secure_zero\(\)](#).

12.80.3.24 security_main_get_tpm_ready()

```
bool security_main_get_tpm_ready (
    bool * ready_out)
```

Read whether TPM-backed services are ready.

Parametry

<i>ready_out</i>	[out] TPM readiness flag.
------------------	---------------------------

Zwraca

true when the flag was copied.

Definicja w linii 908 pliku [security_main.c](#).

12.80.3.25 security_main_log_message()

```
bool security_main_log_message (
    int16_t rssi_dbm,
    const uint8_t * payload,
    uint8_t payload_len)
```

Append a received message to the protected message log.

Parametry

<i>rssi_dbm</i>	RSSI attached to the payload.
<i>payload</i>	Payload bytes.
<i>payload_len</i>	Payload length.

Zwraca

true when the log operation succeeded.

Definicja w linii 879 pliku [security_main.c](#).

12.81 security_main.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_SECURITY_MAIN_H_
00007 #define APP_SECURITY_MAIN_H_
00008
00009 #include "radio_main.h"
00010
00011 #include <stdbool.h>
00012 #include <stdint.h>
00013
00017 typedef struct
00018 {
00019     bool in_use;
00020     bool is_master;
00021     uint8_t slot;
00022     uint32_t node_id;
00023     uint8_t code_len;
00024     uint8_t code[8];
00025 } trusted_info_t;
00026
00030 typedef enum
00031 {
00032     SECURITY_NOTIFY_POPUP = 0,
00033     SECURITY_NOTIFY_BADGE = 1
00034 } security_notify_mode_t;
00035
00039 typedef enum
00040 {
00041     SECURITY_FRAME_KEY_MODE_SHARED = 0,
00042     SECURITY_FRAME_KEY_MODE_PAIR_V1_32 = 1
00043 } security_frame_key_mode_t;
00044
00048 typedef struct
00049 {
```

```

00050     bool coding_enabled;
00051     bool fh_enabled;
00052     uint32_t fh_period_ms;
00053     bool auto_ping_enabled;
00054     uint32_t auto_ping_period_ms;
00055     radio_main_auto_ping_mode_t auto_ping_mode;
00056     security_notify_mode_t notify_mode;
00057     uint8_t lora_preset;
00058     radio_main_modulation_t active_modulation;
00059     radio_lora_cfg_t lora;
00060     radio_main_fsk_cfg_t fsk;
00061     radio_main_ook_cfg_t ook;
00062     bool radio_profiles_persisted;
00063 } security_runtime_cfg_t;
00064
00068 void security_main_create_task(void);
00069
00077 bool security_main_cmd_add_device(uint32_t node_id, const uint8_t *code, uint8_t len);
00078
00086 bool security_main_cmd_add_gateway(uint32_t node_id, const uint8_t *code, uint8_t len);
00087
00093 bool security_main_cmd_delete_device(uint32_t node_id);
00094
00101 bool security_main_cmd_get_device(uint8_t idx, trusted_info_t *out);
00102
00108 bool security_main_cmd_set_coding(bool enabled);
00109
00114 bool security_main_cmd_rotate_key(void);
00115
00121 bool security_main_cmd_set_fh(bool enabled);
00122
00128 bool security_main_cmd_set_fh_period(uint32_t period_ms);
00129
00135 bool security_main_cmd_set_notify_mode(security_notify_mode_t mode);
00136
00142 bool security_main_cmd_set_lora_preset(uint8_t preset_id);
00143
00149 bool security_main_cmd_set_auto_ping(bool enabled);
00150
00156 bool security_main_cmd_set_auto_ping_period(uint32_t period_ms);
00157
00163 bool security_main_cmd_set_auto_ping_mode(radio_main_auto_ping_mode_t mode);
00164
00170 bool security_main_cmd_set_radio_runtime_cfg(const radio_main_runtime_cfg_t *cfg);
00171
00177 bool security_main_cmd_get_runtime_cfg(security_runtime_cfg_t *cfg_out);
00178
00184 bool security_main_get_network_key(uint8_t key_out[16]);
00185
00193 bool security_main_get_peer_link_key(uint32_t local_node_id, uint32_t peer_node_id, uint8_t
key_out[16]);
00194
00204 bool security_main_get_frame_keys_mode(uint16_t local_id,
uint16_t peer_id,
security_frame_key_mode_t mode,
uint8_t enc_key_out[16],
uint8_t hmac_key_out[32]);
00205
00206
00207
00208
00209
00221 bool security_main_get_frame_keys_for_code(uint16_t local_id,
uint16_t peer_id,
security_frame_key_mode_t mode,
const uint8_t *code,
uint8_t code_len,
uint8_t enc_key_out[16],
uint8_t hmac_key_out[32]);
00222
00223
00224
00225
00226
00227
00228
00238 bool security_main_get_frame_keys(uint16_t local_id,
uint16_t peer_id,
bool use_pair_link,
uint8_t enc_key_out[16],
uint8_t hmac_key_out[32]);
00239
00240
00241
00242
00243
00250 bool security_main_get_gateway_counter(uint32_t *rx_counter_out, uint32_t *tx_counter_out);
00251
00258 bool security_main_commit_gateway_counter(uint32_t rx_counter, uint32_t tx_counter);
00259
00267 bool security_main_log_message(int16_t rssi_dbm, const uint8_t *payload, uint8_t payload_len);
00268
00274 bool security_main_get_tpm_ready(bool *ready_out);
00275
00276 #endif /* APP_SECURITY_MAIN_H */

```

12.82 service.c

```

00001 #include "service.h"
00002
00003 #include "cmsis_os2.h"
00004 #include "FreeRTOS.h"
00005 #include "laviet_frame.h"
00006 #include "main.h"
00007 #include "radio_main.h"
00008 #include "radio_lib/radio_lib.h"
00009 #include "security_main.h"
00010
00011 #include <ctype.h>
00012 #include <limits.h>
00013 #include <stdint.h>
00014 #include <stdio.h>
00015 #include <string.h>
00016
00017 #if SERVICE_UART
00018
00019 #define SERVICE_UART_TASK_STACK_SIZE 2048U
00020 #define SERVICE_UART_TASK_STACK_WORDS (SERVICE_UART_TASK_STACK_SIZE / sizeof(StackType_t))
00021 #define SERVICE_UART_LINE_BUF_SIZE (8U + (RADIO_LIB_MAX_PAYLOAD * 3U))
00022 #define SERVICE_UART_RX_RING_SIZE 1024U
00023 #define SERVICE_UART_POLL_DELAY_MS 1U
00024 #define SERVICE_UART_IRQ_PRIORITY 5U
00025 #define SERVICE_UART_TOKEN_MAX 24U
00026
00027 extern UART_HandleTypeDef huart1;
00028
00029 static osThreadId_t s_service_uart_task = NULL;
00030 static StaticTask_t s_service_uart_task_cb;
00031 static StackType_t s_service_uart_task_stack[SERVICE_UART_TASK_STACK_WORDS];
00032 static char s_service_uart_line[SERVICE_UART_LINE_BUF_SIZE];
00033 static uint8_t s_service_uart_tx_bytes[RADIO_LIB_MAX_PAYLOAD];
00034 static uint8_t s_service_uart_rx_ring[SERVICE_UART_RX_RING_SIZE];
00035 static volatile uint16_t s_service_uart_rx_head = 0U;
00036 static volatile uint16_t s_service_uart_rx_tail = 0U;
00037 static volatile bool s_service_uart_rx_overflow = false;
00038 static uint8_t s_service_uart_rx_byte = 0U;
00039 static bool s_service_uart_unlocked = false;
00040
00041 static const osThreadAttr_t s_service_uart_task_attr =
00042 {
00043     .name = "uart_service",
00044     .priority = (osPriority_t)osPriorityLow,
00045     .stack_mem = s_service_uart_task_stack,
00046     .stack_size = sizeof(s_service_uart_task_stack),
00047     .cb_mem = &s_service_uart_task_cb,
00048     .cb_size = sizeof(s_service_uart_task_cb)
00049 };
00050
00051 static void service_uart_task_fn(void *argument);
00052 static void service_uart_rx_start(void);
00053 static bool service_uart_rx_read_byte(uint8_t *out);
00054 static bool service_uart_rx_take_overflow(void);
00055 static void service_uart_rx_clear(void);
00056 static void service_uart_rx_push_from_isr(uint8_t ch);
00057 static void service_uart_handle_line(const char *line);
00058 static void service_uart_print_locked_help(void);
00059 static void service_uart_print_help(void);
00060 static void service_uart_print_cfg(void);
00061 static void service_uart_print_radio_result(bool ok);
00062 static void service_uart_handle_login(const char *args);
00063 static void service_uart_handle_send_hex(const char *payload);
00064 static void service_uart_handle_text(const char *args);
00065 static void service_uart_handle_mod(const char *args);
00066 static void service_uart_handle_preset(const char *args);
00067 static void service_uart_handle_freq(const char *args);
00068 static void service_uart_handle_bw(const char *args);
00069 static void service_uart_handle_option(const char *args);
00070 static void service_uart_handle_bool_cmd(const char *args, bool (*setter)(bool));
00071 static void service_uart_handle_u32_cmd(const char *args, bool (*setter)(uint32_t));
00072 static void service_uart_handle_auto_ping_mode(const char *args);
00073 static void service_uart_handle_save(void);
00074 static bool service_uart_parse_hex_bytes(const char *text,
00075     uint8_t *data,
00076     uint8_t *len_out,
00077     const char **error_out);
00078 static bool service_uart_parse_u32_token(const char *token, uint32_t *value_out);
00079 static bool service_uart_parse_u32_arg(const char *args, uint32_t *value_out);
00080 static bool service_uart_parse_bool_arg(const char *args, bool *value_out);
00081 static bool service_uart_parse_mod_arg(const char *args, radio_main_modulation_t *mod_out);
00082 static bool service_uart_next_token(const char **cursor, char *out, uint8_t out_size);
00083 static const char *service_uart_skip_space(const char *text);
00084 static bool service_uart_token_equals(const char *a, const char *b);

```

```

00085 static bool service_uart_match_prefix(const char **cursor, const char *prefix);
00086 static int service_uart_hex_nibble(char c);
00087 static uint32_t service_uart_get_service_code(void);
00088 static const char *service_uart_mod_text(radio_main_modulation_t modulation);
00089 static const char *service_uart_ap_mode_text(radio_main_auto_ping_mode_t mode);
00090
00091 void service_uart_create_task(void)
00092 {
00093     if (s_service_uart_task != NULL)
00094     {
00095         return;
00096     }
00097
00098     HAL_NVIC_SetPriority(USART1_IRQn, SERVICE_UART_IRQ_PRIORITY, 0U);
00099     HAL_NVIC_EnableIRQ(USART1_IRQn);
00100     service_uart_rx_start();
00101
00102     s_service_uart_task = osThreadNew(service_uart_task_fn, NULL, &s_service_uart_task_attr);
00103     if (s_service_uart_task == NULL)
00104     {
00105         printf("SERVICE UART: task create failed\r\n");
00106     }
00107 }
00108
00110 static void service_uart_task_fn(void *argument)
00111 {
00112     uint16_t line_len = 0U;
00113     bool line_overflow = false;
00114
00115     (void)argument;
00116     printf("SERVICE UART locked. Use LOGIN <code>\r\n");
00117
00118     for (;;)
00119     {
00120         uint8_t ch = 0U;
00121         bool did_work = false;
00122
00123         if (service_uart_rx_take_overflow())
00124         {
00125             service_uart_rx_clear();
00126             line_len = 0U;
00127             line_overflow = false;
00128             printf("SERVICE ERR: rx overflow\r\n");
00129         }
00130
00131         while (service_uart_rx_read_byte(&ch))
00132         {
00133             did_work = true;
00134             if ((ch == (uint8_t)'\r') || (ch == (uint8_t)'\n'))
00135             {
00136                 if (line_overflow)
00137                 {
00138                     printf("SERVICE ERR: line too long\r\n");
00139                 }
00140                 else if (line_len > 0U)
00141                 {
00142                     s_service_uart_line[line_len] = '\0';
00143                     service_uart_handle_line(s_service_uart_line);
00144                 }
00145
00146                 line_len = 0U;
00147                 line_overflow = false;
00148             }
00149             else if ((ch == 0x08U) || (ch == 0x7FU))
00150             {
00151                 if (!line_overflow && (line_len > 0U))
00152                 {
00153                     line_len--;
00154                 }
00155             }
00156             else if (line_overflow)
00157             {
00158                 /* Drop input until end-of-line. */
00159             }
00160             else if (line_len >= (uint16_t)(sizeof(s_service_uart_line) - 1U))
00161             {
00162                 line_overflow = true;
00163             }
00164             else if (isprint((unsigned char)ch) || (ch == (uint8_t)'\t'))
00165             {
00166                 s_service_uart_line[line_len] = (char)ch;
00167                 line_len++;
00168             }
00169         }
00170
00171         if (!did_work)
00172         {

```

```

00173         osDelay(SERVICE_UART_POLL_DELAY_MS);
00174     }
00175 }
00176 }
00177
00179 static void service_uart_rx_start(void)
00180 {
00181     (void)HAL_UART_Receive_IT(&huart1, &s_service_uart_rx_byte, 1U);
00182 }
00183
00185 static bool service_uart_rx_read_byte(uint8_t *out)
00186 {
00187     uint32_t primask;
00188     bool ok = false;
00189
00190     if (out == NULL)
00191     {
00192         return false;
00193     }
00194
00195     primask = __get_PRIMASK();
00196     __disable_irq();
00197     if (s_service_uart_rx_tail != s_service_uart_rx_head)
00198     {
00199         *out = s_service_uart_rx_ring[s_service_uart_rx_tail];
00200         s_service_uart_rx_tail = (uint16_t)((s_service_uart_rx_tail + 1U) %
SERVICE_UART_RX_RING_SIZE);
00201         ok = true;
00202     }
00203     __set_PRIMASK(primask);
00204
00205     return ok;
00206 }
00207
00209 static bool service_uart_rx_take_overflow(void)
00210 {
00211     uint32_t primask;
00212     bool overflow;
00213
00214     primask = __get_PRIMASK();
00215     __disable_irq();
00216     overflow = s_service_uart_rx_overflow;
00217     s_service_uart_rx_overflow = false;
00218     __set_PRIMASK(primask);
00219
00220     return overflow;
00221 }
00222
00224 static void service_uart_rx_clear(void)
00225 {
00226     uint32_t primask;
00227
00228     primask = __get_PRIMASK();
00229     __disable_irq();
00230     s_service_uart_rx_head = 0U;
00231     s_service_uart_rx_tail = 0U;
00232     __set_PRIMASK(primask);
00233 }
00234
00236 static void service_uart_rx_push_from_isr(uint8_t ch)
00237 {
00238     uint16_t next = (uint16_t)((s_service_uart_rx_head + 1U) % SERVICE_UART_RX_RING_SIZE);
00239
00240     if (next == s_service_uart_rx_tail)
00241     {
00242         s_service_uart_rx_overflow = true;
00243         return;
00244     }
00245
00246     s_service_uart_rx_ring[s_service_uart_rx_head] = ch;
00247     s_service_uart_rx_head = next;
00248 }
00249
00250 void USART1_IRQHandler(void)
00251 {
00252     HAL_UART_IRQHandler(&huart1);
00253 }
00254
00255 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
00256 {
00257     if ((huart != NULL) && (huart->Instance == USART1))
00258     {
00259         service_uart_rx_push_from_isr(s_service_uart_rx_byte);
00260         service_uart_rx_start();
00261     }
00262 }
00263

```

```

00264 void HAL_UART_ErrorCallback (UART_HandleTypeDef *huart)
00265 {
00266     if ((huart != NULL) && (huart->Instance == USART1))
00267     {
00268         s_service_uart_rx_overflow = true;
00269         service_uart_rx_start();
00270     }
00271 }
00272
00274 static void service_uart_handle_line(const char *line)
00275 {
00276     const char *p;
00277     char cmd[SERVICE_UART_TOKEN_MAX];
00278
00279     if (line == NULL)
00280     {
00281         return;
00282     }
00283
00284     p = service_uart_skip_space(line);
00285     if (service_uart_match_prefix(&p, "SEND:"))
00286     {
00287         if (!s_service_uart_unlocked)
00288         {
00289             printf("SERVICE LOCKED: use LOGIN <code>\r\n");
00290             return;
00291         }
00292         service_uart_handle_send_hex(p);
00293         return;
00294     }
00295
00296     p = line;
00297     if (!service_uart_next_token(&p, cmd, sizeof(cmd)))
00298     {
00299         return;
00300     }
00301
00302     if (service_uart_token_equals(cmd, "LOGIN") ||
00303         service_uart_token_equals(cmd, "AUTH") ||
00304         service_uart_token_equals(cmd, "UNLOCK"))
00305     {
00306         service_uart_handle_login(p);
00307     }
00308     else if (service_uart_token_equals(cmd, "LOGOUT") ||
00309             service_uart_token_equals(cmd, "LOCK"))
00310     {
00311         s_service_uart_unlocked = false;
00312         printf("SERVICE LOCKED\r\n");
00313     }
00314     else if (!s_service_uart_unlocked)
00315     {
00316         if (service_uart_token_equals(cmd, "HELP") || service_uart_token_equals(cmd, "?"))
00317         {
00318             service_uart_print_locked_help();
00319         }
00320         else
00321         {
00322             printf("SERVICE LOCKED: use LOGIN <code>\r\n");
00323         }
00324     }
00325     else if (service_uart_token_equals(cmd, "HELP") || service_uart_token_equals(cmd, "?"))
00326     {
00327         service_uart_print_help();
00328     }
00329     else if (service_uart_token_equals(cmd, "CFG") ||
00330             service_uart_token_equals(cmd, "CFG?") ||
00331             service_uart_token_equals(cmd, "STATUS"))
00332     {
00333         service_uart_print_cfg();
00334     }
00335     else if (service_uart_token_equals(cmd, "SEND") || service_uart_token_equals(cmd, "TX"))
00336     {
00337         service_uart_handle_send_hex(p);
00338     }
00339     else if (service_uart_token_equals(cmd, "TEXT"))
00340     {
00341         service_uart_handle_text(p);
00342     }
00343     else if (service_uart_token_equals(cmd, "MOD"))
00344     {
00345         service_uart_handle_mod(p);
00346     }
00347     else if (service_uart_token_equals(cmd, "PRESET"))
00348     {
00349         service_uart_handle_preset(p);
00350     }
00351     else if (service_uart_token_equals(cmd, "FREQ"))

```

```

00352     {
00353         service_uart_handle_freq(p);
00354     }
00355     else if (service_uart_token_equals(cmd, "BW"))
00356     {
00357         service_uart_handle_bw(p);
00358     }
00359     else if (service_uart_token_equals(cmd, "OPTION") || service_uart_token_equals(cmd, "SET"))
00360     {
00361         service_uart_handle_option(p);
00362     }
00363     else if (service_uart_token_equals(cmd, "FH"))
00364     {
00365         service_uart_handle_bool_cmd(p, radio_main_cmd_set_fh);
00366     }
00367     else if (service_uart_token_equals(cmd, "FH_PERIOD"))
00368     {
00369         service_uart_handle_u32_cmd(p, radio_main_cmd_set_fh_period);
00370     }
00371     else if (service_uart_token_equals(cmd, "CODING"))
00372     {
00373         service_uart_handle_bool_cmd(p, radio_main_cmd_set_coding);
00374     }
00375     else if (service_uart_token_equals(cmd, "AUTOPING"))
00376     {
00377         service_uart_handle_bool_cmd(p, radio_main_cmd_set_auto_ping);
00378     }
00379     else if (service_uart_token_equals(cmd, "AUTOPING_PERIOD"))
00380     {
00381         service_uart_handle_u32_cmd(p, radio_main_cmd_set_auto_ping_period);
00382     }
00383     else if (service_uart_token_equals(cmd, "AUTOPING_MODE"))
00384     {
00385         service_uart_handle_auto_ping_mode(p);
00386     }
00387     else if (service_uart_token_equals(cmd, "RESET"))
00388     {
00389         service_uart_print_radio_result(radio_main_cmd_reset_module());
00390     }
00391     else if (service_uart_token_equals(cmd, "SAVE"))
00392     {
00393         service_uart_handle_save();
00394     }
00395     else
00396     {
00397         printf("SERVICE ERR: unknown command, type HELP\r\n");
00398     }
00399 }
00400
00402 static void service_uart_print_locked_help(void)
00403 {
00404     printf("SERVICE UART locked\r\n");
00405     printf("  LOGIN <device address>\r\n");
00406     printf("  Address may be decimal, hex, or 0x-prefixed hex\r\n");
00407 }
00408
00410 static void service_uart_print_help(void)
00411 {
00412     printf("SERVICE UART commands:\r\n");
00413     printf("  HELP | CFG? | LOGOUT\r\n");
00414     printf("  SEND: <hex bytes> | TX <hex bytes>\r\n");
00415     printf("  TEXT <dst> <ascii up to 16B>\r\n");
00416     printf("  MOD <LORA|FSK|OOK|0|1|2>\r\n");
00417     printf("  PRESET <0..2> | FREQ <Hz> | BW <0..9>\r\n");
00418     printf("  OPTION <id> <value> (radio_main_option_t)\r\n");
00419     printf("  FH <ON|OFF> | FH_PERIOD <ms>\r\n");
00420     printf("  CODING <ON|OFF>\r\n");
00421     printf("  AUTOPING <ON|OFF> | AUTOPING_PERIOD <ms> | AUTOPING_MODE <FRAME|RAW>\r\n");
00422     printf("  RESET | SAVE\r\n");
00423 }
00424
00426 static void service_uart_print_cfg(void)
00427 {
00428     radio_main_runtime_cfg_t cfg;
00429
00430     if (!radio_main_get_runtime_cfg(&cfg))
00431     {
00432         printf("SERVICE ERR: cfg unavailable\r\n");
00433         return;
00434     }
00435
00436     printf("SERVICE CFG: node=0x%04lX mod=%s fh=%u coding=%u ap=%u ap_period=%lu ap_mode=%s\r\n",
00437         (unsigned long)radio_main_get_node_id(),
00438         service_uart_mod_text(cfg.active_modulation),
00439         cfg.fh_enabled ? 1U : 0U,
00440         cfg.coding_enabled ? 1U : 0U,
00441         cfg.auto_ping_enabled ? 1U : 0U,

```

```

00442         (unsigned long)cfg.auto_ping_period_ms,
00443         service_uart_ap_mode_text(cfg.auto_ping_mode));
00444
00445     if (cfg.active_modulation == RADIO_MAIN_MODULATION_FSK)
00446     {
00447         printf("SERVICE FSK: freq=%lu bitrate=%lu bw=%u shape=%u filter=%u pwr=%d pre=%u sync_len=%u
sync=0x%08lX addr=%u crc=%u white=%u\r\n",
00448             (unsigned long)cfg.fsk.frequency_hz,
00449             (unsigned long)cfg.fsk.bitrate_bps,
00450             (unsigned int)cfg.fsk.rx_bandwidth,
00451             (unsigned int)cfg.fsk.shaping,
00452             (unsigned int)cfg.fsk.filter,
00453             (int)cfg.fsk.tx_power_dbm,
00454             (unsigned int)cfg.fsk.preamble_len,
00455             (unsigned int)cfg.fsk.sync_word_len,
00456             (unsigned long)(cfg.fsk.sync_word & 0xFFFFFFFFUL),
00457             (unsigned int)cfg.fsk.address_filter,
00458             (unsigned int)cfg.fsk.crc_type,
00459             cfg.fsk.data_whitening ? 1U : 0U);
00460     }
00461     else if (cfg.active_modulation == RADIO_MAIN_MODULATION_OOK)
00462     {
00463         printf("SERVICE OOK: freq=%lu bitrate=%lu bw=%u pwr=%d pre=%u sync_len=%u sync=0x%08lX
thr=%u/%u\r\n",
00464             (unsigned long)cfg.ook.frequency_hz,
00465             (unsigned long)cfg.ook.bitrate_bps,
00466             (unsigned int)cfg.ook.rx_bandwidth,
00467             (int)cfg.ook.tx_power_dbm,
00468             (unsigned int)cfg.ook.preamble_len,
00469             (unsigned int)cfg.ook.sync_word_len,
00470             (unsigned long)cfg.ook.sync_word,
00471             (unsigned int)cfg.ook.threshold,
00472             (unsigned int)cfg.ook.threshold_value);
00473     }
00474     else
00475     {
00476         printf("SERVICE LORA: freq=%lu bw=%u sf=%u cr=4/%u pwr=%d pre=%u sync=0x%02X crc=%u hdr=%s
iq=%u\r\n",
00477             (unsigned long)cfg.lora.frequency_hz,
00478             (unsigned int)cfg.lora.bandwidth,
00479             (unsigned int)cfg.lora.spreading_factor,
00480             (unsigned int)cfg.lora.coding_rate,
00481             (int)cfg.lora.tx_power_dbm,
00482             (unsigned int)cfg.lora.preamble_len,
00483             (unsigned int)cfg.lora.sync_word,
00484             cfg.lora.crc_on ? 1U : 0U,
00485             cfg.lora.implicit_header ? "implicit" : "explicit",
00486             cfg.lora.invert_iq ? 1U : 0U);
00487     }
00488 }
00489
00491 static void service_uart_print_radio_result(bool ok)
00492 {
00493     char last_error[21];
00494
00495     if (ok)
00496     {
00497         printf("SERVICE OK\r\n");
00498         return;
00499     }
00500
00501     if (radio_main_get_last_error_text(last_error, sizeof(last_error)) && (last_error[0] != '\0'))
00502     {
00503         printf("SERVICE ERR: %s\r\n", last_error);
00504     }
00505     else
00506     {
00507         printf("SERVICE ERR: command failed\r\n");
00508     }
00509 }
00510
00512 static void service_uart_handle_login(const char *args)
00513 {
00514     uint32_t code;
00515     uint32_t expected;
00516
00517     if (!service_uart_parse_u32_arg(args, &code))
00518     {
00519         printf("SERVICE ERR: use LOGIN <device address>\r\n");
00520         return;
00521     }
00522
00523     expected = service_uart_get_service_code();
00524     if ((expected != 0U) && (code == expected))
00525     {
00526         s_service_uart_unlocked = true;
00527         printf("SERVICE OK: panel enabled\r\n");

```



```

00528     service_uart_print_help();
00529 }
00530 else
00531 {
00532     printf("SERVICE ERR: bad code\r\n");
00533 }
00534 }
00535
00537 static void service_uart_handle_send_hex(const char *payload)
00538 {
00539     const char *error = NULL;
00540     uint8_t len = 0U;
00541
00542     if (!service_uart_parse_hex_bytes(payload, s_service_uart_tx_bytes, &len, &error))
00543     {
00544         printf("SERVICE ERR: %s\r\n", (error != NULL) ? error : "bad hex");
00545         return;
00546     }
00547
00548     if (radio_main_cmd_send_raw(s_service_uart_tx_bytes, len))
00549     {
00550         printf("SERVICE SEND OK: %u B\r\n", (unsigned int)len);
00551         return;
00552     }
00553
00554     service_uart_print_radio_result(false);
00555 }
00556
00558 static void service_uart_handle_text(const char *args)
00559 {
00560     const char *text;
00561     uint32_t dst;
00562     char token[SERVICE_UART_TOKEN_MAX];
00563
00564     if (!service_uart_next_token(&args, token, sizeof(token)) ||
00565         !service_uart_parse_u32_arg(token, &dst))
00566     {
00567         printf("SERVICE ERR: use TEXT <dst> <ascii>\r\n");
00568         return;
00569     }
00570
00571     text = service_uart_skip_space(args);
00572     if ((text == NULL) || (*text == '\0'))
00573     {
00574         printf("SERVICE ERR: empty text\r\n");
00575         return;
00576     }
00577
00578     service_uart_print_radio_result(radio_main_cmd_send_user_text(text, dst));
00579 }
00580
00582 static void service_uart_handle_mod(const char *args)
00583 {
00584     radio_main_modulation_t mod;
00585
00586     if (!service_uart_parse_mod_arg(args, &mod))
00587     {
00588         printf("SERVICE ERR: use MOD <LORA|FSK|OOK|0|1|2>\r\n");
00589         return;
00590     }
00591
00592     service_uart_print_radio_result(radio_main_cmd_set_modulation((uint8_t)mod));
00593 }
00594
00596 static void service_uart_handle_preset(const char *args)
00597 {
00598     uint32_t preset;
00599
00600     if (!service_uart_parse_u32_arg(args, &preset) || (preset > 2UL))
00601     {
00602         printf("SERVICE ERR: use PRESET <0..2>\r\n");
00603         return;
00604     }
00605
00606     service_uart_print_radio_result(radio_main_cmd_set_lora_preset((uint8_t)preset));
00607 }
00608
00610 static void service_uart_handle_freq(const char *args)
00611 {
00612     uint32_t freq;
00613
00614     if (!service_uart_parse_u32_arg(args, &freq))
00615     {
00616         printf("SERVICE ERR: use FREQ <Hz>\r\n");
00617         return;
00618     }
00619 }

```

```

00620     service_uart_print_radio_result (radio_main_cmd_set_modulation_freq(freq));
00621 }
00622
00624 static void service_uart_handle_bw(const char *args)
00625 {
00626     uint32_t bw;
00627
00628     if (!service_uart_parse_u32_arg(args, &bw) || (bw > 9UL))
00629     {
00630         printf("SERVICE ERR: use BW <0..9>\r\n");
00631         return;
00632     }
00633
00634     service_uart_print_radio_result (radio_main_cmd_set_modulation_bw((uint8_t)bw));
00635 }
00636
00638 static void service_uart_handle_option(const char *args)
00639 {
00640     uint32_t option;
00641     uint32_t value;
00642     char token[SERVICE_UART_TOKEN_MAX];
00643
00644     if (!service_uart_next_token(&args, token, sizeof(token)) ||
00645         !service_uart_parse_u32_arg(token, &option) ||
00646         (option > (uint32_t)RADIO_MAIN_OPTION_OOK_RESET_DEFAULTS) ||
00647         !service_uart_parse_u32_arg(args, &value))
00648     {
00649         printf("SERVICE ERR: use OPTION <id> <value>\r\n");
00650         return;
00651     }
00652
00653     service_uart_print_radio_result (radio_main_cmd_set_option((radio_main_option_t)option, value));
00654 }
00655
00657 static void service_uart_handle_bool_cmd(const char *args, bool (*setter)(bool))
00658 {
00659     bool value;
00660
00661     if ((setter == NULL) || !service_uart_parse_bool_arg(args, &value))
00662     {
00663         printf("SERVICE ERR: use <cmd> <ON|OFF>\r\n");
00664         return;
00665     }
00666
00667     service_uart_print_radio_result (setter(value));
00668 }
00669
00671 static void service_uart_handle_u32_cmd(const char *args, bool (*setter)(uint32_t))
00672 {
00673     uint32_t value;
00674
00675     if ((setter == NULL) || !service_uart_parse_u32_arg(args, &value))
00676     {
00677         printf("SERVICE ERR: use <cmd> <value>\r\n");
00678         return;
00679     }
00680
00681     service_uart_print_radio_result (setter(value));
00682 }
00683
00685 static void service_uart_handle_auto_ping_mode(const char *args)
00686 {
00687     char token[SERVICE_UART_TOKEN_MAX];
00688     radio_main_auto_ping_mode_t mode;
00689
00690     if (!service_uart_next_token(&args, token, sizeof(token)))
00691     {
00692         printf("SERVICE ERR: use AUTOPING_MODE <FRAME|RAW>\r\n");
00693         return;
00694     }
00695
00696     if (service_uart_token_equals(token, "FRAME") || service_uart_token_equals(token, "0"))
00697     {
00698         mode = RADIO_MAIN_AUTO_PING_FRAME;
00699     }
00700     else if (service_uart_token_equals(token, "RAW") || service_uart_token_equals(token, "1"))
00701     {
00702         mode = RADIO_MAIN_AUTO_PING_RAW;
00703     }
00704     else
00705     {
00706         printf("SERVICE ERR: use AUTOPING_MODE <FRAME|RAW>\r\n");
00707         return;
00708     }
00709
00710     service_uart_print_radio_result (radio_main_cmd_set_auto_ping_mode(mode));
00711 }

```

```

00712
00714 static void service_uart_handle_save(void)
00715 {
00716     radio_main_runtime_cfg_t cfg;
00717
00718     if (!radio_main_get_runtime_cfg(&cfg))
00719     {
00720         printf("SERVICE ERR: cfg unavailable\r\n");
00721         return;
00722     }
00723
00724     if (security_main_cmd_set_radio_runtime_cfg(&cfg))
00725     {
00726         printf("SERVICE OK: saved\r\n");
00727     }
00728     else
00729     {
00730         printf("SERVICE ERR: save failed\r\n");
00731     }
00732 }
00733
00735 static bool service_uart_parse_hex_bytes(const char *text,
00736                                         uint8_t *data,
00737                                         uint8_t *len_out,
00738                                         const char **error_out)
00739 {
00740     const char *p = service_uart_skip_space(text);
00741     uint16_t count = 0U;
00742
00743     if ((data == NULL) || (len_out == NULL))
00744     {
00745         if (error_out != NULL)
00746         {
00747             *error_out = "internal error";
00748         }
00749         return false;
00750     }
00751
00752     while ((p != NULL) && (*p != '\0'))
00753     {
00754         int high;
00755         int low;
00756
00757         while (isspace((unsigned char)*p) || (*p == ',') || (*p == ';'))
00758         {
00759             p++;
00760         }
00761         if (*p == '\0')
00762         {
00763             break;
00764         }
00765         if ((*p == '0') && ((p[1] == 'x') || (p[1] == 'X')))
00766         {
00767             p += 2;
00768         }
00769
00770         high = service_uart_hex_nibble(*p);
00771         if (high < 0)
00772         {
00773             if (error_out != NULL)
00774             {
00775                 *error_out = "bad hex byte";
00776             }
00777             return false;
00778         }
00779         p++;
00780
00781         low = service_uart_hex_nibble(*p);
00782         if (low < 0)
00783         {
00784             if (error_out != NULL)
00785             {
00786                 *error_out = "hex byte needs 2 digits";
00787             }
00788             return false;
00789         }
00790         p++;
00791
00792         if (isxdigit((unsigned char)*p))
00793         {
00794             if (error_out != NULL)
00795             {
00796                 *error_out = "hex byte too long";
00797             }
00798             return false;
00799         }
00800         if (count >= RADIO_LIB_MAX_PAYLOAD)

```

```

00801     {
00802         if (error_out != NULL)
00803         {
00804             *error_out = "too many bytes";
00805         }
00806         return false;
00807     }
00808
00809     data[count] = (uint8_t)((uint8_t)high << 4) | (uint8_t)low;
00810     count++;
00811
00812     if ((*p != '\0') &&
00813         !isspace((unsigned char)*p) &&
00814         (*p != ',') &&
00815         (*p != ';'))
00816     {
00817         if (error_out != NULL)
00818         {
00819             *error_out = "missing separator";
00820         }
00821         return false;
00822     }
00823 }
00824
00825 if (count == 0U)
00826 {
00827     if (error_out != NULL)
00828     {
00829         *error_out = "no bytes";
00830     }
00831     return false;
00832 }
00833
00834 *len_out = (uint8_t)count;
00835 return true;
00836 }
00837
00838 static bool service_uart_parse_u32_token(const char *token, uint32_t *value_out)
00839 {
00840     {
00841         const char *p;
00842         const char *scan;
00843         uint32_t value = 0UL;
00844         uint32_t base = 10UL;
00845
00846         if ((token == NULL) || (value_out == NULL))
00847         {
00848             return false;
00849         }
00850
00851         p = token;
00852         if ((p[0] == '0') && ((p[1] == 'x') || (p[1] == 'X')))
00853         {
00854             base = 16UL;
00855             p += 2;
00856         }
00857         else
00858         {
00859             scan = p;
00860             while (*scan != '\0')
00861             {
00862                 if (((*scan >= 'A') && (*scan <= 'F')) ||
00863                     ((*scan >= 'a') && (*scan <= 'f')))
00864                 {
00865                     base = 16UL;
00866                     break;
00867                 }
00868                 scan++;
00869             }
00870         }
00871         if (*p == '\0')
00872         {
00873             return false;
00874         }
00875
00876         while (*p != '\0')
00877         {
00878             int digit;
00879
00880             if ((*p >= '0') && (*p <= '9'))
00881             {
00882                 digit = (int)(*p - '0');
00883             }
00884             else if (base == 16UL)
00885             {
00886                 digit = service_uart_hex_nibble(*p);
00887             }
00888             else

```

```

00889     {
00890         return false;
00891     }
00892
00893     if ((digit < 0) || ((uint32_t)digit >= base) ||
00894         (value > ((UINT32_MAX - (uint32_t)digit) / base)))
00895     {
00896         return false;
00897     }
00898
00899     value = (value * base) + (uint32_t)digit;
00900     p++;
00901 }
00902
00903 *value_out = value;
00904 return true;
00905 }
00906
00907 static bool service_uart_parse_u32_arg(const char *args, uint32_t *value_out)
00908 {
00909     char token[SERVICE_UART_TOKEN_MAX];
00910
00911     if ((args == NULL) || (value_out == NULL) ||
00912         !service_uart_next_token(&args, token, sizeof(token)))
00913     {
00914         return false;
00915     }
00916
00917     return service_uart_parse_u32_token(token, value_out);
00918 }
00919
00920 static bool service_uart_parse_bool_arg(const char *args, bool *value_out)
00921 {
00922     char token[SERVICE_UART_TOKEN_MAX];
00923
00924     if ((value_out == NULL) ||
00925         !service_uart_next_token(&args, token, sizeof(token)))
00926     {
00927         return false;
00928     }
00929
00930     if (service_uart_token_equals(token, "ON") ||
00931         service_uart_token_equals(token, "1") ||
00932         service_uart_token_equals(token, "TRUE") ||
00933         service_uart_token_equals(token, "ENABLE"))
00934     {
00935         *value_out = true;
00936         return true;
00937     }
00938
00939     if (service_uart_token_equals(token, "OFF") ||
00940         service_uart_token_equals(token, "0") ||
00941         service_uart_token_equals(token, "FALSE") ||
00942         service_uart_token_equals(token, "DISABLE"))
00943     {
00944         *value_out = false;
00945         return true;
00946     }
00947
00948     return false;
00949 }
00950
00951 static bool service_uart_parse_mod_arg(const char *args, radio_main_modulation_t *mod_out)
00952 {
00953     char token[SERVICE_UART_TOKEN_MAX];
00954
00955     if ((mod_out == NULL) ||
00956         !service_uart_next_token(&args, token, sizeof(token)))
00957     {
00958         return false;
00959     }
00960
00961     if (service_uart_token_equals(token, "LORA") || service_uart_token_equals(token, "0"))
00962     {
00963         *mod_out = RADIO_MAIN_MODULATION_LORA;
00964         return true;
00965     }
00966
00967     if (service_uart_token_equals(token, "FSK") ||
00968         service_uart_token_equals(token, "GFSK") ||
00969         service_uart_token_equals(token, "1"))
00970     {
00971         *mod_out = RADIO_MAIN_MODULATION_FSK;
00972         return true;
00973     }
00974
00975     if (service_uart_token_equals(token, "OOK") || service_uart_token_equals(token, "2"))
00976     {
00977         *mod_out = RADIO_MAIN_MODULATION_OOK;
00978     }

```

```
00979         return true;
00980     }
00981
00982     return false;
00983 }
00984
00986 static bool service_uart_next_token(const char **cursor, char *out, uint8_t out_size)
00987 {
00988     const char *p;
00989     uint8_t len = 0U;
00990
00991     if ((cursor == NULL) || (*cursor == NULL) || (out == NULL) || (out_size == 0U))
00992     {
00993         return false;
00994     }
00995
00996     p = service_uart_skip_space(*cursor);
00997     if ((p == NULL) || (*p == '\0'))
00998     {
00999         return false;
01000     }
01001
01002     while ((*p != '\0') && !isspace((unsigned char)*p))
01003     {
01004         if (len < (uint8_t)(out_size - 1U))
01005         {
01006             out[len] = *p;
01007             len++;
01008         }
01009         p++;
01010     }
01011
01012     out[len] = '\0';
01013     *cursor = p;
01014     return (len > 0U);
01015 }
01016
01018 static const char *service_uart_skip_space(const char *text)
01019 {
01020     while ((text != NULL) && isspace((unsigned char)*text))
01021     {
01022         text++;
01023     }
01024
01025     return text;
01026 }
01027
01029 static bool service_uart_token_equals(const char *a, const char *b)
01030 {
01031     if ((a == NULL) || (b == NULL))
01032     {
01033         return false;
01034     }
01035
01036     while ((*a != '\0') && (*b != '\0'))
01037     {
01038         if (toupper((unsigned char)*a) != toupper((unsigned char)*b))
01039         {
01040             return false;
01041         }
01042         a++;
01043         b++;
01044     }
01045
01046     return ((*a == '\0') && (*b == '\0'));
01047 }
01048
01050 static bool service_uart_match_prefix(const char **cursor, const char *prefix)
01051 {
01052     const char *p;
01053
01054     if ((cursor == NULL) || (*cursor == NULL) || (prefix == NULL))
01055     {
01056         return false;
01057     }
01058
01059     p = service_uart_skip_space(*cursor);
01060     while (*prefix != '\0')
01061     {
01062         if ((*p == '\0') ||
01063             (toupper((unsigned char)*p) != toupper((unsigned char)*prefix)))
01064         {
01065             return false;
01066         }
01067         p++;
01068         prefix++;
01069     }
01070 }
```

```

01070
01071     *cursor = p;
01072     return true;
01073 }
01074
01076 static int service_uart_hex_nibble(char c)
01077 {
01078     if ((c >= '0') && (c <= '9'))
01079     {
01080         return (int)(c - '0');
01081     }
01082     if ((c >= 'A') && (c <= 'F'))
01083     {
01084         return (int)(c - 'A' + 10);
01085     }
01086     if ((c >= 'a') && (c <= 'f'))
01087     {
01088         return (int)(c - 'a' + 10);
01089     }
01090
01091     return -1;
01092 }
01093
01095 static uint32_t service_uart_get_service_code(void)
01096 {
01097     uint32_t node_id = radio_main_get_node_id();
01098
01099     if (node_id == 0U)
01100     {
01101         node_id = laviet_local_node_id();
01102     }
01103
01104     return node_id;
01105 }
01106
01108 static const char *service_uart_mod_text(radio_main_modulation_t modulation)
01109 {
01110     switch (modulation)
01111     {
01112         case RADIO_MAIN_MODULATION_FSK:
01113             return "FSK";
01114         case RADIO_MAIN_MODULATION_OOK:
01115             return "OOK";
01116         case RADIO_MAIN_MODULATION_LORA:
01117             default:
01118                 return "LORA";
01119     }
01120 }
01121
01123 static const char *service_uart_ap_mode_text(radio_main_auto_ping_mode_t mode)
01124 {
01125     switch (mode)
01126     {
01127         case RADIO_MAIN_AUTO_PING_RAW:
01128             return "RAW";
01129         case RADIO_MAIN_AUTO_PING_FRAME:
01130             default:
01131                 return "FRAME";
01132     }
01133 }
01134
01135 #else
01136
01137 void service_uart_create_task(void)
01138 {
01139 }
01140
01141 #endif /* SERVICE_UART */

```

12.83 Dokumentacja pliku Core/App/service.h

Optional UART service console task entry point.

```
#include "pager_config.h"
```

Funkcje

- void `service_uart_create_task` (void)

Create the UART service task when `SERVICE_UART` is enabled.

12.83.1 Opis szczegółowy

Optional UART service console task entry point.
Definicja w pliku [service.h](#).

12.83.2 Dokumentacja funkcji

12.83.2.1 `service_uart_create_task()`

```
void service_uart_create_task (
    void )
```

Create the UART service task when `SERVICE_UART` is enabled.

Definicja w linii 91 pliku [service.c](#).

Odwołania w [app_freertos_init\(\)](#).

12.84 `service.h`

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_SERVICE_H_
00007 #define APP_SERVICE_H_
00008
00009 #include "pager_config.h"
00010
00011 #ifdef __cplusplus
00012 extern "C" {
00013 #endif
00014
00018 void service_uart_create_task(void);
00019
00020 #ifdef __cplusplus
00021 }
00022 #endif
00023
00024 #endif /* APP_SERVICE_H_ */
```

12.85 Dokumentacja pliku `Core/App/st33ktpm2x_lib/st33ktpm2x.c`

ST33KTPM2X driver implementation (TIS over I2C).

```
#include "st33ktpm2x.h"
#include "app_delay.h"
#include "main.h"
#include <stdio.h>
#include <string.h>
```

Definicje

- `#define ST33_REG_LOC_SEL` 0x00U
- `#define ST33_REG_ACCESS` 0x04U
- `#define ST33_REG_STS` 0x18U
- `#define ST33_REG_DATA_FIFO` 0x24U
- `#define ST33_REG_INTERFACE_CAPABILITY` 0x30U
- `#define ST33_REG_DID_VID` 0x48U
- `#define ST33_REG_RID` 0x4CU
- `#define ST33_I2C_MAX_RETRIES` 3U
- `#define ST33_ACCESS_VALID` 0x80U
- `#define ST33_ACCESS_ACTIVE_LOCALITY` 0x20U
- `#define ST33_ACCESS_REQUEST_USE` 0x02U
- `#define ST33_ACCESS_READ_ZERO` 0x48U
- `#define ST33_STS_VALID` 0x80U
- `#define ST33_STS_COMMAND_READY` 0x40U

- `#define ST33_STS_TPM_GO` 0x20U
- `#define ST33_STS_DATA_AVAIL` 0x10U
- `#define ST33_STS_EXPECT` 0x08U
- `#define ST33_STS_READ_ZERO` 0x23U
- `#define ST33_TPM_HEADER_SIZE` 10U
- `#define ST33_TPM_RH_OWNER` 0x40000001UL
- `#define ST33_TPM_RH_PLATFORM` 0x4000000CUL
- `#define ST33_TPM_RS_PW` 0x40000009UL
- `#define ST33_TPM_PW_AUTH_SIZE` 9U
- `#define ST33_TPM_RANDOM_MAX_BYTES` 256U
- `#define ST33_TPM_NV_TRANSFER_MAX_BYTES` 512U
- `#define ST33_TRACE(...)`

Funkcje

- static uint16_t `st33_min_u16` (uint16_t a, uint16_t b)
Internal helper: `st33_min_u16`.
- static uint16_t `st33_be16_read` (const uint8_t *buf)
Internal helper: `st33_be16_read`.
- static uint32_t `st33_be32_read` (const uint8_t *buf)
Internal helper: `st33_be32_read`.
- static uint32_t `st33_le32_read` (const uint8_t *buf)
Internal helper: `st33_le32_read`.
- static bool `st33_identity_is_valid` (uint32_t did_vid, uint8_t rid)
Check whether DID_VID/RID values look like an ST33 TPM response.
- static const char * `st33_status_text` (st33ktpm2x_status_t rc)
Internal helper: `st33_status_text`.
- static void `st33_be16_write` (uint8_t *buf, uint16_t v)
Internal helper: `st33_be16_write`.
- static void `st33_be32_write` (uint8_t *buf, uint32_t v)
Internal helper: `st33_be32_write`.
- static uint16_t `st33_write_empty_password_auth` (uint8_t *buf)
Marshal an empty-password authorization session into a TPM2 command.
- static st33ktpm2x_status_t `st33_i2c_read` (st33ktpm2x_t *ctx, uint8_t reg, uint8_t *data, uint16_t len)
Internal helper: `st33_i2c_read`.
- static st33ktpm2x_status_t `st33_i2c_write` (st33ktpm2x_t *ctx, uint8_t reg, const uint8_t *data, uint16_t len)
Internal helper: `st33_i2c_write`.
- static st33ktpm2x_status_t `st33_read_access` (st33ktpm2x_t *ctx, uint8_t *access)
Internal helper: `st33_read_access`.
- static st33ktpm2x_status_t `st33_write_access` (st33ktpm2x_t *ctx, uint8_t value)
Internal helper: `st33_write_access`.
- static st33ktpm2x_status_t `st33_write_loc_sel` (st33ktpm2x_t *ctx, uint8_t locality)
Internal helper: `st33_write_loc_sel`.
- static st33ktpm2x_status_t `st33_read_sts` (st33ktpm2x_t *ctx, uint8_t *sts, uint16_t *burst_count)
Internal helper: `st33_read_sts`.
- static st33ktpm2x_status_t `st33_write_sts` (st33ktpm2x_t *ctx, uint8_t value)
Internal helper: `st33_write_sts`.
- static st33ktpm2x_status_t `st33_wait_access_bits` (st33ktpm2x_t *ctx, uint8_t mask, uint8_t expected, uint32_t timeout_ms)
Poll TPM_ACCESS until required bits are set and forbidden bits are clear.
- static st33ktpm2x_status_t `st33_wait_sts_bits` (st33ktpm2x_t *ctx, uint8_t mask, uint8_t expected, uint32_t timeout_ms)

- Poll TPM_STS until required status bits are set and forbidden bits are clear.*
- static `st33ktpm2x_status_t st33_wait_burst` (`st33ktpm2x_t *ctx`, `uint16_t *burst_out`, `uint8_t require_data_↵` `avail`)
- Wait for a non-zero TPM FIFO burst count, optionally requiring data availability.*
- static void `st33_trace_tis` (`st33ktpm2x_t *ctx`, `const char *tag`)
- Internal helper: `st33_trace_tis`.*
- static `st33ktpm2x_status_t st33_trace_fail` (`st33ktpm2x_t *ctx`, `const char *stage`, `st33ktpm2x_status_t rc`, `bool release_locality`)
- Internal helper: `st33_trace_fail`.*
- static `st33ktpm2x_status_t st33_fifo_write` (`st33ktpm2x_t *ctx`, `const uint8_t *data`, `uint16_t len`)
- Internal helper: `st33_fifo_write`.*
- static `st33ktpm2x_status_t st33_fifo_read` (`st33ktpm2x_t *ctx`, `uint8_t *data`, `uint16_t len`)
- Internal helper: `st33_fifo_read`.*
- static `st33ktpm2x_status_t st33_read_response_stream` (`st33ktpm2x_t *ctx`, `uint8_t *dst`, `uint16_t len`)
- Internal helper: `st33_read_response_stream`.*
- void `st33ktpm2x_default_cfg` (`st33ktpm2x_cfg_t *cfg`, `I2C_HandleTypeDef *hi2c`)
- Fill config with safe defaults (including PB0 reset pin defaults from `main.h`).*
- `st33ktpm2x_status_t st33ktpm2x_init` (`st33ktpm2x_t *ctx`, `const st33ktpm2x_cfg_t *cfg`)
- Initialize TPM driver context.*
- `st33ktpm2x_status_t st33ktpm2x_deinit` (`st33ktpm2x_t *ctx`)
- Deinitialize context and release locality if needed.*
- `st33ktpm2x_status_t st33ktpm2x_hard_reset` (`st33ktpm2x_t *ctx`)
- Perform hardware reset pulse on TPM reset pin.*
- `st33ktpm2x_status_t st33ktpm2x_davint_is_asserted` (`st33ktpm2x_t *ctx`, `bool *asserted`)
- Read TPM_DAVINT# / PIRQ state.*
- `st33ktpm2x_status_t st33ktpm2x_pp_is_pressed` (`st33ktpm2x_t *ctx`, `bool *pressed`)
- Read state of optional Physical Presence button.*
- `st33ktpm2x_status_t st33ktpm2x_pp_wait_pressed` (`st33ktpm2x_t *ctx`, `uint32_t timeout_ms`)
- Wait until Physical Presence button becomes active.*
- `st33ktpm2x_status_t st33ktpm2x_request_locality0` (`st33ktpm2x_t *ctx`)
- Acquire TPM locality 0.*
- `st33ktpm2x_status_t st33ktpm2x_release_locality0` (`st33ktpm2x_t *ctx`)
- Release TPM locality 0.*
- `st33ktpm2x_status_t st33ktpm2x_read_identity` (`st33ktpm2x_t *ctx`, `uint32_t *did_vid`, `uint8_t *rid`)
- Read TPM DID_VID and RID registers.*
- `st33ktpm2x_status_t st33ktpm2x_transceive` (`st33ktpm2x_t *ctx`, `const uint8_t *command`, `uint16_t ↵` `command_len`, `uint8_t *response`, `uint16_t response_capacity`, `uint16_t *response_len`, `uint32_t *tpm_rc`)
- Send one raw TPM command and receive full response.*
- `st33ktpm2x_status_t st33ktpm2x_tpm2_startup` (`st33ktpm2x_t *ctx`, `uint16_t startup_type`, `uint32_t *tpm_rc`)
- Execute TPM2_Startup.*
- `st33ktpm2x_status_t st33ktpm2x_tpm2_self_test` (`st33ktpm2x_t *ctx`, `bool full_test`, `uint32_t *tpm_rc`)
- Execute TPM2_SelfTest.*
- `st33ktpm2x_status_t st33ktpm2x_tpm2_get_random` (`st33ktpm2x_t *ctx`, `uint16_t requested_bytes`, `uint8_t ↵` `*random_out`, `uint16_t random_capacity`, `uint16_t *random_len`, `uint32_t *tpm_rc`)
- Execute TPM2_GetRandom.*
- `st33ktpm2x_status_t st33ktpm2x_tpm2_get_capability` (`st33ktpm2x_t *ctx`, `uint32_t capability`, `uint32_t ↵` `property`, `uint32_t property_count`, `uint8_t *payload_out`, `uint16_t payload_capacity`, `uint16_t *payload_len`, `bool ↵` `*more_data`, `uint32_t *tpm_rc`)
- Execute TPM2_GetCapability and return raw capability payload.*
- `st33ktpm2x_status_t st33ktpm2x_tpm2_pcr_read_sha256` (`st33ktpm2x_t *ctx`, `uint8_t pcr_index`, `uint8_t ↵` `digest_sha256_out[32]`, `uint32_t *update_counter_out`, `uint32_t *tpm_rc`)
- Execute TPM2_PCR_Read for one SHA-256 PCR index.*

- `st33ktpm2x_status_t st33ktpm2x_tpm2_start_auth_session (st33ktpm2x_t *ctx, uint32_t *session_handle, uint32_t *tpm_rc)`
Start TPM2 policy/auth session helper (scaffold API).
- `st33ktpm2x_status_t st33ktpm2x_tpm2_policy_pcr (st33ktpm2x_t *ctx, uint32_t session_handle, uint8_t pcr_index, uint32_t *tpm_rc)`
Apply PolicyPCR in an existing policy session (scaffold API).
- `st33ktpm2x_status_t st33ktpm2x_tpm2_policy_physical_presence (st33ktpm2x_t *ctx, uint32_t session_handle, uint32_t *tpm_rc)`
Apply Physical Presence policy branch (scaffold API).
- `st33ktpm2x_status_t st33ktpm2x_tpm2_policy_command_code (st33ktpm2x_t *ctx, uint32_t session_handle, uint32_t command_code, uint32_t *tpm_rc)`
Apply PolicyCommandCode in an existing policy session (scaffold API).
- `st33ktpm2x_status_t st33ktpm2x_tpm2_policy_or (st33ktpm2x_t *ctx, uint32_t session_handle, const uint8_t *digest_list, uint8_t digest_count, uint8_t digest_size, uint32_t *tpm_rc)`
Apply PolicyOR branch combiner (scaffold API).
- `st33ktpm2x_status_t st33ktpm2x_tpm2_nv_define (st33ktpm2x_t *ctx, uint32_t nv_index, uint16_t data_size, uint32_t attributes, uint32_t *tpm_rc)`
Define NV index using owner empty-password authorization.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_nv_read (st33ktpm2x_t *ctx, uint32_t nv_index, uint16_t offset, uint8_t *out_data, uint16_t out_capacity, uint16_t *out_len, uint32_t *tpm_rc)`
Read bytes from NV index using owner empty-password authorization.
- `static st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write_auth (st33ktpm2x_t *ctx, uint32_t auth_handle, uint32_t nv_index, uint16_t offset, const uint8_t *data, uint16_t data_len, uint32_t *tpm_rc)`
Internal helper: st33_tpm2_nv_write_auth.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write (st33ktpm2x_t *ctx, uint32_t nv_index, uint16_t offset, const uint8_t *data, uint16_t data_len, uint32_t *tpm_rc)`
Write bytes to NV index using owner empty-password authorization.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write_platform_pp (st33ktpm2x_t *ctx, uint32_t nv_index, uint16_t offset, const uint8_t *data, uint16_t data_len, uint32_t *tpm_rc)`
Write bytes to NV index using platform authorization with hardware Physical Presence.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_flush_context (st33ktpm2x_t *ctx, uint32_t handle, uint32_t *tpm_rc)`
Flush transient context/session handle (scaffold API).

Zmienne

- `static uint8_t s_st33_i2c_tx [1U+ST33KTPM2X_CMD_MAX_BYTES]`

12.85.1 Opis szczegółowy

ST33KTPM2X driver implementation (TIS over I2C).
Definicja w pliku `st33ktpm2x.c`.

12.85.2 Dokumentacja definicji

12.85.2.1 ST33_ACCESS_ACTIVE_LOCALITY

```
#define ST33_ACCESS_ACTIVE_LOCALITY 0x20U
```

Definicja w linii 27 pliku `st33ktpm2x.c`.

12.85.2.2 ST33_ACCESS_READ_ZERO

```
#define ST33_ACCESS_READ_ZERO 0x48U
```

Definicja w linii 29 pliku `st33ktpm2x.c`.

12.85.2.3 ST33_ACCESS_REQUEST_USE

```
#define ST33_ACCESS_REQUEST_USE 0x02U
```

Definicja w linii 28 pliku [st33ktpm2x.c](#).

12.85.2.4 ST33_ACCESS_VALID

```
#define ST33_ACCESS_VALID 0x80U
```

Definicja w linii 26 pliku [st33ktpm2x.c](#).

12.85.2.5 ST33_I2C_MAX_RETRIES

```
#define ST33_I2C_MAX_RETRIES 3U
```

Definicja w linii 23 pliku [st33ktpm2x.c](#).

12.85.2.6 ST33_REG_ACCESS

```
#define ST33_REG_ACCESS 0x04U
```

Definicja w linii 16 pliku [st33ktpm2x.c](#).

12.85.2.7 ST33_REG_DATA_FIFO

```
#define ST33_REG_DATA_FIFO 0x24U
```

Definicja w linii 18 pliku [st33ktpm2x.c](#).

12.85.2.8 ST33_REG_DID_VID

```
#define ST33_REG_DID_VID 0x48U
```

Definicja w linii 20 pliku [st33ktpm2x.c](#).

12.85.2.9 ST33_REG_INTERFACE_CAPABILITY

```
#define ST33_REG_INTERFACE_CAPABILITY 0x30U
```

Definicja w linii 19 pliku [st33ktpm2x.c](#).

12.85.2.10 ST33_REG_LOC_SEL

```
#define ST33_REG_LOC_SEL 0x00U
```

Definicja w linii 15 pliku [st33ktpm2x.c](#).

12.85.2.11 ST33_REG_RID

```
#define ST33_REG_RID 0x4CU
```

Definicja w linii 21 pliku [st33ktpm2x.c](#).

12.85.2.12 ST33_REG_STS

```
#define ST33_REG_STS 0x18U
```

Definicja w linii 17 pliku [st33ktpm2x.c](#).

12.85.2.13 ST33_STS_COMMAND_READY

```
#define ST33_STS_COMMAND_READY 0x40U
```

Definicja w linii 33 pliku [st33ktpm2x.c](#).

12.85.2.14 ST33_STS_DATA_AVAIL

```
#define ST33_STS_DATA_AVAIL 0x10U
```

Definicja w linii 35 pliku [st33ktpm2x.c](#).

12.85.2.15 ST33_STS_EXPECT

```
#define ST33_STS_EXPECT 0x08U
```

Definicja w linii 36 pliku [st33ktpm2x.c](#).

12.85.2.16 ST33_STS_READ_ZERO

```
#define ST33_STS_READ_ZERO 0x23U
```

Definicja w linii 37 pliku [st33ktpm2x.c](#).

12.85.2.17 ST33_STS_TPM_GO

```
#define ST33_STS_TPM_GO 0x20U
```

Definicja w linii 34 pliku [st33ktpm2x.c](#).

12.85.2.18 ST33_STS_VALID

```
#define ST33_STS_VALID 0x80U
```

Definicja w linii 32 pliku [st33ktpm2x.c](#).

12.85.2.19 ST33_TPM_HEADER_SIZE

```
#define ST33_TPM_HEADER_SIZE 10U
```

Definicja w linii 39 pliku [st33ktpm2x.c](#).

12.85.2.20 ST33_TPM_NV_TRANSFER_MAX_BYTES

```
#define ST33_TPM_NV_TRANSFER_MAX_BYTES 512U
```

Definicja w linii 45 pliku [st33ktpm2x.c](#).

12.85.2.21 ST33_TPM_PW_AUTH_SIZE

```
#define ST33_TPM_PW_AUTH_SIZE 9U
```

Definicja w linii 43 pliku [st33ktpm2x.c](#).

12.85.2.22 ST33_TPM_RANDOM_MAX_BYTES

```
#define ST33_TPM_RANDOM_MAX_BYTES 256U
```

Definicja w linii 44 pliku [st33ktpm2x.c](#).

12.85.2.23 ST33_TPM_RH_OWNER

```
#define ST33_TPM_RH_OWNER 0x40000001UL
```

Definicja w linii 40 pliku [st33ktpm2x.c](#).

12.85.2.24 ST33_TPM_RH_PLATFORM

```
#define ST33_TPM_RH_PLATFORM 0x4000000CUL
```

Definicja w linii 41 pliku [st33ktpm2x.c](#).

12.85.2.25 ST33_TPM_RS_PW

```
#define ST33_TPM_RS_PW 0x40000009UL
```

Definicja w linii 42 pliku [st33ktpm2x.c](#).

12.85.2.26 ST33_TRACE

```
#define ST33_TRACE(  
    ...)
```

Wartość:

```
do { if (0) { printf(__VA_ARGS__); } } while (0)
```

Definicja w linii 50 pliku [st33ktpm2x.c](#).

12.85.3 Dokumentacja funkcji

12.85.3.1 st33_be16_read()

```
uint16_t st33_be16_read (  
    const uint8_t * buf) [static]
```

Internal helper: [st33_be16_read](#).

Definicja w linii 62 pliku [st33ktpm2x.c](#).

Odwołania w [st33ktpm2x_tpm2_get_random\(\)](#), [st33ktpm2x_tpm2_nv_read\(\)](#) i [st33ktpm2x_tpm2_pcr_read_sha256\(\)](#).

12.85.3.2 st33_be16_write()

```
void st33_be16_write (  
    uint8_t * buf,  
    uint16_t v) [static]
```

Internal helper: [st33_be16_write](#).

Definicja w linii 128 pliku [st33ktpm2x.c](#).

Odwołania w [st33_tpm2_nv_write_auth\(\)](#), [st33_write_empty_password_auth\(\)](#), [st33ktpm2x_tpm2_get_capability\(\)](#), [st33ktpm2x_tpm2_get_random\(\)](#), [st33ktpm2x_tpm2_nv_define\(\)](#), [st33ktpm2x_tpm2_nv_read\(\)](#), [st33ktpm2x_tpm2_pcr_read_sha256\(\)](#), [st33ktpm2x_tpm2_self_test\(\)](#) i [st33ktpm2x_tpm2_startup\(\)](#).

12.85.3.3 st33_be32_read()

```
uint32_t st33_be32_read (  
    const uint8_t * buf) [static]
```

Internal helper: [st33_be32_read](#).

Definicja w linii 68 pliku [st33ktpm2x.c](#).

Odwołania w [st33ktpm2x_tpm2_pcr_read_sha256\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.85.3.4 st33_be32_write()

```
void st33_be32_write (  
    uint8_t * buf,  
    uint32_t v) [static]
```

Internal helper: [st33_be32_write](#).

Definicja w linii 135 pliku [st33ktpm2x.c](#).

Odwołania w [st33_tpm2_nv_write_auth\(\)](#), [st33_write_empty_password_auth\(\)](#), [st33ktpm2x_tpm2_get_capability\(\)](#), [st33ktpm2x_tpm2_get_random\(\)](#), [st33ktpm2x_tpm2_nv_define\(\)](#), [st33ktpm2x_tpm2_nv_read\(\)](#), [st33ktpm2x_tpm2_pcr_read_sha256\(\)](#), [st33ktpm2x_tpm2_self_test\(\)](#) i [st33ktpm2x_tpm2_startup\(\)](#).

12.85.3.5 st33_fifo_read()

```
st33ktpm2x_status_t st33_fifo_read (  
    st33ktpm2x_t * ctx,  
    uint8_t * data,  
    uint16_t len) [static]
```

Internal helper: [st33_fifo_read](#).

Definicja w linii 485 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_i2c_read\(\)](#).

Odwołania w [st33_read_response_stream\(\)](#).

12.85.3.6 st33_fifo_write()

```
st33ktpm2x_status_t st33_fifo_write (
    st33ktpm2x_t * ctx,
    const uint8_t * data,
    uint16_t len) [static]
```

Internal helper: [st33_fifo_write](#).

Definicja w linii 479 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_i2c_write\(\)](#).

Odwołania w [st33ktpm2x_transceive\(\)](#).

12.85.3.7 st33_i2c_read()

```
st33ktpm2x_status_t st33_i2c_read (
    st33ktpm2x_t * ctx,
    uint8_t reg,
    uint8_t * data,
    uint16_t len) [static]
```

Internal helper: [st33_i2c_read](#).

Definicja w linii 154 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::cfg](#), [st33ktpm2x_cfg_t::hi2c](#), [st33ktpm2x_cfg_t::i2c_addr_7bit](#), [st33ktpm2x_cfg_t::io_timeout_ms](#), [ST33KTPM2X_EHAL](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_ETIMEOUT](#) i [ST33KTPM2X_OK](#).

Odwołania w [st33_fifo_read\(\)](#), [st33_read_access\(\)](#), [st33_read_sts\(\)](#) i [st33ktpm2x_read_identity\(\)](#).

12.85.3.8 st33_i2c_write()

```
st33ktpm2x_status_t st33_i2c_write (
    st33ktpm2x_t * ctx,
    uint8_t reg,
    const uint8_t * data,
    uint16_t len) [static]
```

Internal helper: [st33_i2c_write](#).

Definicja w linii 199 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::cfg](#), [st33ktpm2x_cfg_t::hi2c](#), [st33ktpm2x_cfg_t::i2c_addr_7bit](#), [st33ktpm2x_cfg_t::io_timeout_ms](#), [ST33KTPM2X_CMD_MAX_BYTES](#), [ST33KTPM2X_EHAL](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_EOVERFLOW](#), [ST33KTPM2X_ETIMEOUT](#) i [ST33KTPM2X_OK](#).

Odwołania w [st33_fifo_write\(\)](#), [st33_write_access\(\)](#), [st33_write_loc_sel\(\)](#) i [st33_write_sts\(\)](#).

12.85.3.9 st33_identity_is_valid()

```
bool st33_identity_is_valid (
    uint32_t did_vid,
    uint8_t rid) [static]
```

Check whether DID_VID/RID values look like an ST33 TPM response.

Definicja w linii 86 pliku [st33ktpm2x.c](#).

Odwołania w [st33ktpm2x_read_identity\(\)](#).

12.85.3.10 st33_le32_read()

```
uint32_t st33_le32_read (
    const uint8_t * buf) [static]
```

Internal helper: [st33_le32_read](#).

Definicja w linii 77 pliku [st33ktpm2x.c](#).

Odwołania w [st33ktpm2x_read_identity\(\)](#).

12.85.3.11 st33_min_u16()

```
uint16_t st33_min_u16 (
    uint16_t a,
    uint16_t b) [static]
```

Internal helper: [st33_min_u16](#).

Definicja w linii 56 pliku [st33ktpm2x.c](#).

Odwołania w [st33_read_response_stream\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.85.3.12 st33_read_access()

```
st33ktpm2x_status_t st33_read_access (
    st33ktpm2x_t * ctx,
    uint8_t * access) [static]
```

Internal helper: [st33_read_access](#).

Definicja w linii 242 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_i2c_read\(\)](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_EPROTO](#) i [ST33KTPM2X_OK](#).

Odwołania w [st33_trace_tis\(\)](#), [st33_wait_access_bits\(\)](#) i [st33ktpm2x_request_locality0\(\)](#).

12.85.3.13 st33_read_response_stream()

```
st33ktpm2x_status_t st33_read_response_stream (
    st33ktpm2x_t * ctx,
    uint8_t * dst,
    uint16_t len) [static]
```

Internal helper: [st33_read_response_stream](#).

Definicja w linii 491 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_fifo_read\(\)](#), [st33_min_u16\(\)](#), [st33_wait_burst\(\)](#) i [ST33KTPM2X_OK](#).

Odwołania w [st33ktpm2x_transceive\(\)](#).

12.85.3.14 st33_read_sts()

```
st33ktpm2x_status_t st33_read_sts (
    st33ktpm2x_t * ctx,
    uint8_t * sts,
    uint16_t * burst_count) [static]
```

Internal helper: [st33_read_sts](#).

Definicja w linii 278 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_i2c_read\(\)](#), [ST33KTPM2X_EPROTO](#) i [ST33KTPM2X_OK](#).

Odwołania w [st33_trace_tis\(\)](#), [st33_wait_burst\(\)](#), [st33_wait_sts_bits\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.85.3.15 st33_status_text()

```
const char * st33_status_text (
    st33ktpm2x_status_t rc) [static]
```

Internal helper: [st33_status_text](#).

Definicja w linii 100 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_EHAL](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_ENOTSUP](#), [ST33KTPM2X_EOVERFLOW](#), [ST33KTPM2X_EPROTO](#), [ST33KTPM2X_ETIMEOUT](#), [ST33KTPM2X_ETPM_RC](#) i [ST33KTPM2X_OK](#).

Odwołania w [st33_trace_fail\(\)](#), [st33_trace_tis\(\)](#), [st33_wait_access_bits\(\)](#), [st33_wait_burst\(\)](#), [st33_wait_sts_bits\(\)](#), [st33ktpm2x_hard_reset\(\)](#) i [st33ktpm2x_release_locality0\(\)](#).

12.85.3.16 st33_tpm2_nv_write_auth()

```
st33ktpm2x_status_t st33_tpm2_nv_write_auth (
    st33ktpm2x_t * ctx,
    uint32_t auth_handle,
    uint32_t nv_index,
    uint16_t offset,
    const uint8_t * data,
    uint16_t data_len,
    uint32_t * tpm_rc) [static]
```

Internal helper: [st33_tpm2_nv_write_auth](#).

Definicja w linii 1419 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::initialized](#), [st33_be16_write\(\)](#), [st33_be32_write\(\)](#), [st33_write_empty_password_auth\(\)](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_EPROTO](#) i [st33ktpm2x_transceive\(\)](#).

Odwołania w [st33ktpm2x_tpm2_nv_write\(\)](#) i [st33ktpm2x_tpm2_nv_write_platform_pp\(\)](#).

12.85.3.17 st33_trace_fail()

```
st33ktpm2x_status_t st33_trace_fail (
    st33ktpm2x_t * ctx,
    const char * stage,
    st33ktpm2x_status_t rc,
    bool release_locality) [static]
```

Internal helper: [st33_trace_fail](#).

Definicja w linii 461 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_status_text\(\)](#), [st33_trace_tis\(\)](#) i [st33ktpm2x_release_locality0\(\)](#).

Odwołania w [st33ktpm2x_request_locality0\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.85.3.18 st33_trace_tis()

```
void st33_trace_tis (
    st33ktpm2x_t * ctx,
    const char * tag) [static]
```

Internal helper: [st33_trace_tis](#).

Definicja w linii 437 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::cfg](#), [st33ktpm2x_cfg_t::hi2c](#), [st33ktpm2x_t::locality0_acquired](#), [st33_read_access\(\)](#), [st33_read_sts\(\)](#) i [st33_status_text\(\)](#).

Odwołania w [st33_trace_fail\(\)](#), [st33ktpm2x_hard_reset\(\)](#), [st33ktpm2x_request_locality0\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.85.3.19 st33_wait_access_bits()

```
st33ktpm2x_status_t st33_wait_access_bits (
    st33ktpm2x_t * ctx,
    uint8_t mask,
    uint8_t expected,
    uint32_t timeout_ms) [static]
```

Poll TPM_ACCESS until required bits are set and forbidden bits are clear.

Definicja w linii 312 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_read_access\(\)](#), [st33_status_text\(\)](#), [ST33KTPM2X_ETIMEOUT](#) i [ST33KTPM2X_OK](#).

Odwołania w [st33ktpm2x_request_locality0\(\)](#).

12.85.3.20 st33_wait_burst()

```
st33ktpm2x_status_t st33_wait_burst(
    st33ktpm2x_t * ctx,
    uint16_t * burst_out,
    uint8_t require_data_avail) [static]
```

Wait for a non-zero TPM FIFO burst count, optionally requiring data availability.

Definicja w linii 390 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_cfg_t::burst_timeout_ms](#), [st33ktpm2x_t::cfg](#), [st33_read_sts\(\)](#), [st33_status_text\(\)](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_ETIMEOUT](#) i [ST33KTPM2X_OK](#).

Odwołania w [st33_read_response_stream\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.85.3.21 st33_wait_sts_bits()

```
st33ktpm2x_status_t st33_wait_sts_bits (
    st33ktpm2x_t * ctx,
    uint8_t mask,
    uint8_t expected,
    uint32_t timeout_ms) [static]
```

Poll TPM_STS until required status bits are set and forbidden bits are clear.

Definicja w linii 350 pliku `st33ktpm2x.c`.

Odwołuje się do `st33_read_sts()`, `st33_status_text()`, `ST33KTPM2X_ETIMEOUT` i `ST33KTPM2X_OK`.

Odwołania w `st33ktpm2x_transceive()`.

12.85.3.22 `st33_write_access()`

```
st33ktpm2x_status_t st33_write_access (
    st33ktpm2x_t * ctx,
    uint8_t value) [static]
```

Internal helper: `st33_write_access`.

Definicja w linii 266 pliku `st33ktpm2x.c`.

Odwołuje się do `st33_i2c_write()`.

Odwołania w `st33ktpm2x_release_locality0()` i `st33ktpm2x_request_locality0()`.

12.85.3.23 `st33_write_empty_password_auth()`

```
uint16_t st33_write_empty_password_auth (
    uint8_t * buf) [static]
```

Marshal an empty-password authorization session into a TPM2 command.

Definicja w linii 144 pliku `st33ktpm2x.c`.

Odwołuje się do `st33_be16_write()` i `st33_be32_write()`.

Odwołania w `st33_tpm2_nv_write_auth()`, `st33ktpm2x_tpm2_nv_define()` i `st33ktpm2x_tpm2_nv_read()`.

12.85.3.24 `st33_write_loc_sel()`

```
st33ktpm2x_status_t st33_write_loc_sel (
    st33ktpm2x_t * ctx,
    uint8_t locality) [static]
```

Internal helper: `st33_write_loc_sel`.

Definicja w linii 272 pliku `st33ktpm2x.c`.

Odwołuje się do `st33_i2c_write()`.

Odwołania w `st33ktpm2x_hard_reset()` i `st33ktpm2x_request_locality0()`.

12.85.3.25 `st33_write_sts()`

```
st33ktpm2x_status_t st33_write_sts (
    st33ktpm2x_t * ctx,
    uint8_t value) [static]
```

Internal helper: `st33_write_sts`.

Definicja w linii 306 pliku `st33ktpm2x.c`.

Odwołuje się do `st33_i2c_write()`.

Odwołania w `st33ktpm2x_transceive()`.

12.85.3.26 `st33ktpm2x_davint_is_asserted()`

```
st33ktpm2x_status_t st33ktpm2x_davint_is_asserted (
    st33ktpm2x_t * ctx,
    bool * asserted)
```

Read TPM_DAVINT# / PIRQ state.

Parametry

<code>ctx</code>	Driver context.
<code>asserted</code>	[out] <code>true</code> if active-low interrupt line is asserted.

Zwraca

Status code (`ST33KTPM2X_ENOTSUP` if DAVINT pin is not configured).

Definicja w linii 605 pliku `st33ktpm2x.c`.

Odwołuje się do `st33ktpm2x_t::cfg`, `st33ktpm2x_cfg_t::davint_active_state`, `st33ktpm2x_cfg_t::davint_pin`, `st33ktpm2x_cfg_t::davint_port`, `st33ktpm2x_t::initialized`, `ST33KTPM2X_EINVAL`, `ST33KTPM2X_ENOTSUP` i `ST33KTPM2X_OK`.

12.85.3.27 st33ktpm2x_default_cfg()

```
void st33ktpm2x_default_cfg (
    st33ktpm2x_cfg_t * cfg,
    I2C_HandleTypeDef * hi2c)
```

Fill config with safe defaults (including PBO reset pin defaults from `main.h`).

Parametry

<i>cfg</i>	[out] Configuration object to initialize.
<i>hi2c</i>	I2C peripheral handle (use <code>&hi2c3</code>).

Definicja w linii 518 pliku `st33ktpm2x.c`.

Odwołuje się do `st33ktpm2x_cfg_t::burst_timeout_ms`, `st33ktpm2x_cfg_t::davint_active_state`, `st33ktpm2x_cfg_t::davint_pin`, `st33ktpm2x_cfg_t::davint_port`, `st33ktpm2x_cfg_t::hi2c`, `st33ktpm2x_cfg_t::i2c_addr_7bit`, `st33ktpm2x_cfg_t::io_timeout_ms`, `st33ktpm2x_cfg_t::locality_timeout_ms`, `st33ktpm2x_cfg_t::pp_active_state`, `st33ktpm2x_cfg_t::pp_pin`, `st33ktpm2x_cfg_t::pp_port`, `st33ktpm2x_cfg_t::reset_pin`, `st33ktpm2x_cfg_t::reset_port`, `st33ktpm2x_cfg_t::reset_pulse_ms`, `st33ktpm2x_cfg_t::reset_recovery_ms` i `ST33KTPM2X_I2C_ADDR_DEFAULT`.

12.85.3.28 st33ktpm2x_deinit()

```
st33ktpm2x_status_t st33ktpm2x_deinit (
    st33ktpm2x_t * ctx)
```

Deinitialize context and release locality if needed.

Parametry

<i>ctx</i>	Driver context.
------------	-----------------

Zwraca

Status code.

Definicja w linii 565 pliku `st33ktpm2x.c`.

Odwołuje się do `st33ktpm2x_t::initialized`, `ST33KTPM2X_ESTATE`, `ST33KTPM2X_OK` i `st33ktpm2x_release_locality0()`.

12.85.3.29 st33ktpm2x_hard_reset()

```
st33ktpm2x_status_t st33ktpm2x_hard_reset (
    st33ktpm2x_t * ctx)
```

Perform hardware reset pulse on TPM reset pin.

Parametry

<i>ctx</i>	Driver context.
------------	-----------------

Zwraca

Status code.

Definicja w linii 577 pliku `st33ktpm2x.c`.

Odwołuje się do `st33ktpm2x_t::cfg`, `st33ktpm2x_t::initialized`, `st33ktpm2x_t::locality0_acquired`, `st33ktpm2x_cfg_t::reset_pin`, `st33ktpm2x_cfg_t::reset_port`, `st33ktpm2x_cfg_t::reset_pulse_ms`, `st33ktpm2x_cfg_t::reset_recovery_ms`, `st33_status_text()`, `st33_trace_tis()`, `st33_write_loc_sel()`, `ST33KTPM2X_ESTATE` i `ST33KTPM2X_OK`.

12.85.3.30 st33ktpm2x_init()

```
st33ktpm2x_status_t st33ktpm2x_init (
    st33ktpm2x_t * ctx,
    const st33ktpm2x_cfg_t * cfg)
```

Initialize TPM driver context.

Parametry

<i>ctx</i>	Driver context.
<i>cfg</i>	Runtime configuration.

Zwraca

Status code.

Definicja w linii 547 pliku `st33ktpm2x.c`.

Odwołuje się do `st33ktpm2x_t::cfg`, `st33ktpm2x_cfg_t::hi2c`, `st33ktpm2x_t::initialized`, `st33ktpm2x_cfg_t::reset_pin`, `st33ktpm2x_cfg_t::reset_port`, `ST33KTPM2X_EINVAL` i `ST33KTPM2X_OK`.

12.85.3.31 st33ktpm2x_pp_is_pressed()

```
st33ktpm2x_status_t st33ktpm2x_pp_is_pressed (
    st33ktpm2x_t * ctx,
    bool * pressed)
```

Read state of optional Physical Presence button.

Parametry

<i>ctx</i>	Driver context.
<i>pressed</i>	[out] true if button is active.

Zwraca

Status code (`ST33KTPM2X_ENOTSUP` if PP pin is not configured).

Definicja w linii 624 pliku `st33ktpm2x.c`.

Odwołuje się do `st33ktpm2x_t::cfg`, `st33ktpm2x_t::initialized`, `st33ktpm2x_cfg_t::pp_active_state`, `st33ktpm2x_cfg_t::pp_pin`, `st33ktpm2x_cfg_t::pp_port`, `ST33KTPM2X_EINVAL`, `ST33KTPM2X_ENOTSUP` i `ST33KTPM2X_OK`.

Odwołania w `st33ktpm2x_pp_wait_pressed()`.

12.85.3.32 st33ktpm2x_pp_wait_pressed()

```
st33ktpm2x_status_t st33ktpm2x_pp_wait_pressed (
    st33ktpm2x_t * ctx,
    uint32_t timeout_ms)
```

Wait until Physical Presence button becomes active.

Parametry

<i>ctx</i>	Driver context.
<i>timeout_ms</i>	Timeout in ms.

Zwraca

Status code.

Definicja w linii 643 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::initialized](#), [ST33KTPM2X_ESTATE](#), [ST33KTPM2X_ETIMEOUT](#), [ST33KTPM2X_OK](#) i [st33ktpm2x_pp_is_pressed\(\)](#).

12.85.3.33 st33ktpm2x_read_identity()

```
st33ktpm2x_status_t st33ktpm2x_read_identity (  
    st33ktpm2x_t * ctx,  
    uint32_t * did_vid,  
    uint8_t * rid)
```

Read TPM DID_VID and RID registers.

Parametry

<i>ctx</i>	Driver context.
<i>did_vid</i>	[out] 32-bit DID_VID register value.
<i>rid</i>	[out] 8-bit Revision ID value.

Zwraca

Status code.

Definicja w linii 755 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::initialized](#), [st33_i2c_read\(\)](#), [st33_identity_is_valid\(\)](#), [st33_le32_read\(\)](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_EPROTO](#) i [ST33KTPM2X_OK](#).

12.85.3.34 st33ktpm2x_release_locality0()

```
st33ktpm2x_status_t st33ktpm2x_release_locality0 (  
    st33ktpm2x_t * ctx)
```

Release TPM locality 0.

Parametry

<i>ctx</i>	Driver context.
------------	-----------------

Zwraca

Status code.

Definicja w linii 730 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::initialized](#), [st33ktpm2x_t::locality0_acquired](#), [st33_status_text\(\)](#), [st33_write_access\(\)](#), [ST33KTPM2X_ESTATE](#) i [ST33KTPM2X_OK](#).

Odwołania w [st33_trace_fail\(\)](#), [st33ktpm2x_deinit\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.85.3.35 st33ktpm2x_request_locality0()

```
st33ktpm2x_status_t st33ktpm2x_request_locality0 (
    st33ktpm2x_t * ctx)
```

Acquire TPM locality 0.

Parametry

<i>ctx</i>	Driver context.
------------	-----------------

Zwraca

Status code.

Definicja w linii 672 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::cfg](#), [st33ktpm2x_t::initialized](#), [st33ktpm2x_t::locality0_acquired](#), [st33ktpm2x_cfg_t::locality_timeout_ms](#), [st33_read_access\(\)](#), [st33_trace_fail\(\)](#), [st33_trace_tis\(\)](#), [st33_wait_access_bits\(\)](#), [st33_write_access\(\)](#), [st33_write_loc_sel\(\)](#), [ST33KTPM2X_ESTATE](#) i [ST33KTPM2X_OK](#).

Odwołania w [st33ktpm2x_transceive\(\)](#).

12.85.3.36 st33ktpm2x_tpm2_flush_context()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_flush_context (
    st33ktpm2x_t * ctx,
    uint32_t handle,
    uint32_t * tpm_rc)
```

Flush transient context/session handle (scaffold API).

Parametry

<i>ctx</i>	Driver context.
<i>handle</i>	Transient handle.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1515 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_ENOTSUP](#).

12.85.3.37 st33ktpm2x_tpm2_get_capability()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_get_capability (
    st33ktpm2x_t * ctx,
    uint32_t capability,
    uint32_t property,
    uint32_t property_count,
    uint8_t * payload_out,
    uint16_t payload_capacity,
    uint16_t * payload_len,
    bool * more_data,
    uint32_t * tpm_rc)
```

Execute TPM2_GetCapability and return raw capability payload.

Parametry

<i>ctx</i>	Driver context.
<i>capability</i>	TPM capability selector.
<i>property</i>	Capability-specific property/offset.
<i>property_count</i>	Number of properties requested.
<i>payload_out</i>	Raw capability payload output (without <code>moreData</code> byte).
<i>payload_capacity</i>	Capacity of <code>payload_out</code> .
<i>payload_len</i>	[out] Returned payload length.
<i>more_data</i>	[out] <code>true</code> if TPM reports more data available.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1044 pliku `st33ktpm2x.c`.

Odwołuje się do `st33_be16_write()`, `st33_be32_write()`, `ST33KTPM2X_EINVAL`, `ST33KTPM2X_EOVERFLOW`, `ST33KTPM2X_EPROTO`, `ST33KTPM2X_OK`, `ST33KTPM2X_RSP_MAX_BYTES` i `st33ktpm2x_transceive()`.

12.85.338 st33ktpm2x_tpm2_get_random()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_get_random (
    st33ktpm2x_t * ctx,
    uint16_t requested_bytes,
    uint8_t * random_out,
    uint16_t random_capacity,
    uint16_t * random_len,
    uint32_t * tpm_rc)
```

Execute TPM2_GetRandom.

Parametry

<i>ctx</i>	Driver context.
<i>requested_bytes</i>	Number of random bytes requested from TPM.
<i>random_out</i>	Output buffer for random bytes.
<i>random_capacity</i>	Capacity of <code>random_out</code> .
<i>random_len</i>	[out] Actual bytes returned.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 989 pliku `st33ktpm2x.c`.

Odwołuje się do `st33_be16_read()`, `st33_be16_write()`, `st33_be32_write()`, `ST33KTPM2X_EINVAL`, `ST33KTPM2X_EOVERFLOW`, `ST33KTPM2X_EPROTO`, `ST33KTPM2X_OK` i `st33ktpm2x_transceive()`.

12.85.339 st33ktpm2x_tpm2_nv_define()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_nv_define (
    st33ktpm2x_t * ctx,
```

```
uint32_t nv_index,
uint16_t data_size,
uint32_t attributes,
uint32_t * tpm_rc)
```

Define NV index using owner empty-password authorization.

Parametry

<i>ctx</i>	Driver context.
<i>nv_index</i>	NV index handle.
<i>data_size</i>	NV data size.
<i>attributes</i>	NV attributes mask.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1270 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::initialized](#), [st33_be16_write\(\)](#), [st33_be32_write\(\)](#), [st33_write_empty_password_auth\(\)](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_EPROTO](#) i [st33ktpm2x_transceive\(\)](#).

12.85.3.40 st33ktpm2x_tpm2_nv_read()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_nv_read (
    st33ktpm2x_t * ctx,
    uint32_t nv_index,
    uint16_t offset,
    uint8_t * out_data,
    uint16_t out_capacity,
    uint16_t * out_len,
    uint32_t * tpm_rc)
```

Read bytes from NV index using owner empty-password authorization.

Parametry

<i>ctx</i>	Driver context.
<i>nv_index</i>	NV index handle.
<i>offset</i>	Offset in bytes.
<i>out_data</i>	Output buffer.
<i>out_capacity</i>	Output capacity.
<i>out_len</i>	[out] Returned data length.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1330 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::initialized](#), [st33_be16_read\(\)](#), [st33_be16_write\(\)](#), [st33_be32_write\(\)](#), [st33_write_empty_password_auth\(\)](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_EOVERFLOW](#), [ST33KTPM2X_EPROTO](#), [ST33KTPM2X_OK](#) i [st33ktpm2x_transceive\(\)](#).

12.85.3.41 st33ktpm2x_tpm2_nv_write()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write (
    st33ktpm2x_t * ctx,
    uint32_t nv_index,
    uint16_t offset,
    const uint8_t * data,
    uint16_t data_len,
    uint32_t * tpm_rc)
```

Write bytes to NV index using owner empty-password authorization.

Parametry

<i>ctx</i>	Driver context.
<i>nv_index</i>	NV index handle.
<i>offset</i>	Offset in bytes.
<i>data</i>	Input data.
<i>data_len</i>	Input data length.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1483 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_tpm2_nv_write_auth\(\)](#).

12.85.3.42 st33ktpm2x_tpm2_nv_write_platform_pp()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write_platform_pp (
    st33ktpm2x_t * ctx,
    uint32_t nv_index,
    uint16_t offset,
    const uint8_t * data,
    uint16_t data_len,
    uint32_t * tpm_rc)
```

Write bytes to NV index using platform authorization with hardware Physical Presence.

Parametry

<i>ctx</i>	Driver context.
<i>nv_index</i>	NV index handle.
<i>offset</i>	Offset in bytes.
<i>data</i>	Input data.
<i>data_len</i>	Input data length.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1499 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_tpm2_nv_write_auth\(\)](#).

12.85.3.43 st33ktpm2x_tpm2_pcr_read_sha256()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_pcr_read_sha256 (
    st33ktpm2x_t * ctx,
    uint8_t pcr_index,
    uint8_t digest_sha256_out[32],
    uint32_t * update_counter_out,
    uint32_t * tpm_rc)
```

Execute TPM2_PCR_Read for one SHA-256 PCR index.

Parametry

<i>ctx</i>	Driver context.
<i>pcr_index</i>	PCR index in range 0..23.
<i>digest_sha256_out</i>	[out] 32-byte SHA-256 PCR digest.
<i>update_counter_out</i>	[out] PCR update counter.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1096 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_be16_read\(\)](#), [st33_be16_write\(\)](#), [st33_be32_read\(\)](#), [st33_be32_write\(\)](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_EPROTO](#), [ST33KTPM2X_OK](#), [ST33KTPM2X_RSP_MAX_BYTES](#) i [st33ktpm2x_transceive\(\)](#).

12.85.3.44 st33ktpm2x_tpm2_policy_command_code()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_policy_command_code (
    st33ktpm2x_t * ctx,
    uint32_t session_handle,
    uint32_t command_code,
    uint32_t * tpm_rc)
```

Apply PolicyCommandCode in an existing policy session (scaffold API).

Parametry

<i>ctx</i>	Driver context.
<i>session_handle</i>	Policy session handle.
<i>command_code</i>	TPM command code to authorize.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1236 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_ENOTSUP](#).

12.85.3.45 st33ktpm2x_tpm2_policy_or()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_policy_or (
    st33ktpm2x_t * ctx,
```

```
uint32_t session_handle,
const uint8_t * digest_list,
uint8_t digest_count,
uint8_t digest_size,
uint32_t * tpm_rc)
```

Apply PolicyOR branch combiner (scaffold API).

Parametry

<i>ctx</i>	Driver context.
<i>session_handle</i>	Policy session handle.
<i>digest_list</i>	Pointer to concatenated digest list.
<i>digest_count</i>	Number of digests.
<i>digest_size</i>	Single digest size.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1251 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_ENOTSUP](#).

12.85.3.46 st33ktpm2x_tpm2_policy_pcr()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_policy_pcr (
    st33ktpm2x_t * ctx,
    uint32_t session_handle,
    uint8_t pcr_index,
    uint32_t * tpm_rc)
```

Apply PolicyPCR in an existing policy session (scaffold API).

Parametry

<i>ctx</i>	Driver context.
<i>session_handle</i>	Policy session handle.
<i>pcr_index</i>	PCR index.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1208 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_ENOTSUP](#).

12.85.3.47 st33ktpm2x_tpm2_policy_physical_presence()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_policy_physical_presence (
    st33ktpm2x_t * ctx,
    uint32_t session_handle,
    uint32_t * tpm_rc)
```

Apply Physical Presence policy branch (scaffold API).

Parametry

<i>ctx</i>	Driver context.
<i>session_handle</i>	Policy session handle.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1223 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_ENOTSUP](#).

12.85.3.48 st33ktpm2x_tpm2_self_test()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_self_test (
    st33ktpm2x_t * ctx,
    bool full_test,
    uint32_t * tpm_rc)
```

Execute TPM2_SelfTest.

Parametry

<i>ctx</i>	Driver context.
<i>full_test</i>	true for full self-test, false for incremental self-test.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 973 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_be16_write\(\)](#), [st33_be32_write\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.85.3.49 st33ktpm2x_tpm2_start_auth_session()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_start_auth_session (
    st33ktpm2x_t * ctx,
    uint32_t * session_handle_out,
    uint32_t * tpm_rc)
```

Start TPM2 policy/auth session helper (scaffold API).

Parametry

<i>ctx</i>	Driver context.
<i>session_handle_out</i>	[out] Created session handle.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1192 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_ENOTSUP](#).

12.85.3.50 st33ktpm2x_tpm2_startup()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_startup (
    st33ktpm2x_t * ctx,
    uint16_t startup_type,
    uint32_t * tpm_rc)
```

Execute TPM2_Startup.

Parametry

<i>ctx</i>	Driver context.
<i>startup_type</i>	Use ST33KTPM2X_TPM2_SU_CLEAR or ST33KTPM2X_TPM2_SU_STATE.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 957 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_be16_write\(\)](#), [st33_be32_write\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.85.3.51 st33ktpm2x_transceive()

```
st33ktpm2x_status_t st33ktpm2x_transceive (
    st33ktpm2x_t * ctx,
    const uint8_t * command,
    uint16_t command_len,
    uint8_t * response,
    uint16_t response_capacity,
    uint16_t * response_len,
    uint32_t * tpm_rc)
```

Send one raw TPM command and receive full response.

This is the primary extension point for advanced TPM2 features.

Parametry

<i>ctx</i>	Driver context.
<i>command</i>	Command buffer (TPM2 packet, complete header+payload).
<i>command_len</i>	Command length in bytes.
<i>response</i>	Buffer for response packet.
<i>response_capacity</i>	Response buffer capacity in bytes.
<i>response_len</i>	[out] Actual response length.
<i>tpm_rc</i>	[out] TPM2 response code parsed from response header.

Zwraca

Status code.

Definicja w linii 786 pliku `st33ktpm2x.c`.

Odwwołuje się do `st33ktpm2x_cfg_t::burst_timeout_ms`, `st33ktpm2x_t::cfg`, `st33ktpm2x_t::initialized`, `st33_be32_read()`, `st33_fifo_write()`, `st33_min_u16()`, `st33_read_response_stream()`, `st33_read_sts()`, `st33_trace_fail()`, `st33_trace_tis()`, `st33_wait_burst()`, `st33_wait_sts_bits()`, `st33_write_sts()`, `ST33KTPM2X_CMD_MAX_BYTES`, `ST33KTPM2X_EINVAL`, `ST33KTPM2X_EOVERFLOW`, `ST33KTPM2X_EPROTO`, `ST33KTPM2X_ETPM_RC`, `ST33KTPM2X_OK`, `st33ktpm2x_release_locality0()`, `st33ktpm2x_request_locality0()` i `ST33KTPM2X_RSP_MAX_BYTES`.

Odwwołania w `st33_tpm2_nv_write_auth()`, `st33ktpm2x_tpm2_get_capability()`, `st33ktpm2x_tpm2_get_random()`, `st33ktpm2x_tpm2_nv_define()`, `st33ktpm2x_tpm2_nv_read()`, `st33ktpm2x_tpm2_pcr_read_sha256()`, `st33ktpm2x_tpm2_self_test()` i `st33ktpm2x_tpm2_startup()`.

12.85.4 Dokumentacja zmiennych**12.85.4.1 s_st33_i2c_tx**

```
uint8_t s_st33_i2c_tx[1U+ST33KTPM2X_CMD_MAX_BYTES] [static]
```

Definicja w linii 53 pliku `st33ktpm2x.c`.

12.86 st33ktpm2x.c

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #include "st33ktpm2x.h"
00007
00008 #include "app_delay.h"
00009 #include "main.h"
00010
00011 #include <stdio.h>
00012 #include <string.h>
00013
00014 /* I2C TPM register map (locality 0). */
00015 #define ST33_REG_LOC_SEL 0x00U
00016 #define ST33_REG_ACCESS 0x04U
00017 #define ST33_REG_STS 0x18U
00018 #define ST33_REG_DATA_FIFO 0x24U
00019 #define ST33_REG_INTERFACE_CAPABILITY 0x30U
00020 #define ST33_REG_DID_VID 0x48U
00021 #define ST33_REG_RID 0x4CU
00022
00023 #define ST33_I2C_MAX_RETRIES 3U
00024
00025 /* ACCESS bits. */
00026 #define ST33_ACCESS_VALID 0x80U
00027 #define ST33_ACCESS_ACTIVE_LOCALITY 0x20U
00028 #define ST33_ACCESS_REQUEST_USE 0x02U
00029 #define ST33_ACCESS_READ_ZERO 0x48U
00030
00031 /* STS bits. */
00032 #define ST33_STS_VALID 0x80U
00033 #define ST33_STS_COMMAND_READY 0x40U
00034 #define ST33_STS_TPM_GO 0x20U
00035 #define ST33_STS_DATA_AVAIL 0x10U
00036 #define ST33_STS_EXPECT 0x08U
00037 #define ST33_STS_READ_ZERO 0x23U
00038
00039 #define ST33_TPM_HEADER_SIZE 10U
00040 #define ST33_TPM_RH_OWNER 0x40000001UL
00041 #define ST33_TPM_RH_PLATFORM 0x4000000CUL
00042 #define ST33_TPM_RS_PW 0x40000009UL
00043 #define ST33_TPM_PW_AUTH_SIZE 9U
00044 #define ST33_TPM_RANDOM_MAX_BYTES 256U
00045 #define ST33_TPM_NV_TRANSFER_MAX_BYTES 512U
00046
00047 #if defined(TPM_INIT_LOG)
00048 #define ST33_TRACE(...) printf(__VA_ARGS__)
00049 #else
00050 #define ST33_TRACE(...) do { if (0) { printf(__VA_ARGS__); } } while (0)
00051 #endif
00052
00053 static uint8_t s_st33_i2c_tx[1U + ST33KTPM2X_CMD_MAX_BYTES];
00054
00056 static uint16_t st33_min_u16(uint16_t a, uint16_t b)
```

```

00057 {
00058     return (a < b) ? a : b;
00059 }
00060
00062 static uint16_t st33_be16_read(const uint8_t *buf)
00063 {
00064     return (uint16_t)((uint16_t)buf[0] << 8) | buf[1];
00065 }
00066
00068 static uint32_t st33_be32_read(const uint8_t *buf)
00069 {
00070     return ((uint32_t)buf[0] << 24) |
00071            ((uint32_t)buf[1] << 16) |
00072            ((uint32_t)buf[2] << 8) |
00073            (uint32_t)buf[3];
00074 }
00075
00077 static uint32_t st33_le32_read(const uint8_t *buf)
00078 {
00079     return ((uint32_t)buf[3] << 24) |
00080            ((uint32_t)buf[2] << 16) |
00081            ((uint32_t)buf[1] << 8) |
00082            (uint32_t)buf[0];
00083 }
00084
00086 static bool st33_identity_is_valid(uint32_t did_vid, uint8_t rid)
00087 {
00088     if ((did_vid == 0xFFFFFFFFUL) || (did_vid == 0x00000000UL))
00089     {
00090         return false;
00091     }
00092     if (rid == 0xFFU)
00093     {
00094         return false;
00095     }
00096     return true;
00097 }
00098
00100 static const char *st33_status_text(st33ktpm2x_status_t rc)
00101 {
00102     switch (rc)
00103     {
00104         case ST33KTPM2X_OK:
00105             return "OK";
00106         case ST33KTPM2X_EINVAL:
00107             return "EINVAL";
00108         case ST33KTPM2X_ESTATE:
00109             return "ESTATE";
00110         case ST33KTPM2X_EHAL:
00111             return "EHAL";
00112         case ST33KTPM2X_ETIMEOUT:
00113             return "ETIMEOUT";
00114         case ST33KTPM2X_EOVERFLOW:
00115             return "EOVERFLOW";
00116         case ST33KTPM2X_EPROTO:
00117             return "EPROTO";
00118         case ST33KTPM2X_ETPM_RC:
00119             return "ETPM_RC";
00120         case ST33KTPM2X_ENOTSUP:
00121             return "ENOTSUP";
00122         default:
00123             return "?";
00124     }
00125 }
00126
00128 static void st33_be16_write(uint8_t *buf, uint16_t v)
00129 {
00130     buf[0] = (uint8_t)(v >> 8);
00131     buf[1] = (uint8_t)v;
00132 }
00133
00135 static void st33_be32_write(uint8_t *buf, uint32_t v)
00136 {
00137     buf[0] = (uint8_t)(v >> 24);
00138     buf[1] = (uint8_t)(v >> 16);
00139     buf[2] = (uint8_t)(v >> 8);
00140     buf[3] = (uint8_t)v;
00141 }
00142
00144 static uint16_t st33_write_empty_password_auth(uint8_t *buf)
00145 {
00146     st33_be32_write(&buf[0], ST33_TPM_RS_PW);
00147     st33_be16_write(&buf[4], 0U);
00148     buf[6] = 0U;
00149     st33_be16_write(&buf[7], 0U);
00150     return ST33_TPM_PW_AUTH_SIZE;
00151 }

```

```

00152
00154 static st33ktpm2x_status_t st33_i2c_read(st33ktpm2x_t *ctx,
00155                                           uint8_t reg,
00156                                           uint8_t *data,
00157                                           uint16_t len)
00158 {
00159     HAL_StatusTypeDef hal_st = HAL_ERROR;
00160     uint8_t attempt;
00161
00162     if ((ctx == NULL) || (data == NULL) || (len == 0U))
00163     {
00164         return ST33KTPM2X_EINVAL;
00165     }
00166
00167     for (attempt = 0U; attempt < ST33_I2C_MAX_RETRIES; attempt++)
00168     {
00169         hal_st = HAL_I2C_Master_Transmit(ctx->cfg.hi2c,
00170                                           (uint16_t)(ctx->cfg.i2c_addr_7bit < 1),
00171                                           &reg,
00172                                           1U,
00173                                           ctx->cfg.io_timeout_ms);
00174
00175         if (hal_st == HAL_OK)
00176         {
00177             app_delay_ms(1U);
00178             hal_st = HAL_I2C_Master_Receive(ctx->cfg.hi2c,
00179                                              (uint16_t)(ctx->cfg.i2c_addr_7bit < 1),
00180                                              data,
00181                                              len,
00182                                              ctx->cfg.io_timeout_ms);
00183
00184             if (hal_st == HAL_OK)
00185             {
00186                 app_delay_ms(1U);
00187                 return ST33KTPM2X_OK;
00188             }
00189             app_delay_ms(1U);
00190
00191             if (hal_st == HAL_TIMEOUT)
00192             {
00193                 return ST33KTPM2X_ETIMEOUT;
00194             }
00195             return ST33KTPM2X_EHAL;
00196         }
00197
00199 static st33ktpm2x_status_t st33_i2c_write(st33ktpm2x_t *ctx,
00200                                           uint8_t reg,
00201                                           const uint8_t *data,
00202                                           uint16_t len)
00203 {
00204     HAL_StatusTypeDef hal_st = HAL_ERROR;
00205     uint8_t attempt;
00206
00207     if ((ctx == NULL) || (data == NULL) || (len == 0U))
00208     {
00209         return ST33KTPM2X_EINVAL;
00210     }
00211     if (len > ST33KTPM2X_CMD_MAX_BYTES)
00212     {
00213         return ST33KTPM2X_EOVERFLOW;
00214     }
00215
00216     s_st33_i2c_tx[0] = reg;
00217     memcpy(&s_st33_i2c_tx[1], data, len);
00218
00219     for (attempt = 0U; attempt < ST33_I2C_MAX_RETRIES; attempt++)
00220     {
00221         hal_st = HAL_I2C_Master_Transmit(ctx->cfg.hi2c,
00222                                           (uint16_t)(ctx->cfg.i2c_addr_7bit < 1),
00223                                           s_st33_i2c_tx,
00224                                           (uint16_t)(len + 1U),
00225                                           ctx->cfg.io_timeout_ms);
00226
00227         if (hal_st == HAL_OK)
00228         {
00229             app_delay_ms(1U);
00230             return ST33KTPM2X_OK;
00231         }
00232         app_delay_ms(1U);
00233
00234         if (hal_st == HAL_TIMEOUT)
00235         {
00236             return ST33KTPM2X_ETIMEOUT;
00237         }
00238         return ST33KTPM2X_EHAL;
00239     }
00240

```



```

00242 static st33ktpm2x_status_t st33_read_access(st33ktpm2x_t *ctx, uint8_t *access)
00243 {
00244     uint8_t value = 0U;
00245     st33ktpm2x_status_t rc;
00246
00247     if (access == NULL)
00248     {
00249         return ST33KTPM2X_EINVAL;
00250     }
00251
00252     rc = st33_i2c_read(ctx, ST33_REG_ACCESS, &value, 1U);
00253     *access = value;
00254     if (rc != ST33KTPM2X_OK)
00255     {
00256         return rc;
00257     }
00258     if ((value & ST33_ACCESS_READ_ZERO) != 0U)
00259     {
00260         return ST33KTPM2X_EPROTO;
00261     }
00262     return ST33KTPM2X_OK;
00263 }
00264
00266 static st33ktpm2x_status_t st33_write_access(st33ktpm2x_t *ctx, uint8_t value)
00267 {
00268     return st33_i2c_write(ctx, ST33_REG_ACCESS, &value, 1U);
00269 }
00270
00272 static st33ktpm2x_status_t st33_write_loc_sel(st33ktpm2x_t *ctx, uint8_t locality)
00273 {
00274     return st33_i2c_write(ctx, ST33_REG_LOC_SEL, &locality, 1U);
00275 }
00276
00278 static st33ktpm2x_status_t st33_read_sts(st33ktpm2x_t *ctx, uint8_t *sts, uint16_t *burst_count)
00279 {
00280     uint8_t raw[3];
00281     st33ktpm2x_status_t rc;
00282
00283     rc = st33_i2c_read(ctx, ST33_REG_STS, raw, sizeof(raw));
00284     if (rc != ST33KTPM2X_OK)
00285     {
00286         return rc;
00287     }
00288
00289     if (sts != NULL)
00290     {
00291         *sts = raw[0];
00292     }
00293     if (burst_count != NULL)
00294     {
00295         *burst_count = (uint16_t)raw[1] | ((uint16_t)raw[2] << 8);
00296     }
00297     if ((raw[0] & ST33_STS_READ_ZERO) != 0U)
00298     {
00299         return ST33KTPM2X_EPROTO;
00300     }
00301
00302     return ST33KTPM2X_OK;
00303 }
00304
00306 static st33ktpm2x_status_t st33_write_sts(st33ktpm2x_t *ctx, uint8_t value)
00307 {
00308     return st33_i2c_write(ctx, ST33_REG_STS, &value, 1U);
00309 }
00310
00312 static st33ktpm2x_status_t st33_wait_access_bits(st33ktpm2x_t *ctx,
00313                                                  uint8_t mask,
00314                                                  uint8_t expected,
00315                                                  uint32_t timeout_ms)
00316 {
00317     uint32_t start_ms;
00318     uint8_t access = 0U;
00319     uint8_t last_access = 0U;
00320     st33ktpm2x_status_t rc;
00321     st33ktpm2x_status_t last_rc = ST33KTPM2X_ETIMEOUT;
00322
00323     start_ms = HAL_GetTick();
00324     do
00325     {
00326         rc = st33_read_access(ctx, &access);
00327         last_rc = rc;
00328         last_access = access;
00329         if (rc == ST33KTPM2X_OK)
00330         {
00331             if ((access & mask) == expected)
00332             {
00333                 return ST33KTPM2X_OK;

```

```

00334     }
00335     }
00336     app_delay_ms(1U);
00337     } while ((HAL_GetTick() - start_ms) < timeout_ms);
00338
00339     ST33_TRACE("TPM TIS wait ACCESS timeout mask=0x%02X expected=0x%02X last_rc=%d(%s) last=0x%02X
timeout=%lu\r\n",
00340               (unsigned int)mask,
00341               (unsigned int)expected,
00342               (int)last_rc,
00343               st33_status_text(last_rc),
00344               (unsigned int)last_access,
00345               (unsigned long)timeout_ms);
00346     return ST33KTPM2X_ETIMEOUT;
00347 }
00348
00350 static st33ktpm2x_status_t st33_wait_sts_bits(st33ktpm2x_t *ctx,
00351                                               uint8_t mask,
00352                                               uint8_t expected,
00353                                               uint32_t timeout_ms)
00354 {
00355     uint32_t start_ms;
00356     uint8_t sts = 0U;
00357     uint8_t last_sts = 0U;
00358     uint16_t last_burst = 0U;
00359     st33ktpm2x_status_t rc;
00360     st33ktpm2x_status_t last_rc = ST33KTPM2X_ETIMEOUT;
00361
00362     start_ms = HAL_GetTick();
00363     do
00364     {
00365         rc = st33_read_sts(ctx, &sts, &last_burst);
00366         last_rc = rc;
00367         last_sts = sts;
00368         if (rc == ST33KTPM2X_OK)
00369         {
00370             if ((sts & mask) == expected)
00371             {
00372                 return ST33KTPM2X_OK;
00373             }
00374             app_delay_ms(1U);
00375         } while ((HAL_GetTick() - start_ms) < timeout_ms);
00376
00377         ST33_TRACE("TPM TIS wait STS timeout mask=0x%02X expected=0x%02X last_rc=%d(%s) sts=0x%02X
burst=%u timeout=%lu\r\n",
00378                 (unsigned int)mask,
00379                 (unsigned int)expected,
00380                 (int)last_rc,
00381                 st33_status_text(last_rc),
00382                 (unsigned int)last_sts,
00383                 (unsigned int)last_burst,
00384                 (unsigned long)timeout_ms);
00385         return ST33KTPM2X_ETIMEOUT;
00386     }
00387 }
00388
00390 static st33ktpm2x_status_t st33_wait_burst(st33ktpm2x_t *ctx, uint16_t *burst_out, uint8_t
require_data_avail)
00391 {
00392     uint32_t start_ms;
00393     uint8_t sts = 0U;
00394     uint16_t burst = 0U;
00395     uint8_t last_sts = 0U;
00396     uint16_t last_burst = 0U;
00397     st33ktpm2x_status_t rc;
00398     st33ktpm2x_status_t last_rc = ST33KTPM2X_ETIMEOUT;
00399
00400     if (burst_out == NULL)
00401     {
00402         return ST33KTPM2X_EINVAL;
00403     }
00404
00405     start_ms = HAL_GetTick();
00406     do
00407     {
00408         rc = st33_read_sts(ctx, &sts, &burst);
00409         last_rc = rc;
00410         last_sts = sts;
00411         last_burst = burst;
00412         if (rc == ST33KTPM2X_OK)
00413         {
00414             if ((sts & ST33_STS_VALID) != 0U)
00415             {
00416                 if ((burst > 0U) && (((sts & ST33_STS_DATA_AVAIL) != 0U) || (require_data_avail ==
0U)))
00417                 {
00418                     *burst_out = burst;

```

```

00419         return ST33KTPM2X_OK;
00420     }
00421 }
00422 }
00423     app_delay_ms(1U);
00424 } while ((HAL_GetTick() - start_ms) < ctx->cfg.burst_timeout_ms);
00425
00426     ST33_TRACE("TPM TIS wait burst timeout data_avail=%u last_rc=%d(%s) sts=0x%02X burst=%u
timeout=%lu\r\n",
00427         (unsigned int)require_data_avail,
00428         (int)last_rc,
00429         st33_status_text(last_rc),
00430         (unsigned int)last_sts,
00431         (unsigned int)last_burst,
00432         (unsigned long)ctx->cfg.burst_timeout_ms);
00433     return ST33KTPM2X_ETIMEOUT;
00434 }
00435
00437 static void st33_trace_tis(st33ktpm2x_t *ctx, const char *tag)
00438 {
00439     uint8_t access = 0U;
00440     uint8_t sts = 0U;
00441     uint16_t burst = 0U;
00442     st33ktpm2x_status_t access_rc;
00443     st33ktpm2x_status_t sts_rc;
00444
00445     access_rc = st33_read_access(ctx, &access);
00446     sts_rc = st33_read_sts(ctx, &sts, &burst);
00447     ST33_TRACE("TPM TIS %s access_rc=%d(%s) access=0x%02X sts_rc=%d(%s) sts=0x%02X burst=%u loc=%u
hal_err=0x%08lX\r\n",
00448         (tag != NULL) ? tag : "state",
00449         (int)access_rc,
00450         st33_status_text(access_rc),
00451         (unsigned int)access,
00452         (int)sts_rc,
00453         st33_status_text(sts_rc),
00454         (unsigned int)sts,
00455         (unsigned int)burst,
00456         ctx->locality0_acquired ? 1U : 0U,
00457         (unsigned long)HAL_I2C_GetError(ctx->cfg.hi2c));
00458 }
00459
00461 static st33ktpm2x_status_t st33_trace_fail(st33ktpm2x_t *ctx,
00462     const char *stage,
00463     st33ktpm2x_status_t rc,
00464     bool release_locality)
00465 {
00466     ST33_TRACE("TPM TIS fail stage=%s rc=%d(%s)\r\n",
00467         (stage != NULL) ? stage : "?",
00468         (int)rc,
00469         st33_status_text(rc));
00470     st33_trace_tis(ctx, stage);
00471     if (release_locality)
00472     {
00473         (void)st33ktpm2x_release_locality0(ctx);
00474     }
00475     return rc;
00476 }
00477
00479 static st33ktpm2x_status_t st33_fifo_write(st33ktpm2x_t *ctx, const uint8_t *data, uint16_t len)
00480 {
00481     return st33_i2c_write(ctx, ST33_REG_DATA_FIFO, data, len);
00482 }
00483
00485 static st33ktpm2x_status_t st33_fifo_read(st33ktpm2x_t *ctx, uint8_t *data, uint16_t len)
00486 {
00487     return st33_i2c_read(ctx, ST33_REG_DATA_FIFO, data, len);
00488 }
00489
00491 static st33ktpm2x_status_t st33_read_response_stream(st33ktpm2x_t *ctx, uint8_t *dst, uint16_t len)
00492 {
00493     uint16_t got = 0U;
00494     uint16_t burst = 0U;
00495     uint16_t chunk = 0U;
00496     st33ktpm2x_status_t rc;
00497
00498     while (got < len)
00499     {
00500         rc = st33_wait_burst(ctx, &burst, 1U);
00501         if (rc != ST33KTPM2X_OK)
00502         {
00503             return rc;
00504         }
00505
00506         chunk = st33_min_u16((uint16_t)(len - got), burst);
00507         rc = st33_fifo_read(ctx, &dst[got], chunk);
00508         if (rc != ST33KTPM2X_OK)

```

```

00509     {
00510         return rc;
00511     }
00512     got = (uint16_t)(got + chunk);
00513 }
00514
00515 return ST33KTPM2X_OK;
00516 }
00517
00518 void st33ktpm2x_default_cfg(st33ktpm2x_cfg_t *cfg, I2C_HandleTypeDef *hi2c)
00519 {
00520     if (cfg == NULL)
00521     {
00522         return;
00523     }
00524
00525     memset(cfg, 0, sizeof(*cfg));
00526     cfg->hi2c = hi2c;
00527     cfg->i2c_addr_7bit = ST33KTPM2X_I2C_ADDR_DEFAULT;
00528
00529     cfg->reset_port = TPM_RESET__GPIO_Port;
00530     cfg->reset_pin = TPM_RESET__Pin;
00531     cfg->reset_pulse_ms = 5U;
00532     cfg->reset_recovery_ms = 100U;
00533
00534     cfg->davint_port = TPM_DAVINT__GPIO_Port;
00535     cfg->davint_pin = TPM_DAVINT__Pin;
00536     cfg->davint_active_state = GPIO_PIN_RESET;
00537
00538     cfg->pp_port = NULL;
00539     cfg->pp_pin = 0U;
00540     cfg->pp_active_state = GPIO_PIN_SET;
00541
00542     cfg->io_timeout_ms = 100U;
00543     cfg->locality_timeout_ms = 1000U;
00544     cfg->burst_timeout_ms = 1000U;
00545 }
00546
00547 st33ktpm2x_status_t st33ktpm2x_init(st33ktpm2x_t *ctx, const st33ktpm2x_cfg_t *cfg)
00548 {
00549     if ((ctx == NULL) || (cfg == NULL) || (cfg->hi2c == NULL))
00550     {
00551         return ST33KTPM2X_EINVAL;
00552     }
00553
00554     if ((cfg->reset_port == NULL) || (cfg->reset_pin == 0U))
00555     {
00556         return ST33KTPM2X_EINVAL;
00557     }
00558
00559     memset(ctx, 0, sizeof(*ctx));
00560     ctx->cfg = *cfg;
00561     ctx->initialized = true;
00562     return ST33KTPM2X_OK;
00563 }
00564
00565 st33ktpm2x_status_t st33ktpm2x_deinit(st33ktpm2x_t *ctx)
00566 {
00567     if ((ctx == NULL) || (!ctx->initialized))
00568     {
00569         return ST33KTPM2X_ESTATE;
00570     }
00571
00572     (void)st33ktpm2x_release_locality0(ctx);
00573     memset(ctx, 0, sizeof(*ctx));
00574     return ST33KTPM2X_OK;
00575 }
00576
00577 st33ktpm2x_status_t st33ktpm2x_hard_reset(st33ktpm2x_t *ctx)
00578 {
00579     st33ktpm2x_status_t rc;
00580     uint8_t locality = 0U;
00581
00582     if ((ctx == NULL) || (!ctx->initialized))
00583     {
00584         return ST33KTPM2X_ESTATE;
00585     }
00586
00587     HAL_GPIO_WritePin(ctx->cfg.reset_port, ctx->cfg.reset_pin, GPIO_PIN_RESET);
00588     app_delay_ms(ctx->cfg.reset_pulse_ms);
00589     HAL_GPIO_WritePin(ctx->cfg.reset_port, ctx->cfg.reset_pin, GPIO_PIN_SET);
00590     app_delay_ms(ctx->cfg.reset_recovery_ms);
00591     ctx->locality0_acquired = false;
00592
00593     rc = st33_write_loc_sel(ctx, locality);
00594     ST33_TRACE("TPM TIS LOC_SEL after reset rc=%d(%s) locality=%u\r\n",
00595         (int)rc,

```

```

00596         st33_status_text(rc),
00597         (unsigned int)locality);
00598     if (rc == ST33KTPM2X_OK)
00599     {
00600         st33_trace_tis(ctx, "after_loc_sel");
00601     }
00602     return rc;
00603 }
00604
00605 st33ktpm2x_status_t st33ktpm2x_davint_is_asserted(st33ktpm2x_t *ctx, bool *asserted)
00606 {
00607     GPIO_PinState state;
00608
00609     if ((ctx == NULL) || (!ctx->initialized) || (asserted == NULL))
00610     {
00611         return ST33KTPM2X_EINVAL;
00612     }
00613
00614     if ((ctx->cfg.davint_port == NULL) || (ctx->cfg.davint_pin == 0U))
00615     {
00616         return ST33KTPM2X_ENOTSUP;
00617     }
00618
00619     state = HAL_GPIO_ReadPin(ctx->cfg.davint_port, ctx->cfg.davint_pin);
00620     *asserted = (state == ctx->cfg.davint_active_state);
00621     return ST33KTPM2X_OK;
00622 }
00623
00624 st33ktpm2x_status_t st33ktpm2x_pp_is_pressed(st33ktpm2x_t *ctx, bool *pressed)
00625 {
00626     GPIO_PinState state;
00627
00628     if ((ctx == NULL) || (!ctx->initialized) || (pressed == NULL))
00629     {
00630         return ST33KTPM2X_EINVAL;
00631     }
00632
00633     if ((ctx->cfg.pp_port == NULL) || (ctx->cfg.pp_pin == 0U))
00634     {
00635         return ST33KTPM2X_ENOTSUP;
00636     }
00637
00638     state = HAL_GPIO_ReadPin(ctx->cfg.pp_port, ctx->cfg.pp_pin);
00639     *pressed = (state == ctx->cfg.pp_active_state);
00640     return ST33KTPM2X_OK;
00641 }
00642
00643 st33ktpm2x_status_t st33ktpm2x_pp_wait_pressed(st33ktpm2x_t *ctx, uint32_t timeout_ms)
00644 {
00645     uint32_t start_ms;
00646     bool pressed = false;
00647     st33ktpm2x_status_t rc;
00648
00649     if ((ctx == NULL) || (!ctx->initialized))
00650     {
00651         return ST33KTPM2X_ESTATE;
00652     }
00653
00654     start_ms = HAL_GetTick();
00655     do
00656     {
00657         rc = st33ktpm2x_pp_is_pressed(ctx, &pressed);
00658         if (rc != ST33KTPM2X_OK)
00659         {
00660             return rc;
00661         }
00662         if (pressed)
00663         {
00664             return ST33KTPM2X_OK;
00665         }
00666         app_delay_ms(1U);
00667     } while ((HAL_GetTick() - start_ms) < timeout_ms);
00668
00669     return ST33KTPM2X_ETIMEOUT;
00670 }
00671
00672 st33ktpm2x_status_t st33ktpm2x_request_locality0(st33ktpm2x_t *ctx)
00673 {
00674     uint8_t access = 0U;
00675     uint8_t locality = 0U;
00676     st33ktpm2x_status_t rc;
00677
00678     if ((ctx == NULL) || (!ctx->initialized))
00679     {
00680         return ST33KTPM2X_ESTATE;
00681     }
00682

```

```

00683     if (ctx->locality0_acquired)
00684     {
00685         st33_trace_tis(ctx, "request_locality_cached");
00686         return ST33KTPM2X_OK;
00687     }
00688
00689     rc = st33_write_loc_sel(ctx, locality);
00690     if (rc != ST33KTPM2X_OK)
00691     {
00692         return st33_trace_fail(ctx, "write_loc_sel", rc, false);
00693     }
00694
00695     rc = st33_read_access(ctx, &access);
00696     if (rc != ST33KTPM2X_OK)
00697     {
00698         return st33_trace_fail(ctx, "read_access_initial", rc, false);
00699     }
00700     ST33_TRACE("TPM TIS request locality initial access=0x%02X timeout=%lu\r\n",
00701               (unsigned int)access,
00702               (unsigned long)ctx->cfg.locality_timeout_ms);
00703
00704     if ((access & ST33_ACCESS_ACTIVE_LOCALITY) == 0U)
00705     {
00706         rc = st33_write_access(ctx, ST33_ACCESS_REQUEST_USE);
00707         if (rc != ST33KTPM2X_OK)
00708         {
00709             return st33_trace_fail(ctx, "write_request_use", rc, false);
00710         }
00711         st33_trace_tis(ctx, "after_request_use");
00712     }
00713
00714     rc = st33_wait_access_bits(ctx,
00715                               (uint8_t)(ST33_ACCESS_VALID | ST33_ACCESS_ACTIVE_LOCALITY),
00716                               (uint8_t)(ST33_ACCESS_VALID | ST33_ACCESS_ACTIVE_LOCALITY),
00717                               ctx->cfg.locality_timeout_ms);
00718     if (rc == ST33KTPM2X_OK)
00719     {
00720         ctx->locality0_acquired = true;
00721         st33_trace_tis(ctx, "locality_acquired");
00722     }
00723     else
00724     {
00725         (void)st33_trace_fail(ctx, "locality_timeout", rc, false);
00726     }
00727     return rc;
00728 }
00729
00730 st33ktpm2x_status_t st33ktpm2x_release_locality0(st33ktpm2x_t *ctx)
00731 {
00732     st33ktpm2x_status_t rc;
00733
00734     if ((ctx == NULL) || (!ctx->initialized))
00735     {
00736         return ST33KTPM2X_ESTATE;
00737     }
00738
00739     if (!ctx->locality0_acquired)
00740     {
00741         return ST33KTPM2X_OK;
00742     }
00743
00744     rc = st33_write_access(ctx, ST33_ACCESS_ACTIVE_LOCALITY);
00745     ST33_TRACE("TPM TIS release locality rc=%d(%s)\r\n",
00746               (int)rc,
00747               st33_status_text(rc));
00748     if (rc == ST33KTPM2X_OK)
00749     {
00750         ctx->locality0_acquired = false;
00751     }
00752     return rc;
00753 }
00754
00755 st33ktpm2x_status_t st33ktpm2x_read_identity(st33ktpm2x_t *ctx, uint32_t *did_vid, uint8_t *rid)
00756 {
00757     uint8_t id_raw[4];
00758     st33ktpm2x_status_t rc;
00759
00760     if ((ctx == NULL) || (!ctx->initialized) || (did_vid == NULL) || (rid == NULL))
00761     {
00762         return ST33KTPM2X_EINVAL;
00763     }
00764
00765     rc = st33_i2c_read(ctx, ST33_REG_DID_VID, id_raw, sizeof(id_raw));
00766     if (rc != ST33KTPM2X_OK)
00767     {
00768         return rc;
00769     }

```

```

00770
00771     *did_vid = st33_le32_read(id_raw);
00772     rc = st33_i2c_read(ctx, ST33_REG_RID, rid, 1U);
00773     if (rc != ST33KTPM2X_OK)
00774     {
00775         return rc;
00776     }
00777
00778     if (!st33_identity_is_valid(*did_vid, *rid))
00779     {
00780         return ST33KTPM2X_EPROTO;
00781     }
00782
00783     return ST33KTPM2X_OK;
00784 }
00785
00786 st33ktpm2x_status_t st33ktpm2x_transceive(st33ktpm2x_t *ctx,
00787                                             const uint8_t *command,
00788                                             uint16_t command_len,
00789                                             uint8_t *response,
00790                                             uint16_t response_capacity,
00791                                             uint16_t *response_len,
00792                                             uint32_t *tpm_rc)
00793 {
00794     uint8_t sts = 0U;
00795     uint16_t burst = 0U;
00796     uint16_t sent = 0U;
00797     uint16_t chunk = 0U;
00798     uint16_t rsp_total = 0U;
00799     uint32_t command_code = 0UL;
00800     st33ktpm2x_status_t rc;
00801
00802     if ((ctx == NULL) || (!ctx->initialized) || (command == NULL) || (response == NULL) ||
(response_len == NULL))
00803     {
00804         return ST33KTPM2X_EINVAL;
00805     }
00806     if ((command_len < ST33_TPM_HEADER_SIZE) || (command_len > ST33KTPM2X_CMD_MAX_BYTES))
00807     {
00808         return ST33KTPM2X_EINVAL;
00809     }
00810
00811     *response_len = 0U;
00812     if (tpm_rc != NULL)
00813     {
00814         *tpm_rc = 0UL;
00815     }
00816
00817     command_code = st33_be32_read(&command[6]);
00818     ST33_TRACE("TPM TIS transceive start cc=0x%08lX cmd_len=%u rsp_cap=%u\r\n",
00819               (unsigned long)command_code,
00820               (unsigned int)command_len,
00821               (unsigned int)response_capacity);
00822
00823     rc = st33ktpm2x_request_locality0(ctx);
00824     if (rc != ST33KTPM2X_OK)
00825     {
00826         return st33_trace_fail(ctx, "request_locality", rc, false);
00827     }
00828     st33_trace_tis(ctx, "transceive_after_locality");
00829
00830     rc = st33_write_sts(ctx, ST33_STS_COMMAND_READY);
00831     if (rc != ST33KTPM2X_OK)
00832     {
00833         return st33_trace_fail(ctx, "write_command_ready", rc, true);
00834     }
00835     st33_trace_tis(ctx, "after_write_command_ready");
00836
00837     rc = st33_wait_sts_bits(ctx,
00838                            (uint8_t)(ST33_STS_VALID | ST33_STS_COMMAND_READY),
00839                            (uint8_t)(ST33_STS_VALID | ST33_STS_COMMAND_READY),
00840                            ctx->cfg.burst_timeout_ms);
00841     if (rc != ST33KTPM2X_OK)
00842     {
00843         return st33_trace_fail(ctx, "wait_command_ready", rc, true);
00844     }
00845     st33_trace_tis(ctx, "command_ready");
00846
00847     while (sent < command_len)
00848     {
00849         rc = st33_wait_burst(ctx, &burst, 0U);
00850         if (rc != ST33KTPM2X_OK)
00851         {
00852             return st33_trace_fail(ctx, "wait_write_burst", rc, true);
00853         }
00854
00855         chunk = st33_min_u16((uint16_t)(command_len - sent), burst);

```

```

00856     ST33_TRACE("TPM TIS write FIFO chunk=%u sent_before=%u burst=%u\r\n",
00857               (unsigned int)chunk,
00858               (unsigned int)sent,
00859               (unsigned int)burst);
00860     rc = st33_fifo_write(ctx, &command[sent], chunk);
00861     if (rc != ST33KTPM2X_OK)
00862     {
00863         return st33_trace_fail(ctx, "fifo_write", rc, true);
00864     }
00865
00866     sent = (uint16_t)(sent + chunk);
00867     rc = st33_read_sts(ctx, &sts, NULL);
00868     if (rc != ST33KTPM2X_OK)
00869     {
00870         return st33_trace_fail(ctx, "read_sts_after_fifo_write", rc, true);
00871     }
00872     ST33_TRACE("TPM TIS after FIFO sent=%u/%u sts=0x%02X expect=%u\r\n",
00873               (unsigned int)sent,
00874               (unsigned int)command_len,
00875               (unsigned int)sts,
00876               ((sts & ST33_STS_EXPECT) != 0U) ? 1U : 0U);
00877     if ((sent < command_len) && ((sts & ST33_STS_EXPECT) == 0U))
00878     {
00879         return st33_trace_fail(ctx, "expect_cleared_before_command_end", ST33KTPM2X_EPROTO, true);
00880     }
00881 }
00882
00883 rc = st33_write_sts(ctx, ST33_STS_TPM_GO);
00884 if (rc != ST33KTPM2X_OK)
00885 {
00886     return st33_trace_fail(ctx, "write_tpm_go", rc, true);
00887 }
00888 st33_trace_tis(ctx, "after_tpm_go");
00889
00890 rc = st33_wait_sts_bits(ctx,
00891                        (uint8_t)(ST33_STS_VALID | ST33_STS_DATA_AVAIL),
00892                        (uint8_t)(ST33_STS_VALID | ST33_STS_DATA_AVAIL),
00893                        ctx->cfg.burst_timeout_ms);
00894 if (rc != ST33KTPM2X_OK)
00895 {
00896     return st33_trace_fail(ctx, "wait_data_avail", rc, true);
00897 }
00898 st33_trace_tis(ctx, "data_avail");
00899
00900 rc = st33_read_response_stream(ctx, response, ST33_TPM_HEADER_SIZE);
00901 if (rc != ST33KTPM2X_OK)
00902 {
00903     return st33_trace_fail(ctx, "read_response_header", rc, true);
00904 }
00905
00906 rsp_total = (uint16_t)st33_be32_read(&response[2]);
00907 ST33_TRACE("TPM TIS response header cc=0x%08lX total=%u tpm_rc=0x%08lX\r\n",
00908           (unsigned long)command_code,
00909           (unsigned int)rsp_total,
00910           (unsigned long)st33_be32_read(&response[6]));
00911 if ((rsp_total < ST33_TPM_HEADER_SIZE) || (rsp_total > ST33KTPM2X_RSP_MAX_BYTES))
00912 {
00913     (void)st33_write_sts(ctx, ST33_STS_COMMAND_READY);
00914     return st33_trace_fail(ctx, "response_size_invalid", ST33KTPM2X_EPROTO, true);
00915 }
00916 if (rsp_total > response_capacity)
00917 {
00918     (void)st33_write_sts(ctx, ST33_STS_COMMAND_READY);
00919     return st33_trace_fail(ctx, "response_capacity", ST33KTPM2X_EOVERFLOW, true);
00920 }
00921
00922 if (rsp_total > ST33_TPM_HEADER_SIZE)
00923 {
00924     rc = st33_read_response_stream(ctx,
00925                                   &response[ST33_TPM_HEADER_SIZE],
00926                                   (uint16_t)(rsp_total - ST33_TPM_HEADER_SIZE));
00927     if (rc != ST33KTPM2X_OK)
00928     {
00929         (void)st33_write_sts(ctx, ST33_STS_COMMAND_READY);
00930         return st33_trace_fail(ctx, "read_response_body", rc, true);
00931     }
00932 }
00933
00934 *response_len = rsp_total;
00935 if (tpm_rc != NULL)
00936 {
00937     *tpm_rc = st33_be32_read(&response[6]);
00938 }
00939
00940 (void)st33_write_sts(ctx, ST33_STS_COMMAND_READY);
00941 (void)st33ktpm2x_release_locality0(ctx);
00942

```



```

00943     if ((tpm_rc != NULL) && (*tpm_rc != ST33KTPM2X_TPM2_RC_SUCCESS))
00944     {
00945         ST33_TRACE("TPM TIS command TPM_RC cc=0x%08lX tpm_rc=0x%08lX\r\n",
00946             (unsigned long)command_code,
00947             (unsigned long)*tpm_rc);
00948         return ST33KTPM2X_ETPM_RC;
00949     }
00950
00951     ST33_TRACE("TPM TIS transceive OK cc=0x%08lX rsp_len=%u\r\n",
00952         (unsigned long)command_code,
00953         (unsigned int)*response_len);
00954     return ST33KTPM2X_OK;
00955 }
00956
00957 st33ktpm2x_status_t st33ktpm2x_tpm2_startup(st33ktpm2x_t *ctx,
00958     uint16_t startup_type,
00959     uint32_t *tpm_rc)
00960 {
00961     uint8_t cmd[12];
00962     uint8_t rsp[ST33_TPM_HEADER_SIZE];
00963     uint16_t rsp_len = 0U;
00964
00965     st33_bel6_write(&cmd[0], ST33KTPM2X_TPM2_ST_NO_SESSIONS);
00966     st33_be32_write(&cmd[2], (uint32_t)sizeof(cmd));
00967     st33_be32_write(&cmd[6], ST33KTPM2X_TPM2_CC_STARTUP);
00968     st33_bel6_write(&cmd[10], startup_type);
00969
00970     return st33ktpm2x_transceive(ctx, cmd, (uint16_t)sizeof(cmd), rsp, (uint16_t)sizeof(rsp),
00971         &rsp_len, tpm_rc);
00972 }
00973 st33ktpm2x_status_t st33ktpm2x_tpm2_self_test(st33ktpm2x_t *ctx,
00974     bool full_test,
00975     uint32_t *tpm_rc)
00976 {
00977     uint8_t cmd[11];
00978     uint8_t rsp[ST33_TPM_HEADER_SIZE];
00979     uint16_t rsp_len = 0U;
00980
00981     st33_bel6_write(&cmd[0], ST33KTPM2X_TPM2_ST_NO_SESSIONS);
00982     st33_be32_write(&cmd[2], (uint32_t)sizeof(cmd));
00983     st33_be32_write(&cmd[6], ST33KTPM2X_TPM2_CC_SELF_TEST);
00984     cmd[10] = full_test ? 1U : 0U;
00985
00986     return st33ktpm2x_transceive(ctx, cmd, (uint16_t)sizeof(cmd), rsp, (uint16_t)sizeof(rsp),
00987         &rsp_len, tpm_rc);
00988 }
00989 st33ktpm2x_status_t st33ktpm2x_tpm2_get_random(st33ktpm2x_t *ctx,
00990     uint16_t requested_bytes,
00991     uint8_t *random_out,
00992     uint16_t random_capacity,
00993     uint16_t *random_len,
00994     uint32_t *tpm_rc)
00995 {
00996     uint8_t cmd[12];
00997     uint8_t rsp[ST33_TPM_HEADER_SIZE + 2U + ST33_TPM_RANDOM_MAX_BYTES];
00998     uint16_t rsp_len = 0U;
00999     uint16_t rnd_len = 0U;
01000     st33ktpm2x_status_t rc;
01001
01002     if ((random_out == NULL) ||
01003         (random_len == NULL) ||
01004         (requested_bytes == 0U) ||
01005         (requested_bytes > ST33_TPM_RANDOM_MAX_BYTES) ||
01006         (random_capacity < requested_bytes))
01007     {
01008         return ST33KTPM2X_EINVAL;
01009     }
01010
01011     *random_len = 0U;
01012
01013     st33_bel6_write(&cmd[0], ST33KTPM2X_TPM2_ST_NO_SESSIONS);
01014     st33_be32_write(&cmd[2], (uint32_t)sizeof(cmd));
01015     st33_be32_write(&cmd[6], ST33KTPM2X_TPM2_CC_GET_RANDOM);
01016     st33_bel6_write(&cmd[10], requested_bytes);
01017
01018     rc = st33ktpm2x_transceive(ctx, cmd, (uint16_t)sizeof(cmd), rsp, (uint16_t)sizeof(rsp), &rsp_len,
01019         tpm_rc);
01019     if (rc != ST33KTPM2X_OK)
01020     {
01021         return rc;
01022     }
01023
01024     if (rsp_len < (ST33_TPM_HEADER_SIZE + 2U))
01025     {
01026         return ST33KTPM2X_EPROTO;
01027     }

```

```

01027     }
01028
01029     rnd_len = st33ktpm2x_read(&rsp[ST33_TPM_HEADER_SIZE]);
01030     if ((uint16_t)(ST33_TPM_HEADER_SIZE + 2U + rnd_len) > rsp_len)
01031     {
01032         return ST33KTPM2X_EPROTO;
01033     }
01034     if (rnd_len > random_capacity)
01035     {
01036         return ST33KTPM2X_EOVERFLOW;
01037     }
01038
01039     memcpy(random_out, &rsp[ST33_TPM_HEADER_SIZE + 2U], rnd_len);
01040     *random_len = rnd_len;
01041     return ST33KTPM2X_OK;
01042 }
01043
01044 st33ktpm2x_status_t st33ktpm2x_tpm2_get_capability(st33ktpm2x_t *ctx,
01045                                                     uint32_t capability,
01046                                                     uint32_t property,
01047                                                     uint32_t property_count,
01048                                                     uint8_t *payload_out,
01049                                                     uint16_t payload_capacity,
01050                                                     uint16_t *payload_len,
01051                                                     bool *more_data,
01052                                                     uint32_t *tpm_rc)
01053 {
01054     uint8_t cmd[22];
01055     uint8_t rsp[ST33KTPM2X_RSP_MAX_BYTES];
01056     uint16_t rsp_len = 0U;
01057     uint16_t capability_data_len = 0U;
01058     st33ktpm2x_status_t rc;
01059
01060     if ((payload_out == NULL) || (payload_len == NULL) || (more_data == NULL))
01061     {
01062         return ST33KTPM2X_EINVAL;
01063     }
01064     *payload_len = 0U;
01065     *more_data = false;
01066
01067     st33ktpm2x_write(&cmd[0], ST33KTPM2X_TPM2_ST_NO_SESSIONS);
01068     st33ktpm2x_write(&cmd[2], (uint32_t)sizeof(cmd));
01069     st33ktpm2x_write(&cmd[6], ST33KTPM2X_TPM2_CC_GET_CAPABILITY);
01070     st33ktpm2x_write(&cmd[10], capability);
01071     st33ktpm2x_write(&cmd[14], property);
01072     st33ktpm2x_write(&cmd[18], property_count);
01073
01074     rc = st33ktpm2x_transceive(ctx, cmd, (uint16_t)sizeof(cmd), rsp, (uint16_t)sizeof(rsp), &rsp_len,
01075                                tpm_rc);
01076     if (rc != ST33KTPM2X_OK)
01077     {
01078         return rc;
01079     }
01080     if (rsp_len < (ST33_TPM_HEADER_SIZE + 1U))
01081     {
01082         return ST33KTPM2X_EPROTO;
01083     }
01084
01085     *more_data = (rsp[ST33_TPM_HEADER_SIZE] != 0U);
01086     capability_data_len = (uint16_t)(rsp_len - ST33_TPM_HEADER_SIZE - 1U);
01087     if (capability_data_len > payload_capacity)
01088     {
01089         return ST33KTPM2X_EOVERFLOW;
01090     }
01091     memcpy(payload_out, &rsp[ST33_TPM_HEADER_SIZE + 1U], capability_data_len);
01092     *payload_len = capability_data_len;
01093     return ST33KTPM2X_OK;
01094 }
01095
01096 st33ktpm2x_status_t st33ktpm2x_tpm2_pcr_read_sha256(st33ktpm2x_t *ctx,
01097                                                        uint8_t pcr_index,
01098                                                        uint8_t digest_sha256_out[32],
01099                                                        uint32_t *update_counter_out,
01100                                                        uint32_t *tpm_rc)
01101 {
01102     uint8_t cmd[20];
01103     uint8_t rsp[ST33KTPM2X_RSP_MAX_BYTES];
01104     uint16_t rsp_len = 0U;
01105     uint16_t off = ST33_TPM_HEADER_SIZE;
01106     uint32_t sel_count;
01107     uint32_t dig_count;
01108     uint8_t sizeof_select;
01109     uint16_t dig_len;
01110     uint32_t i;
01111     st33ktpm2x_status_t rc;
01112

```

```

01113     if ((digest_sha256_out == NULL) || (update_counter_out == NULL) || (pcr_index > 23U))
01114     {
01115         return ST33KTPM2X_EINVAL;
01116     }
01117
01118     memset(digest_sha256_out, 0, 32U);
01119     *update_counter_out = 0UL;
01120
01121     st33_be16_write(&cmd[0], ST33KTPM2X_TPM2_ST_NO_SESSIONS);
01122     st33_be32_write(&cmd[2], (uint32_t)sizeof(cmd));
01123     st33_be32_write(&cmd[6], ST33KTPM2X_TPM2_CC_PCR_READ);
01124
01125     st33_be32_write(&cmd[10], 1UL); /* TPML_PCR_SELECTION.count */
01126     st33_be16_write(&cmd[14], ST33KTPM2X_TPM2_ALG_SHA256);
01127     cmd[16] = 3U; /* sizeofSelect */
01128     cmd[17] = 0U;
01129     cmd[18] = 0U;
01130     cmd[19] = 0U;
01131     cmd[17U + (uint8_t)(pcr_index / 8U)] = (uint8_t)(1U << (pcr_index % 8U));
01132
01133     rc = st33ktpm2x_transceive(ctx, cmd, (uint16_t)sizeof(cmd), rsp, (uint16_t)sizeof(rsp), &rsp_len,
tpm_rc);
01134     if (rc != ST33KTPM2X_OK)
01135     {
01136         return rc;
01137     }
01138
01139     if (rsp_len < (ST33_TPM_HEADER_SIZE + 4U + 4U + 4U + 2U))
01140     {
01141         return ST33KTPM2X_EPROTO;
01142     }
01143
01144     *update_counter_out = st33_be32_read(&rsp[off]);
01145     off = (uint16_t)(off + 4U);
01146
01147     sel_count = st33_be32_read(&rsp[off]);
01148     off = (uint16_t)(off + 4U);
01149     for (i = 0UL; i < sel_count; i++)
01150     {
01151         if ((uint16_t)(off + 3U) > rsp_len)
01152         {
01153             return ST33KTPM2X_EPROTO;
01154         }
01155         off = (uint16_t)(off + 2U); /* hash */
01156         sizeof_select = rsp[off];
01157         off = (uint16_t)(off + 1U); /* sizeofSelect */
01158         if ((uint16_t)(off + sizeof_select) > rsp_len)
01159         {
01160             return ST33KTPM2X_EPROTO;
01161         }
01162         off = (uint16_t)(off + sizeof_select);
01163     }
01164
01165     if ((uint16_t)(off + 4U + 2U) > rsp_len)
01166     {
01167         return ST33KTPM2X_EPROTO;
01168     }
01169
01170     dig_count = st33_be32_read(&rsp[off]);
01171     off = (uint16_t)(off + 4U);
01172     if (dig_count == 0UL)
01173     {
01174         return ST33KTPM2X_EPROTO;
01175     }
01176
01177     dig_len = st33_be16_read(&rsp[off]);
01178     off = (uint16_t)(off + 2U);
01179     if (dig_len != 32U)
01180     {
01181         return ST33KTPM2X_EPROTO;
01182     }
01183     if ((uint16_t)(off + dig_len) > rsp_len)
01184     {
01185         return ST33KTPM2X_EPROTO;
01186     }
01187
01188     memcpy(digest_sha256_out, &rsp[off], 32U);
01189     return ST33KTPM2X_OK;
01190 }
01191
01192 st33ktpm2x_status_t st33ktpm2x_tpm2_start_auth_session(st33ktpm2x_t *ctx,
01193                                                         uint32_t *session_handle_out,
01194                                                         uint32_t *tpm_rc)
01195 {
01196     (void)ctx;
01197     if (session_handle_out != NULL)
01198     {

```

```

01199     *session_handle_out = 0UL;
01200 }
01201 if (tpm_rc != NULL)
01202 {
01203     *tpm_rc = 0UL;
01204 }
01205 return ST33KTPM2X_ENOTSUP;
01206 }
01207
01208 st33ktpm2x_status_t st33ktpm2x_tpm2_policy_pcr(st33ktpm2x_t *ctx,
01209                                                uint32_t session_handle,
01210                                                uint8_t pcr_index,
01211                                                uint32_t *tpm_rc)
01212 {
01213     (void)ctx;
01214     (void)session_handle;
01215     (void)pcr_index;
01216     if (tpm_rc != NULL)
01217     {
01218         *tpm_rc = 0UL;
01219     }
01220     return ST33KTPM2X_ENOTSUP;
01221 }
01222
01223 st33ktpm2x_status_t st33ktpm2x_tpm2_policy_physical_presence(st33ktpm2x_t *ctx,
01224                                                                uint32_t session_handle,
01225                                                                uint32_t *tpm_rc)
01226 {
01227     (void)ctx;
01228     (void)session_handle;
01229     if (tpm_rc != NULL)
01230     {
01231         *tpm_rc = 0UL;
01232     }
01233     return ST33KTPM2X_ENOTSUP;
01234 }
01235
01236 st33ktpm2x_status_t st33ktpm2x_tpm2_policy_command_code(st33ktpm2x_t *ctx,
01237                                                           uint32_t session_handle,
01238                                                           uint32_t command_code,
01239                                                           uint32_t *tpm_rc)
01240 {
01241     (void)ctx;
01242     (void)session_handle;
01243     (void)command_code;
01244     if (tpm_rc != NULL)
01245     {
01246         *tpm_rc = 0UL;
01247     }
01248     return ST33KTPM2X_ENOTSUP;
01249 }
01250
01251 st33ktpm2x_status_t st33ktpm2x_tpm2_policy_or(st33ktpm2x_t *ctx,
01252                                                  uint32_t session_handle,
01253                                                  const uint8_t *digest_list,
01254                                                  uint8_t digest_count,
01255                                                  uint8_t digest_size,
01256                                                  uint32_t *tpm_rc)
01257 {
01258     (void)ctx;
01259     (void)session_handle;
01260     (void)digest_list;
01261     (void)digest_count;
01262     (void)digest_size;
01263     if (tpm_rc != NULL)
01264     {
01265         *tpm_rc = 0UL;
01266     }
01267     return ST33KTPM2X_ENOTSUP;
01268 }
01269
01270 st33ktpm2x_status_t st33ktpm2x_tpm2_nv_define(st33ktpm2x_t *ctx,
01271                                                  uint32_t nv_index,
01272                                                  uint16_t data_size,
01273                                                  uint32_t attributes,
01274                                                  uint32_t *tpm_rc)
01275 {
01276     uint8_t cmd[45];
01277     uint8_t rsp[ST33_TPM_HEADER_SIZE + 4U + ST33_TPM_PW_AUTH_SIZE];
01278     uint16_t off = 0U;
01279     uint16_t rsp_len = 0U;
01280
01281     if ((ctx == NULL) || (!ctx->initialized) || (data_size == 0U))
01282     {
01283         return ST33KTPM2X_EINVAL;
01284     }
01285 }

```

```

01286     st33_bel6_write(&cmd[off], ST33KTPM2X_TPM2_ST_SESSIONS);
01287     off = (uint16_t)(off + 2U);
01288     st33_be32_write(&cmd[off], (uint32_t)sizeof(cmd));
01289     off = (uint16_t)(off + 4U);
01290     st33_be32_write(&cmd[off], ST33KTPM2X_TPM2_CC_NV_DEFINE_SPACE);
01291     off = (uint16_t)(off + 4U);
01292
01293     st33_be32_write(&cmd[off], ST33_TPM_RH_OWNER);
01294     off = (uint16_t)(off + 4U);
01295
01296     st33_be32_write(&cmd[off], ST33_TPM_PW_AUTH_SIZE);
01297     off = (uint16_t)(off + 4U);
01298     off = (uint16_t)(off + st33_write_empty_password_auth(&cmd[off]));
01299
01300     st33_bel6_write(&cmd[off], 0U); /* TPM2B_AUTH userAuth */
01301     off = (uint16_t)(off + 2U);
01302
01303     st33_bel6_write(&cmd[off], 14U); /* TPM2B_NV_PUBLIC.size */
01304     off = (uint16_t)(off + 2U);
01305     st33_be32_write(&cmd[off], nv_index);
01306     off = (uint16_t)(off + 4U);
01307     st33_bel6_write(&cmd[off], ST33KTPM2X_TPM2_ALG_SHA256);
01308     off = (uint16_t)(off + 2U);
01309     st33_be32_write(&cmd[off], attributes);
01310     off = (uint16_t)(off + 4U);
01311     st33_bel6_write(&cmd[off], 0U); /* TPM2B_DIGEST authPolicy */
01312     off = (uint16_t)(off + 2U);
01313     st33_bel6_write(&cmd[off], data_size);
01314     off = (uint16_t)(off + 2U);
01315
01316     if (off != sizeof(cmd))
01317     {
01318         return ST33KTPM2X_EPROTO;
01319     }
01320
01321     return st33ktpm2x_transceive(ctx,
01322                                  cmd,
01323                                  (uint16_t)sizeof(cmd),
01324                                  rsp,
01325                                  (uint16_t)sizeof(rsp),
01326                                  &rsp_len,
01327                                  tpm_rc);
01328 }
01329
01330 st33ktpm2x_status_t st33ktpm2x_tpm2_nv_read(st33ktpm2x_t *ctx,
01331                                               uint32_t nv_index,
01332                                               uint16_t offset,
01333                                               uint8_t *out_data,
01334                                               uint16_t out_capacity,
01335                                               uint16_t *out_len,
01336                                               uint32_t *tpm_rc)
01337 {
01338     uint8_t cmd[35];
01339     uint8_t rsp[ST33_TPM_HEADER_SIZE + 4U + 2U + ST33_TPM_NV_TRANSFER_MAX_BYTES +
01340 ST33_TPM_PW_AUTH_SIZE];
01341     uint16_t cmd_off = 0U;
01342     uint16_t rsp_off = ST33_TPM_HEADER_SIZE;
01343     uint16_t rsp_len = 0U;
01344     uint16_t data_len = 0U;
01345     st33ktpm2x_status_t rc;
01346
01347     if ((ctx == NULL) ||
01348         (!ctx->initialized) ||
01349         (out_data == NULL) ||
01350         (out_len == NULL) ||
01351         (out_capacity == 0U) ||
01352         (out_capacity > ST33_TPM_NV_TRANSFER_MAX_BYTES))
01353     {
01354         return ST33KTPM2X_EINVAL;
01355     }
01356
01357     *out_len = 0U;
01358
01359     st33_bel6_write(&cmd[cmd_off], ST33KTPM2X_TPM2_ST_SESSIONS);
01360     cmd_off = (uint16_t)(cmd_off + 2U);
01361     st33_be32_write(&cmd[cmd_off], (uint32_t)sizeof(cmd));
01362     cmd_off = (uint16_t)(cmd_off + 4U);
01363     st33_be32_write(&cmd[cmd_off], ST33KTPM2X_TPM2_CC_NV_READ);
01364     cmd_off = (uint16_t)(cmd_off + 4U);
01365
01366     st33_be32_write(&cmd[cmd_off], ST33_TPM_RH_OWNER);
01367     cmd_off = (uint16_t)(cmd_off + 4U);
01368     st33_be32_write(&cmd[cmd_off], nv_index);
01369     cmd_off = (uint16_t)(cmd_off + 4U);
01370
01371     st33_be32_write(&cmd[cmd_off], ST33_TPM_PW_AUTH_SIZE);
01372     cmd_off = (uint16_t)(cmd_off + 4U);

```

```

01372     cmd_off = (uint16_t)(cmd_off + st33_write_empty_password_auth(&cmd[cmd_off]));
01373
01374     st33_bel6_write(&cmd[cmd_off], out_capacity);
01375     cmd_off = (uint16_t)(cmd_off + 2U);
01376     st33_bel6_write(&cmd[cmd_off], offset);
01377     cmd_off = (uint16_t)(cmd_off + 2U);
01378
01379     if (cmd_off != sizeof(cmd))
01380     {
01381         return ST33KTPM2X_EPROTO;
01382     }
01383
01384     rc = st33ktpm2x_transceive(ctx,
01385                               cmd,
01386                               (uint16_t)sizeof(cmd),
01387                               rsp,
01388                               (uint16_t)sizeof(rsp),
01389                               &rsp_len,
01390                               tpm_rc);
01391
01392     if (rc != ST33KTPM2X_OK)
01393     {
01394         return rc;
01395     }
01396
01397     if (rsp_len < (ST33_TPM_HEADER_SIZE + 4U + 2U))
01398     {
01399         return ST33KTPM2X_EPROTO;
01400     }
01401
01402     rsp_off = (uint16_t)(rsp_off + 4U); /* parameterSize */
01403     data_len = st33_bel6_read(&rsp[rsp_off]);
01404     rsp_off = (uint16_t)(rsp_off + 2U);
01405     if ((uint16_t)(rsp_off + data_len) > rsp_len)
01406     {
01407         return ST33KTPM2X_EPROTO;
01408     }
01409     if (data_len > out_capacity)
01410     {
01411         return ST33KTPM2X_EOVERFLOW;
01412     }
01413     memcpy(out_data, &rsp[rsp_off], data_len);
01414     *out_len = data_len;
01415     return ST33KTPM2X_OK;
01416 }
01417
01418 static st33ktpm2x_status_t st33_tpm2_nv_write_auth(st33ktpm2x_t *ctx,
01419                                                     uint32_t auth_handle,
01420                                                     uint32_t nv_index,
01421                                                     uint16_t offset,
01422                                                     const uint8_t *data,
01423                                                     uint16_t data_len,
01424                                                     uint32_t *tpm_rc)
01425 {
01426     uint8_t cmd[35U + ST33_TPM_NV_TRANSFER_MAX_BYTES];
01427     uint8_t rsp[ST33_TPM_HEADER_SIZE + 4U + ST33_TPM_PW_AUTH_SIZE];
01428     uint16_t off = 0U;
01429     uint16_t rsp_len = 0U;
01430     uint16_t cmd_len;
01431
01432     if ((ctx == NULL) ||
01433         (!ctx->initialized) ||
01434         ((data == NULL) && (data_len > 0U)) ||
01435         (data_len > ST33_TPM_NV_TRANSFER_MAX_BYTES))
01436     {
01437         return ST33KTPM2X_EINVAL;
01438     }
01439
01440     cmd_len = (uint16_t)(35U + data_len);
01441
01442     st33_bel6_write(&cmd[off], ST33KTPM2X_TPM2_ST_SESSIONS);
01443     off = (uint16_t)(off + 2U);
01444     st33_be32_write(&cmd[off], cmd_len);
01445     off = (uint16_t)(off + 4U);
01446     st33_be32_write(&cmd[off], ST33KTPM2X_TPM2_CC_NV_WRITE);
01447     off = (uint16_t)(off + 4U);
01448
01449     st33_be32_write(&cmd[off], auth_handle);
01450     off = (uint16_t)(off + 4U);
01451     st33_be32_write(&cmd[off], nv_index);
01452     off = (uint16_t)(off + 4U);
01453
01454     st33_be32_write(&cmd[off], ST33_TPM_PW_AUTH_SIZE);
01455     off = (uint16_t)(off + 4U);
01456     off = (uint16_t)(off + st33_write_empty_password_auth(&cmd[off]));
01457
01458     st33_bel6_write(&cmd[off], data_len);

```

```

01460     off = (uint16_t)(off + 2U);
01461     if (data_len > 0U)
01462     {
01463         memcpy(&cmd[off], data, data_len);
01464         off = (uint16_t)(off + data_len);
01465     }
01466     st33_bel6_write(&cmd[off], offset);
01467     off = (uint16_t)(off + 2U);
01468
01469     if (off != cmd_len)
01470     {
01471         return ST33KTPM2X_EPROTO;
01472     }
01473
01474     return st33ktpm2x_transceive(ctx,
01475                                   cmd,
01476                                   cmd_len,
01477                                   rsp,
01478                                   (uint16_t)sizeof(rsp),
01479                                   &rsp_len,
01480                                   tpm_rc);
01481 }
01482
01483 st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write(st33ktpm2x_t *ctx,
01484                                                uint32_t nv_index,
01485                                                uint16_t offset,
01486                                                const uint8_t *data,
01487                                                uint16_t data_len,
01488                                                uint32_t *tpm_rc)
01489 {
01490     return st33_tpm2_nv_write_auth(ctx,
01491                                     ST33_TPM_RH_OWNER,
01492                                     nv_index,
01493                                     offset,
01494                                     data,
01495                                     data_len,
01496                                     tpm_rc);
01497 }
01498
01499 st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write_platform_pp(st33ktpm2x_t *ctx,
01500                                                            uint32_t nv_index,
01501                                                            uint16_t offset,
01502                                                            const uint8_t *data,
01503                                                            uint16_t data_len,
01504                                                            uint32_t *tpm_rc)
01505 {
01506     return st33_tpm2_nv_write_auth(ctx,
01507                                     ST33_TPM_RH_PLATFORM,
01508                                     nv_index,
01509                                     offset,
01510                                     data,
01511                                     data_len,
01512                                     tpm_rc);
01513 }
01514
01515 st33ktpm2x_status_t st33ktpm2x_tpm2_flush_context(st33ktpm2x_t *ctx,
01516                                                     uint32_t handle,
01517                                                     uint32_t *tpm_rc)
01518 {
01519     (void)ctx;
01520     (void)handle;
01521     if (tpm_rc != NULL)
01522     {
01523         *tpm_rc = 0UL;
01524     }
01525     return ST33KTPM2X_ENOTSUP;
01526 }

```

12.87 Dokumentacja pliku Core/App/st33ktpm2x_lib/st33ktpm2x.h

High-level and transport-level driver for ST33KTPM2X over I2C (TIS/FIFO).

```

#include "stm32u5xx_hal.h"
#include <stdbool.h>
#include <stdint.h>

```

Struktury danych

- struct [st33ktpm2x_cfg_t](#)

Runtime configuration for ST33KTPM2X instance.

- struct `st33ktpm2x_t`
Driver context for one TPM device.

Definicje

Default Configuration

- #define `ST33KTPM2X_I2C_ADDR_DEFAULT` 0x2EU
Default 7-bit TPM I2C address.
- #define `ST33KTPM2X_CMD_MAX_BYTES` 1024U
Safe default max command size.
- #define `ST33KTPM2X_RSP_MAX_BYTES` 1024U
Safe default max response size.

TPM2 Common Constants

- #define `ST33KTPM2X_TPM2_ST_NO_SESSIONS` 0x8001U
- #define `ST33KTPM2X_TPM2_ST_SESSIONS` 0x8002U
- #define `ST33KTPM2X_TPM2_RC_SUCCESS` 0x00000000UL
- #define `ST33KTPM2X_TPM2_CC_STARTUP` 0x00000144UL
- #define `ST33KTPM2X_TPM2_CC_SELF_TEST` 0x00000143UL
- #define `ST33KTPM2X_TPM2_CC_GET_RANDOM` 0x0000017BUL
- #define `ST33KTPM2X_TPM2_CC_GET_CAPABILITY` 0x0000017AUL
- #define `ST33KTPM2X_TPM2_CC_PCR_READ` 0x0000017EUL
- #define `ST33KTPM2X_TPM2_CC_NV_DEFINE_SPACE` 0x0000012AUL
- #define `ST33KTPM2X_TPM2_CC_NV_WRITE` 0x00000137UL
- #define `ST33KTPM2X_TPM2_CC_NV_READ` 0x0000014EUL
- #define `ST33KTPM2X_TPM2_SU_CLEAR` 0x0000U
- #define `ST33KTPM2X_TPM2_SU_STATE` 0x0001U
- #define `ST33KTPM2X_TPM2_ALG_SHA256` 0x000BU
- #define `ST33KTPM2X_TPM2_CAP_ALGS` 0x00000000UL
- #define `ST33KTPM2X_TPM2_CAP_HANDLES` 0x00000001UL
- #define `ST33KTPM2X_TPM2_CAP_COMMANDS` 0x00000002UL
- #define `ST33KTPM2X_TPM2_CAP_PCERS` 0x00000005UL
- #define `ST33KTPM2X_TPM2_CAP_TPM_PROPERTIES` 0x00000006UL

Wyliczenia

- enum `st33ktpm2x_status_t` {
 `ST33KTPM2X_OK` = 0 , `ST33KTPM2X_EINVAL` , `ST33KTPM2X_ESTATE` , `ST33KTPM2X_EHAL` ,
 `ST33KTPM2X_ETIMEOUT` , `ST33KTPM2X_EOVERFLOW` , `ST33KTPM2X_EPROTO` , `ST33KTPM2X_ETPM_RC`
 ,
 `ST33KTPM2X_ENOTSUP` }
Driver status/result codes.

Funkcje

- void `st33ktpm2x_default_cfg` (`st33ktpm2x_cfg_t` *cfg, `I2C_HandleTypeDef` *hi2c)
Fill config with safe defaults (including PB0 reset pin defaults from `main.h`).
- `st33ktpm2x_status_t` `st33ktpm2x_init` (`st33ktpm2x_t` *ctx, const `st33ktpm2x_cfg_t` *cfg)
Initialize TPM driver context.
- `st33ktpm2x_status_t` `st33ktpm2x_deinit` (`st33ktpm2x_t` *ctx)
Deinitialize context and release locality if needed.
- `st33ktpm2x_status_t` `st33ktpm2x_hard_reset` (`st33ktpm2x_t` *ctx)
Perform hardware reset pulse on TPM reset pin.
- `st33ktpm2x_status_t` `st33ktpm2x_davint_is_asserted` (`st33ktpm2x_t` *ctx, bool *asserted)
Read TPM_DAVINT# / PIRQ state.
- `st33ktpm2x_status_t` `st33ktpm2x_pp_is_pressed` (`st33ktpm2x_t` *ctx, bool *pressed)
Read state of optional Physical Presence button.

- `st33ktpm2x_status_t st33ktpm2x_pp_wait_pressed (st33ktpm2x_t *ctx, uint32_t timeout_ms)`
Wait until Physical Presence button becomes active.
- `st33ktpm2x_status_t st33ktpm2x_request_locality0 (st33ktpm2x_t *ctx)`
Acquire TPM locality 0.
- `st33ktpm2x_status_t st33ktpm2x_release_locality0 (st33ktpm2x_t *ctx)`
Release TPM locality 0.
- `st33ktpm2x_status_t st33ktpm2x_read_identity (st33ktpm2x_t *ctx, uint32_t *did_vid, uint8_t *rid)`
Read TPM DID_VID and RID registers.
- `st33ktpm2x_status_t st33ktpm2x_transceive (st33ktpm2x_t *ctx, const uint8_t *command, uint16_t ↵
t command_len, uint8_t *response, uint16_t response_capacity, uint16_t *response_len, uint32_t *tpm_rc)`
Send one raw TPM command and receive full response.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_startup (st33ktpm2x_t *ctx, uint16_t startup_type, uint32_t *tpm_rc)`
Execute TPM2_Startup.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_self_test (st33ktpm2x_t *ctx, bool full_test, uint32_t *tpm_rc)`
Execute TPM2_SelfTest.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_get_random (st33ktpm2x_t *ctx, uint16_t requested_bytes, uint8_t ↵
*random_out, uint16_t random_capacity, uint16_t *random_len, uint32_t *tpm_rc)`
Execute TPM2_GetRandom.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_get_capability (st33ktpm2x_t *ctx, uint32_t capability, uint32_t pro-
perty, uint32_t property_count, uint8_t *payload_out, uint16_t payload_capacity, uint16_t *payload_len, bool
*more_data, uint32_t *tpm_rc)`
Execute TPM2_GetCapability and return raw capability payload.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_pcr_read_sha256 (st33ktpm2x_t *ctx, uint8_t pcr_index, uint8_t ↵
t digest_sha256_out[32], uint32_t *update_counter_out, uint32_t *tpm_rc)`
Execute TPM2_PCR_Read for one SHA-256 PCR index.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_start_auth_session (st33ktpm2x_t *ctx, uint32_t *session_handle ↵
_out, uint32_t *tpm_rc)`
Start TPM2 policy/auth session helper (scaffold API).
- `st33ktpm2x_status_t st33ktpm2x_tpm2_policy_pcr (st33ktpm2x_t *ctx, uint32_t session_handle, uint8_t ↵
t pcr_index, uint32_t *tpm_rc)`
Apply PolicyPCR in an existing policy session (scaffold API).
- `st33ktpm2x_status_t st33ktpm2x_tpm2_policy_physical_presence (st33ktpm2x_t *ctx, uint32_t session ↵
handle, uint32_t *tpm_rc)`
Apply Physical Presence policy branch (scaffold API).
- `st33ktpm2x_status_t st33ktpm2x_tpm2_policy_command_code (st33ktpm2x_t *ctx, uint32_t session ↵
handle, uint32_t command_code, uint32_t *tpm_rc)`
Apply PolicyCommandCode in an existing policy session (scaffold API).
- `st33ktpm2x_status_t st33ktpm2x_tpm2_policy_or (st33ktpm2x_t *ctx, uint32_t session_handle, const
uint8_t *digest_list, uint8_t digest_count, uint8_t digest_size, uint32_t *tpm_rc)`
Apply PolicyOR branch combiner (scaffold API).
- `st33ktpm2x_status_t st33ktpm2x_tpm2_nv_define (st33ktpm2x_t *ctx, uint32_t nv_index, uint16_t data_size,
uint32_t attributes, uint32_t *tpm_rc)`
Define NV index using owner empty-password authorization.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_nv_read (st33ktpm2x_t *ctx, uint32_t nv_index, uint16_t offset,
uint8_t *out_data, uint16_t out_capacity, uint16_t *out_len, uint32_t *tpm_rc)`
Read bytes from NV index using owner empty-password authorization.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write (st33ktpm2x_t *ctx, uint32_t nv_index, uint16_t offset,
const uint8_t *data, uint16_t data_len, uint32_t *tpm_rc)`
Write bytes to NV index using owner empty-password authorization.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write_platform_pp (st33ktpm2x_t *ctx, uint32_t nv_index,
uint16_t offset, const uint8_t *data, uint16_t data_len, uint32_t *tpm_rc)`
Write bytes to NV index using platform authorization with hardware Physical Presence.
- `st33ktpm2x_status_t st33ktpm2x_tpm2_flush_context (st33ktpm2x_t *ctx, uint32_t handle, uint32_t *tpm_rc)`
Flush transient context/session handle (scaffold API).

12.87.1 Opis szczegółowy

High-level and transport-level driver for ST33KTPM2X over I2C (TIS/FIFO).
This module provides:

- low-level TPM TIS over I2C access,
- locality handling,
- command/response transport,
- selected TPM2 helper commands for security feature development.

The implementation is designed to be extensible. For advanced commands (sessions, HMAC, policy, NV authorization), use raw command transport APIs and add dedicated marshaling helpers in your application layer.
Definicja w pliku [st33ktpm2x.h](#).

12.87.2 Dokumentacja definicji

12.87.2.1 ST33KTPM2X_CMD_MAX_BYTES

```
#define ST33KTPM2X_CMD_MAX_BYTES 1024U
```

Safe default max command size.
Definicja w linii 32 pliku [st33ktpm2x.h](#).
Odwołania w [st33_i2c_write\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.87.2.2 ST33KTPM2X_I2C_ADDR_DEFAULT

```
#define ST33KTPM2X_I2C_ADDR_DEFAULT 0x2EU
```

Default 7-bit TPM I2C address.
Definicja w linii 31 pliku [st33ktpm2x.h](#).
Odwołania w [st33ktpm2x_default_cfg\(\)](#).

12.87.2.3 ST33KTPM2X_RSP_MAX_BYTES

```
#define ST33KTPM2X_RSP_MAX_BYTES 1024U
```

Safe default max response size.
Definicja w linii 33 pliku [st33ktpm2x.h](#).
Odwołania w [st33ktpm2x_tpm2_get_capability\(\)](#), [st33ktpm2x_tpm2_pcr_read_sha256\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.87.2.4 ST33KTPM2X_TPM2_ALG_SHA256

```
#define ST33KTPM2X_TPM2_ALG_SHA256 0x000BU
```

Definicja w linii 55 pliku [st33ktpm2x.h](#).

12.87.2.5 ST33KTPM2X_TPM2_CAP_ALGS

```
#define ST33KTPM2X_TPM2_CAP_ALGS 0x00000000UL
```

Definicja w linii 57 pliku [st33ktpm2x.h](#).

12.87.2.6 ST33KTPM2X_TPM2_CAP_COMMANDS

```
#define ST33KTPM2X_TPM2_CAP_COMMANDS 0x00000002UL
```

Definicja w linii 59 pliku [st33ktpm2x.h](#).

12.87.2.7 ST33KTPM2X_TPM2_CAP_HANDLES

```
#define ST33KTPM2X_TPM2_CAP_HANDLES 0x00000001UL
```

Definicja w linii 58 pliku [st33ktpm2x.h](#).

12.87.2.8 ST33KTPM2X_TPM2_CAP_PCRS

```
#define ST33KTPM2X_TPM2_CAP_PCRS 0x00000005UL
```

Definicja w linii 60 pliku [st33ktpm2x.h](#).

12.87.2.9 ST33KTPM2X_TPM2_CAP_TPM_PROPERTIES

```
#define ST33KTPM2X_TPM2_CAP_TPM_PROPERTIES 0x00000006UL
```

Definicja w linii 61 pliku [st33ktpm2x.h](#).

12.87.2.10 ST33KTPM2X_TPM2_CC_GET_CAPABILITY

```
#define ST33KTPM2X_TPM2_CC_GET_CAPABILITY 0x0000017AUL
```

Definicja w linii 46 pliku [st33ktpm2x.h](#).

12.87.2.11 ST33KTPM2X_TPM2_CC_GET_RANDOM

```
#define ST33KTPM2X_TPM2_CC_GET_RANDOM 0x0000017BUL
```

Definicja w linii 45 pliku [st33ktpm2x.h](#).

12.87.2.12 ST33KTPM2X_TPM2_CC_NV_DEFINE_SPACE

```
#define ST33KTPM2X_TPM2_CC_NV_DEFINE_SPACE 0x0000012AUL
```

Definicja w linii 48 pliku [st33ktpm2x.h](#).

12.87.2.13 ST33KTPM2X_TPM2_CC_NV_READ

```
#define ST33KTPM2X_TPM2_CC_NV_READ 0x0000014EUL
```

Definicja w linii 50 pliku [st33ktpm2x.h](#).

12.87.2.14 ST33KTPM2X_TPM2_CC_NV_WRITE

```
#define ST33KTPM2X_TPM2_CC_NV_WRITE 0x00000137UL
```

Definicja w linii 49 pliku [st33ktpm2x.h](#).

12.87.2.15 ST33KTPM2X_TPM2_CC_PCR_READ

```
#define ST33KTPM2X_TPM2_CC_PCR_READ 0x0000017EUL
```

Definicja w linii 47 pliku [st33ktpm2x.h](#).

12.87.2.16 ST33KTPM2X_TPM2_CC_SELF_TEST

```
#define ST33KTPM2X_TPM2_CC_SELF_TEST 0x00000143UL
```

Definicja w linii 44 pliku [st33ktpm2x.h](#).

12.87.2.17 ST33KTPM2X_TPM2_CC_STARTUP

```
#define ST33KTPM2X_TPM2_CC_STARTUP 0x00000144UL
```

Definicja w linii 43 pliku [st33ktpm2x.h](#).

12.87.2.18 ST33KTPM2X_TPM2_RC_SUCCESS

```
#define ST33KTPM2X_TPM2_RC_SUCCESS 0x00000000UL
```

Definicja w linii 41 pliku [st33ktpm2x.h](#).

12.87.2.19 ST33KTPM2X_TPM2_ST_NO_SESSIONS

```
#define ST33KTPM2X_TPM2_ST_NO_SESSIONS 0x8001U
```

Definicja w linii 39 pliku [st33ktpm2x.h](#).

12.87.2.20 ST33KTPM2X_TPM2_ST_SESSIONS

```
#define ST33KTPM2X_TPM2_ST_SESSIONS 0x8002U
```

Definicja w linii 40 pliku [st33ktpm2x.h](#).

12.87.2.21 ST33KTPM2X_TPM2_SU_CLEAR

```
#define ST33KTPM2X_TPM2_SU_CLEAR 0x0000U
```

Definicja w linii 52 pliku [st33ktpm2x.h](#).

12.87.2.22 ST33KTPM2X_TPM2_SU_STATE

```
#define ST33KTPM2X_TPM2_SU_STATE 0x0001U
```

Definicja w linii 53 pliku [st33ktpm2x.h](#).

12.87.3 Dokumentacja typów wyliczanych

12.87.3.1 st33ktpm2x_status_t

```
enum st33ktpm2x_status_t
```

Driver status/result codes.

Wartości wyliczeń

ST33KTPM2X_OK	Operation completed successfully.
ST33KTPM2X_EINVAL	Invalid argument.
ST33KTPM2X_ESTATE	Invalid driver state.
ST33KTPM2X_EHAL	HAL I2C or GPIO operation failed.
ST33KTPM2X_ETIMEOUT	Timeout while waiting for TPM state.
ST33KTPM2X_EOVERFLOW	Provided buffer too small for response.
ST33KTPM2X_EPROTO	TPM protocol/TIS response format error.
ST33KTPM2X_ETPM_RC	TPM responded with non-success TPM2_RC.
ST33KTPM2X_ENOTSUP	Optional feature not configured/supported.

Definicja w linii 67 pliku [st33ktpm2x.h](#).

12.87.4 Dokumentacja funkcji

12.87.4.1 st33ktpm2x_davint_is_asserted()

```
st33ktpm2x_status_t st33ktpm2x_davint_is_asserted (
    st33ktpm2x_t * ctx,
    bool * asserted)
```

Read TPM_DAVINT# / PIRQ state.

Parametry

<i>ctx</i>	Driver context.
<i>asserted</i>	[out] <code>true</code> if active-low interrupt line is asserted.

Zwraca

Status code (`ST33KTPM2X_ENOTSUP` if DAVINT pin is not configured).

Definicja w linii 605 pliku `st33ktpm2x.c`.

Odwołuje się do `st33ktpm2x_t::cfg`, `st33ktpm2x_cfg_t::davint_active_state`, `st33ktpm2x_cfg_t::davint_pin`, `st33ktpm2x_cfg_t::davint_port`, `st33ktpm2x_t::initialized`, `ST33KTPM2X_EINVAL`, `ST33KTPM2X_ENOTSUP` i `ST33KTPM2X_OK`.

12.87.4.2 st33ktpm2x_default_cfg()

```
void st33ktpm2x_default_cfg (
    st33ktpm2x_cfg_t * cfg,
    I2C_HandleTypeDef * hi2c)
```

Fill config with safe defaults (including PBO reset pin defaults from `main.h`).

Parametry

<i>cfg</i>	[out] Configuration object to initialize.
<i>hi2c</i>	I2C peripheral handle (use <code>&hi2c3</code>).

Definicja w linii 518 pliku `st33ktpm2x.c`.

Odwołuje się do `st33ktpm2x_cfg_t::burst_timeout_ms`, `st33ktpm2x_cfg_t::davint_active_state`, `st33ktpm2x_cfg_t::davint_pin`, `st33ktpm2x_cfg_t::davint_port`, `st33ktpm2x_cfg_t::hi2c`, `st33ktpm2x_cfg_t::i2c_addr_7bit`, `st33ktpm2x_cfg_t::io_timeout_ms`, `st33ktpm2x_cfg_t::locality_timeout_ms`, `st33ktpm2x_cfg_t::pp_active_state`, `st33ktpm2x_cfg_t::pp_pin`, `st33ktpm2x_cfg_t::pp_port`, `st33ktpm2x_cfg_t::reset_pin`, `st33ktpm2x_cfg_t::reset_port`, `st33ktpm2x_cfg_t::reset_pulse_ms`, `st33ktpm2x_cfg_t::reset_recovery_ms` i `ST33KTPM2X_I2C_ADDR_DEFAULT`.

12.87.4.3 st33ktpm2x_deinit()

```
st33ktpm2x_status_t st33ktpm2x_deinit (
    st33ktpm2x_t * ctx)
```

Deinitialize context and release locality if needed.

Parametry

<i>ctx</i>	Driver context.
------------	-----------------

Zwraca

Status code.

Definicja w linii 565 pliku `st33ktpm2x.c`.

Odwołuje się do `st33ktpm2x_t::initialized`, `ST33KTPM2X_ESTATE`, `ST33KTPM2X_OK` i `st33ktpm2x_release_locality0()`.

12.87.4.4 st33ktpm2x_hard_reset()

```
st33ktpm2x_status_t st33ktpm2x_hard_reset (
    st33ktpm2x_t * ctx)
```

Perform hardware reset pulse on TPM reset pin.

Parametry

<i>ctx</i>	Driver context.
------------	-----------------

Zwraca

Status code.

Definicja w linii 577 pliku `st33ktpm2x.c`.

Odwołuje się do `st33ktpm2x_t::cfg`, `st33ktpm2x_t::initialized`, `st33ktpm2x_t::locality0_acquired`, `st33ktpm2x_cfg_t::reset_pin`, `st33ktpm2x_cfg_t::reset_port`, `st33ktpm2x_cfg_t::reset_pulse_ms`, `st33ktpm2x_cfg_t::reset_recovery_ms`, `st33_status_text()`, `st33_trace_tis()`, `st33_write_loc_sel()`, `ST33KTPM2X_ESTATE` i `ST33KTPM2X_OK`.

12.87.4.5 st33ktpm2x_init()

```
st33ktpm2x_status_t st33ktpm2x_init (
    st33ktpm2x_t * ctx,
    const st33ktpm2x_cfg_t * cfg)
```

Initialize TPM driver context.

Parametry

<i>ctx</i>	Driver context.
<i>cfg</i>	Runtime configuration.

Zwraca

Status code.

Definicja w linii 547 pliku `st33ktpm2x.c`.

Odwołuje się do `st33ktpm2x_t::cfg`, `st33ktpm2x_cfg_t::hi2c`, `st33ktpm2x_t::initialized`, `st33ktpm2x_cfg_t::reset_pin`, `st33ktpm2x_cfg_t::reset_port`, `ST33KTPM2X_EINVAL` i `ST33KTPM2X_OK`.

12.87.4.6 st33ktpm2x_pp_is_pressed()

```
st33ktpm2x_status_t st33ktpm2x_pp_is_pressed (
    st33ktpm2x_t * ctx,
    bool * pressed)
```

Read state of optional Physical Presence button.

Parametry

<i>ctx</i>	Driver context.
<i>pressed</i>	[out] true if button is active.

Zwraca

Status code (`ST33KTPM2X_ENOTSUP` if PP pin is not configured).

Definicja w linii 624 pliku `st33ktpm2x.c`.

Odwołuje się do `st33ktpm2x_t::cfg`, `st33ktpm2x_t::initialized`, `st33ktpm2x_cfg_t::pp_active_state`, `st33ktpm2x_cfg_t::pp_pin`, `st33ktpm2x_cfg_t::pp_port`, `ST33KTPM2X_EINVAL`, `ST33KTPM2X_ENOTSUP` i `ST33KTPM2X_OK`.

Odwołania w `st33ktpm2x_pp_wait_pressed()`.

12.87.4.7 st33ktpm2x_pp_wait_pressed()

```
st33ktpm2x_status_t st33ktpm2x_pp_wait_pressed (
    st33ktpm2x_t * ctx,
    uint32_t timeout_ms)
```

Wait until Physical Presence button becomes active.

Parametry

<i>ctx</i>	Driver context.
<i>timeout_ms</i>	Timeout in ms.

Zwraca

Status code.

Definicja w linii 643 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::initialized](#), [ST33KTPM2X_ESTATE](#), [ST33KTPM2X_ETIMEOUT](#), [ST33KTPM2X_OK](#) i [st33ktpm2x_pp_is_pressed\(\)](#).

12.87.4.8 st33ktpm2x_read_identity()

```
st33ktpm2x_status_t st33ktpm2x_read_identity (  
    st33ktpm2x_t * ctx,  
    uint32_t * did_vid,  
    uint8_t * rid)
```

Read TPM DID_VID and RID registers.

Parametry

<i>ctx</i>	Driver context.
<i>did_vid</i>	[out] 32-bit DID_VID register value.
<i>rid</i>	[out] 8-bit Revision ID value.

Zwraca

Status code.

Definicja w linii 755 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::initialized](#), [st33_i2c_read\(\)](#), [st33_identity_is_valid\(\)](#), [st33_le32_read\(\)](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_EPROTO](#) i [ST33KTPM2X_OK](#).

12.87.4.9 st33ktpm2x_release_locality0()

```
st33ktpm2x_status_t st33ktpm2x_release_locality0 (  
    st33ktpm2x_t * ctx)
```

Release TPM locality 0.

Parametry

<i>ctx</i>	Driver context.
------------	-----------------

Zwraca

Status code.

Definicja w linii 730 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::initialized](#), [st33ktpm2x_t::locality0_acquired](#), [st33_status_text\(\)](#), [st33_write_access\(\)](#), [ST33KTPM2X_ESTATE](#) i [ST33KTPM2X_OK](#).

Odwołania w [st33_trace_fail\(\)](#), [st33ktpm2x_deinit\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.87.4.10 st33ktpm2x_request_locality0()

```
st33ktpm2x_status_t st33ktpm2x_request_locality0 (
    st33ktpm2x_t * ctx)
```

Acquire TPM locality 0.

Parametry

<i>ctx</i>	Driver context.
------------	-----------------

Zwraca

Status code.

Definicja w linii 672 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::cfg](#), [st33ktpm2x_t::initialized](#), [st33ktpm2x_t::locality0_acquired](#), [st33ktpm2x_cfg_t::locality_timeout_ms](#), [st33_read_access\(\)](#), [st33_trace_fail\(\)](#), [st33_trace_tis\(\)](#), [st33_wait_access_bits\(\)](#), [st33_write_access\(\)](#), [st33_write_loc_sel\(\)](#), [ST33KTPM2X_ESTATE](#) i [ST33KTPM2X_OK](#).

Odwołania w [st33ktpm2x_transceive\(\)](#).

12.87.4.11 st33ktpm2x_tpm2_flush_context()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_flush_context (
    st33ktpm2x_t * ctx,
    uint32_t handle,
    uint32_t * tpm_rc)
```

Flush transient context/session handle (scaffold API).

Parametry

<i>ctx</i>	Driver context.
<i>handle</i>	Transient handle.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1515 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_ENOTSUP](#).

12.87.4.12 st33ktpm2x_tpm2_get_capability()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_get_capability (
    st33ktpm2x_t * ctx,
    uint32_t capability,
    uint32_t property,
    uint32_t property_count,
    uint8_t * payload_out,
    uint16_t payload_capacity,
    uint16_t * payload_len,
    bool * more_data,
    uint32_t * tpm_rc)
```

Execute TPM2_GetCapability and return raw capability payload.

Parametry

<i>ctx</i>	Driver context.
<i>capability</i>	TPM capability selector.
<i>property</i>	Capability-specific property/offset.
<i>property_count</i>	Number of properties requested.
<i>payload_out</i>	Raw capability payload output (without <code>moreData</code> byte).
<i>payload_capacity</i>	Capacity of <code>payload_out</code> .
<i>payload_len</i>	[out] Returned payload length.
<i>more_data</i>	[out] <code>true</code> if TPM reports more data available.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1044 pliku `st33ktpm2x.c`.

Odwołuje się do `st33_be16_write()`, `st33_be32_write()`, `ST33KTPM2X_EINVAL`, `ST33KTPM2X_EOVERFLOW`, `ST33KTPM2X_EPROTO`, `ST33KTPM2X_OK`, `ST33KTPM2X_RSP_MAX_BYTES` i `st33ktpm2x_transceive()`.

12.87.4.13 st33ktpm2x_tpm2_get_random()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_get_random (
    st33ktpm2x_t * ctx,
    uint16_t requested_bytes,
    uint8_t * random_out,
    uint16_t random_capacity,
    uint16_t * random_len,
    uint32_t * tpm_rc)
```

Execute TPM2_GetRandom.

Parametry

<i>ctx</i>	Driver context.
<i>requested_bytes</i>	Number of random bytes requested from TPM.
<i>random_out</i>	Output buffer for random bytes.
<i>random_capacity</i>	Capacity of <code>random_out</code> .
<i>random_len</i>	[out] Actual bytes returned.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 989 pliku `st33ktpm2x.c`.

Odwołuje się do `st33_be16_read()`, `st33_be16_write()`, `st33_be32_write()`, `ST33KTPM2X_EINVAL`, `ST33KTPM2X_EOVERFLOW`, `ST33KTPM2X_EPROTO`, `ST33KTPM2X_OK` i `st33ktpm2x_transceive()`.

12.87.4.14 st33ktpm2x_tpm2_nv_define()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_nv_define (
    st33ktpm2x_t * ctx,
```

```
uint32_t nv_index,
uint16_t data_size,
uint32_t attributes,
uint32_t * tpm_rc)
```

Define NV index using owner empty-password authorization.

Parametry

<i>ctx</i>	Driver context.
<i>nv_index</i>	NV index handle.
<i>data_size</i>	NV data size.
<i>attributes</i>	NV attributes mask.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1270 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::initialized](#), [st33_be16_write\(\)](#), [st33_be32_write\(\)](#), [st33_write_empty_password_auth\(\)](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_EPROTO](#) i [st33ktpm2x_transceive\(\)](#).

12.87.4.15 st33ktpm2x_tpm2_nv_read()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_nv_read (
    st33ktpm2x_t * ctx,
    uint32_t nv_index,
    uint16_t offset,
    uint8_t * out_data,
    uint16_t out_capacity,
    uint16_t * out_len,
    uint32_t * tpm_rc)
```

Read bytes from NV index using owner empty-password authorization.

Parametry

<i>ctx</i>	Driver context.
<i>nv_index</i>	NV index handle.
<i>offset</i>	Offset in bytes.
<i>out_data</i>	Output buffer.
<i>out_capacity</i>	Output capacity.
<i>out_len</i>	[out] Returned data length.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1330 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33ktpm2x_t::initialized](#), [st33_be16_read\(\)](#), [st33_be16_write\(\)](#), [st33_be32_write\(\)](#), [st33_write_empty_password_auth\(\)](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_EOVERFLOW](#), [ST33KTPM2X_EPROTO](#), [ST33KTPM2X_OK](#) i [st33ktpm2x_transceive\(\)](#).

12.87.4.16 st33ktpm2x_tpm2_nv_write()

```

st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write (
    st33ktpm2x_t * ctx,
    uint32_t nv_index,
    uint16_t offset,
    const uint8_t * data,
    uint16_t data_len,
    uint32_t * tpm_rc)

```

Write bytes to NV index using owner empty-password authorization.

Parametry

<i>ctx</i>	Driver context.
<i>nv_index</i>	NV index handle.
<i>offset</i>	Offset in bytes.
<i>data</i>	Input data.
<i>data_len</i>	Input data length.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1483 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_tpm2_nv_write_auth\(\)](#).

12.87.4.17 st33ktpm2x_tpm2_nv_write_platform_pp()

```

st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write_platform_pp (
    st33ktpm2x_t * ctx,
    uint32_t nv_index,
    uint16_t offset,
    const uint8_t * data,
    uint16_t data_len,
    uint32_t * tpm_rc)

```

Write bytes to NV index using platform authorization with hardware Physical Presence.

Parametry

<i>ctx</i>	Driver context.
<i>nv_index</i>	NV index handle.
<i>offset</i>	Offset in bytes.
<i>data</i>	Input data.
<i>data_len</i>	Input data length.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1499 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_tpm2_nv_write_auth\(\)](#).

12.87.4.18 st33ktpm2x_tpm2_pcr_read_sha256()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_pcr_read_sha256 (
    st33ktpm2x_t * ctx,
    uint8_t pcr_index,
    uint8_t digest_sha256_out[32],
    uint32_t * update_counter_out,
    uint32_t * tpm_rc)
```

Execute TPM2_PCR_Read for one SHA-256 PCR index.

Parametry

<i>ctx</i>	Driver context.
<i>pcr_index</i>	PCR index in range 0..23.
<i>digest_sha256_out</i>	[out] 32-byte SHA-256 PCR digest.
<i>update_counter_out</i>	[out] PCR update counter.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1096 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_be16_read\(\)](#), [st33_be16_write\(\)](#), [st33_be32_read\(\)](#), [st33_be32_write\(\)](#), [ST33KTPM2X_EINVAL](#), [ST33KTPM2X_EPROTO](#), [ST33KTPM2X_OK](#), [ST33KTPM2X_RSP_MAX_BYTES](#) i [st33ktpm2x_transceive\(\)](#).

12.87.4.19 st33ktpm2x_tpm2_policy_command_code()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_policy_command_code (
    st33ktpm2x_t * ctx,
    uint32_t session_handle,
    uint32_t command_code,
    uint32_t * tpm_rc)
```

Apply PolicyCommandCode in an existing policy session (scaffold API).

Parametry

<i>ctx</i>	Driver context.
<i>session_handle</i>	Policy session handle.
<i>command_code</i>	TPM command code to authorize.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1236 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_ENOTSUP](#).

12.87.4.20 st33ktpm2x_tpm2_policy_or()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_policy_or (
    st33ktpm2x_t * ctx,
```

```

uint32_t session_handle,
const uint8_t * digest_list,
uint8_t digest_count,
uint8_t digest_size,
uint32_t * tpm_rc)

```

Apply PolicyOR branch combiner (scaffold API).

Parametry

<i>ctx</i>	Driver context.
<i>session_handle</i>	Policy session handle.
<i>digest_list</i>	Pointer to concatenated digest list.
<i>digest_count</i>	Number of digests.
<i>digest_size</i>	Single digest size.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1251 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_ENOTSUP](#).

12.87.4.21 st33ktpm2x_tpm2_policy_pcr()

```

st33ktpm2x_status_t st33ktpm2x_tpm2_policy_pcr (
    st33ktpm2x_t * ctx,
    uint32_t session_handle,
    uint8_t pcr_index,
    uint32_t * tpm_rc)

```

Apply PolicyPCR in an existing policy session (scaffold API).

Parametry

<i>ctx</i>	Driver context.
<i>session_handle</i>	Policy session handle.
<i>pcr_index</i>	PCR index.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1208 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_ENOTSUP](#).

12.87.4.22 st33ktpm2x_tpm2_policy_physical_presence()

```

st33ktpm2x_status_t st33ktpm2x_tpm2_policy_physical_presence (
    st33ktpm2x_t * ctx,
    uint32_t session_handle,
    uint32_t * tpm_rc)

```

Apply Physical Presence policy branch (scaffold API).

Parametry

<i>ctx</i>	Driver context.
<i>session_handle</i>	Policy session handle.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1223 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_ENOTSUP](#).

12.87.4.23 st33ktpm2x_tpm2_self_test()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_self_test (
    st33ktpm2x_t * ctx,
    bool full_test,
    uint32_t * tpm_rc)
```

Execute TPM2_SelfTest.

Parametry

<i>ctx</i>	Driver context.
<i>full_test</i>	true for full self-test, false for incremental self-test.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 973 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_be16_write\(\)](#), [st33_be32_write\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.87.4.24 st33ktpm2x_tpm2_start_auth_session()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_start_auth_session (
    st33ktpm2x_t * ctx,
    uint32_t * session_handle_out,
    uint32_t * tpm_rc)
```

Start TPM2 policy/auth session helper (scaffold API).

Parametry

<i>ctx</i>	Driver context.
<i>session_handle_out</i>	[out] Created session handle.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 1192 pliku [st33ktpm2x.c](#).

Odwołuje się do [ST33KTPM2X_ENOTSUP](#).

12.87.4.25 st33ktpm2x_tpm2_startup()

```
st33ktpm2x_status_t st33ktpm2x_tpm2_startup (
    st33ktpm2x_t * ctx,
    uint16_t startup_type,
    uint32_t * tpm_rc)
```

Execute TPM2_Startup.

Parametry

<i>ctx</i>	Driver context.
<i>startup_type</i>	Use ST33KTPM2X_TPM2_SU_CLEAR or ST33KTPM2X_TPM2_SU_STATE.
<i>tpm_rc</i>	[out] TPM response code.

Zwraca

Status code.

Definicja w linii 957 pliku [st33ktpm2x.c](#).

Odwołuje się do [st33_be16_write\(\)](#), [st33_be32_write\(\)](#) i [st33ktpm2x_transceive\(\)](#).

12.87.4.26 st33ktpm2x_transceive()

```
st33ktpm2x_status_t st33ktpm2x_transceive (
    st33ktpm2x_t * ctx,
    const uint8_t * command,
    uint16_t command_len,
    uint8_t * response,
    uint16_t response_capacity,
    uint16_t * response_len,
    uint32_t * tpm_rc)
```

Send one raw TPM command and receive full response.

This is the primary extension point for advanced TPM2 features.

Parametry

<i>ctx</i>	Driver context.
<i>command</i>	Command buffer (TPM2 packet, complete header+payload).
<i>command_len</i>	Command length in bytes.
<i>response</i>	Buffer for response packet.
<i>response_capacity</i>	Response buffer capacity in bytes.
<i>response_len</i>	[out] Actual response length.
<i>tpm_rc</i>	[out] TPM2 response code parsed from response header.

Zwraca

Status code.

Definicja w linii 786 pliku `st33ktpm2x.c`.

Odwwołuje się do `st33ktpm2x_cfg_t::burst_timeout_ms`, `st33ktpm2x_t::cfg`, `st33ktpm2x_t::initialized`, `st33_be32_read()`, `st33_fifo_write()`, `st33_min_u16()`, `st33_read_response_stream()`, `st33_read_sts()`, `st33_trace_fail()`, `st33_trace_tis()`, `st33_wait_burst()`, `st33_wait_sts_bits()`, `st33_write_sts()`, `ST33KTPM2X_CMD_MAX_BYTES`, `ST33KTPM2X_EINVAL`, `ST33KTPM2X_EOVERFLOW`, `ST33KTPM2X_EPROTO`, `ST33KTPM2X_ETPM_RC`, `ST33KTPM2X_OK`, `st33ktpm2x_release_locality0()`, `st33ktpm2x_request_locality0()` i `ST33KTPM2X_RSP_MAX_BYTES`.

Odwwołania w `st33_tpm2_nv_write_auth()`, `st33ktpm2x_tpm2_get_capability()`, `st33ktpm2x_tpm2_get_random()`, `st33ktpm2x_tpm2_nv_define()`, `st33ktpm2x_tpm2_nv_read()`, `st33ktpm2x_tpm2_pcr_read_sha256()`, `st33ktpm2x_tpm2_self_test()` i `st33ktpm2x_tpm2_startup()`.

12.88 st33ktpm2x.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00015
00016 #ifndef APP_ST33KTPM2X_LIB_ST33KTPM2X_H_
00017 #define APP_ST33KTPM2X_LIB_ST33KTPM2X_H_
00018
00019 #include "stm32u5xx_hal.h"
00020
00021 #include <stdbool.h>
00022 #include <stdint.h>
00023
00024 #ifdef __cplusplus
00025 extern "C" {
00026 #endif
00027
00031 #define ST33KTPM2X_I2C_ADDR_DEFAULT      0x2EU
00032 #define ST33KTPM2X_CMD_MAX_BYTES        1024U
00033 #define ST33KTPM2X_RSP_MAX_BYTES        1024U
00035
00039 #define ST33KTPM2X_TPM2_ST_NO_SESSIONS   0x8001U
00040 #define ST33KTPM2X_TPM2_ST_SESSIONS     0x8002U
00041 #define ST33KTPM2X_TPM2_RC_SUCCESS      0x00000000UL
00042
00043 #define ST33KTPM2X_TPM2_CC_STARTUP       0x00000144UL
00044 #define ST33KTPM2X_TPM2_CC_SELF_TEST    0x00000143UL
00045 #define ST33KTPM2X_TPM2_CC_GET_RANDOM   0x0000017BUL
00046 #define ST33KTPM2X_TPM2_CC_GET_CAPABILITY 0x0000017AUL
00047 #define ST33KTPM2X_TPM2_CC_PCR_READ     0x0000017EUL
00048 #define ST33KTPM2X_TPM2_CC_NV_DEFINE_SPACE 0x0000012AUL
00049 #define ST33KTPM2X_TPM2_CC_NV_WRITE     0x00000137UL
00050 #define ST33KTPM2X_TPM2_CC_NV_READ      0x0000014EUL
00051
00052 #define ST33KTPM2X_TPM2_SU_CLEAR         0x0000U
00053 #define ST33KTPM2X_TPM2_SU_STATE        0x0001U
00054
00055 #define ST33KTPM2X_TPM2_ALG_SHA256      0x000BU
00056
00057 #define ST33KTPM2X_TPM2_CAP_ALGS        0x00000000UL
00058 #define ST33KTPM2X_TPM2_CAP_HANDLES     0x00000001UL
00059 #define ST33KTPM2X_TPM2_CAP_COMMANDS    0x00000002UL
00060 #define ST33KTPM2X_TPM2_CAP_PCERS      0x00000005UL
00061 #define ST33KTPM2X_TPM2_CAP_TPM_PROPERTIES 0x00000006UL
00063
00067 typedef enum
00068 {
00069     ST33KTPM2X_OK = 0,
00070     ST33KTPM2X_EINVAL,
00071     ST33KTPM2X_ESTATE,
00072     ST33KTPM2X_EHAL,
00073     ST33KTPM2X_ETIMEOUT,
00074     ST33KTPM2X_EOVERFLOW,
00075     ST33KTPM2X_EPROTO,
00076     ST33KTPM2X_ETPM_RC,
00077     ST33KTPM2X_ENOTSUP
00078 } st33ktpm2x_status_t;
00079
00083 typedef struct
00084 {
00085     I2C_HandleTypeDef *hi2c;
00086     uint16_t i2c_addr_7bit;
00087
00088     GPIO_TypeDef *reset_port;
00089     uint16_t reset_pin;
```



```

00090     uint32_t reset_pulse_ms;
00091     uint32_t reset_recovery_ms;
00092
00093     GPIO_TypeDef *davint_port;
00094     uint16_t davint_pin;
00095     GPIO_PinState davint_active_state;
00096
00097     GPIO_TypeDef *pp_port;
00098     uint16_t pp_pin;
00099     GPIO_PinState pp_active_state;
00100
00101     uint32_t io_timeout_ms;
00102     uint32_t locality_timeout_ms;
00103     uint32_t burst_timeout_ms;
00104 } st33ktpm2x_cfg_t;
00105
00109 typedef struct
00110 {
00111     st33ktpm2x_cfg_t cfg;
00112     bool initialized;
00113     bool locality0_acquired;
00114 } st33ktpm2x_t;
00115
00121 void st33ktpm2x_default_cfg(st33ktpm2x_cfg_t *cfg, I2C_HandleTypeDef *hi2c);
00122
00129 st33ktpm2x_status_t st33ktpm2x_init(st33ktpm2x_t *ctx, const st33ktpm2x_cfg_t *cfg);
00130
00136 st33ktpm2x_status_t st33ktpm2x_deinit(st33ktpm2x_t *ctx);
00137
00143 st33ktpm2x_status_t st33ktpm2x_hard_reset(st33ktpm2x_t *ctx);
00144
00151 st33ktpm2x_status_t st33ktpm2x_davint_is_asserted(st33ktpm2x_t *ctx, bool *asserted);
00152
00159 st33ktpm2x_status_t st33ktpm2x_pp_is_pressed(st33ktpm2x_t *ctx, bool *pressed);
00160
00167 st33ktpm2x_status_t st33ktpm2x_pp_wait_pressed(st33ktpm2x_t *ctx, uint32_t timeout_ms);
00168
00174 st33ktpm2x_status_t st33ktpm2x_request_locality0(st33ktpm2x_t *ctx);
00175
00181 st33ktpm2x_status_t st33ktpm2x_release_locality0(st33ktpm2x_t *ctx);
00182
00190 st33ktpm2x_status_t st33ktpm2x_read_identity(st33ktpm2x_t *ctx, uint32_t *did_vid, uint8_t *rid);
00191
00206 st33ktpm2x_status_t st33ktpm2x_transceive(st33ktpm2x_t *ctx,
00207     const uint8_t *command,
00208     uint16_t command_len,
00209     uint8_t *response,
00210     uint16_t response_capacity,
00211     uint16_t *response_len,
00212     uint32_t *tpm_rc);
00213
00221 st33ktpm2x_status_t st33ktpm2x_tpm2_startup(st33ktpm2x_t *ctx,
00222     uint16_t startup_type,
00223     uint32_t *tpm_rc);
00224
00232 st33ktpm2x_status_t st33ktpm2x_tpm2_self_test(st33ktpm2x_t *ctx,
00233     bool full_test,
00234     uint32_t *tpm_rc);
00235
00246 st33ktpm2x_status_t st33ktpm2x_tpm2_get_random(st33ktpm2x_t *ctx,
00247     uint16_t requested_bytes,
00248     uint8_t *random_out,
00249     uint16_t random_capacity,
00250     uint16_t *random_len,
00251     uint32_t *tpm_rc);
00252
00266 st33ktpm2x_status_t st33ktpm2x_tpm2_get_capability(st33ktpm2x_t *ctx,
00267     uint32_t capability,
00268     uint32_t property,
00269     uint32_t property_count,
00270     uint8_t *payload_out,
00271     uint16_t payload_capacity,
00272     uint16_t *payload_len,
00273     bool *more_data,
00274     uint32_t *tpm_rc);
00275
00285 st33ktpm2x_status_t st33ktpm2x_tpm2_pcr_read_sha256(st33ktpm2x_t *ctx,
00286     uint8_t pcr_index,
00287     uint8_t digest_sha256_out[32],
00288     uint32_t *update_counter_out,
00289     uint32_t *tpm_rc);
00290
00298 st33ktpm2x_status_t st33ktpm2x_tpm2_start_auth_session(st33ktpm2x_t *ctx,
00299     uint32_t *session_handle_out,
00300     uint32_t *tpm_rc);
00301
00310 st33ktpm2x_status_t st33ktpm2x_tpm2_policy_pcr(st33ktpm2x_t *ctx,

```

```

00311         uint32_t session_handle,
00312         uint8_t pcr_index,
00313         uint32_t *tpm_rc);
00314
00322 st33ktpm2x_status_t st33ktpm2x_tpm2_policy_physical_presence(st33ktpm2x_t *ctx,
00323                                                                uint32_t session_handle,
00324                                                                uint32_t *tpm_rc);
00325
00334 st33ktpm2x_status_t st33ktpm2x_tpm2_policy_command_code(st33ktpm2x_t *ctx,
00335                                                            uint32_t session_handle,
00336                                                            uint32_t command_code,
00337                                                            uint32_t *tpm_rc);
00338
00349 st33ktpm2x_status_t st33ktpm2x_tpm2_policy_or(st33ktpm2x_t *ctx,
00350                                                  uint32_t session_handle,
00351                                                  const uint8_t *digest_list,
00352                                                  uint8_t digest_count,
00353                                                  uint8_t digest_size,
00354                                                  uint32_t *tpm_rc);
00355
00365 st33ktpm2x_status_t st33ktpm2x_tpm2_nv_define(st33ktpm2x_t *ctx,
00366                                                 uint32_t nv_index,
00367                                                 uint16_t data_size,
00368                                                 uint32_t attributes,
00369                                                 uint32_t *tpm_rc);
00370
00382 st33ktpm2x_status_t st33ktpm2x_tpm2_nv_read(st33ktpm2x_t *ctx,
00383                                               uint32_t nv_index,
00384                                               uint16_t offset,
00385                                               uint8_t *out_data,
00386                                               uint16_t out_capacity,
00387                                               uint16_t *out_len,
00388                                               uint32_t *tpm_rc);
00389
00400 st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write(st33ktpm2x_t *ctx,
00401                                                uint32_t nv_index,
00402                                                uint16_t offset,
00403                                                const uint8_t *data,
00404                                                uint16_t data_len,
00405                                                uint32_t *tpm_rc);
00406
00417 st33ktpm2x_status_t st33ktpm2x_tpm2_nv_write_platform_pp(st33ktpm2x_t *ctx,
00418                                                           uint32_t nv_index,
00419                                                           uint16_t offset,
00420                                                           const uint8_t *data,
00421                                                           uint16_t data_len,
00422                                                           uint32_t *tpm_rc);
00423
00431 st33ktpm2x_status_t st33ktpm2x_tpm2_flush_context(st33ktpm2x_t *ctx,
00432                                                     uint32_t handle,
00433                                                     uint32_t *tpm_rc);
00434
00435 #ifdef __cplusplus
00436 }
00437 #endif
00438
00439 #endif /* APP_ST33KTPM2X_LIB_ST33KTPM2X_H_ */

```

12.89 tof_main.c

```

00001 #include "tof_main.h"
00002
00003 #include "app.h"
00004 #include "cmsis_os2.h"
00005 #include "FreeRTOS.h"
00006 #include "task.h"
00007 #include "vl53l3cx_lib/vl53l3cx_lib.h"
00008
00009 #include <stdio.h>
00010
00011 static osThreadId_t s_tof_task = NULL;
00012 static volatile int32_t s_tof_last_distance_mm = -1;
00013 static StaticTask_t s_tof_task_cb;
00014 #define TOF_TASK_STACK_SIZE 6144U
00015 #define TOF_TASK_STACK_WORDS (TOF_TASK_STACK_SIZE / sizeof(StackType_t))
00016 static StackType_t s_tof_task_stack[TOF_TASK_STACK_WORDS];
00017
00018 static void tof_main_task_fn(void *argument);
00019
00020 static const osThreadAttr_t s_tof_task_attr =
00021 {
00022     .name = "tof_task",
00023     .priority = (osPriority_t)osPriorityBelowNormal,
00024     .stack_mem = s_tof_task_stack,

```

```

00025     .stack_size = sizeof(s_tof_task_stack),
00026     .cb_mem = &s_tof_task_cb,
00027     .cb_size = sizeof(s_tof_task_cb)
00028 };
00029
00030 void tof_main_create_task(void)
00031 {
00032     if (s_tof_task == NULL)
00033     {
00034         s_tof_task = osThreadNew(tof_main_task_fn, NULL, &s_tof_task_attr);
00035     }
00036 }
00037
00038 int32_t tof_main_get_last_distance(void)
00039 {
00040     return s_tof_last_distance_mm;
00041 }
00042
00044 static void tof_main_task_fn(void *argument)
00045 {
00046     bool init_ok;
00047
00048     (void)argument;
00049
00050     if (app_i2c_lock(0U))
00051     {
00052         init_ok = tof_init();
00053         app_i2c_unlock();
00054     }
00055     else
00056     {
00057         init_ok = false;
00058     }
00059
00060     if (!init_ok)
00061     {
00062         printf("ToF task: init failed\r\n");
00063     }
00064     else
00065     {
00066         printf("ToF task: init OK\r\n");
00067     }
00068
00069     for (;;)
00070     {
00071         if (init_ok && app_i2c_lock(0U))
00072         {
00073             s_tof_last_distance_mm = tof_get_distance();
00074             app_i2c_unlock();
00075         }
00076         else
00077         {
00078             s_tof_last_distance_mm = -1;
00079         }
00080
00081         osDelay(1000U);
00082     }
00083 }

```

12.90 Dokumentacja pliku Core/App/tof_main.h

RTOS task facade for the VL53L3CX time-of-flight sensor.

```
#include <stdint.h>
```

Funkcje

- void [tof_main_create_task](#) (void)
Create the ToF periodic sampling task.
- int32_t [tof_main_get_last_distance](#) (void)
Return the last cached distance measurement.

12.90.1 Opis szczegółowy

RTOS task facade for the VL53L3CX time-of-flight sensor.

Definicja w pliku [tof_main.h](#).

12.90.2 Dokumentacja funkcji

12.90.2.1 tof_main_create_task()

```
void tof_main_create_task (
    void )
```

Create the ToF periodic sampling task.

Definicja w linii 30 pliku [tof_main.c](#).

Odwołania w [app_freertos_init\(\)](#).

12.90.2.2 tof_main_get_last_distance()

```
int32_t tof_main_get_last_distance (
    void )
```

Return the last cached distance measurement.

Zwraca

Distance in millimeters, or a negative value when unavailable.

Definicja w linii 38 pliku [tof_main.c](#).

12.91 tof_main.h

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_TOF_MAIN_H_
00007 #define APP_TOF_MAIN_H_
00008
00009 #include <stdint.h>
00010
00014 void tof_main_create_task(void);
00015
00020 int32_t tof_main_get_last_distance(void);
00021
00022 #endif /* APP_TOF_MAIN_H_ */
```

12.92 vl53l3cx_lib.c

```
00001 #include "vl53l3cx_lib.h"
00002
00003 #include "../app.h"
00004 #include "app_delay.h"
00005 #include "main.h"
00006 #include "vl53l3cx.h"
00007 #include "vl53lx_api.h"
00008
00009 #include <stdio.h>
00010 #include <string.h>
00011
00012 extern I2C_HandleTypeDef hi2c1;
00013
00014 #define TOF_I2C_ADDRESS_8BIT      VL53L3CX_DEVICE_ADDRESS
00015 #define TOF_I2C_TIMEOUT_MS        250U
00016 #define TOF_TIMING_BUDGET_MS       30U
00017 #define TOF_RESET_PULSE_MS        5U
00018 #define TOF_BOOT_TIME_MS          10U
00019
00020 static VL53L3CX_Object_t s_tof_obj;
00021 static VL53L3CX_Result_t result;
00022 static uint8_t s_tof_initialized = 0U;
00023 static uint8_t s_tof_started = 0U;
00024 static bool s_first_reading = true;
00025
00026 static int32_t tof_bus_init(void);
00027 static int32_t tof_bus_deinit(void);
00028 static int32_t tof_bus_write(uint16_t address, uint8_t *data, uint16_t length);
00029 static int32_t tof_bus_read(uint16_t address, uint8_t *data, uint16_t length);
00030 static int32_t tof_bus_get_tick(void);
00031 static bool tof_activate(void);
00032 static bool tof_reset_and_boot(void);
00033 static bool tof_low_level_init(void);
00034 static bool tof_is_non_fatal_ref_spad_status(VL53LX_Error status);
```

```

00035
00037 static int32_t tof_bus_init(void)
00038 {
00039     /* I2C1 is configured once by CubeMX in MX_I2C1_Init(). */
00040     return 0;
00041 }
00042
00044 static int32_t tof_bus_deinit(void)
00045 {
00046     return 0;
00047 }
00048
00050 static int32_t tof_bus_write(uint16_t address, uint8_t *data, uint16_t length)
00051 {
00052     return (app_i2c_master_transmit(&hi2c1, address, data, length, TOF_I2C_TIMEOUT_MS) == HAL_OK) ? 0 :
-1;
00053 }
00054
00056 static int32_t tof_bus_read(uint16_t address, uint8_t *data, uint16_t length)
00057 {
00058     return (app_i2c_master_receive(&hi2c1, address, data, length, TOF_I2C_TIMEOUT_MS) == HAL_OK) ? 0 :
-1;
00059 }
00060
00062 static int32_t tof_bus_get_tick(void)
00063 {
00064     return (int32_t)HAL_GetTick();
00065 }
00066
00068 static bool tof_reset_and_boot(void)
00069 {
00070     HAL_GPIO_WritePin(VL53L3CX_xshout_GPIO_Port, VL53L3CX_xshout_Pin, GPIO_PIN_RESET);
00071     app_delay_ms(TOF_RESET_PULSE_MS);
00072     HAL_GPIO_WritePin(VL53L3CX_xshout_GPIO_Port, VL53L3CX_xshout_Pin, GPIO_PIN_SET);
00073     app_delay_ms(TOF_BOOT_TIME_MS);
00074
00075     return true;
00076 }
00077
00079 static bool tof_is_non_fatal_ref_spad_status(VL53LX_Error status)
00080 {
00081     return (status == VL53LX_WARNING_REF_SPAD_CHAR_NOT_ENOUGH_SPADS) ||
00082            (status == VL53LX_WARNING_REF_SPAD_CHAR_RATE_TOO_HIGH) ||
00083            (status == VL53LX_WARNING_REF_SPAD_CHAR_RATE_TOO_LOW);
00084 }
00085
00087 static bool tof_low_level_init(void)
00088 {
00089     VL53LX_Error status;
00090
00091     status = VL53LX_WaitDeviceBooted(&s_tof_obj);
00092     if (status != VL53LX_ERROR_NONE)
00093     {
00094         printf("ToF init: WaitDeviceBooted failed (%ld)\r\n", (long)status);
00095         return false;
00096     }
00097
00098     status = VL53LX_DataInit(&s_tof_obj);
00099     if (status != VL53LX_ERROR_NONE)
00100     {
00101         printf("ToF init: DataInit failed (%ld)\r\n", (long)status);
00102         return false;
00103     }
00104
00105     status = VL53LX_PerformRefSpadManagement(&s_tof_obj);
00106     if ((status != VL53LX_ERROR_NONE) && !tof_is_non_fatal_ref_spad_status(status))
00107     {
00108         printf("ToF init: RefSpadManagement failed (%ld)\r\n", (long)status);
00109         return false;
00110     }
00111
00112     if (tof_is_non_fatal_ref_spad_status(status))
00113     {
00114         printf("ToF init: RefSpadManagement warning (%ld), continuing\r\n", (long)status);
00115     }
00116
00117     s_tof_obj.IsRanging = 0U;
00118     s_tof_obj.IsBlocking = 0U;
00119     s_tof_obj.IsContinuous = 0U;
00120     s_tof_obj.IsAmbientEnabled = 0U;
00121     s_tof_obj.IsSignalEnabled = 0U;
00122     s_tof_obj.IsInitialized = 1U;
00123
00124     return true;
00125 }
00126
00127 bool tof_init(void)

```

```

00128 {
00129     VL53L3CX_IO_t io_ctx;
00130     VL53L3CX_ProfileConfig_t profile;
00131     uint32_t sensor_id;
00132
00133     if (s_tof_initialized == 1U)
00134     {
00135         return true;
00136     }
00137
00138     s_tof_started = 0U;
00139     s_first_reading = true;
00140     memset(&s_tof_obj, 0, sizeof(s_tof_obj));
00141
00142     (void)tof_reset_and_boot();
00143
00144     io_ctx.Init = tof_bus_init;
00145     io_ctx.DeInit = tof_bus_deinit;
00146     io_ctx.Address = TOF_I2C_ADDRESS_8BIT;
00147     io_ctx.WriteReg = tof_bus_write;
00148     io_ctx.ReadReg = tof_bus_read;
00149     io_ctx.GetTick = tof_bus_get_tick;
00150
00151     if (VL53L3CX_RegisterBusIO(&s_tof_obj, &io_ctx) != VL53L3CX_OK)
00152     {
00153         return false;
00154     }
00155
00156     sensor_id = 0U;
00157     if (VL53L3CX_ReadID(&s_tof_obj, &sensor_id) != VL53L3CX_OK)
00158     {
00159         printf("ToF init: ReadID failed\r\n");
00160         return false;
00161     }
00162
00163     if (sensor_id != VL53L3CX_ID)
00164     {
00165         printf("ToF init: unexpected sensor ID 0x%04lX\r\n", (unsigned long)sensor_id);
00166         return false;
00167     }
00168
00169     if (!tof_low_level_init())
00170     {
00171         return false;
00172     }
00173
00174     profile.RangingProfile = VL53L3CX_PROFILE_LONG;
00175     profile.TimingBudget = TOF_TIMING_BUDGET_MS;
00176     profile.Frequency = 0U;
00177     profile.EnableAmbient = 0U;
00178     profile.EnableSignal = 0U;
00179
00180     if (VL53L3CX_ConfigProfile(&s_tof_obj, &profile) != VL53L3CX_OK)
00181     {
00182         return false;
00183     }
00184
00185     s_tof_initialized = 1U;
00186     return true;
00187 }
00188
00190 static bool tof_activate(void)
00191 {
00192     if (s_tof_started == 1U)
00193     {
00194         return true;
00195     }
00196
00197     /*
00198      * Blocking mode prevents timeout/stale reads when the caller polls faster
00199      * than the sensor produces new samples.
00200      */
00201     if (VL53L3CX_Start(&s_tof_obj, VL53L3CX_MODE_BLOCKING_CONTINUOUS) != VL53L3CX_OK)
00202     {
00203         return false;
00204     }
00205
00206     s_tof_started = 1U;
00207     return true;
00208 }
00209
00210 int32_t tof_get_distance(void)
00211 {
00212     int32_t status;
00213     uint32_t target_count;
00214
00215     if (!tof_init())

```

```

00216 {
00217     return -1;
00218 }
00219
00220 if (!tof_activate())
00221 {
00222     return -1;
00223 }
00224
00225 memset(&result, 0, sizeof(result));
00226
00227 if (s_first_reading)
00228 {
00229     VL53L3CX_GetDistance(&s_tof_obj, &result);
00230     s_first_reading = false;
00231     app_delay_ms(100U);
00232 }
00233
00234 status = VL53L3CX_GetDistance(&s_tof_obj, &result);
00235 if (status != VL53L3CX_OK)
00236 {
00237     return -1;
00238 }
00239
00240 target_count = result.ZoneResult[0].NumberOfTargets;
00241
00242 if ((result.ZoneResult[0].Status[0] == 0U) &&
00243     (result.ZoneResult[0].Distance[0] > 0U) &&
00244     target_count > 0)
00245 {
00246     return (int32_t)result.ZoneResult[0].Distance[0];
00247 }
00248 else
00249 {
00250     return -1;
00251 }
00252 return -1;
00253 }
00254
00255 void VL53L3CX_TestOnce(void)
00256 {
00257     int32_t distance_mm;
00258
00259     distance_mm = tof_get_distance();
00260
00261     if (distance_mm < 0)
00262     {
00263         printf("VL53L3CX: read failed\r\n");
00264         return;
00265     }
00266
00267     printf("VL53L3CX: %ld mm\r\n", distance_mm);
00268 }

```

12.93 Dokumentacja pliku Core/App/vl53l3cx_lib/vl53l3cx_lib.h

Application wrapper for the VL53L3CX time-of-flight sensor.

```
#include <stdbool.h>
```

```
#include <stdint.h>
```

Funkcje

- bool `tof_init` (void)
Initialize the onboard VL53L3CX sensor.
- int32_t `tof_get_distance` (void)
Read one distance sample in millimeters.
- void `VL53L3CX_TestOnce` (void)
Diagnostic helper that reads and prints one VL53L3CX sample.

12.93.1 Opis szczegółowy

Application wrapper for the VL53L3CX time-of-flight sensor.

Definicja w pliku `vl53l3cx_lib.h`.

12.93.2 Dokumentacja funkcji

12.93.2.1 `tof_get_distance()`

```
int32_t tof_get_distance (
    void )
```

Read one distance sample in millimeters.

Nota

Funkcja jest blokująca: czeka na świeżą próbkę z czujnika. Czas blokowania zależy od `TimingBudget` (aktualnie ~30 ms + narzut magistrali).

Zwraca

Distance in millimeters, or a negative value on failure.

Definicja w linii 210 pliku `vl53l3cx_lib.c`.

Odwołuje się do `tof_init()`.

Odwołania w `VL53L3CX_TestOnce()`.

12.93.2.2 `tof_init()`

```
bool tof_init (
    void )
```

Initialize the onboard VL53L3CX sensor.

Zwraca

`true` when the sensor was detected and configured.

Definicja w linii 127 pliku `vl53l3cx_lib.c`.

Odwołania w `tof_get_distance()`.

12.93.2.3 `VL53L3CX_TestOnce()`

```
void VL53L3CX_TestOnce (
    void )
```

Diagnostic helper that reads and prints one VL53L3CX sample.

Definicja w linii 255 pliku `vl53l3cx_lib.c`.

Odwołuje się do `tof_get_distance()`.

12.94 `vl53l3cx_lib.h`

[Idź do dokumentacji tego pliku.](#)

```
00001
00005
00006 #ifndef APP_VL53L3CX_LIB_VL53L3CX_LIB_H_
00007 #define APP_VL53L3CX_LIB_VL53L3CX_LIB_H_
00008
00009 #include <stdbool.h>
00010 #include <stdint.h>
00011
00016 bool tof_init(void);
00017
00026 int32_t tof_get_distance(void);
00027
00031 void VL53L3CX_TestOnce(void);
00032
00033 #endif /* APP_VL53L3CX_LIB_VL53L3CX_LIB_H_ */
```


Skorowidz

- action
 - menu_item_t, [87](#)
 - menu_radio_action_binding_t, [89](#)
- active
 - radio_main_ctx_t, [105](#)
- active_brightness
 - led_array_ctx_t, [85](#)
- active_low
 - button_hw_t, [73](#)
- active_modulation
 - radio_main_runtime_cfg_t, [115](#)
 - security_runtime_cfg_t, [126](#)
- addr
 - BMP280_HandleTypedef, [69](#)
- addr_7bit
 - lcd_state_t, [84](#)
- address_filter
 - radio_fsk_cfg_t, [95](#)
 - radio_main_fsk_cfg_t, [112](#)
- admin_unlock_until_ms
 - menu_state_t, [90](#)
- app.h
 - app_freertos_init, [142](#)
 - app_i2c_lock, [142](#)
 - app_i2c_master_receive, [142](#)
 - app_i2c_master_transmit, [143](#)
 - app_i2c_mem_read, [143](#)
 - app_i2c_mem_write, [144](#)
 - app_i2c_unlock, [144](#)
 - app_init, [144](#)
 - app_main, [145](#)
- app_freertos_init
 - app.h, [142](#)
- app_i2c_lock
 - app.h, [142](#)
- app_i2c_master_receive
 - app.h, [142](#)
- app_i2c_master_transmit
 - app.h, [143](#)
- app_i2c_mem_read
 - app.h, [143](#)
- app_i2c_mem_write
 - app.h, [144](#)
- app_i2c_unlock
 - app.h, [144](#)
- app_init
 - app.h, [144](#)
- app_main
 - app.h, [145](#)
- Architektura firmware, [3](#)
- auto_ping_enabled
 - radio_main_ctx_t, [105](#)
 - radio_main_runtime_cfg_t, [115](#)
 - security_runtime_cfg_t, [126](#)
- auto_ping_mode
 - radio_main_ctx_t, [106](#)
 - radio_main_runtime_cfg_t, [116](#)
 - security_runtime_cfg_t, [126](#)
- auto_ping_period_ms
 - radio_main_ctx_t, [106](#)
 - radio_main_runtime_cfg_t, [116](#)
 - security_runtime_cfg_t, [126](#)
- backend_cfg
 - radio_main_ctx_t, [106](#)
- backend_modulation_id
 - radio_main_ctx_t, [106](#)
- backlight
 - lcd_state_t, [84](#)
- bandwidth
 - radio_lora_cfg_t, [100](#)
- BEKO Pager Firmware, [1](#)
- BEKO Pager Network (<tt>pager-rtos</tt>), [7](#)
- BEKO Security Architecture, [15](#)
- Bezpieczeństwo, [5](#)
- bitlen
 - laviet_sha256_ctx_t, [82](#)
- bitrate_bps
 - radio_fsk_cfg_t, [95](#)
 - radio_main_fsk_cfg_t, [112](#)
 - radio_main_ook_cfg_t, [114](#)
 - radio_ook_cfg_t, [117](#)
- BME280_CHIP_ID
 - bmp280.h, [150](#)
- bmp280.h
 - BME280_CHIP_ID, [150](#)
 - BMP280_CHIP_ID, [150](#)
 - BMP280_Filter, [151](#)
 - BMP280_FILTER_16, [151](#)
 - BMP280_FILTER_2, [151](#)
 - BMP280_FILTER_4, [151](#)
 - BMP280_FILTER_8, [151](#)
 - BMP280_FILTER_OFF, [151](#)
 - bmp280_force_measurement, [152](#)
 - BMP280_I2C_ADDRESS_0, [151](#)
 - BMP280_I2C_ADDRESS_1, [151](#)
 - bmp280_init, [152](#)
 - bmp280_init_default_params, [152](#)
 - bmp280_is_measuring, [153](#)

- BMP280_Mode, [151](#)
- BMP280_Oversampling, [151](#)
- bmp280_read_fixed, [153](#)
- bmp280_read_float, [153](#)
- BMP280_StandbyTime, [151](#)
- bmp280_api.h
 - bmp280_api_get_last_data, [159](#)
 - bmp280_api_init, [159](#)
 - bmp280_api_measure_all, [160](#)
 - bmp280_api_measure_pressure, [160](#)
 - bmp280_api_measure_temperature, [161](#)
 - bmp280_api_sleep, [161](#)
- bmp280_api_data_t, [67](#)
 - pressure_hpa, [67](#)
 - pressure_pa, [67](#)
 - temperature_c, [67](#)
 - timestamp_ms, [67](#)
 - valid, [68](#)
- bmp280_api_get_last_data
 - bmp280_api.h, [159](#)
- bmp280_api_init
 - bmp280_api.h, [159](#)
- bmp280_api_measure_all
 - bmp280_api.h, [160](#)
- bmp280_api_measure_pressure
 - bmp280_api.h, [160](#)
- bmp280_api_measure_temperature
 - bmp280_api.h, [161](#)
- bmp280_api_sleep
 - bmp280_api.h, [161](#)
- BMP280_CHIP_ID
 - bmp280.h, [150](#)
- BMP280_Filter
 - bmp280.h, [151](#)
- BMP280_FILTER_16
 - bmp280.h, [151](#)
- BMP280_FILTER_2
 - bmp280.h, [151](#)
- BMP280_FILTER_4
 - bmp280.h, [151](#)
- BMP280_FILTER_8
 - bmp280.h, [151](#)
- BMP280_FILTER_OFF
 - bmp280.h, [151](#)
- bmp280_force_measurement
 - bmp280.h, [152](#)
- BMP280_HandleTypeDef, [68](#)
 - addr, [69](#)
 - dig_H1, [69](#)
 - dig_H2, [69](#)
 - dig_H3, [69](#)
 - dig_H4, [69](#)
 - dig_H5, [69](#)
 - dig_H6, [69](#)
 - dig_P1, [70](#)
 - dig_P2, [70](#)
 - dig_P3, [70](#)
 - dig_P4, [70](#)
 - dig_P5, [70](#)
 - dig_P6, [70](#)
 - dig_P7, [70](#)
 - dig_P8, [70](#)
 - dig_P9, [70](#)
 - dig_T1, [70](#)
 - dig_T2, [71](#)
 - dig_T3, [71](#)
 - i2c, [71](#)
 - id, [71](#)
 - params, [71](#)
- BMP280_I2C_ADDRESS_0
 - bmp280.h, [151](#)
- BMP280_I2C_ADDRESS_1
 - bmp280.h, [151](#)
- bmp280_init
 - bmp280.h, [152](#)
- bmp280_init_default_params
 - bmp280.h, [152](#)
- bmp280_is_measuring
 - bmp280.h, [153](#)
- bmp280_main.h
 - bmp280_main_create_task, [164](#)
 - bmp280_main_get_last, [164](#)
- bmp280_main_create_task
 - bmp280_main.h, [164](#)
- bmp280_main_get_last
 - bmp280_main.h, [164](#)
- BMP280_Mode
 - bmp280.h, [151](#)
- BMP280_Oversampling
 - bmp280.h, [151](#)
- bmp280_params_t, [71](#)
 - filter, [72](#)
 - mode, [72](#)
 - oversampling_humidity, [72](#)
 - oversampling_pressure, [72](#)
 - oversampling_temperature, [72](#)
 - standby, [72](#)
- bmp280_read_fixed
 - bmp280.h, [153](#)
- bmp280_read_float
 - bmp280.h, [153](#)
- BMP280_StandbyTime
 - bmp280.h, [151](#)
- broadcast_group_aes_key
 - radio_main_ctx_t, [106](#)
- broadcast_group_epoch
 - radio_main_ctx_t, [106](#)
- broadcast_group_hmac_key
 - radio_main_ctx_t, [106](#)
- broadcast_group_key_valid
 - radio_main_ctx_t, [106](#)
- broadcast_group_pending_epoch
 - radio_main_ctx_t, [106](#)
- broadcast_group_pending_key
 - radio_main_ctx_t, [106](#)
- broadcast_group_pending_mask

- radio_main_ctx_t, 106
- broadcast_group_pending_valid
 - radio_main_ctx_t, 106
- burst_timeout_ms
 - st33ktpm2x_cfg_t, 131
- bus
 - radio_context_t, 93
 - radio_fsk_context_t, 96
 - radio_ook_context_t, 119
- BUTTON_EVENT_DOWN_SHORT
 - button_main.h, 168
- BUTTON_EVENT_NONE
 - button_main.h, 168
- BUTTON_EVENT_OK_LONG
 - button_main.h, 168
- BUTTON_EVENT_OK_SHORT
 - button_main.h, 168
- button_event_t
 - button_main.h, 168
- BUTTON_EVENT_UP_SHORT
 - button_main.h, 168
- button_hw_t, 72
 - active_low, 73
 - pin, 73
 - port, 73
- button_main.h
 - BUTTON_EVENT_DOWN_SHORT, 168
 - BUTTON_EVENT_NONE, 168
 - BUTTON_EVENT_OK_LONG, 168
 - BUTTON_EVENT_OK_SHORT, 168
 - button_event_t, 168
 - BUTTON_EVENT_UP_SHORT, 168
 - button_main_create_task, 168
 - button_main_get_event, 168
- button_main_create_task
 - button_main.h, 168
- button_main_get_event
 - button_main.h, 168
- button_state_t, 73
 - long_reported, 73
 - pressed_since_ms, 73
 - raw_pressed, 73
 - stable_count, 73
 - stable_pressed, 73
- callback
 - radio_context_t, 93
 - radio_fsk_context_t, 96
 - radio_ook_context_t, 119
- callback_ctx
 - radio_context_t, 93
 - radio_fsk_context_t, 97
 - radio_ook_context_t, 119
- cfg
 - i2c_mem_store_t, 78
 - radio_fsk_context_t, 97
 - radio_ook_context_t, 119
 - security_cmd_t, 123
 - st33ktpm2x_t, 133
- child_page
 - menu_item_t, 87
- code
 - i2c_mem_store_trusted_device_t, 80
 - security_cmd_t, 123
 - security_trusted_entry_t, 129
 - trusted_info_t, 135
- code_len
 - i2c_mem_store_trusted_device_t, 80
 - security_cmd_t, 123
 - security_trusted_entry_t, 129
 - trusted_info_t, 135
- coding_enabled
 - radio_main_ctx_t, 107
 - radio_main_runtime_cfg_t, 116
 - security_runtime_cfg_t, 126
- coding_rate
 - radio_lora_cfg_t, 100
- Core/App/app.c, 137
- Core/App/app.h, 141, 145
- Core/App/bmp280_lib/bmp280.c, 146
- Core/App/bmp280_lib/bmp280.h, 149, 154
- Core/App/bmp280_lib/bmp280_api.c, 155
- Core/App/bmp280_lib/bmp280_api.h, 159, 161
- Core/App/bmp280_main.c, 162
- Core/App/bmp280_main.h, 164
- Core/App/button_main.c, 165
- Core/App/button_main.h, 167, 169
- Core/App/i2c_mem_store_lib/i2c_mem_store.c, 169, 181
- Core/App/i2c_mem_store_lib/i2c_mem_store.h, 190, 198
- Core/App/laviet_crypto.c, 200, 209
- Core/App/laviet_crypto.h, 222, 226
- Core/App/laviet_frame.c, 226, 232
- Core/App/laviet_frame.h, 237, 246
- Core/App/lcd_library/lcd.c, 248
- Core/App/lcd_library/lcd.h, 255, 257
- Core/App/lcd_main.c, 257
- Core/App/lcd_main.h, 265, 269
- Core/App/led_array_lib/led_array_lib.c, 270, 278
- Core/App/led_array_lib/led_array_lib.h, 287, 292
- Core/App/led_array_lib/led_array_lib_config.h, 293, 294
- Core/App/led_array_main.c, 294
- Core/App/led_array_main.h, 295
- Core/App/menu_main.c, 296
- Core/App/menu_main.h, 349, 350
- Core/App/pager_config.h, 351
- Core/App/radio_lib/common/sx1276/radio_sx1276.c, 352, 359
- Core/App/radio_lib/common/sx1276/radio_sx1276.h, 363, 370
- Core/App/radio_lib/common/sx1276/radio_sx1276_regs.h, 371, 386
- Core/App/radio_lib/modulations/fsk/radio_fsk.c, 388, 397
- Core/App/radio_lib/modulations/fsk/radio_fsk.h, 407, 411

Core/App/radio_lib/modulations/lora/radio_lora.c, [412](#), [422](#)
Core/App/radio_lib/modulations/lora/radio_lora.h, [432](#), [438](#)
Core/App/radio_lib/modulations/ook/radio_ook.c, [439](#), [447](#)
Core/App/radio_lib/modulations/ook/radio_ook.h, [457](#), [461](#)
Core/App/radio_lib/radio_lib.c, [461](#), [470](#)
Core/App/radio_lib/radio_lib.h, [474](#), [486](#)
Core/App/radio_lib/radio_lib_config.h, [489](#), [491](#)
Core/App/radio_lib/test/radio_test.c, [492](#), [496](#)
Core/App/radio_lib/test/radio_test.h, [501](#), [503](#)
Core/App/radio_main.c, [503](#)
Core/App/radio_main.h, [560](#), [574](#)
Core/App/security_main.c, [576](#)
Core/App/security_main.h, [613](#), [623](#)
Core/App/service.c, [625](#)
Core/App/service.h, [637](#), [638](#)
Core/App/st33ktpm2x_lib/st33ktpm2x.c, [638](#), [660](#)
Core/App/st33ktpm2x_lib/st33ktpm2x.h, [677](#), [694](#)
Core/App/tof_main.c, [696](#)
Core/App/tof_main.h, [697](#), [698](#)
Core/App/vl53l3cx_lib/vl53l3cx_lib.c, [698](#)
Core/App/vl53l3cx_lib/vl53l3cx_lib.h, [701](#), [702](#)
counter
 laviet_frame_t, [81](#)
 radio_main_ctx_t, [107](#)
crc
 security_key_wire_t, [124](#)
 security_settings_wire_t, [128](#)
crc16
 i2c_mem_store_meta_wire_t, [77](#)
crc_on
 radio_lora_cfg_t, [100](#)
crc_type
 radio_fsk_cfg_t, [95](#)
 radio_main_fsk_cfg_t, [112](#)
crypto_key
 i2c_mem_store_cfg_t, [74](#)
crypto_ready
 radio_main_ctx_t, [107](#)
current_page
 menu_state_t, [90](#)
data
 laviet_sha256_ctx_t, [82](#)
 radio_main_cmd_t, [103](#)
 radio_packet_t, [120](#)
data_whitening
 radio_fsk_cfg_t, [95](#)
 radio_main_fsk_cfg_t, [112](#)
datalen
 laviet_sha256_ctx_t, [82](#)
davint_active_state
 st33ktpm2x_cfg_t, [131](#)
davint_pin
 st33ktpm2x_cfg_t, [131](#)
davint_port
 st33ktpm2x_cfg_t, [131](#)
 deadline_ms
 radio_main_ctx_t, [107](#)
device_code
 menu_notification_t, [87](#)
dig_H1
 BMP280_HandleTypeDef, [69](#)
dig_H2
 BMP280_HandleTypeDef, [69](#)
dig_H3
 BMP280_HandleTypeDef, [69](#)
dig_H4
 BMP280_HandleTypeDef, [69](#)
dig_H5
 BMP280_HandleTypeDef, [69](#)
dig_H6
 BMP280_HandleTypeDef, [69](#)
dig_P1
 BMP280_HandleTypeDef, [70](#)
dig_P2
 BMP280_HandleTypeDef, [70](#)
dig_P3
 BMP280_HandleTypeDef, [70](#)
dig_P4
 BMP280_HandleTypeDef, [70](#)
dig_P5
 BMP280_HandleTypeDef, [70](#)
dig_P6
 BMP280_HandleTypeDef, [70](#)
dig_P7
 BMP280_HandleTypeDef, [70](#)
dig_P8
 BMP280_HandleTypeDef, [70](#)
dig_P9
 BMP280_HandleTypeDef, [70](#)
dig_T1
 BMP280_HandleTypeDef, [70](#)
dig_T2
 BMP280_HandleTypeDef, [71](#)
dig_T3
 BMP280_HandleTypeDef, [71](#)
dio
 radio_hw_cfg_t, [99](#)
dio_pending_mask
 radio_context_t, [93](#)
 radio_fsk_context_t, [97](#)
 radio_ook_context_t, [119](#)
Dokumentacja katalogu Core/App, [55](#)
Dokumentacja katalogu Core/App/i2c_mem_store_lib, [57](#)
Dokumentacja katalogu Core/App/led_array_lib, [58](#)
Dokumentacja katalogu Core/App/radio_lib, [60](#)
Dokumentacja katalogu Core/App/security, [62](#)
Dokumentacja katalogu Core/App/st33ktpm2x_lib, [64](#)
done
 radio_main_cmd_sync_t, [102](#)
 security_cmd_sync_t, [121](#)
dst_id

- laviat_frame_t, [81](#)
- radio_main_cmd_t, [103](#)
- duration_ms
 - led_array_ctx_t, [85](#)
- effect
 - led_array_ctx_t, [85](#)
- elapsed_ms
 - led_array_ctx_t, [85](#)
- enabled
 - radio_main_cmd_t, [103](#)
 - security_cmd_t, [123](#)
- err_text
 - menu_radio_action_binding_t, [89](#)
- event_flags
 - radio_context_t, [93](#)
 - radio_fsk_context_t, [97](#)
 - radio_ook_context_t, [119](#)
- expires_ms
 - radio_main_ctx_t, [107](#)
- fh_enabled
 - radio_main_ctx_t, [107](#)
 - radio_main_runtime_cfg_t, [116](#)
 - security_runtime_cfg_t, [126](#)
- fh_period_ms
 - radio_main_ctx_t, [107](#)
 - radio_main_runtime_cfg_t, [116](#)
 - security_runtime_cfg_t, [126](#)
- filter
 - bmp280_params_t, [72](#)
 - radio_fsk_cfg_t, [95](#)
 - radio_main_fsk_cfg_t, [112](#)
- flags
 - laviat_frame_t, [81](#)
 - security_settings_wire_t, [128](#)
 - security_store_wire_t, [129](#)
- frequency_hz
 - radio_fsk_cfg_t, [95](#)
 - radio_lora_cfg_t, [100](#)
 - radio_main_fsk_cfg_t, [112](#)
 - radio_main_ook_cfg_t, [114](#)
 - radio_ook_cfg_t, [117](#)
- fsk
 - radio_main_runtime_cfg_t, [116](#)
 - security_runtime_cfg_t, [126](#)
- fsk_cfg
 - radio_main_ctx_t, [107](#)
- gateway_key_mode
 - radio_main_ctx_t, [107](#)
- gateway_key_mode_known
 - radio_main_ctx_t, [107](#)
- gateway_pair_code
 - radio_main_ctx_t, [107](#)
- gateway_pair_code_len
 - radio_main_ctx_t, [107](#)
- gateway_pair_code_valid
 - radio_main_ctx_t, [108](#)
- gateway_rx_counter
 - radio_main_ctx_t, [108](#)
- gateway_slot
 - security_cmd_t, [123](#)
- gateway_tx_counter
 - radio_main_ctx_t, [108](#)
- Generowanie dokumentacji, [2](#)
- group_id
 - radio_main_cmd_t, [103](#)
- HAL_GPIO_EXTI_Callback
 - radio_lib.c, [463](#)
- hi2c
 - i2c_mem_store_cfg_t, [74](#)
 - st33ktpm2x_cfg_t, [131](#)
- hop_idx
 - radio_main_ctx_t, [108](#)
- hspi
 - radio_hw_cfg_t, [99](#)
 - sx1276_bus_t, [134](#)
- hw
 - radio_context_t, [93](#)
 - radio_fsk_context_t, [97](#)
 - radio_main_ctx_t, [108](#)
 - radio_ook_context_t, [119](#)
- i2c
 - BMP280_HandleTypeDef, [71](#)
- i2c_addr_7bit
 - i2c_mem_store_cfg_t, [74](#)
 - st33ktpm2x_cfg_t, [131](#)
- i2c_mem_store.c
 - i2c_mem_store_append_message, [172](#)
 - i2c_mem_store_crc16_ccitt, [173](#)
 - i2c_mem_store_default_cfg, [173](#)
 - i2c_mem_store_default_cfg_m24c01r, [173](#)
 - i2c_mem_store_format, [173](#)
 - i2c_mem_store_get_log_count, [174](#)
 - i2c_mem_store_hal_to_status, [174](#)
 - i2c_mem_store_init, [174](#)
 - i2c_mem_store_layout_valid, [175](#)
 - i2c_mem_store_le16_read, [175](#)
 - i2c_mem_store_le16_write, [175](#)
 - i2c_mem_store_le32_read, [175](#)
 - i2c_mem_store_le32_write, [175](#)
 - i2c_mem_store_log_base_addr, [176](#)
 - I2C_MEM_STORE_LOG_CRC_OFFSET, [171](#)
 - I2C_MEM_STORE_LOG_HEADER_SIZE, [171](#)
 - I2C_MEM_STORE_LOG_MAGIC, [171](#)
 - i2c_mem_store_log_slot_addr, [176](#)
 - I2C_MEM_STORE_LOG_START_ADDR, [171](#)
 - i2c_mem_store_meta_decode, [176](#)
 - i2c_mem_store_meta_encode, [176](#)
 - i2c_mem_store_meta_load, [176](#)
 - I2C_MEM_STORE_META_MAGIC, [171](#)
 - I2C_MEM_STORE_META_MIRROR_ADDR, [172](#)
 - I2C_MEM_STORE_META_PRIMARY_ADDR, [172](#)
 - i2c_mem_store_meta_save, [176](#)
 - I2C_MEM_STORE_META_VERSION, [172](#)

I2C_MEM_STORE_META_WIRE_SIZE, 172
 i2c_mem_store_min_u16, 177
 i2c_mem_store_read_message, 177
 i2c_mem_store_read_raw, 177
 i2c_mem_store_secret_base_addr, 177
 I2C_MEM_STORE_SECRET_CRC_OFFSET, 172
 i2c_mem_store_secret_erase, 178
 I2C_MEM_STORE_SECRET_HEADER_SIZE, 172
 I2C_MEM_STORE_SECRET_MAGIC, 172
 i2c_mem_store_secret_read, 178
 i2c_mem_store_secret_slot_addr, 178
 i2c_mem_store_secret_write, 179
 i2c_mem_store_trusted_device_read, 179
 i2c_mem_store_trusted_device_write, 180
 i2c_mem_store_write_raw, 180
 i2c_mem_store_xtea_ctr_crypt, 180
 i2c_mem_store_xtea_encrypt_block, 180
 i2c_mem_store.h
 i2c_mem_store_append_message, 193
 i2c_mem_store_default_cfg, 194
 i2c_mem_store_default_cfg_m24c01r, 194
 I2C_MEM_STORE_ECRC, 193
 I2C_MEM_STORE_EFULL, 193
 I2C_MEM_STORE_EHAL, 193
 I2C_MEM_STORE_EINVAL, 193
 I2C_MEM_STORE_ENOTFOUND, 193
 I2C_MEM_STORE_EOVERFLOW, 193
 I2C_MEM_STORE_EPROTO, 193
 I2C_MEM_STORE_ESTATE, 193
 I2C_MEM_STORE_ETIMEOUT, 193
 i2c_mem_store_format, 194
 i2c_mem_store_get_log_count, 195
 I2C_MEM_STORE_I2C_ADDR_DEFAULT, 192
 i2c_mem_store_init, 195
 I2C_MEM_STORE_LOG_PAYLOAD_MAX, 192
 I2C_MEM_STORE_LOG_RECORD_SIZE, 192
 I2C_MEM_STORE_OK, 193
 I2C_MEM_STORE_PAGE_BYTES_DEFAULT, 192
 i2c_mem_store_read_message, 195
 I2C_MEM_STORE_SECRET_AREA_BYTES_DEFAULT, 192
 i2c_mem_store_secret_erase, 196
 I2C_MEM_STORE_SECRET_PAYLOAD_MAX, 192
 i2c_mem_store_secret_read, 196
 I2C_MEM_STORE_SECRET_SLOT_SIZE, 192
 i2c_mem_store_secret_write, 197
 i2c_mem_store_status_t, 193
 I2C_MEM_STORE_TOTAL_BYTES_DEFAULT, 192
 I2C_MEM_STORE_TRUSTED_CODE_MAX, 192
 i2c_mem_store_trusted_device_read, 197
 i2c_mem_store_trusted_device_write, 198
 I2C_MEM_STORE_TRUSTED_ID_MAX, 192
 I2C_MEM_STORE_WRITE_CYCLE_MS_DEFAULT, 193
 i2c_mem_store_append_message
 i2c_mem_store.c, 172
 i2c_mem_store.h, 193
 i2c_mem_store_cfg_t, 74
 crypto_key, 74
 hi2c, 74
 i2c_addr_7bit, 74
 mem_addr_size, 75
 page_bytes, 75
 secret_area_bytes, 75
 total_bytes, 75
 write_cycle_ms, 75
 i2c_mem_store_crc16_ccitt
 i2c_mem_store.c, 173
 i2c_mem_store_default_cfg
 i2c_mem_store.c, 173
 i2c_mem_store.h, 194
 i2c_mem_store_default_cfg_m24c01r
 i2c_mem_store.c, 173
 i2c_mem_store.h, 194
 I2C_MEM_STORE_ECRC
 i2c_mem_store.h, 193
 I2C_MEM_STORE_EFULL
 i2c_mem_store.h, 193
 I2C_MEM_STORE_EHAL
 i2c_mem_store.h, 193
 I2C_MEM_STORE_EINVAL
 i2c_mem_store.h, 193
 I2C_MEM_STORE_ENOTFOUND
 i2c_mem_store.h, 193
 I2C_MEM_STORE_EOVERFLOW
 i2c_mem_store.h, 193
 I2C_MEM_STORE_EPROTO
 i2c_mem_store.h, 193
 I2C_MEM_STORE_ESTATE
 i2c_mem_store.h, 193
 I2C_MEM_STORE_ETIMEOUT
 i2c_mem_store.h, 193
 i2c_mem_store_format
 i2c_mem_store.c, 173
 i2c_mem_store.h, 194
 i2c_mem_store_get_log_count
 i2c_mem_store.c, 174
 i2c_mem_store.h, 195
 i2c_mem_store_hal_to_status
 i2c_mem_store.c, 174
 I2C_MEM_STORE_I2C_ADDR_DEFAULT
 i2c_mem_store.h, 192
 i2c_mem_store_init
 i2c_mem_store.c, 174
 i2c_mem_store.h, 195
 i2c_mem_store_layout_valid
 i2c_mem_store.c, 175
 i2c_mem_store_le16_read
 i2c_mem_store.c, 175
 i2c_mem_store_le16_write
 i2c_mem_store.c, 175
 i2c_mem_store_le32_read
 i2c_mem_store.c, 175

i2c_mem_store_le32_write
i2c_mem_store.c, [175](#)

i2c_mem_store_log_base_addr
i2c_mem_store.c, [176](#)

I2C_MEM_STORE_LOG_CRC_OFFSET
i2c_mem_store.c, [171](#)

I2C_MEM_STORE_LOG_HEADER_SIZE
i2c_mem_store.c, [171](#)

I2C_MEM_STORE_LOG_MAGIC
i2c_mem_store.c, [171](#)

I2C_MEM_STORE_LOG_PAYLOAD_MAX
i2c_mem_store.h, [192](#)

I2C_MEM_STORE_LOG_RECORD_SIZE
i2c_mem_store.h, [192](#)

i2c_mem_store_log_record_t, [75](#)
length, [76](#)
payload, [76](#)
rssi_dbm, [76](#)
sequence, [76](#)
timestamp_ms, [76](#)

i2c_mem_store_log_slot_addr
i2c_mem_store.c, [176](#)

I2C_MEM_STORE_LOG_START_ADDR
i2c_mem_store.c, [171](#)

i2c_mem_store_meta_decode
i2c_mem_store.c, [176](#)

i2c_mem_store_meta_encode
i2c_mem_store.c, [176](#)

i2c_mem_store_meta_load
i2c_mem_store.c, [176](#)

I2C_MEM_STORE_META_MAGIC
i2c_mem_store.c, [171](#)

I2C_MEM_STORE_META_MIRROR_ADDR
i2c_mem_store.c, [172](#)

I2C_MEM_STORE_META_PRIMARY_ADDR
i2c_mem_store.c, [172](#)

i2c_mem_store_meta_save
i2c_mem_store.c, [176](#)

I2C_MEM_STORE_META_VERSION
i2c_mem_store.c, [172](#)

I2C_MEM_STORE_META_WIRE_SIZE
i2c_mem_store.c, [172](#)

i2c_mem_store_meta_wire_t, [77](#)
crc16, [77](#)
magic, [77](#)
next_sequence, [77](#)
reserved, [77](#)
reserved2, [77](#)
secret_counter, [77](#)
valid_count, [77](#)
version, [77](#)
write_slot, [77](#)

i2c_mem_store_min_u16
i2c_mem_store.c, [177](#)

I2C_MEM_STORE_OK
i2c_mem_store.h, [193](#)

I2C_MEM_STORE_PAGE_BYTES_DEFAULT
i2c_mem_store.h, [192](#)

i2c_mem_store_read_message
i2c_mem_store.c, [177](#)
i2c_mem_store.h, [195](#)

i2c_mem_store_read_raw
i2c_mem_store.c, [177](#)

I2C_MEM_STORE_SECRET_AREA_BYTES_DEFAULT
i2c_mem_store.h, [192](#)

i2c_mem_store_secret_base_addr
i2c_mem_store.c, [177](#)

I2C_MEM_STORE_SECRET_CRC_OFFSET
i2c_mem_store.c, [172](#)

i2c_mem_store_secret_erase
i2c_mem_store.c, [178](#)
i2c_mem_store.h, [196](#)

I2C_MEM_STORE_SECRET_HEADER_SIZE
i2c_mem_store.c, [172](#)

I2C_MEM_STORE_SECRET_MAGIC
i2c_mem_store.c, [172](#)

I2C_MEM_STORE_SECRET_PAYLOAD_MAX
i2c_mem_store.h, [192](#)

i2c_mem_store_secret_read
i2c_mem_store.c, [178](#)
i2c_mem_store.h, [196](#)

i2c_mem_store_secret_slot_addr
i2c_mem_store.c, [178](#)

I2C_MEM_STORE_SECRET_SLOT_SIZE
i2c_mem_store.h, [192](#)

i2c_mem_store_secret_write
i2c_mem_store.c, [179](#)
i2c_mem_store.h, [197](#)

i2c_mem_store_status_t
i2c_mem_store.h, [193](#)

i2c_mem_store_t, [78](#)
cfg, [78](#)
initialized, [78](#)
next_sequence, [78](#)
secret_counter, [79](#)
secret_slot_count, [79](#)
slot_count, [79](#)
valid_count, [79](#)
write_slot, [79](#)

I2C_MEM_STORE_TOTAL_BYTES_DEFAULT
i2c_mem_store.h, [192](#)

I2C_MEM_STORE_TRUSTED_CODE_MAX
i2c_mem_store.h, [192](#)

i2c_mem_store_trusted_device_read
i2c_mem_store.c, [179](#)
i2c_mem_store.h, [197](#)

i2c_mem_store_trusted_device_t, [79](#)
code, [80](#)
code_len, [80](#)
id, [80](#)
id_len, [80](#)

i2c_mem_store_trusted_device_write
i2c_mem_store.c, [180](#)
i2c_mem_store.h, [198](#)

I2C_MEM_STORE_TRUSTED_ID_MAX
i2c_mem_store.h, [192](#)

I2C_MEM_STORE_WRITE_CYCLE_MS_DEFAULT
 i2c_mem_store.h, [193](#)
 i2c_mem_store_write_raw
 i2c_mem_store.c, [180](#)
 i2c_mem_store_xtea_ctr_crypt
 i2c_mem_store.c, [180](#)
 i2c_mem_store_xtea_encrypt_block
 i2c_mem_store.c, [180](#)
 id
 BMP280_HandleTypeDef, [71](#)
 i2c_mem_store_trusted_device_t, [80](#)
 radio_main_cmd_t, [103](#)
 security_cmd_t, [123](#)
 id_len
 i2c_mem_store_trusted_device_t, [80](#)
 implicit_header
 radio_lora_cfg_t, [100](#)
 in_use
 security_trusted_entry_t, [130](#)
 trusted_info_t, [135](#)
 index
 security_cmd_t, [123](#)
 initialized
 i2c_mem_store_t, [78](#)
 lcd_state_t, [84](#)
 led_array_ctx_t, [85](#)
 radio_context_t, [93](#)
 radio_fsk_context_t, [97](#)
 radio_main_ctx_t, [108](#)
 radio_ook_context_t, [119](#)
 st33ktpm2x_t, [133](#)
 invert_iq
 radio_lora_cfg_t, [100](#)
 io_timeout_ms
 st33ktpm2x_cfg_t, [131](#)
 is_master
 security_trusted_entry_t, [130](#)
 trusted_info_t, [135](#)
 item_count
 menu_page_t, [88](#)
 items
 menu_page_t, [88](#)
 label
 menu_item_t, [87](#)
 last_error_text
 radio_main_ctx_t, [108](#)
 last_hop_ms
 radio_main_ctx_t, [108](#)
 last_input_ms
 menu_state_t, [90](#)
 last_packet
 radio_context_t, [93](#)
 radio_fsk_context_t, [97](#)
 radio_ook_context_t, [119](#)
 last_ping_ms
 radio_main_ctx_t, [108](#)
 last_tick_ms
 led_array_ctx_t, [85](#)
 LAVIET_ACK_PAYLOAD_LEN
 laviet_frame.h, [239](#)
 laviet_aes_add_round_key
 laviet_crypto.c, [202](#)
 laviet_aes_ctr_crypt
 laviet_crypto.c, [202](#)
 laviet_crypto.h, [224](#)
 laviet_aes_ctr_crypt_hw_locked
 laviet_crypto.c, [203](#)
 laviet_aes_ctr_crypt_sw
 laviet_crypto.c, [203](#)
 laviet_aes_encrypt_block
 laviet_crypto.c, [203](#)
 laviet_aes_key_expansion
 laviet_crypto.c, [203](#)
 LAVIET_AES_KEY_LEN
 laviet_crypto.h, [223](#)
 laviet_aes_mix_columns
 laviet_crypto.c, [203](#)
 laviet_aes_shift_rows
 laviet_crypto.c, [204](#)
 laviet_aes_sub_bytes
 laviet_crypto.c, [204](#)
 laviet_aes_xtime
 laviet_crypto.c, [204](#)
 laviet_be16_read
 laviet_frame.c, [227](#)
 laviet_be16_write
 laviet_frame.c, [227](#)
 laviet_be32_read
 laviet_frame.c, [228](#)
 laviet_be32_write
 laviet_frame.c, [228](#)
 LAVIET_BROADCAST_ID
 laviet_frame.h, [239](#)
 LAVIET_COUNTER_SYNC_PAYLOAD_LEN
 laviet_frame.h, [239](#)
 laviet_crypto.c
 laviet_aes_add_round_key, [202](#)
 laviet_aes_ctr_crypt, [202](#)
 laviet_aes_ctr_crypt_hw_locked, [203](#)
 laviet_aes_ctr_crypt_sw, [203](#)
 laviet_aes_encrypt_block, [203](#)
 laviet_aes_key_expansion, [203](#)
 laviet_aes_mix_columns, [203](#)
 laviet_aes_shift_rows, [204](#)
 laviet_aes_sub_bytes, [204](#)
 laviet_aes_xtime, [204](#)
 laviet_crypto_init, [204](#)
 laviet_crypto_lock, [204](#)
 laviet_crypto_log_mismatch, [204](#)
 laviet_crypto_self_test_locked, [204](#)
 LAVIET_CRYPT_TIMEOUT_MS, [202](#)
 laviet_crypto_unlock, [205](#)
 laviet_frame_hmac_sha256, [205](#)
 laviet_hal_status_text, [205](#)
 laviet_hmac_sha256, [205](#)
 laviet_hmac_sha256_hw_locked, [206](#)

- laviet_hmac_sha256_sw, [206](#)
- laviet_mac_equal, [206](#)
- laviet_make_counter_block, [207](#)
- laviet_rotr32, [207](#)
- laviet_secure_zero, [207](#)
- laviet_sha256_final, [207](#)
- laviet_sha256_init, [207](#)
- laviet_sha256_transform, [208](#)
- laviet_sha256_update, [208](#)
- LAVIET_TEST_FIXED_FRAME_MAC, [202](#)
- LAVIET_TEST_FIXED_FRAME_MAC_BYTE, [202](#)
- s_aes_rcon, [208](#)
- s_aes_sbox, [208](#)
- s_crypto_hw_ready, [208](#)
- s_crypto_mutex, [208](#)
- s_sha256_k, [208](#)
- laviet_crypto.h
 - laviet_aes_ctr_crypt, [224](#)
 - LAVIET_AES_KEY_LEN, [223](#)
 - laviet_crypto_init, [224](#)
 - laviet_frame_hmac_sha256, [224](#)
 - LAVIET_HMAC_KEY_LEN, [223](#)
 - laviet_hmac_sha256, [225](#)
 - laviet_mac_equal, [225](#)
 - laviet_secure_zero, [225](#)
 - LAVIET_SHA256_LEN, [223](#)
- laviet_crypto_init
 - laviet_crypto.c, [204](#)
 - laviet_crypto.h, [224](#)
- laviet_crypto_lock
 - laviet_crypto.c, [204](#)
- laviet_crypto_log_mismatch
 - laviet_crypto.c, [204](#)
- laviet_crypto_self_test_locked
 - laviet_crypto.c, [204](#)
- LAVIET_CRYPTO_TIMEOUT_MS
 - laviet_crypto.c, [202](#)
- laviet_crypto_unlock
 - laviet_crypto.c, [205](#)
- LAVIET_ERROR_PAYLOAD_LEN
 - laviet_frame.h, [239](#)
- LAVIET_FLAG_ACK_REQUIRED
 - laviet_frame.h, [240](#)
- LAVIET_FLAG_BROADCAST
 - laviet_frame.h, [240](#)
- LAVIET_FLAG_CONFIG_ACCESS
 - laviet_frame.h, [240](#)
- LAVIET_FLAG_COUNTER_OVERRIDE
 - laviet_frame.h, [240](#)
- LAVIET_FLAG_ENCRYPTED
 - laviet_frame.h, [240](#)
- LAVIET_FLAG_IS_ACK
 - laviet_frame.h, [240](#)
- LAVIET_FLAG_KEY_UPDATE
 - laviet_frame.h, [240](#)
- LAVIET_FLAG_PAIRING
 - laviet_frame.h, [240](#)
- laviet_frame.c
 - laviet_be16_read, [227](#)
 - laviet_be16_write, [227](#)
 - laviet_be32_read, [228](#)
 - laviet_be32_write, [228](#)
 - laviet_frame_build_mac_input, [228](#)
 - laviet_frame_decode, [228](#)
 - laviet_frame_encode, [229](#)
 - laviet_frame_is_broadcast, [229](#)
 - laviet_frame_read_ack_payload, [230](#)
 - laviet_frame_type, [230](#)
 - laviet_frame_type_known, [230](#)
 - laviet_frame_validate_plain, [230](#)
 - laviet_frame_ver_type, [231](#)
 - laviet_frame_version_from_ver_type, [231](#)
 - laviet_frame_write_ack_payload, [231](#)
 - laviet_local_node_id, [232](#)
- laviet_frame.h
 - LAVIET_ACK_PAYLOAD_LEN, [239](#)
 - LAVIET_BROADCAST_ID, [239](#)
 - LAVIET_COUNTER_SYNC_PAYLOAD_LEN, [239](#)
 - LAVIET_ERROR_PAYLOAD_LEN, [239](#)
 - LAVIET_FLAG_ACK_REQUIRED, [240](#)
 - LAVIET_FLAG_BROADCAST, [240](#)
 - LAVIET_FLAG_CONFIG_ACCESS, [240](#)
 - LAVIET_FLAG_COUNTER_OVERRIDE, [240](#)
 - LAVIET_FLAG_ENCRYPTED, [240](#)
 - LAVIET_FLAG_IS_ACK, [240](#)
 - LAVIET_FLAG_KEY_UPDATE, [240](#)
 - LAVIET_FLAG_PAIRING, [240](#)
 - laviet_frame_build_mac_input, [242](#)
 - laviet_frame_decode, [243](#)
 - laviet_frame_encode, [243](#)
 - LAVIET_FRAME_HEADER_LEN, [241](#)
 - laviet_frame_is_broadcast, [244](#)
 - LAVIET_FRAME_MAX_LEN, [241](#)
 - LAVIET_FRAME_MIN_LEN, [241](#)
 - laviet_frame_read_ack_payload, [244](#)
 - laviet_frame_status_t, [242](#)
 - laviet_frame_type, [244](#)
 - laviet_frame_type_t, [242](#)
 - laviet_frame_validate_plain, [245](#)
 - laviet_frame_ver_type, [245](#)
 - LAVIET_FRAME_VERSION, [241](#)
 - laviet_frame_write_ack_payload, [246](#)
 - LAVIET_GATEWAY_ID, [241](#)
 - laviet_local_node_id, [246](#)
 - LAVIET_MAC_TAG_LEN, [241](#)
 - LAVIET_MAX_PAYLOAD, [241](#)
 - LAVIET_PAIR_PAYLOAD_LEN, [241](#)
 - LAVIET_STATUS_FLAGS, [242](#)
 - LAVIET_STATUS_ID, [242](#)
 - LAVIET_STATUS_LENGTH, [242](#)
 - LAVIET_STATUS_NULL, [242](#)
 - LAVIET_STATUS_OK, [242](#)
 - LAVIET_STATUS_PAYLOAD, [242](#)
 - LAVIET_STATUS_TYPE, [242](#)
 - LAVIET_STATUS_VERSION, [242](#)
 - LAVIET_TYPE_ACK, [242](#)

LAVIET_TYPE_CFG, [242](#)
 LAVIET_TYPE_COUNTER_SYNC, [242](#)
 LAVIET_TYPE_DATA, [242](#)
 LAVIET_TYPE_ERROR, [242](#)
 LAVIET_TYPE_KEY_ROTATE, [242](#)
 LAVIET_TYPE_NONE, [242](#)
 LAVIET_TYPE_PAIR_REQ, [242](#)
 LAVIET_TYPE_PAIR_RESP, [242](#)
 LAVIET_TYPE_RESP, [242](#)
 laviet_frame_build_mac_input
 laviet_frame.c, [228](#)
 laviet_frame.h, [242](#)
 laviet_frame_decode
 laviet_frame.c, [228](#)
 laviet_frame.h, [243](#)
 laviet_frame_encode
 laviet_frame.c, [229](#)
 laviet_frame.h, [243](#)
 LAVIET_FRAME_HEADER_LEN
 laviet_frame.h, [241](#)
 laviet_frame_hmac_sha256
 laviet_crypto.c, [205](#)
 laviet_crypto.h, [224](#)
 laviet_frame_is_broadcast
 laviet_frame.c, [229](#)
 laviet_frame.h, [244](#)
 LAVIET_FRAME_MAX_LEN
 laviet_frame.h, [241](#)
 LAVIET_FRAME_MIN_LEN
 laviet_frame.h, [241](#)
 laviet_frame_read_ack_payload
 laviet_frame.c, [230](#)
 laviet_frame.h, [244](#)
 laviet_frame_status_t
 laviet_frame.h, [242](#)
 laviet_frame_t, [80](#)
 counter, [81](#)
 dst_id, [81](#)
 flags, [81](#)
 mac_tag, [81](#)
 msg_id, [81](#)
 payload, [81](#)
 payload_len, [82](#)
 src_id, [82](#)
 ver_type, [82](#)
 laviet_frame_type
 laviet_frame.c, [230](#)
 laviet_frame.h, [244](#)
 laviet_frame_type_known
 laviet_frame.c, [230](#)
 laviet_frame_type_t
 laviet_frame.h, [242](#)
 laviet_frame_validate_plain
 laviet_frame.c, [230](#)
 laviet_frame.h, [245](#)
 laviet_frame_ver_type
 laviet_frame.c, [231](#)
 laviet_frame.h, [245](#)
 LAVIET_FRAME_VERSION
 laviet_frame.h, [241](#)
 laviet_frame_version_from_ver_type
 laviet_frame.c, [231](#)
 laviet_frame_write_ack_payload
 laviet_frame.c, [231](#)
 laviet_frame.h, [246](#)
 LAVIET_GATEWAY_ID
 laviet_frame.h, [241](#)
 laviet_hal_status_text
 laviet_crypto.c, [205](#)
 LAVIET_HMAC_KEY_LEN
 laviet_crypto.h, [223](#)
 laviet_hmac_sha256
 laviet_crypto.c, [205](#)
 laviet_crypto.h, [225](#)
 laviet_hmac_sha256_hw_locked
 laviet_crypto.c, [206](#)
 laviet_hmac_sha256_sw
 laviet_crypto.c, [206](#)
 laviet_local_node_id
 laviet_frame.c, [232](#)
 laviet_frame.h, [246](#)
 laviet_mac_equal
 laviet_crypto.c, [206](#)
 laviet_crypto.h, [225](#)
 LAVIET_MAC_TAG_LEN
 laviet_frame.h, [241](#)
 laviet_make_counter_block
 laviet_crypto.c, [207](#)
 LAVIET_MAX_PAYLOAD
 laviet_frame.h, [241](#)
 LAVIET_PAIR_PAYLOAD_LEN
 laviet_frame.h, [241](#)
 laviet_rotr32
 laviet_crypto.c, [207](#)
 laviet_secure_zero
 laviet_crypto.c, [207](#)
 laviet_crypto.h, [225](#)
 laviet_sha256_ctx_t, [82](#)
 bitlen, [82](#)
 data, [82](#)
 datalen, [82](#)
 state, [83](#)
 laviet_sha256_final
 laviet_crypto.c, [207](#)
 laviet_sha256_init
 laviet_crypto.c, [207](#)
 LAVIET_SHA256_LEN
 laviet_crypto.h, [223](#)
 laviet_sha256_transform
 laviet_crypto.c, [208](#)
 laviet_sha256_update
 laviet_crypto.c, [208](#)
 LAVIET_STATUS_FLAGS
 laviet_frame.h, [242](#)
 LAVIET_STATUS_ID
 laviet_frame.h, [242](#)

LAVIET_STATUS_LENGTH
 laviet_frame.h, [242](#)

LAVIET_STATUS_NULL
 laviet_frame.h, [242](#)

LAVIET_STATUS_OK
 laviet_frame.h, [242](#)

LAVIET_STATUS_PAYLOAD
 laviet_frame.h, [242](#)

LAVIET_STATUS_TYPE
 laviet_frame.h, [242](#)

LAVIET_STATUS_VERSION
 laviet_frame.h, [242](#)

LAVIET_TEST_FIXED_FRAME_MAC
 laviet_crypto.c, [202](#)

LAVIET_TEST_FIXED_FRAME_MAC_BYTE
 laviet_crypto.c, [202](#)

LAVIET_TYPE_ACK
 laviet_frame.h, [242](#)

LAVIET_TYPE_CFG
 laviet_frame.h, [242](#)

LAVIET_TYPE_COUNTER_SYNC
 laviet_frame.h, [242](#)

LAVIET_TYPE_DATA
 laviet_frame.h, [242](#)

LAVIET_TYPE_ERROR
 laviet_frame.h, [242](#)

LAVIET_TYPE_KEY_ROTATE
 laviet_frame.h, [242](#)

LAVIET_TYPE_NONE
 laviet_frame.h, [242](#)

LAVIET_TYPE_PAIR_REQ
 laviet_frame.h, [242](#)

LAVIET_TYPE_PAIR_RESP
 laviet_frame.h, [242](#)

LAVIET_TYPE_RESP
 laviet_frame.h, [242](#)

lcd.h
 lcd_animation_hello_beko, [255](#)
 lcd_backlight, [255](#)
 lcd_clear, [255](#)
 lcd_demo, [256](#)
 lcd_init, [256](#)
 lcd_set_cursor, [256](#)
 lcd_write_line, [256](#)
 lcd_write_string, [257](#)

lcd_animation_hello_beko
 lcd.h, [255](#)

lcd_backlight
 lcd.h, [255](#)

lcd_clear
 lcd.h, [255](#)

lcd_demo
 lcd.h, [256](#)

lcd_init
 lcd.h, [256](#)

lcd_main.h
 lcd_main_create_task, [266](#)
 lcd_main_mode_t, [266](#)
 lcd_main_monitor_scroll_down, [266](#)
 lcd_main_monitor_scroll_up, [266](#)
 lcd_main_push_message, [266](#)
 lcd_main_push_message_from, [267](#)
 lcd_main_set_line, [267](#)
 lcd_main_set_lines, [268](#)
 lcd_main_set_mode, [268](#)
 lcd_main_show_boot_hello, [268](#)
 lcd_main_show_menu, [268](#)
 lcd_main_show_popup, [269](#)
 LCD_MODE_MENU, [266](#)
 LCD_MODE_MONITOR, [266](#)
 LCD_MODE_POPUP, [266](#)
 lcd_main_create_task
 lcd_main.h, [266](#)
 lcd_main_mode_t
 lcd_main.h, [266](#)
 lcd_main_monitor_scroll_down
 lcd_main.h, [266](#)
 lcd_main_monitor_scroll_up
 lcd_main.h, [266](#)
 lcd_main_msg_t, [83](#)
 line_index, [83](#)
 mode, [83](#)
 text0, [83](#)
 text1, [83](#)
 text2, [83](#)
 text3, [83](#)
 type, [83](#)
 lcd_main_push_message
 lcd_main.h, [266](#)
 lcd_main_push_message_from
 lcd_main.h, [267](#)
 lcd_main_set_line
 lcd_main.h, [267](#)
 lcd_main_set_lines
 lcd_main.h, [268](#)
 lcd_main_set_mode
 lcd_main.h, [268](#)
 lcd_main_show_boot_hello
 lcd_main.h, [268](#)
 lcd_main_show_menu
 lcd_main.h, [268](#)
 lcd_main_show_popup
 lcd_main.h, [269](#)
 LCD_MODE_MENU
 lcd_main.h, [266](#)
 LCD_MODE_MONITOR
 lcd_main.h, [266](#)
 LCD_MODE_POPUP
 lcd_main.h, [266](#)
 lcd_set_cursor
 lcd.h, [256](#)
 lcd_state_t, [84](#)
 addr_7bit, [84](#)
 backlight, [84](#)
 initialized, [84](#)
 lcd_write_line

- lcd.h, 256
- lcd_write_string
 - lcd.h, 257
- led_array_apply_outputs
 - led_array_lib.c, 272
- led_array_clamp_pct
 - led_array_lib.c, 272
- LED_ARRAY_COUNT
 - led_array_lib.c, 272
- led_array_ctx_t, 84
 - active_brightness, 85
 - duration_ms, 85
 - effect, 85
 - elapsed_ms, 85
 - initialized, 85
 - last_tick_ms, 85
 - manual_brightness, 85
 - mask, 85
 - max_pct, 86
 - min_pct, 86
 - period_ms, 86
 - phase_accum_ms, 86
 - phase_base, 86
 - step_ms, 86
 - tick_period_ms, 86
 - timer_running, 86
- led_array_deinit
 - led_array_lib.c, 272
 - led_array_lib.h, 288
- led_array_effect_breath_update
 - led_array_lib.c, 272
- led_array_effect_fade_update
 - led_array_lib.c, 273
- led_array_effect_none
 - led_array_lib.c, 273
- led_array_effect_rainbow_update
 - led_array_lib.c, 273
- led_array_effect_t
 - led_array_lib.h, 288
- led_array_fire_fade_out
 - led_array_lib.c, 273
 - led_array_lib.h, 289
- led_array_get_effect
 - led_array_lib.c, 273
 - led_array_lib.h, 289
- led_array_get_tim_clock
 - led_array_lib.c, 274
- led_array_hw_init
 - led_array_lib.c, 274
- led_array_init
 - led_array_lib.c, 274
 - led_array_lib.h, 289
- led_array_is_initialized
 - led_array_lib.c, 274
 - led_array_lib.h, 289
- LED_ARRAY_LED_1
 - led_array_lib.h, 288
- LED_ARRAY_LED_2
 - led_array_lib.h, 288
- led_array_lib.h, 288
- LED_ARRAY_LED_3
 - led_array_lib.h, 288
- LED_ARRAY_LED_4
 - led_array_lib.h, 288
- LED_ARRAY_LED_5
 - led_array_lib.h, 288
- led_array_led_mask_t
 - led_array_lib.h, 288
- led_array_lib.c
 - led_array_apply_outputs, 272
 - led_array_clamp_pct, 272
 - LED_ARRAY_COUNT, 272
 - led_array_deinit, 272
 - led_array_effect_breath_update, 272
 - led_array_effect_fade_update, 273
 - led_array_effect_none, 273
 - led_array_effect_rainbow_update, 273
 - led_array_fire_fade_out, 273
 - led_array_get_effect, 273
 - led_array_get_tim_clock, 274
 - led_array_hw_init, 274
 - led_array_init, 274
 - led_array_is_initialized, 274
 - led_array_mask_valid, 274
 - led_array_now_ms, 274
 - led_array_off, 274
 - led_array_on, 275
 - led_array_process, 275
 - LED_ARRAY_PWM_FREQ_HZ, 272
 - LED_ARRAY_PWM_STEPS, 272
 - LED_ARRAY_RAINBOW_PHASE_INC, 272
 - led_array_set_brightness, 275
 - led_array_set_hw_duty, 276
 - led_array_start_breath, 276
 - led_array_start_rainbow, 276
 - led_array_stop_effect, 277
 - led_array_tick_core, 277
 - led_array_timer_base_init, 277
 - led_array_timer_deinit, 277
 - led_array_timer_init, 277
 - led_array_timer_pwm_ch_init, 278
 - led_array_timer_tick, 278
 - led_array_tri8, 278
 - LED_ARRAY_VALID_MASK, 272
 - led_array_write_pin, 278
 - s_led, 278
- led_array_lib.h
 - led_array_deinit, 288
 - led_array_effect_t, 288
 - led_array_fire_fade_out, 289
 - led_array_get_effect, 289
 - led_array_init, 289
 - led_array_is_initialized, 289
 - LED_ARRAY_LED_1, 288
 - LED_ARRAY_LED_2, 288
 - LED_ARRAY_LED_3, 288
 - LED_ARRAY_LED_4, 288

LED_ARRAY_LED_5, [288](#)
 led_array_led_mask_t, [288](#)
 led_array_off, [289](#)
 led_array_on, [290](#)
 led_array_process, [290](#)
 led_array_set_brightness, [290](#)
 led_array_start_breath, [291](#)
 led_array_start_rainbow, [291](#)
 led_array_status_t, [288](#)
 led_array_stop_effect, [291](#)
 led_array_timer_deinit, [292](#)
 led_array_timer_init, [292](#)
 led_array_timer_tick, [292](#)
 led_array_lib_config.h
 LED_ARRAY_LIB_DEFAULT_TICK_MS, [294](#)
 LED_ARRAY_LIB_OWNS_HAL_SYSTICK_CALLBACK, [294](#)
 LED_ARRAY_LIB_DEFAULT_TICK_MS
 led_array_lib_config.h, [294](#)
 LED_ARRAY_LIB_OWNS_HAL_SYSTICK_CALLBACK
 led_array_lib_config.h, [294](#)
 led_array_main.h
 led_array_main_create_task, [295](#)
 led_array_main_create_task
 led_array_main.h, [295](#)
 led_array_mask_valid
 led_array_lib.c, [274](#)
 led_array_now_ms
 led_array_lib.c, [274](#)
 led_array_off
 led_array_lib.c, [274](#)
 led_array_lib.h, [289](#)
 led_array_on
 led_array_lib.c, [275](#)
 led_array_lib.h, [290](#)
 led_array_process
 led_array_lib.c, [275](#)
 led_array_lib.h, [290](#)
 LED_ARRAY_PWM_FREQ_HZ
 led_array_lib.c, [272](#)
 LED_ARRAY_PWM_STEPS
 led_array_lib.c, [272](#)
 LED_ARRAY_RAINBOW_PHASE_INC
 led_array_lib.c, [272](#)
 led_array_set_brightness
 led_array_lib.c, [275](#)
 led_array_lib.h, [290](#)
 led_array_set_hw_duty
 led_array_lib.c, [276](#)
 led_array_start_breath
 led_array_lib.c, [276](#)
 led_array_lib.h, [291](#)
 led_array_start_rainbow
 led_array_lib.c, [276](#)
 led_array_lib.h, [291](#)
 led_array_status_t
 led_array_lib.h, [288](#)
 led_array_stop_effect
 led_array_lib.c, [277](#)
 led_array_lib.h, [291](#)
 led_array_tick_core
 led_array_lib.c, [277](#)
 led_array_timer_base_init
 led_array_lib.c, [277](#)
 led_array_timer_deinit
 led_array_lib.c, [277](#)
 led_array_lib.h, [292](#)
 led_array_timer_init
 led_array_lib.c, [277](#)
 led_array_lib.h, [292](#)
 led_array_timer_pwm_ch_init
 led_array_lib.c, [278](#)
 led_array_timer_tick
 led_array_lib.c, [278](#)
 led_array_lib.h, [292](#)
 led_array_tri8
 led_array_lib.c, [278](#)
 LED_ARRAY_VALID_MASK
 led_array_lib.c, [272](#)
 led_array_write_pin
 led_array_lib.c, [278](#)
 led_mode
 menu_state_t, [90](#)
 len
 radio_main_cmd_t, [103](#)
 security_cmd_t, [123](#)
 length
 i2c_mem_store_log_record_t, [76](#)
 radio_packet_t, [120](#)
 line_index
 lcd_main_msg_t, [83](#)
 locality0_acquired
 st33ktpm2x_t, [133](#)
 locality_timeout_ms
 st33ktpm2x_cfg_t, [132](#)
 long_reported
 button_state_t, [73](#)
 lora
 radio_context_t, [93](#)
 radio_main_runtime_cfg_t, [116](#)
 security_runtime_cfg_t, [127](#)
 lora_cfg
 radio_main_ctx_t, [108](#)
 lora_preset
 radio_main_ctx_t, [108](#)
 security_runtime_cfg_t, [127](#)
 security_settings_wire_t, [128](#)
 security_store_wire_t, [129](#)
 mac_tag
 laviot_frame_t, [81](#)
 magic
 i2c_mem_store_meta_wire_t, [77](#)
 security_key_wire_t, [124](#)
 security_radio_store_wire_t, [125](#)
 security_settings_wire_t, [128](#)
 security_store_wire_t, [129](#)

- manual_brightness
 - led_array_ctx_t, [85](#)
- Mapa modulow, [4](#)
- mask
 - led_array_ctx_t, [85](#)
- max_pct
 - led_array_ctx_t, [86](#)
- mem_addr_size
 - i2c_mem_store_cfg_t, [75](#)
- menu_item_t, [86](#)
 - action, [87](#)
 - child_page, [87](#)
 - label, [87](#)
- menu_main.h
 - menu_main_create_task, [350](#)
 - menu_main_post_notification, [350](#)
 - MENU_NOTIFICATION_DELIVERY, [350](#)
 - MENU_NOTIFICATION_ERROR, [349](#)
 - MENU_NOTIFICATION_PAIRING, [349](#)
 - MENU_NOTIFICATION_RX, [349](#)
 - MENU_NOTIFICATION_SECURITY, [350](#)
 - menu_notification_type_t, [349](#)
 - MENU_NOTIFICATION_WARNING, [349](#)
- menu_main_create_task
 - menu_main.h, [350](#)
- menu_main_post_notification
 - menu_main.h, [350](#)
- MENU_NOTIFICATION_DELIVERY
 - menu_main.h, [350](#)
- MENU_NOTIFICATION_ERROR
 - menu_main.h, [349](#)
- MENU_NOTIFICATION_PAIRING
 - menu_main.h, [349](#)
- MENU_NOTIFICATION_RX
 - menu_main.h, [349](#)
- MENU_NOTIFICATION_SECURITY
 - menu_main.h, [350](#)
- menu_notification_t, [87](#)
 - device_code, [87](#)
 - reply_device_code, [87](#)
 - rss_i_dbm, [87](#)
 - text, [88](#)
 - type, [88](#)
- menu_notification_type_t
 - menu_main.h, [349](#)
- MENU_NOTIFICATION_WARNING
 - menu_main.h, [349](#)
- menu_page_t, [88](#)
 - item_count, [88](#)
 - items, [88](#)
 - parent, [88](#)
 - title, [88](#)
- menu_radio_action_binding_t, [89](#)
 - action, [89](#)
 - err_text, [89](#)
 - modulation, [89](#)
 - ok_text, [89](#)
 - option, [89](#)
 - persist_lora_preset, [89](#)
 - set_modulation, [89](#)
 - set_option, [89](#)
 - value, [89](#)
- menu_state_t, [90](#)
 - admin_unlock_until_ms, [90](#)
 - current_page, [90](#)
 - last_input_ms, [90](#)
 - led_mode, [90](#)
 - modal, [91](#)
 - operator_unlock_until_ms, [91](#)
 - pairing_network_mode, [91](#)
 - pending_action, [91](#)
 - pending_auth_level, [91](#)
 - pending_auth_page, [91](#)
 - pending_target_node_id, [91](#)
 - pin_digits, [91](#)
 - pin_index, [91](#)
 - popup_enabled, [91](#)
 - quick_reply_deadline_ms, [91](#)
 - quick_reply_last_seconds, [91](#)
 - quick_reply_src_id, [92](#)
 - quick_reply_text, [92](#)
 - selected_device_slot, [92](#)
 - selected_idx, [92](#)
 - send_target_action, [92](#)
 - transient_parent_page, [92](#)
 - tx_ack_waiting, [92](#)
 - tx_sent_text, [92](#)
- min_pct
 - led_array_ctx_t, [86](#)
- modal
 - menu_state_t, [91](#)
- mode
 - bmp280_params_t, [72](#)
 - lcd_main_msg_t, [83](#)
 - security_cmd_t, [123](#)
- modulation
 - menu_radio_action_binding_t, [89](#)
- modulation_fh
 - security_store_wire_t, [129](#)
- modulation_id
 - radio_main_ctx_t, [108](#)
- msg_id
 - laviet_frame_t, [81](#)
 - radio_main_cmd_t, [103](#)
 - radio_main_ctx_t, [109](#)
- network_mode
 - radio_main_cmd_t, [103](#)
- next_msg_id
 - radio_main_ctx_t, [109](#)
- next_sequence
 - i2c_mem_store_meta_wire_t, [77](#)
 - i2c_mem_store_t, [78](#)
- node_id
 - radio_main_ctx_t, [109](#)
 - security_cmd_t, [123](#)
 - security_trusted_entry_t, [130](#)

- trusted_info_t, 135
- notify_mode
 - security_runtime_cfg_t, 127
 - security_settings_wire_t, 128
 - security_store_wire_t, 129
- nss
 - radio_hw_cfg_t, 99
- nss_pin
 - sx1276_bus_t, 134
- nss_port
 - sx1276_bus_t, 134
- ok_text
 - menu_radio_action_binding_t, 89
- ook
 - radio_main_runtime_cfg_t, 116
 - security_runtime_cfg_t, 127
- ook_cfg
 - radio_main_ctx_t, 109
- operator_unlock_until_ms
 - menu_state_t, 91
- option
 - menu_radio_action_binding_t, 89
 - radio_main_cmd_t, 103
- oversampling_humidity
 - bmp280_params_t, 72
- oversampling_pressure
 - bmp280_params_t, 72
- oversampling_temperature
 - bmp280_params_t, 72
- packed
 - security_radio_store_wire_t, 125
- page_bytes
 - i2c_mem_store_cfg_t, 75
- pager_config.h
 - PAGER_CONFIG_BMP280_MOUNTED, 351
 - PAGER_CONFIG_UART_SENSITIVE_LOGS, 351
 - SERVICE_UART, 351
- PAGER_CONFIG_BMP280_MOUNTED
 - pager_config.h, 351
- PAGER_CONFIG_UART_SENSITIVE_LOGS
 - pager_config.h, 351
- pairing_active
 - radio_main_ctx_t, 109
- pairing_network_mode
 - menu_state_t, 91
 - radio_main_ctx_t, 109
- pairing_outgoing_code
 - radio_main_ctx_t, 109
- pairing_outgoing_code_len
 - radio_main_ctx_t, 109
- pairing_outgoing_network
 - radio_main_ctx_t, 109
- pairing_outgoing_pending
 - radio_main_ctx_t, 109
- pairing_pending
 - radio_main_ctx_t, 109
- pairing_pending_code
 - radio_main_ctx_t, 109
- pairing_pending_code_len
 - radio_main_ctx_t, 110
- pairing_pending_network
 - radio_main_ctx_t, 110
- pairing_pending_node
 - radio_main_ctx_t, 110
- pairing_until_ms
 - radio_main_ctx_t, 110
- params
 - BMP280_HandleTypeDef, 71
- parent
 - menu_page_t, 88
- payload
 - i2c_mem_store_log_record_t, 76
 - laviet_frame_t, 81
 - security_cmd_t, 123
- payload_len
 - laviet_frame_t, 82
 - radio_lora_cfg_t, 101
- peer_id
 - radio_main_ctx_t, 110
- pending_action
 - menu_state_t, 91
- pending_auth_level
 - menu_state_t, 91
- pending_auth_page
 - menu_state_t, 91
- pending_target_node_id
 - menu_state_t, 91
- period_ms
 - led_array_ctx_t, 86
- persist_lora_preset
 - menu_radio_action_binding_t, 89
- phase_accum_ms
 - led_array_ctx_t, 86
- phase_base
 - led_array_ctx_t, 86
- pin
 - button_hw_t, 73
 - radio_gpio_t, 98
- pin_digits
 - menu_state_t, 91
- pin_index
 - menu_state_t, 91
- popup_enabled
 - menu_state_t, 91
- port
 - button_hw_t, 73
 - radio_gpio_t, 98
- pp_active_state
 - st33ktpm2x_cfg_t, 132
- pp_pin
 - st33ktpm2x_cfg_t, 132
- pp_port
 - st33ktpm2x_cfg_t, 132
- preamble_len
 - radio_fsk_cfg_t, 95

- radio_lora_cfg_t, [101](#)
- radio_main_fsk_cfg_t, [112](#)
- radio_main_ook_cfg_t, [114](#)
- radio_ook_cfg_t, [117](#)
- preset_id
 - security_cmd_t, [123](#)
- pressed_since_ms
 - button_state_t, [73](#)
- pressure_hpa
 - bmp280_api_data_t, [67](#)
- pressure_pa
 - bmp280_api_data_t, [67](#)
- quick_reply_deadline_ms
 - menu_state_t, [91](#)
- quick_reply_last_seconds
 - menu_state_t, [91](#)
- quick_reply_src_id
 - menu_state_t, [92](#)
- quick_reply_text
 - menu_state_t, [92](#)
- radio_address_filter_t
 - radio_lib.h, [477](#)
- radio_apply_fsk_config
 - radio_fsk.c, [391](#)
- radio_apply_lora_config
 - radio_lora.c, [414](#)
- radio_apply_ook_config
 - radio_ook.c, [441](#)
- radio_apply_threshold
 - radio_ook.c, [441](#)
- radio_cfg_valid
 - radio_fsk.c, [391](#)
 - radio_lora.c, [415](#)
 - radio_ook.c, [442](#)
- radio_config_dio_for_rx
 - radio_lora.c, [415](#)
- radio_config_dio_for_tx
 - radio_lora.c, [415](#)
- radio_context_t, [92](#)
 - bus, [93](#)
 - callback, [93](#)
 - callback_ctx, [93](#)
 - dio_pending_mask, [93](#)
 - event_flags, [93](#)
 - hw, [93](#)
 - initialized, [93](#)
 - last_packet, [93](#)
 - lora, [93](#)
 - rx_single_timeout_symbols, [93](#)
 - state, [94](#)
 - tx_resume_state, [94](#)
- radio_default_fsk_cfg
 - radio_lib.c, [463](#)
 - radio_lib.h, [479](#)
- radio_default_hw_cfg
 - radio_lib.c, [463](#)
 - radio_lib.h, [479](#)
- radio_default_lora_cfg
 - radio_lib.c, [463](#)
 - radio_lib.h, [479](#)
- radio_default_ook_cfg
 - radio_lib.c, [464](#)
 - radio_lib.h, [480](#)
- RADIO_DEFAULT_RX_TIMEOUT_SYMBOLS
 - radio_lora.c, [414](#)
- radio_deinit
 - radio_lib.c, [464](#)
 - radio_lib.h, [480](#)
- RADIO_EBUS
 - radio_lib.h, [479](#)
- RADIO_EHW
 - radio_lib.h, [479](#)
- RADIO_EINVAL
 - radio_lib.h, [479](#)
- RADIO_ESTATE
 - radio_lib.h, [479](#)
- RADIO_ETIMEOUT
 - radio_lib.h, [479](#)
- RADIO_EVENT_CAD_DETECTED
 - radio_lib.h, [477](#)
- RADIO_EVENT_CAD_DONE
 - radio_lib.h, [477](#)
- radio_event_cb_t
 - radio_lib.h, [477](#)
- RADIO_EVENT_CRC_ERR
 - radio_lib.h, [477](#)
- RADIO_EVENT_FIFO_OVERRUN
 - radio_lib.h, [477](#)
- RADIO_EVENT_HW_ERROR
 - radio_lib.h, [478](#)
- radio_event_mask_t
 - radio_lib.h, [477](#)
- RADIO_EVENT_NONE
 - radio_lib.h, [477](#)
- RADIO_EVENT_RX_DONE
 - radio_lib.h, [477](#)
- RADIO_EVENT_RX_TIMEOUT
 - radio_lib.h, [477](#)
- RADIO_EVENT_TX_DONE
 - radio_lib.h, [477](#)
- radio_fsk.c
 - radio_apply_fsk_config, [391](#)
 - radio_cfg_valid, [391](#)
 - RADIO_FSK_BROADCAST_ADDRESS, [390](#)
 - radio_fsk_default_hw_cfg, [391](#)
 - radio_fsk_default_lora_cfg, [391](#)
 - radio_fsk_deinit, [391](#)
 - RADIO_FSK_FIFO_MAX_BYTES, [390](#)
 - RADIO_FSK_FIFO_THRESH_LEVEL, [390](#)
 - radio_fsk_get_last_packet, [391](#)
 - radio_fsk_get_state, [391](#)
 - radio_fsk_init, [392](#)
 - RADIO_FSK_NODE_ADDRESS_DEFAULT, [390](#)
 - radio_fsk_on_exti, [392](#)
 - radio_fsk_process, [392](#)

- radio_fsk_raw_read_burst, 392
- radio_fsk_raw_read_reg, 392
- radio_fsk_raw_write_burst, 393
- radio_fsk_raw_write_reg, 393
- RADIO_FSK_RESET_HIGH_DELAY_MS, 390
- RADIO_FSK_RESET_LOW_DELAY_MS, 390
- radio_fsk_send_async, 393
- radio_fsk_set_runtime_cfg, 393
- radio_fsk_sleep, 393
- radio_fsk_standby, 393
- radio_fsk_start_rx_continuous, 394
- radio_fsk_start_rx_single, 394
- radio_fsk_take_events, 394
- RADIO_FSK_TX_GUARD_MS, 390
- radio_get_frequency_deviation, 394
- radio_get_packet_config_1, 394
- radio_get_rx_bw_reg_value, 394
- radio_get_shaping_bits, 395
- radio_handle_exti_pin, 395
- radio_hw_reset, 395
- radio_irq_restore, 395
- radio_irq_save, 395
- radio_read_rx_packet, 395
- radio_resume_after_tx, 396
- radio_rx_single_timeout_ms, 396
- radio_set_event_flags, 396
- radio_set_op_mode_ready, 396
- radio_set_state, 396
- radio_tx_timeout_ms, 396
- radio_write_sync_word, 396
- s_radio, 397
- s_runtime_cfg, 397
- s_runtime_cfg_ready, 397
- radio_fsk.h
 - radio_fsk_default_hw_cfg, 408
 - radio_fsk_default_lora_cfg, 408
 - radio_fsk_deinit, 408
 - radio_fsk_get_last_packet, 409
 - radio_fsk_get_state, 409
 - radio_fsk_init, 409
 - radio_fsk_on_exti, 409
 - radio_fsk_process, 409
 - radio_fsk_raw_read_burst, 409
 - radio_fsk_raw_read_reg, 410
 - radio_fsk_raw_write_burst, 410
 - radio_fsk_raw_write_reg, 410
 - radio_fsk_send_async, 410
 - radio_fsk_set_runtime_cfg, 410
 - radio_fsk_sleep, 411
 - radio_fsk_standby, 411
 - radio_fsk_start_rx_continuous, 411
 - radio_fsk_start_rx_single, 411
 - radio_fsk_take_events, 411
- RADIO_FSK_BROADCAST_ADDRESS
 - radio_fsk.c, 390
- radio_fsk_cfg_t, 94
 - address_filter, 95
 - bitrate_bps, 95
 - crc_type, 95
 - data_whitening, 95
 - filter, 95
 - frequency_hz, 95
 - preamble_len, 95
 - rx_bandwidth, 95
 - shaping, 95
 - sync_word, 96
 - sync_word_len, 96
 - tx_power_dbm, 96
- radio_fsk_context_t, 96
 - bus, 96
 - callback, 96
 - callback_ctx, 97
 - cfg, 97
 - dio_pending_mask, 97
 - event_flags, 97
 - hw, 97
 - initialized, 97
 - last_packet, 97
 - rx_single_deadline_ms, 97
 - state, 97
 - tx_deadline_ms, 97
 - tx_resume_state, 97
- radio_fsk_default_hw_cfg
 - radio_fsk.c, 391
 - radio_fsk.h, 408
- radio_fsk_default_lora_cfg
 - radio_fsk.c, 391
 - radio_fsk.h, 408
- radio_fsk_deinit
 - radio_fsk.c, 391
 - radio_fsk.h, 408
- RADIO_FSK_FIFO_MAX_BYTES
 - radio_fsk.c, 390
- RADIO_FSK_FIFO_THRESH_LEVEL
 - radio_fsk.c, 390
- radio_fsk_filter_t
 - radio_lib.h, 478
- radio_fsk_get_last_packet
 - radio_fsk.c, 391
 - radio_fsk.h, 409
- radio_fsk_get_state
 - radio_fsk.c, 391
 - radio_fsk.h, 409
- radio_fsk_init
 - radio_fsk.c, 392
 - radio_fsk.h, 409
- RADIO_FSK_NODE_ADDRESS_DEFAULT
 - radio_fsk.c, 390
- radio_fsk_on_exti
 - radio_fsk.c, 392
 - radio_fsk.h, 409
- radio_fsk_process
 - radio_fsk.c, 392
 - radio_fsk.h, 409
- radio_fsk_raw_read_burst
 - radio_fsk.c, 392

- radio_fsk.h, [409](#)
- radio_fsk_raw_read_reg
 - radio_fsk.c, [392](#)
 - radio_fsk.h, [410](#)
- radio_fsk_raw_write_burst
 - radio_fsk.c, [393](#)
 - radio_fsk.h, [410](#)
- radio_fsk_raw_write_reg
 - radio_fsk.c, [393](#)
 - radio_fsk.h, [410](#)
- RADIO_FSK_RESET_HIGH_DELAY_MS
 - radio_fsk.c, [390](#)
- RADIO_FSK_RESET_LOW_DELAY_MS
 - radio_fsk.c, [390](#)
- radio_fsk_send_async
 - radio_fsk.c, [393](#)
 - radio_fsk.h, [410](#)
- radio_fsk_set_runtime_cfg
 - radio_fsk.c, [393](#)
 - radio_fsk.h, [410](#)
- RADIO_FSK_SHAPING_FSK
 - radio_lib.h, [478](#)
- RADIO_FSK_SHAPING_GFSK
 - radio_lib.h, [478](#)
- RADIO_FSK_SHAPING_GMSK
 - radio_lib.h, [478](#)
- RADIO_FSK_SHAPING_MSK
 - radio_lib.h, [478](#)
- radio_fsk_shaping_t
 - radio_lib.h, [478](#)
- radio_fsk_sleep
 - radio_fsk.c, [393](#)
 - radio_fsk.h, [411](#)
- radio_fsk_standby
 - radio_fsk.c, [393](#)
 - radio_fsk.h, [411](#)
- radio_fsk_start_rx_continuous
 - radio_fsk.c, [394](#)
 - radio_fsk.h, [411](#)
- radio_fsk_start_rx_single
 - radio_fsk.c, [394](#)
 - radio_fsk.h, [411](#)
- radio_fsk_take_events
 - radio_fsk.c, [394](#)
 - radio_fsk.h, [411](#)
- RADIO_FSK_TX_GUARD_MS
 - radio_fsk.c, [390](#)
- radio_get_backend
 - radio_lib.c, [464](#)
 - radio_lib.h, [480](#)
- radio_get_frequency_deviation
 - radio_fsk.c, [394](#)
- radio_get_last_packet
 - radio_lib.c, [464](#)
 - radio_lib.h, [480](#)
- radio_get_packet_config_1
 - radio_fsk.c, [394](#)
- radio_get_rx_bw_reg_value
 - radio_fsk.c, [394](#)
 - radio_ook.c, [442](#)
- radio_get_shaping_bits
 - radio_fsk.c, [395](#)
- radio_get_state
 - radio_lib.c, [465](#)
 - radio_lib.h, [481](#)
- radio_gpio_t, [98](#)
 - pin, [98](#)
 - port, [98](#)
- radio_handle_exti_pin
 - radio_fsk.c, [395](#)
 - radio_lora.c, [415](#)
 - radio_ook.c, [442](#)
- radio_hw_cfg_t, [98](#)
 - dio, [99](#)
 - hspi, [99](#)
 - nss, [99](#)
 - reset, [99](#)
 - spi_timeout_ms, [99](#)
- radio_hw_reset
 - radio_fsk.c, [395](#)
 - radio_lora.c, [415](#)
 - radio_ook.c, [442](#)
- radio_init
 - radio_lib.c, [465](#)
 - radio_lib.h, [481](#)
- radio_irq_restore
 - radio_fsk.c, [395](#)
 - radio_lora.c, [415](#)
 - radio_ook.c, [442](#)
- radio_irq_save
 - radio_fsk.c, [395](#)
 - radio_lora.c, [415](#)
 - radio_ook.c, [442](#)
- radio_lib.c
 - HAL_GPIO_EXTI_Callback, [463](#)
 - radio_default_fsk_cfg, [463](#)
 - radio_default_hw_cfg, [463](#)
 - radio_default_lora_cfg, [463](#)
 - radio_default_ook_cfg, [464](#)
 - radio_deinit, [464](#)
 - radio_get_backend, [464](#)
 - radio_get_last_packet, [464](#)
 - radio_get_state, [465](#)
 - radio_init, [465](#)
 - radio_normalize_backend, [465](#)
 - radio_on_exti, [465](#)
 - radio_process, [466](#)
 - radio_raw_read_burst, [466](#)
 - radio_raw_read_reg, [466](#)
 - radio_raw_write_burst, [467](#)
 - radio_raw_write_reg, [467](#)
 - radio_select_backend, [467](#)
 - radio_send_async, [468](#)
 - radio_set_fsk_cfg, [468](#)
 - radio_set_ook_cfg, [468](#)
 - radio_sleep, [469](#)

- radio_standby, [469](#)
- radio_start_rx_continuous, [469](#)
- radio_start_rx_single, [469](#)
- radio_take_events, [470](#)
- s_active_backend, [470](#)
- radio_lib.h
 - radio_address_filter_t, [477](#)
 - radio_default_fsk_cfg, [479](#)
 - radio_default_hw_cfg, [479](#)
 - radio_default_lora_cfg, [479](#)
 - radio_default_ook_cfg, [480](#)
 - radio_deinit, [480](#)
 - RADIO_EBUS, [479](#)
 - RADIO_EHW, [479](#)
 - RADIO_EINVAL, [479](#)
 - RADIO_ESTATE, [479](#)
 - RADIO_ETIMEOUT, [479](#)
 - RADIO_EVENT_CAD_DETECTED, [477](#)
 - RADIO_EVENT_CAD_DONE, [477](#)
 - radio_event_cb_t, [477](#)
 - RADIO_EVENT_CRC_ERR, [477](#)
 - RADIO_EVENT_FIFO_OVERRUN, [477](#)
 - RADIO_EVENT_HW_ERROR, [478](#)
 - radio_event_mask_t, [477](#)
 - RADIO_EVENT_NONE, [477](#)
 - RADIO_EVENT_RX_DONE, [477](#)
 - RADIO_EVENT_RX_TIMEOUT, [477](#)
 - RADIO_EVENT_TX_DONE, [477](#)
 - radio_fsk_filter_t, [478](#)
 - RADIO_FSK_SHAPING_FSK, [478](#)
 - RADIO_FSK_SHAPING_GFSK, [478](#)
 - RADIO_FSK_SHAPING_GMSK, [478](#)
 - RADIO_FSK_SHAPING_MSK, [478](#)
 - radio_fsk_shaping_t, [478](#)
 - radio_get_backend, [480](#)
 - radio_get_last_packet, [480](#)
 - radio_get_state, [481](#)
 - radio_init, [481](#)
 - radio_lora_bw_t, [478](#)
 - RADIO_OK, [479](#)
 - radio_on_exti, [481](#)
 - radio_ook_threshold_t, [478](#)
 - radio_packet_crc_t, [478](#)
 - radio_process, [482](#)
 - radio_raw_read_burst, [482](#)
 - radio_raw_read_reg, [482](#)
 - radio_raw_write_burst, [483](#)
 - radio_raw_write_reg, [483](#)
 - radio_select_backend, [483](#)
 - radio_send_async, [484](#)
 - radio_set_fsk_cfg, [484](#)
 - radio_set_ook_cfg, [484](#)
 - radio_sleep, [485](#)
 - radio_standby, [485](#)
 - radio_start_rx_continuous, [485](#)
 - radio_start_rx_single, [485](#)
 - RADIO_STATE_RX_CONT, [478](#)
 - RADIO_STATE_RX_SINGLE, [478](#)
 - RADIO_STATE_STANDBY, [478](#)
 - radio_state_t, [478](#)
 - RADIO_STATE_TX, [478](#)
 - RADIO_STATE_UNINIT, [478](#)
 - radio_status_t, [479](#)
 - radio_take_events, [486](#)
- RADIO_LIB_ACTIVE_MODULATION
 - radio_lib_config.h, [489](#)
- radio_lib_config.h
 - RADIO_LIB_ACTIVE_MODULATION, [489](#)
 - RADIO_LIB_DEFAULT_FREQ_HZ, [489](#)
 - RADIO_LIB_DEFAULT_SYNC_WORD, [490](#)
 - RADIO_LIB_ENABLE_DIO2_EXTI, [490](#)
 - RADIO_LIB_ENABLE_DIO3_EXTI, [490](#)
 - RADIO_LIB_ENABLE_DIO4_EXTI, [490](#)
 - RADIO_LIB_ENABLE_DIO5_EXTI, [490](#)
 - RADIO_LIB_MAX_PAYLOAD, [490](#)
 - RADIO_LIB_MODULATION_FSK, [490](#)
 - RADIO_LIB_MODULATION_LORA, [490](#)
 - RADIO_LIB_MODULATION_OOK, [491](#)
 - RADIO_LIB_OWNS_HAL_EXTI_CALLBACK, [491](#)
 - RADIO_LIB_SPI_TIMEOUT_MS, [491](#)
- RADIO_LIB_DEFAULT_FREQ_HZ
 - radio_lib_config.h, [489](#)
- RADIO_LIB_DEFAULT_SYNC_WORD
 - radio_lib_config.h, [490](#)
- RADIO_LIB_ENABLE_DIO2_EXTI
 - radio_lib_config.h, [490](#)
- RADIO_LIB_ENABLE_DIO3_EXTI
 - radio_lib_config.h, [490](#)
- RADIO_LIB_ENABLE_DIO4_EXTI
 - radio_lib_config.h, [490](#)
- RADIO_LIB_ENABLE_DIO5_EXTI
 - radio_lib_config.h, [490](#)
- RADIO_LIB_MAX_PAYLOAD
 - radio_lib_config.h, [490](#)
- RADIO_LIB_MODULATION_FSK
 - radio_lib_config.h, [490](#)
- RADIO_LIB_MODULATION_LORA
 - radio_lib_config.h, [490](#)
- RADIO_LIB_MODULATION_OOK
 - radio_lib_config.h, [491](#)
- RADIO_LIB_OWNS_HAL_EXTI_CALLBACK
 - radio_lib_config.h, [491](#)
- RADIO_LIB_SPI_TIMEOUT_MS
 - radio_lib_config.h, [491](#)
- radio_lora.c
 - radio_apply_lora_config, [414](#)
 - radio_cfg_valid, [415](#)
 - radio_config_dio_for_rx, [415](#)
 - radio_config_dio_for_tx, [415](#)
 - RADIO_DEFAULT_RX_TIMEOUT_SYMBOLS, [414](#)
 - radio_handle_exti_pin, [415](#)
 - radio_hw_reset, [415](#)
 - radio_irq_restore, [415](#)
 - radio_irq_save, [415](#)
 - radio_lora_default_hw_cfg, [416](#)

- radio_lora_default_lora_cfg, 416
- radio_lora_deinit, 416
- radio_lora_get_last_packet, 416
- radio_lora_get_state, 417
- radio_lora_init, 417
- radio_lora_on_exti, 417
- radio_lora_process, 418
- radio_lora_raw_read_burst, 418
- radio_lora_raw_read_reg, 418
- radio_lora_raw_write_burst, 419
- radio_lora_raw_write_reg, 419
- radio_lora_send_async, 419
- radio_lora_sleep, 420
- radio_lora_standby, 420
- radio_lora_start_rx_continuous, 420
- radio_lora_start_rx_single, 421
- radio_lora_take_events, 421
- radio_low_data_rate_optimize_required, 421
- radio_read_irq_flags_retry, 421
- radio_read_rx_packet, 421
- RADIO_RESET_HIGH_DELAY_MS, 414
- RADIO_RESET_LOW_DELAY_MS, 414
- radio_resume_after_tx, 422
- radio_set_event_flags, 422
- radio_set_state, 422
- RADIO_SX1276_FIFO_RX_BASE_ADDR, 414
- RADIO_SX1276_FIFO_TX_BASE_ADDR, 414
- radio_wait_for_tx_enter_or_done, 422
- s_radio, 422
- radio_lora.h
 - radio_lora_default_hw_cfg, 433
 - radio_lora_default_lora_cfg, 433
 - radio_lora_deinit, 433
 - radio_lora_get_last_packet, 433
 - radio_lora_get_state, 434
 - radio_lora_init, 434
 - radio_lora_on_exti, 434
 - radio_lora_process, 435
 - radio_lora_raw_read_burst, 435
 - radio_lora_raw_read_reg, 435
 - radio_lora_raw_write_burst, 436
 - radio_lora_raw_write_reg, 436
 - radio_lora_send_async, 436
 - radio_lora_sleep, 437
 - radio_lora_standby, 437
 - radio_lora_start_rx_continuous, 437
 - radio_lora_start_rx_single, 437
 - radio_lora_take_events, 438
- radio_lora_bw_t
 - radio_lib.h, 478
- radio_lora_cfg_t, 99
 - bandwidth, 100
 - coding_rate, 100
 - crc_on, 100
 - frequency_hz, 100
 - implicit_header, 100
 - invert_iq, 100
 - payload_len, 101
 - preamble_len, 101
 - spreading_factor, 101
 - sync_word, 101
 - tx_power_dbm, 101
- radio_lora_default_hw_cfg
 - radio_lora.c, 416
 - radio_lora.h, 433
- radio_lora_default_lora_cfg
 - radio_lora.c, 416
 - radio_lora.h, 433
- radio_lora_deinit
 - radio_lora.c, 416
 - radio_lora.h, 433
- radio_lora_get_last_packet
 - radio_lora.c, 416
 - radio_lora.h, 433
- radio_lora_get_state
 - radio_lora.c, 417
 - radio_lora.h, 434
- radio_lora_init
 - radio_lora.c, 417
 - radio_lora.h, 434
- radio_lora_on_exti
 - radio_lora.c, 417
 - radio_lora.h, 434
- radio_lora_process
 - radio_lora.c, 418
 - radio_lora.h, 435
- radio_lora_raw_read_burst
 - radio_lora.c, 418
 - radio_lora.h, 435
- radio_lora_raw_read_reg
 - radio_lora.c, 418
 - radio_lora.h, 435
- radio_lora_raw_write_burst
 - radio_lora.c, 419
 - radio_lora.h, 436
- radio_lora_raw_write_reg
 - radio_lora.c, 419
 - radio_lora.h, 436
- radio_lora_send_async
 - radio_lora.c, 419
 - radio_lora.h, 436
- radio_lora_sleep
 - radio_lora.c, 420
 - radio_lora.h, 437
- radio_lora_standby
 - radio_lora.c, 420
 - radio_lora.h, 437
- radio_lora_start_rx_continuous
 - radio_lora.c, 420
 - radio_lora.h, 437
- radio_lora_start_rx_single
 - radio_lora.c, 421
 - radio_lora.h, 437
- radio_lora_take_events
 - radio_lora.c, 421
 - radio_lora.h, 438

radio_low_data_rate_optimize_required
 radio_lora.c, [421](#)
radio_main.h
 RADIO_MAIN_ADDRESS_FILTER_NODE, [562](#)
 RADIO_MAIN_ADDRESS_FILTER_NODE_BROADCAST, [562](#)
 RADIO_MAIN_ADDRESS_FILTER_NONE, [562](#)
 radio_main_address_filter_t, [562](#)
 RADIO_MAIN_AUTO_PING_FRAME, [563](#)
 radio_main_auto_ping_mode_t, [562](#)
 RADIO_MAIN_AUTO_PING_RAW, [563](#)
 radio_main_cmd_pairing_accept, [565](#)
 radio_main_cmd_reset_module, [566](#)
 radio_main_cmd_send_network_pair_req, [566](#)
 radio_main_cmd_send_pair_error, [566](#)
 radio_main_cmd_send_pair_req, [566](#)
 radio_main_cmd_send_raw, [566](#)
 radio_main_cmd_send_template, [567](#)
 radio_main_cmd_send_user_text, [567](#)
 radio_main_cmd_set_auto_ping, [568](#)
 radio_main_cmd_set_auto_ping_mode, [568](#)
 radio_main_cmd_set_auto_ping_period, [568](#)
 radio_main_cmd_set_coding, [568](#)
 radio_main_cmd_set_fh, [569](#)
 radio_main_cmd_set_fh_period, [569](#)
 radio_main_cmd_set_lora_preset, [569](#)
 radio_main_cmd_set_modulation, [570](#)
 radio_main_cmd_set_modulation_bw, [570](#)
 radio_main_cmd_set_modulation_freq, [570](#)
 radio_main_cmd_set_option, [570](#)
 radio_main_cmd_start_network_pairing, [571](#)
 radio_main_cmd_start_pairing, [571](#)
 RADIO_MAIN_CRC_CCITT, [563](#)
 RADIO_MAIN_CRC_IBM, [563](#)
 RADIO_MAIN_CRC_OFF, [563](#)
 RADIO_MAIN_CRC_SX1276, [563](#)
 radio_main_crc_type_t, [563](#)
 radio_main_create_task, [571](#)
 RADIO_MAIN_FILTER_BT_03, [563](#)
 RADIO_MAIN_FILTER_BT_05, [563](#)
 RADIO_MAIN_FILTER_BT_07, [563](#)
 RADIO_MAIN_FILTER_BT_10, [563](#)
 RADIO_MAIN_FILTER_NONE, [563](#)
 radio_main_filter_t, [563](#)
 RADIO_MAIN_FSK_SHAPING_FSK, [564](#)
 RADIO_MAIN_FSK_SHAPING_GFSK, [564](#)
 RADIO_MAIN_FSK_SHAPING_GMSK, [564](#)
 RADIO_MAIN_FSK_SHAPING_MSK, [564](#)
 radio_main_fsk_shaping_t, [563](#)
 radio_main_get_auto_ping_period_ms, [571](#)
 radio_main_get_last_error_text, [572](#)
 radio_main_get_node_id, [572](#)
 radio_main_get_runtime_cfg, [572](#)
 RADIO_MAIN_HEADER_EXPLICIT, [564](#)
 RADIO_MAIN_HEADER_IMPLICIT, [564](#)
 radio_main_header_mode_t, [564](#)
 radio_main_load_default_fsk_profile, [573](#)
 radio_main_load_default_lora_preset, [573](#)
 radio_main_load_default_ook_profile, [573](#)
 RADIO_MAIN_MODULATION_FSK, [564](#)
 RADIO_MAIN_MODULATION_LORA, [564](#)
 RADIO_MAIN_MODULATION_OOK, [564](#)
 radio_main_modulation_t, [564](#)
 RADIO_MAIN_OOK_THRESHOLD_AVERAGE, [564](#)
 RADIO_MAIN_OOK_THRESHOLD_FIXED, [564](#)
 RADIO_MAIN_OOK_THRESHOLD_PEAK, [564](#)
 radio_main_ook_threshold_t, [564](#)
 RADIO_MAIN_OPTION_FSK_ADDRESS_FILTER, [565](#)
 RADIO_MAIN_OPTION_FSK_BITRATE, [565](#)
 RADIO_MAIN_OPTION_FSK_CRC, [565](#)
 RADIO_MAIN_OPTION_FSK_FILTER, [565](#)
 RADIO_MAIN_OPTION_FSK_FREQ, [565](#)
 RADIO_MAIN_OPTION_FSK_PREAMBLE, [565](#)
 RADIO_MAIN_OPTION_FSK_RESET_DEFAULTS, [565](#)
 RADIO_MAIN_OPTION_FSK_RX_BW, [565](#)
 RADIO_MAIN_OPTION_FSK_SHAPING, [565](#)
 RADIO_MAIN_OPTION_FSK_SYNC_LEN, [565](#)
 RADIO_MAIN_OPTION_FSK_SYNC_WORD, [565](#)
 RADIO_MAIN_OPTION_FSK_TX_POWER, [565](#)
 RADIO_MAIN_OPTION_FSK_WHITENING, [565](#)
 RADIO_MAIN_OPTION_LORA_BW, [565](#)
 RADIO_MAIN_OPTION_LORA_CR, [565](#)
 RADIO_MAIN_OPTION_LORA_CRC, [565](#)
 RADIO_MAIN_OPTION_LORA_FREQ, [565](#)
 RADIO_MAIN_OPTION_LORA_HEADER_MODE, [565](#)
 RADIO_MAIN_OPTION_LORA_IQ_INVERT, [565](#)
 RADIO_MAIN_OPTION_LORA_PREAMBLE, [565](#)
 RADIO_MAIN_OPTION_LORA_PRESET, [565](#)
 RADIO_MAIN_OPTION_LORA_RESET_DEFAULTS, [565](#)
 RADIO_MAIN_OPTION_LORA_SF, [565](#)
 RADIO_MAIN_OPTION_LORA_SYNC_WORD, [565](#)
 RADIO_MAIN_OPTION_LORA_TX_POWER, [565](#)
 RADIO_MAIN_OPTION_OOK_BITRATE, [565](#)
 RADIO_MAIN_OPTION_OOK_FREQ, [565](#)
 RADIO_MAIN_OPTION_OOK_PREAMBLE, [565](#)
 RADIO_MAIN_OPTION_OOK_RESET_DEFAULTS, [565](#)
 RADIO_MAIN_OPTION_OOK_RX_BW, [565](#)
 RADIO_MAIN_OPTION_OOK_SYNC_LEN, [565](#)
 RADIO_MAIN_OPTION_OOK_SYNC_WORD, [565](#)
 RADIO_MAIN_OPTION_OOK_THRESHOLD_TYPE, [565](#)
 RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE, [565](#)
 RADIO_MAIN_OPTION_OOK_TX_POWER, [565](#)
 radio_main_option_t, [564](#)
RADIO_MAIN_ADDRESS_FILTER_NODE
 radio_main.h, [562](#)
RADIO_MAIN_ADDRESS_FILTER_NODE_BROADCAST

radio_main.h, [562](#)
 RADIO_MAIN_ADDRESS_FILTER_NONE
 radio_main.h, [562](#)
 radio_main_address_filter_t
 radio_main.h, [562](#)
 RADIO_MAIN_AUTO_PING_FRAME
 radio_main.h, [563](#)
 radio_main_auto_ping_mode_t
 radio_main.h, [562](#)
 RADIO_MAIN_AUTO_PING_RAW
 radio_main.h, [563](#)
 radio_main_cmd_pairing_accept
 radio_main.h, [565](#)
 radio_main_cmd_reset_module
 radio_main.h, [566](#)
 radio_main_cmd_send_network_pair_req
 radio_main.h, [566](#)
 radio_main_cmd_send_pair_error
 radio_main.h, [566](#)
 radio_main_cmd_send_pair_req
 radio_main.h, [566](#)
 radio_main_cmd_send_raw
 radio_main.h, [566](#)
 radio_main_cmd_send_template
 radio_main.h, [567](#)
 radio_main_cmd_send_user_text
 radio_main.h, [567](#)
 radio_main_cmd_set_auto_ping
 radio_main.h, [568](#)
 radio_main_cmd_set_auto_ping_mode
 radio_main.h, [568](#)
 radio_main_cmd_set_auto_ping_period
 radio_main.h, [568](#)
 radio_main_cmd_set_coding
 radio_main.h, [568](#)
 radio_main_cmd_set_fh
 radio_main.h, [569](#)
 radio_main_cmd_set_fh_period
 radio_main.h, [569](#)
 radio_main_cmd_set_lora_preset
 radio_main.h, [569](#)
 radio_main_cmd_set_modulation
 radio_main.h, [570](#)
 radio_main_cmd_set_modulation_bw
 radio_main.h, [570](#)
 radio_main_cmd_set_modulation_freq
 radio_main.h, [570](#)
 radio_main_cmd_set_option
 radio_main.h, [570](#)
 radio_main_cmd_start_network_pairing
 radio_main.h, [571](#)
 radio_main_cmd_start_pairing
 radio_main.h, [571](#)
 radio_main_cmd_sync_t, [101](#)
 done, [102](#)
 result, [102](#)
 runtime_cfg, [102](#)
 radio_main_cmd_t, [102](#)
 data, [103](#)
 dst_id, [103](#)
 enabled, [103](#)
 group_id, [103](#)
 id, [103](#)
 len, [103](#)
 msg_id, [103](#)
 network_mode, [103](#)
 option, [103](#)
 sync, [103](#)
 text, [103](#)
 timeout_ms, [104](#)
 value, [104](#)
 RADIO_MAIN_CRC_CCITT
 radio_main.h, [563](#)
 RADIO_MAIN_CRC_IBM
 radio_main.h, [563](#)
 RADIO_MAIN_CRC_OFF
 radio_main.h, [563](#)
 RADIO_MAIN_CRC_SX1276
 radio_main.h, [563](#)
 radio_main_crc_type_t
 radio_main.h, [563](#)
 radio_main_create_task
 radio_main.h, [571](#)
 radio_main_ctx_t, [104](#)
 active, [105](#)
 auto_ping_enabled, [105](#)
 auto_ping_mode, [106](#)
 auto_ping_period_ms, [106](#)
 backend_cfg, [106](#)
 backend_modulation_id, [106](#)
 broadcast_group_aes_key, [106](#)
 broadcast_group_epoch, [106](#)
 broadcast_group_hmac_key, [106](#)
 broadcast_group_key_valid, [106](#)
 broadcast_group_pending_epoch, [106](#)
 broadcast_group_pending_key, [106](#)
 broadcast_group_pending_mask, [106](#)
 broadcast_group_pending_valid, [106](#)
 coding_enabled, [107](#)
 counter, [107](#)
 crypto_ready, [107](#)
 deadline_ms, [107](#)
 expires_ms, [107](#)
 fh_enabled, [107](#)
 fh_period_ms, [107](#)
 fsk_cfg, [107](#)
 gateway_key_mode, [107](#)
 gateway_key_mode_known, [107](#)
 gateway_pair_code, [107](#)
 gateway_pair_code_len, [107](#)
 gateway_pair_code_valid, [108](#)
 gateway_rx_counter, [108](#)
 gateway_tx_counter, [108](#)
 hop_idx, [108](#)
 hw, [108](#)
 initialized, [108](#)

last_error_text, 108
last_hop_ms, 108
last_ping_ms, 108
lora_cfg, 108
lora_preset, 108
modulation_id, 108
msg_id, 109
next_msg_id, 109
node_id, 109
ook_cfg, 109
pairing_active, 109
pairing_network_mode, 109
pairing_outgoing_code, 109
pairing_outgoing_code_len, 109
pairing_outgoing_network, 109
pairing_outgoing_pending, 109
pairing_pending, 109
pairing_pending_code, 109
pairing_pending_code_len, 110
pairing_pending_network, 110
pairing_pending_node, 110
pairing_until_ms, 110
peer_id, 110
raw, 110
raw_len, 110
retries_done, 110
timed_out, 110
tx_deadline_ms, 110
tx_in_progress, 110
tx_silent, 110
valid, 111
wait_ms, 111
RADIO_MAIN_FILTER_BT_03
radio_main.h, 563
RADIO_MAIN_FILTER_BT_05
radio_main.h, 563
RADIO_MAIN_FILTER_BT_07
radio_main.h, 563
RADIO_MAIN_FILTER_BT_10
radio_main.h, 563
RADIO_MAIN_FILTER_NONE
radio_main.h, 563
radio_main_filter_t
radio_main.h, 563
radio_main_fsk_cfg_t, 111
address_filter, 112
bitrate_bps, 112
crc_type, 112
data_whitening, 112
filter, 112
frequency_hz, 112
preamble_len, 112
rx_bandwidth, 112
shaping, 112
sync_word, 113
sync_word_len, 113
tx_power_dbm, 113
RADIO_MAIN_FSK_SHAPING_FSK
radio_main.h, 564
RADIO_MAIN_FSK_SHAPING_GFSK
radio_main.h, 564
RADIO_MAIN_FSK_SHAPING_GMSK
radio_main.h, 564
RADIO_MAIN_FSK_SHAPING_MSK
radio_main.h, 564
radio_main_fsk_shaping_t
radio_main.h, 563
radio_main_get_auto_ping_period_ms
radio_main.h, 571
radio_main_get_last_error_text
radio_main.h, 572
radio_main_get_node_id
radio_main.h, 572
radio_main_get_runtime_cfg
radio_main.h, 572
RADIO_MAIN_HEADER_EXPLICIT
radio_main.h, 564
RADIO_MAIN_HEADER_IMPLICIT
radio_main.h, 564
radio_main_header_mode_t
radio_main.h, 564
radio_main_load_default_fsk_profile
radio_main.h, 573
radio_main_load_default_lora_preset
radio_main.h, 573
radio_main_load_default_ook_profile
radio_main.h, 573
RADIO_MAIN_MODULATION_FSK
radio_main.h, 564
RADIO_MAIN_MODULATION_LORA
radio_main.h, 564
RADIO_MAIN_MODULATION_OOK
radio_main.h, 564
radio_main_modulation_t
radio_main.h, 564
radio_main_ook_cfg_t, 113
bitrate_bps, 114
frequency_hz, 114
preamble_len, 114
rx_bandwidth, 114
sync_word, 114
sync_word_len, 114
threshold, 114
threshold_value, 114
tx_power_dbm, 115
RADIO_MAIN_OOK_THRESHOLD_AVERAGE
radio_main.h, 564
RADIO_MAIN_OOK_THRESHOLD_FIXED
radio_main.h, 564
RADIO_MAIN_OOK_THRESHOLD_PEAK
radio_main.h, 564
radio_main_ook_threshold_t
radio_main.h, 564
RADIO_MAIN_OPTION_FSK_ADDRESS_FILTER
radio_main.h, 565
RADIO_MAIN_OPTION_FSK_BITRATE

[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_FSK_CRC
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_FSK_FILTER
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_FSK_FREQ
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_FSK_PREAMBLE
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_FSK_RESET_DEFAULTS
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_FSK_RX_BW
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_FSK_SHAPING
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_FSK_SYNC_LEN
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_FSK_SYNC_WORD
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_FSK_TX_POWER
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_FSK_WHITENING
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_LORA_BW
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_LORA_CR
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_LORA_CRC
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_LORA_FREQ
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_LORA_HEADER_MODE
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_LORA_IQ_INVERT
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_LORA_PREAMBLE
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_LORA_PRESET
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_LORA_RESET_DEFAULTS
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_LORA_SF
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_LORA_SYNC_WORD
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_LORA_TX_POWER
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_OOK_BITRATE
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_OOK_FREQ
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_OOK_PREAMBLE
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_OOK_RESET_DEFAULTS
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_OOK_RX_BW
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_OOK_SYNC_LEN
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_OOK_SYNC_WORD
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_OOK_THRESHOLD_TYPE
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_OOK_THRESHOLD_VALUE
[radio_main.h](#), [565](#)
 RADIO_MAIN_OPTION_OOK_TX_POWER
[radio_main.h](#), [565](#)
[radio_main_option_t](#)
[radio_main.h](#), [564](#)
[radio_main_runtime_cfg_t](#), [115](#)
 [active_modulation](#), [115](#)
 [auto_ping_enabled](#), [115](#)
 [auto_ping_mode](#), [116](#)
 [auto_ping_period_ms](#), [116](#)
 [coding_enabled](#), [116](#)
 [fh_enabled](#), [116](#)
 [fh_period_ms](#), [116](#)
 [fsk](#), [116](#)
 [lora](#), [116](#)
 [ook](#), [116](#)
[radio_normalize_backend](#)
[radio_lib.c](#), [465](#)
 RADIO_OK
[radio_lib.h](#), [479](#)
[radio_on_exti](#)
[radio_lib.c](#), [465](#)
[radio_lib.h](#), [481](#)
[radio_ook.c](#)
 [radio_apply_ook_config](#), [441](#)
 [radio_apply_threshold](#), [441](#)
 [radio_cfg_valid](#), [442](#)
 [radio_get_rx_bw_reg_value](#), [442](#)
 [radio_handle_exti_pin](#), [442](#)
 [radio_hw_reset](#), [442](#)
 [radio_irq_restore](#), [442](#)
 [radio_irq_save](#), [442](#)
 [radio_ook_default_hw_cfg](#), [443](#)
 [radio_ook_default_lora_cfg](#), [443](#)
 [radio_ook_deinit](#), [443](#)
 RADIO_OOK_FIFO_MAX_BYTES, [441](#)
 RADIO_OOK_FIFO_THRESH_LEVEL, [441](#)
 [radio_ook_get_last_packet](#), [443](#)
 [radio_ook_get_state](#), [443](#)
 [radio_ook_init](#), [443](#)
 [radio_ook_on_exti](#), [443](#)
 [radio_ook_process](#), [444](#)
 [radio_ook_raw_read_burst](#), [444](#)
 [radio_ook_raw_read_reg](#), [444](#)
 [radio_ook_raw_write_burst](#), [444](#)
 [radio_ook_raw_write_reg](#), [444](#)
 RADIO_OOK_RESET_HIGH_DELAY_MS, [441](#)
 RADIO_OOK_RESET_LOW_DELAY_MS, [441](#)
 [radio_ook_send_async](#), [445](#)
 [radio_ook_set_runtime_cfg](#), [445](#)
 [radio_ook_sleep](#), [445](#)
 [radio_ook_standby](#), [445](#)

- radio_ook_start_rx_continuous, 445
- radio_ook_start_rx_single, 445
- radio_ook_take_events, 446
- RADIO_OOK_TX_GUARD_MS, 441
- radio_read_rx_packet, 446
- radio_resume_after_tx, 446
- radio_rx_single_timeout_ms, 446
- radio_set_event_flags, 446
- radio_set_op_mode_ready, 446
- radio_set_state, 446
- radio_tx_timeout_ms, 447
- radio_write_sync_word, 447
- s_radio, 447
- s_runtime_cfg, 447
- s_runtime_cfg_ready, 447
- radio_ook.h
 - radio_ook_default_hw_cfg, 458
 - radio_ook_default_lora_cfg, 458
 - radio_ook_deinit, 458
 - radio_ook_get_last_packet, 458
 - radio_ook_get_state, 458
 - radio_ook_init, 458
 - radio_ook_on_exti, 458
 - radio_ook_process, 459
 - radio_ook_raw_read_burst, 459
 - radio_ook_raw_read_reg, 459
 - radio_ook_raw_write_burst, 459
 - radio_ook_raw_write_reg, 459
 - radio_ook_send_async, 460
 - radio_ook_set_runtime_cfg, 460
 - radio_ook_sleep, 460
 - radio_ook_standby, 460
 - radio_ook_start_rx_continuous, 460
 - radio_ook_start_rx_single, 460
 - radio_ook_take_events, 461
- radio_ook_cfg_t, 117
 - bitrate_bps, 117
 - frequency_hz, 117
 - preamble_len, 117
 - rx_bandwidth, 117
 - sync_word, 118
 - sync_word_len, 118
 - threshold, 118
 - threshold_value, 118
 - tx_power_dbm, 118
- radio_ook_context_t, 118
 - bus, 119
 - callback, 119
 - callback_ctx, 119
 - cfg, 119
 - dio_pending_mask, 119
 - event_flags, 119
 - hw, 119
 - initialized, 119
 - last_packet, 119
 - rx_single_deadline_ms, 119
 - state, 119
 - tx_deadline_ms, 120
 - tx_resume_state, 120
- radio_ook_default_hw_cfg
 - radio_ook.c, 443
 - radio_ook.h, 458
- radio_ook_default_lora_cfg
 - radio_ook.c, 443
 - radio_ook.h, 458
- radio_ook_deinit
 - radio_ook.c, 443
 - radio_ook.h, 458
- RADIO_OOK_FIFO_MAX_BYTES
 - radio_ook.c, 441
- RADIO_OOK_FIFO_THRESH_LEVEL
 - radio_ook.c, 441
- radio_ook_get_last_packet
 - radio_ook.c, 443
 - radio_ook.h, 458
- radio_ook_get_state
 - radio_ook.c, 443
 - radio_ook.h, 458
- radio_ook_init
 - radio_ook.c, 443
 - radio_ook.h, 458
- radio_ook_on_exti
 - radio_ook.c, 443
 - radio_ook.h, 458
- radio_ook_process
 - radio_ook.c, 444
 - radio_ook.h, 459
- radio_ook_raw_read_burst
 - radio_ook.c, 444
 - radio_ook.h, 459
- radio_ook_raw_read_reg
 - radio_ook.c, 444
 - radio_ook.h, 459
- radio_ook_raw_write_burst
 - radio_ook.c, 444
 - radio_ook.h, 459
- radio_ook_raw_write_reg
 - radio_ook.c, 444
 - radio_ook.h, 459
- RADIO_OOK_RESET_HIGH_DELAY_MS
 - radio_ook.c, 441
- RADIO_OOK_RESET_LOW_DELAY_MS
 - radio_ook.c, 441
- radio_ook_send_async
 - radio_ook.c, 445
 - radio_ook.h, 460
- radio_ook_set_runtime_cfg
 - radio_ook.c, 445
 - radio_ook.h, 460
- radio_ook_sleep
 - radio_ook.c, 445
 - radio_ook.h, 460
- radio_ook_standby
 - radio_ook.c, 445
 - radio_ook.h, 460
- radio_ook_start_rx_continuous

- radio_ook.c, [445](#)
- radio_ook.h, [460](#)
- radio_ook_start_rx_single
 - radio_ook.c, [445](#)
 - radio_ook.h, [460](#)
- radio_ook_take_events
 - radio_ook.c, [446](#)
 - radio_ook.h, [461](#)
- radio_ook_threshold_t
 - radio_lib.h, [478](#)
- RADIO_OOK_TX_GUARD_MS
 - radio_ook.c, [441](#)
- radio_packet_crc_t
 - radio_lib.h, [478](#)
- radio_packet_t, [120](#)
 - data, [120](#)
 - length, [120](#)
 - rssi_dbm, [121](#)
 - snr_db, [121](#)
 - timestamp_ms, [121](#)
 - valid, [121](#)
- radio_process
 - radio_lib.c, [466](#)
 - radio_lib.h, [482](#)
- radio_profiles_persisted
 - security_runtime_cfg_t, [127](#)
- radio_raw_read_burst
 - radio_lib.c, [466](#)
 - radio_lib.h, [482](#)
- radio_raw_read_reg
 - radio_lib.c, [466](#)
 - radio_lib.h, [482](#)
- radio_raw_write_burst
 - radio_lib.c, [467](#)
 - radio_lib.h, [483](#)
- radio_raw_write_reg
 - radio_lib.c, [467](#)
 - radio_lib.h, [483](#)
- radio_read_irq_flags_retry
 - radio_lora.c, [421](#)
- radio_read_rx_packet
 - radio_fsk.c, [395](#)
 - radio_lora.c, [421](#)
 - radio_ook.c, [446](#)
- RADIO_RESET_HIGH_DELAY_MS
 - radio_lora.c, [414](#)
- RADIO_RESET_LOW_DELAY_MS
 - radio_lora.c, [414](#)
- radio_resume_after_tx
 - radio_fsk.c, [396](#)
 - radio_lora.c, [422](#)
 - radio_ook.c, [446](#)
- radio_rx_single_timeout_ms
 - radio_fsk.c, [396](#)
 - radio_ook.c, [446](#)
- radio_select_backend
 - radio_lib.c, [467](#)
 - radio_lib.h, [483](#)
- radio_send_async
 - radio_lib.c, [468](#)
 - radio_lib.h, [484](#)
- radio_set_event_flags
 - radio_fsk.c, [396](#)
 - radio_lora.c, [422](#)
 - radio_ook.c, [446](#)
- radio_set_fsk_cfg
 - radio_lib.c, [468](#)
 - radio_lib.h, [484](#)
- radio_set_ook_cfg
 - radio_lib.c, [468](#)
 - radio_lib.h, [484](#)
- radio_set_op_mode_ready
 - radio_fsk.c, [396](#)
 - radio_ook.c, [446](#)
- radio_set_state
 - radio_fsk.c, [396](#)
 - radio_lora.c, [422](#)
 - radio_ook.c, [446](#)
- radio_sleep
 - radio_lib.c, [469](#)
 - radio_lib.h, [485](#)
- radio_standby
 - radio_lib.c, [469](#)
 - radio_lib.h, [485](#)
- radio_start_rx_continuous
 - radio_lib.c, [469](#)
 - radio_lib.h, [485](#)
- radio_start_rx_single
 - radio_lib.c, [469](#)
 - radio_lib.h, [485](#)
- RADIO_STATE_RX_CONT
 - radio_lib.h, [478](#)
- RADIO_STATE_RX_SINGLE
 - radio_lib.h, [478](#)
- RADIO_STATE_STANDBY
 - radio_lib.h, [478](#)
- radio_state_t
 - radio_lib.h, [478](#)
- RADIO_STATE_TX
 - radio_lib.h, [478](#)
- RADIO_STATE_UNINIT
 - radio_lib.h, [478](#)
- radio_status_t
 - radio_lib.h, [479](#)
- radio_sx1276.c
 - sx1276_clear_irq_flags, [353](#)
 - sx1276_deselect, [353](#)
 - sx1276_get_irq_flags, [353](#)
 - sx1276_get_packet_rssi_raw, [353](#)
 - sx1276_get_packet_snr_raw, [354](#)
 - sx1276_get_rx_current_addr, [354](#)
 - sx1276_get_rx_nb_bytes, [354](#)
 - sx1276_get_version, [354](#)
 - sx1276_map_dio, [354](#)
 - sx1276_read_burst, [354](#)
 - sx1276_read_fifo, [355](#)

- sx1276_read_reg, [355](#)
 - sx1276_select, [355](#)
 - sx1276_set_fifo_addr_ptr, [356](#)
 - sx1276_set_fifo_base_addrs, [356](#)
 - sx1276_set_frequency, [356](#)
 - sx1276_set_invert_iq, [356](#)
 - sx1276_set_lora_modem, [356](#)
 - sx1276_set_op_mode, [357](#)
 - sx1276_set_pa_output_power, [357](#)
 - sx1276_set_payload_length, [357](#)
 - sx1276_set_preamble, [357](#)
 - sx1276_set_symbol_timeout, [357](#)
 - sx1276_set_sync_word, [357](#)
 - sx1276_write_burst, [358](#)
 - sx1276_write_fifo, [358](#)
 - sx1276_write_reg, [358](#)
- radio_sx1276.h
 - sx1276_clear_irq_flags, [365](#)
 - sx1276_get_irq_flags, [365](#)
 - sx1276_get_packet_rssi_raw, [365](#)
 - sx1276_get_packet_snr_raw, [365](#)
 - sx1276_get_rx_current_addr, [365](#)
 - sx1276_get_rx_nb_bytes, [365](#)
 - sx1276_get_version, [366](#)
 - sx1276_map_dio, [366](#)
 - sx1276_read_burst, [366](#)
 - sx1276_read_fifo, [366](#)
 - sx1276_read_reg, [367](#)
 - sx1276_set_fifo_addr_ptr, [367](#)
 - sx1276_set_fifo_base_addrs, [367](#)
 - sx1276_set_frequency, [367](#)
 - sx1276_set_invert_iq, [367](#)
 - sx1276_set_lora_modem, [368](#)
 - sx1276_set_op_mode, [368](#)
 - sx1276_set_pa_output_power, [368](#)
 - sx1276_set_payload_length, [368](#)
 - sx1276_set_preamble, [368](#)
 - sx1276_set_symbol_timeout, [369](#)
 - sx1276_set_sync_word, [369](#)
 - sx1276_write_burst, [369](#)
 - sx1276_write_fifo, [369](#)
 - sx1276_write_reg, [370](#)
- RADIO_SX1276_FIFO_RX_BASE_ADDR
 - radio_lora.c, [414](#)
- RADIO_SX1276_FIFO_TX_BASE_ADDR
 - radio_lora.c, [414](#)
- radio_sx1276_regs.h
 - SX1276_DIO0_MAP_CAD_DONE, [374](#)
 - SX1276_DIO0_MAP_FSK_PACKET_SENT, [374](#)
 - SX1276_DIO0_MAP_FSK_PAYLOAD_READY, [374](#)
 - SX1276_DIO0_MAP_RX_DONE, [374](#)
 - SX1276_DIO0_MAP_TX_DONE, [374](#)
 - SX1276_DIO1_MAP_CAD_DETECTED, [374](#)
 - SX1276_DIO1_MAP_FHSS_CHANGE, [374](#)
 - SX1276_DIO1_MAP_RX_TIMEOUT, [374](#)
 - SX1276_FIFO_THRESH_TX_START_NOT_EMPTY, [374](#)
 - SX1276_IRQ1_MODE_READY, [374](#)
 - SX1276_IRQ1_PLL_LOCK, [375](#)
 - SX1276_IRQ1_PREAMBLE_DETECT, [375](#)
 - SX1276_IRQ1_RSSI, [375](#)
 - SX1276_IRQ1_RX_READY, [375](#)
 - SX1276_IRQ1_SYNC_ADDRESS_MATCH, [375](#)
 - SX1276_IRQ1_TIMEOUT, [375](#)
 - SX1276_IRQ1_TX_READY, [375](#)
 - SX1276_IRQ2_CRC_OK, [375](#)
 - SX1276_IRQ2_FIFO_FULL, [375](#)
 - SX1276_IRQ2_FIFO_LEVEL, [375](#)
 - SX1276_IRQ2_FIFO_NOT_EMPTY, [375](#)
 - SX1276_IRQ2_FIFO_OVERRUN, [375](#)
 - SX1276_IRQ2_LOW_BAT, [376](#)
 - SX1276_IRQ2_PACKET_SENT, [376](#)
 - SX1276_IRQ2_PAYLOAD_READY, [376](#)
 - SX1276_IRQ_CAD_DETECTED, [376](#)
 - SX1276_IRQ_CAD_DONE, [376](#)
 - SX1276_IRQ_FHSS_CHANGE_CHANNEL, [376](#)
 - SX1276_IRQ_PAYLOAD_CRC_ERROR, [376](#)
 - SX1276_IRQ_RX_DONE, [376](#)
 - SX1276_IRQ_RX_TIMEOUT, [376](#)
 - SX1276_IRQ_TX_DONE, [376](#)
 - SX1276_IRQ_VALID_HEADER, [376](#)
 - SX1276_MODE_CAD, [376](#)
 - SX1276_MODE_FSRX, [377](#)
 - SX1276_MODE_FSTX, [377](#)
 - SX1276_MODE_RXCONTINUOUS, [377](#)
 - SX1276_MODE_RXSINGLE, [377](#)
 - SX1276_MODE_SLEEP, [377](#)
 - SX1276_MODE_STDBY, [377](#)
 - SX1276_MODE_TX, [377](#)
 - SX1276_OPMODE_ACCESS_SHARED_REG, [377](#)
 - SX1276_OPMODE_LONG_RANGE_MODE, [377](#)
 - SX1276_OPMODE_LOW_FREQ_MODE_ON, [377](#)
 - SX1276_OPMODE_MODE_MASK, [377](#)
 - SX1276_OPMODE_MODULATION_FSK, [377](#)
 - SX1276_OPMODE_MODULATION_MASK, [378](#)
 - SX1276_OPMODE_MODULATION_OOK, [378](#)
 - SX1276_OPMODE_SHAPING_FSK_BT03, [378](#)
 - SX1276_OPMODE_SHAPING_FSK_BT05, [378](#)
 - SX1276_OPMODE_SHAPING_FSK_BT10, [378](#)
 - SX1276_OPMODE_SHAPING_MASK, [378](#)
 - SX1276_OPMODE_SHAPING_NONE, [378](#)
 - SX1276_OPMODE_SHAPING_OOK_FILTER, [378](#)
 - SX1276_PA_DAC_DISABLE_17DBM, [378](#)
 - SX1276_PA_DAC_ENABLE_20DBM, [378](#)
 - SX1276_PACKET_ADDR_FILTER_NODE, [378](#)
 - SX1276_PACKET_ADDR_FILTER_NODE_BC, [378](#)
 - SX1276_PACKET_ADDR_FILTER_OFF, [379](#)
 - SX1276_PACKET_CRC_OFF, [379](#)
 - SX1276_PACKET_CRC_ON, [379](#)
 - SX1276_PACKET_CRC_WHITENING_CCITT, [379](#)
 - SX1276_PACKET_CRC_WHITENING_IBM, [379](#)

- SX1276_PACKET_DATA_MODE_CONTINUOUS, [379](#)
- SX1276_PACKET_DATA_MODE_PACKET, [379](#)
- SX1276_PACKET_DC_FREE_NONE, [379](#)
- SX1276_PACKET_DC_FREE_WHITENING, [379](#)
- SX1276_PACKET_FORMAT_FIXED, [379](#)
- SX1276_PACKET_FORMAT_VARIABLE, [379](#)
- SX1276_REG_AFC_BW, [379](#)
- SX1276_REG_BITRATE_LSB, [380](#)
- SX1276_REG_BITRATE_MSB, [380](#)
- SX1276_REG_BROADCAST_ADDRESS, [380](#)
- SX1276_REG_DETECTION_OPTIMIZE, [380](#)
- SX1276_REG_DETECTION_THRESHOLD, [380](#)
- SX1276_REG_DIO_MAPPING_1, [380](#)
- SX1276_REG_DIO_MAPPING_2, [380](#)
- SX1276_REG_FDEV_LSB, [380](#)
- SX1276_REG_FDEV_MSB, [380](#)
- SX1276_REG_FIFO, [380](#)
- SX1276_REG_FIFO_ADDR_PTR, [380](#)
- SX1276_REG_FIFO_RX_BASE_ADDR, [380](#)
- SX1276_REG_FIFO_RX_CURRENT_ADDR, [381](#)
- SX1276_REG_FIFO_THRESH, [381](#)
- SX1276_REG_FIFO_TX_BASE_ADDR, [381](#)
- SX1276_REG_FRF_LSB, [381](#)
- SX1276_REG_FRF_MID, [381](#)
- SX1276_REG_FRF_MSB, [381](#)
- SX1276_REG_INVERT_IQ, [381](#)
- SX1276_REG_INVERT_IQ2, [381](#)
- SX1276_REG_IRQ_FLAGS, [381](#)
- SX1276_REG_IRQ_FLAGS_1, [381](#)
- SX1276_REG_IRQ_FLAGS_2, [381](#)
- SX1276_REG_IRQ_FLAGS_MASK, [381](#)
- SX1276_REG_LNA, [382](#)
- SX1276_REG_MODEM_CONFIG_1, [382](#)
- SX1276_REG_MODEM_CONFIG_2, [382](#)
- SX1276_REG_MODEM_CONFIG_3, [382](#)
- SX1276_REG_NODE_ADDRESS, [382](#)
- SX1276_REG_OOK_AVG, [382](#)
- SX1276_REG_OOK_FIX, [382](#)
- SX1276_REG_OOK_PEAK, [382](#)
- SX1276_REG_OP_MODE, [382](#)
- SX1276_REG_PA_CONFIG, [382](#)
- SX1276_REG_PA_DAC, [382](#)
- SX1276_REG_PACKET_CONFIG_1, [382](#)
- SX1276_REG_PACKET_CONFIG_2, [383](#)
- SX1276_REG_PAYLOAD_LENGTH, [383](#)
- SX1276_REG_PAYLOAD_LENGTH_FSK, [383](#)
- SX1276_REG_PKT_RSSI_VALUE, [383](#)
- SX1276_REG_PKT_SNR_VALUE, [383](#)
- SX1276_REG_PREAMBLE_DETECT, [383](#)
- SX1276_REG_PREAMBLE_LSB, [383](#)
- SX1276_REG_PREAMBLE_LSB_FSK, [383](#)
- SX1276_REG_PREAMBLE_MSB, [383](#)
- SX1276_REG_PREAMBLE_MSB_FSK, [383](#)
- SX1276_REG_RSSI_CONFIG, [383](#)
- SX1276_REG_RSSI_VALUE_FSK, [383](#)
- SX1276_REG_RX_BW, [384](#)
- SX1276_REG_RX_CONFIG, [384](#)
- SX1276_REG_RX_NB_BYTES, [384](#)
- SX1276_REG_SYMB_TIMEOUT_LSB, [384](#)
- SX1276_REG_SYNC_CONFIG, [384](#)
- SX1276_REG_SYNC_VALUE_1, [384](#)
- SX1276_REG_SYNC_WORD, [384](#)
- SX1276_REG_VERSION, [384](#)
- SX1276_RXCONFIG_AFC_AUTO_ON, [384](#)
- SX1276_RXCONFIG_AGC_AUTO_ON, [384](#)
- SX1276_RXCONFIG_RESTART_RX_NO_PLL_LOCK, [384](#)
- SX1276_RXCONFIG_TRIGGER_RSSI, [384](#)
- SX1276_SPI_READ_MASK, [385](#)
- SX1276_SPI_WRITE_MASK, [385](#)
- SX1276_SYNC_AUTO_RESTART_NO_PLL, [385](#)
- SX1276_SYNC_AUTO_RESTART_OFF, [385](#)
- SX1276_SYNC_AUTO_RESTART_PLL, [385](#)
- SX1276_SYNC_FIFOFILL_AUTO, [385](#)
- SX1276_SYNC_FIFOFILL_MANUAL, [385](#)
- SX1276_SYNC_OFF, [385](#)
- SX1276_SYNC_ON, [385](#)
- SX1276_SYNC_PREAMBLE_POLARITY_55, [385](#)
- SX1276_SYNC_PREAMBLE_POLARITY_AA, [385](#)
- SX1276_VERSION_ID, [385](#)
- radio_take_events
 - radio_lib.c, [470](#)
 - radio_lib.h, [486](#)
- radio_test.c
 - radio_test_demo_init, [493](#)
 - radio_test_demo_process, [493](#)
 - radio_test_dump_basic, [493](#)
 - radio_test_dump_tx_debug, [493](#)
 - radio_test_event_cb, [494](#)
 - radio_test_irq_restore, [494](#)
 - radio_test_irq_save, [494](#)
 - radio_test_now_ms, [494](#)
 - radio_test_print_packet, [494](#)
 - radio_test_print_packet_text, [495](#)
 - radio_test_probe, [495](#)
 - radio_test_publish_packet_to_lcd, [495](#)
 - radio_test_send_ping, [495](#)
 - radio_test_store_packet_in_i2c_memory, [496](#)
 - s_demo_initialized, [496](#)
 - s_msg_store, [496](#)
 - s_msg_store_ready, [496](#)
 - s_radio_cb_events, [496](#)
 - s_radio_last_ping_ms, [496](#)
 - s_radio_last_tx_start_ms, [496](#)
- radio_test.h
 - radio_test_demo_init, [501](#)
 - radio_test_demo_process, [502](#)
 - radio_test_dump_basic, [502](#)
 - radio_test_probe, [502](#)
 - radio_test_send_ping, [502](#)
- radio_test_demo_init
 - radio_test.c, [493](#)
 - radio_test.h, [501](#)
- radio_test_demo_process
 - radio_test.c, [493](#)

- radio_test.h, [502](#)
- radio_test_dump_basic
 - radio_test.c, [493](#)
 - radio_test.h, [502](#)
- radio_test_dump_tx_debug
 - radio_test.c, [493](#)
- radio_test_event_cb
 - radio_test.c, [494](#)
- radio_test_irq_restore
 - radio_test.c, [494](#)
- radio_test_irq_save
 - radio_test.c, [494](#)
- radio_test_now_ms
 - radio_test.c, [494](#)
- radio_test_print_packet
 - radio_test.c, [494](#)
- radio_test_print_packet_text
 - radio_test.c, [495](#)
- radio_test_probe
 - radio_test.c, [495](#)
 - radio_test.h, [502](#)
- radio_test_publish_packet_to_lcd
 - radio_test.c, [495](#)
- radio_test_send_ping
 - radio_test.c, [495](#)
 - radio_test.h, [502](#)
- radio_test_store_packet_in_i2c_memory
 - radio_test.c, [496](#)
- radio_tx_timeout_ms
 - radio_fsk.c, [396](#)
 - radio_ook.c, [447](#)
- radio_wait_for_tx_enter_or_done
 - radio_lora.c, [422](#)
- radio_write_sync_word
 - radio_fsk.c, [396](#)
 - radio_ook.c, [447](#)
- raw
 - radio_main_ctx_t, [110](#)
- raw_len
 - radio_main_ctx_t, [110](#)
- raw_pressed
 - button_state_t, [73](#)
- reply_device_code
 - menu_notification_t, [87](#)
- reserved
 - i2c_mem_store_meta_wire_t, [77](#)
- reserved0
 - security_settings_wire_t, [128](#)
- reserved1
 - security_settings_wire_t, [128](#)
- reserved2
 - i2c_mem_store_meta_wire_t, [77](#)
- reset
 - radio_hw_cfg_t, [99](#)
- reset_pin
 - st33ktpm2x_cfg_t, [132](#)
- reset_port
 - st33ktpm2x_cfg_t, [132](#)
- reset_pulse_ms
 - st33ktpm2x_cfg_t, [132](#)
- reset_recovery_ms
 - st33ktpm2x_cfg_t, [132](#)
- result
 - radio_main_cmd_sync_t, [102](#)
 - security_cmd_sync_t, [121](#)
- retries_done
 - radio_main_ctx_t, [110](#)
- rssi_dbm
 - i2c_mem_store_log_record_t, [76](#)
 - menu_notification_t, [87](#)
 - radio_packet_t, [121](#)
 - security_cmd_t, [124](#)
- runtime_cfg
 - radio_main_cmd_sync_t, [102](#)
- runtime_out
 - security_cmd_sync_t, [122](#)
- rx_bandwidth
 - radio_fsk_cfg_t, [95](#)
 - radio_main_fsk_cfg_t, [112](#)
 - radio_main_ook_cfg_t, [114](#)
 - radio_ook_cfg_t, [117](#)
- rx_single_deadline_ms
 - radio_fsk_context_t, [97](#)
 - radio_ook_context_t, [119](#)
- rx_single_timeout_symbols
 - radio_context_t, [93](#)
- s_active_backend
 - radio_lib.c, [470](#)
- s_aes_rcon
 - laviet_crypto.c, [208](#)
- s_aes_sbox
 - laviet_crypto.c, [208](#)
- s_crypto_hw_ready
 - laviet_crypto.c, [208](#)
- s_crypto_mutex
 - laviet_crypto.c, [208](#)
- s_demo_initialized
 - radio_test.c, [496](#)
- s_led
 - led_array_lib.c, [278](#)
- s_msg_store
 - radio_test.c, [496](#)
- s_msg_store_ready
 - radio_test.c, [496](#)
- s_radio
 - radio_fsk.c, [397](#)
 - radio_lora.c, [422](#)
 - radio_ook.c, [447](#)
- s_radio_cb_events
 - radio_test.c, [496](#)
- s_radio_last_ping_ms
 - radio_test.c, [496](#)
- s_radio_last_tx_start_ms
 - radio_test.c, [496](#)
- s_runtime_cfg
 - radio_fsk.c, [397](#)

- radio_ook.c, 447
- s_runtime_cfg_ready
 - radio_fsk.c, 397
 - radio_ook.c, 447
- s_sha256_k
 - laviet_crypto.c, 208
- s_st33_i2c_tx
 - st33ktpm2x.c, 660
- secret_area_bytes
 - i2c_mem_store_cfg_t, 75
- secret_counter
 - i2c_mem_store_meta_wire_t, 77
 - i2c_mem_store_t, 79
- secret_slot_count
 - i2c_mem_store_t, 79
- Security, 45
- security_cmd_sync_t, 121
 - done, 121
 - result, 121
 - runtime_out, 122
 - trusted_out, 122
- security_cmd_t, 122
 - cfg, 123
 - code, 123
 - code_len, 123
 - enabled, 123
 - gateway_slot, 123
 - id, 123
 - index, 123
 - len, 123
 - mode, 123
 - node_id, 123
 - payload, 123
 - preset_id, 123
 - rssi_dbm, 124
 - sync, 124
 - value, 124
- SECURITY_FRAME_KEY_MODE_PAIR_V1_32
 - security_main.h, 615
- SECURITY_FRAME_KEY_MODE_SHARED
 - security_main.h, 615
- security_frame_key_mode_t
 - security_main.h, 614
- security_key_wire_t, 124
 - crc, 124
 - magic, 124
 - seed, 124
- security_main.h
 - SECURITY_FRAME_KEY_MODE_PAIR_V1_32, 615
 - SECURITY_FRAME_KEY_MODE_SHARED, 615
 - security_frame_key_mode_t, 614
 - security_main_cmd_add_device, 615
 - security_main_cmd_add_gateway, 615
 - security_main_cmd_delete_device, 616
 - security_main_cmd_get_device, 616
 - security_main_cmd_get_runtime_cfg, 616
 - security_main_cmd_rotate_key, 617
 - security_main_cmd_set_auto_ping, 617
 - security_main_cmd_set_auto_ping_mode, 617
 - security_main_cmd_set_auto_ping_period, 617
 - security_main_cmd_set_coding, 618
 - security_main_cmd_set_fh, 618
 - security_main_cmd_set_fh_period, 618
 - security_main_cmd_set_lora_preset, 618
 - security_main_cmd_set_notify_mode, 619
 - security_main_cmd_set_radio_runtime_cfg, 619
 - security_main_commit_gateway_counter, 619
 - security_main_create_task, 620
 - security_main_get_frame_keys, 620
 - security_main_get_frame_keys_for_code, 620
 - security_main_get_frame_keys_mode, 621
 - security_main_get_gateway_counter, 621
 - security_main_get_network_key, 622
 - security_main_get_peer_link_key, 622
 - security_main_get_tpm_ready, 622
 - security_main_log_message, 623
 - SECURITY_NOTIFY_BADGE, 615
 - security_notify_mode_t, 615
 - SECURITY_NOTIFY_POPUP, 615
 - security_main_cmd_add_device
 - security_main.h, 615
 - security_main_cmd_add_gateway
 - security_main.h, 615
 - security_main_cmd_delete_device
 - security_main.h, 616
 - security_main_cmd_get_device
 - security_main.h, 616
 - security_main_cmd_get_runtime_cfg
 - security_main.h, 616
 - security_main_cmd_rotate_key
 - security_main.h, 617
 - security_main_cmd_set_auto_ping
 - security_main.h, 617
 - security_main_cmd_set_auto_ping_mode
 - security_main.h, 617
 - security_main_cmd_set_auto_ping_period
 - security_main.h, 617
 - security_main_cmd_set_coding
 - security_main.h, 618
 - security_main_cmd_set_fh
 - security_main.h, 618
 - security_main_cmd_set_fh_period
 - security_main.h, 618
 - security_main_cmd_set_lora_preset
 - security_main.h, 618
 - security_main_cmd_set_notify_mode
 - security_main.h, 619
 - security_main_cmd_set_radio_runtime_cfg
 - security_main.h, 619
 - security_main_commit_gateway_counter
 - security_main.h, 619
 - security_main_create_task
 - security_main.h, 620
 - security_main_get_frame_keys
 - security_main.h, 620

security_main_get_frame_keys_for_code
 security_main.h, 620
 security_main_get_frame_keys_mode
 security_main.h, 621
 security_main_get_gateway_counter
 security_main.h, 621
 security_main_get_network_key
 security_main.h, 622
 security_main_get_peer_link_key
 security_main.h, 622
 security_main_get_tpm_ready
 security_main.h, 622
 security_main_log_message
 security_main.h, 623
 SECURITY_NOTIFY_BADGE
 security_main.h, 615
 security_notify_mode_t
 security_main.h, 615
 SECURITY_NOTIFY_POPUP
 security_main.h, 615
 security_radio_store_wire_t, 125
 magic, 125
 packed, 125
 version, 125
 security_runtime_cfg_t, 125
 active_modulation, 126
 auto_ping_enabled, 126
 auto_ping_mode, 126
 auto_ping_period_ms, 126
 coding_enabled, 126
 fh_enabled, 126
 fh_period_ms, 126
 fsk, 126
 lora, 127
 lora_preset, 127
 notify_mode, 127
 ook, 127
 radio_profiles_persisted, 127
 security_settings_wire_t, 127
 crc, 128
 flags, 128
 lora_preset, 128
 magic, 128
 notify_mode, 128
 reserved0, 128
 reserved1, 128
 version, 128
 security_store_wire_t, 128
 flags, 129
 lora_preset, 129
 magic, 129
 modulation_fh, 129
 notify_mode, 129
 seed, 129
 version, 129
 security_trusted_entry_t, 129
 code, 129
 code_len, 129
 in_use, 130
 is_master, 130
 node_id, 130
 seed
 security_key_wire_t, 124
 security_store_wire_t, 129
 selected_device_slot
 menu_state_t, 92
 selected_idx
 menu_state_t, 92
 send_target_action
 menu_state_t, 92
 sequence
 i2c_mem_store_log_record_t, 76
 service.h
 service_uart_create_task, 638
 SERVICE_UART
 pager_config.h, 351
 service_uart_create_task
 service.h, 638
 set_modulation
 menu_radio_action_binding_t, 89
 set_option
 menu_radio_action_binding_t, 89
 shaping
 radio_fsk_cfg_t, 95
 radio_main_fsk_cfg_t, 112
 slot
 trusted_info_t, 135
 slot_count
 i2c_mem_store_t, 79
 snr_db
 radio_packet_t, 121
 spi_timeout_ms
 radio_hw_cfg_t, 99
 sx1276_bus_t, 134
 spreading_factor
 radio_lora_cfg_t, 101
 src_id
 laviet_frame_t, 82
 ST33_ACCESS_ACTIVE_LOCALITY
 st33ktpm2x.c, 641
 ST33_ACCESS_READ_ZERO
 st33ktpm2x.c, 641
 ST33_ACCESS_REQUEST_USE
 st33ktpm2x.c, 641
 ST33_ACCESS_VALID
 st33ktpm2x.c, 642
 st33_be16_read
 st33ktpm2x.c, 644
 st33_be16_write
 st33ktpm2x.c, 644
 st33_be32_read
 st33ktpm2x.c, 644
 st33_be32_write
 st33ktpm2x.c, 644
 st33_fifo_read
 st33ktpm2x.c, 644

st33_fifo_write
 st33ktpm2x.c, [644](#)
 ST33_I2C_MAX_RETRIES
 st33ktpm2x.c, [642](#)
 st33_i2c_read
 st33ktpm2x.c, [645](#)
 st33_i2c_write
 st33ktpm2x.c, [645](#)
 st33_identity_is_valid
 st33ktpm2x.c, [645](#)
 st33_le32_read
 st33ktpm2x.c, [645](#)
 st33_min_u16
 st33ktpm2x.c, [645](#)
 st33_read_access
 st33ktpm2x.c, [646](#)
 st33_read_response_stream
 st33ktpm2x.c, [646](#)
 st33_read_sts
 st33ktpm2x.c, [646](#)
 ST33_REG_ACCESS
 st33ktpm2x.c, [642](#)
 ST33_REG_DATA_FIFO
 st33ktpm2x.c, [642](#)
 ST33_REG_DID_VID
 st33ktpm2x.c, [642](#)
 ST33_REG_INTERFACE_CAPABILITY
 st33ktpm2x.c, [642](#)
 ST33_REG_LOC_SEL
 st33ktpm2x.c, [642](#)
 ST33_REG_RID
 st33ktpm2x.c, [642](#)
 ST33_REG_STS
 st33ktpm2x.c, [642](#)
 st33_status_text
 st33ktpm2x.c, [646](#)
 ST33_STS_COMMAND_READY
 st33ktpm2x.c, [642](#)
 ST33_STS_DATA_AVAIL
 st33ktpm2x.c, [642](#)
 ST33_STS_EXPECT
 st33ktpm2x.c, [642](#)
 ST33_STS_READ_ZERO
 st33ktpm2x.c, [643](#)
 ST33_STS_TPM_GO
 st33ktpm2x.c, [643](#)
 ST33_STS_VALID
 st33ktpm2x.c, [643](#)
 st33_tpm2_nv_write_auth
 st33ktpm2x.c, [646](#)
 ST33_TPM_HEADER_SIZE
 st33ktpm2x.c, [643](#)
 ST33_TPM_NV_TRANSFER_MAX_BYTES
 st33ktpm2x.c, [643](#)
 ST33_TPM_PW_AUTH_SIZE
 st33ktpm2x.c, [643](#)
 ST33_TPM_RANDOM_MAX_BYTES
 st33ktpm2x.c, [643](#)
 ST33_TPM_RH_OWNER
 st33ktpm2x.c, [643](#)
 ST33_TPM_RH_PLATFORM
 st33ktpm2x.c, [643](#)
 ST33_TPM_RS_PW
 st33ktpm2x.c, [643](#)
 ST33_TRACE
 st33ktpm2x.c, [643](#)
 st33_trace_fail
 st33ktpm2x.c, [647](#)
 st33_trace_tis
 st33ktpm2x.c, [647](#)
 st33_wait_access_bits
 st33ktpm2x.c, [647](#)
 st33_wait_burst
 st33ktpm2x.c, [647](#)
 st33_wait_sts_bits
 st33ktpm2x.c, [647](#)
 st33_write_access
 st33ktpm2x.c, [648](#)
 st33_write_empty_password_auth
 st33ktpm2x.c, [648](#)
 st33_write_loc_sel
 st33ktpm2x.c, [648](#)
 st33_write_sts
 st33ktpm2x.c, [648](#)
 st33ktpm2x.c
 s_st33_i2c_tx, [660](#)
 ST33_ACCESS_ACTIVE_LOCALITY, [641](#)
 ST33_ACCESS_READ_ZERO, [641](#)
 ST33_ACCESS_REQUEST_USE, [641](#)
 ST33_ACCESS_VALID, [642](#)
 st33_be16_read, [644](#)
 st33_be16_write, [644](#)
 st33_be32_read, [644](#)
 st33_be32_write, [644](#)
 st33_fifo_read, [644](#)
 st33_fifo_write, [644](#)
 ST33_I2C_MAX_RETRIES, [642](#)
 st33_i2c_read, [645](#)
 st33_i2c_write, [645](#)
 st33_identity_is_valid, [645](#)
 st33_le32_read, [645](#)
 st33_min_u16, [645](#)
 st33_read_access, [646](#)
 st33_read_response_stream, [646](#)
 st33_read_sts, [646](#)
 ST33_REG_ACCESS, [642](#)
 ST33_REG_DATA_FIFO, [642](#)
 ST33_REG_DID_VID, [642](#)
 ST33_REG_INTERFACE_CAPABILITY, [642](#)
 ST33_REG_LOC_SEL, [642](#)
 ST33_REG_RID, [642](#)
 ST33_REG_STS, [642](#)
 st33_status_text, [646](#)
 ST33_STS_COMMAND_READY, [642](#)
 ST33_STS_DATA_AVAIL, [642](#)
 ST33_STS_EXPECT, [642](#)

ST33_STS_READ_ZERO, 643
ST33_STS_TPM_GO, 643
ST33_STS_VALID, 643
st33_tpm2_nv_write_auth, 646
ST33_TPM_HEADER_SIZE, 643
ST33_TPM_NV_TRANSFER_MAX_BYTES, 643
ST33_TPM_PW_AUTH_SIZE, 643
ST33_TPM_RANDOM_MAX_BYTES, 643
ST33_TPM_RH_OWNER, 643
ST33_TPM_RH_PLATFORM, 643
ST33_TPM_RS_PW, 643
ST33_TRACE, 643
st33_trace_fail, 647
st33_trace_tis, 647
st33_wait_access_bits, 647
st33_wait_burst, 647
st33_wait_sts_bits, 647
st33_write_access, 648
st33_write_empty_password_auth, 648
st33_write_loc_sel, 648
st33_write_sts, 648
st33ktpm2x_davint_is_asserted, 648
st33ktpm2x_default_cfg, 649
st33ktpm2x_deinit, 649
st33ktpm2x_hard_reset, 649
st33ktpm2x_init, 650
st33ktpm2x_pp_is_pressed, 650
st33ktpm2x_pp_wait_pressed, 650
st33ktpm2x_read_identity, 651
st33ktpm2x_release_locality0, 651
st33ktpm2x_request_locality0, 651
st33ktpm2x_tpm2_flush_context, 652
st33ktpm2x_tpm2_get_capability, 652
st33ktpm2x_tpm2_get_random, 653
st33ktpm2x_tpm2_nv_define, 653
st33ktpm2x_tpm2_nv_read, 654
st33ktpm2x_tpm2_nv_write, 654
st33ktpm2x_tpm2_nv_write_platform_pp, 655
st33ktpm2x_tpm2_pcr_read_sha256, 655
st33ktpm2x_tpm2_policy_command_code, 656
st33ktpm2x_tpm2_policy_or, 656
st33ktpm2x_tpm2_policy_pcr, 657
st33ktpm2x_tpm2_policy_physical_presence, 657
st33ktpm2x_tpm2_self_test, 658
st33ktpm2x_tpm2_start_auth_session, 658
st33ktpm2x_tpm2_startup, 659
st33ktpm2x_transceive, 659
st33ktpm2x.h
ST33KTPM2X_CMD_MAX_BYTES, 680
st33ktpm2x_davint_is_asserted, 682
st33ktpm2x_default_cfg, 683
st33ktpm2x_deinit, 683
ST33KTPM2X_EHAL, 682
ST33KTPM2X_EINVAL, 682
ST33KTPM2X_ENOTSUP, 682
ST33KTPM2X_EOVERFLOW, 682
ST33KTPM2X_EPROTO, 682
ST33KTPM2X_ESTATE, 682
ST33KTPM2X_ETIMEOUT, 682
ST33KTPM2X_ETPM_RC, 682
st33ktpm2x_hard_reset, 683
ST33KTPM2X_I2C_ADDR_DEFAULT, 680
st33ktpm2x_init, 684
ST33KTPM2X_OK, 682
st33ktpm2x_pp_is_pressed, 684
st33ktpm2x_pp_wait_pressed, 684
st33ktpm2x_read_identity, 685
st33ktpm2x_release_locality0, 685
st33ktpm2x_request_locality0, 685
ST33KTPM2X_RSP_MAX_BYTES, 680
st33ktpm2x_status_t, 682
ST33KTPM2X_TPM2_ALG_SHA256, 680
ST33KTPM2X_TPM2_CAP_ALGS, 680
ST33KTPM2X_TPM2_CAP_COMMANDS, 680
ST33KTPM2X_TPM2_CAP_HANDLES, 680
ST33KTPM2X_TPM2_CAP_PCERS, 680
ST33KTPM2X_TPM2_CAP_TPM_PROPERTIES, 681
ST33KTPM2X_TPM2_CC_GET_CAPABILITY, 681
ST33KTPM2X_TPM2_CC_GET_RANDOM, 681
ST33KTPM2X_TPM2_CC_NV_DEFINE_SPACE, 681
ST33KTPM2X_TPM2_CC_NV_READ, 681
ST33KTPM2X_TPM2_CC_NV_WRITE, 681
ST33KTPM2X_TPM2_CC_PCR_READ, 681
ST33KTPM2X_TPM2_CC_SELF_TEST, 681
ST33KTPM2X_TPM2_CC_STARTUP, 681
st33ktpm2x_tpm2_flush_context, 686
st33ktpm2x_tpm2_get_capability, 686
st33ktpm2x_tpm2_get_random, 687
st33ktpm2x_tpm2_nv_define, 687
st33ktpm2x_tpm2_nv_read, 688
st33ktpm2x_tpm2_nv_write, 688
st33ktpm2x_tpm2_nv_write_platform_pp, 689
st33ktpm2x_tpm2_pcr_read_sha256, 689
st33ktpm2x_tpm2_policy_command_code, 690
st33ktpm2x_tpm2_policy_or, 690
st33ktpm2x_tpm2_policy_pcr, 691
st33ktpm2x_tpm2_policy_physical_presence, 691
ST33KTPM2X_TPM2_RC_SUCCESS, 681
st33ktpm2x_tpm2_self_test, 692
ST33KTPM2X_TPM2_ST_NO_SESSIONS, 681
ST33KTPM2X_TPM2_ST_SESSIONS, 681
st33ktpm2x_tpm2_start_auth_session, 692
st33ktpm2x_tpm2_startup, 693
ST33KTPM2X_TPM2_SU_CLEAR, 682
ST33KTPM2X_TPM2_SU_STATE, 682
st33ktpm2x_transceive, 693
st33ktpm2x_cfg_t, 130
burst_timeout_ms, 131
davint_active_state, 131
davint_pin, 131
davint_port, 131
hi2c, 131
i2c_addr_7bit, 131

- io_timeout_ms, [131](#)
- locality_timeout_ms, [132](#)
- pp_active_state, [132](#)
- pp_pin, [132](#)
- pp_port, [132](#)
- reset_pin, [132](#)
- reset_port, [132](#)
- reset_pulse_ms, [132](#)
- reset_recovery_ms, [132](#)
- ST33KTPM2X_CMD_MAX_BYTES
 - st33ktpm2x.h, [680](#)
- st33ktpm2x_davint_is_asserted
 - st33ktpm2x.c, [648](#)
 - st33ktpm2x.h, [682](#)
- st33ktpm2x_default_cfg
 - st33ktpm2x.c, [649](#)
 - st33ktpm2x.h, [683](#)
- st33ktpm2x_deinit
 - st33ktpm2x.c, [649](#)
 - st33ktpm2x.h, [683](#)
- ST33KTPM2X_EHAL
 - st33ktpm2x.h, [682](#)
- ST33KTPM2X_EINVAL
 - st33ktpm2x.h, [682](#)
- ST33KTPM2X_ENOTSUP
 - st33ktpm2x.h, [682](#)
- ST33KTPM2X_EOVERFLOW
 - st33ktpm2x.h, [682](#)
- ST33KTPM2X_EPROTO
 - st33ktpm2x.h, [682](#)
- ST33KTPM2X_ESTATE
 - st33ktpm2x.h, [682](#)
- ST33KTPM2X_ETIMEOUT
 - st33ktpm2x.h, [682](#)
- ST33KTPM2X_ETPM_RC
 - st33ktpm2x.h, [682](#)
- st33ktpm2x_hard_reset
 - st33ktpm2x.c, [649](#)
 - st33ktpm2x.h, [683](#)
- ST33KTPM2X_I2C_ADDR_DEFAULT
 - st33ktpm2x.h, [680](#)
- st33ktpm2x_init
 - st33ktpm2x.c, [650](#)
 - st33ktpm2x.h, [684](#)
- ST33KTPM2X_OK
 - st33ktpm2x.h, [682](#)
- st33ktpm2x_pp_is_pressed
 - st33ktpm2x.c, [650](#)
 - st33ktpm2x.h, [684](#)
- st33ktpm2x_pp_wait_pressed
 - st33ktpm2x.c, [650](#)
 - st33ktpm2x.h, [684](#)
- st33ktpm2x_read_identity
 - st33ktpm2x.c, [651](#)
 - st33ktpm2x.h, [685](#)
- st33ktpm2x_release_locality0
 - st33ktpm2x.c, [651](#)
 - st33ktpm2x.h, [685](#)
- st33ktpm2x_request_locality0
 - st33ktpm2x.c, [651](#)
 - st33ktpm2x.h, [685](#)
- ST33KTPM2X_RSP_MAX_BYTES
 - st33ktpm2x.h, [680](#)
- st33ktpm2x_status_t
 - st33ktpm2x.h, [682](#)
- st33ktpm2x_t, [133](#)
 - cfg, [133](#)
 - initialized, [133](#)
 - locality0_acquired, [133](#)
- ST33KTPM2X_TPM2_ALG_SHA256
 - st33ktpm2x.h, [680](#)
- ST33KTPM2X_TPM2_CAP_ALGS
 - st33ktpm2x.h, [680](#)
- ST33KTPM2X_TPM2_CAP_COMMANDS
 - st33ktpm2x.h, [680](#)
- ST33KTPM2X_TPM2_CAP_HANDLES
 - st33ktpm2x.h, [680](#)
- ST33KTPM2X_TPM2_CAP_PCERS
 - st33ktpm2x.h, [680](#)
- ST33KTPM2X_TPM2_CAP_TPM_PROPERTIES
 - st33ktpm2x.h, [681](#)
- ST33KTPM2X_TPM2_CC_GET_CAPABILITY
 - st33ktpm2x.h, [681](#)
- ST33KTPM2X_TPM2_CC_GET_RANDOM
 - st33ktpm2x.h, [681](#)
- ST33KTPM2X_TPM2_CC_NV_DEFINE_SPACE
 - st33ktpm2x.h, [681](#)
- ST33KTPM2X_TPM2_CC_NV_READ
 - st33ktpm2x.h, [681](#)
- ST33KTPM2X_TPM2_CC_NV_WRITE
 - st33ktpm2x.h, [681](#)
- ST33KTPM2X_TPM2_CC_PCR_READ
 - st33ktpm2x.h, [681](#)
- ST33KTPM2X_TPM2_CC_SELF_TEST
 - st33ktpm2x.h, [681](#)
- ST33KTPM2X_TPM2_CC_STARTUP
 - st33ktpm2x.h, [681](#)
- st33ktpm2x_tpm2_flush_context
 - st33ktpm2x.c, [652](#)
 - st33ktpm2x.h, [686](#)
- st33ktpm2x_tpm2_get_capability
 - st33ktpm2x.c, [652](#)
 - st33ktpm2x.h, [686](#)
- st33ktpm2x_tpm2_get_random
 - st33ktpm2x.c, [653](#)
 - st33ktpm2x.h, [687](#)
- st33ktpm2x_tpm2_nv_define
 - st33ktpm2x.c, [653](#)
 - st33ktpm2x.h, [687](#)
- st33ktpm2x_tpm2_nv_read
 - st33ktpm2x.c, [654](#)
 - st33ktpm2x.h, [688](#)
- st33ktpm2x_tpm2_nv_write
 - st33ktpm2x.c, [654](#)
 - st33ktpm2x.h, [688](#)
- st33ktpm2x_tpm2_nv_write_platform_pp

st33ktpm2x.c, [655](#)
st33ktpm2x.h, [689](#)
st33ktpm2x_tpm2_pcr_read_sha256
st33ktpm2x.c, [655](#)
st33ktpm2x.h, [689](#)
st33ktpm2x_tpm2_policy_command_code
st33ktpm2x.c, [656](#)
st33ktpm2x.h, [690](#)
st33ktpm2x_tpm2_policy_or
st33ktpm2x.c, [656](#)
st33ktpm2x.h, [690](#)
st33ktpm2x_tpm2_policy_pcr
st33ktpm2x.c, [657](#)
st33ktpm2x.h, [691](#)
st33ktpm2x_tpm2_policy_physical_presence
st33ktpm2x.c, [657](#)
st33ktpm2x.h, [691](#)
ST33KTPM2X_TPM2_RC_SUCCESS
st33ktpm2x.h, [681](#)
st33ktpm2x_tpm2_self_test
st33ktpm2x.c, [658](#)
st33ktpm2x.h, [692](#)
ST33KTPM2X_TPM2_ST_NO_SESSIONS
st33ktpm2x.h, [681](#)
ST33KTPM2X_TPM2_ST_SESSIONS
st33ktpm2x.h, [681](#)
st33ktpm2x_tpm2_start_auth_session
st33ktpm2x.c, [658](#)
st33ktpm2x.h, [692](#)
st33ktpm2x_tpm2_startup
st33ktpm2x.c, [659](#)
st33ktpm2x.h, [693](#)
ST33KTPM2X_TPM2_SU_CLEAR
st33ktpm2x.h, [682](#)
ST33KTPM2X_TPM2_SU_STATE
st33ktpm2x.h, [682](#)
st33ktpm2x_transceive
st33ktpm2x.c, [659](#)
st33ktpm2x.h, [693](#)
stable_count
button_state_t, [73](#)
stable_pressed
button_state_t, [73](#)
standby
bmp280_params_t, [72](#)
state
laviet_sha256_ctx_t, [83](#)
radio_context_t, [94](#)
radio_fsk_context_t, [97](#)
radio_ook_context_t, [119](#)
step_ms
led_array_ctx_t, [86](#)
STRIDE / CIA Analysis, [39](#)
sx1276_bus_t, [134](#)
hspi, [134](#)
nss_pin, [134](#)
nss_port, [134](#)
spi_timeout_ms, [134](#)
sx1276_clear_irq_flags
radio_sx1276.c, [353](#)
radio_sx1276.h, [365](#)
sx1276_deselect
radio_sx1276.c, [353](#)
SX1276_DIO0_MAP_CAD_DONE
radio_sx1276_regs.h, [374](#)
SX1276_DIO0_MAP_FSK_PACKET_SENT
radio_sx1276_regs.h, [374](#)
SX1276_DIO0_MAP_FSK_PAYLOAD_READY
radio_sx1276_regs.h, [374](#)
SX1276_DIO0_MAP_RX_DONE
radio_sx1276_regs.h, [374](#)
SX1276_DIO0_MAP_TX_DONE
radio_sx1276_regs.h, [374](#)
SX1276_DIO1_MAP_CAD_DETECTED
radio_sx1276_regs.h, [374](#)
SX1276_DIO1_MAP_FHSS_CHANGE
radio_sx1276_regs.h, [374](#)
SX1276_DIO1_MAP_RX_TIMEOUT
radio_sx1276_regs.h, [374](#)
SX1276_FIFO_THRESH_TX_START_NOT_EMPTY
radio_sx1276_regs.h, [374](#)
sx1276_get_irq_flags
radio_sx1276.c, [353](#)
radio_sx1276.h, [365](#)
sx1276_get_packet_rssi_raw
radio_sx1276.c, [353](#)
radio_sx1276.h, [365](#)
sx1276_get_packet_snr_raw
radio_sx1276.c, [354](#)
radio_sx1276.h, [365](#)
sx1276_get_rx_current_addr
radio_sx1276.c, [354](#)
radio_sx1276.h, [365](#)
sx1276_get_rx_nb_bytes
radio_sx1276.c, [354](#)
radio_sx1276.h, [365](#)
sx1276_get_version
radio_sx1276.c, [354](#)
radio_sx1276.h, [366](#)
SX1276_IRQ1_MODE_READY
radio_sx1276_regs.h, [374](#)
SX1276_IRQ1_PLL_LOCK
radio_sx1276_regs.h, [375](#)
SX1276_IRQ1_PREAMBLE_DETECT
radio_sx1276_regs.h, [375](#)
SX1276_IRQ1_RSSI
radio_sx1276_regs.h, [375](#)
SX1276_IRQ1_RX_READY
radio_sx1276_regs.h, [375](#)
SX1276_IRQ1_SYNC_ADDRESS_MATCH
radio_sx1276_regs.h, [375](#)
SX1276_IRQ1_TIMEOUT
radio_sx1276_regs.h, [375](#)
SX1276_IRQ1_TX_READY
radio_sx1276_regs.h, [375](#)
SX1276_IRQ2_CRC_OK

radio_sx1276_regs.h, [375](#)
 SX1276_IRQ2_FIFO_FULL
 radio_sx1276_regs.h, [375](#)
 SX1276_IRQ2_FIFO_LEVEL
 radio_sx1276_regs.h, [375](#)
 SX1276_IRQ2_FIFO_NOT_EMPTY
 radio_sx1276_regs.h, [375](#)
 SX1276_IRQ2_FIFO_OVERRUN
 radio_sx1276_regs.h, [375](#)
 SX1276_IRQ2_LOW_BAT
 radio_sx1276_regs.h, [376](#)
 SX1276_IRQ2_PACKET_SENT
 radio_sx1276_regs.h, [376](#)
 SX1276_IRQ2_PAYLOAD_READY
 radio_sx1276_regs.h, [376](#)
 SX1276_IRQ_CAD_DETECTED
 radio_sx1276_regs.h, [376](#)
 SX1276_IRQ_CAD_DONE
 radio_sx1276_regs.h, [376](#)
 SX1276_IRQ_FHSS_CHANGE_CHANNEL
 radio_sx1276_regs.h, [376](#)
 SX1276_IRQ_PAYLOAD_CRC_ERROR
 radio_sx1276_regs.h, [376](#)
 SX1276_IRQ_RX_DONE
 radio_sx1276_regs.h, [376](#)
 SX1276_IRQ_RX_TIMEOUT
 radio_sx1276_regs.h, [376](#)
 SX1276_IRQ_TX_DONE
 radio_sx1276_regs.h, [376](#)
 SX1276_IRQ_VALID_HEADER
 radio_sx1276_regs.h, [376](#)
 sx1276_map_dio
 radio_sx1276.c, [354](#)
 radio_sx1276.h, [366](#)
 SX1276_MODE_CAD
 radio_sx1276_regs.h, [376](#)
 SX1276_MODE_FSRX
 radio_sx1276_regs.h, [377](#)
 SX1276_MODE_FSTX
 radio_sx1276_regs.h, [377](#)
 SX1276_MODE_RXCONTINUOUS
 radio_sx1276_regs.h, [377](#)
 SX1276_MODE_RXSINGLE
 radio_sx1276_regs.h, [377](#)
 SX1276_MODE_SLEEP
 radio_sx1276_regs.h, [377](#)
 SX1276_MODE_STDBY
 radio_sx1276_regs.h, [377](#)
 SX1276_MODE_TX
 radio_sx1276_regs.h, [377](#)
 SX1276_OPMODE_ACCESS_SHARED_REG
 radio_sx1276_regs.h, [377](#)
 SX1276_OPMODE_LONG_RANGE_MODE
 radio_sx1276_regs.h, [377](#)
 SX1276_OPMODE_LOW_FREQ_MODE_ON
 radio_sx1276_regs.h, [377](#)
 SX1276_OPMODE_MODE_MASK
 radio_sx1276_regs.h, [377](#)
 SX1276_OPMODE_MODULATION_FSK
 radio_sx1276_regs.h, [377](#)
 SX1276_OPMODE_MODULATION_MASK
 radio_sx1276_regs.h, [378](#)
 SX1276_OPMODE_MODULATION_OOK
 radio_sx1276_regs.h, [378](#)
 SX1276_OPMODE_SHAPING_FSK_BT03
 radio_sx1276_regs.h, [378](#)
 SX1276_OPMODE_SHAPING_FSK_BT05
 radio_sx1276_regs.h, [378](#)
 SX1276_OPMODE_SHAPING_FSK_BT10
 radio_sx1276_regs.h, [378](#)
 SX1276_OPMODE_SHAPING_MASK
 radio_sx1276_regs.h, [378](#)
 SX1276_OPMODE_SHAPING_NONE
 radio_sx1276_regs.h, [378](#)
 SX1276_OPMODE_SHAPING_OOK_FILTER
 radio_sx1276_regs.h, [378](#)
 SX1276_PA_DAC_DISABLE_17DBM
 radio_sx1276_regs.h, [378](#)
 SX1276_PA_DAC_ENABLE_20DBM
 radio_sx1276_regs.h, [378](#)
 SX1276_PACKET_ADDR_FILTER_NODE
 radio_sx1276_regs.h, [378](#)
 SX1276_PACKET_ADDR_FILTER_NODE_BC
 radio_sx1276_regs.h, [378](#)
 SX1276_PACKET_ADDR_FILTER_OFF
 radio_sx1276_regs.h, [379](#)
 SX1276_PACKET_CRC_OFF
 radio_sx1276_regs.h, [379](#)
 SX1276_PACKET_CRC_ON
 radio_sx1276_regs.h, [379](#)
 SX1276_PACKET_CRC_WHITENING_CCITT
 radio_sx1276_regs.h, [379](#)
 SX1276_PACKET_CRC_WHITENING_IBM
 radio_sx1276_regs.h, [379](#)
 SX1276_PACKET_DATA_MODE_CONTINUOUS
 radio_sx1276_regs.h, [379](#)
 SX1276_PACKET_DATA_MODE_PACKET
 radio_sx1276_regs.h, [379](#)
 SX1276_PACKET_DC_FREE_NONE
 radio_sx1276_regs.h, [379](#)
 SX1276_PACKET_DC_FREE_WHITENING
 radio_sx1276_regs.h, [379](#)
 SX1276_PACKET_FORMAT_FIXED
 radio_sx1276_regs.h, [379](#)
 SX1276_PACKET_FORMAT_VARIABLE
 radio_sx1276_regs.h, [379](#)
 sx1276_read_burst
 radio_sx1276.c, [354](#)
 radio_sx1276.h, [366](#)
 sx1276_read_fifo
 radio_sx1276.c, [355](#)
 radio_sx1276.h, [366](#)
 sx1276_read_reg
 radio_sx1276.c, [355](#)
 radio_sx1276.h, [367](#)
 SX1276_REG_AFC_BW

radio_sx1276_regs.h, [379](#)
SX1276_REG_BITRATE_LSB
radio_sx1276_regs.h, [380](#)
SX1276_REG_BITRATE_MSB
radio_sx1276_regs.h, [380](#)
SX1276_REG_BROADCAST_ADDRESS
radio_sx1276_regs.h, [380](#)
SX1276_REG_DETECTION_OPTIMIZE
radio_sx1276_regs.h, [380](#)
SX1276_REG_DETECTION_THRESHOLD
radio_sx1276_regs.h, [380](#)
SX1276_REG_DIO_MAPPING_1
radio_sx1276_regs.h, [380](#)
SX1276_REG_DIO_MAPPING_2
radio_sx1276_regs.h, [380](#)
SX1276_REG_FDEV_LSB
radio_sx1276_regs.h, [380](#)
SX1276_REG_FDEV_MSB
radio_sx1276_regs.h, [380](#)
SX1276_REG_FIFO
radio_sx1276_regs.h, [380](#)
SX1276_REG_FIFO_ADDR_PTR
radio_sx1276_regs.h, [380](#)
SX1276_REG_FIFO_RX_BASE_ADDR
radio_sx1276_regs.h, [380](#)
SX1276_REG_FIFO_RX_CURRENT_ADDR
radio_sx1276_regs.h, [381](#)
SX1276_REG_FIFO_THRESH
radio_sx1276_regs.h, [381](#)
SX1276_REG_FIFO_TX_BASE_ADDR
radio_sx1276_regs.h, [381](#)
SX1276_REG_FRF_LSB
radio_sx1276_regs.h, [381](#)
SX1276_REG_FRF_MID
radio_sx1276_regs.h, [381](#)
SX1276_REG_FRF_MSB
radio_sx1276_regs.h, [381](#)
SX1276_REG_INVERT_IQ
radio_sx1276_regs.h, [381](#)
SX1276_REG_INVERT_IQ2
radio_sx1276_regs.h, [381](#)
SX1276_REG_IRQ_FLAGS
radio_sx1276_regs.h, [381](#)
SX1276_REG_IRQ_FLAGS_1
radio_sx1276_regs.h, [381](#)
SX1276_REG_IRQ_FLAGS_2
radio_sx1276_regs.h, [381](#)
SX1276_REG_IRQ_FLAGS_MASK
radio_sx1276_regs.h, [381](#)
SX1276_REG_LNA
radio_sx1276_regs.h, [382](#)
SX1276_REG_MODEM_CONFIG_1
radio_sx1276_regs.h, [382](#)
SX1276_REG_MODEM_CONFIG_2
radio_sx1276_regs.h, [382](#)
SX1276_REG_MODEM_CONFIG_3
radio_sx1276_regs.h, [382](#)
SX1276_REG_NODE_ADDRESS
radio_sx1276_regs.h, [382](#)
SX1276_REG_OOK_AVG
radio_sx1276_regs.h, [382](#)
SX1276_REG_OOK_FIX
radio_sx1276_regs.h, [382](#)
SX1276_REG_OOK_PEAK
radio_sx1276_regs.h, [382](#)
SX1276_REG_OP_MODE
radio_sx1276_regs.h, [382](#)
SX1276_REG_PA_CONFIG
radio_sx1276_regs.h, [382](#)
SX1276_REG_PA_DAC
radio_sx1276_regs.h, [382](#)
SX1276_REG_PACKET_CONFIG_1
radio_sx1276_regs.h, [382](#)
SX1276_REG_PACKET_CONFIG_2
radio_sx1276_regs.h, [383](#)
SX1276_REG_PAYLOAD_LENGTH
radio_sx1276_regs.h, [383](#)
SX1276_REG_PAYLOAD_LENGTH_FSK
radio_sx1276_regs.h, [383](#)
SX1276_REG_PKT_RSSI_VALUE
radio_sx1276_regs.h, [383](#)
SX1276_REG_PKT_SNR_VALUE
radio_sx1276_regs.h, [383](#)
SX1276_REG_PREAMBLE_DETECT
radio_sx1276_regs.h, [383](#)
SX1276_REG_PREAMBLE_LSB
radio_sx1276_regs.h, [383](#)
SX1276_REG_PREAMBLE_LSB_FSK
radio_sx1276_regs.h, [383](#)
SX1276_REG_PREAMBLE_MSB
radio_sx1276_regs.h, [383](#)
SX1276_REG_PREAMBLE_MSB_FSK
radio_sx1276_regs.h, [383](#)
SX1276_REG_RSSI_CONFIG
radio_sx1276_regs.h, [383](#)
SX1276_REG_RSSI_VALUE_FSK
radio_sx1276_regs.h, [383](#)
SX1276_REG_RX_BW
radio_sx1276_regs.h, [384](#)
SX1276_REG_RX_CONFIG
radio_sx1276_regs.h, [384](#)
SX1276_REG_RX_NB_BYTES
radio_sx1276_regs.h, [384](#)
SX1276_REG_SYMB_TIMEOUT_LSB
radio_sx1276_regs.h, [384](#)
SX1276_REG_SYNC_CONFIG
radio_sx1276_regs.h, [384](#)
SX1276_REG_SYNC_VALUE_1
radio_sx1276_regs.h, [384](#)
SX1276_REG_SYNC_WORD
radio_sx1276_regs.h, [384](#)
SX1276_REG_VERSION
radio_sx1276_regs.h, [384](#)
SX1276_RXCONFIG_AFC_AUTO_ON
radio_sx1276_regs.h, [384](#)
SX1276_RXCONFIG_AGC_AUTO_ON

radio_sx1276_regs.h, [384](#)
 SX1276_RXCONFIG_RESTART_RX_NO_PLL_LOCK
 radio_sx1276_regs.h, [384](#)
 SX1276_RXCONFIG_TRIGGER_RSSI
 radio_sx1276_regs.h, [384](#)
 sx1276_select
 radio_sx1276.c, [355](#)
 sx1276_set_fifo_addr_ptr
 radio_sx1276.c, [356](#)
 radio_sx1276.h, [367](#)
 sx1276_set_fifo_base_addrs
 radio_sx1276.c, [356](#)
 radio_sx1276.h, [367](#)
 sx1276_set_frequency
 radio_sx1276.c, [356](#)
 radio_sx1276.h, [367](#)
 sx1276_set_invert_iq
 radio_sx1276.c, [356](#)
 radio_sx1276.h, [367](#)
 sx1276_set_lora_modem
 radio_sx1276.c, [356](#)
 radio_sx1276.h, [368](#)
 sx1276_set_op_mode
 radio_sx1276.c, [357](#)
 radio_sx1276.h, [368](#)
 sx1276_set_pa_output_power
 radio_sx1276.c, [357](#)
 radio_sx1276.h, [368](#)
 sx1276_set_payload_length
 radio_sx1276.c, [357](#)
 radio_sx1276.h, [368](#)
 sx1276_set_preamble
 radio_sx1276.c, [357](#)
 radio_sx1276.h, [368](#)
 sx1276_set_symbol_timeout
 radio_sx1276.c, [357](#)
 radio_sx1276.h, [369](#)
 sx1276_set_sync_word
 radio_sx1276.c, [357](#)
 radio_sx1276.h, [369](#)
 SX1276_SPI_READ_MASK
 radio_sx1276_regs.h, [385](#)
 SX1276_SPI_WRITE_MASK
 radio_sx1276_regs.h, [385](#)
 SX1276_SYNC_AUTO_RESTART_NO_PLL
 radio_sx1276_regs.h, [385](#)
 SX1276_SYNC_AUTO_RESTART_OFF
 radio_sx1276_regs.h, [385](#)
 SX1276_SYNC_AUTO_RESTART_PLL
 radio_sx1276_regs.h, [385](#)
 SX1276_SYNC_FIFOFILL_AUTO
 radio_sx1276_regs.h, [385](#)
 SX1276_SYNC_FIFOFILL_MANUAL
 radio_sx1276_regs.h, [385](#)
 SX1276_SYNC_OFF
 radio_sx1276_regs.h, [385](#)
 SX1276_SYNC_ON
 radio_sx1276_regs.h, [385](#)
 SX1276_SYNC_PREAMBLE_POLARITY_55
 radio_sx1276_regs.h, [385](#)
 SX1276_SYNC_PREAMBLE_POLARITY_AA
 radio_sx1276_regs.h, [385](#)
 SX1276_VERSION_ID
 radio_sx1276_regs.h, [385](#)
 sx1276_write_burst
 radio_sx1276.c, [358](#)
 radio_sx1276.h, [369](#)
 sx1276_write_fifo
 radio_sx1276.c, [358](#)
 radio_sx1276.h, [369](#)
 sx1276_write_reg
 radio_sx1276.c, [358](#)
 radio_sx1276.h, [370](#)
 sync
 radio_main_cmd_t, [103](#)
 security_cmd_t, [124](#)
 sync_word
 radio_fsk_cfg_t, [96](#)
 radio_lora_cfg_t, [101](#)
 radio_main_fsk_cfg_t, [113](#)
 radio_main_ook_cfg_t, [114](#)
 radio_ook_cfg_t, [118](#)
 sync_word_len
 radio_fsk_cfg_t, [96](#)
 radio_main_fsk_cfg_t, [113](#)
 radio_main_ook_cfg_t, [114](#)
 radio_ook_cfg_t, [118](#)
 temperature_c
 bmp280_api_data_t, [67](#)
 text
 menu_notification_t, [88](#)
 radio_main_cmd_t, [103](#)
 text0
 lcd_main_msg_t, [83](#)
 text1
 lcd_main_msg_t, [83](#)
 text2
 lcd_main_msg_t, [83](#)
 text3
 lcd_main_msg_t, [83](#)
 Threat Model, [43](#)
 threshold
 radio_main_ook_cfg_t, [114](#)
 radio_ook_cfg_t, [118](#)
 threshold_value
 radio_main_ook_cfg_t, [114](#)
 radio_ook_cfg_t, [118](#)
 tick_period_ms
 led_array_ctx_t, [86](#)
 timed_out
 radio_main_ctx_t, [110](#)
 timeout_ms
 radio_main_cmd_t, [104](#)
 timer_running
 led_array_ctx_t, [86](#)
 timestamp_ms

- bmp280_api_data_t, [67](#)
 - i2c_mem_store_log_record_t, [76](#)
 - radio_packet_t, [121](#)
 - title
 - menu_page_t, [88](#)
 - tof_get_distance
 - vl53l3cx_lib.h, [702](#)
 - tof_init
 - vl53l3cx_lib.h, [702](#)
 - tof_main.h
 - tof_main_create_task, [698](#)
 - tof_main_get_last_distance, [698](#)
 - tof_main_create_task
 - tof_main.h, [698](#)
 - tof_main_get_last_distance
 - tof_main.h, [698](#)
 - total_bytes
 - i2c_mem_store_cfg_t, [75](#)
 - transient_parent_page
 - menu_state_t, [92](#)
 - trusted_info_t, [134](#)
 - code, [135](#)
 - code_len, [135](#)
 - in_use, [135](#)
 - is_master, [135](#)
 - node_id, [135](#)
 - slot, [135](#)
 - trusted_out
 - security_cmd_sync_t, [122](#)
 - tx_ack_waiting
 - menu_state_t, [92](#)
 - tx_deadline_ms
 - radio_fsk_context_t, [97](#)
 - radio_main_ctx_t, [110](#)
 - radio_ook_context_t, [120](#)
 - tx_in_progress
 - radio_main_ctx_t, [110](#)
 - tx_power_dbm
 - radio_fsk_cfg_t, [96](#)
 - radio_lora_cfg_t, [101](#)
 - radio_main_fsk_cfg_t, [113](#)
 - radio_main_ook_cfg_t, [115](#)
 - radio_ook_cfg_t, [118](#)
 - tx_resume_state
 - radio_context_t, [94](#)
 - radio_fsk_context_t, [97](#)
 - radio_ook_context_t, [120](#)
 - tx_sent_text
 - menu_state_t, [92](#)
 - tx_silent
 - radio_main_ctx_t, [110](#)
 - type
 - lcd_main_msg_t, [83](#)
 - menu_notification_t, [88](#)
 - valid
 - bmp280_api_data_t, [68](#)
 - radio_main_ctx_t, [111](#)
 - radio_packet_t, [121](#)
 - valid_count
 - i2c_mem_store_meta_wire_t, [77](#)
 - i2c_mem_store_t, [79](#)
 - value
 - menu_radio_action_binding_t, [89](#)
 - radio_main_cmd_t, [104](#)
 - security_cmd_t, [124](#)
 - ver_type
 - laviet_frame_t, [82](#)
 - version
 - i2c_mem_store_meta_wire_t, [77](#)
 - security_radio_store_wire_t, [125](#)
 - security_settings_wire_t, [128](#)
 - security_store_wire_t, [129](#)
 - vl53l3cx_lib.h
 - tof_get_distance, [702](#)
 - tof_init, [702](#)
 - VL53L3CX_TestOnce, [702](#)
 - VL53L3CX_TestOnce
 - vl53l3cx_lib.h, [702](#)
 - wait_ms
 - radio_main_ctx_t, [111](#)
 - write_cycle_ms
 - i2c_mem_store_cfg_t, [75](#)
 - write_slot
 - i2c_mem_store_meta_wire_t, [77](#)
 - i2c_mem_store_t, [79](#)